

Principia Softwarica: The C– Compiler Backend **qc** version 0.1

Yoann Padioleau
`yoann.padioleau@gmail.com`

September 3, 2026

Copyright © 2010, 2015, 2026 Yoann Padioleau

Permission is granted to copy, distribute and/or modify this document, except all the source code it contains, under the terms of the GNU Free Documentation License, Version 1.3.

Contents

1	Introduction	17
1.1	Motivations	17
1.2	C--	17
1.3	Other backend code generators	17
1.4	Getting started	17
1.5	Requirements	17
1.6	About this document	17
1.7	Copyright	17
1.8	Acknowledgments	17
2	Overview	18
2.1	Compiler back end principles	18
2.2	qc services	18
2.3	Input C-- language	18
2.4	Output assembly language(s)	18
2.5	Code organization	18
2.6	Software architecture	18
3	Core Data Structures	19
3.1	Source Code Locations	19
3.1.1	Interface	19
3.1.2	Implementation	20
3.2	Abstract Syntax Definition	22
3.2.1	The ASDL-definition	22
3.3	Normalized Abstract-Syntax Tree	25
3.4	Fenv	27
3.5	RTL	27
3.6	Nelab	27
3.7	Target	27
3.8	Asm	27
4	main()	28
4.1	Driver	28
4.1.1	Caml Implementation	29
4.1.2	Driver support for the new front end	30
5	Parsing	32
5.1	Lexical Analysis	32
5.1.1	Declarations for Regular Expressions	33
5.1.2	The Main Lexer	34

5.1.3	Character Literals	35
5.1.4	Escape Sequences	36
5.1.5	Comments	37
5.1.6	Strings	37
5.1.7	Pragmas	38
5.1.8	Debugging the Scanner	40
5.2	Grammatical Analysis	41
6	Normalizing	51
6.1	Implementation	51
7	IR	55
8	Elaborating	56
9	Control-flow Graph	57
9.1	Dataflow pass for available expressions	57
9.1.1	Implementation	57
10	Liveness	59
10.1	Liveness Analysis	59
10.1.1	Interface	59
10.2	Implementation of liveness	59
10.3	Dead-assignment elimination	61
11	Register Allocators	63
11.1	A register allocator written as a dataflow algorithm	63
11.1.1	Useful boilerplate and a few utilities	63
11.1.2	The big picture	65
11.1.3	Gathering related dataflow information	66
11.1.4	Allocating registers in an RTL	67
11.2	Framing the allocator as a dataflow analysis	72
11.3	Optimization: Choosing spills	74
11.4	Optimization: Allocation Preferences	78
11.5	Graph Coloring Register Allocation	79
11.5.1	Overview	79
11.5.2	Data Structures	81
11.5.3	Graph Coloring State	83
11.5.4	Graph Coloring Stages	87
11.5.5	Lua registration	103
11.5.6	Debugging support	109
11.5.7	Register Class Trees	112
11.5.8	Register Class Trees	113
11.5.9	Utilities to implement	115
11.5.10	Lua registration	116
11.6	DFS Linear Scan Register Allocation	117
11.7	DFS Linear Scan Register Allocation	117
11.7.1	Preferences	118
11.7.2	The Basic Algorithm	120
11.7.3	Spilling	127
11.7.4	Choosing a Register for a Single Temp	127

11.7.5	Shuffling Establishes Consistency	129
11.7.6	Variable Maps for the Runtime System	133
11.7.7	Utilities	134
11.7.8	Exporting to Lua	136
12	Layout	137
13	Code Generation	138
13.1	Widening	138
13.1.1	Floating-point widener	139
13.1.2	Integer widening by dynamic programming	142
13.1.3	Counting operations and testing	165
14	ARM target	167
15	Runtime	168
15.1	C-- Run-time Interface	168
15.2	Implementation	169
15.2.1	Walking the stack	171
15.2.2	Unwinding to a continuation	176
15.2.3	Span Data	178
15.2.4	Accessing Local Variables	178
15.2.5	Run-time functions written in C--	180
15.2.6	Threads	180
15.2.7	Putting it together	182
15.3	PC Map	183
15.3.1	Implementation	185
15.3.2	Linker script	191
15.4	Walking GCC-Linux Stack Frames	191
15.5	x86cont	195
16	Optimizations	196
16.1	Simple Optimization	196
16.1.1	The Optimizations	196
16.1.2	The Implementations	196
16.2	Peephole Optimization	199
16.2.1	Implementation	200
16.3	Graph information	203
16.4	The dominator tree interface	204
16.5	The Lengauer-Tarjan algorithm	205
16.6	A translator module for the zipcfg	207
17	Advanced Topics	211
18	Conclusion	212
A	Debugging	213
A.1	Diagnostic output	213
A.2	AST Pretty Printer	214
A.2.1	Interface	214
A.2.2	Implementation	214

B Profiling	225
C Error Management	226
C.1 Error Handling	226
C.1.1 Interface	226
C.1.2 Implementation	227
C.2 Impossible	230
C.2.1 Implementation	231
C.3 Announcing unsupported constructs	231
D Utilities	235
E Mini stdlib	236
F Extra Code	237
G prelude	238
G.1 prelude/auxfunns.nw	238
G.2 Auxiliaries	238
G.2.1 Implementation	239
G.3 prelude/pp.nw	240
G.4 Pretty Printer	240
G.4.1 Interface	240
G.4.2 Implementation	241
G.5 prelude/strutil.nw	244
G.6 String Utilities	244
G.7 Implementation	244
G.8 prelude/verbose.nw	245
G.9 Diagnostic output	245
H bitops/	246
H.1 bitops/alignment.nw	246
H.2 Interface to manipulate alignment constraints	246
H.3 Implementation of alignment manipulation	246
H.4 commons3/bits.nw	246
H.5 Bits – Low-Level Representation of Values	247
H.5.1 Interface	247
H.5.2 Arithmetic	247
H.5.3 Signed Constructors	248
H.5.4 Signed Observers	248
H.5.5 Unsigned Constructors	248
H.5.6 Unsigned Observers	248
H.5.7 Implementation	248
H.5.8 Sign and zero extension	249
H.5.9 Strings	249
H.5.10 Arithmetic	249
H.5.11 Signed Interpretation	251
H.5.12 Unsigned Interpretation	252
H.6 Residualizing instantiation	255
H.6.1 Residualizing instantiation — Interface	255
H.6.2 Residualizing instantiation — Implementation	255

H.7	bitops/bitset64.nw	258
H.8	Small sets of bits	258
H.9	Implementation	259
H.10	bitops/cell.nw	259
H.11	Cells	259
H.12	bitops/ctypes.nw	260
H.13	codegen/idcode.nw	262
H.14	Code arbitrary string as an identifier	262
H.15	codegen/idgen.nw	263
H.16	Name Generator	263
	H.16.1 Interface	263
	H.16.2 Implementation	263
H.17	prelude/reinit.nw	263
H.18	Reinitialization	264
H.19	regalloc/tx.nw	264
H.20	commons3/uint64.nw	264
H.21	UInt64 – unsigned operations on int64	265
	H.21.1 Interface	265
	H.21.2 Implementation	265
	H.21.3 C implementation of the primitives	266
I	error	269
I.1	error/debug.nw	269
I.2	error/error.nw	269
I.3	error/impossible.nw	269
I.4	error/srcmap.nw	269
I.5	error/unsupported.nw	269
J	parsing	270
J.1	parsing/ast.nw	270
J.2	parsing/astpp.nw	270
J.3	parsing/parser.nw	270
K	front_rtl	271
K.1	ir/register.nw	271
K.2	Register	271
K.3	Implementation of registers	272
K.4	ir/reloc.nw	275
K.5	Relocatable Addresses	275
	K.5.1 Implementation	276
K.6	ir/rtl.nw	277
K.7	Register-transfer Lists (RTL)	277
	K.7.1 Interface	277
	K.7.2 Implementation	280
K.8	front_rtl/rtldebug.nw	282
K.9	Functions for Debugging RTLs	282
	K.9.1 Implementation	282
K.10	front_rtl/rtlop.nw	283
K.11	Support for RTL operators	283
	K.11.1 Implementation	284

K.11.2	RTL-Op Types	284
K.11.3	Translation from C-- operators to RTL operators	286
K.12	<code>front_rtl/rtlutil.nw</code>	288
K.13	RTL Utilities	288
K.13.1	Important types	288
K.13.2	Common operations on RTLs	288
K.13.3	Equality and comparison	289
K.13.4	Width	289
K.13.5	Evaluation Time Classification	289
K.13.6	Read-Write Sets	289
K.13.7	Search	290
K.13.8	Aliasing	290
K.13.9	Substitution	291
K.13.10	Classification	291
K.13.11	RTL AST Representation	291
K.13.12	RTL String Representation	292
K.13.13	Implementation	292
K.13.14	Substitutions inside RTLs (Second Try)	295
K.13.15	RTL Classification	297
K.13.16	Conversion to AST	298
K.13.17	Conversion to Readable String	300
K.13.18	Evaluation Time Classification	301
K.13.19	Conversion to String	302
K.13.20	Aliasing	303
K.14	Reassociating arithmetic	305
K.14.1	Operator search	306
K.15	<code>front_rtl/symbol.nw</code>	310
K.16	Symbol	310
K.17	<code>front_rtl/types.nw</code>	311
K.18	Types	311
K.18.1	Interface (Types)	311
K.18.2	Interface (Functions)	311
K.18.3	Useful abbreviations	312
K.18.4	Implementation	312
L	ir	316
L.1	<code>ir/asm.nw</code>	316
L.2	Abstract Assembler Interface	316
L.2.1	Assembler Methods	316
M	ir	319
M.1	<code>ir/block.nw</code>	319
M.2	Memory Blocks	319
M.2.1	Lua Interface	320
M.2.2	Implementation	320
M.3	<code>front_fenv/eqn.nw</code>	322
M.4	Linear Equations over Integers	322
M.4.1	Interface	322
M.4.2	Implementation	323
M.4.3	Test	326

M.5	<code>front_fenv/fenv.nw</code>	328
M.6	The Fat Environment	328
M.6.1	Interface: Clean vs. Dirty	328
M.6.2	Getting Started: creating an Environment	328
M.6.3	Scopes and the Fat Environment	329
M.6.4	Bindings for Values and Types	329
M.6.5	Binding and Finding Names and Values	330
M.6.6	Error Flag	330
M.6.7	Imports, Exports, and Assembler Symbols	330
M.6.8	The Details of C-- names and Assembler Symbols	330
M.6.9	Implementation: Large Scale Structure	330
M.6.10	The Heart of the Implementation: <code>Env</code>	331
M.6.11	Auxiliaries	332
M.6.12	Mapping	332
M.6.13	Creating <code>env</code> and <code>scope</code> values	333
M.6.14	Binding and Finding Names and Values	334
M.6.15	The Error Flag	335
M.6.16	Imports, Exports, Symbols	335
M.6.17	Register Index	336
M.6.18	Cleaning a fat environment	336
M.7	<code>front_fenv/metrics.nw</code>	337
M.8	Target Platform Metrics	337
M.9	Implementation	337
M.10	<code>front_fenv/rtleqn.nw</code>	339
M.11	Equations over RTL-Expressions	339
M.12	Implementation of equations	339
N	<code>cfg/legacy</code>	342
N.1	<code>cfg/legacy/cfg.nw</code>	342
N.2	Control-Flow Graph Revisions	342
N.3	Goals	342
N.4	The Control-Flow Graph	342
N.5	Invariants	342
N.6	Interface	342
N.6.1	Interface summary	343
N.6.2	Observers	343
N.6.3	Mutators	344
N.6.4	Constructors	345
N.7	Implementation	346
N.7.1	Node Type	347
N.8	<code>front_cfg/cfgx.nw</code>	367
N.9	<code>cfg/legacy/dag.nw</code>	367
N.10	Directed Acyclic Graph	367
N.10.1	Interface	367
N.10.2	Implementation	368
N.11	<code>cfg/legacy/ep.nw</code>	369
N.12	Embedding-projection pairs	369
N.13	<code>front_cfg/mflow.nw</code>	369
N.14	Machine-level control flow	369
N.15	<code>front_cfg/spans.nw</code>	372

N.16 Representing spans, including internal spans	372
O elab	374
O.1 elab/elabexp.nw	374
O.2 Elaborating Expressions	374
O.2.1 Implementation	374
O.2.2 The translation of types, names, and expressions	375
O.3 front_nelab/elabstmt.nw	377
O.4 Elaborating Statements	378
O.4.1 Implementation	379
O.4.2 Value-passing statements: calls and so on	383
O.5 front_nelab/memalloc.nw	387
O.6 Automaton-Based Memory Allocation	387
O.6.1 Implementation	387
O.7 front_nelab/nast.nw	388
O.8 front_nelab/nelab.nw	388
O.9 Checking Static Semantics	389
O.9.1 Assembler Symbols	389
O.9.2 Implementation – The 30 000 feet overview	389
O.9.3 Auxiliaries	390
O.9.4 Sorting and binding type and constant declarations	390
O.9.5 Building the environment	392
O.9.6 Translation of <code>stackdata</code> declarations, accumulating labels	395
O.10 front_nelab/simplify.nw	398
O.11 RTL Constant Folding	398
O.11.1 Implementation	398
O.11.2 RTL Traversal	400
O.11.3 Unsafe Simplification	402
O.11.4 RTL Operator Implementations	403
O.11.5 Machine-dependent simplifications	408
O.12 front_nelab/topsort.nw	408
O.13 Topological Sort	408
O.13.1 Interface	408
O.13.2 Implementation	409
O.13.3 Test	409
P target	411
P.1 target/automaton.nw	411
P.2 Automaton-Based Slot Allocation	411
P.2.1 Standard kinds	411
P.2.2 Observer-Mutator interface	411
P.2.3 Constructor interface	412
P.3 Implementation	413
P.3.1 Wrapping etc.	414
P.4 front_target/box.nw	419
P.5 Boxing	419
P.6 front_target/float.nw	420
P.7 Support for floating-point numbers	420
P.8 front_target/space.nw	421
P.9 RTL-Spaces	421

P.9.1	Implementation	422
P.10	front_target/target.nw	425
P.11	Target Platform Description	425
P.11.1	Calling Convention	425
P.11.2	Bits and Pieces	426
P.11.3	Identify Special Locations	426
P.11.4	Named Hardware Registers	426
P.11.5	Temporaries, Register indices, and Register Allocation	426
P.11.6	The recognizer	426
P.11.7	Control-flow operations	427
P.11.8	Code-generation assist	427
P.11.9	Code-generation capabilities	428
P.11.10	Implementation	428
P.11.11	Lua support for exposing target capabilities	430
Q	cfg/zipcfg	434
Q.1	front_zipcfg/avail.nw	434
Q.2	Available expressions	434
Q.2.1	Implementation	434
Q.3	front_zipcfg/property.nw	436
Q.4	front_zipcfg/unique.nw	441
Q.5	Unique identifiers, maps, and sets	441
Q.5.1	Implementation	442
Q.6	front_zipcfg/varmap.nw	443
Q.6.1	Variable Maps	443
Q.7	cfg/zipcfg/zipcfg.nw	448
Q.8	Applicative Control-Flow Graph, Based on Huet's Zipper	449
Q.8.1	Implementation	452
R	codegen	464
R.1	codegen/ast2ir.nw	464
R.2	Translation to Intermediate Representation	464
R.3	Implementation: Translate to intermediate representation	464
R.3.1	Abbreviations, utilities, and type definitions	464
R.3.2	Overall structure of the translation	465
R.3.3	The translation of actual parameters, formal parameters, and results	466
R.3.4	The translation of continuations and flow annotations	466
R.3.5	Translations of statements	469
R.3.6	Translating a procedure	472
R.3.7	Initialized and uninitialized data	476
R.3.8	Global Registers	476
R.3.9	Translating a full compilation unit	477
R.4	front_ir/automatongraph.nw	478
R.5	Automaton-Graph	478
R.5.1	Implementation	478
R.5.2	automatongraph.ml	480
R.6	front_ir/call.nw	481
R.7	Calling conventions	482
R.7.1	Discussion	482
R.7.2	Interface	482

R.7.3	Registration of Calling Conventions from Lua	483
R.7.4	Implementation	484
R.7.5	Potential new constructors for <code>Call.t</code>	486
R.8	<code>front_ir/context.nw</code>	487
R.9	Operator Contexts	487
R.9.1	Implementation	488
R.10	<code>front_ir/contn.nw</code>	490
R.11	Continuations	490
R.11.1	Implementation	491
R.12	<code>front_ir/expander.nw</code>	491
R.13	A generic, parameterized code expander	492
R.13.1	Support for expansion—the postexpander module	492
R.13.2	The target-dependent postexpander interface	493
R.13.3	Implementation of the postexpander module	496
R.13.4	A generic expander	498
R.14	Things yet to be covered in this document	520
R.15	<code>front_ir/opshape.nw</code>	520
R.16	Operator-shape analysis	520
R.16.1	Shapes	521
R.16.2	Obsolete, unsupported operators	521
R.17	<code>front_ir/preast2ir.nw</code>	524
R.18	Target and proc definitions	524
R.19	<code>front_ir/proc.nw</code>	525
R.20	Procedure Intermediate Representation	525
R.21	<code>front_ir/rewrite.nw</code>	525
R.22	Rewriting RTL operators (and related utilities)	526
R.22.1	Basic operator interface	526
R.22.2	Rewriting functions	526
R.22.3	Other related utilities	527
R.22.4	Implementation	527
R.23	Things not to be looked at	528
R.23.1	Automatically generated helper functions	528
R.23.2	C code to test division rules	531
R.24	<code>front_ir/runtimedata.nw</code>	535
R.25	Collecting and emitting data for the run-time system	535
R.25.1	The interface	535
R.25.2	Imports, type definitions, and utilities	535
R.26	<code>front_ir/talloc.nw</code>	540
R.27	Allocator for temporaries	540
R.27.1	Implementation	540
R.28	<code>front_ir/typecheck.nw</code>	542
R.29	Type Checker for the Control-Flow Graph	542
R.29.1	Implementation	542
S	asm	543
S.1	<code>asm/astasm.nw</code>	543
S.2	C-- Assembler	543
S.2.1	Implementation	543
S.3	Assembler Constructor Function	547
S.4	<code>assembler/cfgutil.nw</code>	547

S.5	Control-Flow Graph Utilities	547
S.5.1	Implementation	548
S.6	assembler/dotasm.nw	551
S.7	Assembler Interface to DOT	551
S.7.1	Implementation	551
S.8	assembler/dummyasm.nw	553
S.9	Dummy Assembler	553
S.9.1	Implementation	553
S.10	assembler/mangle.nw	554
S.11	Name Mangling	554
S.11.1	Implementation	555
S.11.2	Example	556
T	front_last	557
T.1	front_last/automatonutil.nw	557
T.2	Automaton utilities	557
T.2.1	Implementation	557
T.3	front_last/callspec.nw	557
T.4	Constructing a Calling Convention	558
T.4.1	Implementation	559
T.5	front_last/dataflow.nw	561
T.6	A polymorphic infrastructure for dataflow problems	561
T.6.1	Descriptions of dataflow passes	562
T.7	Implementation	564
T.7.1	Utilities	564
T.7.2	Backward problems	565
T.7.3	Forward	569
T.8	front_last/mvalidate.nw	574
T.9	RTL Validation	575
T.10	front_last/placevar.nw	577
T.11	Placing variables into suitable temporary locations	577
T.11.1	Variable Placement by Execution Estimate	578
T.12	layout/vfp.nw	582
T.13	Virtual frame pointer	582
U	arch/dummy	586
U.1	arch/dummy/dummy.nw	586
U.2	The Dummy Target	586
U.2.1	Implementation	586
U.2.2	Global Register Allocation	589
U.2.3	Spaces	589
U.2.4	Control Flow	589
U.2.5	Spills and Reloads	590
U.2.6	Calling Conventions	590
U.2.7	New-Style Calling Convention	590
U.3	arch/dummy/dummyexpander.nw	592
U.4	Code-Expander for the Dummy Target	592
U.4.1	Code Expansion with BURG	592
U.4.2	Utilities for BURG actions	592
U.5	Code Expansion Rules	594

U.5.1	Interfacing RTLs with the Expander	598
U.5.2	Expanding an entire CFG	599
U.5.3	Exporting the Expander to Lua	599
V	arch/x86	600
V.1	arch/x86/x86.nw	600
V.2	Back end for a Intel x86 (32-bit subset)	600
V.3	Implementation	600
V.3.1	Storage spaces	601
V.3.2	Postexpander	601
V.3.3	Stack operations	609
V.3.4	Building the target	612
V.3.5	Variable placement	615
V.4	Tentative example Lua code	615
V.5	arch/x86/x86all.nw	616
V.6	arch/x86/x86asm.nw	616
V.7	X86 Assembler	616
V.7.1	Implementation	616
V.8	arch/x86/x86call.nw	619
V.9	X86 calling conventions	620
V.10	Implementation of X86 calling conventions	620
V.10.1	Remove? Implementation using Callspec	626
V.11	arch/x86/x86rec.nw	630
V.12	Intel x86 Recognizer	630
V.12.1	Utilities	632
V.12.2	Register names	633
V.12.3	Other machine info	633
V.12.4	Recognizer terminals, nonterminals, and constructors	633
V.12.5	Recognizer Rules	633
V.12.6	Interfacing RTLs with the Expander	641
V.12.7	The exported recognizers	648
V.13	arch/x86/x86regs.nw	648
V.14	X86 registers	648
V.15	Implementation	649
W	arch/ppc	650
W.1	arch/ppc/ppc.nw	650
W.2	Module Structure	650
W.3	Spaces	651
W.4	Post Expander	653
W.5	Calling Conventions	659
W.6	Target Specification	663
W.7	Variable Placer	664
W.8	arch/ppc/ppcasm.nw	664
W.9	PPC Assembler	665
W.9.1	Implementation	665
W.10	arch/ppc/ppcrec.nw	668
W.11	PPC Recognizer	668
W.11.1	Utilities	669
W.11.2	Recognizer Rules	669

W.11.3	Interfacing RTLs with the Expander	673
W.11.4	The exported recognizers	675
X	arch/sparc	676
X.1	Back end for the SPARC	676
X.1.1	Storage spaces	676
X.1.2	Registers	677
X.1.3	Utilities	677
X.1.4	Control flow	677
X.1.5	Postexpander	678
X.1.6	Expander	686
X.1.7	Target	686
X.1.8	Variable Placement	689
X.2	SPARC Assembler	689
X.2.1	Implementation	689
X.3	SPARC calling conventions	692
X.4	Implementation of SPARC calling conventions	692
X.4.1	Implementation based on Callspec	695
X.5	SPARC Recognizer	698
X.5.1	Utilities	699
X.5.2	Recognizer terminals, nonterminals, and constructors	700
X.5.3	Recognizer Rules	700
X.5.4	Interfacing RTLs with the Expander	707
X.5.5	The exported recognizer	710
X.6	SPARC spaces and registers	710
Y	arch/alpha	714
Y.1	Back end for the Alpha	714
Y.1.1	Name and storage spaces	714
Y.1.2	Registers	715
Y.1.3	Utilities	715
Y.1.4	Control-flow RTLs	715
Y.1.5	Postexpander	716
Y.1.6	Expander	724
Y.1.7	Spill and reload	724
Y.1.8	Global Variables	724
Y.1.9	The target record	725
Y.2	Alpha Assembler	727
Y.2.1	Name Mangling	727
Y.2.2	Implementation	727
Y.2.3	Public Methods	728
Y.3	Alpha calling conventions	730
Y.3.1	Implementation of Alpha calling conventions	730
Y.3.2	Putting it together	733
Y.4	Alpha Recognizer	736
Y.4.1	Implementation	737
Y.4.2	Recognizer Rules	738
Y.4.3	Debugging Support	742
Y.4.4	Interfacing RTLs with the Expander	742
Y.4.5	Special cases	743

Y.4.6	The exported recognizers	744
Z	arch/mips	745
Z.1	arch/mips/mips.nw	745
Z.2	Back end for the MIPS	745
Z.2.1	Name and storage spaces	745
Z.2.2	Registers	746
Z.2.3	Utilities	746
Z.2.4	Control-flow RTLs	746
Z.2.5	Postexpander	747
Z.2.6	Expander	750
Z.2.7	Spill and reload	750
Z.2.8	Global Variables	750
Z.2.9	The target record	751
Z.3	arch/mips/mipsasm.nw	752
Z.4	MIPS Assembler	752
Z.4.1	Name Mangling	753
Z.4.2	Implementation	753
Z.4.3	Public Methods	754
Z.5	arch/mips/mipscall.nw	756
Z.6	MIPS calling conventions	756
Z.6.1	Implementation of MIPS calling conventions	756
Z.6.2	Putting it together	759
Z.6.3	Implementation using Callspec	760
Z.7	arch/mips/mipsrec.nw	761
Z.8	MIPS Recognizer	761
Z.8.1	Implementation	762
Z.8.2	Recognizer Rules	763
Z.8.3	Debugging Support	766
Z.8.4	Interfacing RTLs with the Expander	766
Z.8.5	Special cases	767
Z.8.6	The exported recognizers	768
Z.9	arch/mips/mipsregs.nw	768
Z.10	MIPS spaces and registers	768
AA	arch/arm	770
AA.1	arch/arm/arm.nw	770
AA.2	Back end for the ARM	770
AA.2.1	Abbreviations and utility functions	770
AA.2.2	Name and storage spaces	771
AA.2.3	Registers	771
AA.2.4	Variable Placer	771
AA.2.5	Control-flow RTLs	772
AA.2.6	Postexpander	772
AA.2.7	Expander	776
AA.2.8	Spill and reload	776
AA.2.9	Global Variables	776
AA.2.10	The target record	776
AB	runtime/	779

AC.	780
AC.1driver.nw	780
AC.2main2.nw	780
AC.3Main	780
AC.4This	781
ADChangelog	782
AEGlossary	783
Index	784
References	784

Chapter 1

Introduction

1.1 Motivations

The goal of this book is to present in full details the source code of a compiler backend. Why? Because I think you are a better programmer if you fully understand how things work under the hood.

1.2 C--

1.3 Other backend code generators

Here are other candidates that were considered but ultimately discarded:

- GNU RTL
- LLVM IR
- ML-RISC

1.4 Getting started

1.5 Requirements

1.6 About this document

1.7 Copyright

Most of this document is actually source code from Quick C-, so those parts are public domain. The prose is mine and is licensed under the GNU Free Documentation License.

1.8 Acknowledgments

Chapter 2

Overview

- 2.1 Compiler back end principles
- 2.2 qc services
- 2.3 Input C-- language
- 2.4 Output assembly language(s)
- 2.5 Code organization
- 2.6 Software architecture

Chapter 3

Core Data Structures

3.1 Source Code Locations

3.1.1 Interface

```
<srcmap.mli 19a>≡ (269h) 19b▷
  type pos          = int
  type rgn          = pos * pos
  [@@deriving show]

<srcmap.mli 19b>+≡ (269h) <19a 19c>▷
  val null          : rgn

<srcmap.mli 19c>+≡ (269h) <19b 19d>▷
  type location     = string    (* file    *)
                      * int      (* line    *)
                      * int      (* column  *)

<srcmap.mli 19d>+≡ (269h) <19c 19e>▷
  type map          [@@deriving show]

  val mk:            unit -> map (* empty map *)

<srcmap.mli 19e>+≡ (269h) <19d 19f>▷
  val sync :         map -> pos -> location -> unit
  val nl  :         map -> pos -> unit

<srcmap.mli 19f>+≡ (269h) <19e 19g>▷
  val last :         map -> location

<srcmap.mli 19g>+≡ (269h) <19f 19h>▷
  val location :     map -> pos -> location
  val dump:         map -> unit

<srcmap.mli 19h>+≡ (269h) <19g 19i>▷
  type point        = map * pos
  type region       = map * rgn

<srcmap.mli 19i>+≡ (269h) <19h>▷
  module Str:
    sig
      val point      : point -> string
      val region     : region -> string
    end
```

3.1.2 Implementation

```
<srcmap.ml 20a>≡ (269g) 20b▷
  type pos          = int
  [@@deriving show]
  type rgn          = pos * pos

  (* coupling: Ast.region printer *)
  let pp_rgn fmt _rgn =
    Format.fprintf fmt "<>"
  let show_rgn _map = "<>"

  type location      = string    (* file    *)
                        * int      (* line    *)
                        * int      (* column  *)

<srcmap.ml 20b>+≡ (269g) <20a 20c>
  let null = (0,0)

<srcmap.ml 20c>+≡ (269g) <20b 20d>
  type syncpoint     = pos * location

<srcmap.ml 20d>+≡ (269g) <20c 20e>
  type map =          { mutable points:      syncpoint array
                        ; mutable top:        int
                        ; files :             (string, string) Hashtbl.t
                        }

  let pp_map fmt _map =
    Format.fprintf fmt "<srcmap>"
  let show_map _map = "<srcmap>"

  type point          = map * pos
  type region         = map * rgn

<srcmap.ml 20e>+≡ (269g) <20d 20f>
  let size            = 2      (* small to test alloc *)
  let undefined       = (0, ("undefined", -1, -1))

<srcmap.ml 20f>+≡ (269g) <20e 20g>
  let mk () =
    { points = Array.create size undefined
    ; top    = 0
    ; files  = Hashtbl.create 17
    }

<srcmap.ml 20g>+≡ (269g) <20f 21a>

  let alloc srcmap =
    let length = Array.length srcmap.points in
    if srcmap.top < length then
      ()
    else
      let points' = Array.create length undefined in
      srcmap.points <- Array.append srcmap.points points'
```

<srcmap.ml 21a>+≡ (269g) <20g 21b>

```
let sync srcmap pos (file,line,col) =
  let _ = alloc srcmap in
  let file' = try Hashtbl.find srcmap.files file
               with Not_found -> ( Hashtbl.add srcmap.files file file
                                   ; file
                                   )
  in
  let location' = (file', line, col) in
  let top = srcmap.top in
  ( assert ((pos = 0) || (fst srcmap.points.(top-1) < pos))
    ; srcmap.points.(top) <- (pos,location')
    ; srcmap.top <- srcmap.top + 1
  )
```

<srcmap.ml 21b>+≡ (269g) <21a 21d>

```
let last map =
  ( assert (map.top > 0 && map.top <= Array.length map.points)
    ; snd map.points.(map.top-1)
  )
```

<nl specification 21c>≡

```
let nl map pos =
  let (file, line, _) = last map in
  sync map pos (file, line+1, 1)
```

<srcmap.ml 21d>+≡ (269g) <21b 21e>

```
let nl srcmap pos =
  let _ = alloc srcmap in
  let (file, line, _) = last srcmap in
  let location' = (file, line+1,1) in
  let top = srcmap.top in
  ( assert ((pos = 0) || (fst srcmap.points.(top-1) < pos))
    ; srcmap.points.(top) <- (pos,location')
    ; srcmap.top <- srcmap.top + 1
  )
```

<srcmap.ml 21e>+≡ (269g) <21d 21f>

```
let cmp x (y,_) = compare x y
```

<srcmap.ml 21f>+≡ (269g) <21e 22a>

```
let search x array length cmp =
  let rec loop left right =
    if left > right then
      ( assert (0 <= right && right < Array.length array)
        ; array.(right)
      )
    else
      let pivot = (left + right)/2 in
      let res = cmp x array.(pivot) in
      let _ = assert (0 <= pivot && pivot < Array.length array) in
      if res = 0 then
        array.(pivot)
      else if res < 0 then
        loop left (pivot-1)
      else
        loop (pivot+1) right
  in
  ( assert (length > 0)
    ; loop 0 (length-1)
  )
```

```

<srcmap.ml 22a>+≡ (269g) <21f 22b>
  let location map pos =
    let pos',(file,line,col) = search pos map.points map.top cmp in
    (file,line,pos - pos' + col)

<srcmap.ml 22b>+≡ (269g) <22a 22c>
  let dump map =
    let point (pos,(file,line,col)) =
      Printf.printf "%5d: %-32s %4d %3d\n" pos file line col
    in
    for i=0 to map.top-1 do
      point map.points.(i)
    done

<srcmap.ml 22c>+≡ (269g) <22b>
  module Str = struct
    let point (map,pos) =
      let (file,line,column) = location map pos in
      Printf.sprintf "File \"%s\", line %d, character %d" file line column

    let region (map,rgn) =
      match rgn with
      | (0,0) -> Printf.sprintf "<unknown location>"
      | (left,right) ->
        let (file1,l1,col1) = location map left in
        let (file2,l2,col2) = location map right in
        let (=$=) : string -> string -> bool = Pervasives.(=) in
        if file1 =$= file2 && l1 = l2 then
          Printf.sprintf
            "File \"%s\", line %d, characters %d-%d" file1 l1 col1 col2
        else if file1 =$= file2 then
          Printf.sprintf
            "File \"%s\", line %d, character %d - line %d, character %d"
            file1 l1 col1 l2 col2
        else
          Printf.sprintf
            "File \"%s\", line %d, character %d - file %s, line %d, character %d"
            file1 l1 col2 file2 l2 col2

  end
end

```

3.2 Abstract Syntax Definition

```

<ast.asdl 22d>≡
  module ast {
    <ast declaration 22e>
  }

```

3.2.1 The ASDL-definition

```

<ast declaration 22e>≡ (22d) 23a▷
  name      = (string)

  conv      = (string) -- names for calling conventions
  hint      = (string) -- hints for register allocation
  reg       = (string) -- names for global registers
  target    = (string)

```

```

alias_set    = (string)  -- names for aliasing assertions

size         = (int)
align        = (int)      -- power of 2
aligned       = (int)      -- power of 2

in_alias     = (string)
op           = (string)

<ast declaration 23a>+≡ (22d) <22e 23b>
  region      = (int,int) -- (* Srcmap.reg *)

<ast declaration 23b>+≡ (22d) <23a 23c>
  program     = (toplevel*)

  toplevel    = ToplevelAt    (toplevel, region)
               | Section      (name, section*)
               | TopDecl      (decl)
               | TopProcedure (proc)

<ast declaration 23c>+≡ (22d) <23b 23d>
  section     = SectionAt      (section, region)
               | Decl          (decl)
               | Procedure     (proc)
               | Datum         (datum)
               | SSpan         (expr key, expr value, section*)

<ast declaration 23d>+≡ (22d) <23c 23e>
  decl        = DeclAt         (decl, region)
               | Import        (ty?, import*)
               | Export        (ty?, export*)
               | Const         (ty?, name, expr)
               | Typedef       (ty, name*)
               | Registers     (register*)
               | Pragma        (arch*)
               | Target        (arch*)

<ast declaration 23e>+≡ (22d) <23d 23f>
  arch        = Memsize        (int)
               | ByteorderBig
               | ByteorderLittle
               | FloatRepr     (string) -- "ieee754"
               | Charset       (string) -- "latin1"
               | WordSize      (int)    -- 32 (bits)
               | PointerSize   (int)    -- 32 (bits)

  import      = (string?, name)
  export      = (name, string?)

<ast declaration 23f>+≡ (22d) <23e 24a>
  register    = (variance, hint?, ty, name, reg?)
  proc        = (conv?, name, formal*, body*, region)

  body        = BodyAt         (body, region)
               | DeclBody      (decl)
               | StmtBody      (stmt)
               | DataBody      (datum*)

```



```

memsize      = NoSize
              | DynSize
              | FixSize      (expr)

datum        = DatumAt      (datum, region)
              | Label       (name)
              | Align       (align)
              | MemDecl     (ty, memsize, init?)

⟨ast declaration 24a⟩+≡ (22d) ◁23f 24b▷
  init       = InitAt      (init, region)
              | InitExprs  (expr*)
              | InitStr    (string)
              | InitUStr   (string)

⟨ast declaration 24b⟩+≡ (22d) ◁24a 24c▷
  ty         = TyAt        (ty, region)
              | BitsTy     (size)
              | TypeSynonym (name)

⟨ast declaration 24c⟩+≡ (22d) ◁24b 24d▷
  variance   = Invariant
              | Invisible
              | Variant

formal       = (region, bare_formal)
bare_formal  = (hint?, variance, ty, name, aligned?)
actual       = (hint?, expr, aligned?)
cformal      = (region, hint?, name, aligned?)

⟨ast declaration 24d⟩+≡ (22d) ◁24c 24e▷
  flow       = FlowAt      (flow, region)
              | CutsTo     (name*)
              | UnwindsTo  (name*)
              | ReturnsTo  (name*)
              | NeverReturns
              | Aborts

mem          = AliasAt     (mem, region)
              | Reads      (name*)
              | Writes     (name*)

procann      = Flow (flow)
              | Alias (mem)

name_or_mem  = NameOrMemAt (name_or_mem, region)
              | Name       (hint?,name,aligned?)
              | Mem        (ty, expr, aligned?, in_alias*)

⟨ast declaration 24e⟩+≡ (22d) ◁24d 25a▷
  altcont    = (expr, expr)

range        = Point       (expr)
              | Range      (expr,expr)

arm          = ArmAt       (arm, region)
              | Case       (range*,body*)

```

$\langle \text{ast declaration } 25a \rangle + \equiv$ (22d) $\triangleleft 24e \ 25b \triangleright$
 guarded = (expr? guard, expr value)

$\langle \text{ast declaration } 25b \rangle + \equiv$ (22d) $\triangleleft 25a \ 25c \triangleright$
 stmt = StmtAt (stmt, region)
 | IfStmt (expr, body*, body*)
 | SwitchStmt (range?, expr, arm*)
 | LabelStmt (name)
 | ContStmt (name, cformal*)
 | SpanStmt (expr key, expr value, body*)
 | AssignStmt (name_or_mem*, guarded*)
 | CallStmt (name_or_mem*, conv?, expr, actual*, target*, procann*)
 | PrimStmt (name_or_mem*, conv?, name, actual*, flow*)
 | GotoStmt (expr, target*)
 | JumpStmt (conv?, expr, actual*, target*)
 | CutStmt (expr, actual*, flow*)
 | ReturnStmt (conv?, altcont?, actual*)
 | EmptyStmt
 | CommentStmt (string)
 | LimitcheckStmt(expr, expr?)

$\langle \text{ast declaration } 25c \rangle + \equiv$ (22d) $\triangleleft 25b \triangleright$
 expr = ExprAt (expr, region)
 | Sint (string,ty?)
 | Uint (string,ty?)
 | Float (string,ty?)
 | Char (int,ty?)
 | Fetch (name_or_mem)
 | BinOp (expr, op, expr)
 | UnOp (op, expr)
 | PrimOp (name, actual*)

3.3 Normalized Abstract-Syntax Tree

$\langle \text{exposed types}(\text{nast.nw}) \ 25d \rangle \equiv$ (51a 27b) $26a \triangleright$
 type ty = Ast.ty
 [@@deriving show]
 type exp = Ast.expr
 [@@deriving show]

 type loc = Ast.name_or_mem
 [@@deriving show]

 type 'a marked = Ast.region * 'a
 [@@deriving show]

 type name = string
 [@@deriving show]
 type kind = string
 [@@deriving show]
 type convention = string
 [@@deriving show]
 type aligned = int
 [@@deriving show]

```

(exposed types(nast.nw) 26a)+≡ (51a 27b) <25d 26b>
type cformal = Ast.region * kind * name * aligned option
[@@deriving show]
type actual = kind * exp * aligned option
[@@deriving show]

type flow = Ast.flow list
[@@deriving show]
type alias = Ast.mem list
[@@deriving show]
type range = Ast.range
[@@deriving show]

type procname = string
[@@deriving show]
type label = string
[@@deriving show]

type stmt = | S of stmt * Ast.region
| If of exp * stmt list * stmt list
| Switch of range option * exp * (range list * stmt list) list
| Label of label
| Cont of name * convention * cformal list
| Span of exp * exp * stmt list
| Assign of loc list * Ast.guarded list
| Call of loc list * convention * exp * actual list * procname list * flow * alias
| Prim of loc list * convention * name * actual list * flow
| Goto of exp * label list
| Jump of convention * exp * actual list * procname list
| Cut of convention * exp * actual list * flow
| Return of convention * (exp * exp) option * actual list
| Limitcheck of convention * exp * (exp * name) option (* (cookie,(failk,recname)) *)
[@@deriving show]

(exposed types(nast.nw) 26b)+≡ (51a 27b) <26a 26c>
type typedefn = ty * name list
[@@deriving show]
type constdefn = ty option * name * exp
[@@deriving show]
type compile_time_defns = {
  types : typedefn marked list;
  constants : constdefn marked list;
}
[@@deriving show]

(exposed types(nast.nw) 26c)+≡ (51a 27b) <26b 27a>
type proc = {
  region : Ast.region;
  cc : convention;
  name : name;
  formals : (kind * Ast.variance * ty * name * aligned option) marked list;
  locals : Ast.register marked list;
  pdecls : compile_time_defns;
  continuations : (name * convention * cformal list) marked list;
  labels : name marked list; (* code labels *)
  stackdata : datum marked list;
  code : stmt list;
}
and datum =
| Datalabel of name

```

```

| Align      of int
| ReserveMem of ty * Ast.memsize * Ast.init option (*init always none on stackdata*)
| Procedure  of proc                                (* never on stackdata *)
| SSpan      of exp * exp * datum marked list      (* never on stackdata *)
[@@deriving show]

⟨exposed types(nast.nw) 27a⟩+≡ (51a 27b) ◁26c
type section = name * datum marked list
[@@deriving show]

type t = {
  target    : Ast.arch marked list;
  imports   : (Ast.region * Ast.ty option * Ast.import list) list;
  exports   : (Ast.region * Ast.ty option * Ast.export list) list;
  globals   : Ast.register marked list;
  code_labels : name marked list list;
  udecls    : compile_time_defns;
  sections  : section list
}
[@@deriving show]

⟨nast.mli 27b⟩≡ (388b)
⟨exposed types(nast.nw) 25d⟩
val program : Ast.toplevel list -> t

```

3.4 Fenv

3.5 RTL

3.6 Nelab

3.7 Target

3.8 Asm

Chapter 4

main()

4.1 Driver

`<driver.mli 28>`≡

```
(* the lexer *)
val scan      : string -> unit

(* the parser *)
val parse     : string -> Srcmap.map * Ast.program

(* sexp-based AST printer *)
val emit_asdl : Srcmap.map * Ast.program -> unit

(* basic pretty printer *)
val pretty    : Srcmap.map * Ast.program -> Pp.doc
val print     : Pp.doc -> int -> out_channel -> unit

val elab :
  swap:bool ->
  (Rtl.rtl -> string option) ->
  Srcmap.map * Ast.program ->
  'proc Asm.assembler ->
  ('proc Fenv.Dirty.env' * 'proc Nelab.compunit) Error.error

val compile :
  Ast2ir.tgt ->
  (Ast2ir.proc -> unit) ->
  exportglobals:bool ->
  src:(Srcmap.map * Ast.program) ->
  asm:Ast2ir.proc Asm.assembler ->
  validate:bool ->
  swap:bool ->
  unit
  (* raises Error.ErrorExn *)

val version  : unit -> unit

val metrics_ok : Metrics.t -> ('a, 'b, 'c) Target.t -> bool
```

4.1.1 Caml Implementation

```
<driver.ml content 29a>≡ (780a) 29b▷  
module E = Error  
module M = Metrics  
module PA = Preast2ir  
module T = Target
```

```
<driver.ml content 29b>+≡ (780a) <29a 29c>  
let version () =  
  ( This.name      stdout  
  ; output_string stdout " "  
  ; This.version   stdout  
  ; output_string stdout  
    " (see also http://www.cminusminus.org)\n"  
  )
```

```
<driver.ml content 29c>+≡ (780a) <29b 29d>  
let scan file =  
  let fd          = try open_in file  
                    with Sys_error(msg) -> E.error msg      in  
  let finally ()  = close_in fd                              in  
  let lexbuf      = Lexing.from_channel fd                   in  
  let map         = Srcmap.mk ()                             in  
  let scanner     = Scan.scan map                            in  
  let location lb = Srcmap.location map (Lexing.lexeme_start lb) in  
  let rec loop lb =  
    match scanner lb with  
    | Parse.EOF -> ()  
    | tok      ->  
      let (file,line,col) = location lb      in  
      let tok            = Scan.tok2str tok  in  
      ( Printf.printf "%-16s %3d %2d %s\n" file line col tok  
        ; loop lb  
      )  
  in  
  ( Srcmap.sync map 0 (file,1,1)  
  ; loop lexbuf  
  ; finally ()  
  )
```

```
<driver.ml content 29d>+≡ (780a) <29c 30a>  
let parse (file:string) =  
  let fd          = try open_in file  
                    with Sys_error(msg) -> E.error msg      in  
  let finally ()  = close_in fd                              in  
  let lexbuf      = Lexing.from_channel fd                   in  
  let map         = Srcmap.mk ()                             in  
  let scanner     = Scan.scan map                            in  
  try  
    ( Srcmap.sync map 0 (file,1,1)  
    ; (map, Parse.program scanner lexbuf)  
    )  
  with  
  | Parsing.Parse_error ->  
    ( finally()  
    ; E.errorPointPrt (map, Lexing.lexeme_start lexbuf) "parse error"  
    ; E.error "parse error - compilation aborted"  
    )  
  | E.ErrorExn msg ->
```

```

    ( finally()
    ; E.errorPointPrt (map, Lexing.lexeme_start lexbuf) msg
    ; E.error "parse error - compilation aborted"
    )
| e ->
    ( finally()
    ; raise e
    )

```

```

⟨driver.ml content 30a⟩+≡ (780a) <29d 30b>
    let emit_asdl (map,ast) =
        failwith "OLD: AstUtil.sexp_wr_program ast stdout"

```

```

⟨driver.ml content 30b⟩+≡ (780a) <30a 30c>
    let pretty (map,ast) = Astpp.program ast
    let print doc width channel = Pp.ppToFile channel width doc

```

4.1.2 Driver support for the new front end

```

⟨driver.ml content 30c⟩+≡ (780a) <30b 30d>
    let metrics_ok src tgt =
        let int i = string_of_int i in
        let string s = s in
        let outcome = ref true in
        let unequal property to_string source target =
            outcome := false;
            Printf.eprintf "source code specifies %s %s, but target %s specifies %s %s\n"
                property (to_string source) tgt.T.name property (to_string target) in
        let (<>) = Pervasives.<> in
        if src.M.byteorder <> tgt.T.byteorder then
            unequal "byteorder" Rtlutil.ToUnreadableString.agg src.M.byteorder tgt.T.byteorder;
        if src.M.wordsize <> tgt.T.wordsize then
            unequal "wordsize" int src.M.wordsize tgt.T.wordsize;
        if src.M.pointersize <> tgt.T.pointersize then
            unequal "pointersize" int src.M.pointersize tgt.T.pointersize;
        if src.M.memsize <> tgt.T.memsize then
            unequal "memsize" int src.M.memsize tgt.T.memsize;
        if src.M.float <> Float.name tgt.T.float then
            unequal "float" string src.M.float (Float.name tgt.T.float);
        if src.M.charset <> tgt.T.charset then
            unequal "charset" string src.M.charset tgt.T.charset;
        !outcome

```

```

⟨driver.ml content 30d⟩+≡ (780a) <30c>
    let elab ~swap validate (map,ast) asm =
        Nelab.program ~swap validate map asm (Nast.program ast)

let compile (PA.T target) opt ~exportglobals ~src ~asm ~validate ~swap =
    (* old: failwith "TODO: pad: Driver.compile and mvalidate" *)

    let validate = if validate then Mvalidate.rtl target else (fun _ -> None) in
    let abort () = Error.error "compilation aborted because of errors" in
    let as_ok = function Error.Ok x -> x | Error.Error -> abort () in
    as_ok (Error.ematch (elab swap validate src asm) (fun (env,compunit) ->
        if metrics_ok (Fenv.Dirty.metrics env) target then
            if Fenv.Dirty.errorFlag env then
                abort ()
            else
                Ast2ir.translate (PA.T target) (Fenv.clean env) opt exportglobals compunit

```

```
else
  Error.error "metrics of source code don't match the target"))
```


Chapter 5

Parsing

5.1 Lexical Analysis

```
<scan.mli 32a>≡
  val scan : Srcmap.map -> Lexing.lexbuf -> Parse.token
  val tok2str : Parse.token -> string

<scan.mli 32b>≡
  {
    <prolog 32c>
  }

<prolog 32c>≡
  module P = Parse      (* tokens are defined here *)
  module E = Error

  let nl lexbuf map =
    let next = (Lexing.lexeme_start lexbuf) + 1      in
    Srcmap.nl map next

<prolog 32d>+≡
  let tab lexbuf map = ()
(32b) <32c 32e>

<prolog 32e>+≡
  let location lexbuf map =
    Srcmap.location map (Lexing.lexeme_start lexbuf)

  let error lexbuf msg = Error.error msg
(32b) <32d 32f>

<prolog 32f>+≡
  let get          = Lexing.lexeme
  let getchar      = Lexing.lexeme_char
  let strlen       = String.length
  let pos_start    = Lexing.lexeme_start
  let pos_end      = Lexing.lexeme_end
  let substr       = Auxfun.substr
(32b) <32e 32g>

<prolog 32g>+≡
  let keywords     = Hashtbl.create 127
  let keyword s    = Hashtbl.find keywords s
(32b) <32f 36b>

let _ = Array.iter (fun (str,tok) -> Hashtbl.add keywords str tok)
  [|("aborts"      , P.ABORTS)
   ; ("align"      , P.ALIGN)
```

```

; ("aligned"          , P.ALIGNED)
; ("also"             , P.ALSO)
; ("as"               , P.AS)
; ("big"              , P.BIG)
; ("byteorder"        , P.BYTEORDER)
; ("case"             , P.CASE)
; ("const"            , P.CONST)
; ("continuation"     , P.CONTINUATION)
; ("cut"              , P.CUT)
; ("cuts"             , P.CUTS)
; ("else"             , P.ELSE)
; ("equal"            , P.EQUAL)
; ("export"           , P.EXPORT)
; ("fails"            , P.FAILS)
; ("foreign"          , P.FOREIGN)
; ("goto"             , P.GOTO)
; ("if"               , P.IF)
; ("import"           , P.IMPORT)
; ("in"               , P.IN)
; ("invariant"        , P.INVARIANT)
; ("jump"             , P.JUMP)
; ("limitcheck"       , P.LIMITCHECK)
; ("little"           , P.LITTLE)
; ("memsize"          , P.MEMSIZE)
; ("never"            , P.NEVER)
; ("pragma"           , P.PRAGMA)
; ("register"         , P.REGISTER)
; ("reads"            , P.READS)
; ("return"           , P.RETURN)
; ("returns"          , P.RETURNS)
; ("section"          , P.SECTION)
; ("semi"             , P.SEMI)
; ("span"             , P.SPAN)
; ("stackdata"        , P.STACKDATA)
; ("switch"           , P.SWITCH)
; ("target"           , P.TARGET)
; ("targets"          , P.TARGETS)
; ("to"               , P.TO)
; ("typedef"          , P.TYPDEF)
; ("unicode"          , P.UNICODE)
; ("unwinds"          , P.UNWINDS)
; ("writes"           , P.WRITES)

; ("float"            , P.FLOATREPR)
; ("charset"          , P.CHARSET)
; ("pointersize"      , P.PTRSIZE)
; ("wordsize"         , P.WRDSIZE)

```

l]

5.1.1 Declarations for Regular Expressions

<scan.mll 33>+≡

```

let digit      = ['0'-'9']
let octal      = ['0'-'7']
let hex        = ['0'-'9' 'A'-'F' 'a'-'f']

let printable  = [' ' '~']      (* add 8bit chars *)
let alpha      = ['a'-'z' 'A'-'Z']

```

<32b 34b>

```

let misc      = ['.' ' ' '_' '$' '@']

let escape    = ['\\' '\n' '\r' '\a' '\b' '\f' '\n' '\r' '\t' ' '? ]

let sign      = ['+' '-']
let nat       = digit+
let uint      = ['1'-'9'] digit* ['u' 'U']      (* unsigned decimal *)
let zerou     = '0' ['u' 'U']                  (* unsigned decimal zero *)
let hexint    = '0' ['x' 'X'] hex+              (* hex *)
let octint    = '0' digit*                      (* octal *)
let sint      = ['1'-'9'] digit*                (* signed decimal *)
let frac      = nat '.' nat
let exp       = ['e' 'E'] sign? nat
let float     = frac exp?
              | nat exp

let cxxcomment = "//" [^ '\n']*

let id        = (alpha | misc) (alpha | misc | digit)*
let ws        = [' ' '\012' '\r'] (* SP FF CR *)
let nl        = '\n'
let tab       = '\t'

```

5.1.2 The Main Lexer

<scanner entry point 34a>≡ (39b)

```

let scan map lexbuf =
  token lexbuf map

```

<scan.mll 34b>+≡ <33 35a>

```

rule token = parse
  eof      { fun map -> P.EOF          }
| ws+      { fun map -> token lexbuf map }
| tab      { fun map -> tab lexbuf map; token lexbuf map }
| nl       { fun map -> nl lexbuf map ; token lexbuf map }
| nl ws* '#' { fun map -> line lexbuf map 0; token lexbuf map }
| ws* '#'   { fun map ->
    if Lexing.lexeme_start lexbuf = 0 then
      ( line lexbuf map 0
        ; token lexbuf map
      )
    else
      error lexbuf "illegal character"
  }

| ";"      { fun map -> P.SEMI          }
| ":"      { fun map -> P.COLON          }
| "::"     { fun map -> P.CCOLON          }
| ","      { fun map -> P.COMMA          }
| ".."     { fun map -> P.DOTDOT          }

| "("      { fun map -> P.LPAREN          }
| ")"      { fun map -> P.RPAREN          }
| "{"      { fun map -> P.LBRACE          }
| "}"      { fun map -> P.RBRACE          }
| "["      { fun map -> P.LBRACKET        }
| "]"      { fun map -> P.RBRACKET        }
| "%"      { fun map -> P.PPERCENT        }

```

```

| "="          { fun map -> P.EQUAL          }

(* infix/prefix operators *)

| "+"          { fun map -> P.PLUS(get lexbuf)    }
| "-"          { fun map -> P.MINUS(get lexbuf)   }
| "*"          { fun map -> P.STAR(get lexbuf)    }
| "/"          { fun map -> P.SLASH(get lexbuf)   }
| "%"          { fun map -> P.PERCENT(get lexbuf) }
| ">>"         { fun map -> P.GGREATER(get lexbuf) }
| "<<"         { fun map -> P.LLESS(get lexbuf)   }
| "&"          { fun map -> P.AMPERSAND(get lexbuf) }
| "|"          { fun map -> P.BAR(get lexbuf)     }
| "^"          { fun map -> P.CARET(get lexbuf)   }
| "~"          { fun map -> P.TILDE(get lexbuf)   }
| "=="         { fun map -> P.EEQ(get lexbuf)     }
| "!="         { fun map -> P.NEQ(get lexbuf)     }
| "<"          { fun map -> P.LT(get lexbuf)      }
| "<="         { fun map -> P.LEQ(get lexbuf)     }
| ">"          { fun map -> P.GT(get lexbuf)      }
| ">="         { fun map -> P.GEQ(get lexbuf)     }

| "`" id "`"   { fun map -> P.INFIXOP(substr 1 (-1) (get lexbuf)) }

| "bits" nat   { fun map ->
    let s = substr 4 0 (get lexbuf) in
    P.BITSn (int_of_string s)
  }

⟨scan.mll 35a⟩+≡
| id           { fun map ->
    let s = get lexbuf in
    let k = try keyword s with Not_found -> P.ID s in
    if k = P.PRAGMA then pragma1 lexbuf map else k
  }
| '%' id       { fun map ->
    let s = substr 1 0 (get lexbuf)
    in P.PRIMOP(s)
  }
| float        { fun map -> P.FLT (get lexbuf) }
| sint         { fun map -> P.SINT (get lexbuf) }
| uint         { fun map -> P.UINT (get lexbuf) }
| zerou        { fun map -> P.UINT (get lexbuf) }
| hexint       { fun map -> P.UINT (get lexbuf) }
| octint       { fun map -> P.UINT (get lexbuf) }

| "/*"         { fun map -> comment1 lexbuf map }
| cxxcomment   { fun map -> token lexbuf map (* skip comment *) }

| "\""         { fun map -> string lexbuf map (Buffer.create 80) }
| "'"          { fun map -> character lexbuf map }

| _            { fun map -> error lexbuf "illegal character" }

```

5.1.3 Character Literals

```

⟨scan.mll 35b⟩+≡
and character = parse

```

```

'\\'
{ fun map ->
  let i = escape lexbuf in
  if i >= 256 then
    error lexbuf "character literal too large"
  else
    character_end lexbuf map i
}

| printable
{ fun map ->
  let c = getchar lexbuf 0 in
  character_end lexbuf map (Char.code c)
}

| _
{ fun map ->
  error lexbuf
    ( "illegal character literal: "
    ^ get lexbuf
    )
}

and character_end = parse
  ""
  | _
    { fun map i -> P.CHAR(i)
      { fun map i ->
        error lexbuf
          ( "illegal character literal (too many characters): "
          ^ get lexbuf
          )
      }
    }

```

5.1.4 Escape Sequences

```

<scan.mll 36a>+≡
and escape = parse
  | escape
  | ['x' 'X'] hex
  | ['x' 'X'] hex hex
  | octal
  | octal octal
  | octal octal octal
  | _
    { decode_escape (getchar lexbuf 0) }
    { decode_hex (getchar lexbuf 1) }
    { decode_hex (getchar lexbuf 1) * 16 +
      decode_hex (getchar lexbuf 2) }
    { decode_hex (getchar lexbuf 0) }
    { decode_hex (getchar lexbuf 0) * 8 +
      decode_hex (getchar lexbuf 1) }
    { decode_hex (getchar lexbuf 0) * 64 +
      decode_hex (getchar lexbuf 1) * 8 +
      decode_hex (getchar lexbuf 2) }
    { error lexbuf "illegal escape sequence" }

  <35b 37a>

<prolog 36b>+≡
let rec decode_escape = function
  | 'a' -> 7
  | 'b' -> 8
  | 'n' -> 10
  | 'r' -> 13
  | 't' -> 9
  | '\\' -> 92
  | '\'' -> 39
  | '"' -> 34
  | '?' -> 63
  | _ -> Impossible.impossible "unknown escape sequence"

  (32b) <32g

let decode_hex c = match c with
  | 'a' .. 'f' -> Char.code c - Char.code 'a' + 10

```

```

| 'A' .. 'F' -> Char.code c - Char.code 'A' + 10
| '0' .. '9' -> Char.code c - Char.code '0'
| _          -> Impossible.impossible
              ("not a hexadecimal character: "^Char.escaped c)

```

5.1.5 Comments

```

<scan.mll 37a>+≡
and comment1 = parse
  eof
  { fun map ->
    error lexbuf "unterminated comment"
  }
  | [^ '*' '\n' '\t' '/' ]+
  { fun map ->
    comment1 lexbuf map
  }
  | nl
  { fun map ->
    nl lexbuf map; comment1 lexbuf map
  }
  | nl '#'
  { fun map ->
    line lexbuf map 0; comment1 lexbuf map
  }
  | tab
  { fun map ->
    tab lexbuf map; comment1 lexbuf map
  }
  | "*"
  { fun map ->
    comment1 lexbuf map
  }
  | "*/"
  { fun map ->
    token lexbuf map
  }
  | _
  { fun map ->
    comment1 lexbuf map
  }

```

5.1.6 Strings

```

<scan.mll 37b>+≡
and string = parse
  eof
  { fun map buf ->
    error lexbuf "unterminated string"
  }
  | "\""
  { fun map buf -> P.STR (Buffer.contents buf)
    (* we are done *)
  }
  | [^ '\000'-' \031'
    '\128'-' \255'
    '"' '\\' ]+
  { fun map buf ->
    let s = get lexbuf in
    ( Buffer.add_string buf s
    ; string lexbuf map buf
    )
  }
  | '\\\
  { fun map buf ->
    let i = escape lexbuf in
    if i >= 256 then
      error lexbuf "character literal too large"
  }

```

```

        else
            ( Buffer.add_char buf (Char.chr(i))
              ; string lexbuf map buf
            )
        }
    | _
    { fun map buf ->
      error lexbuf "illegal character in string"
    }
}

```

5.1.7 Pragmas

```

<scan.mll 38a>+≡
and pragma1 = parse
  eof
  | ws+
  | tab
  | nl
  | id
  { fun map -> P.EOF }
  { fun map -> pragma1 lexbuf map }
  { fun map -> tab lexbuf map; pragma1 lexbuf map }
  { fun map -> nl lexbuf map; pragma1 lexbuf map }
  { fun map ->
    let s = get lexbuf in
    try ( match keyword s with
      | _ -> pragma2 lexbuf map s
    )
    with Not_found -> pragma2 lexbuf map s
  }
  | _
  { fun map ->
    error lexbuf "id for pragma expected"
  }

and pragma2 = parse
  eof
  | ws+
  | tab
  | nl
  | '{'
  { fun map id ->
    error lexbuf "pragma body expected"
  }
  { fun map id -> pragma2 lexbuf map id }
  { fun map id -> tab lexbuf map; pragma2 lexbuf map id }
  { fun map id -> nl lexbuf map; pragma2 lexbuf map id }
  { fun map id ->
    pragma3 lexbuf map 0
  }
  | _
  { fun map id ->
    error lexbuf "pragma body expected"
  }

<scan.mll 38b>+≡
and pragma3 = parse
  eof
  | [^ '{' '}' '\n' '\t'
    '/' '\'', '"'] +
  { fun map level ->
    error lexbuf "unterminated pragma"
  }
  { fun map level ->
    pragma3 lexbuf map level
  }
  | nl
  { fun map level ->
    nl lexbuf map
    ; pragma3 lexbuf map level
  }
  | tab
  { fun map level ->
    tab lexbuf map; pragma3 lexbuf map level
  }
}

```

```

| '{'
    { fun map level ->
      pragma3 lexbuf map (level+1)
    }

| '}'
    { fun map level ->
      if level = 0
      then token lexbuf map
      else pragma3 lexbuf map (level-1)
    }

| cxxcomment
| "/*"
    { fun map -> pragma3 lexbuf map (* ignore *) }

| "\"
    { fun map level ->
      ignore (string lexbuf map (Buffer.create 80))
      ; pragma3 lexbuf map level
    }

| "'"
    { fun map level ->
      ignore (character lexbuf map)
      ; pragma3 lexbuf map level
    }

| _
    { fun map level ->
      pragma3 lexbuf map level
    }

```

<scan.mll 39a>+≡ <38b 39b>

```

and line = parse
  eof
    { fun map l ->
      error lexbuf "unterminated line directive"
    }

| ws+
| tab
| '"'
    { fun map l -> line lexbuf map l }

| nat
    { fun map l ->
      let buf      = Buffer.create 80 in
      let _        = string lexbuf map buf in
      let file     = Buffer.contents buf in
      let pos      = Lexing.lexeme_start lexbuf in
      let location = file, l-1, 1 in
      ( Srcmap.sync map pos location
        ; () (* return *)
      )
    }

| id
    { fun map l ->
      line lexbuf map l
    }

| _
    { fun map l ->
      error lexbuf
        "illegal character in line directive"
    }

```

<scan.mll 39b>+≡ <39a 40>


```
{ (* start of epilog *)
  <scanner entry point 34a>
```

5.1.8 Debugging the Scanner

```
<scan.mll 40>+≡ <39b
let tok2str = function
```

```
| P.ABORTS      -> "ABORTS"
| P.ALIGN       -> "ALIGN"
| P.ALIGNED     -> "ALIGNED"
| P.ALSO        -> "ALSO"
| P.AS          -> "AS"
| P.COLON       -> "COLON"
| P.CCOLON      -> "CCOLON"
| P.COMMA       -> "COMMA"
| P.CONST       -> "CONST"
| P.CONTINUATION -> "CONTINUATION"
| P.CUT         -> "CUT"
| P.CUTS        -> "CUTS"
| P.ELSE        -> "ELSE"
| P.EOF         -> "EOF"
| P.EQUAL       -> "EQUAL"
| P.EXPORT      -> "EXPORT"
| P.FAILS       -> "FAILS"
| P.FOREIGN     -> "FOREIGN"
| P.GOTO        -> "GOTO"
| P.IF          -> "IF"
| P.IMPORT      -> "IMPORT"
| P.IN          -> "IN"
| P.INVARIANT   -> "INVARIANT"
| P.JUMP        -> "JUMP"
| P.LBRACE      -> "LBRACE"
| P.LBRACKET    -> "LBRACKET"
| P.LPAREN      -> "LPAREN"
| P.NEVER       -> "NEVER"
| P.PPERCENT    -> "PPERCENT"
| P.PRAGMA      -> "PRAGMA"
| P.RBRACE      -> "RBRACE"
| P.RBRACKET    -> "RBRACKET"
| P.READS       -> "READS"
| P.REGISTER    -> "REGISTER"
| P.RETURN      -> "RETURN"
| P.RETURNS     -> "RETURNS"
| P.RPAREN      -> "RPAREN"
| P.SECTION     -> "SECTION"
| P.SEMI        -> "SEMI"
| P.SPAN        -> "SPAN"
| P.STACKDATA   -> "STACKDATA"
| P.TARGETS     -> "TARGETS"
| P.TO          -> "TO"
| P.UNICODE     -> "UNICODE"
| P.UNWINDS     -> "UNWINDS"
| P.WRITES      -> "WRITES"

| P.TYPEDEF     -> "TYPEDEF"
| P.MEMSIZE     -> "MEMSIZE"
| P.BYTEORDER   -> "BYTEORDER"
| P.LIMITCHECK  -> "LIMITCHECK"
```

```

| P.LITTLE          -> "LITTLE"
| P.BIG             -> "BIG"
| P.CASE            -> "CASE"
| P.DEFAULT         -> "DEFAULT"
| P.TARGET          -> "TARGET"
| P.DOTDOT          -> "DOTDOT"
| P.SWITCH          -> "SWITCH"

| P.WRDSIZE         -> "WORDSIZE"
| P.PTRSIZE         -> "POINTERSIZE"
| P.FLOATREPR       -> "FLOAT"
| P.CHARSET         -> "CHARSET"

| P.AMPERSAND(s)    -> "AMPERSAND(" ^ s ^ ")"
| P.BAR(s)          -> "BAR(" ^ s ^ ")"
| P.CARET(s)        -> "CARET(" ^ s ^ ")"
| P.EEQ(s)          -> "EEQ(" ^ s ^ ")"
| P.GEQ(s)          -> "GEQ(" ^ s ^ ")"
| P.GGREATER(s)     -> "GGREATER(" ^ s ^ ")"
| P.GT(s)           -> "GT(" ^ s ^ ")"
| P.LEQ(s)          -> "LEQ(" ^ s ^ ")"
| P.LLESS(s)        -> "LLESS(" ^ s ^ ")"
| P.LT(s)           -> "LT(" ^ s ^ ")"
| P.MINUS(s)        -> "MINUS(" ^ s ^ ")"
| P.NEQ(s)          -> "NEQ(" ^ s ^ ")"
| P.PERCENT(s)      -> "PERCENT(" ^ s ^ ")"
| P.PLUS(s)         -> "PLUS(" ^ s ^ ")"
| P.SLASH(s)        -> "SLASH(" ^ s ^ ")"
| P.STAR(s)         -> "STAR(" ^ s ^ ")"
| P.TILDE(s)        -> "TILDE(" ^ s ^ ")"
| P.UMINUS(s)       -> "UMINUS" ^ s ^ ")"

| P.ID(s)           -> "ID(" ^ s ^ ")"
| P.STR(s)          -> "STR(" ^ String.escaped s ^ ")"
| P.INFIXOP(s)      -> "INFIXOP(" ^ s ^ ")"
| P.PRIMOP(s)       -> "PRIMOP(" ^ s ^ ")"
| P.SINT(s)         -> "SINT(" ^ s ^ ")"
| P.UINT(s)         -> "UINT(" ^ s ^ ")"
| P.FLT(s)          -> "FLT(" ^ s ^ ")"
| P.CHAR(i)         -> "CHAR(" ^ Char.escaped (Char.chr i) ^ ")"

| P.BITSn(i)        -> "BITSn(" ^ string_of_int i ^ ")"

} (* end of epilog *)

```

5.2 Grammatical Analysis

```

⟨parse.mly 41⟩≡
%{
  module A = Ast
  module E = Error

  (* pad: syncweb does not support multi-lang; can't have special comments for
   * yacc (C) and OCaml so have to inline the ocaml code here manually
   *)
  let p () = (Parsing.symbol_start (), Parsing.symbol_end())

```

43b▷

```

let pn n    = (Parsing.rhs_start n, Parsing.rhs_end n)
let px ()   = (Parsing.symbol_start ())

let rev     = List.rev

let dep msg =
  let deprecated = Reinit.ref false in
  fun x ->
    if not (!deprecated) then
      begin
        Printf.eprintf "C-- warning: %s\n" msg;
        deprecated := true;
      end;
    x

let mkRegs v (hint,ty,regs) = List.map (fun (id,reg) -> (v, hint, ty, id, reg)) regs
let mkTypedefs ty names = List.map (fun n -> A.Typedef(n,ty)) names

let rdep = dep "Use of 'register' keyword is deprecated"
let noeqdep = dep "Hardware register name without '=' is deprecated"

let str2uint str =
  let b = Bits.U.of_string str Nativeint.size in
  try Bits.U.to_int b with Bits.Overflow ->
    Error.errorf "constant %s overflows %d-bit native integer" str (Nativeint.size-1)

let ast_mem ty exp (align, alias) = A.Mem(ty, exp, align, alias)

let uminus e =
  let rec strip = function
    | A.ExprAt(e, _) -> strip e
    | A.Sint(s, w) -> A.Sint("-" ^ s, w)
    | _ -> A.UnOp("-", e) in
  strip e

%}

⟨helper functions 42a⟩≡
let p () = (Parsing.symbol_start (), Parsing.symbol_end())
let pn n = (Parsing.rhs_start n, Parsing.rhs_end n)
let px () = (Parsing.symbol_start ())

let rev = List.rev

42b▷

⟨helper functions 42b⟩+≡
let dep msg =
  let deprecated = Reinit.ref false in
  fun x ->
    if not (!deprecated) then
      begin
        Printf.eprintf "C-- warning: %s\n" msg;
        deprecated := true;
      end;
    x

42a 42c▷

let rdep = dep "Use of 'register' keyword is deprecated"
let noeqdep = dep "Hardware register name without '=' is deprecated"

⟨helper functions 42c⟩+≡
let mkRegs v (hint,ty,regs) = List.map (fun (id,reg) -> (v, hint, ty, id, reg)) regs
let mkTypedefs ty names = List.map (fun n -> A.Typedef(n,ty)) names
42b 43a▷

```

```

(helper functions 43a)+≡ <42c 47a>
let str2uint str =
  let b = Bits.U.of_string str Nativeint.size in
  try Bits.U.to_int b with Bits.Overflow ->
    Error.errorf "constant %s overflows %d-bit native integer" str (Nativeint.size-1)

(parse.mly 43b)+≡ <41 43c>
%token ABORTS ALIGN ALIGNED ALSO AS AMPERSAND BIG BYTEORDER CASE COLON CCOLON
%token COMMA CONST CONTINUATION CUT CUTS DEFAULT DOTDOT ELSE EOF EQUAL
%token EXPORT FAILS FOREIGN GOTO IF IMPORT IN INFIXOP INVARIANT JUMP LBRACE
%token LBRACKET LIMITCHECK LITTLE LPAREN MEMSIZE NEVER PPERCENT RBRACE RBRACKET
%token READS REGISTER RETURN RETURNS RPAREN SECTION SEMI SPAN STACKDATA SWITCH
%token TARGET TARGETS TO TYPEDEF UNICODE UNWINDS WRITES
%token WRDSIZE PTRSIZE FLOATREPR CHARSET

/* pragmas */

%token PRAGMA

(parse.mly 43c)+≡ <43b 43d>
/* infix and prefix operators */

%token <string> EEQ NEQ LT LEQ GT GEQ
%token <string> BAR
%token <string> CARET
%token <string> AMPERSAND
%token <string> LLESS GGREATER
%token <string> PLUS MINUS
%token <string> PERCENT STAR SLASH
%token <string> TILDE UMINUS
%token <string> INFIXOP
%token <string> PRIMOP

%token <string> ID
%token <string> STR
%token <int> BITSn

%token <string> SINT
%token <string> UINT
%token <string> FLT
%token <int> CHAR

(parse.mly 43d)+≡ <43c 44a>
/* lvalue conflict resolution */
%right ID
%right LBRACKET

/* conflict resolution for %foo() and %foo. We ensure the '(' has
   higher precedence and gets shifted. Take the following two
   declarations out and you get a shift/reduce conflict which defaults
   to shifting. This is correct, but we also like to get rid of the
   warning. */
%nonassoc PRIMOP
%nonassoc LPAREN

%nonassoc INFIXOP
%nonassoc EEQ NEQ LT LEQ GT GEQ
%left BAR
%left CARET
%left AMPERSAND

```

```

%left      LLESS GGREATER
%left      PLUS MINUS
%left      PERCENT STAR SLASH
%right     TILDE UMINUS

%start program
%type <Ast.program>program

%%

program      :   toplevels                                { rev $1 }

toplevels    :   toplevels toplevelAt                    { $2::$1 }
              |   /**/                                    { []      }

<parse.mly 44a>+≡
toplevelAt   :   toplevel                                { A.ToplevelAt($1,p()) }
toplevel     :   SECTION STR LBRACE sections RBRACE { A.Section($2, rev $4) }
              |   procedure                              { A.TopProcedure($1) }
              |   declAt                                { A.TopDecl($1) }

<parse.mly 44b>+≡
sections     :   sections sectionAt                      { $2 :: $1 }
              |   /**/                                    { []      }

sectionAt    :   section                                { A.SectionAt($1,p()) }
section      :   procedure                              { A.Procedure($1) }
              |   datum                                { A.Datum($1) }
              |   span                                  { A.SSpan($1) }
              |   declAt                                { A.Decl($1) }

<parse.mly 44c>+≡
declAt       :   decl                                    { A.DeclAt($1,p()) }
decl         :   INVARIANT /*-----*/ registers SEMI
              |   { A.Registers(mkRegs A.Invariant $2) }
              |   /*-----*/ /*-----*/ registers SEMI
              |   { A.Registers(mkRegs A.Variant $1) }
              |   INVARIANT REGISTER registers SEMI
              |   { rdep (A.Registers(mkRegs A.Invariant $3)) }
              |   /*-----*/ REGISTER registers SEMI
              |   { rdep (A.Registers(mkRegs A.Variant $2)) }

              |   EXPORT ty exports SEMI                { A.Export(Some $2, $3) }
              |   EXPORT   exports SEMI                { A.Export(None,$2) }
              |   IMPORT ty imports SEMI                { A.Import(Some $2,$3) }
              |   IMPORT   imports SEMI                { A.Import(None,$2) }
              |   CONST   ID EQUAL exprAt SEMI          { A.Const(None,$2,$4) }
              |   CONST ty ID EQUAL exprAt SEMI         { A.Const(Some $2,$3,$5) }
              |   TYPEDEF ty names SEMI                 { A.Typedef($2,rev $3) }
              |   TARGET properties SEMI                { A.Target(rev $2) }

<parse.mly 44d>+≡
uint         :   SINT { str2uint $1 }
              |   UINT { str2uint $1 }

property     :   MEMSIZE uint                          { A.Memsize $2 }
              |   BYTEORDER BIG                        { A.ByteorderBig }
              |   BYTEORDER LITTLE                    { A.ByteorderLittle }
              |   FLOATREPR STR                        { A.FloatRepr $2 }

```

	CHARSET STR	{ A.Charset \$2 }
	PTRSIZE uint	{ A.PointerSize \$2 }
	WRDSIZE uint	{ A.WordSize \$2 }

<parse.mly 45a>+≡ <44d 45b>
 properties : properties property { \$2 :: \$1 }
 | /**/ { [] }

<parse.mly 45b>+≡ <45a 45c>
 span : SPAN sexprAt exprAt { \$2, \$3, rev \$5 }
 LBRACE sections RBRACE

<parse.mly 45c>+≡ <45b 45d>
 datumAt : datum { A.DatumAt(\$1,p()) }
 datum : ID COLON { A.Label \$1 }
 | ALIGN uint SEMI { A.Align \$2 }
 | tyAt size opt_initAt SEMI { A.MemDecl(\$1,\$2,\$3)}

 opt_initAt : initAt { Some \$1 } | { None }

<parse.mly 45d>+≡ <45c 45e>
 initAt : init { A.InitAt(\$1,p()) }
 init : LBRACE exprs RBRACE { A.InitExprs(\$2) }
 | LBRACE RBRACE { A.InitExprs([]) }
 | STR { A.InitStr(\$1) }
 | string16 { A.InitUStr(\$1) }

<parse.mly 45e>+≡ <45d 45f>
 registers : STR tyAt regs opt_comma { Some \$1, \$2, rev \$3 }
 | tyAt regs opt_comma { None , \$1, rev \$2 }

 regs : regs COMMA ID optEq STR { (\$3, Some \$5):: \$1 }
 | regs COMMA ID { (\$3, None):: \$1 }
 | ID optEq STR { [(\$1,Some \$3)] }
 | ID { [(\$1,None)] }

 optEq : { noeqdep () }
 | EQUAL { () }

<parse.mly 45f>+≡ <45e 46a>
 size : LBRACKET exprAt RBRACKET { A.FixSize(\$2) }
 | LBRACKET RBRACKET { A.DynSize }
 | { A.NoSize }

 procedure : conv ID frmls body { \$1, \$2, \$3, \$4, p() }
 | ID frmls body {None, \$1, \$2, \$3, p() }

 body : LBRACE body0 RBRACE { rev \$2 }

 body0 : body0 bodyAt { \$2 :: \$1 }
 | /**/ { [] }

 bodyAt : body1 { A.BodyAt(\$1,p()) }
 body1 : declAt { A.DeclBody \$1 }
 | stackdecl { A.DataBody \$1 }
 | stmtAt { A.StmtBody \$1 }

 frmls : LPAREN formals RPAREN { \$2 }
 | LPAREN RPAREN { [] }

```

actls      : LPAREN  actls RPAREN      { $2 }
            | LPAREN          RPAREN   { [] }

formals    : formals_ opt_comma        { rev $1  }
formals_   : formals_ COMMA formal     { $3 :: $1 }
            | formal                   { [$1]   }

actuals    : actuals_ opt_comma        { rev $1  }
actuals_   : actuals_ COMMA actual     { $3 :: $1 }
            | actual                    { [$1]   }

formal     : bare_formal                { p(), $1 }
bare_formal : opt_kind opt_invariant opt_register tyAt ID opt_aligned
            { $1, $2, $4, $5, $6 }

opt_kind   : STR { Some $1 } | { None }
opt_invariant : INVARIANT { A.Invariant } | { A.Variant }
opt_register : REGISTER { ( ) } | { ( ) }

cformal    : opt_kind ID opt_aligned    { p(), $1, $2, $3 }

cformals   : cformals_ opt_comma        { rev $1  }
            | /**/      opt_comma       { []       }
cformals_  : cformals_ COMMA cformal     { $3 :: $1 }
            | cformal                    { [$1]   }

actual     : opt_kind exprAt opt_aligned { $1, $2, $3 }

<parse.mly 46a>+≡ <45f 46b>
stackdecl  : STACKDATA LBRACE data RBRACE { rev $3 }

data       : data datumAt               { $2 :: $1 }
            | /**/                       { []       }

conv       : FOREIGN STR                 { Some $2 }

aligned    : ALIGNED uint                { $2 }

flowAt     : flow                       { A.FlowAt($1,p()) }
flow       : ALSO CUTS      TO names    { A.CutsTo($4)   }
            | ALSO UNWINDS  TO names    { A.UnwindsTo($4) }
            | ALSO RETURNS  TO names    { A.ReturnsTo($4) }
            | ALSO ABORTS                    { A.Aborts      }
            | NEVER RETURNS                    { A.NeverReturns }

flows      : flows flowAt                { $2 :: $1 }
            | /**/                       { []       }

targets    : TARGETS names               { $2 }
            | /**/                       { [] }

aliasAt    : alias                       { A.AliasAt($1,p()) }
alias      : READS  opt_names            { A.Reads ($2)   }
            | WRITES opt_names           { A.Writes($2)  }

procanns   : procanns flowAt             { A.Flow  $2 :: $1 }
            | procanns aliasAt           { A.Alias $2 :: $1 }
            | /**/                       { []       }

<parse.mly 46b>+≡ <46a 47b>

```

```

nameOrMemAt :   nameOrMem                                { A.NameOrMemAt($1,p()) }
nameOrMem    :   nameOrMem0                               { $1 }
              |   ID aligned                             { A.Name(None, $1, Some $2) }
              |   STR ID opt_aligned                     { A.Name(Some $1,$2,$3) }

⟨helper functions 47a⟩+≡                                     <43a 49b>
  let ast_mem ty exp (align, alias) = A.Mem(ty, exp, align, alias)

⟨parse.mly 47b⟩+≡                                           <46b 47c>
  nameOrMemAt0:   nameOrMem0                               { A.NameOrMemAt($1,p()) }
  nameOrMem0    :   ID                                     { A.Name(None, $1, None) }
              |   mem_type LBRACKET exprAt mem_properties RBRACKET { ast_mem $1 $3 $4 }

  mem_type : sty { $1 }
            | ID { A.TypeSynonym($1) }

  mem_properties : aligned opt_in_ids { (Some $1, $2) }
                | IN ids opt_aligned { ($3, $2) }
                | { (None, []) }

  opt_aligned : { None } | aligned { Some $1 }
  opt_in_ids  : { [] } | IN ids { $2 }
  ids         : ID { [$1] }
              | ID COMMA ids { $1 :: $3 }

  nameOrMems :   nameOrMems_ opt_comma                     { rev $1 }
  nameOrMems_ :   nameOrMems_ COMMA nameOrMemAt            { $3 :: $1 }
              |   nameOrMemAt                             { [$1] }

  tyAt      :   ty                                         { A.TyAt($1,p()) }
  ty        :   sty                                         { $1 }
              |   ID                                         { A.TypeSynonym($1) }

  sty       :   BITSn                                       { A.BitsTy($1) }

  returnto  :   LT sexprAt SLASH sexprAt GT                { Some($2,$4) }
              |   /**/                                       { None }

⟨parse.mly 47c⟩+≡                                           <47b 48a>
  stmtAt    :   stmt                                         { A.StmtAt($1,p()) }
  stmt      :   SEMI                                         { A.EmptyStmt }
              |   ID COLON                                    { A.LabelStmt $1 }
              |   SPAN sexprAt sexprAt body                 { A.SpanStmt($2,$3,$4) }
              |   nameOrMems EQUAL exprs SEMI
                { A.AssignStmt($1,List.map (fun e -> None,e) $3) }

              |   nameOrMems EQUAL conv PPERCENT ID actls flows SEMI
                { A.PrimStmt($1, $3, $5, $6, rev $7) }

              |   conv PPERCENT ID actls flows SEMI
                { A.PrimStmt([], $1, $3, $4, rev $5) }

              |   nameOrMems EQUAL conv expr actls targets procanns SEMI
                { A.CallStmt($1, $3, $4, $5, $6, rev $7) }

              |   conv expr actls targets procanns SEMI
                { A.CallStmt([], $1, $2, $3, $4, rev $5) }

              |   nameOrMems EQUAL PPERCENT ID actls flows SEMI

```



```

    { A.PrimStmt($1, None, $4, $5, rev $6)    }

|
|           PPERCENT ID actls flows SEMI
| { A.PrimStmt( [], None, $2, $3, rev $4)}

| nameOrMems EQUAL      expr actls targets procanns SEMI
| { A.CallStmt($1, None, $3, $4, $5, rev $6)}

|
|           expr actls targets procanns SEMI
| { A.CallStmt([], None, $1, $2, $3, rev $4)}

| IF exprAt body                {A.IfStmt($2,$3,[])}
| IF exprAt body ELSE body      {A.IfStmt($2,$3,$5)}
| GOTO exprAt targets SEMI      {A.GotoStmt($2,$3)}
| CONTINUATION ID LPAREN cformals RPAREN COLON
|                               {A.ContStmt($2,$4)}
| CUT TO expr actls flows SEMI {A.CutStmt($3,$4, rev $5) }
| LIMITCHECK exprAt limitfailure SEMI {A.LimitcheckStmt($2,$3) }
| conv JUMP exprAt      targets SEMI {A.JumpStmt($1 , $3, [], $4) }
|      JUMP exprAt      targets SEMI {A.JumpStmt(None,$2, [], $3) }
| conv JUMP exprAt actls targets SEMI {A.JumpStmt($1 , $3, $4, $5) }
|      JUMP exprAt actls targets SEMI {A.JumpStmt(None,$2, $3, $4) }
| conv RETURN returnto      SEMI     {A.ReturnStmt($1 , $3, []) }
|      RETURN returnto      SEMI     {A.ReturnStmt(None,$2, []) }
| conv RETURN returnto actls SEMI     {A.ReturnStmt($1 , $3, $4) }
|      RETURN returnto actls SEMI     {A.ReturnStmt(None,$2, $3) }
| SWITCH srange expr LBRACE arms RBRACE
|      {A.SwitchStmt($2,$3, rev $5)}

```

⟨parse.mly 48a⟩+≡ ◁47c 48b▷

```

limitfailure : FAILS TO exprAt {Some $3}
| /**/      {None}

```

⟨parse.mly 48b⟩+≡ ◁48a 48c▷

```

srange      : LBRACKET range RBRACKET      {Some $2}
| /**/      {None}

range       : expr                          {A.Point $1      }
| expr DOTDOT expr                          {A.Range ($1,$3)}

ranges      : ranges_                      { rev $1 }
| ranges_ COMMA                            { rev $1 }
| /**/                                        { [] }

ranges_     : ranges_ COMMA range           { $3 :: $1 }
| range                                           { [$1]      }

arms        : arms armAt                   { $2 :: $1 }
| /**/                                           { []        }

armAt       : arm                          { A.ArmAt($1,p()) }
arm         : CASE ranges COLON body         { A.Case($2,$4)  }

```

⟨parse.mly 48c⟩+≡ ◁48b 49a▷

```

sexprAt     : sexpr                        { A.ExprAt($1, p()) }
sexpr       : SINT opt_colon_ty            { A.Sint ($1, $2)  }
|           UINT opt_colon_ty              { A.Uint ($1, $2)  }
|           FLT  opt_colon_ty              { A.Float($1, $2)  }
|           CHAR opt_colon_ty              { A.Char ($1, $2)  }
|           nameOrMemAt0                   { A.Fetch $1      }

```

```

/* allow to write nullary OPs like constants */
| PRIMOP { A.PrimOp($1,[]) }
| LPAREN exprAt RPAREN { $2 }
opt_colon_ty : CCOLON tyAt { Some $2 } | { None }

⟨parse.mly 49a⟩+≡
exprAt : expr { A.ExprAt($1, p()) }
expr: | sexpr { $1 }
| PRIMOP actls { A.PrimOp($1,$2) }

| exprAt PLUS exprAt { A.BinOp($1,$2,$3) }
| exprAt MINUS exprAt { A.BinOp($1,$2,$3) }
| exprAt PERCENT exprAt { A.BinOp($1,$2,$3) }
| exprAt STAR exprAt { A.BinOp($1,$2,$3) }
| exprAt SLASH exprAt { A.BinOp($1,$2,$3) }
| exprAt LLESS exprAt { A.BinOp($1,$2,$3) }
| exprAt GGREATER exprAt { A.BinOp($1,$2,$3) }
| exprAt CARET exprAt { A.BinOp($1,$2,$3) }
| exprAt BAR exprAt { A.BinOp($1,$2,$3) }
| exprAt AMPERSAND exprAt { A.BinOp($1,$2,$3) }
| exprAt EEQ exprAt { A.BinOp($1,$2,$3) }
| exprAt NEQ exprAt { A.BinOp($1,$2,$3) }
| exprAt LT exprAt { A.BinOp($1,$2,$3) }
| exprAt LEQ exprAt { A.BinOp($1,$2,$3) }
| exprAt GEQ exprAt { A.BinOp($1,$2,$3) }
| exprAt GT exprAt { A.BinOp($1,$2,$3) }
| exprAt INFIXOP exprAt { A.PrimOp($2,[None,$1,None;
None,$3,None]) }
| TILDE exprAt %prec TILDE { A.UnOp($1,$2) }
| MINUS exprAt %prec UMINUS { uminus $2 }

exprs : exprs_ opt_comma { rev $1 }
exprs_ : exprs_ COMMA exprAt { $3 :: $1 }
| exprAt { [$1] }

⟨helper functions 49b⟩+≡
let uminus e =
  let rec strip = function
    | A.ExprAt(e, _) -> strip e
    | A.Sint(s, w) -> A.Sint("-" ^ s, w)
    | _ -> A.UnOp("-", e) in
  strip e

⟨parse.mly 49c⟩+≡
imports : imports_ opt_comma { rev $1 }
imports_ : imports_ COMMA STR AS ID { ($3,$5):: $1 }
| imports_ COMMA ID { (None, $3):: $1 }
| STR AS ID { [$1,$3] }
| ID { [None, $1] }

exports : exports_ opt_comma { rev $1 }
exports_ : exports_ COMMA ID AS STR { ($3,Some $5):: $1 }
| exports_ COMMA ID { ($3,None):: $1 }
| ID AS STR { [$1,Some $3] }
| ID { [$1,None] }

names : names_ opt_comma { rev $1 }
names_ : names_ COMMA ID { $3:: $1 }
| ID { [$1] }

```

```

opt_names      :   names { $1 }
                |   /**/ { [] }

opt_comma      :   COMMA                {}
                |   /**/                {}

string16       :   UNICODE LPAREN STR RPAREN { $3 }

```

Chapter 6

Normalizing

6.1 Implementation

```
<nast.ml 51a>≡ (388a) 51b▷
  module A = Ast
  <exposed types(nast.nw) 25d>
  let default_proc_section = "text"

<nast.ml 51b>+≡ (388a) <51a 51c>
  let id ss = ss
  let null = function [] -> true | _ :: _ -> false

  let rflatten xs =
    let add (r, xs) tail = List.fold_right (fun a t -> (r, a) :: t) xs tail in
    List.fold_right add xs []

  let cformal (r, kind, name, aligned) = (r, Auxfun.Option.get "" kind, name, aligned)
  let formal (r, (kind, variance, ty, name, aligned)) =
    (r, (Auxfun.Option.get "" kind, variance, ty, name, aligned))
  let actual (kind, name, aligned) = (Auxfun.Option.get "" kind, name, aligned)
  let convention = Auxfun.Option.get "C--"

<nast.ml 51c>+≡ (388a) <51b 51d>
  let rec add_datum r d ds = match d with
  | A.Da (d, r) -> add_datum r d ds
  | A.Label l -> (r, Datalabel l) :: ds
  | A.Align n -> (r, Align n) :: ds
  | A.MemDecl (t, s, init) -> (r, ReserveMem (t, s, init)) :: ds

  let fdata r = List.fold_right (add_datum r)

<nast.ml 51d>+≡ (388a) <51c 52a>
  type ('a) accumulation_disaster =
  { impls : (A.region * A.ty option * A.import list) list
  ; exps : (A.region * A.ty option * A.export list) list
  ; lbls : 'a
  ; ks : (Ast.region * (A.name * convention * (A.region * A.hint * A.name * aligned option) list)) list
  ; consts : (A.region * (A.ty option * A.name * A.expr)) list
  ; tys : (A.region * (A.ty * A.name list)) list
  ; regs : (A.region * A.register list) list
  ; archs : (A.region * A.arch list) list
  ; data : ((A.region * datum) list)
  }
```

```

let rec decl r accums d k = match d with
| A.D(x,r)    -> decl r accums x k
| A.Typedef d  -> k {accums with tys = ((r,d) :: accums.tys)}
| A.Import (t,ii) -> k {accums with imps = ((r,t,ii)::accums.imps)}
| A.Export (t,ee) -> k {accums with exps = ((r,t,ee)::accums.exps)}
| A.Const d     -> k {accums with consts = ((r,d) :: accums.consts)}
| A.Registers rs -> k {accums with regs = ((r, rs) :: accums.regs)}
| A.Target t    -> k {accums with archs = ((r, t) :: accums.archs)}
| A.Pragma      -> k accums

⟨nast.ml 52a⟩+≡ (388a) <51d 52b>
let rec kmap f cons r accums xs k = match xs with
| []          -> k [] accums
| x :: xs ->
    kmap f cons r accums xs
    (fun xs accums -> f r accums x (fun x accums -> k (cons x xs) accums))

let kmap_none f r accums xs k =
    let xk f r accums x k = f r accums x (k []) in
    kmap (xk f) (fun _ _ -> []) r accums xs (fun _ -> k)

⟨nast.ml 52b⟩+≡ (388a) <52a 53a>
let rec body r accums b k = match b with
| A.B(b,r) -> body r accums b k
| A.DeclBody d -> decl r accums d (k id)
| A.StmtBody s -> (
    ⟨match stmt s at r and continue with k 52c⟩
  )
| A.DataBody ds -> k id {accums with data = (fdata r ds accums.data)}
and bodies r = kmap body (fun add ss -> add ss) r

⟨match stmt s at r and continue with k 52c⟩≡ (52b)
let rec stmt r s k =
    let cons s ss = S (s, r) :: ss in
    let atomic s = k (cons s) accums in
    match s with
    | A.S(s,r) -> stmt r s k
    | A.IfStmt (c,b1,b2) ->
        bodies r accums b2
        (fun ss2 accums ->
            bodies r accums b1 (fun ss1 accums -> k (cons(If(c,ss1,ss2))) accums))
    | A.SwitchStmt (range,e,aa) ->
        arms r accums aa
        (fun aa accums ->
            let stmt = cons (Switch (range, e, aa)) in
            k stmt accums)
    | A.LabelStmt l ->
        k (cons (Label l)) {accums with lbls = ((r, l) :: accums.lbls)}
    | A.ContStmt (n, formals) ->
        let formals = List.map cformal formals in
        let cc      = convention None in
        let stmt    = cons (Cont (n, cc, formals)) in
        k stmt {accums with ks = ((r, (n,cc,formals)) :: accums.ks)}
    | A.SpanStmt(key,v,bs) ->
        bodies r accums bs (fun ss accums -> k (cons (Span (key,v,ss))) accums)
    | A.AssignStmt (ls, rs) ->
        atomic (Assign (ls, rs))
    | A.CallStmt (ls, cc, p, a's, tgts, flows) ->
        let add a (flows, mems) = match a with
        | A.Flow f -> (f :: flows, mems)

```

```

| A.Alias m -> (flows, m :: mems) in
  let flows, mems = List.fold_right add flows ([], []) in
  atomic (Call (ls, convention cc, p, List.map actual a's, tgts, flows, mems))
| A.PrimStmt (ls, cc, p, a's, flows) ->
  atomic (Prim (ls, convention cc, p, List.map actual a's, flows))
| A.GotoStmt (e, tgts) ->
  atomic (Goto (e, tgts))
| A.JumpStmt (cc, p, a's, tgts) ->
  atomic (Jump (convention cc, p, List.map actual a's, tgts))
| A.CutStmt (e, a's, flows) ->
  atomic (Cut (convention None, e, List.map actual a's, flows))
| A.ReturnStmt (cc, alt, a's) ->
  atomic (Return (convention cc, alt, List.map actual a's))
| A.EmptyStmt
| A.CommentStmt _ -> k id accums
| A.LimitcheckStmt (cookie, cont) ->
  let cc = convention None in
  match cont with
  | None -> atomic (Limitcheck (cc, cookie, None))
  | Some cont ->
    (* construct continuation for recovery *)
    let formals = [] in
    let name = Idgen.label "overflow-recovery continuation" in
    let stmt = cons (Limitcheck (cc, cookie, Some (cont, name))) in
    k stmt {accums with ks = ((r, (name,cc,formals)) :: accums.ks)} in
stmt r s k

```

$\langle \text{nast.ml } 53a \rangle + \equiv \quad (388a) \triangleleft 52b \ 53b \triangleright$

```

and arm r accums a k = match a with
| A.ArmAt (a,r) -> arm r accums a k
| A.Case (ranges,bs) ->
  bodies r accums bs (fun stmts accums -> k ((ranges, stmts)) accums)
and arms r = kmap arm (fun x y -> x::y) r

```

$\langle \text{nast.ml } 53b \rangle + \equiv \quad (388a) \triangleleft 53a \ 54 \triangleright$

```

let rec section r accums s k = match s with
| A.Sec (s,r) ->
  section r accums s k
| A.Decl d ->
  decl r accums d k
| A.SSpan(key, v, ss) ->
  sections r {accums with data = []} ss
  (fun saccums ->
    let data' = (r, SSpan(key,v, saccums.data)) :: accums.data in
    k {saccums with data = data'})
| A.Datum d ->
  k {accums with data = (add_datum r d accums.data)}
| A.Procedure(cc,p,fs,bs,r) ->
  bodies r { imps = accums.imps ; exps = accums.exps
            ; lbls = [] ; ks = [] ; consts = [] ; tys = []
            ; regs = [] ; archs = accums.archs ; data = [] }
  bs
(fun ss paccums ->
  let cdecls = { constants = paccums.consts; types = paccums.tys } in
  let p = { cc = convention cc; name = p; formals = List.map formal fs; code = ss;
            continuations = paccums.ks; labels = paccums.lbls; region = r;
            locals = rflatten paccums.regs; pdecls = cdecls;
            stackdata = paccums.data; } in
  k {accums with imps = paccums.imps;
      exps = paccums.exps;

```

```

        lbls = paccums.lbls :: accums.lbls;
        archs = paccums.archs;
        data = (r, Procedure p)::accums.data})
and sections r = kmap_none section r

<nast.ml 54>+≡ (388a) <53b
let rec toplevel r accums t k =
  let cons s ss = (s, r) :: ss in
  match t with
  | A.Top(t,r) ->
    toplevel r accums t k
  | A.Section(name, ss) ->
    sections r {accums with data = []} ss
    (fun saccums -> k (cons (name, saccums.data))
      {saccums with data = accums.data})
  | A.TopDecl d ->
    decl r accums d (k id)
  | A.TopProcedure p ->
    let t = A.Section(default_proc_section, [A.Procedure p]) in
    toplevel r accums t k

let program ts =
  let checknull what l =
    if not (null l) then Impossible.impossible ("some toplevel " ^ what) in
  kmap toplevel (fun add t -> add t)
  Srcmap.null
  { imps = [] ; exps = [] ; lbls = [] ; ks = [] ; consts = []
  ; tys = [] ; regs = [] ; archs = [] ; data = [] }
  ts
  (fun ss saccums ->
    let _ = checknull "data" saccums.data in
    let _ = checknull "continuations" saccums.ks in
    { target = rflatten saccums.archs
    ; imports = saccums.imps; exports = saccums.exps
    ; code_labels = saccums.lbls
    ; globals = rflatten saccums.regs
    ; sections = List.map fst ss
    ; udecls = { types = saccums.tys; constants = saccums.consts }
    })

```

Chapter 7

IR

Chapter 8

Elaborating

Chapter 9

Control-flow Graph

9.1 Dataflow pass for available expressions

For dependency reasons, this module has to be split from the Avail module.

```
<availpass.mli 57a>≡  
  val analysis : Avail.t Dataflow.F.analysis
```

9.1.1 Implementation

```
<availpass.ml 57b>≡  
  module D = Dataflow  
  module G = Zipcfg  
  module GR = Zipcfg.Rep  
  module P = Property  
  let matcher = { P.embed = (fun a -> P.Avail a);  
                  P.project = (function P.Avail a -> Some a | _ -> None);  
                  P.is = (function P.Avail a -> true | _ -> false);  
                }  
  
  let prop = Unique.Prop.prop matcher  
  
  let _ = Debug.register "avail-changed" "show avail exps that change"  
  
  let debug_smaller ~old ~new' =  
    let b = Avail.smaller ~old ~new' in  
    if b then  
      Printf.eprintf "*** Available expressions have changed from %s\n** to new avail%s\n"  
        (Avail.to_string old) (Avail.to_string new');  
    b  
  
  let debug_join a a' =  
    let a'' = Avail.join a a' in  
    Printf.eprintf "*** Joined avails are %s\n" (Avail.to_string a'');  
    a''  
  
  let smaller = if Debug.on "avail-changed" then debug_smaller else Avail.smaller  
  let join     = if Debug.on "avail-changed" then debug_join     else Avail.join  
  
  let fact = {  
    D.fact_name = "available expressions";  
    D.init_info = Avail.unknown;  
    D.add_info = join;  
    D.changed = smaller;  
    D.prop = prop;
```

```

}

let last_outs in' l set =
  let out = Avail.forward (GR.last_instr l) in' in
  let set_edge e =
    let _out = Avail.invalidate e.G.defs out in
    let _out = Avail.invalidate e.G.kills out in
    let out = Avail.unknown (* paranoia *) in
    set (fst e.G.node) out in
  G.iter_outedges l ~noflow:(fun u -> set u out) ~flow:set_edge

let comp = {
  D.F.name = "avail";
  D.F.middle_out = (fun a m -> Avail.forward (GR.mid_instr m) a);
  D.F.last_outs = last_outs;
}

let analysis = fact, comp

```

Chapter 10

Liveness

10.1 Liveness Analysis

Liveness analysis determines what locations are live on entry to and exit from every node in a flow graph. Liveness is actually an edge property: a location x is live on an edge e iff that edge flows to a use of x without crossing a definition or kill of x . Since our edges are simple pointers, we put liveness information at the nodes, in the form of *live-in sets*. The liveness of an edge is the live-in set of that edge's head.

10.1.1 Interface

Here's the new stuff; what's below is old stuff.

```
<live.mli 59a>≡
  type uid = Zipcfg.uid
  type liveset = Register.SetX.t

  val live_in      : liveset Dataflow.B.analysis (* live in to each block *)
  val live_in_last  : Zipcfg.Rep.last -> liveset
  val live_in_middle : liveset -> Zipcfg.Rep.middle -> liveset
    (* map live out to live in *)

  val live_out_last : Zipcfg.Rep.last -> liveset
```

10.2 Implementation of liveness

```
<live.ml 59b>≡
  <live.ml 59c>
```

Following a suggestion of John Dias, we cache live-out information at each node. Although our liveness analysis is performed over the type `Register.x`, we store only live_out sets of type `Register.t`. It is important to perform the liveness at the finer granularity of slices to appropriately capture liveness. For example, consider the code

```
%a1 = e
...
%ah = e'
{use of %a1}
```

If we calculate liveness at the register level, we will think the between the assignments. But if we perform liveness at the slice level, then promote the slices to registers, we will see that

```
<live.ml 59c>≡ (59b) 60a▷
```

```

module D    = Dataflow
module G    = Zipcfg
module GR   = Zipcfg.Rep
module P    = Property
module RSX  = Register.SetX
module R    = Rtl

```

```

type uid = Zipcfg.uid
type liveset = Register.SetX.t

```

```

⟨live.ml 60a⟩+≡ (59b) <59c 60b>
let irwk = Rtlutil.ReadWriteKill.sets

```

N.B. `irwk` stands for “instruction read write kill.”

We use these set operations. When we use `--` to subtract a register r , we have to be sure to remove not only r but also any slices completely contained in r . Furthermore, we expect it never to happen that there is a slice that overlaps with r but is not completely contained in it. But we don’t check.

```

⟨live.ml 60b⟩+≡ (59b) <60a 60c>
let diff live killed =
  let is_killed r = RSX.exists (fun r' -> Register.contains r' r) killed in
  RSX.filter (fun r -> not (is_killed r)) live

```

The `--*` operation expresses our expectation that a killed register should never be live.

```

⟨live.ml 60c⟩+≡ (59b) <60b 60e>
let ( ++ ) = RSX.union
let ( -- ) = diff
let ( --* ) live killed =
  let is_killed r = RSX.exists (fun r' -> Register.contains r' r) killed in
  if RSX.exists is_killed live then
    (
      ⟨complain of live variables in killed set 60d⟩
      ; live--killed
    )
    (*Impossible.impossible "live variable is in killed set"*)
  else
    live

```

```

⟨complain of live variables in killed set 60d⟩≡ (60c)
Printf.eprintf "Live vars %s in killed set { %s }\n"
  (RSX.to_string (RSX.filter is_killed live)) (RSX.to_string killed)

```

The `live_out` function uses the cache. The `new_live_out` function computes the live-out set afresh; it is used to fill the cache. It works by taking the union over outedges of the live-in sets of its successors, also accounting for any defs or kills that may be on the outedges. Finally, the `live_in` function starts with the cached live-out set and accounts for any registers used, defined, and killed by the instruction in the node.

$$live_{in} = (live_{out} \setminus defined \setminus killed) \cup used.$$

```

⟨live.ml 60e⟩+≡ (59b) <60c 61a>
let matcher = { P.embed = (fun a -> P.Live_in a);
  P.project = (function P.Live_in a -> Some a | _ -> None);
  P.is = (function P.Live_in a -> true | _ -> false);
}

let prop = Unique.Prop.prop matcher
let get = Unique.Prop.get prop

let live_in_middle out mid =
  let uses, defs, kills = irwk (GR.mid_instr mid) in

```

```

out -- defs --* kills ++ uses

let live_out_last last =
  G.union_over_outedges last get
  (fun {G.node = (u, l); G.defs = d; G.kills = k} -> get u -- d --* k)

let live_in_last last =
  let uses, defs, kills = irwk (GR.last_instr last) in
  let live = live_out_last last -- defs --* kills ++ uses in
  G.add_live_spansl last (G.add_inedge_uses last live)

let live_in_first out first =
  G.add_live_spansf first out

```

N.B. we could operate more efficiently if, instead of using `irwk` to build sets, we simply folded over the proper functions directly.

```

⟨live.ml 61a⟩+≡ (59b) <60e
let live_in =
{ D.fact_name = "live vars";
  D.add_info   = (++);
  D.changed    = (fun ~old ~new' -> RSX.cardinal new' > RSX.cardinal old);
  D.init_info  = RSX.empty;
  D.prop       = prop
},
{ D.B.name      = "liveness";
  D.B.last_in   = live_in_last;
  D.B.middle_in = live_in_middle;
  D.B.first_in  = live_in_first;
}

```

10.3 Dead-assignment elimination

```

⟨dead.mli 61b⟩≡
val elim_assignments : Live.liveset Dataflow.B.transformation

⟨dead.ml 61c⟩≡
⟨dead.ml 61d⟩

⟨dead.ml 61d⟩≡ (61c) 61e▷
module D = Dataflow.B
module G = Zipcfg
module GR = Zipcfg.Rep
module R = Register
module RS = Register.SetX
module RP = Rtl.Private
module S = Rtlutil.ToString

⟨dead.ml 61e⟩+≡ (61c) <61d 62a▷
let () = Debug.register "dead" "dead-assignment elimination"
let debug fmt = Debug.eprintf "dead" fmt
let epr = Printf.eprintf

⟨Dead utilities 62e⟩

```

We must not eliminate potentially dead stack adjustments!

```

⟨dead.ml 62a⟩+≡ (61c) <61e 62d>
let last_in l = None
let middle_in live m = match m with
| GR.Instruction rtl -> (* don't eliminate dead stack adjustments or assertions *)
  let is_dead r =
    let r = Register.Reg r in
    not (RS.exists (fun r' -> Register.contains r r') live) in
  all_regs_assigned rtl
  (fun regs ->
    if List.for_all is_dead regs then
      (
        ⟨announce dead elimination 62b⟩
        ; Some G.empty)
    else
      (
        ⟨announce dead non-elimination 62c⟩
        ; None))
  (fun () -> None)
| _ -> None

let first_in live f = None

⟨announce dead elimination 62b⟩≡ (62a)
(if Debug.on "dead" then
begin
  epr "dead: eliminating dead assignment %s\n" (S.rtl rtl);
  epr "dead: assigned to registers %s\n" (String.concat ", " (List.map S.reg regs));
  epr "dead: live = { %s }\n" (RS.to_string live);
end)

⟨announce dead non-elimination 62c⟩≡ (62a)
(if Debug.on "dead" then
begin
  epr "dead: something is live out\n";
  epr "dead: live = { %s }\n" (RS.to_string live);
end)

⟨dead.ml 62d⟩+≡ (61c) <62a
let elim_assignments =
{ D.name = "dead";
  D.last_in = last_in; D.middle_in = middle_in; D.first_in = first_in; }

```

CPS to find if an RTL assigns only registers. We're making pessimistic assumptions, because if we do see a slice, we don't want to have to check all the may-alias relations. THIS IS PROBABLY A BAD IDEA—WE'LL HAVE TO BITE THE MAY-ALIAS BULLET SOONER OR LATER.

```

⟨Dead utilities 62e⟩≡ (61e)
let all_regs_assigned rtl succ fail =
let fail () = Debug.eprintf "dead" "assigned to non-register\n"; fail () in
let RP.Rtl effs = Rtl.Dn.rtl rtl in
let rec effects regs = function
| [] -> succ regs
| (_, RP.Store(RP.Reg r, _, _)) :: es -> effects (r::regs) es
| (_, RP.Store((RP.Mem _ | RP.Var _ | RP.Global _), _, _)) :: es -> fail ()
| (_, RP.Kill _) :: es -> effects regs es
| (g, RP.Store(RP.Slice(_, _, l), r, w)) :: es ->
  effects regs ((g, RP.Store(l, r, w)) :: es) in
effects [] effs

```

Chapter 11

Register Allocators

11.1 A register allocator written as a dataflow algorithm

The basic idea is to write a register allocator as a pass in our dataflow framework. As it turns out, we can use multiple passes to gather information used to improve the quality of the generated code.

Status:

- Output code isn't terrible could be improved.
- The attempt to track copies through registers doesn't seem to add much benefit when the peephole optimizer is activated. It's probably worth throwing that mess away.

Ideas that interest me:

- After computing varmaps, incorporate the varmaps into the allocation preferences.
- Spilling by moving to an available register.
- Pre-spilling. If you know a temp will be spilled, maybe you can do better by spilling it early? (By "do better", I mean either spill the temp in a basic block that is executed less frequently or end up with a better register allocation because other temps are allocated with the knowledge that the spilled register will be empty.)

Only one function is exported: a register allocator.

```
<flowra.mli 63a>≡  
  val ralloc : 'a -> Ast2ir.proc -> Ast2ir.proc * bool
```

11.1.1 Useful boilerplate and a few utilities

```
<flowra.ml 63b>≡  
  <flowra.ml 63c>
```

Lots of boilerplate, none of it new:

```
<flowra.ml 63c>≡ (63b) 64a▷  
  module AU = Automatonutil  
  module C  = Cfgutil  
  module D  = Dataflow  
  module Dn = Rtl.Dn  
  module F  = Dataflow.F  
  module G  = Zipcfg  
  module GR = Zipcfg.Rep  
  module P  = Property
```



```

module R    = Register
module RM   = Register.Map
module RP   = Rtl.Private
module RS   = Register.Set
module RU   = Rtlutil
module T    = Target
module U    = Unique
module UP   = Unique.Prop
module VM   = VarMap

let () = Debug.register "ralloc-cfg" "show the CFG before and after register allocation"
let eprintf x      = Debug.eprintf "flowra" x
let print_vm x y = if Debug.on "flowra" then VM.print' x y else ()
let imposs        = Impossible.impossible
let impossf fmt   = Printf.kprintf Impossible.impossible fmt
let ( ++ ) = RS.union
let ( -- ) = RS.diff
let isNone = function None -> true | Some _ -> false
let isNil  = function [] -> true | _ -> false
let pr_reg = RU.ToString.reg
let pr_reg_lst s l = eprintf "%s [%s]\n" s (String.concat ", " (List.map pr_reg l))

```

And a module for sets of integer pairs:

```

⟨flowra.ml 64a⟩+≡ (63b) ◁63c 64b▷
module IPS = Set.Make (struct (* Int Pair Set *)
  type t = (int * int)
  let compare (i1, i2) (i1', i2') =
    match compare i1 i1' with 0 -> compare i2 i2' | n -> n
end)

```

We use four different types of dataflow information:

- Maps of distances to the next uses and defs for each temp.
- A register allocation map from temps to the registers they have been assigned.
- Live variable sets.
- Allocation preferences: sometimes we think we can improve the register allocation by assigning a temp to a specific register. We use the *preferences map* to note these preferred assignments.

```

⟨flowra.ml 64b⟩+≡ (63b) ◁64a 65a▷
type 'a propRec = {get : U.uid -> 'a; set : U.uid -> 'a -> unit}
let propRecord (embed, project) =
  let matcher =
    { P.embed = embed; P.project = project;
      P.is = (fun x -> match project x with Some _ -> true | _ -> false) } in
  let prop = U.Prop.prop matcher in
  {get = U.Prop.get prop; set = U.Prop.set prop}
let dists = propRecord ((fun a -> P.Distances a),
  (function P.Distances a -> Some a | _ -> None))
let vmr = propRecord ((fun a -> P.VarOutMap' a),
  (function P.VarOutMap' a -> Some a | _ -> None))
let lir = propRecord ((fun a -> P.Live_in a),
  (function P.Live_in a -> Some a | _ -> None))
let pir = propRecord ((fun a -> P.AllocPrefs a),
  (function P.AllocPrefs a -> Some a | _ -> None))

```

To ensure we allocate a temp to a valid register, we look up the set of available registers using the temp's space as an index.

```
<flowra.ml 65a>+≡ (63b) <64b 65b>
module SM = Map.Make (struct type t = Rtl.space let compare = RU.Compare.space end)
(* Premature(?) optimization. *)
let space_regmap = ref SM.empty
let get_regs_for_space tgt regs s =
  try SM.find s (!space_regmap)
  with Not_found -> let r = List.filter (T.fits tgt s) regs in
    (space_regmap := SM.add s r (!space_regmap); r)
```

For middle and last nodes, we need to find the sets of registers that are defined and used.

```
<flowra.ml 65b>+≡ (63b) <65a 65d>
let p = R.promote_rxset
let irwk = Rtlutil.ReadWriteKill.sets_promote
let defs_uses_m middle =
  let uses, defs, kills = irwk (GR.mid_instr middle) in
  (defs ++ kills, uses)
let defs_uses_l last =
  let uses, defs, kills = irwk (GR.last_instr last) in
  let defs = defs ++ kills in
  (p (G.union_over_outedges last (fun _ -> R.rset_to_rxset defs)
    (fun {G.node = n'; G.defs = d; G.kills = k} ->
      (R.rset_to_rxset (defs ++ p d ++ p k))))),
  p (G.add_inedge_uses last (R.rset_to_rxset uses)))
```

For each temp that we spill, we allocate a spill slot on the stack; the `spillMap` maps each temp to its spill slot. Of course, this mutable data is defined in the `ralloc` function so that it is reallocated each time we perform register allocation on a new procedure.

```
<set up the map from temps to spill locations 65c>≡ (65d)
let spillMap = ref RM.empty in
let get_spill_loc ((_, _, ms), _, c as temp) =
  try RM.find temp (!spillMap)
  with Not_found ->
    let l = Automaton.allocate proc.priv ~width:(Cell.to_width ms c)
      ~kind:"" ~align:1 in
    spillMap := RM.add temp l (!spillMap);
    l in
```

11.1.2 The big picture

The main function has three main parts: setting up a few more helper functions (`is_tmp`, `is_vfp`, ...), computing related dataflow facts (distances to uses and defs, allocation preferences), and finally running the register allocator's dataflow pass and updating the spans.

The register allocator's dataflow pass is unique because an analysis of the graph after it is rewritten with the allocated registers will produce different dataflow results (without any temps). Therefore, after the register assignments are computed, we have to use a rewriting phase that doesn't check for convergence.

```
<flowra.ml 65d>+≡ (63b) <65b>
<type definitions used internally by the register allocator 66a>
let ralloc v (g, (Procedure.{cc = cc; target = Preast2ir.T tgt} as proc)) =
  let debug = Debug.on "flowra" in
  let m = (proc, tgt.T.machine, proc.exp_of_lbl) in
  let regs = RS.elements (cc.Call.volregs ++ cc.Call.pre_nvregs) in
  let is_tmp (s,_,_) = T.is_tmp tgt s in
  let is_vfp t = Vfp.is_vfp (Dn.loc (Rtl.reg t)) in
  if debug then (eprintf "ralloc begin\n"; Cfgutil.print_cfg g);
  (* claude: QCDEBUG=ralloc-cfg is a quieter before/after showcase of just
```

```

* the CFG (no per-instruction dataflow trace) - see tests/phases/regalloc/. *)
if Debug.on "ralloc-cfg" then
  (Debug.eprintf "ralloc-cfg" "BEFORE register allocation:\n"; Cfgutil.print_cfg g);
  <set up the map from temps to spill locations 65c>
  <compute distances to uses 75a>
  <compute allocation prefs 78>
  <define functions for manipulating related dataflow info 66b>
  <define code for rewriting spans 74a>
  <set up dataflow analysis for register allocation 67c>
  F.run_anal' analysis entry_fact g;
  let g, b = F.modify_solved' (F.a_t' analysis tx) ~entry_fact g in
  <update entry label spans 74b>
  if debug then (eprintf "ralloc begin\n"; Cfgutil.print_cfg g);
  if Debug.on "ralloc-cfg" then
    (Debug.eprintf "ralloc-cfg" "AFTER register allocation:\n"; Cfgutil.print_cfg g);
  (g, proc), b

```

11.1.3 Gathering related dataflow information

As mentioned earlier, we use three related dataflow facts: liveness, distances to uses and defs, and allocation preferences.

```

<type definitions used internally by the register allocator 66a>≡ (65d)
type info = {liveset : RS.t;
             dists : int * (int * VM.def_dist RM.t * VM.use_dist RM.t);
             prefs : Register.t Register.Map.t * Register.t list Register.Map.t}

```

These dataflow facts are computed before the register allocator is run. To access these facts from the register allocator, we define a function to fetch all the precomputed dataflow information for the basic block with id uid, and we gather a list of the dataflow facts for each instruction in the basic block:

```

<define functions for manipulating related dataflow info 66b>≡ (65d) 66c>
let get_info g uid =
  let (block, _) = G.openz (G.focus uid g) in
  let (f, _) = GR.goto_start block in
  let (head, last) = GR.goto_end block in
  let rec get_info (liveout, ud, prefs) rst = function
    | GR.First f ->
      {liveset = p (G.add_live_spansf f liveout); dists = ud_first_in ud f;
       prefs = prefs_first_in prefs f} :: rst
    | GR.Head (h, m) ->
      get_info (Live.live_in_middle liveout m, ud_middle_in ud m,
               prefs_middle_in prefs m)
      ({liveset = p liveout; dists = ud; prefs = prefs} :: rst) h in
  let lo = get_info (Live.live_in_last last, ud_last_in last, prefs_last_in last)
    [{liveset = p (G.add_live_spansl last (Live.live_out_last last));
     dists = ud_out_last last; prefs = prefs_last_out last}] head in
  let blockname = match f with GR.Entry -> "<entry>" | GR.Label ((_, l), _, _) -> l in
  eprintf "For block %s, liveouts are:\n" blockname;
  if debug then List.iter (fun {liveset=ls} -> eprintf " %s\n" (RS.to_string ls)) lo;
  eprintf "For block %s, use distances are:\n" blockname;
  if debug then List.iter (fun {dists=ud} -> prmaps ud) lo;
  lo in

```

Currently, our compiler has a flaw in the handling of parameters to unwind continuations: it doesn't tell us when they are defined. The result is that the live range of such a parameter is extended to the nearest reaching definition, or to the entry of the cfg if no such definition exists. But they might not show up in the use or def sets of any instructions, so instead, we change the variable map for the entry block to assign those poor bastards a space in memory.

```

<define functions for manipulating related dataflow info 66c>+≡ (65d) <66b 67a>

```

```

let (_, entry_vm as entry_fact) =
  match get_info g GR.entry_uid with
  | {liveset=li}::_ as liveouts ->
    (liveouts,
     RS.fold (fun t vm -> if is_tmp t then VM.add_mem' t vm else vm) li VM.empty)
  | _ -> imposs "no livesets for entry block" in

```

Invariants are good. The variable map should contain information about all the live temps and nothing else.

<define functions for manipulating related dataflow info 67a>+≡ (65d) <66c 67b>

```

let check_vm vm li =
  if Debug.on "flowra" then
    (print_vm "check_vm" vm;
     eprintf "li: %s\n" (RS.to_string li);
     if not (VM.is_empty' vm) then
       (VM.fold' (fun t _ () -> assert (not (is_tmp t) || RS.mem t li))
        (fun t _ () -> assert (not (is_tmp t) || RS.mem t li)) vm ();
       try RS.iter (fun t -> if is_tmp t then ignore (VM.var_locs'' vm t)) li
        with Not_found -> assert false)) in

```

Finally, a few functions we use to fetch the dataflow information computed by the other dataflow passes (as well as the computed variable map) and store the variable map.

<define functions for manipulating related dataflow info 67b>+≡ (65d) <67a

```

let get_vm uid = if U.eq uid GR.entry_uid then entry_vm else vmr.get uid in
let get g uid = (get_info g uid, get_vm uid) in
let set uid (_, vm) =
  print_vm "set prev: " (try get_vm uid with Not_found -> VM.empty);
  print_vm "set vm: " vm;
  check_vm vm (R.promote_rxset (lir.get uid));
  vmr.set uid vm in

```

11.1.4 Allocating registers in an RTL

`allocate` is responsible for finding a register allocation (represented by a variable map) for an `rtl`, given an input variable map representing the register allocation at predecessors in the `cfg`. The helper function `alloc_one` is responsible for the hard work, allocating one temp at a time. The register allocation takes place in about four steps:

1. If a hardware register is defined in the `rtl`, then any temps in that hardware register must be spilled.
2. Uses are allocated; expire (remove from the variable map) any use that is not live out.
3. Defs are allocated; expire any def that is not live out.
4. Reconcile the variable map with its successors (necessary only for last instructions, explained below).

<set up dataflow analysis for register allocation 67c>≡ (65d) 71c>

```

let allocate reconcile_succs upd_midmap is_last vm
  ({liveset=li},
   {liveset=lo; dists=(_, (_, dm, um) as maps); prefs = (pmap,_) as prefs})
  rtl (defs, uses) =
  check_vm vm li; eprintf "lo: %s\n" (RS.to_string lo); prmaps maps; prprefs prefs;
  (* a def of a hardware reg requires us to spill the h/w reg *)
  let spill_hw r z =
    if is_tmp r then z
    else let spill_temp (vm, spills as z) t =
      if RS.mem t lo then
        let vm' = VM.spill' t vm in
        if (VM.var_locs'' vm t).VM.mem' then (vm', spills)
        else (eprintf "spilling reg\n"; (vm', t :: spills))

```

```

    else z in
      List.fold_left spill_temp z (VM.reg_contents' vm r) in
    let inmap, spills = RS.fold spill_hw defs (vm, []) in
    (* allocate each use in turn *)
    <define allocation helpers 68a>
    let _, usemap, spills, moves, reloads =
      RS.fold (alloc_one false rtl) uses (VM.empty, inmap, spills, [], []) in
    let midmap = upd_midmap (expire_last_uses usemap) in
    let _, defmap_early, spills, moves, reloads =
      RS.fold (alloc_one true rtl) defs (usemap, midmap, spills, moves, reloads) in
    let defmap, spills = reconcile_succs (defmap_early, spills) in
    let outmap = expire_last_defs defmap in
    pr_reg_lst "spills: " spills;
    pr_reg_lst "moves: " moves;
    pr_reg_lst "reloads: " reloads;
    print_vm "inmap: " vm; print_vm "usemap: " usemap;
    print_vm "midmap: " midmap; print_vm "defmap_early: " defmap_early;
    print_vm "defmap: " defmap; print_vm "outmap: " outmap;
    (usemap, midmap, defmap, outmap, spills, moves, reloads) in

```

For each temp used or defined in the instruction, we call `alloc_one` to allocate the register. If the temp is really a hardware register or it's already in a register, we're done. Otherwise, we try allocating it in 3 steps:

1. If the temp is involved in a copy, we try allocating it to the other register.
2. If we can allocate the temp without spilling, we do so.
3. We spill a register for the temp.

```

<define allocation helpers 68a>≡ (67c) 71b▷
let alloc_one is_def rtl ((s, _, _) as t) (usemap, vm, spills, moves, reloads as z) =
  eprintf "alloc_%s %s\n" (if is_def then "def" else "use") (pr_reg t);
  (* IS THIS NEXT LINE REALLY NECESSARY? *)
  let vm = if is_def then VM.remove_mem' t vm else vm in
  if is_tmp t && isNone (VM.temp_loc' vm t) then (* unallocated temp *)
    let regs = get_regs_for_space tgt regs s in
    <functions for choosing a temp to spill 70b>
    <functions to try finding a register without spilling 69>
    <functions to try a register assignment for coalescing copies 68b>
    try match VM.var_locs'' vm t with
      | {VM.reg' = Some _} -> z
      | {VM.reg' = None}   -> try_copy ()
    with Not_found -> try_copy ()
  else z in

```

If the other half of the copy instruction has already been allocated or if it has a preferred register, we try to allocate the temp to that register. Before allocating to the register, we make sure the register won't be redefined during the lifetime of the temp we're allocating.

```

<functions to try a register assignment for coalescing copies 68b>≡ (68a)
(* Assign move-related temps to the same registers. *)
let try_copy () =
  let fail () = try_regs regs in
  match RU.RTLType.singleAssignment rtl with
  | Some (dst, src) ->
    let alloc r =
      let def = next_def t dm in
      if List.for_all (fun r -> last_use r um <= def) (VM.reg_contents' vm r) then
        alloc_to r
      else fail () in
    let target = if is_def then src else dst in

```

```

(* claude: neither branch below (is_tmp target's usemap
 * lookup, or the plain hardware-register "else alloc
 * target") ever checked lo/li on target itself - alloc's own
 * "List.for_all (... VM.reg_contents' vm r)" only accounts
 * for OTHER tmps the allocator has already placed in
 * target's register, not target's own continued need as
 * itself. Coalescing t straight into target's register is
 * only sound if target's current value isn't needed again
 * later - e.g. unsound here whenever target gets redefined
 * (a multiple assignment's "new" side, or an incoming
 * argument register) before some later, independent use of
 * its "old" value that this move was never meant to
 * disturb. Reproduced on two shapes: tests/cmm/bool.c--
 * (target = a hardware argument register, $r[0]/n, fetched
 * again for a printf argument after Peephole.subst_forward
 * propagated a direct reference to it) and tests/cmm/
 * multasn.c-- (target = a tmp holding "hp", redefined to
 * hp+8 by the same multiple assignment that "x := hp" is
 * part of). Mirrors can_use's own lo/li check just above. *)
let target_needed_later =
  (is_def && RS.mem target lo) || (not is_def && RS.mem target li) in
if is_vfp target || target_needed_later then fail ()
else if is_tmp target then
  try match (VM.var_locs'' usemap target).VM.reg' with
    | Some r -> alloc r
    | None   -> fail ()
  with Not_found -> fail ()
(* claude: target is precolored (not a tmp, not vfp). For an
 * ordinary middle instruction that's fine even when target is
 * a symbolic pseudo-register - the move degenerates to a
 * harmless self-move, and Optimize.remove_nops deletes it.
 * But for is_last, target may be a pseudo-register that is
 * never a real allocation destination at all, like the
 * control/pc register (Reg('c', pc-index), arch/x86/
 * x86rec.mlb's "eip : Reg('c', n) [...]") standing for this
 * very Jump's own target: t6 := ...; $c[0] := t6 (t6's own
 * USE here has is_def=false, target=dst=$c[0]) coalesced
 * this way makes the jump's *own* defining move degenerate
 * into "$c[0] := $c[0]" once both sides share a location -
 * an unselectable last-node RTL this time (no
 * remove_nops-style cleanup exists for last nodes, and
 * arch/x86/x86rec.mlb has no rule for a bare register/self
 * fetch as a jump target - there isn't supposed to be one).
 * Restrict only the is_last case to target actually in regs,
 * the real allocatable set this function is choosing among
 * everywhere else (try_regs, spill, ...) - restricting the
 * middle case too regressed ppc's altread/altret/altret2,
 * which coalesce into some non-regs precolored target
 * legitimately at -O0, no peephole/dead-elim involved.
 * Reproduced on tests/cmm/call3.c-- (tail2/tailnot), needing
 * Peephole.subst_forward and Optimize.elim_dead_assignments
 * together upstream of here to produce this exact shape. *)
else if is_last && not (List.exists (R.eq target) regs) then fail ()
else alloc target
| None -> fail () in

```

The `can_use` function decides whether a register `r` can be allocated to `t` for the entire lifetime of `t` (i.e. without spilling). I have tried some more complicated (now commented out) code to track copies through registers; it's a bit of a mess and it didn't improve the code much, if any.

(functions to try finding a register without spilling 69)≡

(68a) 70a▷

```

let can_use r =
  (* attempted optimization: tracking copies through registers
  let end_t = last_use t um in
  let ndef = next_def t dm in
  let checkReDefs = function
    | VM.HW (VM.NonCopy i) -> end_t <= i
    | VM.HW (VM.Copy (_, i, t', _)) -> end_t <= i || (R.eq t t' && i < ndef)
    | VM.Temp _ -> imposs "checkReDefs expected h/w reg" in
  (try checkReDefs (RM.find r dm) with Not_found -> true) &&
  *)
  last_use t um <= next_def r dm &&
  ((is_def && isNil (VM.reg_contents' vm r) && not (RS.mem r lo)) ||
   (not is_def && not (RS.mem r li) && isNil (VM.reg_contents' vm r) &&
    (not (RS.mem r lo) || RS.mem t defs || not (RS.mem t lo)))) in

```

The `try_regs'` function looks through the list of register to find one that can be allocated to `t` without spilling; if none can be found, we spill. The `try_regs` function first tries to allocate the temp to a register indicated by the preferences map, then falls back to `try_regs'`.

(functions to try finding a register without spilling 70a)+≡ (68a) <69

```

let alloc_to r =
  let reloads = if is_def then reloads else t :: reloads in
  (usemap, VM.add_reg' t r vm, spills, moves, reloads) in
let rec try_regs' = function
  | [] -> spill regs (* give up and spill *)
  | r::rst -> if can_use r then alloc_to r else try_regs' rst in
let try_regs regs =
  try let r = RM.find t pmap in
    if can_use r then alloc_to r
    else (eprintf "can't use %s\n" (pr_reg r); try_regs' regs)
  with Not_found -> try_regs' regs in

```

A register can be spilled if it (and all its contents) are not otherwise needed in the RTL. The function `spill'` spills the first possible register.

We might want to consider spilling by moving the spilled value to another register.

(functions for choosing a temp to spill 70b)≡ (68a) 71a▷

```

let spillable r =
  not (RS.mem r lo) && (is_def || not (RS.mem r uses)) && not (RS.mem r defs) in
let rec spill' = function
  | [] -> imposs "alloc_one: no registers to spill?"
  | r::rst ->
    if spillable r then
      let canSpill t = (is_def && not (RS.mem t defs)) ||
        (not is_def && not (RS.mem t uses)) in
      let makeSpill (vm, spills) t =
        if try (VM.var_locs'' vm t).VM.mem' with Not_found -> false
        then (VM.remove_reg' t vm, spills) (* already in memory *)
        else (eprintf "spill'\n";
          (VM.add_mem' t (VM.remove_reg' t vm), t::spills)) in
      let ts = VM.reg_contents' vm r in
      if List.for_all canSpill ts then
        let (vm, spills) = List.fold_left makeSpill (vm, spills) ts in
        let reloads = if is_def then reloads else t :: reloads in
        (usemap, VM.add_reg' t r vm, spills, moves, reloads)
      else spill' rst
    else spill' rst in

```

But some spills are better than others. So we define a comparison function `du_comp'` that find compares two registers according to which register is defined or used next. The function `spill` then orders the register first according to which spills will be cheaper (because the register's contents are already in memory or because

the spill was inevitable due to an upcoming definition of the hardware register), then according to the order imposed by the `du_comp'` function. Finally, we call `spill'` to effect the spill.

(functions for choosing a temp to spill 71a) +≡ (68a) <70b

```

let du_comp' r1 r2 =
  let getDist proj r m = try proj (RM.find r m) with Not_found -> max_int in
  let dist' r = min (getDist proj_dstd r dm) (getDist proj_distu r um) in
  let dist r =
    List.fold_left (fun m t -> min m (dist' t)) (dist' r) (VM.reg_contents' vm r) in
  eprintf "dist %s: %d, %s: %d\n" (pr_reg r2) (dist r2) (pr_reg r1) (dist r1);
  dist r2 - dist r1 in
let spill regs =
  (* We want to order forced spills (Def) before unwanted spills (Use). *)
  let cheap_spill r =
    let cheap t' = (try (VM.var_locs'' vm t').VM.mem' with Not_found -> false)
      || next_use t' um > next_def r dm
      || last_use t' um <= next_def r dm in
    spillable r && List.for_all cheap (VM.reg_contents' vm r) in
  let cheap, expensive = List.partition cheap_spill regs in
  eprintf "cheap regs: [%s]\n      expensive: [%s]\n"
    (String.concat " ", (List.map pr_reg cheap))
    (String.concat " ", (List.map pr_reg expensive));
  let regs' = (List.sort du_comp' cheap @ List.sort du_comp' expensive) in
  eprintf "spill regs: [%s]\n      regs': [%s]\n"
    (String.concat " ", (List.map pr_reg regs))
    (String.concat " ", (List.map pr_reg regs'));
  prmaps (0, (0, dm, um));
  spill' regs' in

```

At the end of a temp's live range, we remove it from the variable map.

(define allocation helpers 71b) +≡ (67c) <68a

```

let expire_last_uses vm =
  let expire_one t vm =
    if is_tmp t then
      if RS.mem t defs then VM.remove_mem' t vm (* keep and reuse the reg assignment *)
      else if RS.mem t lo then vm
      else VM.remove_mem' t (VM.remove_reg' t vm)
    else vm in
  RS.fold expire_one uses vm in
let expire_last_defs vm =
  let expire_one t vm =
    if is_tmp t && not (RS.mem t lo) then VM.remove_reg' t vm else vm in
  RS.fold expire_one defs vm in

```

For middle nodes, the `allocate` function takes care of everything. For last nodes, there are a few complications:

- We need to reconcile the variable map with the successors of the basic block.
- The run-time system doesn't currently have support for storing variables in multiple locations (e.g. a register and a stack slot). So, at possible stopping points (call sites and their successors), we need to make sure live variables are only in a single location. So, for each temp allocated to both a register and a stack slot, we drop the stack slot with `upd_midmap`.

(set up dataflow analysis for register allocation 71c) +≡ (65d) <67c 72a>

```

let allocate_mid      = allocate (fun x -> x) (fun x -> x) false in
let allocate_last l =
  let reconcile z =
    let add_spill uid (defmap, spills) =
      let succmap = get_vm uid in
      let spill t (dm, spills as z) =

```



```

    try if (VM.var_locs'' dm t).VM.mem' then z
    else (eprintf "reconcile spill\n"; (VM.add_mem' t dm, t :: spills))
    with Not_found -> z in
  VM.fold' (fun _ _ x -> x) spill succmap (defmap, spills) in
  GR.fold_succs add_spill l z in
let upd_midmap vm =
  match l with GR.Call _ ->
    let rem t vm = try match (VM.var_locs'' vm t).VM.reg' with
      | Some _ -> VM.remove_mem' t vm
      | None -> vm
      with Not_found -> vm in
    VM.fold' (fun _ _ m -> m) rem vm vm
  | _ -> vm in
allocate reconcile upd_midmap true in

```

11.2 Framing the allocator as a dataflow analysis

Now, we use the `allocate` functions in putting together our dataflow analysis. The case for `last` nodes is slightly more interesting, as we set the dataflow fact (variable maps) for the successors.

(set up dataflow analysis for register allocation 72a) + ≡ (65d) <71c 72b>

```

let middle_out (live_outs, vm) m =
  if Debug.on "flowra" then (eprintf "\nmiddle: "; C.pr_mid m; eprintf "\n");
  match live_outs with
  | li::(lo::_ as rst) ->
    let _,_,_, outmap, _,_,_ =
      allocate_mid vm (li, lo) (GR.mid_instr m) (defs_uses_m m) in
    (rst, outmap)
  | _ -> imposs "no liveout for a middle node" in
let last_out (live_outs, vm) l set =
  if Debug.on "flowra" then (eprintf "\nlast: "; C.pr_last g l; eprintf "\n");
  match live_outs with
  | [li; lo] ->
    print_vm "last varmap: " vm;
    let usemap, midmap, defmap, outmap, spills, moves, reloads =
      allocate_last l vm (li, lo) (GR.last_instr l) (defs_uses_l l) in
    let set' uid =
      let li = R.promote_rxset (lir.get uid) in
      let rem_reg t _ vm = if RS.mem t li then vm else VM.remove_reg' t vm in
      let rem_mem t _ vm = if RS.mem t li then vm else VM.remove_mem' t vm in
      set uid ([], VM.fold' rem_reg rem_mem outmap outmap) in
    G.iter_outedges l ~noflow:(fun u -> set' u) ~flow:(fun e -> set' (fst e.G.node))
  | _ -> imposs "!= 1 liveout for a last node" in

```

And formally defining up the analysis:

(set up dataflow analysis for register allocation 72b) + ≡ (65d) <72a 73a>

```

let changed ~old:(_, x) ~new':(_, y) = not (VM.eq' ~old:x ~new':y) in
let join (_, vm1) (_, vm2) = ([], VM.join vm1 vm2) in
let fact = { D.fact_name' = "varmaps"
  ; D.init_info' = ([], VM.emptyy)
  ; D.add_info' = join
  ; D.changed' = changed
  ; D.get' = get g
  ; D.set' = set
  } in
let computation = { F.name = "flowra"
  ; F.middle_out = middle_out
  ; F.last_outs = last_out
  } in

```

```
let analysis = (fact, computation) in
```

And now setting up the transformation. For the most part, it's the same as the analysis, but when we get the results, we rewrite the RTL and the spans attached to the last nodes.

There's probably an opportunity for refactoring here.

```
<set up dataflow analysis for register allocation 73a>+≡ (65d) <72b>
let tx =
  <helper functions for rewriting the rtls and inserting spills/reloads 73b>
  let middle_out (live_outs, inmap) m =
    if Debug.on "flowra" then (eprintf "\nmiddle: "; C.pr_mid m; eprintf "\n");
    match live_outs with
    | li::lo::_ ->
      let rtl = GR.mid_instr m in
      let usemap, midmap, defmap, outmap, spills, moves, reloads =
        allocate_mid inmap (li, lo) rtl (defs_uses_m m) in
      let upd, rtl' = rewrite ~usemap ~defmap rtl in
      (match (upd, reloads, moves, spills) with
       | (false, [], [], []) -> None
       | _ -> Some (make_moves ~inmap ~usemap ~reloads ~moves ~spills
                           (G.single_middle (G.new_rtlm rtl' m))))
    | _ -> imposs "tx: no liveout for a middle node" in
  let last_outs (live_outs, inmap) l =
    if Debug.on "flowra" then (eprintf "\nlast: "; C.pr_last g l; eprintf "\n");
    match live_outs with
    | [li; lo] ->
      let rtl = GR.last_instr l in
      let usemap, midmap, defmap, outmap, spills, moves, reloads =
        allocate_last l inmap (li, lo) rtl (defs_uses_l l) in
      let upd, rtl' = rewrite ~usemap ~defmap rtl in
      (match l with GR.Call c -> upd_spans c.GR.cal_spans midmap | _ -> ());
      Some (make_moves ~inmap ~usemap ~reloads ~moves ~spills
                  (G.single_last (G.new_rtl l rtl' l)))
    | _ -> imposs "!= 1 liveout for a last node" in
  { F.name = "flowralloc"; F.middle_out = middle_out; F.last_outs = last_outs } in
```

Just a few helpers for rewriting the RTLs, with support for declaring a map from temps to registers for rewriting the RTL (`make_map`) and support for spilling, reloading, and moving values between registers.

```
<helper functions for rewriting the rtls and inserting spills/reloads 73b>≡ (73a)
let make_map upd vm t =
  let fail () = print_vm "" vm; impossf "DLS: didn't ralloc %s" (pr_reg t) in
  if is_tmp t then
    try match (VM.var_locs'' vm t).VM.reg' with Some r -> (upd := true; r)
        | None -> fail ()
    with Not_found -> fail ()
  else t in
let rewrite ~usemap ~defmap rtl =
  let upd = ref false in
  (!upd, RU.Subst.reg_def ~map:(make_map upd defmap)
    (RU.Subst.reg_use ~map:(make_map upd usemap) rtl)) in
let spill ~inmap g t =
  let fail () = impossf "can't spill temp %s" (pr_reg t) in
  try match VM.var_locs'' inmap t with
  | {VM.mem' = true} -> g
  | {VM.reg' = Some src} ->
    let spillg = tgt.T.machine.T.spill proc src (get_spill_loc t) in
    G.splice_focus_entry g (G.unfocus (fst (G.block2cfg m spillg)))
  | _ -> fail ()
  with Not_found -> fail () in
let reload ~usemap g t =
  let fail () = imposs "can't reload temp" in
```

```

let dst = try match (VM.var_locs'' usemap t).VM.reg' with Some r -> r
                                     | None      -> fail ()
      with Not_found -> fail () in
let reloadg = tgt.T.machine.T.reload proc (get_spill_loc t) dst in
G.splice_focus_entry g (G.unfocus (fst (G.block2cfg m reloadg))) in
let make_moves ~inmap ~usemap ~reloads ~moves ~spills g =
  if not (isNil moves) then imposs "not prepared for moves";
  G.unfocus (List.fold_left (spill ~inmap)
    (List.fold_left (reload ~usemap) (G.entry g) reloads) spills) in

```

Old hat: rewriting the spans. Surely some code like this should be shared among all the register allocators.

```

⟨define code for rewriting spans 74a⟩≡ (65d)
let upd_spans s vm = match s with
| Some spans ->
  let rewrite l =
    let guard = function RP.Reg t -> is_tmp t | _ -> false in
    let map = function
      | RP.Reg t ->
        eprintf "span in %s\n" (pr_reg t);
        (try match VM.var_locs'' vm t with
          | {VM.mem' = true} -> Dn.loc (AU.alloc (get_spill_loc t) (R.width t))
          | {VM.reg' = Some r} -> RP.Reg r
          | _ -> raise Runtimedata.DeadValue
        with Not_found -> raise Runtimedata.DeadValue)
      | _ -> impossf "DLS reg allocator emitting RT data: guard failed" in
    RU.Subst.loc_of_loc ~guard ~map l in
  Runtimedata.upd_spans rewrite spans
| None -> () in

```

The dataflow framework doesn't feed us the first nodes in a forward dataflow analysis, so we step outside the framework to update the spans on the entry nodes.

```

⟨update entry label spans 74b⟩≡ (65d)
let upd_entry_span b =
  match GR.first (GR.unzip b) with
  | GR.Label ((id, _), _, s) -> upd_spans (!s) (vmr.get id)
  | GR.Entry -> () in
G.iter_blocks upd_entry_span g;

```

11.3 Optimization: Choosing spills

To improve the quality of the register allocation, we need to carefully choose the registers we spill. We can use the distance to next definition or use of a register as a heuristic: we can choose a register that won't be touched again for a while. We compute the distances using a backward dataflow pass.

N.B. We should really take the loop-nest depth of the next use into account because we will have to insert a reload before the use.

Note: in our world, a single live range may include multiple defs. "What???", you say. "Think about $t[0] := t[0] + t[1]$ ", I respond. If a temp is used and defined in the instruction, both instances must be allocated to the same register.

As an attempt at yet another optimization, I'm not keeping track not only of the next use but also the last use of a temp. Also, instead of the next definition, I keep a sequence of copy definitions (possibly empty) ending in a non-copy definition.

Additionally, we have to consider how this will converge: we're computing the distance from each node to the defs of a given temp. But if there's a loop, we can see the same copy instruction repeatedly, leading to an unbounded sequence of copies. The insight is that we only want to include each static definition of a temp once in a sequence of copies, which means that we need to identify each definition site. We use the a map to associate

each basic block with an integer, and we count each definition in a basic block as we find it. The pair of basic block number and def index identifies each static definition site.

<compute distances to uses 75a>≡ (65d) 75b▷

```
let (_, block_nums) =
  let add_num b (n, map) = (n + 1, U.Map.add (GR.id b) n map) in
  G.fold_blocks add_num (0, U.Map.empty) g in
let sprintf = Printf.sprintf in
let prmaps (_, (i, dm, um)) =
  eprintf "map %d: " i;
  let rec prhw = function
    | VM.Copy ((n, Some b), i, r, hw) ->
      sprintf "copy %d(%d,%d):%s, %s" i n b (pr_reg r) (prhw hw)
    | VM.Copy ((n, None), i, r, hw) ->
      sprintf "copy %d(%d,%d):%s, %s" i n (pr_reg r) (prhw hw)
    | VM.NonCopy i -> sprintf "noncopy %d\n" i in
  let pr_d = function
    | VM.HW hw -> prhw hw
    | VM.Temp (i, i') -> sprintf "defs %d,%d\n" i i' in
  let pr_u (VM.Use (next, last)) = sprintf "use %d (%d)" next last in
  RM.iter (fun r i -> eprintf "%s -> %s, " (pr_reg r) (pr_d i)) dm;
  RM.iter (fun r i -> eprintf "%s -> %s, " (pr_reg r) (pr_u i)) um;
  eprintf "\n" in
let empty = (0, (0, RM.empty, RM.empty)) in
```

We use the “seen” maps to keep track of the static definitions we’ve seen.

<compute distances to uses 75b>+≡ (65d) ◁75a 75c▷

```
let emptySeen = IPS.empty in
let seen (n, o) set = match o with
| Some blockNum -> IPS.mem (n, blockNum) set
| None -> imposs "seen: unknown block num for def" in
let addSeen (n, o) set = match o with
| Some blockNum -> IPS.add (n, blockNum) set
| None -> imposs "addSeen: unknown block num for def" in
```

We compute the join for a definition by merging the sequences up to the nearest non-copy definition. Also, we only include the nearest distance for each definition site.

<compute distances to uses 75c>+≡ (65d) ◁75b 75d▷

```
let rec hwdefmin copiesSeen = function
| (VM.NonCopy i, VM.NonCopy i') -> VM.NonCopy (min i i')
| (VM.Copy (_, i, _, _) as x, (VM.NonCopy i' as y)) ->
  if i < i' then addCopy copiesSeen (x, y) else y
| (VM.NonCopy i as x, (VM.Copy (_, i', _, _) as y)) ->
  if i < i' then x else addCopy copiesSeen (y, x)
| (VM.Copy (_, i, _, _) as x, (VM.Copy (_, i', _, _) as y)) ->
  if i < i' then addCopy copiesSeen (x, y) else addCopy copiesSeen (y, x)
and addCopy copiesSeen = function
| (VM.Copy (n, i, r, rst), y) ->
  if seen n copiesSeen then hwdefmin copiesSeen (rst, y)
  else VM.Copy (n, i, r, hwdefmin (addSeen n copiesSeen) (rst, y))
| _ -> imposs "addCopy applied to non-copy" in
let rec dmin = function
| (VM.Temp (i1,i2), VM.Temp (i1',i2')) ->
  if i1 < i1' then VM.Temp (i1, min i1' i2) else VM.Temp (i1', min i1 i2')
| (VM.HW x, VM.HW y) -> VM.HW (hwdefmin emptySeen (x, y))
| _ -> imposs "mismatched temp vs. hardware reg" in
```

Uses are much simpler.

<compute distances to uses 75d>+≡ (65d) ◁75c 76a▷

```
let umin (VM.Use (i,l), VM.Use (i',l')) = VM.Use (min i i', min l l') in
```

A few convenience functions for fetching definition and use distances.

```

⟨compute distances to uses 76a⟩+≡ (65d) <75d 76b>
let next_def r dm = try match RM.find r dm with | VM.Temp (i,_) -> i
                                                | VM.HW (VM.NonCopy i) -> i
                                                | VM.HW (VM.Copy (_, i, _, _)) -> i
                                                with Not_found -> max_int in
let next_use r um = try match RM.find r um with | VM.Use (i,_) -> i
                                                with Not_found -> min_int in
let last_use r um = try match RM.find r um with | VM.Use (_,l) -> l
                                                with Not_found -> min_int in
let proj_distd = function
  | (VM.Temp (i, _)) -> i
  | (VM.HW (VM.NonCopy i)) -> i
  | (VM.HW (VM.Copy (_, i, _, _))) -> i in
let proj_distu (VM.Use (u, _)) = u in

```

And finally, the complete join operation: join definitions and uses in turn.

```

⟨compute distances to uses 76b⟩+≡ (65d) <76a 76c>
let join (_, (i1, dm1, um1 as x)) (_, (i2, dm2, um2 as y)) =
  assert (i1=0 && i2=0);
  let addd t d1 m = try RM.add t (dmin (d1, (RM.find t dm2))) m
                    with Not_found -> RM.add t d1 m in
  let addu t u1 m = try RM.add t (umin (u1, (RM.find t um2))) m
                    with Not_found -> RM.add t u1 m in
  (0, (0, RM.fold addd dm1 dm2, RM.fold addu um1 um2)) in

```

Given an RTL, we update the definition and use distances. The first interesting case is a move instruction, which may extend the last use of the used instruction to the last use of the defined instruction (because we expect to assign both temps to the same register). A definition in a move instruction also extends the known sequence of definitions for the defined temp. The second interesting case is when a temp is both defined and used in an instruction. In our compiler, such a temp must be assigned the same register in both use and definition, so we do not treat the definition as the end of the live range for the temp; instead, both the def and use share a single live range.

```

⟨compute distances to uses 76c⟩+≡ (65d) <76b 77a>
let upd_maps rtl (defs, uses) (def_num, (i, dm_out, um_out)) =
  let def_id = (def_num, None) in
  let add_use t (def_num, ud, um) =
    let i' = try match RM.find t um with | VM.Use (_, i') -> max i i'
              with Not_found -> i in
    (* If it's a copy, pass on the last use info from the destination to the source. *)
    let i' = match RU.RTLType.singleAssignment rtl with
      | Some (dst, _) when is_tmp dst ->
        (try match RM.find dst um_out with
          | VM.Use (_, def_i') -> max i' def_i'
          with Not_found -> i')
      | _ -> i' in
    (def_num, ud, RM.add t (VM.Use (i, i')) um) in
  let add_def t (def_num, dm, um) =
    let dn' = def_num + 1 in
    let add_hw_def () =
      match RU.RTLType.singleAssignment rtl with
      | Some (_, r) -> (try match RM.find t dm with
        | VM.HW d ->
          (dn', RM.add t (VM.HW (VM.Copy (def_id, i, r, d))) dm, um)
        | _ -> imposs "expected hw reg"
        with Not_found ->
          (dn', RM.add t (VM.HW (VM.Copy (def_id, i, r,
            VM.NonCopy max_int)))) dm, um))
      | None -> (dn', RM.add t (VM.HW (VM.NonCopy i)) dm, um) in

```

```

    if RS.mem t uses then (* t is both defined and used *)
      (if is_tmp t then (dn', RM.add t (VM.Temp (i, next_def t dm)) dm, um)
       else add_hw_def ())
    else if is_tmp t then (dn', RM.remove t dm, RM.remove t um)
    else add_hw_def () in
let (def_num, dm, um) =
  RS.fold add_use uses (RS.fold add_def defs (def_num, dm_out, um_out)) in
(def_num, (i - 1, dm, um)) in

```

Now, we put together all our machinery for tracking definition and use distances and wrap it up for the dataflow framework. The one complication is the case for **first** nodes, in which we update all the distances. As we calculated the distances, we start the **last** node with the index 0, and we decrement the index as we move back toward the first node. When we reach the first node, we update all the distances so that the first node is 0, the second node is 1, etc. We also update the block number on definitions, which we can only compute from the **first** node.

```

⟨compute distances to uses 77a⟩+≡ (65d) ◁76c 77b▷
let ud_middle_in out mid = upd_maps (GR.mid_instr mid) (defs_uses_m mid) out in
let ud_out_last last =
  GR.fold_succs (fun uid m -> join (dists.get uid) m) last empty in
let ud_last_in l = upd_maps (GR.last_instr l) (defs_uses_l l) (ud_out_last l) in
let ud_first_in (_, (z, dm, um)) f =
  (* normalize to 0 *)
  let bid = Some (U.Map.find (GR.fid f) block_nums) in
  let upd i = if i = max_int then max_int else i - z in
  let rec upd_hw = function
    | VM.NonCopy i -> VM.NonCopy (upd i)
    | VM.Copy ((def_num, None), i, r, z) -> VM.Copy ((def_num, bid), upd i, r, upd_hw z)
    | VM.Copy (n, i, r, z) -> VM.Copy (n, upd i, r, upd_hw z) in
  let updd = function
    | VM.Temp (i, i') -> VM.Temp (upd i, upd i')
    | VM.HW hw -> VM.HW (upd_hw hw) in
  let updu (VM.Use (i, i')) = VM.Use (upd i, upd i') in
  (0, (0, RM.map updd dm, RM.map updu um)) in

```

Define equality and a function to set the dataflow fact, and we're ready to apply the dataflow framework.

```

⟨compute distances to uses 77b⟩+≡ (65d) ◁77a
let changed ~old:(_, (oi, (odm : VM.def_dist RM.t), oum as o))
  ~new':(_, (ni, (ndm : VM.def_dist RM.t), num as n)) =
  let optEq = function (Some b, Some b') -> b = b' | (None, None) -> true
    | _ -> false in
  let rec hweq = function
    | (VM.NonCopy i, VM.NonCopy i') -> i = i'
    | (VM.Copy ((n, b), i, r, hw), VM.Copy ((n', b'), i', r', hw')) ->
      n == n' && optEq (b, b') && i = i' && Register.eq r r' && hweq (hw, hw')
    | _ -> false in
  let deq x y = match (x, y) with
    | (VM.Temp (i1, i1'), VM.Temp (i2, i2')) -> i1 = i2 && i1' = i2'
    | (VM.HW hw1, VM.HW hw2) -> hweq (hw1, hw2)
    | _ -> false in
  let ueq (VM.Use (i, l)) (VM.Use (i', l')) = i = i' && l = l' in
  oi <> ni || not (RM.equal deq odm ndm) || not (RM.equal ueq oum num) in

let set_ud uid (_, (i, dm, um) as v) =
  (*let prev = (try dists.get uid with Not_found -> empty) in
  dprintf "set prev:\n"; pr prev; eprintf "set useDist:\n"; pr v;*)
  dists.set uid v in

let useDistance =
  Dataflow.B.anal' ({ D.fact_name' = "use distance"

```

```

    ; D.add_info' = join
    ; D.changed' = changed
    ; D.init_info' = empty
    ; D.get' = dists.get
    ; D.set' = set_ud
  },
  { D.B.name = "compute use distance"
    ; D.B.last_in = ud_last_in
    ; D.B.middle_in = ud_middle_in
    ; D.B.first_in = ud_first_in
  }) in
let (g, _) = Dataflow.B.rewrite' useDistance g in

```

11.4 Optimization: Allocation Preferences

Sometimes we know the register we would like to use for a particular temp. The priority is avoiding spills, but when possible, we'd also like to eliminate some copies.

Two maps: the first from temps to the hardware registers we'd like them to have; the second from each hardware register to the list of temps we want there.

For now, I'm only bothering with hardware registers. It would be nice to use the variable maps from the dataflow analysis (before reconciling differences between a block and its successors) to generate preferences.

```

⟨compute allocation prefs 78⟩≡ (65d)
let prprefs (prefs, revmap) =
  eprintf "Prefs: ";
  RM.iter (fun t r -> eprintf ", %s -> %s" (pr_reg t) (pr_reg r)) prefs;
  eprintf "\nRevmap: ";
  RM.iter (fun r ts -> eprintf ", %s -> " (pr_reg r); pr_reg_lst "" ts) revmap;
  eprintf "\n" in
let empty = (RM.empty, RM.empty) in
let join _ _ = empty in
let changed ~old:(o1, o2) ~new':(n1, n2) =
  not (RM.equal Register.eq o1 n1) ||
  not (RM.equal (List.for_all2 Register.eq) o2 n2) in
let prefs_last_out l = empty in
let prefs_last_in l = prefs_last_out l in
let prefs_middle_in (prefs, rev_prefs as out) m =
  match RU.RTLType.singleAssignment (GR.mid_instr m) with
  | Some (dst, src) ->
    let add_pref (prefs, rev_prefs) r =
      (RM.add src r prefs,
       RM.add r (src :: try RM.find r rev_prefs with Not_found -> []) rev_prefs) in
    if not (is_tmp src) || is_vfp dst then out
    else if is_tmp dst then
      try add_pref out (RM.find dst prefs) with Not_found -> out
    else
      let prefs = try List.fold_left (fun p t -> RM.remove t p)
        prefs (RM.find dst rev_prefs)
        with Not_found -> prefs in
      let rev_prefs = RM.remove dst rev_prefs in
      add_pref (prefs, rev_prefs) dst
  | None -> out in
let prefs_first_in out f = out in
let allocPrefs =
  Dataflow.B.anal' ({ D.fact_name' = "alloc prefs"
    ; D.add_info' = join
    ; D.changed' = changed
    ; D.init_info' = empty

```



```

; D.get'      = pir.get
; D.set'      = pir.set
},
{ D.B.name     = "compute alloc prefs"
; D.B.last_in  = prefs_last_in
; D.B.middle_in = prefs_middle_in
; D.B.first_in = prefs_first_in
}) in
let (g, _) = Dataflow.B.rewrite' allocPrefs g in
  Early spilling if the spill will happen in a frequently executed block?

```

11.5 Graph Coloring Register Allocation

Register allocation is responsible for replacing temporary registers (temps) in the control-flow graph (CFG) with hardware registers. Liveness analysis is performed prior to register allocation, annotating each node of the CFG with the list of registers (temps and pre-assigned hardware registers) that are alive at the node. Using this information, we construct an interference graph with nodes for each of the temporaries and edges between temporaries that are live at the same time.

Temporaries that interfere must be placed in different registers. This requirement reduces directly to the graph-coloring goal of assigning different colors to nodes that share graph edges. Register allocation is carried out by performing graph coloring on the interference graph, with the hardware registers serving as the colors. If the graph cannot be colored, we spill the uncolorable temps to stack slots and attempt to color the new graph.

11.5.1 Overview

We implement Lal George and Andrew Appel's iterated register coalescing algorithm. Before the graph coloring begins, liveness analysis is performed, and the interference graph is built. The interference graph is redundantly represented by an adjacency set and by adjacency lists. The adjacency set maintains a set of all the edges in the graph. The adjacency lists are stored in a hash table, with each node mapped to the list of its neighbors. The purpose of the redundancy is to optimize both the process of testing whether two nodes are adjacent and of finding all neighbors of a node.

In addition to building the graph, we also initialize the degree of each temp in the graph. The degree of a temp is the number of registers (both temps and hardware registers) with which the temp interferes.

The degree is also overloaded to deal with register constraints. The trick is that we express register constraints by ensuring that each temporary interferes with registers that cannot hold it. But there's no need to actually put the edges in the graph—instead, you can pre-seed the degree of the temp to think it interferes with all those registers.¹ The point is that all temp spaces agree on a single degree k that distinguishes high-degree from low-degree temps. There are surely other ways to achieve this goal, but they probably require modifying more of the algorithm. I think I picked up this technique from Tucker Taft back in 153.

Before the actual graph coloring process begins, temps are separated into worklists, based on their degree and whether they are involved in a move instruction (move-related).

The basis of the graph coloring algorithm is the observation that low-degree temps (temps that have fewer than K neighbors, where K is the number of colors) can be colored easily, so they can be removed from the graph. If the rest of the graph is colorable, then the graph will remain colorable when we restore the temps we removed. As we remove these temps, we place them on the `selectStack`, which provides the order we will assign colors.

If there are no low-degree, non-move-related temps remaining in the graph, we attempt to coalesce. We can coalesce two temps into a single temp by removing move instructions involving two temps that do not interfere, and using one of the temps in place of the other. By using conservative heuristics, we can ensure that the

¹Provided you choose registers carefully when you name the specific hardware register.

graph will not be rendered uncolorable by the coalescences. Two heuristics have been developed for conservative coalescing: the Briggs heuristic and the George heuristic.

If there are no low-degree, non-move-related temps in the graph, we freeze one of the move-related temps. Freezing a temp means that we will no longer attempt to coalesce moves in which the temp is involved; this also frees the temp for removal from the graph, since it becomes a low-degree, non-move-related temp.

The graph may be uncolorable if only high-degree temps remain; however, instead of assuming that we must spill a node, we choose a high-degree node, remove it from the graph, and place it on the `selectStack`.

This process of removing temps from the graph is summarized by the following options. The first applicable choice is always selected, and the decision is repeated until the graph is empty:

1. Attempt to remove non-move-related, low-degree temps from the graph and place them on the `selectStack`.
2. If there are no low-degree temps, then attempt to coalesce move-related temps.
3. If there is a move-related temp of low-degree, then we **freeze** the moves in which this temp is involved (meaning we will no longer try to coalesce them), allowing this temp and possibly others to be placed on the `selectStack`.
4. Otherwise, a significant-degree temp will be chosen as a potential spill candidate and placed on the `selectStack`.

After all the temps have been placed on the `selectStack`, they must be assigned colors. If a temp cannot be assigned a color (its neighbors are using all the available colors), then it must be spilled to memory. `rewriteProgram` is called to add spill temps, fetch instructions, and store instructions to the CFG. Graph coloring must start over from the point of liveness analysis if any temps are spilled. If all the temps can be assigned colors without requiring any spilling, then the temps are replaced by the assigned registers in the CFG, and register allocation is complete.

Additionally, register allocation must handle temporaries in different register spaces and constructs of irregular architectures, including register pairs. With multiple register spaces, we want to restrict each temp to the set of hardware registers in which it will fit. This is equivalent to adding interference edges between each temp and the hardware registers in which it cannot be placed. However, we can avoid the trouble of adding these edges by simply updating the degree of the node to indicate that it interferes with physical registers in other register spaces.

Traditionally, register pairs have been handled by adding extra edges in the interference graph, but this is not yet implemented in this allocator. Additionally, Smith and Holloway suggest that this provides an inaccurate representation - more thought must be committed here.

Note: I have augmented the Appel-George algorithm with the improvements by Allen Leung and Lal George, as described in section 3 of <http://smlnj.org/compiler-notes/new-ra.ps>.

from Appel 244:

```
procedure Main()
  LivenessAnalysis()
  Build()
  MakeWorklist()
  repeat
    if simplifyWorklist != {} then Simplify()
    else if worklistMoves != {} then Coalesce()
    else if freezeWorklist != {} then Freeze()
    else if spillWorklist != {} then SelectSpill()
  until simplifyWorklist = {} and worklistMoves = {}
    and simplifyWorklist = {} and spillWorklist = {}
  AssignColors()
```

```

if spilledTemps != {} then
  RewriteProgram(spilledTemps)
  Main()

```

11.5.2 Data Structures

```

⟨graph coloring types 81a⟩≡ (104b) 81b▷
  module T = Target

```

```

⟨graph coloring types 81b⟩+≡ (104b) <81a 84▷
  module CompareSpace = struct
    type t = Rtl.space
    let compare = Rtlutil.Compare.space
  end
  module SpaceMap = Map.Make(CompareSpace)

  module CompareEdge = struct
    type t = Register.t * Register.t
    let compare (x, y) (x', y') =
      match Register.compare x x' with
      | 0 -> Register.compare y y'
      | d -> d
  end
  module EdgeSet = Set.Make(CompareEdge)

  let is_tmp target (s,_,_) = Target.is_tmp target s

```

Temps and moves are stored in worklists. A temp can only be in one of these worklists at a time. We use imperative lists to improve the efficiency of removing items. WHAT ARE THE MEASUREMENTS THAT JUSTIFY THIS USE?

Worklists are gathered in related groups called `impListGroup`; a list item can only be on one of these lists at a time. Each `impListGroup` is polymorphic over the type of the item it carries and a `list` type. Each item has a `list` value that indicates which list (of the list group) to which the item belongs. The `impListGroup` is represented as a function from the `list` type to an imperative list. It is a dynamic invariant that each member will have the same `list` type as every other member of its imperative list.

IS IT WORTH PUTTING IMPERATIVE LISTS IN A SEPARATE MODULE?

```

⟨imperative lists 81c⟩≡ (103e) 81d▷
  type ('a,'b) impListItem = { v : 'b
                               ; mutable list : 'a
                               ; mutable next : ('a,'b) impListItem option
                               ; mutable prev : ('a,'b) impListItem option
                             }

```

```

type ('a,'b) impListGroup = 'a -> ('a,'b) impListItem option ref

```

We must not loop forever chasing pointers in a doubly-linked list. I'm pretty sure that with the new `eq_item`, the stack never gets more than three frames deep.

```

⟨imperative lists 81d⟩+≡ (103e) <81c 82▷
  let rec eq_item a b i i' =
    let eq_here i i' = a i.list i'.list && b i.v i'.v in
    let opt_eq o o' = match o, o' with
      | None, None -> true
      | Some i, Some i' -> eq i i'
      | None, Some _ -> false
      | Some _, None -> false in
    let rec eq_left i i' = eq_here i i' && opt_eq_left i.prev i'.prev in

```

```

let rec eq_right i i' = eq_here i i' && opt eq_right i.next i'.next in
eq_here i i' && opt eq_right i.next i'.next && opt eq_left i.prev i'.prev
let eq_item a b i i' =
  b i.v i'.v

```

ilg_add makes a new item and adds it to the front of the given list. The new list item is returned. ilg_switch removes a list item from its current list and placed it on a new list. ilg_iter, ilg_map, ilg_fold, and ilg_filter act like their functional counterparts.

(imperative lists 82)+≡ (103e) <81d 83>

```

let ilg_add group list value =
  let item = { v = value
               ; list = list
               ; next = !(group list)
               ; prev = None
               } in
  let () = match item.next with
    | Some n -> n.prev <- Some item
    | None    -> () in
  let () = group list := Some item in
  item

let ilg_switch group list item =
  begin
    (match item.prev with
     | Some p -> p.next <- item.next
     | None   -> group item.list := item.next)
    ; (match item.next with
     | Some n -> n.prev <- item.prev
     | None   -> ())
    ; item.list <- list
    ; item.prev <- None
    ; item.next <- !(group list)
    ; group list := Some item
    ; (match item.next with
     | Some n -> n.prev <- Some item
     | None   -> ())
  end

let rec ilg_iter fn lst =
  match lst with
  | Some n ->
    let next = n.next in
    let () = fn n in
    ilg_iter fn next
  | None    -> ()

let rec ilg_map fn lst =
  match lst with
  | Some n -> fn n :: ilg_map fn n.next
  | None    -> []

let rec ilg_fold fn lst rst =
  match lst with
  | Some n -> fn n (ilg_fold fn n.next rst)
  | None    -> rst

let rec ilg_filter fn lst =
  match lst with
  | Some n when fn n -> n :: ilg_filter fn n.next
  | Some n -> ilg_filter fn n.next

```

```
| None    -> []
```

For George-Appel iterated register coalescing, we need two list groups: one for temps and one for move instructions.

```
(imperative lists 83)+≡ (103e) <82
type tempList = Precolored
                | Initial
                | SimplifyWorklist
                | FreezeWorklist
                | SpillWorklist
                | Spilled
                | Coalesced
                | Colored
                | SelectStack
let str_of_list = function
| Precolored    -> "Precolored"
| Initial       -> "Initial"
| SimplifyWorklist -> "Simplify"
| FreezeWorklist -> "Freeze"
| SpillWorklist  -> "Spill Wlist"
| Spilled        -> "Spilled Temp"
| Coalesced      -> "Coalesced"
| Colored        -> "Colored"
| SelectStack    -> "Select"

type moveList = CoalescedMove
                | ConstrainedMove
                | FrozenMove
                | WorklistMove
                | ActiveMove
```

11.5.3 Graph Coloring State

The `colorGraph` record type represents the interference graph and maintains the current state of the graph coloring.

- `adjSet` - a set of all edges in the interference graph, where an edge is represented by a pair of registers
- `adjListMap` - a map from a temp `t` to the list of temps that interfere with `t`
- `degreeMap` - a map containing the current degree of each temp
- `moveListMap` - a map from a temp to the list of move instructions it is involved in
- `aliasMap` - when a move is coalesced, the coalesced temp must be aliased to the temp we maintain in the graph
- `colorMap` - map from temp to assigned color (no h/w register is ever in the domain??)

Every temp is a member of exactly one of these lists:

- `precolored` - preassigned machine registers
- `initial` - temps, not yet preprocessed for graph coloring
- `simplifyWorklist` - low-degree non-move-related temps
- `freezeWorklist` - low-degree move-related temps

- `spillWorklist` - high-degree temps
- `spilledTemps` - potential spills
- `coalescedTemps` - temps that have been coalesced out of the graph
- `coloredTemps` - temps successfully colored
- `selectStack` - stack of temps removed from the graph

These lists are collected in the `tempLG` list group.

Every move instruction is in one and only one of these lists:

- `coalescedMoves` - moves that have been coalesced
- `constrainedMoves` - moves whose source and target interfere
- `frozenMoves` - move that will no longer be considered for coalescing
- `worklistMoves` - moves enabled for possible coalescing
- `activeMoves` - moves not yet ready for coalescing

These lists are collected in the `moveLG` list-group.

Additional information is also stored:

- `regToItem` - map from temps to list-group items
- `numUses` - map from temps to number of uses and defs
- `liveRange` - map from temps to number of nodes over which it is live
- `spills` - short-living temps introduced for spilling (spill temps)
- `spillMap` - map from the original temps to spill temps
- `allColors` - list of all hardware registers (or proxies thereof)
- `colors` - map from register spaces to a list of available registers
- `k` - the number of physical registers

```

⟨graph coloring types 84⟩+≡ (104b) <81b 87a>
type move = Register.t * Register.t
module CompareMoves = struct
  type t = move
  let compare (r1, r2) (r1', r2') =
    match Register.compare r1 r1' with
    | 0 -> Register.compare r2 r2'
    | n -> n
end
module MoveSet = Set.Make (CompareMoves)
module MoveMap = Map.Make (CompareMoves)
type color = Color.t
type igNode = (tempList, Register.t) implistItem
type moveWorklist = (moveList, move) implistGroup
type tempWorklist = (tempList, Register.t) implistGroup
type 'a colorGraph = { mutable adjSet : EdgeSet.t
                      ; mutable adjListMap : igNode list RM.t

```

```

; mutable degreeMap : int RM.t
; mutable moveListMap : (moveList, move) impListItem list RM.t
; mutable aliasMap : Register.t RM.t
; mutable colorMap : color RM.t
; mutable spills : RS.t
; mutable spillMap : (Automaton.loc * Register.t list) RM.t

; mutable regToItem : igNode RM.t
; mutable regToClass : RC.t RM.t
; mutable tempLG : tempWorklist
; mutable moveLG : moveWorklist

; mutable allColors : Color.set
; mutable colors : Color.set SpaceMap.t
; mutable numUses : int RM.t
; mutable liveRange : int RM.t
; mutable hi : int MoveMap.t
; mutable k : int
}

```

`cgInfo` is the mutable data structure passed around the register allocator. A function with access to the target must be called to initialize the information about registers.

```

<init cgInfo 85>≡ (107a) 86▷
let cgInfo =
  let pre = ref None in
  let init = ref None in
  let simp = ref None in
  let freez = ref None in
  let spillw = ref None in
  let spilld = ref None in
  let coal = ref None in
  let colo = ref None in
  let sel = ref None in
  let tempFn = function
    | Precolored -> pre
    | Initial -> init
    | SimplifyWorklist -> simp
    | FreezeWorklist -> freez
    | SpillWorklist -> spillw
    | Spilled -> spilld
    | Coalesced -> coal
    | Colored -> colo
    | SelectStack -> sel in
  let coalm = ref None in
  let constm = ref None in
  let frzm = ref None in
  let workm = ref None in
  let actvm = ref None in
  let moveFn = function
    | CoalescedMove -> coalm
    | ConstrainedMove -> constm
    | FrozenMove -> frzm
    | WorklistMove -> workm
    | ActiveMove -> actvm in
  { adjSet = EdgeSet.empty
  ; regToItem = RM.empty
  ; regToClass = RM.empty
  ; adjListMap = RM.empty
  ; degreeMap = RM.empty
  ; moveListMap = RM.empty

```

```

; aliasMap = RM.empty
; colorMap = RM.empty
; tempLG = tempFn
; spills = RS.empty
; spillMap = RM.empty
; moveLG = moveFn

; colors = SpaceMap.empty
; allColors = CS.empty
; k = 0
; numUses = RM.empty
; liveRange = RM.empty
; hi = MoveMap.empty
}

```

clearCGInfo clears the data structure, leaving only the register lists.

(init cgInfo 86) +≡ (107a) <85

```

let allocatable cc = CS.of_regs (cc.Call.pre_nvregs ++ cc.Call.volregs)
let clearCGInfo cg (cfg, (Procedure.{cc = colors} as proc)) =
  let pre = ref None in
  let init = ref None in
  let simp = ref None in
  let freez = ref None in
  let spillw = ref None in
  let spilld = ref None in
  let coal = ref None in
  let colo = ref None in
  let sel = ref None in
  let tempFn = function
    | Precolored -> pre
    | Initial -> init
    | SimplifyWorklist -> simp
    | FreezeWorklist -> freez
    | SpillWorklist -> spillw
    | Spilled -> spilld
    | Coalesced -> coal
    | Colored -> colo
    | SelectStack -> sel      in
  let coalm = ref None in
  let constm = ref None in
  let frzm = ref None in
  let workm = ref None in
  let actvm = ref None in
  let moveFn = function
    | CoalescedMove -> coalm
    | ConstrainedMove -> constm
    | FrozenMove -> frzm
    | WorklistMove -> workm
    | ActiveMove -> actvm in
  let () = cg.adjSet <- EdgeSet.empty in
  let () = cg.regToItem <- RM.empty in
  let () = cg.adjListMap <- RM.empty in
  let () = cg.degreeMap <- RM.empty in
  let () = cg.moveListMap <- RM.empty in
  let () = cg.aliasMap <- RM.empty in
  let () = cg.colorMap <- RM.empty in
  let () = cg.tempLG <- tempFn in
  let () = cg.spills <- RS.empty in
  let () = cg.spillMap <- RM.empty in
  let () = cg.moveLG <- moveFn in

```

```

let () = cg.allColors <- allocatable colors in
let () = cg.numUses <- RM.empty in
let () = cg.liveRange <- RM.empty in
let () = cg.hi <- MoveMap.empty in
let () = cg.k <- CS.cardinal cg.allColors in
((cfg, proc), false)

```

We include two observers for the `colorGraph` type. `possibleColors` returns a list of the registers in which the argument temp could be placed. This list is cached for future lookup. This cache assumes that the space determines the available registers.

```

⟨graph coloring types 87a⟩+≡ (104b) <84 87b>
let possibleColors target cg ((s,_,_) as tmp) =
  try SpaceMap.find s cg.colors
  with Not_found ->
    if is_tmp target tmp then
      let colSet = CS.filter (Color.fits target s) cg.allColors in
      let () = cg.colors <- SpaceMap.add s colSet cg.colors in
      colSet
    else
      let _ = impossf "asked for possible colors of a h/w register" in
      CS.singleton (Color.of_reg tmp)

```

`degree_for_temp` returns the initial degree for a temporary, which for an allocable temporary is the total number of registers less the number of registers in which the temporary could be placed. For a non-allocable temporary, the initial degree is zero. WHY? DOES THIS FORCE A NON-ALLOCABLE TEMPORARY TO BE AUTOMATICALLY A LOW-DEGREE NODE?

```

⟨graph coloring types 87b⟩+≡ (104b) <87a
let degree_for_temp target cg temp =
  if is_tmp target temp then
    cg.k - CS.cardinal (possibleColors target cg temp)
  else
    0

```

11.5.4 Graph Coloring Stages

Build

Using the liveness information, we can build the interference graph by scanning the CFG and recording temps that interfere. The build stage requires that liveness analysis has been performed on the CFG and the `cgInfo` data structure has been initialized.

`resetDegree` resets the degree of a temp. It should not necessarily be reset to 0 because the temp must interfere with registers in other register spaces. Instead, we set $degree[t] = K - R$, where K is the total number of registers and R is the number of registers that could be used to color temp t 's register space.

The predicate `compete_for_registers` determines whether two registers compete for hardware resources.

```

⟨build 87c⟩≡ (107a) 88a>
let resetDegree target cg temp =
  cg.degreeMap <- RM.add temp (degree_for_temp target cg temp) cg.degreeMap

let compete_for_registers t1 t2 target cg = (* code from KH not yet used *)
  let c1 = RM.find t1 cg.regToClass in
  let c2 = RM.find t2 cg.regToClass in
  let competes = RC.may_alias c1 c2 in
  Debug.eprintf "colorgraph" "Interferes? %b\n" competes;
  competes

let compete_for_registers (s1,_,_ as t1) (s2,_,_ as t2) target cg =

```



```

let aliases t cs = CS.exists (Color.may_alias (Color.of_reg t)) cs in
if not (is_tmp target t1) then
  if not (is_tmp target t2) then
    RU.MayAlias.regs t1 t2
  else (* t2 is temporary, t1 is not *)
    aliases t1 (possibleColors target cg t2)
else (* t1 is a temporary *)
  if not (is_tmp target t2) then
    aliases t2 (possibleColors target cg t1)
  else (* both are temporaries *)
    if RU.Eq.space s1 s2 then
      true (* temporaries in the same space compete for registers *)
    else
      CS.overlap (possibleColors target cg t1) (possibleColors target cg t2)

```

We don't add an edge to the interference graph if it already exists or if the variables do not actually interfere (e.g. they are in non-intersecting register spaces). Also, we don't need to keep track of the neighbors of precolored temps.

```

<build 88a>+≡ (107a) <87c 88b>
let addEdge target cg u v =
  if not ((EdgeSet.mem (u, v) cg.adjSet) || (Register.eq u v)) &&
    compete_for_registers u v target cg
  then begin
    let () = cg.adjSet <- List.fold_right EdgeSet.add [(u, v); (v, u)] cg.adjSet in
    let addToAdjList t1 t2 =
      if is_tmp target t1 (* t1 <> Precolored *)
      then begin
        cg.adjListMap <- listMapAdd (RM.find t2 cg.regToItem) t1 cg.adjListMap;
        cg.degreeMap <- RM.add t1 (RM.find t1 cg.degreeMap + 1) cg.degreeMap
      end in
    let () = addToAdjList u v in
    let () = addToAdjList v u in
    ()
  end
end

```

Before building the interference graph, we must reset the degree of every temp in the CFG. Also, we reset the initial and precolored sets of the cg record. Then, we build the interference graph by folding over every node of the CFG, adding interferences as necessary. We also build a map (`regToItem`) from a register to a listgroup item.

We also want to map each interference graph node to its register class.

```

<build 88b>+≡ (107a) <88a
(* claude: [[last]] used to union in only
* "add_live_spansl 1 RSX.empty" (defs/uses/spans local to this
* node), but addInterference's own vis_last below computes its
* live set as "add_live_spansl last (Live.live_out_last last)"
* - the real backward-liveness fact, a strictly bigger set
* whenever a register is live out of this block's terminator
* without being defined or used anywhere in the whole
* procedure (a pure pass-through, e.g. a convention-mandated
* live register on a short, flat function with nothing else to
* touch it). That gap meant regToItem/regToClass below could
* be missing a register addInterference then calls addEdge on,
* crashing addToAdjList's "RM.find t2 cg.regToItem" with
* Not_found - reproduced on a small 7-argument function with no
* branches; the existing native.tests corpus never hit it
* because its multi-block programs happen to touch the same
* registers elsewhere too. Fixed by mirroring vis_last's own
* live expression exactly, so both passes agree on what a
* "last" node's registers are. *)

```

```

let get_regs cfg =
  let first f rst = R.promote_rxset (G.add_live_spansf f RSX.empty) ++ rst in
  let middle m rst = defsm m ++ usesm m ++ rst in
  let last l rst = defsl l ++ usesl l ++
    R.promote_rxset (G.add_live_spansl l (Live.live_out_last l)) ++
    rst in
  let block b rst = GR.fold_fwd_block first middle last b rst in
  G.fold_blocks block RS.empty cfg

let build cg (cfg, (Procedure.{target = PA.T target} as proc)) =
  Debug.eprintf "color" "color::build()\n"; flush stderr;
  let () = if is_some (!(cg.tempLG Initial)) then
    Impossible.impossible "Temps on initial list prematurely" in
  let regs = get_regs cfg in
  (*Debug.eprintf "color" "build regs: %s" (Register.Set.to_string regs);*)
  let regMap =
    RS.fold (fun r map -> resetDegree target cg r;
      cg.numUses <- RM.add r 0 cg.numUses;
      cg.liveRange <- RM.add r 0 cg.liveRange;
      let lst = if is_tmp target r then Initial else Precolored in
      RM.add r (ilg_add cg.tempLG lst r) map)
    regs cg.regToItem in
  let () = cg.regToItem <- regMap in
  let classMap =
    RS.fold (fun ((s,_,_) as r) map -> RM.add r (RC.mkClass s target) map)
    regs cg.regToClass in
  let () = cg.regToClass <- classMap in
  if Debug.on "colorgraph" then begin
    Debug.eprintf "colorgraph" "registerclass map \n ";
    RM.iter (fun tmp rclass ->
      Debug.eprintf "colorgraph" "%s -> %s\n"
        (RU.ToString.reg tmp) (RC.to_string rclass))
      cg.regToClass;
    Debug.eprintf "colorgraph" "\n";
  end;
  <addInterference 89>
  G.iter_blocks addInterference cfg;
  (cfg, proc), true

```

We add an edge between the temps defined in an instruction and those that are live after this instruction **node**. Note that the defined temps must interfere with each other. If **node** is a move instruction of the form $t_1 = t_2$, then t_2 will not be live after this instruction; furthermore, we track this instruction and the temps involved for possible coalescing.

TEST CASE: try an instruction with multiple defs. Each def must interfere with the other defs.

IT WOULD REALLY BE GREAT TO HAVE REAL EXECUTION COUNTS FOR THE numUses ADJUSTMENT.

```

<addInterference 89>≡ (88b)
let intMapAdd t v map = RM.add t ((try RM.find t map with Not_found -> 0) + v) map in
let addInterference block =
  let (head, last) = GR.goto_end (GR.unzip block) in
  let addEdges live defs uses =
    let live = live ++ defs in
    let addAllEdges def =
      RS.iter (fun liveNode -> addEdge target cg liveNode def) live in
    RS.iter addAllEdges defs in
  let add_and_upd live defs uses =
    ( RS.iter (fun t -> cg.numUses <- intMapAdd t 1 cg.numUses) uses
    ; RS.iter (fun t -> cg.numUses <- intMapAdd t 1 cg.numUses) defs
    ; RS.iter (fun t -> cg.liveRange <- intMapAdd t 1 cg.liveRange) live

```

```

; addEdges live defs uses
; uses ++ (live -- defs)
) in
let rec vis_head h live = match h with
| GR.First f -> ()
| GR.Head (h, m) ->
  let defs = defsm m in
  let uses = usesm m in
  let live =
    if isMoveInstruction target m then
      match getMove m with
      | Some move ->
        (let item = ilg_add cg.moveLG WorklistMove move in
         cg.moveListMap <- RS.fold (listMapAdd item) (defs ++ uses) cg.moveListMap;
         live -- uses)
      | None -> impossf "non-move passed isMoveInstruction()"
    else
      live in
  vis_head h (add_and_upd live defs uses) in
let vis_last l =
  let live = R.promote_rxset (G.add_live_spansl last (Live.live_out_last last)) in
  let defs = defsl l in
  let uses = usesl l in
  add_and_upd live defs uses in
vis_head head (vis_last last) in

```

MakeWorkList

Before beginning the graph coloring algorithm, the temps must be separated into worklists:

- Temps with high degree are placed on the `spillWorklist`
- Move-related temps are placed on the `freezeWorklist`
- Low-degree, non-move-related temps are placed on the `simplifyWorklist`

$\langle \text{makeWorklist } 90 \rangle \equiv \quad (107a)$
 $\langle \text{temp observers } 91a \rangle$

```

let makeWorklist cg proc =
  let addToLists = function
  (*
    | temp when RM.find temp.v cg.degreeMap >=
      RC.cardinal (RM.find temp.v cg.regToClass) ->
  *)
  (* JD MERGE
  *)
    | temp when RM.find temp.v cg.degreeMap >= cg.k ->
  (*
  *)
    ilg_switch cg.tempLG SpillWorklist temp
  | temp when moveRelated temp cg ->
    ilg_switch cg.tempLG FreezeWorklist temp
  | temp ->
    ilg_switch cg.tempLG SimplifyWorklist temp in
  let () = ilg_iter addToLists (!(cg.tempLG Initial)) in
  proc, true

```

It is useful to have observers that take a `temp` as an argument and return other temps with which `temp` interferes, moves in which `temp` is involved, or whether `temp` is move-related.

```

⟨temp observers 91a⟩≡ (90)
let adjacent temp cg =
  let (<>) = Pervasives.<> in
  List.filter (fun t -> t.list <> SelectStack && t.list <> Coalesced)
    (listMapFind temp cg.adjListMap)

let nodeMoves temp cg =
  List.filter (fun t -> t.list == ActiveMove || t.list == WorklistMove)
    (listMapFind temp.v cg.moveListMap)

let moveRelated temp cg =
  not (empty (nodeMoves temp cg))

```

Simplify

The `simplify` stage of graph coloring removes the easy-to-color temps in the `simplifyWorklist` from the interference graph. As temps are removed from the graph, the degrees of adjacent temps will decrease, possibly changing high-degree temps into easily colored low-degree temps and allowing previously constrained moves to be coalesced.

```

⟨simplify 91b⟩≡ (107a)
⟨remove temp 91c⟩

let simplify cg (cfg, (Procedure.{target = PA.T t} as proc)) =
  (*Debug.eprintf "color" "color::simplify()\n"; flush stderr;*)
  (match !(cg.tempLG SimplifyWorklist) with
  | Some temp ->
    let () = ilg_switch cg.tempLG SelectStack temp in
    let adj = adjacent temp.v cg in
    let () = if RM.find temp.v cg.degreeMap >= cg.k then enableMoves adj cg in
    let () = List.iter (decrementDegree cg t) (adjacent temp.v cg) in
    (cfg, proc), true
  | None -> (cfg, proc), false)

```

For each of the temps, `enableMoves` will allow previously uncoalescable moves to be reconsidered in the next coalescing phase.

```

⟨remove temp 91c⟩≡ (91b) 91d>
let enableMoves temps cg =
  let enableMove move =
    if move.list == ActiveMove then
      let hi_m = MoveMap.find move.v cg.hi - 1 in
      ( cg.hi <- MoveMap.add move.v hi_m cg.hi
      ; if hi_m <= 0 then ilg_switch cg.moveLG WorklistMove move
      ) in
  List.iter (fun temp -> List.iter enableMove (nodeMoves temp cg)) temps

```

When the degree of a temp is decremented, we also check whether the temp is now of low-degree. If so, we enable moves in which this temp is involved, and we place the temp on the appropriate worklist.

```

⟨remove temp 91d⟩+≡ (91b) <91c
let decrementDegree cg target temp =
  let d = RM.find temp.v cg.degreeMap in
  let () = cg.degreeMap <- RM.add temp.v (d - 1) cg.degreeMap in
  if d = cg.k
  then begin
    enableMoves (adjacent temp.v cg) cg;

```

```

    if moveRelated temp cg
    then ilg_switch cg.tempLG FreezeWorklist temp
    else ilg_switch cg.tempLG SimplifyWorklist temp
end

```

Coalesce

When two temps are connected by a move instruction, it is often possible to coalesce them into a single temp and eliminate the move instruction. It may not be possible to do this if the two temps interfere or if coalescing the temps will yield a graph that can no longer be colored using K colors. We use the Appel and George coalescing heuristic when attempting to coalesce a pre-colored node with a non-precolored temp, and we use the Briggs coalescing test for two non-precolored temps.

There are four possible cases for moves on the `worklistMoves` list:

- The source and destination of the move are the same temp
- The source and destination of the move are both precolored, or they interfere; we will never be able to coalesce this move
- The move may be safely coalesced
- It is not yet safe to coalesce this move

<coalesce 92> $\equiv$ (107a)
<definition of uniq 103a>
<coalesce move 93a>

```

let coalesce cg (cfg, (Procedure.{target = PA.T target} as proc)) =
  (*Debug.eprintf "color" "color::coalesce()\n"; flush stderr;*)
  (match !(cg.moveLG WorklistMove) with
  | Some move ->
  (*Debug.eprintf "color" "coalesces: %d\n" (ilg_fold (fun _ n -> n + 1)
    (!(cg.moveLG WorklistMove)) 0); flush stderr;*)
    let x = get (source move.v) cg in
    let y = get (dest move.v) cg in
    let (u, v) = if y.list == Precolored then (y, x) else (x, y) in
    let () =
      if eq_item (==) Register.eq u v
      then begin
        ilg_switch cg.moveLG CoalescedMove move
        ; addWorklist u cg
      end else if (v.list == Precolored) || (EdgeSet.mem (u.v, v.v) cg.adjSet)
      then begin
        ilg_switch cg.moveLG ConstrainedMove move
        ; addWorklist u cg
        ; addWorklist v cg
      end else if george move u v cg || briggs move u v cg
    (*
      end else if ((u.list == Precolored) &&
        (List.fold_left (fun rst t -> (ok t u cg) && rst)
          true (adjacent v.v cg))
        || ((not (u.list == Precolored)) &&
          (conservative (uniq ((adjacent u.v cg) @ (adjacent v.v cg)))
            cg)))
    *)
    then begin
      let (keep, coal) = if RS.mem u.v cg.spills then (v, u) else (u, v) in
      ( ilg_switch cg.moveLG CoalescedMove move

```

```

        ; combine target keep coal cg
        ; addWorklist keep cg
    )
end else
    ilg_switch cg.moveLG ActiveMove move in
    ((cfg, proc), true)
| None -> ((cfg, proc), false))

```

When moves are coalesced or constrained, the temps involved are placed on the `simplifyWorklist`.

```

⟨coalesce move 93a⟩≡ (92) 93b▷
let addWorklist temp cg =
  if not ((temp.list == Precolored) || (temp.list == Coalesced) ||
    (moveRelated temp cg) || (RM.find temp.v cg.degreeMap >= cg.k))
  then ilg_switch cg.tempLG SimplifyWorklist temp

```

`ok` is the heuristic for coalescing pre-colored temps. `tempr` is the pre-colored temp, and `tempt` is a temp adjacent to `tempv`, where we are trying to coalesce `tempv` with `tempr`, and `tempv` is not pre-colored.

```

⟨coalesce move 93b⟩+≡ (92) <93a 93c▷
(*
let ok tempt tempr cg =
  (RM.find tempt.v cg.degreeMap < cg.k)
  || (tempt.list == Precolored)
  || (EdgeSet.mem (tempt.v, tempr.v) cg.adjSet)
*)
let george move u v cg =
  u.list == Precolored && not (v.list == Precolored) &&
  let k = List.fold_left (fun k w -> if RM.find w.v cg.degreeMap >= cg.k &&
    not (EdgeSet.mem (u.v, w.v) cg.adjSet)
    then k + 1 else k) 0 (adjacent v.v cg) in
  if k > 0 then (cg.hi <- MoveMap.add move.v k cg.hi; false) else true

```

`conservative` implements the Briggs coalescing heuristic, which verifies that the temps we are trying to coalesce have fewer than `K` significant-degree neighbors.

```

⟨coalesce move 93c⟩+≡ (92) <93b 93d▷
let conservative neighbors cg =
  cg.k > (List.fold_left
    (fun k temp ->
      if RM.find temp.v cg.degreeMap >= cg.k then k + 1 else k)
    0 neighbors)
let briggs move u v cg =
  let k =
    List.fold_left (fun k n -> if RM.find n.v cg.degreeMap >= cg.k then k+1 else k)
    0 (uniq ((adjacent u.v cg) @ (adjacent v.v cg))) in
  if k < cg.k then true else (cg.hi <- MoveMap.add move.v (k - (cg.k - 1)) cg.hi; false)

```

`getAlias` returns the temp with which `node` has been coalesced.

```

⟨coalesce move 93d⟩+≡ (92) <93c 93e▷
let rec getAlias node cg =
  try
    if node.list == Coalesced
    then getAlias (RM.find (RM.find node.v cg.aliasMap) cg.regToItem) cg
    else node.v
  with Not_found -> Impossible.impossible "Not_found in Colorgraph.getAlias"

```

`combine` coalesces the temp `v` with the temp `u`. `v`'s neighbors and moves must be added to `u`.

```

⟨coalesce move 93e⟩+≡ (92) <93d
let get m cg = RM.find (getAlias (RM.find m cg.regToItem) cg) cg.regToItem
let combine target u v cg =
  let () = ilg_switch cg.tempLG Coalesced v in

```

```

let () = cg.aliasMap <- RM.add v.v u.v cg.aliasMap in

let () =
  if u.list == Precolored then
    let upd move =
      let x = get (source move.v) cg in
      let y = get (dest move.v) cg in
      if not (x.list == Precolored || y.list == Precolored) then
        ( cg.hi <- MoveMap.add move.v 0 cg.hi
          ; cg.moveListMap <- RM.add u.v (move :: listMapFind u.v cg.moveListMap)
            cg.moveListMap
        ) in
    List.iter upd (listMapFind v.v cg.moveListMap)
  else
    (* NOTE, UNION WOULD BE BETTER THAN @ HERE and above *)
    cg.moveListMap <- RM.add u.v (listMapFind u.v cg.moveListMap @
                                  listMapFind v.v cg.moveListMap) cg.moveListMap in
let losingHiDegNode = RM.find u.v cg.degreeMap >= cg.k &&
                      RM.find v.v cg.degreeMap >= cg.k in
(*
let moves = listMapFind u.v cg.moveListMap @ listMapFind v.v cg.moveListMap in
let () = cg.moveListMap <- RM.add u.v moves cg.moveListMap in
let () = enableMoves [v] cg in
*)
let () = List.iter (fun temp -> addEdge target cg temp.v u.v
                    ; decrementDegree cg target temp)
              (adjacent v.v cg) in
( if (RM.find u.v cg.degreeMap >= cg.k) && (u.list == FreezeWorklist)
  then ilg_switch cg.tempLG SpillWorklist u
; if losingHiDegNode then enableMoves (adjacent u.v cg) cg
)

```

Freeze

If we cannot simplify or coalesce a move, we choose a move-related, low-degree temp and freeze the moves it is involved in. We will no longer consider the frozen moves for coalescing, which will allow this temp to be simplified. Also, other temps may become non-move-related, allowing them to be simplified.

There is a policy decision of which temp to freeze; currently, the choice is rather arbitrary.

$\langle \text{freeze } 94a \rangle \equiv$ (107a)
 $\langle \text{freeze move } 94b \rangle$

```

let freeze cg proc =
  Debug.eprintf "color" "color::freeze()\n"; flush stderr;
  (match !(cg.tempLG FreezeWorklist) with
   | Some temp ->
     ( ilg_switch cg.tempLG SimplifyWorklist temp
       ; freezeMoves temp cg
       ; (proc, true)
     )
   | None -> (proc, false))

```

When a move is frozen, it must be removed from the `activeMoves` list. Also, if a low-degree temp becomes non-move-related when a move is frozen, this temp can be simplified from the interference graph.

$\langle \text{freeze move } 94b \rangle \equiv$ (94a)
 let freezeMoves temp cg =
 let freezeMove move =
 let x = RM.find (source move.v) cg.regToItem in
 let y = RM.find (dest move.v) cg.regToItem in

```

let v = if Register.eq (getAlias y cg) (getAlias temp cg)
      then RM.find (getAlias x cg) cg.regToItem
      else RM.find (getAlias y cg) cg.regToItem in
let () = ilg_switch cg.moveLG FrozenMove move in
if (Pervasives.<>) v.list Precolored && empty (nodeMoves v cg)) &&
  (RM.find v.v cg.degreeMap < cg.k)
then ilg_switch cg.tempLG SimplifyWorklist v in
List.iter freezeMove (nodeMoves temp cg)

```

Select Spill

We perform optimistic spilling, allowing high-degree nodes to be removed from the graph as a last option, in the hope that we will still be able to assign colors to these nodes.

The choice of which node to spill here is an important policy decision.

<selectSpill 95>≡ (107a)

```

(* claude: cg.liveRange/numUses (used by cheaper below) are
 * accumulated by addInterference's single linear backward walk
 * over each block, seeded from the real (correctly
 * fixed-pointed) Live.live_out_last - so they're accurate as a
 * *static* "how many program points is this live across"
 * count, but that count has no idea a block is inside a loop:
 * a value carried around a back-edge (e.g. a self-recursive
 * tail call's own induction variables) gets exactly the same
 * modest liveRange as any other value that merely spans one
 * block, even though spilling it costs a reload every runtime
 * iteration, not once. Confirmed via
 * docs/claude_notes/notes_debugging_techniques.txt entry 27:
 * without this, selectSpill kept *preferring* exactly such a
 * value round after round (their liveRange/degree ratio looks
 * "cheap"), and each spill's own reload is itself loop-carried
 * the same way, so it needed spilling next round too -
 * |get_regs cfg| grew by one forever on tests/cmm/
 * tail_from_c.c--'s self-recursive sp2_help, never converging.
 *
 * Real loop-nesting-depth-weighted cost would need dominator/
 * natural-loop analysis this file doesn't have (see
 * TODO/dominator.nw, still unintegrated). Approximating
 * instead: a block whose last has a successor at or before its
 * own position in a postorder DFS is a back edge (standard
 * test - a tree/forward edge always goes to a strictly later
 * postorder position, since postorder finishes a node only
 * after all its descendants), and Live.live_out_last of that
 * *source* block already contains (is a superset of) whatever
 * is live-in to the loop header on the other end - no need to
 * separately walk into the target block. Loop-carried
 * candidates are heavily deprioritized, not excluded: they can
 * still be chosen as a last resort if nothing else is on the
 * worklist, so this cannot itself get selectSpill stuck with
 * nothing to pick. *)
let loop_carried_regs cfg =
  let order = G.postorder_dfs cfg in
  let (_, posnum) =
    List.fold_left (fun (i, m) b -> (i + 1, UM.add (GR.id b) i m))
      (0, UM.empty) order in
  let pos u = try UM.find u posnum with Not_found -> -1 in
  G.fold_blocks (fun b regs ->
    let (_, last) = GR.goto_end (GR.unzip b) in
    let bpos = pos (GR.id b) in

```



```

    if List.exists (fun s -> pos s >= bpos) (GR.succs last)
    then R.promote_rxset (Live.live_out_last last) ++ regs
    else regs)
  RS.empty cfg
let selectSpill cg (cfg, proc) =
  Debug.eprintf "color" "color::selectSpill()\n"; flush stderr;
  let lst = !(cg.tempLG SpillWorklist) in
  match lst with
  | None -> ((cfg, proc), false)
  | Some t ->
    let loop_carried = loop_carried_regs cfg in
    let cheaper ({v = v} as t) (cost, _ as best) =
      let i2f = float_of_int in
      let cost' = (i2f (RM.find v cg.numUses)) /.
        (i2f (RM.find v cg.liveRange) *. i2f (RM.find v cg.degreeMap)) in
      let cost' = if RS.mem v loop_carried then cost' *. 1000. else cost' in
      if cost' <. cost then (cost', t) else best in
    let (_, spillee) = ilg_fold cheaper lst (infinity, t) in
    if RS.mem spillee.v cg.spills then
      impossf "Graph coloring ralloc chose to spill a previously spilled temp...."
    else
      ( ilg_switch cg.tempLG SimplifyWorklist spillee
      ; freezeMoves spillee cg
      ; ((cfg, proc), true)
      )

```

Assign Colors

We attempt to color the graph by popping temps off the select stack and choosing a color that is not used by the temp's neighbors. If there is no such color, we must spill the temp. After coloring the temps on the select stack, we assign colors to the coalesced nodes if there were no .

The `GraphColoring` exception is raised if the graph is uncolorable.

(assignColors 96a) ≡ (107a)
 exception GraphColoring

```

let assignColors cg (cfg, ({Procedure.target = PA.T target} as proc)) =
  Debug.eprintf "color" "color::assignColors()\n"; flush stderr;
  (colorTemp 96b)
  let colorSelectStack () =
    let () = ilg_iter colorTemp (!(cg.tempLG SelectStack)) in
  if is_some (!(cg.tempLG SelectStack)) then
    Impossible.impossible "Failed to color a temp?" in
  let colorCoalescedTemps () =
    let setColor temp =
      cg.colorMap <- RM.add temp.v (colorLookup temp.v cg.colorMap) cg.colorMap in
    ilg_iter setColor (!(cg.tempLG Coalesced)) in
  let () = colorSelectStack () in
  let () = match !(cg.tempLG Spilled) with
    | None -> colorCoalescedTemps ()
    | _ -> () in
  ((cfg, proc), true)

```

`colorTemp` is responsible for coloring a single temp. The possible colors (`okColors`) are calculated by excluding colors that used by neighbors temps. If any colors remain, we color the temp with one of them; otherwise, we must spill a temp.

The choices of colors assigned and neighbors spilled is a policy decision, currently arbitrary.

(colorTemp 96b) ≡ (96a)
 let colorLookup temp cmap =

```

let t = getAlias (RM.find temp cg.regToItem) cg in
if is_tmp target t then
  try RM.find t cmap
  with Not_found -> impossf "colorLookup(): uncolored temp %s" (RU.ToString.reg temp)
else
  Color.of_reg t in

let colorTemp temp =
  let removeUsedColor colors adjTemp =
    let t = RM.find (getAlias adjTemp cg) cg.regToItem in
    if t.list== Colored || t.list== Precolored
    then CS.remove (colorLookup t.v cg.colorMap) colors
    else colors in
  let possColors = possibleColors target cg temp.v in
  let okColors =
    List.fold_left removeUsedColor possColors (listMapFind temp.v cg.adjListMap) in

  if CS.is_empty possColors then
    impossf "No hardware register available for %s " (printReg temp.v)
  else if CS.is_empty okColors then
    if RS.mem temp.v cg.spills then
      <complain of spilling a spill temp, then die 97a>
    else ilg_switch cg.tempLG Spilled temp
  else
    begin
      cg.colorMap <- RM.add temp.v (CS.choose okColors) cg.colorMap;
      ilg_switch cg.tempLG Colored temp;
    end in

<complain of spilling a spill temp, then die 97a>≡ (96b)
begin
  Cfgutil.print_cfg cfg;
  print_cgInfo cg;
  impossf "Spilling a spill temp: %s" (printReg temp.v);
end

```

Apply Colors

`applyColors` is used to replace the temps in the CFG with the colors they are assigned. If a temp has not been assigned a color yet, the impossible exception will be raised. This exception indicates an error in the graph-coloring implementation.

```

<applyColors 97b>≡ (107a)
<define span updating functions 98a>
let applyColors cg (cfg, ({Procedure.target = PA.T target; Procedure.var_map = vMap} as proc)) =
  let find ((s,_,_), i, _) as t) =
    try RM.find t cg.colorMap
    with Not_found -> impossf "Uncolored temp %c%d" s i in
  let subst_reg t = if is_tmp target t then find t else Color.of_reg t in
  let rewrite_rtl = Color.subst subst_reg in
  let cfg = G.map_rtls rewrite_rtl cfg in
  let rewrite_span_loc live_in l =
    let guard = function RP.Reg r -> is_tmp target r | _ -> false in
    let map l = match l with
      | RP.Reg tmp ->
          if RSX.mem (Register.Reg tmp) live_in then
            Color.reg (find tmp)
          else
            raise RTD.DeadValue
      | _ -> impossf "GC reg allocator emitting RT data: guard failed" in

```

```

    Rtlutil.Subst.loc_of_loc ~guard ~map 1 in
  ( update_spans rewrite_span_loc cfg
  ; ((cfg, proc), true)
  )

```

It's useful to have a generic function to replace temporaries in the spans. We parameterize this function over an update function that takes a set of registers `live_in` to a node, as well as an rtl to rewrite.

```

⟨define span updating functions 98a⟩≡ (97b)
let update_spans upd cfg =
  let block b =
    let (head, last) = GR.goto_end (GR.unzip b) in
    let live_inl = Live.live_in_last last in
    let rec vis_head h live_out = match h with
      | GR.First (GR.Label (_, lbl), _, {contents = Some ss}) as f ->
        RTD.upd_spans (upd (G.add_live_spansf f live_out)) ss
      | GR.First _ -> ()
      | GR.Head (h, m) -> vis_head h (Live.live_in_middle live_out m) in
    ( match last with
      | GR.Call {GR.cal_spans = Some ss; cal_i = i} -> RTD.upd_spans (upd live_inl) ss
      | _ -> ()
      ; vis_head head live_inl
    ) in
  G.iter_blocks block cfg

```

Reset/Update Program

After an iteration of graph coloring in which spilled temps are generated, `resetProgram` inserts the necessary loads and stores to spill the temps in the CFG and resets the `cg` structure for another round of graph coloring.

```

⟨resetProgram 98b⟩≡ (107a) 99a▷
⟨insert spills 99c⟩

let resetProgram cg (cfg, proc) =
  Debug.eprintf "color" "color::resetProgram()\n"; flush stderr;
  (* handle addition of new temps to the cfg (those in spilledTemps) *)
  let (cfg, newSpillMap, newTemps) = rewrite cfg proc (!(cg.tempLG Spilled)) in
  Debug.eprintf "color" " color::resetProgram() -- done with rewrite\n"; flush stderr;
  let () = if is_some (!(cg.tempLG Initial))
    then Impossible.impossible "temps still on initial list" in
  let () = List.iter (fun t -> cg.spills <- RS.add t cg.spills) newTemps in
  let () = cg.spillMap <- RM.fold RM.add newSpillMap cg.spillMap in

  let () = cg.tempLG Precolored := None in
  let () = cg.tempLG Initial := None in
  let () = cg.tempLG SimplifyWorklist := None in
  let () = cg.tempLG FreezeWorklist := None in
  let () = cg.tempLG SpillWorklist := None in
  let () = cg.tempLG Spilled := None in
  let () = cg.tempLG Coalesced := None in
  let () = cg.tempLG Colored := None in
  let () = cg.tempLG SelectStack := None in

  let () = cg.moveLG CoalescedMove := None in
  let () = cg.moveLG ConstrainedMove := None in
  let () = cg.moveLG FrozenMove := None in
  let () = cg.moveLG WorklistMove := None in
  let () = cg.moveLG ActiveMove := None in

  let () = cg.adjSet <- EdgeSet.empty in
  let () = cg.regToItem <- RM.empty in

```

```

let () = cg.adjListMap <- RM.empty in
let () = cg.degreeMap <- RM.empty in
let () = cg.moveListMap <- RM.empty in
let () = cg.aliasMap <- RM.empty in
let () = cg.colorMap <- RM.empty in
((cfg, proc), true)

```

The simple predicate `haveSpilledTemps` may be used to determine whether the graph-coloring algorithm has terminated.

```

⟨resetProgram 99a⟩+≡ (107a) <98b 99b>
let haveSpilledTemps cg proc = (proc, is_some (!(cg.tempLG Spilled)))

```

`updateProgram` removes coalesced moves from the CFG. It may be desirable to call this stage even after graph coloring has succeeded in placing every temp.

```

⟨resetProgram 99b⟩+≡ (107a) <99a
let updateProgram cg (cfg, proc) =
  let coalesced = ilg_fold (fun n set -> MoveSet.add n.v set)
    (!(cg.moveLG CoalescedMove)) MoveSet.empty in
  let filter_copies (first, tail) =
    let is_copy middle =
      match getMove middle with
      | Some rs -> MoveSet.mem rs coalesced
      | _ -> false in
    let rec filt head tail = match tail with
      | GR.Tail (m, t) when is_copy m -> filt head t
      | GR.Tail (m, t) -> filt (GR.Head (head, m)) t
      | GR.Last _ -> GR.zipht head tail in
    filt (GR.First first) tail in
  ((G.of_blocks (UM.map filter_copies (G.to_blocks cfg)), proc), true)

```

The `reads_writes` function checks whether a location `loc` is read or written by an RTL. It returns a pair (r, w) of bool values. The first component r is true iff `loc` is read by `rtl`, the second w iff `loc` is written by `rtl`.

```

⟨insert spills 99c⟩≡ (98b) 99d>
let reads_writes rtl loc =
  let read reg (r,w) = (r || Register.eq loc reg, w) in
  let write reg (r,w) = (r, w || Register.eq loc reg) in
  Rtlutil.ReadWrite.fold_promote ~read ~write rtl (false,false)

```

We fold over each block to insert spills and loads, and to replace spilled temps in the rtl's.

```

⟨insert spills 99d⟩+≡ (98b) <99c 102a>
let updateBlock proc spilllees uid block (blockMap, spillMap, newTemps) =
  let len = ilg_fold (fun _ rst -> rst + 1) spilllees 0 in
  Debug.eprintf "color" " color::updateBlock(%d)\n" len; flush stderr;
  let PA.T tgt = proc.Procedure.target in
  ⟨define update functions for each type of node in the cfg 100b⟩

```

```

let (head, last) = GR.goto_end (GR.unzip block) in
let m = (proc, tgt.T.machine, proc.exp_of_lbl) in
let add_nodes shuffles (g : G.zgraph) =
  let add_subgraph g block =
    let (subg, _) = G.block2cfg m block in
    G.splice_focus_entry g (G.unfocus subg) in
  List.fold_left add_subgraph g shuffles in
let rec fold_block g h spillMap newTemps = match h with
| GR.First _ -> (G.to_blocks (G.unfocus g), spillMap, newTemps)
| GR.Head (h, m) ->
  let (middle, loads, spillMap, spills, newTemps) =
    ilg_fold updateMiddle spilllees (m, [], spillMap, [], newTemps) in

```

```

    let g = add_nodes spills g in
    let ((head, tail), blockmap) = G.openz g in
    let tail = GR.Tail (middle, tail) in
    let g = G.tozgraph ((head, tail), blockmap) in
    let g = add_nodes loads g in
    fold_block g h spillMap newTemps in
let (last, loads, spillMap, spills, newTemps) =
  ilg_fold updateLast spilltees (last, [], spillMap, [], newTemps) in
⟨verify that there are no spills 100a⟩
let g = G.tozgraph ((GR.First (GR.first (GR.unzip block)), GR.Last last), blockMap) in
fold_block (add_nodes loads g) head spillMap newTemps

⟨verify that there are no spills 100a⟩≡ (99d)
(match spills with
| [] -> ()
| _ -> imposs "colorgraph: unexpected spill while allocating last node");

(*
let (head, last) = GR.goto_end (GR.unzip block) in
let add_inst_lst = List.fold_left (fun tl ld -> GR.Tail (GR.Instruction ld, tl)) in
let rec vis_head h tail spillMap newTemps = match h with
| GR.First f -> (UM.add uid (f, tail) blockMap, spillMap, newTemps)
| GR.Head (h, m) ->
  let (middle, loads, spillMap, spills, newTemps) =
    ilg_fold updateMiddle spilltees (m, [], spillMap, [], newTemps) in
  let tail = List.fold_left add_inst_lst tail spills in
  let tail = GR.Tail (middle, tail) in
  let tail = List.fold_left add_inst_lst tail loads in
  vis_head h tail spillMap newTemps in
let (last, loads, spillMap, _, newTemps) =
  ilg_fold updateLast spilltees (last, [], spillMap, [], newTemps) in
let tail = List.fold_left add_inst_lst (GR.Last last) loads in
vis_head head tail spillMap newTemps
*)

```

The heart of the spiller is `update` that is applied to every node in the CFG. If node reads or writes the spilltee, it must be updated such that it uses a new temporary `tmp` instead. Before and after the node, `tmp` must be read from or written to the memory location `mem`, where the value of the spilltee is held.

```

⟨define update functions for each type of node in the cfg 100b⟩≡ (99d)
let updateInstr map_rtl instr spilltee
  (node, loads, spillMap, spills, newTemps as accum) =
let spilltee = spilltee.v in
let reads, writes = reads_writes instr spilltee in
if reads || writes then
  let (mem, spill_regs) = RM.find spilltee spillMap in
  let tmp = Talloc.Multiple.reg_like proc.temps spilltee in
  let subst = fun x -> if Register.eq x spilltee then tmp else x in
  let map = Rtlutil.Subst.reg ~map:subst in
  let loads =
    if reads then tgt.T.machine.T.reload proc mem tmp :: loads else loads in
  let spills =
    if writes then tgt.T.machine.T.spill proc tmp mem :: spills else spills in
  let spillMap = RM.add tmp (mem, tmp::spill_regs) spillMap in
  let newTemps = tmp::newTemps in
  (map_rtl map, loads, spillMap, spills, newTemps)
else accum in

```

(* claude: a Call/Cut/Jump/Return's uses of real registers are
 * not always expressed in its own RTL (GR.last_instr):

```

* - Call carries a separate cal_uses field; Cut/Jump/Return
*   their own positional uses:regs (zipcfg.ml's
*   add_inedge_uses, which build()/addInterference already
*   read through usesl/defsl, folds these in).
* - Call/Cut's continuation edges (cal_contedges / Cut's own
*   edgelist) each carry their own defs:regs and kills:regs
*   (zipcfg.ml's union_over_outedges, which defsl also reads)
*   - registers considered defined/killed across that edge,
*   independent of cal_i's own RTL.
* reads_writes/updateInstr above only inspect the RTL, and
* Zipcfg.map_rtl1 (used below to rewrite cal_i) only touches
* a contedge's assertion field, never its defs/kills. So a
* spilllee referenced *only* via one of these extra fields was
* never substituted: the old register survived the rewrite
* untouched, so the very next build() saw it as still
* needing a color and picked it right back out of the spill
* worklist - forever (reproduced on tests/cmm/add.c--, a
* trivial one-call function: selectSpill chose the same temp
* every single round; resetProgram's "already spilled" check
* never caught it because that check only tracks the
* *replacement* temps from past rounds, not the original -
* confirmed the spilled temp appeared in a continuation
* edge's kills/defs, not cal_uses, by tracing every
* candidate field by hand). Fixed by also substituting
* spilllee for a fresh reload temp inside all of these
* fields directly, mirroring what updateInstr already does
* for the RTL itself. *)
let contedges_of = function
  | GR.Call c -> c.GR.cal_contedges
  | GR.Cut (_, es, _) -> es
  | _ -> [] in
let last_extra_uses last =
  let ce_regs =
    List.fold_left
      (fun r (ce : G.contedge) -> RSX.union r (RSX.union ce.G.defs ce.G.kills))
      RSX.empty (contedges_of last) in
  let own_uses = match last with
    | GR.Call c -> c.GR.cal_uses
    | GR.Cut (_, _, u) | GR.Jump (_, u, _) | GR.Return (_, _, u) -> u
    | _ -> RSX.empty in
  RSX.union own_uses ce_regs in
let subst_last_extra_uses spilllee tmp last =
  let subst_elt = function
    | R.Reg r when Register.eq r spilllee -> R.Reg tmp
    | R.Slice (w, i, r) when Register.eq r spilllee -> R.Slice (w, i, tmp)
    | x -> x in
  let subst_ce (ce : G.contedge) =
    { ce with G.defs = RSX.map subst_elt ce.G.defs
      ; G.kills = RSX.map subst_elt ce.G.kills } in
  match last with
  | GR.Call c -> GR.Call { c with GR.cal_uses = RSX.map subst_elt c.GR.cal_uses
    ; GR.cal_contedges = List.map subst_ce c.GR.cal_contedges }
  | GR.Cut (r, es, u) -> GR.Cut (r, List.map subst_ce es, RSX.map subst_elt u)
  | GR.Jump (r, u, tg) -> GR.Jump (r, RSX.map subst_elt u, tg)
  | GR.Return (i, r, u) -> GR.Return (i, r, RSX.map subst_elt u)
  | _ -> last in
let updateLast spilllee (last, loads, spillMap, spills, newTemps) =
  let spilllee_v = spilllee.v in
  let last, loads, spillMap, newTemps =
    if RSX.mem (R.Reg spilllee_v) (last_extra_uses last) then

```

```

    let (mem, spill_regs) = RM.find spill_v spillMap in
    let tmp = Talloc.Multiple.reg_like proc.temps spill_v in
    let loads = tgt.T.machine.T.reload proc mem tmp :: loads in
    let spillMap = RM.add tmp (mem, tmp :: spill_regs) spillMap in
    (subst_last_extra_uses spill_v tmp last, loads, spillMap, tmp :: newTemps)
  else (last, loads, spillMap, newTemps) in
match updateInstr (fun map -> G.map_rtl map map last) (GR.last_instr last) spill_v
  (last, loads, spillMap, [], newTemps) with
| (_, _, _, [], _) as x -> x
| _ -> impossf "GC reg allocator trying to spill from a last node" in

let updateMiddle spill_v (middle, loads, spillMap, spills, newTemps) =
  updateInstr (fun map -> G.map_rtl map middle) (GR.mid_instr middle) spill_v
    (middle, loads, spillMap, spills, newTemps) in

```

In addition to adding store and fetch instructions to replace the spilled temps in the CFG, `rewrite` returns the new spill temps that are generated.

```

⟨insert spills 102a⟩+≡ (98b) <99d
let rewrite cfg (Procedure.{target = PA.T target} as proc) spill_v =
  let spillMap =
    ilg_fold (fun s map -> RM.add s.v (spill_slot_for proc s.v, []) map)
      spill_v RM.empty in

  let rewrite_span_loc live_in l =
    let guard = function RP.Reg r -> is_tmp target r && RM.mem r spillMap
      | _ -> false in
    let map l = match l with
      | RP.Reg tmp ->
        if RSX.mem (Register.Reg tmp) live_in then
          Rtl.Dn.loc (AU.aloc (fst (RM.find tmp spillMap)) (Register.width tmp))
        else raise RTD.DeadValue
      | _ -> impossf "guard failed" in
    Rtlutil.Subst.loc_of_loc ~guard ~map l in
  let () = update_spans rewrite_span_loc cfg in
  let blocks = G.to_blocks cfg in
  Debug.eprintf "color" " color::rewrite() -- done with updateSpans\n"; flush stderr;
  let (blocks, spillMap, newTemps) =
    UM.fold (updateBlock proc spill_v) blocks (UM.empty, spillMap, []) in
  (G.of_blocks blocks, spillMap, newTemps)

```

Common Utilities

(* check whether two temps interfere - whether they have overlapping reg * sets. * this may also be misleading - if they only partially overlap, this * may or may not be a good way to indicate interference - see the * smith, holloway paper.... *)

The node observers are mostly straightforward. `isMoveInstruction` verifies that the instruction is an assignment from one temp to another, where both are in the same register space.

```

⟨node observers 102b⟩≡ (107a)
let source = fst
let dest   = snd

let is_some = function Some _ -> true | None -> false

(*
* singleAssignment returns a (Register.t * Register.t) option
* First get the assignment, then verify that the source and dest are
* in the same reg. space

```

```

*)
let isMoveInstruction t mid =
  (* WE NEED TO DISCUSS HOW TO HANDLE THIS PROPERLY: V IS NOT A SPACE
     IN THE TARGET, WHICH MEANS THE FUNCTIONS IN TARGET2.NW DON'T WORK *)
  let is_vfp r = Vfp.is_vfp (RP.Reg r) in
  match Rtlutil.RTLType.singleAssignment (GR.mid_instr mid) with
  | Some (((s1,_,_) as r1), ((s2,_,_) as r2)) ->
    (not (is_vfp r1 || is_vfp r2)) &&
    (RU.Eq.space s1 s2 || (is_tmp t r1 && Target.fits t s1 r2)
      || (is_tmp t r2 && Target.fits t s2 r1))
  | _ -> false
let getMove mid = Rtlutil.RTLType.singleAssignment (GR.mid_instr mid)

```

We'd like to sort before checking uniqueness, but there's no safe, easy way to compare nodes in an imperative list. So this algorithm is quadratic. There's probably a better way (maybe hashing?).

<definition of uniq 103a>≡ (92)

```

let uniq nodes =
  let map = List.fold_left (fun map i -> RM.add i.v i map) RM.empty nodes in
  RM.fold (fun _ v rst -> v :: rst) map []

```

The interface provides the modules to embed the graph coloring functions in the Lua interpreter.

<colorgraph.mli 103b>≡

```

val ralloc : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

```

A few convenience functions for handling lists. Perhaps the lists that use the set operations should be converted to register sets.

<list as set 103c>≡ (107a)

```

let empty lst = match lst with
| [] -> true
| _ -> false

let listMapFind key listMap =
  try RM.find key listMap with Not_found -> []
let listMapAdd value key listMap =
  RM.add key (value::(listMapFind key listMap)) listMap

```

11.5.5 Lua registration

<colorgraph.ml 103d>≡
<colorgraph.ml 103e>

<colorgraph.ml 103e>≡ (103d) 104a▷

```

module AU = Automatonutil
module Dn = Rtl.Dn
module RTD = RuntimeData
module P = Proc
module PA = Preast2ir
module R = Register
module RP = Rtl.Private
module RC = Registerclass.RegisterClass
module RM = Register.Map
module RS = Register.Set
module RSX = Register.SetX
module RU = Rtlutil

```

```

(* claude: Nopoly.(==) below needs this open; every other .nw-tangled
   * .ml in this tree gets it prepended automatically by the (now-dropped)
   * generator, so it has to be added by hand here. *)
open Nopoly
let ( ++ ) = RS.union

```



```

let ( -- ) = RS.diff
let imposs = Impossible.impossible
let impossf fmt = Printf.kprintf Impossible.impossible fmt

let () = Debug.register "colorgraph" "generalized graph coloring allocator"
<imperative lists 81c>

<colorgraph.ml 104a>+≡ (103d) <103e 104b>
module G = Zipcfg
module GR = Zipcfg.Rep
module UM = Unique.Map

let irwk = Rtlutil.ReadWriteKill.sets_promote
let defsm middle =
  let uses, defs, kills = irwk (GR.mid_instr middle) in
  defs ++ kills

let defsl last =
  let uses, defs, kills = irwk (GR.last_instr last) in
  let defs = defs ++ kills in
  let p = R.promote_rxset in
  p (G.union_over_outedges last (fun _ -> R.rset_to_rxset defs)
    (fun {G.node = n'; G.defs = d; G.kills = k} ->
      (R.rset_to_rxset (defs ++ p d ++ p k))))

let usesm middle =
  let uses, _, _ = irwk (GR.mid_instr middle) in
  R.promote_rxset (R.rset_to_rxset uses)
let usesl last =
  let uses, _, _ = irwk (GR.last_instr last) in
  R.promote_rxset (G.add_inedge_uses last (R.rset_to_rxset uses))

<colorgraph.ml 104b>+≡ (103d) <104a
let spill_slot_for proc ((_,_,ms), _, c) =
  Automaton.allocate proc.Procedure.priv (Cell.to_width ms c) "" 1
module type COLOR = sig
  type t
  type set
  val reg : t -> Rtl.Private.loc
  val of_reg : Register.t -> t
  val fits : ('a, 'b, 'c) Target.t -> Rtl.space -> t -> bool
  val to_string : t -> string
  val subst : (Register.t -> t) -> Rtl.rtl -> Rtl.rtl
  val may_alias : t -> t -> bool (* share hardware resources *)
module Set : sig
  val empty : set
  val singleton : t -> set
  val of_list : t list -> set
  val of_regs : Register.Set.t -> set

  val remove : t -> set -> set
  val filter : (t -> bool) -> set -> set
  val exists : (t -> bool) -> set -> bool

  val is_empty : set -> bool
  val choose : set -> t
  val cardinal : set -> int (* can't work with infinite sets; must eliminate *)
  val overlap : set -> set -> bool (* true iff intersection is not empty *)
end
end

```

```

module RegColor : COLOR with type t = Register.t and type set = RS.t = struct
  type t = Register.t
  type set = RS.t
  let reg r = RP.Reg r
  let of_reg r = r
  let fits = Target.fits
  let to_string = RU.ToString.reg
  let subst f = Rtlutil.Subst.reg ~map:f
  let may_alias = RU.MayAlias.regs
  module Set = struct
    include RS
    let of_list l = List.fold_left (fun set reg -> RS.add reg set) RS.empty l
    let of_regs s = s
    let overlap s1 s2 = RS.is_empty (RS.inter s1 s2)
  end
end

module PreMake (Color : COLOR) = struct
  module CS = Color.Set
  <graph coloring types 81a>
  (* claude: this used to be wrapped in a Lua/Backplane binding shell,
   * mirroring x86backend.ml's decision not to port Backplane - the real
   * algorithm below never referenced Lua/Interp/V/register itself (only
   * the deleted stageFn table at the very end of this chunk did), so it
   * moves out unchanged into a plain PreMake, and [[ralloc]] at the end of
   * this functor is the new straight-line OCaml driver replacing whatever
   * Lua policy script used to sequence build/makeWorklist/simplify/coalesce/
   * freeze/selectSpill/assignColors/resetProgram/applyColors (that script
   * itself, referenced as Ralloc.color = ColorGraph.color in
   * LUA/lua-cmm-driver/luacompiled.nw:667, is not present anywhere in this
   * fork or the upstream checkout - reconstructed from the stage names/
   * signatures here, which match Appel's iterated-register-coalescing
   * algorithm exactly). Old shell, kept for comparison against the port:
   *
   * module GCT : Lua.Lib.USERTYPE with type 'a t = 'a colorGraph = struct
   *   type 'a t = 'a colorGraph
   *   let tname = "Graph Coloring"
   *   let eq _ x y = x == y (* may be dangerous *)
   *   let to_string vs _ = "<cgInfo>"
   * end
   * module Make (BackplaneT : Lua.Lib.TYPEVIEW with type 'a t = 'a Backplane.M.action)
   *   (GCT : Lua.Lib.TYPEVIEW with type 'a t = 'a GCT.t
   *     and type 'a combined = 'a BackplaneT.combined)
   *   (ProcT : Lua.Lib.TYPEVIEW with type 'a t = Ast2ir.proc
   *     and type 'a combined = 'a BackplaneT.combined)
   *   : Lua.Lib.USERCODE with type 'a userdata' = 'a BackplaneT.combined =
   *   struct
   *     type 'a userdata' = 'a BackplaneT.combined
   *     module M (Interp : Lua.Lib.CORE
   *       with type 'a V.userdata' = 'a BackplaneT.combined) = struct
   *       module V = Interp.V
   *       let cgmap = GCT.makemap V.userdata V.projection
   *
   *       module RT = Register.RT (Interp)
   *       let register = RT.map
   *     *)
   *
   *   <graph coloring builtins 107a>
   *
   *   (* claude: the driver that used to be a Lua policy script (see the comment

```

```

* at the top of [[graph coloring builtins]] - Appel's iterated register
* coalescing main loop: build the interference graph, repeatedly prefer
* simplify over coalesce over freeze over spilling a candidate until none
* of those can make progress, assign colors, and - if that produced any
* spills - rewrite the program to materialize them and start over from
* build; otherwise remove the now-redundant coalesced moves and rewrite
* every temp to its assigned color. *)
(* claude: "ralloc-cfg" is already Debug.register'd by regalloc/flowra.ml
* - both link into the same qc binary, and Debug.register errors on a
* second registration of the same word - so this reuses it via
* Debug.on/Debug.eprintf only, matching Flowra.ralloc's own before/after
* dump exactly. *)
let ralloc _ (cfg, proc) =
  let cg = cgInfo in
  if Debug.on "ralloc-cfg" then
    (Debug.eprintf "ralloc-cfg" "BEFORE register allocation:\n"; Cfgutil.print_cfg cfg);
  let rec fixpoint (cfg, proc) =
    let (cfg, proc), changed = simplify cg (cfg, proc) in
    if changed then fixpoint (cfg, proc) else
    let (cfg, proc), changed = coalesce cg (cfg, proc) in
    if changed then fixpoint (cfg, proc) else
    let (cfg, proc), changed = freeze cg (cfg, proc) in
    if changed then fixpoint (cfg, proc) else
    let (cfg, proc), changed = selectSpill cg (cfg, proc) in
    if changed then fixpoint (cfg, proc) else
    (cfg, proc) in
  let rec main (cfg, proc) =
    (* claude: build()/get_regs below calls Live.live_out_last directly,
    * which only *reads* a uid-keyed property table (Unique.Prop) - it
    * does not compute liveness itself. That table is populated once,
    * before ralloc even runs (arch/x86/x86backend.ml's Phases.liveness),
    * and nothing in this retry loop ever re-solves it, even though
    * resetProgram rewrites the cfg every round (inserting spill/reload
    * code). So build() on round 2+ was reading round-0's liveness
    * facts against a structurally different cfg: a temp whose every
    * real reference had already been correctly rewritten away by the
    * spill insertion could still show up as "live" per the stale
    * table, so selectSpill kept re-picking the exact same temp every
    * round with no way to ever converge - reproduced as a genuine
    * infinite loop on tests/cmm/add.c--, a trivial one-call function
    * (confirmed by ruling out every other place a spilled register
    * could still be referenced - RTL, cal_uses/cut/jump/return uses,
    * continuation-edge defs/kills, spans - all correctly cleared by
    * the spill rewrite; only the separately-cached liveness fact
    * survived). Re-solving fresh here, every round (round 1 included,
    * for consistency, even though it happens to already be fresh from
    * Phases.liveness), is what actually fixes it. *)
    let cfg, _ = Dataflow.B.rewrite (Dataflow.B.anal Live.live_in) cfg in
    let (cfg, proc), _ = clearCGInfo cg (cfg, proc) in
    let (cfg, proc), _ = build cg (cfg, proc) in
    let (cfg, proc), _ = makeWorklist cg (cfg, proc) in
    let (cfg, proc) = fixpoint (cfg, proc) in
    let (cfg, proc), _ = assignColors cg (cfg, proc) in
    let (cfg, proc), spilled = haveSpilledTemps cg (cfg, proc) in
    if spilled then
      let (cfg, proc), _ = resetProgram cg (cfg, proc) in
      main (cfg, proc)
    else
      let (cfg, proc), _ = updateProgram cg (cfg, proc) in
      applyColors cg (cfg, proc) in

```

```

let (cfg, proc), changed as result = main (cfg, proc) in
if Debug.on "ralloc-cfg" then
  (Debug.eprintf "ralloc-cfg" "AFTER register allocation:\n"; Cfgutil.print_cfg cfg);
result
end

```

```

module XXX = PreMake(RegColor)
include XXX

```

```

<graph coloring builtins 107a>≡ (104b) 107b▷
let verb    = 18
let cfgSay  = Verbose.say verb
let cgSay   = cfgSay

<list as set 103c>
<node observers 102b>
<print CGInfo 109a>
<init cgInfo 85>
<build 87c>
<makeWorklist 90>
<simplify 91b>
<coalesce 92>
<freeze 94a>
<selectSpill 95>
<assignColors 96a>
<applyColors 97b>
<resetProgram 98b>

```

Embedding the functions in the Lua interpreter.

```

<graph coloring builtins 107b>+≡ (104b) <107a
(* claudie: dropped the Lua module-registration table this chunk used
 * to end with, exposing build/makeWorklist/.../applyColors to Lua
 * under "ColorGraph" - see the comment above [[graph coloring
 * builtins]]'s opening. [[ralloc]] below calls the same functions
 * directly. Old table, kept for comparison against the port:
 *
 * let proc = ProcT.makemap V.userdata V.projection
 * let ( **-> ) = V.( **-> )
 * let stageFn = V.efunc (cgmap **-> proc **-> V.resultpair proc V.bool)
 * let graph_coloring_module =
 *   [ "build",          stageFn build
 *     ; "makeWorklist", stageFn makeWorklist
 *     ; "simplify",     stageFn simplify
 *     ; "coalesce",     stageFn coalesce
 *     ; "freeze",       stageFn freeze
 *     ; "selectSpill",  stageFn selectSpill
 *     ; "assignColors", stageFn assignColors
 *     ; "haveSpilledTemps", stageFn haveSpilledTemps
 *     ; "clearCGInfo",  stageFn clearCGInfo
 *     ; "resetProgram", stageFn resetProgram
 *     ; "updateProgram", stageFn updateProgram
 *     ; "applyColors",  stageFn applyColors
 *     ; "printCG",      stageFn printCGInfo
 *     ; "cgInfo",       cgmap.V.embed cgInfo
 *     (* not used anywhere except maybe for debugging:
 *     "setRegisters", V.efunc (V.value **-> V.result V.unit)
 *       (fun value -> cgInfo.luaColorOverride <- value;
 *         cgInfo.colors <- SpaceMap.empty) *)
 *   ]
 *

```

```
* let init = Interp.register_module "ColorGraph" graph_coloring_module
*)
```

N.B. I've removed the infrastructure supporting `setRegisters`. If it is restored, the correct way to do it is with dynamic type dispatch. Also note I've moved some code into `Aux.List.take`. —NR

Lua startup code

This startup code sets up the right passes.

(Lua startup code for Colorgraph module 108)≡

```
-----
-- Graph-Coloring Register Allocator
-----
```

```
function ColorGraph_init (CG, B) -- carefully don't define table
  CG = CG or error('predefined ColorGraph is nil??') -- has some primitives

  CG.makeGraph =
    B.seq {
      Liveness.liveness,
      { fn = CG.build, uses = {'liveness', 'no vars'}, creates = 'no temps' }
    }

  CG.pcg = B.ignore(CG.printCG)

  CG.orderVars = B.seq
    { CG.makeWorklist
      , B.fix(B.unless_do(CG.simplify,
                          B.unless_do(CG.coalesce,
                                      B.unless_do(CG.freeze,
                                                  CG.selectSpill))))
    }

  CG.color =
    B.share
      ( CG.cgInfo
        , "cgInfo"
        , B.seq
          { CG.clearCGInfo
            , B.fix
              ( B.seq
                { B.ignore (B.seq
                          { CG.makeGraph
                            , CG.pcg
                            , CG.orderVars
                            , CG.pcg
                            , CG.assignColors
                          })
                , B.when_do
                  ( CG.haveSpilledTemps
                    , CG.resetProgram
                  )
              }
            )
          , B.seq
            { CG.updateProgram
              , CG.applyColors
              , CG.pcg
            }
          )
      )
```

```

    }
  }
)
end

ColorGraph_init (ColorGraph, Backplane)

```

11.5.6 Debugging support

Printing functions used for debugging purposes. Functions are provided for printing the `cgInfo` data structure.

```

⟨print CGInfo 109a⟩≡ (107a) 109b▷
let regLists cgInfo =
  [ ("Precolored", !(cgInfo.tempLG Precolored))
    ; ("Initial", !(cgInfo.tempLG Initial))
    ; ("SimplifyWorklist", !(cgInfo.tempLG SimplifyWorklist))
    ; ("FreezeWorklist", !(cgInfo.tempLG FreezeWorklist))
    ; ("SpillWorklist", !(cgInfo.tempLG SpillWorklist))
    ; ("spilledTemps", !(cgInfo.tempLG Spilled))
    ; ("coalescedTemps", !(cgInfo.tempLG Coalesced))
    ; ("coloredTemps", !(cgInfo.tempLG Colored))
    ; ("SelectStack", !(cgInfo.tempLG SelectStack))
  ]
let regLists cgInfo =
  [ ("Spills", RS.elements cgInfo.spills)
    (* ; ("AllColors", cgInfo.allColors) *)
  ]
let moveLists cgInfo =
  [ ("coalescedMoves", !(cgInfo.moveLG CoalescedMove))
    ; ("ConstrainedMoves", !(cgInfo.moveLG ConstrainedMove))
    ; ("FrozenMoves", !(cgInfo.moveLG FrozenMove))
    ; ("WorklistMoves", !(cgInfo.moveLG WorklistMove))
    ; ("ActiveMoves", !(cgInfo.moveLG ActiveMove))
  ]

```

Several helper functions are provided for printing parts of the `cgInfo` record.

```

⟨print CGInfo 109b⟩+≡ (107a) <109a 109c>
let printReg ((s,_,_), i, w) = Printf.sprintf "%c%d" s i
let printColor = Color.to_string
let printMove move = printReg (source move)
                        ^ " -> "
                        ^ printReg (dest move)
let printMap p1 p2 mapName sm =
  cgSay [" " ^ mapName ^ ":\n"];
  RM.iter (fun tmp s -> cgSay [ " "
                                ; p1 tmp
                                ; " -> "
                                ; p2 s
                                ; "\n"
                              ]) sm
let printStringMap mapName sm = printMap printReg (fun x -> x) mapName sm
let printAliasMap mapName am = printMap printReg printReg mapName am
let printColorMap mapName am = printMap printReg printColor mapName am

```

The `print_cgInfo` function prints the graph coloring information in a readable format. `printCGInfo` is embedded in the Lua interpreter to provide access to `print_cgInfo`. Printing functions for the `cfInfo` record.

```

⟨print CGInfo 109c⟩+≡ (107a) <109b>
let print_cgInfo cgInfo =
  let _ = cgSay ["\nCGINFO\n"] in

```

```

let printList iter pr (name, itemlist) =
  let indent = "      " in
  cgSay ["      "; name; ":\n"];
  iter (fun r -> cgSay [indent; pr r; "\n"])
        itemlist in
let printItemList pr (name, itemlist) =
  match itemlist with
  | [] -> ()
  | i -> printList List.iter pr (name, itemlist) in
let printItemIList pr (name, itemlist) =
  match itemlist with
  | None -> ()
  | Some _ -> printList (fun fn -> ilg_iter (fun i -> fn i.v))
                        pr (name, itemlist) in
let printRegList lst = printItemList printReg lst in
let printRegIList lst = printItemIList printReg lst in
let printMoveIList lst = printItemIList printMove lst in
let printAdjListMember key lst =
  cgSay ["      "; printReg key; ": "];
  List.iter (fun r -> cgSay [", "; printReg r.v]) lst;
  cgSay ["\n"]; in
let printAdjListMap am =
  let _ = cgSay ["      AdjListMap:\n"] in
  RM.iter printAdjListMember am
in
  printAdjListMap cgInfo.adjListMap;
  List.iter printRegList (regLists cgInfo);
  List.iter printRegIList (regILists cgInfo);
  cgSay ["      K: "; string_of_int cgInfo.k; "\n"];
  List.iter printMoveIList (moveILists cgInfo);
  printAliasMap "aliasMap" cgInfo.aliasMap;
  printColorMap "colorMap" cgInfo.colorMap;
  printStringMap "listMap"
    (RM.fold (fun r i map -> RM.add r (str_of_list i.list) map)
             cgInfo.regToItem RM.empty);
  cgSay ["      spills: "; RS.to_string cgInfo.spills; "\n"]

let printCGInfo param proc = print_cgInfo param; (proc, false)

⟨cg stages 110⟩≡
  (*
let selectSpillStage = { Backplane.name = "selectSpill"
                        ; Backplane.fn = selectSpill
                        ; Backplane.paramExpected = Some "cgInfo"
                        ; Backplane.stateCreated = Backplane.list2SSet []
                        ; Backplane.stateDestroyed = Backplane.list2SSet []
                        ; Backplane.stateUsed = Backplane.list2SSet []
                        }

let assignColorsStage = { Backplane.name = "assignColors"
                        ; Backplane.fn = assignColors
                        ; Backplane.paramExpected = Some "cgInfo"
                        ; Backplane.stateCreated = Backplane.list2SSet []
                        ; Backplane.stateDestroyed = Backplane.list2SSet []
                        ; Backplane.stateUsed = Backplane.list2SSet ["interferenceGraph"]
                        }

let rewriteProgramStage = { Backplane.name = "rewriteProgram"
                        ; Backplane.fn = rewriteProgram
                        ; Backplane.paramExpected = Some "cgInfo"

```

```

        ; Backplane.stateCreated = Backplane.list2SSet []
        ; Backplane.stateDestroyed = Backplane.list2SSet []
        ; Backplane.stateUsed = Backplane.list2SSet []
    }

*)
(*
let haveSpilledTempsStage = { Backplane.name = "haveSpilledTemps"
    ; Backplane.fn = haveSpilledTemps
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet []
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet ["cfg"]
}

*)
(*
let livenessStage = { Backplane.name = "liveness"
    ; Backplane.fn = liveness
    ; Backplane.paramExpected = None
    ; Backplane.stateCreated = Backplane.list2SSet ["live_in sets"]
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet ["cfg"]
}

*)
(*
let buildStage = {Backplane.name = "build"
    ; Backplane.fn = build
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet ["interferenceGraph"]
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet ["cfg"; "live_in sets"]
}

*)
(*
let makeWorklistStage = { Backplane.name = "makeWorklist"
    ; Backplane.fn = makeWorklist
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet ["worklists"]
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet ["interferenceGraph"]
}

*)
(*
let simplifyStage = { Backplane.name = "simplify"
    ; Backplane.fn = simplify
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet []
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet []
}

*)
(*
let coalesceStage = { Backplane.name = "coalesce"
    ; Backplane.fn = coalesce
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet []
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet []
}

let recklessCoalesceStage = { Backplane.name = "recklessCoalesce"

```



```

        ; Backplane.fn = recklessCoalesce
        ; Backplane.paramExpected = Some "cgInfo"
        ; Backplane.stateCreated = Backplane.list2SSet []
        ; Backplane.stateDestroyed = Backplane.list2SSet []
        ; Backplane.stateUsed = Backplane.list2SSet []
      }
*)
(*
let freezeStage = { Backplane.name = "freeze"
    ; Backplane.fn = freeze
    ; Backplane.paramExpected = Some "cgInfo"
    ; Backplane.stateCreated = Backplane.list2SSet []
    ; Backplane.stateDestroyed = Backplane.list2SSet []
    ; Backplane.stateUsed = Backplane.list2SSet []
  }
*)

```

11.5.7 Register Class Trees

A register class is simply a set of hardware registers. We would like to be able to ask things such as

- What is the alias set of a class, ie what is the set of registers that the class overlaps with in hardware?
- Are two classes alias equivalent? ie $alias(C_1) = alias(C_2)$?
- Is one class a subset of another under aliasing? ie $alias(C_1) \subset alias(C_2)$?
- Is it possible two assignments from two different classes to alias? ie

$$alias(class_{n1}) \cap class_{n2} \neq \emptyset \vee class_{n1} \cap alias(class_{n2}) \neq \emptyset$$

$\langle register\ class\ interface\ 112a \rangle \equiv$ (116)

```

module type REGISTERCLASS = sig
  type t = Register.Set.t
  val is_empty: t -> bool
  val aliases: t -> t
  val alias_eq: t -> t -> bool
  val alias_contained: t -> t -> bool
  val may_alias: t -> t -> bool
  val inter: t -> t -> t
  val eq: t -> t -> bool
  val to_string: t -> string
  val cardinal: t -> int
  val space_to_class: Rtl.space -> t
  val map_class: Rtl.space -> t -> unit
  val mkClass: Rtl.space -> ('a, 'b, 'c) Target.t -> t
  val init: Register.t list -> unit
end

```

$\langle register\ classes\ 112b \rangle \equiv$ (116d) 113a▷

```

module RegisterClass:REGISTERCLASS = struct
  type t = RS.t
  let is_empty = RS.is_empty
  let inter c1 c2 = RS.inter c1 c2
  let eq c1 c2 = RS.equal c1 c2
  let to_string = RS.to_string

```

```

let cardinal = RS.cardinal
let class_map = Spacetbl.create 5
let map_class s c = Spacetbl.add class_map s c
let space_to_class s = Spacetbl.find class_map s

```

We need to keep around the universe of registers. Sigh.

(register classes 113a) $\equiv$ (116d) $\triangleleft$ 112b 113b $\triangleright$

```

let fromList l = List.fold_left (fun set reg -> RS.add reg set) RS.empty l
let all_regs = ref RS.empty
let init allregs =
  all_regs := List.fold_left (fun set reg -> RS.add reg set) RS.empty allregs

```

To get the aliases for one class, filter the global set of registers on whether or not there is a register in *c* that aliases. This does result in scanning the register class set *c* quite a bit, but I think it should be better than creating several mostly equal sets (the set of registers that alias with each register in the class) and then unioning them.

(register classes 113b) $\equiv$ (116d) $\triangleleft$ 113a

```

let aliases c =
  let alias_filter reg = RS.exists (fun elt -> RU.MayAlias.regs reg elt) c in
  RS.filter alias_filter !all_regs

let alias_eq c1 c2 = RS.equal (aliases c1) (aliases c2)
let alias_contained c1 c2 = RS.subset (aliases c1) (aliases c2)

let may_alias c1 c2 = not (is_empty (inter (aliases c1) (aliases c2)))

let mkClass space target =
  RS.fold
    (fun reg set -> if Target.fits target space reg then
      RS.add reg set else set)
    !all_regs RS.empty
end

```

11.5.8 Register Class Trees

We represent the register classes in a tree structure to better compute the squeeze estimate. I'm sure that the list operations I'm doing are horribly slow; if this worries you and you have suggestions please let me know!

(class tree interface 113c) $\equiv$ (116)

```

module type CLASSTREE = sig
  type t = Vertex of RegisterClass.t list ref * t list ref
  val classes: t -> RegisterClass.t list
  val down: t -> RegisterClass.t list
  val children: t -> t list
  val mkTree: RegisterClass.t list -> t
  val mkTreeList: Rtl.space list -> Register.t list -> ('a, 'b, 'c) Target.t -> t list
  val space_to_tree: Rtl.space -> t
end

```

(class trees 113d) $\equiv$ (116d)

```

module ClassTree:CLASSTREE = struct
  type t = Vertex of RegisterClass.t list ref * t list ref
  let tree_map = Spacetbl.create 5
  let space_to_tree s = Spacetbl.find tree_map s
  let classes = function Vertex(c, _) -> !c

```

```

let rec down = function Vertex(c, kids) ->
  List.flatten (!c::List.map down !kids)
let children = function Vertex(c, kids) -> !kids
⟨make tree 114⟩
end

```

A good class tree should have the following properties:

- Each vertex should contain *only* alias equivalent classes
- There should be no empty vertex.
- If two vertices have the same parent, their alias sets are disjoint
- The alias set of a child node should be a subset of the alias set of its parent node.

When classes can overlap without being equivalent or contained (with respect to aliasing), then we do not know how to construct a class tree that always gives the best approximation. For now, we punt this issue (it is not a realistic issue in actual architectures) since eventually we want to move the class tree construction to be part of the target.

Alias equivalence is transitive, therefore we only check the head of the vertex list.

```

⟨make tree 114⟩≡ (113d) 115a▷
let mkTree (xs:RegisterClass.t list) = match xs with
(x::classL) ->
  let first = Vertex(ref [x], ref []) in
  let rec addToTree root c = match root with
    Vertex(classes, children) -> let rep = List.hd (!classes) in
    if RegisterClass.alias_eq c rep then begin
      classes := c::(!classes);
      root
    end else if RegisterClass.alias_contained rep c then
      Vertex (ref [c], ref [root])
    else if RegisterClass.alias_contained c rep then begin
      match (!children) with
      [] -> let () = children := [Vertex(ref [c], ref [])] in root
      | _ -> let eqNode = function Vertex(cs, _) ->
          RegisterClass.alias_eq c (List.hd !cs) in
          let eqCd = function Vertex(cs, _) ->
              RegisterClass.alias_contained c (List.hd !cs) in
          let eqCs = function Vertex(cs, _) ->
              RegisterClass.alias_contained (List.hd !cs) c in
          let newKids =
            if List.exists (fun n -> eqNode n || eqCd n || eqCs n)
              !children then
              (List.map (fun r -> addToTree r c) !children)
            else Vertex(ref [c], ref [])::!children in
          children := newKids;
          root
        end
      else begin
        Impossible.impossible "Adding a space to a class tree it doesn't belong in"
      end
    in List.fold_left addToTree first classL
  | [] -> Impossible.impossible "Cannot build class tree from empty list"

```

`mkTreeList` initializes the class trees given a target. The class trees and register classes should hold the information about what registers a temp can be assigned to now rather than `cginfo`.

```

<make tree 115a>+≡ (113d) <114 115b>
let mkTreeList spaces allRegs target =
  RegisterClass.init allRegs;
  let splitAliases classL1 space =
    let rClass = RegisterClass.mkClass space target in
    RegisterClass.map_class space rClass;
    let belongs c l = List.exists
      (fun elt -> RegisterClass.is_empty (RegisterClass.inter c elt)) l in
    let rec addToList l2 = match l2 with
      [] -> [[rClass]]
    | head::rest -> if belongs rClass head then
      let newList = rClass::head in newList::rest
    else head::(addToList rest)
    in
    addToList classL1
  in
  let partitionedList = List.fold_left splitAliases [] spaces in
  let treeList = List.map mkTree partitionedList in

```

It should always be the case that if `c` is alias contained in the register class of the root, then there is a vertex `v` in this tree that contains class `c`.

```

<make tree 115b>+≡ (113d) <115a
List.iter
  (fun s -> let sClass = RegisterClass.space_to_class s in
    let tree = List.find (fun aTree ->
      RegisterClass.alias_contained sClass
      (List.hd (classes aTree))) treeList in
    Spacetbl.add tree_map s tree)
spaces;
treeList

```

11.5.9 Utilities to implement

We keep a mapping from each space to the register class for that space.

```

<tracked state 115c>≡ (116d)

module HashableSpace = struct
  type t = Rtl.space
  let equal a b = Rtlutil.Compare.space a b = 0
  let hash (name, _, size) = int_of_char name
end

module Spacetbl = Hashtbl.Make(HashableSpace)

```

Estimating worst

In order to efficiently estimate $worst^m(N, C)$ we observe that $worst^n(N, C) \leq worst^1(N, C)$ or more simply, the number of registers that can be blocked for a node of class N by m temporaries is no more than the number of registers that can be blocked for a node of class N by one temporary, multiplied by m .

We need to create a table of $worst^1(N, C)$

We don't want to form dependencies with the register allocation scheme, so perhaps the best interface is to present the node to measure and a list of its neighbors, having the register allocator decide what set of neighbors to use for worst.

First split the neighbors up by register class. Since the classes in the tree were determined from the spaces on the target architecture, we know that for any temporary, its space (and resulting class) is in the tree, and will get counted.

If it turns out the `eq` comparison here is getting costly, we know that any two tmps coming from the same space have the same class. We could either identify classes with an optional space tag, or associate each class with its space in some way.

<worst 116a>≡ (116d) 116b▷

```
let temp_to_space (s, _, _) = s

let degree_neighbors c =
  List.length
    (List.filter (fun tmp -> RegisterClass.eq
      (RegisterClass.space_to_class(temp_to_space tmp))
      c) neighbors)
```

`worst_1` computes the worst case number of tmp's registers that could be blocked by a coloring of 1 neighbor of class `c`. This needs to be cached once everything is working!

We could compute this very early (static table in the target early) rather than computing it once and caching the result.

<worst 116b>+≡ (116d) ◁116a

```
let worst_1 (s,_,_) c =
  let n_ = RegisterClass.space_to_class s in
  RS.fold (fun reg worst ->
    let blocked = RegisterClass.inter n_ (RegisterClass.aliases (RS.singleton reg)) in
    max (RegisterClass.cardinal blocked) worst)
  c 0

let bound (s,_,_) classes =
  let alias_sets = List.fold_left (fun union rClass -> RS.union (RegisterClass.aliases rClass) union) RS.empty
  RS.cardinal (RS.inter (RegisterClass.space_to_class s) alias_sets)

let rec rawZ tmp neighbors vertex =
  let rawZ_sum tmp neighbors vertex =
    List.fold_left (fun sum c_ -> let degree_n_C = degree_neighbors c_ in
      let term = degree_n_C * (worst_1 tmp c_) in
      sum + term)
    0 (ClassTree.classes vertex) in
  let z tmp neighbors vertex =
    min (bound tmp (ClassTree.down vertex)) (rawZ tmp neighbors vertex) in
  let filtered_children tmp neighbors vertex =
    List.fold_left (fun sum v -> (z tmp neighbors v) + sum) 0 (ClassTree.children vertex) in
  rawZ_sum tmp neighbors vertex + (filtered_children tmp neighbors vertex)
```

<registerclass.mli 116c>≡

```
<register class interface 112a>
module RegisterClass : REGISTERCLASS
<class tree interface 113c>
module ClassTree:CLASSTREE
```

11.5.10 Lua registration

<registerclass.ml 116d>≡

```
module R = Register
module RM = R.Map
module RS = R.Set
module RSX = R.SetX
module RU = Rtlutil
```

```

let imposs = Impossible.impossible

let () = Debug.register "registerclass" "generalized register classes"

  <tracked state 115c>
  <register class interface 112a>
  <register classes 112b>
  <class tree interface 113c>
  <class trees 113d>
  <worst 116a>

```

11.6 DFS Linear Scan Register Allocation

The goal of this register allocator is to allocate registers in linear time, generating as few load and store instructions as possible. We make two traversals of the cfg; the first traversal determines the order of the second traversal, which allocates the registers.

This algorithm is very different from graph coloring. Graph coloring builds up an abstract model of how temps interfere in a procedure. Our algorithm builds no such representation; instead, liveness information provides all the information we need to allocate registers.

The important invariant of the graph traversal is that we visit the predecessors of a node before we visit the node. Of course, this invariant does not apply to predecessors along backedges. Also, from join points, we make sure that the information from the allocation of one predecessor flows to the allocation of the other predecessors. This information flow allows us to allocate a variable to the same register, so no extra move instructions are required at the join point.

This algorithm takes its ideas primarily from two sources: linear scan and SSA. Linear scan register allocation offers the idea of the reverse postorder depth-first search. Our contribution is to realize that this graph traversal, coupled only with liveness information, allows us to treat the graph as if it has the SSA property of static single-assignment. Static single-assignment allows us to treat each definition as a separate temp. Of course, it is a good idea to ensure that if two definitions of a temp are live into a join point, they should be allocated to the same location; if not, shuffles must be inserted to place them in the same location. This behavior at join points is exactly what we would expect from a phi-node in SSA.

Because we take a local, graph-node-by-graph-node perspective, we never spill a temp unless there are more live temps than registers. Of course, the big concerns are whether we spill the right temps and whether we have too many shuffle instructions at join points. We use preferencing information as heuristics for each of these problems. More on these later...

11.7 DFS Linear Scan Register Allocation

Before digging into the algorithm, we have a few shorthands for module names and some type declarations. The utilities are largely uninteresting, so we leave their exposition until later.

```

<dls type declarations and utilities 117>≡ (136c)
module A = Automaton
module R = Register
module PA = Preast2ir
module RP = Rtl.Private
module Dn = Rtl.Dn
module RU = Rtlutil
module RTD = Runtimedata
module G = Zipcfg
module GR = Zipcfg.Rep
module IntMod = struct type t = int let compare x y = x - y end

```

```

module IS = Set.Make (IntMod)
module IM = Map.Make (IntMod)
module U  = Unique
module UM = Unique.Map
module US = Unique.Set
module RM = R.Map
module RS = R.Set
module SM = Map.Make (struct type t = Rtl.space let compare = RU.Compare.space end)
module P = Proc
module T = Target

<register utilities 134c>
<Dls printers 134b>
module VM = Varmap

type tgt = Ast2ir.tgt
<Dls type declarations 118a>
<Dls utilities 135a>

```

11.7.1 Preferences

A `varMap` maps a temp (temporary register) to a hardware register. It is a dynamic invariant that the domain of this function contains only temps and the range includes hardware registers and memory locations.

```

<Dls type declarations 118a>≡ (117) 121a▷
type preferences = { varMap : VM.t
                    ; regs   : Register.t list
                    }

```

Printers are our debugging friends.

```

(define print_prefs to print preferences 118b)≡ (121b)
let _print_prefs msg prefs =
  Printf.eprintf "%s\nPreferences VarMap:\n" msg;
  VM.print "" prefs.varMap;
  Printf.eprintf "Preferences RegQueue:\n";
  List.iter (fun r -> Printf.eprintf "%s " (printReg r)) prefs.regs;
(*
  SM.iter (fun (s,_,_) regs ->
    Printf.eprintf "Space %c: " s;
    List.iter (fun r -> Printf.eprintf "%s " (printReg r)) regs)
  prefs.regs;
*)
Printf.eprintf "\n" in

```

The preferences come in two forms: the register order and the variable map. The register order indicates the order in which we assign registers. The variable map suggests where a variable should be placed (if there is a choice). We make no assumptions that the input `allocated_preds` have already been allocated (and therefore have useful outmaps) – this way, the callee doesn't have to allocate a new list with only the allocated predecessors.

Currently, the choice of predecessor is arbitrary.

THIS IS REALLY BAD: I'M DROPPING THE (USEFUL) PREFERENCE HINTS FROM THE SUCCESSOR

```

<updating preferences 118c>≡ (121b) 119▷
let update_pref_varmap prefs preds =
  let rec use_allocated_pred preds = match preds with
  | [] -> prefs
  | p::rst -> try { prefs with varMap = getVarOut p }
              with Not_found -> use_allocated_pred rst in
  use_allocated_pred preds in

```

```

⟨define avoid and aim_for to decrease or increase the likelihood of choosing a register 120a⟩
let update_prefs_at_block prefs bid blockmap =
  prefs in
  (*
    let temp_defs, hw_defs = partition_regs tgt (defs node)
    let app f v z = match v with Some s -> f s z | None -> z in
    let to_avoid =
      RS.fold (fun t rst -> try app RS.add (VM.var_locs' prefs.varMap t).reg rst
        with Not_found -> rst)
        temp_defs (RS.inter hw_defs alloc_regset)
    (*let get_regs = get_regs prefs.varMap
    let to_avoid = RS.fold (fun t rst -> List.fold_right RS.add (get_regs t) rst)
      temp_defs (RS.inter hw_defs alloc_regset)
  *)

  let vm = match get_copy_regs node with
    | Some (d, s) when is_tmp tgt s ->
      if is_tmp tgt d then
        try VM.add_reg s (choose_reg prefs.varMap d) prefs.varMap
        with Not_found -> prefs.varMap
      else if RS.mem d alloc_regset then
        VM.add_reg s d prefs.varMap
      else prefs.varMap
    | _ -> prefs.varMap in
  { regs = avoid prefs.reg to_avoid
  ; varMap = vm
  } in
  *)

```

After a node has been allocated, we need to update the register queue in the prefs record. When a temp expires, we send its register to the front of the queue. When we allocate a temp to a register, we send that register to the end of the queue. Note that this function is called after the node has been rewritten to replace all temps with hardware registers.

```

⟨updating preferences 119⟩+≡ (121b) <118c
let _post_update_prefs node prefs outmap =
  prefs in
  (*
  (*
    print_prefs (Printf.sprintf "Prefs before node %d" (G.num node)) prefs;
    *)
    let live_in = regs_in_vm tgt outmap (get_live_in node) in
    let live_out = regs_in_vm tgt outmap (get_live_out node) in
    let defs = regs_in_vm tgt outmap (defs node) in
    let expired = (defs ++ live_in) -- live_out in
    let live_out_defs = RS.inter alloc_regset (RS.inter defs live_out) in
    let expired_regs = RS.inter alloc_regset (expired -- live_out_defs) in
    (*printTempSet "expired_regs" expired_regs;
    Printf.eprintf "\n";
    printTempSet "live_out_def_regs" live_out_defs;
    Printf.eprintf "\n";
    *)
    let prefs = {prefs with regs = avoid (aim_for prefs.reg expired_regs) live_out_defs} in
    (*
    print_prefs (Printf.sprintf "Prefs after node %d" (G.num node)) prefs;
    *)
    prefs in
  *)

```


We decrease the likelihood that a set of registers are selected by moving them to the end of the queue of registers.

```

⟨define avoid and aim_for to decrease or increase the likelihood of choosing a register 120a⟩≡      (118c)
(*)
let move_regs move reg_map move_map =
  let move_map = separate_set move_map in
  let move_space s regs =
    try let to_move = SM.find s move_map in
      move (diff regs to_move) (RS.elements to_move)
    with Not_found -> regs in
  SM.mapi move_space reg_map in
let avoid = move_regs (@) in
let aim_for = move_regs (fun a b -> b @ a) in
*)

```

11.7.2 The Basic Algorithm

```

⟨dls.ml 120b⟩≡
⟨dls.ml 120d⟩

```

The `state` is used to carry information as we traverse the flow graph.

- `spillMap`: a map from a temp to its spill location
- `varInMaps`: a map from a node (by node number) to the `varMap` representing the location of each temp `live_in` to the node
- `varOutMaps`: a map from a node (by node number) to the `varMap` representing the location of each temp `live_out` from the node

I used to pass it around as a record, but that's really tedious. Instead, property lists are used for the variable maps and a `spillMap` is bound in the top-level `dls` function for all to use.

```

⟨set up spillMap 120c⟩≡      (122)
  let spillMap = ref RM.empty in

⟨dls.ml 120d⟩≡      (120b) 134a▷
module Pr = Property
let varInMatcher = { Pr.embed = (fun a -> Pr.VarInMap a);
  Pr.project = (function Pr.VarInMap a -> Some a | _ -> None);
  Pr.is = (function Pr.VarInMap a -> true | _ -> false);
}
let varOutMatcher = { Pr.embed = (fun a -> Pr.VarOutMap a);
  Pr.project = (function Pr.VarOutMap a -> Some a | _ -> None);
  Pr.is = (function Pr.VarOutMap a -> true | _ -> false);
}
let varCallInMatcher = { Pr.embed = (fun a -> Pr.VarCallInMap a)
  ; Pr.project = (function Pr.VarCallInMap a -> Some a | _ -> None)
  ; Pr.is = (function Pr.VarCallInMap a -> true | _ -> false)
}

let varInProp = Unique.Prop.prop varInMatcher
let getVarIn = Unique.Prop.get varInProp
let setVarIn = Unique.Prop.set varInProp
let remVarIn = Unique.Prop.remove varInProp
let varOutProp = Unique.Prop.prop varOutMatcher
let getVarOut = Unique.Prop.get varOutProp
let setVarOut = Unique.Prop.set varOutProp
let remVarOut = Unique.Prop.remove varOutProp

```

```

let varCallInProp = Unique.Prop.prop   varCallInMatcher
let getCallIn     = Unique.Prop.get    varCallInProp
let setCallIn     = Unique.Prop.set    varCallInProp
let remCallIn     = Unique.Prop.remove varCallInProp

```

We use the `fits tgt s` function to find the registers that hold temps from the temporary space `s`.

<Dls type declarations 121a>+≡ (117) <118a
`let () = Debug.register "dls-regs" "mapping of registers to temporaries"`

```

let fits =
  if Debug.on "dls-regs" then
    (fun t ((sn, _, _) as s) r ->
      let answer = Target.fits t s r in
      Printf.eprintf "register %s %s space '%c'\n" (RU.ToString.reg r)
        (if answer then "fits" else "does not fit") sn;
      answer)
  else
    Target.fits

```

```

let get_regs_for_space tgt regs s =
  List.filter (fits tgt s) regs

```

```

let init_prefs regs = { varMap = VM.empty; regs = regs }

```

We need to visit every node in the `cfg`, which is trickier than one might think: some nodes are only reachable from the entry node or by reverse pointers from the exit node. Since nodes that are reachable only by reverse pointers from the exit node are unreachable, I first trim the unreachable code. We still need to deal with nodes that are reachable from the entry node but not from the exit node. Furthermore, we need to make sure we always allocate a node's predecessors before allocating the node.

First, I establish a post-order on the nodes of the graph; this order includes all nodes, since they are now all reachable from the entry node. This order is maintained in both a map and a list. We establish the reverse post-order by reversing the list.

From here on out, we can start the actual graph traversal, following predecessor edges from the exit node. Of course, some nodes can not be reached from the exit node, so we have to start over with them. We do this easily by taking the reverse post-order list and restarting our graph traversal from the first node on the list that has not yet been allocated. Note: we could start with this policy from the very beginning, instead of starting with the exit node.

<rpo_dfs determines the order for visiting nodes 121b>≡ (122)
<define print_prefs to print preferences 118b>
<updating preferences 118c>
`let rpo_dfs cfg =`
 `let blocks = G.postorder_dfs cfg in`
 `let predmap =`
 `let add_preds m b =`
 `let ladd k v m = UM.add k (v :: try UM.find k m with Not_found -> []) m in`
 `GR.fold_succs (fun uid m -> ladd uid (GR.id b) m) (GR.last (GR.unzip b)) m in`
 `List.fold_left add_preds (UM.add GR.entry_uid [] UM.empty) blocks in`
 `let (_, po_map, rpo_bid_lst) =`
 `List.fold_left (fun (i, m, l) b -> let bid = GR.id b in`
 `(i + 1, UM.add bid i m, bid :: l))`
 `(0, UM.empty, []) blocks in`

`(* Pre-condition: node has not been visited. *)`
`let rec visit_block visited bid k blockmaps prefs =`
 `let visited = US.add bid visited in`
 `let prefs = update_prefs_at_block prefs bid blockmaps in`
 `let pred_ids = UM.find bid predmap in`

```

let node_num = UM.find bid po_map in

let visit_pred visited bid blockmaps prefs k =
  (* THIS DOES SOMETHING SOMEWHAT DUMB AND NOT QUITE WHAT WE WANT *)
  let prefs = update_pref_varmap prefs pred_ids in
  visit_block visited bid k blockmaps prefs in
let alloc_curr_block visited blockmaps =
  let blockmaps = alloc_block prefs blockmaps bid pred_ids in
  (* MAYBE SOME PREFERENCING SHOULD HAPPEN HERE, BUT IS IT THIS?: *)
  (*let prefs =
    post_update_prefs block prefs (getVarOut (GR.id block)) in
  prefs in*)
  k visited blockmaps in

(* we pass the same prefs to all preds *)
let visit_preds visited pred_ids blockmaps prefs =
  let rec loop pred_ids visited blockmaps = match pred_ids with
  | [] -> alloc_curr_block visited blockmaps
  | pid::rst ->
    if UM.find pid po_map < node_num && not (US.mem pid visited) then
      visit_pred visited pid blockmaps prefs (loop rst)
    else loop rst visited blockmaps in
  loop pred_ids visited blockmaps in
visit_preds visited pred_ids blockmaps prefs in
let rec finish rpo_bid_lst visited blockmaps = match rpo_bid_lst with
| bid::rst ->
  if US.mem bid visited then
    finish rst visited blockmaps
  else
    visit_block visited bid (finish rst) blockmaps empty_prefs
| [] -> blockmaps in
let (blockmap, predmap) =
  finish rpo_bid_lst US.empty (G.to_blocks cfg, predmap) in
G.of_blocks blockmap in

```

<dls algorithm 122> $\equiv$ (136c)

```

let dls _ (cfg, (Procedure.{cc = cc; target = PA.T tgt} as proc)) =
  let machine = tgt.T.machine in
  let l2e = proc.exp_of_lbl in
  let m = (proc, machine, l2e) in
  let () = Debug.eprintf "dls" "DLS(%s)\n" proc.symbol#original_text in
  <set up spillMap 120c>
  let ((cfg, proc), _) = Optimize.trim_unreachable_code () (cfg, proc) in
  if Debug.on "dls" then
    <print cfg plus live-in and live-out sets 135b>
  ;
  let alloc_regset = cc.Call.volregs ++ cc.Call.pre_nvregs in
  if Debug.on "dls-regs" then
    Printf.eprintf "alloc_regset = { %s }\n" (Register.Set.to_string alloc_regset);
  <define spill functions 127b>
  <definition of shuffle 129b>
  <choose_varMaps finds varIn and varOut maps for a node 126a>
  <alloc_block allocates registers at a single block 123>
  let empty_prefs = init_prefs (RS.elements alloc_regset) in
  <rpo_dfs determines the order for visiting nodes 121b>
  let cfg = rpo_dfs cfg in
  let cfg = cleanup cfg in
  <add spans for variable locations 133b>
  ;
  if Debug.on "dls" then (

```

```

    <print cfg plus live-in and live-out sets 135b>
    ;
    <print varmaps 136a>
  );
  (cfg, proc), true
(* Diagnostic:
let dls a proc = time (proc.Proc.symbol#original_text) (dls a) proc
*)

```

To allocate registers at a single block, we follow a simple process:

1. Find predecessors and successors of node.
2. Choose a varMap for node.
3. Rewrite the instruction to reflect the placement of temps in the varMaps.
4. Insert a shuffle of moves, loads, and stores to settle differences between varMaps in adjacent nodes.

```

<alloc_block allocates registers at a single block 123>≡ (122) 125c>
let alloc_block prefs (blockmap, predmap) bid pred_ids =
  let (first, tail as block) = UM.find bid blockmap in
  let () = Debug.eprintf "dls" "allocating block %s\n" (blockname block) in
  let (head, last) = GR.goto_end (GR.unzip block) in
  (*Printf.eprintf "dls adding varmap for block %d\n" id;*)
  <get live_outs 125b>
  (* THESE PROMOTIONS NEED TO BE STAMPED OUT, BUT FOR NOW, I DON'T HAVE THE
     MACHINERY TO PROPERLY HANDLE SLICES -- SETS USING PERVASIVES.COMPARE ARE NOT
     SUFFICIENT -- THEY DON'T INDICATE OVERLAP *)
  let live_in = match live_outs with
    | lo::_ -> R.promote_rxset (G.add_live_spansf first lo)
    | [] -> impossf "DLS: Block has no live_out sets" in
  <define functions entry_inmap and label_inmap to help find the inmap 125a>

  let add_nodes shuffles (g : G.zgraph) =
    let add_subgraph g block =
      let (subg, _) = G.block2cfg m block in
      G.splice_focus_exit g (G.unfocus subg) in
    List.fold_left add_subgraph g shuffles in
  (* first: *)
  let () = Debug.eprintf "dls" "choosing varmaps for first\n" in
  let varOutf = match first with
    | GR.Entry -> let vm = entry_inmap () in (setVarIn bid vm; vm)
    | GR.Label _ ->
      match live_outs with
      | lo::_ ->
        let inmap = label_inmap pred_ids in
        let live_out = R.promote_rxset lo in
        let live_out_w_spans = R.promote_rxset (G.add_live_spansf first lo) in
        let (varIn, midMap, defMap, varOut, spills, reloads) =
          choose_varMaps prefs None inmap ~defs:RS.empty ~uses:RS.empty
            ~live_in ~live_out ~live_out_w_spans in
        let () = setVarIn bid varIn in
        varOut
      | [] -> impossf "DLS: no more live_out sets" in

  (* PREFS AREN'T PROPERLY THREADED HERE *)
  (* tail: *)
  let rec alloc_tail g inmap live_outs (*head*) tail =
    let (li, lo, live_outs) = match live_outs with

```

```

    | li::((lo::_) as rst) -> (li, lo, rst)
    | _ -> impossf "DLS: wrong number of live_out sets" in
let live_in = R.promote_rxset li in (* repeated promotion done here *)
let live_out = R.promote_rxset lo in
match tail with
| GR.Tail (m, tail) ->
    let live_out_w_spans = live_out in
    let rtl = GR.mid_instr m in
    let () = Debug.eprintf "dls" "choosing varmaps for middle: %s\n"
        (Rtlutil.ToString.rtl rtl) in

    let defs = defsm m in
    let uses = usesm m in
    let () = if Debug.on "dls" then printTempSet "liveIn: " live_in in
    let (varIn, midMap, varDefs, varOut, spills, reloads) =
        choose_varMaps prefs (Some rtl) inmap
            ~defs ~uses ~live_in ~live_out ~live_out_w_spans in
    let () = if Debug.on "dls" then begin
        Printf.eprintf "spills: ";
        let pr =
            Dag.pr_block (fun f -> RU.ToString.rtl (f (Rtl.bool true))) in
        List.iter (fun s -> Printf.eprintf " %s" (pr s)) spills;
        Printf.eprintf "\n" end in
    let ((head, t), blockmap) = G.openz (add_nodes reloads (add_nodes spills g)) in
    let head = GR.Head (head, G.new_rtlm (rewrite_rtl tgt ~varIn ~varDefs rtl) m) in
    alloc_tail (G.tozgraph ((head, t), blockmap)) varOut live_outs tail
| GR.Last l ->
    let live_out_w_spans = R.promote_rxset (G.add_live_spansl l lo) in
    let rtl = GR.last_instr l in
    let () = Debug.eprintf "dls" "choosing varmaps for last: %s\n"
        (Rtlutil.ToString.rtl rtl) in

    let defs = defsl l in
    let uses = usesl l in
    let (varIn, midMap, varDefs, varOut, spills, reloads) =
        choose_varMaps prefs (Some rtl) inmap
            ~defs ~uses ~live_in ~live_out ~live_out_w_spans in
    let upd = rewrite_rtl tgt ~varIn ~varDefs in
    let l' = G.new_rtlm (upd rtl) l in
    let l' = match l' with
    | GR.Call c ->
        let upd_assn ce = { ce with G.assertion = upd ce.G.assertion } in
        GR.Call { c with GR.cal_contedges = List.map upd_assn c.GR.cal_contedges }
    | _ -> l' in
    let () = setVarOut bid varOut in
    let () = match l' with GR.Call _ -> setCallIn bid midMap | _ -> () in

    let ((head, _), blockmap) = G.openz (add_nodes reloads (add_nodes spills g)) in
    l', G.tozgraph ((head, GR.Last l'), blockmap) in
let last, g = alloc_tail (G.tozgraph ((GR.First first, GR.Last GR.Exit), blockmap))
    varOutf live_outs tail in
let blockmap = G.to_blocks (G.unfocus g) in
let maps =
    List.fold_left (fun maps p ->
        try ignore (getVarOut p); shuffle maps p bid with Not_found -> maps)
        (blockmap, predmap) pred_ids in
let (blockmap, predmap) =
    List.fold_left (fun maps s ->
        try ignore (getVarIn s); shuffle maps bid s with Not_found -> maps)
        maps (GR.succs last) in
(blockmap, predmap) in

```

```

(* Not sufficient; doesn't save any time *)
(*
let free_unused_maps n =
  let not_all_allocated lst = List.exists (fun p -> not (is_allocated p)) lst in
  let k = G.kind n in
  if not (is_allocated n) ||
    G.is_call n || G.is_join n || G.is_fork n || k == G.Assertion ||
    k == G.Branch || not_all_allocated (G.preds n) ||
    not_all_allocated (G.succs n)
  then
    ()
  else
    ((*Printf.eprintf "dls removing varmap for node %d\n" (G.num n);*)
     remVarIn (GR.id n);
     remVarOut (GR.id n)
    ) in
let () = List.iter free_unused_maps preds in
let () = free_unused_maps node in
List.iter free_unused_maps succs in
*)

```

In `block_inmap`, we find a plausible variable map on entry to the basic block. If the first node is the entry node, then we just put all the variables somewhere; otherwise, we take the outmap from a predecessor.

(define functions entry_inmap and label_inmap to help find the inmap 125a) ≡ (123)

```

let rec label_inmap pred_ids = match pred_ids with
| pid::rst ->
  (try let inmap = getVarOut pid in
   VM.filter (fun t -> RS.mem t live_in) inmap
   with Not_found -> label_inmap rst)
| [] -> impossf "DLS: No allocated predecessor found" in
let entry_inmap () =
  let alloc_remaining (s,_,_ as t) (map, used as z) =
    if is_tmp tgt t then
      let rec loop = function
        | r::rst when RS.mem r used -> loop rst
        | r::rst -> (VM.add_reg t r map, RS.add r used)
        | [] -> let l = get_spill_loc t in
                 (VM.add_mem t l map, used) in
      loop (get_regs_for_space tgt prefs.regs s)
    else z in
  let (inmap, _) = RS.fold alloc_remaining live_in (VM.empty, live_in) in
  inmap in

```

For each instruction in the basic block, we accumulate the set of `live_out` registers.

(get live_outs 125b) ≡ (123)

```

let last_out = Live.live_out_last last in
let rec get_liveouts liveout rst = function
| GR.First _ -> liveout :: rst
| GR.Head (h, m) -> get_liveouts (Live.live_in_middle liveout m) (liveout :: rst) h in
let live_outs = get_liveouts (Live.live_in_last last) [last_out] head in

```

We clean up any move instruction where the source register is the same as the destination register. We accumulate the list of nodes we intend to delete before deleting them because we do not want to modify the `cfg` while traversing it.

(alloc_block allocates registers at a single block 125c) + ≡ (122) < 123

```

let cleanup cfg =
  let filter_copies (first, tail) =
    let is_copy middle =
      match get_copy_regs (Some (GR.mid_instr middle)) with

```

```

    | Some (r1,r2) -> Register.eq r1 r2
    | _           -> false in
let rec filt head tail = match tail with
| GR.Tail (m, t) when is_copy m -> filt head t
| GR.Tail (m, t)                -> filt (GR.Head (head, m)) t
| GR.Last _                    -> GR.zipht head tail in
filt (GR.First first) tail in
G.of_blocks (UM.map filter_copies (G.to_blocks cfg)) in

```

`choose_varMaps` is responsible for choosing the `varMaps` in and out of each node. This particular version does not do any preferencing. We begin by gathering some useful information about registers at the node.

```

⟨choose_varMaps finds varIn and varOut maps for a node 126a⟩≡      (122) 126b▷
let choose_varMaps prefs instr inmap ~defs ~uses
    ~live_in ~live_out ~live_out_w_spans =
  ⟨define remove_last_uses, alloc, and alloc_hw_regs 126e⟩
  (*
    ⟨check pred map invariant 127a⟩
  *)

```

If any temps are in an unavailable register, then we need to spill the temp from that register. There is an opportunity for optimization here: even if the temp is not used in this instruction, we could try to allocate it to a register if there are any free registers. This optimization would insert a move instruction instead of a store.

```

⟨choose_varMaps finds varIn and varOut maps for a node 126b⟩+≡      (122) <126a 126c▷
  ⟨choose_register 128b⟩
  (* We need to spill a hardware register if it is defined *)
  let spill r z =
    if is_tmp tgt r then z else spill_reg (fun t -> RS.mem t live_out_w_spans) z r in
  let inmap, spills = RS.fold spill defs (inmap, []) in
  let inmap, spills, reloads =
    alloc_temps alloc_uses prefs.reg uses (inmap, spills, []) in

```

Now, we expire any temps that die at this node and move on to the defs at the node.

```

⟨choose_varMaps finds varIn and varOut maps for a node 126c⟩+≡      (122) <126b 126d▷
  let midmap = expire_last_uses inmap in
  let outmap, spills, reloads =
    alloc_temps (alloc_defs inmap) prefs.reg defs (midmap, spills, reloads) in

```

Here's an interesting case. If we have a def that is never used again, then we must assign it a register, but we don't want it in the outmap. So, we have a separate defmap and outmap, where the defmap keeps any dead definitions.

```

⟨choose_varMaps finds varIn and varOut maps for a node 126d⟩+≡      (122) <126c
  let inmap, outmap = VM.sync_maps inmap outmap in (* NECESSARY? *)
  let defmap = outmap in
  let outmap = expire_dead_defs outmap in
  if Debug.on "dls" then VM.print "inmap" inmap;
  if Debug.on "dls" then VM.print "defmap" defmap;
  if Debug.on "dls" then VM.print "outmap" outmap;
  inmap, midmap, defmap, outmap, spills, reloads in

```

In `expire_last_uses`, the guard supports instructions such as $x := x + y$; where both x 's must be placed in the same register. It is the expander's responsibility to place the x 's in different temps if more flexibility is desired. In any case, we still have to remove obsolete memory references from the variable map.

```

⟨define remove_last_uses, alloc, and alloc_hw_regs 126e⟩≡      (126a)
let app f v z = match v with Some s -> f s z | None -> z in
let expire_last_uses vm =
  let expire_one t vm =
    if is_tmp tgt t then
      let lp = VM.var_locs' vm t in
      if RS.mem t defs then app (VM.remove_mem t) lp.VM.mem vm

```

```

    else if RS.mem t live_out then vm
    else app (VM.remove_mem t) lp.VM.mem (app (VM.remove_reg t) lp.VM.reg vm)
  else vm in
  RS.fold expire_one uses vm in
let expire_dead_defs vm =
  let expire_one t vm =
    if is_tmp tgt t && not (RS.mem t live_out) then
      (* THE EXCEPTIONAL CASE SHOULD BE AN IMPOSSIBLE.IMPOSSIBLE *)
      try app (VM.remove_reg t) (VM.var_locs' vm t).VM.reg vm
      with Not_found -> vm
    else vm in
  RS.fold expire_one defs vm in

```

The predecessor's outmap must map each live_in temp to some location.

<check pred map invariant 127a> $\equiv$ (126a)

```

let () = let check t =
  let fail () = impossf "Live-in temp not in pred's outmap" in
  if is_tmp tgt t then
    try match VM.var_locs' inmap t with
      | {reg = None; mem = None} -> fail ()
      | _ -> ()
    with Not_found -> fail () in
  RS.iter check live_in in

```

11.7.3 Spilling

Spilling a temp from a register requires that we modify the state in addition to the varmaps because we may have to set a spill location for the temp.

<define spill functions 127b> $\equiv$ (122)

```

let get_spill_loc ((_, _, ms), _, c as temp) =
  try RM.find temp (!spillMap)
  with Not_found ->
    let l = Automaton.allocate proc.priv ~width:(Cell.to_width ms c)
      ~kind:"" ~align:1 in
    let () = spillMap := RM.add temp l (!spillMap) in
    l in

```

<define assign, move, store, and load 133a>

```

let spill temp map = VM.spill temp (get_spill_loc temp) map in
let spill_reg guard (map, spills as z) reg =
  match VM.reg_contents map reg with
  | Some t when guard t ->
    let map' = spill t map in
    (match (VM.var_locs' map t).VM.mem with
    | Some _ -> (map', spills)
    | None -> (map', store' t reg :: spills))
  | _ -> z in

```

11.7.4 Choosing a Register for a Single Temp

<define allocation functions 127c> $\equiv$ (128b) 128a▷

```

let default_alloc (map, spills, reloads, regs_used) t =
  let live_past_use = RS.mem t live_out_w_spans in
  let reg = alloc_reg map regs_used live_past_use t in
  let regs_used = RS.add reg regs_used in
  let map, spills = match VM.reg_contents map reg with

```



```

    | Some _ -> let map, spills = spill_reg true_fun (map, spills) reg in
      (VM.add_reg t reg map, spills)
    | None    -> (VM.add_reg t reg map, spills) in
let reloads = if is_use then load' t reg :: reloads else reloads in
if live_past_use && (RS.mem reg live_out || RS.mem reg defs) then
  let spill_loc = get_spill_loc t in
  (VM.add_mem t spill_loc map, store' t reg :: spills, reloads, regs_used)
else (map, spills, reloads, regs_used) in

```

Note that by calling `get_avail_reg`, I am disallowing copies in which the used temp lives past this node. Is there a good reason to do this? I could allocate them to the same register, and as long as I: 1.) have the variable `map` remember that multiple temps are in the same register and 2.) when spilling, spill all temps in the register to separate spill slots (b/c spill slots are allocated by temp name, not value, and one of the temps may be redefined and spilled, overwriting the other).

```

⟨define allocation functions 128a⟩+≡ (128b) <127c
let get_avail_reg ((map, spills, reloads, regs_used) as s) t reg fail =
  if free_reg map regs_used (RS.mem t live_out_w_spans) reg then
    let reloads =
      if is_use then (load' t reg :: reloads) else reloads in
    (VM.add_reg t reg map, spills, reloads, RS.add reg regs_used)
  else fail s t in

(* include preferences *)
let alloc_prefs state t =
  try get_avail_reg state t (choose_reg prefs.varMap t) default_alloc
  with Not_found -> default_alloc state t in
(* include copy propagation *)
let alloc_copy order map state t =
  match get_copy_regs instr with
  | Some regpair ->
    let (to_alloc, other) = order regpair in
    if R.eq to_alloc t && not (Vfp.is_vfp (RP.Reg other)) then
      try let r = if is_tmp tgt other then
        choose_reg map other
      else if List.exists (R.eq other) regs then
        other
      else raise Not_found in
      get_avail_reg state t r alloc_prefs
    with Not_found -> alloc_prefs state t
    else alloc_prefs state t
  | None -> alloc_prefs state t in

⟨choose_register 128b⟩≡ (126b) 129a▷
let allocate_regs is_use free_reg order premap alloc_reg state to_alloc =
  ⟨define allocation functions 127c⟩
  alloc_copy order premap state to_alloc in
let free_in = VM.free_reg_inregs defs live_in live_out in
let alloc_uses regs =
  allocate_regs true free_in (fun (d,s) -> (s,d)) prefs.varMap
  (VM.alloc_inreg regs defs live_in live_out) in
let free_out = VM.free_reg_outregs defs live_out in
let alloc_defs uses_map regs =
  allocate_regs false free_out (fun (d,s) -> (d,s)) uses_map
  (VM.alloc_outreg regs defs live_out) in

```

If any temps are in an unavailable register, then we need to spill the temp from that register. There is an opportunity for optimization here: even if the temp is not used in this instruction, we could try to allocate it to

a register if there are any free registers. This optimization would insert a move instruction instead of a store.

```

⟨choose_register 129a⟩+≡ (126b) <128b
  let alloc_temps alloc regs to_alloc (map, spills, reloads) =
    (* I'm valuing the cost of allocation over the cost of verbose code here.... *)
    let fold_allocated f z r vm =
      try match (VM.var_locs' vm r).VM.reg with None -> z | Some r -> f r z
      with Not_found -> z in
    let regs_used = (* for (t = HW reg), Not_found should suffice *)
      RS.fold (fun t set -> fold_allocated RS.add set t map) to_alloc RS.empty in

    let one_alloc (s,_,_ as r) rst =
      if is_tmp tgt r && fold_allocated (fun _ _ -> false) true r map then
        let regs = get_regs_for_space tgt regs s in
        alloc regs rst r
      else rst in
    let (map, spills, reloads, _) =
      RS.fold one_alloc to_alloc (map, spills, reloads, regs_used) in
    (map, spills, reloads) in

```

11.7.5 Shuffling Establishes Consistency

Hmmmm... the aliasing call is okay only if we have registers on both sides of the assignment – I should really do something more general about shuffling here and just treat all the moves, stores, and loads as parallel effects.

```

⟨definition of shuffle 129b⟩≡ (122) 129d▷
  let rec shuffle_moves = function
    | ((t, dst, src) :: rest) as effs ->
      ⟨definition of Dls.try_first_effect 129c⟩
      try_first_effect effs
      (fun (t, dst, src) rest ->
        let mv = move t dst src in
        let shuffs = shuffle_moves rest in
        mv :: shuffs)
    (fun () ->
      (* NOT IDEAL -- SHOULD TRY TO MOVE THROUGH A REG FIRST.... *)
      let spill = store t src in
      let shuffs = shuffle_moves rest in
      let reload = load t dst in
      spill :: shuffs @ [reload])
    | [] -> [] in

```

```

⟨definition of Dls.try_first_effect 129c⟩≡ (129b)
  let try_first_effect effects succ fail =
    let rec maybe_bad = function
      | [] -> fail () (* Circular register moves *)
      | (t, dst, src) :: rest ->
        let alias (_, _, r) = RU.MayAlias.regs dst r in
        if not (List.exists alias bad || List.exists alias rest) then
          succ (t, dst, src) (List.rev_append bad rest)
        else
          maybe ((t, dst, src) :: bad) rest in
    maybe [] effects in

```

When we want to insert a shuffle between two nodes, we first have to check whether we can insert between the nodes. If not (as in multiway-branch, call, and cut-to instructions), we need to insert loads and stores before and after the nodes.

```

⟨definition of shuffle 129d⟩+≡ (122) <129b
  let shuffle (blockmap, predmap) pred_id succ_id =

```

```

Debug.eprintf "dls" "Shuffling pred %s and succ %s\n"
  (blockname (UM.find pred_id blockmap)) (blockname (UM.find succ_id blockmap));
let pred_vm = getVarOut pred_id in
let succ_vm = getVarIn succ_id in
if Debug.on "dls" then VM.print "pred:" pred_vm;
if Debug.on "dls" then VM.print "succ:" succ_vm;
<define shuffle_around and shuffle_between 130a>
match GR.last (GR.unzip (UM.find pred_id blockmap)) with
| GR.Mbranch _ | GR.Call _ | GR.Cut _ ->
  shuffle_around blockmap predmap pred_id succ_id
| _ -> shuffle_between blockmap predmap pred_id succ_id in

```

First, we find the temps that are placed inconsistently between the nodes. We insert the necessary load instructions after the successor, and we insert the necessary store instructions before the successor's predecessors. We also update each variable map that is changed.

```

<define shuffle_around and shuffle_between 130a>≡ (129d) 130b▷
let shuffle_around blockmap predmap pred_id succ_id =
  let () = Debug.eprintf "dls" "shuffle_around\n" in
  let _pred = UM.find pred_id blockmap in
  let _succ = UM.find succ_id blockmap in

  let regs temp succ_reg (ts, loads as rst) =
    match (VM.var_locs' pred_vm temp).VM.reg with
    | Some r when R.eq r succ_reg -> rst
    | _ -> let l = load temp succ_reg in
          (RS.add temp ts, l::loads) in
  let mems temp succ_mem (ts, loads as rst) =
    match (VM.var_locs' pred_vm temp).VM.mem with
    | None -> (RS.add temp ts, loads)
    | _ -> rst in
  let (temps,loads) = VM.fold regs mems succ_vm (RS.empty,[]) in

```

We update the successor's variable map to have each inconsistent temp in memory.

```

<define shuffle_around and shuffle_between 130b>+≡ (129d) <130a 130c▷
let succ_vm =
  let app f t v z = match v with Some s -> f t s z | None -> z in
  RS.fold (fun t vm -> let lp = VM.var_locs' vm t in
    let vm' = app VM.remove_reg t lp.VM.reg vm in
    VM.add_mem t (RM.find t (!spillMap)) vm')
    temps succ_vm in
let () = setVarIn succ_id succ_vm in

```

```

<define shuffle_around and shuffle_between 130c>+≡ (129d) <130b 131▷
(* make the successor consistent *)
let g = G.focus succ_id (G.of_blocks blockmap) in
let (vm_out, g) = add_updates succ_vm loads g in
let blockmap = G.to_blocks (G.unfocus g) in
let succ = UM.find succ_id blockmap in

(* make the predecessors consistent *)
(*
Cfgutil.print_cfg cfg;
Printf.eprintf "#preds\n";
List.iter (fun n -> Printf.eprintf "%d " (G.num n)) (G.preds succ);
Printf.eprintf "\n#allocated preds\n";
List.iter (fun n -> Printf.eprintf "%d " (G.num n))
  (List.filter (is_allocated ) (G.preds succ));
Printf.eprintf "\n";
*)

```

```

let handle_pred blockmap pred_id =
  let pred = UM.find pred_id blockmap in
  let () = Debug.eprintf "dls" "handle_pred: pred is %s\n" (blockname pred) in
  let pred_vm =
    try getVarOut pred_id
    with Not_found ->
      (Cfgutil.print_cfg cfg;
       Printf.eprintf "node %s; pred %s\n" (blockname succ) (blockname pred);
       impossf "shuffling: pred node with no outmap") in
  if Debug.on "dls" then VM.print "pred:" pred_vm;
  if Debug.on "dls" then VM.print "succ:" succ_vm;
  (* printTempSet "pred_live_out\n" (get_live_out pred); *)
  (* printTempSet "succ_live_in\n" (get_live_in succ); *)
  let stores =
    RS.fold (fun t stores ->
      match VM.var_locs' pred_vm t with
      | {VM.mem = Some _} -> stores
      | {VM.reg = Some r} ->
        let s = store t r in
        let () = Debug.eprintf "dls" " spilling %s\n" (printReg r) in
        s::stores
      | _ -> impossf "Unallocated temp in pred's outmap")
    temps [] in
  let (head, last) = GR.goto_end (GR.unzip (UM.find pred_id blockmap)) in
  let g = G.tozgraph ((head, GR.Last last), blockmap) in
  let (pred_vm, g) = add_updates pred_vm stores g in
  let blockmap = G.to_blocks (G.unfocus g) in

  (*VM.print "new maps" pred_vm;*)
  let () = setVarOut pred_id pred_vm in
  blockmap in
let handle_pred blockmap pred_id =
  try ignore (getVarOut pred_id); handle_pred blockmap pred_id
  with Not_found -> blockmap in
let blockmap = List.fold_left handle_pred blockmap (UM.find succ_id predmap) in
blockmap, predmap in

```

<define shuffle_around and shuffle_between 131>+≡

(129d) <130c

```

let shuffle_between blockmap predmap pred_id succ_id =
  let () = Debug.eprintf "dls" "shuffle_between\n" in
  let pred = UM.find pred_id blockmap in
  let succ = UM.find succ_id blockmap in
  let shuffle_err temp =
    impossf "%s %s %s %s %s %s %s %s"
      "DLS register allocator error in shuffle(): Temp" (printReg temp)
      "allocated in succ" (blockname succ) "but not pred" (blockname pred)
      "map in function" (proc.symbol#original_text) in
  let _fail () = VM.print "Pred VM:" pred_vm;
    VM.print "Succ VM:" succ_vm;
    impossf "Varmap has temp in unexpected locations" in
  let regs temp succ_reg (ms, ls, ss as state_lsts) =
    match VM.var_locs' pred_vm temp with
    | {VM.reg = Some r} when R.eq succ_reg r -> state_lsts
    | {VM.reg = Some r} -> ((temp, succ_reg, r)::ms, ls, ss)
    | {VM.mem = Some m} -> let l = load temp succ_reg in
      (ms, l::ls, ss)
    | {VM.reg = None; VM.mem = None} -> shuffle_err temp in
  let mems temp succ_mem (ms, ls, ss as state_lsts) =
    match VM.var_locs' pred_vm temp with

```

```

| {VM.mem = Some m} -> state_lsts
| {VM.reg = Some r} -> let s = store temp r in
    (ms, ls, s:ss)
| {VM.reg = None; VM.mem = None} -> shuffle_err temp in
let moves, loads, stores = VM.fold regs mems succ_vm ([],[],[]) in
if null loads && null moves && null stores
then let () = Debug.eprintf "dls" " --> no shuffling\n" in
    blockmap, predmap
else
    let moves      = shuffle_moves moves in
    let shuffles   = List.concat [ stores ; moves ; loads] in

    (* Make a new block containing all the shuffle instructions *)
    let (_, name as succ_lbl), succ_spans = match GR.goto_start (GR.unzip succ) with
        | GR.Label (lbl, _, spans), _ -> lbl, spans
        | GR.Entry, _ -> impossf "successor is entry block" in
    (* Make a new block containing all the shuffle instructions *)
    let new_uid = G.uid () in
    let new_lbl = (new_uid, Idgen.label name) in
    let (b,r) = tgt.T.machine.T.goto.T.embed proc (l2e succ_lbl) in
    let g = G.tozgraph ((GR.First (GR.Label (new_lbl, GR.Genlabel, succ_spans)),
        GR.Last (GR.Branch (r, succ_lbl))),
        blockmap) in
    let (vm, g) = add_updates pred_vm shuffles g in
    let (g,_) = G.block_before (proc, tgt.T.machine, l2e) b g in
    let (new_block, blockmap) = G.openz g in

    (* Add the new block to the graph and fix the predmap *)
    let blockmap = UM.add new_uid (GR.zip new_block) blockmap in
    let () = setVarIn new_uid pred_vm in
    let () = setVarOut new_uid succ_vm in
    let predmap =
        let preds = new_uid :: List.filter (fun x -> not (U.eq x pred_id))
            (UM.find succ_id predmap) in
        UM.add succ_id preds predmap in

    (* Update the last instruction of the pred block to link to the new block *)
    (* Note: [[last]] must be either a [[Branch]] or [[CBranch]];
        other variants should be handled elsewhere or not have successors. *)
    let (head, last) = GR.goto_end (GR.unzip pred) in
    let blockmap =
        match last with
        | GR.Branch (rtl, (uid, _)) ->
            assert (U.eq uid succ_id);
            let (b,r) = machine.T.goto.T.embed proc (l2e new_lbl) in
            let g = G.tozgraph ((head, GR.Last (GR.Branch (r, new_lbl))), blockmap) in
            let (g,_) = G.block_before (proc, machine, l2e) b g in
            G.to_blocks (G.unfocus g)
        | GR.Cbranch (rtl, (uidt, _ as lt), (uidf, _ as lf)) ->
            assert (U.eq uidt succ_id || U.eq uidf succ_id);
            let lt = if U.eq uidt succ_id then new_lbl else lt in
            let lf = if U.eq uidf succ_id then new_lbl else lf in
            let cbr = machine.T.retgt_br rtl in
            let g = G.tozgraph ((GR.First GR.Entry, GR.Last (GR.Exit)), blockmap) in
            let (g,_) = G.cbranch2cfg m cbr ~ifso:lt ~ifnot:lf g in
            let (new_block, blockmap) =
                match G.openz g with
                | ((GR.First GR.Entry, tail), bm) -> ((head, tail), bm)
                | _ -> Impossible.impossible "block not inserted at entry?" in
            UM.add pred_id (GR.zip new_block) blockmap

```

```

    | _ -> impossf "DLS trying to shuffle_between with non-branch pred block" in
    blockmap, predmap in

```

To reconcile variable maps, we need to create move, store, and load rtl's.

```

⟨define assign, move, store, and load 133a⟩≡ (127b)
let make_reg_empty r vm =
  match VM.reg_contents vm r with
  | Some t -> VM.remove_reg t r vm
  | None    -> vm          in
let move temp ((_,_,ms),_,c as dst) src =
  ((fun vm -> VM.add_reg temp dst (make_reg_empty dst vm)),
   tgt.T.machine.T.move proc ~src ~dst) in
let store' temp src =
  let () = Debug.eprintf "dls" "spilling %s from %s\n" (printReg temp) (printReg src) in
  let loc = get_spill_loc temp in
  tgt.T.machine.T.spill proc src loc in
let store temp src =
  let loc = get_spill_loc temp in
  ((fun vm -> VM.add_mem temp loc vm),
   tgt.T.machine.T.spill proc src loc) in
let load' temp dst =
  let loc = get_spill_loc temp in
  tgt.T.machine.T.reload proc loc dst in
let load temp dst =
  let loc = get_spill_loc temp in
  ((fun vm -> VM.add_reg temp dst (make_reg_empty dst vm)),
   tgt.T.machine.T.reload proc loc dst) in
let add_updates start_vm shuffles (g : G.zgraph) =
  let add_subgraph (vm, g) (upd_vm, block) =
    let (subg, _) = G.block2cfg m block in
    (upd_vm vm, G.splice_focus_exit g (G.unfocus subg)) in
  List.fold_left add_subgraph (start_vm, g) shuffles in

```

11.7.6 Variable Maps for the Runtime System

After we have set all the variable maps, we need to propagate them to the runtime system. For each span, we replace temporaries with the registers they have been assigned to.

```

⟨add spans for variable locations 133b⟩≡ (122)
let rewrite_block (first, _ as block) =
  let _, last = GR.goto_end (GR.unzip block) in
  let bid = GR.id block in
  let rewrite_spans ss getProp =
    let vm =
      try getProp bid
      with Not_found -> impossf "DLS: Can't modify runtime data for unknown node" in
    let rewrite l =
      let guard = function RP.Reg r -> is_tmp tgt r | _ -> false in
      let map l = match l with
        | RP.Reg r ->
          (try match VM.var_locs' vm r with
            | {VM.mem = Some m} -> Dn.loc (Automatonutil.aloc m (Register.width r))
            | {VM.reg = Some r} -> RP.Reg r
            | _ -> raise RTD.DeadValue
          with Not_found -> raise RTD.DeadValue)
        | _ -> impossf "DLS reg allocator emitting RT data: guard failed" in
      Rtlutil.Subst.loc_of_loc ~guard ~map l in
    Runtimedata.upd_spans rewrite ss in
  let span s vm = match s with Some s -> rewrite_spans s vm | None -> () in

```

```

    ( (match first with GR.Label (_, _, s) -> span (!s)  getVarIn  | _ -> ())
    ; (match last  with GR.Call c -> span c.GR.cal_spans getCallIn | _ -> ())
    ) in
G.iter_blocks rewrite_block cfg

```

```

⟨dls.ml 134a⟩+≡ (120b) <120d 136c>
let () = Debug.register "dls" "(downstairs) linear-scan register allocator"

```

11.7.7 Utilities

```

⟨Dls printers 134b⟩≡ (117)
let blockname block =
  match GR.blocklabel block with
  | Some (_, s) -> s
  | None -> "<entry block>"

let impossf fmt = Printf.kprintf Impossible.impossible fmt
let indent = "  "
let printTemps iter msg collection =
  Printf.eprintf "%s {" msg;
  iter (fun t -> Printf.eprintf ", %s%s" indent (printReg t)) collection;
  Printf.eprintf "}\n";
  flush stderr
let printTempSet  = printTemps RS.iter
let printTempList = printTemps List.iter
let printTempMap msg map =
  Printf.eprintf "%s" msg;
  RM.iter (fun t r -> Printf.eprintf "%s%s -> %s\n" indent (printReg t)
                                     (printReg r)) map;
  flush stderr

```

Simple utilities used for register allocation. Most of the functions are not even specific to this particular algorithm.

```

⟨register utilities 134c⟩≡ (117)
let printReg ((s,_,_), i, R.C n) =
  if n = 1 then Printf.sprintf "%c%d" s i
  else Printf.sprintf "%c%d:%d" s i n
let true_fun _ = true
let diff lst set = List.filter (fun r -> not (RS.mem r set)) lst
let inter lst set = List.filter (fun r -> RS.mem r set) lst

exception Evict of Register.t * Register.t
let is_tmp tgt (s,_,_) = Target.is_tmp tgt s
let rem_regs tgt temps = RS.filter (is_tmp tgt) temps
let get_hw_regs tgt temps = RS.filter (fun t -> not (is_tmp tgt t)) temps
let partition_regs tgt temps = RS.partition (is_tmp tgt) temps
(* I HOPE TO ELIMINATE THIS CASE WITH SOME BETTER PREFERENCING *)
let get_copy_regs rtl =
  match rtl with
  | Some i -> Rtlutil.RTLType.singleAssignment i
  | None -> None
let ( ++ ) = RS.union
let ( -- ) = RS.diff
let irwk = Rtlutil.ReadWriteKill.sets_promote
let defsm middle =
  let uses, defs, kills = irwk (GR.mid_instr middle) in
  defs ++ kills

let defsl last =

```

```

let uses, defs, kills = irwk (GR.last_instr last) in
let defs = defs ++ kills in
let p = R.promote_rxset in
p (G.union_over_outedges last (fun _ -> R.rset_to_rxset defs)
  (fun {G.node = n'; G.defs = d; G.kills = k} ->
    (R.rset_to_rxset (defs ++ p d ++ p k))))

let usesm middle =
  let uses, _, _ = irwk (GR.mid_instr middle) in
  R.promote_rxset (R.rset_to_rxset uses)
let usesl last =
  let uses, _, _ = irwk (GR.last_instr last) in
  R.promote_rxset (G.add_inedge_uses last (R.rset_to_rxset uses))

⟨Dls utilities 135a⟩≡ (117)
let null = function [] -> true | _ -> false

let choose_reg vm t =
  match (VM.var_locs' vm t).VM.reg with
  | Some r -> r
  | None -> raise Not_found

let regs_in_vm tgt vm temps =
  RS.fold (fun t set ->
    if is_tmp tgt t then
      match (VM.var_locs' vm t).VM.reg with Some r -> RS.add r set
      | None -> set
    else RS.add t set)
  temps RS.empty

let make_map tgt reg_map reg =
  if is_tmp tgt reg
  then try choose_reg reg_map reg
    with Not_found ->
      ( VM.print "" reg_map
      ; flush stderr
      ; impossf "DLS: failed to allocate temp %s" (printReg reg)
      )
  else reg

let rewrite_rtl tgt ~varIn ~varDefs rtl =
  Rtlutil.Subst.reg_def ~map:(make_map tgt varDefs)
  (Rtlutil.Subst.reg_use ~map:(make_map tgt varIn) rtl)

(* (time fn written by John Harrison) *)
let time str f x =
  let start_time = Sys.time() in
  let result = f x in
  let finish_time = Sys.time() in
  Printf.eprintf "CPU time (user) used by dls(%s): %f\n"
    str (finish_time -. start_time);
  result

⟨print cfg plus live-in and live-out sets 135b⟩≡ (122)
begin
  Cfgutil.print_cfg cfg;
  let print_block b =
    let _, last = GR.goto_end (GR.unzip b) in
    let set = Register.SetX.to_string in

```



```

    let live_out = try set (Live.live_out_last last)
                      with Not_found -> "$$Not_found$$" in
    Printf.eprintf "Block %s: live_out_last { %s }\n" (blockname b) live_out in
    G.iter_blocks print_block cfg
end

```

```

⟨print varmaps 136a⟩≡ (122)
    let print_vms b =
      let id = GR.id b in
    Printf.eprintf "block %s\n" (blockname b);
    (VM.print (Printf.sprintf "InMap for %s" (blockname b)) (getVarIn id);
     VM.print (Printf.sprintf "OutMap for %s" (blockname b)) (getVarOut id)) in
    G.iter_blocks print_vms cfg

```

11.7.8 Exporting to Lua

Boilerplate for exposing this register allocator to Lua.

```

⟨dls.mli 136b⟩≡
    val dls: 'a -> Ast2ir.proc -> Ast2ir.proc * bool

    More code for exposing the register allocator to Lua.

```

```

⟨dls.ml 136c⟩+≡ (120b) ◁134a
    ⟨dls type declarations and utilities 117⟩
    ⟨dls algorithm 122⟩

```

Chapter 12

Layout

Chapter 13

Code Generation

13.1 Widening

`<codegen/widen.ml 138a>≡`
`<widen.ml 138c>`

`<codegen/widen.mli 138b>≡`
`<widen.mli 138d>`

`<widen.ml 138c>≡` `(138a) 139d>`
`(* claude: Nopoly.(=$=)/(==) below need this open; every other .nw-tangled`
`* .ml in this tree gets it prepended automatically by the (now-dropped)`
`* generator, so it has to be added by hand here. *)`
`open Nopoly`
`let () =`
`(* claude: Debug.register takes labeled args now (error/debug.mli),`
`* upstream's positional call form no longer type-checks. *)`
`Debug.register ~word:"widen" ~meaning:"show results of widener";`
`Debug.register ~word:"widen-pre" ~meaning:"show RTL before widening";`
`Debug.register ~word:"widen-vars" ~meaning:"show when a variable is widened";`
`Debug.register ~word:"gamma" ~meaning:"show how Gamma is constructed";`
`Debug.register ~word:"widen-op-solns" ~meaning:"show operator solutions during widening"`
`let not_null = function [] -> false | _ :: _ -> true`

float is a simple widener for floating-point values. Parameter `rm` contains the hardware rounding modes.

`<widen.mli 138d>≡` `(138b) 138e>`
`val float : rm:Rtl.exp -> int list -> Rtl.rtl -> Rtl.rtl`
`(* widen all floating-point operations *)`

`<widen.mli 138e>+≡` `(138b) <138d 138f>`
`val store_const : int -> Rtl.rtl -> Rtl.rtl`
`(* l := k => l := lobits(k') *)`

A dynamic programming implementation of integer widening.

`<widen.mli 138f>+≡` `(138b) <138e 138g>`
`val dpwiden : Ast2ir.proc -> Rtl.rtl -> Rtl.rtl`

Before running the integer widener, `widenlocs` should be called to make each store and fetch an acceptable width.

`<widen.mli 138g>+≡` `(138b) <138f 139a>`
`val widenlocs : ('a, 'b, 'c) Target.t -> Rtl.rtl -> Rtl.rtl`

Check if widths are less than the target width. The `lualink.nw` file uses `Doesn't_need_widening` to avoid updating RTLs that don't need integer widening. The `needs_widening` function is called to determine if an RTL needs integer widening.

```
<widen.mli 139a>+≡ (138b) <138g 139b>
exception Doesn't_need_widening
val needs_widening : ('a, 'b, 'c) Target.t -> Rtl.rtl -> bool
```

`init_gamma_counts` and `update_gamma_counts` are used by `lualink.nw` to count the context in which local variables are used in a procedure. The function `create_gamma` is called after updating all the counts for the procedure and before doing integer widening. The counts are used to suggest fill types for widening of variables. The `gamma` environment is also used to get the original width of widened locations—unlike the fill type hints, this needs to be accurate.

```
<widen.mli 139b>+≡ (138b) <139a 139c>
val init_gamma_counts : unit -> unit
val update_gamma_counts : ('a, 'b, 'c) Target.t -> Rtl.rtl -> unit
val create_gamma : unit -> unit
```

Finally, `width_cost` and `app_count` can be used to gauge the effectiveness of the dynamic programming algorithm in keeping widened expressions small.

```
<widen.mli 139c>+≡ (138b) <139b>
val width_cost : Rtl.rtl -> (int * int * int)
(* count extension and truncation operations (#sign, #zero, #lobits) *)
(* we don't count these operations applied to locations or constants *)

val app_count : Rtl.rtl -> int
(* this probably shouldn't be here...it just counts the number of
   RP.App present in the rtl *)
```

13.1.1 Floating-point widener

```
<widen.ml 139d>+≡ (138a) <138c 139e>
module PA = Preast2ir
module R = Rtl
module RP = Rtl.Private
module RU = Rtlutil

module Dn = Rtl.Dn
module Up = Rtl.Up

(* THIS IS PRETTY CHEESY. WE SHOULD BE FIXING THE BUG IN THE COMPILER
   THAT CAUSES MACHINE-DEPENDENT RTL OPERATORS TO BE UNKNOWN. *)
let safe_has_floating_result op =
  try Rtlop.has_floating_result op with Not_found -> false

let print_debug = false
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let null = function [] -> true | _ :: _ -> false

exception Abort_unknown
exception Doesn't_need_widening
```

We're given `goodwidths`, and our goal is to change all floating-point operations in an RTL to be using one of the widths in `goodwidths`. (In practice we choose `bestwidth w`, which is the smallest width in `goodwidths` that is at least as large as `w`.)

```
<widen.ml 139e>+≡ (138a) <139d 142b>
let float ~rm goodwidths =
  let rm = Dn.exp rm in
  let goodwidths = List.sort compare goodwidths in
```

```

let isgood w = List.exists (fun w' -> w' = w) goodwidths in
let bestwidth w =
  try List.find (fun w' -> w' >= w) goodwidths
  with Not_found -> Unsupported.widen_float w in
⟨floating-point widening 140a⟩
rtl

```

I'm a little shaky what is the postcondition regarding float-to-float conversions. It's embarrassing!! For example, suppose the source program tries to narrow a floating-point value to single precision. We owe it to ourselves to arrange to store and reload it at that precision, but I'm sure that doesn't happen. There may be other errors lurking as well.

Anyway `f2f'` implements a float-to-float conversion, which may or may not combine with a conversion underneath it.

⟨floating-point widening 140a⟩≡ (139e) 140b▷

```

let rec make_cvt cvt s d e rm =
  if isgood s || isgood d then
    RP.App((cvt, [s; d]), [e; rm])
  else
    let w = bestwidth (max s d) in
    let cvtleft, cvtright =
      match cvt with
      | "f2f" -> "f2f", "f2f"
      | "i2f" -> "f2f", "i2f"
      | "f2i" -> "f2i", "f2f"
      | _ -> Impossible.impossible "unknown conversion" in
    make_cvt cvtleft w d (make_cvt cvtright s w e rm) rm in
let f2f' combine srcw dstw e =
  let make_f2f = make_cvt "f2f" in
  if srcw = dstw then
    e
  else
    match e with
    | RP.App(("f2f", [from; to']), [e; rm]) when combine ->
if to' <> srcw then
  Printf.eprintf "f2f[%d->%d] applied to f2f[%d->%d]\n" srcw dstw from to';
  assert (to' = srcw);
  make_f2f from dstw e rm
  | RP.App(("f2f_implicit_round", [from; to']), [e]) when combine ->
  assert (to' = srcw);
  make_f2f from dstw e rm
  | e ->
  make_f2f srcw dstw e rm in
let f2f = f2f' true in
let force_f2f = f2f' true in

```

Function `convert` takes an expression that is known to produce a floating-point value, and it changes the width of the expression, while rewriting all subexpressions.

⟨floating-point widening 140b⟩+≡ (139e) <140a 141a▷

```

let rec convert ~(from:int) ~(to':int) e =
  let cvt = convert ~from ~to' in
  match e with
  | RP.Const _ ->
    f2f from to' e
  | RP.Fetch(l, w) ->
    f2f from to' (RP.Fetch(loc l, w))
  | RP.App(("f2f", [f; t]), [x; rm]) ->
    convert ~from:f ~to' x
  | RP.App(("f2f_implicit_round", [f; t]), [x]) ->
    convert ~from:f ~to' x

```

```

| RP.App (("i2f", [f; t]), [x; rm]) ->
  make_cvt "i2f" f to' (rewrite x) rm
| RP.App (((("pinf"|"minf"|"pzero"|"mzero") as op, [w]), []) ->
  assert (w=from);
  f2f from to' (RP.App((op, [w]), []))
| RP.App (("NaN" as op, [n; w]), [x]) ->
  assert (w=from);
  f2f from to' (RP.App((op, [n; w]), [x]))
| RP.App (((("fabs"|"fneg") as op, [w]), [x]) ->
  assert (w=from);
  RP.App((op, [to']), [cvt x])
| RP.App (((("fadd"|"fsub"|"fmul"|"fdiv") as op, [w]), [x; y; rm]) ->
  RP.App((op, [to']), [cvt x; cvt y; rm])
| RP.App (((("fsqrt") as op, [w]), [x; rm]) ->
  assert (w=from);
  RP.App((op, [to']), [cvt x; rm])
| RP.App ((op, _), _) ->
  Impossible.impossible
  ("float widener widening non-float value (applied " ^ op ^ ")")

```

Function `rewrite` rewrites all subexpressions to ensure the postcondition. Its major job is to call `convert` when it spots a floating-point subexpression.

The third case (with the warning) might occur if we pass a widened floating-point variable to an integer comparison operation.

```

⟨floating-point widening 141a⟩+≡ (139e) <140b 141b>
and rewrite e = match e with
| RP.Const _ -> e
| RP.Fetch (l, e) -> RP.Fetch(loc l, e)
| RP.App (op, args) when safe_has_floating_result (Up.opr op) ->
  Debug.eprintf "widen"
  "Warning -- rewriting (not converting) floating-point result";
  let w = Rtlutil.Width.exp' e in
  convert w w e
| RP.App (("f2i", [fw; iw]), [x; rm]) ->
  let fw' = bestwidth fw in
  make_cvt "f2i" fw' iw (convert ~from:fw ~to':fw' x) rm
| RP.App (((("fcmp"|"flt"|"fle"|"fgt"|"fge"|"feq"|"fne"|"fordered"|"funordered")
  as opname, [w]), [x; y]) ->
  let w' = bestwidth w in
  RP.App ((opname, [w']), [convert ~from:w ~to':w' x; convert ~from:w ~to':w' y])
| RP.App (op, args) ->
  RP.App (op, List.map rewrite args)

```

Function `loc` rewrites all subexpressions that appear in a location (i.e., in an addressing expression).

```

⟨floating-point widening 141b⟩+≡ (139e) <141a 141c>
and loc l = match l with
| RP.Mem(sp, w, addr, assn) -> RP.Mem(sp, w, rewrite addr, assn)
| RP.Slice(w, lsb, l) -> RP.Slice(w, lsb, loc l)
| RP.Reg(_, _, _)
| RP.Var(_, _, _)
| RP.Global(_, _, _) -> l in

```

Assignment requires that we know whether the right-hand side is a floating-point expression. If so, `convert`, otherwise `rewrite`.

```

⟨floating-point widening 141c⟩+≡ (139e) <141b 142a>
let is_floating_exp = function
| RP.App(op, args) -> safe_has_floating_result (Up.opr op)
| _ -> false in
let guarded (g, eff) =
  let eff = match eff with

```

```

| RP.Store(l, r, w) ->
  if is_floating_exp r then (* && bestwidth w <> w then *)
    let r = convert ~from:w ~to':(bestwidth w) r in
    RP.Store(loc l, force_f2f (bestwidth w) w r, bestwidth w)
  else
    RP.Store(loc l, rewrite r, w)
| RP.Kill l -> RP.Kill (loc l) in
(rewrite g, eff) in

```

In the common case where we have no operators that need conversion, we avoid rewriting the RTL, thereby reducing the load on the major heap.

```

⟨floating-point widening 142a⟩+≡ (139e) <141c
let rtl r =
  let needs_conversion op =
    safe_has_floating_result op ||
    match fst (Dn.opr op) with
    | "fcmp"|"flt"|"fle"|"fgt"|"fge"|"feq"|"fne"|"fordered"|"funordered"|"f2i" -> true
    | _ -> false in
  if Rtlutil.Exists.Opr.rtl needs_conversion r then
    let rtl = Rtlutil.ToString.rtl in
    let _ = if Debug.on "widen-pre" then Printf.eprintf "Widening %s...\r" (rtl r) in
    let r' =
      let RP.Rtl es = Dn.rtl r in Up.rtl (RP.Rtl (List.map guarded es)) in
    if Debug.on "widen" then Printf.eprintf "Widened %s into\n %s\n" (rtl r) (rtl r');
    r'
  else
    r in

```

```

⟨widen.ml 142b⟩+≡ (138a) <139e 160a>
let store_const goodwidth rtl = match Dn.rtl rtl with
| RP.Rtl [(g, RP.Store(l, (RP.Const (RP.Bits b) as k), w))] when w < goodwidth ->
  let widek = RP.App(("zx", [w; goodwidth]), [k]) in
  let widek = Dn.exp (Simplify.exp (Up.exp widek)) in
  let lobits = RP.App(("lobits", [goodwidth; w]), [widek]) in
  Up.rtl (RP.Rtl [(g, RP.Store(l, lobits, w))])
| _ -> rtl

```

13.1.2 Integer widening by dynamic programming

Some basic types used in the integer widener:

```

⟨Widen types 142c⟩≡ (160a) 142d>
type fill = G | Z | S | H | F
type index = int
type width = int

```

Type `fill` represents the kind of the high bits produced by a widened expression: garbage, zeros, sign bits, and “high-bits contain the value”.

The dynamic programming algorithm works by generating all possible solutions based on a set of rules and the operations available on the target machine. A solution is represented as a type `solution` which is a record containing the original expression `e`, the width of the original expression `m`, the cost of this solution `cost`, the widened expression `e'`, the fill type of the widened expression `fill`, the index of the fill type `n`, and finally the width of the widened expression `w`.

```

⟨Widen types 142d⟩+≡ (160a) <142c 143a>
type solution = {e : RP.exp; m : width; cost : int; e' : RP.exp;
  fill : fill; n : index; w : width}

```

There are three types of environments used by the widener: type `machine_env` describes the abilities of the target machine, type `fill_env` gives the fill type of each operator, and `gamma` gives a fill type to each local variable.

```

⟨Widen types 143a⟩+≡ (160a) ◁142d 143b▷
  type machine_env = { ops      : (string * (width list)) list
                      ; literals : width list
                      ; pointer  : width
                      ; word     : width
                      ; rm       : RP.exp
                      ; itemps   : width list
                      ; ftemps   : width list
                      ; is_hardware_register : RP.loc -> bool
                      }

  type fill_env      = (string * fill * index) list
  let (gamma : (string, (fill * int)) Hashtbl.t) = Hashtbl.create 50

⟨Widen types 143b⟩+≡ (160a) ◁143a
  type tcount = {z:int; s:int; h:int; ni:int}
  let (gamma_counts : (string, tcount) Hashtbl.t) = Hashtbl.create 50

```

Expanding the solution frontier

The dynamic programming algorithm works bottom-up by applying two phases at each AST node.

Apply syntax translation rules: In the first phase we apply each rule that can match the syntax of the original expression—there is exactly one rule for each kind of expression. The first phase is implemented in the function `ast_combine`, which simply pattern-matches the incoming AST and applies the appropriate rule. A description of this rule comes a bit later. Some rules are non-deterministic and thus can produce several possible solutions for a given input. Therefore each rule returns a list of all possible solutions for a given input.

Apply width-changing and subtyping rules: In the second phase we take all the solutions from the first phase and apply several rules that needn't match particular input syntax. These second-phase rules can be applied to any input solution, although they have conditions that must be met to produce a non-empty list of result solutions. This second phase is implemented in the function `find_reachable` in the next code chunk. These two phases are applied bottom up at each AST node to produce all possible solutions.

The function `find_reachable` implements the second phase applied at each AST node. The rules we apply at this phase may usefully be reapplied to the solutions from other rules in the set. We want to apply each rule to the set of solutions until no new solutions are returned. The function takes two parameters: `known`, the entire set of solutions, and `frontier`, the set of solutions that have just been generated. `find_reachable` applies each rule to the `frontier` to create a fresh frontier. It then recurs using the “relevant” solutions in the fresh frontier as the `frontier` and the union of the old frontier with the old known solutions as `known`.

```

⟨find reachable solutions 143c⟩≡ (160a)
  ⟨relations 144c⟩
  ⟨solution set utilities 144a⟩
  ⟨applying rules 145b⟩
  let rec find_reachable m known frontier =
    let frontier' = solutions_unique (List.concat (List.map (apply_rules m) frontier)) in
    let known     = solutions_union frontier known in
    let frontier  = List.filter (fun soln -> relevant_given soln known) frontier'
    in
    if not_null frontier then find_reachable m known frontier else known

```


`find_reachable` relies on the ability to create a set of solutions from a list of solutions using the function `solutions_unique`.

$\langle \text{solution set utilities } 144a \rangle \equiv (143c) \ 144b \triangleright$

```
let solutions_union xs ys =
  let xs' = List.filter (fun soln -> relevant_given soln ys) xs in
  xs' @ ys
```

`find_reachable` also needs to construct the union of two sets of solutions using the function `solutions_union`.

$\langle \text{solution set utilities } 144b \rangle + \equiv (143c) \ \triangleleft 144a$

```
let solutions_unique =
  let rec make_unique result = function
    | [] -> result
    | soln :: rest ->
        if relevant_given soln rest && relevant_given soln result then
          make_unique (soln :: result) rest
        else make_unique result rest
  in make_unique []
```

All three of these functions rely on being able to tell if a solution is *relevant* given a set of known solutions. A solution is relevant if there does not exist a known solution that makes it redundant.

$\langle \text{relations } 144c \rangle \equiv (143c)$

```
 $\langle \text{redundant } 144d \rangle$ 
let relevant_given soln known = not (List.exists (redundant_given1 soln) known)
```

The function `redundant_given1` determines if a solution `a` is redundant given that we already know solution `b`.

$\langle \text{redundant } 144d \rangle \equiv (144c)$

```
 $\langle \text{comparable } 145a \rangle$ 
let rec redundant_given1 a b =
  if comparable a b then
    if is_tau a then
      if b.m <= index b then
        if index a > index b && cost a < cost b then false
        else if index a < index b && cost a > cost b then false
        else if index a >= index b then cost a >= cost b
        else (* index a < index b *) false
      else if a.m > index a then
        if index a = index b then cost a >= cost b
        else false
      else if a.m <= index a then false
      else impossf "comparing solutions for redundancy: missing case"
    else if a.fill == H then
      (* pretend we have converted back to some sigma type and compare *)
      redundant_given1 {a with n = width a - index a; fill = S}
                        {b with n = width b - index b; fill = S}
    else (* a.fill == F *)
      cost b < cost a
  else false
```

The first thing that `redundant_given1` checks is that the two solutions are actually comparable at all—for solutions to be comparable, they must widen the same original expression to the same width with the same fill type. We also check that the original expression have the same width, although this should be unnecessary. Note that to be comparable they do not need to have the same fill *index*. The index is part of how we decide if a solution is redundant. Conditions from the SUBSUME-INDEX rule are baked into `redundant_given1`.

$$\frac{\text{SUBSUME-INDEX } (n < n' \leq w) \quad \Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n] : w \quad M^\infty \vdash e : n}{\Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n'] : w}$$

There should probably be a subsumption rule for high bits types. We deal with high bits redundancy by just pretending that the high-bits solutions have been converted back to sign-bits solutions and compare them as usual.

Determining of a solution is redundant is the way we trim the search space, so it's important to throw away as many solutions as we can without actually ignoring a potential desired solution. The dynamic programming algorithm is designed to take the minimum cost solution, so `redundant_given1` throws away solutions that are essentially the same as a known solutions but with higher cost.

```

⟨comparable 145a⟩≡ (144d)
  let comparable a b =
    RU.Eq.exp (ast a) (ast b) && width a = width b && fill a == fill b && a.m = b.m

```

Width-changing and subtyping rules

The SUBSUME-INDEX rule is implicit in the definition of other rules—other rules search for solutions at a desired index and may accept a smaller index if the preconditions of this rule are met. This rule is also partly implemented in the function `redundant_given1`.

$$\frac{\text{SUBSUME-INDEX } (n < n' \leq w) \quad \Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n] : w \quad M^\infty \vdash e : n}{\Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n'] : w}$$

We use the function `subsume_index` to check if a solution `soln` can be used in place of a solution with index `n'`. The `subsume_index` function *assumes* that the solution you have and the solution you are asking for have the same fill type even though you only pass the index `n'` in instead of an entire solution.

```

⟨applying rules 145b⟩≡ (143c) 145c▷
  let rec subsume_index soln n' =
    match fill soln with
    | S | G | Z -> (soln.n < n' && soln.m <= soln.n) || soln.n = n'
    | H          -> subsume_index {soln with fill = S; n = soln.w - soln.n} n'
    | F          -> true

```

Again, the high-bits type is dealt with by pretending it has been converted back to sign-bits type and comparing as usual.

$$\frac{\text{SUBSUME-FILL } (n \leq w) \quad \Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n] : w}{\Gamma, M \vdash e \xrightarrow{c} e' :: g[n] : w}$$

```

⟨applying rules 145c⟩+≡ (143c) <145b 145d▷
  let subsume_fill_rule _ soln =
    if is_sigma soln then
      [subst_fill soln G]
    else []

```

$$\frac{\text{NATURAL} \quad \Gamma, M \vdash e \xrightarrow{c} e' :: g[w] : w}{\Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[w] : w}$$

The NATURAL rule tells us that if the fill index is already at the high end of the bitvector then we can switch to any fill type desired (it's vacuously true). The first part of the `natural_rule` function implements NATURAL as follows:

```

⟨applying rules 145d⟩+≡ (143c) <145c 146a▷
  let natural_rule _ soln =
    let create fill' = {soln with fill = fill'; n = soln.w} in
    if width soln = index soln && fill soln == G then

```

```

[create Z; create S;
 (* {soln with fill = H; n = 0} *)
]

```

However, we also need to take into account the possibility that NATURAL is combined with the SUBSUME-INDEX rule, which says that we can increase the fill index under certain circumstances—the next branch of the conditional gives solutions for the combined rule case.

$\langle \text{applying rules 146a} \rangle + \equiv$ (143c) $\triangleleft 145d \ 146b \triangleright$

```

else match fill soln with
| S | Z -> List.filter (fun s -> subsume_index soln (index s)) [create Z; create S]
| H | G | F -> []

```

$$\frac{\text{FILL } (\mathcal{U}(k_w) = n, n \leq w \leq w') \quad \Gamma, M \vdash e \xrightarrow{c} e' :: g[n] : w \quad \sigma x_{lo_{w'} \leftarrow w} \in M}{\Gamma, M \vdash e \xrightarrow{c+1} \sigma x_{lo_{w'} \leftarrow w}(k_w, e') :: \sigma[n] : w'}$$

$\langle \text{applying rules 146b} \rangle + \equiv$ (143c) $\triangleleft 146a \ 146c \triangleright$

```

let fill_rule m soln =
  match fill soln with
  | G ->
    let w = width soln in
    let lo_exists = not_null (find_exts_from m w "lobits") in
    if lo_exists then
      let n = index soln in
      let zx_exists = not_null (find_exts_to m w "zx") in
      let sx_exists = not_null (find_exts_to m w "sx") in
      let create fill' ext =
        { soln with
          cost = soln.cost + 2;
          e' = RP.App((ext, [n;w]), [RP.App(("lobits", [w;n]), [soln.e'])]);
          fill = fill' }
      in
      let zx_solns = if zx_exists then [create Z "zx"] else [] in
      let sx_solns = if sx_exists then [create S "sx"] else [] in
      let solns = zx_solns @ sx_solns in
      List.filter (fun s -> s.n <= s.w) solns (* this is necessary *)
    else []
  | _ -> []

```

$$\frac{\text{WIDEN-}\sigma \ (n \leq w \leq w') \quad \Gamma, M \vdash e \xrightarrow{c} e' :: \sigma[n] : w \quad \sigma x_{w' \leftarrow w} \in M}{\Gamma, M \vdash e \xrightarrow{c+1} \sigma x_{w' \leftarrow w} e' :: \sigma[n] : w'}$$

$\langle \text{applying rules 146c} \rangle + \equiv$ (143c) $\triangleleft 146b \ 147a \triangleright$

```

let widen_sigma m soln =
  if is_sigma soln then
    let ext = ext_of_fill soln.fill in
    let w = width soln in
    let ext_widths = find_exts_from m w ext in
    let create ws =
      match Rtl.op.mono (Rtl.opr ext ws) with
      | ([Types.Bits _], Types.Bits w') ->
        { soln with
          e' = RP.App((ext, ws), [soln.e']);
          cost = soln.cost + 1;
          w = w' }
      | _ -> impossf "%s operator should have some type bits#n->bits#w" ext
    in
    List.map create ext_widths

```

else []

$$\frac{\text{WIDEN-g } (n \leq w \leq w') \quad \Gamma, M \vdash e \xrightarrow{c} e' :: g[n] : w \quad \sigma x_{w' \leftarrow w} \in M}{\Gamma, M \vdash e \xrightarrow{c+1} \sigma x_{w' \leftarrow w} e' :: g[n] : w'}$$

<applying rules 147a>+≡ (143c) <146c 147b>

```

let widen_garbage m soln =
  match fill soln with
  | G ->
    let w = width soln in
    let zx_widths = find_exts_from m w "zx" in
    let sx_widths = find_exts_from m w "sx" in
    let create_fill' ext ws =
      match Rtlop.mono (Rtl.opr ext ws) with
      | ([Types.Bits _], Types.Bits w') ->
        { soln with
          e' = RP.App((ext, ws), [soln.e']);
          cost = soln.cost + 1;
          w = w' }
    | _ -> impossf "%s operator should have some type bits#n->bits#w" ext in
    List.map (create Z "zx") zx_widths @ List.map (create S "sx") sx_widths
  | _ -> []

```

<applying rules 147b>+≡ (143c) <147a 147c>

```

let insert_f2f m soln =
  match fill soln with
  | F ->
    let srcw = width soln in
    let dstwss = find_exts_from m srcw "f2f" in
    let create_dstws =
      match Rtlop.mono (Rtl.opr "f2f" dstwss) with
      | ([Types.Bits _; Types.Bits _], Types.Bits dstw) ->
        { soln with
          e' = RP.App(("f2f", dstws), [soln.e'; m.rm]);
          cost = soln.cost + 1;
          w = dstw }
    | _ -> impossf "%f2f should be a binary operator" in
    List.map create_dstwss
  | _ -> []

```

<applying rules 147c>+≡ (143c) <147b 148a>

```

let insert_i2f m soln =
  match fill soln with
  | S ->
    let srcw = width soln in
    let dstwss = find_exts_from m srcw "i2f" in
    let create_dstws =
      match Rtlop.mono (Rtl.opr "i2f" dstwss) with
      | ([Types.Bits _; Types.Bits _], Types.Bits dstw) ->
        { soln with
          e' = RP.App(("i2f", dstws), [soln.e'; m.rm]);
          cost = soln.cost + 1;
          fill = F;
          w = dstw }
    | _ -> impossf "%i2f should be a binary operator" in
    List.map create_dstwss
  | _ -> []

```

$$\begin{array}{c}
\text{WIDEN-HW-}\sigma \ (n \leq n' \leq w, x_{n'} \text{ IS HARDWARE REG}) \\
w \in \text{itemwidth} \quad \Gamma, M \vdash e \xrightarrow{c}_c x_{n'} :: \sigma[n] : n' \\
\hline
\Gamma, M \vdash e \xrightarrow{c+1}_c \sigma x_{w \leftarrow n'} x_{n'} :: \sigma[n] : w \\
\\
\text{WIDEN-HW-}g \ (n \leq n' \leq w, x_{n'} \text{ IS HARDWARE REG}) \\
w \in \text{itemwidth} \quad \Gamma, M \vdash e \xrightarrow{c}_c x_{n'} :: g[n] : n' \\
\hline
\Gamma, M \vdash e \xrightarrow{c+1}_c \sigma x_{w \leftarrow n'} x_{n'} :: g[n] : w
\end{array}$$

<applying rules 148a>+≡ (143c) <147c 148b>

```

let widen_hw m soln =
(* if Debug.on "widen" then
  Printf.eprintf "widen_hw: %s -> %s :: %s[%i] : %i\n"
    (Rtlutil.ToString.exp (Up.exp soln.e)) (Rtlutil.ToString.exp (Up.exp soln.e'))
    (string_of_fill soln.fill) (index soln) (width soln); *)
match soln.e' with
| RP.Fetch(1,_) when m.is_hardware_register 1 ->
  let n' = width soln in
  let ws = List.filter (fun w -> n' <= w) m.items in
  let create_ext f w = {soln with e' = RP.App((ext, [n';w]),[soln.e'])};
    fill = f;
    w = w;
    cost = soln.cost + 1}

  in
  let solns' =
    match fill soln with
    | G -> List.map (create "zx" G) ws
    | Z -> List.map (create "zx" Z) ws
    | S -> List.map (create "sx" S) ws
    | H -> []
    | F -> []

  in
  (* if Debug.on "widen" then
    List.iter (fun s -> Printf.eprintf
      "widen_hw success: %s -> %s :: %s[%i] : %i\n"
      (RU.ToString.exp (Up.exp s.e))
      (RU.ToString.exp (Up.exp s.e'))
      (string_of_fill s.fill)
      (index s) (width s)) solns'; *)

  solns'
| _ -> []

```

$$\begin{array}{c}
\text{NARROW} \ (n \leq w \leq w') \\
\Gamma, M \vdash e \xrightarrow{c}_c e' :: \tau[n] : w' \quad \text{lo}_{w \leftarrow w'} \in M \\
\hline
\Gamma, M \vdash e \xrightarrow{c+1}_c \text{lo}_{w \leftarrow w'} e' :: \tau[n] : w
\end{array}$$

<applying rules 148b>+≡ (143c) <148a 149a>

```

let narrow_rule m soln =
  if is_tau soln then
    let w' = width soln in
    let trunc_widths = find_exts_from m w' "lobits" in
    let create ws =
      match Rtltop.mono (Rtl.opr "lobits" ws) with
      | ([Types.Bits _], Types.Bits w) ->
        { soln with
          e' = RP.App(("lobits", ws), [soln.e']);
          cost = soln.cost + 1;
          w = w }

    | _ -> impossf "lobits operator should have some type bits#w->bits#n"

```

```

in
List.filter (fun s -> s.n <= s.w) (List.map create trunc_widths)
else []

```

$$\begin{array}{c}
\text{TO-HIGH}(\mathcal{U}(k_w) = w - n, n \leq w) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: g[n] \quad \text{shl} : w \times w \rightarrow w}{\Gamma, M \vdash e \xrightarrow{c+1} \text{shl}_w(e', k_w) :: h[w - n] : w}
\end{array}$$

(applying rules 149a) $\vdash \equiv$ (143c) $\triangleleft$ 148b 149b $\triangleright$

```

let to_high_rule m soln =
  match fill soln with
  | G when
    List.exists (function [w] -> w = soln.w | _ -> false) (machine_widths m "shl") ->
    let w = soln.w in
    let n = soln.n in
    [{ soln with
      e' = RP.App(("shl", [w]), [soln.e';
        RP.Const(RP.Bits(Bits.S.of_int (w-n) w))]);
      cost = soln.cost + 1;
      n = w - n;
      fill = H }]
  | _ -> []

```

$$\begin{array}{c}
\text{FROM-HIGH-S}(\mathcal{U}(k_w) = n) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: h[n] : w \quad \text{shra} : w \times w \rightarrow w}{\Gamma, M \vdash e \xrightarrow{c+1} \text{shra}(e', k_w) :: s[w - n] : w}
\end{array}$$

(applying rules 149b) $\vdash \equiv$ (143c) $\triangleleft$ 149a 149c $\triangleright$

```

let from_high_s_rule m soln =
  match fill soln with
  | H when
    List.exists (function [w] -> w = soln.w | _ -> false) (machine_widths m "shra") ->
    let w = soln.w in
    let n = soln.n in
    [{ soln with
      e' = RP.App(("shra", [w]), [soln.e';
        RP.Const(RP.Bits(Bits.S.of_int n w))]);
      cost = soln.cost + 1;
      n = w - n;
      fill = S }]
  | _ -> []

```

$$\begin{array}{c}
\text{FROM-HIGH-Z}(\mathcal{U}(k_w) = n) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: h[n] : w \quad \text{shra} : w \times w \rightarrow w}{\Gamma, M \vdash e \xrightarrow{c+1} \text{shrl}(e', k_w) :: z[w - n] : w}
\end{array}$$

(applying rules 149c) $\vdash \equiv$ (143c) $\triangleleft$ 149b 150a $\triangleright$

```

let from_high_z_rule m soln =
  match fill soln with
  | H when
    List.exists (function [w] -> w = soln.w | _ -> false) (machine_widths m "shrl") ->
    let w = soln.w in
    let n = soln.n in
    [{ soln with
      e' = RP.App(("shrl", [w]), [soln.e';
        RP.Const(RP.Bits(Bits.S.of_int n w))]);
      cost = soln.cost + 1;
      n = w - n;

```

```

    fill = Z }]
  | _ -> []

```

All the width-changing and subtyping rules are applied by the function `apply_rule`. Each rule returns a list of solutions when applied to a solution. `apply_rule` concatenates all the lists of solutions from applying relevant rules to the argument `soln`. Usually the relevant rules are all the rules, but there are two special cases: boolean constants shouldn't have their width changed since they aren't really bitvectors, and bitvector constants can be widened at compile time making it unnecessary to apply most of the rules.

```

⟨applying rules 150a⟩≡ (143c) <149c
let apply_rules m soln =
  let all_general_rules = [subsume_fill_rule;
                           natural_rule;
                           fill_rule;
                           widen_sigma;
                           widen_garbage;
                           widen_hw;
                           narrow_rule;
                           insert_f2f;
                           insert_i2f THIS IS BOGUS *)
  (*
    ]
  (*
    ; from_high_s_rule; from_high_z_rule; to_high_rule]
  *)
  in
  let all_general_rules = List.map (fun r -> r m) all_general_rules in
  let const_rules = [subsume_fill_rule; natural_rule; narrow_rule; insert_f2f] in
  let const_rules = List.map (fun r -> r m) const_rules in
  let relevant_rules =
    match ast soln with
    | RP.Const (RP.Bool _) -> []
    | RP.Const (RP.Bits _) -> const_rules
    | _ -> all_general_rules
  in List.concat (List.map (apply_with soln) relevant_rules)

```

Syntax translation rules

$$\begin{array}{c}
 \text{LIT } (k'_w = \sigma_{x_{w \leftarrow n}} k_n, n \leq w) \\
 \hline
 \Gamma, M \vdash k_n \xrightarrow{0} k'_w :: \sigma[n] : w \\
 \text{LIT-H } (k'_w = 2^{w-n} k_n, n \leq w) \\
 \hline
 \Gamma, M \vdash k_n \xrightarrow{0} k'_w :: h[w-n] : w
 \end{array}$$

```

⟨combine solutions 150b⟩≡ (160a) 151a▷
let mksolutions_lit_bool e = match e with
| (RP.Const (RP.Bool b)) -> [{e = e; m = 1; cost = 0; e' = e; fill = Z; n = 1; w = 1}]
| _ -> impossf "mksolutions_lit_bool requires a bool const"

let mksolutions_lit_bits m tau e = match e with
| (RP.Const (RP.Bits b)) ->
  let n = Bits.width b in
  let litwidths = List.filter (fun w' -> n <= w') m.literals in
  (match tau with
  | S | Z ->
    let f w' =
      let ext = if tau == S then Bits.Ops.sx else Bits.Ops.zx in
      {e = e; m = n; cost = 0; e' = RP.Const(RP.Bits (ext w' b)); fill = tau; n = n; w = w'} in
    List.map f litwidths
  | H ->

```

```

let f w' =
  let ext = Bits.Ops.shl (Bits.Ops.zx w' b) (Bits.S.of_int (w' - n) w') in
  (*
    if Debug.on "widen" then
      Printf.eprintf "%s -> %s :: H[%i] : %i\n"
        (Bits.to_string b) (Bits.to_string ext) n w'; *)
    {e = e; m = n; cost = 0; e' = RP.Const(RP.Bits ext); fill = H; n = w' - n; w = w'}
  in
  List.map f litwidths
| F -> if List.mem n m.ftemps then
    [{e = e; m = n; cost = 0; e' = e; fill = F; n = n; w = n}]
  else []
| G -> impossf "widening failed: literals cannot have garbage type"
| _ -> impossf "mksolutions_lit_bits requires a bitvector const"

```

$$\begin{array}{c}
\text{VAR } (n \leq w) \\
\frac{\Gamma(x_w) = \tau[n]}{\Gamma, M \vdash x_w \xrightarrow{o} x_w :: \tau[n] : w}
\end{array}$$

Use gamma to find the appropriate fill type for this variable.

$\langle \text{combine solutions } 151a \rangle + \equiv$ (160a) $\langle 150b \ 151b \rangle$

```

let lookup_gamma name w =
  try
    let (fill, n) = Hashtbl.find gamma name in
    if Debug.on "gamma" then
      Printf.eprintf "Found %s :: %s[%i]\n" name (string_of_fill fill) n;
    (fill, n)
  with Not_found ->
    if Debug.on "gamma" then
      Printf.eprintf "%s not found in gamma\n" name;
    (G, w)

let mksolutions_var name e w =
  let (fill, n) = lookup_gamma name w in
  let create f = {e = e; m = n; cost = 0; e' = e; fill = f; n = n; w = w} in
  match fill with
  | H -> [{e = e; m = n; cost = 0; e' = e; fill = H; n = w - n; w = w}]
  | _ -> [create fill; create F]

```

$$\begin{array}{c}
\text{BINOP } (n_1 \leq w_1, n_2 \leq w_2, n \leq w) \\
\frac{\oplus :: \tau_1 \times \tau_2 \rightarrow \tau \quad \oplus : n_1 \times n_2 \rightarrow n \in M^\infty \quad \Gamma, M \vdash e_1 \xrightarrow{c_1} e'_1 :: \tau_1[n_1] : w_1 \quad \Gamma, M \vdash e_2 \xrightarrow{c_2} e'_2 :: \tau_2[n_2] : w_2}{\Gamma, M \vdash \oplus_{n \leftarrow n_1 \times n_2} (e_1, e_2) \xrightarrow{c_1 + c_2} \oplus_{w \leftarrow w_1 \times w_2} (e'_1, e'_2) :: \tau[n] : w}
\end{array}$$

$\langle \text{combine solutions } 151b \rangle + \equiv$ (160a) $\langle 151a \ 153a \rangle$

```

let mksolutions_op m args_solns opname ws args =
  let dbg_print_args_solns solnss =
    Printf.eprintf "\t %s\n"
      (strconcatmap ";\n\t " (strconcatmap "\n\t " string_of_soln) args_solns)
  in

  if Debug.on "widen-op-solns" then
    (Printf.eprintf "Solving %%%s " opname;
     Printf.eprintf "ws = [%s]\n" (String.concat " " (List.map string_of_int ws));
     Printf.eprintf "\targs_solns =\n";
     dbg_print_args_solns args_solns);

```

(* here we just want to pretend that booleans are 1-bit values *)

```
let monomorphic opname ws =
```



```

let to_int = function
  | Types.Bits n -> n
  | Types.Bool   -> 1
in
let (a, b) =
  if List.mem_assoc opname op_fill_types then
    Rtlop.mono (Rtl.opr opname ws)
  else raise Not_found
in
(List.map to_int a, to_int b)
in

let (args_indices, result_index) =
  try monomorphic opname ws
  with Not_found -> raise Abort_unknown
in
let args_solns =
  map2_find_all (fun n' s -> subsume_index s n') args_indices args_solns
in
if Debug.on "widen-op-solns" then
  (Printf.eprintf "\targs_solns subsume_index\n";
   dbg_print_args_solns args_solns);

let m_widths = machine_widths m opname in
let m_widths = List.map (fork (monomorphic opname) id) m_widths in
if Debug.on "widen-op-solns" then
  Printf.eprintf "\tm_widths = %s\n"
    (strconcatmap ", "
     (fun (x,y) -> "["^strconcatmap "->" string_of_int x^->"^string_of_int y^"]"))
    (List.map fst m_widths));

let solns_combinations = cross_list_all args_solns in

let has_widths solns widths =
  List.for_all2 (fun s w -> s.w = w) solns (fst (fst widths)) in
let solns_and_widths = cross_pair_filter has_widths solns_combinations m_widths in

if Debug.on "widen-op-solns" then
  (Printf.eprintf "\tmap fst solns_and_widths =\n";
   dbg_print_args_solns (List.map fst solns_and_widths));

let taus = try List.assoc opname op_fill_types with _ -> [] in
let has_taus (solns, _) (taus, _) =
  List.for_all2 (fun s t -> s.fill == t) solns taus in
let solns_widths_and_taus = cross_pair_filter has_taus solns_and_widths taus in

let mkopsoln ((solns, ((_,w'), ws')), (_,t)) =
  (* THIS MIGHT BE BOGUS *)
  (* I suspect what this is trying to do is get rid of widths that
   are 1 because we were replacing a boolean.  However, there are operators
   that actually do operate on one bit values.  On the other hand,
   since they are special purpose, the width probably doesn't
   appear as a parameter for the op.  Can we show that we never
   want to accept a parameter width 1?
  *)
  let ws' = List.filter (fun x -> x > 1) ws' in
  { e      = RP.App((opname, ws'), args);
    m      = result_index;
    cost   = List.fold_left (+) 0 (List.map cost solns);
    e'     = RP.App((opname, ws'), List.map ast' solns);
  }

```

```

    fill = t;
    n     = result_index;
    w     = w' }
in
let solns = List.map mkopsoln solns_widths_and_taus in
if Debug.on "widen-op-solns" then
  Printf.eprintf "\tsolutions:\n\t%s\n"
    (String.concat "\n\t" (List.map string_of_soln solns));
solns
⟨combine solutions 153a⟩+≡ (160a) ◁151b 153b▷
let mksolutions_nullary_op m opname ws =
  let monomorphic opname ws =
    let to_int = function Types.Bits n -> n | Types.Bool -> 1 in
    let (a, b) =
      if List.mem_assoc opname op_fill_types then Rtl.op.mono (Rtl.opr opname ws)
      else raise Not_found
    in (List.map to_int a, to_int b)
  in
  let snds xs = List.map snd xs in
  let (_, result_index) = try monomorphic opname ws
    with Not_found -> raise Abort_unknown in
  let m_widths = machine_widths m opname in
  let result_widths = snds (List.map (monomorphic opname) m_widths) in
  let result_fills = try snds (List.assoc opname op_fill_types) with _ -> [] in
  let widths_and_fills = cross_pair result_widths result_fills in
  let create (ws', (w, t)) = { e = RP.App((opname, ws), []);
    m = result_index;
    cost = 0;
    e' = RP.App((opname, ws'), []);
    fill = t;
    n = result_index;
    w = w' } in
  match ws with
  | [] -> List.map (fun x -> create ([], x))
    (List.filter (fun (w,_) -> w = result_index) widths_and_fills)
  | [_] -> List.map create (List.map (fun (w,t) -> ([w],(w,t))) widths_and_fills)
  | _ -> impossf "Unary operator could have one width parameter at most"

```

Drop extention rule from the paper:

$$\begin{array}{c}
 \text{DROP-EXT } (n \leq n' \leq w) \\
 \frac{\Gamma, M \vdash e \hookrightarrow_c e' :: \sigma[n] : w}{\Gamma, M \vdash \sigma x_{n' \leftarrow n} e \hookrightarrow_c e' :: \sigma[n] : w}
 \end{array}$$

More flexible revised rule that is implemented:

$$\begin{array}{c}
 \text{DROP-EXT } (n \leq n' \leq n'' \leq w) \\
 \frac{\Gamma, M \vdash e \hookrightarrow_c e' :: \sigma[n] : w}{\Gamma, M \vdash \sigma x_{n'' \leftarrow n} e \hookrightarrow_c e' :: \sigma[n'] : w}
 \end{array}$$

```

⟨combine solutions 153b⟩+≡ (160a) ◁153a 154▷
let mksolutions_ext ext ws arg_solnss =
  let arg_solns = match arg_solnss with [s] -> s | _ -> impossf "extension arity" in

  (* find all solutions that have the appropriate sigma *)
  let sigma = fill_of_ext ext in
  let arg_solns = List.filter (fun s -> s.fill == sigma) arg_solns in

```

```

(* find all solutions with index equal to the input width of the extension *)
let (n, n'') =
  match Rtlop.mono (Rtl.opr ext ws) with
  | ([Types.Bits n], Types.Bits n'') -> (n, n'')
  | _ -> impossf "%s operator should have some type bits#n->bits#w" ext in
(* assert that n <= n'' *)

let arg_solns = List.filter (fun s -> subsume_index s n && n <= s.w) arg_solns in

let create1 arg_soln = { arg_soln with e = RP.App((ext,ws),[arg_soln.e]);
                        m = n''; n = n } in
let create2 arg_soln = { arg_soln with e = RP.App((ext,ws),[arg_soln.e]);
                        m = n''; n = n'' } in
List.map create2 arg_solns @ List.map create1 arg_solns

```

Dropping f2f may not be a great idea since we sort of lose the rounding mode information. We will try to pick the correct rounding mode back up when inserting an f2f or other operation.

```

⟨combine solutions 154⟩+≡ (160a) <153b 155a>
let mksolutions_drop_f2f ws args_solns rm =
  let s = match args_solns with [s;_] -> s | _ -> impossf "f2f arity" in
  let arg_solns = List.filter (fun s -> s.fill == F) s in
  let (srcw, dstw) =
    match Rtlop.mono (Rtl.opr "f2f" ws) with
    | ([Types.Bits n; Types.Bits _], Types.Bits n') -> (n, n')
    | _ -> impossf "%f2f should be a binary operator"
  in
  let arg_solns = List.filter (fun s -> s.w <= dstw && s.w <= srcw) arg_solns in
  let create arg_soln = { arg_soln with
                        e = RP.App(("f2f",ws),[arg_soln.e; rm]); m = srcw }
  in
  List.map create arg_solns

```

Implementation of the drop rules for truncation.

These rules need to be revised, I think. They are sound in the paper, but they could be less restrictive. In their original versions:

$$\begin{array}{c}
\text{DROP-LO-COPY } (n \leq n' \leq w) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: \tau[n'] : w}{\Gamma, M \vdash \text{lo}_{n \leftarrow n'} e \xrightarrow{c} e' :: \tau[n'] : w} \\
\\
\text{DROP-LO-IGNORE } (n \leq n' \leq w) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: \tau[n'] : w}{\Gamma, M \vdash \text{lo}_{n \leftarrow n'} e \xrightarrow{c} e' :: g[n] : w}
\end{array}$$

Modified versions I've implemented:

$$\begin{array}{c}
\text{DROP-LO-COPY } (n \leq n' \leq n'' \leq w) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: \tau[n'] : w}{\Gamma, M \vdash \text{lo}_{n \leftarrow n''} e \xrightarrow{c} e' :: \tau[n'] : w} \\
\\
\text{DROP-LO-IGNORE } (n \leq n' \leq n'' \leq w) \\
\frac{\Gamma, M \vdash e \xrightarrow{c} e' :: \tau[n'] : w}{\Gamma, M \vdash \text{lo}_{n \leftarrow n''} e \xrightarrow{c} e' :: g[n] : w}
\end{array}$$

In the new versions, the index of the input solution doesn't have to be exactly the same as the input width to the lobits operation. This is still sound because the fill type is correct (same as input solution) and the value in

the low n' bits is required (by precondition) to be the same as the original expression—the fact that the original expression has lost some bits doesn't matter.

(combine solutions 155a) + $\equiv$ (160a) $\triangleleft$ 154 155b $\triangleright$

```
let mksolutions_trunc ws arg_solnss =
  let arg_solns =
    assert (List.length arg_solnss = 1);
    List.filter is_tau (List.hd arg_solnss)
  in
  (* [[e]] just updates the original expression *)
  let e arg = RP.App(("lobits", ws), [arg]) in
  let (n'', n) =
    match Rtl.op mono (Rtl.opr "lobits" ws) with
    | ([Types.Bits n''], Types.Bits n) -> (n'', n)
    | _ -> impossf "lobits operator should have some type bits#w->bits#n"
  in
  let drop_lo_copy arg_soln = { arg_soln with e = e arg_soln.e; m = n } in
  let drop_lo_ignore arg_soln =
    { arg_soln with e = e arg_soln.e; fill = G; m = n; n = n }
  in
  let ok_soln s =
    let n' = s.n in
    n' <= n'' && (n <= n' || subsume_index s n)
  in
  let arg_solns = List.filter ok_soln arg_solns in
  List.map drop_lo_copy arg_solns @ List.map drop_lo_ignore arg_solns
```

The following rule doesn't occur because we don't really have a sxlo or zxlo operator.

$$\frac{\text{EXTLO } (n \leq n' \leq w \leq w') \quad \sigma\text{xlo}_{w' \leftarrow w} \in M \quad \Gamma, M \vdash b \xrightarrow{c_1} b' :: z[n] : w \quad \Gamma, M \vdash e \xrightarrow{c_2} e' :: g[n] : w}{\Gamma, M \vdash \sigma\text{xlo}_{n' \leftarrow n}(b, e) \xrightarrow{c_1 + c_2 + 1} \sigma\text{xlo}_{w' \leftarrow w}(b', e') :: \sigma[n] : w'}$$

(combine solutions 155b) + $\equiv$ (160a) $\triangleleft$ 155a

```
let var_name = function
  | RP.Const(RP.Link(nm,_,_)) -> nm#original_text
  | RP.Const(RP.Late(nm,_))   -> nm
  | RP.Fetch((RP.Var(nm,_,_) | RP.Global(nm,_,_)),_) -> nm
  | _ -> "/" (* impossible name *)

let ast_combine m solns = function
  | RP.Const(RP.Bool b) as e -> mksolutions_lit_bool e
  | RP.Const(RP.Bits b) as e ->
    (mksolutions_lit_bits m Z e)
    @ (mksolutions_lit_bits m S e)
    @ (mksolutions_lit_bits m F e)
    (* @ (mksolutions_lit_bits m.literals H e) *)
  (* claude: upstream matched Mem as a stale 6-field shape; the current
   * Rtl.Private.loc's Mem is 'space * count * exp * assertion' with space
   * itself 'char * aggregation * Cell.t' (see the working match a few
   * hundred lines below at 'Mem((_,_,ms),c,e,_)'). ms is the whole space,
   * mcell just its cell component, needed for Cell.to_width. *)
  | (RP.Fetch(RP.Mem(((_,_,mcell) as ms), c, addr, a),_)) as e ->
    let all = match solns with [s] -> s | _ -> impossf "mem arity" in
    let best = List.filter (fun s -> fill s == Z && s.w = m.pointer) all in
    if null best then
      impossf "couldn't widen address in %s" (Rtlutil.ToString.exp (Up.exp e));
    let best = minimum_cost best in
    let w = Cell.to_width mcell c in
    let create f = { best with
```

```

        e      = e;
        e'     = RP.Fetch(RP.Mem(ms,c,best.e',a),w);
        fill   = f;
        w      = w;
        n      = w } in
    [create G; create F]
  | (RP.Const(RP.Link(_,_,w)|RP.Late(_,w)) | RP.Fetch(_,w)) as e ->
    mksolutions_var (var_name e) e w
  | RP.App(("sx"|"zx" as ext), ws), [arg])    -> mksolutions_ext ext ws solns
(*  | RP.App(("f2f",
            ws), [arg;rm]) ->
    mksolutions_drop_f2f ws solns rm *)
  | RP.App(("lobits",
            ws), [arg])    -> mksolutions_trunc ws solns
  | RP.App((opname,
            ws), [])       -> mksolutions_nullary_op m opname ws
  | RP.App((opname,
            ws), args)     -> mksolutions_op m solns opname ws args
  | _ -> impossf "Widen.ast_combine couldn't handle the rtl"

```

⟨bottom-up transform 156a⟩ ≡ (160a)

```

let rec bottom_up f = function
  | RP.App(_,es) as e -> f (List.map (bottom_up f) es) e
  | RP.Fetch(RP.Mem(_, _, addr, _), _) as e -> f [bottom_up f addr] e
  | e -> f [] e

```

`all_solutions` is where the two phases of the dynamic programming algorithm are applied bottom-up on each AST node.

⟨dynamic programming 156b⟩ ≡ (160a)

```

let all_solutions m =
  let solve_node known e =
    find_reachable m [] (solutions_unique (ast_combine m known e))
  in
  bottom_up solve_node

```

⟨solution utilities 156c⟩ ≡ (160a) 156d▷

```

let cost s = s.cost
let index s = s.n
let width s = s.w
let ast s = s.e
let ast' s = s.e'
let fill s = s.fill

```

```

let string_of_fill = function
  | S -> "S"
  | Z -> "Z"
  | G -> "G"
  | H -> "H"
  | F -> "F"

```

```

let is_sigma s = match fill s with (S | Z) -> true | (G | H | F) -> false
let is_tau s = match fill s with (S | Z | G) -> true | (H | F) -> false

```

⟨solution utilities 156d⟩ +≡ (160a) <156c 156e▷

```

let subst_fill s tau = {s with fill = tau}

```

⟨solution utilities 156e⟩ +≡ (160a) <156d 157a▷

```

let apply_with x f = f x
let map2_find_all prop xss ys = List.map2 (fun x -> List.filter (prop x)) xss ys
let fork f g x = (f x, g x)
let id x = x
let compose f g x = f (g x)

```

```

⟨solution utilities 157a⟩+≡ (160a) <156e 157b>
  let rec cross_list xs ys =
    match xs, ys with
    | [], _ -> []
    | _, [] -> []
    | x::rest, ys -> List.map (fun y -> [x;y]) ys @ cross_list rest ys

  let rec cross_pair xs ys =
    match xs, ys with
    | [], _ -> []
    | _, [] -> []
    | x::rest, ys ->
      List.map (fun y -> (x, y)) ys @ cross_pair rest ys

⟨solution utilities 157b⟩+≡ (160a) <157a 157c>
  let rec cross_pair_filter p xs ys =
    match xs, ys with
    | [], _ -> []
    | _, [] -> []
    | x::rest, ys ->
      List.map (fun y -> (x, y)) (List.filter (p x) ys) @ cross_pair_filter p rest ys

⟨solution utilities 157c⟩+≡ (160a) <157b 157d>
  let cross_list_all xss =
    let nest_each = List.map (fun x -> [x]) in
    let rec combinations current = function
      | [] -> current
      | xs :: rest ->
        let current' = List.map List.concat (cross_list current (nest_each xs)) in
        combinations current' rest
    in
    match xss with
    | [] -> []
    | xs :: rest -> combinations (nest_each xs) rest

⟨solution utilities 157d⟩+≡ (160a) <157c 157e>
  let rec minimum_cost = function
    | [] -> impossf "can't take minimum of empty list"
    | [x] -> x
    | x :: rest -> let m = minimum_cost rest in if cost x < cost m then x else m

  let rec maximum_cost = function
    | [] -> impossf "can't take maximum of empty list"
    | [x] -> x
    | x :: rest -> let m = minimum_cost rest in if cost x > cost m then x else m

  let minimum = function
    | [] -> impossf "empty list"
    | x :: rest -> List.fold_left min x rest

⟨solution utilities 157e⟩+≡ (160a) <157d
  let string_of_soln s =
    Printf.sprintf "%s -> %s :: %s[%i] : %i"
      (RU.ToString.exp (Up.exp s.e))
      (RU.ToString.exp (Up.exp s.e'))
      (string_of_fill s.fill)
    s.n
    s.w

```

<helpers 158a>≡ (160a) 160b▷

```

let ext_of_fill = function
| S -> "sx"
| Z -> "zx"
| _ -> raise Not_found

let fill_of_ext = function
| "sx" -> S
| "zx" -> Z
| ext -> raise Not_found

let strconcatmap x y z = String.concat x (List.map y z)

```

<environments 158b>≡ (160a)

```

let op_fill_types =
[ "add", [[G;G],G;
          [H;H],H]
; "addc", [[G;G;G], G]
; "and", [[S;S],S;
          [Z;G],Z;
          [G;Z],Z;
          [G;G],G;
          [H;H],H]
; "borrow", [[S;S;G], Z;
             [Z;Z;G], Z]
; "carry", [[S;S;G], Z]
; "com", [[S],S;
          [G],G]
; "div", [[S;S],S]
; "divu", [[Z;Z],Z]
; "mod", [[S;S],S]
; "modu", [[Z;Z],Z]
; "mul", [[S;S],S] (* maybe garbage instead? *)
; "mulx", [[S;S],S]
; "mulux", [[Z;Z],Z]
; "neg", [[G],G;
          [H],H]
; "or", [[S;S],S;
         [Z;Z],Z;
         [G;G],G;
         [H;H],H]
; "quot", [[S;S],S]
; "popcnt", [[Z], Z]
; "rem", [[S;S],S]
(* rotl, rotr must be rewritten *)
; "shl", [[G;Z],G]
; "shra", [[S;Z],S]
; "shrl", [[Z;Z],Z]
; "sub", [[G;G],G;
          [H;H],H]
; "subb", [[G;G;G], G]
; "xor", [[G;G],G
          ; [S;S],S
          ; [Z;Z],Z
          ; [H;H],H]
; "eq", [[S;S],Z;
         [Z;Z],Z;
         [H;H],Z]
; "ge", [[S;S],S;
         [H;H],Z]

```

```

; "geu",    [[S;S],Z;
              [Z;Z],Z;
              [H;H],Z]
; "gt",     [[S;S],Z;
              [H;H],Z]
; "gtu",    [[S;S],Z;
              [Z;Z],Z;
              [H;H],Z]
; "le",     [[S;S],Z;
              [H;H],Z]
; "leu",    [[S;S],Z;
              [Z;Z],Z;
              [H;H],Z]
; "lt",     [[S;S],Z;
              [H;H],Z]
; "ltu",    [[S;S],Z;
              [Z;Z],Z;
              [H;H],Z]
; "ne",     [[S;S],Z;
              [Z;Z],Z;
              [H;H],Z]

; "fabs", [[F],F]
; "fadd", [[F;F;Z],F]
; "fdiv", [[F;F;Z],F]
; "feq", [[F;F],Z]
; "fge", [[F;F],Z]
; "fgt", [[F;F],Z]
; "fle", [[F;F],Z]
; "flt", [[F;F],Z]
; "fne", [[F;F],Z]
; "fordered", [[F;F],Z]
; "fmul", [[F;F;Z],F]
; "fneg", [[F],F]
; "funordered", [[F;F],Z]
; "fsqrt", [[F;Z],F]
; "fsub", [[F;F;Z],F]
; "f2f", [[F;Z],F]
; "i2f", [[S;Z],F]
; "f2i", [[F;Z],S]
; "minf", [[],F]
; "pinf", [[],F]
; "mzero", [[],F]
; "pzero", [[],F]
; "round_up",    [[],Z]
; "round_down",  [[],Z]
; "round_zero",  [[],Z]
; "round_nearest", [[],Z]
(*)
; "NaN",
; "add_overflows",
; "div_overflows",
; "mul_overflows",
; "mulu_overflows",
; "quot_overflows",
; "sub_overflows",
*)
; "not",    [[Z],Z]
; "bool",   [[Z],Z]
; "disjoin", [[Z;Z],Z]

```



```

; "conjoin", [[Z;Z],Z]
; "bit",      [[Z],G]
]

```

```
let machine_has m opname ws = List.mem (opname,ws) m.ops
```

```
let machine_widths m opname =
  let all = List.filter (fun (opname',_) -> opname' = opname) m.ops in
  List.map (fun (opname,ws) -> ws) all
```

```
let find_exts_from m w extname =
  let ext_widths = machine_widths m extname in
  List.filter (function (w1::_) -> w1 = w
    | _ -> impossf "ill formed machine env") ext_widths
```

```
let find_exts_to m w extname =
  let ext_widths = machine_widths m extname in
  List.filter (function [_;w2] -> w2 = w
    | _ -> impossf "ill formed machine env") ext_widths
```

```

<widen.ml 160a>+≡ (138a) <142b 160c>
  <Widen types 142c>
  <solution utilities 156c>
  <helpers 158a>
  <environments 158b>
  <find reachable solutions 143c>
  <combine solutions 150b>
  <bottom-up transform 156a>
  <dynamic programming 156b>

```

To know if a global variable is a hardware register, we have to look it up in the map associated with the procedure. We are careful to avoid infinite recursion for the case in which a global variable is mapped to itself, as is done by the interpreter back end, for example.

The general idea is that if a back end allows something to get into a hardware register, the expander will assume the obligation of extending it if needed.

```

<helpers 160b>+≡ (160a) <158a
  let is_hardware_register proc loc =
    match loc with
    | RP.Global (_, i, w) ->
      (match Dn.exp (proc.Procedure.global_map.(i).Automaton.fetch w) with
      | RP.Fetch (l, _) -> RU.is_hardware l
      | _ -> false)
    | _ -> RU.is_hardware loc

```

```

<widen.ml 160c>+≡ (138a) <160a 161>
  let needs_widening t r =
    let targetwidth = t.Target.wordsize in
    let rec loc = function
      | RP.Mem((_, _, ms),c,e,_) -> Cell.to_width ms c < targetwidth || exp e
      | RP.Reg((_,_,ms),_,c) -> Cell.to_width ms c < targetwidth
      | RP.Var(.,_,w) -> w < targetwidth
      | RP.Global(.,_,w) -> w < targetwidth
      | RP.Slice(w,_,l) -> w < targetwidth || loc l
    and exp = function
      | RP.Fetch(l, w) -> w < targetwidth || loc l
      | RP.App(., ws), es ->
        List.exists (fun w -> w < targetwidth) ws || List.exists exp es
      | RP.Const c -> const c
    and const = function
      | RP.Link(.,_,w) -> w < targetwidth

```

```

| RP.Diff(x,y)    -> const x || const y
| RP.Late(_,w)    -> w < targetwidth
| RP.Bits b       -> Bits.width b < targetwidth
| RP.Bool _       -> false
in
let guard (g, eff) =
  let eff' =
    match eff with
    | RP.Store(l, r, w) -> w < targetwidth || loc l || exp r
    | RP.Kill l -> loc l
  in
  eff' || exp g
in
let RP.Rtl es = Dn.rtl r in
List.fold_left (fun b e -> b || guard e) false es

```

I'll choose in the `widen_guarded` code to find a solution of type `Z`, just to match what some C- frontends seem to want to do...not that it matters. Calling `widen_loc` never changes the width of the location—it simply fixes up any code in an addressing expression that might need widening.

```

(widen.ml 161)+≡ (138a) <160c 162>
let dpwiden ((g,proc) : Ast2ir.proc) r =
  let PA.T t      = proc.target in
  let targetwidth = t.Target.wordsize in
  let ptrwidth    = t.Target.pointersize in
  let rm          =
    let fetch t = R.fetch t (R.locwidth t) in
    Dn.exp (fetch t.Target.rounding_mode) in
  let ops          = List.map Dn.opr t.Target.capabilities.Target.operators in
  let litwidths    = t.Target.capabilities.Target.literals in
  let m = {ops=ops; literals=litwidths; pointer=ptrwidth; word=targetwidth;
    itemps=t.Target.capabilities.Target.itemps;
    ftemps=t.Target.capabilities.Target.ftemps;
    rm=rm;
    is_hardware_register=is_hardware_register proc} in

  let find_best_soln' r filter =
    let construct r =
      let solns = all_solutions m r in
      List.filter filter solns in
    let solns = construct r in
    let solns =
      if null solns then construct (Dn.exp (Simplify.exp (Up.exp r)))
      else solns in
    let () = if null solns then
      impossf "dpwiden couldn't widen %s" (Rtlutil.ToString.exp (Up.exp r))
    in
    let soln = minimum_cost solns in
    if print_debug then Printf.eprintf "cost: %i\n" soln.cost;
    soln.e'
  in

  let find_best_soln_index r tau n w =
    find_best_soln' r (fun s -> index s = n && width s = w && fill s == tau) in
  let find_best_soln r tau w =
    find_best_soln' r (fun s -> width s = w && fill s == tau) in

  let rec widen_loc = function
    | RP.Mem(sp,w,e,a) -> RP.Mem(sp,w,find_best_soln e Z ptrwidth, a)
    | RP.Slice(w,i,l)  -> RP.Slice(w,i, widen_loc l)
    | l                -> l in

```

```

(*)
let find_best_soln_index r tau n w =
  let solns = all_solutions m r in
  let solns = List.filter (fun s -> index s = n && width s = w && fill s == tau) solns in
  let () = if null solns then
    (* try simplifier and run again *)
    impossf "dpwiden couldn't widen %s" (Rtlutil.ToString.exp (Up.exp r)) in
  let soln = minimum_cost solns in
  if print_debug then Printf.eprintf "cost: %i\n" soln.cost;
  soln.e' in
let find_best_soln r tau w =
  let construct r =
    let solns = all_solutions m r in
    let solns = List.filter (fun s -> width s = w && fill s == tau) solns in
    solns
  in
  let solns = construct r in
  let solns =
    if null solns then (* try the simplifier and run again *)
      construct (simplify r)
    else solns
  in
  let () = if null solns then
    impossf "dpwiden couldn't widen %s" (Rtlutil.ToString.exp (Up.exp r)) in
  let soln = minimum_cost solns in
  if print_debug then Printf.eprintf "cost: %i\n" soln.cost;
  soln.e' in
*)

```

$$\begin{array}{c}
\text{ASSIGN } (n \leq w \leq w', x_w \text{ IS A HARDWARE REG}) \\
\frac{\Gamma(x_w) = \tau[n] \quad \Gamma, M \vdash e \xrightarrow{c}_c e' :: \tau[n] : w'}{\Gamma, M \vdash x_w :=_w \text{lo}_{w \leftarrow w'} e \xrightarrow{c}_c x_w :=_w \text{lo}_{w \leftarrow w'} e'} \\
\text{ASSIGN } (n \leq w) \\
\frac{\Gamma(x_w) = \tau[n] \quad \Gamma, M \vdash e \xrightarrow{c}_c e' :: \tau[n] : w}{\Gamma, M \vdash x_w :=_w e \xrightarrow{c}_c x_w :=_w e'}
\end{array}$$

Note that `widen_guarded` doesn't seem to be checking the index of the result!

```

⟨widen.ml 162⟩+≡ (138a) <161 163>
let widen_guarded (g, eff) =
  try
    let effsoln =
      match eff with
      | RP.Store(l, (RP.App(op, args) as e), w)
      when safe_has_floating_result (Up.opr op) ->
        if Debug.on "widen" then
          Printf.eprintf " looking for %s -> ... :: F : %i\n"
            (RU.ToString.exp (Up.exp e)) w;
          RP.Store(l, find_best_soln e F w, w)
      | RP.Store(l, RP.App(("lobits", [w';_]), [e]), w)
      when is_hardware_register proc l && w <= w' ->
        let (fill, _) = lookup_gamma (var_name (RP.Fetch(l,w))) w in
        if Debug.on "widen" then
          Printf.eprintf " looking for %s -> ... :: %s[_] : %i\n"
            (RU.ToString.exp (Up.exp e)) (string_of_fill fill) w';
          RP.Store(l, RP.App(("lobits", [w';w]), [find_best_soln e fill w']), w)
      | RP.Store(l, r, w) ->
        let (fill, n) = lookup_gamma (var_name (RP.Fetch(l,w))) w in

```

```

    if Debug.on "widen" then
        Printf.eprintf " looking for %s -> ... :: %s[%i] : %i\n"
            (RU.ToString.exp (Up.exp r)) (string_of_fill fill) n w;
        RP.Store(widen_loc 1, find_best_soln_index r fill n w, w)
    | RP.Kill 1          -> RP.Kill (widen_loc 1)
in
let gsoln = find_best_soln g Z 1 in
(gsoln, effsoln)
with Abort_unknown -> (g, eff)
in
let rtl = Rtlutil.ToString.rtl in
let () = if Debug.on "widen-pre" then
    (Printf.eprintf "DP widening %s\n" (rtl r); flush stderr) in
let r' =
    let RP.Rtl es = Dn.rtl r in
    Up.rtl (RP.Rtl (List.map widen_guarded es)) in
if Debug.on "widen" && not (rtl r' = $ = rtl r) then
    (Printf.eprintf "'->      %s\n" (rtl r')); flush stderr;
r'

```

We don't touch registers because actual hardware registers should already be at the right width—otherwise there is something wrong with the frontend. We also don't touch memory references or slices. I'm not sure what the right way to deal with globals is—maybe they should be widened and maybe not. So for now, we only handle the “Var” case. For now, we always do zero extension when extension is necessary. I choose to do `zx` instead of `sx` here to be consistent with choices that seem to be made by frontends for QC-. (So I don't start failing tests).

```

(widen.ml 163)+≡ (138a) <162 164a>
let widenlocs t r =
    let targetwidth = t.Target.wordsize in
    let tostr = Rtlutil.ToString.rtl in
    let lo n e = RP.App(("lobits", [targetwidth; n]), [e]) in
    let rec widen_loc = function
        | RP.Mem(sp, w, e, a) -> RP.Mem(sp, w, widen_exp e, a)
        | RP.Slice(w, i, 1)   -> RP.Slice(w, i, widen_loc 1)
        | 1                   -> 1
    and widen_exp = function
        | (RP.Fetch(RP.Var(name, i, vw), fw)) as e ->
            if vw <> fw then
                impossf "Fetch and var widths are not the same in Fetch(%s::bits%i, %i)"
                    name vw fw
            else if vw < targetwidth
            then lo vw (RP.Fetch(RP.Var(name, i, targetwidth), targetwidth))
            else e
        | RP.Fetch(l, w) -> RP.Fetch(widen_loc l, w)
        | RP.App(opr, es) -> RP.App(opr, List.map widen_exp es)
        | e -> e
in
let widen_guarded (g, eff) =
    let eff' =
        match eff with
        | RP.Store(RP.Var(name, i, vw), r, sw) ->
            if vw <> sw then
                impossf "Store and var widths are not the same in %s::bits%i :=_%i %s"
                    name vw sw (RU.ToString.exp (Up.exp r))
            else if vw < targetwidth
            then
                let ext =
                    try ext_of_fill (fst (lookup_gamma name vw)) with Not_found -> "zx"
                in

```

```

        RP.Store(RP.Var(name, i, targetwidth),
                RP.App((ext, [vw; targetwidth]), [widen_exp r]),
                targetwidth)
    else RP.Store(RP.Var(name, i, vw), widen_exp r, sw)
  | RP.Store(l, r, w) -> RP.Store(widen_loc l, widen_exp r, w)
  | RP.Kill l          -> RP.Kill (widen_loc l)
in
(widen_exp g, eff') in
let r' =
  let RP.Rtl es = Dn.rtl r in
  Up.rtl (RP.Rtl (List.map widen_guarded es)) in
let () =
  if Debug.on "widen-vars" && not (RU.Eq.rtl (Dn.rtl r') (Dn.rtl r)) then
    (Printf.eprintf "rewritten to %s\n" (tostr r'); flush stderr) in
r'

```

Needs work: this function should only count those variables that need widening. NOTE: REMOVING THE H TYPE VOTING—SEE ANDREW WILKINS BUG FROM THE C- LIST.

```

⟨widen.ml 164a⟩+≡ (138a) <163 164b>
let update_gamma_counts t r =
  let add c i n = function
    | G -> c
    | Z -> {c with z=c.z+i; ni = n}
    | S -> {c with s=c.s+i; ni = n}
    | H -> c (* {c with h=c.h+i; ni = n} *)
    | F -> c in
  let update h x n t =
    if Hashtbl.mem h x then Hashtbl.replace h x (add (Hashtbl.find h x) 1 n t)
    else Hashtbl.add h x (add {z=0; s=0; h=0; ni=n} 1 n t) in
  let targetwidth = t.Target.wordsize in
  let rec loc fty = function
    | (RP.Var(s,_,w)) ->
      if w < targetwidth then
        (if Debug.on "gamma" then
          Printf.eprintf "Updating count of %s: +%s\n" s (string_of_fill fty);
          update gamma_counts s w fty)
    | RP.Mem(_,_,e,_) -> ignore(exp fty e)
    | RP.Slice(_,_,l) -> loc fty l
    | _ -> ()
  and exp fty = function
    | RP.Fetch(l, w) -> loc fty l; [fty]
    | RP.App((opname, ws), es) ->
      (try
        let ftys = List.assoc opname op_fill_types in
        ignore(List.map (fun (arg_ftys, _) -> List.map2 exp arg_ftys es) ftys);
        List.map snd ftys
      with _ -> [G])
    | _ -> [G] in
  let guard (g, eff) =
    (match eff with
    | RP.Store(l, r, w) -> List.iter (fun fty -> loc fty l) (exp G r)
    | RP.Kill l -> ());
    ignore(exp Z g) in
  let RP.Rtl es = Dn.rtl r in
  List.iter guard es

```

```

⟨widen.ml 164b⟩+≡ (138a) <164a 165a>
let init_gamma_counts () = Hashtbl.clear gamma_counts

let create_gamma () =

```

```

let argmax z s h =
  if z > h then
    if z > s then Z
    else S
  else if h > s then H
  else S
in
Hashtbl.clear gamma;
Hashtbl.iter (fun s c -> Hashtbl.add gamma s (argmax c.z c.s c.h, c.ni)) gamma_counts;
if Debug.on "gamma" then
  (Printf.eprintf "Printing gamma:\n";
   Hashtbl.iter (fun s (t,n) -> Printf.eprintf "Gamma |- %s :: %s[%i]\n" s (string_of_fill t) n) gamma)

```

13.1.3 Counting operations and testing

```

⟨widen.ml 165a⟩+≡ (138a) <164b 165b>
let width_cost r =
  let rec count_loc c = function
    | RP.Mem(_,_,e,_) -> do_count c e
    | RP.Slice(_,_,l) -> count_loc c l
    | _ -> impossf "count_loc can only handle mem and slice"
  and do_count ((sx, zx, lo) as c) = function
    | RP.Const _ -> c
    | RP.Fetch(l, w) -> c
  (* considering sx(const) the same as const *)
  | RP.App (("sx"|"zx"), _, [(RP.Const _ | RP.Fetch _)]) -> c
  | RP.App ("sx", _, [arg]) -> do_count (sx+1, zx, lo) arg
  | RP.App ("zx", _, [arg]) -> do_count (sx, zx+1, lo) arg
  | RP.App ("lobits", _, [RP.Const(RP.Bits _)]) -> c
  | RP.App ("lobits", _, [arg]) -> do_count (sx, zx, lo+1) arg
  | RP.App ((opname, _), args) -> List.fold_left do_count c args
in
let do_count_guard counts (g, eff) =
  let counts' =
    match eff with
    | RP.Store(l, r, w) -> do_count counts r
    | RP.Kill l -> counts
  in
  do_count counts' g
in
let rtl = Rtlutil.ToString.rtl in
let _ = if print_debug then (Printf.eprintf "Counting %s\n" (rtl r); flush stderr) in
let (sx, zx, lo) =
  let RP.Rtl es = Dn.rtl r in
  List.fold_left do_count_guard (0, 0, 0) es
in
Printf.eprintf "Counted %i sx, %i zx, %i lo\n" sx zx lo;
(sx, zx, lo)

```

```

⟨widen.ml 165b⟩+≡ (138a) <165a
let app_count r =
  let rec count_loc c = function
    | RP.Mem(_,_,e,_) -> do_count c e
    | RP.Slice(_,_,l) -> count_loc c l
    | _ -> impossf "count_loc can only handle mem and slice"
  and do_count c = function
    | RP.Const _ -> c
    | RP.Fetch(l, w) -> c

```

```

| RP.App (((("sx"|"zx"), _), [(RP.Const _ | RP.Fetch _)])) -> c
| RP.App (("lobits", _), [RP.Const(RP.Bits _)]) -> c

| RP.App (_, args) -> 1 + List.fold_left do_count c args
in
let do_count_guard c (g, eff) =
  let c' =
    match eff with
    | RP.Store(l, r, w) -> do_count c r
    | RP.Kill l -> c
  in
  do_count c' g
in
let rtl = Rtlutil.ToString.rtl in
let _ = if print_debug then (Printf.eprintf "Counting %s\n" (rtl r); flush stderr) in
let count =
  let RP.Rtl es = Dn.rtl r in
  List.fold_left do_count_guard 0 es
in
Printf.eprintf "Counted %i ops\n" count;
count

```

Chapter 14

ARM target

Chapter 15

Runtime

15.1 C-- Run-time Interface

This is essentially the interface in the specification, except we don't provide access to global variables.

```
<qc--runtime.h 168a>≡
  #ifndef QCMM_RUNTIME_H
  #define QCMM_RUNTIME_H

  #include <stdint.h>

  <machine-dependent macro definitions for the public interface 170>

  <data structures 168b>
  <exposed private data structures 171e>
  <public functions 169a>
  <exposed private functions 180e>
  #endif /* QCMM_RUNTIME_H */
```

The definitions of the basic C-- types are public.

```
<data structures 168b>≡ (168a) 168c>
/* claude: qc-- emits every pc_map_entry field (locations, register/local/
 * span counts, the location-encoded offsets themselves) at the target's
 * native pointer width, not a fixed 32 bits - confirmed by hand from a
 * -riscv64 .s dump, where inalloc/outalloc/return_addressp/num_registers/
 * etc. are all ".dword" (8 bytes), and MKOFFSET(0) shows up as
 * -9223372036854775808 (bit 63 set), not bit 31. uintptr_t equals plain
 * "unsigned" on every existing 32-bit backend, so this is a no-op there.
 * pemap.c's Cmm_loctype/Cmm_as_register/Cmm_as_offset decode this
 * generically off sizeof() of their own local variable, so they need no
 * further change beyond using a same-width signed type (see that file).
 * This is a different table from fork-tiger's own "bits32[] {...}" GC
 * descriptor tables (frontend/frame.ml's output_footer there is a
 * deliberate, separate choice to stay 32-bit regardless of target) -
 * those are addressed *through* a span value (itself pointer-width, as
 * elab/nelab.ml's own span-value check requires), not part of this
 * struct, so they are unaffected by this typedef. */
typedef uintptr_t Cmm_Word;
typedef void* Cmm_Dataptr;
typedef void (*Cmm_Codeptr)();
```

The layouts of continuations and activations are considered private.

```
<data structures 168c>+≡ (168a) <168b
typedef struct cmm_cont Cmm_Cont;
typedef struct cmm_activation Cmm_Activation;
```

Because we wish for a client to be able to allocate an activation on the stack as a local variable, the representation of an activation is actually exposed below. It is nevertheless an *unchecked* run-time error to depend in any way on the definition of `struct cmm_activation`. Similarly, accidents of implementation require us to expose `struct cmm_cont`, but again, it is an *unchecked* run-time error to depend in any way on its definition.

We provide a function to help debug the run-time system.

```
<public functions 169a>≡ (168a) 169b▷
extern void Cmm_show_activation(const Cmm_Activation *a); /* for debugging */
```

Functions for walking the stack.

```
<public functions 169b>+≡ (168a) <169a 169c▷
Cmm_Activation Cmm_YoungestActivation (const Cmm_Cont *t);
int Cmm_isOldestActivation (const Cmm_Activation *a);
Cmm_Activation Cmm_NextActivation (const Cmm_Activation *a);
int Cmm_ChangeActivation (Cmm_Activation *a);
Cmm_Cont* Cmm_MakeUnwindCont (Cmm_Activation *a, Cmm_Word index, ...);
```

```
<public functions 169c>+≡ (168a) <169b 169d▷
Cmm_Dataptr Cmm_GetDescriptor(const Cmm_Activation *a, Cmm_Word token);
```

Functions for finding local variables.

```
<public functions 169d>+≡ (168a) <169c 169e▷
void Cmm_LocalVarWritten (const Cmm_Activation *a, unsigned n);
unsigned Cmm_LocalVarCount (const Cmm_Activation *a);
void* Cmm_FindLocalVar (const Cmm_Activation *a, unsigned n);
void* Cmm_FindDeadLocalVar (const Cmm_Activation *a, unsigned n);
void* Cmm_FindStackLabel (const Cmm_Activation *a, unsigned n);
```

Convenience.

```
<public functions 169e>+≡ (168a) <169d 169f▷
void Cmm_CutTo (const Cmm_Cont *k);
```

Thread creation.

```
<public functions 169f>+≡ (168a) <169e
Cmm_Cont* Cmm_CreateThread (Cmm_Codeptr f, Cmm_Dataptr x, void *s, unsigned n);
void Cmm_Yield (Cmm_Cont *k, Cmm_Cont *kold);
```

These functions are currently unimplemented.

```
<unimplemented public functions 169g>≡
void* Cmm_FindGlobalVar (int n);
```

15.2 Implementation

First, the machine-dependent register information.

```
<machine-dependent macro definitions for the public interface ((x86-linux)) 169h>≡ (170)
#define NUM_REGS 12
```

```
<machine-dependent macro definitions for the implementation ((x86-linux)) 169i>≡ (171a)
#define ESP 8 /* r[4] */
#define PC 0 /* c[0] */
```

```

(machine-dependent macro definitions for the public interface 170)≡ (168a)
/* claude: NUM_REGS/FP_REG must match the TARGET's own flat register-index
 * numbering (target/target.ml's mk_reg_ix_map: every Reg/Fixed-classified
 * space, sorted by space character, concatenated - see each arch's own
 * Spaces module for its per-space counts/order). Cmm_Activation.regs[] is
 * a fixed-size array with no bounds check in update_saved_regs, so if
 * NUM_REGS is too small for a given target, writes past the end silently
 * corrupt whatever follows it on the stack - this is what "ppc/sparc/
 * riscv/alpha were all silently running with x86's NUM_REGS=12" actually
 * meant each time it was found (see git log). A platform this chain
 * doesn't recognize gets a #error instead of silently reusing x86's
 * values, which would reintroduce exactly that bug for whatever's next. */
#ifdef __powerpc__
#define NUM_REGS 96 /* c(7)->0-7, f(32)->8-40, r(32)->41-73 - confirmed via gdb */
#elif defined(__sparc__)
#define NUM_REGS 128 /* r/f/c/k, ~49 indices - generous, not pinned down exactly */
#elif defined(__riscv)
#define NUM_REGS 96 /* c(6)->0-5, f(32)->6-37, r(32)->38-69 = 70; both RV32/RV64 */
#elif defined(__alpha__)
#define NUM_REGS 96 /* c(6)->0-5, f(32)->6-37, r(32)->38-69 = 70, same shape as riscv's */
#elif defined(__mips__)
#define NUM_REGS 96 /* c(6)->0-5, f(32)->6-37, r(32)->38-69 = 70, same shape as riscv/alpha */
#elif defined(__arm__)
#define NUM_REGS 32 /* c(3)->0-2, r(16)->3-18 = 19; no float space (arm.ml: no FPU) */
#elif defined(__m68k__)
/* claude: verified via a throwaway OCaml harness calling the real
 * Target.mk_reg_ix_map on m68k.ml's actual T.spaces (same method as
 * __aarch64__/_x86_64__ below, not hand-derived - see those comments for
 * why hand-deriving misses the off-by-one). Confirmed result: c(3)->0-3 (4
 * slots), r(11)->4-15 (12 slots, the same count+1 off-by-one every other
 * target's r space hits) = 16 total. m68k.ml folds d0-d7/a0(scratch)/a6(fp)/
 * a7(sp) into this one 'r' space rather than a genuine split D/A-register
 * space - see arch/m68k/m68kregs.ml's own header comment - so there is no
 * separate float/address-register range the way ppc/riscv/alpha/mips have
 * one; no float space either (m68k.ml: no FPU). */
#define NUM_REGS 32 /* generous margin over the confirmed total=16, same margin style as arm's own 32 */
#elif defined(__aarch64__)
/* claude: verified via a throwaway OCaml harness calling the real
 * Target.mk_reg_ix_map on arm64.ml's actual T.spaces (not hand-derived by
 * inspecting the SS.c/SS.r/SS.f 'count' arguments the way the other
 * targets' comments above were - that misses an off-by-one:
 * Target.mk_reg_ix_map's space_to_regs generates indices 0..count
 * INCLUSIVE, i.e. count+1 registers per space, not count. Confirmed
 * result: x0->37, x28->65, x29(fp)->66, x30(lr)->67, sp(x31)->68, total=70 -
 * c(3)->0-3 (4 slots, no npc/fp_mode/fp_fcmp: arm64regs.ml declares SS.c 3
 * like arm.ml, not riscv64regs.ml's SS.c 6), f(32)->4-36 (33 slots),
 * r(32)->37-69 (33 slots) = 70 total. */
#define NUM_REGS 96 /* generous margin over the confirmed total=70, same margin style as riscv64/alpha/mips's o
#elif defined(__x86_64__)
/* claude: NOT the same branch as __i386__ below - amd64.ml's T.spaces
 * (Space.Standard64-based: c(3), r(16), no float space) is a completely
 * different register file from x86.ml's 32-bit one (Space.Standard32-based:
 * c(3), r(7)), so i386's NUM_REGS=12/FP_REG=9 do not apply here at all -
 * verified via the same throwaway-OCaml-harness approach as __aarch64__
 * above (calling the real Target.mk_reg_ix_map on Amd64.target's actual
 * T.spaces). Confirmed result: rax->4, rbp->9, r15->19, total=21 - c(3)->0-3
 * (4 slots, same shape as every other target's c space), r(16)->4-20 (17
 * slots, the same count+1 off-by-one every other target's r space hits). */
#define NUM_REGS 96 /* generous margin over the confirmed total=21, same margin style as every other 64-bit tar
#elif defined(__i386__)

```

```

<machine-dependent macro definitions for the public interface ((x86-linux)) 169h>
#else
#error "NUM_REGS/FP_REG not defined for this platform - see qc--runtime.h/gcc-linux.c"
#endif

<machine-dependent macro definitions for the implementation 171a>≡ (182c)
<machine-dependent macro definitions for the implementation ((x86-linux)) 169i>

<machine-dependent macro definitions for the public interface ((x86-cygwin)) 171b>≡
#define NUM_REGS 12

<machine-dependent macro definitions for the implementation ((x86-cygwin)) 171c>≡
#define ESP 8 /* r[4] */
#define PC 0 /* c[0] */

```

15.2.1 Walking the stack

This implementation assumes that a C-- thread can only be suspended at a continuation. Particularly, this means we do not expect that the runtime system will be entered in response to an interrupt or some other method that preempts the C-- thread at an arbitrary point. (At some future time, it may be worthwhile for the run-time system to be able to capture a Unix signal context and synthesize a suitable C-- continuation.)

Walking the stack always begins at a continuation. A C-- continuation contains the program counter and stack pointer at the point of return to the continuation. From this an initial virtual frame pointer (vfp) can be derived. The process of walking the stack involves computing the vfp of each caller, and tracking the locations of all saved registers along the way.

In order to compute the virtual frame pointer for an activation, the runtime system needs to know how it relates to the stack pointer at each call site and continuation in the C-- program. In addition, the run-time system needs to know the locations of caller- and callee-saved registers. Happily, for C-- functions, all of this information is emitted by the C-- compiler as part of the run-time data. For non-C-- functions, including C functions, we must rely on the target-specific ABI to skip over unknown areas of the stack.¹

Stack-walking data structures and internal interfaces

We dispatch between generic C-- code and target-specific code using function pointers. At present, there is only one function pointer, which implements `ChangeActivation`. At a future point, to make things more efficient, we may add an implementation of `IsOldestActivation`.

```

<definitions of private data structures 171d>≡ (182c)
struct cmm_activation_methods {
    int (*change)(Cmm_Activation *a);
};

```

The activation contains a pointer to the method suite, a virtual frame pointer, and a pointer to the run-time system's data. It also contains a pointer to each machine register; if the value of a register is not known, the corresponding pointer is `NULL`.

```

<exposed private data structures 171e>≡ (168a)
<definition of struct cmm_cont 182b>
struct cmm_activation {
    struct cmm_activation_methods *methods;
    char *vfp; /* declared char* so that arithmetic works on addresses */
    struct cmm_pc_map_entry *rtdata;
    Cmm_Word* regs[NUM_REGS];
    <other fields of an activation 172a>
};

```

¹If the ABI does not specify the locations in which registers are saved, we make pessimistic assumptions.

In addition to the fields above, an activation also holds a program counter `pc` (used only for debugging), plus space for specialized run-time system functions.

- For an activation built from a continuation, the `pc` is the program counter of that continuation. For an activation suspended at a call site, it is the normal return address.
- The space is stored in a union `u`; the `unwind` field is used by the interface function `MakeUnwindContinuation`, and the `cwalk` field contains a counter that may be useful to some target-specific stack walkers.

```

⟨other fields of an activation 172a⟩≡ (171e)
Cmm_Codeptr pc; /* used only for debugging */
union { /* following fields are temporary space to avoid allocation */
    struct {
        Cmm_Cont k;
    } unwind;
    struct {
        unsigned tries;
    } cwalk;
} u;

```

The target-specific stack-walking code must export these two functions:

```

⟨external functions 172b⟩≡ (182c) 172d▷
extern int Cmm_c_change_activation(Cmm_Activation *a);
extern void Cmm_init_c_frame(Cmm_Activation *a, char *young_in_overflow);

```

The function `Cmm_c_change_activation` is the change method (i.e., it implements `ChangeActivation`) for an activation suspended at a call site in a C function. The function `Cmm_init_c_frame` is used to initialize a frame for such an activation. For such a frame, the `regs` field is as accurate as we can make it, and the `pc` field is initialized to the return address. The `methods` and `rtdata` fields are initialized appropriately. The pointer `young_in_overflow` points to the young end of the callee's incoming overflow parameters. N.B. in an activation representing a target-specific frame, the meaning of the `vfp` field is target-dependent.

The implementations of `Cmm_c_change_activation` and `Cmm_init_c_frame` live in target-specific files—one for each compiler-machine combination that we support. The rest of the stack-walking code lives here, in this file.

Here are the methods that are used by the stack walker. A pointer to the C-- method suite is exported to target-specific code.

```

⟨definitions of method suites 172c⟩≡ (182c)
static int change_nothing(Cmm_Activation *a) { return 0; }
static struct cmm_activation_methods cmm_methods = { normal_change };
static struct cmm_activation_methods c_methods = { Cmm_c_change_activation };
static struct cmm_activation_methods oldest_methods = { change_nothing };
struct cmm_activation_methods *Cmm_cmm_frame_methods = &cmm_methods;

```

Both generic and target-specific code need to be able to identify a thread-start frame, because it does not actually appear on the (virtual) stack.

```

⟨external functions 172d⟩+≡ (182c) <172b 180c▷
int Cmm_is_thread_start_frame(struct cmm_pc_map_entry *rtdata, Cmm_Codeptr pc);

```

Initializing and walking a stack

The run-time system can begin walking a call stack only at a C-- continuation. The youngest activation is constructed by looking up the run-time data associated with the given continuation. The initial activation record is returned by value, which forces the caller to allocate space for it.

Almost every continuation can be dealt with simply by looking up the appropriate entry in the PC map. The exception is a continuation that represents a freshly created thread. Such a continuation points to

`Cmm_start_thread_helper`, which is a procedure, not a call site or a C-- continuation, and therefore is *not* in the PC map. We treat that case specially, as required by the C-- specification. There are two bits of magic: methods that treat it as the oldest activation, and a special, hand-written PC-map entry that makes the two parameters visible.

⟨implementations of public functions 173a⟩≡ (182c) 176c▷

```
Cmm_Activation Cmm_YoungestActivation(const Cmm_Cont* k) {
    int i;
    Cmm_Activation a;

    if (k->pc == Cmm_start_thread_helper) {
        a.methods = &oldest_methods;
        a.vfp      = (char *)k->sp;
        a.pc       = k->pc;
        if (Cmm_stack_growth > 0)
            a.rtdata = Cmm_thread_start_up_pcmap_entry;
        else
            a.rtdata = Cmm_thread_start_dn_pcmap_entry;
    } else {
        pc_map_entry* entry = Cmm_lookup_entry(k->pc);
        assert(entry);
        a.methods = &cmm_methods;
        a.vfp      = (char *)k->sp - Cmm_as_offset(entry->outalloc);
        a.pc       = k->pc;
        a.rtdata   = entry;
    }
    for (i = 0; i < NUM_REGS; i++)
        a.regs[i] = NULL;
    return a;
}
```

The function for walking past a C-- frame is `normal_change`.

⟨declarations of private functions 173b⟩≡ (182c) 176a▷

```
static int normal_change(Cmm_Activation *a);
```

The reasoning required to get from one `vfp` to another is slightly involved. The key idea is that in the presence of tail calls, the only thing we can count on is the location of the *deallocation point*. This point marks the location on the stack below which it is the callee's responsibility to deallocate, whether on a return or on a tail call. The PC-map entry for a call site stores the incoming deallocation point of its procedure, as well as the outgoing deallocation point of the call site. The caller's outgoing point is the callee's incoming point, and by solving the equation, we can compute the caller's `vfp` from the callee's `vfp`.

⟨private functions 173c⟩≡ (182c) 176b▷

```
static int normal_change(Cmm_Activation *a) {
    char *vfp;
    pc_map_entry* calleedata, *callerdata;
    intptr_t calleedata_inalloc;

    assert(a && a->vfp);
    calleedata = a->rtdata;

    vfp = a->vfp;
    a->pc = *(Cmm_Codeptr*)locptr(a, calleedata->return_addressp);
    /* claude: real SPARC "call" hardware stores the call instruction's own
     * address (not +4/+8) into the return register - the same fact
     * Post.return already compensates for via its "jmpl %i7+8" epilogue
     * (see arch/sparc/sparc.ml's 'return' comment). But return_addressp
     * hands back that same raw, unadjusted value, and this is a SEPARATE
     * consumer: Cmm_lookup_entry below needs the actual resumption pc
     * (call+8, skipping the call and its delay-slot nop) to match the
     * pcmap table's own keys, which are built from the real continuation
```

```

* label placed after both instructions. Without this, every SPARC
* frame-walk (tig_gc's root scan, exception unwinding) looked up the
* wrong pc, got NULL back, and silently treated a live C-- frame as
* the C/host boundary - skipping its GC roots entirely. Confirmed via
* gdb: a real qsort.tig run had callee return_addressp = 0x1d69c (the
* call instruction itself) while the pcmap table's real entry for
* that call site's continuation was 0x1d6a4 = 0x1d69c + 8. */
#ifdef __sparc__
    a->pc = (Cmm_Codeptr)((char *)a->pc + 8);
#endif
/* claude: a MIPS-specific mismatch in the opposite direction from
* sparc's above: here it's the RUNTIME value that's correct and the
* COMPILER's own pcmap-table key that's wrong. arch/mips's real "jal"
* hardware sets $31 (ra) to callsite+8 (skipping both the call and its
* one delay slot) - a fact runtime code reading return_addressp gets
* "for free," correctly, with no adjustment. But
* middle/translate/runtimedata.ml's emit_site_spans (target-
* independent, shared by every backend) keys the pcmap table entry off
* the raw "call successor" CFG label placed by cfg/zipcfg/zipcfg.ml's
* 'call' (also shared, target-independent) - and on this backend that
* label lands at callsite+4 (right after the delay-slot nop), one
* MIPS instruction *before* where $31 actually points, because
* whatever compiles the call's "successor" block (the result-consuming
* move, or a placeholder nop when the result is discarded, as for
* call_gc()) is emitted as ordinary block content *after* the label
* rather than being accounted for in the label's own placement.
* Adjusting that label's placement is a target-independent, shared
* code path this fork wasn't able to isolate a safe local fix in - so,
* matching the sparc case above (same file, same function, opposite
* side of the mismatch), compensate at the one point that actually
* needs the two values to agree: subtract the same 4 bytes back off
* here so Cmm_lookup_entry's key matches what the table really has.
* Confirmed via gdb: qsort.tig's tiger_main resumed at 0x4037b4 (real
* $31, correct) while the pcmap table's actual entry for that call
* site was keyed at 0x4037b0 = 0x4037b4 - 4. Every non-hello/sieve
* tiger-mips failure at the time this was found showed the identical
* "array of size 0" / stale-GC-root symptom family this causes. */
#ifdef __mips__
    a->pc = (Cmm_Codeptr)((char *)a->pc - 4);
#endif
    update_saved_regs(a, a);
    <possibly shout about caller's EBP 175>
    callerdata = Cmm_lookup_entry(a->pc);
    a->rtdata = callerdata;

    calleedata_inalloc = Cmm_as_offset(calleedata->inalloc);
/* claude: amd64.ml's vfp sits AT the return-address word itself (see
* amd64call.ml's mem_ra, "the return address lives on the stack at the
* vfp location" - x86-64's 'call' has no link register, unlike every
* other 64-bit backend here, so it pushes that 8-byte word there,
* unconditionally, on every single call). amd64call.ml's own incoming-
* parameter automaton (its 'prolog') starts allocating stack-passed
* arguments at plain vfp instead of vfp+8 though - modeled on
* arm64call.ml's identically-shaped prolog, which is correct for
* AArch64 (no pushed return address to skip over) but not ported from
* x86call.ml's own analogous 32-bit prolog, which DOES start its own
* automaton at "addk vfp 4" for exactly this reason. The result: every
* amd64 procedure's own inalloc (computed from that automaton's own
* young_end, see middle/translate/ast2ir.ml's 'inalloc') comes out 8
* bytes short of where it should be, uniformly, regardless of whether

```



```

* the procedure has any stack-passed args at all. Reworking the
* automaton wiring to fix this at the source touches the same Block/Eqn
* simultaneous-equation solver every OTHER call-site automaton for the
* same procedure (in ovfl results, out call parms, stackdata, private,
* empty block, ...) is solved together with (confirmed empirically: a
* 'Block.srelative vfp ... -> autoAt (addk vfp 8) ...' swap in
* amd64call.ml's prolog alone left main's own frame with an
* unsolvable equation system, "Impossible: can't solve these eqns" -
* real regression risk to reworking that shared machinery further)
* - so, matching the SPARC/MIPS return-address compensations just above
* (same function, same "runtime-side patch at the one point that
* actually needs the two values to agree" reasoning their own comments
* give), the 8 bytes are added back here instead: the ONE place both of
* calleedata->inalloc's consumers (the normal in-callerdata branch just
* below, and Cmm_init_c_frame's own C-boundary branch) read it.
* Confirmed via the trace in docs/claude_notes/notes_amd64.txt's dated
* follow-up: without this, every vfp-stepping hop through amd64 code
* came up 8 bytes short, harmless for a hop or two (still landed on
* plausible-looking stack data) but fatal once enough nested calls
* (tig_alloc/new_string's own repeated-allocation chains, as
* tests/tiger64/colmajor.c--/rc4.c--/rb.c--/wf.c--/wff.c-- all trigger)
* accumulated the drift into unrelated stack memory - a GC root-scan
* that silently walked off the rails after only 4 real hops, or a
* segfault reading a garbage pointer as a Tiger string. NOT needed by
* i386/x86 (arch/x86/x86call.ml already gets its own 4-byte version of
* this right in the automaton itself, so its inalloc is already
* correct - adding this there too would double-count it). */
#ifdef __x86_64__
    calleedata_inalloc += 8;
#endif

if (callerdata) {
    if (Cmm_is_thread_start_frame(callerdata, a->pc))
        return 0;
    /* dealloopt = vfp          + calleedata->inalloc
       dealloopt = callervfp + callerdata->outalloc

       therefore callerfvp = vfp + calleedata->inalloc - callerdata->outalloc
    */
    a->vfp = vfp + calleedata_inalloc
            - Cmm_as_offset(callerdata->outalloc);
} else {
    a->methods = &c_methods;
    a->rtdata = Cmm_empty_pemap_entry;
    Cmm_init_c_frame(a, vfp + calleedata_inalloc);
}
return 1;
}



(possibly shout about caller's EBP 175)≡ (173c)


#define NOISY 0
#if NOISY
    fprintf(stderr, "leaving C-- activation; caller's EBP == 0x%08x\n",
        a->regs[9] ? (unsigned)*a->regs[9] : 0xdeadbeef);
#endif

```


Tracking register contents

As new activations are created during a walk of the stack, the locations of saved registers are recorded within the activations. The locations of all saved registers for an activation can be computed using information in the pc map table along with the previous activation.

```
<declarations of private functions 176a>+≡ (182c) <173b 179c>
void update_saved_regs(Cmm_Activation* new, const Cmm_Activation* prev);

<private functions 176b>+≡ (182c) <173c 179d>
void update_saved_regs(Cmm_Activation* new, const Cmm_Activation* prev) {
    int i;
    int r = prev->rtdata->num_registers;
    struct reg* regs = registers(prev->rtdata);

    for (i = 0; i < NUM_REGS; i++)
        new->regs[i] = prev->regs[i];

    for (i = 0; i < r; i++)
        new->regs[regs[i].index] = (Cmm_Word*)(prev->vfp + Cmm_as_offset(regs[i].saved));
}
```

This code will fail loudly if a register is ever saved in another register (instead of on the stack). This is as it should be, since a worse problem would be for the code to fail silently in the presence of a cycle among registers (e.g., register 5 is saved in register 7 and vice versa).

Exported stack-walking functions

At present, all the exported functions are built on top of the `change` method. The efficient path is `ChangeActivation`. As long as we don't have a very mixed stack, branch-prediction hardware should do well with the indirect call.

```
<implementations of public functions 176c>+≡ (182c) <173a 176d>
int Cmm_ChangeActivation(Cmm_Activation *a) {
    return a->methods->change(a);
}
```

`NextActivation` is not too bad, as most of the copying is necessary.

```
<implementations of public functions 176d>+≡ (182c) <176c 176e>
Cmm_Activation Cmm_NextActivation(const Cmm_Activation *a) {
    Cmm_Activation na = *a;
    int rc = na.methods->change(&na);
    assert(rc);
    return na;
}
```

`isOldestActivation` is wildly inefficient—in the common case, we copy the activation, mutate it, and then throw all that work away.

```
<implementations of public functions 176e>+≡ (182c) <176d 177a>
int Cmm_isOldestActivation(const Cmm_Activation *a) {
    Cmm_Activation na = *a;
    return (!na.methods->change(&na));
}
```

15.2.2 Unwinding to a continuation

To unwind to a continuation, we pass an activation to some assembly code that restores the registers from the contents of the activation. We do this by synthesizing the continuation `a->u.unwind.k`, in which the PC points

to the assembly code and the SP points to the relevant part of the activation. Although this assembly code is nothing like a C function, we import it as such.

```

<implementations of public functions 177a>+≡ (182c) <176e 178a>
extern void Cmm_unwindcont_pc(); /* assembly code; not a C function */
Cmm_Cont* Cmm_MakeUnwindCont(Cmm_Activation *a, Cmm_Word index, ...) {
    int i;
    struct conts *conts;
    struct contblock *block;

    assert(a->rtdata);
    conts = continuations(a->rtdata);
    if (index >= conts->num_entries)
        die("C-- run-time error: index %d out of bounds in Cmm_MakeUnwindCont", index);
    block = (struct contblock *) ((Cmm_Word *) conts + conts->entries[index]);
    <write variadic parameters into block->vars 177b>

    /* We initialize unknown registers to a bogus (but clearly live) address:
       that of the activation. Otherwise, assembly code dereferences 0. */
    for (i = 0; i < NUM_REGS; i++)
        if (a->regs[i] == NULL)
            a->regs[i] = (Cmm_Word *)a;

    /* This activation can't be passed into this interface again, so it
       is safe to overwrite the PC and ESP registers with the values that
       belong to the unwind continuation */

    a->regs[PC] = (Cmm_Word *) block->pc;
    a->regs[ESP] = (Cmm_Word *) (a->vfp + Cmm_as_offset(block->sp));
    a->u.unwind.k.pc = Cmm_unwindcont_pc;
    a->u.unwind.k.sp = (Cmm_Word *) a->regs;

    return(&a->u.unwind.k);
}

```

This part of the interface is unsafe, but it can't be helped.

```

<write variadic parameters into block->vars 177b>≡ (177a)
{
    va_list ap;
    va_start(ap, index);
    for (i = 0; i < block->num_vars; i++) {
        <assign block->vars[i] from ap 177c>
        Cmm_LocalVarWritten(a, block->vars[i].localnum);
    }
    va_end(ap);
}

```

Because passing to a varargs function involves implicit promotions, we do the promotions that are needed. The L argument is the type used internally; the R version is the same type as promoted for passing to a varargs function. Because somebody might generate code in which a formal parameter of a continuation is not used, we have to consider the possibility that the variable being assigned to is dead. In that case, we consume the actual parameter but make no assignment.

```

<assign block->vars[i] from ap 177c>≡ (177b)
#define ASSIGN(L,R) \
{ L *formal = (L*) Cmm_FindLocalVar(a, block->vars[i].localnum); \
  R actual = va_arg(ap, R); \
  if (formal) *formal = (L) actual; \
}
switch(block->vars[i].ctype) {
    case CHAR:          ASSIGN (char,          int);          break;

```

```

case DOUBLE:      ASSIGN (double,      double);      break;
case FLOAT:       ASSIGN (float,       double);       break;
case INT:         ASSIGN (int,         int);           break;
case LONGDOUBLE:  ASSIGN (long double, long double);   break;
case LONGINT:     ASSIGN (long int,    long int);      break;
case LONGLONGINT: ASSIGN (long long    int, long long int); break;
case SHORT:      ASSIGN (short,       int);           break;
case SIGNEDCHAR:  ASSIGN (signed char, int);           break;
case UNSIGNEDCHAR: ASSIGN (unsigned char, int);        break;
case UNSIGNEDLONG: ASSIGN (unsigned long, unsigned long); break;
case UNSIGNEDSHORT: ASSIGN (unsigned short, int);      break;
case UNSIGNEDINT:  ASSIGN (unsigned int, unsigned int); break;
case UNSIGNEDLONGLONG: ASSIGN (unsigned long long, unsigned long long); break;
case ADDRESS:     ASSIGN (void *,     void *);        break;
default: assert(0);
}

```

15.2.3 Span Data

For any activation, the client may ask for the innermost enclosing span associated with a given token. The user spans are contained in the run-time data emitted by the compiler. Tokens must be integers greater than or equal to zero. A table of descriptors is stored in the pc map entry immediately following the locals data. Using the activation record, the table of descriptors is looked up, and the appropriate entry returned.

(implementations of public functions 178a)+≡ (182c) <177a 178b>

```

Cmm_Dataptr Cmm_GetDescriptor(const Cmm_Activation *a, Cmm_Word token) {
    unsigned num_spans;
    Cmm_Word* descs;
    pc_map_entry* entry = a->rtdata;
    if (!entry)
        return 0;

    num_spans = entry->num_spans;
    if ((unsigned) token < num_spans) {
        descs = spans(entry);
        return (Cmm_Dataptr)descs[token];
    } else {
        return 0;
    }
}

```

15.2.4 Accessing Local Variables

Local variables for an activation can be found either on the stack, or in registers. The location of parameters is computed using information generated by the C-- compiler. Each C-- call site has associated with it the number of locals, and an array containing information about each local. If a local lives on the stack, an offset from the frame pointer is given. If a local is in a register, then the register number is given. The number of registers, and their indexes are architecture dependent.

In addition to information about locals, each call site has information about saved registers at the time of the call. If a register has been saved on the stack, then the offset from the frame pointer is given. Otherwise, a zero indicates that the register has not been saved by this activation.

Clients of the runtime system must report when a local variable is written.

(implementations of public functions 178b)+≡ (182c) <178a 179a>

```

void Cmm_LocalVarWritten(const Cmm_Activation *a, unsigned n) {
    return;
}

```

Clients of the runtime system can ask for the number of locals for a given activation. This information is stored in the activation structure.

```

<implementations of public functions 179a>+≡ (182c) <178b 179b>
unsigned Cmm_LocalVarCount(const Cmm_Activation *a) {
    assert(a);
    if (a->rtdata)
        return a->rtdata->num_locals;
    else
        return 0;
}

```

Local variables are asked for by index. A pointer is returned to the local by looking it up in the array of locals for this activation.

```

<implementations of public functions 179b>+≡ (182c) <179a 179e>
void* Cmm_FindLocalVar(const Cmm_Activation *a, unsigned n) {
    pc_map_entry* entry = a->rtdata;
    assert(entry);
    if (n >= entry->num_locals)
        die("local var index %d out of range", n);
    return locptr(a, locals(entry)[n]);
}

```

If a location is in a register, a pointer into the saved register is returned. Otherwise, a pointer into the stack where the register is saved is returned.

```

<declarations of private functions 179c>+≡ (182c) <176a 183a>
static Cmm_Word *locptr(const Cmm_Activation *a, location l);

<private functions 179d>+≡ (182c) <176b 183b>
static Cmm_Word *locptr(const Cmm_Activation *a, location l) {
    int reg;
    switch (Cmm_loctype(l)) {
        case REGISTER:
            reg = Cmm_as_register(l);
            assert(a);
            assert(a->regs);
            if (a->regs[reg] == NULL)
                die("Tried to find a value in an unrecoverable register.\n"
                    "Probable cause: calling C without using the \"paranoid C\" convention.");
            return a->regs[reg];
        case DEAD:
            return NULL;
        case OFFSET:
            return (Cmm_Word *) (a->vfp + Cmm_as_offset(l));
        default:
            die("this can't happen --- badly coded location");
            return 0;
    }
}

```

In the current implementation it is not possible to find the location of a dead variable.

```

<implementations of public functions 179e>+≡ (182c) <179b 179f>
void* Cmm_FindDeadLocalVar(const Cmm_Activation *a, unsigned n) {
    return Cmm_FindLocalVar(a, n);
}

```

Stack data is stored in a table per procedure. Each PC map entry links to the correct stack data table.

```

<implementations of public functions 179f>+≡ (182c) <179e 180d>
void* Cmm_FindStackLabel(const Cmm_Activation *a, unsigned n) {
    pc_map_entry* entry = a->rtdata;
    struct sd *sdt = entry->stackdata_table;
}

```

```

assert(sdt);
if (n >= sdt->num_entries)
    die("Stack data index %d out of range [0..%d]\n", n, sdt->num_entries);
return locptr(a, sdt->entries[n]);
}

```

15.2.5 Run-time functions written in C--

Function `Cmm_CutTo` is a convenience; it is easy to write in C--.

```

<cut.c- 180a>≡
target byteorder little;
export Cmm_CutTo;
foreign "C" Cmm_CutTo(bits32 k) {
    cut to k();
}

<yield.c- 180b>≡
target byteorder little;
export Cmm_Yield;
foreign "C" Cmm_Yield(bits32 k, bits32 kold) {
    bits32[kold] = bits32[knew];
    bits32[kold+4] = bits32[knew+4];
    cut to k() also cuts to knew also aborts;
    continuation knew(): foreign "C" return ;
}

```

15.2.6 Threads

```

<external functions 180c>+≡ (182c) <172d
extern void Cmm_start_thread_helper();
extern void Cmm_thread_helper_fence();
extern int Cmm_stack_growth; /* generated by qc-- -globals */

<implementations of public functions 180d>+≡ (182c) <179f 180f>
int Cmm_is_thread_start_frame(struct cmm_pc_map_entry *rtdata, Cmm_Codeptr pc) {
    return spans(rtdata)[0] == 0xefffaced &&
        (unsigned) Cmm_start_thread_helper <= (unsigned) pc &&
        (unsigned) pc <= (unsigned) Cmm_thread_helper_fence;
}

```

Because the run-time system is split across multiple C files, this function needs to be exposed—but it is private to the C-- run-time system, and it is an unchecked run-time error for client code to call this function.

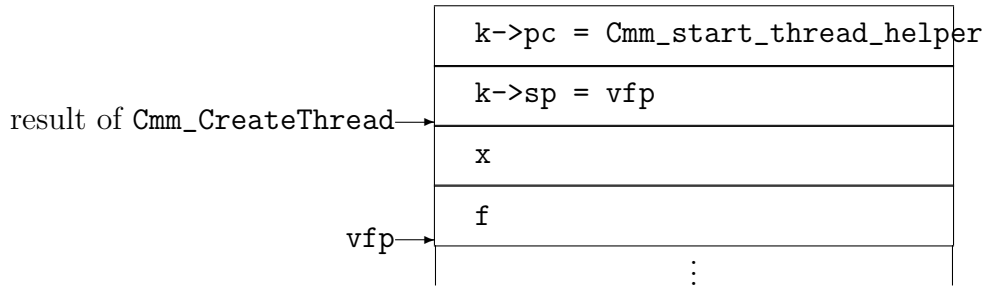
```

<exposed private functions 180e>≡ (168a)
int Cmm_is_thread_start_frame(struct cmm_pc_map_entry *rtdata, Cmm_Codeptr pc);

<implementations of public functions 180f>+≡ (182c) <180d 181d>
Cmm_Cont* Cmm_CreateThread(Cmm_Codeptr f, Cmm_Dataptr x, void *s, unsigned n) {
    char *stack = s;
    char *young, *old, *vfp;
    Cmm_Cont *k;
    if (Cmm_stack_growth < 0) {
        <set up stack for growth downward 181a>
        return k;
    } else {
        <set up stack for growth upward 181b>
        return k;
    }
}

```

Here is the layout of a thread with a down-going stack:



I set up the stack pointer to point to the first empty word.

<set up stack for growth downward 181a>≡ (180f)

```

young = stack;
old    = stack + n;
k = (Cmm_Cont *) old;
k--;
k->sp = (Cmm_Word*) (vfp = old - sizeof(*k) - sizeof(f) - sizeof(x));
k->pc = Cmm_start_thread_helper;
*(Cmm_Codeptr*)vfp = f;
vfp += sizeof(f);
*(Cmm_Dataptr*)vfp = x;

```

<set up stack for growth upward 181b>≡ (180f)

```

old    = stack;
young = stack + n;
k = (Cmm_Cont *) old;
k++;
k->sp = (Cmm_Word*) (vfp = old + sizeof(*k) + sizeof(f) + sizeof(x));
k->pc = Cmm_start_thread_helper;
*(Cmm_Codeptr*)vfp = f;
vfp -= sizeof(f);
*(Cmm_Dataptr*)vfp = x;

```

<thread.c- 181c>≡

```

target byteorder little;
export Cmm_start_thread_helper, Cmm_thread_helper_fence;
import write, abort;
section "text" {
    span 0 0xeffaced {
        foreign "C-- thread" Cmm_start_thread_helper(bits32 f, bits32 x) {
            f(x) also aborts;
            foreign "C" write(2, "address" msg, msg_end - msg);
            foreign "C" abort() never returns;
        }
    }
    Cmm_thread_helper_fence:
}
section "data" {
    msg: bits8[] "fatal C-- error: function passed to Cmm_CreateStack returned!\n";
    msg_end:
}

```

<implementations of public functions 181d>+≡ (182c) <180f

```

static void show_span(unsigned key, void *value, void *closure) {
    if (value)
        printf("    span %d == %p\n", key, value);
}

```

```

}

void Cmm_show_activation(const Cmm_Activation *a) {
    printf("activation @ %p = {\n", (void*)a);
    printf("  methods = ");
    if (a->methods == Cmm_cmm_frame_methods)
        printf("C-- methods,\n");
    else if (a->methods == &c_methods)
        printf("C methods,\n");
    else
        printf("unknown methods at %p,\n", (void*)a->methods);
    printf("  vfp = %p, pc = %p,\n", a->vfp, (void *)a->pc);
    <print registers in activation a 182a>
    printf("  entry = {\n");
    Cmm_show_map_entry(a->rtdata, -1, a->pc, show_span, NULL);
    printf("  }\n}\n");
}

```

<print registers in activation a 182a>≡ (181d)

```

{ int any, i;
  char *pfx;
  any = 0;
  for (i = 0; i < NUM_REGS; i++)
      if (a->regs[i]) {
          any = 1;
          break;
      }
  if (any) {
      printf("  regs = { ");
      pfx = "";
      for (i = 0; i < NUM_REGS; i++)
          if (a->regs[i]) {
              printf("%sREG %d @ %p", pfx, i, (void*)a->regs[i]);
              pfx = ", ";
          }
      printf(" },\n");
  } else {
      printf("  regs = (no registers recoverable),\n");
  }
}

```

<definition of struct cmm_cont 182b>≡ (171e)

```

struct cmm_cont {
    Cmm_Codeptr pc;
    Cmm_Word*   sp;
};

```

15.2.7 Putting it together

<runtime.c 182c>≡

```

#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

#include "qc--runtime.h"
#include "pemap.h"

```

<machine-dependent macro definitions for the implementation 171a>

<external functions 172b>
 <definitions of private data structures 171d>
 <declarations of private functions 173b>
 <definitions of method suites 172c>
 <private functions 173c>
 <implementations of public functions 173a>

<declarations of private functions 183a>+≡ (182c) <179c
 static void die(char *msg,...);

<private functions 183b>+≡ (182c) <179d
 static void die(char *msg,...) { /* see K&R 2nd ed, page 174 */
 va_list args;
 va_start(args,msg);
 fprintf(stderr, "C-- run-time error: ");
 vfprintf(stderr, msg, args);
 fprintf(stderr, "\n");
 va_end(args);
 abort();
 }

15.3 PC Map

This file provides an interface to the run-time data emitted by the compiler.

<pcmap.h 183c>≡
 #ifndef PC_MAP_H
 #define PC_MAP_H

 <exported type, procedure, and macro declarations 183d>

 #endif /* PC_MAP_H */

Locations are coded as described in `src/runtimedata.nw`. A location has one of three types: dead, register, or offset. To get the register number of a location with register type, pass the location to `as_register`; similarly with `as_offset`. To use one of these functions with a location of the wrong type is a checked run-time error.

<exported type, procedure, and macro declarations 183d>≡ (183c) 183e▷
 typedef struct coded_location { Cmm_Word l; } location;
 typedef enum { DEAD=0, REGISTER=1, OFFSET=2 } loctype;
 loctype Cmm_loctype(location l);
 unsigned Cmm_as_register(location l);
 intptr_t Cmm_as_offset (location l);
 #define isdead(LOC) ((LOC).l == 0)

The `pc_map_index` and `pc_map_entry` structs below are used to access the PC map table and run-time data entries emitted by the compiler. A more detailed description of the layout and contents of the emitted data can be found in the file `src/runtimedata.nw`. N.B. although the `pc_map_index` definition is exact, the `pc_map_entry` is only an approximation. The actual data structure has fields of varying size; hence the `num_*` fields and the macros. The documentation of the fields needs work.

<exported type, procedure, and macro declarations 183e>+≡ (183c) <183d 184a▷
 typedef struct {
 Cmm_Codeptr ra;
 struct cmm_pc_map_entry* entry;
 } pc_map_index;

 typedef struct cmm_pc_map_entry pc_map_entry;
 struct cmm_pc_map_entry {
 location inalloc; /* incoming dealloc point on stack (always offset) */


```

location    outalloc;          /* outgoing dealloc point on stack (always offset) */
location    return_addressp; /* where return address is saved */
struct sd *stackdata_table;
Cmm_Word    num_registers;
Cmm_Word    num_locals;
Cmm_Word    num_spans;
Cmm_Word    cont_block_size;
location    data[2]; /* registers, locals, continuations, spans */
};

```

Here are the individual pieces. The stack-data array stores the location of each `stackdata` label.

(exported type, procedure, and macro declarations 184a)+≡ (183c) <183e 184b>

```

struct sd {
    unsigned num_entries;
    location entries[1]; /* variable length */
};

```

Information about callee-saved registers gives the index of a register and the location in which the value of the register is saved.

(exported type, procedure, and macro declarations 184b)+≡ (183c) <184a 184c>

```

struct reg {
    unsigned index;          /* machine-specific index */
    location saved;          /* where saved (or dead) */
};

```

The structure for `also unwinds` to continuations is more complicated. Each call site contains a list of entries. A single entry is represented by a `contblock`, which is of variable size because each continuation may take a different number of parameters.

(exported type, procedure, and macro declarations 184c)+≡ (183c) <184b 184d>

```

struct conts {
    unsigned num_entries;
    int entries[1]; /* variable length (offset of contblock) */
};

struct contblock {
    unsigned num_vars;
    Cmm_Codeptr pc;
    location sp;
    struct contarg {
        Cmm_Word localnum;
        Cmm_Word ctype;
    } vars[1]; /* variable length */
};

```

The compiler and run-time system must agree on the representation of C types. The `ctype` field above would be declared `enum ctypes` except that we can't rely on a C compiler to give an enum a predictable size and alignment.

(exported type, procedure, and macro declarations 184d)+≡ (183c) <184c 184e>

```

enum ctypes { CHAR = 0, DOUBLE = 1, FLOAT = 2, INT = 3, LONGDOUBLE = 4, LONGINT = 5
    , LONGLONGINT = 6, SHORT = 7, SIGNEDCHAR = 8, UNSIGNEDCHAR = 9
    , UNSIGNEDLONG = 10, UNSIGNEDSHORT = 11, UNSIGNEDINT = 12
    , UNSIGNEDLONGLONG = 13, ADDRESS = 14
};

```

Here is the address arithmetic that provides access to different parts of an entry. We do the address arithmetic using macros whose names end in `A`. We then provide a second set of macros that contain suitable casts. Using macros is a bit dodgy, but NR was worried about procedure-call overhead for the arithmetic.

(exported type, procedure, and macro declarations 184e)+≡ (183c) <184d 185a>

```

#define registersA(e)    ((e)->data)
#define localsA(e)      (registersA(e) + 2 * (e)->num_registers)

```

```

#define continuationsA(e) (localsA(e)          +      (e)->num_locals)
#define spansA(e)         (continuationsA(e) +      (e)->cont_block_size)

#define registers(e)       ((struct reg*)registersA(e))
#define locals(e)         ((location *)localsA(e))
#define continuations(e)  ((struct conts *)continuationsA(e))
#define spans(e)          ((Cmm_Word*)spansA(e))

```

Finally, here is where we get entries: we can look them up by PC, or we have special entries for unknown procedures (the empty entry) and for start-of-thread activations.

```

⟨exported type, procedure, and macro declarations 185a⟩+≡ (183c) <184e 185b>
pc_map_entry* Cmm_lookup_entry(const Cmm_Codeptr caller);
pc_map_entry* Cmm_empty_pmap_entry;
pc_map_entry* Cmm_thread_start_up_pmap_entry;
pc_map_entry* Cmm_thread_start_dn_pmap_entry;

```

We provide some functions for debugging. If not NULL, the “span shower” is called for each user span.

```

⟨exported type, procedure, and macro declarations 185b⟩+≡ (183c) <185a
typedef void (*Cmm_span_shower)(unsigned key, void *value, void *closure);
void Cmm_show_map(Cmm_span_shower, void *); /* for debugging */
void Cmm_show_map_entry(pc_map_entry *entry, int index, Cmm_Codeptr ra,
                        Cmm_span_shower show_span, void *closure);

```

15.3.1 Implementation

```

⟨pmap.c 185c⟩≡
⟨pmap.c 185d⟩

⟨pmap.c 185d⟩≡ (185c) 185e>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <string.h>
#include "qc--runtime.h"
#include "pmap.h"

```

Hand-written map entries

Here are the hand-written map entries, together with some support for creating locations.

```

⟨pmap.c 185e⟩+≡ (185c) <185d 185f>
/* claude: (Cmm_Word)1, not a bare 1 (int) - shifting by sizeof(location)*8-1
 * is 63 on a 64-bit target now that Cmm_Word is uintptr_t, which is
 * undefined behavior for a 32-bit int shift. */
#define MKOFFSET(N) { ((Cmm_Word)1 << (sizeof(location)*8 - 1)) | (N) }
#define MKDEAD      { 0 }
static struct sd nostackdata = { 0, { MKDEAD } };

```

An unknown functions has no stackdata, registers, locals, spans, or continuations, and we don’t know any relations on allocation points.

```

⟨pmap.c 185f⟩+≡ (185c) <185e 186a>
static struct cmm_pc_map_entry empty = {
    MKOFFSET(0),
    MKOFFSET(0),
    MKOFFSET(0),
    &nostackdata,
    0, 0, 0, 0, { MKDEAD }
};
pc_map_entry* Cmm_empty_pmap_entry = &empty;

```

A thread-start activation contains two locals, whose locations depend on the direction of stack growth. Because such a frame is never walked (it has a special, trivial `change` method), we don't store anything about deallocation points.

```

⟨pcmap.c 186a⟩+≡ (185c) <185f 186b>
static struct cmm_pc_map_entry startdn = {
    MKOFFSET(0),
    MKOFFSET(0),
    MKDEAD,
    &nostackdata,
    0, 2, 0, 0, { MKOFFSET(0), MKOFFSET(sizeof(Cmm_Word)) }
};
static struct cmm_pc_map_entry startup = {
    MKOFFSET(0),
    MKOFFSET(0),
    MKDEAD,
    &nostackdata,
    0, 2, 0, 0, { MKOFFSET(0), MKOFFSET(-sizeof(Cmm_Word)) }
};
pc_map_entry* Cmm_thread_start_up_pcmap_entry = &startup;
pc_map_entry* Cmm_thread_start_dn_pcmap_entry = &startdn;

```

Entry lookup by PC

The labels `Cmm_pc_map` and `Cmm_pc_map_limit` are inserted by the linker script in chunk <<pcmap.ld>>.

```

⟨pcmap.c 186b⟩+≡ (185c) <186a 186c>
/* claude: on every ELF host, Cmm_pc_map/Cmm_pc_map_limit are bound by
 * pcmap.ld (a GNU-ld "-T" linker-script fragment that places every input
 * object's .pcmap section together and defines these two symbols around
 * it - see that file). Mach-O has no equivalent to "-T": ld64 does not
 * support linker scripts at all. Empirically confirmed replacement: ld64
 * already auto-synthesizes a "section$start$SEGMENT$section"/
 * "section$end$SEGMENT$section" symbol pair for every section, with no
 * script needed (verified with a standalone hand-assembled test .s/.c
 * pair before relying on it here - see docs/claude_notes/notes_arm64.txt).
 * arm64asm.ml emits the pcmap section as ".section __DATA,pcmap", so the
 * two symbols below are exactly its start/end. No macOS counterpart to
 * pcmap.ld is needed at all - this #ifdef is the whole story. */
#ifdef __APPLE__
extern pc_map_index Cmm_pc_map[] __asm("section$start$__DATA$pcmap");
extern pc_map_index Cmm_pc_map_limit[] __asm("section$end$__DATA$pcmap");
#else
extern pc_map_index Cmm_pc_map[];
extern pc_map_index Cmm_pc_map_limit[];
#endif
#define pc_map_size (Cmm_pc_map_limit - Cmm_pc_map)

```

If we could rely on the compiler to sort the entries by PC, then the lookup function could always do binary search. But there have been bugs, so we have to check, and if entries are not sorted, we use linear search.

Here's where we check for sorting. Two adjacent entries with *equal* PCs are considered OK. This can happen if a call site annotated with `never returns` is followed immediately by a continuation, for example.

```

⟨pcmap.c 186c⟩+≡ (185c) <186b 187a>
static int is_sorted(void) {
    pc_map_index *p;
    for (p = Cmm_pc_map+1; p < Cmm_pc_map_limit; p++)
        if ((unsigned)p[-1].ra > (unsigned)p[0].ra) {
            ⟨conditionally spray information about the pc_map array 187d⟩
            return 0;
        }
}

```

```

    <conditionally announce that pc_map is sorted 187e>
    return 1;
}

```

Here's the binary search for the sorted case.

```

<pcmap.c 187a>+≡ (185c) <186c 187b>
static int compare(const void *x, const void *y) {
    const pc_map_index *p = y;
    return (unsigned) x - (unsigned) p->ra;
}

static pc_map_entry *binlookup(const Cmm_Codeptr caller) {
    pc_map_index *p;
    p = bsearch((void*)caller, Cmm_pc_map, pc_map_size, sizeof(Cmm_pc_map[0]), compare);
    if (p)
        return p->entry;
    else
        return NULL;
}

```

Here's the linear search for the unsorted case.

```

<pcmap.c 187b>+≡ (185c) <187a 187c>
static pc_map_entry* linlookup(const Cmm_Codeptr caller) {
    unsigned i = 0;
    for(i = 0; i < pc_map_size; ++i)
        if (Cmm_pc_map[i].ra == caller)
            return Cmm_pc_map[i].entry;
    return NULL;
}

```

We cache the search function in lookup.

```

<pcmap.c 187c>+≡ (185c) <187b 188a>
pc_map_entry* Cmm_lookup_entry(const Cmm_Codeptr caller)
{
    static pc_map_entry *(*lookup)(const Cmm_Codeptr caller) = NULL;
    if (lookup == NULL)
        lookup = is_sorted() ? binlookup : linlookup;
    return lookup(caller);
}

```

And this is for debugging!

```

<conditionally spray information about the pc_map array 187d>≡ (186c)
#define SHOW_UNSORTED 1
{ char *debug = getenv("QCDEBUG");
  if (SHOW_UNSORTED || (debug && strstr(debug, "pcmap"))) {
      fprintf(stderr, "C-- Surprise! PCMAP array is unsorted!\n");
      for (p = Cmm_pc_map; p < Cmm_pc_map_limit; p++) {
          fprintf(stderr, "  ra = %8p", (void*)p->ra);
          if (p > Cmm_pc_map && (unsigned)p[-1].ra > (unsigned)p[0].ra)
              fprintf(stderr, " *");
          fprintf(stderr, "\n");
      }
  }
}

```

```

<conditionally announce that pc_map is sorted 187e>≡ (186c)
{ char *debug = getenv("QCDEBUG");
  if (debug && strstr(debug, "pcmap"))
      fprintf(stderr, "C-- info: PCMAP array is sorted, as expected\n");
}

```

Locations

The compiler encodes the type in the top two bits. Registers are given the encoding 01, dead variables are given 00, and offsets are prefixed with 1. See `emit_data` in `src/runtimedata.nw` for more details.

```
<pcmap.c 188a>+≡ (185c) <187c 188b>
/* claude: local's own width, not a fixed 32 bits - GOODBITS has to match
 * whatever width qc-- actually packed this location as (see qc--runtime.h's
 * Cmm_Word claude: comment): 30 on every existing 32-bit backend
 * (unchanged), 62 on a 64-bit one. */
#define GOODBITS ((intptr_t)sizeof(local)*8 - 2)

loctype Cmm_loctype(location l) {
    intptr_t local = (intptr_t) l.l;
    local = local >> GOODBITS;
    if (local & 2)
        return OFFSET;
    else
        return (loctype) local;
}

<pcmap.c 188b>+≡ (185c) <188a 188c>
unsigned Cmm_as_register(location l) {
    intptr_t local = (intptr_t) l.l;
    intptr_t mask = GOODBITS / 2;
    intptr_t slice = (local & (((intptr_t)1 << GOODBITS) - 1))
        >> mask;
    intptr_t offset = local & (((intptr_t)1 << mask) - 1);
    assert(Cmm_loctype(l) == REGISTER);

    if (slice) {
        fprintf(stderr, "register slices not supported.\n");
        assert(0);
    }
    return (unsigned) offset;
}

<pcmap.c 188c>+≡ (185c) <188b 188d>
intptr_t Cmm_as_offset(location l) {
    intptr_t local = (intptr_t) l.l;
    return (local << 1)
        >> 1;
}
```

Debugging support

`show_map` dumps out all of the run-time data. It is only useful for debugging the `qc--` run-time system.

```
<pcmap.c 188d>+≡ (185c) <188c 189a>
static void printloc(location loc) {
    switch (Cmm_loctype(loc)) {
        case REGISTER:
            printf ("REG %d", Cmm_as_register(loc));
            break;
        case DEAD:
            printf ("DEAD");
            break;
        case OFFSET:
            printf ("OFFSET %d", Cmm_as_offset(loc));
            break;
        default:
```

```

        printf("<MALFORMED %X>", loc.l);
        break;
    }
}

⟨pcmap.c 189a⟩+≡ (185c) ◁188d 189b▷
⟨private show functions 190e⟩

void Cmm_show_map(Cmm_span_shower show_span, void *closure) {
    pc_map_index *idx;
    printf("pc_map_size %d\n", pc_map_size);
    for (idx = Cmm_pc_map; idx < Cmm_pc_map_limit; idx++) {
        Cmm_show_map_entry(idx->entry, idx - Cmm_pc_map, idx->ra, show_span, closure);
    }
}

⟨pcmap.c 189b⟩+≡ (185c) ◁189a
void Cmm_show_map_entry(pc_map_entry *entry, int index, Cmm_Codeptr ra,
                        Cmm_span_shower show_span, void *closure)
{
    struct sd *sdt;
    char *indent = index < 0 ? "    " : "";
    if (index >= 0)
        printf("%sentry%3d @ %p (ra = %p):\n", indent, index, (void *)entry, (void *) ra);
    else
        printf("%sentry @ %p (ra = %p):\n", indent, (void *)entry, (void *) ra);
    printf("%s  inalloc = %d (coded %X), outalloc = %d (coded %X)\n"
           "%s  num_regs = %d, num_locals = %d, num_spans = %d,"
           " size of cont block = %d\n",
           indent,
           Cmm_as_offset(entry->inalloc),
           entry->inalloc.l,
           Cmm_as_offset(entry->outalloc),
           entry->outalloc.l,
           indent,
           entry->num_registers,
           entry->num_locals,
           entry->num_spans,
           entry->cont_block_size);
    printf("%s  return address at ", indent);
    printloc(entry->return_addressp);
    printf("\n");
    sdt = entry->stackdata_table;
    ⟨show locals 190b⟩
    ⟨show registers 190a⟩
    ⟨show continuations 190c⟩
    ⟨show stack-data table sdt 189c⟩
    ⟨show spans 190d⟩
}

⟨show stack-data table sdt 189c⟩≡ (189b)
assert(sdt);
if (sdt->num_entries == 0) {
    printf("%s    (no stackdata)\n", indent);
} else {
    int i;
    for (i = 0; i < sdt->num_entries; i++) {
        printf("%s    stacklabel%3d = ", indent, i);
        printloc(sdt->entries[i]);
        printf("\n");
    }
}

```

```

    }
}

⟨show registers 190a⟩≡ (189b)
{
    int r = entry->num_registers;
    int i;
    struct reg* regs = registers(entry);
    for (i = 0; i < r; i++) {
        printf("%s      pair %2d: caller register %d at ", indent, i, regs[i].index);
        printloc(regs[i].saved);
        printf("\n");
    }
}

```

```

⟨show locals 190b⟩≡ (189b)
{
    int n = entry->num_locals;
    if (n == 0) {
        printf("%s      (no locals)\n", indent);
    } else {
        int i;
        for (i = 0; i < n; i++) {
            printf("%s      local%3d at ", indent, i);
            printloc((locals(entry))[i]);
            printf("\n");
        }
    }
}

```

```

⟨show continuations 190c⟩≡ (189b)
{
    int i;
    struct conts *conts;
    conts = continuations(entry);

    if (conts->num_entries == 0) {
        printf("%s      (no continuations)\n", indent);
    } else {
        for (i = 0; i < conts->num_entries; i++)
            show_cont(conts, i, indent);
    }
}

```

```

⟨show spans 190d⟩≡ (189b)
{ unsigned i;
  Cmm_Word *descs = spans(entry);
  for (i = 0; i < entry->num_spans; i++)
      show_span(i, (void *)descs[i], closure);
}

```

```

⟨private show functions 190e⟩≡ (189a) 191a▷
const char *typestring(enum ctypes t) {
#define xx(T) case T: return #T;
    switch(t) {
        xx(CHAR) xx(DOUBLE) xx(FLOAT) xx(INT) xx(LONGDOUBLE) xx(LONGINT)
        xx(LONGLONGINT) xx(SHORT) xx(SIGNEDCHAR) xx(UNSIGNEDCHAR)
        xx(UNSIGNEDLONG) xx(UNSIGNEDSHORT) xx(UNSIGNEDINT)
        xx(UNSIGNEDLONGLONG) xx(ADDRESS)
        default: return "unknown-type";
    }
}

```

```

⟨private show functions 191a⟩+≡ (189a) <190e
static void show_cont(struct conts *conts, int contnum, const char *indent) {
    struct contblock *block;
    int i;

    printf("%s    unwind cont%2d (", indent, contnum);
    block = (struct contblock *) ((Cmm_Word *) conts + conts->entries[contnum]);
    for (i = 0; i < block->num_vars; i++) {
        printf("%s%s local%d", i > 0 ? ", " : "", typestring(block->vars[i].ctype),
            block->vars[i].localnum);
    }
    printf(") = <pc=%p, sp=", (void*)block->pc);
    printloc(block->sp);
    printf(">\n");
}

```

15.3.2 Linker script

The run-time data is in a table indexed by PC. For each call site and continuation, there is an entry in the `pcmap` section which points to the run-time data. The actual run-time data is placed in the `pcmap_data` section. Both of these are collected up and placed in the a data section by the system linker. Below is an example linker script for x86 Linux that makes the PC map available to the run-time system.

```

⟨pcmap.ld 191b⟩≡
SECTIONS {
    .rodata : {
        . = (. + 3) & ~ 3;
        Cmm_pc_map = .;
        *(.pcmap)
        Cmm_pc_map_limit = .;
        *(.pcmap_data)
    }
}

```

If the compiler is careful to sort PC-map entries within each compilation unit, this linker script will ensure they are sorted overall.

15.4 Walking GCC-Linux Stack Frames

This version of the gcc stack walker is based on the System V ABI for the i386 platform. We assume that a frame pointer is being used (no `--fomit-frame-pointer`). Every function stores the frame pointer in `EBP`, and this register is saved at the top of each stack frame on entry.

```

⟨gcc-linux.c 191c⟩≡
⟨gcc-linux.c 191d⟩
⟨gcc-linux.c 191d⟩≡ (191c) 192▷
#include "qc--runtime.h"
#include "pcmap.h"
#include <assert.h>
#include <stdio.h>

/* claude: FP_REG is the activation-index (qc--runtime.h's NUM_REGS
 * numbering) of whichever callee-saved register real gcc-compiled C code
 * (compiled -fno-omit-frame-pointer) uses as its own frame-pointer chain
 * - this is what lets the walk below continue from our outermost C--
 * frame into genuine C frames above it (main's caller, libc, ...). Each
 * value below was either confirmed by compiling a probe with the target

```



```

* cross-gcc -fno-omit-frame-pointer and reading which register its own
* prologue/.frame directive names, or derived from qc--runtime.h's own
* NUM_REGS numbering when a probe wasn't (yet) run - noted per case. An
* unrecognized platform gets a #error, not x86's FP_REG=9 by default -
* see qc--runtime.h's own NUM_REGS comment for why silently reusing
* another target's value is the bug this guards against. */
#ifdef __powerpc__
#define FP_REG 72 /* r31 - confirmed via "mr 31,1" in a probe's prologue */
#elif defined(__sparc__)
/* claude: never actually populated, by design - SPARC's register windows
* save everything in hardware, so update_saved_regs's "for each callee-
* saved register" loop never runs and a->regs[FP_REG] is always NULL
* (see the graceful-bail comment on the assert below) - any in-range
* placeholder works identically since the slot is always NULL. */
#define FP_REG 30 /* %fp/%i6 */
#elif defined(__riscv)
#define FP_REG 46 /* s0/x8, flat index 38+8 - derived, not probe-confirmed */
#elif defined(__alpha__)
#define FP_REG 53 /* $15/s6/fp, flat index 38+15 - confirmed via ".frame $15,..." */
#elif defined(__mips__)
#define FP_REG 68 /* $30/fp, flat index 38+30 - confirmed via ".frame $fp,..." */
#elif defined(__arm__)
#define FP_REG 10 /* r7, flat index 3+7 - confirmed via "push {r7}"/frame_needed=1 in a probe (this toolchain d
#elif defined(__m68k__)
#define FP_REG 13 /* %a6/%fp, flat index 4+9 - confirmed via "link.w %fp,#-4"/"unlk %fp" in a probe (m68k-linux
#elif defined(__aarch64__)
#define FP_REG 66 /* x29/fp, flat index 37+29 - confirmed via a throwaway OCaml harness calling Target.mk_reg_i
#elif defined(__x86_64__)
#define FP_REG 9 /* %rbp, flat index 4+5 - confirmed via the same throwaway
OCaml harness as qc--runtime.h's own __x86_64__ NUM_REGS
comment (rbp is amd64regs.ml's r-space index 5, r-space
starts at flat index 4) - NOT the same value as i386's
FP_REG=9 below by coincidence of the number alone, that
one is r[5] within a different, smaller register file
(c(3) is only 4 slots wide either way, so both land on
"c-space-width + r-space-index-5", but the two c/r
spaces are unrelated register files - do not assume this
numeric coincidence generalizes to any other value on
this branch). Standard SysV AMD64 frame-pointer
convention: clang/gcc -fno-omit-frame-pointer emit
"push %rbp; mov %rsp,%rbp", same role x86-64's rbp plays
as i386's ebp/AArch64's x29 - not independently probe-
confirmed via a compiled prologue here, but this is
universal, well-documented ABI convention, not a
toolchain-specific choice the way ARM32's Thumb-vs-ARM
r7-vs-r11 pick was. */
#elif defined(__i386__)
#define FP_REG 9 /* %ebp, r[5] */
#else
#error "NUM_REGS/FP_REG not defined for this platform - see qc--runtime.h/gcc-linux.c"
#endif

```

On initialization, the `young_in_overflow` pointer is the callee's deallocation point. Because the x86 uses a frame pointer, we don't actually need this information.

```

<gcc-linux.c 192>+= (191c) <191d 194>
void Cmm_init_c_frame(Cmm_Activation *a, char *young_in_overflow) {
    Cmm_Word fp;
    /* claude: was a bare "assert(a->regs[FP_REG])", which is fine for x86/
    * ppc (their calling conventions mark the frame-pointer register non-
    * volatile, so qc--'s own register-preservation bookkeeping - see

```

```

* runtime.c's update_saved_regs - always records a slot for it before
* we get here) but wrong for SPARC: its calling convention
* (arch/sparc/sparccall.ml's pre_nvregs) marks NOTHING as callee-saved,
* because SPARC's register windows preserve registers across calls
* automatically, in hardware, with no software cooperation needed -
* so a->regs[FP_REG] is *always* NULL there, not just occasionally.
* Confirmed empirically: a temporary debug print in update_saved_regs
* showed its "for each saved register" loop never running at all for
* a tiger program's outermost C-- frame on SPARC (num_registers == 0).
* Treat "we don't know the C caller's frame pointer" as "nothing more
* to walk" (same outcome as the implausible-value case just below,
* and the same philosophy as Cmm_c_change_activation's own guard a
* few lines down) instead of asserting - a no-op for x86/ppc, since
* their assert never fired in practice, but the only thing that lets
* a windowed architecture's C-- programs finish a GC stack walk that
* runs off the end of the outermost C-- frame. */
if (a->regs[FP_REG] == NULL) {
    a->vfp = 0;
    a->u.cwalk.tries = 0;
    return;
}
fp = *a->regs[FP_REG];
/* claude: main (a foreign "C" C-- procedure) blanket-preserves FP_REG
* like any other callee-saved register, with no notion that it is
* "the" frame pointer - so what we recover here is genuinely just
* whatever FP_REG held when something called main, which is only a
* valid frame pointer if that something was ALSO compiled with frame
* pointers on (see Cmm_c_change_activation's big comment below for
* the precedent: modern glibc/crt usually is not). The x86 case
* happened to produce an obviously-wrong value (1, unaligned); on
* ppc it produced a plausible-looking one (r31 left holding main's
* own code address by whatever loaded it before the call) - aligned
* and non-null, so alignment alone doesn't catch it. A real caller
* frame has to live at or above our own frame's stack extent
* (stacks grow down), so bound-check against that instead. Treat
* "not a frame pointer" the same as "nothing more to walk": a->vfp=0
* makes the very next Cmm_c_change_activation call return 0 via its
* own existing guard. */
if (fp < (Cmm_Word) young_in_overflow) {
    a->vfp = 0;
} else {
    a->vfp = (char *) fp;
}
a->u.cwalk.tries = 0;    /* logging only */
}
extern struct cmm_activation_methods *Cmm_cmm_frame_methods;

```

The most important information in the ABI is in Chapter 3, “C Stack Frame,” page 3-42, Figure 3-47. (This information is slightly different in both placement and content from the draft ABI on the web.)

The ABI says that the caller's EDI, ESI, and EBX must be saved, and it even says where to look for them on the stack, but there is no specified way to determine whether they have actually been saved on the stack or simply left untouched. In any case, it is crystal clear that gcc does not conform to this part of the specification. For this reason, we can restore only EBP—not the other registers. If it needs access to variables held in registers, a C-- program will need to kill all registers, e.g., by using the “**paranoid C**” convention to call into the run-time system.

We identify the oldest activation by seeing a “caller's EBP” equal to zero. (This as claimed by the “Process Stack and Registers” part of the “Process Initialization” section of the “Operating-System Interface” chapter of the ABI, pages 3-28 and 3-29.)

```

#define NOISY 0

int Cmm_c_change_activation(Cmm_Activation *a) {
    Cmm_Word callerfp;
    Cmm_Codeptr ra;
    pc_map_entry *entry;
    int i;

    /* claude: this must come before the assert below, and must accept 0.
     *
     * a->vfp here is a frame-pointer value recovered from a C frame (see
     * FP_REG above), and the whole C walk assumes it chains: word 0 of the
     * frame is the caller's own frame pointer, ending in 0. That held when
     * this was written for x86, but modern glibc and crt are built with
     * -fomit-frame-pointer and use %ebp as an ordinary register, so whatever
     * it holds when main is called is not a frame pointer. What that is
     * varies by libc: 0x1 on this machine's glibc 2.39, 0 on the 2.35 in the
     * Docker image - and 0 tripped the assert, which is why the suite passed
     * here and failed there. (ppc's r31, by contrast, is the ABI's own
     * back-chain register whenever -fno-omit-frame-pointer is on - see
     * runtime/regenerate-ppc.sh - so this degenerate case shouldn't arise
     * there, but the same guard costs nothing to keep for both.)
     *
     * There is no way to walk a C stack that has no frame pointers, so an
     * implausible one ends the walk instead. A real frame pointer is a
     * non-null word-aligned stack address.
     *
     * For a C-- program called from main this loses nothing: the frames above
     * the outermost C-- one belong to crt and libc and hold no C-- data.
     */
    if (a->vfp == 0 || ((Cmm_Word)a->vfp & (sizeof(Cmm_Word) - 1)) != 0)
        return 0;

    assert(a->vfp != 0); /* protect against an unexpected error */
    <possibly shout about our departure, showing arguments 195a>
    callerfp = *(Cmm_Word *)a->vfp;
    <possibly announce caller's ebp 195b>
    if (callerfp == 0) {
        <possibly shout about finishing tries 195c>
        return 0;
    }
    for(i = 0; i < NUM_REGS; i++) /* registers cannot be restored */
        a->regs[i] = NULL;
    a->regs[FP_REG] = (Cmm_Word*)a->vfp; /* point to location of caller's fp */

    ra = ((Cmm_Codeptr *)a->vfp)[1];
    entry = Cmm_lookup_entry(ra);
    if (entry) { /* next frame is a C-- frame */
        if (Cmm_is_thread_start_frame(entry, ra))
            return 0; /* such a frame must not be seen */
        a->rtdata = entry;
        /* deallocation point = vfp + 8 (because caller deallocates args)
         deallocation point = callervfp + entry->outalloc

         so callervfp = vfp + 8 - entry->outalloc */
        a->vfp += 8 - Cmm_as_offset(entry->outalloc);
        a->pc = ra; /* for debugging only */
        a->methods = Cmm_cmm_frame_methods;
    } else {

```

```

    a->vfp = (char *)callerfp;
    a->pc = ra; /* for debugging only */
}
return 1;
}

```

<possibly shout about our departure, showing arguments 195a>≡ (194)

```

#if NOISY
{
    Cmm_Word *myfp = (Cmm_Word*) a->vfp;
    fprintf(stderr, "Leaving C activation; my fp = %p, ra = %p, caller's fp = %p\n",
        (void*)myfp, (void*)myfp[1], (void*)myfp[0]);
    for (i = 0; i < 3; i++) fprintf(stderr, " arg[%d] = 0x%08x\n", i, myfp[i+2]);
}
#endif

```

<possibly announce caller's ebp 195b>≡ (194)

```

#if 0
    fprintf(stderr, "walked to caller fp == 0x%08x\n", callerfp);
#endif

```

<possibly shout about finishing tries 195c>≡ (194)

```

#if NOISY
    fprintf(stderr, "finished C walk with callerfp = 0x%08x, tries = %d\n",
        (unsigned)callerfp, a->u.cwalk.tries);
#endif

```

15.5 x86cont

For `Cmm_MakeUnwindContinuation`, we need a continuation that restores the callee-saves registers and cuts to the unwind-target. The precondition here is that the SP points to an array of pointers to register values. There is one exception—the slot for the SP holds the SP value directly, not a pointer to that value.

```

<x86cont.s 195d>≡
.globl Cmm_unwindcont_pc
.section .data
__cmm_unwind_pc:
.long 0
.section .text
Cmm_unwindcont_pc:
    movl 0(%esp), %eax # grab the unwind pc
    movl %eax, __cmm_unwind_pc # move it to a well-known location (not thread-safe!)
    movl 16(%esp), %eax; movl (%eax), %eax # restore the registers (skipping %esp)
    movl 20(%esp), %ecx; movl (%ecx), %ecx
    movl 24(%esp), %edx; movl (%edx), %edx
    movl 28(%esp), %ebx; movl (%ebx), %ebx
    movl 36(%esp), %ebp; movl (%ebp), %ebp
    movl 40(%esp), %esi; movl (%esi), %esi
    movl 44(%esp), %edi; movl (%edi), %edi
    movl 32(%esp), %esp # restore %esp
    jmp *__cmm_unwind_pc

```

N.B. The code above is not thread-safe. The right solution is to pick a designated register as a scratch register for unwind continuations. This register should be used to hold the program counter. The compiler should mark the flow edge (from the call site to the unwind continuation) as killing the scratch register.

Chapter 16

Optimizations

16.1 Simple Optimization

If you are looking for real optimizations, they don't exist yet. But we hope to make some effort in that direction in the near future. For now, we have a few simple transformations of the control-flow graph.

16.1.1 The Optimizations

$\langle \text{optimize.mli } 196a \rangle \equiv$
 $\langle \text{optimize.mli } 196b \rangle$

We can simplify the expressions in the rtl's in a flow graph by calling `simplify_exps`.

$\langle \text{optimize.mli } 196b \rangle \equiv$ (196a) 196c $\triangleright$
val simplify_exps : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

It is possible to build a flow graph such that a node cannot be reached by following successors from the entry node, but it can be reached by following predecessors from the exit node. The nodes that are unreachable by following successor edges from the entry node are removed by `trim_unreachable_code`.

$\langle \text{optimize.mli } 196c \rangle + \equiv$ (196a) $\triangleleft 196b \ 196d \triangleright$
val trim_unreachable_code : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

It's also easy to collapse simple branch chains. We eliminate a branch to a join point if the branch is the only predecessor of the join point.

$\langle \text{optimize.mli } 196d \rangle + \equiv$ (196a) $\triangleleft 196c \ 196e \triangleright$
val collapse_branch_chains : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

Remove any instruction that leaves the machine in the same state in which it started. This includes not only `Rtl.null` but also assignments of the form `x := x`.

$\langle \text{optimize.mli } 196e \rangle + \equiv$ (196a) $\triangleleft 196d \ 196f \triangleright$
val remove_nops : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

Function `validate` does not actually do anything; it just checks the machine invariant.

$\langle \text{optimize.mli } 196f \rangle + \equiv$ (196a) $\triangleleft 196e$
val validate : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

val elim_dead_assignments : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

16.1.2 The Implementations

$\langle \text{optimize.ml } 196g \rangle \equiv$
 $\langle \text{optimize.ml } 197a \rangle$

We provide each of the optimization in turn.

```
<optimize.ml 197a>≡ (196g) 197b▷
module OG = Cfgx.M
module G = Zipcfg
module GR = Zipcfg.Rep
module DB = Dataflow.B
module P = Proc
module PA = Preast2ir
module RP = Rtl.Private
module RU = Rtlutil
module Dn = Rtl.Dn
module Up = Rtl.Up
module SS = Strutil.Set
module T = Target

(* claude: Nopoly.(==) below needs this open; every other .nw-tangled
 * .ml in this tree gets it prepended automatically by the (now-dropped)
 * generator, so it has to be added by hand here. *)
open Nopoly
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let not_null = function [] -> false | _ :: _ -> true
```

We traverse the flow graph and simplify rtl's.

```
<optimize.ml 197b>+≡ (196g) <197a 197c>
let simplify_exps _ (g, proc) =
  let changed = ref false in
  let g = G.map_rtls (fun r -> changed := true; Simplify.rtl r) g in
  (g, proc), !changed
```

A forward dfs followed by a backwards dfs will find nodes reachable from the entry (following successors) and nodes reachable from the exit (following predecessors). Since we want to catch loops, we can't just look for nodes that have no successors. Instead, we compare the nodes we can reach from the entry node with the nodes we can reach from the exit node. We break any loops among these nodes by setting the successor of each join node to the illegal node. Finally, we can delete any nodes that have no successors.

When we delete a node, we also need to remove any spans that are associated with the node. Because spans are associated with labels, we also collect the labels that are accessible from the entry node. Then we remove the unreachable spans. I DON'T SEE WHERE UNREACHABLE SPANS ARE EVER REMOVED. CAN THE OLD CODE SIMPLY BE REMOVED?

```
<optimize.ml 197c>+≡ (196g) <197b 198a>
let trim_unreachable_code _ {Procedure.cfg = cfg} =
  let module IS = Set.Make (struct type t = int let compare x y = x - y end) in
  let nodes_from_entry =
    let _add_labels n lset =
      if OG.is_join n then List.fold_right SS.add (OG.labels n) lset else lset in
    OG.postorder_dfs (fun n nset -> IS.add (OG.num n) nset)
    IS.empty cfg in
  let nodes_from_exit = OG.reverse_podfs (fun n rst -> n :: rst) [] cfg in
  let to_delete = List.filter (fun n -> not (IS.mem (OG.num n) nodes_from_entry))
    nodes_from_exit in
  let () = List.iter (fun n -> if OG.kind n == OG.Join then OG.set_succ cfg n (OG.illegal cfg))
    to_delete in
  let hanging_nodes =
    List.filter (fun n -> Pervasives.<> (OG.kind n) OG.Entry &&
      List.for_all (fun p -> OG.kind p == OG.Illegal) (OG.preds n)) in
  let rec delete_nodes = function
    | [] -> ()
    | n :: rst ->
      let succs = OG.succs n in
      (OG.delete cfg n;
```

```

    delete_nodes (hanging_nodes succs @ rst)) in
delete_nodes (hanging_nodes to_delete);
not_null to_delete

```

```

let trim_unreachable_code _ (g, proc) =
  let g' = G.postorder_dfs g in
  let changed = List.length g' < Unique.Map.size (G.to_blocks g) in
  (G.of_block_list g', proc), changed

```

We find each join point with one predecessor, and if the predecessor is a branch to the join point, we eliminate the branch and the join point. I THOUGHT A BRANCH CHAIN WAS A BRANCH TO A BRANCH?

```

<optimize.ml 198a>+≡ (196g) <197c 198b>
(* claude: ported to Zipcfg (the old version above operated on the
 * retired mutable Cfgx.M graph and was already dead code, shadowed by
 * a stub -- see docs/claude_notes/ for why). Since a [[Branch]]'s RTL is
 * baked in by the target's own [[goto]] embedder at construction time
 * (Zipcfg.branch), we cannot retarget a branch to a new label by editing
 * fields -- only the embedder that built it knows how to encode a jump.
 * Instead, for a chain of empty blocks that are nothing but an
 * unconditional branch, we chase the chain to the [[last]] of the final,
 * non-trivial hop and reuse that value verbatim (its RTL already jumps
 * straight to the real target) as the new [[last]] of every block earlier
 * in the chain. A cycle of such empty blocks (an infinite loop with no
 * side effects) is left alone rather than collapsed past, guarded by
 * [[seen]]. *)
let () = Debug.register "collapse-branch-chains"
      "count the branch hops Optimize.collapse_branch_chains removes"
let collapse_branch_chains _ (g, proc) =
  let blocks = G.to_blocks g in
  let mem_uid u l = List.exists (Unique.eq u) l in
  let rec chase uid seen =
    if mem_uid uid seen then None
    else
      match Unique.Map.find uid blocks with
      | (_, GR.Last (GR.Branch (_, (target, _)) as last)) ->
        (match chase target (uid :: seen) with
         | Some _ as deeper -> deeper
         | None -> Some (target, last))
      | _ -> None
  in
  let n = ref 0 in
  let blocks =
    Unique.Map.map (fun (first, tail) ->
      match tail with
      | GR.Last (GR.Branch (_, (target, _))) ->
        (match chase target [ GR.fid first ] with
         | Some (_, final_last) -> incr n; (first, GR.Last final_last)
         | None -> (first, tail))
      | _ -> (first, tail))
    blocks
  in
  if !n > 0 then Debug.eprintf "collapse-branch-chains" "collapsed %d branch(es)\n" !n;
  (G.of_blocks blocks, proc), !n > 0

```

```

<optimize.ml 198b>+≡ (196g) <198a 199>
type limit = { mutable lim : int }
let remove_nops _ (g, proc) =
  let changed = ref false in
  let is_nop rtl =
    let unneeded = function

```

```

| (_, RP.Kill _) -> true
| (_, RP.Store (l, RP.Fetch(l', w'), w)) -> (* assert (w=w'); *)
    if w<>w' then
        impossf "width of fetch and store don't match in %s" (RU.ToString.rtl rtl)
    else
        RU.Eq.loc l l'
| _ -> false in
let RP.Rtl effs = Dn.rtl rtl in
List.for_all unneeded effs in
let remaining = Tx.remaining() in
let tx = { lim = remaining } in
let block (f, t) =
    let rec finish t h = match t with
    | GR.Last _ -> GR.zipht h t
    | GR.Tail (m, t) when tx.lim > 0 && GR.is_executable m && is_nop (GR.mid_instr m) ->
        (changed := true; tx.lim <- tx.lim - 1; finish t h)
    | GR.Tail (m, t) -> finish t (GR.Head (h, m)) in
    finish t (GR.First f) in
let g = G.of_blocks (Unique.Map.map block (G.to_blocks g)) in
let () = if tx.lim < remaining then Tx.decrement "remove_nops" remaining tx.lim in
(g, proc), !changed

⟨optimize.ml 199⟩+≡ (196g) <198b
let badrtl r =
    Printf.eprintf "non-target RTL: %s\n" (RU.ToString.rtl r)
    (* if you want to do the following, extend Target.t with a to_asm function *)
    (* Printf.eprintf "x86 rec says %s\n" (X86rec.M.to_asm r) *)

let validate _ ((g, proc) : Ast2ir.proc) =
    let PA.T tgt = proc.target in
    let check r = if not (tgt.T.is_instruction r) then badrtl r else () in
    let first _ = () in
    let middle m = if GR.is_executable m then check (GR.mid_instr m) in
    let last l = check (GR.last_instr l) in
    G.iter_nodes first middle last g;
    (g, proc), false

(* claude: wraps Dead.elim_assignments (cfg/dataflow/dead.ml) the way
 * opti/peephole.ml's subst_forward wraps Availpass.analysis - combine the
 * transformation with the backward liveness analysis it needs
 * (Live.live_in) via Dataflow.B.a_t, then run it to a fixpoint with
 * Dataflow.B.rewrite. No entry_fact, unlike Peephole's forward passes:
 * a backward analysis starts from the exit, not the entry. *)
let elim_dead_assignments _ (g, proc) =
    let pass = DB.a_t Live.live_in Dead.elim_assignments in
    let g, changed = DB.rewrite pass g in
    (g, proc), changed

```

16.2 Peephole Optimization

The idea behind peephole optimization is simple. Suppose you have the following program:

```

t1 := 12
t2 := sp + t1

```

After the first assignment, you know as an “available expression” that `t1 == 12`. You can therefore do a forward substitution of 12 for `t1`, yielding


```
t1 := 12
t2 := sp + 12
```

With luck, the assignment to `t1` becomes dead, and a later dead-assignment elimination will eliminate it. Congratulations! You have a peephole optimizer.

This module implements a forward dataflow pass that does the forward substitution. It's very pessimistic about available expressions, assuming that they are empty at the start of each basic block. For cleaning up the output of the generic expander, this is just fine.

```
<peephole.mli 200a>≡
val subst_forward : 'a -> Ast2ir.proc -> Ast2ir.proc * bool
  (* composed; probably should be sequential *)

val sequential : 'a -> Ast2ir.proc -> Ast2ir.proc * bool
```

16.2.1 Implementation

```
<peephole.ml 200b>≡
  <peephole.ml 200c>

<peephole.ml 200c>≡ (200b) 200d▷
module D = Dataflow.F
module G = Zipcfg
module GR = Zipcfg.Rep
module P = Proc
module PA = Preast2ir
module RP = Rtl.Private
module RU = Rtlutil
module Dn = Rtl.Dn
module Up = Rtl.Up
module SS = Strutil.Set
module T = Target

module ToString = RU.ToString
<Peephole utilities 203a>

<peephole.ml 200d>+≡ (200b) <200c 200e▷
let peephole = "peephole" (* avoid allocating again and again... *)
let () = Debug.register peephole "peephole optimizer"
let debug = Debug.on peephole
```

This is a forward dataflow problem. We propagate dataflow facts and use these facts to mutate RTLs at nodes. Two key functions are wrapped for debugging:

- `ok` tests to see if an RTL is an instruction on the target
- `replace` produces the new version of an RTL

```
<peephole.ml 200e>+≡ (200b) <200d
let tx ((g, proc) : Ast2ir.proc) =
  let PA.T tgt = proc.target in
  let replace =
    if debug then
      (fun rtl old ->
        Printf.eprintf "Replacing old rtl %s\n" with new rtl %s\n"
          (ToString.rtl old) (ToString.rtl rtl);
        rtl)
    else
      (fun rtl _ -> rtl) in
```

```

let ok =
  if debug then
    (fun i -> if tgt.T.is_instruction i then true
      else (Printf.eprintf "### rejected substitution %s\n"
        (ToString.rtl i); false))
  else
    tgt.T.is_instruction in
  <propagation of available expressions and replacement of RTLs 201a>
  { D.name = "peephole"; D.middle_out = middle_out; D.last_outs = last_outs }

let _ = Debug.register "avail-dataflow" "computation of available expressions"

(* claudie: upstream's Backplane.of_dataflow (LUA/luacmm-driver/backplane.nw:899)
 * is just this lift from a graph rewrite to a procedure rewrite. This fork
 * dropped the rest of Backplane (see arch/x86/x86backend.ml's header
 * comment on why), so inline the one function actually needed here
 * instead of porting the whole module. *)
let of_dataflow graph_rewrite _ (g, proc) =
  let g, changed = graph_rewrite g in
  (g, proc), changed

let subst_forward v gproc =
  let pass = D.a_t Availpass.analysis (tx gproc) in
  let pass = if Debug.on "avail-dataflow" then D.debug Avail.to_string pass else pass in
  of_dataflow (D.rewrite pass ~entry_fact:Avail.unknown) v gproc

let sequential v (g,proc) =
  let pass = D.a_t Availpass.analysis (tx (g,proc)) in
  D.run_anal Availpass.analysis Avail.unknown g;
  of_dataflow (D.rewrite_solved pass ~entry_fact:Avail.unknown) v (g,proc)

```

In one pass, we simultaneously propagate dataflow information and mutate RTLs. The dataflow information is a set of available expressions, which is passed in as parameter `avail`. The propagation is shown here; the real work is done by `Avail.forward`. This code is very stupid; the moment it encounters any nontrivial control flow, it forgets all it knows and resumes with `Avail.empty`.

<propagation of available expressions and replacement of RTLs 201a>≡ (200e)
 <utility functions that substitute expressions for locations 202a>
 <code that may mutate the RTL at node 201b>

Here's the mutation code. The `update` function replaces an RTL, provided we haven't run out of transactions.
 <code that may mutate the RTL at node 201b>≡ (201a)

```

let search avail update rtl =
  <do our best to use update to replace rtl 202c>
  in
  let middle_out avail m = match m with
  | GR.Stack_adjust _ -> None
  | _ ->
    let rtl = GR.mid_instr m in
    if debug then
      Printf.eprintf "=> Considering %s\n" (ToString.rtl rtl);
    let update new_rtl = Some (G.single_middle (G.new_rtlm (replace new_rtl rtl) m)) in
    search avail update rtl in
  let last_outs avail l =
    let rtl = GR.last_instr l in
    if debug then
      Printf.eprintf "=> Considering %s\n" (ToString.rtl rtl);
    let update new_rtl = Some (G.single_last (G.new_rtl1 (replace new_rtl rtl) l)) in
    search avail update rtl in

```

How do we find a new RTL to pass to `update`? The basic idea is to substitute `Fetch(e)` for `loc` in the

existing `rtl`. According to Jack Davidson, when we are peephole optimizing we should try two substitutions, not just one. This is a bit like using a three-instruction window for peephole optimization instead of just a two-instruction window.

Instead of using single substitution twice, we define `subst2` to do two substitutions at once. This way we avoid allocating yet another copy of the RTL.

```
<utility functions that substitute expressions for locations 202a>≡ (201a) 202b▷
let slocs = RU.Subst.Fetch.rtl in
let subst1 (loc, e) rtl =
  Simplify.rtl (slocs ~guard:(RU.Eq.loc loc) ~fetch:(fun _ _ -> e) rtl) in
let subst2 (loc,e) (loc',e') rtl =
  Simplify.rtl (slocs ~guard:(fun l _ -> RU.Eq.loc l loc || RU.Eq.loc l loc')
    ~fetch:(fun l _ -> if RU.Eq.loc l loc then e else e') rtl) in
```

What substitutions will we try? We want locations that are fetched in the current RTL and that have known values in `avail`. For the time being, we're only interested in register locations, but if we eventually get good may-alias information, we will want to check memory locations as well.

Function `candidates` finds all `(loc, e)` pairs that might be suitable for substitutions.

```
<utility functions that substitute expressions for locations 202b>+≡ (201a) ◁202a 202e▷
let candidates avail rtl =
  let add loc pairs = match loc with
  | RP.Reg _ when not (List.mem_assoc loc pairs) -> (* don't make duplicates *)
    (match Avail.in_loc avail loc with
    | Some e -> (loc, e) :: pairs
    | None -> pairs)
  | _ -> pairs in
  RU.Fold.LocFetched.rtl add rtl [] in
```

Now we actually attempt the update. We use a backtracking search, which is implemented in continuation-passing style. The search repeatedly attempts first `try_pair` and then `try_one`. Each `try` function uses one or two (location, expression) pairs (named `le` or `le'`), substitutes in `rtl`, and checks the result to see if it is OK, that is, if it is representable on the target machine. If so, we are done. If not, we backtrack by tail-calling the `resume` continuation.

```
<do our best to use update to replace rtl 202c>≡ (201b) 202d▷
let try_pair le le' resume =
  let rtl' = subst2 le le' rtl in
  if ok rtl' then update rtl' else resume () in
let try_one le resume =
  let rtl' = subst1 le rtl in
  if ok rtl' then update rtl' else resume () in
```

To call these functions, we generate the list of all candidates, try all pairs, then try all single candidates. Functions `wrap2` and `wrap` add debugging information.

```
<do our best to use update to replace rtl 202d>+≡ (201b) ◁202c
let cs = candidates avail (Dn.rtl rtl) in
search_pairs cs (wrap2 try_pair) (fun () -> search cs (wrap try_one) (fun () -> None))
```

The search functions are the typical style with success and failure continuations. (Examples and explanations can be found in the undergraduate textbook by Ramsey and Kamin.)

```
<utility functions that substitute expressions for locations 202e>+≡ (201a) ◁202b
let rec search list succ fail = match list with
| [] -> fail()
| x :: xs -> succ x (fun () -> search xs succ fail) in

let rec search_pairs list succ fail = match list with
| [] -> fail ()
| x :: xs -> search xs (fun x' resume -> succ x x' resume)
    (fun () -> search_pairs xs succ fail) in
```

These wrapper functions help diagnose failures of the search for substitutions.

```

⟨Peephole utilities 203a⟩≡ (200c) 203b▷
let () =
  Debug.register "peephole-search"
    "search for a substitutable location in peephole optimizer"

let wrap =
  if Debug.on "peephole-search" then
    (fun succ l resume ->
      Printf.eprintf "Substituting for %s\n" (ToString.loc (Up.loc (fst l)));
      succ l (fun () ->
        (Printf.eprintf "Abandoning %s\n" (ToString.loc (Up.loc (fst l))); resume()))))
  else
    (fun succ -> succ)

let wrap2 =
  if Debug.on "peephole-search" then
    (fun succ l l' resume ->
      Printf.eprintf "Substituting for %s and %s\n" (ToString.loc (Up.loc (fst l)))
        (ToString.loc (Up.loc (fst l')));
      succ l l' (fun () ->
        (Printf.eprintf "Abandoning %s and %s\n" (ToString.loc (Up.loc (fst l)))
          (ToString.loc (Up.loc (fst l'))); resume()))))
  else
    (fun succ -> succ)

```

It's very often possible to peephole-optimize a copy instruction, but it's not clear if it's a good idea. This predicate could easily be used to turn off people optimization of copy instructions (the idea being that the register allocator should take care of them). A possible refinement would be to optimize only copy instructions between different register spaces.

```

⟨Peephole utilities 203b⟩+≡ (200c) <203a
let is_copy rtl = match Dn.rtl rtl with
| RP.Rtl [(RP.Const (RP.Bool true),
            RP.Store(RP.Reg _, RP.Fetch(RP.Reg _, _), _))] -> true
| _ -> false

```

16.3 Graph information

```

⟨dominator.mli 203c⟩≡
  ⟨dominator.mli 203e⟩

⟨dominator.ml 203d⟩≡
  ⟨dominator.ml 204a⟩

```

We don't want to force any particular representation of a graph so you have to provide some values in order to be able to use the dominator tree algorithm. Those values must be packages into a module that respect the following signature :

```

⟨dominator.mli 203e⟩≡ (203c) 204b▷
module type GRAPHINFO = sig

  type t
  type result
  val getNodeNumber : t -> int
  val getSuccs       : t -> int list array
  val getPreds       : t -> int list array
  val translate       : int array -> t -> result

end

```

The type `t` is the type of your graph. The value `getNodeNumber` must give the number of node in your graph. The value `getSuccs` must provide an array of interger list, each index of the array correspond to a node, each element of the array is the list of successors. The value `getPreds` is the same but for predecessors.

```

⟨dominator.ml 204a⟩≡ (203d) 204c▷
module type GRAPHINFO = sig

  type t
  type result
  val getNodeNumber : t -> int
  val getSuccs       : t -> int list array
  val getPreds       : t -> int list array
  val translate      : int array -> t -> result

end

```

16.4 The dominator tree interface

The dominator tree has the following interface :

```

⟨dominator.mli 204b⟩+≡ (203c) <203e 204e▷
module type DOMINATORTREE =
  functor (G : GRAPHINFO) -> sig

    val dominatorTree : G.t -> G.result

  end

```

```

⟨dominator.ml 204c⟩+≡ (203d) <204a 204d▷
module type DOMINATORTREE =
  functor (G : GRAPHINFO) -> sig

    val dominatorTree : G.t -> G.result

  end

```

```

⟨dominator.ml 204d⟩+≡ (203d) <204c 205a▷
type tree =
  | Leaf of Zipcfg.label option * Zipcfg.Rep.labelkind option
  | Node of Zipcfg.label option * Zipcfg.Rep.labelkind option * tree list

```

I need to declare stuff in the mli file

```

⟨dominator.mli 204e⟩+≡ (203c) <204b
type tree =
  | Leaf of Zipcfg.label option * Zipcfg.Rep.labelkind option
  | Node of Zipcfg.label option * Zipcfg.Rep.labelkind option * tree list
module ZipGraph : GRAPHINFO with type t = Zipcfg.graph
                                and type result = tree
module LengauerTarjan : DOMINATORTREE

(* claude: not part of TODO/dominator.nw - see the matching comment in
 * dominator.ml. Spliced into the tail of the existing "dominator.mli"
 * chunk (no new chunk of its own) since it postdates the original .nw. *)
module Query : sig
  type t = {
    blocks : Zipcfg.Rep.block array;
    idom    : int array;
    succs   : int list array;
    preds   : int list array;
  }

```

```

val analyze : Zipcfg.graph -> t
val dominates_idx : t -> int -> int -> bool

type loop = {
  header    : Zipcfg.Rep.block;
  body      : Zipcfg.Rep.block list;
  preheader : Zipcfg.Rep.block option;
}

val loops : t -> loop list
end

```

To get a usable `dominatorTree` value, you first need to apply the functor using your translation module that respect the signature `GRAPHINFO`.

Then the only value in the module takes your graph as an argument and gives an array of integers representing the dominator tree.

16.5 The Lengauer-Tarjan algorithm

The first Dominator module use the Lengauer-Tarjan algorithm described in the book : 'modern compiler implementation in ML' by Andrew W. Appel.

```

⟨dominator.ml 205a⟩+≡ (203d) <204d 205b>
module LengauerTarjan : DOMINORTREE =
  functor (G : GRAPHINFO) -> struct

```

First, we alias the module `Array` and the module `List` for convenience

```

⟨dominator.ml 205b⟩+≡ (203d) <205a 205c>
  module A = Array
  module L = List

```

Then we implement the `dominatorTree` value.

```

⟨dominator.ml 205c⟩+≡ (203d) <205b 205d>
  let dominatorTree graph =

```

We first take the information we need about the graph using the module `G`

```

⟨dominator.ml 205d⟩+≡ (203d) <205c 205e>
    let nodeNumber = G.getNodeNumber graph
    and succs =      G.getSuccs graph
    and preds =      G.getPreds graph in

```

And set all the structures we need for the algorithm

```

⟨dominator.ml 205e⟩+≡ (203d) <205d 205f>
  let vertex =      A.init nodeNumber (fun i -> -1)
  and parent =      A.init nodeNumber (fun i -> -1)
  and bucket =      A.init nodeNumber (fun i -> [])
  and dfnum =       A.init nodeNumber (fun i -> -1)
  and semi =        A.init nodeNumber (fun i -> -1)
  and ancestor =    A.init nodeNumber (fun i -> -1)
  and idom =        A.init nodeNumber (fun i -> -1)
  and samedom =     A.init nodeNumber (fun i -> -1)
  and counter =     ref 0 in

```

We need a value that number the nodes in a depth first search way and set the parent of each node according to the depth first search tree. We also keep track of the ranking of the nodes using `vertex`.

```

⟨dominator.ml 205f⟩+≡ (203d) <205e 206a>
(* claude: unvisited nodes are marked with dfnum = -1, not 0 - 0 is dfnum's
 * own root's *real* number, so testing "= 0" here would treat the root as
 * still-unvisited on any later edge back into it and re-run the whole dfs

```

```

* from there, corrupting vertex/parent/counter (TODO/dominator.nw bug). *)
let rec dfs p n =
  if A.get dfnum n = -1 then (
    A.set dfnum n !counter ;
    A.set vertex !counter n ;
    A.set parent n p ;
    counter := !counter + 1 ;
    L.iter (fun w -> dfs n w) (A.get succs n)
  ) in

```

We also need to know what is the ancestor that is the lowest semidominator of a particular node, say n. For this we follow the ancestors searching for the first one that a greater dfnum than n.

```

⟨dominator.ml 206a⟩+≡ (203d) <205f 206b>
let ancestorWithLowestSemi v =
  let u = ref v
  and v = ref v in
  while A.get ancestor !v != -1 do
    if A.get dfnum (A.get semi !v) < A.get dfnum (A.get semi !u)
    then u := !v ;
    v := A.get ancestor !v ;
  done ;
  !u

```

And the last utility value is link which does something really interesting and important.

```

⟨dominator.ml 206b⟩+≡ (203d) <206a 206c>
and link p n =
  A.set ancestor n p in

```

We finally compute the dominators.

```

⟨dominator.ml 206c⟩+≡ (203d) <206b 206d>
let dominators () =

```

We first number the graph from the root (-1 means that the root has no parent)

```

⟨dominator.ml 206d⟩+≡ (203d) <206c 206e>
dfs (-1) 0 ;

```

Now, for each dfnum : Get the node with the highest dfnum, it parent and declare a storage variable.

```

⟨dominator.ml 206e⟩+≡ (203d) <206d 206f>
for i = !counter - 1 downto 1 do

  let n = A.get vertex i in
  let p = A.get parent n in
  let s = ref p
  and s' = ref p in

```

Make the calculation of the semidominator of the node.

```

⟨dominator.ml 206f⟩+≡ (203d) <206e 207a>
(* claude: semi/bucket used to be set *inside* this L.iter, once per
 * predecessor instead of once after considering all of them - bucket
 * then got [[n]] added once per predecessor (duplicated, and into
 * whatever intermediate, not-yet-final [[s]] each iteration had) -
 * TODO/dominator.nw bug. Appel's algorithm sets both once, after the
 * loop over preds(n) has settled on the true minimum. *)
L.iter
  (fun v ->
    if dfnum.(v) <= dfnum.(n)
    then s' := v
    else s' := A.get semi (ancestorWithLowestSemi v) ;
    if A.get dfnum !s' < A.get dfnum !s then s := !s' ;
  ) (A.get preds n) ;
A.set semi n !s ;
A.set bucket !s (n :: A.get bucket !s) ;

```

Link the path from s to n into the forest.

```
<dominator.ml 207a>+≡ (203d) <206f 207b>
  link p n ;
```

And we are able to compute the dominators or defer the calculation until y dominator is known.

```
<dominator.ml 207b>+≡ (203d) <207a 207c>
  L.iter
    (fun v ->
      let y = ancestorWithLowestSemi v in
      if A.get semi y = A.get semi v
      then A.set idom v p
      else A.set samedom v y ;
    ) (A.get bucket p) ;

  A.set bucket p [] ;
```

This is it for a node.

```
<dominator.ml 207c>+≡ (203d) <207b 207d>
  done;
```

Once every node has been done, we can make the calculations of dominators that had been deferred earlier.

```
<dominator.ml 207d>+≡ (203d) <207c 207e>
  for i = 1 to !counter - 1 do

    let n = A.get vertex i in
    if A.get samedom n != -1
    then A.set idom n (A.get idom (A.get samedom n))

  done ; in
```

Now, we call this function and return the idom array that represents the dominator tree.

```
<dominator.ml 207e>+≡ (203d) <207d 207f>
  dominators () ;
  (* We should not have the graph appearing here ... *)
  G.translate idom graph

end
```

16.6 A translator module for the zipcfg

```
<dominator.ml 207f>+≡ (203d) <207e
(* claude: rewritten against the current Zipcfg (this file predates the
 * fork's reorg): [[Branch]]/[[Cbranch]]/[[Mbranch]] no longer carry
 * [[label option]]s (see zipcfg.mli), and label-list search with a
 * hand-rolled [[*=]] equality is no longer needed - [[Rep.succs]] already
 * gives a block's successor uids directly, and [[Rep.id]]/[[Unique.Map]]
 * give a proper uid -> index map instead of an O(n) position search. Also
 * fixed: [[getPreds]]'s "for i = 0 to max" was one past the end of a
 * [[max]]-length array (Invalid_argument on any non-trivial graph), and it
 * was O(n^2) besides; [[translate]]'s "find idom num" filtered idom's
 * *values* for ones equal to num, i.e. it returned [num; num; ...] (one
 * copy of num per child) instead of the children's own indices, so the
 * tree it built recursed into [[num]] again forever. *)
(* claude: kept at file level, not nested inside [[ZipGraph]], so that
 * [[Query]] further down can number a graph's blocks the exact same way
 * [[ZipGraph]] does - [[ZipGraph]]'s own signature ascription (below)
 * hides everything but [[GRAPHINFO]]'s four values, so nested helpers
 * would not be reachable from outside it. *)
```



```

(* claude: node 0 must be the entry block - LengauerTarjan's [[dfs]] starts
* numbering at node 0 and treats it as the tree's root. Despite the name,
* [[Zipcfg.postorder_dfs]] lists the entry *first*: its own local [[next]]
* only conses a node once every remaining sibling in [[children]] has
* been fully walked via CPS, and the outermost call's remaining-siblings
* list is what's left of *entry's* children - so entry's own cons is the
* last one to fire, and since each cons prepends to the accumulator
* threaded through every deeper call, "last to cons" means "ends up at
* the head of the final list". (A first version of this file assumed the
* opposite - "postorder" reads as "children before parent" - and
* reversed the list, which silently put some other node at index 0 and
* made every dominates_idx/back-edge query here wrong; verified against
* the actual block order by tracing a 2-node graph by hand, and against
* fold_layout's use of postorder_dfs, which only makes sense - laying
* a procedure's blocks out for emission - if the entry comes first.) *)
let blocks_of graph = Array.of_list (Zipcfg.postorder_dfs graph)

let index_of blocks =
  let n = Array.length blocks in
  let m = ref Unique.Map.empty in
  for i = 0 to n - 1 do m := Unique.Map.add (Zipcfg.Rep.id blocks.(i)) i !m done;
  !m

module ZipGraph : GRAPHINFO with type t = Zipcfg.graph
  and type result = tree = struct

  module A = Array
  module L = List
  module G = Zipcfg
  module B = Zipcfg.Rep

  type t = G.graph
  type result = tree

  let getNodeNumber graph = A.length (blocks_of graph)

  let getSuccs graph =
    let bs = blocks_of graph in
    let idx = index_of bs in
    let succs_of b =
      L.map (fun u -> Unique.Map.find u idx) (B.succs (B.last (B.unzip b))) in
    A.map succs_of bs

  let getPreds graph =
    let succs = getSuccs graph in
    let preds = A.make (A.length succs) [] in
    A.iteri (fun p ss -> L.iter (fun s -> preds.(s) <- p :: preds.(s)) ss) succs;
    preds

  let translate idom graph =
    let bs = blocks_of graph in
    let n = A.length bs in
    let name i = (B.blocklabel bs.(i), B.blockkind bs.(i)) in
    let children_of p =
      let acc = ref [] in
      for i = n - 1 downto 0 do
        if i <> p && idom.(i) = p then acc := i :: !acc
      done;
      !acc in
    let rec build i =

```

```

    let (lbl, kind) = name i in
    match children_of i with
    | [] -> Leaf (lbl, kind)
    | cs -> Node (lbl, kind, L.map build cs) in
  build 0
end

(* claude: not part of TODO/dominator.nw - a friendlier query layer on top
 * of [[ZipGraph]]/[[LengauerTarjan]] for consumers that want dominance
 * questions answered ("does A dominate B?", "what are this loop's back
 * edges?") rather than the label tree [[ZipGraph.translate]] builds, e.g.
 * opti/licm.ml and opti/strength_reduction.ml's natural-loop detection.
 * Spliced into the tail of the existing "dominator.ml" chunk (no new
 * chunk of its own) since it postdates the original .nw. *)
module Query = struct
  module A = Array
  module L = List
  module G = Zipcfg
  module B = Zipcfg.Rep
  module IS = Set.Make (struct type t = int let compare = compare end)

  (* Reuses ZipGraph's node numbering (getNodeNumber/getSuccs/getPreds) but
   * keeps the raw idom array instead of folding it into a [[tree]]. *)
  module IdomGraph = struct
    include ZipGraph
    type result = int array
    let translate idom (_graph : Zipcfg.graph) = idom
  end
  module Idom = LengauerTarjan (IdomGraph)

  type t = {
    blocks : B.block array;      (* index -> block, entry at index 0 *)
    idom    : int array;         (* idom.(i) = index of immediate dominator, -1 for the entry *)
    succs   : int list array;
    preds   : int list array;
  }

  let analyze graph =
    { blocks = blocks_of graph;
      idom   = Idom.dominatorTree graph;
      succs  = ZipGraph.getSuccs graph;
      preds  = ZipGraph.getPreds graph }

  (* does node [[a]] dominate node [[b]]? (every node dominates itself) *)
  let dominates_idx t a b =
    let rec walk i = i = a || (t.idom.(i) >= 0 && walk t.idom.(i)) in
    walk b

  (* A loop's [[body]] includes its [[header]]. [[preheader]] is set only
   * when the header has exactly one predecessor outside the loop, and
   * that predecessor has no other successor - i.e. control can only ever
   * reach the loop by falling straight through it, so an invariant
   * computation can be moved there verbatim with no new block and no
   * edge to redirect. Anything less tidy (a header reachable from
   * several places outside the loop, or a would-be preheader that also
   * branches elsewhere) is reported with [[preheader = None]]; it still
   * describes a real loop, just not one this simple scheme can hoist
   * into. *)
  type loop = {

```

```

    header      : B.block;
    body        : B.block list;
    preheader   : B.block option;
}

(* nodes that can reach [[n]] without going through [[h]], plus [[h]]
* itself - the standard natural-loop construction for a back edge
* n -> h with h dominating n (Aho/Sethi/Ullman, ch. 9). *)
let natural_loop t n h =
  let body = ref (IS.add h (IS.singleton n)) in
  let rec add = function
    | [] -> ()
    | m :: rest ->
        if IS.mem m !body then add rest
        else (body := IS.add m !body; add (t.preds.(m) @ rest))
  in
  add t.preds.(n);
  !body

let loops t =
  let back_edges = ref [] in
  A.iteri (fun n succs ->
    L.iter (fun h -> if dominates_idx t h n then back_edges := (n, h) :: !back_edges) succs)
    t.succs;
  let by_header = Hashtbl.create 16 in
  L.iter (fun (n, h) ->
    let prev = try Hashtbl.find by_header h with Not_found -> [] in
    Hashtbl.replace by_header h (n :: prev))
    !back_edges;
  Hashtbl.fold (fun h ns acc ->
    let body = L.fold_left (fun acc n -> IS.union acc (natural_loop t n h)) IS.empty ns in
    let external_preds = L.filter (fun p -> not (IS.mem p body)) t.preds.(h) in
    let preheader = match external_preds with
      | [p] when t.succs.(p) = [h] -> Some t.blocks.(p)
      | _ -> None
    in
    { header = t.blocks.(h);
      body = L.map (fun i -> t.blocks.(i)) (IS.elements body);
      preheader } :: acc)
    by_header []
end

```

Chapter 17

Advanced Topics

Chapter 18

Conclusion

Appendix A

Debugging

A.1 Diagnostic output

```
<debug.mli 213a>≡ (269b) 213b▷
  val on : string -> bool    (* debugging on for this word *)
  val eprintf : string -> ('a, out_channel, unit) format -> 'a
    (* if debugging on, print to stderr *)

<debug.mli 213b>+≡ (269b) <213a
  val explain : unit -> unit  (* print known words to stderr *)
  val register : word:string -> meaning:string -> unit

<debug.ml 213c>≡ (269a) 213d▷
  let debugging =
    try Str.split (Str.regexp ",") (Sys.getenv "QCDEBUG") with _ -> []
  let on s = List.mem s debugging

  let refs = ref ([] : (string * string) list)
  let register ~word ~meaning =
    let (<) : string -> string -> bool = Pervasives.< in
    let rec insert word meaning = function
      | [] -> [(word, meaning)]
      | ((w, m) as p) :: ps when word < w -> (word, meaning) :: p :: ps
      | p :: ps -> p :: insert word meaning ps in
    if List.mem_assoc word (!refs) then
      Impossible.impossible ("Debug.register: multiple registrations of " ^ word)
    else
      refs := insert word meaning (!refs)

  let rec ign x = Obj.magic ign
  let print_and_flush s = (prerr_string s; flush stderr)
  let eprintf s = if on s then Printf.kprintf print_and_flush else ign
    (* for reasons known only to INRIA, eprintf and kprintf have different types,
       so the above will not typecheck *)
  let eprintf s = if on s then Printf.eprintf else ign

<debug.ml 213d>+≡ (269a) <213c
  let explain () =
    Printf.eprintf "Words recognized in QCDEBUG:\n";
    List.iter (fun (w, e) -> Printf.eprintf "  %-15s  %s\n" w e) (!refs)
```

A.2 AST Pretty Printer

A.2.1 Interface

```
(astpp.mli 214a)≡ (270b)
val decl      : bool -> Ast.decl  -> Pp.doc
val stmt      : Ast.stmt         -> Pp.doc
val program   : Ast.toplevel list -> Pp.doc

val emit      : out_channel -> width:int -> Ast.toplevel list -> unit
```

A.2.2 Implementation

```
(astpp.ml 214b)≡ (270a) 214d▷
module A = Ast
module P = Pp
module E = Error
```

```
open Nopoly
```

```
let (^~)  = P.(^~)
let (^/)  = P.(^/)
let (^~) x = x
let nonempty = function [] -> false | _ :: _ -> true
```

```
(example(astpp.nw) 214c)≡
let f (x,y,z) =
  nest begin
    ~~ x
    ^/ y
    ^/ z
  end
```

```
(astpp.ml 214d)+≡ (270a) <214b 214e>
let nest      = P.nest 4
```

```
(astpp.ml 214e)+≡ (270a) <214d 214f>
let commablock f xs =
  nest begin
    ~~ P.list (P.text "," ^~ P.break) f xs
  end
```

```
(astpp.ml 214f)+≡ (270a) <214e 215a>
let angrp x =
  P.agrp begin
    ~~ nest begin
      ~~ x
    end
  end

let fngroup x =
  P.fgroup begin
    ~~ nest begin
      ~~ x
    end
  end
```

```

<astpp.ml 215a>+≡ (270a) <214f 215b>
  let unzip = List.split
  let zip   = List.combine

  let id i      = P.text i
  let int n     = P.text (string_of_int n)
  let semi      = P.text ";"

<astpp.ml 215b>+≡ (270a) <215a 215c>
  let comment x = P.vgrp (P.text "// " ^^ P.text x)

<astpp.ml 215c>+≡ (270a) <215b 215d>
  let str s     = P.text ("\"" ^ String.escaped s ^ "\"")
  let char i    = P.text ("'" ^ Char.escaped(Char.chr i) ^ "'")

<astpp.ml 215d>+≡ (270a) <215c 215e>
  let rec ty = function
    | A.T(x,_)      -> ty x
    | A.BitsTy(n)   -> P.text "bits" ^^ int n
    | A.TypeSynonym(name) -> P.text name

<astpp.ml 215e>+≡ (270a) <215d 215f>
  let aligned = function
    | Some i -> P.text " aligned" ^/ int i
    | None   -> P.empty

  let rec lvalue = function
    | A.NM(x,_) -> lvalue x
    | A.Name(None,x,a) -> P.agrp (id x ^^ aligned a)
    | A.Name(Some h,x,a) -> P.agrp (str h ^/ id x ^^ aligned a)
    | A.Mem(t,e,a,aliasing) ->
      ~~ ty t
      ^^ P.text "["
      ^^ expr e
      ^^ aligned a
      ^^ begin match aliasing with
        | [] -> P.empty
        | n :: ns ->
          let head = P.break ^^ P.text "in" ^/ P.text n in
          List.fold_left (fun h n -> h ^^ P.text "," ^/ P.text n) head ns
      end
      ^^ P.text "]"

<astpp.ml 215f>+≡ (270a) <215e 216a>
  and glvalue = function
    | x, None -> lvalue x
    | x, Some e -> P.agrp(lvalue x ^/ P.text "when" ^/ expr e)

  and actual = function
    | (Some hint, e, a) -> P.agrp (str hint ^/ expr e ^^ aligned a)
    | (None, e, a) -> P.agrp (expr e ^^ aligned a)

  and actuals xs = P.agrp (P.text "(" ^^ P.commalist actual xs ^^ P.text ")")

```



```

<astpp.ml 216a>+≡ (270a) <215f 216b>
and expr e =
  let with_ty t p = match t with None -> p | Some t -> p ^^ P.text ":@" ^^ ty t in
  match e with
  | A.E(x,_)      -> expr x
  | A.Sint( i, t)  -> with_ty t (P.text i)
  | A.Uint( i, t)  ->
    with_ty t (if String.get i 0 <= '0' then P.text i else P.text (i^"U"))
  | A.Float( f, t) -> with_ty t (P.text f)
  | A.Char( i, t)  -> with_ty t (char i)
  | A.Fetch(v)     -> lvalue v
  | A.BinOp(l,op,r) -> P.agrp begin
    ^^ P.text "(" ^^ expr l
    ^^ nest begin
    ^^ P.text op
    ^^ (if op = $= "%" then P.break else P.empty)
    ^^ P.breakWith ""
    ^^ expr r
    end
    ^^ P.text ")"
  end
  | A.UnOp(op,e)    -> P.agrp (P.text op ^^ expr e)
  | A.PrimOp(n,xs)  -> P.agrp (P.text "%" ^^ id n ^^ actuals xs)

```

```

<astpp.ml 216b>+≡ (270a) <216a 216c>
let memsize = function
  | A.NoSize      -> P.empty
  | A.DynSize     -> P.text "[]"
  | A.FixSize(e)  -> P.text "[" ^^ expr e ^^ P.text "]"

let rec init = function
  | A.I(x,_) -> init x
  | A.InitExprs(es) ->
    P.fgrp begin
    ^^ P.text "{"
    ^/ nest begin
    ^^ P.commalist expr es
    end
    ^/ P.text "}"
  end
  | A.InitStr(s) -> str s
  | A.InitUStr(s) ->
    P.agrp begin
    ^^ P.text "unicode"
    ^/ P.text "("
    ^/ nest begin
    ^^ str s
    end
    ^/ P.text ")"
  end

let rec datum = function
  | A.Da(x,_)      -> datum x
  | A.Label(n)     -> id n ^^ P.text ":"
  | A.Align(a)     -> P.agrp (P.text "align" ^/ int a ^^ semi)
  | A.MemDecl(t,m,Some i) -> P.agrp (ty t ^^ memsize m ^/ init i ^^ semi)
  | A.MemDecl(t,m,None)  -> P.agrp (ty t ^^ memsize m ^^ semi)

```

```

<astpp.ml 216c>+≡ (270a) <216b 217a>
let formal (_, (h, v, t, n, a)) =

```

```

P.agrp begin
~~ (match h with Some hint -> str hint ^^ P.break | None -> P.empty)
~~ (if v == A.Invariant then P.text "invariant" ^^ P.break else P.empty)
~~ ty t
~/ id n
^^ aligned a
end

let formals xs = P.agrp (P.text "(" ^^ P.commalist formal xs ^^ P.text ")")

<astpp.ml 217a>+≡ (270a) <216c 217b>
let cformal (_, h, n, a) =
  P.agrp begin
  ~~ (match h with Some hint -> str hint ^^ P.break | None -> P.empty)
  ^^ id n
  ^^ aligned a
  end

let cformals xs = P.agrp (P.text "(" ^^ P.commalist cformal xs ^^ P.text ")")

<astpp.ml 217b>+≡ (270a) <217a 217c>
let register is_global (v, hint, t, n, reg) =
  angrp begin
  ~~ (if v == A.Invariant then P.text "invariant" ^^ P.break else P.empty)
  ^^ (if is_global then P.text "register" ^^ P.break else P.empty)
  ^^ (match hint with Some h -> P.break ^^ str h | None -> P.empty)
  ^^ ty t
  ~/ id n
  ^^ (match reg with Some r -> P.break ^^ str r | None -> P.empty)
  ^^ semi
  end

<astpp.ml 217c>+≡ (270a) <217b 218a>
let altcont (e1,e2) =
  P.agrp begin
  ~~ P.text "<"
  ^^ expr e1
  ^^ P.text "/"
  ~/ expr e2
  ^^ P.text ">"
  end

let targets = function
| [] -> P.empty
| ts -> P.agrp (P.text "targets" ~/ nest (P.commalist id ts))

let rec flow f =
  let also s ns = P.agrp (P.text "also" ~/ P.text s ~/ P.text "to"
    ~/ nest (P.commalist id ns)) in
  match f with
  | A.Fl(x,_) -> flow x
  | A.CutsTo(ns) when nonempty ns -> also "cuts" ns
  | A.UnwindsTo(ns) when nonempty ns -> also "unwinds" ns
  | A.ReturnsTo(ns) when nonempty ns -> also "returns" ns
  | A.Aborts -> P.text "also aborts"
  | _ -> P.empty

let rec alias f =
  let ann s ns = P.agrp (P.text s ~/ nest (P.commalist id ns)) in
  match f with

```

```

| A.Al(x,_)      -> alias x
| A.Reads ns     -> ann "reads" ns
| A.Writes ns    -> ann "writes" ns

let pann = function
| A.Alias a -> alias a
| A.Flow f  -> flow f

let flows = function
| []      -> P.empty
| xs      -> nest (P.agrp (P.list P.break flow xs))

let panns = function
| []      -> P.empty
| xs      -> nest (P.agrp (P.list P.break pann xs))

let conv = function
| Some cc      -> P.agrp (P.text "foreign" ^/ str cc)
| None         -> P.empty

<astpp.ml 218a>+≡ (270a) <217c 218b>
let opt_ty t = ( match t with
| Some t -> P.break ^^ ty t
| None   -> P.empty
)

let export t ns =
let export' = function
| (x, Some y) -> P.agrp (id x ^/ P.text "as" ^/ str y)
| (x, None   ) -> id x
in
P.agrp begin
~~ P.agrp begin
~~ P.text "export"
^^ opt_ty t
end
^/ commablock export' ns
^^ semi
end

let architecture = function
| A.Memsize(i)      -> P.agrp (P.text "memsize" ^/ int i)
| A.ByteorderBig    -> P.agrp (P.text "byteorder" ^/ P.text "big")
| A.ByteorderLittle -> P.agrp (P.text "byteorder" ^/ P.text "little")
| A.WordSize i      -> P.agrp (P.text "wordsize" ^/ int i)
| A.PointerSize i    -> P.agrp (P.text "pointersize" ^/ int i)
| A.FloatRepr s      -> P.agrp (P.text "float" ^/ str s)
| A.Charset s        -> P.agrp (P.text "charset" ^/ str s)

let range = function
| A.Point(e)      -> expr e
| A.Range(e1,e2)  -> P.agrp(expr e1 ^/ P.text ".." ^/ expr e2)

<astpp.ml 218b>+≡ (270a) <218a 219a>
let rec stmt = function
| A.S(x,_)      -> stmt x

| A.IfStmt ( e, ss1, ss2)  ->

```

```

P.agrp begin
~~ P.agrp (P.text "if" ^/ expr e)
~/ P.block (body false) ss1
~~ begin match ss2 with
  | []      -> P.empty
  | ss      -> P.break ^^ P.text "else" ^/ P.block (body false) ss
end
end
| A.LabelStmt(n)          ->
  P.agrp begin
  ~~ id n ^^ P.text ":"
  end

| A.ContStmt(n,cf)        ->
  P.agrp begin
  ~~ P.text "continuation"
  ^^ angrp begin
    ~~ P.break
    ^^ id n
    ^^ cformals cf
    ^^ P.text ":"
  end
end

| A.SpanStmt(e1,e2,ss)    ->
  P.agrp begin
  ~~ P.agrp (P.text "span" ^/ expr e1 ^/ expr e2)
  ~/ P.block (body false) ss
end

```

<astpp.ml 219a>+≡ (270a) <218b 219b>

```

| A.AssignStmt(lhs,rhs)   ->
  let rec combine = function (* error tolerant *)
    | [] , []      -> []
    | x , []      -> []
    | [] , x      -> []
    | x::xx, y::yy -> (x,y)::combine (xx,yy)
                                in
  let guards, rhs = List.split rhs      in
  let lhs         = combine (lhs,guards) in
  P.agrp begin
  ~~ fngroup begin
    ~~ P.commalist glvalue lhs
    ~/ P.text "="
  end
  ^^ fngroup begin
    ~~ P.break
    ^^ P.commalist expr rhs
    ^^ semi
  end
end
end

```

<astpp.ml 219b>+≡ (270a) <219a 220>

```

| A.CallStmt(lhs, cc, e, args, ts, fs) ->
  angrp begin
  ~~ ( if nonempty lhs
    then fngroup begin
      ~~ P.commalist lvalue lhs
      ~/ P.text "="
      ^^ P.break
    end
  )
end

```

```

        end
    else P.empty
    )
^^ ( match cc with
  | Some c -> P.agrp (P.text "foreign" ^/ str c ^^ P.break)
  | None   -> P.empty
)
^^ angrp begin
  ^^ expr e
  ^^ fnggrp begin
    ^^ P.break
    ^^ P.text "("
    ^^ P.commalist actual args
    ^^ P.text ")"
  end
end
^^ (if nonempty ts then P.break ^^ targets ts else P.empty)
^^ (if nonempty fs then P.break ^^ panns fs else P.empty)
^^ semi
end

```

⟨astpp.ml 220⟩+≡ (270a) ◁219b 222▷

```

| A.PrimStmt(lhs, cc, n, args, fs)    ->
  angrp begin
  ^^ ( if nonempty lhs then
    ^^ P.commalist lvalue lhs
    ^/ P.text "="
    ^^ P.break
  else P.empty
  )
^^ ( match cc with
  | Some c -> P.agrp (P.text "foreign" ^/ str c ^^ P.break)
  | None   -> P.empty
)
^^ fnggrp begin
  ^^ P.text "%%"
  ^^ id n
  ^^ P.break
  ^^ P.text "("
  ^^ P.commalist actual args
  ^^ P.text ")"
end
^^ (if nonempty fs then P.break ^^ flows fs else P.empty)
^^ semi
end

| A.GotoStmt(e,ts)                    ->
  angrp begin
  ^^ P.text "goto"
  ^/ expr e
  ^^ (if nonempty ts then P.break ^^ targets ts else P.empty)
  ^^ semi
end

| A.CutStmt(e, args, fs)              ->
  angrp begin
  ^^ P.text "cut to"
  ^/ expr e
  ^^ fnggrp begin
    ^^ P.break
    ^^ P.text "("

```

```

    ^^ P.commalist actual args
    ^^ P.text ")"
    end
    ^^ (if nonempty fs then P.break ^^ flows fs else P.empty)
    ^^ semi
    end
| A.ReturnStmt(cc, alt, args)          ->
    angrp begin
    ^^ ( match cc with
        | Some c -> P.text "foreign" ^/ str c ^^ P.break
        | None   -> P.empty
        )
    ^^ P.text "return"
    ^^ ( match alt with
        | Some a -> P.break ^^ altcont a ^^ P.break
        | None   -> P.empty
        )
    ^^ fnggrp (P.break ^^ P.text "(" ^^ P.commalist actual args ^^ P.text ")")
    ^^ semi
    end
| A.JumpStmt(cc,e,args,ts)            ->
    angrp begin
    ^^ ( match cc with
        | Some c -> P.agrp (P.text "foreign" ^/ str c) ^^ P.break
        | None   -> P.empty
        )
    ^^ fnggrp begin
    ^^ (P.text "jump" ^/ expr e)
    ^^ ( if nonempty args then
        ^^ P.break
        ^^ P.text "("
        ^^ P.commalist actual args
        ^^ P.text ")"
        else
        P.empty
        )
    ^^ (if nonempty ts then P.break ^^ targets ts else P.empty)
    end
    ^^ semi
    end
| A.EmptyStmt          -> semi
| A.CommentStmt(s) -> comment s

| A.LimitcheckStmt (cookie, cont)    ->
    P.agrp begin
    ^^ P.agrp (P.text "limitcheck" ^/ expr cookie)
    ^/ begin match cont with
        | None   -> P.empty
        | Some e -> P.text "fails to" ^/ expr e
    end
    end
    end

| A.SwitchStmt (r,e,arms) ->
    P.agrp begin
    ^^ P.agrp begin
    ^^ P.text "switch"
    ^^ ( match r with
        | Some r -> P.agrp begin
            ^^ P.break
            ^^ P.text "["

```

```

        ^^ range r
        ^^ P.text "]"
      end
    | None    -> P.empty
  )
  ^/ expr e
end
~/ P.agrp (P.block arm arms)
end

and body is_global = function
| A.B(x, _)    -> body is_global x
| A.DeclBody(d) -> decl is_global d
| A.StmtBody(s) -> stmt s
| A.DataBody(dd) ->
    P.agrp begin
    ^^ P.text "stackdata"
    ^/ P.block datum dd
    end

and proc (cc,n,fs,ss,_) =
  P.agrp begin
  ^^ P.agrp begin
    ^^ begin match cc with
    | Some c -> P.text "foreign" ^/ str c ^^ P.break
    | None   -> P.empty
    end
    ^^ id n
    ^^ formals fs
    end
  ^/ P.text "{"
  ^^ nest begin
    ^^ P.break
    ^^ P.list P.break (body false) ss
    end
  ^/ P.text "}"
end

```

<astpp.ml 222>+≡ (270a) <220 223a>

```

and decl is_global = function
| A.D(x,_) -> decl is_global x
| A.Import( t, ns) ->
  let import' = function
  | (Some x, y) -> P.agrp (str x ^/ P.text "as" ^/ P.text y)
  | (None  , y) -> id y
  in
    P.agrp begin
    ^^ P.agrp (P.text "import" ^/ opt_ty t)
    ^/ commablock import' ns
    ^^ semi
    end
| A.Export( t, ns) -> export t ns
| A.Const (t,n,e) ->
  P.agrp begin
  ^^ P.text "const"
  ^^ ( match t with
    | Some t -> P.break ^^ ty t
    | None   -> P.empty
    )
  ^/ id n

```

```

        ~/ P.text "="
        ~/ expr e
        ^^ semi
    end
| A.Registers( rs) ->
    P.vgrp begin
    ^^ P.list P.break (register is_global) rs
    end
| A.Typedef (t,nn) ->
    angrp begin
    ^^ P.text "typedef"
    ~/ ty t
    ~/ P.text "="
    ~/ commablock id nn
    ^^ semi
    end

| A.Target (arch) ->
    angrp begin
    ^^ P.text "target"
    ~/ P.list P.break architecture arch
    ^^ semi
    end
| A.Pragma          -> comment "pragma"

<astpp.ml 223a>+≡ (270a) <222 223b>
and arm = function
| A.ArmAt(x,_)      -> arm x
| A.Case(ranges, stmts) ->
    P.agrp begin
    ^^ P.text "case"
    ~/ P.agrp (P.list (P.text "," ^^ P.break) range ranges)
    ^^ P.text ":"
    ~/ P.block (body false) stmts
    end

and section = function (* inside a section *)
| A.Sec(x,_) -> section x
| A.Decl(d)   -> decl true d
| A.Datum( d) -> datum d
| A.Procedure(p) -> proc p
| A.SSpan( e1, e2, ss) ->
    P.agrp begin
    ^^ P.agrp (P.text "span" ~/ expr e1 ~/ expr e2)
    ~/ P.block section ss
    end

<astpp.ml 223b>+≡ (270a) <223a 224>
let rec toplevel = function
| A.Top(x, _) -> toplevel x
| A.Section(name, ss) ->
    P.agrp begin
    ^^ P.agrp (P.text "section" ~/ str name)
    ~/ P.block section ss
    end
| A.TopDecl(d) -> decl true d
| A.TopProcedure(p) -> proc p

let program ds = P.vgrp begin
    ^^ P.list (P.break ^^ P.break) toplevel ds

```



```

        ^^ P.break
    end

    let pp = program

<astpp.ml 224>+≡ (270a) <223b
    let emit fd ~width tl =
        List.iter (fun t -> ( P.ppToFile fd width (toplevel t)
                               ; output_string fd "\n\n"
                               )) tl

```

Appendix B

Profiling

Appendix C

Error Management

C.1 Error Handling

C.1.1 Interface

```
(error.mli 226a)≡ (269d) 226b▷
  type 'a error    = Error          (* bad result *)
                    | Ok             of 'a      (* good result *)

  [@@deriving show]

  exception ErrorExn of string
  val error :        string -> 'a (* ErrorExn *)
  val errorf : ('a, unit, string, 'b) format4 -> 'a

(error.mli 226b)+≡ (269d) <226a 226c>
  val combine : 'a error error -> 'a error
  val ematch  : 'a error -> ('a -> 'b) -> 'b error
  val ematch2 : 'a error -> 'b error -> ('a -> 'b -> 'c) -> 'c error
  val ematch3 : 'a error -> 'b error -> 'c error ->
    ('a -> 'b -> 'c -> 'd) -> 'd error
  val ematch4 : 'a error -> 'b error -> 'c error -> 'd error ->
    ('a -> 'b -> 'c -> 'd -> 'e) -> 'e error
  val ematch5 : 'a error -> 'b error -> 'c error -> 'd error -> 'e error ->
    ('a -> 'b -> 'c -> 'd -> 'e -> 'f) -> 'f error
  val ematch6 : 'a error -> 'b error -> 'c error -> 'd error -> 'e error -> 'f error ->
    ('a -> 'b -> 'c -> 'd -> 'e -> 'f -> 'g) -> 'g error
  val ematchPair  : 'a error * 'b error -> ('a * 'b -> 'c) -> 'c error
  val ematchTriple: 'a error * 'b error * 'c error ->
    ('a * 'b * 'c -> 'd) -> 'd error
  val ematchQuad  : 'a error * 'b error * 'c error * 'd error ->
    ('a * 'b * 'c * 'd -> 'e) -> 'e error
  val seq         : 'a error -> ('a -> 'b error) -> 'b error
  val seq2        : 'a error -> 'b error -> ('a -> 'b -> 'c error) -> 'c error
  val seqPair     : 'a error * 'b error -> ('a * 'b -> 'c error) -> 'c error
  val seq'        : 'b -> 'a error -> ('a -> 'b) -> 'b

(error.mli 226c)+≡ (269d) <226b 227a>
  module Raise :
    sig
      val option : 'a error option -> 'a option error
      val list   : 'a error list  -> 'a list error
      val pair   : 'a error * 'b error -> ('a * 'b) error
```

```

<error.mli 227a>+≡ (269d) <226c 227b>
  val left   : 'a error * 'b          -> ('a * 'b) error
  val right  : 'a * 'b error          -> ('a * 'b) error

  val triple : 'a error * 'b error * 'c error -> ('a * 'b * 'c) error
  val quad   : 'a error * 'b error * 'c error * 'd error
                -> ('a * 'b * 'c * 'd) error

end

<error.mli 227b>+≡ (269d) <227a 227c>
  module Lower :
    sig
      val pair   : ('a * 'b)          error -> 'a error * 'b error
      val triple : ('a * 'b * 'c)      error -> 'a error * 'b error * 'c error
      val quad   : ('a * 'b * 'c * 'd) error -> 'a error
                                                * 'b error
                                                * 'c error
                                                * 'd error
    end

end

<error.mli 227c>+≡ (269d) <227b 227d>
  module Implode :
    sig
      val singleton : 'a error          -> unit error
      val pair      : 'a error * 'b error -> unit error
      val triple    : 'a error * 'b error * 'c error -> unit error
      val quad      : 'a error * 'b error * 'c error * 'd error -> unit error
      val list      : 'a error list      -> unit error
      val map       : ('a -> 'b error) -> 'a list      -> unit error
    end

end

<error.mli 227d>+≡ (269d) <227c 227e>
  val catch   : (string -> unit)
                -> ('a -> 'b error) -> 'a -> 'b error
  val catch'  : 'b -> (string -> unit)
                -> ('a -> 'b      ) -> 'a -> 'b

end

<error.mli 227e>+≡ (269d) <227d 227f>
  val warningPrt : string -> unit
  val errorPrt   : string -> unit

end

<error.mli 227f>+≡ (269d) <227e
  val errorPointPrt : Srcmap.point -> string -> unit
  val errorRegionPrt : Srcmap.region -> string -> unit
  val errorPosPrt    : Srcmap.pos -> string -> unit
  val errorRegPrt    : Srcmap.rgn -> string -> unit

```

C.1.2 Implementation

```

<error.ml 227g>≡ (269c) 227h>
  type 'a error = Error (* bad result *)
                  | Ok   of 'a (* good result *)

  [@@deriving show { with_path = false }]

  exception ErrorExn of string

end

<error.ml 227h>+≡ (269c) <227g 228h>
  module Raise = struct
    <raise module 228a>
  end

```

```

⟨raise module 228a⟩≡ (227h) 228b▷
  let option = function
    | Some Error      -> Error
    | Some (Ok x)     -> Ok (Some x)
    | None            -> Ok (None)

⟨raise module 228b⟩+≡ (227h) ◁228a 228c▷
  let list xs =
    let rec loop acc = function
      | []           -> Ok (List.rev acc)
      | (Ok x)::xs   -> loop (x::acc) xs
      | Error::xs    -> Error
    in
      loop [] xs

⟨raise module 228c⟩+≡ (227h) ◁228b 228d▷
  let pair = function
    | (Ok x, Ok y)     -> Ok (x,y)
    | _               -> Error

⟨raise module 228d⟩+≡ (227h) ◁228c 228e▷
  let left = function
    | (Ok x, y)        -> Ok (x,y)
    | _               -> Error

⟨raise module 228e⟩+≡ (227h) ◁228d 228f▷
  let right = function
    | (x, Ok y)        -> Ok (x,y)
    | _               -> Error

⟨raise module 228f⟩+≡ (227h) ◁228e 228g▷
  let triple = function
    | (Ok x, Ok y, Ok z) -> Ok (x,y,z)
    | _               -> Error

⟨raise module 228g⟩+≡ (227h) ◁228f
  let quad = function
    | (Ok a,Ok b,Ok c,Ok d) -> Ok (a,b,c,d)
    | _               -> Error

⟨error.ml 228h⟩+≡ (269c) ◁227h 229a▷
  module Lower = struct
    ⟨lower module 228i⟩
  end

⟨lower module 228i⟩≡ (228h) 228j▷
  let pair = function
    | Ok (x,y)         -> Ok x , Ok y
    | Error            -> Error, Error

⟨lower module 228j⟩+≡ (228h) ◁228i 228k▷
  let triple = function
    | Ok (x,y,z)       -> Ok x , Ok y , Ok z
    | Error            -> Error, Error, Error

⟨lower module 228k⟩+≡ (228h) ◁228j
  let quad = function
    | Ok (a,b,c,d)     -> Ok a , Ok b , Ok c , Ok d
    | Error            -> Error, Error, Error, Error

```

```

⟨error.ml 229a⟩+≡ (269c) ◁228h 229h▷
  module Implode = struct
    ⟨implode module 229b⟩
  end

⟨implode module 229b⟩≡ (229a) 229c▷
  let singleton = function
    | Ok _ -> Ok ()
    | _ -> Error

⟨implode module 229c⟩+≡ (229a) ◁229b 229d▷
  let pair = function
    | Ok _, Ok _ -> Ok ()
    | _ -> Error

⟨implode module 229d⟩+≡ (229a) ◁229c 229e▷
  let triple = function
    | Ok _, Ok _, Ok _ -> Ok ()
    | _ -> Error

⟨implode module 229e⟩+≡ (229a) ◁229d 229f▷
  let quad = function
    | Ok _, Ok _, Ok _, Ok _ -> Ok ()
    | _ -> Error

⟨implode module 229f⟩+≡ (229a) ◁229e 229g▷
  let rec list = function
    | [] -> Ok ()
    | (Ok _)::xs -> list xs
    | _ -> Error

⟨implode module 229g⟩+≡ (229a) ◁229f
  let map f l =
    let ok = Ok () in
    let rec loop res = function
      | [] -> res
      | x::xs -> ( match res, f x with
                    | Error, _ -> loop Error xs
                    | _ , Error -> loop Error xs
                    | _ -> loop ok xs
                  )
    in
      loop ok l

⟨error.ml 229h⟩+≡ (269c) ◁229a 229i▷
  let combine = function
    | Ok x -> x
    | Error -> Error

⟨error.ml 229i⟩+≡ (269c) ◁229h 230a▷
  let ematch x f = match x with Ok x -> Ok (f x) | Error -> Error
  let ematch2 x y f =
    match x with Ok x -> (match y with Ok y -> Ok (f x y) | _ -> Error) | _ -> Error
  let ematch3 x y z f = match x with Ok x -> ematch2 y z (f x) | Error -> Error
  let ematch4 x y z w f = match x with Ok x -> ematch3 y z w (f x) | Error -> Error
  let ematch5 x y z w v f = match x with Ok x -> ematch4 y z w v (f x) | Error -> Error
  let ematch6 x y z w v u f =
    match x with Ok x -> ematch5 y z w v u (f x) | Error -> Error
  let ematchPair x2 = ematch (Raise.pair x2)
  let ematchTriple x3 = ematch (Raise.triple x3)
  let ematchQuad x4 = ematch (Raise.quad x4)

```

```

⟨error.ml 230a⟩+≡ (269c) <229i 230b>
  let seq x f = match x with Ok x -> f x | Error -> Error
  let seq2 x y f =
    match x with Ok x -> (match y with Ok y -> f x y | _ -> Error) | _ -> Error
  let seq' default x f = match x with Ok x -> f x | Error -> default
  let seqPair x2 = seq (Raise.pair x2)

```

```

⟨error.ml 230b⟩+≡ (269c) <230a 230c>
  let catch' default printer f arg =
    try f arg with
    | ErrorExn(msg) -> ( printer msg
                        ; default
                      )

```

```

⟨error.ml 230c⟩+≡ (269c) <230b 230d>
  let catch printer f arg = catch' Error printer f arg
  let error msg = raise (ErrorExn msg)
  let errorf fmt = Printf.kprintf error fmt

```

```

⟨error.ml 230d⟩+≡ (269c) <230c 230e>
  let warningPrt msg = Printf.eprintf "Warning: %s\n" msg
  let errorPrt msg = Printf.eprintf "Error: %s\n" msg

```

```

⟨error.ml 230e⟩+≡ (269c) <230d 230f>
  let errorPointPrt p msg =
    ( Printf.eprintf "%s " (Srcmap.Str.point p)
    ; errorPrt msg
    )

```

```

⟨error.ml 230f⟩+≡ (269c) <230e 230g>
  let errorRegionPrt r msg =
    ( Printf.eprintf "%s " (Srcmap.Str.region r)
    ; errorPrt msg
    )

```

```

⟨error.ml 230g⟩+≡ (269c) <230f 230h>
  let errorPosPrt p msg =
    ( Printf.eprintf "Character %d " p
    ; errorPrt msg
    )

```

```

⟨error.ml 230h⟩+≡ (269c) <230g>
  let errorRegPrt (l,r) msg =
    ( Printf.eprintf "Character %d-%d " l r
    ; errorPrt msg
    )

```

C.2 Impossible

```

⟨impossible.mli 230i⟩≡ (269f)
  val impossible: string -> 'a
  val unimp: string -> 'a

```

C.2.1 Implementation

```
<impossible.ml 231a>≡ (269e)
exception Impossible of string

let impossible msg =
  prerr_endline ("This can't happen: " ^ msg);
  raise (Impossible msg)
let unimp msg =
  prerr_endline ("Not implemented in qc--: " ^ msg);
  raise (Impossible msg)
```

C.3 Announcing unsupported constructs

```
<unsupported.mli 231b>≡ (269j)
exception Unsupported
val explain : int -> int list -> unit
```

(* Each function below always raises Unsupported *)

```
val widen_float : int -> 'a
val stack_width : have:int -> want:int -> 'a
val calling_convention : string -> 'a
val automaton_widths : int -> 'a
val automaton_widen : have:int -> want:int -> 'a
val div_and_mod : unit -> 'a
val div_overflows : unit -> 'a
val floatlit : int -> 'a
val mulx_and_mlux : unit -> 'a
val singlebit : op:string -> 'a
val popcnt : notok:int -> ok:int -> 'a
```

```
<unsupported.ml 231c>≡ (269i) 231d▷
exception Unsupported
type state = { mutable n : int; mutable explanations : (int list -> string list) list }
let state = { n = 0; explanations = [] }
```

```
let getn f =
  state.n <- state.n + 1;
  let n = state.n in
  state.explanations <- f :: state.explanations;
  n
```

```
<unsupported.ml 231d>+≡ (269i) <231c 231e▷
let fail msg code args =
  (* let binname = Sys.argv.(0) in *)
  let binname = "qc--" in
  let args = String.concat " " (List.map string_of_int (code :: args)) in
  Printf.eprintf "%s\n" msg;
  Printf.eprintf "For a longer explanation run\n %s -e 'Unsupported.explain(%s)'\n"
    binname args;
  raise Unsupported
```

```
<unsupported.ml 231e>+≡ (269i) <231d 232a▷
let explain n ns =
  let rec ex m l = match l with
  | [] -> Printf.printf "There is no explanation numbered %d\n" n
  | x :: xs ->
    if m = n then
```



```

        List.iter (Printf.printf "%s\n") (x ns)
    else
        ex (m-1) xs
    in ex state.n state.explanations

<unsupported.ml 232a>+≡ (269i) <231e 232b>
let exs ss = (fun _ -> ss)
let ex0 f = function
| [] -> f()
| _ ->
    ["For this explanation code, Unsupported.explain expects exactly one argument"]
let ex1 f = function
| [n] -> f n
| _ ->
    ["For this explanation code, Unsupported.explain expects exactly two arguments"]
let ex2 f = function
| [n; m] -> f n m
| _ ->
    ["For this explanation code, Unsupported.explain expects exactly three arguments"]

<unsupported.ml 232b>+≡ (269i) <232a 232c>
let s = Printf.sprintf
let nosupport s = "This back end does not support " ^ s

<unsupported.ml 232c>+≡ (269i) <232b 232d>
let widen_n = getn (ex1 (fun n ->
    [ s "On this target, floating-point operations can be at most %d bits wide." n;
      "Narrower operations can be widened using one of the stages in the Widen";
      s "module, but anything wider than %d bits is unsupported." n;
    ])))

let widen_float d =
    fail (nosupport (s "%d-bit floating-point computation" d)) widen_n [d]

<unsupported.ml 232d>+≡ (269i) <232c 232e>
let stack_width_n = getn (ex2 (fun h w ->
    [ s "Your code tried to pass a %d-bit argument to an operator that takes its" h;
      s "arguments on the machine stack, but on this target, only %d-bit values can" w;
      "go on the machine stack.";
      "(This message could be triggered if you tried to use software rounding modes";
      "instead of hardware rounding modes in a floating-point instruction on the";
      s "Pentium, or if you tried to use something other than %d-bit floats.)" w;
    ])))
let stack_width ~have ~want =
    fail (nosupport (s "%d-bit value on the machine stack" have))
        stack_width_n [have; want]

<unsupported.ml 232e>+≡ (269i) <232d 233a>
let calling_convention_n = getn (ex0 (fun _ ->
    [ "Your code tried to make a procedure call, return, or cut to using an"
      ; "unsupported calling convention. Please verify the correct spelling of the"
      ; "calling convention or get a wizard to extend the compiler with this"
      ; "calling convention."
    ])))
let calling_convention name =
    fail (nosupport (s "the '%s' calling convention" name))
        calling_convention_n []

```

```

<unsupported.ml 233a>+≡ (269i) <232e 233b>
  let automaton_widths_n = getn (ex1 (fun x ->
    [ s "Your code tried to pass a %d-bit value as a parameter or return" x
      ; "value to/from a procedure although it was not supported by the calling"
      ; "convention being used for the call/return/cut to."
    ]))
  let automaton_widths w =
    fail (s "The current calling convention could not support a %d-bit value" w)
          automaton_widths_n [w]

  let automaton_widen_n = getn (ex2 (fun have want ->
    [ s "Your code tried to pass a %d-bit value as a parameter or return" have
      ; "value to/from a procedure although the calling convention being"
      ; s "used expected a value %d-bits wide or potentially narrower." want
    ]))
  let automaton_widen ~have ~want =
    fail (s "The current calling convention either could not widen a " ^
      s "%d-bit value to %d-bits or fit it in a %d-bit register"
      have want want)
          automaton_widen_n [ have ; want ]

<unsupported.ml 233b>+≡ (269i) <233a 233c>
  let div_and_mod_n = getn (fun _ ->
    ["On this target, signed division and modulus round toward zero.";
     "You must use the operators %quot and %rem, not %div and %mod.";
    ])

  let div_and_mod () = fail (nosupport "%div and %mod") div_and_mod_n []

  let div_overflows_n = getn (fun _ ->
    ["This code generator cannot test for %div_overflows and %quot_overflows"])

  let div_overflows () =
    fail (nosupport "overflow detection for division") div_overflows_n []

<unsupported.ml 233c>+≡ (269i) <233b 233d>
  let mulx_and_mulux_n = getn (fun _ ->
    ["This target does not support extended integer-multiply operators."])
  )

  let mulx_and_mulux () = fail (nosupport "%mulx and %mulux") mulx_and_mulux_n []

<unsupported.ml 233d>+≡ (269i) <233c 233e>
  let floatlit = getn (ex1 (fun n ->
    [ "On this target, floating-point literals can be either 32 or 64 bits wide.";
      s "You asked for a literal of %d bits. You could try %f2f%d(<literal>)." n n;
    ]))

  let floatlit d =
    fail (nosupport (s "%d-bit floating-point literal" d)) floatlit [d]

<unsupported.ml 233e>+≡ (269i) <233d 234>
  let singlebit_n = getn (fun _ ->
    ["This target does not support extended addition and subtraction using";
     "single-bit values with the %addc, %carry, %subb, and %borrow operators.";
    ])
  let singlebit ~op =
    fail (nosupport (s "the %s operator" op)) singlebit_n []

```

```

<(unsupported.ml 234)+≡ (269i) <233e
let popcnt_n = getn (ex2 (fun good bad ->
  [ s "Your code tried to use the %%popcnt operator on a %d-bit argument," bad;
    s "but on this target, %%popcnt applies only to %d-bit values." good;
  ]))
let popcnt ~notok ~ok =
  fail (nosupport (s "%%popcnt(bits%d)" notok))
  popcnt_n [ok; notok]

```

Appendix D

Utilities

Appendix E

Mini stdlib

Appendix F

Extra Code

Appendix G

prelude

G.1 prelude/auxfunns.nw

$\langle \text{commons2/auxfunns.ml } 238a \rangle \equiv$
 $\langle \text{auxfunns.ml } 239b \rangle$

$\langle \text{commons2/auxfunns.mli } 238b \rangle \equiv$
 $\langle \text{auxfunns.mli } 238c \rangle$

G.2 Auxiliaries

$\langle \text{auxfunns.mli } 238c \rangle \equiv$ (238b) 239a $\triangleright$

```
val round_up_to : multiple_of:int -> int -> int
  (* round_up_to n k rounds k up to the nearest multiple of n.
     n must be positive and k must be nonnegative *)

val foldri : (int -> 'a -> 'b -> 'b) -> 'a list -> 'b -> 'b
  (* fold list elements right to left, passing index for List.nth *)

val map_partial : ('a -> 'b option) -> 'a list -> 'b list

val last : 'a list -> 'a (* raises Invalid_argument on empty list *)

val from: int -> upto:int -> int list
  (* from x ~upto:y is the list of integers x, ..., y *)

val compare_list : ('a -> 'a -> int) -> 'a list -> 'a list -> int

module List : sig
  val take : int -> 'a list -> 'a list
    (* take the first n elements of a list, or if there are fewer
       than n elements, take the whole list (viva Haskell!) *)
end

module Option : sig
  val is_some : 'a option -> bool
  val is_none : 'a option -> bool
  val get: 'a -> 'a option -> 'a
  val map : ('a -> 'b) -> ('a option -> 'b option)
end

module String : sig
  val foldr : (char -> 'a -> 'a) -> string -> 'a -> 'a
```

end

type void = Void of void (* used as placeholder for polymorphic variable *)

<auxfuns.mli 239a>+≡ (238b) <238c
val substr: int -> int -> string -> string

G.2.1 Implementation

<auxfuns.ml 239b>≡ (238a) 239c>
type void = Void of void (* used as placeholder for polymorphic variable *)

```
module Option = struct
  let is_some = function Some _ -> true | None -> false
  let is_none = function Some _ -> false | None -> true
  let get x = function
    | Some y -> y
    | None -> x
  let map f = function Some x -> Some (f x) | None -> None
end
```

<auxfuns.ml 239c>+≡ (238a) <239b 239d>
let round_up_to ~multiple_of:n k = n * ((k+(n-1)) / n)

<auxfuns.ml 239d>+≡ (238a) <239c 239e>
let foldri f l z =
 let rec next n = function
 | [] -> z
 | x :: xs -> f n x (next (n+1) xs) in
 next 0 l

```
let rec from first ~upto =  
  if first > upto then [] else first :: from (first+1) ~upto
```

```
let substr start stop str =  
  let start = if start < 0 then String.length str + start else start in  
  let stop = if stop <= 0 then String.length str + stop else stop in  
  String.sub str start (stop - start)
```

<auxfuns.ml 239e>+≡ (238a) <239d 239f>
module String = struct
 let foldr f s z =
 let rec down_from n z =
 if n < 0 then z else down_from (n-1) (f (String.get s n) z) in
 down_from (String.length s - 1) z
end

<auxfuns.ml 239f>+≡ (238a) <239e 239g>
let rec last = function
 | [] -> raise (Invalid_argument "empty list")
 | [x] -> x
 | x :: xs -> last xs

<auxfuns.ml 239g>+≡ (238a) <239f 240a>
let rec map_partial f = function
 | [] -> []
 | x :: xs ->
 match f x with
 | Some y -> y :: map_partial f xs
 | None -> map_partial f xs


```

⟨auxfuns.ml 240a⟩+≡ (238a) ◁239g 240b▷
  let rec compare_list cmp x y = match x, y with
  | [], [] -> 0
  | [], _ :: _ -> -1
  | _ :: _, [] -> 1
  | x :: xs, y :: ys ->
      match cmp x y with
      | 0 -> compare_list cmp xs ys
      | diff -> diff

```

```

⟨auxfuns.ml 240b⟩+≡ (238a) ◁240a
  module List = struct
    let rec take n xs = match xs with
    | [] -> []
    | _ :: _ when n = 0 -> []
    | x :: xs -> x :: take (n-1) xs
  end

```

G.3 prelude/pp.nw

```

⟨commons2/pp.ml 240c⟩≡
  ⟨pp.ml 241i⟩

```

```

⟨commons2/pp.mli 240d⟩≡
  ⟨pp.mli 240e⟩

```

G.4 Pretty Printer

G.4.1 Interface

```

⟨pp.mli 240e⟩≡ (240d) 240f▷
  type doc

```

```

⟨pp.mli 240f⟩+≡ (240d) ◁240e 240g▷
  val empty : doc

```

```

⟨pp.mli 240g⟩+≡ (240d) ◁240f 240h▷
  val (^) : doc -> doc -> doc

```

```

⟨pp.mli 240h⟩+≡ (240d) ◁240g 240i▷
  val text : string -> doc

```

```

⟨pp.mli 240i⟩+≡ (240d) ◁240h 240j▷
  val break : doc

```

```

⟨pp.mli 240j⟩+≡ (240d) ◁240i 240k▷
  val breakWith : string -> doc

```

```

⟨pp.mli 240k⟩+≡ (240d) ◁240j 241a▷
  val nest : int -> doc -> doc

```

$\langle \text{pp.mli } 241a \rangle + \equiv$ (240d) $\triangleleft 240k \ 241b \triangleright$
`val hgrp : doc -> doc`

$\langle \text{pp.mli } 241b \rangle + \equiv$ (240d) $\triangleleft 241a \ 241c \triangleright$
`val vgrp : doc -> doc`

$\langle \text{pp.mli } 241c \rangle + \equiv$ (240d) $\triangleleft 241b \ 241d \triangleright$
`val agrp : doc -> doc`

$\langle \text{pp.mli } 241d \rangle + \equiv$ (240d) $\triangleleft 241c \ 241e \triangleright$
`val fgrp : doc -> doc`

$\langle \text{pp.mli } 241e \rangle + \equiv$ (240d) $\triangleleft 241d \ 241f \triangleright$
`val ppToString : int -> doc -> string`
`val ppToFile : out_channel -> int -> doc -> unit`

$\langle \text{pp.mli } 241f \rangle + \equiv$ (240d) $\triangleleft 241e \ 241g \triangleright$
`val list : doc -> ('a -> doc) -> 'a list -> doc`
`val commalist : ('a -> doc) -> 'a list -> doc`

$\langle \text{pp.mli } 241g \rangle + \equiv$ (240d) $\triangleleft 241f \ 241h \triangleright$
`val (^/) : doc -> doc -> doc`

$\langle \text{pp.mli } 241h \rangle + \equiv$ (240d) $\triangleleft 241g$
`val block : ('a -> doc) -> 'a list -> doc`

G.4.2 Implementation

$\langle \text{pp.ml } 241i \rangle \equiv$ (240c) $242a \triangleright$
 $\langle \text{auxiliaries}(\text{pp.nw}) \ 244a \rangle$
 $\langle \text{gmode } 241j \rangle$

```

type doc =
  | DocNil
  | DocCons      of doc * doc
  | DocText      of string
  | DocNest      of int * doc
  | DocBreak     of string
  | DocGroup     of gmode * doc

```

$\langle \text{gmode } 241j \rangle \equiv$ (241i)

```

type gmode =
  | GFlat      (* hgrp *)
  | GBreak     (* vgrp *)
  | GFill      (* fgrp *)
  | GAuto      (* agrp *)

```

<pp.ml 242a>+≡ (240c) <241i 242b>

```

let (^^) x y      = DocCons(x,y)
let empty         = DocNil
let text s        = DocText(s)
let nest i x      = DocNest(i,x)
let break         = DocBreak(" ")
let breakWith s   = DocBreak(s)

let hgrp d        = DocGroup(GFlat, d)
let vgrp d        = DocGroup(GBreak,d)
let agrp d        = if debug
                     then DocGroup(GAuto, text "[" ^^ d ^^ text "]")
                     else DocGroup(GAuto, d)

let fgrp d        = if debug
                     then DocGroup(GFill, text "{" ^^ d ^^ text "}")
                     else DocGroup(GFill, d)

```

<pp.ml 242b>+≡ (240c) <242a 242d>

```

type sdock =
  | SNil
  | SText      of string * sdock
  | SLine      of int      * sdock    (* newline + spaces *)

```

<pp.ml original 242c>≡

```

let rec sdockToString = function
  | SNil          -> ""
  | SText(s,d)    -> s ^ sdockToString d
  | SLine(i,d)    -> let prefix = String.make i ' '
                     in  nl ^ prefix ^ sdockToString d

```

<pp.ml 242d>+≡ (240c) <242b 243a>

```

let sdockToString sdock =
  let buf = Buffer.create 256 in
  let rec loop = function
    | SNil          -> ()
    | SText(s,d)    -> ( Buffer.add_string buf s
                        ; loop d
                        )
    | SLine(i,d)    -> let prefix = String.make i ' ' in
                        ( Buffer.add_char   buf '\n'
                        ; Buffer.add_string buf prefix
                        ; loop d
                        )
  in

```

```

  ( loop sdock
    ; Buffer.contents buf
    )

```

```

let sdockToFile oc doc =
  let pstr = output_string oc in
  let rec loop = function
    | SNil          -> ()
    | SText(s,d)    -> pstr s; loop d
    | SLine(i,d)    -> let prefix = String.make i ' '
                        in  pstr nl;
                           pstr prefix;
                           loop d

```

```
in
  loop doc
```

```
<pp.ml 243a>+≡ (240c) <242d 243b>
  type mode =
    | Flat
    | Break
    | Fill
```

```
<pp.ml 243b>+≡ (240c) <243a 243d>
  <fits 243c>
```

```
(* format is cps to avoid stack overflow *)
let cons s post z = post (SText (s, z))
let consl i post z = post (SLine (i, z))
let rec format w k l post = match l with
| [] -> post SNil
| (i,m,DocNil) :: z -> format w k z post
| (i,m,DocCons(x,y)) :: z -> format w k ((i,m,x)::(i,m,y)::z) post
| (i,m,DocNest(j,x)) :: z -> format w k ((i+j,m,x)::z) post
| (i,m,DocText(s)) :: z -> format w (k + strlen s) z (cons s post)
| (i,Flat, DocBreak(s)) :: z -> format w (k + strlen s) z (cons s post)
| (i,Fill, DocBreak(s)) :: z -> let l = strlen s in
                                if fits (w - k - l) z
                                then format w (k+l) z (cons s post)
                                else format w i z (consl i post)
| (i,Break,DocBreak(s)) :: z -> format w i z (consl i post)
| (i,m,DocGroup(GFlat ,x)) :: z -> format w k ((i,Flat ,x)::z) post
| (i,m,DocGroup(GFill ,x)) :: z -> format w k ((i,Fill ,x)::z) post
| (i,m,DocGroup(GBreak,x)) :: z -> format w k ((i,Break,x)::z) post
| (i,m,DocGroup(GAuto, x)) :: z -> if fits (w-k) ((i,Flat,x)::z)
                                then format w k ((i,Flat ,x)::z) post
                                else format w k ((i,Break,x)::z) post
```

```
<fits 243c>≡ (243b)
  let rec fits w = function
    | _ when w < 0 -> false
    | [] -> true
    | (i,m,DocNil) :: z -> fits w z
    | (i,m,DocCons(x,y)) :: z -> fits w ((i,m,x)::(i,m,y)::z)
    | (i,m,DocNest(j,x)) :: z -> fits w ((i+j,m,x)::z)
    | (i,m,DocText(s)) :: z -> fits (w - strlen s) z
    | (i,Flat, DocBreak(s)) :: z -> fits (w - strlen s) z
    | (i,Fill, DocBreak(_)) :: z -> true
    | (i,Break,DocBreak(_)) :: z -> true
    | (i,m,DocGroup(_,x)) :: z -> fits w ((i,Flat,x)::z)
```

```
<pp.ml 243d>+≡ (240c) <243b 243e>
  let ppToString w doc = format w 0 [0,Flat,agrp(doc)] sdocToString
  let ppToFile oc w doc = format w 0 [0,Flat,agrp(doc)] (sdocToFile oc)
```

```
<pp.ml 243e>+≡ (240c) <243d>
  let rec list sep f xs =
    let rec loop acc = function
      | [] -> acc
      | [x] -> acc ^^ f x
      | x::xs -> loop (acc ^^ f x ^^ sep) xs
```

```

in
  loop empty xs

let commalist f = list (text "," ^^ break) f

let (^/) x y    = x ^^ break ^^ y
let (~~) x      = x

let block f xs =
  text "{"
  ^^ nest 4 begin
    ~~ break
    ^^ list break f xs
  end
  ^/ text "}"

```

```

⟨auxiliaries(pp.nw) 244a⟩≡ (241i) 244b▷
  let debug    = false
  let strlen   = String.length

```

```

⟨auxiliaries(pp.nw) 244b⟩+≡ (241i) ◁244a
  let nl       = "\n"

```

G.5 prelude/strutil.nw

G.6 String Utilities

```

⟨strutil.mli 244c⟩≡
(* old:
module Set: Set.S with type elt = string
module Map: Map.S with type key = string
*)

module Set : sig
  include Set.S with type elt = string
  val pp : Format.formatter -> t -> unit
end
module Map : sig
  include Map.S with type key = string
  val pp : (Format.formatter -> 'a -> unit) -> Format.formatter -> 'a t -> unit
end

val assoc2map: (string * 'a) list -> 'a Map.t
val from_list: string list -> Set.t

```

G.7 Implementation

```

⟨strutil.ml 244d⟩≡
  let compares (x:string) (y:string) = compare x y

  module Compare = struct type t = string let compare=compares end

  (* old:

```

```

module Set = Set.Make(Compare)
module Map = Map.Make(Compare)
*)

module Set = struct
  include Set.Make(Compare)

  let pp fmt s =
    Format.fprintf fmt "{%a}"
      (Format.pp_print_list
        ~pp_sep:(fun fmt () -> Format.fprintf fmt ", ")
        (fun fmt s -> Format.fprintf fmt "%S" s))
        (elements s))
end

module Map = struct
  include Map.Make(Compare)

  let pp pp_value fmt m =
    Format.fprintf fmt "@[<v 2>{";
    List.iter
      (fun (k, v) ->
        Format.fprintf fmt "@,";
        Format.fprintf fmt "@[<hov 0>%S -> %a@" k pp_value v;
        Format.fprintf fmt ";")
        (bindings m);
    Format.fprintf fmt "@]@,}"
end

let assoc2map pairs =
  let f map (key,value) = Map.add key value map in
  List.fold_left f Map.empty pairs

let from_list xs = List.fold_left (fun set x -> Set.add x set) Set.empty xs

```

G.8 prelude/verbose.nw

G.9 Diagnostic output

```

<verbose.mli 245a>≡
val say : int -> string list -> unit
  (* if VERBOSITY >= k, then say k l writes every string in l to stderr *)
val eprintf : int -> ('a, out_channel, unit) format -> 'a
  (* if VERBOSITY >= k, then say k l writes every string in l to stderr *)
val verbosity : int (* current verbosity *)

```

```

<verbose.ml 245b>≡
let verbosity = try int_of_string (Sys.getenv "VERBOSITY") with _ -> 0
let err l = List.iter prerr_string l; flush stderr
let say k = if verbosity >= k then err else ignore

let rec ign x = Obj.magic ign
let eprintf k = if verbosity >= k then Printf.eprintf else ign

```

Appendix H

bitops/

H.1 bitops/alignment.nw

H.2 Interface to manipulate alignment constraints

```
<alignment.mli 246a>≡  
  type t  
  
  val init   : int -> t  
  
  val add    : int -> t -> t  
  val align  : int -> t -> t  
  
  val alignment : t -> int  
  
  val gcd : int -> int -> int  
    (* will disappear when Alignment.t becomes RTL.assertion *)
```

H.3 Implementation of alignment manipulation

```
<alignment.ml 246b>≡  
  type t = int * int  
  
  let init k = (0, k)  
  let add i (n, k) = (n + i, k)  
  let align k (n', k') = (0, k)  
  let rec gcd n m =  
    if n > m then gcd m n  
    else if n = 0 then m  
    else gcd (m - n) n  
  let alignment (n, k) = gcd n k
```

H.4 commons3/bits.nw

```
<commons3/bits.ml 246c>≡  
  <bits.ml 248g>  
  
<commons3/bits.mli 246d>≡  
  <bits.mli 247a>
```

H.5 Bits – Low-Level Representation of Values

H.5.1 Interface

```
⟨bits.mli 247a⟩≡ (246d) 247b▷  
  type width = int  
  
  (* pad: this was previously an abstract type, but it's less convenient  
   * with ocamldebug. update: less of a problem with deriving show?  
   *)  
  type bits = Int64.t * int  
  [@@deriving show]  
  
  type t = bits  
  [@@deriving show]  
  
  val width : t -> width      (* observer *)  
  val zero  : width -> t      (* constructor *)  
  val is_zero : t -> bool  
  
⟨bits.mli 247b⟩+≡ (246d) ◁247a 247c▷  
  val compare : t -> t -> int  
  val eq      : t -> t -> bool  
  
⟨bits.mli 247c⟩+≡ (246d) ◁247b 247d▷  
  exception Overflow  
  module S: sig (* signed *)  
    ⟨interface S 248a⟩  
  end  
  module U: sig (* unsigned *)  
    ⟨interface U 248d⟩  
  end  
  
⟨bits.mli 247d⟩+≡ (246d) ◁247c 247e▷  
  val to_string      : bits -> string  
  val to_decimal_string : bits -> string  
  val to_hex_or_decimal_string : declimit:int -> bits -> string
```

H.5.2 Arithmetic

```
⟨bits.mli 247e⟩+≡ (246d) ◁247d  
  module Ops : sig  
    val add      : bits -> bits -> bits  
    val and'     : bits -> bits -> bits  
    val com      : bits -> bits  
    val divu     : bits -> bits -> bits  
    val mul      : bits -> bits -> bits  
    val neg      : bits -> bits  
    val or'      : bits -> bits -> bits  
    val sub      : bits -> bits -> bits  
    val shra     : bits -> bits -> bits  
    val shr1     : bits -> bits -> bits  
    val shl      : bits -> bits -> bits  
    val xor      : bits -> bits -> bits  
  
    val sx       : int  -> bits -> bits  
    val zx       : int  -> bits -> bits  
    val lobits   : int  -> bits -> bits
```



```

val eq      : bits -> bits -> bool
val ne      : bits -> bits -> bool
val lt      : bits -> bits -> bool
val gt      : bits -> bits -> bool
val ltu     : bits -> bits -> bool
val gtu     : bits -> bits -> bool
end

```

H.5.3 Signed Constructors

```

⟨interface S 248a⟩≡ (247c) 248b▷
  val of_int:    int          -> width -> t    (* raises Overflow *)
  val of_native: nativeint    -> width -> t    (* raises Overflow *)
  val of_int32:  int32        -> width -> t    (* raises Overflow *)
  val of_int64:  int64        -> width -> t    (* raises Overflow *)
  val of_string: string       -> width -> t    (* raises Overflow *)

```

H.5.4 Signed Observers

```

⟨interface S 248b⟩+≡ (247c) ◁248a 248c▷
  val to_int:    t -> int          (* Overflow *)
  val to_native: t -> nativeint     (* Overflow *)
  val to_int64:  t -> int64        (* Overflow *)

⟨interface S 248c⟩+≡ (247c) ◁248b
  val fits : width -> t -> bool    (* fits w b == (b = sxlo w b) *)

```

H.5.5 Unsigned Constructors

```

⟨interface U 248d⟩≡ (247c) 248e▷
  val of_int:    int          -> width -> t
  val of_native: nativeint    -> width -> t
  val of_int64:  int64        -> width -> t
  val of_int32:  int32        -> width -> t
  val of_string: string       -> width -> t    (* raises Overflow *)

```

H.5.6 Unsigned Observers

```

⟨interface U 248e⟩+≡ (247c) ◁248d 248f▷
  val to_int:    t -> int          (* Overflow *)
  val to_native: t -> nativeint     (* Overflow *)
  val to_int64:  t -> int64        (* Overflow *)

⟨interface U 248f⟩+≡ (247c) ◁248e
  val fits : width -> t -> bool    (* fits w b == (b = zxlo w b) *)

```

H.5.7 Implementation

```

⟨bits.ml 248g⟩≡ (246c) 249a▷
  let (==) = (=)
  let (<=) (x:char) (y:char) = (=) x y

module I = Int64      (* signed operations *)
exception Overflow
let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

```

⟨bits.ml 249a⟩+≡ (246c) ◁248g 249b▷
  let (<=) = Pervasives.<=)
  let (>=) = Pervasives.>=)
  let (<) = Pervasives.<)
  let (>) = Pervasives.>)

```

```

⟨bits.ml 249b⟩+≡ (246c) ◁249a 249c▷

```

```

type width      = int
[@@deriving show]
type bits       = int64 * width
[@@deriving show]
type t          = bits
[@@deriving show]

let width (_,w) = w
let check w = if w <= 0 || w > 64 then impossf "unsupported bitwidth %d" w
let zero w = check w; (I.zero, w)

```

H.5.8 Sign and zero extension

```

⟨bits.ml 249c⟩+≡ (246c) ◁249b 249d▷
  let sx64 i w =
    if w = 64 then i
    else
      let w' = 64-w in
      I.shift_right (I.shift_left i w') w'

```

```

⟨bits.ml 249d⟩+≡ (246c) ◁249c 249e▷
  let zx64 i w =
    if w = 64 then i
    else
      let w' = 64-w in
      I.shift_right_logical (I.shift_left i w') w'

```

H.5.9 Strings

```

⟨bits.ml 249e⟩+≡ (246c) ◁249d 249f▷
  let to_string (i,w) = Printf.sprintf "0x%Lx::bits%d" i w
  let to_decimal_string (i,w) = Printf.sprintf "%Ld" i
  let to_hex_or_decimal_string ~declimit (i,w) =
    assert (declimit >= 0);
    if UInt64.lt i (I.of_int declimit) then
      Printf.sprintf "%Ld" i
    else
      Printf.sprintf "0x%Lx" (zx64 i w)

```

H.5.10 Arithmetic

```

⟨bits.ml 249f⟩+≡ (246c) ◁249e 251a▷
  module Ops = struct
    ⟨bitwise operators 250⟩
  end
  let eq (b, w) (b', w') = (w = w') && (sx64 b w) == (sx64 b' w)
  let compare (b, w) (b', w') =
    match compare w w' with
    | 0 -> Pervasives.compare (sx64 b w) (sx64 b' w)
    | diff -> diff
  let is_zero (b, w) = zx64 b w == I.zero

```

(bitwise operators 250)≡

(249f)

```
let add (b, w) (b', w') =
  assert (w = w');
  (Int64.add (sx64 b w) (sx64 b' w), w)

let eq (b, w) (b', w') =
  assert (w = w');
  (sx64 b w) == (sx64 b' w)

let lt (b, w) (b', w') =
  assert (w = w');
  let n = sx64 b w in
  let n' = sx64 b' w' in
  Pervasives.< n n'
let gt x y = lt y x

let ltu (b, w) (b', w') =
  assert (w = w');
  let n = zx64 b w in
  let n' = zx64 b' w' in
  UInt64.lt n n'
let gtu x y = ltu y x

let neg (b,w) = (Int64.neg (sx64 b w), w)
  (* THIS SX SHOULD BE UNNECESSARY -- CONSULT KEVIN *)

let sub (b,w) (b', w') =
  assert (w = w');
  let n = sx64 b w in
  let n' = sx64 b' w' in
  (Int64.sub n n', w)

let mul (b,w) (b', w') =
  assert (w = w');
  let n = sx64 b w in
  let n' = sx64 b' w' in
  (Int64.mul n n', w)

let divu (b,w) (b', w') =
  assert (w = w');
  let n = zx64 b w in
  let n' = zx64 b' w' in
  (UInt64.div n n', w)

let eq (b,w) (b',w') =
  assert (w = w');
  let n = zx64 b w in
  let n' = zx64 b' w' in
  n == n'

let ne x y = not (eq x y)

let sx k (b,w) =
  assert (k >= w);
  let n = sx64 b w in
  (n ,k)

let zx k (b,w) =
  assert (k >= w);
  let n = zx64 b w in
```

```

    (n, k)

let lobits k (b, w) =
  if k <= w then (sx64 b k, k)
  else (Printf.eprintf "lobits error: k: %d, w: %d\n" k w;
        Impossible.impossible "lobits: k > w")

let shra (b,w) (b',w') =
  (* todo: pad: bug ? _n is unused ? *)
  let _n = zx64 b w in
    (Int64.shift_right b (Int64.to_int b'), w)

let shr1 (b,w) (b',w') =
  (Int64.shift_right_logical b (Int64.to_int b'), w)

let shl (b,w) (b',w') =
  (zx64 (Int64.shift_left b (Int64.to_int b')) w, w)

let or' (b,w) (b',w') =
  assert (w = w');
  (zx64 (Int64.logor b b') w, w)

let and' (b,w) (b',w') =
  assert (w = w');
  (zx64 (Int64.logand b b') w, w)

let xor (b,w) (b',w') =
  assert (w = w');
  (zx64 (Int64.logxor b b') w, w)

let com (b,w) = (Int64.lognot (zx64 b w), w)

⟨bits.ml 251a⟩+≡ (246c) <249f
  ⟨string scanning 253⟩
  module S = struct
    ⟨implementation S 251b⟩
  end

  module U = struct
    ⟨implementation U 252a⟩
  end

```

H.5.11 Signed Interpretation

```

⟨implementation S 251b⟩≡ (251a)
  let fits_signed i w =
    w = 64 or
    let i' = I.shift_right i (w-1) in i' == I.zero || i' == I.minus_one

  let fits w' (i, w) = w <= w' || fits_signed i w'

  let of_int i w =
    check w;
    let i' = I.of_int i in
      if fits_signed i' w then (i',w) else raise Overflow

  let of_int32 i w =

```

```

    check w;
    let i' = I.of_int32 i in
    if fits_signed i' w then (i',w) else raise Overflow

let of_native i w =
    check w;
    let i' = I.of_nativeint i in
    if fits_signed i' w then (i',w) else raise Overflow

let of_int64 i w =
    check w;
    if fits_signed i w then (i,w) else raise Overflow

let of_string str w =
    check w;
    let i = string sint str w in
    if fits_signed i w then (i,w) else raise Overflow

let to_int (i,w) =
    if I.of_int min_int <= i && i <= I.of_int max_int then
        I.to_int i
    else
        raise Overflow

let to_native (i,w) =
    if I.of_nativeint Nativeint.min_int <= i &&
        i <= I.of_nativeint Nativeint.max_int
    then
        I.to_nativeint i
    else
        raise Overflow

let to_int64 (i,w) = i

```

H.5.12 Unsigned Interpretation

⟨implementation U 252a⟩ $\equiv$ (251a) 252b $\triangleright$

```

let mask i w =
    if w < 64 then
        let m = I.shift_right_logical I.minus_one (64-w) in
        I.logand i m
    else
        i

```

⟨implementation U 252b⟩ $\equiv$ (251a) $\triangleleft$ 252a

```

let fits_unsigned i w =
    assert (w = 64 || i >= I.zero);
    w = 64 or (I.shift_right_logical i w) == I.zero

let fits w' (i, w) = w <= w' || fits_unsigned i w'

let of_int i w =
    check w;
    assert (i >= 0);
    let i' = I.of_int i in
    let i' = if i' < I.zero
        then I.sub i' (I.shift_left (I.of_int min_int) 1)
        else i'
    in

```

```

        if fits_unsigned i' w then (sx64 i' w, w) else raise Overflow

let of_int32 i w =
  check w;
  assert (i >= Int32.zero);
  let i' = I.of_int32 i in
  let i' = if i' < I.zero
    then I.sub i' (I.shift_left (I.of_int32 Int32.min_int) 1)
    else i'
  in
    if fits_unsigned i' w then (sx64 i' w, w) else raise Overflow

let of_native i w =
  check w;
  assert (i >= Nativeint.zero);
  let i' = I.of_nativeint i in
  let i' = if i' < I.zero
    then I.sub i' (I.shift_left (I.of_nativeint Nativeint.min_int) 1)
    else i'
  in
    if fits_unsigned i' w then (sx64 i' w, w) else raise Overflow

let of_int64 i w =
  check w;
  if fits_unsigned i w then (sx64 i w, w) else raise Overflow

let of_string str w =
  check w;
  let i = string uint str w in
  if fits_unsigned i w then (i, w) else raise Overflow

let to_int (i,w) =
  let i = mask i w in
  if i <= I.of_int max_int then
    I.to_int i
  else
    let i' = I.add i (I.shift_left (I.of_int min_int) 1) in
    if i' < I.of_int max_int then
      I.to_int i'
    else
      raise Overflow

let to_native (i,w) =
  let i = mask i w in
  if i <= I.of_nativeint Nativeint.max_int then
    I.to_nativeint i
  else
    let i' = I.add i (I.shift_left (I.of_nativeint Nativeint.min_int) 1) in
    if i' < I.of_nativeint Nativeint.max_int then
      I.to_nativeint i'
    else
      raise Overflow

let to_int64 (i,w) =
  let i = mask i w in
  i

```

<string scanning 253>≡
 <parse int 254a>
 <parse float 254c>

(251a)

<parse char 254d>

```
let string int (s:string) (w:int) =
  assert (String.length s > 0);
  if s.[0] <= '\\' then
    char s w
  else if ( String.length s >= 2
    && s.[0] <= '0'
    && (s.[1] <= 'x' || s.[1] <= 'X')
    ) then
    int s w
  else if (String.contains s '.' || String.contains s 'e' ||
    String.contains s 'E') then
    float s w
  else
    int s w
```

<parse int 254a>≡

(253) 254b▷

```
let sint str w =
  check w;
  let len = String.length str in
  try
    if len > 2 && str.[0] <= '0' && (str.[1] <= 'x' || str.[1] <= 'X') then
      I.of_string str
    else if len > 2 && str.[0] <= '0' then
      I.of_string ("0o"^str)
    else
      I.of_string str
  with Failure s -> raise Overflow (* either that, or we let through bad syntax *)
```

<parse int 254b>+≡

(253) <254a

```
let uint str w =
  check w;
  try Uint64.of_string str
  with
  | Failure "overflow" -> raise Overflow
  | Failure s -> impossf "bad unsigned integer literal '%s'" str
```

<parse float 254c>≡

(253)

```
let float str w =
  try
    let x = float_of_string str in
    match w with
    | 32 -> Uint64.Cast.float32 x
    | 64 -> Int64.bits_of_float x
    | _ -> Unsupported.floatlit w
  with Failure s -> impossf "caml function '%s' failed on literal %s" s str
```

<parse char 254d>≡

(253)

```
let char str w =
  let str = String.sub str 1 (String.length str - 2) in
  let len = String.length str in
  try
    if len = 0
    then assert false
    else if str.[0] <= '\\\ ' && len > 1
    then match str.[1] with
    | 'a' when len = 2 -> I.of_int 7
    | 'b' when len = 2 -> I.of_int 8
    | 'n' when len = 2 -> I.of_int 10
```

```

| 'r'  when len = 2 -> I.of_int 13
| 't'  when len = 2 -> I.of_int 9
| '\\', when len = 2 -> I.of_int 92
| '\'', when len = 2 -> I.of_int 39
| '\"', when len = 2 -> I.of_int 34
| '??' when len = 2 -> I.of_int 63
| 'x'  when len = 2 -> I.of_string ("0" ^ str)
| '0' .. '7'          -> I.of_string ("0o" ^ str)
| _              -> impossf "bad character literal '%s'" str
else              I.of_int (Char.code str.[0])
with Failure s -> impossf "caml function '%s' failed on literal %s" s str

```

H.6 Residualizing instantiation

H.6.1 Residualizing instantiation — Interface

```

⟨bits.mli ((residualizing)) 255a⟩≡
  type tag = INT | NINT | INT64 | BV
  type width = int
  type bitvector = (tag * Tdpe.dynamic) (* abstract *)
  type bits = bitvector * width

```

H.6.2 Residualizing instantiation — Implementation

```

⟨bits.ml ((residualizing)) 255b⟩≡ 255c▷
  module E = Error
  module I = Int64 (* signed operations *)
  module U = UInt64 (* unsigned operations *)

  module Sy = Syntax
  module T = Tdpe

  exception Overflow
  exception Size
  exception BTA of string

```

```

⟨bits.ml ((residualizing)) 255c⟩+≡ <255b 255d>

  type tag = INT | NINT | INT64 | BV
  type width = int
  type bitvector = (tag * Tdpe.dynamic)
  type bits = bitvector * width (* width * int64 *)

```

```

⟨bits.ml ((residualizing)) 255d⟩+≡ <255c 255e>
  let maxwidth = 64

```

```

⟨bits.ml ((residualizing)) 255e⟩+≡ <255d 256a>

  exception NotAvailable

  let mkBitsINT width bits = ((INT,Sy.INT bits),width)
  let mkBitsNINT width bits = ((NINT,Sy.NINT bits),width)
  let mkBitsINT64 width bits = ((INT64,Sy.INT64 bits),width)
  let mkBitsBV width bits = raise NotAvailable

  let width (_,w) = w

```


⟨bits.ml ((residualizing)) 256a⟩+≡ <255e 256b>

```
let fits (bits,width') width =
  let (tg,bits) = bits in
  if (width <= 0) || (width > 64)
  then raise Size
  else (width = 64) ||
    (T.nbe' (T.arrowN(T.pair(T.a',T.a'),T.arrow(T.a',T.boonNone)))
      (Sy.VAR "Bits.fits")
      (Sy.INT width',bits)
      (Sy.INT width))
```

⟨bits.ml ((residualizing)) 256b⟩+≡ <256a 256c>

```
let check width uint64 =
  if (not(
    if (width <= 0) || (width > 64)
    then raise Size
    else (width = 64) || (U.le uint64 (U.shl width I.one))))
  then raise Overflow
  else ()
```

⟨bits.ml ((residualizing)) 256c⟩+≡ <256b 256d>

```
let setSize (b,w as bits) width =
  if not (fits bits width)
  then
    raise Overflow
  else (b,width)
```

⟨bits.ml ((residualizing)) 256d⟩+≡ <256c 256e>

```
let of_stringi str width =
  let len = String.length str in
  let uint64 = ( match str.[0] with
    | '0' when len > 1 -> U.of_string ("0o"^str)
    | _ -> U.of_string str
  ) in
  try
    ( check width uint64
      ; mkBitsINT64 width uint64
    )
  with
    Failure _ -> syntax ("syntax error in constant: "^str)
```

⟨bits.ml ((residualizing)) 256e⟩+≡ <256d 256f>

```
let of_stringf str width =
  if width <> 64
  then raise Size
  else try mkBitsINT64 width (U.of_float (float_of_string str)) with
    | Failure _ -> syntax ("syntax error in constant: "^str)
```

⟨bits.ml ((residualizing)) 256f⟩+≡ <256e 257a>

```
let of_stringc str width =
  let len = String.length str in
  let c =
    try
      if len = 0
      then assert false
      else if str.[0] <= '\\\ ' && len > 1
      then match str.[1] with
```

```

      | 'a'  when len = 2 -> U.of_int 7
      | 'b'  when len = 2 -> U.of_int 8
      | 'n'  when len = 2 -> U.of_int 10
      | 'r'  when len = 2 -> U.of_int 13
      | 't'  when len = 2 -> U.of_int 9
      | '\\\'' when len = 2 -> U.of_int 92
      | '\\\'' when len = 2 -> U.of_int 39
      | '\"'  when len = 2 -> U.of_int 34
      | '?'  when len = 2 -> U.of_int 63
      | 'x'  when len = 2 -> U.of_string ("0" ^ str)
      | '0' .. '7'      -> U.of_string ("0o" ^ str)
      | _             -> syntax ("syntax error: "^str)
      else             U.of_int (Char.code str.[0])
    with Failure _     -> syntax ("syntax error: "^str)
  in
    ( check width c;
      mkBitsINT64 width c
    )

```

⟨bits.ml ((residualizing)) 257a⟩+≡ ‹256f 257b›

```

  let of_int i width =
    let uint64 = (*U.of_int i*) Int64.of_int i in
      ( check width uint64 ;
        mkBitsINT width i
      )
  let of_std_int x width = ((BV,x),width)

```

⟨bits.ml ((residualizing)) 257b⟩+≡ ‹257a 257c›

```

  let of_int64 i width =
    ( check width i ;
      mkBitsINT64 width i
    )

```

⟨bits.ml ((residualizing)) 257c⟩+≡ ‹257b 257d›

```

  let of_nativeint i width =
    ((NINT,i),width)

```

⟨bits.ml ((residualizing)) 257d⟩+≡ ‹257c 258a›

```

  let to_int ((tag,bitv),w) =
    match tag with
    | INT -> bitv
    | NINT ->
      (match bitv with
       | Sy.INT64 i64 -> Sy.INT (Int64.to_int i64)
       | Sy.NINT ni -> Sy.INT (ENativeint.to_int ni)
       | Sy.INT _ -> bitv
       | _ -> T.nbe' T.arra' (Sy.VAR "Nativeint.to_int") bitv
      )
  | INT64 ->
    (match bitv with
     | Sy.INT64 i64 -> Sy.INT (Int64.to_int i64)
     | Sy.NINT ni -> Sy.INT (ENativeint.to_int ni)
     | Sy.INT i -> bitv
     | _ ->
       let (v,w1) = T.nbe' (T.arrowN(T.a',T.arrow(T.a',T.paira'))))
         (Sy.VAR "Bits.of_int64") bitv (Sy.INT w) in
       T.nbe' (T.arrow(T.paira',T.a'))
         (Sy.VAR "Bits.to_int") (v,Sy.INT w)
    )

```

```

    )
| BV -> T.nbe' (T.arrow(T.paira',T.a'))
    (Sy.VAR "Bits.to_int") (bitv,Sy.INT w)

⟨bits.ml ((residualizing)) 258a⟩+≡                                <257d 258b>
let to_nativeint ((tag,bitv),w) =
  match tag with
  | INT ->
    (match bitv with
     | Sy.INT64 i64 -> Sy.NINT (Int64.to_nativeint i64)
     | Sy.NINT ni -> bitv
     | Sy.INT i -> Sy.NINT (ENativeint.of_int i)
     | _ -> T.nbe' T.arr' (Sy.VAR "Nativeint.to_int") bitv
    )
  | NINT -> bitv
  | INT64 ->
    (match bitv with
     | Sy.INT64 i64 -> Sy.NINT (Int64.to_nativeint i64)
     | Sy.NINT ni -> bitv
     | Sy.INT i -> Sy.NINT (ENativeint.of_int i)
     | _ ->
        let (v,w1) = T.nbe' (T.arrowN(T.a',T.arrow(T.a',T.paira'))))
        (Sy.VAR "Bits.of_int64") bitv (Sy.INT w) in (* Bogus? *)
        T.nbe' (T.arrow(T.paira',T.a')) (Sy.VAR "Bits.to_nativeint") (v,Sy.INT w)
    )
| BV -> T.nbe' (T.arrow(T.paira',T.a')) (Sy.VAR "Bits.to_nativeint") (bitv,Sy.INT w)

```

```

⟨bits.ml ((residualizing)) 258b⟩+≡                                <258a 258c>
type radix      = Oct
                | Dec
                | Hex

```

```

let to_string radix dBits = raise (BTA "Can't convert to string")

```

```

⟨bits.ml ((residualizing)) 258c⟩+≡                                <258b>
let zero w =
  let z = U.of_int 0 in
  ( check w z
  ; mkBitsINT w 0
  )

```

H.7 bitops/bitset64.nw

H.8 Small sets of bits

```

⟨bitset64.mli 258d⟩≡
type elt = int    (* range 0..63 *)
type t    (* set of elements *)

val empty : t
val is_empty : t -> bool
val mem: elt -> t -> bool
val add: elt -> t -> t
val add_range: lsb:elt -> width:int -> t -> t
val singleton: elt -> t

```

```

val single_range: lsb:elt -> width:int -> t
val remove: elt -> t -> t
val remove_range: lsb:elt -> width:int -> t -> t
val union: t -> t -> t
val inter: t -> t -> t
val diff: t -> t -> t
val subset: t -> t -> bool
val eq: t -> t -> bool
val overlap: t -> t -> bool (* nonempty intersection *)

```

H.9 Implementation

```

⟨bitset64.ml 259a⟩≡
  module B = Bits
  module B0 = Bits.Ops
  type elt = int
  type t = Bits.bits

  let w = 64

  let int i = B.U.of_int i w
  let one = int 1
  let mask = B0.com (B.zero w)

  let empty = B.zero w
  let is_empty = B.is_zero
  let inter = B0.and'
  let union = B0.or'
  let diff s s' = B0.and' s (B0.com s')

  let overlap s s' = not (is_empty (inter s s'))

  let singleton i = (assert (i < w); B0.shl one (int i))

  let mem i s = not (B.is_zero (B0.and' (B0.shl one (int i)) s))
  let add i s = union (singleton i) s

  let single_range ~lsb ~width =
    assert (lsb + width < w);
    B0.shl (B0.shrl mask (int (w-width))) (int lsb)

  let add_range ~lsb ~width s = union (single_range ~lsb ~width) s

  let remove i s = diff s (singleton i)
  let remove_range ~lsb ~width s = diff s (single_range ~lsb ~width)

  let eq = B.eq
  let subset s s' = B.eq (inter s s') s

```

H.10 bitops/cell.nw

H.11 Cells

```

⟨cell.mli 259b⟩≡
  type t

```

```

[@@deriving show]

type count = C of int    (* a number of cells *)
type width = int          (* a number of bits *)

val to_width : t -> count -> width
val to_count : t -> width -> count
val divides  : t -> width -> bool    (* width is an even number of cells *)
val size     : t -> width (* number of bits in one cell *)

val of_size : int -> t

<cell.ml 260a>≡
type count = C of int    (* a number of cells *)
[@@deriving show]
type width = int          (* a number of bits *)
[@@deriving show]

type t = int * (count -> width) * (width -> count) * (width -> bool)
[@@deriving show]

let to_width (_, f, _, _) = f
let to_count (_, _, f, _) = f
let size      (w, _, _, _) = w
let divides   (_, _, _, f) = f

let c1  = ( 1, (fun (C c) ->      c), (fun w -> C w), (fun w -> true))
let c8  = ( 8, (fun (C c) -> 8 * c), (fun w -> C (w / 8)), (fun w -> w mod 8 = 0))
let c16 = (16, (fun (C c) -> 16 * c), (fun w -> C (w / 16)), (fun w -> w mod 16 = 0))
let c32 = (32, (fun (C c) -> 32 * c), (fun w -> C (w / 32)), (fun w -> w mod 32 = 0))
let c64 = (64, (fun (C c) -> 64 * c), (fun w -> C (w / 64)), (fun w -> w mod 64 = 0))

let of_size n = match n with
| 1  -> c1
| 8  -> c8
| 16 -> c16
| 32 -> c32
| 64 -> c64
| _  -> (n, (fun (C c) -> n * c), (fun w -> C (w / n)), (fun w -> w mod n = 0))

```

H.12 bitops/ctypes.nw

```

<commons3/ctypes.ml 260b>≡
  <ctypes.ml 261b>

<ctypes.mli 260c>≡
  <types 260d>
  val enum_int : string -> int
  (*
  val ctypes_vararg_enum : string
  val ctypes_vararg_str  : string
  *)

<types 260d>≡
  type 'a ctypes = { char      : 'a
                    ; double    : 'a
                    ; float     : 'a
                    ; int       : 'a

```

(261b 260c) 261a▷

```

; long_double      : 'a
; long_int          : 'a
; long_long_int    : 'a
; short            : 'a
; signed_char       : 'a
; unsigned_char     : 'a
; unsigned_long     : 'a
; unsigned_short    : 'a
; unsigned_int      : 'a
; unsigned_long_long : 'a
; address           : 'a
}

```

$\langle \text{types } 261a \rangle + \equiv$ (261b 260c) $\triangleleft$ 260d

```

type width = int
type metrics = { w : width; va_w : width }

```

$\langle \text{ctypes.ml } 261b \rangle \equiv$ (260b) 261c $\triangleright$

```

<types 260d>
module SM = Strutil.Map
let sprintf = Printf.sprintf
let fetch_ct ct str = match str with
| "char"          -> ct.char
| "double"        -> ct.double
| "float"         -> ct.float
| ""              -> ct.int
| "int"           -> ct.int
| "long double"   -> ct.long_double
| "long int"      -> ct.long_int
| "long long int" -> ct.long_long_int
| "short"         -> ct.short
| "signed char"   -> ct.signed_char
| "unsigned char" -> ct.unsigned_char
| "unsigned long" -> ct.unsigned_long
| "unsigned short" -> ct.unsigned_short
| "unsigned int"  -> ct.unsigned_int
| "unsigned long long" -> ct.unsigned_long_long
| "address"       -> ct.address
| s -> Impossible.impossible (sprintf "Unexpected C type %s\n" s)

let ct_foldi f ct z =
  let f str = f str (fetch_ct ct str) in
  f "char" (f "double" (f "float" (f "int" (f "long double"
    (f "long int" (f "long long int" (f "short" (f "signed char"
      (f "unsigned char" (f "unsigned long" (f "unsigned short"
        (f "unsigned int" (f "unsigned long long" (f "address" z))))))))))))))
let ct_fold f = ct_foldi (fun _ x -> f x)

```

$\langle \text{ctypes.ml } 261c \rangle + \equiv$ (260b) $\triangleleft$ 261b 262a $\triangleright$

```

let x86_ctypes =
{ char          = {w = 8 ; va_w = 32}
; double        = {w = 64; va_w = 64}
; float         = {w = 32; va_w = 64}
; int           = {w = 32; va_w = 32}
; long_double   = {w = 96; va_w = 96}
; long_int      = {w = 32; va_w = 32}
; long_long_int = {w = 64; va_w = 64}
; short         = {w = 16; va_w = 32}
; signed_char   = {w = 8 ; va_w = 32}
; unsigned_char = {w = 8 ; va_w = 32}

```

```

; unsigned_long      = {w = 32; va_w = 32}
; unsigned_short     = {w = 16; va_w = 32}
; unsigned_int       = {w = 32; va_w = 32}
; unsigned_long_long = {w = 64; va_w = 64}
; address            = {w = 32; va_w = 32}
}

```

<ctypes.ml 262a>+≡

(260b) <261c

```

let enum_ct =
  { char          = ("CHAR"          , 0)
  ; double        = ("DOUBLE"        , 1)
  ; float         = ("FLOAT"         , 2)
  ; int           = ("INT"           , 3)
  ; long_double   = ("LONGDOUBLE"    , 4)
  ; long_int      = ("LONGINT"       , 5)
  ; long_long_int = ("LONGLONGINT"   , 6)
  ; short         = ("SHORT"         , 7)
  ; signed_char   = ("SIGNEDCHAR"    , 8)
  ; unsigned_char = ("UNSIGNEDCHAR"  , 9)
  ; unsigned_long = ("UNSIGNEDLONG"  , 10)
  ; unsigned_short = ("UNSIGNEDSHORT", 11)
  ; unsigned_int  = ("UNSIGNEDINT"   , 12)
  ; unsigned_long_long = ("UNSIGNEDLONGLONG", 13)
  ; address       = ("ADDRESS",      14)
}

let ctypes_vararg_enum ct =
  let str_lst = ct_fold (fun (s,id) rst -> (sprintf "%s = %d" s id)::rst) enum_ct [] in
  Printf.sprintf "{ %s }" (String.concat ", " str_lst)
let enum_int h = snd (fetch_ct enum_ct h)

```

H.13 codegen/idcode.nw

H.14 Code arbitrary string as an identifier

<idcode.mli 262b>≡

```
val encode : string -> string
```

<idcode.ml 262c>≡

```

let codes = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789_."
let () = assert (String.length codes = 64)

let encode s =
  let b = Buffer.create 32 in
  let n = String.length s in
  let rec walk i accum bits =
    if bits >= 5 then
      let idx = accum lsr (bits-5) in
      Buffer.add_char b codes.[idx land 0x1f];
      walk i accum (bits-5)
    else if i = n then
      if bits = 0 then ()
      else walk i (accum lsl 1) (bits+1)
    else
      walk (i+1) ((accum lsl 8) lor (Char.code s.[i])) (bits+8) in
  begin
    walk 0 0 0;
    Buffer.contents b
  end

```

H.15 codegen/idgen.nw

```
<commons3/idgen.ml 263a>≡  
  <idgen.ml 263c>
```

H.16 Name Generator

H.16.1 Interface

```
<idgen.mli 263b>≡  
  type generator = string -> string  
  val label:      generator      (* denotes pc value *)  
  val offset:     generator      (* offset in stack      *)  
  val exit:       generator      (* label for exit node in procedure *)  
  val cont:       generator      (* symbol for dynamic continuation value *)  
  val slot:       generator      (* slot address on stack *)  
  val block:      generator      (* offset in Lua-generated Block.t address *)  
  
  module ContEntry : sig  
    val cut      : generator      (* label for cut-to entry point *)  
    val unwind   : generator      (* label for unwind entry point *)  
    val return    : generator      (* label for alternate-return entry point *)  
  end  
  (* add more as needed *)
```

H.16.2 Implementation

```
<idgen.ml 263c>≡ (263a) 263d▷  
  type generator = string -> string  
  let count = Reinit.ref 0  
  
<idgen.ml 263d>+≡ (263a) <263c 263e▷  
  let id kind s =  
    ( incr count  
      ; Printf.sprintf "%s%s:%c%d" (if kind == 'l' then ".L" else "") s kind !count  
      )  
  
<idgen.ml 263e>+≡ (263a) <263d  
  let label   = id 'l'  
  let offset  = id 'o'  
  let exit    = id 'x'  
  let cont    = id 'c'  
  let slot    = id 's'  
  let block   = id 'b'  
  module ContEntry = struct  
    let cut    = id 'C'  
    let unwind = id 'U'  
    let return = id 'R'  
  end  
  (* add more here as needed *)
```

H.17 prelude/reinit.nw

```
<commons3/reinit.mli 263f>≡  
  <reinit.mli 264a>
```


H.18 Reinitialization

```
<reinit.mli 264a>≡ (263f) 264b▷  
  val at : (unit -> unit) -> unit  
  val reset : unit -> unit  
  
<reinit.mli 264b>+≡ (263f) <264a  
  val ref : 'a -> 'a ref  
  
<reinit.ml 264c>≡  
  let tasks = ref ([] : (unit -> unit) list)  
  let at f = tasks := f :: !tasks  
  let reset () = List.iter (fun f -> f ()) (!tasks)  
  let ref x =  
    let r = ref x in  
    at (fun () -> r := x);  
    r
```

H.19 regalloc/tx.nw

```
<tx.mli 264d>≡  
  val decrement : name:string -> from:int -> to':int -> unit  
  val remaining : unit -> int  
  val set_limit : int -> unit  
  val used : unit -> int  
  val last : unit -> string  
  
<tx.ml 264e>≡  
  type t = { mutable limit : int; mutable remaining : int; mutable last : string; }  
  let ts = { limit = max_int; remaining = max_int; last = "<none>"; }  
  let _ = Reinit.at (fun () -> begin  
    ts.limit <- max_int;  
    ts.remaining <- ts.limit;  
    ts.last <- "<none>";  
  end)  
  
  let () = Debug.register "tx" "watch transaction counts"  
  
  let set_limit n = ts.limit <- n; ts.remaining <- n  
  let remaining () = ts.remaining  
  let decrement ~name ~from ~to' =  
    assert (ts.remaining = from);  
    if from <> to' then  
      begin  
        Debug.eprintf "tx" "'%s' decrementing tx's from %d to %d (diff is %d)\n"  
          name from to' (from-to');  
        ts.remaining <- to';  
        ts.last <- name  
      end  
  
  let used _ = ts.limit - ts.remaining  
  let last _ = ts.last
```

H.20 commons3/uint64.nw

```
<commons3/uint64.ml 264f>≡  
  <uint64.ml 265i>
```

```
⟨commons3/uint64.mli 265a⟩≡
  ⟨uint64.mli 265b⟩
```

H.21 UInt64 – unsigned operations on int64

H.21.1 Interface

```
⟨uint64.mli 265b⟩≡ (265a) 265c▷
  ⟨UInt64 external functions 265d⟩
  ⟨internal functions 265g⟩

⟨uint64.mli 265c⟩+≡ (265a) ◁265b
  module Cast : sig
    external float64 : float -> int64 = "uint64_float64"
    external float32 : float -> int64 = "uint64_float32"
  end

⟨UInt64 external functions 265d⟩≡ (265) 265e▷
  (* external of_int: int -> int64 = "uint64_i2i" *)

⟨UInt64 external functions 265e⟩+≡ (265) ◁265d 265f▷
  external cmp: int64 -> int64 -> int = "uint64_compare"
  external add: int64 -> int64 -> int64 = "uint64_add"
  external sub: int64 -> int64 -> int64 = "uint64_sub"
  external mul: int64 -> int64 -> int64 = "uint64_mul"
  external div: int64 -> int64 -> int64 = "uint64_div"
  external modu: int64 -> int64 -> int64 = "uint64_mod"

⟨UInt64 external functions 265f⟩+≡ (265) ◁265e
  external of_string: string -> int64 = "uint64_of_string"

⟨internal functions 265g⟩≡ (265b) 265h▷
  val eq: int64 -> int64 -> bool (* equal *)
  val lt: int64 -> int64 -> bool (* less than *)
  val gt: int64 -> int64 -> bool (* greather than *)
  val le: int64 -> int64 -> bool (* less equal *)
  val ge: int64 -> int64 -> bool (* greater equal *)

⟨internal functions 265h⟩+≡ (265b) ◁265g
  val shl: int -> int64 -> int64 (* shift left *)
  val shr: int -> int64 -> int64 (* shift right *)
```

H.21.2 Implementation

```
⟨uint64.ml 265i⟩≡ (264f) 265j▷
  module I = Int64
  ⟨UInt64 external functions 265d⟩
  module Cast = struct
    external float64 : float -> int64 = "uint64_float64"
    external float32 : float -> int64 = "uint64_float32"
  end

⟨uint64.ml 265j⟩+≡ (264f) ◁265i 266a▷
  let eq x y = (cmp x y) = 0
  let lt x y = (cmp x y) < 0
  let gt x y = (cmp x y) > 0
  let le x y = (lt x y) || (eq x y)
  let ge x y = (gt x y) || (eq x y)

  let le x y = not (gt x y)
  let ge x y = not (lt x y)
```

```

<uint64.ml 266a>+≡ (264f) <265j
  let shl n x =      I.shift_left x n
  let shr n x =      I.shift_right_logical x n

```

H.21.3 C implementation of the primitives

```

<uint64p.c 266b>≡ 266c>
#include <caml/fail.h>
#include <caml/mlvalues.h>
#include <caml/alloc.h>
#include <caml/config.h>

```

```

<uint64p.c 266c>+≡ <266b 266d>
value uint64_compare(value v1, value v2)
{
  uint64 i1 = Int64_val(v1);
  uint64 i2 = Int64_val(v2);
  return i1 == i2 ? Val_int(0) : i1 < i2 ? Val_int(-1) : Val_int(1);
}

```

```

<uint64p.c 266d>+≡ <266c 266e>
value uint64_float64(value f)
{
  union { double d; int64 i; } buffer;
  buffer.d = Double_val(f);
  return copy_int64(buffer.i);
}

value uint64_float32(value f)
{
  union { float f; unsigned n; } buffer;
  int64 i;
  buffer.f = (float) Double_val(f);
  i = (int64) buffer.n;
  return copy_int64(i);
}

```

```

<uint64p.c 266e>+≡ <266d 266f>
value uint64_i2f(value i)
{
  union { double d; int64 i; } buffer;
  buffer.i = Int64_val(i);
  return copy_double(buffer.d);
}

```

```

<uint64p.c 266f>+≡ <266e 267a>
value uint64_i2i(value i)
{
  union { int32 i[2]; int64 j; } buffer;

  buffer.i[0] = Int_val(i);
  buffer.i[1] = 0;

  return copy_int64(buffer.j);
}

```

```

<uint64p.c 267a>+≡ <266f 267b>
value uint64_add(value v1, value v2) /* ML */
{ return copy_int64((uint64)Int64_val(v1) + (uint64)Int64_val(v2)); }

value uint64_sub(value v1, value v2) /* ML */
{ return copy_int64((uint64)Int64_val(v1) - (uint64)Int64_val(v2)); }

value uint64_mul(value v1, value v2) /* ML */
{ return copy_int64((uint64)Int64_val(v1) * (uint64)Int64_val(v2)); }

<uint64p.c 267b>+≡ <267a 267c>
value uint64_div(value v1, value v2) /* ML */
{
    int64 divisor = Int64_val(v2);
    if (divisor == 0) raise_zero_divide();
    return copy_int64((uint64)Int64_val(v1) / (uint64)divisor);
}

value uint64_mod(value v1, value v2) /* ML */
{
    int64 divisor = Int64_val(v2);
    if (divisor == 0) raise_zero_divide();
    return copy_int64((uint64)Int64_val(v1) % (uint64)divisor);
}

<uint64p.c 267c>+≡ <267b 267d>
static int parse_digit(char * p)
{
    int c = *p;
    if (c >= '0' && c <= '9')
        return c - '0';
    else if (c >= 'A' && c <= 'F')
        return c - 'A' + 10;
    else if (c >= 'a' && c <= 'f')
        return c - 'a' + 10;
    else
        return -1;
}

<uint64p.c 267d>+≡ <267c 268>
static char* parse_base (char *p, int *base)
{
    *base = 10;
    if (*p == '0') {
        switch (p[1]) {
            case 'x': case 'X':
                *base = 16; return p + 2;
            case 'o': case 'O':
                *base = 8; return p + 2;
            case 'b': case 'B':
                *base = 2; return p + 2;
            default:
                *base = 8; return p;
        }
    } else {
        return p;
    }
}

```

```

<uint64p.c 268>+≡
value uint64_of_string(value s)          /* ML */
{
    uint64 max_uint64 = ~(uint64)0;
    char * p;
    uint64 res, threshold;
    int base, d;

    p = parse_base (String_val(s), &base);
    threshold = max_uint64 / (uint64) base;
    for (res = 0; /*nothing*/; p++) {
        d = parse_digit(p);
        if (d < 0 || d >= base) break;
        /* Detect overflow in multiplication base * res */
        if (threshold < res) caml_failwith("overflow");
        res = res * (uint64)base + (uint64)d;
        /* Detect overflow in addition (base * res) + d */
        if (res < (uint64)d) caml_failwith("overflow");
    }
    if ((base == 10 || (base == 8 && res == 0)) && (*p == 'u' || *p == 'U')) p++;
    if (p != String_val(s) + caml_string_length(s)) failwith("syntax");
    return copy_int64(res);
}

```

<267d

Appendix I

error

I.1 error/debug.nw

$\langle \text{error/debug.ml } 269\text{a} \rangle \equiv$
 $\langle \text{debug.ml } 213\text{c} \rangle$

$\langle \text{error/debug.mli } 269\text{b} \rangle \equiv$
 $\langle \text{debug.mli } 213\text{a} \rangle$

I.2 error/error.nw

$\langle \text{error/error.ml } 269\text{c} \rangle \equiv$
 $\langle \text{error.ml } 227\text{g} \rangle$

$\langle \text{error/error.mli } 269\text{d} \rangle \equiv$
 $\langle \text{error.mli } 226\text{a} \rangle$

I.3 error/impossible.nw

$\langle \text{error/impossible.ml } 269\text{e} \rangle \equiv$
 $\langle \text{impossible.ml } 231\text{a} \rangle$

$\langle \text{error/impossible.mli } 269\text{f} \rangle \equiv$
 $\langle \text{impossible.mli } 230\text{i} \rangle$

I.4 error/srcmap.nw

$\langle \text{error/srcmap.ml } 269\text{g} \rangle \equiv$
 $\langle \text{srcmap.ml } 20\text{a} \rangle$

$\langle \text{error/srcmap.mli } 269\text{h} \rangle \equiv$
 $\langle \text{srcmap.mli } 19\text{a} \rangle$

I.5 error/unsupported.nw

$\langle \text{error/unsupported.ml } 269\text{i} \rangle \equiv$
 $\langle \text{unsupported.ml } 231\text{c} \rangle$

$\langle \text{error/unsupported.mli } 269\text{j} \rangle \equiv$
 $\langle \text{unsupported.mli } 231\text{b} \rangle$

Appendix J

parsing

J.1 parsing/ast.nw

J.2 parsing/astpp.nw

$\langle \text{parsing/astpp.ml } 270a \rangle \equiv$
 $\langle \text{astpp.ml } 214b \rangle$

$\langle \text{parsing/astpp.mli } 270b \rangle \equiv$
 $\langle \text{astpp.mli } 214a \rangle$

J.3 parsing/parser.nw

Appendix K

front_rtl

K.1 ir/register.nw

`<front_rtl/register.ml 271a>≡`
`<register.ml content 272g>`

`<front_rtl/register.mli 271b>≡`
`<register.mli content 271e>`

K.2 Register

`<exported types(register.nw) 271c>≡` `(272g 271e) 271d▷`
`open Nopoly`

```
type aggregation =
  | BigEndian
  | LittleEndian
  | Identity
[@@deriving show]
type space = char * aggregation * Cell.t    (* name, byte order, cell size *)
[@@deriving show]
type count = Cell.count = C of int
[@@deriving show]
type width = int
[@@deriving show]
type reg = space * int * count (* Rtl.space, index, number of cells *)
[@@deriving show]
type t = reg
[@@deriving show]
type x = Reg    of t
          | Slice of width * int * t
```

`<exported types(register.nw) 271d>+≡` `(272g 271e) <271c`
`module type SETX = sig`
 `include Set.S`
 `val of_list : elt list -> t`
 `val to_string : t -> string (* elements sep. by commas (no braces) *)`
`end`

`<register.mli content 271e>≡` `(271b) 272a▷`
`<exported types(register.nw) 271c>`
`val width : t -> int`
`val eq : t -> t -> bool`
`val compare : t -> t -> int`


```

<register.mli content 272a>+≡ (271b) <271e 272b>
  val widthx : x -> int
  val eqx    : x -> x -> bool

<register.mli content 272b>+≡ (271b) <272a 272c>
  module SetX: SETX with type elt = x
  module MapX: Map.S with type key = x
  module Set:  SETX with type elt = t
  module Map:  Map.S with type key = t

<register.mli content 272c>+≡ (271b) <272b 272d>
  val promote_x      : x -> t
  val rset_to_rxset  : Set.t -> SetX.t
  val promote_rxset  : SetX.t -> Set.t

<register.mli content 272d>+≡ (271b) <272c 272e>
  val reg_int_map : t list -> int * int Map.t

<register.mli content 272e>+≡ (271b) <272d
  val contains : outer:x -> inner:x -> bool

<register.mli LUA 272f>≡
  module RT (C : Lua.Lib.CORE) : sig
    val map : t C.V.map
  end

```

K.3 Implementation of registers

```

<register.ml content 272g>≡ (271a) 273a>
  <exported types(register.nw) 271c>
  module Compare = struct
    type t = reg
    let compare (((xs,_,_),xi,C xc):t) (((ys,_,_),yi,C yc):t) =
      let x = comparec xs ys in
      if x <> 0 then x
      else let i = comparei xi yi in if i <> 0 then i
      else comparei xc yc
    let compare' x y = compare y x

    let compare = try ignore (Sys.getenv "QCREVERSE"); compare' with _ -> compare

  (*
    let compare (((xs,_,_),xi,C xc) as x:t) (((ys,_,_),yi,C yc) as y:t) =
      let answer = compare x y in
      let prime = compare y x in
      if prime <> -answer then
        Printf.kprintf Impossible.impossible "$%c[%d:%d] ? $%c[%d:%d] == %d (neg %d)\n" xs xi xc ys yi yc answer
      answer
  *)
  end
  let compare = Compare.compare
  let eq r r' = Compare.compare r r' = 0
  module CompareX = struct
    type t = x
    let compare x y = match x, y with
    | Slice (w, lsb, r), Slice (w', lsb', r') ->
      let x = comparei w w' in
      if x <> 0 then x

```

```

        else
            let x = comparei lsb lsb' in
            if x <> 0 then x
            else Compare.compare r r'
| Slice _, Reg _ -> 1
| Reg _, Slice _ -> -1
| Reg r, Reg r' -> Compare.compare r r'
end

⟨register.ml content 273a⟩+≡ (271a) <272g 273b>
module SetX = struct
    module S = Set.Make(CompareX)
    include S
    let of_list l = List.fold_right add l empty
    let to_string s =
        let elt = function
            | Reg ((s,_,_), i, C 1) -> Printf.sprintf "%c%d" s i
            | Reg ((s,_,_), i, C n) -> Printf.sprintf "%c%d:%d" s i n
            | Slice (w, i, ((s,_,_), i', _)) ->
                Printf.sprintf "%c%d@[%d..%d]" s i' i (i + w - 1) in
        String.concat ", " (List.map elt (elements s))
end
module Set = struct
    module S = Set.Make(Compare)
    include S
    let of_list l = List.fold_right add l empty
    let to_string s =
        let elt ((s,_,_), i, C n) =
            if n = 1 then
                Printf.sprintf "%c%d" s i
            else
                Printf.sprintf "%c%d:%d" s i n in
        String.concat ", " (List.map elt (elements s))
end

⟨register.ml content 273b⟩+≡ (271a) <273a 273c>
module MapX = Map.Make(CompareX)
module Map = Map.Make(Compare)

⟨register.ml content 273c⟩+≡ (271a) <273b 273d>
let promote_x = function Reg r | Slice (_, _, r) -> r
let rset_to_rxset set = Set.fold (fun r rst -> SetX.add (Reg r) rst) set SetX.empty
let promote_rxset set =
    SetX.fold (fun r rst -> match r with Reg r | Slice (_,_,r) -> Set.add r rst)
        set Set.empty

⟨register.ml content 273d⟩+≡ (271a) <273c 275a>
let reg_int_map regs =
    let cmp ((s1,_,_),i1,_) ((s2,_,_),i2,_) =
        match comparec s1 s2 with 0 -> comparei i1 i2 | n -> n in
    let order = List.sort cmp regs in
    (List.fold_left (fun (i,map) r -> (i+1, Map.add r i map)) (0, Map.empty) order)

⟨register.ml LUA 273e⟩≡
module RT (C : Lua.Lib.CORE) = struct
    module V = C.V

    let agg =
        V.default Identity
        (V.enum "aggregation"

```

```
["big", BigEndian; "little", LittleEndian; "identity", Identity])
```

```
let embedRegister ((s,a,cell),i,C c) =
  (V.record V.value).V.embed
  [ "space", V.string.V.embed (Char.escaped s)
  ; "agg",    agg.V.embed a
  ; "cellsize", V.int.V.embed (Cell.size cell)
  ; "index", V.int.V.embed i
  ; "width", V.int.V.embed (Cell.to_width cell (C c))
  ; "count", V.int.V.embed c
  ]
```

```
let projectRegister reg = match reg with
| V.Table reg ->
  (try
    let field f = V.Table.find reg (V.String f) in
    let s      = String.get (V.string.V.project (field "space")) 0 in
    let cell = Cell.of_size (V.int.V.project (field "cellsize")) in
    let i     = V.int.V.project (field "index") in
    let a     = agg.V.project (field "agg") in
    let c     = match field "count" with
    | V.Nil -> let w = V.int.V.project (field "width") in
               Cell.to_count cell w
    | cf      -> C (V.int.V.project cf) in
    ((s,a,cell),i,c)
    with V.Projection _ -> raise (V.Projection (V.Table reg, "register")))
| _ -> raise (V.Projection (reg, "register"))
```

```
let map = {
  V.embed = embedRegister ; V.project = projectRegister;
  V.is = (fun r -> try ignore (projectRegister r); true
            with V.Projection (_, _) -> false); }
```

```
end
```

⟨Lua code for registers 274⟩≡

```
Register = { }
```

```
function Register.create(t)
  if not t.width and not t.count then t.count = 1 end
  assert(t.space and t.cellsize and t.index)
  assert(t.count > 0)
  t["I am a register"] = 1
  return t
end
```

```
function Register.is(r)
  return r["I am a register"]
end
```

```
function Register.range(r1, r2)
  assert(Register.is(r1) and Register.is(r2))
  assert(r1.space == r1.space and r1.agg == r2.agg and r1.cellsize == r2.cellsize
    and r1.count == r2.count)
  assert(r1.index <= r2.index)
  local l = { }
  local next = 1
  local i = r1.index
  local space, cellsize, agg, count = r1.space, r1.cellsize, r1.agg, r1.count
  while i <= r2.index do
    l[next] =
```

```

      Register.create { space = space, cellsize = cellsize, agg = agg, count = count,
                      index = i }

    i = i + count
    next = next + 1
  end
  return l
end

⟨register.ml content 275a⟩+≡ (271a) <273d 275b>
let width ((_, _, ms), _, c) = Cell.to_width ms c
let widthx = function
  | Reg r -> width r
  | Slice (w, _, _) -> w

let rec eqx x x' = match x, x' with
| Reg r, Reg r' -> eq r r'
| Slice (w, lsb, r), Slice (w', lsb', r') -> w = w && lsb = lsb' && eq r r'
| Reg _, Slice _ -> false
| Slice _, Reg _ -> false

⟨register.ml content 275b⟩+≡ (271a) <275a
let contains ~outer ~inner = match outer, inner with
| Reg ((s, _, _), i, C c), Reg ((s', _, _), i', C c') ->
  s <= s' && i' >= i && i' + c' <= i + c
| Reg ((s, _, _), i, C c), Slice (w', lsb', ((s', _, _), i', C c')) ->
  s <= s' && i' >= i && i' + c' <= i + c
| Slice (w, lsb, ((s, _, _), i, C c)), Reg ((s', _, cell), i', C c') ->
  if c <> 1 then
    Impossible.impossible "slice of register aggregate";
    lsb = 0 && w = Cell.to_width cell (C c') && s <= s' && i = i' && c = c'
| Slice (w, lsb, ((s, _, _), i, C c)), Slice (w', lsb', ((s', _, _), i', C c')) ->
  if c <> 1 || c' <> 1 then
    Impossible.impossible "slice of register aggregate";
    s <= s' && i = i' && lsb <= lsb' && lsb+w >= lsb'+w'

```

K.4 ir/reloc.nw

```

⟨front_rtl/reloc.ml 275c⟩≡
  ⟨reloc.ml content 276a⟩

```

```

⟨front_rtl/reloc.mli 275d⟩≡
  ⟨reloc.mli content 275e⟩

```

K.5 Relocatable Addresses

```

⟨reloc.mli content 275e⟩≡ (275d)
type symbol = Symbol.t * (Symbol.t -> Rtl.width -> Rtl.exp)

type exp = Pos of symbol * Rtl.width | Neg of symbol * Rtl.width
type t = exp list * Bits.bits
[@@deriving show]
(* pad: was previously an abstract type
 *   'type t'
 * but it forbids to see the data from the debugger
 *)

```

```

(* constructors *)
val of_const : Bits.bits -> t
val of_sym   : symbol -> Rtl.width -> t
val add : t -> t -> t
val sub : t -> t -> t

(* observers *)
val fold : const:(Bits.bits -> 'a) -> sym:(symbol -> 'a) ->
          add:('a -> 'a -> 'a) -> sub:('a -> 'a -> 'a) -> t -> 'a

val width : t -> Rtl.width
val if_bare : t -> Bits.bits option (* if not a bare value, returns None *)
val as_simple : t -> Symbol.t option * Bits.bits
          (* checked RTE if not simple *)

```

K.5.1 Implementation

```

⟨reloc.ml content 276a⟩≡ (275c) 276b▷

open Nopoly

type symbol = Symbol.t * (Symbol.t -> Rtl.width -> Rtl.exp)
[@@deriving show]

let eqsym (s,_) (s',_) = s#mangled_text = $= s'#mangled_text

type exp = Pos of symbol * Rtl.width | Neg of symbol * Rtl.width
[@@deriving show]

let neg = function Pos (s, w) -> Neg (s, w) | Neg (s, w) -> Pos (s, w)

type t = exp list * Bits.bits
[@@deriving show]

let of_const c = ([], c)
let of_sym s w = ([Pos (s, w)], Bits.zero w)
let add (xs, xc) (ys, yc) = (xs @ ys, Bits.Ops.add xc yc)
let sub (xs, xc) (ys, yc) = (xs @ List.map neg ys, Bits.Ops.sub xc yc)
let fold ~const ~sym ~add ~sub (ss, c) =
  let extend e = function Pos (s, w) -> add e (sym s)
                  | Neg (s, w) -> sub e (sym s) in
  match ss with
  | Pos (s, w) :: ss when Bits.is_zero c -> List.fold_left extend (sym s) ss
  | _ -> List.fold_left extend (const c) ss

⟨reloc.ml content 276b⟩+≡ (275c) ◁276a

let width (_, b) = Bits.width b
let if_bare = function ([], b) -> Some b | (_,_, _) -> None
let as_simple a =
  let w = Bits.width (snd a) in
  let const bits = (None, bits) in
  let sym (s, _) = (Some s, Bits.zero w) in
  let add (s, b) (s', b') = match s, s' with
  | Some s, None -> (Some s, Bits.Ops.add b b')
  | None, Some s -> (Some s, Bits.Ops.add b b')
  | None, None -> (None, Bits.Ops.add b b')
  | Some _, Some _ -> Impossible.impossible "added symbols in simple reloc" in

```

```

let sub (s, b) (s', b') = match s' with
| None -> (s, Bits.Ops.sub b b')
| Some _ -> Impossible.impossible "subtracted symbols in simple reloc" in
fold ~const ~sym ~add ~sub a

```

K.6 ir/rtl.nw

$\langle \text{front_rtl/rtl.ml } 277a \rangle \equiv$
 $\langle \text{rtl.ml content } 280a \rangle$

$\langle \text{front_rtl/rtl.mli } 277b \rangle \equiv$
 $\langle \text{rtl.mli content } 277c \rangle$

K.7 Register-transfer Lists (RTL)

K.7.1 Interface

$\langle \text{rtl.mli content } 277c \rangle \equiv$ (277b) 277d▷
 $\langle \text{definitions of exported, exposed types } 277e \rangle$

```

module Private: sig
   $\langle \text{representation exposed in the private interface } 279a \rangle$ 
end

```

$\langle \text{types and functions exported at top level } 277f \rangle$

$\langle \text{rtl.mli content } 277d \rangle + \equiv$ (277b) ◁ 277c

```

module Dn: sig
   $\langle \text{DN } 278h \rangle$ 
end

```

```

module Up: sig
   $\langle \text{UP } 278i \rangle$ 
end

```

$\langle \text{definitions of exported, exposed types } 277e \rangle \equiv$ (280a 277c) 278a▷

```

type aggregation = Register.aggregation =
  | BigEndian
  | LittleEndian
  | Identity
[ $\text{@@deriving show}$ ]

```

```

type space = char * aggregation * Cell.t    (* name, byte order, cell size *)
[ $\text{@@deriving show}$ ]

```

```

type width = int
[ $\text{@@deriving show}$ ]

```

$\langle \text{types and functions exported at top level } 277f \rangle \equiv$ (277c) 278b▷

```

type exp          (* denotes a compile-time or run-time value *)
[ $\text{@@deriving show}$ ]
type loc          (* mutable container of a bit vector *)
[ $\text{@@deriving show}$ ]
type rtl          (* effect of a computation *)

```

```

[@@deriving show]
type opr      (* a pure function on values *)
type assertion (* a claim about the run-time value of an address *)

val bool      : bool -> exp
val bits      : Bits.bits -> width -> exp
val codesym   : Symbol.t -> width -> exp    (* locally defined code label (incl proc) *)
val datasym   : Symbol.t -> width -> exp    (* locally defined data label *)
val impsym    : Symbol.t -> width -> exp    (* imported symbol *)
val late      : string -> width -> exp      (* late compile time constant *)
val fetch     : loc -> width -> exp
val app       : opr -> exp list -> exp

val opr       : string -> width list -> opr

<definitions of exported, exposed types 278a>+≡ (280a 277c) <277e
  type count = Register.count = C of int

<types and functions exported at top level 278b>+≡ (277c) <277f 278c>
  val none      : assertion
  val aligned   : int -> assertion
  val shift     : int -> assertion -> assertion
  val shift_multiple : int -> assertion -> assertion
  val alignment : assertion -> int
  val mem       : assertion -> space -> count -> exp -> loc
  val reg       : Register.t -> loc
  val regx      : Register.x -> loc
  val var       : string -> index:int -> width -> loc
  val global    : string -> index:int -> width -> loc
  val slice     : width -> lsb:int -> loc -> loc

<types and functions exported at top level 278c>+≡ (277c) <278b 278d>
  val store     : loc -> exp -> width -> rtl
  val kill      : loc -> rtl

<types and functions exported at top level 278d>+≡ (277c) <278c 278e>
  val guard     : exp -> rtl -> rtl
  val par       : rtl list -> rtl

<types and functions exported at top level 278e>+≡ (277c) <278d 278f>
  val null      : rtl

<types and functions exported at top level 278f>+≡ (277c) <278e 278g>
  val fetch_cvt : loc -> width -> exp
  val store_cvt : loc -> exp -> width -> rtl

<types and functions exported at top level 278g>+≡ (277c) <278f>
  val locwidth  : loc -> width

<DN 278h>≡ (277d)
  val exp:      exp      -> Private.exp
  val loc:      loc      -> Private.loc
  val rtl:      rtl      -> Private.rtl
  val opr:      opr      -> Private.opr
  val assertion: assertion -> Private.assertion

<UP 278i>≡ (277d)
  val exp:      Private.exp      -> exp
  val loc:      Private.loc      -> loc
  val rtl:      Private.rtl      -> rtl
  val opr:      Private.opr      -> opr
  val assertion: Private.assertion -> assertion
  val effect :   Private.effect   -> rtl
  val const :    Private.const     -> exp

```

```

⟨representation exposed in the private interface 279a⟩≡ (280b 277c) 279b▷
type aligned = int (* alignment guaranteed *)
[@@deriving show]
type assertion = aligned (* may one day include alias info *)
[@@deriving show]

⟨representation exposed in the private interface 279b⟩+≡ (280b 277c) ◁279a 279c▷
type opr = string * width list
[@@deriving show]

⟨representation exposed in the private interface 279c⟩+≡ (280b 277c) ◁279b 279d▷
type const = Bool of bool
              | Bits of Bits.bits (* literal constant *)
              | Link of Symbol.t * symkind * width (* link-time constant *)
              | Diff of const * const (* difference of two constants *)
              | Late of string * width (* late compile time constant *)
and symkind = Code | Data | Imported (* three kinds of symbol, needed for PIC *)
[@@deriving show]

type exp = Const of const
          | Fetch of loc * width
          | App of opr * exp list

⟨representation exposed in the private interface 279d⟩+≡ (280b 277c) ◁279c 279e▷
and count = Register.count = C of int
and loc = Mem of space
              * count
              * exp
              * assertion
              | Reg of Register.t (* space * int * count *)
              | Var of string (* name from C-- source *)
                  * int (* index for run-time API *)
                  * width
              | Global of string * int * width (* global C-- variable *)
              | Slice of width (* number of bits in loc *)
                  * int (* index of least-significant bit of slice *)
                  * loc (* location from which slice is drawn *)
[@@deriving show]

⟨representation exposed in the private interface 279e⟩+≡ (280b 277c) ◁279d 279f▷
type effect = Store of loc * exp * width
            | Kill of loc
[@@deriving show]

⟨representation exposed in the private interface 279f⟩+≡ (280b 277c) ◁279e 279g▷
type guarded = exp * effect
[@@deriving show]

⟨representation exposed in the private interface 279g⟩+≡ (280b 277c) ◁279f
type rtl = Rtl of guarded list
[@@deriving show]

```


K.7.2 Implementation

$\langle \text{rtl.ml content } 280a \rangle \equiv$ (277a)
 $\langle \text{definitions of exported, exposed types } 277e \rangle$

```
module Private = struct
   $\langle \text{Private } 280b \rangle$ 
end

 $\langle \text{definitions of types and functions exported at top level } 280c \rangle$ 

module Dn = struct
   $\langle \text{Dn } 282a \rangle$ 
end

module Up = struct
   $\langle \text{Up } 282b \rangle$ 
end
```

$\langle \text{Private } 280b \rangle \equiv$ (280a)
 $\langle \text{representation exposed in the private interface } 279a \rangle$

$\langle \text{definitions of types and functions exported at top level } 280c \rangle \equiv$ (280a) 280d▷

```
open Private    (* never do that *)
type exp        = Private.exp
[@@deriving show]
type loc        = Private.loc
[@@deriving show]
type rtl        = Private.rtl
[@@deriving show]
type opr        = Private.opr
[@@deriving show]
type assertion = Private.assertion
```

$\langle \text{definitions of types and functions exported at top level } 280d \rangle + \equiv$ (280a) <280c 280e▷

```
let aligned k = k
let alignment k = k
let shift k' k =
  let rec gcd (m:int) (n:int) =
    let rec g m n = if n = 0 then m else g n (m mod n) in
    if n < 0 then gcd m (- n)
    else if m < n then gcd n m
    else g m n in
  gcd k k'
let shift_multiple = shift (* OK for this rep, which is lossy *)
let none = aligned 1
```

$\langle \text{definitions of types and functions exported at top level } 280e \rangle + \equiv$ (280a) <280d 280f▷

```
let par rtls = Rtl (List.concat (List.map (fun (Rtl x) -> x) rtls))
let slice width ~lsb loc = Slice (width, lsb, loc)
```

$\langle \text{definitions of types and functions exported at top level } 280f \rangle + \equiv$ (280a) <280e 281a▷

```
let conjunction = ("conjoin", []) (* logical and *)
let conjoin g g' = match g with
| Const(Bool true)  -> g'
| Const(Bool false) -> g
| _ -> match g' with
| Const(Bool true)  -> g
| Const(Bool false) -> g'
| _ -> App(conjunction, [g; g'])
```

<definitions of types and functions exported at top level 281a>+≡ (280a) <280f 281b>

```
let guard expr (Rtl geffects) =
  let conjunct (guard,effect) = (conjoin expr guard, effect) in
  Rtl (List.map conjunct geffects)
```

<definitions of types and functions exported at top level 281b>+≡ (280a) <281a 281c>

```
let true' = Const (Bool true)
let false' = Const (Bool false)
let bool p = if p then true' else false'
```

<definitions of types and functions exported at top level 281c>+≡ (280a) <281b 281d>

```
let bits b width = ( assert (Bits.width b = width)
                     ; Const (Bits b)
                     )

let codesym name width = Const (Link(name,Code,width))
let datasym name width = Const (Link(name,Data,width))
let impsym name width = Const (Link(name,Imported,width))
let diff const1 const2 = Const (Diff(const1,const2))
let late name width = Const (Late(name,width))
let fetch loc w = Fetch(loc, w)
let app op exprs = App(op,exprs)
let opr name widths = (name,widths)
```

(location *)*

```
let mem assertion sp count expr = Mem(sp,count,expr,assertion)
let var name ~index width = Var (name,index,width)
let global name ~index width = Global (name,index,width)
let reg r = Reg r
let regx = function
  | Register.Reg r -> Reg r
  | Register.Slice (w, lsb, r) -> Slice(w, lsb, Reg r)
```

(effect *)*

```
let rtl effect = Rtl [(bool true, effect)]
let store loc expr width = rtl (Store(loc, expr, width))
let kill loc = rtl (Kill(loc))
let null = Rtl []
```

<definitions of types and functions exported at top level 281d>+≡ (280a) <281c 281e>

```
let locwidth = function
  | Mem((_,_,ms),c,_,_) -> Cell.to_width ms c
  | Reg((_,_,ms),_,c) -> Cell.to_width ms c
  | Var(_,_,w) -> w
  | Global(_,_,w) -> w
  | Slice(w, _, _) -> w
```

```
let rec fetch_cvt loc n =
  match loc with
  | Slice(n', lsb, loc) ->
    assert (n'=n);
    let w = locwidth loc in
    if lsb = 0 then
      App(("lobits", [w; n]), [fetch loc n])
    else
      App(("bitExtract", [w; n]), [Const (Bits (Bits.U.of_int lsb w)); fetch loc n])
  | _ -> Fetch(loc,n)
```

<definitions of types and functions exported at top level 281e>+≡ (280a) <281d>

```
let store_cvt loc expr n =
  match loc with
```

```

| Slice(n', lsb, loc) ->
  assert (n'=n);
  let w = locwidth loc in
  let wide = Fetch(loc, w) in
  let lsb = Const (Bits (Bits.U.of_int lsb w)) in
  let wide' = App(("bitInsert", [w; n]), [lsb; wide; expr]) in
  rtl (Store(loc, wide', w))
| _ -> rtl (Store(loc,expr,n))

```

$\langle Dn\ 282a \rangle \equiv$ (280a)

```

let id          = fun x -> x
let exp         = id
let loc         = id
let rtl         = id
let opr         = id
let assertion   = id

```

$\langle Up\ 282b \rangle \equiv$ (280a)

```

let effect      = rtl (* from above *)
let id          = fun x -> x
let exp         = id
let loc         = id
let rtl         = id
let opr         = id
let assertion   = id
let const c     = Const c

```

K.8 front_rtl/rtldebug.nw

$\langle \text{front_rtl/rtldebug.ml } 282c \rangle \equiv$
 $\langle \text{rtldebug.ml content } 282f \rangle$

$\langle \text{front_rtl/rtldebug.mli } 282d \rangle \equiv$
 $\langle \text{rtldebug.mli content } 282e \rangle$

K.9 Functions for Debugging RTLs

$\langle \text{rtldebug.mli content } 282e \rangle \equiv$ (282d)

```

exception TypeCheck of Rtl.rtl
val typecheck:      Rtl.rtl -> unit      (* raises TypeCheck *)

```

K.9.1 Implementation

$\langle \text{rtldebug.ml content } 282f \rangle \equiv$ (282c)

```

open Nopoly

module RP = Rtl.Private
exception TypeCheck of Rtl.rtl
let fail rtl = raise (TypeCheck rtl)

let typecheck (r:Rtl.rtl) =
  let width n ty = match ty with
    | Types.Bits k when n = k -> ty
    | _ -> fail r in
  let bits = function

```

```

      | Types.Bits n          -> n
      | _                    -> fail r in
let bool = function
  | Types.Bool              -> ()
  | _                      -> fail r in

let rec exp = function
  | RP.Fetch (l, w)          -> width w (loc l)
  | RP.Const c               -> const c
  | RP.App(opr, args)        ->
    let argtys               = List.map exp args in
    let opty, result = Rtlop.mono (Rtl.Up.opr opr) in
    if opty == argtys then (* comparing lists of types! *)
      result
    else
      fail r

and const = function
  | RP.Bool _                -> Types.Bool
  | RP.Bits b                 -> Types.Bits (Bits.width b)
  | RP.Link (_,_,w)           -> Types.Bits w
  | RP.Diff (c,c')           -> let t = const c and t' = const c' in
    if t == t' then t else fail r
  | RP.Late (_,w)             -> Types.Bits w

and loc = function
  | RP.Mem ((_,_, ms),c,e,ass) ->
let _ = bits (exp e) in Types.Bits (Cell.to_width ms c)
  | RP.Reg ((_,_, ms),i,c)     -> Types.Bits (Cell.to_width ms c)
  | RP.Slice (w,i,l)           -> let _ = bits (loc l) in Types.Bits w
  | RP.Var (_,_,w)             -> Types.Bits w
  | RP.Global (_,_,w)          -> Types.Bits w

and effect = function
  | RP.Store (l,e,w)           -> let _ = width w (loc l) in
    let _ = width w (exp e) in
    ()
  | RP.Kill (l)                -> let _ = bits (loc l) in ()

and guarded (e,eff)            = ( bool (exp e)
    ; effect eff
    )

and rtl (RP.Rtl rtl)           = List.iter guarded rtl in
rtl (Rtl.Dn.rtl r)

```

K.10 front_rtl/rtlop.nw

$\langle \text{front_rtl/rtlop.ml } 283a \rangle \equiv$
 $\langle \text{rtlop.ml content } 284e \rangle$

$\langle \text{front_rtl/rtlop.mli } 283b \rangle \equiv$
 $\langle \text{rtlop.mli content } 283c \rangle$

K.11 Support for RTL operators

$\langle \text{rtlop.mli content } 283c \rangle \equiv$ (283b) 284a▷
 val add_operator : name:string -> result_is_float:bool -> Types.tyscheme -> unit

```

<rtlop.mli content 284a>+≡ (283b) <283c 284b>
  val mono : Rtl.opr -> Types.monotype      (* Not_found *)
  val has_floating_result : Rtl.opr -> bool
  val fold : (string -> Types.tyscheme -> 'a -> 'a) -> 'a -> 'a
  val print_shapes : unit -> unit

<rtlop.mli content 284b>+≡ (283b) <284a 284c>
  val opnames : unit -> string list (* names of all source-language operators *)

<rtlop.mli content 284c>+≡ (283b) <284b 284d>
  module Translate : sig
    val prefix : string -> Types.ty list -> Types.ty * Rtl.opr (*ErrorExn*)
    val binary : string -> Types.ty list -> Types.ty * Rtl.opr (*ErrorExn*)
    val unary  : string -> Types.ty list -> Types.ty * Rtl.opr (*ErrorExn*)
  end

<rtlop.mli content 284d>+≡ (283b) <284c
  module Emit : sig
    val creators : unit -> unit
  end

```

K.11.1 Implementation

```

<rtlop.ml content 284e>≡ (283a) 284f>
  module T = Types      (* save external Types module here *)
  let (-->) = T.proc
  let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

K.11.2 RTL-Op Types

```

<rtlop.ml content 284f>+≡ (283a) <284e 286a>
  let predefined =
    (* the four values below characterize the proper interpretation of a
       result, but the only information we actually use is a Boolean
       telling us if the result is a floating-point value *)
    let float = true in
    let int    = false in
    let code2 = false in
    let bool   = false in
    [ "NaN"      , int,    [T.var 1] --> T.var 2
    ; "add"      , int,    [T.var 1; T.var 1] --> T.var 1
    ; "addc"     , int,    [T.var 1; T.var 1; T.fixbits 1] --> T.var 1
    ; "add_overflows", bool, [T.var 1; T.var 1] --> T.bool
    ; "and"      , int,    [T.var 1; T.var 1] --> T.var 1
    ; "bit"      , int,    [T.bool] --> T.fixbits 1
    ; "bool"     , bool,   [T.fixbits 1] --> T.bool
    ; "borrow"   , int,    [T.var 1; T.var 1; T.fixbits 1] --> T.fixbits 1
    ; "carry"    , int,    [T.var 1; T.var 1; T.fixbits 1] --> T.fixbits 1
    ; "com"      , int,    [T.var 1] --> T.var 1
    ; "conjoin"  , bool,   [T.bool; T.bool] --> T.bool
    ; "disjoin"  , bool,   [T.bool; T.bool] --> T.bool
    ; "div"      , int,    [T.var 1; T.var 1] --> T.var 1
    ; "div_overflows", bool, [T.var 1; T.var 1] --> T.bool
    ; "divu"     , int,    [T.var 1; T.var 1] --> T.var 1
    ; "eq"       , bool,   [T.var 1; T.var 1] --> T.bool
    ; "f2f"      , float,  [T.var 1; T.fixbits 2] --> T.var 2
    ; "f2f_implicit_round", float, [T.var 1] --> T.var 2
    ; "f2i"      , int,    [T.var 1; T.fixbits 2] --> T.var 2

```

```

; "fabs"           , float, [T.var 1] --> T.var 1
; "fadd"           , float, [T.var 1; T.var 1; T.fixbits 2] --> T.var 1
; "false"          , bool, [] --> T.bool
; "fcmp"           , code2, [T.var 1; T.var 1] --> T.fixbits 2
; "fdiv"           , float, [T.var 1; T.var 1; T.fixbits 2] --> T.var 1
; "feq"            , bool, [T.var 1; T.var 1] --> T.bool
; "fge"            , bool, [T.var 1; T.var 1] --> T.bool
; "fgt"            , bool, [T.var 1; T.var 1] --> T.bool
; "fle"            , bool, [T.var 1; T.var 1] --> T.bool
; "float_eq"        , code2, [] --> T.fixbits 2
; "float_gt"        , code2, [] --> T.fixbits 2
; "float_lt"        , code2, [] --> T.fixbits 2
; "flt"            , bool, [T.var 1; T.var 1] --> T.bool
; "fmul"           , float, [T.var 1; T.var 1; T.fixbits 2] --> T.var 1
; "fmulx"          , float, [T.var 1; T.var 1] --> T.double 1
; "fne"            , bool, [T.var 1; T.var 1] --> T.bool
; "fneg"           , float, [T.var 1] --> T.var 1
; "fordered"        , bool, [T.var 1; T.var 1] --> T.bool
; "fsqrt"          , float, [T.var 1; T.fixbits 2] --> T.var 1
; "fsub"           , float, [T.var 1; T.var 1; T.fixbits 2] --> T.var 1
; "funordered"      , bool, [T.var 1; T.var 1] --> T.bool
; "ge"             , bool, [T.var 1; T.var 1] --> T.bool
; "geu"            , bool, [T.var 1; T.var 1] --> T.bool
; "gt"             , bool, [T.var 1; T.var 1] --> T.bool
; "gtu"            , bool, [T.var 1; T.var 1] --> T.bool
; "i2f"            , float, [T.var 1; T.fixbits 2] --> T.var 2
; "le"             , bool, [T.var 1; T.var 1] --> T.bool
; "leu"            , bool, [T.var 1; T.var 1] --> T.bool
; "lobits"          , int, [T.var 1] --> T.var 2
; "lt"             , bool, [T.var 1; T.var 1] --> T.bool
; "ltu"            , bool, [T.var 1; T.var 1] --> T.bool
; "minf"           , float, [] --> T.var 1
; "mod"            , int, [T.var 1; T.var 1] --> T.var 1
; "modu"           , int, [T.var 1; T.var 1] --> T.var 1
; "mul"            , int, [T.var 1; T.var 1] --> T.var 1
; "mulux"          , int, [T.var 1; T.var 1] --> T.double 1
; "mulx"           , int, [T.var 1; T.var 1] --> T.double 1
; "mul_overflows"  , bool, [T.var 1; T.var 1] --> T.bool
; "mulu_overflows" , bool, [T.var 1; T.var 1] --> T.bool
; "mzero"          , float, [] --> T.var 1
; "ne"             , bool, [T.var 1; T.var 1] --> T.bool
; "neg"            , int, [T.var 1] --> T.var 1
; "not"            , bool, [T.bool] --> T.bool
; "or"             , int, [T.var 1; T.var 1] --> T.var 1
; "pinf"           , float, [] --> T.var 1
; "popcnt"         , int, [T.var 1] --> T.var 1
; "pzero"          , float, [] --> T.var 1
; "quot"           , int, [T.var 1; T.var 1] --> T.var 1
; "quot_overflows" , bool, [T.var 1; T.var 1] --> T.bool
; "rem"            , int, [T.var 1; T.var 1] --> T.var 1
; "rotr"           , int, [T.var 1; T.var 1] --> T.var 1
; "round_down"     , code2, [] --> T.fixbits 2
; "round_nearest"  , code2, [] --> T.fixbits 2
; "round_up"       , code2, [] --> T.fixbits 2
; "round_zero"     , code2, [] --> T.fixbits 2
; "shl"            , int, [T.var 1; T.var 1] --> T.var 1
; "shra"           , int, [T.var 1; T.var 1] --> T.var 1
; "shrl"           , int, [T.var 1; T.var 1] --> T.var 1
; "sub"            , int, [T.var 1; T.var 1] --> T.var 1

```

```

; "subb"          , int,    [T.var 1; T.var 1; T.fixbits 1] --> T.var 1
; "sub_overflows", bool,  [T.var 1; T.var 1] --> T.bool
; "sx"           , int,    [T.var 1] --> T.var 2
; "true"         , bool,   [] --> T.bool
; "unordered"    , code2, [] --> T.fixbits 2
; "xor"          , int,    [T.var 1; T.var 1] --> T.var 1
; "zx"          , int,    [T.var 1] --> T.var 2
(* not C-- operators, but might show up in RTLs *)
; "bitExtract"   , int,    [T.var 1; T.var 1] --> T.var 2
; "bitInsert"    , int,    [T.var 1; T.var 1; T.var 2] --> T.var 1
; "bitTransfer"  , int,    [T.var 1; T.var 1; T.var 1; T.var 1; T.var 1] --> T.var 1

]

<rtlop.ml content 286a>+≡ (283a) <284f 286b>
let optypes = ref (Strutil.assoc2map (List.map (fun (o, _, t) -> (o, t)) predefined))
let opfloats = ref (Strutil.assoc2map (List.map (fun (o, f, _) -> (o, f)) predefined))

<rtlop.ml content 286b>+≡ (283a) <286a 286c>
let add_operator ~name:o ~result_is_float:f t =
  if Strutil.Map.mem o (!optypes) then
    impossf "registered a duplicate RTL operator %%%s" o;
  optypes := Strutil.Map.add o t (!optypes);
  opfloats := Strutil.Map.add o f (!opfloats)

<rtlop.ml content 286c>+≡ (283a) <286b 286d>
let print_shapes () =
  Strutil.Map.iter (fun o t ->
    Printf.printf "%16s : %s\n" o (Types.scheme_string t)) (!optypes)

<rtlop.ml content 286d>+≡ (283a) <286c 286e>
let visible = function
  | "bitExtract" | "bitInsert" | "bitTransfer" | "f2f_implicit_round" -> false
  | "fcmp" | "float_lt" | "float_eq" | "float_gt" | "unordered" -> false (*expunged*)
  | _ -> true

let fold f z =
  Strutil.Map.fold (fun o t z -> if visible o then f o t z else z) (!optypes) z

let opnames () = fold (fun o _ os -> o :: os) []

<rtlop.ml content 286e>+≡ (283a) <286d 286f>
let findopname name t =
  try Strutil.Map.find name t
  with Not_found ->
    ( Printf.eprintf "unknown RTL operator '%%s'" name
      ; raise Not_found
    )
let findop op = findopname (fst (Rtl.Dn.opr op))
let mono op = let _, ws = Rtl.Dn.opr op in Types.instantiate (findop op (!optypes)) ws
let has_floating_result op = findop op (!opfloats)

```

K.11.3 Translation from C-- operators to RTL operators

```

<rtlop.ml content 286f>+≡ (283a) <286e 287a>
module Translate = struct
  let prefix opname argtys =
    try
      let op, retsize = T.split opname in

```

```

    let opty          = findopname op (!optypes) in
    let argwidths     = T.widthlist opname opty argtys in
    ( match retsize with
    | None    -> T.appl opname opty argtys, Rtl.opr op argwidths
    | Some n -> T.bits n,                Rtl.opr op (argwidths@[n])
    )
  with
  | Not_found -> Error.errorf "unknown operator %%%s" opname

<rtlop.ml content 287a>+≡ (283a) <286f 287b>
  let binops = Strutil.assoc2map
    [ "+"      , "add"
    ; "-"      , "sub"
    ; "*"      , "mul"
    ; "/"      , "divu"
    ; "%"      , "modu"
    ; "<<"     , "shl"
    ; ">>"     , "shrl"
    ; "=="     , "eq"
    ; "<="     , "leu"
    ; ">="     , "geu"
    ; ">"      , "gtu"
    ; "<"      , "ltu"
    ; "!="     , "ne"
    ; "&"      , "and"
    ; "^"      , "xor"
    ; "|"      , "or"
    ; "!="     , "ne"
    ]

  let unops = Strutil.assoc2map
    [ "-"      , "neg"
    ; "~"      , "com"
    ]

  let binary op =
    try prefix (Strutil.Map.find op binops)
    with Not_found -> Impossible.impossible ("unknown binary infix operator "^op)

  let unary op =
    try prefix (Strutil.Map.find op unops)
    with Not_found -> Impossible.impossible ("unknown unary symbolic operator "^op)
end

<rtlop.ml content 287b>+≡ (283a) <287a
  let ws = ["w"; "w'"; "w3"; "w4"; "w5"]
  let args = ["x"; "y"; "z"; "u"; "v"]

  let width n = try List.nth ws n with _ -> Impossible.impossible "too many widths"
  let arg n = try List.nth args n with _ -> Impossible.impossible "too many args"

module Emit = struct
  let pf = Printf.printf
  let mangle = function
    | "and" -> "_and"
    | "NaN" -> "_Nan"
    | "mod" -> "_mod"
    | "or" -> "_or"
    | n -> n

```



```

let emitop name (parms, _ as tyscheme) =
  let nargs    = List.length parms in
  let nwidths = T.largest_key tyscheme in
  begin
    pf "let %s" (mangle name);
    for i = 0 to nwidths - 1 do pf " %s" (width i) done;
    for i = 0 to nargs    - 1 do pf " %s" (arg  i) done;
    pf " = Rtl.app (Rtl.opr \"%s\" [" name;
    for i = 0 to nwidths - 1 do pf "%s;" (width i) done;
    pf "]) [";
    for i = 0 to nargs    - 1 do pf "%s; " (arg  i) done;
    pf "]\n";
  end
let creators () =
  pf "(* This code generated automatically by Rtlop.Emit.creators *)\n";
  Strutil.Map.iter emitop (!optypes)
end

```

K.12 front_rtl/rtlutil.nw

`<front_rtl/rtlutil.ml 288a>≡`
`<rtlutil.ml content 292b>`

`<front_rtl/rtlutil.mli 288b>≡`
`<rtlutil.mli content 288c>`

K.13 RTL Utilities

K.13.1 Important types

`<rtlutil.mli content 288c>≡` (288b) 288d▷
 type alloc =
 { fetch : Rtl.width -> Rtl.exp
 ; store : Rtl.exp -> Rtl.width -> Rtl.rtl
 }

K.13.2 Common operations on RTLs

`<rtlutil.mli content 288d>+≡` (288b) <288c 288e▷
 val add : Rtl.width -> Rtl.exp -> Rtl.exp -> Rtl.exp
 val addk : Rtl.width -> Rtl.exp -> int -> Rtl.exp

`<rtlutil.mli content 288e>+≡` (288b) <288d 288f▷
 val fetch : Rtl.loc -> Rtl.exp
 val store : Rtl.loc -> Rtl.exp -> Rtl.rtl

`<rtlutil.mli content 288f>+≡` (288b) <288e 288g▷
 val reloc : Reloc.t -> Rtl.width -> Rtl.exp

`<rtlutil.mli content 288g>+≡` (288b) <288f 289a▷
 val is_hardware : Rtl.Private.loc -> bool
 val to_hardware : Rtl.Private.loc -> Register.x

K.13.3 Equality and comparison

```
(rtlutil.mli content 289a)+≡ (288b) <288g 289b>
module Eq : sig
  val space : Rtl.space      -> Rtl.space      -> bool
  val const : Rtl.Private.const -> Rtl.Private.const -> bool
  val loc   : Rtl.Private.loc   -> Rtl.Private.loc   -> bool
  val exp   : Rtl.Private.exp   -> Rtl.Private.exp   -> bool
  val rtl   : Rtl.Private.rtl   -> Rtl.Private.rtl   -> bool
end
module Compare : sig
  val space : Rtl.space      -> Rtl.space      -> int
  val const : Rtl.Private.const -> Rtl.Private.const -> int
  val loc   : Rtl.Private.loc   -> Rtl.Private.loc   -> int
  val exp   : Rtl.Private.exp   -> Rtl.Private.exp   -> int
  val rtl   : Rtl.Private.rtl   -> Rtl.Private.rtl   -> int
end
```

K.13.4 Width

```
(rtlutil.mli content 289b)+≡ (288b) <289a 289c>
module Width: sig
  val loc:    Rtl.loc -> int
  val exp:    Rtl.exp -> int
  val exp':   Rtl.Private.exp -> int
end
```

K.13.5 Evaluation Time Classification

```
(rtlutil.mli content 289c)+≡ (288b) <289b 289d>
module Time: sig
  val compile: Rtl.exp -> bool (* true, iff compile-time const *)
  val link:    Rtl.exp -> bool (* true, off compile-time or link-time *)
end
```

K.13.6 Read-Write Sets

```
(rtlutil.mli content 289d)+≡ (288b) <289c 289e>
module ReadWrite: sig
  type 'a observert = Register.t -> 'a -> 'a
  type 'a observerx = Register.x -> 'a -> 'a
  val fold: read: 'a observerx -> write: 'a observerx -> Rtl.rtl -> 'a -> 'a
  val fold_promote: read: 'a observert -> write: 'a observert -> Rtl.rtl -> 'a -> 'a
  val sets: Rtl.rtl -> (Register.SetX.t * Register.SetX.t)
  val sets_promote: Rtl.rtl -> (Register.Set.t * Register.Set.t)
end
```

```
(rtlutil.mli content 289e)+≡ (288b) <289d 290a>
module ReadWriteKill: sig
  type 'a observert = Register.t -> 'a -> 'a
  type 'a observerx = Register.x -> 'a -> 'a
  val fold: read: 'a observerx -> write: 'a observerx -> kill: 'a observerx ->
    Rtl.rtl -> 'a -> 'a
  val fold_promote: read: 'a observert -> write: 'a observert -> kill: 'a observert ->
    Rtl.rtl -> 'a -> 'a
  val sets: Rtl.rtl -> Register.SetX.t * Register.SetX.t * Register.SetX.t
  val sets_promote: Rtl.rtl -> Register.Set.t * Register.Set.t * Register.Set.t
end
```

```

<rtlutil.mli content 290a>+≡ (288b) <289e 290b>
module FullReadWriteKill: sig
  type 'a observer = Rtl.Private.loc -> 'a -> 'a
  val fold: read: 'a observer -> write: 'a observer -> kill: 'a observer ->
    Rtl.rtl -> 'a -> 'a
end

```

K.13.7 Search

```

<rtlutil.mli content 290b>+≡ (288b) <290a 290c>
module Exists : sig
  module Opr : sig
    val rtl : (Rtl.opr -> bool) -> Rtl.rtl -> bool
  end
  module Loc : sig
    val exp : (Rtl.Private.loc -> bool) -> Rtl.Private.exp -> bool
    val rtl : (Rtl.Private.loc -> bool) -> Rtl.Private.rtl -> bool
  end
  module Const : sig
    val rtl : (Rtl.Private.const -> bool) -> Rtl.rtl -> bool
  end
end

```

```

<rtlutil.mli content 290c>+≡ (288b) <290b 290d>
module Find : sig
  module Loc : sig
    val exp : (Rtl.Private.loc -> bool) -> Rtl.Private.exp -> Rtl.Private.loc option
  end
end

```

```

<rtlutil.mli content 290d>+≡ (288b) <290c 290e>
module Fold : sig
  module RegX : sig
    val loc : (Register.x -> 'a -> 'a) -> Rtl.loc -> 'a -> 'a
  end
  module LocFetched : sig
    val rtl : (Rtl.Private.loc -> 'a -> 'a) -> Rtl.Private.rtl -> 'a -> 'a
  end
end

```

K.13.8 Aliasing

```

<rtlutil.mli content 290e>+≡ (288b) <290d 291a>
module MayAlias : sig
  val regs : Register.t -> Register.t -> bool
  val locs : Rtl.loc -> Rtl.loc -> bool (* useful to partially apply *)
  val locs' : Rtl.Private.loc -> Rtl.Private.loc -> bool (* partially apply *)
  val exp : Rtl.loc -> Rtl.exp -> bool (* useful to partially apply *)
  val exp' : Rtl.Private.loc -> Rtl.Private.exp -> bool (* partially apply *)
  val store_uses' :
    Rtl.Private.loc -> (Rtl.Private.loc * Rtl.Private.exp * int) -> bool
end

```

K.13.9 Substitution

```
<rtlutil.mli content 291a>+≡ (288b) <290e 291b>
module Subst: sig
  val loc:      guard:(Rtl.Private.loc -> bool)
                -> map:(Rtl.Private.loc -> Rtl.Private.loc)
                -> Rtl.rtl -> Rtl.rtl

  val alloc:    guard:(Rtl.Private.loc -> bool)
                -> map:(Rtl.Private.loc -> alloc)
                -> Rtl.rtl -> Rtl.rtl

  val exp:      guard:(Rtl.Private.exp -> bool)
                -> map:(Rtl.Private.exp -> Rtl.Private.exp)
                -> Rtl.rtl -> Rtl.rtl

  val exp_of_exp: guard:(Rtl.Private.exp -> bool)
                  -> map:(Rtl.Private.exp -> Rtl.Private.exp)
                  -> Rtl.exp -> Rtl.exp

  val exp_of_loc: guard:(Rtl.Private.exp -> bool)
                  -> map:(Rtl.Private.exp -> Rtl.Private.exp)
                  -> Rtl.loc -> Rtl.loc

  val loc_of_loc: guard:(Rtl.Private.loc -> bool)
                  -> map:(Rtl.Private.loc -> Rtl.Private.loc)
                  -> Rtl.loc -> Rtl.loc

  val reg:      map:(Register.t -> Register.t) -> Rtl.rtl -> Rtl.rtl

  val reg_def:  map:(Register.t -> Register.t) -> Rtl.rtl -> Rtl.rtl

  val reg_use:  map:(Register.t -> Register.t) -> Rtl.rtl -> Rtl.rtl

  val reg_to_mem: map:(Register.t -> Rtl.Private.loc) -> Rtl.rtl -> Rtl.rtl

  module Fetch : sig (* substitute for Fetch(l, w) *)
    val rtl:      guard:(Rtl.Private.loc -> bool) ->
                  fetch:(Rtl.Private.loc -> Rtl.width -> Rtl.Private.exp) ->
                  Rtl.rtl -> Rtl.rtl

    val exp':     guard:(Rtl.Private.loc -> bool) ->
                  fetch:(Rtl.Private.loc -> Rtl.width -> Rtl.Private.exp) ->
                  Rtl.Private.exp -> Rtl.Private.exp
  end
end
```

K.13.10 Classification

```
<rtlutil.mli content 291b>+≡ (288b) <291a 291c>
module RTLType: sig
  val singleAssignment: Rtl.rtl -> (Register.t * Register.t) option
end
```

K.13.11 RTL AST Representation

```
<rtlutil.mli content 291c>+≡ (288b) <291b 292a>
module ToAST: sig
  type verbosity = Low | High (* default is High *)
  val verbosity: verbosity -> verbosity (* returns old value *)
end
```

```

    val expr:          Rtl.exp  -> Ast.expr
    val rtl:           Rtl.rtl  -> Ast.stmt
end

```

K.13.12 RTL String Representation

```

⟨rtlutil.mli content 292a⟩+≡ (288b) ◁291c
module ToUnreadableString: sig
    val regx: Register.x -> string
    val reg:  Register.t -> string
    val rtl:  Rtl.rtl -> string
    val exp:  Rtl.exp -> string
    val loc:  Rtl.loc -> string
    val agg:  Rtl.aggregation -> string
end
module ToString: sig
    val reg: Register.t -> string
    val rtl: Rtl.rtl -> string
    val exp: Rtl.exp -> string
    val loc: Rtl.loc -> string
    val const: Rtl.Private.const -> string
end

```

K.13.13 Implementation

```

⟨rtlutil.ml content 292b⟩≡ (288a) 292c▷
open Nopoly

module R    = Rtl
module RP   = Rtl.Private
module Up   = Rtl.Up
module Dn   = Rtl.Dn
module Rg   = Register
module RS   = Register.Set
module RSX  = Register.SetX

let id      x = x
let falsef x = false

let unimpf fmt = Printf.kprintf Impossible.unimp fmt
let sprintf = Printf.sprintf

type aloc =
{ fetch  : Rtl.width -> Rtl.exp
; store  : Rtl.exp -> Rtl.width -> Rtl.rtl
}

module Width = struct
    ⟨Width 293c⟩
end

let fetch l    = R.fetch l    (Width.loc l)
let store l e  = R.store l e  (Width.loc l)

⟨rtlutil.ml content 292c⟩+≡ (288a) ◁292b 293a▷
⟨define common ReadWriteKill Higher Order Fns 294a⟩

module ReadWrite = struct

```

```

    <ReadWrite 294b>
end

module ReadWriteKill = struct
    <ReadWriteKill 295a>
end

module FullReadWriteKill = struct
    <FullReadWriteKill 295b>
end

module Subst = struct
    <Subst 295c>
end

module ToString = struct
    <ToString 300a>
end

module ToAST = struct
    <ToAST 298b>
end

module ToUnreadableString = struct
    <ToUnreadableString 302>
end

module Time = struct
    <Time 301b>
end

<rtlutil.ml content 293a>+≡ (288a) <292c 293b>
let rec is_hardware = function
  | RP.Reg r -> true
  | RP.Slice (w, lsb, r) -> is_hardware r
  | _ -> false

let rec to_hardware = function
  | RP.Reg r -> Rg.Reg r
  | RP.Slice (w, lsb, r) ->
    (match to_hardware r with
     | Rg.Reg r -> Rg.Slice (w, lsb, r)
     | Rg.Slice (w', lsb', r) -> Rg.Slice(w', lsb+lsb', r))
  | l -> Printf.kprintf Impossible.impossible
    "to_hardware given non-hardware location %s" (ToString.location' l)

<rtlutil.ml content 293b>+≡ (288a) <293a 303>
module RTLType = struct
    <RTLType 298a>
end

<Width 293c>≡ (292b) 293d>
let loc = Rtl.locwidth

<Width 293d>+≡ (292b) <293c
let rec const = function
  | RP.Bool _ -> Impossible.impossible "asked width of Boolean"
  | RP.Bits b -> Bits.width b
  | RP.Link (_,_,w) -> w
  | RP.Diff (c,_) -> const c

```

```

| RP.Late (_,w)    -> w

let exp' = function
| RP.Const(c)      -> const c
| RP.Fetch(_,w)    -> w
| RP.App (op,_)    ->
  ( match snd (Rtlop.mono (Rtl.Up.opr op)) with
  | Types.Bits n   -> n
  | Types.Bool     -> Impossible.impossible "asked width of Boolean operator"
  )

let exp e = exp' (Rtl.Dn.exp e)

⟨define common ReadWriteKill Higher Order Fns 294a⟩≡ (292c)
let fold handle_reg handle_slice handle_mem ~read ~write ~kill r z =
  let rec rtl (z:'a) (RP.Rtl gs) = List.fold_left guarded z gs
    and guarded z (g, eff) = exp (effect z eff) g
    and effect z eff = match eff with
    | RP.Store (lhs, rhs, _)      -> loc write (exp z rhs) lhs
    | RP.Kill lhs                 -> loc kill z lhs
    and loc which z l = match l with
    | RP.Mem (space, w, addr, _) as m -> exp (handle_mem which z m) addr
    | RP.Reg r                       -> handle_reg      which z r
    | RP.Slice (w, i, l)             -> handle_slice loc which z (w, i, l)
    | RP.Var _                      -> z
    | RP.Global _                   -> z
    and exp z e = match e with
    | RP.Const _                   -> z
    | RP.Fetch (l, w)              -> loc read z l
    | RP.App (_, es)               -> List.fold_left exp z es
  in
  rtl z (Rtl.Dn.rtl r)

let fold_regx ~read ~write ~kill r z =
  fold (fun which z r -> which (Rg.Reg r) z)
    (fun loc which z (w, i, l) -> match l with
    | RP.Reg r -> which (Rg.Slice (w, i, r)) z
    | _       -> loc which z l)
    (fun _ z _ -> z)
  ~read ~write ~kill r z

let fold_regt ~read ~write ~kill r z =
  fold (fun which z r -> which r z)
    (fun loc which z (w, i, l) -> loc which z l)
    (fun _ z _ -> z)
  ~read ~write ~kill r z

⟨ReadWrite 294b⟩≡ (292c)
type 'a observert = Register.t -> 'a -> 'a
type 'a observerx = Register.x -> 'a -> 'a

let fold      ~read ~write r z = fold_regx ~read ~write ~kill:write r z
let fold_promote ~read ~write r z = fold_regt ~read ~write ~kill:write r z

let mk_sets fold add empty rtl =
  let add_left  r (left,right) = (add r left, right) in
  let add_right r (left,right) = (left, add r right) in
  let empty = (empty, empty) in
  fold ~read:add_left ~write:add_right rtl empty
let sets      rtl = mk_sets fold      RSX.add RSX.empty rtl
let sets_promote rtl = mk_sets fold_promote RS.add RS.empty rtl

```

ReadWriteKill 295a)≡ (292c)

```

type 'a observert = Register.t -> 'a -> 'a
type 'a observerx = Register.x -> 'a -> 'a
let fold          = fold_regx
let fold_promote  = fold_regt

let mk_sets fold add empty rtl =
  let read  reg (r, w, k) = (add reg r, w, k) in
  let write reg (r, w, k) = (r, add reg w, k) in
  let kill  reg (r, w, k) = (r, w, add reg k) in
  fold ~read ~write ~kill rtl (empty, empty, empty)

let sets          rtl = mk_sets fold          RSX.add RSX.empty rtl
let sets_promote rtl = mk_sets fold_promote RS.add  RS.empty  rtl

```

FullReadWriteKill 295b)≡ (292c)

```

type 'a observer = RP.loc -> 'a -> 'a
let fold ~read ~write ~kill r z =
  fold (fun which z r -> which (RP.Reg r) z)
    (fun loc which z (w, i, l) -> (* ??? *))
    match l with
    | RP.Reg r -> which (RP.Slice (w, i, l)) z
    | _         -> loc which z l)
    (fun which z m -> which m z)
  ~read ~write ~kill r z

```

K.13.14 Substitutions inside RTLs (Second Try)

Subst 295c)≡ (292c) 295d▷

```

module DownUp = struct
  let rtl f r = Rtl.Up.rtl (f (Rtl.Dn.rtl r))
  let loc f l = Rtl.Up.loc (f (Rtl.Dn.loc l))
  let exp f e = Rtl.Up.exp (f (Rtl.Dn.exp e))
end

```

Subst 295d) +≡ (292c) ◁295c 296▷

```

let loc_gen' guard map rtl ~def ~use =
  let rec exp = function
    | RP.Fetch (l, width) -> RP.Fetch(loc use l, width)
    | RP.App(opr, exprs)  -> RP.App(opr, List.map exp exprs)
    | x                   -> x
  and loc act l = if act && guard l then map l else match l with
    | RP.Mem (sp,w,e,ass) -> RP.Mem(sp, w, exp e, ass)
    | RP.Slice (w,i,l)    -> RP.Slice(w,i,loc act l)
    | x                   -> x
  and effect = function
    | RP.Store (l,e,width) -> RP.Store(loc def l, exp e, width)
    | RP.Kill  (l)         -> RP.Kill (loc def l)
  and guarded (e,eff)      = (exp e, effect eff)
  and subst (RP.Rtl rtl)   = RP.Rtl (List.map guarded rtl)
  in
    subst rtl

let loc'      guard map rtl = loc_gen' guard map rtl ~def:true ~use:true
let loc_def'  guard map rtl = loc_gen' guard map rtl ~def:true ~use:false
let loc_use'  guard map rtl = loc_gen' guard map rtl ~def:false ~use:true

let alloc guard map rtl =
  let rec exp = function
    | RP.Fetch (l, width) -> fetch l width

```



```

  | RP.App(opr, exprs)  -> RP.App(opr, List.map exp exprs)
  | RP.Const c as e     -> e
and fetch l  w = if guard l then Dn.exp((map l).fetch w) else RP.Fetch (loc l, w)
and store l e w =
  if guard l then
    match Dn.rtl ((map l).store (Up.exp e) w) with
    | RP.Rtl [RP.Const (RP.Bool true), eff] -> eff
    | _ -> unimpf "aloc substitution on fancy location"
  else
    RP.Store (loc l, e, w)
and loc = function
  | RP.Mem (sp,w,e,ass) -> RP.Mem(sp, w, exp e, ass)
  | RP.Slice (w,i,l)    -> if guard l then unimpf "aloc slice"
                           else RP.Slice(w,i,loc l)
  | (RP.Global _ | RP.Var _ | RP.Reg _) as l -> l
and effect = function
  | RP.Store (l,e,width) -> store l (exp e) width
  | RP.Kill (l)          -> if guard l then unimpf "aloc kill " else RP.Kill (loc l)
and guarded (e,eff)      = (exp e, effect eff)
and subst (RP.Rtl rtl)   = RP.Rtl (List.map guarded rtl) in
subst rtl

```

```

let subst_exp_loc ~eguard ~emap ~lguard ~lmap =
  let rec subst_exp e = if eguard e then emap e else match e with
    | RP.Fetch (l, width) -> RP.Fetch(subst_loc l, width)
    | RP.App(opr, exprs)  -> RP.App(opr, List.map subst_exp exprs)
    | x                   -> x
  and subst_loc l = if lguard l then lmap l else match l with
    | RP.Mem (sp,w,e,ass) -> RP.Mem(sp, w, subst_exp e, ass)
    | RP.Slice (w,i,l)    -> RP.Slice(w,i,subst_loc l)
    | x                   -> x
  (subst_exp, subst_loc)
let exp' eguard emap rtl =
  let (exp, loc) = subst_exp_loc ~eguard ~emap ~lguard:falsef ~lmap:id in
  let effect = function
    | RP.Store (l,e,width) -> RP.Store(loc l, exp e, width)
    | RP.Kill (l)          -> RP.Kill (loc l)
  in
  let guarded (e,eff)      = (exp e, effect eff)
  in
  let subst (RP.Rtl rtl)   = RP.Rtl (List.map guarded rtl)
  in
  subst rtl

```

$\langle \text{Subst } 296 \rangle + \equiv$ (292c) $\triangleleft 295d \ 297a \triangleright$

```

let reg_gen' map rtl loc' =
  let map = function RP.Reg r -> RP.Reg(map r) | _ -> assert false in
  let is_register = function RP.Reg _ -> true | _ -> false in
  loc' is_register map rtl

```

```

let reg_to_mem_gen' map rtl loc' =
  let map = function RP.Reg r -> map r | _ -> assert false in
  let is_register = function RP.Reg _ -> true | _ -> false in
  loc' is_register map rtl

```

```

let reg' map rtl      = reg_gen' map rtl loc'
let reg_def' map rtl = reg_gen' map rtl loc_def'
let reg_use' map rtl = reg_gen' map rtl loc_use'
let reg_to_mem' map rtl      = reg_to_mem_gen' map rtl loc'

```

```

let exp      ~guard ~map rtl = DownUp.rtl (exp' guard map) rtl
let exp_of_exp ~guard ~map exp =

```

```

let (subst_exp,_) = subst_exp_loc ~eguard:guard ~emap:map ~lguard:falsef ~lmap:id in
DownUp.exp subst_exp exp

let exp_of_loc ~guard ~map loc =
  let (_,subst_loc) = subst_exp_loc ~eguard:guard ~emap:map ~lguard:falsef ~lmap:id in
  DownUp.loc subst_loc loc

let loc_of_loc ~guard ~map loc =
  let (_,subst_loc) = subst_exp_loc ~eguard:falsef ~emap:id ~lguard:guard ~lmap:map in
  DownUp.loc subst_loc loc

let loc      ~guard ~map rtl = DownUp.rtl (loc' guard map) rtl
let aloc     ~guard ~map rtl = DownUp.rtl (aloc guard map) rtl
let reg      ~map rtl = DownUp.rtl (reg' map) rtl
let reg_def  ~map rtl = DownUp.rtl (reg_def' map) rtl
let reg_use  ~map rtl = DownUp.rtl (reg_use' map) rtl
let reg_to_mem ~map rtl = DownUp.rtl (reg_to_mem' map) rtl

⟨Subst 297a⟩+≡ (292c) <296
module Fetch = struct
  let rec exp ~guard ~fetch = function
    | RP.Fetch (l, width) when guard l -> fetch l width
    | RP.Fetch (l, width) -> RP.Fetch (loc ~guard ~fetch l, width)
    | RP.App(opr, exprs) -> RP.App(opr, List.map (exp ~guard ~fetch) exprs)
    | RP.Const _ as e -> e
  and loc ~guard ~fetch l = match l with
  | RP.Mem (sp, w, e, ass) -> RP.Mem(sp, w, exp ~guard ~fetch e, ass)
  | RP.Slice (w,i,l) -> RP.Slice(w,i,loc ~guard ~fetch l)
  | RP.Reg _ | RP.Var _ | RP.Global _ -> l
  let rtl ~guard ~fetch rtl =
    let RP.Rtl effs = Dn.rtl rtl in
    let loc = loc ~guard ~fetch in
    let exp = exp ~guard ~fetch in
    let effect = function
      | RP.Store (l,e,width) -> RP.Store(loc l, exp e, width)
      | RP.Kill (l) -> RP.Kill (loc l) in
    let guarded (e,eff) = (exp e, effect eff) in
    Up.rtl (RP.Rtl (List.map guarded effs))
  let exp' = exp
end

```

K.13.15 RTL Classification

```

⟨old RTLType 297b⟩≡
let singleAssignment rtl =
  let rec firstAssignment rtls = match rtls with
  | [] -> (None, [])
  | (RP.Const(RP.Bool true), RP.Store(loc1, RP.Fetch(loc2, _), _))::rst ->
    (try (Some (Register.of_loc (Rtl.Up.loc loc1),
      Register.of_loc (Rtl.Up.loc loc2)),
      rst)
    with Invalid_argument _ -> (None, []))
  | _::rst -> firstAssignment rst in
  let getFirstAssignment (RP.Rtl guarded) = firstAssignment guarded in

  let isKill rtl = match rtl with
  | (RP.Const(RP.Bool true), RP.Kill _) -> true
  | _ -> false in
  let rec allKills rtls = List.for_all isKill rtls in

```

```

let (assign, rst) = getFirstAssignment (Rtl.Dn.rtl rtl) in
  match assign with
  | Some _ when allKills rst -> assign
  | _ -> None

```

$\langle \text{RTLType } 298a \rangle \equiv$ (293b)

```

let singleAssignment rtl =
  let is_truth = function RP.Const(RP.Bool true) -> true | _ -> false in
  let is_move = function (* effect is an uncond. move *)
    | (g, RP.Store(RP.Reg(dst), RP.Fetch(RP.Reg(src), _), _))
      when is_truth g -> Some(dst, src)
    | _ -> None in
  let is_kill = function (* effect is an uncond. kill *)
    | (g, RP.Kill _) -> is_truth g
    | _ -> false in
  match Rtl.Dn.rtl rtl with
  | RP.Rtl([]) -> None
  | RP.Rtl(fst::rst) -> if List.for_all is_kill rst then is_move fst else None

```

K.13.16 Conversion to AST

$\langle \text{ToAST } 298b \rangle \equiv$ (292c) 298c▷

```

module R = Rtl.Private
module A = Ast
module T = Types

```

$\langle \text{ToAST } 298c \rangle + \equiv$ (292c) ◁298b 298d▷

```

type verbosity = Low | High (* default is High --- more accurate *)
let the_verbosity = ref High
let verbosity v =
  let v' = !the_verbosity in
  the_verbosity := v;
  v'

```

$\langle \text{ToAST } 298d \rangle + \equiv$ (292c) ◁298c 298e▷

```

let bits n = A.BitsTy n
let rec const = function
| R.Bool(true) -> A.PrimOp ("true", [])
| R.Bool(false) -> A.PrimOp ("false", [])
| R.Bits b ->
  let w = match !the_verbosity with
  | High -> Some (bits (Bits.width b))
  | Low -> None in
  (try A.Sint (string_of_int (Bits.S.to_int b), w)
  with Bits.Overflow -> A.Uint (Bits.to_string b, w))
| R.Link(sym, _, w) -> A.Fetch(A.Name(None, sym#mangled_text, None))
| R.Diff(c1, c2) -> A.PrimOp("-", [(None, const c1, None); (None, const c2, None)])
| R.Late(name, w) -> A.Fetch(A.Name(None, name, None)) (*XXX ok? *)

```

$\langle \text{ToAST } 298e \rangle + \equiv$ (292c) ◁298d 299a▷

```

let opr op w =
  sprintf "%s[%s]" op (String.concat "," (List.map string_of_int w))

let rec expr' = function
| R.Const c -> const c
| R.Fetch(loc, _) -> rvalue loc
| R.App((op, ww), ee) -> let actual e = (None, expr' e, None) in
  match ee with

```

```

| [l; r] when op == "add" || op == "sub" ->
  let op = if op == "add" then "+" else "-" in
  A.BinOp(expr' l, op, expr' r)
| _ -> A.PrimOp(opr op ww, List.map actual ee)

```

(ToAST 299a) +=

(292c) <298e 299b>

```

and rvalue = function
| R.Slice (w, i, loc) ->
  let e = rvalue loc in
  let e = if i = 0 then e
    else A.PrimOp("shrl", [None, e, None;
      None, A.Sint (string_of_int i, None), None]) in
  let e = A.PrimOp("lobits" ^ string_of_int w, [None, e, None]) in
  e
| loc -> A.Fetch(location' loc)

and location' =
let name n = A.Name (None, n, None) in
function
| R.Mem ((sp,agg,ms),c,e,ass) ->
  let width = Cell.to_width ms c in
  (match !the_verbosity with
  | High ->
    let a = match agg with
      | Rtl.Identity -> 'I'
      | Rtl.BigEndian -> 'B'
      | Rtl.LittleEndian -> 'L' in
    let space = sprintf "%c(%d%c)" sp width a in
    let aligned = if ass = 1 then None else Some ass in
    A.Mem(A.TypeSynonym space,expr' e,aligned,
      if sp <= 'm' then [] else ["'" ^ Char.escaped sp ^ "'"])
  | Low ->
    A.Mem(A.BitsTy width,expr' e, (if ass = 1 then None else Some ass),
      if sp <= 'm' then [] else ["'" ^ Char.escaped sp ^ "'"]))
| R.Reg((sp,_,ms),i,c) ->
  (match !the_verbosity with
  | High ->
    let space = sprintf "%c(%d)" sp (Cell.to_width ms c) in
    let s = string_of_int i in
    A.Mem(A.TypeSynonym space,A.Sint(s,None), None, [])
  | Low -> name (sprintf "%c%d" sp i))
| R.Var (n,_,width) -> name n
| R.Global (n,_,width) -> name n
| R.Slice (w,i,loc) ->
  name (sprintf "Slice(%d, %d, %s)" w i (ToString.location' loc))

let (<<) f g = fun x -> f (g x)
let expr = expr' << Rtl.Dn.exp
let location = location' << Rtl.Dn.loc

```

(ToAST 299b) +=

(292c) <299a

```

let rec effect guard = function
| R.Kill loc ->
  let width = Width.loc (Rtl.Up.loc loc) in
  let undef = R.App(("undef",[width]),[]) in
  effect guard (R.Store(loc, undef, width))
| R.Store (loc,e,w) ->
  (location' loc,(guard, expr' e))

```

```

let guard = function
  | R.Const (R.Bool true) -> None
  | g -> Some (expr' g)

let guarded (g, eff) = effect (guard g) eff

let rtl' (R.Rtl gg) =
  match gg with
  | [] -> A.EmptyStmt
  | _ :: _ ->
    let pairs = List.map (guarded ) gg in
    let lhs, rhs = List.split pairs in
    A.AssignStmt(lhs,rhs)
let rtl = rtl' << Rtl.Dn.rtl

```

K.13.17 Conversion to Readable String

<ToString 300a>≡ (292c) 300b▷

```

module R = Rtl.Private
module A = Ast
module T = Types

```

<ToString 300b>+≡ (292c) <300a 300c>

```

let rec const = function
  | R.Bool true -> "true"
  | R.Bool false -> "false"
  | R.Bits b -> Int64.to_string (Bits.S.to_int64 b)
  | R.Link(sym,_,w) -> sym#mangled_text
  | R.Diff(c1,c2) -> (const c1) ^ "-" ^ (const c2)
  | R.Late(name,w) -> "<" ^ name ^ ">"

```

<ToString 300c>+≡ (292c) <300b 300d>

```

let opr op w = match op, w with
| ("sx" | "zx" | "lobits" |
   "i2f" | "f2i" | "f2f" ), [w1;w2] -> "%" ^ op ^ string_of_int w2
| _, _ -> "%" ^ op

let join = String.concat

let rec expr' = function
  | R.Const c -> const c
  | R.Fetch(loc,_) -> location' loc
  | R.App((op,ww),ee) -> match ee with
    | [l; r] when op =$= "add" || op =$= "sub" ->
      let op = if op =$= "add" then " + " else " - " in
      brexpr' l ^ op ^ brexpr' r
    | _ -> opr op ww ^ "(" ^ join ", " (List.map expr' ee) ^ ")"
and brexpr' e = match e with (* bracketed expression *)
  | R.App ((op,ww), _) when op =$= "add" || op =$= "sub" -> "(" ^ expr' e ^ ")"
  | _ -> expr' e

```

<ToString 300d>+≡ (292c) <300c 301a>

```

and agg = function
  | Rtl.BigEndian -> "B"
  | Rtl.LittleEndian -> "L"
  | Rtl.Identity -> "I"
and location' = function
  | R.Mem ((sp,a,ms),c,e,ass) ->
    let space = if sp <= 'm' then "bits" ^ string_of_int (Cell.to_width ms c)

```

```

        else "$" ^ Char.escaped sp in
          sprintf "%s%s[%s]" space (agg a) (expr' e)
      | R.Reg r -> reg r
      | R.Var (name,_,width) -> name
      | R.Global (name,_,width) -> name
      | R.Slice (w,i,loc) ->

        location' loc ^ "@[" ^ string_of_int i ^ ".." ^ string_of_int (i+w-1) ^ "]"

and reg ((sp,a,_),i,R.C n) =
  if n = 1 then
    sprintf "$%s[%d]" (Char.escaped sp) i
  else
    sprintf "$%s[%d:%d%s]" (Char.escaped sp) i n (agg a)

let (<<) f g = fun x -> f (g x)
let expr' = expr' << Rtl.Dn.exp
let loc = location' << Rtl.Dn.loc

<ToString 301a>+≡ (292c) <300d
let rec effect = function
  | R.Kill loc -> "kill " ^ location' loc
  | R.Store (loc,e,w) -> location' loc ^ " := " ^ expr' e

let guard = function
  | R.Const (R.Bool true) -> ""
  | g -> expr' g ^ " --> "

let guarded (g, eff) = guard g ^ effect eff

let rtl' (R.Rtl gg) =
  match gg with
  | [] -> "skip"
  | _ :: _ -> join " | " (List.map guarded gg)
let rtl = rtl' << Rtl.Dn.rtl
let exp = expr

```

K.13.18 Evaluation Time Classification

```

<Time 301b>≡ (292c)
let compile e = match Dn.exp e with
  | RP.Const(RP.Bits _) -> true
  | RP.Const(RP.Bool _) -> true
  | RP.Const(RP.Late _) -> true
  | _ -> false

let link e = match Dn.exp e with
  | RP.Const(RP.Bits _) -> true
  | RP.Const(RP.Bool _) -> true
  | RP.Const(RP.Late _) -> true
  | RP.Const(RP.Link _) -> true
  | RP.Const(RP.Diff _) -> true
  | RP.App(("add",_), [RP.Const(RP.Link _); RP.Const(RP.Bits _)])
    -> true
  | RP.App(("add",_), [RP.Const(RP.Diff _); RP.Const(RP.Bits _)])
    -> true
  | _ -> false

```

K.13.19 Conversion to String

```
(ToUnreadableString 302)≡ (292c)
module P = Pp
module R = Rtl
module RP = Rtl.Private

let (^^) = P.(^^)
let (^/) = P.(^/)

let indent x = P.nest 4 (P.break ^^ x)
let int i = P.text (string_of_int i)
let str s = P.text s
let char c = P.text (sprintf "'%c'" c)

let tuple docs =
  P.agrp (P.text "(" ^^ P.list (P.text "," ^^ P.break) id docs ^^ P.text ")")

let apply c args = P.agrp(str c ^^ indent(tuple args))

let aggregation' = function
| R.BigEndian -> P.text "BE"
| R.LittleEndian -> P.text "LE"
| R.Identity -> P.text "ID"

let agg = function
| R.BigEndian -> "big"
| R.LittleEndian -> "little"
| R.Identity -> "none"

let opr (name, ws) =
  tuple [str name; tuple (List.map int ws)]

let rec const = function
| RP.Bool(b) -> apply "Bool" [str (if b then "true" else "false")]
| RP.Bits(b) -> apply "Bits" [ str (Int64.to_string (Bits.U.to_int64 b))
                               ; int (Bits.width b)
                               ]
| RP.Link(r,k,w) -> apply "Link" [str r#mangled_text; P.text (kind k); int w]
| RP.Diff(c1,c2) -> apply "Diff" [const c1; const c2]
| RP.Late(s,w) -> apply "Late" [str s; int w]
and kind = function RP.Code -> "Code" | RP.Data -> "Data" | RP.Imported -> "Imported"

let rec exp = function
| RP.Const(k) -> apply "Const" [const k]
| RP.Fetch(l,w) -> apply "Fetch" [loc l; int w]
| RP.App(o,es) -> apply "App" [opr o; tuple (List.map exp es)]

and loc = function
| RP.Mem((sp, agg, ms), c, e, ass) ->
  apply "Mem" [char sp; aggregation' agg; int (Cell.to_width ms c); exp e]
| RP.Reg((sp,agg,ms),i,R.C c) ->
  apply "Reg" [char sp; int i; str (sprintf "C %d" c)]
| RP.Var(name, index, w) ->
  apply "Var" [str name; int index; int w]
| RP.Global(name, index, w) ->
  apply "Global" [str name; int index; int w]
| RP.Slice(w, i, l) ->
  apply "Slice" [int w; int i; loc l]
```

```

let effect = function
  | RP.Store(l,e,w) -> apply "Store" [loc l; exp e; int w]
  | RP.Kill(l)      -> apply "Kill"  [loc l]

let guarded (g, e) =
  tuple [exp g; effect e]

let rtl' (RP.Rtl x) =
  apply "Rtl" (List.map guarded x)

let rtl r =
  let pp = rtl' (R.Dn.rtl r) in
  Pp.ppToString 66 (* line width *) pp

let exp e =
  let pp = exp (R.Dn.exp e) in
  Pp.ppToString 66 (* line width *) pp

let reg' ((sp,agg,ms),i,R.C c) =
  tuple [char sp; int i; str (sprintf "C %d" c)]
let reg r =
  Pp.ppToString 66 (P.agrp (reg' r))

let regx r = match r with
  | Rg.Reg r -> Pp.ppToString 66 (P.agrp (reg' r))
  | Rg.Slice (w, lsb, r) ->
    Pp.ppToString 66 (P.agrp ( reg' r ^^ str "@" ^^ int lsb ^^ str ":" ^^ int w))

let loc l = Pp.ppToString 66 (loc (Dn.loc l))

```

K.13.20 Aliasing

```

<rtlutil.ml content 303>+≡ (288a) <293b 305a>
module Down = Rtl.Dn
module MayAlias = struct
  let regs ((s,_,_), i, R.C c) ((s',_,_), i', R.C c') =
    s <= s' && not (i + c <= i' || i' + c' <= i)

  let () = Debug.register "may-alias" "check register may-alias function"

  let regs =
    if Debug.on "may-alias" then
      let regs ((s,_,_), i, R.C c as r) ((s',_,_), i', R.C c' as r') =
        let good = regs r r' in
        let what p = if p then "may" else "may not" in
        ( Printf.eprintf "Comparing %c[%d:%d] and %c[%d:%d]: %s alias\n"
          s i c s' i' c' (what good)
          ; good
          ) in
      regs
    else
      regs

  let with_reg r = function
    | RP.Reg r' -> regs r r'
    | _ -> false

```



```

let with_vari vi = function
| RP.Var (_, vi', _) -> vi = vi'
| _ -> false

let with_globali vi = function
| RP.Global (_, vi', _) -> vi = vi'
| _ -> false

type slot = string * int (* symbol and offset *)

let stack_slot = function
| RP.Fetch(RP.Reg(('V',_,_), 0, _), _) -> Some("", 0)
| RP.App(("add", [_]), [RP.Fetch(RP.Reg(('V',_,_), 0, _), _); offset]) ->
  (match offset with
  | RP.Const (RP.Bits k') -> Some("", Bits.S.to_int k')
  | RP.Const (RP.Late (s, _)) -> Some(s, 0)
  | RP.App(("add", [_]), [RP.Const l; RP.Const r]) ->
    (match l, r with
    | RP.Late (s, _), RP.Bits k -> Some(s, Bits.S.to_int k)
    | RP.Bits k, RP.Late (s, _) -> Some(s, Bits.S.to_int k)
    | _ -> None)
  | _ -> None)
| _ -> None

let rec is_initialized_data = function
| RP.Const (RP.Link(_, _, _)) -> true
| RP.Const (RP.Diff(_, _)) -> true
| RP.App(("add"|"sub", [_]), [RP.Const _ as l; RP.Const _ as r]) ->
  is_initialized_data l || is_initialized_data r
| _ -> false

let with_mem (s,_,_) (R.C c) e = function (* different stack slots don't alias *)
| RP.Mem ((s',_,_), R.C c', e', _) ->
  s <= s' &&
  (match stack_slot e, stack_slot e' with
  | None, Some _ -> not (is_initialized_data e)
  | Some _, None -> not (is_initialized_data e')
  | None, None -> true
  | Some (sym, n), Some(sym', n') ->
    (*
    Printf.eprintf "--> comparing stack slots %s+%d and %s+%d\n" sym n sym' n';
    *)
    sym = $ = sym' && not (n + c <= n' || n' + c' <= n))
| _ -> false

let unslice f l =
  let rec un = function
  | RP.Slice (_, _, l) -> un l
  | l -> l
  in f (un l)

let rec locs' l = match l with
| RP.Reg r -> unslice (with_reg r)
| RP.Var (_, i, _) -> unslice (with_vari i)
| RP.Global (_, i, _) -> unslice (with_globali i)
| RP.Mem (s, w, e, _) -> unslice (with_mem s w e)
| RP.Slice (_, _, l) -> locs' l
and locs l =
  let alias = locs' (Down.loc l) in
  fun l' -> alias (Down.loc l')
```

```

let has_loc f e =
  let rec has es ess = match es with
  | [] -> (match ess with [] -> false | es :: ess -> has es ess)
  | RP.Const _ :: es -> has es ess
  | RP.Fetch (l, w) :: es ->
    f l || (match l with
    | RP.Mem (_, _, e, _) -> has (e :: es) ess
    | _ -> has es ess)
  | RP.App (_, es') :: es -> has es (es' :: ess) in
  has [e] []

let exp' l =
  let may_alias = has_loc (locs' l) in
  fun e ->
    let answer = may_alias e in
    let module S = ToString in
    let _debug () =
      Printf.eprintf "*** Expression %s %s alias with location %s\n"
        (S.exp (Rtl.Up.exp e)) (if answer then " may " else " may not ")
        (S.loc (Rtl.Up.loc l)) in
    answer

let exp l =
  let alias = exp' (Down.loc l) in
  fun e -> alias (Down.exp e)

let store_uses' l (l', r', _) =
  exp' l r' || match l' with RP.Mem (_, _, e, _) -> exp' l e | _ -> false
end

```

<rtlutil.ml content 305a>+≡

(288a) <303 305b>

K.14 Reassociating arithmetic

<rtlutil.ml content 305b>+≡

(288a) <305a 306>

```

let add' w =
  let addop = R.opr "add" [w] in
  fun x y -> R.app addop [x; y]

let rec is_sum_of_constants = function
| RP.Const (RP.Bits _ | RP.Late (_, _)) -> true
| RP.App (((("add"), [w])), es) -> List.for_all is_sum_of_constants es
| _ -> false

let addc w =
  let add' = add' w in
  let addconst x y = match x, y with
  | RP.Bits b, RP.Bits b' -> R.bits (Bits.Ops.add b b') w
  | c, c' -> add' (Up.const c) (Up.const c') in
  fun x y ->
    match Down.exp x with
    | RP.App (("add", [w']), [x1; RP.Const c]) -> add' (Up.exp x1) (addconst c y)
    | RP.App (("add", [w']), [x1; x2]) when is_sum_of_constants x2 ->
      add' (Up.exp x1) (add' (Up.exp x2) (Up.const y))
    | _ -> add' x (Up.const y)

```

```

let addk w =
  let add = addc w in
  fun x k -> if k = 0 then x else add x (RP.Bits (Bits.S.of_int k w))

let add w =
  let add = add' w in
  fun x y -> match (Down.exp y) with
  | RP.Const c -> addc w x c
  | _ -> add x y

```

K.14.1 Operator search

```

<rtlutil.ml content 306>+≡ (288a) <305b 307a>
module Exists = struct
  module Loc = struct
    let exp = MayAlias.has_loc
    let rtl p =
      let exp = exp p in
      let effect = function
        | RP.Store (l, r, w) -> exp r || exp (RP.Fetch (l, w))
        | RP.Kill l -> exp (RP.Fetch (l, 0)) in (* width cheat is OK *)
      let ge (guard, eff) = exp guard || effect eff in
      fun (RP.Rtl ges) -> List.exists ge ges
    end
  end
  module Opr = struct
    let rtl p =
      let rec rtl (RP.Rtl gs) = List.exists guarded gs
      and guarded (g, eff) = exp g || effect eff
      and effect = function
        | RP.Store (lhs, rhs, _) -> loc lhs || exp rhs
        | RP.Kill lhs -> loc lhs
      and loc = function
        | RP.Mem (_, _, addr, _) -> exp addr
        | RP.Reg r -> false
        | RP.Slice (_, _, l) -> loc l
        | RP.Var _ -> false
        | RP.Global _ -> false
      and exp = function
        | RP.Const _ -> false
        | RP.Fetch (l, w) -> loc l
        | RP.App (opr, es) -> p (Rtl.Up.opr opr) || List.exists exp es in
      fun r -> rtl (Rtl.Dn.rtl r)
    end
  end
  module Const = struct
    let rtl const_ftn =
      let rec rtl (RP.Rtl gs) = List.exists guarded gs
      and guarded (e, eff) = exp e || effect eff
      and effect eff = match eff with
        | RP.Store(l, e, _) -> loc l || exp e
        | RP.Kill l -> loc l
      and loc l = match l with
        | RP.Mem(_,_,e,_) -> exp e
        | RP.Slice(_,_,l') -> loc l'
        | RP.Reg _ | RP.Var _ | RP.Global _ -> false
      and exp e = match e with
        | RP.Const c -> const c
        | RP.Fetch(l, _) -> loc l
        | RP.App(_, es) -> List.exists exp es
      and const c = const_ftn c in

```

```

    fun r -> rtl (R.Dn.rtl r)
  end
end

⟨rtlutil.ml content 307a⟩+≡ (288a) <306 307b>
module Find = struct
  module Loc = struct
    let exp f e =
      let rec has es ess = match es with
      | [] -> (match ess with [] -> None | es :: ess -> has es ess)
      | RP.Const _ :: es -> has es ess
      | RP.Fetch (l, w) :: es ->
          if f l then Some l
          else ( match l with
                  | RP.Mem (_, _, e, _) -> has (e :: es) ess
                  | _ -> has es ess)
      | RP.App (_, es') :: es -> has es (es' :: ess) in
      has [e] []
    end
  end
end

⟨rtlutil.ml content 307b⟩+≡ (288a) <307a 307c>
module Fold = struct
  module RegX = struct
    let loc f l z =
      let rec exp z = function
      | RP.Const _ -> z
      | RP.Fetch (l, _) -> loc z l
      | RP.App (_, es') -> List.fold_left exp z es'
      and loc z = function
      | RP.Reg r -> f (Rg.Reg r) z
      | RP.Slice (w, i, RP.Reg r) -> f (Rg.Slice (w, i, r)) z
      | RP.Slice (_, _, l) -> loc z l
      | _ -> z in
      loc z (Rtl.Dn.loc l)
    end
  end
  module LocFetched = struct
    let rtl f rtl z =
      let rec exp z e =
        let rec fold z es ess = match es with
        | [] -> (match ess with [] -> z | es :: ess -> fold z es ess)
        | RP.Const _ :: es -> fold z es ess
        | RP.Fetch (l, w) :: es -> let z = f l z in fold (loc z l) es ess
        | RP.App (_, es') :: es -> fold z es (es' :: ess) in
        fold z [e] []
      and loc z = function
      | RP.Mem (_, _, e, _) -> exp z e
      | _ -> z in
      let effect z = function
      | RP.Kill l -> loc z l
      | RP.Store (l, r, w) -> exp (loc z l) r in
      let guarded z (g, e) = effect (exp z g) e in
      let RP.Rtl effs = rtl in
      List.fold_left guarded z effs
    end
  end
end

⟨rtlutil.ml content 307c⟩+≡ (288a) <307b 310a>
⟨comparisons 308⟩
module Eq = struct

```

```

let space ((s:char), _, _) ((s':char), _, _) = s <= s'
let const c c' = Compare.const c c' = 0
let loc l l' = Compare.loc l l' = 0
let exp e e' = Compare.exp e e' = 0
let rtl r r' = Compare.rtl r r' = 0
end

```

<comparisons 308>≡

(307c)

```

module Compare = struct
  let less    = -1
  let equal   = 0
  let greater = 1

  let space (s, _, _) (s', _, _) = comparec s s'

  let rec const c c' = match c, c' with
  | (RP.Bool l, RP.Bool r) -> compareb l r
  | (RP.Bits l, RP.Bits r) -> Bits.compare l r
  | (RP.Link (l1, l2, l3), RP.Link (r1, r2, r3)) ->
    (match Pervasives.compare l1 r1 with
    | 0 ->
      (match Pervasives.compare l2 r2 with
      | 0 -> compare l3 r3
      | diff -> diff)
    | diff -> diff)
  | (RP.Diff (l1, l2), RP.Diff (r1, r2)) ->
    (match const l1 r1 with
    | 0 -> const l2 r2
    | diff -> diff)
  | (RP.Late (l1, l2), RP.Late (r1, r2)) ->
    (match compares l1 r1 with
    | 0 -> comparei l2 r2
    | diff -> diff)
  | (RP.Bool _, RP.Bits _) -> less
  | (RP.Bool _, RP.Link _) -> less
  | (RP.Bool _, RP.Diff _) -> less
  | (RP.Bool _, RP.Late _) -> less
  | (RP.Bits _, RP.Bool _) -> greater
  | (RP.Bits _, RP.Link _) -> less
  | (RP.Bits _, RP.Diff _) -> less
  | (RP.Bits _, RP.Late _) -> less
  | (RP.Link _, RP.Bool _) -> greater
  | (RP.Link _, RP.Bits _) -> greater
  | (RP.Link _, RP.Diff _) -> less
  | (RP.Link _, RP.Late _) -> less
  | (RP.Diff _, RP.Bool _) -> greater
  | (RP.Diff _, RP.Bits _) -> greater
  | (RP.Diff _, RP.Link _) -> greater
  | (RP.Diff _, RP.Late _) -> less
  | (RP.Late _, RP.Bool _) -> greater
  | (RP.Late _, RP.Bits _) -> greater
  | (RP.Late _, RP.Link _) -> greater
  | (RP.Late _, RP.Diff _) -> greater
  and symkind k k' = match k, k' with
  | (RP.Code, RP.Code) -> equal
  | (RP.Data, RP.Data) -> equal
  | (RP.Imported, RP.Imported) -> equal
  | (RP.Code, RP.Data) -> less
  | (RP.Code, RP.Imported) -> less
  | (RP.Data, RP.Code) -> greater

```

```

| (RP.Data, RP.Imported) -> less
| (RP.Imported, RP.Code) -> greater
| (RP.Imported, RP.Data) -> greater

let rec exp e e' = match e, e' with
| (RP.Const l, RP.Const r) -> const l r
| (RP.Fetch (l1, l2), RP.Fetch (r1, r2)) ->
  (match loc l1 r1 with
  | 0 -> compare l2 r2
  | diff -> diff)
| (RP.App (l1, l2), RP.App (r1, r2)) ->
  (match Pervasives.compare l1 r1 with
  | 0 -> Auxfun.compare_list exp l2 r2
  | diff -> diff)
| (RP.Const _, RP.Fetch _) -> less
| (RP.Const _, RP.App _) -> less
| (RP.Fetch _, RP.Const _) -> greater
| (RP.Fetch _, RP.App _) -> less
| (RP.App _, RP.Const _) -> greater
| (RP.App _, RP.Fetch _) -> greater
and count (RP.C l) (RP.C r) = compare l r
and loc l l' = match l, l' with
| (RP.Mem (l1, l2, l3, l4), RP.Mem (r1, r2, r3, r4)) ->
  (match space l1 r1 with
  | 0 ->
    (match count l2 r2 with
    | 0 ->
      (match exp l3 r3 with
      | 0 -> Pervasives.compare l4 r4
      | diff -> diff)
    | diff -> diff)
  | diff -> diff)
| (RP.Reg l, RP.Reg r) -> Register.compare l r
| (RP.Var (l1, l2, l3), RP.Var (r1, r2, r3)) -> comparei l2 r2
| (RP.Global (l1, l2, l3), RP.Global (r1, r2, r3)) -> comparei l2 r2
| (RP.Slice (l1, l2, l3), RP.Slice (r1, r2, r3)) ->
  (match comparei l2 r2 with
  | 0 ->
    (match comparei l1 r1 with
    | 0 -> loc l3 r3
    | diff -> diff)
  | diff -> diff)
| (RP.Mem _, RP.Reg _) -> less
| (RP.Mem _, RP.Var _) -> less
| (RP.Mem _, RP.Global _) -> less
| (RP.Mem _, RP.Slice _) -> less
| (RP.Reg _, RP.Mem _) -> greater
| (RP.Reg _, RP.Var _) -> less
| (RP.Reg _, RP.Global _) -> less
| (RP.Reg _, RP.Slice _) -> less
| (RP.Var _, RP.Mem _) -> greater
| (RP.Var _, RP.Reg _) -> greater
| (RP.Var _, RP.Global _) -> less
| (RP.Var _, RP.Slice _) -> less
| (RP.Global _, RP.Mem _) -> greater
| (RP.Global _, RP.Reg _) -> greater
| (RP.Global _, RP.Var _) -> greater
| (RP.Global _, RP.Slice _) -> less
| (RP.Slice _, RP.Mem _) -> greater
| (RP.Slice _, RP.Reg _) -> greater

```

```

| (RP.Slice _, RP.Var _) -> greater
| (RP.Slice _, RP.Global _) -> greater

let effect e e' = match e, e' with
| (RP.Store (l1, l2, l3), RP.Store (r1, r2, r3)) ->
  (match loc l1 r1 with
  | 0 ->
    (match exp l2 r2 with
    | 0 -> compare l3 r3
    | diff -> diff)
  | diff -> diff)
| (RP.Kill l, RP.Kill r) -> loc l r
| (RP.Store _, RP.Kill _) -> -1
| (RP.Kill _, RP.Store _) -> 1
let guarded (l1, l2) (r1, r2) =
  (match exp l1 r1 with
  | 0 -> effect l2 r2
  | diff -> diff)
let rtl (RP.Rtl l) (RP.Rtl r) = Auxfun.compare_list guarded l r
end

```

```

⟨rtlutil.ml content 310a⟩≡ (288a) <307c
let reloc addr w =
  let const b = R.bits b w in
  let sym (s, mk) = mk s w in
  let infix op a b = R.app (R.opr op [w]) [a; b] in
  let add, sub = infix "add", infix "sub" in
  Reloc.fold ~const ~sym ~add ~sub addr

```

K.15 front_rtl/symbol.nw

```

⟨front_rtl/symbol.ml 310b⟩≡
  ⟨symbol.ml content 311a⟩

```

```

⟨front_rtl/symbol.mli 310c⟩≡
  ⟨symbol.mli content 310e⟩

```

K.16 Symbol

```

⟨class type t 310d⟩≡ (311a 310e)
class type t = object
  method mangled_text: string
  method original_text: string
end

```

```

⟨symbol.mli content 310e⟩≡ (310c)
⟨class type t 310d⟩
val pp: Format.formatter -> t -> unit
val unmangled : string -> t
val with_mangler : (string -> string) -> string -> t

```

$\langle \text{symbol.ml content 311a} \rangle \equiv$ (310b)

$\langle \text{class type } t \text{ 310d} \rangle$

```
(* does not work: [@@deriving show] *)
(* manual *)
let pp fmt (x : t) =
  if x#mangled_text <> x#original_text
  then
    Format.fprintf fmt
      "{ mangled_text = %S; original_text = %S }"
      x#mangled_text
      x#original_text
  else
    Format.fprintf fmt "%S" x#original_text

class unmangled (n:string) : t =
object(this)
  method original_text = n
  method mangled_text  = n
end
class mangled (mangle:string->string) (n:string) : t = object
  method mangled_text  = mangle n
  method original_text = n
end
let unmangled n = new unmangled n
let with_mangler m n = new mangled m n
```

K.17 front_rtl/types.nw

$\langle \text{front_rtl/types.ml 311b} \rangle \equiv$

$\langle \text{types.ml content 312f} \rangle$

$\langle \text{front_rtl/types.mli 311c} \rangle \equiv$

$\langle \text{types.mli content 312d} \rangle$

K.18 Types

K.18.1 Interface (Types)

$\langle \text{exported type definitions(types.nw) 311d} \rangle \equiv$

(312) 311e▷

$\langle \text{definition of type size 312e} \rangle$

```
type 'a t = Bool
        | Bits of 'a
```

```
[@@deriving show]
```

```
type ty  = int t
```

```
[@@deriving show]
```

$\langle \text{exported type definitions(types.nw) 311e} \rangle + \equiv$

(312) ◁311d

```
type tyscheme = (size t) list * (size t)
```

```
type monotype = (int t) list * (int t)
```

K.18.2 Interface (Functions)

$\langle \text{appl 311f} \rangle \equiv$

(312d) 312a▷

```
val appl      : string -> tyscheme -> ty list -> ty      (* raises Error.ErrorExn *)
```

```
val widthlist : string -> tyscheme -> ty list -> int list (* raises Error.ErrorExn *)
```



```

⟨appl 312a⟩+≡ (312d) <311f 312b>
  val split : string -> string * int option (* RTL op name, return width *)

⟨appl 312b⟩+≡ (312d) <312a
  val instantiate: tyscheme -> widths:int list -> monotype

```

K.18.3 Useful abbreviations

```

⟨abbrevs 312c⟩≡ (312d)
  val fixbits : int -> size t (* fixed/constant size *)
  val var : key -> size t (* variable size *)
  val double : key -> size t (* doubled width - see above *)
  val half : key -> size t (* halfed width - see above *)
  val bool : 'a t (* bool *)
  val bits : 'a -> 'a t
  val proc : size t list -> size t -> tyscheme (* build proc type *)
  (* keys used in sizes must be dense, or an unchecked RTE [SHOULD BE CHECKED] *)
  val largest_key : tyscheme -> key
  (* return the largest key in the type scheme, or 0 if no keys *)

⟨types.mli content 312d⟩≡ (311c)
  ⟨exported type definitions(types.nw) 311d⟩
  ⟨appl 311f⟩
  ⟨abbrevs 312c⟩
  val to_string : ty -> string
  val scheme_string : tyscheme -> string

```

K.18.4 Implementation

```

⟨definition of type size 312e⟩≡ (311d)
  type key = int
  type size = Const of int
               | Var of key
               | Double of key
               | Half of key

⟨types.ml content 312f⟩≡ (311b) 312g>
  ⟨exported type definitions(types.nw) 311d⟩
  (* types from the interface definition *)

⟨types.ml content 312g⟩+≡ (311b) <312f 313a>
  module E = Error
  module S = Map.Make(struct type t=key let compare=compare end)

  let impossf fmt = Printf.kprintf Impossible.impossible fmt

  let to_string = function
    | Bool -> "bool"
    | Bits n -> "bits" ^ string_of_int n

  let lookup key env = S.find key env
  let dump env = let f key data res = (key,data)::res in S.fold f env []

```

```

(types.ml content 313a)+≡ (311b) <312g 313b>
let match' opname sigma expected actual =
  let to_string' = function
    | Bool -> to_string Bool
    | Bits (Const x) -> to_string (Bits x)
    | Bits (Var x) when S.mem x sigma -> to_string (Bits (lookup x sigma))
    | Bits _ -> "bitsxxx (not easily identified)" in
  let badarg () =
    E.errorf
      "operator %%%s expected argument of type %s, got %s (in unspecified position)"
      opname (to_string' expected) (to_string' actual) in
  match expected, actual with
  | Bool , Bool -> sigma
  | Bits(Const x) , Bits y when x = y -> sigma

  | Bits(Var x) , Bits y when S.mem x sigma -> if lookup x sigma = y then sigma
                                              else badarg()
  | Bits(Var x) , Bits y -> S.add x y sigma

  | Bits(Double x), Bits y when S.mem x sigma ->
    if (lookup x sigma) * 2 = y then sigma else badarg()
  | Bits(Double x), Bits y -> S.add x (y/2) sigma

  | Bits(Half x) , Bits y when S.mem x sigma ->
    if (lookup x sigma) / 2 = y then sigma else badarg()
  | Bits(Half x) , Bits y -> S.add x (y*2) sigma

  | _ , _ -> badarg()

```

```

(types.ml content 313b)+≡ (311b) <313a 313c>
let subst opname sigma = function
  | Bool -> Bool
  | Bits(Const x) -> Bits x
  | Bits(Var x) -> (try Bits(lookup x sigma) with Not_found ->
    E.errorf "operator %%%s is polymorphic;\n    to indicate the size, \
    use a suffix such as %%%s32" opname opname)
  | Bits(Double x) -> (try Bits(2*(lookup x sigma)) with Not_found ->
    impossf "internal error (2) in application")
  | Bits(Half x) -> (try Bits((lookup x sigma)/2) with Not_found ->
    impossf "internal error (3) in application")

```

```

(types.ml content 313c)+≡ (311b) <313b 313d>
let wrongargs opname args' args =
  E.errorf "operator %s expected %d arguments, got %d"
    opname (List.length args') (List.length args)

let appl opname (args',r) args =
  let sigma = try List.fold_left2 (match' opname) S.empty args' args with
    | Invalid_argument _ -> wrongargs opname args' args in
  subst opname sigma r

```

```

(types.ml content 313d)+≡ (311b) <313c 314a>
let widthlist opname (args',r) args =
  let sigma = try List.fold_left2 (match' opname) S.empty args' args with
    | Invalid_argument _ -> wrongargs opname args' args in
  let sorted = List.sort (fun (key1,_) (key2,_) -> compare key1 key2) (dump sigma) in
  List.map snd sorted

```

<types.ml content 314a>+≡ (311b) <313d 314b>

```
let rtlop = Str.regexp "^\\([A-Za-z0-9_]*[A-Za-z_]\\)\\([0-9]+\\)?\$"
```

```
let split op =
  let matched n l = Str.matched_group n l in
  if Str.string_match rtlop op 0 then
    let basename = matched 1 op in
    let size      = try Some (int_of_string (matched 2 op))
                     with Not_found -> None in
    (basename, size)
  else
    impossf "illegal operator %s?" op
```

<types.ml content 314b>+≡ (311b) <314a 314c>

<key size 314d>

```
let instantiate ((args,ret):tyscheme) ~widths =
  if List.length widths <> largest_key (args, ret) then
    impossf "instantiated %d-key type scheme with %d widths"
      (largest_key (args, ret)) (List.length widths);
  let inst = function
    | Bits (Var i)   -> ( try Bits (List.nth widths (i-1)) with
                          | Failure _ -> assert false
                        )
    | Bits (Double i)-> ( try Bits (2 * (List.nth widths (i-1))) with
                          | Failure _ -> assert false
                        )
    | Bits (Half i)  -> ( try Bits ((List.nth widths (i-1))/2) with
                          | Failure _ -> assert false
                        )
    | Bits (Const k) -> Bits k
    | Bool           -> Bool in
  (List.map inst args, inst ret)
```

<types.ml content 314c>+≡ (311b) <314b 314e>

```
let fixbits x      = assert (x > 0); Bits(Const x)
let var    x      = assert (x > 0); Bits(Var x)
let double x      = assert (x > 0); Bits(Double x)
let half   x      = assert (x > 0); Bits(Half x)
let bool    = Bool
let bits x    = Bits x
let proc args res = (args,res)
```

<key size 314d>≡ (314b)

```
let largest_key (args, res) =
  let rec count k = function
    | [] -> k
    | (Bool | Bits (Const _)) :: ws -> count k ws
    | (Bits (Var n) | Bits (Double n) | Bits (Half n)) :: ws -> count (max k n) ws in
  count 0 (res :: args)
```

<types.ml content 314e>+≡ (311b) <314c>

```
let keyname = function
  | 1 -> "n"
  | 2 -> "m"
  | n -> Printf.sprintf "t%d" (n-2)

let scheme_string (args, result) =
  let spr = Printf.sprintf in
  let ty = function
    | Bits (Var i)   -> spr "%s bits" (keyname i)
```

```

    | Bits (Double i) -> spr "2*#%s bits" (keyname i)
    | Bits (Half i)   -> spr "%s/2 bits" (keyname i)
    | Bits (Const k)  -> spr "%d bits"    k
    | Bool            -> "bool" in
let scheme = match args with
| _ :: _ -> String.concat " * " (List.map ty args) ^ " -> " ^ ty result
| [] -> ty result in
match largest_key (args, result) with
| 0 -> scheme
| n -> spr "\\ / %s . %s" (String.concat ", " (List.map keyname (Auxfuns.from 1 ~upto:n)))
                        scheme

```

Appendix L

ir

L.1 ir/asm.nw

`<front_asm/asm.ml 316a>≡`
`<asm.ml content 317>`

`<front_asm/asm.mli 316b>≡`
`<asm.mli content 316c>`

L.2 Abstract Assembler Interface

`<asm.mli content 316c>≡` (316b)
`<exported type definitions(asm.nw) 316d>`

```
val pp_assembler :  
  (Format.formatter -> 'proc -> unit) ->  
  Format.formatter ->  
  'proc assembler ->  
  unit
```

```
val map : ('a -> 'b) -> 'b assembler -> 'a assembler  
val reloc_string : (Bits.bits -> string) -> Reloc.t -> string  
  (* make string form of relocatable address using mangled text of symbols *)
```

`<exported type definitions(asm.nw) 316d>≡` (317 316c)
`class type ['proc] assembler = object`
`<assembler methods 316e>`
`end`

L.2.1 Assembler Methods

`<assembler methods 316e>≡` (316d)
`(* declarations *)`
`method import : string -> Symbol.t           (* any name is legal *)`
`method export : string -> Symbol.t          (* any name is legal *)`
`method local : string -> Symbol.t          (* any name is legal *)`

`(* definitions *)`
`method label : Symbol.t -> unit   (*bind symbol to current location counter*)`
`method const : Symbol.t -> Bits.bits -> unit (*bind symbol to constant*)`

`(* section, location counter *)`

```

method section : string -> unit
method current : string          (* req: section called *)

method org      : int -> unit      (* set location counter      *)
method align    : int -> unit      (* align location counter    *)
method addloc   : int -> unit      (* increment location counter *)

(* size (bytes) of long jump instruction *)
method longjmp_size : unit -> int

(* emit instructions - add more methods for different instr types *)
method cfg_instr   : 'proc -> unit

(* emit data *)
method value       : Bits.bits -> unit
method addr        : Reloc.t -> unit
method zeroes      : int -> unit      (* n bytes of zeroes *)

(* announce number of global variables -- used only in interpreter *)
method globals     : int -> unit      (* allocate space for n globals (not bytes) *)

(* comment *)
method comment:    : string -> unit

(* emit *)
method emit:        : unit              (* finalize *)
(* should probably be called progend *)

⟨asm.ml content 317⟩≡ (316a) 318▷
⟨exported type definitions(asm.nw) 316d⟩

let pp_assembler pp_proc fmt (a : 'proc assembler) =
  Format.fprintf fmt "<assembler>"

class ['proc] mapped_asm f (asm : 'a assembler) : ['proc] assembler =
object
  (* declarations *)
  method import s = asm#import s
  method export s = asm#export s
  method local  s = asm#local s

  (* sections *)
  method section s = asm#section s
  method current = asm#current

  (* definitions *)
  method label s = asm#label s
  method const s b = asm#const s b

  (* locations *)

  method org n = asm#org n
  method align n = asm#align n
  method addloc n = asm#addloc n

  (* instructions *)
  method longjmp_size () = asm#longjmp_size ()
  method cfg_instr proc = asm#cfg_instr (f proc)

```

```

method globals n = asm#globals n
method zeroes n = asm#zeroes n
method value v = asm#value v
method addr a = asm#addr a
method comment s = asm#comment s
method emit = asm#emit
end
let map f asm = new mapped_asm f asm

⟨asm.ml content 318⟩+≡ (316a) ◁317
let reloc_string const =
  let sym (s, _) = s#mangled_text in
  let infix op a b = String.concat "" [a; " "; op; " "; b] in
  Reloc.fold ~const ~sym ~add:(infix "+") ~sub:(infix "-")

```

Appendix M

ir

M.1 ir/block.nw

$\langle \text{front_fenv/block.ml } 319a \rangle \equiv$
 $\langle \text{block.ml content } 320b \rangle$

$\langle \text{front_fenv/block.mli } 319b \rangle \equiv$
 $\langle \text{block.mli content } 319c \rangle$

M.2 Memory Blocks

$\langle \text{block.mli content } 319c \rangle \equiv$ (319b) 319d▷
type t
[*@@deriving show*]

```
val base:      t -> Rtl.exp
val size:      t -> int
val alignment: t -> int
val constraints: t -> Rtleqn.t list
```

$\langle \text{block.mli content } 319d \rangle + \equiv$ (319b) <319c 319e▷
val at: base:Rtl.exp -> size:int -> alignment:int -> t
val with_constraint: t -> Rtleqn.t -> t

$\langle \text{block.mli content } 319e \rangle + \equiv$ (319b) <319d 319f▷
val relative : Rtl.exp -> string -> (base:Rtl.exp -> 'a) -> 'a
(* construct address at unknown offset and call function *)
val srelative : Rtl.exp -> string -> (start:Rtl.exp -> 'a) -> 'a
(* construct address at unknown offset and call function *)

$\langle \text{block.mli content } 319f \rangle + \equiv$ (319b) <319e 319g▷
exception OverlapHigh
type placement = High | Low
val cath1: t -> t -> t
val overlap: placement -> t -> t -> t (* OverlapHigh *)

$\langle \text{block.mli content } 319g \rangle + \equiv$ (319b) <319f 320a▷
val adjust: t -> t (* size is multiple of alignment *)
val cath1_list: Rtl.width -> t list -> t
val overlap_list: Rtl.width -> placement -> t list -> t

M.2.1 Lua Interface

```
⟨block.mli content 320a⟩+≡ (319b) <319g
module Lua: sig
  val relative: t -> string -> int (*size*) -> int (* align *) -> t
  (* observer *)
  val base: t -> string (* show base address *)
  val constraints: t -> string list (* for debugging *)
  val size: t -> int
  val alignment: t -> int
  (* operations *)
  val adjust: t -> t
  val cat: Rtl.width -> t list -> t
  val overlap: Rtl.width -> placement (* "high"|"low"*) -> t list -> t

  val eq : t -> t -> bool
end
```

M.2.2 Implementation

```
⟨block.ml content 320b⟩≡ (319a) 320c>
module C = Rtleqn
module RU = Rtlutil

type t =
  { base: Rtl.exp
  ; size: int
  ; alignment: int
  ; constraints: C.t list
  }

[@@deriving show]

let base t = t.base
let size t = t.size
let alignment t = t.alignment
let constraints t = t.constraints

let at ~base ~size ~alignment =
  { base = base
  ; size = size
  ; alignment = alignment
  ; constraints = []
  }

let with_constraint t c = {t with constraints = c :: t.constraints}

let relative anchor dbg f =
  let w = RU.Width.exp anchor in
  f ~base:(RU.add w anchor (Rtl.late (Idgen.offset dbg) w))
let srelative anchor dbg f = relative anchor dbg (fun ~base -> f ~start:base)

⟨block.ml content 320c⟩+≡ (319a) <320b 320d>
let align x n = Auxfuns.round_up_to ~multiple_of:n x
let add_exp i = RU.addk (RU.Width.exp exp) exp i

⟨block.ml content 320d⟩+≡ (319a) <320c 321a>
let offset base name ptrwidth =
  RU.add ptrwidth base (Rtl.late (Idgen.offset name) ptrwidth)

(* claude: this was a _empty_vfp_hook ref, filled in by the caller, because
```

```

* Vfp then lived in layout/ and could not be reached from here. Its core is
* in ir/vfp.ml now, so the original one-liner works again.
*)
let empty ptrwidth =
  relative (Vfp.mk ptrwidth) "empty block" at ~size:0 ~alignment:1

⟨block.ml content 321a⟩+≡ (319a) <320d 321b>
let cathl hi lo =
  let size' = align (size lo) (alignment hi) in
  { base      = base lo
  ; size      = size' + size hi
  ; alignment  = max (alignment lo) (alignment hi)
  ; constraints = C.equate (add (base lo) size') (base hi)
  :: (constraints lo) @ (constraints hi)
  }

⟨block.ml content 321b⟩+≡ (319a) <321a 321c>
type placement = High | Low
exception OverlapHigh

let overlap place x y = match place with
| Low ->
  { base      = base x
  ; size      = max (size x) (size y)
  ; alignment  = max (alignment x) (alignment y)
  ; constraints = C.equate (base x) (base y)
  :: (constraints x) @ (constraints y)
  }
| High -> let x,y = if size x < size y then x,y else y,x in
  if (size y - size x) mod (alignment x) <> 0 then raise OverlapHigh else
  { base      = base y      (* y is the larger block *)
  ; size      = size y
  ; alignment  = max (alignment x) (alignment y)
  ; constraints = C.equate (add (base x) (size x))
  (add (base y) (size y))
  :: (constraints x) @ (constraints y)
  }

⟨block.ml content 321c⟩+≡ (319a) <321b 321d>
let adjust t = { t with size = align (size t) (alignment t) }
let cathl_list w = function
| [] -> empty w
| b :: bs -> List.fold_left cathl b bs
let overlap_list w p = function
| [] -> empty w
| b :: bs -> List.fold_left (overlap p) b bs

⟨block.ml content 321d⟩+≡ (319a) <321c>
module Lua = struct
  let size      = size
  let alignment = alignment
  let adjust    = adjust
  let cat       = cathl_list
  let overlap   = overlap_list

  let constraints block = List.map Rtleqn.to_string (constraints block)

  let relative block name size alignment =
    relative (base block) name at ~size ~alignment
  let base block = Rtlutil.ToString.exp (base block)

```

```

let eq b1 b2 =
  b1.size = b2.size && b1.alignment = b1.alignment &&
  Rtlutil.Eq.exp (Rtl.Dn.exp b1.base) (Rtl.Dn.exp b2.base) (* ignore constraints! *)

end

```

M.3 front_fenv/eqn.nw

$\langle \text{front_fenv/eqn.ml } 322a \rangle \equiv$
 $\langle \text{eqn.ml content } 323a \rangle$

$\langle \text{front_fenv/eqn.mli } 322b \rangle \equiv$
 $\langle \text{eqn.mli content } 322e \rangle$

M.4 Linear Equations over Integers

M.4.1 Interface

$\langle \text{EXP } 322c \rangle \equiv$ (323a 322e)

```

module type EXP = sig
  type t                                (* a term *)
  val variable: t -> string option
  val compare: t -> t -> int            (* -1/0/1 *)
  val print: t -> string                (* for debugging *)
end

```

$\langle S(\text{eqn.nw}) 322d \rangle \equiv$ (323a 322e)

```

module type S = sig
  type t                                (* set of equations *)
  type term
  type sum      = (int * term) list

  exception Can'tSolve of t

  type solution =
    { known:      (string * sum) list
    ; dependent: (string * sum) list
    }

  val empty:      t                                (* empty set of equations *)
  val make_zero:  sum -> t -> t                    (* add equation *)
  val solve:      t -> solution                    (* Can'tSolve *)
end

```

$\langle \text{eqn.mli content } 322e \rangle \equiv$ (322b)

$\langle \text{EXP } 322c \rangle$
 $\langle S(\text{eqn.nw}) 322d \rangle$

```

module Make (E: EXP): S with type term = E.t

```

M.4.2 Implementation

```

⟨eqn.ml content 323a⟩≡ (322a)
  open Nopoly

  ⟨EXP 322c⟩
  ⟨S(eqn.nw) 322d⟩
  module Make (E: EXP) = struct
    type term      = E.t

    ⟨Make(eqn.nw) 323b⟩
  end

  module Test = struct
    ⟨Test 326d⟩
  end

  ⟨Make(eqn.nw) 323b⟩≡ (323a) 323c▷
  type sum      = (int * E.t) list (* invariant: ordered *)
  type assoc    = string * sum

  type solution =
    { known      : (string * sum) list
    ; dependent  : (string * sum) list
    }

  module SM = Map.Make (struct type t = string let compare = compares end)
  type t    =
    { set      : sum list (* invariant: normalized *)
    ; env      : sum SM.t
    ; deps     : string list SM.t
    }

  let empty =
    { set = []
    ; env = SM.empty
    ; deps = SM.empty
    }

  exception Can'tSolve of t
  let error msg = raise (Can'tSolve msg)

  ⟨Make(eqn.nw) 323c⟩+≡ (323a) <323b 324a▷
  let dump t =
    let spr = Printf.sprintf in
    let product (i,term) =
      if i = 1 then
        E.print term
      else
        spr "%d * %s" i (E.print term) in
    let sum = function
      | [] -> "0"
      | s -> String.concat " + " (List.map product s) in
    let eqn s =
      let pos = List.filter (fun (i, t) -> i >= 0) s in
      let neg = List.filter (fun (i, t) -> i < 0) s in
      let neg = List.map (fun (i, t) -> (-i, t)) neg in
      spr " %s = %s" (sum pos) (sum neg) in
    let assoc v s rst = spr " %s :-> %s" v (sum s) :: rst in
    let multiple f s = String.concat "\n" (List.map f s) in

```

```

let set f s = String.concat "\n" (SM.fold assoc s []) in
Printf.printf
  "Eqn.t: unsolved equations\n%s\n----- solved variables:\n%s\n----- Eqn.t ends\n"
  (multiple eqn t.set) (set assoc t.env);
flush stdout

⟨Make(eqn.nw) 324a⟩+≡ (323a) <323c 324b>
let rec gcd (m:int) (n:int) =
  let rec g m n = if n = 0 then m else g n (m mod n) in
  if n < 0 then
    gcd m (- n)
  else if m < n then gcd n m
  else g m n

let normalize = function
| [] -> []
| ((c,_)::rest) as sum ->
  let g = List.fold_left (fun k (c,_) -> gcd k c) c rest in
  if g > 1 then
    List.map (fun (c,t) -> (c / g, t)) sum
  else
    sum

⟨Make(eqn.nw) 324b⟩+≡ (323a) <324a 324c>
let combine (k1:int) (sum1:sum) (sum2:sum) =
  let rec loop = function
  | [] , [] -> []
  | ((c1,x1)::s1 as _x) , ((c2,x2)::s2 as y) when E.compare x1 x2 < 0 ->
    let k = k1 * c1 in
    if k = 0 then loop (s1, y) else
      (k, x1) :: loop (s1, y)
  | ((c1,x1)::s1) , ((c2,x2)::s2) when E.compare x1 x2 = 0 ->
    let k = k1 * c1 + c2 in
    if k = 0 then loop (s1, s2) else
      (k, x1) :: loop (s1, s2)
  | ((c1,x1)::s1 as x) , ((c2,x2)::s2 as _y) (* x1 > x2 *) ->
    let k = c2 in
    if k = 0 then loop (x, s2) else
      (k, x2) :: loop (x, s2)
  | ((c1,x1)::s1) , [] ->
    let k = k1 * c1 in
    if k = 0 then loop (s1, []) else
      (k, x1) :: loop (s1, [])
  | [] , ((c2,x2)::s2) ->
    let k = c2 in
    if k = 0 then loop ([], s2) else
      (k, x2) :: loop ([], s2)
  in
  loop (sum1,sum2)

⟨Make(eqn.nw) 324c⟩+≡ (323a) <324b 325a>
let split (v:string) (sum:sum) =
  let rec loop a = function
  | [] -> (0, sum)
  | (c,t as ct)::s ->
    ( match E.variable t with
    | Some v' when v == v' -> (c, List.rev a @ s)
    | _ -> loop (ct::a) s
    )
  in
  loop [] sum

```

```

⟨Make(eqn.nw) 325a⟩+≡ (323a) <324c 325b>
  let elim (v:string) (vsum:sum) (sum:sum) =
    match split v sum with
    | (0,_)    -> sum
    | (c,sum') -> combine c vsum sum'

⟨Make(eqn.nw) 325b⟩+≡ (323a) <325a 325c>
  let vars (sum:sum) =
    let rec loop vs = function
      | (_, t)::s ->
        ( match E.variable t with
          | Some v -> loop (v::vs) s
          | _      -> loop vs s
        )
      | [] -> vs in
    loop [] sum

⟨Make(eqn.nw) 325c⟩+≡ (323a) <325b 325d>
  let elim_all env (sum:sum) =
    let vs = vars sum in
    List.fold_left (fun sum v -> try elim v (SM.find v env) sum
                                with Not_found -> sum) sum vs

⟨Make(eqn.nw) 325d⟩+≡ (323a) <325c 325e>
  let rec zero = function
    | []                -> true
    | (0,_)::rest      -> zero rest
    | _                -> false

⟨Make(eqn.nw) 325e⟩+≡ (323a) <325d 325f>
  let candidate t found finished =
    let negate      = List.map (fun (c,trm) -> (-c,trm))
    let unitvar (c,trm) = Auxfuns.Option.is_some (E.variable trm) && (c = 1 || c = -1) in
    let rec loop accum_set = function
      | [] -> (* all eqns scanned *)
        (match accum_set with
         | [] -> finished { t with set = accum_set }
         | _ -> error    { t with set = accum_set }
        )
      | sum::sums ->
        let sum = normalize (elim_all t.env sum) in
        if zero sum then
          loop accum_set sums
        else
          try let v      = match E.variable (snd (List.find unitvar sum)) with
            | Some v -> v
            | None   -> assert false in
            let c,vsum = split v sum in
            match c with
            | -1 -> found (v, vsum, { t with set = accum_set @ sums })
            | 1  -> found (v, negate vsum, { t with set = accum_set @ sums })
            | _  -> assert false (* c is a unit, i.e. +/-1 *)
          with Not_found -> loop (sum::accum_set) sums (* check next equation *)
    in
    loop [] t.set

⟨Make(eqn.nw) 325f⟩+≡ (323a) <325e 326a>
  let update (v:string) (vsum:sum) t =
    let find x map = try SM.find x map with Not_found -> [] in
    let depends_on_v = find v t.deps in

```

```

let env = List.fold_left (fun env v' -> SM.add v' (elim v vsum (SM.find v' env)) env)
                      t.env depends_on_v in

let vs = vars vsum in
let new_deps = v :: depends_on_v in
let add_deps v' = SM.add v' (new_deps @ find v' deps) deps in
{ set = t.set
; env = SM.add v vsum env
; deps = List.fold_left add t.deps vs
}

⟨Make(eqn.nw) 326a⟩+≡ (323a) <325f 326b>
let solver t =
  let rec loop t =
    candidate t (fun (v, vsum, t') -> loop (update v vsum t')) (fun t' -> t') in
  loop t

⟨Make(eqn.nw) 326b⟩+≡ (323a) <326a 326c>
let solve t =
  (* let () = dump t in *)
  let t = try solver t with Can'tSolve x -> (dump x; error x) in
  (* let () = dump t in *)
  let known sum = List.for_all (fun (_,e) -> Auxfun.Option.is_none (E.variable e)) sum in
  let k,d =
    SM.fold (fun s sum (k,d) -> if known sum then ((s,sum)::k,d) else (k, (s,sum)::d))
          t.env ([], []) in
  { known = k
  ; dependent = d
  }

⟨Make(eqn.nw) 326c⟩+≡ (323a) <326b>
let rec merge sum = match sum with
| [] -> []
| [_] as x -> x
| (c1,t1 as ct1)::((c2,t2)::rest1 as rest2) ->
  if E.compare t1 t2 = 0 then
    if c1+c2 <> 0 then
      merge ((c1+c2,t1)::rest1)
    else
      merge rest1 (* drop zero coefficient *)
  else
    ct1 :: merge rest2

let make_zero (sum:sum) (t:t) =
  let cmp (_,tx) (_,ty) = E.compare tx ty in
  let eqn = normalize (merge (List.sort cmp sum)) in
  let r = { t with set = eqn::t.set } in
  (* dump t; *)
  match eqn with [] -> t | _ :: _ -> r

```

M.4.3 Test

```

⟨Test 326d⟩≡ (323a)
module E = struct
  type t = Var of string
        | Unit
        | Const of string

```

```

let variable = function
  | Var s   -> Some s
  | _      -> None

let compare : t -> t -> int = Pervasives.compare

let print = function
  | Var(s) -> s
  | Unit   -> "1"
  | Const(s) -> s
end

module S = Make(E)

let test eqns =
  let t = List.fold_left (fun t eqn -> S.make_zero eqn t) S.empty eqns
  in
    S.solve t

let eqns1 =
  [ [3,E.Var "x";5   ,E.Var "y"]
  ; [2,E.Var "y";3   ,E.Var "z"]
  ; [3,E.Var "x";(-3),E.Var "x"]
  ; [2,E.Var "x";(-5),E.Var "z"]
  ; [3,E.Unit  ;1   ,E.Var "y"]
  ]

let eqns2 =
  [ [-3,E.Unit;1, E.Var "x"; 1, E.Var "y"; 1, E.Var "z"]
  ; [-1,E.Unit;1, E.Var "x";-1, E.Var "y";-1, E.Var "z"]
  ; [4, E.Unit;1, E.Var "z"]
  ]

let eqns3 =
  [ [3,E.Var "x";5   ,E.Var "y"; 3,E.Const "k1"]
  ; [2,E.Var "y";3   ,E.Var "z"]
  ; [3,E.Var "x";(-3),E.Var "x"]
  ; [2,E.Var "x";(-5),E.Var "z"; 2,E.Const "k1"]
  ; [3,E.Unit  ;1   ,E.Var "y"; 3,E.Const "k3"]
  ]

(*
let _ = assert
  (test eqns1 =
    { S.known = ["x", [5, E.Unit]; "z", [2, E.Unit]; "y", [-3, E.Unit]]
    ; S.dependent = []
    })

let _ = assert
  (test eqns2 =
    { S.known = ["y", [5, E.Unit]; "x", [2, E.Unit]; "z", [-4, E.Unit]]
    ; S.dependent = []
    })
*)

```


M.5 front_fenv/fenv.nw

```
<front_fenv/fenv.ml 328a>≡  
  <fenv.ml content 330i>
```

```
<front_fenv/fenv.mli 328b>≡  
  <fenv.mli content 328d>
```

M.6 The Fat Environment

M.6.1 Interface: Clean vs. Dirty

```
<exported signature Env 328c>≡ (331b 328d)  
  module type Env = sig  
    type 'a info  
    [@@deriving show]  
    type 'a partial  
    val bad: unit -> 'a info  
  
    <bindings appearing in signature Env 328e>  
  end  
  
<fenv.mli content 328d>≡ (328b) 330b▷  
  <exposed types shared by clean and dirty environments 329c>  
  (* pad: now put also those private types in mli *)  
  <private types shared by clean and dirty environments 332b>  
  <exported signature Env 328c>  
  
  module Dirty : Env  
    with type 'a info = 'a Error.error  
    with type 'a partial = 'a option  
  
  module Clean : Env  
    with type 'a info = 'a  
    with type 'a partial = 'a  
  
  val clean : 'proc Dirty.env' -> 'proc Clean.env'
```

M.6.2 Getting Started: creating an Environment

```
<bindings appearing in signature Env 328e>≡ (328c) 329a▷  
  (* pad: was previously an abstract type *)  
  type 'proc env' = { scopes      : scope list (* top = hd scopes *)  
                    ; srcmap      : Srcmap.map  
                    (* pad: the assembler is embeded in the fat env !! *)  
                    ; asm         : 'proc Asm.assembler  
  
                    ; error       : bool  
                    ; metrics     : Metrics.t  
                    ; extern      : extern  
                    ; globals     : string list (* global registers *)  
                    ; stackdata   : stackdata  
                    }  
  
  (* type env = Proc.t env' *)  
  and scope      = { mutable venv: ventry Strutil.Map.t  
                    ; tenv:   tentry Strutil.Map.t  
                    ; rindex: int   (* getIndex, nextIndex *)
```

```
}

```

<types exposed in signature Env 330a>

```
val map : ('b -> 'a) -> 'a env' -> 'b env'

val empty : Srcmap.map -> Metrics.t -> 'proc Asm.assembler -> 'proc env'
(* empty scope stack *)
val srcmap : 'proc env' -> Srcmap.map
(* pad: allow to access the assembler embeded in the fat env *)
val asm : 'proc env' -> 'proc Asm.assembler
val metrics : 'proc env' -> Metrics.t

```

M.6.3 Scopes and the Fat Environment

<bindings appearing in signature Env 329a>+≡ (328c) <328e 329b>

```
val emptyscope: scope
val top: 'p env' -> scope (* top empty = assert false *)
val pop: 'p env' -> 'p env' (* pop empty = assert false *)
val push: 'p env' -> scope -> 'p env'

```

<bindings appearing in signature Env 329b>+≡ (328c) <329a 330d>

```
val foldv: (string -> ventry -> 'a -> 'a) -> scope -> 'a -> 'a

```

M.6.4 Bindings for Values and Types

<exposed types shared by clean and dirty environments 329c>≡ (331b 328d) 329d>

```
type regkind = RReg of string (* hardware reg *)
              | RKind of string (* calling convention kind *)
              | RNone (* none of above *)

[@@deriving show]

```

<exposed types shared by clean and dirty environments 329d>+≡ (331b 328d) <329c 329e>

```
type variable = { index: int
                  ; rkind: regkind
                  ; loc: Rtl.loc
                  ; variance: Ast.variance
                  }

[@@deriving show]

```

<exposed types shared by clean and dirty environments 329e>+≡ (331b 328d) <329d

```
type symclass = Proc of Symbol.t
              | Code of Symbol.t
              | Data of Symbol.t
              | Stack of Rtl.exp (* address of slot *)

[@@deriving show]

type denotation = Constant of Bits.bits
                | Label of symclass
                | Import of string * Symbol.t
                  (* external name, assembly symbol *)
                | Variable of variable
                | Continuation of continuation

and continuation = { base : Block.t; (* always empty; used only for address *)
                    convention : string;
                    formals : (string * variable * aligned) list;
                      (* kinded, aligned formals *)

```

```

        mutable escapes      : bool;  (* used as rvalue *)
        mutable cut_to       : bool;  (* mentioned in annotation *)
        mutable unwound_to   : bool;  (* mentioned in annotation *)
        mutable returned_to  : convention list;
            (* list every convention cc such that there exists
               a call site with convention cc and that call site
               'also returns to' the continuation *)
    } (* might need a convention here *)

and convention = string
and aligned    = int
[@@deriving show]

<types exposed in signature Env 330a>≡ (328e) 330c>
and ventry     = Srcmap.rgn * (denotation * Types.ty) info

<fenv.mli content 330b>+≡ (328b) <328d
    val denotation's_category : denotation -> string

<types exposed in signature Env 330c>+≡ (328e) <330a
and tentry     = Srcmap.rgn * Types.ty info
[@@deriving show]

```

M.6.5 Binding and Finding Names and Values

```

<bindings appearing in signature Env 330d>+≡ (328c) <329b 330e>
val bindv      : string -> ventry -> 'p env' -> 'p env'
val rebindv    : string -> ventry -> 'p env' -> 'p env'
val rebindv'   : string -> ventry -> 'p env' -> unit (* mutates *)
val bindt      : string -> tentry -> 'p env' -> 'p env'
val findv      : string -> 'p env' -> ventry (* Error.ErrorExn *)
val findt      : string -> 'p env' -> tentry (* Error.ErrorExn *)

<bindings appearing in signature Env 330e>+≡ (328c) <330d 330f>
val is_localv  : string -> 'p env' -> bool (* *)

```

M.6.6 Error Flag

```

<bindings appearing in signature Env 330f>+≡ (328c) <330e 330g>
val flagError  : 'p env' -> 'p env'
val errorFlag  : 'p env' -> bool

```

M.6.7 Imports, Exports, and Assembler Symbols

```

<bindings appearing in signature Env 330g>+≡ (328c) <330f 330h>
val import: Srcmap.rgn -> string -> string -> 'p env' -> 'p env' (* import g as f *)
val export: Srcmap.rgn -> string -> string -> 'p env' -> 'p env' (* export f as g *)

<bindings appearing in signature Env 330h>+≡ (328c) <330g
val symbol: 'p env' -> string -> Symbol.t

```

M.6.8 The Details of C-- names and Assembler Symbols

M.6.9 Implementation: Large Scale Structure

```

<fenv.ml content 330i>≡ (328a) 331a>
module E      = Error      (* handy abbreviations *)
module T      = Types
module Asm    = Asm

```

```

⟨fenv.ml content 331a⟩+≡ (328a) <330i 331b>
  module type Arg = sig
    type 'a partial
    type 'a info
    [@@deriving show]

    val good    : 'a    -> 'a info
    val asgood  : 'a info -> 'a option
    val bad     : unit -> 'a info
  end

⟨fenv.ml content 331b⟩+≡ (328a) <331a 331c>
  ⟨exposed types shared by clean and dirty environments 329c⟩
  ⟨private types shared by clean and dirty environments 332b⟩
  ⟨exported signature Env 328c⟩
  module Env (Arg: Arg) = struct
    ⟨body of functor Env 331e⟩
  end

⟨fenv.ml content 331c⟩+≡ (328a) <331b 331d>
  module Dirty = Env (struct
    type 'a partial = 'a option
    type 'a info    = 'a Error.error
    [@@deriving show]

    let good  x  = Error.Ok(x)
    let bad   x  = Error.Error
    let asgood = function
      | Error.Ok(x) -> Some x
      | Error.Error -> None
  end)

⟨fenv.ml content 331d⟩+≡ (328a) <331c 332f>
  module Clean = Env (struct
    type 'a partial      = 'a
    type 'a info         = 'a
    [@@deriving show]

    let good x          = x
    let asgood x        = Some x
    let bad  x          = assert false
  end)

```

M.6.10 The Heart of the Implementation: Env

```

⟨body of functor Env 331e⟩≡ (331b) 332a>
  type 'a partial      = 'a Arg.partial
  type 'a info         = 'a Arg.info
  [@@deriving show]
  let bad              = Arg.bad

  (* pad: *)
  type ventry          = Srcmap.rgn * (denotation * Types.ty) info
  [@@deriving show]
  type tentry          = Srcmap.rgn * Types.ty info
  [@@deriving show]

```

```

⟨body of functor Env 332a⟩+≡ (331b) <331e 332d>
type 'proc env' = { scopes      : scope list (* top = hd scopes *)
                  ; srcmap      : Srcmap.map
                  ; asm         : 'proc Asm.assembler
                  ; error       : bool
                  ; metrics     : Metrics.t
                  ; extern      : extern
                  ; globals     : string list (* global registers *)
                  ; stackdata   : stackdata
                  }

```

```

⟨private types shared by clean and dirty environments 332b⟩≡ (331b 328d) 332c>
type stackdata = { soffset      : int      (* current offset *)
                  ; smaxalign    : int      (* max stackdata align constr*)
                  ; sname       : string   (* label for offset *)
                  }

```

[@@deriving show]

```

⟨private types shared by clean and dirty environments 332c⟩+≡ (331b 328d) <332b
type extern      = { imported:   Strutil.Set.t
                  ; exported:   Strutil.Set.t
                  ; nam2sym:     Symbol.t Strutil.Map.t
                  }

```

[@@deriving show]

```

⟨body of functor Env 332d⟩+≡ (331b) <332a 332e>
and scope        = { mutable venv: ventry Strutil.Map.t
                  ; tenv:   tentry Strutil.Map.t
                  ; rindex: int   (* getIndex, nextIndex *)
                  }

```

[@@deriving show]

M.6.11 Auxiliaries

```

⟨body of functor Env 332e⟩+≡ (331b) <332d 332g>
let error r map msg = E.errorRegionPrt (map,r) msg

```

```

⟨fenv.ml content 332f⟩+≡ (328a) <331d 336d>
let denotation's_category = function
| Label (Proc _) -> "procedure"
| Label (Code _) -> "code label"
| Label (Data _) -> "data label"
| Label (Stack _) -> "stackdata label"
| Constant _      -> "constant"
| Continuation _  -> "continuation"
| Import (_, _)   -> "imported symbol"
| Variable _      -> "register variable"

```

M.6.12 Mapping

```

⟨body of functor Env 332g⟩+≡ (331b) <332e 333a>
let map f { scopes      = scopes
          ; srcmap      = srcmap
          ; asm         = asm
          ; error       = error
          ; metrics     = metrics
          ; extern      = extern
          ; globals     = globals

```

```

; stackdata = stackdata
} =
{ scopes    = scopes
; srcmap    = srcmap
; asm       = Asm.map f asm
; error     = error
; metrics   = metrics
; extern    = extern
; globals   = globals
; stackdata = stackdata
}

```

M.6.13 Creating env and scope values

⟨body of functor Env 333a⟩+≡ (331b) <332g 333b>

```

let empty map metrics asm =
  { scopes    = []
; srcmap     = map
; asm        = asm
; error      = false
; globals    = []
; stackdata  = { smaxalign = 1
                ; soffset  = 0
                ; sname    = "can't happen"
                }
; metrics    = metrics
; extern     = { imported  = Strutil.Set.empty
                ; exported  = Strutil.Set.empty
                ; nam2sym   = Strutil.Map.empty
                }
}

```

⟨body of functor Env 333b⟩+≡ (331b) <333a 333c>

```

let emptyscope = { venv = Strutil.Map.empty
                  ; tenv = Strutil.Map.empty
                  ; rindex = 0
                  }

```

```

let srcmap {srcmap=m} = m
let asm    {asm=a}    = a
let metrics {metrics=m} = m

```

```

let top env = match env.scopes with
| []    -> assert false
| s::ss -> s

```

```

let pop env = match env.scopes with
| []    -> assert false
| s::ss -> { env with scopes = ss }

```

```

let push env scope = { env with scopes = scope :: env.scopes }

```

⟨body of functor Env 333c⟩+≡ (331b) <333b 334a>

```

let foldv f {venv=v} z = Strutil.Map.fold f v z

```

M.6.14 Binding and Finding Names and Values

```

<body of functor Env 334a>+≡ (331b) <333c 334b>
  type typedefn = (Ast.ty * string list) * Ast.region
  let addTypedefn t env = assert false
  let typedefns env = assert false
  let setTypedefns ts env = assert false

<body of functor Env 334b>+≡ (331b) <334a 334c>
  type constdefn = (Ast.ty option * string * Ast.expr) * Ast.region
  let addConstdefn t env = assert false
  let constdefns env = assert false
  let setConstdefns ts env = assert false

<body of functor Env 334c>+≡ (331b) <334b 334d>
  let bindv name (rgn,x as ventry) env =
    let scope = ( match env.scopes with
      | []      -> assert false
      | s::ss   -> s
    ) in
  try let (rgn',x) = Strutil.Map.find name scope.venv in
    ( error rgn  env.srcmap ("re-declaration of value "^name)
    ; error rgn' env.srcmap ("previously declared here")
    ; let scope =
      { scope with venv = Strutil.Map.add name (rgn',Arg.bad()) scope.venv }
    in
      { env with
        scopes = scope :: List.tl env.scopes
        ; error = true
      }
    )
  with Not_found ->
    let scope = { scope with venv = Strutil.Map.add name ventry scope.venv } in
    let env = { env with scopes = scope :: List.tl env.scopes } in
      ( match Arg.asgood x with
      | Some(Variable _,_) when List.length env.scopes = 1 ->
        (* this is a global register declaration *)
        { env with globals = name :: env.globals }
      | _ -> env
      )

<body of functor Env 334d>+≡ (331b) <334c 335a>
  let bindt name (rgn,x as tentry) env =
    let scope = ( match env.scopes with
      | []      -> assert false
      | s::ss   -> s
    ) in
  try let (rgn',x) = Strutil.Map.find name scope.tenv in
    ( error rgn  env.srcmap ("re-declaration of type "^name)
    ; error rgn' env.srcmap ("previously declared here")
    ; let scope =
      { scope with tenv = Strutil.Map.add name (rgn',Arg.bad()) scope.tenv }
    in
      { env with
        scopes = scope :: List.tl env.scopes
        ; error = true
      }
    )
  with Not_found ->
    let scope = { scope with tenv = Strutil.Map.add name tentry scope.tenv } in
    let env = { env with scopes = scope :: List.tl env.scopes } in
      env

```

```

⟨body of functor Env 335a⟩+≡ (331b) <334d 335b>
  let findv name env =
    let rec loop = function
      | []    -> E.error ("unknown value: "^name)
      | s:ss -> (try Strutil.Map.find name s.venv with Not_found -> loop ss) in
    loop env.scopes

⟨body of functor Env 335b⟩+≡ (331b) <335a 335c>
  let findt name env =
    let rec loop = function
      | []    -> E.error ("unknown type: "^name)
      | s:ss -> ( try Strutil.Map.find name s.tenv with Not_found -> loop ss ) in
    loop env.scopes

⟨body of functor Env 335c⟩+≡ (331b) <335b 335d>
  let rebinding name x env =
    let rec loop = function
      | []    -> Impossible.impossible ("can't rebinding "^name)
      | s:ss -> if Strutil.Map.mem name s.venv
                  then { s with venv = Strutil.Map.add name x s.venv } :: ss
                  else s :: loop ss in
    { env with scopes = loop env.scopes }

  let rebinding' name x env =
    let rec loop = function
      | []    -> Impossible.impossible ("can't rebinding "^name)
      | s:ss -> if Strutil.Map.mem name s.venv
                  then s.venv <- Strutil.Map.add name x s.venv
                  else loop ss in
    loop env.scopes

⟨body of functor Env 335d⟩+≡ (331b) <335c 335e>
  let is_localv name env =
    ( match env.scopes with
    | []    -> assert false
    | [s]   -> false
    | s:ss -> Strutil.Map.mem name s.venv
    )

```

M.6.15 The Error Flag

```

⟨body of functor Env 335e⟩+≡ (331b) <335d 335f>
  let flagError env = if env.error then env else { env with error = true }
  let errorFlag env = env.error

```

M.6.16 Imports, Exports, Symbols

```

⟨body of functor Env 335f⟩+≡ (331b) <335e 336a>
  let import r sym name env =
    if Strutil.Set.mem sym env.extern.exported
    then
      ( error r env.srcmap ("import of an exported name: "^name)
      ; flagError env
      )
    else
      let sym'   = env.asm#import sym in
      let extern = { exported = env.extern.exported
                    ; imported = Strutil.Set.add sym  env.extern.imported

```



```

        ; nam2sym = Strutil.Map.add name sym' env.extern.nam2sym
      }
    in
      { env with extern = extern }

let export r name sym env =
  if Strutil.Set.mem sym env.extern.imported
  then
    ( error r env.srcmap ("export of an imported name: "^name)
    ; flagError env
    )
  else
    let sym' = env.asm#export sym in
    let extern = { imported = env.extern.imported
                  ; exported = Strutil.Set.add sym env.extern.exported
                  ; nam2sym = Strutil.Map.add name sym' env.extern.nam2sym
                  }
    in
      { env with extern = extern }

⟨body of functor Env 336a⟩+≡ (331b) ◁335f 336b▷
let symbol env n =
  try Strutil.Map.find n env.extern.nam2sym
  with Not_found ->
    let rec avoid_collision n =
      if Strutil.Set.mem n env.extern.exported ||
        Strutil.Set.mem n env.extern.imported
      then
        avoid_collision (n ^ "@")
      else
        n
    in
      env.asm#local (avoid_collision n)

```

M.6.17 Register Index

```

⟨body of functor Env 336b⟩+≡ (331b) ◁336a 336c▷
let nextIndex env = match env.scopes with
  | top::t -> { env with scopes =
                {top with rindex = top.rindex + 1}::t
              }
  | _ -> assert false (* no scope *)

let getIndex env = match env.scopes with
  | top::_ -> top.rindex
  | _ -> assert false (* no scope *)

⟨body of functor Env 336c⟩+≡ (331b) ◁336b
let globals env = List.rev env.globals

```

M.6.18 Cleaning a fat environment

```

⟨fenv.ml content 336d⟩+≡ (328a) ◁332f 337a▷
let clean_env map =
  let copy key data map = match data with
  | pos, E.Ok den -> Strutil.Map.add key (pos, den) map
  | _, E.Error -> assert false in
  Strutil.Map.fold copy map Strutil.Map.empty

```

```

⟨fenv.ml content 337a⟩+≡ (328a) <336d 337b>
  let clean_scope s =
    { Clean.tenv    = clean_env s.Dirty.tenv
    ; Clean.venv    = clean_env s.Dirty.venv
    ; Clean.rindex  = s.Dirty.rindex
    }

```

```

⟨fenv.ml content 337b⟩+≡ (328a) <337a
  let clean env =
    { Clean.scopes    = List.map clean_scope env.Dirty.scopes
    ; Clean.srcmap    = env.Dirty.srcmap
    ; Clean.asm       = env.Dirty.asm
    ; Clean.error     = env.Dirty.error
    ; Clean.metrics   = env.Dirty.metrics
    ; Clean.stackdata = env.Dirty.stackdata
    ; Clean.globals   = env.Dirty.globals
    ; Clean.extern    = env.Dirty.extern
    }

```

M.7 front_fenv/metrics.nw

```

⟨front_fenv/metrics.ml 337c⟩≡
  ⟨metrics.ml content 337g⟩

```

```

⟨front_fenv/metrics.mli 337d⟩≡
  ⟨metrics.mli content 337e⟩

```

M.8 Target Platform Metrics

```

⟨metrics.mli content 337e⟩≡ (337d)
  ⟨exposed types(metrics.nw) 337f⟩
  val default : t
  val of_ast  :
    swap:bool -> Srcmap.map -> (Ast.region * Ast.arch) list ->
    t Error.error

```

```

⟨exposed types(metrics.nw) 337f⟩≡ (337)
  type t = {
    byteorder   : Rtl.aggregation ; (* big/little endian, id *)
    wordsize    : int              ; (* bits *)
    pointersize : int              ; (* bits *)
    memsize     : int              ; (* smallest addressable unit, typically 8 *)
    float       : string           ; (* name of float representation (def "ieee754") *)
    charset     : string           ; (* "latin1" character encoding *)
  }
  [@@deriving show]

```

M.9 Implementation

```

⟨metrics.ml content 337g⟩≡ (337c) 339a>
  (*open Nopoly*)
  let (==) = (=)

  module A = Ast
  ⟨exposed types(metrics.nw) 337f⟩

```

```

let default = {
  byteorder  = Rtl.Identity;
  wordsize   = 32;
  pointersize = 32;
  memsize    = 8;
  float      = "ieee754";
  charset    = "latin1";
}

let of_ast ~swap map =
  let errorf rgn =
    Printf.kprintf (fun s -> (Error.errorRegionPrt (map, rgn) s; Error.Error)) in
  let def default = function
    | Error.Ok (Some (r, v)) -> v
    | Error.Ok None         -> default
    | Error.Error           -> assert false in
  let merge r prt what v = function (* used at int, string, and Rtl.aggregation *)
    | Error.Ok (Some (r', v')) as m when v == v' -> m
    | Error.Ok (Some (r', v')) ->
      errorf r "%s %s inconsistent with previous declaration of %s at %s"
        what (prt v) (prt v') (Srcmap.Str.region (map, r'))
    | Error.Ok None -> Error.Ok (Some (r, v))
    | Error.Error -> Error.Error in
  let mergei r what i v =
    if i > 0 then merge r string_of_int what i v
    else errorf r "%s must be a positive integer" what in
  let bos = function Rtl.LittleEndian -> "little" | _ -> "big" in
  let mergebo r v = merge r bos "byte order" v in
  let merges r = merge r (fun s -> s) in
  let is_ok = function Error.Ok _ -> true | Error.Error -> false in

  let rec metrics bo w p m f c aa =
    let big_endian    r aa = metrics (mergebo r Rtl.BigEndian    bo) w p m f c aa in
    let little_endian r aa = metrics (mergebo r Rtl.LittleEndian bo) w p m f c aa in
    match aa with
    | [] -> if is_ok bo && is_ok w && is_ok p && is_ok m && is_ok f && is_ok c then
      match bo with
      | Error.Ok (Some (_, bo)) ->
        Error.Ok { byteorder  = bo;
                    wordsize   = def default.wordsize    w;
                    pointersize = def default.pointersize p;
                    memsize    = def default.memsize      m;
                    float      = def default.float        f;
                    charset    = def default.charset      c;
                  }
      | Error.Ok None ->
        (Error.errorPrt "target byte order not specified"; Error.Error)
      | Error.Error ->
        assert false
    else
      Error.Error
  | (r,A.Memsize i)      :: aa -> metrics bo w p (mergei r "memsize" i m) f c aa
  | (r,A.ByteorderBig)  :: aa -> (if swap then little_endian else big_endian) r aa
  | (r,A.ByteorderLittle):: aa -> (if swap then big_endian else little_endian) r aa
  | (r,A.FloatRepr s)   :: aa -> metrics bo w p m (merges r "float" s f) c aa
  | (r,A.WordSize i)    :: aa -> metrics bo (mergei r "wordsize" i w) p m f c aa
  | (r,A.PointerSize i) :: aa -> metrics bo w (mergei r "pointersize" i p) m f c aa
  | (r,A.Charset s)     :: aa -> metrics bo w p m f (merges r "charset" s c) aa in
  let n = Error.Ok None in

```

```

    metrics n n n n n n
⟨metrics.ml content 339a⟩+≡ (337c) <337g

```

M.10 front_fenv/rtleqn.nw

```

⟨front_fenv/rtleqn.ml 339b⟩≡
  ⟨rtleqn.ml 339e⟩
⟨front_fenv/rtleqn.mli 339c⟩≡
  ⟨rtleqn.mli 339d⟩

```

M.11 Equations over RTL-Expressions

```

⟨rtleqn.mli 339d⟩≡ (339c)
  exception Can'tSolve

  type t (* equation *)
  [@@deriving show]

  type solution =
    { known:      (string * Rtl.exp) list (* fully solved *)
    ; dependent:  (string * Rtl.exp) list (* depend on other variables *)
    }

  val equate : Rtl.exp -> Rtl.exp -> t (* e1 == e2 *)

  val solve : width:int -> t list -> solution (* Can'tSolve *)
  val to_string : t -> string (* for debugging *)

```

M.12 Implementation of equations

```

⟨rtleqn.ml 339e⟩≡ (339b) 339f▷
  open Nopoly

  module Dn = Rtl.Dn
  module Up = Rtl.Up
  module RP = Rtl.Private

  exception Can'tSolve
  type solution =
    { known:      (string * Rtl.exp) list
    ; dependent:  (string * Rtl.exp) list
    }

⟨rtleqn.ml 339f⟩+≡ (339b) <339e 340a▷
  type term =
    | Const of Rtl.exp
    | Var   of string
    | Unit
  [@@deriving show]

  type sum = (int * term) list
  [@@deriving show]
  type t = sum (* equation, sum == 0 *)
  [@@deriving show]

```

<rtleqn.ml 340a>+≡ (339b) <339f 340b>

```

module ToString = struct
  let term = function
    | Const(exp) -> "(" ^ Rtlutil.ToString.exp exp ^ ")"
    | Var(s)      -> s
    | Unit        -> "1"
  let summand (i,t) = match i with
    | 1 -> term t
    | n ->
      match t with
      | Unit -> Printf.sprintf "%d" i
      | _    -> Printf.sprintf "%d * %s" i (term t)
  let t sum = (* no thought for efficiency *)
    let pos = List.filter (fun (i, n) -> i >= 0) sum in
    let neg = List.filter (fun (i, n) -> i < 0) sum in
    let neg = List.map (fun (i, n) -> (-i, n)) neg in
    let side = function
      | [] -> "0"
      | sum -> String.concat " + " (List.map summand sum) in
    side pos ^ " = " ^ side neg
end
let to_string = ToString.t

```

<rtleqn.ml 340b>+≡ (339b) <340a 340c>

```

let sym x = [(1, Var x)]
let int i = if i = 0 then [] else [(i, Unit)]
let const k = [(1, Const (Up.exp k))]
let add x y = x @ y
let sub x y = x @ List.map (fun (i,y) -> (-i,y)) y

```

<rtleqn.ml 340c>+≡ (339b) <340b 340d>

```

let exp e =
  let rec exp = function
    | RP.Const(RP.Bits(b))      -> int (Bits.S.to_int b)
    | RP.Const(RP.Late(x,_))    -> sym x
    | RP.Const(_) as x         -> const x
    | RP.App(("add",_),[e1;e2]) -> add (exp e1) (exp e2)
    | RP.App(("sub",_),[e1;e2]) -> sub (exp e1) (exp e2)
    | x                         -> const x
  in
    exp (Dn.exp e)

```

<rtleqn.ml 340d>+≡ (339b) <340c 340e>

```

let equate e1 e2 =
  let t = sub (exp e1) (exp e2) in
  let () = if Debug.on "rtleqn" then
    Printf.eprintf "Rtleqn.equate: %s === %s\n"
      (ToString.t (exp e1)) (ToString.t (exp e2)) in
  t
let () = Debug.register "rtleqn" "RTL equation solver"

```

<rtleqn.ml 340e>+≡ (339b) <340d 341a>

```

module T = struct
  type t = term
  let variable = function (* identify variable *)
    | Var s -> Some s
    | _     -> None
  let compare t t' = match t, t' with
    | (Const l, Const r) -> Rtlutil.Compare.exp (Dn.exp l) (Dn.exp r)
    | (Var l, Var r) -> compares l r

```

```

| (Unit, Unit) -> 0
| (Const _, Var _) -> -1
| (Const _, Unit) -> -1
| (Var _, Const _) -> 1
| (Var _, Unit) -> -1
| (Unit, Const _) -> 1
| (Unit, Var _) -> 1

let print    = ToString.term
end

module Solver = Eqn.Make(T)          (* equation solver over terms *)

<rtleqn.ml 341a>+≡                      (339b) <340e 341b>
let rtl ~(width:int) (e:sum) =
  let add    = Rtlutil.add width in
  let mult   = Rtl.opr "mul" [width] in
  let bits i = Rtl.bits (Bits.S.of_int i width) width in
  let late x = Rtl.late x width in
  let summand = function
    | (1,Var x)   -> late x
    | (i,Var x)   -> Rtl.app mult [bits i; late x]
    | (i,Unit)    -> bits i
    | (1,Const k) -> k
    | (i,Const k) -> Rtl.app mult [bits i;k] in
  let rec loop e = function
    | []    -> e
    | [s]   -> add e (summand s)
    | s::ss -> loop (add e (summand s)) ss
  in
  match e with
  | []    -> Rtl.bits (Bits.zero width) width
  | s::ss -> loop (summand s) ss

<rtleqn.ml 341b>+≡                      (339b) <341a
let solve ~width sums =
  try
    let eqns   = List.fold_right Solver.make_zero sums Solver.empty in
    let result = Solver.solve eqns in
    let to_rtl = function (v,sum) -> (v,rtl width sum) in
      { known      = List.map to_rtl result.Solver.known
        ; dependent = List.map to_rtl result.Solver.dependent
        }
  with
  Solver.Can'tSolve _ -> raise Can'tSolve

```

Appendix N

cfg/legacy

N.1 cfg/legacy/cfg.nw

$\langle \text{front_cfg/cfg.mli } 342a \rangle \equiv$
 $\langle \text{cfg.mli } 342b \rangle$

N.2 Control-Flow Graph Revisions

N.3 Goals

N.4 The Control-Flow Graph

N.5 Invariants

N.6 Interface

$\langle \text{cfg.mli } 342b \rangle \equiv$ (342a) 343a $\triangleright$
 $\langle \text{exported signatures } 342c \rangle$

$\langle \text{exported signatures } 342c \rangle \equiv$ (346c 342b)

```
module type X = sig
  type jx (* extension at join point *)
  type fx (* extension at fork point *)
  type nx (* extension at non-fork/join nodes *)
  val jx : unit -> jx
  val fx : unit -> fx
  val nx : unit -> nx
end

module type S = sig
  module X : X

  type label = string
  type 'i cfg
  type 'i t = 'i cfg
  type 'i node
  type regs = Register.Set.t (* sets of regs for dataflow *)
  type xregs = Register.SetX.t (* sets of regs for dataflow *)
  type 'i contedge = { kills:regs; defs:regs; node:'i node }
  type kind = (* all the kinds of nodes *)
```

```

      | Join | Instruction | StackAdjust | Branch | Cbranch | Mbranch
      | Jump | Call | Return | CutTo
      | Entry | Exit | Assertion | Illegal | Impossible
    <graph and node observers 343b>
    <graph and node constructors 343d>
    <graph and node mutators 343c>
  end

<cfg.mli 343a>+≡ (342a) <342b>
  module Make (X:X) : S with module X = X

```

N.6.1 Interface summary

```

<graph and node observers 343b>≡ (342c) 343e▷
<graph and node mutators 343c>≡ (342c) 344i▷
<graph and node constructors 343d>≡ (342c) 345g▷

```

N.6.2 Observers

```

<graph and node observers 343e>+≡ (342c) <343b 343f>
  val of_node      : 'i node -> 'i cfg

<graph and node observers 343f>+≡ (342c) <343e 343g>
  val kind         : 'i node -> kind
  val num          : 'i node -> int
  val eq           : 'i node -> 'i node -> bool

<graph and node observers 343g>+≡ (342c) <343f 343h>
  val is_join      : 'i node -> bool
  val is_non_local_join : 'i node -> bool
  val is_fork      : 'i node -> bool
  val is_head      : 'i node -> bool

<graph and node observers 343h>+≡ (342c) <343g 343i>
  val label        : 'i node -> label      (* defined on any join node except exit *)
  val labels       : 'i node -> label list (* defined on any join node except exit *)

<graph and node observers 343i>+≡ (342c) <343h 343j>
  val to_instr      : 'i node -> 'i option
  val to_executable : 'i node -> 'i option

<graph and node observers 343j>+≡ (342c) <343i 343k>
  val pred          : 'i node -> 'i node (* defined on non-join, non-exit *)
  val succ          : 'i node -> 'i node (* defined on non-fork *)

<graph and node observers 343k>+≡ (342c) <343j 343l>
  val tsucc         : 'i node -> 'i node (* defined on conditional branch *)
  val fsucc         : 'i node -> 'i node (* defined on conditional branch *)

<graph and node observers 343l>+≡ (342c) <343k 343m>
  val succ_n        : 'i node -> int -> 'i node (* defined on any node *)

<graph and node observers 343m>+≡ (342c) <343l 344a>
  val preds         : 'i node -> 'i node list (* defined on any node *)
  val succs         : 'i node -> 'i node list (* defined on any node *)

```


⟨graph and node observers 344a⟩+≡ (342c) <343m 344b>
 val altrets : 'i node -> int (* defined only on call nodes *)
 val cuts_to : 'i node -> int (* defined only on call nodes *)
 val unwinds_to : 'i node -> int (* defined only on call nodes *)

⟨graph and node observers 344b⟩+≡ (342c) <344a 344c>
 val is_cti : 'i node -> bool (* is any control-flow 'i node *)
 val is_br : 'i node -> bool (* is direct branch *)
 val is_cbr : 'i node -> bool (* is conditional branch *)
 val is_mbr : 'i node -> bool (* is multiway branch *)
 val is_call : 'i node -> bool
 val is_cut : 'i node -> bool

⟨graph and node observers 344c⟩+≡ (342c) <344b 344d>
 val spans : 'i node -> Spans.t option

⟨graph and node observers 344d⟩+≡ (342c) <344c 344e>
 val jx : 'i node -> X.jx (* defined only on join nodes *)
 val fx : 'i node -> X.fx (* defined only on fork nodes *)
 val nx : 'i node -> X.nx (* defined only on non-fork, non-join *)

⟨graph and node observers 344e⟩+≡ (342c) <344d 344f>
 val entry : 'i cfg -> 'i node
 val exit : 'i cfg -> 'i node

⟨graph and node observers 344f⟩+≡ (342c) <344e 344g>
 val iter_nodes : ('i node -> unit) -> 'i cfg -> unit
 val iter_heads : ('i node -> unit) -> 'i cfg -> unit
 val fold_nodes : ('i node -> 'a -> 'a) -> 'a -> 'i cfg -> 'a
 val fold_heads : ('i node -> 'a -> 'a) -> 'a -> 'i cfg -> 'a
 val fold_layout : ('i node -> 'a -> 'a) -> 'a -> 'i cfg -> 'a
 val postorder_dfs : ('i node -> 'a -> 'a) -> 'a -> 'i cfg -> 'a
 val reverse_podfs : ('i node -> 'a -> 'a) -> 'a -> 'i cfg -> 'a

⟨graph and node observers 344g⟩+≡ (342c) <344f 344h>
 val union_over_outedges :
 'i node -> noflow:('i node -> xregs) -> flow:('i contedge -> xregs) -> xregs
 val add_inedge_uses : 'i node -> xregs -> xregs
 val add_live_spans : 'i node -> xregs -> xregs

⟨graph and node observers 344h⟩+≡ (342c) <344g>
 val print_node : Rtl.rtl node -> string

N.6.3 Mutators

⟨graph and node mutators 344i⟩+≡ (342c) <343c 344j>

⟨graph and node mutators 344j⟩+≡ (342c) <344i 345a>
 module Tx : sig
 exception Exhausted
 val start_cond : string -> bool (* returns true iff transaction OK *)
 val start_exn : string -> unit (* raises Exhausted iff transaction not OK *)
 val finish : unit -> unit (* marks end of transaction *)

 val set_limit : int -> unit (* set the transaction limit *)
 val used : unit -> int (* say how many transactions were used *)
 val last : unit -> string (* string passed to the last transaction *)
 end

$\langle$ graph and node mutators 345a $\rangle + \equiv$ (342c) $\triangleleft$ 344j 345b $\triangleright$
 val set_succ : 'i cfg -> 'i node -> succ:'i node -> unit (* defined on non-fork *)
 val set_tsucc : 'i cfg -> 'i node -> succ:'i node -> unit
 (* defined on conditional branch *)
 val set_fsucc : 'i cfg -> 'i node -> succ:'i node -> unit
 (* defined on conditional branch *)
 val set_succ_n : 'i cfg -> 'i node -> int -> succ:'i node -> unit
 (* defined on any node *)

$\langle$ graph and node mutators 345b $\rangle + \equiv$ (342c) $\triangleleft$ 345a 345c $\triangleright$
 val invert_cbr : 'i node -> unit (* defined on conditional branch *)

$\langle$ graph and node mutators 345c $\rangle + \equiv$ (342c) $\triangleleft$ 345b 345d $\triangleright$
 val splice_on_every_edge_between : 'i cfg -> entry:'i node -> exit:'i node ->
 pred:'i node -> succ:'i node -> unit
 (* [[pred]] is not multiway branch, call, or cutto *)
 val splice_before : 'i cfg -> entry:'i node -> exit:'i node -> 'i node -> unit
 (* node is not join *)
 val splice_after : 'i cfg -> entry:'i node -> exit:'i node -> 'i node -> unit
 (* node is not fork *)

$\langle$ graph and node mutators 345d $\rangle + \equiv$ (342c) $\triangleleft$ 345c 345e $\triangleright$
 val delete : 'i cfg -> 'i node -> unit (* must have unique pred and succ *)

$\langle$ graph and node mutators 345e $\rangle + \equiv$ (342c) $\triangleleft$ 345d 345f $\triangleright$
 val update_instr : ('i -> 'i) -> 'i node -> unit

$\langle$ graph and node mutators 345f $\rangle + \equiv$ (342c) $\triangleleft$ 345e $\triangleright$
 val set_spans : 'i node -> Spans.t -> unit

N.6.4 Constructors

$\langle$ graph and node constructors 345g $\rangle + \equiv$ (342c) $\triangleleft$ 343d 345h $\triangleright$
 type label_supply = string -> string
 type 'i instruction_info = { goto : (label, 'i) Ep.map
 ; branch : (Rtl.exp * label, 'i) Ep.map
 ; bnegate : 'i -> 'i
 }
 val mk : 'i instruction_info -> label_supply -> 'i cfg

$\langle$ graph and node constructors 345h $\rangle + \equiv$ (342c) $\triangleleft$ 345g 345i $\triangleright$
 val copy : 'j instruction_info -> ('i -> 'j) -> 'i cfg -> 'j cfg

$\langle$ graph and node constructors 345i $\rangle + \equiv$ (342c) $\triangleleft$ 345h 345j $\triangleright$
 val node_labeled : 'i cfg -> label -> 'i node
 val non_local_join_labeled : 'i cfg -> label -> 'i node

$\langle$ graph and node constructors 345j $\rangle + \equiv$ (342c) $\triangleleft$ 345i 345k $\triangleright$
 val join_leading_to : succ:'i node -> 'i node

$\langle$ graph and node constructors 345k $\rangle + \equiv$ (342c) $\triangleleft$ 345j 346a $\triangleright$
 val instruction : 'i cfg -> 'i -> succ:'i node -> 'i node
 val stack_adjust : 'i cfg -> 'i -> succ:'i node -> 'i node
 val assertion : 'i cfg -> 'i -> succ:'i node -> 'i node

```

⟨graph and node constructors 346a⟩+≡ (342c) <345k 346b>
val branch      : 'i cfg -> target:'i node -> 'i node
val jump        : 'i cfg -> 'i -> uses:regs -> targets:string list -> 'i node
val cbranch     : 'i cfg -> Rtl.exp -> ifso:'i node -> ifnot:'i node -> 'i node
val mbranch     : 'i cfg -> 'i -> targets:'i node list -> 'i node
val call        : 'i cfg -> 'i -> altrets:'i contedge list -> succ:'i node ->
    unwinds_to:'i contedge list -> cuts_to:'i contedge list ->
    aborts:bool -> uses:regs -> defs:regs -> kills:regs ->
    reads:string list option -> writes:string list option ->
    spans:Spans.t option -> 'i node
val cut_to      : 'i cfg -> 'i -> cuts_to:'i contedge list -> aborts:bool ->
    uses:regs -> 'i node
val return      : 'i cfg -> 'i -> uses:regs -> 'i node

⟨graph and node constructors 346b⟩+≡ (342c) <346a
val impossible  : 'i cfg -> succ:'i node -> 'i node
val illegal     : 'i cfg -> 'i node (* a node that must never be reached *)

```

N.7 Implementation

```

⟨cfg.ml 346c⟩≡
open Nopoly
⟨exported signatures 342c⟩
module Make (X:X) : S with module X = X = struct
  module R    = Register
  module RS   = R.Set
  type regs   = Register.Set.t (* sets of regs for dataflow *)
  type xregs  = Register.SetX.t (* sets of regs for dataflow *)
  module IS   = Set.Make (struct type t = int let compare = compare end)
  module RSX  = R.SetX
  module SM   = Strutil.Map
  module X    = X

  type label = string
  type label_supply = string -> label
  type 'i instruction_info = { goto      : (label, 'i) Ep.map
    ; branch : (Rtl.exp * label, 'i) Ep.map
    ; bnegate : 'i -> 'i
    }

  type kind = (* all the kinds of nodes *)
    | Join | Instruction | StackAdjust | Branch | Cbranch | Mbranch
    | Jump  | Call  | Return | CutTo
    | Entry | Exit  | Assertion | Illegal | Impossible
  ⟨node type 347⟩
  and 'i contedge = { kills:regs; defs:regs; node:'i node }
  and 'i cfg = { mutable entry      : 'i node
    ; mutable exit      : 'i node (*mutable for bootstrapping cfg*)
    ; mutable ill       : 'i node (*mutable for bootstrapping cfg*)
    ; inst_info : 'i instruction_info
    ; mutable label_map : 'i node SM.t
    ; mk_label  : label_supply
    }
  type 'i t = 'i cfg

  ⟨graph utilities 364a⟩
  ⟨graph and node observers implementation 349⟩
  ⟨printing utilities 366b⟩

```

```

    <remove node predecessors 354c>
    <set node successors 355a>

    <simple node constructors 362>
    <node constructors with interesting control flow edges 361>
    <graph construction and copying 360a>

    <graph and node mutators implementation 358a>
end

```

N.7.1 Node Type

```

<node type 347>≡
    type ('i, 'n)      ent = {
                        ent_num      : int
                        ; mutable ent_succ : 'n
                        ; ent_nx      : X.nx
                        }
    type ('i, 'n, 'c) exi = {
                        exi_num      : int
                        ; mutable exi_cfg  : 'c
                        ; mutable exi_labels : label list
                        ; mutable exi_preds : 'n list
                        ; mutable exi_lpred : 'n
                        ; exi_jx      : X.jx
                        }
    type ('i, 'n, 'c) joi = {
                        joi_num      : int
                        ; joi_local  : bool
                        ; mutable joi_labels : label list
                        ; mutable joi_cfg  : 'c
                        ; mutable joi_preds : 'n list
                        ; mutable joi_succ  : 'n
                        ; mutable joi_lpred : 'n
                        ; joi_jx      : X.jx
                        ; mutable joi_spans : Spans.t option
                        }
    type ('i, 'n)      ins = {
                        ins_num      : int
                        ; mutable ins_i   : 'i
                        ; mutable ins_pred : 'n
                        ; mutable ins_succ : 'n
                        ; ins_nx      : X.nx
                        }
    type ('i, 'n)      sta = {
                        sta_num      : int
                        ; mutable sta_i   : 'i
                        ; mutable sta_pred : 'n
                        ; mutable sta_succ : 'n
                        ; sta_nx      : X.nx
                        }
    type ('i, 'n)      ass = {
                        ass_num      : int
                        ; mutable ass_i   : 'i
                        ; mutable ass_pred : 'n
                        ; mutable ass_succ : 'n
                        ; ass_nx      : X.nx
                        }
    type ('i, 'n)      bra = {
                        bra_num      : int
                        ; mutable bra_i   : 'i
                        ; mutable bra_pred : 'n
                        ; mutable bra_succ : 'n
                        ; mutable bra_lsucc : 'n
                        ; bra_nx      : X.nx
                        }

```

(346c) 348▷

```

type ('i, 'n)      cbr = {
                    cbr_num      : int
                    ; mutable cbr_i      : 'i
                    ; mutable cbr_pred   : 'n
                    ; mutable cbr_true   : 'n
                    ; mutable cbr_false  : 'n
                    ;      cbr_fx      : X.fx
                    }

```

```

type ('i, 'n)      mbr = {
                    mbr_num      : int
                    ; mutable mbr_i      : 'i
                    ; mutable mbr_pred   : 'n
                    ; mutable mbr_succs  : 'n array
                    ; mutable mbr_lsucc  : 'n
                    ;      mbr_fx      : X.fx
                    }

```

⟨node type 348⟩+≡

(346c) ◁347

```

type ('i, 'n, 'e) cal = {
                    cal_num      : int
                    ; mutable cal_i      : 'i
                    ; mutable cal_pred   : 'n
                    ; mutable cal_contedges : 'e array
                    ; mutable cal_lsucc  : 'n
                    ;      cal_fx      : X.fx
                    ; mutable cal_spans  : Spans.t option
                    ; mutable cal_uses   : regs
                    ;      cal_altrets  : int
                    ;      cal_unwinds_to : int
                    ;      cal_cuts_to  : int
                    ;      cal_reads   : string list option
                    ;      cal_writes  : string list option
                    }

```

```

type ('i, 'n, 'e) cut = {
                    cut_num      : int
                    ; mutable cut_i      : 'i
                    ; mutable cut_pred   : 'n
                    ; mutable cut_lsucc  : 'n
                    ; mutable cut_contedges : 'e array
                    ;      cut_fx      : X.fx
                    ; mutable cut_uses   : regs
                    }

```

```

type ('i, 'n)      jum = {
                    jum_num      : int
                    ; mutable jum_i      : 'i
                    ; mutable jum_pred   : 'n
                    ; mutable jum_succ   : 'n
                    ; mutable jum_lsucc  : 'n
                    ;      jum_nx      : X.nx
                    ; mutable jum_uses   : regs
                    }

```

```

type ('i, 'n)      ret = {
                    ret_num      : int
                    ; mutable ret_i      : 'i
                    ; mutable ret_pred   : 'n
                    ; mutable ret_succ   : 'n
                    ; mutable ret_lsucc  : 'n
                    ;      ret_nx      : X.nx
                    ; mutable ret_uses   : regs
                    }

```

```

type ('i, 'n)      imp = {
                    imp_num      : int
                    ; mutable imp_pred   : 'n
                    ; mutable imp_succ   : 'n
                    ; mutable imp_exit_succ : 'n
                    ;      imp_fx      : X.fx
                    }

```

```

type ('i, 'n, 'c) ill = {
    ill_num      : int
    ; ill_nx     : X.nx
    ; mutable ill_cfg : 'c
}

```

```

type 'i node =
| Boot
| Ent of ('i, 'i node)      ent
| Exi of ('i, 'i node, 'i cfg) exi
| Joi of ('i, 'i node, 'i cfg) joi
| Ins of ('i, 'i node)      ins
| Sta of ('i, 'i node)      sta
| Ass of ('i, 'i node)      ass
| Bra of ('i, 'i node)      bra
| Cbr of ('i, 'i node)      cbr
| Mbr of ('i, 'i node)      mbr
| Cal of ('i, 'i node, 'i contedge) cal
| Cut of ('i, 'i node, 'i contedge) cut
| Ret of ('i, 'i node)      ret
| Jum of ('i, 'i node)      jum
| Imp of ('i, 'i node)      imp
| Ill of ('i, 'i node, 'i cfg) ill

```

(graph and node observers implementation 349)≡

(346c) 350a▷

```

let kind = kind_util
let num = num_util

let is_join node = match node with
| Joi _ | Exi _ -> true
| _          -> false
let is_non_local_join node = match node with
| Joi j -> not j.joi_local
| _      -> false
let is_head node = match node with
| Joi _ | Ent _ -> true
| _      -> false
let is_fork node = match node with
| Cbr _ | Mbr _ | Cal _ | Cut _ | Imp _ -> true
| _      -> false

let to_instr node = match node with
| Ins i -> Some i.ins_i
| Sta i -> Some i.sta_i
| Ass a -> Some a.ass_i
| Bra b -> Some b.bra_i
| Cbr c -> Some c.cbr_i
| Mbr m -> Some m.mbr_i
| Cal c -> Some c.cal_i
| Cut c -> Some c.cut_i
| Jum j -> Some j.jum_i
| Ret r -> Some r.ret_i
| Ent _ -> None
| Exi _ -> None
| Joi _ -> None
| Imp _ -> None
| Ill _ -> None
| Boot -> undef "to_instr or to_executable" "boot"
let to_executable node = match node with
| Ass _ -> None

```

```

| _      -> to_instr node

⟨graph and node observers implementation 350a⟩+≡ (346c) <349 350b>
let label node = match node with
| Joi j -> (try List.hd j.joi_labels
           with Failure _ -> imposs "no labels at join")
| Exi e -> ( log "label called on exit node"
           ; try List.hd e.exi_labels
           with Failure _ -> imposs "no labels at exit"
           )
| _      -> undef "label" "non-join"

let labels node = match node with
| Joi j -> j.joi_labels
| Exi e -> ( log "labels called on exit node" ; e.exi_labels )
| _      -> undef "labels" "non-join"

⟨graph and node observers implementation 350b⟩+≡ (346c) <350a 350c>
let pred node = match node with
| Ins i -> i.ins_pred
| Sta i -> i.sta_pred
| Ass a -> a.ass_pred
| Bra b -> b.bra_pred
| Cbr c -> c.cbr_pred
| Mbr m -> m.mbr_pred
| Cal c -> c.cal_pred
| Cut c -> c.cut_pred
| Jum j -> j.jum_pred
| Ret r -> r.ret_pred
| Imp i -> i.imp_pred
| Ent _ -> undef "pred" "entry"
| Joi _ -> undef "pred" "join"
| Exi _ -> undef "pred" "exit"
| Ill _ -> undef "pred" "illegal"
| Boot -> undef "pred" "boot"

⟨graph and node observers implementation 350c⟩+≡ (346c) <350b 350d>
let tsucc node = match node with
| Cbr c -> c.cbr_true
| _      -> undef "tsucc" "non-cond-branch"
let fsucc node = match node with
| Cbr c -> c.cbr_false
| _      -> undef "fsucc" "non-cond-branch"

⟨graph and node observers implementation 350d⟩+≡ (346c) <350c 351a>
let succ_n node n =
  let fail ()      = raise (Invalid_argument "succ_n: illegal index") in
  let single s n = if n = 0 then s else fail ()          in
  let mult a n     = try a.(n) with Invalid_argument _ -> fail ()      in
  match node with
  | Joi j -> single j.joi_succ n
  | Ins i -> single i.ins_succ n
  | Sta i -> single i.sta_succ n
  | Ass a -> single a.ass_succ n
  | Bra b -> single b.bra_succ n
  | Jum j -> single j.jum_succ n
  | Ret r -> single r.ret_succ n
  | Ent e -> single e.ent_succ n
  | Mbr m -> mult m.mbr_succs n
  | Cal c -> mult (ce_getnodes c.cal_contedges) n

```

```

| Cut c -> mult (ce_getnodes c.cut_contedges) n
| Cbr c ->
  if n = 0 then c.cbr_true
  else if n = 1 then c.cbr_false
  else fail ()
| Imp i ->
  if n = 0 then i.imp_succ
  else if n = 1 then i.imp_exit_succ
  else fail ()
| Exi _ -> undef "succ_n" "exit"
| Ill _ -> undef "succ_n" "illegal"
| Boot _ -> undef "succ_n" "boot"

```

<graph and node observers implementation 351a>+≡ (346c) <350d 351b>

```

let succ node = match node with
| Ins i -> i.ins_succ
| Sta i -> i.sta_succ
| Ass a -> a.ass_succ
| Joi j -> j.joi_succ
| Bra b -> b.bra_succ
| Jum j -> j.jum_succ
| Ret r -> r.ret_succ
| Ent e -> e.ent_succ
| Mbr _ -> undef_with_node (num node) "succ" "multi branch"
| Cbr _ -> undef_with_node (num node) "succ" "cond branch"
| Cal _ -> undef_with_node (num node) "succ" "call"
| Imp _ -> undef_with_node (num node) "succ" "impossible"
| Cut _ -> undef_with_node (num node) "succ" "cut"
| Exi _ -> undef_with_node (num node) "succ" "exit"
| Ill _ -> undef_with_node (num node) "succ" "illegal"
| Boot _ -> undef_with_node (num node) "succ" "boot"

```

<graph and node observers implementation 351b>+≡ (346c) <351a 351c>

```

let get_cfg start_node =
  let rec search n badlist =
    if List.exists (eq n) badlist then
      imposs "cycle in cfg with no join node"
    else match n with
      | Exi e -> e.exi_cfg
      | Joi j -> j.joi_cfg
      | Ill i -> i.ill_cfg
      | n      -> search (succ_n n 0) (n::badlist) in
  search start_node []
let of_node = get_cfg

```

<graph and node observers implementation 351c>+≡ (346c) <351b 352a>

```

let all_incoming node = match node with
| Ins i -> [i.ins_pred]
| Sta i -> [i.sta_pred]
| Ass a -> [a.ass_pred]
| Bra b -> [b.bra_pred]
| Cbr c -> [c.cbr_pred]
| Mbr m -> [m.mbr_pred]
| Cal c -> [c.cal_pred]
| Cut c -> [c.cut_pred]
| Jum j -> [j.jum_pred]
| Ret r -> [r.ret_pred]
| Imp i -> [i.imp_pred]
| Joi j -> j.joi_preds
| Exi e -> e.exi_preds

```



```

| Ent _ -> []
| Ill _ -> []
| Boot  -> undef "preds" "boot"

let preds node = uniq (all_incoming node)

let all_outgoing node = match node with
| Joi j -> [j.joi_succ]
| Ins i -> [i.ins_succ]
| Sta i -> [i.sta_succ]
| Ass a -> [a.ass_succ]
| Bra b -> [b.bra_succ]
| Jum j -> [j.jum_succ]
| Ret r -> [r.ret_succ]
| Ent e -> [e.ent_succ]
| Cbr c -> [c.cbr_false; c.cbr_true]
| Imp i -> [i.imp_succ; i.imp_exit_succ]
| Mbr m -> (Array.to_list m.mbr_succs)
| Cal c -> (Array.to_list (ce_getnodes c.cal_contedges))
| Cut c -> (Array.to_list (ce_getnodes c.cut_contedges))
| Exi _ -> []
| Ill _ -> []
| Boot  -> undef "succs" "boot"
let succs node = uniq (all_outgoing node)

⟨graph and node observers implementation 352a⟩+≡ (346c) <351c 352b>
let altrets node = match node with
| Cal c -> c.cal_altrets
| _      -> undef "altrets" "non-call"
let cuts_to node = match node with
| Cal c -> c.cal_cuts_to
| _      -> undef "cuts" "non-call"
let unwinds_to node = match node with
| Cal c -> c.cal_unwinds_to
| _      -> undef "unwinds" "non-call"

⟨graph and node observers implementation 352b⟩+≡ (346c) <352a 353a>
let is_cti node = match node with
| Bra _ | Cbr _ | Mbr _ | Cal _ | Cut _ -> true
| _                                     -> false
let is_br node = match node with
| Bra _ -> true
| _      -> false
let is_cbr node = match node with
| Cbr _ -> true
| _      -> false
let is_mbr node = match node with
| Mbr _ -> true
| _      -> false
let is_call node = match node with
| Cal _ -> true
| _      -> false
let is_cut node = match node with
| Cut _ -> true
| _      -> false
let spans node = match node with
| Cal c -> c.cal_spans
| Joi j -> j.joi_spans
| _      -> undef_with_node (num node) "spans" "non-call or non-join"

```

```

⟨graph and node observers implementation 353a⟩+≡ (346c) <352b 353b>
  let jx node = match node with
  | Joi j -> j.joi_jx
  | Exi e -> e.exi_jx
  | _      -> undef "jx" "non-join"

  let fx node = match node with
  | Cbr c -> c.cbr_fx
  | Mbr m -> m.mbr_fx
  | Cal c -> c.cal_fx
  | Cut c -> c.cut_fx
  | Imp i -> i.imp_fx
  | _      -> undef "fx" "non-fork"

  let nx node = match node with
  | Ins i -> i.ins_nx
  | Sta i -> i.sta_nx
  | Ass a -> a.ass_nx
  | Bra b -> b.bra_nx
  | Jum j -> j.jum_nx
  | Ret r -> r.ret_nx
  | Ent e -> e.ent_nx
  | Ill i -> i.ill_nx
  | _      -> undef "nx" "fork and join"

  let entry cfg = cfg.entry
  let exit  cfg = cfg.exit

⟨graph and node observers implementation 353b⟩+≡ (346c) <353a 353c>
  let fold_heads f zero cfg = List.fold_left (fun x y -> f y x) zero (heads cfg)

⟨graph and node observers implementation 353c⟩+≡ (346c) <353b 353d>
  ⟨join node set operations 367b⟩
  let fold_nodes cons nil g =
    let cons x y = cons y x in
    let rec vchildren children k set lst =
      match children with
      | c::rst -> vnode c (vchildren rst k) set lst
      | []      -> k set lst
    and vnode n k set lst =
      if IS.mem (num n) set then
        k set lst
      else
        let preds = preds n in
        let succs = succs n in
        vchildren succs (vchildren preds k) (IS.add (num n) set) (n::lst) in
    vnode (entry g) (fun _ lst -> List.fold_left cons nil lst)
    IS.empty []
  let iter_nodes f = fold_nodes (fun n () -> f n) ()
  let iter_heads f = fold_heads (fun n () -> f n) ()

⟨graph and node observers implementation 353d⟩+≡ (346c) <353c 354a>
  let cps_postorder_dfs heads get_children g cons nil cont =
    let rec vnode node cont acc visited =
      if member node visited then
        cont acc visited
      else
        vchildren node (get_children node) cont acc (add node visited)
    and vchildren node children cont acc visited =
      let rec next children acc visited = match children with

```

```

    | [] -> cont (cons node acc) visited
    | n::rst -> vnode n (next rst) acc visited in
  next children acc visited in
let rec fold_heads heads cont acc visited =
  match heads with
  | h::rst -> vnode h (fold_heads rst cont) acc visited
  | []      -> cont acc visited in
fold_heads heads cont nil emptyset

⟨graph and node observers implementation 354a⟩+≡ (346c) <353d 354b>
let postorder_dfs cons nil g =
  cps_postorder_dfs [entry g] succs g cons nil (fun x _ -> x)
let reverse_podfs cons nil g =
  cps_postorder_dfs [exit g] preds g cons nil (fun x _ -> x)

⟨graph and node observers implementation 354b⟩+≡ (346c) <354a
(* should this fn be defined on exi and ill?
   when there are cont edges, do we still need to cover the regular edges?*)
let (++) = RSX.union
let union_over_outedges node ~noflow ~flow =
  let union_contedges ce =
    Array.fold_left (fun r s -> r ++ flow s) RSX.empty ce in
  match node with
  | Cal c -> union_contedges c.cal_contedges
  | Cut c -> union_contedges c.cut_contedges
  | Cbr c -> noflow c.cbr_true ++ noflow c.cbr_false
  | Imp i -> noflow i.imp_succ ++ noflow i.imp_exit_succ
  | Mbr m -> Array.fold_right (fun s -> (++) (noflow s)) m.mbr_succs RSX.empty
  | Ins i -> noflow i.ins_succ
  | Sta i -> noflow i.sta_succ
  | Ass a -> noflow a.ass_succ
  | Bra b -> noflow b.bra_succ
  | Jum j -> noflow j.jum_succ
  | Ret r -> noflow r.ret_succ
  | Ent e -> noflow e.ent_succ
  | Joi j -> noflow j.joi_succ
  | Exi e -> RSX.empty
  | Ill i -> RSX.empty
  | Boot -> imposs "union_over_outedges undefined on boot nodes"

let add_inedge_uses node regs =
  let reg_add = RS.fold (fun r rst -> RSX.add (R.Reg r) rst) in
  match node with
  | Cal c -> reg_add c.cal_uses regs
  | Cut c -> reg_add c.cut_uses regs
  | Jum j -> reg_add j.jum_uses regs
  | Ret r -> reg_add r.ret_uses regs
  | _      -> regs

let add_live_spans node regs =
  let span_add spans rst = match spans with
    | Some ss -> Spans.fold_live_locs RSX.add ss rst
    | None     -> rst in
  match node with
  | Cal c -> span_add c.cal_spans regs
  | Joi j -> span_add j.joi_spans regs
  | _      -> regs

⟨remove node predecessors 354c⟩≡ (346c)
let remove_pred cfg node ~old_succ =

```

```

let rec rem_1_list lst = match lst with
| [] -> imposs "remove_pred: predecessor to be removed not found"
| n::rst when eq n node -> rst
| n::rst -> n :: rem_1_list rst in
match old_succ with
| Joi j -> j.joi_preds <- rem_1_list j.joi_preds
| Exi e -> e.exi_preds <- rem_1_list e.exi_preds
| Ins i -> i.ins_pred <- cfg.ill
| Sta i -> i.sta_pred <- cfg.ill
| Bra b -> b.bra_pred <- cfg.ill
| Cbr c -> c.cbr_pred <- cfg.ill
| Mbr m -> m.mbr_pred <- cfg.ill
| Jum j -> j.jum_pred <- cfg.ill
| Cal c -> c.cal_pred <- cfg.ill
| Ret r -> r.ret_pred <- cfg.ill
| Cut c -> c.cut_pred <- cfg.ill
| Ass a -> a.ass_pred <- cfg.ill
| Imp i -> i.imp_pred <- cfg.ill
| Ill _ -> ()
| Ent _ -> imposs "entry node was a successor"
| Boot -> imposs "boot nodes should not live to remove_pred"

```

<set node successors 355a>≡ (346c) 355b▷

```

let mk_join cfg labels ~preds ~lpred ~succ ~local ~spans =
  let join = Joi { joi_num      = new_num ()
                  ; joi_local   = local
                  ; joi_labels  = labels
                  ; joi_cfg     = cfg
                  ; joi_preds   = preds
                  ; joi_succ    = succ
                  ; joi_lpred   = lpred
                  ; joi_jx      = X.jx ()
                  ; joi_spans   = spans
                  } in
  List.iter (fun l -> cfg.label_map <- SM.add l join cfg.label_map) labels;
  join

let node_labeled cfg label =
  try SM.find label cfg.label_map
  with Not_found -> mk_join cfg [label] ~preds:[] ~lpred:cfg.ill ~succ:cfg.ill
    ~local:true ~spans:None

let non_local_join_labeled cfg label =
  try let n = SM.find label cfg.label_map in
    match n with
    | Joi j when j.joi_local ->
      imposs "CFG: non_local_join_labeled called on local join"
    | Joi _ -> n
    | _ -> imposs "CFG: non-join found in label_map"
  with Not_found -> mk_join cfg [label] ~preds:[] ~lpred:cfg.ill ~succ:cfg.ill
    ~local:false ~spans:None

```

<set node successors 355b>+≡ (346c) <355a 356a>

```

let rec join_leading_to ~succ =
  if is_join succ then succ
  else
    let cfg = get_cfg succ in
    if is_ill succ then
      let join = node_labeled cfg (cfg.mk_label "join") in
      let () = set_succ cfg join ~succ:succ in
      join

```

```

else
  let p = pred succ in (* p must not be a fork b/c succ is not a join *)
  if is_join p then p
  else let join = node_labeled cfg (cfg.mk_label "join") in
        let () = if not (is_ill p) then set_succ cfg p ~succ:join in
        let () = set_succ cfg join ~succ:succ in
        join

⟨set node successors 356a⟩+≡ (346c) <355b 357c>
and set_succ' cfg ~single ~ce_mult ~mult ~cbr ~node ~old_succ ~succ =
  if not (is_join succ) && not (is_ill succ) && not (is_ill (pred succ)) then
    set_succ' cfg ~single ~ce_mult ~mult ~cbr ~node ~old_succ
    ~succ:(join_leading_to succ)
else
  begin
    ( (
  ⟨modify the nth successor of node because node is no longer a predecessor 357a⟩
    )
    ; let succ = (
      ⟨update node's nth successor field to point to succ 356b⟩
    ) in
    (if not (is_ill succ)
      then
        ⟨modify succ because node has become a predecessor 357b⟩
      )
    )
  end

⟨update node's nth successor field to point to succ 356b⟩≡ (356a)
match node with
| Joi j -> j.joi_succ <- single succ; succ
| Ins i -> i.ins_succ <- single succ; succ
| Sta i -> i.sta_succ <- single succ; succ
| Ass a -> a.ass_succ <- single succ; succ
| Jum j -> j.jum_succ <- single succ; succ
| Ret r -> r.ret_succ <- single succ; succ
| Ent e -> e.ent_succ <- single succ; succ
| Mbr m -> ignore (mult m.mbr_succs (fun _ -> succ)); succ
| Cal c ->
  let succ = join_leading_to succ in
  ignore (ce_mult c.cal_contedges (fun s -> {s with node = succ})); succ
| Cut c -> ignore (ce_mult c.cut_contedges (fun s -> {s with node = succ})); succ
| Bra b ->
  let succ = join_leading_to succ in
  ( b.bra_i <- (get_cfg node).inst_info.goto.Ep.embed (label succ)
    ; b.bra_succ <- single succ
    ; succ
  )
| Cbr c ->
  let succ = join_leading_to succ in
  let cfg = get_cfg node in
  (if cbr () then
    c.cbr_true <- succ
  else
    c.cbr_false <- succ
  );
  let (cond, _) = cfg.inst_info.branch.Ep.project c.cbr_i in
  c.cbr_i <- cfg.inst_info.branch.Ep.embed (cond, label c.cbr_true);
  succ
| Imp i ->

```

```

let succ = join_leading_to succ in
if cbr () then i.imp_succ <- succ
else if is_exi succ then i.imp_exit_succ <- succ
else imposs "exit_succ must be an exit node";
succ
| Exi _ -> undef "set_succ" "exit"
| Ill _ -> undef "set_succ" "illegal"
| Boot _ -> undef "set_succ" "boot"

```

$\langle \text{modify the } n\text{th successor of node because node is no longer a predecessor } 357a \rangle \equiv (356a)$

```

remove_pred cfg node old_succ

```

$\langle \text{modify succ because node has become a predecessor } 357b \rangle \equiv (356a)$

```

let join_pred cur_preds = match cur_preds with
| [Ill _] -> [node]
| _ -> node::cur_preds in
match succ with
| Ins i -> i.ins_pred <- node
| Sta i -> i.sta_pred <- node
| Ass a -> a.ass_pred <- node
| Bra b -> b.bra_pred <- node
| Jum j -> j.jum_pred <- node
| Ret r -> r.ret_pred <- node
| Imp i -> i.imp_pred <- node
| Cbr c -> c.cbr_pred <- node
| Mbr m -> m.mbr_pred <- node
| Cal c -> c.cal_pred <- node
| Cut c -> c.cut_pred <- node
| Exi e -> e.exi_preds <- join_pred e.exi_preds
| Joi j -> j.joi_preds <- join_pred j.joi_preds
| Ent e -> imposs "something tried to become a predecessor of the entry node"
| Ill i -> imposs "client can make a pointer dangle, but believed to be handled above"
| Boot _ -> undef "add_pred" "boot"

```

$\langle \text{set node successors } 357c \rangle + \equiv (346c) \triangleleft 356a \ 357d \triangleright$

```

and set_succ_n cfg node n ~succ:succ =
  set_succ' cfg
    ~single: (if n = 0 then (fun s -> s) else (fun s -> invalid_arg "successor index"))
    ~ce_mult: (fun a f -> let s = a.(n) in a.(n) <- f s; s)
    ~mult: (fun a f -> let s = a.(n) in a.(n) <- f s; s)
    ~cbr: (if n = 0 then (fun () -> true) else if n = 1 then (fun () -> false)
           else fun () -> invalid_arg "successor index")
    ~node ~old_succ: (succ_n node n) ~succ
and set_succ cfg node ~succ:the_succ =
  set_succ' cfg
    ~single: (fun s -> s)
    ~ce_mult: (fun a f -> undef "set_succ" "multiple node not o/w specified")
    ~mult: (fun a f -> undef "set_succ" "multiple node not o/w specified")
    ~cbr: (fun () -> undef "set_succ" "fork")
    ~node ~old_succ: (succ node) ~succ:the_succ

```

$\langle \text{set node successors } 357d \rangle + \equiv (346c) \triangleleft 357c$

```

let set_tsucc cfg node ~succ = match node with
| Cbr _ -> set_succ_n cfg node 0 ~succ
| _ -> undef "set_tsucc" "non-cond-branch"
let set_fsucc cfg node ~succ = match node with
| Cbr _ -> set_succ_n cfg node 1 ~succ
| _ -> undef "set_fsucc" "non-cond-branch"

```

```

⟨graph and node mutators implementation 358a⟩≡ (346c) 358b▷
module Tx = struct
  exception Exhausted
  type t = { mutable limit : int; mutable remaining : int; mutable last : string; }
  let ts = { limit = max_int; remaining = max_int; last = "<none>"; }

  let start_cond who =
    if ts.remaining > 0 then
      ( ts.remaining <- ts.remaining - 1; ts.last <- who; true )
    else
      false

  let start_exn who = if start_cond who then () else raise Exhausted

  let finish _ = ()

  let set_limit n = ts.limit <- n; ts.remaining <- n
  let used _ = ts.limit - ts.remaining
  let last _ = ts.last

  let _ = Reinit.at (fun () -> begin ts.remaining <- ts.limit; ts.last <- "<none>" end)
end

```

```

⟨graph and node mutators implementation 358b⟩+≡ (346c) <358a 358c>
let invert_cbr node = match node with
| Cbr c ->
  let cfg = get_cfg node in
  let new_true = c.cbr_false in
  ( set_fsucc cfg node ~succ:c.cbr_true
  ; set_tsucc cfg node ~succ:new_true
  ; c.cbr_i <- cfg.inst_info.bnegate c.cbr_i
  )
| _ -> undef "invert_cbr" "non-cond-branch"

```

```

⟨graph and node mutators implementation 358c⟩+≡ (346c) <358b 359a>
let splice_on_every_edge_between cfg ~entry ~exit ~pred ~succ =
  let () = if not (List.exists (eq pred) (preds succ))
    then (Printf.eprintf "Splicing between pred:%d succ:%d\n" (num pred) (num
      succ)); assert false)
    else () in
  let () =
    if not (List.exists (eq succ) (succs pred))
    then imposs "splice_on_every_edge_between: succ is not a successor of pred" in
  let update_succ () = set_succ cfg exit ~succ in
  let update_pred () =
    match pred with
    | Cbr c ->
      let entry = join_leading_to entry in
      if eq succ c.cbr_true then set_tsucc cfg pred entry;
      if eq succ c.cbr_false then set_fsucc cfg pred entry
    | Imp i ->
      let upd_succ n entry = if eq n succ then entry else n in
      ( i.imp_succ <- upd_succ i.imp_succ entry
      ; i.imp_exit_succ <- upd_succ i.imp_exit_succ entry
      )
    | Mbr m -> undef "splice_on_every_edge_between" "mbranch"
    | Cal c -> undef "splice_on_every_edge_between" "call"
    | Cut c -> undef "splice_on_every_edge_between" "cut-to"
    | _ -> set_succ cfg pred ~succ:entry in
  (update_pred (); update_succ ())

```

```

let splice_before cfg ~entry ~exit node =
  let p = pred node in
  if Pervasives.<> (kind p) Illegal then set_succ cfg p ~succ:entry;
  set_succ cfg exit ~succ:node

let splice_after cfg ~entry ~exit node =
  let succ = succ node in
  (set_succ cfg node ~succ:entry ; set_succ cfg exit ~succ)

let delete cfg node =
  ( if List.for_all (fun p -> kind p == Illegal) (all_incoming node)
    then List.iter (fun s -> remove_pred cfg node ~old_succ:s) (all_outgoing node)
  ; if kind node == Join
    then cfg.label_map <- List.fold_right SM.remove (labels node) cfg.label_map
  )

let update_instr upd node = match node with
| Ins i -> i.ins_i <- upd i.ins_i
| Sta i -> i.sta_i <- upd i.sta_i
| Ass a -> a.ass_i <- upd a.ass_i
| Bra b -> b.bra_i <- upd b.bra_i
| Jum j -> j.jum_i <- upd j.jum_i
| Ret r -> r.ret_i <- upd r.ret_i
| Mbr m -> m.mbr_i <- upd m.mbr_i
| Cal c -> c.cal_i <- upd c.cal_i
| Cut c -> c.cut_i <- upd c.cut_i
| Cbr c -> c.cbr_i <- upd c.cbr_i
| Joi j -> ()
| Ent e -> ()
| Exi _ -> ()
| Imp i -> ()
| Ill _ -> ()
| Boot -> imposs "update_instr undefined on boot nodes"

```

<graph and node mutators implementation 359a>+≡ (346c) <358c 359b>

```

let set_spans n spans = match n with
| Cal c -> c.cal_spans <- Some spans
| Joi j -> j.joi_spans <- Some spans
| _ -> undef_with_node (num n) "set_spans" "non-call or non-join"

```

<graph and node mutators implementation 359b>+≡ (346c) <359a

```

let fold_layout f zero cfg =
  let multiple_preds = function (* N.B. branch to exit is never needed *)
  | Joi j -> (match j.joi_preds with _ :: _ :: _ -> true | _ -> false)
  | _ -> false in
  let ins_branch node = match node with
  | Cbr c ->
    if multiple_preds c.cbr_false then
      set_succ cfg node ~succ:(join_leading_to (branch cfg c.cbr_false))
  | Cal c ->
    if multiple_preds (succ_n node 0) then
      set_succ_n cfg node 0 ~succ:(join_leading_to (branch cfg (succ_n node 0)))
  | Imp i ->
    if multiple_preds i.imp_succ then
      set_succ cfg node ~succ:(join_leading_to (branch cfg i.imp_succ))
  | Joi _ | Ins _ | Sta _ | Ass _ | Ret _ | Ent _ ->
    if multiple_preds (succ node) then
      set_succ cfg node ~succ:(branch cfg (succ node))

```



```

    | Mbr _ | Bra _ | Cut _ | Jum _ | Exi _ | Ill _ | Boot -> () in
let () = iter_nodes ins_branch cfg in
let snoc node build = fun tail -> build (f node tail) in
cps_postorder_dfs [entry cfg] succs cfg snoc (fun x -> x) (fun build _ -> build zero)

```

⟨graph construction and copying 360a⟩≡ (346c) 360b▷

```

let mk_entry succ =
  Ent { ent_num = new_num ()
        ; ent_succ = succ
        ; ent_nx = X.nx ()
      }
let mk_exit cfg labels ~preds ~lpred =
  let exit = Exi { exi_num = new_num ()
                  ; exi_cfg = cfg
                  ; exi_labels = labels
                  ; exi_preds = []
                  ; exi_lpred = cfg.ill
                  ; exi_jx = X.jx ()
                } in
  List.iter (fun l -> cfg.label_map <- SM.add l exit cfg.label_map) labels;
  exit

```

⟨graph construction and copying 360b⟩+≡ (346c) ◁360a 360c▷

```

let mk_i_info label_supply =
  let entry = match mk_entry Boot with
    | Ent e -> e
    | _ -> imposs "mk_entry returned non-entry node" in
  let cfg = { entry = Ent entry
              ; exit = Boot
              ; ill = Boot
              ; inst_info = i_info
              ; label_map = SM.empty
              ; mk_label = label_supply
            } in
  let ill = illegal cfg in
  let exit = mk_exit cfg [label_supply "exit"] ~preds:[] ~lpred:ill in
  let () = cfg.ill <- ill in
  let () = entry.ent_succ <- ill in
  let () = cfg.exit <- exit in
  cfg

```

⟨graph construction and copying 360c⟩+≡ (346c) ◁360b

```

let copy_info transform cfg =
  assert false (* FOLLOWING NEVER TESTED; DON'T BELIEVE THE HYPE *)
(*
  let new_cfg = { entry = Boot
                  ; exit = Boot
                  ; ill = Boot
                  ; inst_info = info
                  ; label_map = SM.empty
                  ; mk_label = cfg.mk_label
                } in
  let () = new_cfg.ill <- illegal new_cfg in
  let lookup join =
    try SM.find (label join) new_cfg.label_map
    with Not_found ->
      mk_join new_cfg (labels join) ~preds:[] ~lpred:new_cfg.ill
      ~succ:new_cfg.ill ~spans:join.joi_spans in
  let ce_lookup ce =
    let n' = lookup ce.node in

```

```

{ce with node = n'} in
let convert_contedges contedges = Array.map ce_lookup contedges in
let convert node succ =
  match node with
  | Ins i -> instruction new_cfg (transform i.ins_i) ~succ
  | Sta i -> stack_adjust new_cfg (transform i.sta_i) ~succ
  | Ass a -> assertion new_cfg (transform a.ass_i) ~succ
  | Bra b -> branch new_cfg ~target:(lookup b.bra_succ)
  | Jum j -> jump new_cfg (transform j.jum_i) ~uses:j.jum_uses
  | Cbr c ->
    let (cond, _) = cfg.inst_info.branch.Ep.project c.cbr_i in
    cbranch new_cfg cond ~ifso:(lookup c.cbr_true)
    ~ifnot:(lookup c.cbr_false)
  | Ret r -> return new_cfg (transform r.ret_i) ~uses:r.ret_uses
  | Imp i -> impossible new_cfg ~succ
  | Ill i -> new_cfg.ill
  | Mbr m ->
    mbranch new_cfg (transform m.mbr_i)
    ~targets:(Array.fold_right (fun t rst -> lookup t :: rst)
    m.mbr_succs [])
  | Cal c ->
    let ce_lst = convert_contedges c.cal_contedges in
    mk_call (transform c.cal_i) ~pred:new_cfg.ill
    ~contedges:ce_lst ~lsucc:new_cfg.ill ~uses:c.cal_uses
    ~altrets:c.cal_altrets ~unwinds_to:c.cal_unwinds_to
    ~cuts_to:c.cal_cuts_to ~reads:c.cal_reads ~writes:c.cal_writes
    ~spans:c.cal_spans
  | Cut c ->
    let cuts_to = Array.to_list (convert_contedges c.cut_contedges) in
    cut_to new_cfg (transform c.cut_i) ~cuts_to ~aborts:false ~uses:c.cut_uses
  | Joi j ->
    let join = lookup node in
    (set_succ join succ; join)
  | Exi e ->
    let exit = mk_exit new_cfg (labels cfg.exit) ~preds:[]
    ~lpred:new_cfg.ill in
    (new_cfg.exit <- exit; exit)
  | Ent e ->
    let ent = mk_entry succ in
    (new_cfg.entry <- ent; ent)
  | Boot -> undef "copy" "boot" in
let () = match postorder_dfs convert new_cfg.ill cfg with
  | Ent _ -> ()
  | _ -> imposs "postorder_dfs didn't end with entry node" in
new_cfg
*)

```

(node constructors with interesting control flow edges 361) (346c)

```
let join_leading_to_ce ce = {ce with node = join_leading_to ce.node}
```

```

let mk_call i ~pred ~contedges ~lsucc ~uses ~altrets ~unwinds_to ~cuts_to
~reads ~writes ~spans =
  Cal { cal_num      = new_num ()
      ; cal_i        = i
      ; cal_pred     = pred
      ; cal_contedges = contedges
      ; cal_lsucc    = lsucc
      ; cal_fx       = X.fx ()
      ; cal_spans    = spans

```

```

    ; cal_uses      = uses
    ; cal_altrrets  = altrrets
    ; cal_unwinds_to = unwinds_to
    ; cal_cuts_to    = cuts_to
    ; cal_reads     = reads
    ; cal_writes     = writes
  }

let call cfg i ~altrrets ~succ ~unwinds_to ~cuts_to ~aborts
    ~uses ~defs ~kills ~reads ~writes ~spans =
  let new_cedge n = {kills = kills; defs = defs; node = n} in
  let succ = match kind succ with
    | Exit -> let label = Idgen.label "postcall" in
      let join = node_labeled cfg label in
      set_succ cfg join ~succ;
      join
    | _ -> join_leading_to ~succ in
  let succ      = [new_cedge succ] in
  let altrrets  = List.map join_leading_to_ce altrrets in
  let unwinds_to = List.map join_leading_to_ce unwinds_to in
  let cuts_to    = List.map join_leading_to_ce cuts_to in
  let abort      = if aborts then [new_cedge cfg.exit] else [] in
  let edgelist   = List.flatten [succ; altrrets; unwinds_to; cuts_to; abort] in
  (*let edgelist = List.flatten [altrrets; succ; unwinds_to; cuts_to; abort] in*)
  let illedge    = fun {kills=k; defs=d; node=n} -> {kills=k; defs=d; node=cfg.ill} in
  let contedges  = Array.of_list (List.map illedge edgelist) in
  let cal =
    mk_call i ~pred:cfg.ill ~contedges ~lsucc:cfg.ill ~uses ~reads ~writes
      ~altrrets:(List.length altrrets) ~unwinds_to:(List.length unwinds_to)
      ~cuts_to:(List.length cuts_to) ~spans in
  let () = Auxfuns.foldri (fun i n () -> set_succ_n cfg cal i ~succ:n.node)
    edgelist () in
  cal

let cut_to cfg i ~cuts_to ~aborts ~uses =
  let cuts_to    = List.map join_leading_to_ce cuts_to in
  let new_cedge n = {kills = RS.empty; defs = RS.empty; node = n} in
  let aborts     = if aborts then [new_cedge cfg.exit] else [] in
  (* UNKNOWN WHETHER EDGE TO EXIT MUST KILL NVR'S. LET US HOPE NOT. *)
  let edgelist   = List.flatten [cuts_to; aborts] in
  let illedge    = fun {kills=k; defs=d; node=n} -> {kills=k; defs=d; node=cfg.ill} in
  let cut = Cut { cut_num      = new_num ()
    ; cut_i        = i
    ; cut_pred     = cfg.ill
    ; cut_lsucc    = cfg.ill
    ; cut_contedges = Array.of_list (List.map illedge edgelist)
    ; cut_fx       = X.fx ()
    ; cut_uses     = uses
  } in
  let () = Auxfuns.foldri (fun i n () -> set_succ_n cfg cut i ~succ:n.node) edgelist () in
  cut

⟨simple node constructors 362⟩≡ (346c)
let instruction cfg i ~succ =
  let ins = Ins { ins_num = new_num ()
    ; ins_i    = i
    ; ins_pred = cfg.ill
    ; ins_succ = cfg.ill
    ; ins_nx   = X.nx ()
  } in

```

```

let () = set_succ cfg ins ~succ:succ in
ins

let stack_adjust cfg i ~succ =
  let sta = Sta { sta_num = new_num ()
                  ; sta_i   = i
                  ; sta_pred = cfg.ill
                  ; sta_succ = cfg.ill
                  ; sta_nx   = X.nx ()
                  } in
  let () = set_succ cfg sta ~succ:succ in
  sta

let assertion cfg i ~succ =
  let ass = Ass { ass_num = new_num ()
                  ; ass_i   = i
                  ; ass_pred = cfg.ill
                  ; ass_succ = cfg.ill
                  ; ass_nx   = X.nx ()
                  } in
  let () = set_succ cfg ass ~succ:succ in
  ass

let branch cfg ~target =
  let succ = join_leading_to ~succ:target in
  let bra = Bra { bra_num = new_num ()
                  ; bra_i   = cfg.inst_info.goto.Ep.embed (label succ)
                  ; bra_pred = cfg.ill
                  ; bra_succ = cfg.ill
                  ; bra_lsucc = succ
                  ; bra_nx   = X.nx ()
                  } in
  let () = set_succ cfg bra ~succ in
  bra

(* SHOULD WE ASSERT THAT THESE TARGETS ARE JOIN POINTS? *)
let jump cfg i ~uses ~targets =
  let jum = Jum { jum_num = new_num ()
                  ; jum_i   = i
                  ; jum_pred = cfg.ill
                  ; jum_succ = cfg.ill
                  ; jum_lsucc = cfg.ill
                  ; jum_nx   = X.nx ()
                  ; jum_uses = uses
                  } in
  let () = set_succ cfg jum ~succ:cfg.exit in
  jum

let cbranch cfg cond ~ifso ~ifnot =
  let so_succ = join_leading_to ~succ:ifso in
  let not_succ = join_leading_to ~succ:ifnot in
  let cbr = Cbr { cbr_num = new_num ()
                  ; cbr_i   = cfg.inst_info.branch.Ep.embed
                        (cond, (label so_succ))
                  ; cbr_pred = cfg.ill
                  ; cbr_true = cfg.ill
                  ; cbr_false = cfg.ill
                  ; cbr_fx   = X.fx ()
                  } in
  let () = set_tsucc cfg cbr ~succ:so_succ in

```

```

let () = set_succ cfg cbr ~succ:not_succ in
cbr

let mbranch cfg i ~targets =
  let targets = List.map (fun succ -> join_leading_to ~succ) targets in
  let succs = Array.make (List.length targets) cfg.ill in
  let mbr = Mbr { mbr_num = new_num ()
                  ; mbr_i = i
                  ; mbr_pred = cfg.ill
                  ; mbr_succs = succs
                  ; mbr_lsucc = cfg.ill
                  ; mbr_fx = X.fx ()
                } in
  let () = Auxfuns.foldr1 (fun i n () -> set_succ_n cfg mbr i ~succ:n) targets () in
  mbr

let return cfg i ~uses =
  let ret = Ret { ret_num = new_num ()
                 ; ret_i = i
                 ; ret_pred = cfg.ill
                 ; ret_succ = cfg.ill
                 ; ret_lsucc = cfg.ill
                 ; ret_nx = X.nx ()
                 ; ret_uses = uses
               } in
  let () = set_succ cfg ret ~succ:cfg.exit in
  ret

let impossible cfg ~succ =
  let succ = join_leading_to succ in
  let imp = Imp { imp_num = new_num ()
                 ; imp_pred = cfg.ill
                 ; imp_succ = cfg.ill
                 ; imp_exit_succ = cfg.ill
                 ; imp_fx = X.fx ()
               } in
  let () = set_succ cfg imp ~succ:succ in
  let () = set_succ_n cfg imp 1 ~succ:cfg.exit in
  imp

let illegal cfg = Ill { ill_num = new_num ()
                       ; ill_nx = X.nx ()
                       ; ill_cfg = cfg
                     }

```

<graph utilities 364a>≡ (346c) 364b▷

```

let imposs = Impossible.impossible
let undef f n = imposs (Printf.sprintf "%s: undefined on %s nodes" f n)
let undef_with_node node_num f n =
  imposs (Printf.sprintf "%s: undefined on %s node %d" f n node_num)

```

<graph utilities 364b>+≡ (346c) <364a 365a>

```

let kind_util = function
| Joi _ -> Join
| Ins _ -> Instruction
| Sta _ -> StackAdjust
| Bra _ -> Branch
| Cbr _ -> Cbranch
| Mbr _ -> Mbranch
| Jum _ -> Jump

```

```

| Cal _ -> Call
| Ret _ -> Return
| Cut _ -> CutTo
| Ent _ -> Entry
| Exi _ -> Exit
| Ass _ -> Assertion
| Ill _ -> Illegal
| Imp _ -> Impossible
| Boot -> undef "kind" "boot"

```

```
let num_util node = match node with
```

```

| Joi j -> j.joi_num
| Ins i -> i.ins_num
| Sta s -> s.sta_num
| Bra b -> b.bra_num
| Cbr c -> c.cbr_num
| Mbr m -> m.mbr_num
| Jum j -> j.jum_num
| Cal c -> c.cal_num
| Ret r -> r.ret_num
| Cut c -> c.cut_num
| Ent e -> e.ent_num
| Exi e -> e.exi_num
| Ass a -> a.ass_num
| Ill i -> i.ill_num
| Imp i -> i.imp_num
| Boot -> undef "num" "boot"

```

graph utilities 365a) +≡

(346c) <364b 365b>

```
let name = function
```

```

| Joi _ -> "Join"
| Ins _ -> "Instruction"
| Sta _ -> "StackAdjust"
| Bra _ -> "Branch"
| Cbr _ -> "Cbranch"
| Mbr _ -> "Mbranch"
| Jum _ -> "Jump"
| Cal _ -> "Call"
| Ret _ -> "Return"
| Cut _ -> "CutTo"
| Ent _ -> "Entry"
| Exi _ -> "Exit"
| Ass _ -> "Assertion"
| Ill _ -> "Illegal"
| Imp _ -> "Impossible"
| Boot -> "Boot"

```

graph utilities 365b) +≡

(346c) <365a 366a>

```
let eq node node' = match node, node' with
```

```

| Joi r, Joi r' -> r == r'
| Ins r, Ins r' -> r == r'
| Sta r, Sta r' -> r == r'
| Bra r, Bra r' -> r == r'
| Cbr r, Cbr r' -> r == r'
| Mbr r, Mbr r' -> r == r'
| Jum r, Jum r' -> r == r'
| Cal r, Cal r' -> r == r'
| Ret r, Ret r' -> r == r'
| Cut r, Cut r' -> r == r'
| Ent r, Ent r' -> r == r'

```

```

| Exi r, Exi r' -> r == r'
| Ass r, Ass r' -> r == r'
| Ill r, Ill r' -> r == r'
| Imp r, Imp r' -> r == r'
| Boot, Boot   -> true
| _,          -> false

```

<graph utilities 366a> $\equiv$ (346c) *<365b 366c>*

```

let node_num = ref 0
let new_num () = let n = !node_num in node_num := n + 1; n

```

<printing utilities 366b> $\equiv$ (346c)

```

let show_rtl = function
  Some r -> Rtlutil.ToString.rtl r
| None   -> ""

let printReg (s, i, w) = Printf.sprintf "%c%d" s i
let print_node node =
  String.concat "" (List.flatten
    [ [ name node; ": " ]
    ; List.flatten [ [ string_of_int (num node)
                    ; ": " ]
                  ]
    ; if is_join node then labels node
      else [show_rtl (to_instr node)]
    ; [ "\n" ] ]
  ; [" Preds: "]
  ; List.fold_right (fun n rst -> Printf.sprintf "%d, " (num n) :: rst)
                    (preds node) []
  ; ["Succs: "]
  ; List.fold_right (fun n rst -> Printf.sprintf "%d, " (num n) :: rst)
                    (succs node) []
  ])

```

<graph utilities 366c> $\equiv$ (346c) *<366a 367a>*

```

let log m = ()
let uniq lst = match lst with
| [] | [_] -> lst (* premature optimization? *)
| _ -> fst (List.fold_left (fun (ns,set as rst) n1 ->
  let i = num_util n1 in
  if IS.mem i set then rst else (n1::ns, IS.add i set))
([],IS.empty) lst)

```

```

let is_ill node = match node with
| Ill _ -> true
| _      -> false

```

```

let is_exi node = match node with
| Exi _ -> true
| _      -> false

```

```

let ce_getnodes ce = Array.map (fun ce -> ce.node) ce

```

<UNDEFINED graph utilities 366d> $\equiv$

```

(*
let merge_joins j1 j2 succ lpred = match j1, j2 with
| Joi j1, Joi j2 ->
{ joi_num      = new_num ()
; joi_labels = j1.joi_labels @ j2.joi_labels
; joi_cfg      = j1.joi_cfg

```

```

    ; joi_preds  = j1.joi_preds  @ j2.joi_preds
    ; joi_succ   = j2.joi_succ
    ; joi_lpred  = j2.joi_lpred
    ; joi_jx     = j2.joi_jx
  }
  | _           -> undef "merge_joins" "non-join"
*)

```

```

⟨graph utilities 367a⟩+≡ (346c) <366c
  let heads cfg =
    List.filter (fun n -> Pervasives.<> (kind_util n) Exit)
      (uniq (cfg.entry :: SM.fold (fun _ h rst -> h::rst) cfg.label_map []))

```

```

⟨join node set operations 367b⟩≡ (353c)
  let emptyset      = IS.empty
  let add    n set  = IS.add (num n) set
  let member n set  = IS.mem (num n) set

```

N.8 front_cfg/cfgx.nw

```

⟨cfgx.mli 367c⟩≡
  module X : Cfg.X
  module M : Cfg.S with module X = X

```

```

⟨cfgx.ml 367d⟩≡
  module X = struct
    type jx = unit
    type fx = unit
    type nx = unit
    let jx () = ()
    let fx () = ()
    let nx () = ()
  end
  module M = Cfg.Make(X)

```

N.9 cfg/legacy/dag.nw

N.10 Directed Acyclic Graph

N.10.1 Interface

```

⟨abstract types for dags 367e⟩≡ (368b)
  type uid

```

```

⟨types for dags 367f⟩≡ (368)
  type 'a block = Rtl    of Rtl.rtl
                  | Seq   of 'a block * 'a block
                  | If     of 'a cbranch * 'a block * 'a block
                  | While of 'a cbranch * 'a block
                  | Nop
  (* block ending in branch, including multiway branch *)
  and 'a branch  = 'a block * Rtl.rtl
  and 'a cbranch = Exit    of bool
                  | Test   of 'a block * 'a cbinst
                  | Shared of uid * 'a cbranch (* don't duplicate this node *)
  and 'a cbinst  = 'a * 'a cbranch * 'a cbranch

```



```

<exported utility functions for dags 368a>≡ (368b) 368c▷
  val (<:>) : 'a block -> 'a block -> 'a block
  val pr_block : ('a -> string) -> 'a block -> string

<dag.mli 368b>≡
  <abstract types for dags 367e>
  <types for dags 367f>
  <exported utility functions for dags 368a>

<exported utility functions for dags 368c>+≡ (368b) <368a 368d>
  val shared : 'a cbranch -> 'a cbranch

<exported utility functions for dags 368d>+≡ (368b) <368c 368f>
  val cond : 'a -> 'a cbranch

```

N.10.2 Implementation

```

<implementation of utility functions for dags 368e>≡ (368j) 368g▷
  let (<:>) b b' = match b, b' with
  | Nop, b' -> b'
  | b, Nop -> b
  | _, _ -> Seq (b, b')

<exported utility functions for dags 368f>+≡ (368b) <368d
  type 'a nodeset
  val empty : 'a nodeset
  val lookup : uid -> 'a nodeset -> 'a (* raises Not_found *)
  val insert : uid -> 'a -> 'a nodeset -> 'a nodeset

<implementation of utility functions for dags 368g>+≡ (368j) <368e 368h>
  type 'a nodeset = (uid * 'a) list
  let empty = []
  let lookup = List.assoc
  let insert i x l = (i, x) :: l

<implementation of utility functions for dags 368h>+≡ (368j) <368g 368i>
  module Shared = struct
    let n = Reinit.ref 0
    let shared c = match c with
    | Shared _ -> c
    | _ -> (n := !n + 1; Shared (!n, c))
  end
  let shared = Shared.shared

<implementation of utility functions for dags 368i>+≡ (368j) <368h
  let cond g = Test (Nop, (g, Exit true, Exit false))

<dag.ml 368j>≡
  module RU = Rtlutil
  module TS = RU.ToString
  type uid = int
  <types for dags 367f>
  <implementation of utility functions for dags 368e>

  let sprintf = Printf.sprintf

  let rec pr_block pr = function
  | Rtl r -> TS.rtl r
  | Seq (b1, b2) -> pr_block pr b1 ^ pr_block pr b2

```

```

| If (c, t, f) -> sprintf "if (%s) { %s; } else { %s; }"
                    (pr_cbranch pr c) (pr_block pr t) (pr_block pr f)
| While (c, b) -> sprintf "while (%s) { %s; }" (pr_cbranch pr c) (pr_block pr b)
| Nop -> "<nop>"
and pr_cbranch pr c = match c with
| Exit p          -> if p then "true" else "false"
| Shared (_, c) -> sprintf "[%s]" (pr_cbranch pr c)
| Test (Nop, c) -> pr_cbi pr c
| Test (b, c)    -> sprintf "{%s; %s}" (pr_block pr b) (pr_cbi pr c)
and pr_cbi pr c = match c with
| a, Exit true,  Exit false -> pr a
| a, Exit false, Exit true  -> sprintf "!(%s)" (pr a)
| a, Exit true,  p -> sprintf "(%s || %s)" (pr a) (pr_cbranch pr p)
| a, p, Exit false -> sprintf "(%s && %s)" (pr a) (pr_cbranch pr p)
| a, p, q -> sprintf "(%s ? %s : %s)" (pr a) (pr_cbranch pr p) (pr_cbranch pr q)

```

N.11 cfg/legacy/ep.nw

N.12 Embedding-projection pairs

```

⟨ep.mli 369a⟩≡
type ('em, 'pr) pre_map =
  { embed   : 'em
    ; project : 'pr
  }
type ('lo, 'hi) map = ('lo -> 'hi, 'hi -> 'lo) pre_map

⟨ep.ml 369b⟩≡
type ('em, 'pr) pre_map =
  { embed   : 'em
    ; project : 'pr
  }
type ('lo, 'hi) map = ('lo -> 'hi, 'hi -> 'lo) pre_map

```

N.13 front_cfg/mflow.nw

```

⟨front_cfg/mflow.mli 369c⟩≡
  ⟨mflow.mli 369d⟩

```

N.14 Machine-level control flow

```

⟨mflow.mli 369d⟩≡ (369c) 370b▷
  ⟨signatures 369e⟩

⟨signatures 369e⟩≡ (370c 369d) 370a▷
type ('em, 'pr) map' = ('em, 'pr) Ep.pre_map =
  { embed   : 'em
    ; project : 'pr
  }
type brtl = Rtl.exp -> Rtl.rtl
type ('a, 'b) map = ('a -> 'b -> brtl Dag.branch, Rtl.rtl -> 'b) map'
type ('a, 'b) mapc = ('a -> 'b -> brtl Dag.cbranch, Rtl.rtl -> 'b) map'

type cut_args = { new_sp : Rtl.exp; new_pc : Rtl.exp }

```

```

type 'a machine =
{ bnegate:    Rtl.rtl -> Rtl.rtl
; goto:      ('a, Rtl.exp) map
; jump:      ('a, Rtl.exp) map
; call:      ('a, Rtl.exp) map
; branch:    ('a, Rtl.exp) mapc (* condition *)
; retgt_br:  Rtl.rtl -> brtl Dag.cbranch
; move:      'a -> src:Register.t -> dst:Register.t -> brtl Dag.block
; spill:     'a -> Register.t      -> Rtlutil.aloc    -> brtl Dag.block
; reload:    'a -> Rtlutil.aloc    -> Register.t     -> brtl Dag.block
; cutto:     ('a, cut_args) map    (* newpc * newsp map*)
; return:    Rtl.rtl
; forbidden: Rtl.rtl    (* causes a run-time error *)
}

⟨signatures 370a⟩+≡ (370c 369d) <369e
module type PC = sig
  val pc_lhs : Rtl.loc
  val pc_rhs : Rtl.loc
  val ra_reg  : Rtl.loc
  val ra_offset : int    (* at call, ra_reg  := PC + ra_offset *)
end

module type S = sig
  val machine : sp:Rtl.loc -> 'a machine
  (* N.B. sp is a dynamic argument because it could differ among call conventions *)
end

⟨mflow.mli 370b⟩+≡ (369c) <369d
module MakeStandard (Pc : PC) : S

⟨mflow.ml 370c⟩≡
  ⟨signatures 369e⟩
  module DG = Dag
  module RU = Rtlutil
  module R  = Rtl
  module RP = Rtl.Private
  module Up  = Rtl.Up
  module Down = Rtl.Dn
  let (=/ ) = RU.Eq.loc
  let (="//") = RU.Eq.exp

  module MakeStandard (P : PC) = struct
    type 'a map = ('a, Rtl.rtl) Ep.map
    ⟨standard machine-level control flow 370d⟩
  end

  ⟨standard machine-level control flow 370d⟩≡ (370c) 370e▷
  let w = RU.Width.loc P.pc_lhs
  let downrtl = R.Dn.rtl
  let uploc   = R.Up.loc
  let upexp   = R.Up.exp

  ⟨standard machine-level control flow 370e⟩+≡ (370c) <370d 371a▷
  let cmpneg w ~cop ~fetch =
    let flip_op flip = RP.App ((flip, [w]), fetch) in
    match cop with
    | "eq"  -> flip_op "ne"
    | "ne"  -> flip_op "eq"
    | "lt"  -> flip_op "ge"

```

```

| "le"    -> flip_op "gt"
| "gt"    -> flip_op "le"
| "ge"    -> flip_op "lt"
| "ltu"   -> flip_op "geu"
| "leu"   -> flip_op "gtu"
| "gtu"   -> flip_op "leu"
| "geu"   -> flip_op "ltu"
| "feq"   -> flip_op "fne"
| "fne"   -> flip_op "feq"
| "flt"   -> flip_op "fge"
| "fle"   -> flip_op "fgt"
| "fgt"   -> flip_op "fle"
| "fge"   -> flip_op "flt"
| "not"   -> (match fetch with
    | [r] -> r
    | _ -> Impossible.impossible "negation of multiple arguments")
| _       -> RP.App (("not", [w]), fetch)
let bnegate r = match Down.rtl r with
| RP.Rtl [RP.App((cop, [w1]), [RP.Fetch (flags, w2)]), RP.Store (pc, tgt, w3)]
  when pc =/ Down.loc P.pc_lhs && w1 = w && w2 = w && w3 = w ->
    Up.rtl (RP.Rtl [cmpneg w ~cop ~fetch:[RP.Fetch (flags, w)],
      RP.Store (pc, tgt, w)])
| _ -> Impossible.impossible "ill-formed conditional branch"

⟨standard machine-level control flow 371a⟩+≡ (370c) <370e 371b>
let gotor = { Ep.embed    = (fun e -> R.store P.pc_lhs e w)
    ; Ep.project = (fun r -> match downrtl r with
        | RP.Rtl [(_, RP.Store(_, e, _))] -> upexp e
        | _ -> Impossible.impossible "projected non-goto")
    }
let goto = { Ep.embed    = (fun _ e -> (DG.Nop, gotor.Ep.embed e))
    ; Ep.project = gotor.Ep.project
    }
let jump = goto

⟨standard machine-level control flow 371b⟩+≡ (370c) <371a 371c>
let cutto ~sp =
{ Ep.embed    = (fun _ {new_sp=new_sp; new_pc=new_pc} ->
    let assign loc e = Rtl.store loc e w in
    (DG.Nop, Rtl.par [assign P.pc_lhs new_pc; assign sp new_sp]))
; Ep.project = (fun r -> match downrtl r with
    | RP.Rtl [ (_, RP.Store(_, npc, _))
        ; (_, RP.Store(_ , nsp, _))] ->
        { new_sp=upexp nsp; new_pc= upexp npc }
    | _ -> Impossible.impossible "projected non-cutto")
}

⟨standard machine-level control flow 371c⟩+≡ (370c) <371b 372a>
let ra_val =
  let pc = R.fetch P.pc_rhs w in
  RU.addk w pc P.ra_offset

let call = { Ep.embed    = (fun _ e -> (DG.Nop, R.par [R.store P.pc_lhs e w;
    R.store P.ra_reg ra_val w]))
    ; Ep.project =
    (fun r -> match downrtl r with
        | RP.Rtl [(_, RP.Store(_, e, _)); _] -> upexp e
        | _ -> Impossible.impossible (Printf.sprintf "projected non-call: %s"
            (RU.ToString.rtl r)))
    }

```

```

<standard machine-level control flow 372a>+≡ (370c) <371c 372b>
  let return = R.store P.pc_lhs (R.fetch P.ra_reg w) w

<standard machine-level control flow 372b>+≡ (370c) <372a 372c>
  let branch =
    { Ep.embed = (fun _ cond ->
      DG.cond (fun tgt -> R.guard cond (gotor.Ep.embed tgt)))
    ; Ep.project = (fun r -> match downrtl r with
      | RP.Rtl [(b, RP.Store(_, _, _))] -> upexp b
      | _ -> Impossible.impossible "projected non-branch")
    }

<standard machine-level control flow 372c>+≡ (370c) <372b 372d>
  let forbidden = R.kill P.pc_lhs

<standard machine-level control flow 372d>+≡ (370c) <372c>
  let machine ~sp =
    let fail _ = assert false in
    { bnegate = bnegate; goto = goto; jump = jump; call = call; cutto = cutto sp;
      retgt_br = fail; spill = fail; reload = fail; move = fail;
      return = return; branch = branch; forbidden = forbidden }

```

N.15 front_cfg/spans.nw

```

<front_cfg/spans.ml 372e>≡
  <spans.ml 373b>

<front_cfg/spans.mli 372f>≡
  <spans.mli 372g>

```

N.16 Representing spans, including internal spans

```

<spans.mli 372g>≡ (372f) 373a>
  type t
  type label = string
  type link = Reloc.t
  type aligned = int
  val to_spans : inalloc:Rtl.loc -> outalloc:Rtl.loc -> ra:Rtl.loc ->
    users:((Bits.bits * link) list) ->
    csregs:((Register.t * Rtl.loc option) list) ->
    conts:((label * Rtl.loc * (string * int * aligned) list) list) ->
      (* (ra, sp, parms : (hint, var number, alignment) list) *)
    sds:(Rtl.loc list) ->
    vars:(Rtl.loc option array) ->
    t
  <regrettably exposed rep type 372h>
  val expose : t -> rep

<regrettably exposed rep type 372h>≡ (373b 372g)
  type rep =
    { mutable inalloc : Rtl.loc
    ; mutable outalloc : Rtl.loc
    ; mutable ra : Rtl.loc
    ; mutable users : (Bits.bits * link) list
    ; mutable csregs : (Register.t * Rtl.loc option) list
    ; mutable conts : (label * Rtl.loc * (string * int * aligned) list) list
    ; mutable sds : Rtl.loc list
    ; mutable vars : Rtl.loc option array
    }

```

```

⟨spans.mli 373a⟩+≡ (372f) ◁372g
  val fold_live_locs : (Register.x -> 'a -> 'a) -> t -> 'a -> 'a

⟨spans.ml 373b⟩≡ (372e) 373c▷
  type label = string
  type link = Reloc.t
  type aligned = int
  ⟨regrettably exposed rep type 372h⟩
  type t = rep
  let to_spans ~inalloc ~outalloc ~ra ~users ~csregs ~conts ~sds ~vars =
    { inalloc = inalloc; outalloc = outalloc; ra = ra; users = users; csregs = csregs;
      conts = conts; sds = sds; vars = vars; }
  let expose spans = spans

⟨spans.ml 373c⟩+≡ (372e) ◁373b
  let fold_live_locs f spans z =
    let fold = Rtlutil.Fold.RegX.loc f in
    let foldcsreg z = function (_, Some l) -> fold l z | (_, None) -> z in
    List.fold_left foldcsreg (fold spans.inalloc (fold spans.outalloc (fold spans.ra z)))
      spans.csregs

```

Appendix O

elab

O.1 elab/elabexp.nw

```
<front_nelab/elabexp.ml 374a>≡  
  <elabexp.ml 374e>
```

```
<front_nelab/elabexp.mli 374b>≡  
  <elabexp.mli 374c>
```

O.2 Elaborating Expressions

```
<elabexp.mli 374c>≡ (374b) 374d▷  
  type nm_or_mem = Ast.name_or_mem  
  type link = Reloc.t  
  val aligned : Metrics.t -> Rtl.width -> Ast.aligned option -> Ast.aligned  
                                          (* raises Error *)  
  val elab_ty : 'a Fenv.Dirty.env' -> Ast.ty -> Rtl.width -> Error.error  
  val elab_loc : 'a Fenv.Dirty.env' -> nm_or_mem -> (Rtl.loc * Rtl.width) Error.error  
  val elab_exp : 'a Fenv.Dirty.env' -> Ast.expr -> (Rtl.exp * Types.ty) Error.error  
  val elab_con : 'a Fenv.Dirty.env' -> Ast.expr -> (Bits.bits * Rtl.width) Error.error  
  val elab_link : 'a Fenv.Dirty.env' -> Ast.expr -> (link * Rtl.width) Error.error  
  val elab_kinded_name :  
    'a Fenv.Dirty.env' -> nm_or_mem -> (string * (Rtl.loc * Rtl.width) * int) Error.error
```

```
<elabexp.mli 374d>+≡ (374b) <374c  
  val loc_region : Ast.region -> nm_or_mem -> Ast.region  
  val exp_region : Ast.region -> Ast.expr -> Ast.region
```

O.2.1 Implementation

```
<elabexp.ml 374e>≡ (374a) 375a▷  
  open Nopoly  
  
  module A = Ast  
  module E = Error  
  module F = Fenv.Dirty  
  module FE = Fenv  
  type nm_or_mem = Ast.name_or_mem  
  type link = Reloc.t  
  
  let impossf fmt = Printf.kprintf Impossible.impossible fmt
```

```
let (@<<) f g = fun x -> f (g x) (* function composition *)
```

```
let is2power x = x > 0 && x land (x - 1) = 0
```

O.2.2 The translation of types, names, and expressions

```

<elabexp.ml 375a>+≡ (374a) <374e
  module M = Metrics
  <utilities(elabexp.nw) 375b>
  let loc_region r l = match l with A.NM(_, r) -> r | _ -> r
  let exp_region r l = match l with A.E      (_, r) -> r | _ -> r

  let aligned metrics =
    let wordsize = metrics.M.wordsize in
    let wordmult = Cell.divides (Cell.of_size wordsize) in
    let aligned w = function
      | Some a -> alignment a
      | None   -> if wordmult w then w / wordsize else 1 in
    aligned

  let exprfunfs env =
    let metrics      = F.metrics env in
    let mcell        = Cell.of_size metrics.M.memsize in
    let mspace       = ('m', metrics.M.byteorder, mcell) in
    let eprint r     = E.errorRegionPrt (F.srcmap env, r) in
    let errorf fmt   = Printf.kprintf E.error fmt in
    let catch r f x = E.catch (eprint r) f x in
    let aligned      = aligned metrics in
    <definition of lvalue_name 376a>
    <definition of rvalue_name 376b>
    <mutually recursive nest of typed_expr and lvalue 376c>
    in
    let constant simp e = E.ematch (typed_expr e) (fun (k, kt) ->
      match kt with
      | Types.Bits w -> simp k, w
      | _ -> E.error "constant expression may not be a Boolean") in
    let caught r f x = catch (r x) f x in
    let l, e = loc_region Srcmap.null, exp_region Srcmap.null in
    caught e typed_expr, caught l lvalue, caught l kinded_name,
    caught e (constant Simplify.bits), caught e (constant Simplify.link)

  let elab_exp      env = let f, _, _, _, _ = exprfunfs env in f
  let elab_loc      env = let _, f, _, _, _ = exprfunfs env in f
  let elab_kinded_name env = let _, _, f, _, _ = exprfunfs env in f
  let elab_con      env = let _, _, _, f, _ = exprfunfs env in f
  let elab_link     env = let _, _, _, _, f = exprfunfs env in f

<utilities(elabexp.nw) 375b>≡ (375a) 376d>
  let tywidth a_bad_thing = function
    | Types.Bits n -> n
    | Types.Bool   -> E.error (Printf.sprintf "A boolean may not be %s" a_bad_thing)

  let emap f x = E.ematch x f

  let elab_full_ty env =
    let catch r = E.catch (E.errorRegionPrt (F.srcmap env, r)) in
    let rec elab = function
      | A.T (x,r)   -> catch r elab x
      | A.BitsTy size -> E.Ok (Types.Bits size)

```



```
| A.TypeSynonym n -> snd (F.findt n env) in
catch Srcmap.null elab
```

```
(* pad: was << but I changed it to @<<, typo in .nw file ??? *)
let astwidth msg env = emap (tywidth msg) @<< elab_full_ty env
let elab_ty env = astwidth "expressed in abstract syntax" env
```

```
<definition of lvalue_name 376a>≡ (375a)
let lvalue_name x =
  E.ematch (snd (F.findv x env))
  (fun (den,t) ->
    let w = tywidth "assigned to" t in
    match den with
    | FE.Variable {FE.loc=l} -> l, w
    | _ -> errorf "may not assign to %s %s" (FE.denotation's_category den) x) in
```

```
<definition of rvalue_name 376b>≡ (375a)
let rvalue_name x =
  E.seq (snd (F.findv x env))
  (fun (denot, t) ->
    let w = tywidth "named variable" t in
    let rval = match denot with
    | FE.Constant bits -> E.Ok (Rtl.bits bits w)
    | FE.Label (FE.Proc s) -> E.Ok (Rtl.codesym s w)
    | FE.Label (FE.Code s) -> E.Ok (Rtl.codesym s w)
    | FE.Label (FE.Data s) -> E.Ok (Rtl.datasym s w)
    | FE.Import(_,s) -> E.Ok (Rtl.imp sym s w)
    | FE.Continuation c -> (c.FE.escapes <- true; E.Ok (Block.base c.FE.base))
    | FE.Label(FE.Stack addr) -> E.Ok addr (* a fetch would be wrong *)
    | FE.Variable _ -> emap (fun (l,w) -> Rtl.fetch l w) (lvalue_name x) in
    E.Raise.left (rval, t)) in
```

```
<mutually recursive nest of typed_expr and lvalue 376c>≡ (375a) 377b▷
let rec kind_name lhs = match lhs with
| A.NM(lhs,r) -> catch r kind_name lhs
| A.Name(kind,x,a) -> E.ematch (lvalue_name x) (fun (loc, w) ->
  (Auxfuns.Option.get "" kind, (loc, w), aligned w a))
| A.Mem _ -> E.error "only a name may appear to left of a call" in
let rec lvalue lhs = match lhs with
| A.NM(lhs,r) -> catch r lvalue lhs
| A.Name(k,x,a) -> no_kind_or_alignment k a lvalue_name x
| A.Mem(t,addr,aligned,aliasing) ->
  let w = astwidth "value in memory" env t in
  E.ematch2 w (typed_expr addr) (fun w (addre, addrt) ->
    if tywidth "address" addrt <> metrics.M.pointersize then
      E.error "address's type is not the machine's pointer size"
    else if not (Cell.divides mcell w) then
      E.error "reference's type is not a multiple of target memsize"
    else
      Rtl.mem (assertion aligned) mspace (Cell.to_count mcell w) addre, w)
and rvalue_name_or_mem nm = match nm with
| A.NM(v, r) -> catch r rvalue_name_or_mem v
| A.Name(k, x, a) -> no_kind_or_alignment k a rvalue_name x
| A.Mem(_, _, _, _) ->
  E.ematch (lvalue nm) (fun (loc, w) -> Rtl.fetch loc w, Types.Bits w)
```

```
<utilities(elabexp.nw) 376d>+≡ (375a) <375b 377a▷
let no_kind_or_alignment k a f x = match k, a with
| None, None -> f x
| Some _, None -> E.error "kind permissible only on parameter or result"
```

```

| None, Some _ -> E.errorf "alignment permissible only on parameter or result"
| Some _ ,Some _ ->
    E.errorf "kind and alignment permissible only on parameter or result"

<utilities(elabexp.nw) 377a>+≡ (375a) <376d
let alignment n =
  if is2power n then n else E.errorf "alignment %d is not a power of 2" n
let assertion = function
  | None -> Rtl.none
  | Some n -> Rtl.aligned (alignment n)

<mutually recursive nest of typed_expr and lvalue 377b>+≡ (375a) <376c
and typed_expr exp =
  let apply tx op args =
    E.ematch (E.Raise.list (List.map typed_expr args))
    (fun args ->
      let arges, argtys = List.split args in
      let resty, opr = tx op argtys in
      Rtl.app opr arges, resty) in
  let literal cvt v width = Rtl.bits (cvt v width) width, Types.Bits width in
  let literal cvt v = emap (literal cvt v) in
  let signed, unsigned, float =
    let does_not_overflow f v w =
      try (ignore (f v w); true) with Bits.Overflow -> false in
    let cvt signed what cvt v width =
      try cvt v width
      with Bits.Overflow ->
        if signed && not (v.[0] <= '-') && does_not_overflow Bits.U.of_string v width
        then
          E.errorf "literal %s does not fit in %d signed bits, but it will\n\
            fit in %d unsigned bits---perhaps you want '%sU'?"
            v width width v
        else
          E.errorf "%s overflow in literal %s" what v in
    cvt true "signed-integer" Bits.S.of_string,
    cvt false "unsigned-integer" Bits.U.of_string,
    cvt false "floating-point" Bits.U.of_string in (* surprising but true *)
  let const default_width = function
    | Some ty -> astwidth "literal constant" env ty
    | None -> E.Ok default_width in
  match exp with
  | A.E(x,r) -> catch r typed_expr x
  | A.Sint(str,t) -> literal signed str (const metrics.M.wordsize t)
  | A.Uint(str,t) -> literal unsigned str (const metrics.M.wordsize t)
  | A.Float(str,t) -> literal float str (const metrics.M.wordsize t)
  | A.Char(int,t) -> literal Bits.U.of_int int (const 8 t)
  | A.Fetch(v) -> rvalue_name_or_mem v
  | A.BinOp (l,op,r) -> apply Rtlop.Translate.binary op [l;r]
  | A.UnOp (op,e) -> apply Rtlop.Translate.unary op [e]
  | A.PrimOp(op,args) -> let args = List.map (fun (k, x, a) -> x) args in
    apply Rtlop.Translate.prefix op args

```

O.3 front_nelab/elabstmt.nw

```

<front_nelab/elabstmt.ml 377c>≡
  <elabstmt.ml 379a>

<front_nelab/elabstmt.mli 377d>≡
  <elabstmt.mli 378c>

```

O.4 Elaborating Statements

```
(exposed types(elabstmt.nw) 378a)≡ (379a 378c) 378b▷  
type exp = Rtl.exp  
[@@deriving show]  
type loc = Rtl.loc * Rtl.width  
[@@deriving show]  
type rtl = Rtl.rtl  
[@@deriving show]  
  
type name = string  
[@@deriving show]  
type kind = string  
[@@deriving show]  
type convention = string  
[@@deriving show]  
type aligned = int  
[@@deriving show]  
  
(exposed types(elabstmt.nw) 378b)+≡ (379a 378c) <378a  
type actual = kind * exp * Rtl.width * aligned  
[@@deriving show]  
type 'a kinded = kind * 'a * aligned  
[@@deriving show]  
type 'a flow = { cuts : 'a list; unwinds : 'a list; areturns : 'a list;  
                returns : bool; aborts : bool }  
[@@deriving show { with_path = false }]  
type 'a cflow = { ccuts : 'a list; caborts : bool }  
[@@deriving show { with_path = false }]  
type 'a alias = { reads : 'a; writes : 'a }  
[@@deriving show { with_path = false }]  
type range = Bits.bits * Bits.bits (* lo 'leu' x 'leu' hi, as in manual *)  
[@@deriving show]  
type procname = string  
[@@deriving show]  
type label = string  
[@@deriving show]  
type linktime = Reloc.t  
[@@deriving show]  
  
type stmt =  
| If of exp * stmt list * stmt list  
| Switch of range option * exp * (range list * stmt list) list  
| Label of label  
| Cont of name * convention * Fenv.variable kinded list  
| Span of (Bits.bits * linktime) * stmt list  
| Assign of rtl  
| Call of loc kinded list * convention * exp * actual list * procname list  
      * name flow * name list option alias  
| Call' of convention * exp * actual list * procname list  
      (* the dog ate the annotations *)  
| Goto of exp * label list  
| Jump of convention * exp * actual list * procname list  
| Cut of convention * exp * actual list * name cflow  
| Return of convention * int * int * actual list  
| Limitcheck of convention * exp * limitfailure option  
and limitfailure = { failcont : exp; reccont : exp; recname : name; }  
[@@deriving show { with_path = false }]  
  
(elabstmt.mli 378c)≡ (377d)
```

```

⟨exposed types(elabstmt.nw) 378a⟩
val elab_stmts :
  (Rtl.rtl -> string option) -> Srcmap.map -> Ast.region -> 'a Fenv.Dirty.env' ->
  Nast.stmt list -> stmt list Error.error
val elab_cformals :
  Ast.region -> 'a Fenv.Dirty.env' -> Nast.cformal list ->
  Fenv.variable kinded list Error.error
val codelabels : stmt list -> label list

```

O.4.1 Implementation

```

⟨elabstmt.ml 379a⟩≡ (377c) 379b▷
⟨exposed types(elabstmt.nw) 378a⟩

```

```

module A = Ast
module E = Error
module F = Fenv.Dirty
module FE = Fenv
module N = Nast
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let is2power x = x > 0 && x land (x - 1) = 0

```

```

⟨elabstmt.ml 379b⟩+≡ (377c) <379a 379c>
  module M = Metrics

```

```

⟨elabstmt.ml 379c⟩+≡ (377c) <379b 386c>
  let elab_functions validate srcmap r env =
    let eprint r = E.errorRegionPrt (F.srcmap env, r) in
    let errorf r = Printf.kprintf (fun s -> eprint r s; E.Error) in
    let findve r n = E.catch (eprint r) (fun env -> snd (F.findv n env)) in
    let aligned = Elabexp.aligned (F.metrics env) in
    let aligned r w = E.catch (eprint r) (fun a -> E.Ok (aligned w a)) in
    let exp = Elabexp.elab_exp env in
    let con = Elabexp.elab_con env in
    let loc = Elabexp.elab_loc env in
    let metrics = F.metrics env in
    ⟨elaboration of continuation formal 381a⟩
    let rec full_stmt r reachable s =
      ⟨elaboration utilities 380a⟩
      ⟨definitions of elaboration functions(elabstmt.nw) 381b⟩
      let rec stmt s = match s with
      | N.S (s, r) -> E.catch (eprint r) (full_stmt r reachable) s
      | N.If (c, t, f) ->
          let c = exp c in let t = stmts r true t in let f = stmts r true f in (*order*)
          E.ematch3 c t f (fun (c, cty) t f ->
            match cty with
            | Types.Bool -> If (c, t, f)
            | Types.Bits n -> E.error "condition in 'if' is not a Boolean")
      | N.Switch (range, e, arms) -> switch range e arms
      | N.Label n -> E.Ok (Label n)
      | N.Cont (name, cc, formals) ->
          let c = E.ematch (cformals formals) (fun formals -> Cont (name,cc,formals)) in
          if reachable then
            E.error "control can fall through to continuation"
          else
            c
      | N.Span (k, v, ss) -> span k v (stmts r reachable ss)
      | N.Assign (lhs, rhs) -> assign lhs rhs

```

```

| N.Goto (tgt, tgts) -> goto tgt tgts
| N.Call (lhs, conv, p, args, tgts, f, a) -> call lhs conv p args tgts f a
| N.Jump (conv, p, args, tgts) -> jump conv p args tgts
| N.Cut (conv, k, args, flow) -> cut conv k args flow
| N.Prim (lhs, conv, p, args, flow) -> prim lhs conv p args flow
| N.Return (conv, altcont, args) -> return conv altcont args
| N.Limitcheck (conv, cookie, cont) -> limitcheck conv cookie cont in
stmt s
and stmts r reachable ss =
  let rec stmts reachable = function
    | [] -> []
    | N.Span (_, _, []) :: ss -> stmts reachable ss
    | s :: ss ->
      let s' = full_stmt r reachable s in (* enforce order *)
      s' :: stmts (not (jumps s)) ss in
  Error.Raise.list (stmts reachable ss)
and jumps = function N.Goto _ | N.Jump _ | N.Cut _ | N.Return _ -> true
  | N.Call (_, _, _, _, _, fl, _) -> List.exists never_returns fl
  | N.S (s, _) -> jumps s
  | N.Span (_, _, ss) -> jumpsss ss
  | N.If (_, t, f) -> jumpsss t && jumpsss f
  | N.Switch (_, _, arms) -> List.for_all jumpsarm arms
  | _ -> false
and never_returns = function
  | Ast.Fl (f, _) -> never_returns f
  | Ast.NeverReturns -> true
  | _ -> false
and jumpsss = function
  | [] -> false
  | [s] -> jumps s
  | s :: N.Span (_, _, []) :: ss -> jumpsss (s :: ss)
  | _ :: ss -> jumpsss ss
and jumpsarm (_, ss) = jumpsss ss in
let stmts ss =
  let falls = not (jumpsss ss) in
  let ss = stmts r true ss in
  if falls then errorf r "control falls off end of procedure" else ss
in stmts, cformals
let nullmap = Srcmap.mk ()
let nullv = fun _ -> None
let elab_stmts v m r env = fst (elab_functions v m r env)
let elab_cformals r env = snd (elab_functions nullv nullmap r env)

<elaboration utilities 380a>≡ (379c) 380b▷
let rec exp_as_name = function
  | A.E(x, _) -> exp_as_name x
  | A.Fetch(A.NM(x, _)) -> exp_as_name (A.Fetch x)
  | A.Fetch(A.Name(_, x, _)) -> Some x
  | _ -> None in

<elaboration utilities 380b>+≡ (379c) <380a 380c>
let ispointer =
  function Types.Bits n when n = metrics.M.pointersize -> true | _ -> false in
let insist_pointer what t =
  if ispointer t then ()
  else E.error (what ^ " must be a bit vector of the native pointer type") in

<elaboration utilities 380c>+≡ (379c) <380b 381c>
let vrtl r rtl =
  match validate rtl with

```

```

| None -> rtl
| Some msg ->
  ( Printf.eprintf "%s: Warning: %s\n" (Srcmap.Str.region r) msg; rtl ) in

⟨elaboration of continuation formal 381a⟩≡ (379c)
let cformal (rgn, kind, name, a) =
  E.seq (findve rgn name env) (fun den ->
    match den with
    | FE.Variable v, Types.Bool -> impossf "Boolean variable"
    | FE.Variable v, Types.Bits w when F.is_localv name env ->
      E.ematch (aligned rgn w a) (fun a -> (kind, v, a))
    | FE.Variable _, _ ->
      errorf r "continuation parameter '%s' may not be a global variable" name
    | d, _ ->
      errorf r "continuation parameter '%s' is a %s, not a local register variable"
        name (FE.denotation's_category d)) in
let cformals formals = E.Raise.list (List.map cformal formals) in

⟨definitions of elaboration functions (elabstmt.nw) 381b⟩≡ (379c) 382d▷
let switch rg e arms =
  let rg = E.Raise.option (Auxfuns.Option.map range rg) in (* order matters *)
  let arms = E.Raise.list (List.map arm arms) in
  E.ematch3 (exp e) rg arms (fun (e, et) rg arms ->
    match et with
    | Types.Bool -> E.error "switch statement scrutinee is a Boolean"
    | Types.Bits w ->
      let rg = Auxfuns.Option.map (striprange w) rg in
      let arms = striparms w arms in
      if null arms then E.error "empty switch statement"
      else Switch(rg, e, arms)) in

⟨elaboration utilities 381c⟩+≡ (379c) <380c 381d▷
let range = function
  | A.Range (l, h) ->
    let l = con l in let h = con h in (* order matters *)
    E.ematch2 l h (fun (l, lw) (h, hw) ->
      if lw = hw then (l, h), lw
      else E.errorf "constants in range %s..%s have different types bits%d and bits%d"
        (Bits.to_string l) (Bits.to_string h) lw hw)
  | A.Point e -> E.ematch (con e) (fun (n, w) -> (n, n), w) in

⟨elaboration utilities 381d⟩+≡ (379c) <381c 381e▷
let arm (rgs, ss) =
  let rgs = E.Raise.list (List.map range rgs) in (* order matters *)
  E.ematch2 rgs (stmts r true ss) (fun rgs ss ->
    match rgs with
    | [] -> E.error "case arm has an empty range list"
    | (rg, w) :: rgs ->
      let strip w (rg, w') = if w = w' then rg else
        ⟨exn on range type mismatch 382a⟩
      in
      ((rg :: List.map (strip w) rgs), ss), w) in

⟨elaboration utilities 381e⟩+≡ (379c) <381d 382f▷
let striprange w (rg, w') =
  if w = w' then rg else
    ⟨exn on case range type mismatch 382c⟩
  in
let striparms w =
  let strip (arm, w') = if w = w' then arm else

```

```

    <exn on arm type mismatch 382b>
in
List.map strip in

<exn on range type mismatch 382a>≡ (381d)
E.errorf "ranges in case arm have different types bits%d and bits%d" w w'

<exn on arm type mismatch 382b>≡ (381e)
E.errorf "case arm uses range of type bits%d but scrutinee has type bits%d" w' w

<exn on case range type mismatch 382c>≡ (381e)
E.errorf "switch statement range has type bits%d but scrutinee has type bits%d" w' w

<definitions of elaboration functions(elabstmt.nw) 382d>+≡ (379c) <381b 382e>
let span k v ss =
  let k = con k in (* order matters *)
  E.ematch3 k (Elabexp.elab_link env v) ss (fun (k, kw) (v, vw) ss ->
    if kw <> metrics.M.wordsize then
      E.error "span token (key) must be a bit vector of native word size";
      insist_pointer "span value" (Types.Bits vw);
      Span ((k, v), ss)) in

<definitions of elaboration functions(elabstmt.nw) 382e>+≡ (379c) <382d 383b>
let assign lhs rhs =
  try E.ematch (E.Raise.list (List.map2 effect lhs rhs)) (fun es ->
    Assign (vrtl (srcmap, r) (Rtl.par es)))
  with Invalid_argument _ ->
    <complain about length mismatch in assignment 382g>
in

<elaboration utilities 382f>+≡ (379c) <381e 383a>
let effect lhs (guard,rhs) =
  let guard = Auxfun.Option.map exp guard in
  E.seq2 (loc lhs) (exp rhs) (fun (loc, w) (e, ty) ->
    if Pervasives.<> ty (Types.Bits w) then
      errorf (Elabexp.loc_region r lhs)
        "location of type bits%d cannot hold value of type %s" w (Types.to_string ty)
    else
      let rtl = Rtl.store loc e w in
      match guard with
      | None -> E.Ok rtl
      | Some g -> E.seq g (fun (g, ty) ->
        match ty with Types.Bool -> E.Ok (Rtl.guard g rtl)
        | _ -> errorf r "guard must have type bool, not %s" (Types.to_string ty))) in

<complain about length mismatch in assignment 382g>≡ (382e)
if List.length lhs < List.length rhs then
  errorf r
    "assignment has %d expressions on the right but only %d locations on the left"
    (List.length rhs) (List.length lhs)
else
  errorf r
    "assignment has %d locations on the left but only %d expressions on the right"
    (List.length lhs) (List.length rhs)

```

(elaboration utilities 383a)+≡ (379c) <382f 383c>

```

let limitcheck cconv cookie cont =
  E.seq (exp cookie) (fun (cookie, cookiety) ->
    insist_pointer "limit cookie" cookiety;
    match cont with
    | None -> Error.Ok (Limitcheck (cconv, cookie, None))
    | Some (failk, recname) ->
      let reck = A.Fetch (A.Name (None, recname, None)) in
      E.ematch2 (exp failk) (exp reck) (fun (failk, fty) (reck, rty) ->
        insist_pointer "overflow continuation" fty;
        insist_pointer "automatically generated recovery continuation" rty;
        let f = { failcont = failk; recont = reck; recname = recname; } in
        Limitcheck (cconv, cookie, Some f))) in

```

(definitions of elaboration functions(elabstmt.nw) 383b)+≡ (379c) <382e 383d>

```

let goto tgt tgts =
  let tgts = E.Raise.list (List.map (goto_target r) tgts) in (* order matters *)
  E.ematch2 (exp tgt) tgts (fun (te, t) tgts ->
    insist_pointer "target of goto" t;
    if null tgts && not (exp_is_local_code_label r tgt) then
      E.error "target list omitted and target is not a local code label";
    Goto (te, tgts)) in

```

(elaboration utilities 383c)+≡ (379c) <383a 384d>

```

let goto_target r x = E.seq (findve r x env) (fun (d, t) ->
  match d with
  | FE.Label (FE.Code _) when F.is_localv x env -> E.Ok x
  | FE.Label (FE.Code _) -> errorf r "may not goto %s (label in another procedure)" x
  | _ -> errorf r "may not goto %s %s" (FE.denotation's_category d) x) in
let exp_is_local_code_label r e = match exp_as_name e with
| None -> false
| Some x -> match findve r x env with
| E.Ok (FE.Label (FE.Code _), _) when F.is_localv x env -> true
| E.Ok _ -> false
| E.Error -> impossf "good target name came back as error" in

```

O.4.2 Value-passing statements: calls and so on

(definitions of elaboration functions(elabstmt.nw) 383d)+≡ (379c) <383b 383e>

```

let call left conv p args targets flows alias =
  let left = E.Raise.list (List.map (Elabexp.elab_kinded_name env) left) in
  let args = E.Raise.list (List.map (actual r) args) in
  let tgts = E.Raise.list (List.map (call_target r) targets) in
  let flow = call_flow r conv flows in
  let alias = elab_alias r alias in
  E.seq (exp p) (fun (pe, pt) ->
    reject_uncallable r p;
    E.ematch5 left args tgts flow alias (fun left args tgts flow alias ->
      insist_pointer "procedure value" pt;
      Call (left, conv, pe, args, tgts, flow, alias))) in

```

(definitions of elaboration functions(elabstmt.nw) 383e)+≡ (379c) <383d 384a>

```

let jump conv p args targets =
  let args = E.Raise.list (List.map (actual r) args) in
  let tgts = E.Raise.list (List.map (call_target r) targets) in
  E.seq (exp p) (fun (pe, pt) ->
    reject_uncallable r p;
    E.ematch2 args tgts (fun args tgts ->
      insist_pointer "procedure value" pt;
      Jump (conv, pe, args, tgts))) in

```


⟨definitions of elaboration functions (elabstmt.nw) 384a⟩+≡ (379c) <383e 384b>

```
let cut conv k args flows =
  let args = E.Raise.list (List.map (actual r) args) in
  let flow = cut_flow r conv flows in
  E.ematch3 args (exp k) flow (fun args (k,kt) flow ->
    insist_pointer "target of 'cut to'" kt;
    Cut (conv, k, args, flow)) in
```

⟨definitions of elaboration functions (elabstmt.nw) 384b⟩+≡ (379c) <384a 385d>

```
let return conv alt args =
  let args = E.Raise.list (List.map (actual r) args) in
  let i, n = match alt with
  | None -> E.Ok 0, E.Ok 0
  | Some (i, n) ->
    let okbits f x = E.Ok (Bits.S.to_int (f x)) in
    let const k = E.seq (exp k) (fun (k, kt) ->
      E.catch (eprint r) (okbits Simplify.bits) k) in
    const i, const n in
  E.ematch3 i n args (fun i n args ->
    if i >= 0 && n >= 0 && i <= n then
      Return (conv, i, n, args)
    else
      ⟨raise error exn with bad continuation numbers 384c⟩
  ) in
```

⟨raise error exn with bad continuation numbers 384c⟩≡ (384b)

```
if i < 0 then
  E.errorf "in return <i/n>, i must be nonnegative but you have i = %d" i
else if n < 0 then
  E.errorf "in return <i/n>, n must be nonnegative but you have n = %d" n
else if i > n then
  E.errorf "in return <i/n>, i must be at most n but you have i = %d and n = %d" i n
else
  impossf "bad <i/n> with i = %d and n = %d, but no better error message" i n
```

⟨elaboration utilities 384d⟩+≡ (379c) <383c 384e>

```
let actual r (kind, e, a) = E.seq (exp e) (fun (e, ty) ->
  match ty with
  | Types.Bits w -> E.ematch (aligned r w a) (fun a -> (kind, e, w, a))
  | Types.Bool -> errorf r "an actual parameter may not be a Boolean") in
```

⟨elaboration utilities 384e⟩+≡ (379c) <384d 384f>

```
let reject_uncallable r e = match exp_as_name e with
| None -> ()
| Some x -> match findve r x env with
| E.Ok ((FE.Label (FE.Proc _) | FE.Import (_, _) | FE.Variable _), _) | E.Error -> ()
| E.Ok (d, _) ->
  E.errorf "cannot call or jump to %s %s" (FE.denotation's_category d) x in
```

⟨elaboration utilities 384f⟩+≡ (379c) <384e 385a>

```
let elab_alias r alias =
  let none = function None -> true | Some _ -> false in
  let rec elab r reads writes = function
  | [] -> E.Ok { reads = reads; writes = writes }
  | A.Al(a,r) :: a's -> E.catch (eprint r) (elab r reads writes) (a :: a's)
  | A.Reads ns :: a's when none reads -> elab r (Some ns) writes a's
  | A.Writes ns :: a's when none writes -> elab r reads (Some ns) a's
  | A.Reads _ :: _ -> errorf r "multiple 'reads' annotations on one call"
  | A.Writes _ :: _ -> errorf r "multiple 'writes' annotations on one call" in
  E.catch (eprint r) (elab r None None) alias in
```

$\langle \text{elaboration utilities } 385a \rangle + \equiv$
(379c) $\triangleleft 384f \ 385b \triangleright$

```

let flow r conv default_aborts default_returns =
  let as_cut_to k = k.FE.cut_to <- true in
  let as_unwinds k = k.FE.unwound_to <- true in
  let as_returns k =
    if not (List.mem conv k.FE.returned_to) then
      k.FE.returned_to <- conv :: k.FE.returned_to in
  let rec ks r as_what prev' ns =
    let continuation n = E.seq (findve r n env) (function
      | FE.Continuation k, _ -> as_what k; E.Ok n
      | _ -> errorf r "%s is not a continuation" n) in
    match ns with
    | [] -> prev'
    | n :: ns -> ks r as_what (continuation n :: prev') ns in
  let rec flow r c' u' r' aborts returns = function
    | [] -> finish_flow c' u' r' aborts returns
    | A.Fl(f,r) :: fs -> flow r c' u' r' aborts returns (f :: fs)
    | A.CutsTo ns :: fs -> flow r (ks r as_cut_to c' ns) u' r' aborts returns fs
    | A.UnwindsTo ns :: fs -> flow r c' (ks r as_unwinds u' ns) r' aborts returns fs
    | A.ReturnsTo ns :: fs -> flow r c' u' (ks r as_returns r' ns) aborts returns fs
    | A.Aborts :: fs -> flow r c' u' r' true returns fs
    (* | A.NeverAborts :: fs -> flow r c' u' r' false returns fs *)
    | A.NeverReturns :: fs -> flow r c' u' r' aborts false fs
  and finish_flow c' u' r' aborts returns =
    let list ns = E.Raise.list (List.rev ns) in
    let c = list c' in let u = list u' in let r = list r' in (* order matters *)
    E.ematch3 c u r (fun c u r ->
      { cuts = c; unwinds = u; areturns = r; aborts = aborts; returns = returns }) in
  flow r [] [] [] default_aborts default_returns in

```

$\langle \text{elaboration utilities } 385b \rangle + \equiv$
(379c) $\triangleleft 385a \ 385c \triangleright$

```

let call_flow r cc fs = flow r cc false true fs in
let null = function [] -> true | _ :: _ -> false in
let cut_flow r cc fs =
  E.seq (flow r cc true false fs) (fun f ->
    if not (null f.unwinds) then
      errorf r "'also unwinds to' is meaningless on a 'cut to'"
    else if not (null f.aretains) then
      errorf r "'also returns to' is meaningless on a 'cut to'"
    else if f.returns then
      impossf "a 'cut to' returns? I think not!"
    else
      E.Ok { ccuts = f.cuts; caborts = f.aborts }) in

```

$\langle \text{elaboration utilities } 385c \rangle + \equiv$
(379c) $\triangleleft 385b \ 386b \triangleright$

```

let call_target r x = E.seq (findve r x env) (function
  | (FE.Label (FE.Proc _) | FE.Import _), t when ispointer t -> E.Ok x
  | d, _ -> errorf r "target %s is a %s (must be a procedure or imported pointer)"
    x (FE.denotation's_category d)) in

```

$\langle \text{definitions of elaboration functions}(\text{elabstmt.nw}) \ 385d \rangle + \equiv$
(379c) $\triangleleft 384b \triangleright$

```

let prim lhs conv op args flows =
  let strip_decorations = function
    | "", arg, None -> arg
    | "", arg, Some _ ->
      E.error "call to primitive operator may not use explicit alignment"
    | s, arg, None ->
      E.error "call to primitive operator must use empty kind"
    | s, arg, Some _ ->
      E.error "call to primitive operator must use empty kind and default alignment"

```

```

in
let lhs = E.Raise.list (List.map (Elabexp.elab_kinded_name env) lhs) in
let args = E.Raise.list (List.map (fun arg -> exp (strip_decorations arg)) args) in
let flow = prim_flow r flows in
E.ematch3 lhs args flow (fun lhs args flow ->
  match lhs with
  | [_, (result, w), aligned] ->
    let argtys = List.map snd args in
    let argexps = List.map fst args in
    let t, opr = Rtlop.Translate.prefix op argtys in
    if Pervasives.<> t (Types.Bits w) then
      (raise error exn on primop type mismatch 386a)
    else
      Assign (Rtl.store result (Rtl.app opr argexps) w)
  | _ -> E.error "primitive operator produces exactly one result") in

(raise error exn on primop type mismatch 386a)≡ (385d)
Printf.kprintf E.error
  "left-hand side has type bits%d, but primitive operator produces results of type %s"
  w (Types.to_string t)

(elaboration utilities 386b)+≡ (379c) <385c
let prim_flow r fs =
  E.seq (flow r "C--" false true fs) (fun f ->
    if not (null f.unwinds) then
      errorf r "'also unwinds to' is meaningless on a primitive operator"
    else if not (null f.cuts) then
      errorf r "'also cuts to' is meaningless on a primitive operator"
    else if not (null f.areturns) then
      errorf r "alternate returns on primitive operators are not yet implemented"
    else if f.aborts then
      errorf r "'also aborts' is meaningless on a primitive operator"
    else if not f.returns then
      errorf r "'never returns' is meaningless on a primitive operator"
    else
      E.Ok f) in

(elabstmt.ml 386c)+≡ (377c) <379c
let codelabels ss =
  let rec adds ss labels = List.fold_left add labels ss
  and add labels s = match s with
  | If (_, ts, fs) -> adds ts (adds fs labels)
  | Switch (_, _, arms) ->
    List.fold_left (fun labels (_, ss) -> adds ss labels) labels arms
  | Label l -> l :: labels
  | Cont _ -> labels
  | Span (_, ss) -> adds ss labels
  | Assign _
  | Call _
  | Call' _
  | Goto _
  | Jump _
  | Cut _
  | Return _
  | Limitcheck _ -> labels in
  adds ss []

```

O.5 front_nelab/memalloc.nw

```
<front_nelab/memalloc.ml 387a>≡  
  <memalloc.ml 387d>  
  
<front_nelab/memalloc.mli 387b>≡  
  <memalloc.mli 387c>
```

O.6 Automaton-Based Memory Allocation

```
<memalloc.mli 387c>≡ (387b)  
type t (* immutable *)  
  
type growth = Up | Down  
  
val at : start:Rtl.exp -> growth -> t (* provide start address *)  
val relative : anchor:Rtl.exp -> dbg:string -> growth -> t (* unknown addr relative to anchor *)  
val allocate : t -> size:int -> t (* increase block *)  
val align : t -> int -> t (* align cursor *)  
val alignment : t -> int (* max alignment ever requested *)  
val current : t -> Rtl.exp (* obtain cursor *)  
val freeze : t -> Block.t (* return allocated, aligned block *)  
val num_allocated : t -> int
```

O.6.1 Implementation

```
<memalloc.ml 387d>≡ (387a)  
type growth = Up | Down  
type unchanging =  
  { start : Rtl.exp (* start address of block *)  
    ; growth : growth  
    ; exp_at : int -> Rtl.exp (* exp at distance k from start *)  
  }  
type t =  
  { u : unchanging  
    ; num_allocated : int (* size of block *)  
    ; max_alignment : int (* max alignment ever requested *)  
  }  
  
let at ~start g =  
  let w = Rtlutil.Width.exp start in  
  let addk = Rtlutil.addk w start in  
  { u = { start = start  
        ; growth = g  
        ; exp_at = match g with Up -> addk | Down -> fun k -> addk (-k)  
        }  
    ; num_allocated = 0  
    ; max_alignment = 1  
  }  
  
let relative ~anchor ~dbg g =  
  let w = Rtlutil.Width.exp anchor in  
  at (Rtlutil.add w anchor (Rtl.late (Idgen.offset dbg) w)) g  
  
let allocate t ~size =  
  assert (size >= 0);  
  { t with num_allocated = t.num_allocated + size }
```

```

let align    t n = (* align both size and base alignment *)
  assert (n > 0);
  { t with max_alignment = max t.max_alignment n (*SHOULD BE LEAST COMMON MULTIPLE*)
    ; num_allocated = Auxfun.round_up_to t.num_allocated ~multiple_of:n
  }
let current t = t.u.exp_at t.num_allocated
let alignment t = t.max_alignment

let freeze t = match t.u.growth with
| Up   -> Block.at t.u.start  t.num_allocated t.max_alignment
| Down -> let t = align t t.max_alignment in
  Block.at (current t) t.num_allocated t.max_alignment

let num_allocated t = t.num_allocated

```

O.7 front_nelab/nast.nw

$\langle \text{front_nelab/nast.ml } 388a \rangle \equiv$
 $\langle \text{nast.ml } 51a \rangle$

$\langle \text{front_nelab/nast.mli } 388b \rangle \equiv$
 $\langle \text{nast.mli } 27b \rangle$

O.8 front_nelab/nelab.nw

$\langle \text{front_nelab/nelab.ml } 388c \rangle \equiv$
 $\langle \text{nelab.ml } 389e \rangle$

$\langle \text{front_nelab/nelab.mli } 388d \rangle \equiv$
 $\langle \text{nelab.mli } 389b \rangle$

$\langle \text{exposed types(nelab.nw) } 388e \rangle \equiv$ (389) 388f▷
 type name = string
 [@@deriving show]
 type kind = string
 [@@deriving show]
 type aligned = int
 [@@deriving show]

$\langle \text{exposed types(nelab.nw) } 388f \rangle + \equiv$ (389) ◁388e 389a▷
 type index = int
 [@@deriving show]
 type linktime = Reloc.t
 [@@deriving show]
 type 'a proc =
 { env : 'a Fenv.Dirty.env;
 sym : Symbol.t; (* assembly-language symbol for this procedure *)
 cc : string; (* calling convention *)
 name : name;
 spans : (Bits.bits * linktime) list; (* enclose whole procedure *)
 formals : (index * kind * Ast.variance * Rtl.width * name * aligned) list;
 locals : Fenv.variable list;
 continuations : (name * Fenv.continuation) list;
 stackmem : Block.t;
 stacklabels : Rtl.exp list;

```

    code          : Elabstmt.stmt list;
    basic_block    : bool;  (* procedure represents a basic block from source *)
  }
[@@deriving show { with_path = false} ]

type 'a datum =
| Datalabel      of Symbol.t (* must be asm level not source level *)
| Align          of int
| InitializedData of (linktime * Rtl.width) list
| UninitializedData of int (* counts the number of mems *)
| Procedure      of 'a proc
[@@deriving show { with_path = false} ]

type 'a section = name * 'a datum list
[@@deriving show]

<exposed types(nelab.nw) 389a>+≡ (389) <388f 389c>
type 'a compunit = {
  globals : (name * Fenv.variable) list;
  sections : 'a section list;
}
[@@deriving show { with_path = false} ]

<nelab.mli 389b>≡ (388d) 389d>
<exposed types(nelab.nw) 388e>
val program : swap:bool -> validator -> Srcmap.map -> 'a Asm.assembler -> Nast.t ->
('a Fenv.Dirty.env' * 'a compunit) Error.error

<exposed types(nelab.nw) 389c>+≡ (389) <389a>
type validator = Rtl.rtl -> string option
[@@deriving show]

<nelab.mli 389d>+≡ (388d) <389b>
val rewrite : (Auxfun.void compunit -> Auxfun.void compunit) -> ('a compunit -> 'a compunit)

```

O.9 Checking Static Semantics

O.9.1 Assembler Symbols

O.9.2 Implementation – The 30 000 feet overview

```

<nelab.ml 389e>≡ (388c) 390a>
open Nopoly

<exposed types(nelab.nw) 388e>
module A = Ast
module B = Bits
module E = Error
module F = Fenv.Dirty
module FE = Fenv
module N = Nast
module R = Rtl
module RP = Rtl.Private
module Dn = Rtl.Dn
module Up = Rtl.Up
module T = Types

let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

⟨nelab.ml 390a⟩+≡ (388c) ◁389e 390f▷
 ⟨auxiliaries(nelab.nw) 390b⟩

O.9.3 Auxiliaries

⟨auxiliaries(nelab.nw) 390b⟩≡ (390a) 390c▷
 let foldl: ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a = List.fold_left

⟨auxiliaries(nelab.nw) 390c⟩+≡ (390a) ◁390b 390d▷
 let (--) xx yy = List.filter (fun x -> List.mem x yy) xx

⟨auxiliaries(nelab.nw) 390d⟩+≡ (390a) ◁390c 390e▷
 let error r env msg = E.errorRegionPrt (F.srcmap env,r) msg

⟨auxiliaries(nelab.nw) 390e⟩+≡ (390a) ◁390d
 let is2power x = x > 0 && x land (x - 1) = 0

O.9.4 Sorting and binding type and constant declarations

⟨nelab.ml 390f⟩+≡ (388c) ◁390a 390g▷
 module type SB = sig
 type d
 module Decl : Topsort.Sortable with type decl = d N.marked
 val what : string
 type rhs
 val eval : 'a F.env' -> Decl.decl -> rhs E.error
 val bind : string -> rhs E.error N.marked -> 'a F.env' -> 'a F.env'
 end

⟨nelab.ml 390g⟩+≡ (388c) ◁390f 391b▷
 module SortAndBind (D : SB) = struct
 module Sort = Topsort.Make (D.Decl)
 let bind_sortable_definitions env ds =
 ⟨definitions of sort-binding functions 390h⟩
 let rec bind env ds =
 try
 foldl bind_one env (Sort.sort ds)
 with Sort.Cycle offending ->
 let ds = ds -- offending in
 let env = bind_to_error env offending in
 (report_cycle env offending ; bind env ds) in
 bind env ds
 end

⟨definitions of sort-binding functions 390h⟩≡ (390g) 390i▷
 let bind_each_name rhs env ((r, _) as d) =
 foldl (fun env n -> D.bind n (r, rhs) env) env (D.Decl.defines d) in
 let bind_to_error env decls =
 foldl (bind_each_name E.Error) (F.flagError env) decls in

⟨definitions of sort-binding functions 390i⟩+≡ (390g) ◁390h 391a▷
 let rec bind_one env d =
 match D.eval env d with
 | E.Error -> bind_each_name E.Error (F.flagError env) d
 | E.Ok _ as rhs -> bind_each_name rhs env d in

```

<definitions of sort-binding functions 391a>+≡ (390g) <390i
let report_cycle env ds =
  let report d =
    error (fst d) env
    (D.what ^ " definition for "^(List.hd (D.Decl.defines d))^" is cyclic") in
  List.iter report ds in

<nelab.ml 391b>+≡ (388c) <390g 391c>
module Type = struct
  type d = N.typedefn
  module Decl = struct
    type decl = N.typedefn N.marked
    let defines (_, (_,names)) = names
    let uses (_, (t, _)) =
      let rec u = function
        | A.BitsTy _ -> []
        | A.TypeSynonym x -> [x]
        | A.T (t, _) -> u t in
      u t
    end
    let what = "type"
    type rhs = Types.ty
    let eval env (_, (t, _)) = E.ematch (Elabexp.elab_ty env t) (fun w -> Types.Bits w)
    let bind = F.bindt
  end
  module TypeSortBind = SortAndBind(Type)

<nelab.ml 391c>+≡ (388c) <391b 392b>
module Const = struct
  type d = N.constdefn
  <utility functions for constants 391d>
  module Decl = struct
    type decl = d N.marked
    let defines (_, (t,name,expr)) = [name]
    let uses (_, (t,name,expr)) = freeExprVars expr []
  end
  let what = "constant"
  type rhs = Bits.bits * Types.ty
  let eval env (r, (ty, name, e)) =
    let errorf x =
      Printf.kprintf (fun s -> E.errorRegionPrt (F.srcmap env, r) s; E.Error) x in
    E.seq (Elabexp.elab_con env e) (fun (b, w) ->
      match ty with
      | None -> E.Ok (b, Types.Bits w)
      | Some ty ->
        E.seq (Elabexp.elab_ty env ty) (fun w' ->
          if w = w' then E.Ok (b, Types.Bits w)
          else errorf "constant %s declared with type bits%d has actual type bits%d"
            name w' w))
    let bind name (r, typed_exp) env =
      F.bindv name (r, E.ematch typed_exp (fun (b, t) -> FE.Constant b, t)) env
  end
  module ConstSortBind = SortAndBind(Const)

<utility functions for constants 391d>≡ (391c) 392a>
let rec freeExprVars e tail = match e with
| A.E(x,_) -> freeExprVars x tail
| A.Fetch(lvalue) -> freeLValueVars lvalue tail
| A.BinOp(e1,_,e2) -> freeExprVars e1 (freeExprVars e2 tail)
| A.UnOp(_,e) -> freeExprVars e tail

```



```

| A.PrimOp(_,args) -> foldl (fun t (_,e,_) -> freeExprVars e t) tail args
| A.Sint _
| A.Uint _
| A.Float _
| A.Char _ -> tail

```

```

<utility functions for constants 392a>+≡ (391c) <391d
and freeLValueVars l tail = match l with
| A.NM(x,r) -> freeLValueVars x tail
| A.Mem(_,e,_,_) -> freeExprVars e tail
| A.Name(_,name,_) -> if List.mem name tail then tail else name :: tail

```

O.9.5 Building the environment

```

<nelab.ml 392b>+≡ (388c) <391c 398a>
type scope = Local | Global

```

```

let program ~swap validate srcmap asm ast =
  E.seq (Metrics.of_ast ~swap srcmap ast.N.target) (fun metrics ->
    let flag = function E.Error -> F.flagError | E.Ok _ -> (fun env -> env) in
    let eprint r s = E.errorRegionPrt (srcmap, r) s in
    let errorf r = Printf.kprintf (fun s -> eprint r s; E.Error) in
    let findve r n = E.catch (eprint r) (fun env -> snd (F.findv n env)) in
    let pointerty = Types.Bits metrics.Metrics.pointersize in
    (* claude: was a local copy of Vfp.mk, which used to be unreachable from
       * here; its core is in ir/vfp.ml now. *)
    let vfp = Vfp.mk metrics.Metrics.wordsize in
    let memsize = metrics.Metrics.memsize in
    let aligned = Elabexp.aligned metrics in
    let aligned r w = E.catch (eprint r) (fun a -> E.Ok (aligned w a)) in
    <definitions of name-binding functions 393a>
    <definitions of elaboration functions(nelab.nw) 395b>
    (* phase one: create environment and bind all names *)
    let env = F.empty srcmap metrics asm in
    let env = F.push env (F.emptyscope) in (* weird, but so it is *)
    let env = TypeSortBind.bind_sortable_definitions env ast.N.udecls.N.types in
    let env = ConstSortBind.bind_sortable_definitions env ast.N.udecls.N.constants in
    let env = foldl bind_and_remember_import env ast.N.imports in
    let env = foldl remember_export env ast.N.exports in
    let env, globals = registers_from Global 0 ast.N.globals env in
    let env = foldl (foldl bind_code_label) env ast.N.code_labels in
    let env = foldl bind_section_names env ast.N.sections in
    (* phase two: in environment, elaborate and check the world (only flags env) *)
    let env = foldl check_export env ast.N.exports in
    let ss = E.Raise.list (List.map (section env) ast.N.sections) in
    let env = flag ss env in
    E.ematch2 globals ss (fun globals ss ->
      (env, { globals = globals; sections = ss })))

```

```

<definitions of procedure-processing functions 392c>≡ (395b)
let proc env spans p =
  let env = F.push env F.emptyscope in
  let env = TypeSortBind.bind_sortable_definitions env p.N.pdecls.N.types in
  let env = ConstSortBind.bind_sortable_definitions env p.N.pdecls.N.constants in
  let env, n, formals = formals_from 0 p.N.formals env in
  let env, locals = registers_from Local n p.N.locals env in
  let env = foldl bind_code_label env p.N.labels in (* odd -- explained above *)
  let env, conts = bind_and_return_continuations env p.N.continuations in
  let stackmem, stacklabels, env = stackdata env p.N.stackdata in

```

```

let code = Elabstmt.elab_stmts validate srcmap p.N.region env p.N.code in
let _, den = F.findv p.N.name env in (* lookup cannot fail *)
E.ematch4 code locals (E.Raise.list spans) den (fun code locals spans den ->
  let sym = match den with
  | FE.Label (FE.Proc sym), _ -> sym
  | _ -> impossf "procedure %s is not a procedure?!" p.N.name in
  Procedure
  { env = env; cc = p.N.cc; name = p.N.name; sym = sym; formals = formals;
    locals = List.map snd locals; continuations = conts; spans = spans;
    stackmem = stackmem; stacklabels = stacklabels; code = code;
    basic_block = false; }) in

```

<definitions of name-binding functions 393a>≡ (392b) 393b▷

```

let bind_and_return_continuations env conts =
  let rec bind prev' env = function
  | [] -> env, List.rev prev'
  | (r, (name, cc, formals)) :: ks ->
    match (Elabstmt.elab_cformals r env formals) with
    | E.Error -> bind prev' (F.bindv name (r, E.Error) env) ks
    | E.Ok formals ->
      let addrblock =
        Block.relative vfp ("continuation " ^ name)
        Block.at ~size:0 ~alignment:1 in
      let k = { FE.cut_to = false; FE.unwound_to = false; FE.returned_to = [];
        FE.base = addrblock; FE.escapes = false; FE.convention = cc;
        FE.formals = formals; } in
      let env = F.bindv name (r, E.Ok (FE.Continuation k, pointerty)) env in
      bind ((name, k) :: prev') env ks in
  bind [] env conts in

```

<definitions of name-binding functions 393b>+≡ (392b) ◁393a 393d▷

```

let bind_and_remember_import env (r, ty, imports) =
  let import env (foreign,name) =
    let env = F.import r (Auxfun.Option.get name foreign) name env in
    let sym = F.symbol env name in
    let den = FE.Import (Auxfun.Option.get name foreign, sym) in
    let ty =
      <convert optional importing ty as long as it is pointer size 393c>
    in
    F.bindv name (r, E.Raise.right (den, ty)) env in
  foldl import env imports in

```

<convert optional importing ty as long as it is pointer size 393c>≡ (393b)

```

match ty with None -> E.Ok pointerty
| Some t -> E.seq (Elabexp.elab_ty env t) (fun w ->
  if w == metrics.Metrics.pointersize then E.Ok pointerty
  else errorf r "imported type bits%d is inconsistent with native pointer type %s"
    w (Types.to_string pointerty))

```

<definitions of name-binding functions 393d>+≡ (392b) ◁393b 394a▷

```

let bind_code_label env (r, n) =
  F.bindv n (r, E.Ok (FE.Label(FE.Code (F.symbol env n)), pointerty)) env in

```

<to migrate to variable placer 393e>≡

```

let global_base_sym = F.symbol env "Cmm.global_area" in
let global_sigbuf = Buffer.create 128 in
let global_loc env =
  let global_automaton = target.T.globals (Rtl.link global_base_sym pointersize) in
  fun kind ty name ->
    let kind h =

```

```

let w = tywidth ty in
Printf.bprintf global_sigbuf "[%s %d]" name w
AT.allocate t w h in
let hardware r =
  try
    Printf.bprintf global_sigbuf "[%s %s]" name r;
    E.Ok (AT.of_loc (Strutil.Map.find t target.T.named_locs))
  with Not_found ->
    errorf "unknown hardware register %S" r in
function FE.Rnone -> kinded ""
  | FE.RKind h -> kinded h
  | FE.RReg r -> hardware r in
with Error.ErrorExn msg -> impossible msg
in
  ( List.iter (assign env) names
  ; AT.freeze t, Digest.string (Buffer.contents decls)
  )

```

(definitions of name-binding functions 394a) +≡ (392b) <393d 394b>

```

let registers_from scope n rs env =
  let rec regs_from prev' n rs env = match rs with
  | [] ->
    let regs = E.ematch (E.Raise.list (List.rev prev')) (fun rs ->
      let unreg = function (n, (FE.Variable r, _)) -> (n, r)
        | _ -> impossf "nonregister" in
      List.map unreg rs) in
    env, regs
  | (rgn, (v, kind, t, name, hwreg)) :: rs ->
    let entry = E.seq (Elabexp.elab_ty env t) (fun w ->
      let loc = (match scope with Global -> R.global | Local -> R.var) name n w in
      let reg h = { FE.index = n; FE.rkind = h; FE.loc = loc; FE.variance = v } in
      let den h = E.Ok (FE.Variable (reg h)) in
      let den = match kind, hwreg with
      | Some _, Some _ -> errorf rgn "can't have kind and hardware register"
      | Some h, None ->
        (match scope with Global -> den (FE.RKind h)
        | Local -> errorf rgn
          "kinds permissible only on formal parameters or global variables")
      | None, None -> den FE.RNone
      | None, Some r ->
        (match scope with
        | Local -> errorf rgn "local register can't be mapped to hardware"
        | Global -> den (FE.RReg r)) in
      E.Raise.left (den, Types.Bits w)) in
    let named = E.Raise.right (name, entry) in
    regs_from (named::prev') (n+1) rs (F.bindv name (rgn, entry) env) in
  regs_from [] n rs env in

```

(definitions of name-binding functions 394b) +≡ (392b) <394a 395a>

```

let formals_from n rs env =
  let rec formals prev' n rs env = match rs with
  | [] -> env, n, List.rev prev'
  | (r, (kind, inv, ty, name, a)) :: rs ->
    let error () = formals prev' n rs (F.bindv name (r, E.Error) env) in
    match Elabexp.elab_ty env ty with
    | E.Error -> error()
    | E.Ok w ->
      match aligned r w a with
      | E.Error -> error()
      | E.Ok a ->

```

```

    let i = { FE.index = n; FE.variance = inv;
              FE.loc = Rtl.var name n w; FE.rkind = FE.RKind kind; } in
    let env = F.bindv name (r, E.Ok (FE.Variable i, Types.Bits w)) env in
    formals ((n, kind, inv, w, name, a)::prev') (n+1) rs env in
formals [] n rs env in

```

<definitions of name-binding functions 395a>+≡ (392b) <394b 395c>

```

let bind_section_names env (secname, ds) =
  <definitions of name-binding functions for data 396c>
  foldl datum env ds in

```

<definitions of elaboration functions(nelab.nw) 395b>≡ (392b)

```

let section env (secname, ds) =
  <definitions of procedure-processing functions 392c>
  <definitions of elaboration functions for data 396d>
  E.Raise.right (secname, E.Raise.list (data [] ds [])) in

```

O.9.6 Translation of stackdata declarations, accumulating labels

<definitions of name-binding functions 395c>+≡ (392b) <395a 396a>

```

let stackdata env =
  <definitions of stack-data functions 395d>
  in
  stackdata (Memalloc.relative vfp "stackdata" Memalloc.Up) [] env in

```

<definitions of stack-data functions 395d>≡ (395c) 395e>

```

let rec stackdata mem stacklabels' env ds = match ds with
| [] -> Memalloc.freeze mem, List.rev stacklabels', env
| d :: ds ->
    stackdatum mem stacklabels' env d
    (fun mem stacklabels' env -> stackdata mem stacklabels' env ds)

```

<definitions of stack-data functions 395e>+≡ (395c) <395d

```

and stackdatum mem stacklabels' env (r, d) k =
  let errorf fmt = Printf.kprintf
    (fun s -> eprint r s; k mem stacklabels' (F.flagError env)) fmt in
  match d with
  | N.Datalabel l ->
    let loc = Memalloc.current mem in
    let env = F.bindv l (r, E.Ok (FE.Label (FE.Stack loc), pointerty)) env in
    k mem (loc :: stacklabels') env
  | N.Align n when is2power n ->
    k (Memalloc.align mem n) stacklabels' env
  | N.Align n -> errorf "alignment %d is not a power of 2" n
  | N.ReserveMem(ty,size,Some _) -> errorf "initialization not permitted for stackdata"
  | N.ReserveMem(ty,A.DynSize,_) -> errorf "illegal size for stackdata"
  | N.ReserveMem(ty,size,None) ->
    (match Elabexp.elab_ty env ty with
    | E.Error -> k mem stacklabels' env
    | E.Ok w ->
      let finish size =
        if w mod memsize = 0 then
          k (Memalloc.allocate mem (size * (w / memsize))) stacklabels' env
        else
          errorf "width %d of initialized data is not a multiple of memsize %d"
            w memsize in
      match size with
      | A.NoSize -> finish 1
      | A.DynSize -> impossf "dynamic memory size"

```

```

    | A.FixSize e ->
      match Elabexp.elab_con env e with
      | E.Ok (bits, _) -> finish (Bits.U.to_int bits)
      | E.Error _ -> k mem stacklabels' (F.flagError env)
  )
| N.Procedure _ -> impossf "nested procedures"
| N.SSpan (_, _, _) -> impossf "span in stackdata"

⟨definitions of name-binding functions 396a⟩+≡ (392b) <395c 396b>
let remember_export env (r, _, exports) =
  let export env (name, foreign) =
    F.export r name (Auxfuns.Option.get name foreign) env in
  foldl export env exports in

⟨definitions of name-binding functions 396b⟩+≡ (392b) <396a
let check_export env (r, ty, pairs) =
  let errorf fmt = Printf.kprintf (fun s -> eprint r s; F.flagError env) fmt in
  let check_pair env (name, foreign) =
    E.seq' env (findve r name env) (fun (den, ty) ->
      if Pervasives.<> ty pointerty then
        errorf "exported value must have native pointer type"
      else
        match den with
        | FE.Label (FE.Proc _ | FE.Code _ | FE.Data _) -> env
        | d -> errorf "%s '%s' can't be exported" (FE.denotation's_category d) name) in
  let check_pair env = E.catch' env (eprint r) (check_pair env) in
  let env = match ty with
  | Some t ->
    E.seq' env (Elabexp.elab_ty env t) (fun w ->
      if w <> metrics.Metrics.pointersize then
        errorf "exported value must have native pointer type"
      else
        env)
  | None -> env in
  foldl check_pair env pairs in

⟨definitions of name-binding functions for data 396c⟩≡ (395a)
let rec datum env (r, d) =
  let errorf fmt = Printf.kprintf (fun s -> eprint r s; F.flagError env) fmt in
  match d with
  | N.Datalabel l ->
    F.bindv l (r, E.Ok (FE.Label (FE.Data (F.symbol env l)), pointerty)) env
  | N.Align n when is2power n -> env
  | N.Align n -> errorf "alignment %d is not a power of 2" n
  | N.ReserveMem(ty, size, init) -> env
  | N.Procedure p ->
    let den = FE.Label (FE.Proc (F.symbol env p.N.name)) in
    F.bindv p.N.name (r, E.Ok (den, pointerty)) env
  | N.SSpan (_, _, ds) -> foldl datum env ds in

⟨definitions of elaboration functions for data 396d⟩≡ (395b) 397a>
let metrics = F.metrics env in
let rec data_spans ds rest = match ds with
| [] -> (match rest with [] -> [] | (spans, ds) :: rest -> data_spans ds rest)
| (r, d) :: ds ->
  let datalabel_meaning l = match findve r l env with
  | E.Ok (FE.Label (FE.Data sym), _) -> E.Ok (Datalabel sym)
  | E.Ok _ -> impossf "data label %s stands for something else?!" l
  | E.Error -> E.Error in
  match d with

```

```

| N.Datalabel l -> let d = datalabel_meaning l in d :: data spans ds rest
| N.Align n      -> let d = E.Ok (Align n)      in d :: data spans ds rest
| N.Procedure p -> let d = proc env spans p      in d :: data spans ds rest
| N.ReserveMem(ty,size,init) ->
  let d = reserve_mem r ty size init in d :: data spans ds rest
| N.SSpan (k, v, ds') ->
  let span = E.seq2 (Elabexp.elab_con env k) (Elabexp.elab_link env v)
  (fun (k, kw) (v, vw) ->
    if kw <> metrics.Metrics.wordsize then
      errorf r "span token (key) must be a bit vector of native word size"
    else if vw <> metrics.Metrics.pointersize then
      errorf r "span value must be a bit vector of native pointer size"
    else
      E.Ok ((k, v))) in
  data (span::spans) ds' ((spans, ds) :: rest)

⟨definitions of elaboration functions for data 397a⟩+≡ (395b) <396d 397b>
and reserve_mem r ty size idata =
  E.seq2 (Elabexp.elab_ty env ty) (init_size r size) (fun w n ->
    match idata with
    | None ->
      if w mod memsize = 0 then
        E.Ok (UninitializedData (Auxfuns.Option.get 1 n * (w / memsize)))
      else
        errorf r "width %d of initialized data is not a multiple of memsize = %d"
          w memsize
    | Some i ->
      E.seq (init r (Types.Bits w) i) (fun inits ->
        let rcount = List.length inits in
        let lcount = Auxfuns.Option.get rcount n in
        if rcount > lcount then errorf r "too many initial values"
        else if rcount < lcount then errorf r "too few initial values"
        else E.Ok (InitializedData inits)))

⟨definitions of elaboration functions for data 397b⟩+≡ (395b) <397a 397c>
and init_size r = function
| A.DynSize -> E.Ok (None)
| A.NoSize -> E.Ok (Some 1)
| A.FixSize e ->
  E.seq (Elabexp.elab_con env e) (fun (bits, _) ->
    (try
      let size = Bits.S.to_int bits in
      if size >= 0 then E.Ok (Some size)
      else errorf r "negative count for initialized data"
    with Bits.Overflow -> errorf r "overflow computing size"))

⟨definitions of elaboration functions for data 397c⟩+≡ (395b) <397b>
and init r ty i =
  match i with
  | A.I (x,r) -> E.catch (error r env) (init r ty) x
  | A.InitExprs(es) ->
    E.seq (E.Raise.list (List.map (Elabexp.elab_link env) es)) (fun tes ->
      if List.for_all (fun (e, w) -> Types.Bits w == ty) tes then
        E.Ok tes
      else
        errorf r "type of an initial value does not match declared type %s"
          (Types.to_string ty))
  | A.InitStr s ->
    let to_texpr c = Reloc.of_const (Bits.U.of_int (Char.code c) 8), 8 in
    if ty == T.Bits 8 then

```

```

      E.Ok (Auxfuns.String.foldr (fun c l -> to_expr c :: l) s [])
    else
      errorf r "initialized string must specify type bits8"
  | A.InitUStr(s) ->
    if ty == T.Bits 16 then
      Impossible.unimp "Unicode"
    else
      errorf r "initialized Unicode string must specify type bits16" in

⟨nelab.ml 398a⟩+≡ (388c) <392b
  let rewrite = Obj.magic

```

O.10 front_nelab/simplify.nw

```

⟨front_nelab/simplify.ml 398b⟩≡
  ⟨simplify.ml 398g⟩

⟨front_nelab/simplify.mli 398c⟩≡
  ⟨simplify.mli 398d⟩

```

O.11 RTL Constant Folding

```

⟨simplify.mli 398d⟩≡ (398c) 398e▷
  val rtl:    Rtl.rtl -> Rtl.rtl
  val exp:    Rtl.exp -> Rtl.exp
  val bits:   Rtl.exp -> Bits.bits (* Error.ErrorExn, convenient function *)
  val link:   Rtl.exp -> Reloc.t
  val bool:   Rtl.exp -> bool      (* Error.ErrorExn, convenient function *)

⟨simplify.mli 398e⟩+≡ (398c) <398d 398f▷
  module Unsafe: sig
    val rtl:    Rtl.rtl -> Rtl.rtl
  end

⟨simplify.mli 398f⟩+≡ (398c) <398e
  val compile_time_ops: string list

```

O.11.1 Implementation

```

⟨simplify.ml 398g⟩≡ (398b) 399a▷
  open Nopoly

  module B0 = Bits.Ops
  module R  = Rtl
  module RP = Rtl.Private
  module RU = Rtlutil
  module Dn = Rtl.Dn
  module Up = Rtl.Up

  exception Error of string
  type reloc      = Reloc.t

  let error msg = raise (Error msg)
  let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

```

module ToString = struct
  let exp e = RU.ToString.exp (Up.exp e)
  let exp' e = RU.ToUnreadableString.exp (Up.exp e)
end

module Safe = struct
  <RTL operator implementations 400a>
  <traverse RTL 400b>
end

<simplify.ml 399a>+≡ (398b) <398g 399b>
module Unsafe = struct
  <unsafe RTL simplification 402a>
end

<simplify.ml 399b>+≡ (398b) <399a
  let rtl r = try Up.rtl (Safe.rtl (Dn.rtl r))
               with Error msg -> Impossible.impossible msg

  let exp e = try Up.exp (Safe.exp (Dn.exp e))
               with Error msg -> Impossible.impossible msg

  let rec location_category = function
    | RP.Mem _ -> "memory"
    | RP.Reg _ -> "a machine register"
    | RP.Var (name, _, _) | RP.Global (name, _, _) ->
        Printf.sprintf "register variable '%s'" name
    | RP.Slice (_, _, loc) -> location_category loc

  let bits e =
    let rec cvt = function
      | RP.Const (RP.Bits b) -> b
      | RP.Const (RP.Bool _) -> Error.error "a constant expression may not be a Boolean"
      | RP.Const (RP.Link (s, _, _)) ->
          Error.errorf "constant %s not resolvable until link time" s#original_text
      | RP.Const (RP.Diff (_, _)) ->
          Error.errorf "difference of two constants not resolvable until link time"
      | RP.Const (RP.Late (s, _)) ->
          Error.errorf "late compile-time constant %s is not a constant yet" s
      | RP.Fetch (l, _) ->
          Error.errorf "a constant expression may not refer to %s"
            (location_category l)
      | RP.App ((opr, _), es) ->
          List.iter (fun e -> ignore (cvt e)) es;
          Error.errorf "cannot evaluate operator %%%s at compile time" opr in
    cvt (Safe.exp (Dn.exp e))

  let bool e = match Safe.exp (Dn.exp e) with
    | RP.Const(RP.Bool(b)) -> b
    | _ -> Error.error "not a constant condition"

  let mklink kind s w = Up.exp (RP.Const (RP.Link(s, kind, w)))

  let link e =
    let rec const c = match c with
      | RP.Bits b -> Reloc.of_const b
      | RP.Link(l,kind,w) -> Reloc.of_sym (l, mklink kind) w
      | RP.Diff(c1,c2) -> Reloc.sub (const c1) (const c2)
      | _ -> Error.errorf "Bad link-time constant %s" (Rtlutil.ToString.const c) in
    let rec exp e = match e with

```



```

| RP.App(("add",_),[x; y]) -> Reloc.add (exp x) (exp y)
| RP.App(("sub",_),[x; y]) -> Reloc.sub (exp x) (exp y)
| RP.Const c -> const c
| _ -> Error.errorf "Bad link-time constant expression %s"
      (Rtlutil.ToString.exp (Up.exp e)) in
exp (Safe.exp (Dn.exp e))

```

```

let compile_time_ops =
  [ "add"; "and"; "com"; "divu"; "eq"; "ge"; "geu"; "gt"; "gtu"; "le"; "leu";
    "lobits"; "lt"; "ltu"; "mul"; "ne"; "neg"; "or"; "shl"; "shra";
    "shrl"; " sub"; "sx"; "xor"; "zx"]

```

O.11.2 RTL Traversal

⟨RTL operator implementations 400a⟩ ≡ (398g) 403a▷

```

let default op w args = RP.App((op,w),args)

```

⟨traverse RTL 400b⟩ ≡ (398g)

```

let app o w args = match o with
| "NaN"      -> nan w args
| "add"      -> add w args
| "and"      -> binop o w B0.and' args
| "bit"      -> bit  w args
| "bool"     -> bool w args
| "borrow"   -> default o w args
| "carry"    -> default o w args
| "com"      -> unop o w B0.com args
| "conjoin"  -> conjoin w args
| "disjoin"  -> disjoin w args
| "div"      -> default o w args
| "divu"     -> binop o w B0.divu args
| "eq"       -> cmp_bits o w B0.eq args
| "f2f"      -> default o w args
| "f2i"      -> default o w args
| "fabs"     -> default o w args
| "fadd"     -> default o w args
| "false"    -> false'   w args
| "fcmp"     -> default o w args
| "fdiv"     -> default o w args
| "feq"      -> default o w args
| "fge"      -> default o w args
| "fgt"      -> default o w args
| "fle"      -> default o w args
| "float_eq" -> default o w args
| "float_gt" -> default o w args
| "float_lt" -> default o w args
| "flt"      -> default o w args
| "fmul"     -> default o w args
| "fmulx"    -> default o w args
| "fne"      -> default o w args
| "fneg"     -> default o w args
| "forordered" -> default o w args
| "fsqrt"    -> default o w args
| "fsub"     -> default o w args
| "funordered" -> default o w args
| "ge"       -> cmp_bits o w (fun x y -> B0.lt  y x) args
| "geu"      -> cmp_bits o w (fun x y -> B0.ltu y x) args
| "gt"       -> cmp_bits o w B0.gt args
| "gtu"      -> cmp_bits o w B0.gtu args

```

```

| "i2f"      -> default o w args
| "le"       -> cmp_bits o w (fun x y -> B0.gt y x) args
| "leu"      -> cmp_bits o w (fun x y -> B0.gtu y x) args
| "lobits"   -> lobits w args
| "lt"       -> cmp_bits o w B0.lt args
| "ltu"      -> cmp_bits o w B0.ltu args
| "minf"     -> minf w args
| "mod"      -> default o w args
| "modu"     -> default o w args
| "mul"      -> binop o w B0.mul args
| "mulux"    -> default o w args
| "mulx"     -> default o w args
| "mzero"    -> mzero w args
| "ne"       -> cmp_bits o w B0.ne args
| "neg"      -> unop o w B0.neg args
| "not"      -> not' w args
| "or"       -> binop o w B0.or' args
| "pinf"     -> pinf w args
| "popcnt"   -> default o w args
| "pzero"    -> pzero w args
| "quot"     -> default o w args
| "rem"      -> default o w args
| "rotr"     -> default o w args
| "round_down" -> round_down w args
| "round_nearest" -> round_nearest w args
| "round_up" -> round_up w args
| "round_zero" -> round_zero w args
| "shl"      -> binop o w B0.shl args
|            (* likely to break machine invariant on PPC? *)

| "shra"     -> binop o w B0.shra args
| "shrl"     -> binop o w B0.shrl args
| "sub"      -> sub w args
| "sx"       -> sx w args
| "true"     -> true' w args
| "unordered" -> default o w args
| "xor"      -> binop o w B0.xor args
| "zx"       -> zx w args
| "bitExtract" -> default o w args
| "bitInsert" -> default o w args
| "bitTransfer" -> default o w args
| "x86_carrybit" -> x86_carrybit w args
| o          -> default o w args

```

```

let rec exp e =
  match e with
  | RP.Const _ as c      -> c
  | RP.Fetch (l, w)      -> RP.Fetch (loc l, w)
  | RP.App ((o,w),args)  -> app o w (List.map exp args)

```

```

and loc = function
| RP.Mem(sp, w, e, ass) -> RP.Mem(sp, w, exp e, ass)
| RP.Slice(n,lsb,l)    -> (
  <rewrite slice l@lsb:n 405b>
)
| RP.Reg(sp, i, w) as v -> v
| RP.Var (s,i,w) as v -> v
| RP.Global(s,i,w) as v -> v

```

```

let effect = function
  | RP.Store(l,e,w)      -> RP.Store(loc l, exp e, w)
  | RP.Kill(l)           -> RP.Kill(loc l)

let guarded (e, eff)     = (exp e, effect eff)
let rtl (RP.Rtl(es))     = RP.Rtl(List.map guarded es)

```

O.11.3 Unsafe Simplification

```

<unsafe RTL simplification 402a>≡ (399a) 402b▷
let zero w = RP.Const(RP.Bits (Bits.U.of_int 0 w))
let reg_offset = function
  | RP.App(("add", _), [RP.Fetch(RP.Reg r,_); RP.Const _ as k]) -> Some (r,k)
  | RP.Fetch(RP.Reg r, w) -> Some (r, zero w)
  | _ -> None

<unsafe RTL simplification 402b>+≡ (399a) <402a
let rec app o w args = match Safe.app o w args with
  | RP.App (("ne"|"eq"|"ltu"|"gtu"|"lt"|"gt" as op, w), [left;right]) as x ->
    ( match reg_offset left, reg_offset right with
      | Some(r1,k1), Some(r2,k2) when Register.eq r1 r2 ->
        let r      = app op w [k1;k2] in
        let _dbg() = Printf.eprintf "Simplify2.Unsafe.app: %s '%s' %s = %s\n"
          (ToString.exp k1) op (ToString.exp k2) (ToString.exp r) in
        r
      | _ -> x
    )
  | x -> x

let rec exp = function
  | RP.Const _ as c      -> c
  | RP.Fetch (l, w)      -> RP.Fetch (loc l, w)
  | RP.App ((o,w),args)  -> app o w (List.map exp args)

and loc = function
  | RP.Mem(sp, w, e, ass) -> RP.Mem(sp, w, exp e, ass)
  | RP.Slice(w,i,l)       -> RP.Slice(w, i, loc l)
  | RP.Reg(sp, i, w) as v -> v
  | RP.Var (s,i,w) as v -> v
  | RP.Global(s,i,w) as v -> v

let effect = Safe.effect

let guarded (e, eff)     = (exp e, effect eff)
let rtl' (RP.Rtl(es))    =
  let add_guarded e es = match guarded e with
    | RP.Const(RP.Bool false), _ -> es
    | e -> e :: es in
  RP.Rtl(List.fold_right add_guarded es [])

(*let rtl' (RP.Rtl es) = RP.Rtl (List.map guarded es)*)

let rtl r = try Up.rtl (rtl' (Dn.rtl r))
  with Error msg -> Impossible.impossible msg

```

O.11.4 RTL Operator Implementations

⟨RTL operator implementations 403a⟩+≡ (398g) <400a 403b>
let to_bool b = RP.Const (RP.Bool b)
let const k = RP.Const (RP.Bits k)

⟨RTL operator implementations 403b⟩+≡ (398g) <403a 403c>
let false' w args = RP.Const(RP.Bool false)
let true' w args = RP.Const(RP.Bool true)

⟨RTL operator implementations 403c⟩+≡ (398g) <403b 403d>
let round_down w args = RP.Const(RP.Bits(Bits.U.of_int 1 2))
let round_up w args = RP.Const(RP.Bits(Bits.U.of_int 2 2))
let round_nearest w args = RP.Const(RP.Bits(Bits.U.of_int 0 2))
let round_zero w args = RP.Const(RP.Bits(Bits.U.of_int 3 2))

⟨RTL operator implementations 403d⟩+≡ (398g) <403c 403e>
let rec add ws args =
 let w = match ws with [w] -> w | _ -> Impossible.impossible "ill-typed add" in
 match args with
 | [RP.App(("add", ws'), [x; RP.Const (RP.Bits k)]); RP.Const (RP.Bits k')] ->
 assert (ws==ws');
 add ws [x; RP.Const (RP.Bits (Bits.Ops.add k k'))]
 | [RP.Const (RP.Bits x); RP.Const (RP.Bits y)] ->
 RP.Const (RP.Bits (Bits.Ops.add x y))
 | [x; RP.Const (RP.Bits y)] when Bits.Ops.eq (Bits.U.of_int 0 w) y -> x
 | args -> default "add" ws args

let sub ws args =
 let rec remove_one y xs = match xs with
 | [] -> Impossible.impossible "did not find removable expression"
 | x :: xs when RU.Eq.exp y x -> xs
 | x :: xs -> x :: remove_one y xs in
 let w = match ws with [w] -> w | _ -> Impossible.impossible "ill-typed sub" in
 match args with
 | [RP.Const(RP.Bits x); RP.Const(RP.Bits y)] -> const (Bits.Ops.sub x y)
 | [RP.App(("add",ws'),args); e] when List.mem e args ->
 (match remove_one e args with
 | _::_:_ as rst -> add ws' rst
 | [a] -> a
 | [] -> RP.Const(RP.Bits(Bits.zero w))
)
 | [x; y] when RU.Eq.exp x y -> RP.Const(RP.Bits(Bits.zero w))
 | args -> default "sub" ws args

⟨RTL operator implementations 403e⟩+≡ (398g) <403d 403f>
let binop name w op = function
 | [RP.Const(RP.Bits x); RP.Const(RP.Bits y)] -> const (op x y)
 | args -> default name w args

⟨RTL operator implementations 403f⟩+≡ (398g) <403e 404a>
let disjoin w = function
 | [RP.Const(RP.Bool true); _] -> RP.Const(RP.Bool true)
 | [_; RP.Const(RP.Bool true)] -> RP.Const(RP.Bool true)
 | [RP.Const(RP.Bool false); p] -> p
 | [p; RP.Const(RP.Bool false)] -> p
 | args -> default "disjoin" w args

let conjoin w = function
 | [RP.Const(RP.Bool false); _] -> RP.Const(RP.Bool false)

```

| [_; RP.Const(RP.Bool false)] -> RP.Const(RP.Bool false)
| [RP.Const(RP.Bool true); p] -> p
| [p; RP.Const(RP.Bool true)] -> p
| args -> default "conjoin" w args

⟨RTL operator implementations 404a⟩+≡ (398g) <403f 404b>
let not' w = function
| [RP.Const(RP.Bool b)] -> RP.Const(RP.Bool (not b))
| args -> default "not" w args

⟨RTL operator implementations 404b⟩+≡ (398g) <404a 404c>
let cmp_bits name w op = function
| [RP.Const(RP.Bits x); RP.Const(RP.Bits y)] -> to_bool (op x y)
| args -> default name w args

⟨RTL operator implementations 404c⟩+≡ (398g) <404b 404d>
let unop name w op = function
| [RP.Const(RP.Bits x)] -> const (op x)
| args -> default name w args

⟨RTL operator implementations 404d⟩+≡ (398g) <404c 404e>

⟨RTL operator implementations 404e⟩+≡ (398g) <404d 404f>
let or' ws args =
let w = match ws with [w] -> w | _ -> Impossible.impossible "ill-typed or" in
match args with
| [RP.Const (RP.Bits x); RP.Const (RP.Bits y)] ->
RP.Const (RP.Bits (Bits.Ops.or' x y))
| [x; RP.Const (RP.Bits y)] when Bits.Ops.eq (Bits.U.of_int 0 w) y -> x
| [RP.Const (RP.Bits x); y] when Bits.Ops.eq (Bits.U.of_int 0 w) x -> y
| args -> default "or" ws args

let and' ws args =
let w = match ws with [w] -> w | _ -> Impossible.impossible "ill-typed and" in
match args with
| [RP.Const (RP.Bits x); RP.Const (RP.Bits y)] ->
RP.Const (RP.Bits (Bits.Ops.and' x y))
| [x; RP.Const (RP.Bits y) as zero] when Bits.Ops.eq (Bits.U.of_int 0 w) y -> zero
| [RP.Const (RP.Bits x) as zero; y] when Bits.Ops.eq (Bits.U.of_int 0 w) x -> zero
| args -> default "and" ws args

⟨RTL operator implementations 404f⟩+≡ (398g) <404e 405a>
let rec zx w args = match w, args with
| [n;w],[RP.Const (RP.Bits x)] -> const (Bits.Ops.zx w x)
| [n;w],[RP.App (("zx", [n';w']), [x])] -> assert (w' = n); zx [n';w] [x]
| [n;w], [x] when n = w -> x
| _ -> default "zx" w args

let rec sx w args = match (w,args) with
| [i;j],[RP.Const(RP.Bits x)] -> const (Bits.Ops.sx j x)
| [n;w],[RP.App (("sx", [n';w']), [x])] -> assert (w' = n); sx [n';w] [x]
| [n;w],[RP.App (("zx", [n';w']), [x])] when w' > n' -> assert (w' = n); zx [n';w] [x]
| [i;j], [x] when i = j -> x
| _ -> default "sx" w args

```

```

<RTL operator implementations 405a>+≡ (398g) <404f 407e>
let rec lobits ws args = match (ws,args) with
| [i;j],[RP.Const(RP.Bits x)] -> const (Bits.Ops.lobits j x)
<cases for rewriting aggregates in big-endian or little-endian memory 405c>
<cases for rewriting aggregates in big-endian or little-endian registers 406e>
| [i;j], [x] when i = j -> x
| _ -> default "lobits" ws args

let () = Debug.register "simp-lobits" "simplification of %lobits expressions"

let lobits =
  if Debug.on "simp-lobits" then
    (fun ws args ->
      let e = RP.App (("lobits", ws), args) in
      let answer = lobits ws args in
      Printf.eprintf "simplified %s -> %s\n" (ToString.exp e) (ToString.exp answer);
      answer)
  else
    lobits

<rewrite slice l@lsb:n 405b>≡ (400b)
match loc l with
|
  <big-endian memory location 406d>
  when Cell.divides mcell n && Cell.divides mcell lsb ->
    let w = Cell.to_width mcell ct in
    let R.C offset = Cell.to_count mcell (w - n - lsb) in
    let addr = Dn.exp (RU.addk (RU.Width.exp' addr) (Up.exp addr) offset) in
    let assn = Alignment.gcd assn offset in
    RP.Mem (mspace, Cell.to_count mcell n, addr, assn)
|
  <little-endian memory location 406b>
  when Cell.divides mcell n && Cell.divides mcell lsb ->
    let R.C offset = Cell.to_count mcell lsb in
    let addr = Dn.exp (RU.addk (RU.Width.exp' addr) (Up.exp addr) offset) in
    let assn = Alignment.gcd assn offset in
    RP.Mem (mspace, Cell.to_count mcell n, addr, assn)
|
  <big-endian register location 407d>
  when Cell.divides rcell n && Cell.divides rcell lsb ->
    let w = Cell.to_width rcell count in
    let R.C offset = Cell.to_count rcell (w - n - lsb) in
    RP.Reg (rspace, index + offset, Cell.to_count rcell n)
|
  <little-endian register location 407c>
  when Cell.divides rcell n && Cell.divides rcell lsb ->
    let R.C offset = Cell.to_count rcell lsb in
    RP.Reg (rspace, index + offset, Cell.to_count rcell n)
| l -> RP.Slice(n,lsb,l)

<cases for rewriting aggregates in big-endian or little-endian memory 405c>≡ (405a)
| [w;n],[
  <little-endian memory fetch 406a>
  ] when Cell.divides mcell n ->
  RP.Fetch (RP.Mem (mspace, Cell.to_count mcell n, addr, assn), n)
| [w;n],[
  <big-endian memory fetch 406c>
  ] when Cell.divides mcell (w - n) ->
  let R.C offset = Cell.to_count mcell (w - n) in
  let addr = Dn.exp (RU.addk (RU.Width.exp' addr) (Up.exp addr) offset) in

```

```

    let assn = Alignment.gcd assn offset in
    RP.Fetch (RP.Mem (mspace, Cell.to_count mcell n, addr, assn), n)
| [w;n],[RP.App(("shrl", [_]),
    [
      <little-endian memory fetch 406a>
      ; RP.Const(RP.Bits b)]]] ->
let shamt = Bits.U.to_int b in
if Cell.divides mcell shamt then
  let R.C offset = Cell.to_count mcell shamt in
  let addr = Dn.exp (RU.addk (RU.Width.exp' addr) (Up.exp addr) offset) in
  let assn = Alignment.gcd assn offset in
  lobits [w-shamt; n]
  [RP.Fetch (RP.Mem (mspace, Cell.to_count mcell (w-shamt), addr, assn),w-shamt)]
else
  default "lobits" ws args
| [w;n],[RP.App(("shrl", [_]), [
  <big-endian memory fetch 406c>
  ; RP.Const(RP.Bits b)]]] ->
let shamt = Bits.U.to_int b in
if Cell.divides mcell shamt then
  lobits [w-shamt; n]
  [RP.Fetch (RP.Mem (mspace, Cell.to_count mcell (w-shamt), addr, assn),w-shamt)]
else
  default "lobits" ws args

<little-endian memory fetch 406a>≡ (405)
RP.Fetch(
  <little-endian memory location 406b>
  , w')

<little-endian memory location 406b>≡ (406a 405b)
RP.Mem ((sp, Rtl.LittleEndian, mcell) as mspace, ct, addr, assn)

<big-endian memory fetch 406c>≡ (405)
RP.Fetch(
  <big-endian memory location 406d>
  , w'')

<big-endian memory location 406d>≡ (406c 405b)
RP.Mem ((sp, Rtl.BigEndian, mcell) as mspace, ct, addr, assn)

<cases for rewriting aggregates in big-endian or little-endian registers 406e>≡ (405a)
| [w;n],[
  <little-endian register fetch 407a>
] when Cell.divides rcell n ->
  RP.Fetch (RP.Reg (rspace, index, Cell.to_count rcell n), n)
| [w;n],[
  <big-endian register fetch 407b>
] when Cell.divides rcell (w - n) ->
  let R.C offset = Cell.to_count rcell (w - n) in
  let index      = index + offset in
  RP.Fetch (RP.Reg (rspace, index, Cell.to_count rcell n), n)
| [w;n],[RP.App(("shrl", [_]),
  [
    <little-endian register fetch 407a>
    ; RP.Const(RP.Bits b)]]] ->
let shamt = Bits.U.to_int b in
if Cell.divides rcell shamt then
  let R.C offset = Cell.to_count rcell shamt in
  let index      = index + offset in

```

```

    lobits [w-shamt; n]
    [RP.Fetch (RP.Reg (rspace, index, Cell.to_count rcell (w-shamt)), w-shamt)]
else
    default "lobits" ws args
| [w;n],[RP.App(("shr1", [_]), [
    ⟨big-endian register fetch 407b⟩
    ; RP.Const(RP.Bits b)]]] ->
let shamt = Bits.U.to_int b in
if Cell.divides rcell shamt then
    lobits [w-shamt; n]
    [RP.Fetch (RP.Reg (rspace, index, Cell.to_count rcell (w-shamt)), w-shamt)]
else
    default "lobits" ws args

⟨little-endian register fetch 407a⟩≡ (406)
RP.Fetch(
    ⟨little-endian register location 407c⟩
    , w'')

⟨big-endian register fetch 407b⟩≡ (406)
RP.Fetch(
    ⟨big-endian register location 407d⟩
    , w'')

⟨little-endian register location 407c⟩≡ (407a 405b)
RP.Reg ((sp, Rtl.LittleEndian, rcell) as rspace, index, count)

⟨big-endian register location 407d⟩≡ (407b 405b)
RP.Reg ((sp, Rtl.BigEndian, rcell) as rspace, index, count)

⟨RTL operator implementations 407e⟩+≡ (398g) <405a 407f>
let unsigned n w = RP.Const (RP.Bits (Bits.U.of_int n w))
let null = function [] -> true | _ :: _ -> false

let zero1 = Bits.U.of_int 0 1
let one1 = Bits.U.of_int 1 1
let bool w args = assert (null w); match args with
| [RP.Const (RP.Bits x)] -> RP.Const (RP.Bool (Bits.Ops.ne x zero1))
| [RP.App ("lobits", [w; 1]), [x]] ->
    RP.App ("ne", [w]), [RP.App ("and", [w]), [x; unsigned 1 w]]; unsigned 0 w]
| _ -> default "bool" w args

let bit w args = assert (null w); match args with
| [RP.Const (RP.Bool b)] -> RP.Const (RP.Bits (if b then one1 else zero1))
| _ -> default "bit" w args

⟨RTL operator implementations 407f⟩+≡ (398g) <407e 408a>
let pzero w args = assert (null args); match w with
| [(32|64) as w] -> const (Bits.zero w)
| _ -> default "pzero" w args

let mzero w args = assert (null args); match w with
| [(32|64) as w] -> const (B0.shl (Bits.U.of_int 1 w) (Bits.U.of_int (w-1) w))
| _ -> default "mzero" w args

let pinf w args = assert (null args); match w with
| [32] -> const (B0.shl (Bits.U.of_int 0xff 32) (Bits.U.of_int 23 32))
| [64] -> const (B0.shl (Bits.U.of_int 0x7ff 64) (Bits.U.of_int 52 64))
| _ -> default "pinf" w args

```



```

let minf w args = assert (null args); match w with
| [32|64] -> or' w [pinf w args; mzero w args]
| _ -> default "minf" w args

let nan w args =
  let swidth = function
    | 32 -> 23
    | 64 -> 52
    | _ -> Impossible.unimp "NaN at width other than 32 or 64" in
  match w, args with
  | [n;w], [x] when n = swidth w -> or' [w] [pinf [w] []; zx [n; w] [x]]
  | [n;w], [x] -> impossf "significand in NaN has width %d; should be %d" n (swidth w)
  | _ -> Impossible.impossible "ill-formed NaN"

```

O.11.5 Machine-dependent simplifications

<RTL operator implementations 408a>+≡ (398g) <407f

```

let x86_carrybit w args = match args with
| [RP.App (("x86_setcarry", _), [_; e])] -> e
| _ -> default "x86_carrybit" w args

```

O.12 front_nelab/topsort.nw

*<front_nelab/topsort.ml 408b>≡
<topsort.ml 409a>*

*<front_nelab/topsort.mli 408c>≡
<topsort.mli 408f>*

O.13 Topological Sort

O.13.1 Interface

<signature S 408d>≡ (409a 408f)

```

module type S = sig
  type decl
  exception Cycle of decl list
  val sort: decl list -> decl list (* raises Cycle *)
end

```

<signature Sortable 408e>≡ (409a 408f)

```

module type Sortable = sig
  type decl
  val defines : decl -> string list
  val uses : decl -> string list
end

```

<topsort.mli 408f>≡ (408c)

```

<signature S 408d>
<signature Sortable 408e>

module Make (S: Sortable) : (S with type decl = S.decl)

```

O.13.2 Implementation

```

<topsort.ml 409a>≡ (408b)
  open Nopoly

  <signature S 408d>
  <signature Sortable 408e>

  module ID      = struct type t=string let compare=compares end
  module IDMap   = Map.Make(ID)

  module Make (S: Sortable) = struct
    type decl = S.decl
    exception Cycle of decl list

    <functions 409b>
  end

  <functions 409b>≡ (409a) 409c>
  let declmap decls =
    let find key map = try IDMap.find key map with Not_found -> [] in
    let add d map key = IDMap.add key (d :: find key map) map in
    let add' map d = List.fold_left (add d) map (S.defines d) in
    List.fold_left add' IDMap.empty decls

  <functions 409c>+≡ (409a) <409b 409d>
  let succ map x =
    let find map n = try IDMap.find n map with Not_found -> [] in
    List.concat (List.map (find map) (S.uses x))

  <functions 409d>+≡ (409a) <409c
  let sort decls =
    let m = declmap decls in
    let nv x (v,c) = x::v ,c in
    let rec sort todo path (visited, cycles) =
      match todo with
      | [] -> visited, cycles
      | x::xs ->
          let vc = if List.mem x path then visited, x::cycles
                  else if List.mem x visited then visited, cycles
                  else nv x (sort (succ m x) (x::path) (visited,cycles))
          in
          sort xs path vc
    in
    match sort decls [] ([],[]) with
    | dd, [] -> List.rev dd
    | dd, cc -> raise (Cycle cc)

```

O.13.3 Test

```

<topsorttest.ml 409e>≡ 410a>
  module D = struct
    type decl = string list * string list

    let defines (x,_) = x
    let uses (_,y) = y
  end

  module DSort = Topsort.Make(D)

```

```

(*   DEFINED                               USED BY DEFINITION *)
let regular =
  [ ["wake"]           , ["shower"; "eat" ]
    ; ["shower"]       , ["dress"]
    ; ["dress"]        , ["go"]
    ; ["eat"]          , ["wash-up"]
    ; ["wash-up"]      , ["go"]
    ; ["go"]           , []
  ]

<topsortttest.ml 410a>+≡ <409e 410b>
let multi_lhs1 =
  [ ["wake"]           , ["shower"; "eat" ]
    ; ["shower"; "bath"] , ["dress"]
    ; ["dress"]        , ["go"]
    ; ["eat"; "drink"]  , ["wash-up"]
    ; ["wash-up"]      , ["go"]
    ; ["go"]           , []
  ]

<topsortttest.ml 410b>+≡ <410a 410c>
let multi_lhs2 =
  [ ["wake"]           , ["shower"; "eat" ]
    ; ["shower"; "bath"] , ["dress"]
    ; ["bath"]          , ["dress"]
    ; ["dress"]        , ["go"]
    ; ["eat"; "drink"]  , ["wash-up"]
    ; ["wash-up"]      , ["go"]
    ; ["go"]           , []
    ; ["go"]           , []
  ]

<topsortttest.ml 410c>+≡ <410b 410d>
let undefs =
  [ ["wake"]           , ["shower"; "eat" ]
    ; ["shower"; "bath"] , ["dress"]
    ; ["bath"]          , ["dress"]
    ; ["dress"]        , ["go"]
    ; ["eat"; "drink"]  , ["wash-up"]
    ; ["wash-up"]      , ["go"]
  ]

<topsortttest.ml 410d>+≡ <410c>
let cycle1 =
  [ ["wake"]           , ["shower"; "eat" ]
    ; ["shower"; "bath"] , ["dress"]
    ; ["bath"]          , ["dress"]
    ; ["dress"]        , ["go"; "wake"]
    ; ["eat"; "drink"]  , ["wash-up"]
    ; ["wash-up"]      , ["go"]
    ; ["go"]           , []
    ; ["go"]           , []
  ]

```

Appendix P

target

P.1 target/automaton.nw

```
⟨front_target/automaton.ml 411a⟩≡  
  ⟨automaton.ml 413l⟩  
  
⟨front_target/automaton.mli 411b⟩≡  
  ⟨automaton.mli 411c⟩
```

P.2 Automaton-Based Slot Allocation

```
⟨automaton.mli 411c⟩≡ (411b)  
  ⟨abstract types 411d⟩  
  ⟨exposed types(automaton.nw) 411e⟩  
  ⟨registration types 413h⟩  
  ⟨exported values 411f⟩
```

P.2.1 Standard kinds

P.2.2 Observer-Mutator interface

```
⟨abstract types 411d⟩≡ (411c) 412e▷  
  type t  
  
⟨exposed types(automaton.nw) 411e⟩≡ (413l 411c) 411g▷  
  type result =  
    { overflow      : Block.t  
    ; regs_used    : Register.Set.t  
    ; mems_used    : Rtl.loc list  
    ; align_state  : int    (* final alignment state of overflow block *)  
    }  
  
⟨exported values 411f⟩≡ (411c) 412a▷  
  val allocate : t -> width:int -> kind:string -> align:int -> loc  
  val freeze   : t -> result  
  
⟨exposed types(automaton.nw) 411g⟩+≡ (413l 411c) <411e 412c▷  
  type width = int  
  type kind  = string  
  type loc = Rtlutil.aloc =  
    { fetch  : width -> Rtl.exp  
    ; store  : Rtl.exp -> width -> Rtl.rtl  
    }
```

<exported values 412a>+≡ (411c) <411f 412b>
 val fetch : loc -> width -> Rtl.exp
 val store : loc -> Rtl.exp -> width -> Rtl.rtl

<exported values 412b>+≡ (411c) <412a 412d>
 val of_loc : Rtl.loc -> loc

P.2.3 Constructor interface

<exposed types(automaton.nw) 412c>+≡ (413l 411c) <411g 413a>
 type methods =
 { allocate : width: int -> alignment: int -> kind: string -> loc
 ; freeze : Register.Set.t -> Rtl.loc list -> result
 }

<exported values 412d>+≡ (411c) <412b 412f>
 val at : Rtl.space -> start:Rtl.exp -> stage -> t
 val of_methods : methods -> t

<abstract types 412e>+≡ (411c) <411d 412m>
 type stage

<exported values 412f>+≡ (411c) <412d 412g>
 val (*>) : stage -> stage -> stage

<exported values 412g>+≡ (411c) <412f 412h>
 val wrap : (methods -> methods) -> stage

<exported values 412h>+≡ (411c) <412g 412i>
 val overflow : growth:Memalloc.growth -> max_alignment:int -> stage

<exported values 412i>+≡ (411c) <412h 412j>
 val widths : int list -> stage

<exported values 412j>+≡ (411c) <412i 412k>
 val widen : (int -> int) -> stage

<exported values 412k>+≡ (411c) <412j 412l>
 val align_to : (int -> int) -> stage

<exported values 412l>+≡ (411c) <412k 412n>
 val useregs : Register.t list -> bool -> stage

<abstract types 412m>+≡ (411c) <412e
 type counter = string
 type counterenv

<exported values 412n>+≡ (411c) <412l 412o>
 val bitcounter : counter -> stage
 val regs_by_bits: counter -> Register.t list -> bool -> stage

 val argcounter : counter -> stage
 val regs_by_args: counter -> Register.t list -> bool -> stage

 val pad : counter -> stage

<exported values 412o>+≡ (411c) <412n 413b>
 val postprocess : stage -> (result -> result) -> stage

```

⟨exposed types(automaton.nw) 413a⟩+≡ (413l 411c) <412c
  type choice_predicate = int -> string -> counterenv -> bool

⟨exported values 413b⟩+≡ (411c) <412o 413c>
  val choice : (choice_predicate * stage) list -> stage

⟨exported values 413c⟩+≡ (411c) <413b 413d>
  val counter_is : counter -> (int -> bool) -> choice_predicate
  val is_kind : kind -> choice_predicate
  val is_width : int -> choice_predicate
  val is_any : choice_predicate

⟨exported values 413d⟩+≡ (411c) <413c 413e>
  val as_stage : t -> stage

⟨exported values 413e⟩+≡ (411c) <413d 413f>
  val first_choice : (choice_predicate * stage) list -> stage

⟨exported values 413f⟩+≡ (411c) <413e 413g>
  val unit : stage

⟨exported values 413g⟩+≡ (411c) <413f 413i>
  val debug : counter -> (int -> string -> int -> int -> unit) -> stage

⟨registration types 413h⟩≡ (411c)
  type cc_spec = { call : stage; results : stage; cutto : stage }
  type cc_specs = (string * cc_spec) list

⟨exported values 413i⟩+≡ (411c) <413g
  val init_cc : cc_specs

```

P.3 Implementation

```

⟨hidden types 413j⟩≡ (413l)
  type counter = string
  type counterenv = string -> int ref option

⟨hidden types that refer to methods 413k⟩≡ (413l)
  type t = methods
  type implementation = I of (start:Rtl.exp -> space:Rtl.space -> ctrs:counterenv -> t)
  type stage = S of (implementation -> implementation)

⟨automaton.ml 413l⟩≡ (411a) 414a>
  open Nopoly

⟨hidden types 413j⟩
⟨exposed types(automaton.nw) 411e⟩
⟨hidden types that refer to methods 413k⟩

module R = Rtl
module RS = Register.Set
module B = Block
module M = Memalloc
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let unimp = Impossible.unimp

```

```

⟨automaton.ml 414a⟩+≡ (411a) <413l 414b>
  let ( * ) (S f1) (S f2) = S (fun rhs -> f1 (f2 rhs))

  let pipeline_end = I (
    fun ~start ~space ~ctr ->
      let block = Block.at ~base:start ~size:0 ~alignment:1 in
      { allocate = (fun ~width ~alignment ~kind -> impossf
        "fell off end of pipeline -- multiple return values from C?")
      ; freeze =
        (fun regs mems ->
          {overflow = block; regs_used = regs; mems_used = mems; align_state = 0})
      }
  )

```

```

⟨automaton.ml 414b⟩+≡ (411a) <414a 414c>
  let at space ~start (S s) =
    let I f = s pipeline_end in
    f ~start ~space ~ctr:(fun _ -> None)

  let of_methods t = t
  let allocate t ~width ~kind ~align = t.allocate width align kind
  let freeze t = t.freeze Register.Set.empty []

  let fetch loc = loc.fetch
  let store loc = loc.store
  let of_loc loc = { fetch = R.fetch loc; store = R.store loc }

```

P.3.1 Wrapping etc.

```

⟨automaton.ml 414c⟩+≡ (411a) <414b 414d>
  let wrap f = S (fun (I next) -> I (fun ~start ~space ~ctr ->
    f (next ~start ~space ~ctr)))

  let swrap f =
    S (fun (I next) -> I (fun ~start ~space ~ctr ->
      f (next ~start ~space ~ctr) ~start ~space ~ctr))

  let wrap_choice f choices = S (fun (I next) ->
    let apply (I f) = f in
    let downchoice c ~start ~space ~ctr =
      fun meths -> apply (c (I (fun ~start ~space ~ctr -> meths))) ~start ~space ~ctr in
    I (fun ~start ~space ~ctr ->
      let choices = List.map (fun (p,(S c)) ->
        (p, downchoice c ~start ~space ~ctr))
        choices
      in f choices (next ~start ~space ~ctr) ~ctr))

```

```

⟨automaton.ml 414d⟩+≡ (411a) <414c 415c>
  type state = { mutable mem : M.t; mutable frozen : bool
    ; mutable mems_list : R.loc list }

  let overflow ~growth ~max_alignment next =
    fun ~start ~space:((_, byteorder, memsize) as space) ~ctr ->
      let mem = M.at start growth in
      let state = { mem = mem; frozen = false; mems_list = [] } in
      ⟨functions allocate and freeze for the overflow block 415a⟩
      { allocate = allocate; freeze = freeze }

  let overflow ~growth ~max_alignment =
    swrap (overflow ~growth ~max_alignment)

```

```

⟨functions allocate and freeze for the overflow block 415a⟩≡ (414d) 415b▷
let allocate ~width ~alignment ~kind =
  if not (Cell.divides memsize width) then
    impossf "width %d not multiple of memsize %d" width (Cell.size memsize);
  if max_alignment mod alignment <> 0 then
    impossf "max alignment %d not multiple of alignment %d" max_alignment alignment;
  if state.frozen then impossf "Allocation from a frozen overflow block";
  let (R.C c) as count = Cell.to_count memsize width in
  let mem = state.mem in
  let mem = M.align mem alignment in
  let mem' = M.allocate mem c in
  let addr = M.current (match growth with M.Up -> mem | M.Down -> mem') in
  let loc = (R.mem (R.aligned alignment) space count addr) in
  state.mems_list <- loc :: state.mems_list;
  state.mem <- mem';
  of_loc loc in

```

```

⟨functions allocate and freeze for the overflow block 415b⟩+≡ (414d) <415a
let freeze regs mems =
  state.frozen <- true;
  let mem = state.mem in
  let astate = M.num_allocated mem mod max_alignment in
  let mem = match growth with
    M.Up -> mem | M.Down -> M.align mem (M.alignment mem) in
  let block = M.freeze mem in
  { overflow = block; regs_used = regs; mems_used = state.mems_list @ mems
  ; align_state = astate } in

```

```

⟨automaton.ml 415c⟩+≡ (411a) <414d 415d▷
type allocator = width: int -> alignment: int -> kind: string -> loc
let txstage f = wrap (fun next ->
  { allocate = f next.allocate; freeze = next.freeze })
let _ = (txstage : (allocator -> allocator) -> stage)

```

```

⟨automaton.ml 415d⟩+≡ (411a) <415c 415e▷
let widths ws = txstage
  (fun nalloc ~width ~alignment ~kind ->
    if List.exists ((=) width) ws then nalloc width alignment kind
    else Unsupported.automaton_widths width)

```

```

⟨automaton.ml 415e⟩+≡ (411a) <415d 416a▷
let narrower =
  let narrow nopname wopname ~w ~n loc =
    let widen = R.opr wopname [n; w] in
    let narrow = R.opr nopname [w; n] in
    { fetch = (fun n -> R.app narrow [loc.fetch w])
    ; store = (fun e n -> loc.store (R.app widen [e]) w)
    } in
  function
  | "signed" -> narrow "lobits" "sx"
  | "float" -> narrow "f2f_implicit_round" "f2f_implicit_round"
  | "unsigned" | "address" | "" | _ -> narrow "lobits" "zx"
let narrowf = narrower "float"

let widen f = txstage
  (fun nalloc ~width:n ~alignment ~kind ->
    let w = f n in
    let loc = nalloc w alignment kind in
    if w = n then loc else narrower kind ~n ~w loc)

let align_to f =
  txstage (fun nalloc ~width:w ~alignment ~kind -> nalloc w (f w) kind)

```



```

(automaton.ml 416a)+≡ (411a) <415e 416b>
let choice choices next ~ctr =
  let add (p, stage) alternative =
    let follows_choice = { allocate = next.allocate
                          ; freeze   = alternative.freeze } in
    let choice = stage follows_choice in
    let alloc ~width ~alignment ~kind =
      let alloc = if p width kind ctrs then choice.allocate
                  else alternative.allocate in
      alloc width alignment kind in
    { allocate = alloc; freeze = choice.freeze } in
  List.fold_right add choices
  { allocate = (fun ~width -> impossf "fell off end of choice stage")
    ; freeze = next.freeze }
let choice = wrap_choice choice

let first_choice choices next ~ctr =
  let choices = List.map (fun (p, c) -> (p, c next)) choices in
  let myself = ref next in (* temporary *)
  let allocate ~width ~alignment ~kind =
    (!myself).allocate width alignment kind in
  let freeze regs mems = (!myself).freeze regs mems in
  let first_allocate ~width ~alignment ~kind =
    let choice =
      try snd (List.find (fun (p, _) -> p width kind ctrs) choices)
      with Not_found -> impossf "missing choice" in
    myself := choice;
    allocate width alignment kind in
  let () = try
    myself := { allocate = first_allocate
                ; freeze   = (snd (List.hd choices)).freeze }
  with Failure _ -> impossf "no first choice" in
  { allocate = allocate; freeze = freeze }
let first_choice = wrap_choice first_choice

let counter_is c f _ _ env = match env c with Some i -> f (!i) | None -> false
let is_kind    h' w h _ = h = $= h'
let is_width   w' w h _ = w = w'
let is_any     w h _ = true

(automaton.ml 416b)+≡ (411a) <416a 416c>
let wrap_counter f label =
  S (fun (I next) ->
    I (fun ~start ~space ~ctr ->
      let (ctr, ctrs) = (match ctrs label with
        | Some n -> (n, ctrs)
        | None   ->
          let n = ref 0 in
          (n, fun s -> if s = $= label then Some n else ctrs s)) in
      f space ctr (next ~start ~space ~ctr)))

let counter_of_width =
  let f ctr nalloc ~width ~alignment ~kind =
    let loc = nalloc ~width ~alignment ~kind in
    (ctr := !ctr + of_width width ; loc)
  in (fun ctr next -> { allocate = f ctr next.allocate; freeze = next.freeze })

(automaton.ml 416c)+≡ (411a) <416b 418c>
type regstate = { mutable used : Register.Set.t }

```

<definition of combine_loc 418a>

```
let reg_placer of_width agg n regs reserve next =
  let state = { used = RS.empty } in
  let getreg = function
  | []      -> raise (Failure "get register")
  | r :: rs -> (state.used <- RS.add r state.used; r) in

  let rec drop i regs = if i <= 0 then regs else match regs with
  | r :: rest -> drop (i - of_width (Register.width r)) rest
  | []        -> [] in

  let rec alloc ~width ~alignment ~kind =
    try
      let ((_, _, ms), _, c) as r = getreg (drop !n regs) in
      let w = Cell.to_width ms c in
      if w = width then
        let allocated = of_loc (R.reg r) in
        ( if reserve then ignore (next.allocate width alignment kind)
        ; allocated )
      else if w < width then match agg with
      | Some endianness ->
        let w' = width - w in
        let _ = n := !n + w in
        let ell = combine_loc endianness
          (of_loc (R.reg r)) w
          (alloc ~width:w' ~alignment ~kind) w'
        in ( n := !n - w ; ell )
      | None -> Unsupported.automaton_widen ~have:width ~want:w
      else unimp "auto-widening for register requests"
    with Failure _ -> next.allocate width alignment kind in
  let freeze regs mems = next.freeze (RS.union state.used regs) mems in
  { freeze = freeze; allocate = alloc }

let bitcounter = counter (fun w -> w)
let regs_by_bits = reg_placer (fun w -> w)

let useregs regs reserve =
  swap (fun next ~start ~space(_, _, byteorder, _) ~ctr -> let c = ref 0 in
    (bitcounter c) ((regs_by_bits (Some byteorder) c regs reserve) next))

let bitcounter = wrap_counter (fun _ -> bitcounter)
let regs_by_bits label rs reserve =
  wrap_counter (fun (_, bo, _) ctr -> regs_by_bits (Some bo) ctr rs reserve) label

let argcounter = wrap_counter (fun _ -> counter (fun w -> 1))
let regs_by_args = reg_placer (fun w -> 1) None
let regs_by_args label rs reserve =
  wrap_counter (fun _ ctr -> regs_by_args ctr rs reserve) label

let pad =
  let padcounter space ctr next =
    let (_, _, memsize) = space in
    let f nalloc ~width ~alignment ~kind =
      let al = Cell.to_width memsize (R.C alignment) in
      ctr := Auxfun.round_up_to ~multiple_of:al !ctr;
      nalloc ~width ~alignment ~kind
    in { next with allocate = f next.allocate }
  in wrap_counter padcounter
```

```

⟨definition of combine_loc 418a⟩≡ (416c)
⟨shift and mask functions 418b⟩
let combine msb msw lsb lsw =
  { fetch = (fun ww -> assert (ww = msw + lsw);
              orb ww (zx lsw ww (fetch lsb lsw))
              (shl ww (zx msw ww (fetch msb msw)) lsw))
  ; store = (fun exp ww -> assert (ww = msw + lsw);
              R.par [ store lsb (lobits ww lsw exp) lsw
                    ; store msb (lobits ww msw (shrl ww exp lsw)) msw ])
  }

let combine_loc = function
| Rtl.Identity      -> impossf "split without byte-ordering"
| Rtl.LittleEndian -> (fun b1 w1 b2 w2 -> combine b2 w2 b1 w1)
| Rtl.BigEndian    -> combine

```

```

⟨shift and mask functions 418b⟩≡ (418a)
let const w k = R.bits (Bits.U.of_int k w) w
let zx n w v = R.app (R.opr "zx" [n; w]) [v]
let orb w x y = R.app (R.opr "or" [w]) [x; y]
let shl w x k = R.app (R.opr "shl" [w]) [x; const w k]
let shrl w x k = R.app (R.opr "shrl" [w]) [x; const w k]
let lobits w n x = R.app (R.opr "lobits" [w; n]) [x]

```

```

⟨automaton.ml 418c⟩+≡ (411a) <416c 418d>
let as_stage inner next =
  fun ~start ~space ~ctrs ->
    let freeze regs mems =
      { regs_used = regs; mems_used = mems;
        overflow = Block.at start 0 0; align_state = 0 } in
    { allocate = inner.allocate; freeze = freeze }
let as_stage inner = swrap (as_stage inner)

```

```

⟨automaton.ml 418d⟩+≡ (411a) <418c 418e>
let unit = wrap (fun next -> next)

```

```

⟨automaton.ml 418e⟩+≡ (411a) <418d 418f>
let debug f _ ctr next =
  let allocate ~width ~alignment ~kind =
    f width kind alignment (!ctr);
    next.allocate ~width ~alignment ~kind in
  { allocate = allocate; freeze = next.freeze }

let debug counter f = wrap_counter (debug f) counter

```

```

⟨automaton.ml 418f⟩+≡ (411a) <418e 418g>
let postprocess (S stage) f = S (fun imp ->
  let I i = stage imp in
  I (fun ~start ~space ~ctrs ->
    let m = i start space ctrs in
    { allocate = m.allocate
    ; freeze = (fun rs ms -> f (m.freeze rs ms))
    } ))

```

```

⟨automaton.ml 418g⟩+≡ (411a) <418f
type cc_spec = { call : stage; results : stage; cutto : stage }
type cc_specs = (string * cc_spec) list
let init_cc = []

```

P.4 front_target/box.nw

```
<front_target/box.ml 419a>≡  
  <box.ml 420b>  
  
<front_target/box.mli 419b>≡  
  <box.mli 419d>
```

P.5 Boxing

```
<BOXER type 419c>≡ (420b 419d)  
  module type BOXER = sig  
    type t  
    val box : t -> rtl  
    val unbox : rtl -> t (* possible assertion failure *)  
  end  
  
<box.mli 419d>≡ (419b) 419e▷  
  type rtl = Rtl.rtl  
  <BOXER type 419c>  
  module Exp : BOXER with type t = Rtl.exp  
  module ExpList : BOXER with type t = Rtl.exp list  
  module Guard : BOXER with type t = Rtl.exp  
  module Combine (Box1 : BOXER) (Box2 : BOXER) : BOXER with type t = Box1.t * Box2.t  
  module GuardExp : BOXER with type t = Rtl.exp * Rtl.exp  
  
<box.mli 419e>+≡ (419b) ◁419d  
  val assert_not_boxed : rtl -> unit  
  
<exp boxer 419f>≡ (420b)  
  type t = R.exp  
  let box e = R.store (R.reg (x, 0, (R.C 1))) e 32  
  let unbox r = match Dn.rtl r with  
    | RP.Rtl [(RP.Const (RP.Bool true),  
               RP.Store (RP.Reg (('000',_,_), _, _), e, 32))] -> Up.exp e  
    | _ -> assert false  
  
<explist boxer 419g>≡ (420b)  
  type t = Rtl.exp list  
  let box es = R.par (List.map Exp.box es)  
  let unbox r = match Dn.rtl r with  
    | RP.Rtl gs -> List.map (fun g -> Exp.unbox (Up.rtl (RP.Rtl [g]))) gs  
  
<guard boxer 419h>≡ (420b)  
  type t = R.exp  
  let box g = R.guard g (R.store (R.reg (x, 0, (R.C 1))) (R.bits (Bits.zero 32) 32) 32)  
  let unbox r = match Dn.rtl r with  
    | RP.Rtl [(g, RP.Store (RP.Reg (('000',_,_), _, _), _, 32))] -> Up.exp g  
    | _ -> assert false  
  
<Combine 419i>≡ (420b)  
  module Combine (Box1 : BOXER) (Box2 : BOXER) = struct  
    type t = Box1.t * Box2.t  
    let box (v1, v2) =  
      let b1 = Box1.box v1 in  
      let () = match Dn.rtl b1 with | RP.Rtl [] -> () | _ -> assert false in  
      R.par [b1; Box2.box v2]  
    let unbox r = match Dn.rtl r with  
      | RP.Rtl (g::gs) ->  
        (Box1.unbox (Up.rtl (RP.Rtl [g])), Box2.unbox (Up.rtl (RP.Rtl gs)))  
      | _ -> assert false  
  end
```

```

<assert_not_boxed 420a>≡ (420b)
  let assert_not_boxed r =
    let check = function
      | (_, RP.Store (RP.Reg (('000',_,_), _, _), _, _)) -> assert false
      | _ -> () in
    match Dn.rtl r with
    | RP.Rtl gs -> List.iter check gs

```

```

<box.ml 420b>≡ (419a)
  module Dn = Rtl.Dn
  module R = Rtl
  module RP = Rtl.Private
  module Up = Rtl.Up
  type rtl = R.rtl
  let x = (Space.Standard32.x R.LittleEndian [32]).Space.space
  <BOXER type 419c>
  module Exp = struct
    <exp boxer 419f>
  end
  module ExpList = struct
    <explist boxer 419g>
  end
  module Guard = struct
    <guard boxer 419h>
  end
  <Combine 419i>
  module GuardExp = Combine (Guard) (Exp)
  <assert_not_boxed 420a>

```

P.6 front_target/float.nw

```

<front_target/float.ml 420c>≡
  <float.ml 420f>

```

```

<front_target/float.mli 420d>≡
  <float.mli 420e>

```

P.7 Support for floating-point numbers

```

<float.mli 420e>≡ (420d)
  type t

```

```

  val name      : t -> string
  val of_string : t -> string -> Rtl.width -> Bits.bits

```

```

  val ieee754 : t      (* standard IEEE 754 semantics *)
  val none    : t      (* for machines without floating-point support *)

```

```

<float.ml 420f>≡ (420c)
  type t = string
  let name n = n
  let of_string _ _ _ = Impossible.unimp "Float.t"
  let ieee754 = "ieee754"
  let none = "none"

```

P.8 front_target/space.nw

```
<front_target/space.ml 421a>≡  
  <space.ml 422d>  
  
<front_target/space.mli 421b>≡  
  <space.mli 421c>
```

P.9 RTL-Spaces

```
<space.mli 421c>≡ (421b) 422c▷  
  <definitions of exported types, including t 421e>  
  val checked : t -> t  
  <definition of signature Standard 421d>  
  module Standard32 : Standard  
  module Standard64 : Standard  
  
<definition of signature Standard 421d>≡ (422d 421c)  
  module type Standard = sig  
    type generator = Rtl.aggregation -> Rtl.width list -> t  
    type tgenerator = Rtl.aggregation -> Rtl.width -> t  
    val m : generator (* standard 8-bit memory *)  
    val r : count:int -> generator (* registers *)  
    val t : tgenerator (* register temps *)  
    val f : count:int -> generator (* floats *)  
    val u : tgenerator (* float temps *)  
    val a : count:int -> generator (* address registers *)  
    val v : tgenerator (* address temps *)  
    val p : count:int -> generator (* predicate registers *)  
    val w : tgenerator (* predicate temps *)  
    val c : count:int -> generator (* control and special registers *)  
    val vf : generator (* virtual frame pointer *)  
    val x : generator (* boxed rtls *)  
    val s : int -> tgenerator (* stack-slots temporaries *)  
  
    type 'a locations =  
      { pc: 'a  
        ; npc: 'a  
        ; cc: 'a  
        ; fp_mode: 'a (* FP rounding mode *)  
        ; fp_fcmp: 'a (* FP %fcmp results *)  
      }  
  
    val locations : c:t -> Rtl.loc locations  
      (* apply to c space to get standard locations *)  
    val indices : int locations (* standard indices in c space *)  
    val vfp : Rtl.exp (* the virtual frame pointer, $V[0] *)  
  end  
  
<definitions of exported types, including t 421e>≡ (422d 421c)  
  <definition of type location_set 422b>  
  <definition of type classification 422a>  
  type t =  
    { space: Rtl.space (* space being described *)  
      ; doc: string (* informal doc string *)  
      ; indexwidth: int (* bits *)  
      ; indexlimit: int option (* None = 2 ** indexwidth *)  
      ; widths: int list (* bit widths of supported aggregates *)  
      ; classification: classification  
    }
```

<definition of type classification 422a>≡ (421e)

```
type classification =
  | Mem
  | Reg
  | Fixed
  | Temp of location_set
```

<definition of type location_set 422b>≡ (421e)

```
type location_set =
{ stands_for : Register.t -> bool (* what registers we stand for *)
; set_doc    : string    (* informal description *)
}
```

<space.mli 422c>+≡ (421b) <421c
 val stands_for : char -> Register.aggregation -> Register.width -> (Register.t -> bool)

P.9.1 Implementation

<space.ml 422d>≡ (421a) 422e▷
 open Nopoly

<definitions of exported types, including t 421e>
<definition of signature Standard 421d>

<space.ml 422e>+≡ (421a) <422d 422f▷
 let indexwidth n =
 let rec wid = function
 | 0 -> 0
 | n -> 1 + wid (n lsr 1) in
 wid (n-1)
 let checked s =
 begin
 let (_, agg, cell) = s.space in
 assert (match s.widths with [] -> false | _ :: _ -> true);
 List.iter (fun w -> assert (Cell.divides cell w)) s.widths;
 (match agg with
 | Rtl.Identity -> assert (s.widths == [Cell.to_width cell (Cell.C 1)])
 | _ -> ());
 (match s.indexlimit with
 | Some n -> assert (indexwidth n <= s.indexwidth)
 | _ -> ());
 s
 end
 let stands_for s agg w =
 (fun ((s', agg', c), _, Register.C ct) ->
 s' <= s && agg == agg' && ct = 1 && Cell.size c = w)

<space.ml 422f>+≡ (421a) <422e
 module Standard(A:sig val width: int end) = struct
 type generator = Rtl.aggregation -> Rtl.width list -> t
 type tgenerator = Rtl.aggregation -> Rtl.width -> t
 let m agg ws = checked
 { space = ('m', agg, Cell.of_size 8)
 ; doc = "memory"
 ; indexwidth = A.width
 ; indexlimit = None
 ; widths = ws
 ; classification = Mem
 }

```

let r ~count agg ws = checked
{ space      = ('r', agg, Cell.of_size A.width)
; doc        = "general-purpose registers"
; indexwidth  = indexwidth count
; indexlimit  = Some count
; widths     = ws
; classification = Reg
}

let f ~count agg ws = checked
{ space      = ('f', agg, Cell.of_size A.width)
; doc        = "floating-point registers"
; indexwidth  = indexwidth count
; indexlimit  = Some count
; widths     = ws
; classification = Reg
}

let a ~count agg ws = checked
{ space      = ('a', agg, Cell.of_size A.width)
; doc        = "address registers"
; indexwidth  = indexwidth count
; indexlimit  = Some count
; widths     = ws
; classification = Reg
}

let p ~count agg ws = checked
{ space      = ('p', agg, Cell.of_size 1)
; doc        = "predicate registers"
; indexwidth  = indexwidth count
; indexlimit  = Some count
; classification = Reg
; widths     = ws
}

(* CHECK THAT count IS LARGE ENOUGH TO COVER indicies BELOW -- CL*)
let c ~count agg ws = checked
{ space      = ('c', agg, Cell.of_size A.width)
; doc        = "control and special"
; indexwidth  = indexwidth count
; indexlimit  = Some count
; widths     = ws
; classification = Fixed
}

let vf agg ws =
  let doc = "virtual frame pointer" in
  checked { space      = ('V', agg, Cell.of_size A.width)
          ; doc        = doc
          ; indexwidth  = 1
          ; indexlimit  = Some 1
          ; widths     = ws
          ; classification = Fixed
          }

let x agg ws = checked
{ space      = ('\000', agg, Cell.of_size A.width)
; doc        = "boxed rtls"

```



```

; indexwidth      = 1
; indexlimit      = Some 1
; widths          = ws
; classification  = Reg
}

let temp space letter ~for space doc agg w = checked
{ space          = (letter, Rtl.Identity, Cell.of_size w);
  doc            = doc;
  indexwidth     = A.width;
  indexlimit     = None;
  widths         = [w];
  classification = Temp
    { stands_for = stands_for for space agg w;
      set_doc    = doc;
    };
}

let t = temp space 't' ~for space:'r' "general-purpose temporaries"
let u = temp space 'u' ~for space:'f' "floating-point temporaries"
let v = temp space 'v' ~for space:'a' "address temporaries"
let w = temp space 'w' ~for space:'p' "predicate temporaries"

let s align = temp space (char_of_int align) ~for space:'m'
  (string_of_int align ^ " aligned stack-slots temporaries")

type 'a locations =
{ pc:      'a
; npc:     'a
; cc:      'a
; fp_mode: 'a (* FP rounding mode *)
; fp_fcmp: 'a (* FP condition codes *)
}

let locations ~c = match c with
| { space      = ('c',bo,_) as space
; indexwidth  = iw
} ->
  let reg n = Rtl.reg (space, n, Rtl.C 1) in
  { pc      = reg 0
; npc      = reg 1
; cc       = reg 2
; fp_mode  = Rtl.slice 2 0 (reg 4)
; fp_fcmp  = Rtl.slice 2 0 (reg 5)
}
| { space = (s,_,_) } -> Impossible.impossible
  ( "Standard locations from space " ^ Char.escaped s)

let indices = { pc = 0; npc = 1; cc = 2; fp_mode = 4 ; fp_fcmp = 5 }
let vfp_cell = Cell.of_size A.width
let vfp = Rtl.fetch (Rtl.reg (('V',Rtl.Identity,vfp_cell), 0, Rtl.C 1)) A.width
end
module Standard32 = Standard(struct let width = 32 end)
module Standard64 = Standard(struct let width = 64 end)

```

P.10 front_target/target.nw

<front_target/target.ml 425a>≡
<target.ml 428e>

<front_target/target.mli 425b>≡
<target.mli 425c>

P.11 Target Platform Description

<target.mli 425c>≡ (425b) 428c▷
<map and machine types 427a>
val boxmach : unit machine
<type cc 425f>
<auxiliary types used to define type t 428b>
<type t(target.nw) 425d>

<exported functions 426i>
val space : ('pr, 'au, 'cc) t -> Rtl.space -> Space.t

<type t(target.nw) 425d>≡ (428e 425c)
type ('proc, 'automaton, 'cc) t = {
 <components of t 425e>
}

<components of t 425e>≡ (425d) 425g▷
name: string;

P.11.1 Calling Convention

<type cc 425f>≡ (428e 425c)
type 'automaton cc' =
 { sp: Rtl.loc (* stack pointer *)
 ; return: Rtl.rtl (* machine instr passed to Cfg.return *)
 (* NEEDS TO TAKE ALTS AND INDEX AS PARAMETER *)
 ; proc: 'automaton (* pass parameter to a procedure *)
 ; cont: 'automaton (* pass parameter to a continuation *)
 ; ret: 'automaton (* return values *)
 ; mutable allocatable: Register.t list (* regs for reg-allocation *)
 (* THIS TYPE SHOULD INCLUDE INFORMATION ABOUT ALTERNATE RETURN CONTINUATIONS,
 IN PARTICULAR, HOW BIG IS EACH SLOT, AND DOES IT HOLD AN INSTRUCTION OR
 AN ADDRESS? *)
 ; stack_slots: 'automaton (* where private data go *)
 }

<components of t 425g>+≡ (425d) <425e 425h>
mutable cc_specs: Automaton.cc_specs;
cc_spec_to_auto: string -> Automaton.cc_spec -> 'cc;

<components of t 425h>+≡ (425d) <425g 426a>
vfp : Rtl.exp; (* the (immutable) virtual frame pointer *)
(* always equal to Vfp.mk T.pointersize *)

P.11.2 Bits and Pieces

⟨components of t 426a⟩+≡ (425d) <425h 426b>
byteorder: Rtl.aggregation ; (* big/little endian, id *)
wordsize: int ; (* bits *)
pointersize: int ; (* bits *)
alignment: int ; (* alignment of word access *)
memsize: int ; (* smallest addressable unit, typically 8 *)
memspace: Rtl.space ; (* redundant with byte order, word size *)
float: Float.t ; (* floating pt name and semantics *)
charset: string ; (* "latin1" character encoding *)
globals: 'automaton ; (* Automaton to allocate global vars *)

⟨components of t 426b⟩+≡ (425d) <426a 426c>
max_unaligned_load : Rtl.count; (* how many cells to load unaligned *)

⟨components of t 426c⟩+≡ (425d) <426b 426d>
spaces: Space.t list;

⟨components of t 426d⟩+≡ (425d) <426c 426e>
reg_ix_map : int * int Register.Map.t;

⟨components of t 426e⟩+≡ (425d) <426d 426f>
distinct_addr_sp: bool;

⟨components of t 426f⟩+≡ (425d) <426e 426g>
data_section: string; (* ASM section for global regs *)

P.11.3 Identify Special Locations

⟨components of t 426g⟩+≡ (425d) <426f 426h>
rounding_mode : Rtl.loc;

P.11.4 Named Hardware Registers

⟨components of t 426h⟩+≡ (425d) <426g 426k>
named_locs: Rtl.loc Strutil.Map.t;

P.11.5 Temporaries, Register indices, and Register Allocation

⟨exported functions 426i⟩≡ (425c) 426j>
val is_tmp : ('pr, 'au, 'cc) t -> Rtl.space -> bool (* partially apply me *)
val fits: ('pr, 'au, 'cc) t -> Rtl.space -> Register.t -> bool

⟨exported functions 426j⟩+≡ (425c) <426i
val mk_reg_ix_map : Space.t list -> (int * int Register.Map.t)

P.11.6 The recognizer

⟨components of t 426k⟩+≡ (425d) <426h 427b>
is_instruction: Rtl.rtl -> bool;

P.11.7 Control-flow operations

⟨map and machine types 427a⟩≡ (428e 425c)

```
type ('em, 'pr) map' = ('em, 'pr) Ep.pre_map =
  { embed    : 'em
  ; project  : 'pr
  }

type brtl = Rtl.exp -> Rtl.rtl
type ('a,'b) map = ('a -> 'b -> brtl Dag.branch, Rtl.rtl -> 'b) map'
type ('a,'b) mapc = ('a -> 'b -> brtl Dag.cbranch, Rtl.rtl -> 'b) map'

type 'a machine = 'a Mflow.machine =
  { bnegate:  Rtl.rtl -> Rtl.rtl
  ; goto:    ('a, Rtl.exp) map
  ; jump:    ('a, Rtl.exp) map
  ; call:    ('a, Rtl.exp) map
  ; branch:  ('a, Rtl.exp) mapc (* condition *)
  ; retgt_br: Rtl.rtl -> brtl Dag.cbranch
  ; move:    'a -> src:Register.t -> dst:Register.t -> brtl Dag.block
  ; spill:   'a -> Register.t -> Rtlutil.aloc -> brtl Dag.block
  ; reload:  'a -> Rtlutil.aloc -> Register.t -> brtl Dag.block
  ; cutto:   ('a, Mflow.cut_args) map (* newpc * newsp map*)
  ; return:  Rtl.rtl
  ; forbidden: Rtl.rtl (* causes a run-time error *)
  }
```

⟨components of t 427b⟩+≡ (425d) <426k 427d>

```
machine : 'proc machine;
```

⟨boxed machine 427c⟩≡ (428e)

```
let boxmach =
  let fail _ = assert false in
  let crmap = { embed = (fun () e -> (DG.Nop, Box.Exp.box e))
               ; project = Box.Exp.unbox } in
  let x = (Space.Standard32.x R.LittleEndian [32]).Space.space in
  let bogus = R.kill (R.reg (x, 0, (R.C 1))) in
  { bnegate = (fun _ -> assert false)
  ; goto = crmap
  ; jump = crmap
  ; call = crmap
  ; cutto = { embed =
              (fun _ ca -> (DG.Nop, Box.ExpList.box [ca.M.new_sp; ca.M.new_pc]))
            ; project = (fun box -> match Box.ExpList.unbox box with
                                | [sp; pc] -> {M.new_sp = sp; M.new_pc = pc}
                                | _ -> assert false)}
  ; branch = { embed = (fun _ g -> DG.cond (fun _ -> Box.Guard.box g))
              ; project = (fun box -> Box.Guard.unbox box)}
  ; retgt_br = fail
  ; move = fail
  ; spill = fail
  ; reload = fail
  ; M.return = bogus
  ; forbidden = bogus
  }
```

P.11.8 Code-generation assist

⟨components of t 427d⟩+≡ (425d) <427b 428a>

```
tx_ast : Auxfun.void Nelab.compunit -> Auxfun.void Nelab.compunit;
```

P.11.9 Code-generation capabilities

```
<components of t 428a>+≡ (425d) <427d
capabilities: capabilities;

<auxiliary types used to define type t 428b>≡ (428e 425c)
type capabilities = {
  operators   : Rtl.opr list; (* operators that can be used *)
  literals    : Rtl.width list; (* literals that can be used *)
  litops      : Rtl.opr list; (* operators usable on literals only *)
  memory      : Rtl.width list; (* memory refs that can be used *)
  block_copy  : bool; (* OK to copy large variables, large refs? *)
  itemps      : Rtl.width list; (* what integer temporaries can be computed with *)
  ftemps      : Rtl.width list; (* what floating temporaries can be computed with *)
  iwiden      : bool; (* use int ops and literals at narrow widths? *)
  fwiden      : bool; (* use float ops literals at narrow widths? *)
}

<target.mli 428c>+≡ (425b) <425c 428d>
val incapable : capabilities (* the completely useless back end *)

<target.mli 428d>+≡ (425b) <428c
val minimal_capabilities: int -> capabilities
```

P.11.10 Implementation

```
<target.ml 428e>≡ (425a) 429a>
<map and machine types 427a>
module DG = Dag
module M = Mflow
module R = Rtl
module S = Space
module RP = Rtl.Private
module RU = Rtlutil
<type cc 425f>
<auxiliary types used to define type t 428b>
<type t(target.nw) 425d>
<boxed machine 427c>
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let incapable = { operators = []; literals = []; litops = []; memory = []; itemps = [];
                  ftemps = []; block_copy = false; iwiden = false; fwiden = false; }
let minimal_capabilities wordsz =
{ literals = [wordsz]
  ; block_copy = false
  ; itemps = [wordsz]
  ; ftemps = []
  ; iwiden = false
  ; fwiden = false
  ; litops = []
; memory = [wordsz]
; operators = List.map Rtl.Up.opr [
    "sx", [ 1; wordsz]
    ; "zx", [ 8; wordsz]
    (* for some reason, -interp always generates 8->32 zx instructions *)
    ; "lobits", [wordsz; 1]
    ; "lobits", [wordsz; wordsz]
    ; "add", [wordsz]
    ; "addc", [wordsz]
    ; "and", [wordsz]
```

```

; "borrow", [wordsize]
; "carry", [wordsize]
; "com", [wordsize]
; "div", [wordsize]
; "divu", [wordsize]
; "false", []
; "mod", [wordsize]
; "modu", [wordsize]
; "mul", [wordsize]
; "mulx", [wordsize]
; "mulux", [wordsize]
; "neg", [wordsize]
; "or", [wordsize]
; "quot", [wordsize]
; "popcnt", [wordsize]
; "rem", [wordsize]
; "rotl", [wordsize]
; "rotr", [wordsize]
; "shl", [wordsize]
; "shra", [wordsize]
; "shrl", [wordsize]
; "sub", [wordsize]
; "subb", [wordsize]
; "true", []
; "xor", [wordsize]
; "eq", [wordsize]
; "ge", [wordsize]
; "geu", [wordsize]
; "gt", [wordsize]
; "gtu", [wordsize]
; "le", [wordsize]
; "leu", [wordsize]
; "lt", [wordsize]
; "ltu", [wordsize]
; "ne", [wordsize]

; "add_overflows", [wordsize]
; "div_overflows", [wordsize]
; "mul_overflows", [wordsize]
; "mulu_overflows", [wordsize]
; "quot_overflows", [wordsize]
; "sub_overflows", [wordsize]
; "not", []
; "bool", []
; "disjoin", []
; "conjoin", []
; "bit", []
];}

```

```

<target.ml 429a>+≡ (425a) <428e 429b>
let space t s =
  try List.find (fun x -> RU.Eq.space x.Space.space s) t.spaces
  with Not_found ->
    let (s, _, _) = s in
    impossf "Space '%c' not found in target '%s'\n" s t.name

```

```

<target.ml 429b>+≡ (425a) <429a 430a>
let is_tmp { spaces = spaces } =
  List.fold_right

```

```

    (fun s rest ->
      match s.Space.classification with
      | Space.Temp _ -> (fun c -> RU.Eq.space c s.Space.space || rest c)
      | _ -> rest)
    spaces
    (fun _ -> false)

<target.ml 430a>+≡ (425a) <429b 430b>
  let mk_reg_ix_map spaces =
    let space_to_regs space =
      let rec list_to n =
        if n <= 0 then [0]
        else n :: list_to (n-1) in
      List.map (fun i -> (space.S.space, i, R.C 1))
        (list_to (match space.S.indexlimit with
          | Some l -> l
          | None -> 1 lsl space.S.indexwidth)) in
    let regspaces =
      let keep s = match s.S.classification with S.Reg | S.Fixed -> true | _ -> false in
      List.filter keep spaces in
    Register.reg_int_map (List.concat (List.map space_to_regs regspaces))

<target.ml 430b>+≡ (425a) <430a
  let space_fits tmp =
    match tmp.Space.classification with
    | Space.Temp {Space.stands_for=ok} -> ok
    | _ -> let (sp, _, _) = tmp.Space.space in
      impossf "space_fits called on non-temp of space '%c'" sp

  let fits t space =
    try space_fits (List.find (fun s -> RU.Eq.space s.Space.space space) t.spaces)
    with Not_found -> impossf "space not found in Target.fits"

```

P.11.11 Lua support for exposing target capabilities

```

<Lua utility functions written in Lua 430c>≡ 431a>
function w (...) -- a valuable abbreviation
  write(Util.call(format, arg))
end

function Target.explain(t)
  t = t or backend or error('default back end did not get set')
  if type(t) == 'table' then t = t.target end
  if type(t) ~= 'userdata' then
    error(tostring(t) .. ' is not a target')
  end
  local cap = Target.capabilities(t)
  local name = Target.name(t)

  local ops = cap.operators
  w('==== Advertised capabilities of target %s ====\\n\\n', name)

  w('target %s has these metrics:\\n', name)
  Target.emitmetrics(t, ' ')
  w('\\n')

  w('target %s supports memory references in these sizes:\\n', name)
  Target.showtypes(' %-6s[...]\\n', cap.memory, '\\n')

```

```

w('target %s supports literal bit vectors in these sizes:\n', name)
Target.showtypes('  nnnn :: %-6s\n', cap.literals, '\n')

w('target %s can compute with C-- variables declared at these sizes:\n', name)
Target.showtypes('  %-6s n;    // integer variable\n', cap.items);
Target.showtypes('  %-6s x;    // floating-point variable\n', cap.ftemps, '\n');

w('target %s implements these operators:\n', name)
local defined = { }
local i = 1
while ops[i] do
    defined[ops[i].name] = 1
    Target.showop(ops[i])
    i = i + 1
end
w('\n')

w('target %s can apply these operators to compile-time constant expressions only:\n',
  name)
local defined = { }
local i = 1
local litops = cap.litops
while litops[i] do
    defined[litops[i].name] = 1
    Target.showop(litops[i])
    i = i + 1
end
w('\n')

w('target %s does *not* implement these operators:\n', name)
i = 1
local allops = Rtlop.opnames ()
while allops[i] do
    if not defined[allops[i]] then
        write(format('  %%%s\n', allops[i]))
    end
    i = i + 1
end

if cap.iwiden then Target.widen_ok (name, 'integer')      end
if cap.fwiden then Target.widen_ok (name, 'floating-point') end
if cap.block_copy then <<announce block-copy support>> end
end

```

⟨*Lua utility functions written in Lua 431a*⟩+≡ <430c 431b>

```

function Target.showtypes(fmt, ts, s)
    local i = 1
    while ts[i] do
        w(fmt, Target.showty(ts[i]))
        i = i + 1
    end
    w(s or '')
end

```

⟨*Lua utility functions written in Lua 431b*⟩+≡ <431a 432b>

```

function Target.widen_ok(name, kind)
    w([[

```

The %s back end includes a 'widener' that allows you to declare %s variables and write %s computations at widths smaller than the natural widths shown above. Your code is automatically rewritten to use

the natural width of the machine.

```
]], name, kind, kind)
end
```

<announce block-copy support 432a>+≡

```
w([[
The %s back end supports block copies. This means you can declare
very wide variables and very wide memory references, but the only
thing you can do with them is copy the bits from one place to another.
For example, here is a mighty big swap operation:
```

```
bits128 n, m;
n = bits128[p];
m = bits128[p+16];
bits128[p] = m;
bits128[p+16] = n;
```

It should also be possible to pass such large variables as parameters and return them as results. Attempts to do arithmetic or other operations very wide variables are doomed to failure.

```
]], name)
```

<Lua utility functions written in Lua 432b>+≡

<431b 432c>

```
function Target.showty(t)
  if type(t) == 'number' then
    return 'bits' .. t
  else
    return t
  end
end
```

<Lua utility functions written in Lua 432c>+≡

<432b 432d>

```
Target.width_changers =
  Util.set { 'NaN', 'sx', 'zx', 'lobits', 'f2f', 'f2i', 'i2f' }
```

```
function Target.showop(op)
  local args, result = Rtlop.mono(op)
  if result then
    local ty = Target.showty
    w(' %-6s %%%s', ty(result), op.name)
    if Target.width_changers[op.name] then w(result) end
    w('(')
    local i = 1
    local pfx = ""
    while args[i] do
      w('%s%s', pfx, ty(args[i]))
      pfx = ', '
      i = i + 1
    end
    w(')\n')
  else
    w(' nonstandard operator %%%s\n', op.name)
  end
end
```

<Lua utility functions written in Lua 432d>+≡

<432c>

```
function Target.emitmetrics(t, pfx)
  local m = Target.metrics(t)
  pfx = pfx or ''
  w('%starget byteorder %s;\n', pfx, m.byteorder)
  w('%starget wordsize %d pointersize %d memsize %d;\n',
    pfx, m.wordsize, m.pointersize, m.memsize)
```

```
w('%starget float "%s" charset "%s";\n', pfx, m.float, m.charset)
end
```

Appendix Q

cfg/zipcfg

Q.1 front_zipcfg/avail.nw

```
<front_zipcfg/avail.ml 434a>≡  
  <avail.ml 434g>
```

```
<front_zipcfg/avail.mli 434b>≡  
  <avail.mli 434c>
```

Q.2 Available expressions

```
<avail.mli 434c>≡ (434b) 434d▷  
  type t    (* a set of available expressions *)  
  val forward : Rtl.rtl -> t -> t  
  val invalidate : Register.SetX.t -> t -> t (* anything that depends on the set *)  
  val unknown : t (* the unknown set (aka the infinite set: top) *)  
  val join : t -> t -> t (* effectively set intersection *)  
  val smaller : old:t -> new':t -> bool (* has a set shrunk? (unknown at top) *)
```

```
<avail.mli 434d>+≡ (434b) <434c 434e▷  
  val in_loc : t -> Rtl.Private.loc -> Rtl.Private.exp option  
  val has_exp : t -> Rtl.Private.exp -> Rtl.Private.loc option
```

```
<avail.mli 434e>+≡ (434b) <434d 434f▷  
  val subst_exp : t -> Rtl.Private.loc list -> Rtl.Private.exp -> Rtl.Private.exp
```

```
<avail.mli 434f>+≡ (434b) <434e  
  val to_string : t -> string
```

Q.2.1 Implementation

```
<avail.ml 434g>≡ (434a) 435a▷  
  module R    = Rtl  
  module RP   = Rtl.Private  
  module RU   = Rtlutil  
  module Up   = Rtl.Up  
  module Dn   = Rtl.Dn  
  module S    = RU.ToString  
  let () = Debug.register "avail" "available expressions"  
  let debug = Debug.on "avail"
```

```

⟨avail.ml 435a⟩+≡ (434a) <434g 435b>
type t = Unknown | Known of (RP.loc * RP.exp) list
let unknown = Unknown
let smaller ~old ~new' =
  match old with
  | Unknown -> (match new' with Unknown -> false | Known _ -> true)
  | Known olds ->
    (match new' with
    | Unknown -> false
    | Known news -> List.length news < List.length olds)

⟨avail.ml 435b⟩+≡ (434a) <435a 435c>
let in_loc t l = match t with
| Unknown -> None
| Known pairs -> try Some (List.assoc l pairs) with Not_found -> None

let has_exp t e = match t with
| Unknown -> None
| Known pairs ->
  try Some (fst (List.find (fun (_, e') -> RU.Eq.exp e e') pairs))
  with Not_found -> None

⟨avail.ml 435c⟩+≡ (434a) <435b 435d>
let join a a' = match a, a' with
| Unknown, a' -> a'
| a, Unknown -> a
| Known ps, Known ps' ->
  let primed (l, e) =
    List.exists (fun (l', e') -> RU.Eq.loc l l' && RU.Eq.exp e e') ps' in
  Known (List.filter primed ps)

⟨avail.ml 435d⟩+≡ (434a) <435c 436e>
⟨kills 436b⟩
⟨substitution 436c⟩
⟨invalidation 436d⟩
let forward rtl t =
  let pairs = match t with Unknown -> [] | Known ps -> ps in
  let locs = locs_killed rtl in
  let alocs = List.map RU.MayAlias.locs' locs in
  let aexps = List.map RU.MayAlias.exp' locs in
  let add_new_pair l r new_pairs =
    if List.exists (fun aexp -> aexp r) aexps then
      ⟨if all interfering locations can be substituted, keep l, r; otherwise not 436a⟩
    else
      (l, r) :: new_pairs in
  let pairs = invalidate_pairs alocs aexps pairs in
  (* massive pessimism: do registers only *)
  let extend guarded pairs = match guarded with
  | RP.Const (RP.Bool true), RP.Store(RP.Reg _ as l, r, _) -> add_new_pair l r pairs
  | _ -> pairs in
  let RP.Rtl effects = Dn.rtl rtl in
  let pairs = List.fold_right extend effects pairs in
  if debug then
    begin
      Printf.eprintf "avail: forwarding past %s yields\n" (S.rtl rtl);
      List.iter
        (fun (l, r) -> Printf.eprintf "avail: %s == %s\n"
          (S.loc (Up.loc l)) (S.exp (Up.exp r))) pairs;
    end;
  Known pairs

```

$\langle \text{if all interfering locations can be substituted, keep } l, r; \text{ otherwise not } 436a \rangle \equiv (435d)$

```

let badlocs = List.filter (fun l -> RU.MayAlias.exp' l r) locs in
if List.for_all (fun l -> List.mem_assoc l pairs) badlocs then
  (l, subst_exp (Known pairs) badlocs r) :: new_pairs
else
  new_pairs

```

$\langle \text{kills } 436b \rangle \equiv (435d)$

```

let locs_killed rtl =
  let add l locs = if List.mem l locs then locs else l :: locs in
  RU.FullReadWriteKill.fold ~write:add ~kill:add ~read:(fun _ locs -> locs)
    rtl []

```

$\langle \text{substitution } 436c \rangle \equiv (435d)$

```

let subst_exp t ls e = match t, ls with
| _, [] -> e
| Known pairs, _ :: _ ->
  RU.Subst.Fetch.exp' ~guard:(fun l -> List.mem l ls)
    ~fetch:(fun l w -> List.assoc l pairs) e
| Unknown, _ :: _ -> Impossible.impossible "substitution with no available expressions"

```

$\langle \text{invalidation } 436d \rangle \equiv (435d)$

```

let invalidate_pairs alocs aexps pairs =
  let invalidated (l, r) =
    List.exists (fun alloc -> alloc l) alocs || List.exists (fun aexp -> aexp r) aexps in
  if List.exists invalidated pairs then
    List.filter (fun p -> not (invalidated p)) pairs
  else
    pairs

let invalidate locs_killed t = match t with
| Unknown -> Unknown
| Known pairs ->
  let add l locs = Dn.loc (Rtl.regx l) :: locs in
  let locs = Register.SetX.fold add locs_killed [] in
  let alocs = List.map RU.MayAlias.locs' locs in
  let aexps = List.map RU.MayAlias.exp' locs in
  Known (invalidate_pairs alocs aexps pairs)

```

$\langle \text{avail.ml } 436e \rangle + \equiv (434a) \triangleleft 435d$

```

let to_string = function
| Unknown -> "<?>"
| Known [] -> "<nothing-available>"
| Known les ->
  let print (l, e) =
    Printf.sprintf "\n  %s = %s"
      (RU.ToString.loc (Up.loc l)) (RU.ToString.exp (Up.exp e)) in
  String.concat "" (List.map print les)

```

Q.3 front_zipcfg/property.nw

$\langle \text{front_zipcfg/property.mli } 436f \rangle \equiv$

$\langle \text{property.mli } 436g \rangle$

$\langle \text{property.mli } 436g \rangle \equiv (436f) \triangleleft 437c \triangleright$

$\langle \text{exposed representation } 437b \rangle$

$\langle \text{exported module types } 437a \rangle$

<exported module types 437a>≡ (437d 436g)

```

module type S = sig
  type 'a t    (* a property *)

  type list
  val list  : unit -> list    (* fresh, empty property list *)
  val clear : list -> unit    (* remove all properties *)

  val get    : 'a t -> list -> 'a
  val set    : 'a t -> list -> 'a -> unit
  val remove : 'a t -> list -> unit

  val prop : 'a matcher -> 'a t
end

```

<exposed representation 437b>≡ (437d 436g)

```

type rep =
| Live_in      of Register.SetX.t    (* to be extended... *)
| Vfp          of (Rtl.Private.exp * int)
| Avail        of Avail.t
| VarInMap      of Varmap.t
| VarOutMap     of Varmap.t
| VarOutMap'    of Varmap.y
| VarCallInMap of Varmap.t
| AllocPrefs   of (Register.t Register.Map.t * Register.t list Register.Map.t)
| Distances of
  (int * (int * Varmap.def_dist Register.Map.t * Varmap.use_dist Register.Map.t))

type 'a matcher = { embed : 'a -> rep; project : rep -> 'a option; is : rep -> bool; }

```

<property.mli 437c>+≡ (436f) <436g

```

module M : S

```

<property.ml 437d>≡

```

let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

<exposed representation 437b>

<exported module types 437a>

```

module M = struct
  <implementation 438a>
end

```

<old implementation 437e>≡

```

type hcontainer = { uid : int; update : unit -> unit; }
type 'a container = { make : 'a -> hcontainer;
                      pred : hcontainer -> bool;
                      peek : hcontainer -> 'a option; }

```

```

let container =
  let n = Reinit.ref 0 in
  fun () ->
    let id = !n in
    let () = n := id + 1 in
    let r = ref None in
    let make v = { uid = id; update = (fun () -> r := Some v); } in
    let peek c =
      if c.uid = id then
        let () = c.update() in
        let v = !r in
        let () = r := None in

```

```

        v
    else
        None in
    let pred c = c.uid = id in
    { make = make; pred = pred; peek = peek; }

type void = Void of void

(* let (_ : void -> hcontainer) = container *)

⟨implementation 438a⟩≡ (437d)
type 'a plist = 'a list
type list = { mutable props : rep plist }

let list () = { props = [] }
let length t = List.length t.props
let eq t t' = t == t'
let clear t = t.props <- []

let numPeeks: int ref = Reinit.ref 0
let numLinks: int ref = Reinit.ref 0
let maxLength: int ref = Reinit.ref 0

let stats () =
    Printf.eprintf "numPeeks = %d; maxLength = %d; avg pos = %.2f"
        (!numPeeks) (!maxLength) (float (!numLinks) /. float (!numPeeks))

let get matcher list =
    let rec loop l n =
        let update () =
            begin
                numLinks := n + !numLinks;
                if !numLinks < n then impossf "property list numLinks overflow";
                if n > !maxLength then maxLength := n;
            end in
        match l with
        | [] -> (update (); raise Not_found)
        | e :: l ->
            match matcher.project e with
            | Some r -> (update (); r)
            | None -> loop l (n+1) in
        numPeeks := 1 + !numPeeks;
        if !numPeeks < 1 then impossf "property list numPeeks overflow";
        loop list.props 0

let remove matcher list =
    list.props <- List.filter (fun p -> not (matcher.is p)) list.props
let add matcher list v = list.props <- matcher.embed v :: list.props
let set matcher list v = (remove matcher list; add matcher list v)

type 'a t = 'a matcher
let prop m = m

⟨junk(property.nw) 438b⟩≡

==> het-container.sig <==
(* Copyright (C) 1999-2002 Henry Cejtin, Matthew Fluet, Suresh

```

```

*   Jagannathan, and Stephen Weeks.
*
* MLton is released under the GNU General Public License (GPL).
* Please see the file MLton-LICENSE for license information.
*)
signature HET_CONTAINER =
  sig
    type t

    val new: unit -> {make: 'a -> t,
      pred: t -> bool,
      peek: t -> 'a option}
    end

==> property-list.sig <==
(* Copyright (C) 1999-2002 Henry Cejtin, Matthew Fluet, Suresh
*   Jagannathan, and Stephen Weeks.
*
* MLton is released under the GNU General Public License (GPL).
* Please see the file MLton-LICENSE for license information.
*)
type int = Int.t

signature PROPERTY_LIST =
  sig
    type t

    (* remove all properties from the list *)
    val clear: t -> unit
    (* pointer equality of property lists *)
    val equals: t * t -> bool
    val length: t -> int
    (* create an empty property list *)
    val new: unit -> t
    (* create a new property *)
    val newProperty:
unit -> {
      (* See if a property is in a property list.
      * NONE if it isn't.
      *)
      peek: t -> 'a option,
      (* Add the value of the property -- must not already exist. *)
      add: t * 'a -> unit,
      (* Remove a property from a property list.
      * Noop if the property isn't there.
      *)
      remove: t -> unit
    }
    val stats: unit -> Layout.t
  end

==> het-container.fun <==
(* Copyright (C) 1999-2002 Henry Cejtin, Matthew Fluet, Suresh
*   Jagannathan, and Stephen Weeks.
*
* MLton is released under the GNU General Public License (GPL).
* Please see the file MLton-LICENSE for license information.
*)
functor ExnHetContainer():> HET_CONTAINER =
  struct

```



```

    type t = exn

    fun 'a new() =
let exception E of 'a
in {make = E,
    pred = fn E _ => true | _ => false,
    peek = fn E x => SOME x | _ => NONE}
end
end

functor RefHetContainer():> HET_CONTAINER =
  struct
    type t = unit ref * (unit -> unit)

    fun 'a new() =
let
  val id = ref()
  val r: 'a option ref = ref NONE
  fun make v = (id, fn () => r := SOME v)
  fun peek(id', f) =
    if id = id' then (f()); !r before r := NONE
    else NONE
  fun pred(id', _) = id = id'
in {make = make, pred = pred, peek = peek}
end
end

==> property-list.fun <==
(* Copyright (C) 1999-2002 Henry Cejtin, Matthew Fluet, Suresh
 *   Jagannathan, and Stephen Weeks.
 *
 * MLton is released under the GNU General Public License (GPL).
 * Please see the file MLton-LICENSE for license information.
 *)
functor PropertyList (H: HET_CONTAINER):> PROPERTY_LIST =
  struct

    datatype t = T of H.t list ref

    fun new (): t = T (ref [])

    fun length (T r) = List.length (!r)

    val equals = fn (T r, T r') => Ref.equals (r, r')

    fun clear (T hs) = hs := []

    val numPeeks: int ref = ref 0
    val numLinks: int ref = ref 0
    val maxLength: int ref = ref 0

    fun stats () =
      let open Layout
      in align
        [seq [str "numPeeks = ", Int.layout (!numPeeks)],
         seq [str "maxLength = ", Int.layout (!maxLength)],
         seq [str "average position in property list = ",
              str let open Real
            in format (fromInt (!numLinks) / fromInt (!numPeeks),
                      Format.fix (SOME 3))
            end
          ]
      end
  end

```

```

end]]
end

fun 'a newProperty () =
  let
    val {make, pred, peek = peekH} = H.new ()
    fun peek (T hs) =
  let
    fun loop (l, n) =
      let
        fun update () =
          ((numLinks := n + !numLinks
            handle Overflow => Error.bug "property list numLinks overflow")
          ; if n > !maxLength
            then maxLength := n
          else ())
          in case l of
            [] => (update (); NONE)
          | e :: l =>
              case peekH e of
                r as SOME _ => (update (); r)
                | NONE => loop (l, n + 1)
              end
          val _ =
              numPeeks := 1 + !numPeeks
              handle Overflow => Error.bug "property list numPeeks overflow"
          in
            loop (!hs, 0)
          end
        end
      end
    fun add (T hs, v: 'a): unit = hs := make v :: !hs
    fun remove (T hs) = hs := List.remove (!hs, pred)
  in
    {add = add, peek = peek, remove = remove}
  end
end

end

```

Q.4 front_zipcfg/unique.nw

Q.5 Unique identifiers, maps, and sets

```

⟨unique.mli 441a⟩≡
  type uid
  val eq : uid -> uid -> bool
  val uid : unit -> uid
  val distinguished_uid : uid      (* distinct from any other *)
  ⟨exposed types(unique.nw) 441b⟩
  module Map : MAP
  module Set : SET
  module Array : ARRAY
  module Prop : Property.S with type list = uid

  ⟨exposed types(unique.nw) 441b⟩≡
    module type MAP = sig
      type 'a t
    end
    (442c 441a) 442a▷

```

```

val empty : 'a t
val is_empty : 'a t -> bool
val mem : uid -> 'a t -> bool
val add : uid -> 'a -> 'a t -> 'a t
val find : uid -> 'a t -> 'a
val split : uid -> 'a t -> 'a * 'a t
  (* split k m = (find k m, remove k m) *)
val splitp: (uid -> 'a -> bool) -> 'a t -> 'a * 'a t
  (* split based on predicate *)
val union : 'a t -> 'a t -> 'a t (* keep larger set on the right *)
val fold : (uid -> 'a -> 'b -> 'b) -> 'a t -> 'b -> 'b
val iter : ('a -> unit) -> 'a t -> unit
val size : 'a t -> int

val map : ('a -> 'b) -> ('a t -> 'b t)
end

⟨exposed types(unique.nw) 442a⟩+≡ (442c 441a) <441b 442b>
module type SET = sig
  type t
  val empty : t
  val mem : uid -> t -> bool
  val add : uid -> t -> t
end

⟨exposed types(unique.nw) 442b⟩+≡ (442c 441a) <442a
module type ARRAY = sig
  type 'a t
  val make : uid list -> 'a -> 'a t
  val get : 'a t -> uid -> 'a
  val set : 'a t -> uid -> 'a -> unit
  val update : 'a t -> uid -> ('a -> 'a) -> unit
end

```

Q.5.1 Implementation

```

⟨unique.ml 442c⟩≡
module P = Property.M
type uid = int * P.list
let uid =
  let n = Reinit.ref 1 in
  fun () -> let u = !n in (n := u + 1; (u, P.list()))
let distinguished_uid = (0, P.list())

⟨exposed types(unique.nw) 441b⟩

module Ord = struct
  type t = uid
  let compare : t -> t -> int = fun (u, _) (u', _) -> compare u u'
end
module Set = Set.Make (Ord)
module M = Map.Make (Ord)
module Map : MAP = struct
  include M
  let split u m = (find u m, remove u m)
  let splitp p m =
    let scan u v (yes, no) = match yes with
    | None when p u v -> (Some v, no)
    | _ -> (yes, add u v no) in

```

```

match fold scan m (None, empty) with
| (Some v, m) -> v, m
| (None, _) -> raise Not_found
let union m m' =
  let add u v m =
    if mem u m then
      raise (Invalid_argument (Printf.sprintf "Unique.Map.union dup key %d" (fst u)))
    else
      add u v m in
  fold add m m'
let size m = fold (fun _ _ n -> n + 1) m 0
let iter f m = iter (fun _ v -> f v) m
end

module Array : ARRAY = struct
  type 'a t = int Map.t * 'a array
  let make uids init =
    let rec build uids i map = match uids with
    | [] -> map, Array.make i init
    | u :: us -> build us (i+1) (Map.add u i map) in
    build uids 0 Map.empty

  let get (map, a) u = Array.get a (Map.find u map)
  let set (map, a) u v = Array.set a (Map.find u map) v
  let update (map, a) u f =
    let i = Map.find u map in
    Array.set a i (f (Array.get a i))
end

let eq ((u, _):uid) (u', _) = u = u'

module Prop = struct
  type 'a t = 'a P.t
  type list = uid
  let list = uid
  let clear (_, l) = P.clear l
  let get p (_, l) = P.get p l
  let set p (_, l) v = P.set p l v
  let remove p (_, l) = P.remove p l
  let prop = P.prop
end

```

Q.6 front_zipcfg/varmap.nw

$\langle \text{front_zipcfg/varmap.ml } 443a \rangle \equiv$
 $\langle \text{varmap.ml } 444e \rangle$

$\langle \text{front_zipcfg/varmap.mli } 443b \rangle \equiv$
 $\langle \text{varmap.mli } 443c \rangle$

Q.6.1 Variable Maps

$\langle \text{varmap.mli } 443c \rangle \equiv$ (443b) 444a▷
 type reg = Register.t
 type rset = Register.Set.t
 type temp = Register.t

```

type loc_pair = { reg : Register.t option; mem : Rtlutil.aloc option }
type loc_pair' = { reg' : Register.t option; mem' : bool }
<types for tracking defs and uses 448b>

⟨varmap.mli 444a⟩+≡ (443b) <443c 444b>
  type t
  val empty : t
  val add_reg : temp -> Register.t -> t -> t
  val add_mem : temp -> Rtlutil.aloc -> t -> t
  val spill : temp -> Rtlutil.aloc -> t -> t
  val remove_reg : temp -> Register.t -> t -> t
  val remove_mem : temp -> Rtlutil.aloc -> t -> t

  type y
  val emptyy : y
  val is_empty' : y -> bool
  val add_reg' : temp -> Register.t -> y -> y
  val add_mem' : temp -> y -> y
  val remove_mem' : temp -> y -> y
  val remove_reg' : temp -> y -> y
  val spill' : temp -> y -> y
  val join : y -> y -> y
  val eq' : old:y -> new':y -> bool

⟨varmap.mli 444b⟩+≡ (443b) <444a 444c>
  val fold : (temp -> Register.t -> 'a -> 'a) ->
    (temp -> Rtlutil.aloc -> 'a -> 'a) -> t -> 'a -> 'a
  val fold' : (temp -> Register.t -> 'a -> 'a) ->
    (temp -> 'a -> 'a) -> y -> 'a -> 'a
  val filter : (temp -> bool) -> t -> t
  val var_locs' : t -> temp -> loc_pair
  val var_locs'' : y -> temp -> loc_pair'
  val temp_loc' : y -> temp -> reg option
  val reg_contents : t -> reg -> temp option
  val reg_contents' : y -> reg -> temp list
  val print : string -> t -> unit
  val print' : string -> y -> unit

⟨varmap.mli 444c⟩+≡ (443b) <444b 444d>
  val free_reg_inregs : rset -> rset -> rset -> t -> rset -> bool -> reg -> bool
  val free_reg_outregs : rset -> rset -> t -> rset -> bool -> reg -> bool
  val alloc_inreg : reg list -> rset -> rset -> rset -> t -> rset -> bool ->
    temp -> reg
  val alloc_outreg : reg list -> rset -> rset -> t -> rset -> bool ->
    temp -> reg

⟨varmap.mli 444d⟩+≡ (443b) <444c
  val sync_maps : t -> t -> t * t

⟨varmap.ml 444e⟩≡ (443a) 445a>
  open Nopoly

  type reg = Register.t
  type rset = Register.Set.t
  type temp = Register.t
  type loc_pair = { reg : Register.t option; mem : Rtlutil.aloc option }
  type loc_pair' = { reg' : Register.t option; mem' : bool }
  <types for tracking defs and uses 448b>
  module RM = Register.Map
  module RS = Register.Set

```

```

type t = loc_pair RM.t * temp RM.t
type y = loc_pair' RM.t * temp list RM.t
let impossf fmt = Printf.kprintf Impossible.impossible fmt

⟨varmap.ml 445a⟩+≡ (443a) <444e 445b>
let empty_pair      = {reg = None; mem = None}
let def_reg pair r   = {pair with reg = r}
let def_mem pair m   = {pair with mem = m}
let pair_map_find    t m = try RM.find t m with Not_found -> empty_pair

let empty           = (RM.empty, RM.empty)
let emptyy          = (RM.empty, RM.empty)
let is_empty' (vm, lm) = RM.is_empty vm && RM.is_empty lm
let remove_reg t r (vm, lm) =
  let lm' = RM.remove r lm in
  try match (RM.find t vm).mem with
    | None -> (RM.remove t vm, lm')
    | Some m as m' -> (RM.add t {reg = None; mem = m'} vm, lm')
  with Not_found -> Impossible.impossible "DLS: Tried to remove unknown reg"
let filt r lst = List.filter (fun r' -> not (Register.eq r r')) lst
let lm_rem' r t lm = try RM.add r (filt t (RM.find r lm)) lm with Not_found -> lm
let lm_add' r t lm = RM.add r (t :: try RM.find r lm with Not_found -> []) lm
let remove_reg' t (vm, lm) =
  try let lp = RM.find t vm in
    match lp.reg' with
    | Some r -> let lm' = lm_rem' r t lm in
      if lp.mem' then (RM.add t {reg' = None; mem' = true} vm, lm')
      else (RM.remove t vm, lm')
    | None -> Impossible.impossible "DLS: Tried to remove unknown reg"
  with Not_found -> Impossible.impossible "DLS: Tried to remove unknown reg"

let remove_mem t _ (vm, lm) =
  try match (RM.find t vm).reg with
    | None -> (RM.remove t vm, lm)
    | Some r as r' -> (RM.add t {reg = r'; mem = None} vm, lm)
  with Not_found -> Impossible.impossible "DLS: Tried to remove unknown mem"
let remove_mem' t (vm, lm as m) =
  try match (RM.find t vm).reg' with
    | None -> (RM.remove t vm, lm)
    | Some r as r' -> (RM.add t {reg' = r'; mem' = false} vm, lm)
  with Not_found -> m

⟨varmap.ml 445b⟩+≡ (443a) <445a 446>
let add_reg t r (vm, lm) =
  try let lp = RM.find t vm in
    let lm' = match lp.reg with
      | Some r_old -> RM.remove r_old lm
      | None -> lm in
    (RM.add t {lp with reg = Some r} vm, RM.add r t lm')
  with Not_found -> (RM.add t {reg = Some r; mem = None} vm, RM.add r t lm)
let add_reg' t r (vm, lm) =
  try let lp = RM.find t vm in
    let lm' = match lp.reg' with
      | Some r_old -> lm_rem' r_old t lm
      | None -> lm in
    (RM.add t {lp with reg' = Some r} vm, lm_add' r t lm')
  with Not_found -> (RM.add t {reg' = Some r; mem' = false} vm, lm_add' r t lm)

```

```

let add_mem t m (vm, lm) =
  try (RM.add t {(RM.find t vm) with mem = Some m} vm, lm)
  with Not_found -> (RM.add t {reg = None; mem = Some m} vm, lm)
let add_mem' t (vm, lm) =
  try (RM.add t {(RM.find t vm) with mem' = true} vm, lm)
  with Not_found -> (RM.add t {reg' = None; mem' = true} vm, lm)

let spill t m (vm, lm) =
  let fail () = Impossible.impossible "DLS: can not spill temp" in
  try match RM.find t vm with
    | {reg = Some r} -> (RM.add t {reg = None; mem = Some m} vm, RM.remove r lm)
    | _ -> fail ()
  with Not_found -> fail ()
let spill' t (vm, lm) =
  let fail () = Impossible.impossible "failed to spill temp" in
  try match RM.find t vm with
    | {reg' = Some r} -> (RM.add t {reg' = None; mem' = true} vm,
                        lm_rem' r t lm)
    | _ -> fail ()
  with Not_found -> fail ()

let var_locs' (vm, _) temp = RM.find temp vm
let var_locs'' (vm, _) temp = RM.find temp vm
let temp_loc' (vm, _) t = try (RM.find t vm).reg' with Not_found -> None
let reg_contents (_, lm) reg = try Some (RM.find reg lm) with Not_found -> None
let reg_contents' (_, lm) reg = try RM.find reg lm with Not_found -> []
let fold f_r f_m (vm, _) zero =
  let app f t v z = match v with Some s -> f t s z | None -> z in
  RM.fold (fun t lp z -> app f_m t lp.mem (app f_r t lp.reg z)) vm zero
let fold' f_r f_m (vm, _) zero =
  let app f t v z = match v with Some s -> f t s z | None -> z in
  let app' f t v z = match v with true -> f t z | false -> z in
  RM.fold (fun t lp z -> app' f_m t lp.mem' (app f_r t lp.reg' z)) vm zero

let filter f (vm, _ as maps) =
  let remove temp {reg = reg} (vm, lm as maps) =
    if f temp then maps
    else (RM.remove temp vm, match reg with Some r -> RM.remove r lm | None -> lm) in
  RM.fold remove vm maps

```

```

⟨varmap.ml 446⟩+≡ (443a) <445b 447a>
let printReg ((s,_,_), i, Register.C n) =
  if n = 1 then Printf.sprintf "%c%d" s i
  else Printf.sprintf "%c%d:%d" s i n
let print msg (_, lm as map) =
  ( Printf.eprintf "%s\nOneLocVarMap:\n" msg
  ; fold (fun t (r:Register.t) () -> Printf.eprintf "%s -> %s\n" (printReg t) (printReg r))
        (fun t m () -> Printf.eprintf "%s -> mem_loc\n" (printReg t))
        map ()
  ; Printf.eprintf " Reverse map:\n"
  ; RM.iter (fun r t -> Printf.eprintf " %s -> %s\n" (printReg r) (printReg t)) lm
  ; flush stderr
  )
let print' msg (_, lm as map) =
  ( Printf.eprintf "%s\nOneLocVarMap:\n" msg
  ; fold' (fun t (r:Register.t) () -> Printf.eprintf "%s -> %s\n" (printReg t) (printReg r))
        (fun t () -> Printf.eprintf "%s -> mem_loc\n" (printReg t))
        map ()
  ; Printf.eprintf " Reverse map:\n"
  ; RM.iter (fun r ts -> Printf.eprintf " %s -> " (printReg r);

```

```

        List.iter (fun t -> Printf.eprintf "%s, " (printReg t)) ts;
        Printf.eprintf "\n") lm
    ; flush stderr
)

⟨varmap.ml 447a⟩+≡ (443a) <446 447b>
let sync_maps inmap (om_vm, _ as outmap) =
  (RM.fold (fun temp loc_pair (im_vm', _ as inmap') ->
    match loc_pair.mem with
    | Some m ->
      if Auxfuns.Option.is_none (pair_map_find temp im_vm').mem then
        add_mem temp m inmap'
      else
        inmap'
    | None -> inmap')
    om_vm inmap, outmap)

⟨varmap.ml 447b⟩+≡ (443a) <447a 448a>
let free_reg_inregs defs live_in live_out (_, lm) regs_used t_live_past_uses r =
  let not_in set = not (RS.mem r set) in
  not (RM.mem r lm) && not_in regs_used && not_in live_in &&
  not (t_live_past_uses && (RS.mem r defs || RS.mem r live_out))

let free_reg_outregs defs live_out (_, lm) regs_used _ r =
  let not_in set = not (RS.mem r set) in
  not (RM.mem r lm) && not_in regs_used && not_in defs && not_in live_out

(* If we have found a register to spill, we may only be able to use it for the
   live-in part of the instruction, or it may be good long-term. *)
type spill = NoneYet
  | LiveInOnly of Register.t
  | LongTerm of Register.t
let alloc_inreg allregs defs live_in live_out (_, lm)
  regs_used live_past_use t =
  (*printTempList "alloc_outreg allregs: " allregs;
  printTempSet "regs_used: " regs_used;
  printTempSet "live_out: " live_out;
  printTempSet "defs: " defs;
  *)
  let rec try_regs rs bst = match rs with
  | [] -> (match bst with LongTerm r -> r | LiveInOnly r -> r
    | NoneYet -> impossf "no register available for temp %s" (printReg t))
  | r::rs ->
    let liveinonly r = match bst with NoneYet -> LiveInOnly r | _ -> bst in
    let longterm r = match bst with LongTerm _ -> bst | _ -> LongTerm r in
    let not_in set = not (RS.mem r set) in
    if not_in regs_used && not_in live_in &&
      (not live_past_use || not_in live_out && not_in defs) then
      if RM.mem r lm then try_regs rs (longterm r)
      else r
    (* won't be live out: *)
    else if not_in regs_used && not_in live_in then try_regs rs (liveinonly r)
    else try_regs rs bst in
  try_regs allregs NoneYet

let alloc_outreg allregs defs live_out (_, lm) regs_used _ t =
  (*printTempList "alloc_outreg allregs: " allregs;
  printTempSet "regs_used: " regs_used;
  printTempSet "live_out: " live_out;
```



```

printTempSet "defs: " defs;
*)
let rec try_regs rs bst = match rs with
| [] -> (match bst with LongTerm r -> r | LiveInOnly r -> r
| NoneYet -> impossf "no register available for temp %s" (printReg t))
| r::rs ->
let longterm r = match bst with LongTerm _ -> bst | _ -> LongTerm r in
let not_in set = not (RS.mem r set) in
if not_in regs_used && not_in live_out && not_in defs then
if RM.mem r lm then try_regs rs (longterm r)
else r
else try_regs rs bst in
try_regs allregs NoneYet

⟨varmap.ml 448a⟩+≡ (443a) <447b
let join (m1, _) (m2, _) =
let upd t lp1 lp2 maps = match (lp1, lp2) with
| ({reg' = Some r1; mem' = false}, {reg' = Some r2; mem' = false})
when Register.eq r1 r2 -> add_reg' t r1 maps
| ({reg' = Some r1; mem' = true}, {reg' = Some r2; mem' = true})
when Register.eq r1 r2 -> add_mem' t (add_reg' t r1 maps)
| _ -> add_mem' t maps in
RM.fold (fun temp lp z -> upd temp lp (try RM.find temp m2 with Not_found -> lp) z)
m1 empty

let eq' ~old:(m2, _) ~new':(m1, _) =
let is_eq m' t lp1 b =
b && try let lp2 = RM.find t m' in
(match (lp1.reg', lp2.reg') with
| (Some r1, Some r2) -> Register.eq r1 r2
| (None, None) -> true
| _ -> false) &&
(lp1.mem' := lp2.mem')
with Not_found -> false in
RM.fold (is_eq m2) m1 true (* only equality on a subset... *)

⟨types for tracking defs and uses 448b⟩≡ (444e 443c) 448c▷
type use_dist = Use of (int * int)

⟨types for tracking defs and uses 448c⟩+≡ (444e 443c) <448b 448d▷
type hwdef_num = (int * int option) (* (n, m): nth def in basic block m *)
type hwdef = Copy of (hwdef_num * int * Register.t * hwdef)
| NonCopy of int

⟨types for tracking defs and uses 448d⟩+≡ (444e 443c) <448c
type def_dist = HW of hwdef
| Temp of (int * int)

```

Q.7 cfg/zipcfg/zipcfg.nw

```

⟨front_zipcfg/zipcfg.ml 448e⟩≡
⟨zipcfg.ml 452g⟩

⟨front_zipcfg/zipcfg.mli 448f⟩≡
⟨zipcfg.mli 449a⟩

```

Q.8 Applicative Control-Flow Graph, Based on Huet's Zipper

```

<zipcfg.mli 449a>≡ (448f) 449b▷
  type uid = Unique.uid
  type label = uid * string

  type regs = Register.SetX.t (* sets of regs for dataflow *)
  type contedge = { kills:regs; defs:regs; node:label; assertion:Rtl.rtl }

  module Rep : sig
    <exposed types for Rep module 450d>
    <declarations of Rep's public functions 450h>
  end

  type graph
  type zgraph

  val empty : graph
  val entry : graph -> zgraph (* focus on edge out of entry node *)
  val exit : graph -> zgraph (* focus on edge into default exit node *)
  val focus : uid -> graph -> zgraph (* focus on edge out of node with uid *)
  val unfocus : zgraph -> graph (* lose focus *)

<zipcfg.mli 449b>+≡ (448f) <449a 449c>
  val splice_focus_entry : zgraph -> graph -> zgraph
  val splice_focus_exit : zgraph -> graph -> zgraph

<zipcfg.mli 449c>+≡ (448f) <449b 449d>
  val add_blocks : graph -> graph -> graph

<zipcfg.mli 449d>+≡ (448f) <449c 449e>
  val uid : unit -> uid
  type exp_of_lbl = label -> Rtl.exp (* exp of code label *)
  type 'a machine = 'a * 'a Mflow.machine * exp_of_lbl (* useful pairing *)
  type nodes = zgraph -> zgraph (* sequence of nodes in Hughes's representation *)
  type cbranch = ifso:label -> ifnot:label -> nodes (* ability to branch conditionally *)

  val label : 'a machine -> label -> nodes (* spans? *)
  val instruction : Rtl.rtl -> nodes
  val stack_adjust : Rtl.rtl -> nodes
  val branch : 'a machine -> label -> nodes
  val jump : 'a machine -> Rtl.exp -> uses:regs -> targets:string list -> nodes
  val cbranch : 'a machine -> Rtl.exp -> cbranch
  val mbranch : 'a machine -> Rtl.exp -> targets:label list -> nodes
  val call : 'a machine -> Rtl.exp -> altrets:contedge list ->
    unwinds_to:contedge list -> cuts_to:contedge list ->
    aborts:bool -> uses:regs -> defs:regs -> kills:regs ->
    reads:string list option -> writes:string list option ->
    spans:Spans.t option -> succ_assn:Rtl.rtl -> nodes
  val cut_to : 'a machine -> Mflow.cut_args -> cuts_to:contedge list ->
    aborts:bool -> uses:regs -> nodes
  val return : Rtl.rtl -> exit:int -> uses:regs -> nodes
  val forbidden : 'a machine -> nodes
  (* control should not reach; causes checked RTE *)

<zipcfg.mli 449e>+≡ (448f) <449d 449f>
  val if_then_else : 'a machine -> cbranch -> t:nodes -> f:nodes -> nodes
  val while_do : 'a machine -> cbranch -> body:nodes -> nodes

<zipcfg.mli 449f>+≡ (448f) <449e 450a>
  val limitcheck : 'a machine -> cbranch -> t:nodes -> nodes

```

```

<zipcfg.mli 450a>+≡ (448f) <449f 450b>
  val make_target : 'a machine -> zgraph -> label * zgraph

<zipcfg.mli 450b>+≡ (448f) <450a 450c>
  val set_spans : zgraph -> Spans.t -> unit (* set spans at node preceding focus *)

<zipcfg.mli 450c>+≡ (448f) <450b 451b>
  val single_middle : Rep.middle -> graph
  val single_last   : Rep.last   -> graph

<exposed types for Rep module 450d>≡ (453a 449a) 450e>
  type 'a edgelist = 'a list (* could be array *)
  <node types 453b>
  type labelkind
    = Userlabel (* user-written; cannot be deleted *)
    | Genlabel  (* generated; can be deleted *)
    | Limitlabel (* generated after limit check; cannot be deleted *)

  type first
    = Entry
    | Label of label * labelkind * Spans.t option ref

  type middle
    = Instruction of Rtl.rtl
    | Stack_adjust of Rtl.rtl

  type last
    = Exit
    | Branch of Rtl.rtl * label
    | Cbranch of Rtl.rtl * label * label (* true, false *)
    | Mbranch of Rtl.rtl * label edgelist (* possible successors *)
    | Call of call
    | Cut of Rtl.rtl * contedge edgelist * regs (* out edges, registers used *)
    | Return of exit_num * Rtl.rtl * regs
    | Jump of Rtl.rtl * regs * string list (* inst, registers used, targets *)
    | Forbidden of Rtl.rtl (* cause a run-time error *)
  and exit_num = int

<exposed types for Rep module 450e>+≡ (453a 449a) <450d 450f>
  type head = First of first | Head of head * middle
  type tail = Last of last | Tail of middle * tail

<exposed types for Rep module 450f>+≡ (453a 449a) <450e 450g>
  type zblock = head * tail

<exposed types for Rep module 450g>+≡ (453a 449a) <450f
  type block = first * tail

<declarations of Rep's public functions 450h>≡ (449a) 451a>
  val id : block -> uid
  val blocklabel : block -> label option (* entry block has no label *)
  val blockkind : block -> labelkind option (* entry block has no kind *)
  val fid : first -> uid
  val entry_uid : uid

  val zip : zblock -> block
  val unzip : block -> zblock

  val first : zblock -> first
  val last : zblock -> last
  val goto_start : zblock -> first * tail
  val goto_end : zblock -> head * last

```

```

<declarations of Rep's public functions 451a>+≡ (449a) <450h 451j>
  val ht_to_first : head -> tail -> first * tail
  val ht_to_last  : head -> tail -> head  * last
  val zipht       : head -> tail -> block

<zipcfg.mli 451b>+≡ (448f) <450c 451c>
  val splice_head : Rep.head -> graph -> graph * Rep.head
  val splice_tail : graph -> Rep.tail -> Rep.tail * graph

<zipcfg.mli 451c>+≡ (448f) <451b 451d>
  val splice_head_only : Rep.head -> graph -> graph

<zipcfg.mli 451d>+≡ (448f) <451c 451e>
  val remove_entry : graph -> Rep.tail * graph

<zipcfg.mli 451e>+≡ (448f) <451d 451f>
  val to_blocks : graph -> Rep.block Unique.Map.t
  val of_blocks : Rep.block Unique.Map.t -> graph (* cheap *)
  val of_block_list : Rep.block list -> graph (* expensive *)
  val openz : zgraph -> Rep.zblock * Rep.block Unique.Map.t
  val tozgraph : Rep.zblock * Rep.block Unique.Map.t -> zgraph

<zipcfg.mli 451f>+≡ (448f) <451e 451g>
  val postorder_dfs : graph -> Rep.block list

<zipcfg.mli 451g>+≡ (448f) <451f 451h>
  val fold_layout : (Rep.block -> label option -> 'a -> 'a) -> 'a -> graph -> 'a

<zipcfg.mli 451h>+≡ (448f) <451g 451i>
  val fold_blocks : (Rep.block -> 'a -> 'a) -> 'a -> graph -> 'a
  val iter_blocks : (Rep.block -> unit) -> graph -> unit

<zipcfg.mli 451i>+≡ (448f) <451h 451m>
  val expand : (Rep.middle -> graph) -> (Rep.last -> graph) -> graph -> graph

<declarations of Rep's public functions 451j>+≡ (449a) <451a 451k>
  val succs : last -> uid list
  val fold_succs : (uid -> 'a -> 'a) -> last -> 'a -> 'a
  val iter_succs : (uid -> unit) -> last -> unit

<declarations of Rep's public functions 451k>+≡ (449a) <451j 451l>
  val mid_instr : middle -> Rtl.rtl
  val last_instr : last -> Rtl.rtl (* may be nop for, e.g., [[Exit]] *)

<declarations of Rep's public functions 451l>+≡ (449a) <451k 451n>
  val is_executable : middle -> bool

<zipcfg.mli 451m>+≡ (448f) <451i 451o>
  val iter_spans : (Spans.t -> unit) -> graph -> unit
  val fold_spans : (Spans.t -> 'a -> 'a) -> graph -> 'a -> 'a

<declarations of Rep's public functions 451n>+≡ (449a) <451l
  val fold_fwd_block :
    (first -> 'a -> 'a) -> (middle -> 'a -> 'a) -> (last -> 'a -> 'a) ->
    block -> 'a -> 'a

<zipcfg.mli 451o>+≡ (448f) <451m 452a>
  val iter_nodes :
    (Rep.first -> unit) -> (Rep.middle -> unit) -> (Rep.last -> unit) -> graph -> unit
  val iter_rtls : (Rtl.rtl -> unit) -> graph -> unit

```

```

⟨zipcfg.mli 452a⟩+≡ (448f) <451o 452b>
  val map_rtls : (Rtl.rtl -> Rtl.rtl) -> graph -> graph
  val map_nodes :
    (Rep.first -> Rep.first) -> (Rep.middle -> Rep.middle) -> (Rep.last -> Rep.last) ->
    graph -> graph

⟨zipcfg.mli 452b⟩+≡ (448f) <452a 452c>
  val new_rtlm : Rtl.rtl -> Rep.middle -> Rep.middle
  val new_rtl1 : Rtl.rtl -> Rep.last -> Rep.last
  val map_rtlm : (Rtl.rtl -> Rtl.rtl) -> Rep.middle -> Rep.middle
  val map_rtl1 :
    map_rtl:(Rtl.rtl -> Rtl.rtl) -> map_assn:(Rtl.rtl -> Rtl.rtl) -> Rep.last -> Rep.last

⟨zipcfg.mli 452c⟩+≡ (448f) <452b 452d>
  val union_over_outedges :
    Rep.last -> noflow:(uid -> regs) -> flow:(contedge -> regs) -> regs

⟨zipcfg.mli 452d⟩+≡ (448f) <452c 452e>
  val iter_outedges :
    Rep.last -> noflow:(uid -> unit) -> flow:(contedge -> unit) -> unit

⟨zipcfg.mli 452e⟩+≡ (448f) <452d 452f>
  val add_inedge_uses : Rep.last -> regs -> regs
  val add_live_spansl : Rep.last -> regs -> regs
  val add_live_spansf : Rep.first -> regs -> regs

⟨zipcfg.mli 452f⟩+≡ (448f) <452e
  val block_before : 'a machine -> (Rtl.exp -> Rtl.rtl) Dag.block -> zgraph ->
    (zgraph * bool)
  val block2cfg : 'a machine -> (Rtl.exp -> Rtl.rtl) Dag.block -> (zgraph * bool)
  val cbranch2cfg : 'a machine -> (Rtl.exp -> Rtl.rtl) Dag.cbranch ->
    ifso:label -> ifnot:label -> zgraph -> (zgraph * bool)

```

Q.8.1 Implementation

```

⟨zipcfg.ml 452g⟩≡ (448e) 453a>
  open Nopoly

  module DG = Dag
  module M = Mflow
  module R = Rtl
  module RSX = Register.SetX
  module U = Unique
  module UM = Unique.Map
  module US = Unique.Set

  let impossf fmt = Printf.kprintf Impossible.impossible fmt
  let unimpf fmt = Printf.kprintf Impossible.unimp fmt
  let ( **> ) f x = f x

  type uid = U.uid
  type label = uid * string
  type exp_of_lbl = label -> Rtl.exp (* exp of code label *)
  type 'a machine = 'a * 'a Mflow.machine * exp_of_lbl (* useful pairing *)
  let uid = U.uid

```

```

<zipcfg.ml 453a>+≡ (448e) <452g 454a>
  type regs = Register.SetX.t (* sets of regs for dataflow *)
  type contedge = { kills:regs; defs:regs; node:label; assertion: Rtl.rtl }

  module Rep = struct
    let entry_uid = U.distinguished_uid
    <exposed types for Rep module 450d>
    <definitions of Rep's public functions 453c>
  end
  open Rep

<node types 453b>≡ (450d)
  type call = {
    ; cal_i : Rtl.rtl
    ; cal_contedges : contedge edgelist
    ; cal_spans : Spans.t option
    ; mutable cal_uses : regs
    ; cal_altrets : int
    ; cal_unwinds_to : int
    ; cal_cuts_to : int
    ; cal_reads : string list option
    ; cal_writes : string list option
  }

<definitions of Rep's public functions 453c>≡ (453a) 453d>
  let fid = function Entry -> entry_uid | Label ((u, _), _, _) -> u
  let id (f, t) = fid f
  let blocklabel (f, t) = match f with
  | Entry -> None
  | Label (l, _, _) -> Some l
  let blockkind (f, t) = match f with
  | Entry -> None
  | Label (_, k, _) -> Some k

<definitions of Rep's public functions 453d>+≡ (453a) <453c 453e>
  let rec ht_to_first head tail = match head with
  | First f -> f, tail
  | Head (h, m) -> ht_to_first h (Tail (m, tail))

  let goto_start (h, t) = ht_to_first h t

  let rec ht_to_last head tail = match tail with
  | Last l -> head, l
  | Tail (m, t) -> ht_to_last (Head (head, m)) t

  let goto_end (h, t) = ht_to_last h t

  let zip = goto_start
  let zipht = ht_to_first
  let unzip (n, ns) = (First n, ns)

<definitions of Rep's public functions 453e>+≡ (453a) <453d 457d>
  let rec lastt = function (Last l) -> l | Tail (_, t) -> lastt t
  let last (h, t) = lastt t
  let first =
    let rec first = function (First f) -> f | Head (h, _) -> first h in
    fun (h, t) -> first h

```

```

<zipcfg.ml 454a>+≡ (448e) <453a 454b>
  module type BLOCKS = sig
    type t
    val empty : t
    val insert : block -> t -> t
    val find : t -> uid -> block
    val focus : t -> uid -> block * t
    val focusp : t -> (block -> bool) -> block * t
    val union : t -> t -> t
      (* keep larger set on the right *)

    val fold : (block -> 'a -> 'a) -> t -> 'a -> 'a
    val iter : (block -> unit) -> t -> unit
  end

  module Blocks : BLOCKS with type t = block UM.t = struct
    type t = block UM.t
    let empty = UM.empty
    let insert block = UM.add (id block) block
    let find blocks u = UM.find u blocks
    let focusp blocks p = UM.splitp (fun _ b -> p b) blocks
    let focus blocks u = UM.split u blocks
    let union = UM.union
    let fold f blocks z = UM.fold (fun _ b z -> f b z) blocks z
    let iter = UM.iter
  end

<zipcfg.ml 454b>+≡ (448e) <454a 454c>
  type graph = Blocks.t
  type zgraph = zblock * Blocks.t
  type nodes = zgraph -> zgraph (* sequence of nodes in Hughes's representation *)
  type cbranch = ifso:label -> ifnot:label -> nodes (* ability to branch conditionally *)

  let of_blocks g = g
  let to_blocks g = g
  let openz z = z
  let tozgraph z = z

<zipcfg.ml 454c>+≡ (448e) <454b 454d>
  let empty = Blocks.insert (Entry, Last Exit) Blocks.empty

  let focus uid blocks =
    let (b, bs) = Blocks.focus blocks uid in
    unzip b, bs
  let entry blocks = focus entry_uid blocks
  let exit g =
    let is_exit b = match last (unzip b) with Exit -> true | _ -> false in
    let (b, bs) = Blocks.focusp g is_exit in
    let (h, l) = goto_end (unzip b) in
    ((h, Last l), bs)

  let unfocus (bz, bs) = Blocks.insert (zip bz) bs

<zipcfg.ml 454d>+≡ (448e) <454c 455c>
  let consm middle ((h, t), blocks) = ((h, Tail (middle, t)), blocks)

  let instruction rtl g = consm (Instruction rtl) g
  let stack_adjust rtl g = consm (Stack_adjust rtl) g

  let unreachable = function

```

```

| Last (Branch _ | Forbidden _ | Exit) -> ()
| t ->
  let pr s = Debug.eprintf "zipcfg" s in
  pr "warning: unreachable code?\n";
  let rec warn = function
    | Tail (m, t) -> pr " %s\n" (Rtlutil.ToString.rtl (mid_instr m)); warn t
    | Last l -> pr " %s\n" (Rtlutil.ToString.rtl (last_instr l)) in
  warn t

let consl last ((head, tail), blocks) = unreachable tail; ((head, Last last), blocks)

⟨mutually recursive graph construction 455a⟩≡ (463) 455b▷
let rec consl' m b last ((head, tail), blocks) =
  unreachable tail;
  fst (block_before m b ((head, Last last), blocks))
and branch (p, machine, l2e as m) target =
  let (b, r) = machine.M.goto.M.embed p (l2e target) in
  consl' m b (Branch (r, target))
and jump (p, machine, l2e as m) e ~uses ~targets =
  let (b, r) = machine.M.jump.M.embed p e in
  consl' m b (Jump (r, uses, targets))

⟨mutually recursive graph construction 455b⟩+≡ (463) ◁455a 455d▷
and cbranch (p, machine, l2e as m) guard ~ifso ~ifnot succ =
  let cbr = machine.M.branch.M.embed p guard in
  fst (cbranch2cfg m cbr ~ifso ~ifnot succ)
and mbranch (p, machine, l2e as m) e ~targets ((head, tail), blocks) =
  let (b, r) = machine.M.goto.M.embed p e in
  unreachable tail;
  fst (block_before m b ((head, Last (Mbranch (r, targets))), blocks))
and cut_to (p, machine, l2e as m) cut_args ~cuts_to ~aborts ~uses =
  let (b, r) = machine.M.cutto.M.embed p cut_args in
  consl' m b (Cut (r, cuts_to, uses))

⟨zipcfg.ml 455c⟩+≡ (448e) ◁454d 455f▷
let return rtl ~exit ~uses = consl (Return (exit, rtl, uses))
let forbidden (_, machine, _) = consl (Forbidden machine.M.forbidden)

⟨mutually recursive graph construction 455d⟩+≡ (463) ◁455b 455e▷
and label' user (p, machine, l2e as m) lbl ((head, tail), blocks) =
  let (b, r) = machine.M.goto.M.embed p (l2e lbl) in
  fst (block_before m b ((head, Last (Branch (r, lbl))),
    Blocks.insert (Label (lbl, user, ref None), tail) blocks))

⟨mutually recursive graph construction 455e⟩+≡ (463) ◁455d 456a▷
and label x = label' Userlabel x
and privatelabel x = label' Genlabel x
and limitlabel x = label' Limitlabel x

⟨zipcfg.ml 455f⟩+≡ (448e) ◁455c 456f▷
let check_single_exit g =
  let check block found = match last (unzip block) with
    | Exit when not found -> true
    | _ -> found in
  if not (Blocks.fold check g false) then
    impossf "graph does not have an exit"

```



```

⟨mutually recursive graph construction 456a⟩+≡ (463) <455e 456b>
and make_target machine ((b, bs) as gz) = match b with
| First (Label (u, _, _)), _ -> u, gz
| First Entry, Last (Branch (_, u)) -> u, gz
| First Entry, _ ->
    let lbl = (uid (), Idgen.label "branch target") in
    let gz = branch machine lbl **> privatelabel machine lbl **> gz in
    lbl, gz
| _ -> impossf "focus not on entry"

⟨mutually recursive graph construction 456b⟩+≡ (463) <456a 456c>
and if_then_else machine cbranch ~t ~f g =
    let endif, g = make_target machine g in
    let not, g = make_target machine (f g) in
    let g = branch machine endif g in
    let so, g = make_target machine (t g) in
    cbranch ~ifso:so ~ifnot:not g

⟨mutually recursive graph construction 456c⟩+≡ (463) <456b 456d>
and while_do machine cbranch ~body g =
    let lbl = (uid (), Idgen.label "loop head") in
    let endwhile, g = make_target machine g in
    let body, g = make_target machine (body (branch machine lbl g)) in
    let g = cbranch ~ifso:body ~ifnot:endwhile g in
    label machine lbl g

⟨mutually recursive graph construction 456d⟩+≡ (463) <456c 456e>
and limitcheck machine cbranch ~t g =
    let endif = (uid (), Idgen.label "post-limitcheck label") in
    let g = limitlabel machine endif g in
    let so, g = make_target machine (t g) in
    cbranch ~ifso:so ~ifnot:endif g

⟨mutually recursive graph construction 456e⟩+≡ (463) <456d 462f>
and call (p, machine, l2e as m) exp ~altrets ~unwinds_to ~cuts_to ~aborts
    ~uses ~defs ~kills ~reads ~writes ~spans ~succ_assn succ =
    let lbl = (uid (), Idgen.label "call successor") in
    let succ_ce = { kills = kills; defs = defs; node = lbl; assertion = succ_assn } in
    let edgelist = succ_ce :: List.flatten [altrets; unwinds_to; cuts_to] in
    let (b, r) = machine.M.call.M.embed p exp in
    let call =
        { cal_i = r; cal_contedges = edgelist; cal_spans = spans;
          cal_uses = uses; cal_altrets = List.length altrets;
          cal_unwinds_to = List.length unwinds_to; cal_cuts_to = List.length cuts_to;
          cal_reads = reads; cal_writes = writes; } in
    let succ = privatelabel m lbl succ in
    match succ with
    | (First Entry, Last (Branch (_, lbl'))), blocks when lbl' == lbl ->
        fst (block_before m b ((First Entry, Last (Call call)), blocks))
    | _ -> impossf "internal error in call constructor"

⟨zipcfg.ml 456f⟩+≡ (448e) <455f 457a>
let set_spans (bz, blocks) spans = match bz with
| First (Label (l, u, r)), _ -> r := Some spans
| _ -> impossf "setting spans on non-label"

```

<zipcfg.ml 457a>+≡ (448e) <456f 457b>

```
let iter_spans f g =
  let span s = match s with Some s -> f s | None -> () in
  let first = function Label (_, _, s) -> span (!s) | _ -> () in
  let block (f, t) =
    first f;
    match lastt t with
    | Call c -> span c.cal_spans
    | _ -> () in
  Blocks.iter block g
```

<zipcfg.ml 457b>+≡ (448e) <457a 457c>

```
let fold_spans f g z =
  let span s z = match s with Some s -> f s z | None -> z in
  let first f z = match f with Label (_, _, s) -> span (!s) z | _ -> z in
  let block (f, t) z =
    let z = first f z in
    match lastt t with
    | Call c -> span c.cal_spans z
    | _ -> z in
  Blocks.fold block g z
```

<zipcfg.ml 457c>+≡ (448e) <457b 458a>

```
let add_blocks blocks (focus, newblocks) =
  match focus with
  | First Entry, Last (Branch _ | Forbidden _ | Exit) ->
    let rec add_block blocks = match goto_end (unzip block) with
    | First (Label _), Exit -> blocks
    | _, Exit -> impossf "exit contains nontrivial code"
    | _ -> Blocks.insert block blocks in
    Blocks.fold add newblocks blocks
  | First Entry, _ -> impossf "entry contains nontrivial code"
  | _ -> impossf "focus not on entry"
```

<definitions of Rep's public functions 457d>+≡ (453a) <453e 458c>

```
let succs = function
| Exit -> []
| Branch (_, l) -> [fst l]
| Cbranch (_, t, f) -> [fst f; fst t] (* order meets layout constraint *)
| Mbranch (_, edges) -> List.map fst edges
| Call c -> List.map (fun e -> fst e.node) c.cal_contedges
| Cut (_, edges, _) -> List.map (fun e -> fst e.node) edges
| Return _ -> []
| Jump _ -> []
| Forbidden _ -> []

let fold_succs f t z = match t with
| Exit -> z
| Branch (_, l) -> f (fst l) z
| Cbranch (_, te, fe) -> f (fst te) (f (fst fe) z) (* order meets layout constraint *)
| Mbranch (_, edges) -> List.fold_left (fun z e -> f (fst e) z) z edges
| Call c -> List.fold_left (fun z e -> f (fst e.node) z) z c.cal_contedges
| Cut (_, edges, _) -> List.fold_left (fun z e -> f (fst e.node) z) z edges
| Return _ -> z
| Jump _ -> z
| Forbidden _ -> z
```

```
let iter_succs f t = fold_succs (fun t () -> f t) t ()
```

<zipcfg.ml 458a>+≡ (448e) <457c 458b>

```

let postorder_dfs g =
  let entry, blocks = entry g in
  let rec vnode block cont acc visited =
    let u = id block in
    if US.mem u visited then
      cont acc visited
    else
      vchildren block (get_children block) cont acc (US.add u visited)
  and get_children block =
    let uids = succs (last (unzip block)) in
    (*List.map (Blocks.find blocks) uids*)
    List.fold_left (fun rst bid -> try Blocks.find blocks bid :: rst
                                with Not_found -> rst) [] uids
  and vchildren block children cont acc visited =
    let rec next children acc visited = match children with
      | [] -> cont (block :: acc) visited
      | n::rst -> vnode n (next rst) acc visited in
    next children acc visited in
  vnode (zip entry) (fun acc _visited -> acc) [] US.empty

```

<zipcfg.ml 458b>+≡ (448e) <458a 458d>

```

let fold_layout f z g =
  let nextlabel (f, t) = match f with
    | Entry -> impossf "entry as successor"
    | Label (l, _, _) -> Some l in
  let rec fold blocks z = match blocks with
    | [] -> z
    | [b] -> f b None z
    | b1 :: b2 :: bs -> fold (b2 :: bs) (f b1 (nextlabel b2) z) in
  fold (postorder_dfs g) z

```

<definitions of Rep's public functions 458c>+≡ (453a) <457d 460>

```

let fold_fwd_block first middle last (f, t) z =
  let z = first f z in
  let rec tail t z = match t with
    | Tail (m, t) -> tail t (middle m z)
    | Last l -> last l z in
  tail t z

```

<zipcfg.ml 458d>+≡ (448e) <458b 458e>

```

let iter_nodes first middle last g =
  let block (f, t) =
    let () = first f in
    let rec tail t = match t with
      | Tail (m, t) -> (middle m; tail t)
      | Last l -> last l in
    tail t in
  UM.iter block g

let iter_rtls rfun g =
  iter_nodes
    (fun f -> ()) (fun m -> rfun (mid_instr m)) (fun l -> rfun (last_instr l)) g

```

<zipcfg.ml 458e>+≡ (448e) <458d 459a>

```

let fold_blocks f z bs = UM.fold (fun _ -> f) bs z
let iter_blocks = UM.iter

```

```

<zipcfg.ml 459a>+≡ (448e) <458e 459b>
let new_rtl1 rtl l = match l with
| Exit -> l
| Branch (r, l) -> Branch (rtl, l)
| Cbranch (r, t, f) -> Cbranch (rtl, t, f)
| Mbranch (r, tgts) -> Mbranch (rtl, tgts)
| Call c -> Call { c with cal_i = rtl }
| Cut (r, es, uses) -> Cut (rtl, es, uses)
| Return (i, r, uses) -> Return (i, rtl, uses)
| Jump (r, uses, tgts) -> Jump (rtl, uses, tgts)
| Forbidden r -> Forbidden (rtl)

let new_rtlm rtl m = match m with
| Instruction r -> Instruction rtl
| Stack_adjust r -> Stack_adjust rtl

<zipcfg.ml 459b>+≡ (448e) <459a 459c>
let map_rtl1 ~map_rtl ~map_assn l =
let map_ces ces =
List.map (fun ce -> { ce with assertion = map_assn ce.assertion }) ces in
match l with
| Exit -> l
| Branch (r, l) -> Branch (map_rtl r, l)
| Cbranch (r, t, f) -> Cbranch (map_rtl r, t, f)
| Mbranch (r, tgts) -> Mbranch (map_rtl r, tgts)
| Call c -> Call { c with cal_i = map_rtl c.cal_i ;
cal_contedges = map_ces c.cal_contedges }
| Cut (r, es, uses) -> Cut (map_rtl r, map_ces es, uses)
| Return (i, r, uses) -> Return (i, map_rtl r, uses)
| Jump (r, uses, tgts) -> Jump (map_rtl r, uses, tgts)
| Forbidden r -> Forbidden (map_rtl r)

let map_rtlm map m = match m with
| Instruction r -> Instruction (map r)
| Stack_adjust r -> Stack_adjust (map r)

let map_rtls map g =
let block (f, t) =
let rec tail t = match t with
| Tail (m, t) -> Tail (map_rtlm map m, tail t)
| Last l -> Last (map_rtl1 ~map_rtl:map ~map_assn:map l) in
(f, tail t) in
UM.map block g

<zipcfg.ml 459c>+≡ (448e) <459b 459d>
let map_nodes first middle last g =
let block (f, t) =
let rec tail t = match t with
| Tail (m, t) -> Tail (middle m, tail t)
| Last l -> Last (last l) in
(first f, tail t) in
UM.map block g

<zipcfg.ml 459d>+≡ (448e) <459c 461a>
(* should this fn be defined on exi and ill?
when there are cont edges, do we still need to cover the regular edges?*)
let (++) = RSX.union
let union_over_outedges node ~noflow ~flow =
let noflow (u, l) = noflow u in
let union_contedges ce = List.fold_left (fun r s -> r ++ flow s) RSX.empty ce in

```

```

match node with
| Call c -> union_contedges c.cal_contedges
| Cut (_, es, _) -> union_contedges es
| Cbranch (c, t, f) -> noflow t ++ noflow f
| Mbranch (_, ls) -> List.fold_left (fun r s -> r ++ noflow s) RSX.empty ls
| Branch (_, l) -> noflow l
| Return (_, _, regs) -> regs
| Jump _
| Exit
| Forbidden _ -> RSX.empty

let iter_outedges node ~noflow ~flow =
  let noflow (u, l) = noflow u in
  match node with
  | Call c -> List.iter flow c.cal_contedges
  | Cut (_, es, _) -> List.iter flow es
  | Cbranch (c, t, f) -> (noflow t; noflow f)
  | Mbranch (_, ls) -> List.iter noflow ls
  | Branch (_, l) -> noflow l
  | Jump _
  | Return _
  | Exit
  | Forbidden _ -> ()

let add_inedge_uses node regs =
  let reg_add = RSX.fold RSX.add in
  match node with
  | Call c -> reg_add c.cal_uses regs
  | Cut (_, _, uses) -> reg_add uses regs
  | Jump (_, uses, _) -> reg_add uses regs
  | Return (_, _, uses) -> reg_add uses regs
  | Exit | Branch _ | Cbranch _ | Mbranch _ | Forbidden _ -> regs

let span_add spans rst = match spans with
| Some ss -> Spans.fold_live_locs RSX.add ss rst
| None -> rst

let add_live_spansl node regs = match node with
| Call c -> span_add c.cal_spans regs
| _ -> regs

let add_live_spansf node regs = match node with
| Label (_, _, sp) -> span_add (!sp) regs
| Entry -> regs

```

<definitions of Rep's public functions 460>+≡

(453a) <458c

```

let mid_instr m = match m with
| Instruction r -> r
| Stack_adjust r -> r

```

```

let is_executable _ = true

```

```

let nop = Rtl.par []
let last_instr l = match l with
| Exit -> nop
| Branch (r, _) -> r
| Cbranch (r, _, _) -> r
| Mbranch (r, _) -> r
| Call c -> c.cal_i
| Cut (r, _, _) -> r

```

```

| Return (_, r, _) -> r
| Jump (r, _, _) -> r
| Forbidden r -> r

⟨zipcfg.ml 461a⟩+≡ (448e) <459d 461b>
let of_block_list blocks =
  List.fold_left (fun m b -> Blocks.insert b m) Blocks.empty blocks

⟨zipcfg.ml 461b⟩+≡ (448e) <461a 461c>
let prepare_for_splicing graph single multi =
  let gentry, gblocks = Blocks.focus graph entry_uid in
  if UM.is_empty gblocks then
    ((match last (unzip gentry) with Exit -> () | _ -> impossf "bad single block");
     single (snd gentry)
    )
  else
    let gexit, gblocks = exit gblocks in
    let gh, gl = goto_end gexit in
    (match gl with Exit -> () | _ -> impossf "exit is not exit?!");
    multi ~entry:(snd gentry) ~exit:gh ~others:gblocks

let _ = (prepare_for_splicing :
graph ->
(Rep.tail -> 'answer) ->
(entry:Rep.tail -> exit:Rep.head -> others:Rep.block Unique.Map.t -> 'answer) ->
'answer)

⟨zipcfg.ml 461c⟩+≡ (448e) <461b 461d>
let splice_head head g =
  check_single_exit g;
  let splice_one_block tail' = match ht_to_last head tail' with
  | head, Exit -> Blocks.empty, head
  | _ -> impossf "spliced graph without exit" in
  let splice_many_blocks ~entry ~exit ~others =
    Blocks.insert (zipht head entry) others, exit in
  prepare_for_splicing g splice_one_block splice_many_blocks

⟨zipcfg.ml 461d⟩+≡ (448e) <461c 461e>
let splice_tail g tail =
  check_single_exit g;
  let splice_one_block tail' = (* return tail' .. tail *)
    match ht_to_last (First Entry) tail' with
    | head', Exit ->
      (match ht_to_first head' tail with
      | Entry, t -> (t, Blocks.empty)
      | _ -> impossf "entry in; garbage out")
    | _ -> impossf "spliced single block without Exit" in
  let splice_many_blocks ~entry ~exit ~others =
    (entry, Blocks.insert (zipht exit tail) others) in
  prepare_for_splicing g splice_one_block splice_many_blocks

⟨zipcfg.ml 461e⟩+≡ (448e) <461d 462a>
let splice_focus_entry ((head, tail), blocks) g =
  let tail, blocks' = splice_tail g tail in
  ((head, tail), Blocks.union blocks' blocks)

let splice_focus_exit ((head, tail), blocks) g =
  let blocks', head = splice_head head g in
  ((head, tail), Blocks.union blocks' blocks)

```

```

<zipcfg.ml 462a>+≡ (448e) <461e 462b>
  let splice_head_only head graph =
    let gentry, gblocks = Blocks.focus graph entry_uid in
    match gentry with
    | Entry, tail -> Blocks.insert (zipht head tail) gblocks
    | _ -> impossf "splice graph does not start with entry"

<zipcfg.ml 462b>+≡ (448e) <462a 462c>
  let remove_entry graph =
    let gentry, gblocks = Blocks.focus graph entry_uid in
    match gentry with
    | Entry, tail -> tail, gblocks
    | _ -> impossf "removing nonexistent entry"

<zipcfg.ml 462c>+≡ (448e) <462b 462d>
  let expand expand_middle expand_last graph =
    let expand_block block expanded =
      let rec expand_tail h t expanded = match t with
      | Tail (m, t) ->
          let g, h = splice_head h (expand_middle m) in
          expand_tail h t (Blocks.union g expanded)
      | Last l ->
          Blocks.union (splice_head_only h (expand_last l)) expanded in
      let (f, t) = block in
      expand_tail (First f) t expanded in
    Blocks.fold expand_block graph Blocks.empty

<zipcfg.ml 462d>+≡ (448e) <462c 462e>
  let single_middle m =
    let block = (Entry, Tail (m, Last Exit)) in
    Blocks.insert block Blocks.empty

  let single_last l =
    let block = (Entry, Last l) in
    Blocks.insert block Blocks.empty

<zipcfg.ml 462e>+≡ (448e) <462d 463>
  let modified = ref false

<mutually recursive graph construction 462f>+≡ (463) <456e 462g>
  and block_before' m block succ =
    let rec before b = match b with
    | DG.Seq (DG.Nop, b) -> before b
    | DG.Seq (b, DG.Nop) -> before b
    | DG.Rtl i -> modified := true; instruction i succ
    | DG.Seq (b, b') -> block_before' m b (before b')
    | DG.If (c, t, f) ->
        if_then_else m (cbranch2cfg' m c) ~t:(block_before' m t) ~f:(block_before' m f)
        succ
    | DG.While (c, b) -> while_do m (cbranch2cfg' m c) ~body:(block_before' m b) succ
    | DG.Nop -> succ in
    before block
  and block2cfg' machine block = block_before' machine block (entry empty)

<mutually recursive graph construction 462g>+≡ (463) <462f>
  and cbranch2cfg' (_, _, l2e as machine) c ~ifso ~ifnot succ =
    let nodemap = ref DG.empty in
    let rec cbi (br, tk, fk) succ =
      let tlbl, succ = cbr tk succ in
      let flbl, succ = cbr fk succ in

```

```

    cons1 (Cbranch (br (l2e tlbl), tlbl, flbl)) succ
and cbr c succ = match c with
| DG.Exit p          -> (if p then ifso else ifnot), succ
| DG.Test  (b, i) -> make_target machine (block_before' machine b (cbi i succ))
| DG.Shared (u, c) -> shared u c succ
and shared u c succ =
  try DG.lookup u (!nodemap), succ with
  | Not_found ->
      let lbl, succ = cbr c succ in
      nodemap := DG.insert u lbl (!nodemap);
      (lbl, succ) in
modified := true;
let lbl, g = cbr c succ in
branch machine lbl g
and block_before m b s =
  modified := false;
  (block_before' m b s, !modified)
and block2cfg m b =
  modified := false;
  (block2cfg' m b, !modified)
and cbranch2cfg m c ~ifso ~ifnot s =
  modified := false;
  (cbranch2cfg' m c ifso ifnot s, !modified)

```

⟨zipcfg.ml 463⟩+≡

⟨mutually recursive graph construction 455a⟩

(448e) <462e

Appendix R

codegen

R.1 codegen/ast2ir.nw

```
<front_ir/ast2ir.ml 464a>≡  
  <ast2ir.ml 464e>
```

```
<front_ir/ast2ir.mli 464b>≡  
  <ast2ir.mli 464d>
```

R.2 Translation to Intermediate Representation

```
<type imports 464c>≡ (465e 464d)  
  type tgt = Preast2ir.tgt =  
    T of (basic_proc, (Rtl.exp -> Automaton.t), Call.t) Target.t  
  and basic_proc = Preast2ir.basic_proc  
  type proc      = Preast2ir.proc  
  type old_proc  = Preast2ir.old_proc  
  
<ast2ir.mli 464d>≡ (464b)  
  <type imports 464c>  
  val set_headroom : int -> unit  
  val translate : tgt  
    -> proc Fenv.Clean.env'  
    -> optimizer: (proc -> unit)  
    -> defineglobals: bool  
    -> proc Nelab.compunit  
    -> unit (* side-effects the assembler in the environment *)
```

R.3 Implementation: Translate to intermediate representation

R.3.1 Abbreviations, utilities, and type definitions

```
<ast2ir.ml 464e>≡ (464a) 465a▷  
  module A = Ast  
  module AT = Automaton  
  module C = Call  
  module Dn = Rtl.Dn  
  module E = Error  
  module F = Fenv.Clean  
  module FE = Fenv  
  module G = Zipcfg
```

```

module N = Nelab
module R = Rtl
module RO = Rewrite.Ops
module RP = Rtl.Private
module RU = Rtlutil
module RS = Register.Set
module RSX= Register.SetX
module S = Elabstmt
module SM = Strutil.Map
module SP = Spans
module T = Target
module Up = Rtl.Up
module W = Rtlutil.Width

type aligned = int
type label   = G.uid * string
let genlabel s =
  let s = Idgen.label s in
  (G.uid (), s)

⟨ast2ir.ml 465a⟩+≡ (464a) <464e 465e>
  ⟨utilities(ast2ir.nw) 465b⟩

⟨utilities(ast2ir.nw) 465b⟩≡ (465a) 465c>
  let rsx = Register.rset_to_rxset

⟨utilities(ast2ir.nw) 465c⟩+≡ (465a) <465b 465d>
  let (<<) f g = fun x -> f (g x) (* function composition *)
  let impossf x = Printf.kprintf Impossible.impossible x
  let unimpf x = Printf.kprintf Impossible.unimp x

⟨utilities(ast2ir.nw) 465d⟩+≡ (465a) <465c 476b>
  let ( **> ) f x = f x

⟨ast2ir.ml 465e⟩+≡ (464a) <465a 465f>
  ⟨type imports 464c⟩
  ⟨module K, for continuation info 466b⟩
  ⟨types for nonvolatile registers 473g⟩

```

R.3.2 Overall structure of the translation

```

⟨ast2ir.ml 465f⟩+≡ (464a) <465e 472b>
  (* I have to choose the value *)
  let headroom = ref 1024
  let set_headroom n = headroom := n
  ⟨definition of type blocklists 473b⟩
  let translate target env ~optimizer ~defineglobals =
    let T target = target in
    let pointersize = target.T.pointersize in
    let asm = F.asm env in
    ⟨environment-independent support for formals, actuals, and results 466a⟩
  in
  ⟨definition of proc, which translates one procedure 472e⟩
  in
  ⟨definition of globals 476d⟩
  in
  ⟨function datum, for initialized and uninitialized data 476c⟩
  in
  ⟨function program, which translates an entire program 477b⟩
  in
  program

```

R.3.3 The translation of actual parameters, formal parameters, and results

<environment-independent support for formals, actuals, and results 466a> $\equiv$ (465f)

```

let convert_parms kind_parm_aligned cvt =
  let add x (wkas, parms) =
    let kind, parm, a = kind_parm_aligned x in
    let parm, w = cvt parm in
    (w, kind, a) :: wkas, parm :: parms in
  fun conv l ->
    let hws, parms = List.fold_right add l ([], []) in
    conv hws parms

```

R.3.4 The translation of continuations and flow annotations

<module K, for continuation info 466b> $\equiv$ (465e)

```

module K = struct
  type convention = string
  type 'i t =
    {
      unwind_pc   : label      (* uids may or may not be used *)
    ; unwind_sp   : Rtl.loc
    ; cut_pc      : label
    ; mutable return_pcs : (convention * label list) list
    ; escapes     : bool       (* properties of how it is used *)
    ; cut_to      : bool
    ; unwound_to  : bool
    ; formals     : (string * FE.variable * aligned) list
                  (* args (kind, variable index) to the continuation *)
    ; cut_in      : (Block.t -> Rtl.rtl) C.answer
                  (* move vals at cut (uses rep) *)
    ; return_in   : (Block.t -> Rtl.rtl) C.answer
                  (* move vals at also returns *)
    ; rep         : Contn.t     (* representation as C-- value (incl block) *)
    ; base        : Block.t     (* base address; to be composed with rep *)
    ; convention  : string
    ; succ        : label       (* filled in by 2nd pass time *)
    ; mutable spans : (Bits.bits * Reloc.t) list
                  (* user-defined spans at the continuation *)
    }
  <definition of K.mk, which initializes continuation information 466c>
end

```

<definition of K.mk, which initializes continuation information 466c> $\equiv$ (466b) 467b>

```

let mk to_cc name den rep base ~formals ~cut_in ~return_in ~unwind_sp =
  let ccname cc = (to_cc cc).C.name in
  { escapes      = den.FE.escapes
  ; cut_to       = den.FE.cut_to
  ; unwound_to   = den.FE.unwound_to
  ; unwind_pc    = genlabel "unwind_entry"
  ; unwind_sp    = unwind_sp
  ; cut_pc       = genlabel "cut_entry"
  ; return_pcs   =
    <set the return pcs 467a>
  ; rep          = rep
  ; base         = base
  ; convention    = den.FE.convention
  ; formals      = formals
  ; cut_in       = cut_in
  ; return_in    = return_in
  ; succ         = genlabel "start of continuation code"

```

```

; spans      = []
}

⟨set the return pcs 467a⟩≡ (466c)
List.map (fun cc -> (ccname cc, [])) den.FE.returned_to

⟨definition of K.mk, which initializes continuation information 467b⟩+≡ (466b) ◁466c
let get_return_pc cc k =
  let labels = try List.assoc cc.C.name k.return_pcs
    with Not_found -> impossf "Unknown alt-return convention %s" cc.C.name in
  let new_label = genlabel "return_entry" in
  let new_entry = (cc.C.name, new_label :: labels) in
  k.return_pcs <- new_entry :: (List.remove_assoc cc.C.name k.return_pcs);
  new_label

⟨function extend_cont, for adding flow-graph info to continuations 467c⟩≡ (473f)
let extend_cont name den =
  let cc      = Call.get_cc target den.FE.convention in
  let args    = den.FE.formals in
  let cut_in  = cformals cc.C.cut_parms.C.in' args in
  let return_in = cformals cc.C.results.C.in' args in
  let rep     = Contn.with_overflow target cut_in.C.overflow in
  K.mk (Call.get_cc target) name den rep den.FE.base ~formals:args ~cut_in ~return_in
    ~unwind_sp:(to_mloc proc_cc.C.stable_sp_loc) in

⟨supporting functions for translating statements 467d⟩≡ (469c) 470a▷
let rec contStmt2 label succ =
  let k = continuation label in
  k.K.spans <- props;
  G.forbidden m **>
  G.label m k.K.succ **>
  succ

⟨function continuations, which translates flow annotations 467e⟩≡ (474d)
let continuations cc ast =
  let volregs = cc.C.volregs in (* volatile registers *)
  let allregs = RS.union volregs cc.C.pre_nvregs in (* all registers *)
  let (--)    = RS.diff in
  let as_cut_to k =
    let defs = k.K.cut_in.C.reg in
    { G.defs = rsx defs; G.kills = rsx (allregs -- defs); G.node = k.K.cut_pc
    ; G.assertion = k.K.cut_in.C.insp (Contn.rep k.K.rep) } in
  let as_unwinds k =
    { G.defs = RSX.empty; G.kills = rsx volregs; G.node = k.K.unwind_pc
    ; G.assertion = proc_cc.C.sp_on_unwind proc_cc.C.stable_sp_loc } in
    (* defs are vars and so can't be a register set. See [[splice_in_unwind_to_entry]] *)
  let as_returns k =
    let pc = K.get_return_pc cc k in
    let defs = k.K.return_in.C.reg in
    { G.defs = rsx defs; G.kills = rsx (volregs -- defs); G.node = pc
    ; G.assertion = k.K.return_in.C.insp k.K.return_in.C.overflow } in
  let contmap f = List.map (f << continuation) in
  { S.cuts      = contmap as_cut_to ast.S.cuts;
    S.unwinds   = contmap as_unwinds ast.S.unwinds;
    S.areturns  = contmap as_returns ast.S.areturns;
    S.returns   = ast.S.returns; S.aborts = ast.S.aborts; } in
let ccontinuations cc ast =
  let ast' = { S.cuts = ast.S.ccuts; S.aborts = ast.S.caborts;
    S.unwinds = []; S.areturns = []; S.returns = false; } in
  let c = continuations cc ast' in
  { S.ccuts = c.S.cuts; S.caborts = c.S.aborts }

```

```

⟨definition of insert_init_cont_nodes 468a⟩≡ (474d)
let insert_init_cont_nodes contmap init_label g =
  let one_node cname k g =
    if k.K.escapes then
      let pc = exp_of_code_label k.K.cut_pc in
      let sp = proc_cc.C.stable_sp_loc in
      let spans = SP.to_spans ~inalloc ~outalloc:(to_mloc sp) ~ra:saved_ra
        ~users:k.K.spans ~csregs:nvr_temps ~conts:[]
        ~sds:sd_locs ~vars:(var_array ()) in
      let cut_g = G.focus (fst k.K.cut_pc) (G.unfocus g) in
      let () = G.set_spans cut_g spans in
      let blockname block =
        match G.Rep.blocklabel block with
        | Some (_, s) -> s
        | None -> "<entry block>" in
      let _focused_blockname g =
        let b, _ = G.openz g in
        let b = G.Rep.zip b in
        blockname b in
      let pc_sp = { Mflow.new_pc = pc; Mflow.new_sp = sp } in
      let init = G.instruction (Contn.init_code k.K.rep pc_sp) (G.entry G.empty) in
      G.splice_focus_entry g (G.unfocus init)
    else
      g in
  G.unfocus (Strutil.Map.fold one_node contmap (G.focus (fst init_label) g))

```

```

⟨functions that translate statements, including stmts 468b⟩≡ (474d) 468c▷
let rec finish_compiling_continuation k g =
  let g =
    if k.K.cut_to then
      splice_in_cut_to_entry k g
    else if k.K.escapes then
      impossf "Continuation escapes but is not annotated with also cuts to"
    else g in
  let g = splice_in_return_to_entries k g in
  let g = if k.K.unwound_to then splice_in_unwind_to_entry k g else g in
  g

```

```

⟨functions that translate statements, including stmts 468c⟩+≡ (474d) <468b 468d>
and splice_in_cut_to_entry k g =
  let in' = k.K.cut_in in
  let prolog =
    G.label m k.K.cut_pc **>
    G.stack_adjust in'.C.pre_sp **>
    G.instruction in'.C.shuffle **>
    G.stack_adjust in'.C.post_sp **>
    G.branch m k.K.succ **>
    G.entry G.empty in
  G.add_blocks g prolog

```

```

⟨functions that translate statements, including stmts 468d⟩+≡ (474d) <468c 469a>
and splice_in_return_to_entries k g =
  let gen cc g label =
    let in' = k.K.return_in in
    let () = add_youngblocks cc.C.overflow_alloc.C.result_allocator in'.C.overflow in
    let prolog =
      G.label m label **>
    (* NR IS BEHIND THE TIMES AND NEEDS TO BE REMINDED WHAT HAPPENED TO ASSERTIONS *)
    (* WE LOST: G.assertion g (in'.C.insp in'.C.overflow) *)
    G.stack_adjust in'.C.pre_sp **>

```

```

    G.instruction in'.C.shuffle    **>
    G.stack_adjust in'.C.post_sp   **>
    G.branch m k.K.succ          **>
    G.entry G.empty in
    G.add_blocks g prolog in
let gen_labels g (ccname, labels) =
    List.fold_left (gen (Call.get_cc target ccname)) g labels in
List.fold_left gen_labels g k.K.return_pcs

⟨functions that translate statements, including stmts 469a⟩+≡    (474d) <468d 469b>
and splice_in_unwind_to_entry k g =
    let prolog =
        G.label m k.K.unwind_pc **>
    (* NR IS BEHIND THE TIMES AND NEEDS TO BE REMINDED WHAT HAPPENED TO ASSERTIONS *)
    (* WE LOST: G.assertion g (proc_cc.C.sp_on_unwind proc_cc.C.stable_sp_loc) **> *)
        G.branch m k.K.succ          **>
        G.entry G.empty in
    G.add_blocks g prolog
(*)
let bogus_space = ('z', Rtl.Identity, Cell.of_size 1) in
let use_var (h,v,a) succ =
    let w = RU.Width.loc v.FE.loc in
    G.assertion g (RU.store (R.mem R.none bogus_space (R.C w)
                           (R.bits (Bits.S.of_int 0 w) w))
                 (R.fetch (R.var "" v.FE.index w) w)) succ in
... (List.fold_right use_var k.K.formals k.K.succ) ...
*)

```

R.3.5 Translations of statements

```

⟨functions that translate statements, including stmts 469b⟩+≡    (474d) <469a 469c>
and stmts props ss succ = List.fold_right (stmt props) ss succ
and _stmt' props s succ =
    Printf.eprintf "Translating %s\n" (
    match s with
    | S.If (e, so, not) -> "If"
    | S.Label n          -> "Label " ^ n
    | S.Switch _          -> "Switch"
    | S.Cont (name, _, _) -> "Cont"
    | S.Span (kv, bs)     -> "Span"
    | S.Assign rtl        -> "Assign"
    | S.Call (lhs, cc, e, args, targets, conts, alias) -> "Call"
    | S.Call' _           -> "Call'"
    | S.Goto (e, labels) -> "Goto"
    | S.Jump (cc, e, args, targets) -> "Jump"
    | S.Cut (cc, k, args, conts) -> "Cut"
    | S.Return (cc, i, n, args) -> "Return"
    | S.Limitcheck (cc, cookie, cont) -> "Limitcheck");
    stmt props s succ

⟨functions that translate statements, including stmts 469c⟩+≡    (474d) <469b
and stmt props s succ =
    ⟨supporting functions for translating statements 467d⟩
in
match s with
| S.If (e, so, not) ->
    let cb ~ifso ~ifnot = G.cbranch m e ~ifso ~ifnot in
    G.if_then_else m cb ~t:(stmts props so) ~f:(stmts props not) succ
| S.Label n -> G.label m (uid_of n, n) succ

```

```

| S.Switch (rg, e, arms) -> switchStmt rg e arms props succ
| S.Cont (name, _, _) -> contStmt2 name succ
| S.Span (kv, bs) -> stmts (kv :: props) bs succ
| S.Assign rtl -> G.instruction rtl succ
| S.Call (lhs, cc, e, args, targets, conts, alias) ->
  call lhs cc e args targets conts alias succ
| S.Call' (cc, e, args, targets) ->
  jump ~stack:false cc e args targets succ
| S.Goto (e, labels) ->
  (match Dn.exp e with
  | RP.Const (RP.Link (sym, _, w)) ->
    let lbl = sym#original_text in
    G.branch m (uid_of lbl, lbl) succ
  | _ -> (*let instr = machine.T.goto.T.embed e in*)
    let targets = List.map (fun l -> (uid_of l, l)) labels in
    G.mbranch m e targets succ)
| S.Jump (cc, e, args, targets) -> jump ~stack:true cc e args targets succ
| S.Cut (cc, k, args, conts) -> cut cc k args conts succ
| S.Return (cc, i, n, args) -> return cc i n args succ
| S.Limitcheck (cc, cookie, cont) -> limitcheck cc cookie cont

```

<supporting functions for translating statements 470a>+≡ (469c) <467d 470b>

```

and switchStmt range e arms props stmt_succ =
  let w = RU.Width.exp e in
  let e_in_interval (lo, hi) =
    if Bits.Ops.eq lo hi then
      R0.eq w e (Rtl.bits lo w)
    else
      R0.conjoin (R0.leu w (Rtl.bits lo w) e) (R0.leu w e (Rtl.bits hi w)) in
  let endswitch, g = G.make_target m stmt_succ in
  let default, g = G.make_target m (G.forbidden m g) in
  let do_arm (ranges, body) (next_test, g) =
    let condition =
      Simplify.exp (
        List.fold_left (fun cond range -> R0.disjoin (e_in_interval range) cond)
          (Rtl.bool false) ranges) in
    let arm, g = G.make_target m (stmts props body (G.branch m endswitch g)) in
    G.make_target m (G.cbranch m condition ~ifso:arm ~ifnot:next_test g) in
  let (first_test, g) = List.fold_right do_arm arms (default, g) in
  g

```

<supporting functions for translating statements 470b>+≡ (469c) <470a 471a>

```

and jump ~stack cconv e args targets g =
  let stack_adjust = if stack then G.stack_adjust else fun _ succ -> succ in
  let cc = Call.get_cc target cconv in
  let out = actuals cc.C.call_parms.C.out args in
  let ra_out = cc.C.ra_on_exit saved_ra out.C.overflow temps in
  let jump_sp = cc.C.sp_on_jump out.C.overflow temps in
  let () = add oldblocks cc.C.overflow_alloc.C.parameter_deallocator out.C.overflow in
  stack_adjust out.C.pre_sp **>
  G.instruction out.C.shuffle **>
  G.instruction restore_nvrs **>
  G.instruction (RU.store ra_out (RU.fetch saved_ra)) **>
  G.instruction (RU.store cc.C.jump_tgt_reg e) **>
  stack_adjust jump_sp **>
  G.jump m (RU.fetch cc.C.jump_tgt_reg) ~targets
  ~uses:(rsx (RS.union out.C.regs nvregs)) **>
  g

```

```

⟨supporting functions for translating statements 471a⟩+≡ (469c) <470b 471b>
and call lhs cconv e args _targets conts alias succ =
  let cc = Call.get_cc target cconv in
  let out = actuals cc.C.call_parms.C.out args in
  let in' = results cc.C.results.C.in' lhs in
  let unwind_conts =
    let f_to_cont (h,v,a) = (h,v.FE.index,a) in
    let unwind_k kname =
      let k = continuation kname in
      let args = List.map f_to_cont k.K.formals in
      (k.K.unwind_pc, k.K.unwind_sp, args) in
    List.map unwind_k conts.S.unwinds in
  let conts = continuations cc conts in
  add youngblocks cc.C.overflow_alloc.C.parameter_deallocator out.C.overflow;
                                     (* outgoing overflow parms *)
  add youngblocks cc.C.overflow_alloc.C.result_allocator in'.C.overflow;
                                     (* incoming overflow results *)
  let outalloc = match cc.C.overflow_alloc.C.parameter_deallocator with
    | C.Caller -> to_mloc (young_end cc out.C.overflow)
    | C.Callee -> to_mloc (old_end cc out.C.overflow) in
  let label ((uid, l), x, y) = (l, x, y) in
  let spans = SP.to_spans ~inalloc ~outalloc ~ra:saved_ra
    ~users:props ~csregs:nvr_temps
    ~conts:(List.map label unwind_conts)
    ~sds:sd_locs ~vars:(var_array ()) in
  let succ_assn = if conts.S.returns then in'.C.insp in'.C.overflow else R.par [] in
  G.stack_adjust out.C.pre_sp **>
  G.instruction out.C.shuffle **>
  G.stack_adjust out.C.post_sp **>
  G.call m e
    ~uses:(rsx out.C.regs) ~defs:(rsx in'.C.regs) ~kills:(rsx cc.C.volregs)
    ~altrets:conts.S.aretains ~unwinds_to:conts.S.unwinds
    ~cuts_to:conts.S.cuts ~aborts:conts.S.aborts
    ~reads:alias.S.reads ~writes:alias.S.writes ~spans:(Some spans) ~succ_assn **>
  (if conts.S.returns then
    G.stack_adjust in'.C.pre_sp (* DOUBTS ABOUT THIS *) **>
    G.instruction in'.C.shuffle **>
    G.stack_adjust in'.C.post_sp succ
  else
    G.forbidden m succ)

```

```

⟨supporting functions for translating statements 471b⟩+≡ (469c) <471a 472c>
and return cconv i n args g =
  let cc = Call.get_cc target cconv in
  let out = actuals cc.C.results.C.out args in
  ⟨verbosely announce the registers used by the return 472a⟩
  let ra_out = cc.C.ra_on_exit saved_ra out.C.overflow temps in
  let w = Rtlutil.Width.loc saved_ra in
  let upd_ra = if i > 0 then RU.store saved_ra (RU.addk w (RU.fetch saved_ra)
    (i * asm#longjmp_size ()))
    else R.null in
  let reti = cc.C.return i n (RU.fetch ra_out) in
  add oldblocks cc.C.overflow_alloc.C.result_allocator out.C.overflow;
                                     (* outgoing jump overflow = incoming *)
  G.instruction upd_ra **>
  G.stack_adjust out.C.pre_sp **>
  G.instruction out.C.shuffle **>
  G.instruction (RU.store ra_out (RU.fetch saved_ra)) **>
  G.instruction restore_nvrs **>
  G.stack_adjust out.C.post_sp **>

```



```

G.return      ~exit:i reti ~uses:(rsx (RS.union out.C.regs nvregs)) **>
g

⟨verbosely announce the registers used by the return 472a⟩≡ (471b)
let () =
  if Debug.on "return-regs" then
    let regstring ((s,_,_), i, R.C c) =
      if c = 1 then Printf.sprintf "%c%d" s i else Printf.sprintf "%c%d:%d" s i c in
    let used = List.map regstring (RS.elements out.C.regs) in
    Printf.eprintf "return statement uses regs: %s\n" (String.concat " " used) in

⟨ast2ir.ml 472b⟩+≡ (464a) <465f
let () = Debug.register "return-regs" "show registers used by return statement"

⟨supporting functions for translating statements 472c⟩+≡ (469c) <471b 472d>
and cut cc k args conts g =
  let cc      = Call.get_cc target cc in
  let out     = actuals (cc.C.cut_parms.C.out k) args in
  let cut_args = Contn.cut_args target ~contn:k in
  let ()      = add_constraints (Block.constraints out.C.overflow) in
  let conts   = ccontinuations cc conts in
  G.instruction out.C.shuffle **>
  G.cut_to m cut_args ~cuts_to:conts.S.ccuts ~aborts:conts.S.caborts
    ~uses:(rsx out.C.regs) **>
g

⟨supporting functions for translating statements 472d⟩+≡ (469c) <472c
and limitcheck cconv cookie cont =
  let w = W.exp vfp in
  let cc = Call.get_cc target cconv in
  let growth = cc.Call.stack_growth in
  let younger = match growth with
  | Memalloc.Down -> Rewrite.Ops.lt w
  | Memalloc.Up -> Rewrite.Ops.gt w in
  let _extremum = unimpf "old end of stack frame" in
  let overflows = younger vfp cookie in
  let ovnode ~ifso ~ifnot = G.cbranch m overflows ~ifso ~ifnot in
  G.limitcheck m ovnode (limitcheck_fails cconv cont) succ
and limitcheck_fails cconv cont succ = match cont with
| None -> G.forbidden m succ
| Some failure ->
  let failflow = { S.caborts = true; S.ccuts = [failure.S.recname] } in
  cut cconv failure.S.recont [] failflow **>
  contStmt2 failure.S.recname **>
  succ

```

R.3.6 Translating a procedure

```

⟨definition of proc, which translates one procedure 472e⟩≡ (465f) 472f>
let proc global_map proc =
  let proc_cc = Call.get_cc target proc.N.cc in
  let vfp     = target.T.vfp in (* virtual frame pointer *)
  let temps   = Talloc.Multiple.for_spaces target.T.spaces in

⟨definition of proc, which translates one procedure 472f⟩+≡ (465f) <472e 473a>
let uidmap =
  let add map l = SM.add l (G.uid ()) map in
  List.fold_left add SM.empty (Elabstmt.codelabels proc.N.code) in
let uid_of s =
  try SM.find s uidmap with Not_found -> impossf "unknown code label" in

```

```

⟨definition of proc, which translates one procedure 473a⟩+≡ (465f) <472f 473c>
  let exp_of_code_label (u,l) = R.codesym ((F.asm env)#local l) pointersize in
  let machine = T.boxmach in
  let m = (), machine, exp_of_code_label in

⟨definition of type blocklists 473b⟩≡ (465f)
  type 'a blocklists = { mutable caller : 'a list; mutable callee : 'a list }

⟨definition of proc, which translates one procedure 473c⟩+≡ (465f) <473a 473d>
  let youngblocks      = { caller = []; callee = [] } in
  let oldblocks        = { caller = []; callee = [] } in
  let add blocks party b = match party with
  | C.Caller -> blocks.caller <- b :: blocks.caller
  | C.Callee -> blocks.callee <- b :: blocks.callee in
  let constraints      = ref [] in
  let add_constraints c = constraints := c :: !constraints in
  let ptrcount =
    let _, _, c = target.T.memspace in Cell.to_count c target.T.pointersize in
  let to_mloc exp =
    R.mem (R.aligned target.T.alignment) target.T.memspace ptrcount exp in

⟨definition of proc, which translates one procedure 473d⟩+≡ (465f) <473c 473e>
  let low_end block = Block.base block in
  let high_end block =
    let base = Block.base block in
    Rtlutil.addk (W.exp base) base (Block.size block) in
  let young_end cc = match cc.C.stack_growth with
  | Memalloc.Down -> low_end
  | Memalloc.Up -> high_end in
  let old_end cc = match cc.C.stack_growth with
  | Memalloc.Down -> high_end
  | Memalloc.Up -> low_end in

⟨definition of proc, which translates one procedure 473e⟩+≡ (465f) <473d 473f>
  let formals = convert_parms (fun ((_, k, _, w, _, a) as parm) -> k, parm, a)
    (fun (i, _, _, w, n, _) -> AT.of_loc (R.var n i w), w) in
  let cformals = convert_parms (fun p -> p)
    (fun v -> AT.of_loc v.FE.loc, RU.Width.loc v.FE.loc) in
  let actuals = convert_parms (fun ((k, _, _, a) as parm) -> k, parm, a)
    (fun (_, e, w, _) -> e, w) in
  let results = convert_parms (fun p -> p) (fun (l, w) -> AT.of_loc l, w) in

⟨definition of proc, which translates one procedure 473f⟩+≡ (465f) <473e 473h>
  ⟨function extend_cont, for adding flow-graph info to continuations 467c⟩
  let contenv =
    let add env (n, k) = Strutil.Map.add n (extend_cont n k) env in
    List.fold_left add Strutil.Map.empty proc.N.continuations in
  let continuation l =
    try Strutil.Map.find l contenv with _ -> impossf "lost cont %s" l in

⟨types for nonvolatile registers 473g⟩≡ (465e)
  type 'a nvr = { reg : 'a; tmp : 'a } (* for callee-saves info *)

⟨definition of proc, which translates one procedure 473h⟩+≡ (465f) <473f 474a>
  let proc_in' = formals proc_cc.C.call_parms.C.in' proc.N.formals in
  let nvregs = RS.diff proc_cc.C.pre_nvregs proc_in'.C.regs in
  let nvr_temps =
    RS.fold (fun r i -> (r, Some (proc_cc.C.saved_nvr temps r)) :: i) nvregs [] in
  let convert (r,t) = match t with Some t -> {reg = R.reg r; tmp = t}
    | None -> impossf "Some nvr_temp expected" in

```

```

let nvr_info = List.map convert nvr_temps in
let save_nvrs =
  R.par (List.map (fun i -> RU.store i.tmp (RU.fetch i.reg)) nvr_info) in
let restore_nvrs =
  R.par (List.map (fun i -> RU.store i.reg (RU.fetch i.tmp)) nvr_info) in
let save_ra, saved_ra =
  let ra_in = proc_cc.C.ra_on_entry proc_in'.C.overflow in
  let loc = proc_cc.C.where_to_save_ra ra_in temps in
  R.store loc ra_in pointersize, loc in

⟨definition of proc, which translates one procedure 474a⟩+≡      (465f) <473h 474b>
let inalloc =
  match proc_cc.C.overflow_alloc.C.parameter_deallocator with
  | C.Callee -> to_mloc (old_end proc_cc proc_in'.C.overflow)
  | C.Caller -> to_mloc (young_end proc_cc proc_in'.C.overflow) in

⟨definition of proc, which translates one procedure 474b⟩+≡      (465f) <474a 474c>
let nvars = List.length proc.N.formals + List.length proc.N.locals in
(* I DON'T KNOW HOW TO GET A VAR'S NAME!!!! SEE ELSEWHERE ALSO *)
let var_array () = Array.init nvars (fun i -> Some (R.var "" ~index:i 0)) in

⟨definition of proc, which translates one procedure 474c⟩+≡      (465f) <474b 474d>
let sd_locs = List.map to_mloc proc.N.stacklabels in

⟨definition of proc, which translates one procedure 474d⟩+≡      (465f) <474c 474e>
⟨function continuations, which translates flow annotations 467e⟩
in
⟨functions that translate statements, including stmts 468b⟩
in
⟨definition of insert_init_cont_nodes 468a⟩
in

⟨definition of proc, which translates one procedure 474e⟩+≡      (465f) <474d 474f>
let bodylbl = genlabel "proc body start" in
let contlbl = genlabel "initialize continuations" in
let () = add oldblocks proc_cc.C.overflow_alloc.C.parameter_deallocator
          proc_in'.C.overflow in
let stack_adjust =
  if proc.N.basic_block then (fun _ succ -> succ) else G.stack_adjust in
let g =
  G.unfocus                ***>
  stack_adjust proc_in'.C.pre_sp ***>
  G.instruction proc_in'.C.shuffle ***>
  stack_adjust proc_in'.C.post_sp ***>
  G.instruction save_nvrs   ***>
  G.instruction save_ra     ***>
  G.label m contlbl         ***>
  G.label m bodylbl         ***>
  stmts proc.N.spans proc.N.code ***>
  G.entry G.empty in

⟨definition of proc, which translates one procedure 474f⟩+≡      (465f) <474e 475a>
let contblocks, g =
  if proc.N.basic_block then
    Block.cathl_list pointersize [], g
  else
    let addblock name k (blocks, g) =
      let g = finish_compiling_continuation k g in
      let blocks =
        if k.K.escapes then Block.cathl (Contn.rep k.K.rep) k.K.base :: blocks

```

```

        else blocks in
        blocks, g in
        let blocks, g = Strutil.Map.fold addblock contenv ([], g) in
        Block.cathl_list pointersize blocks, g in
let g =
  if proc.N.basic_block then
    g (* continuations already initialized in prolog of original proc *)
  else
    insert_init_cont_nodes contenv contlbl g in

⟨definition of proc, which translates one procedure 475a⟩+≡      (465f) <474f 475b>
let index = ref 0 in
let _allocate = fun ~width -> fun ~alignment -> fun ~kind ->
  let () = index := !index + 1 in
  let spaceId = char_of_int alignment in
  let cell = Cell.of_size target.Target.wordsize in
  let space = (spaceId, target.Target.byteorder, cell) in
  AT.of_loc (Rtl.reg (space, !index, Cell.to_count cell width))
and _freeze arg = Impossible.impossible "you must not freeze this automaton" in

(*
  let index = ref 0 in
  let blocks = ref [] in
  let allocate = fun ~width -> fun ~alignment -> fun ~kind ->
    let () = index := !index + 1 in
    let spaceId = char_of_int alignment in
    let cell = Cell.of_size target.Target.wordsize in
    let space = (spaceId, target.Target.byteorder, cell) in
    let block = Block.relative vfp "spills block" Block.at ~size:width ~alignment:alignment in
    let () = blocks := block :: !blocks in
    AT.of_loc (Rtl.reg (space, !index, Cell.to_count cell width))
  and freeze arg1 arg2 =
    let l = !blocks in
    let () = blocks := [] in
    { AT.overflow = Block.cathl_list target.Target.pointersize l
    ; AT.regs_used = RS.empty
    ; AT.mems_used = []
    ; AT.align_state = 1 (* oyh oyh oyh *)
    }

  in
*)

⟨definition of proc, which translates one procedure 475b⟩+≡      (465f) <475a 476a>
let formals =
  List.map (fun (i, h, v, w, n, a) -> i, (Some h, v, Ast.BitsTy w, n, Some a))
  proc.N.formals in
let (i : proc) =
  g,
  Procedure.{
    symbol      = proc.N.sym
  ; cc          = proc_cc
  ; target      = T target
  ; formals     = formals
  ; temps       = temps
  ; mk_symbol   = Fenv.Clean.symbol env
  ; cfg         = ()
  ; oldblocks   = (* parms must be first, so call List.rev *)
    { C.callee = List.rev (oldblocks.callee);
      C.caller  = List.rev (oldblocks.caller); }

```

```

; youngblocks = (* order doesn't matter *)
  { C.callee = youngblocks.callee;
    C.caller = youngblocks.caller; }
; stackd      = proc.N.stackmem      (* stack data block *)

; priv        = Block.relative vfp "private"
              (aligned_mem ~align:proc_cc.C.sp_align) target
(*
; Proc.priv    = AT.of_methods {AT.allocate = allocate ; AT.freeze = freeze}
*)
; sp          = Block.at proc_cc.C.stable_sp_loc 0 proc_cc.C.sp_align
; eqns        = List.concat (!constraints)
; conts       = contblocks
; vars        = nvars                (* number of variables *)
; nvregs      = RS.cardinal nvregs (* number of non-volatile regs *)
; var_map     = Array.make nvars None
; global_map  = global_map
; bodylbl     = bodylbl
; headroom    = !headroom
; exp_of_lbl  = exp_of_code_label
} in

```

⟨definition of `proc`, which translates one procedure 476a⟩+≡ (465f) <475b

```

(* claude: upstream called Optimize.trim_unreachable_code right here,
 * unconditionally, before the backend's optimizer. It can't be called
 * from this file: opti/optimize.ml's Optimize depends on cmm_front_ir
 * (this library) for the Ast2ir.proc type its signature names, so
 * cmm_front_ir depending back on cmm_opti would be a cycle. Instead
 * it's the first thing X86backend.optimizer/Ppcbackend.optimizer do
 * (arch/x86/x86backend.ml, arch/ppc/ppcbackend.ml), which sit above
 * both libraries in the dune graph - same position in the pipeline,
 * just moved to where the dependency direction allows it. *)
optimizer i (* runs optimizer, freezes, and assembles proc *)

```

⟨utilities(`ast2ir.nw`) 476b⟩+≡ (465a) <465d

```

let aligned_mem ~align ~base target =
  let ( *> ) = AT.( *> ) in
  AT.at target.T.memspace ~start:base (
    AT.align_to (fun w -> min align (w / target.T.memsize)) *>
    AT.overflow ~growth:Memalloc.Up ~max_alignment:align)

```

R.3.7 Initialized and uninitialized data

⟨function `datum`, for initialized and uninitialized data 476c⟩≡ (465f)

```

let rec datum global_map asm =
  function
  | N.Datalabel l -> asm#label l
  | N.Align n -> asm#align n
  | N.InitializedData es -> List.iter (fun (k, t) -> asm#addr k) es
  | N.UninitializedData n -> asm#zeroes n
  | N.Procedure p -> proc global_map p

```

R.3.8 Global Registers

⟨definition of `globals` 476d⟩≡ (465f)

```

let globals (base:Rtl.exp) vars =
  let t = target.T.globals base in
  let decls = Buffer.create 128 in (* initial size - grows as needed *)

```

```

let rec assign bindings (name, var) = match var.FE.rkind with
| FE.RNone | FE.RKind _ ->
  (* global register w/o hardware annotation *)
  let w      = RU.Width.loc var.FE.loc in
  let sig'   = Printf.sprintf "[%s %d]" name w in
  let ()     = Buffer.add_string decls sig' in
  let loc    = AT.allocate t w "" 1 in
  (var.FE.index, loc) :: bindings  (* could do even better here *)
| FE.RReg hw ->
  (* global register with h/w annotation *)
  (try
    let loc    = AT.of_loc (Strutil.Map.find hw target.T.named_locs) in
    let sig'   = Printf.sprintf "[%s %s]" name hw in
    let ()     = Buffer.add_string decls sig' in
    (var.FE.index, loc) :: bindings
  with Not_found -> impossf "unknown hardware register \"%s\" " hw) in
let bindings = List.fold_left assign [] vars in
let gmap =
  let var n = try List.assoc n bindings
              with Not_found -> impossf "no global register %d" n in
  Array.init (List.length vars) var in
gmap, AT.freeze t, Digest.string (Buffer.contents decls)

```

(supporting functions for fooling with the global-register area 477a)≡ (477b)

```

let emit_global_register_area ~export =
  let areaname = "Cmm.global_area" in
  let base_sym = if export then asm#export areaname else F.symbol env areaname in
  let base     = Rtl.datasym base_sym pointersize in
  let gmap, area, digest = globals base prog.N.globals in
  let area     = area.AT.overflow in
  let digest   = "Cmm.globalsig." ^ Idcode.encode digest in
  let asm      = F.asm env in
  let digest   = if export then asm#export digest else asm#import digest in
  (* must move to placevars and use gmap as a replacement map *)
  let _ =
    if export then
      begin
        asm#section (target.T.data_section);
        asm#comment "memory for global registers";
        asm#align (Block.alignment area);
        asm#label digest;      (* ensures desired definition is present *)
        asm#label base_sym;    (* ensures no multiple inconsistent definitions *)
        asm#addloc (Block.size area);
        asm#globals (List.length prog.N.globals);
      end
    else
      let localref = asm#local "Cmm.ref_to_global_area" in
      begin
        asm#section (target.T.data_section);
        asm#label localref;
        asm#comment "reference to global-register signature";
        asm#addr (Reloc.of_sym (digest, Rtl.imsym) pointersize);
      end in
  gmap

```

R.3.9 Translating a full compilation unit

(function program, which translates an entire program 477b)≡ (465f)

```

let program prog =

```

```

let prog = N.rewrite target.T.tx_ast prog in
⟨supporting functions for fooling with the global-register area 477a⟩
in
let global_map = emit_global_register_area defineglobals in
if defineglobals then
  Runtimedata.emit_global_properties target asm;
let section asm (name, data) =
  asm#section name; List.iter (datum global_map asm) data in
List.iter (section asm) prog.N.sections

```

R.4 front_ir/automatongraph.nw

```

⟨front_ir/automatongraph.ml 478a⟩≡
  ⟨automatongraph.ml 478d⟩

```

```

⟨front_ir/automatongraph.mli 478b⟩≡
  ⟨automatongraph.mli 478c⟩

```

R.5 Automaton-Graph

```

⟨automatongraph.mli 478c⟩≡ (478b)
  type ty = int * string * int (* width * kind * alignment *)
  val print: mk:(unit -> Automaton.t) -> ty list -> unit
  val paths: mk:(unit -> Automaton.t) -> ty list -> unit
  val summary : what:string -> mk:(unit -> Automaton.t) -> ty list -> unit

```

R.5.1 Implementation

```

⟨automatongraph.ml 478d⟩≡ (478a) 478e▷
  module A = Automaton
  let sprintf = Printf.sprintf
  let printf = Printf.printf

```

```

⟨automatongraph.ml 478e⟩+≡ (478a) <478d 478f▷
  type ty = int * string * int (* width * kind * alignment *)
  type path = ty list (* leads to a node *)

```

```

⟨automatongraph.ml 478f⟩+≡ (478a) <478e 478g▷
  type node = { regs: Register.Set.t
                ; align: int
                }

```

```

let compare_nodes (x:node) (y:node) =
  ( match compare x.align y.align with
  | 0 -> Register.Set.compare x.regs y.regs
  | x -> x
  )

```

```

⟨automatongraph.ml 478g⟩+≡ (478a) <478f 479a▷
  type edge = node * ty

```

```

let compare_edges (x:edge) (y:edge) =
  ( match Pervasives.compare (snd x) (snd y) with
  | 0 -> compare_nodes (fst x) (fst y)
  | x -> x
  )

```

```

<automatongraph.ml 479a>+≡ (478a) <478g 479b>
  module NS = Set.Make (struct type t=node let compare=compare_nodes end)
  module ES = Set.Make (struct type t=edge let compare=compare_edges end)
  module T = Map.Make (struct type t=edge let compare=compare_edges end)

  type graph = { nodes: NS.t (* all the nodes *)
                ; start: node (* start node *)
                ; edges: node T.t (* transition: node*ty => node *)
                }

  let graph node = { nodes = NS.add node NS.empty
                    ; start = node
                    ; edges = T.empty
                    }

<automatongraph.ml 479b>+≡ (478a) <479a 479c>
  let mem edge graph = T.mem edge graph.edges (* is edge in graph? *)
  let add (n,t as edge) node graph = (* add endpoint of edge *)
    assert (NS.mem n graph.nodes);
    { graph with edges = T.add edge node graph.edges
      ; nodes = NS.add node graph.nodes }

<automatongraph.ml 479c>+≡ (478a) <479b 479d>
  module ToString = struct
    let register ((sp,_,_),i,_) = sprintf "$%c%i" sp i
    let ty (width,kind,a) = sprintf "%s::%d@%d" kind width a
    let align n = sprintf "%i:" n
    let node s = let regs = Register.Set.elements s.regs in
                  String.concat ""
                    ((align s.align):: List.map register regs)
    let edge src label dst= sprintf "%s --%s--> %s"
                                (node src)
                                (ty label)
                                (node dst)

    let path p = String.concat " " (List.map ty p)
    let paths ps = String.concat "\n" (List.map path ps)
    let graph g =
      let add_edge (src,label) dst str = edge src label dst :: str in
      let edges = T.fold add_edge g.edges [] in
      String.concat "\n" edges
  end

<automatongraph.ml 479d>+≡ (478a) <479c 480a>
  module ToDot = struct
    let size = function
      | 32 -> ""
      | 64 -> "q"
      | n -> sprintf "%d" n

    let register ((sp,_,ms),i,c) = sprintf "$%c%i%s" sp i (size (Cell.to_width ms c))
    let ty (width,kind,a) = sprintf "%s::%d@%d" kind width a
    let node s = let regs = Register.Set.elements s.regs in
                  sprintf "\"%i:%s\""
                    s.align
                    (String.concat ""
                      (List.map register regs))
    let edge src label dst= printf "%s -> %s [label=\"%s\"]\n"
                                (node src)
                                (node dst)
                                (ty label)

```



```

let path p          = String.concat " " (List.map ty p)
let paths ps        = List.iter (fun s -> printf "%s\n" s)
                      (List.map path ps)

let graph g =
  ( printf "digraph \"calling convention\" {\n"
  ; printf "// nodes=%d \n" (NS.cardinal g.nodes)
  ; printf "size=\"9,6\"\n"
  ; printf "ratio=fill\n"
  ; let print_edge (src,label) dst () = edge src label dst in
    T.fold print_edge g.edges ()
  ; printf "}\n"
  )
end

<automatongraph.ml 480a>+≡ (478a) <479d 480b>
let goto mk path =
  let t      = mk () in
  let ()     = List.iter (fun (w,h,a) -> ignore (A.allocate t w h a)) path in
  let res    = A.freeze t in
  { regs     = res.A.regs_used
  ; align    = res.A.align_state
  }

<automatongraph.ml 480b>+≡ (478a) <480a 480c>
let registers loc =
  let c      = Rtl.bits (Bits.zero 32) 32 in
  let rtl    = A.store loc c 32 in
  let (read, written) = Rtlutil.ReadWrite.sets rtl in
  written

<automatongraph.ml 480c>+≡ (478a) <480b 480d>
let rec follow (mk:unit->Automaton.t) (dirs: ty list) graph path node ty =
  let path    = path @ [ty] in
  let node'   = goto mk path in
  let edge    = (node,ty) in
  if mem edge graph then
    graph
  else
    (* assert (Register.Set.subset node.regs node'.regs) *)
    dfs mk dirs (add edge node' graph) path node'

and dfs mk dirs graph path node = (* call this *)
  List.fold_left
    (fun graph ty -> follow mk dirs graph path node ty) graph dirs

<automatongraph.ml 480d>+≡ (478a) <480c 480e>
let _dump g = print_endline (ToString.graph g)

let print ~mk dirs =
  let init    = {regs=Register.Set.empty; align=0} in
  let g       = graph init in
  let g       = dfs mk dirs g [] init in
  ToDot.graph g

```

R.5.2 automatongraph.ml

```

<automatongraph.ml 480e>+≡ (478a) <480d 481a>
let next graph edge = T.find edge graph.edges

```

```

<automatongraph.ml 481a>+≡ (478a) <480e 481b>
  let outgoing_graph node =
    let add_label (src, label) dst labels =
      if compare_nodes src node = 0 then label :: labels else labels
    in
      T.fold add_label graph.edges []

<automatongraph.ml 481b>+≡ (478a) <481a 481c>
  let rec explore graph (visited:ES.t) (path:path) (paths:path list list) node =
    let labels = outgoing_graph node in
    let paths = (List.map (fun l -> l::path) labels) :: paths in
    let follow (node, ty as edge) visited paths =
      if ES.mem edge visited then
        (paths, visited) (* do nothing, return result *)
      else
        explore
          graph (ES.add edge visited) (ty::path) paths (next_graph edge)
    in
      List.fold_left
        (fun (paths,visited) edge -> follow edge visited paths)
        (paths,visited)
        (List.map (fun ty -> (node,ty)) labels)

<automatongraph.ml 481c>+≡ (478a) <481b>
  let interesting_paths graph =
    let (paths,_) = explore graph ES.empty [] [] graph.start in
    List.map List.rev (List.concat paths)

  let paths ~mk_dirs =
    let init = {regs=Register.Set.empty; align=0} in
    let g = graph init in
    let g = dfs mk_dirs g [] init in
    ToDot.paths (interesting_paths g)

  let mapsize m = T.fold (fun _ _ n -> n + 1) m 0

  let summary ~what ~mk_dirs =
    let init = {regs=Register.Set.empty; align=0} in
    let g = graph init in
    let g = dfs mk_dirs g [] init in
    Printf.printf "Automaton graph for %s has %d nodes, %d edges, %d paths\n" what
      (NS.cardinal g.nodes) (mapsize g.edges) (List.length (interesting_paths g))

```

R.6 front_ir/call.nw

```

<front_ir/call.ml 481d>≡
  <call.ml 484c>

<front_ir/call.mli 481e>≡
  <call.mli 482a>

```

R.7 Calling conventions

R.7.1 Discussion

R.7.2 Interface

```
<call.mli 482a>≡ (481e) 483e▷
  <exported type definitions(call.nw) 482b>

<exported type definitions(call.nw) 482b>≡ (484c 482a) 482c▷
  type kind      = string
  type width     = int
  type aligned   = int
  type types     = (width * kind * aligned) list

  type 'insp answer =
  {
    (* locs      : Automaton2.loc list *) (* where passed values reside *)
      overflow : Block.t (* includes all locs for values passed in mem *)
    (* ; sploc   : Rtl.exp (* where sp is required when values are passed *) *)
      ; insp    : 'insp (* specify sp when overflow block is introduced;
                          used in assertions entries from calls, cuts *)
      ; regs    : Register.Set.t (* set of locations defined (used) by partner *)
      ; pre_sp  : Rtl.rtl (* conditional SP adjustment pre-shuffle *)
      ; shuffle : Rtl.rtl (* shuffle parms where they go *)
      ; post_sp : Rtl.rtl (* unconditional SP adjustment post-shuffle *)
  }

<exported type definitions(call.nw) 482c>+≡ (484c 482a) <482b 482d>
  type party = Caller | Callee

<exported type definitions(call.nw) 482d>+≡ (484c 482a) <482c 482e>
  type overflow =
  { parameter_deallocator: party
    ; result_allocator:    party
  }

<exported type definitions(call.nw) 482e>+≡ (484c 482a) <482d 482f>
  type ('a, 'b) split_blocks = { caller : 'a; callee : 'b }

<exported type definitions(call.nw) 482f>+≡ (484c 482a) <482e 483f>
  type outgoing = types -> Rtl.exp list -> unit answer
  type incoming = types -> Automaton.loc list -> (Block.t -> Rtl.rtl) answer
  type ('inc, 'out) pair' = { in' : 'inc ; out : 'out }
  type pair              = (incoming, outgoing) pair'
  type cut_pair          = (incoming, (Rtl.exp -> outgoing)) pair'

  type t = (* part of a calling convention *)
  (* we get 3 dual pairs *)
  { name          : string (* canonical name of this cc *)
    ; overflow_alloc : overflow
    ; call_parms   : pair
    ; results      : pair
    ; cut_parms    : cut_pair
    (* exp is continuation val; used to address overflow block *)

    ; stable_sp_loc : Rtl.exp
    (* address where sp points after prolog, sits between calls,
       and should be set to on arrival at a continuation *)
```

```

; jump_tgt_reg : Rtl.loc
  (* When jumping through a register, we need a hardware register -- otherwise
     we might try to spill a temp after moving the sp, which would be very bad. *)

; stack_growth : Memalloc.growth
; sp_align      : int                (* alignment of stack pointer at call/cut *)
⟨fields to support the return address 483a⟩
⟨fields to support jumps 483d⟩
; sp_on_unwind  : Rtl.exp -> Rtl.rtl
; pre_nvregs    : Register.Set.t      (* registers preserved across calls *)
; volregs       : Register.Set.t      (* registers not preserved across calls *)
; saved_nvr     : Talloc.Multiple.t -> Register.t -> Rtl.loc (* where to save NVR *)
; return        : int -> int -> ra:Rtl.exp -> Rtl.rtl      (* alternate return *)
(* ; alt_return_table : node list -> node *)
(* these next two encapsulate knowledge of which reg. is sp *)
; replace_vfp    : Zipcfg.graph -> Zipcfg.graph * bool
}

⟨fields to support the return address 483a⟩≡ (482f) 483b▷
; ra_on_entry    : Block.t -> Rtl.exp

⟨fields to support the return address 483b⟩+≡ (482f) <483a 483c▷
; where_to_save_ra : Rtl.exp -> Talloc.Multiple.t -> Rtl.loc

⟨fields to support the return address 483c⟩+≡ (482f) <483b
; ra_on_exit      : Rtl.loc -> Block.t -> Talloc.Multiple.t -> Rtl.loc

⟨fields to support jumps 483d⟩≡ (482f)
; sp_on_jump      : Block.t -> Talloc.Multiple.t -> Rtl.rtl

⟨call.mli 483e⟩+≡ (481e) <482a 483g▷
val outgoing :
  growth:Memalloc.growth -> sp:Rtl.loc -> mkauto:valpass ->
  autosp:(Automaton.result -> Rtl.exp) ->
  postsp:(Automaton.result -> Rtl.exp -> Rtl.exp) -> outgoing
val incoming :
  growth:Memalloc.growth -> sp:Rtl.loc -> mkauto:valpass ->
  autosp:(Automaton.result -> Rtl.exp) ->
  postsp:(Automaton.result -> Rtl.exp -> Rtl.exp) ->
  insp:(Automaton.result -> Rtl.exp -> Block.t -> Rtl.exp) -> incoming

⟨exported type definitions(call.nw) 483f⟩+≡ (484c 482a) <482f
type valpass = unit -> Automaton.t

```

R.7.3 Registration of Calling Conventions from Lua

```

⟨call.mli 483g⟩+≡ (481e) <483e 484a▷
type 'a tgt = ('a, (Rtl.exp -> Automaton.t), t) Target.t

val register_cc :
  'a tgt -> string -> call:Automaton.stage ->
  results:Automaton.stage ->
  cutto:Automaton.stage -> unit

(* val get_cc : ('a, 'cc) Target.t -> string -> 'cc *)
val get_cc : ('p, 'a, 'cc) Target.t -> string -> 'cc

```

```

⟨call.mli 484a⟩+≡ (481e) <483g 484b>
  val dump_proc      : 'a tgt -> string -> types -> unit
  val dump_return    : 'a tgt -> string -> types -> unit
  val dump_cutto     : 'a tgt -> string -> types -> unit
  val paths_proc     : 'a tgt -> string -> types -> unit
  val paths_return   : 'a tgt -> string -> types -> unit
  val paths_cutto    : 'a tgt -> string -> types -> unit
  val summary_proc   : 'a tgt -> string -> types -> unit
  val summary_return : 'a tgt -> string -> types -> unit
  val summary_cutto  : 'a tgt -> string -> types -> unit

  val path_2_in_overflow : 'a tgt -> string -> unit

⟨call.mli 484b⟩+≡ (481e) <484a
  val run_cc_on_sig_and_return :
    (Automaton.cc_spec -> Automaton.stage) -> 'a tgt -> string -> types -> string list list
  val run_cc_on_sig_and_print :
    (Automaton.cc_spec -> Automaton.stage) -> 'a tgt -> string -> types -> unit

```

R.7.4 Implementation

```

⟨call.ml 484c⟩≡ (481d) 484d>
  module A = Automaton
  module Dn = Rtl.Dn
  module R = Rtl
  module RP = Rtl.Private
  module RS = Register.SetX
  module RU = Rtlutil
  module Up = Rtl.Up
  ⟨exported type definitions(call.nw) 482b⟩

⟨call.ml 484d⟩+≡ (481d) <484c 485>
  let ignore r s = s

  let too_small growth sp target =
    let w = RU.Width.loc sp in
    let ( >* ) x y = R.app (R.opr "gt" [w]) [x; y] in
    let ( <* ) x y = R.app (R.opr "lt" [w]) [x; y] in
    match growth with
    | Memalloc.Down -> RU.fetch sp >* target
    | Memalloc.Up   -> RU.fetch sp <* target

  let ne sp target = R.app (R.opr "ne" [RU.Width.loc sp]) [RU.fetch sp; target]

  let outgoing ~growth ~sp ~mkauto ~autosp ~postsp types actuals =
    let a = mkauto () in
    let crank effects' (w, k, aligned) actual =
      let l = A.allocate a ~width:w ~kind:k ~align:aligned in
      A.store l actual w :: effects' in
    let shuffle = R.par (List.rev (List.fold_left2 crank [] types actuals)) in
    let a = A.freeze a in
    let autosp = autosp a in
    let postsp = postsp a autosp in
    let setsp = RU.store sp postsp in
    { overflow = a.A.overflow
    ; insp     = ()
    ; regs     = a.A.regs_used
    ; shuffle  = shuffle
    ; post_sp  = R.guard (ne sp postsp)
    }

```

```

; pre_sp    = R.guard (too_small growth sp postsp) setsp
}

let incoming ~growth ~sp ~mkauto ~autosp ~postsp ~insp types formals =
  let a = mkauto() in
  let crank effects' (w, k, aligned) formal =
    let l = A.allocate a ~width:w ~kind:k ~align:aligned in
    A.store formal (A.fetch l w) w :: effects' in
  let shuffle = R.par (List.rev (List.fold_left2 crank [] types formals)) in
  let a = A.freeze a in
  let autosp = autosp a in
  let postsp = postsp a autosp in
  let insp    = insp a autosp in
  let setsp   = RU.store sp postsp in
  { overflow = a.A.overflow
  ; insp     = (fun b -> RU.store sp (insp b))
  ; regs     = a.A.regs_used
  ; shuffle  = shuffle
  ; post_sp  = R.guard (ne sp postsp)          setsp
  ; pre_sp   = R.guard (too_small growth sp postsp) setsp
  }

⟨call.ml 485⟩+≡ (481d) <484d 486a>
type 'a tgt = ('a, (Rtl.exp -> Automaton.t), t) Target.t

let add_cc specs name ~call ~results ~cutto =
  let base = List.remove_assoc name specs in
  (name, { A.call    = call
          ; A.results = results
          ; A.cutto   = cutto
          }
  ) :: base

let register_cc t name ~call ~results ~cutto =
  let newspecs = add_cc t.Target.cc_specs name ~call ~results ~cutto
  in t.Target.cc_specs <- newspecs ; ()

let get_ccspec tgt name =
  try List.assoc name tgt.Target.cc_specs
  with Not_found -> Unsupported.calling_convention name

let get_cc tgt name =
  tgt.Target.cc_spec_to_auto name (get_ccspec tgt name)

let dump what autofun target cname =
  let wordsize = target.Target.wordsz in
  let automaton = autofun (get_ccspec target cname) in
  what ~mk:(fun () ->
    A.at ~start:(R.bits (Bits.zero wordsz) wordsz)
        target.Target.memspace automaton)

let dump_proc    x = dump Automatongraph.print (fun s -> s.A.call) x
let dump_return  x = dump Automatongraph.print (fun s -> s.A.results) x
let dump_cutto   x = dump Automatongraph.print (fun s -> s.A.cutto) x
let paths_proc   x = dump Automatongraph.paths (fun s -> s.A.call) x
let paths_return x = dump Automatongraph.paths (fun s -> s.A.results) x
let paths_cutto  x = dump Automatongraph.paths (fun s -> s.A.cutto) x
let summary_proc x =
  dump (Automatongraph.summary ~what:"parameters") (fun s -> s.A.call) x
let summary_return x =

```

```

    dump (Automatongraph.summary ~what:"results")    (fun s -> s.A.results) x
let summary_cutto x =
    dump (Automatongraph.summary ~what:"cont parms") (fun s -> s.A.cutto) x

⟨call.ml 486a⟩+≡ (481d) <485 486b>
let findpath_1_in_overflow autofun target ccname =
    let wordsize = target.Target.wordszize in
    let automaton = autofun (get_ccspec target ccname) in
    let rec find_overflow_1 tys =
        let an = A.at ~start:(R.bits (Bits.zero wordszize) wordszize)
            target.Target.memspace automaton in
        let tys = (wordszize, "unsigned") :: tys in
        let _ = List.map (fun (w,h) -> A.allocate an ~width:w ~kind:h) tys in
        let res = A.freeze an in
        if res.A.mems_used == [] then find_overflow_1 tys else tys in
    find_overflow_1 []

let findpath_2_in_overflow autofun target ccname =
    (target.Target.wordszize, "unsigned") ::
    (findpath_1_in_overflow autofun target ccname)

let path_2_in_overflow target ccname =
    List.iter (fun (h,w) -> Printf.printf "%d/int " target.Target.wordszize)
    (findpath_2_in_overflow (fun s -> s.A.call) target ccname);
    Printf.printf "\n"

let locs alloc w =
    let RP.Rtl gs = Dn.rtl (A.store alloc (Rtl.bits (Bits.zero w) w) w) in
    let getloc = function RP.Store (l, _, _) -> l | RP.Kill l -> l in
    let rec add_locs l locs = match l with
    | RP.Reg (s, n, RP.C c) when c <= 0 -> locs
    | RP.Reg (s, n, RP.C c) -> Up.loc l :: add_locs (RP.Reg (s, n+1, RP.C (c-1))) locs
    | l -> Up.loc l :: locs in
    List.fold_right (fun (_, e) locs -> add_locs (getloc e) locs) gs []

let run_cc_on_sig_and_return autofun target ccname tys =
    let loc2strs l (w, k, a) = List.map Rtlutil.ToString.loc (locs l w) in
    let wordszize = target.Target.wordszize in
    let auto = autofun (get_ccspec target ccname) in
    let an = A.at ~start:(Rtl.late "ovflw" wordszize) target.Target.memspace auto in
    let allocs = List.map (fun (w,k,a) -> A.allocate an ~width:w ~kind:k ~align:a) tys in
    List.map2 loc2strs allocs tys

let run_cc_on_sig_and_print autofun target ccname tys =
    let locs = run_cc_on_sig_and_return autofun target ccname tys in
    let print (w, k, a) ls =
        Printf.printf "\"%s\":"%d@%d => { %s } \n" k w a (String.concat "," ls) in
    List.iter2 print tys locs

```

R.7.5 Potential new constructors for Call.t

```

⟨call.ml 486b⟩+≡ (481d) <486a
  ⟨specification for a calling convention 486c⟩

⟨specification for a calling convention 486c⟩≡ (486b)
type call_t = t
module type SPEC = sig
  type party = Caller | Callee
  type overflow = { parm_deallocator : party

```

```

        ; result_allocator : party
    }
val c_overflow      : overflow
val tail_overflow  : overflow
type nvr_saver      = Talloc.Multiple.t -> Register.t -> Rtl.loc
val save_nvrs_anywhere : Space.t list -> nvr_saver
    (* save h/w register in suitable temp space from the list *)
    (* one day: add [[save_nvrs_in_conventional_locations]] *)

module ReturnAddress : sig
    type style =
        | KeepInPlace      (* leave the RA where it comes in --- probably on stack *)
        | PutInTemporaries (* put the RA in temporaries *)

    (* values for the three RA-related functions in the Call.t *)
    (* these functions are not needed by a client but will be used to convert
       a Spec.t into a Call.t. Some client may want to use such functions to
       modify a Call.t *)
    val enter_in_loc : Rtl.loc -> Block.t -> Rtl.exp (* in_loc l b = fetch l *)

    val save_in_temp : Rtl.exp -> Talloc.Multiple.t -> Rtl.loc
    val save_as_is   : Rtl.exp -> Talloc.Multiple.t -> Rtl.loc

    val exit_in_temp : Rtl.exp -> Block.t -> Talloc.Multiple.t -> Rtl.loc
    val exit_as_is   : Rtl.exp -> Block.t -> Talloc.Multiple.t -> Rtl.loc
end

type t =
{ name      : string      (* canonical name of this cc *)
; stack_growth : Memalloc.growth
; sp         : Register.t
; sp_align   : int         (* alignment of stack pointer at call/cut *)
; allregs    : Register.Set.t      (* registers visible to the allocator *)
; nvregs     : Register.Set.t      (* registers preserved across calls *)
; saved_nvr  : nvr_saver
; ra         : Rtl.loc * ReturnAddress.style
                        (* where's the RA and what to do with it *)
}

val to_call : t -> call_t
end

```

R.8 front_ir/context.nw

```

⟨front_ir/context.ml 487a⟩≡
  ⟨context.ml 488d⟩

⟨front_ir/context.mli 487b⟩≡
  ⟨context.mli 487c⟩

```

R.9 Operator Contexts

```

⟨context.mli 487c⟩≡                                     (487b) 487d▷
  type 'c op = string * 'c list * 'c                    (* op, arguments, result *)

⟨context.mli 487d⟩≡                                     (487b) <487c 488a▷
  val nonbool : int:'c -> fp:'c -> rm:'c -> overrides:'c op list -> 'c op list
  val full    : int:'c -> fp:'c -> rm:'c -> bool:'c -> overrides:'c op list -> 'c op list

```



```

<context.mli 488a>+≡ (487b) <487d 488b>
  type t = (Talloc.Multiple.t -> int -> Register.t) * (Register.t -> bool)

<context.mli 488b>+≡ (487b) <488a 488c>
  val of_space : Space.t -> t
  val of_spaces : Space.t list -> t

<context.mli 488c>+≡ (487b) <488b>
  val functions : t op list -> (Rtl.Private.opr -> t list) * (Rtl.Private.opr -> t)

```

R.9.1 Implementation

```

<context.ml 488d>≡ (487a) 489>
  open Nopoly

  let impossf fmt = Printf.kprintf Impossible.impossible fmt

  type 'c op = string * 'c list * 'c (* op, arguments, result *)
  let nonboolops ~int ~fp ~rm =
    [ "add"      , [int; int], int
    ; "addc"     , [int; int; int], int
    ; "and"      , [int; int], int
    ; "bitExtract" , [int; int], int
    ; "bitInsert" , [int; int; int], int
    ; "bitTransfer" , [int; int; int; int; int], int
    ; "borrow"    , [int; int; int], int
    ; "carry"     , [int; int; int], int
    ; "com"       , [int], int
    ; "div"       , [int; int], int
    ; "divu"      , [int; int], int
    ; "f2f"       , [fp; rm], fp
    ; "f2f_implicit_round" , [fp], fp
    ; "f2i"       , [fp; rm], fp (* conversion done in float unit?? *)
    ; "fabs"      , [fp], fp
    ; "fadd"      , [fp; fp; rm], fp
    ; "fcmp"      , [fp; fp], fp
    ; "fdiv"      , [fp; fp; rm], fp
    ; "float_eq"  , [], fp
    ; "float_gt"  , [], fp
    ; "float_lt"  , [], fp
    ; "fmul"      , [fp; fp; rm], fp
    ; "fmulx"     , [fp; fp], fp
    ; "fneg"      , [fp], fp
    ; "fsqrt"     , [fp; rm], fp
    ; "fsub"      , [fp; fp; rm], fp
    ; "i2f"       , [fp; rm], fp (* conversion done in float unit? *)
    ; "lobits"    , [int], int
    ; "minf"      , [], fp
    ; "mod"       , [int; int], int
    ; "modu"      , [int; int], int
    ; "mul"       , [int; int], int
    ; "mulux"     , [int; int], int
    ; "mulx"      , [int; int], int
    ; "mzero"     , [], fp
    ; "NaN"       , [int], int (* implemented in simplifier using integer ops *)
    ; "neg"       , [int], int
    ; "or"        , [int; int], int
    ; "pinf"      , [], fp
    ; "popcnt"    , [int], int

```

```

; "pzero"          , [], fp
; "quot"           , [int; int], int
; "rem"            , [int; int], int
; "round_down"     , [], rm
; "round_nearest"  , [], rm
; "round_up"       , [], rm
; "round_zero"     , [], rm
; "rotr"           , [int; int], int
; "rotl"           , [int; int], int
; "shl"            , [int; int], int
; "shra"           , [int; int], int
; "shrl"           , [int; int], int
; "sub"            , [int; int], int
; "subb"           , [int; int; int], int
; "sx"             , [int], int
; "unordered"      , [], fp
; "xor"            , [int; int], int
; "zx"             , [int], int
]

```

```

let boolops ~int ~fp ~bool =
[ "add_overflows" , [int; int], bool
; "bit"           , [bool], int
; "bool"          , [int], bool
; "conjoin"       , [bool; bool], bool
; "disjoin"       , [bool; bool], bool
; "div_overflows" , [int; int], bool
; "eq"            , [int; int], bool
; "feq"           , [fp; fp], bool
; "fge"           , [fp; fp], bool
; "fgt"           , [fp; fp], bool
; "fle"           , [fp; fp], bool
; "flt"           , [fp; fp], bool
; "fne"           , [fp; fp], bool
; "ge"            , [int; int], bool
; "geu"           , [int; int], bool
; "gt"            , [int; int], bool
; "gtu"           , [int; int], bool
; "le"            , [int; int], bool
; "leu"           , [int; int], bool
; "lt"            , [int; int], bool
; "ltu"           , [int; int], bool
; "mul_overflows" , [int; int], bool
; "mulu_overflows", [int; int], bool
; "ne"            , [int; int], bool
; "not"           , [bool], bool
; "quot_overflows", [int; int], bool
; "sub_overflows" , [int; int], bool
]

```

```

let except base exns =
  let keep (o, _, _) = not (List.exists (fun (o', _, _) -> o == o') exns) in
  exns @ List.filter keep base

```

```

let nonbool ~int ~fp ~rm ~overrides = except (nonboolops ~int ~fp ~rm) overrides
let full ~int ~fp ~rm ~bool ~overrides =
  except (nonboolops ~int ~fp ~rm @ boolops ~int ~fp ~bool) overrides

```

<context.ml 489>+≡ (487a) <488d 490a>
 type t = (Talloc.Multiple.t -> int -> Register.t) * (Register.t -> bool)

```

let spacename (s, _, _) = s
let of_space s =
  match s.Space.classification with
  | Space.Temp { Space.stands_for = ok } ->
    let name = spacename s.Space.space in
    let alloc = Talloc.Multiple.reg name in
    let ok ((s', _, _) as r) = spacename s' <= name || ok r in
    alloc, ok
  | _ -> impossf "context from non-temporary space"

let rec of_spaces l = match l with
| [] -> impossf "context from empty list of spaces"
| [s] -> of_space s
| s :: ss ->
  let a, o = of_space s in
  let a', o' = of_spaces ss in
  let acellwidth =
    let (_, _, cell) = s.Space.space in Cell.to_width cell (Cell.C 1) in
  let alloc m =
    let a = a m in
    let a' = a' m in
    fun w -> if w = acellwidth then a w else a' w in
  alloc, (fun r -> o r || o' r)

```

```

<context.ml 490a>+≡ (487a) <489
module SM = Strutil.Map
let functions ops =
  let resmap = List.fold_left (fun m (n, a, r)-> SM.add n r m) SM.empty ops in
  let argmap = List.fold_left (fun m (n, a, r)-> SM.add n a m) SM.empty ops in
  let arg_contexts (n, _) =
    try SM.find n argmap with Not_found -> impossf "no arg context for %s" n in
  let result_context (n, _) =
    try SM.find n resmap with Not_found -> impossf "no result context for %s" n in
  arg_contexts, result_context

```

R.10 front_ir/contn.nw

```

<front_ir/contn.ml 490b>≡
  <contn.ml 491a>

<front_ir/contn.mli 490c>≡
  <contn.mli 490d>

```

R.11 Continuations

```

<contn.mli 490d>≡ (490c) 490e▷
type t

val init_code      : t -> Mflow.cut_args -> Rtl.rtl
val rep            : t -> Block.t      (* entire cont in memory; rep is address *)
val with_overflow  : ('a, 'b, 'c) Target.t -> overflow:Block.t -> t

<contn.mli 490e>+≡ (490c) <490d
val cut_args       : ('a, 'b, 'c) Target.t -> contn:Rtl.exp -> Mflow.cut_args
val ovblock_exp    : Rtl.exp -> int -> int -> int -> Rtl.exp
val get_contn      : Rtl.exp * Rtl.exp -> Rtl.exp

```

R.11.1 Implementation

```
<contn.ml 491a>≡ (490b)
module M = Mflow
module R = Rtl
module T = Target

type t =
  { block      : Block.t      (* memory block for pair + overflow incoming parms *)
  ; sp         : Rtl.loc      (* location inside block for sp *)
  ; pc         : Rtl.loc      (* location in block for code ptr *)
  }

let init_code t vals =
  Rtl.par [ Rtl.store t.pc vals.M.new_pc (Rtlutil.Width.loc t.pc)
           ; Rtl.store t.sp vals.M.new_sp (Rtlutil.Width.loc t.sp)
          ]

let rep t = t.block

let offset base n w = Rtlutil.addk w base n

let pc_sp base t =
  let (_, _, cell) = t.T.memspace in
  let (Rtl.C n) as ct = Cell.to_count cell t.T.pointersize in
  let mem addr = Rtl.mem Rtl.none t.T.memspace ct addr in
  let pc = mem (offset base 0 t.T.pointersize) in
  let sp = mem (offset base n t.T.pointersize) in
  pc, sp

let ovblock_exp e memsize ptrsize alignment =
  let ptrcells = ptrsize / memsize in
  offset e (Auxfuns.round_up_to alignment (2 * ptrcells)) ptrsize

let with_overflow t ~overflow =
  let size = t.T.pointersize / t.T.memsize in
  let my_pc_sp =
    Block.relative (Block.base overflow) "continuation block"
    Block.at ~size:(2 * size) ~alignment:t.T.alignment in
  let my_rep = Block.cath1 overflow my_pc_sp in
  let pc, sp = pc_sp (Block.base my_rep) t in
  { block = my_rep; sp = sp; pc = pc }

let cut_args t ~contn =
  let w = t.T.pointersize in
  let pc, sp = pc_sp contn t in
  {Mflow.new_pc=R.fetch pc w; Mflow.new_sp=R.fetch sp w}

let get_contn (newpc, newsp) = newpc
```

R.12 front_ir/expander.nw

```
<front_ir/expander.ml 491b>≡
  <expander.ml 498d>

<front_ir/expander.mli 491c>≡
  <expander.mli 498c>
```

```

⟨front_ir/postexpander.ml 492a⟩≡
  ⟨postexpander.ml 496b⟩

```

```

⟨front_ir/postexpander.mli 492b⟩≡
  ⟨postexpander.mli 492c⟩

```

R.13 A generic, parameterized code expander

R.13.1 Support for expansion—the postexpander module

```

⟨postexpander.mli 492c⟩≡ (492b) 495m▷
  ⟨types for postexpanders 492d⟩
  ⟨exported utility functions for postexpanders 492f⟩
  ⟨exported utility functions for use by the generic expander 496a⟩
  ⟨signature of a postexpander 493e⟩

⟨types for postexpanders 492d⟩≡ (496b 492c) 492e▷
  type temp      = Register.t
  type rtl       = Rtl.rtl
  type brtl      = Rtl.exp -> Rtl.rtl (* given expression, create branch rtl *)
  type exp       = Rtl.exp
  type address   = Rtl.exp
  type width     = Rtl.width
  type assertion = Rtl.assertion
  type operator  = Rtl.Private.opr

⟨types for postexpanders 492e⟩+≡ (496b 492c) ◁492d 492g▷
  type wtemp = fill * temp
  and fill = HighS | HighZ | HighAny
  and warg = WBits of Bits.bits
             | WTemp of Register.x (* high bits, if any, could contain anything *)

⟨exported utility functions for postexpanders 492f⟩≡ (492c) 492h▷
  val warg_val : warg -> exp

⟨types for postexpanders 492g⟩+≡ (496b 492c) ◁492e 495b▷
  type 'a block = 'a Dag.block
  type 'a branch = 'a Dag.branch
  type 'a cbranch = 'a Dag.cbranch
  type 'a cbinst = 'a Dag.cbinst

⟨exported utility functions for postexpanders 492h⟩+≡ (492c) ◁492f 492i▷
  module Alloc : sig
    val temp : char -> width -> temp
    val slot : width:width -> aligned:int -> Automaton.loc
    val isValid : unit -> bool
  end

⟨exported utility functions for postexpanders 492i⟩+≡ (492c) ◁492h 492j▷
  module Expand : sig
    val block : exp block -> brtl block
    val branch : exp branch -> brtl branch
    val cbranch : exp cbranch -> brtl cbranch
    val cbranch' : exp -> ifso:(brtl cbranch) -> ifnot:(brtl cbranch) -> brtl cbranch
  end

⟨exported utility functions for postexpanders 492j⟩+≡ (492c) ◁492i 493a▷
  val with_hw : hard:Register.x -> soft:warg -> temp:temp -> brtl block -> brtl block

```

```

⟨exported utility functions for postexpanders 493a⟩+≡ (492c) ◁492j 493b▷
  (*
  val (<:>) : 'a block -> 'a block -> 'a block
  *)

⟨exported utility functions for postexpanders 493b⟩+≡ (492c) ◁493a 493c▷
  (*
  val shared : 'a cbranch -> 'a cbranch
  *)

⟨exported utility functions for postexpanders 493c⟩+≡ (492c) ◁493b 493d▷
  (*
  val cond : exp -> exp cbranch (* branch taken iff exp true *)
  *)

⟨exported utility functions for postexpanders 493d⟩+≡ (492c) ◁493c
  (*
  type 'a nodeset
  val empty : 'a nodeset
  val lookup : uid -> 'a nodeset -> 'a (* raises Not_found *)
  val insert : uid -> 'a -> 'a nodeset -> 'a nodeset
  *)

```

R.13.2 The target-dependent postexpander interface

```

⟨signature of a postexpander 493e⟩≡ (496b 492c)
  module type S = sig
    val byte_order : Rtl.aggregation (* used to check access to memory *)
    val exchange_alignment : int (* alignment for an exchange slot *)
    ⟨generic expansion operations for register machines 493f⟩
    ⟨generic expansion operations for stack machines 495c⟩
  end

⟨generic expansion operations for register machines 493f⟩≡ (493e) 493g▷
  val load : dst:temp -> addr:address -> assertion -> brtl block
  val store : addr:address -> src:temp -> assertion -> brtl block

⟨generic expansion operations for register machines 493g⟩+≡ (493e) ◁493f 493h▷
  val sxload : dst:temp -> addr:address -> width -> assertion -> brtl block
  val zxload : dst:temp -> addr:address -> width -> assertion -> brtl block
  val lostore : addr:address -> src:temp -> width -> assertion -> brtl block

⟨generic expansion operations for register machines 493h⟩+≡ (493e) ◁493g 493i▷
  val move : dst:temp -> src:temp -> brtl block

⟨generic expansion operations for register machines 493i⟩+≡ (493e) ◁493h 493j▷
  val extract : dst:temp -> lsb:int -> src:temp -> brtl block
  val aggregate : dst:temp -> src:temp list -> brtl block (* little-endian *)

⟨generic expansion operations for register machines 493j⟩+≡ (493e) ◁493i 493k▷
  val hwset : dst:Register.x -> src:warg -> brtl block
  val hwget : dst:wtemp -> src:Register.x -> brtl block

⟨generic expansion operations for register machines 493k⟩+≡ (493e) ◁493j 493l▷
  val li : dst:temp -> Rtl.Private.const -> brtl block

⟨generic expansion operations for register machines 493l⟩+≡ (493e) ◁493k 494a▷
  val lix : dst:temp -> Rtl.exp -> brtl block

```

$\langle$ generic expansion operations for register machines 494a $\rangle + \equiv$ (493e) $\langle$ 493l 494b $\rangle$
 val block_copy :
 dst:address -> assertion -> src:address -> assertion -> width -> brtl block

$\langle$ generic expansion operations for register machines 494b $\rangle + \equiv$ (493e) $\langle$ 494a 494c $\rangle$
 val unop : dst:temp -> operator -> temp -> brtl block
 val binop : dst:temp -> operator -> temp -> temp -> brtl block

$\langle$ generic expansion operations for register machines 494c $\rangle + \equiv$ (493e) $\langle$ 494b 494d $\rangle$
 val unrm : dst:temp -> operator -> temp -> warg -> brtl block
 val binrm : dst:temp -> operator -> temp -> temp -> warg -> brtl block

$\langle$ generic expansion operations for register machines 494d $\rangle + \equiv$ (493e) $\langle$ 494c 494e $\rangle$
 val dblop : dsthi:temp -> dstlo:temp -> operator -> temp -> temp -> brtl block

$\langle$ generic expansion operations for register machines 494e $\rangle + \equiv$ (493e) $\langle$ 494d 494f $\rangle$
 val wrdop : dst:temp -> operator -> temp -> temp -> warg -> brtl block

$\langle$ generic expansion operations for register machines 494f $\rangle + \equiv$ (493e) $\langle$ 494e 494g $\rangle$
 val wrdrop : dst:wtemp -> operator -> temp -> temp -> warg -> brtl block

$\langle$ generic expansion operations for register machines 494g $\rangle + \equiv$ (493e) $\langle$ 494f 494h $\rangle$
 val icontext : Context.t (* for ints *)
 val fcontext : Context.t (* for floats *)
 val acontext : Context.t (* for addresses *)
 val constant_context : width -> Context.t
 val arg_contexts : operator -> Context.t list
 val result_context : operator -> Context.t

$\langle$ generic expansion operations for register machines 494h $\rangle + \equiv$ (493e) $\langle$ 494g 494i $\rangle$
 val itempwidth : int (* maximum width for one integer temporary *)

$\langle$ generic expansion operations for register machines 494i $\rangle + \equiv$ (493e) $\langle$ 494h 494j $\rangle$
 val pc_lhs : Rtl.loc (* program counter as assigned by branch *)
 val pc_rhs : Rtl.loc (* program counter as captured by call *)

$\langle$ generic expansion operations for register machines 494j $\rangle + \equiv$ (493e) $\langle$ 494i 494k $\rangle$
 val br : tgt:temp -> brtl branch (* branch register *)
 val b : tgt:Rtl.Private.const -> brtl branch (* branch *)

$\langle$ generic expansion operations for register machines 494k $\rangle + \equiv$ (493e) $\langle$ 494j 494l $\rangle$
 val bc_guard : temp -> operator -> temp -> brtl block * Rtl.exp
 val bc_of_guard : brtl block * Rtl.exp -> ifso:(brtl cbranch) -> ifnot:(brtl cbranch)
 -> brtl cbranch
 (* Formerly:
 val bc : temp -> operator -> temp -> ifso:(brtl cbranch) -> ifnot:(brtl cbranch)
 -> brtl cbranch
 *)
 val bnegate : Rtl.rtl -> Rtl.rtl

$\langle$ generic expansion operations for register machines 494l $\rangle + \equiv$ (493e) $\langle$ 494k 494m $\rangle$
 val callr : tgt:temp -> brtl branch
 val call : tgt:Rtl.Private.const -> brtl branch

$\langle$ generic expansion operations for register machines 494m $\rangle + \equiv$ (493e) $\langle$ 494l 494n $\rangle$
 val cut_to : Mflow.cut_args -> brtl branch

$\langle$ generic expansion operations for register machines 494n $\rangle + \equiv$ (493e) $\langle$ 494m 495a $\rangle$
 val return : Rtl.rtl
 val forbidden : Rtl.rtl (* causes a run-time error *)

```

⟨generic expansion operations for register machines 495a⟩+≡ (493e) ◁494n
  val don't_touch_me : Rtl.Private.effect list -> bool

⟨types for postexpanders 495b⟩+≡ (496b 492c) ◁492g
  type operator_class = Register | Stack of push * int
  and push = LeftFirst | RightFirst

⟨generic expansion operations for stack machines 495c⟩≡ (495m 493e) 495d▷
  val opclass : operator -> operator_class
  val stack_depth : int
  val stack_width : int

⟨generic expansion operations for stack machines 495d⟩+≡ (495m 493e) ◁495c 495e▷
  val converts_stack_to_temp : operator -> bool

⟨generic expansion operations for stack machines 495e⟩+≡ (495m 493e) ◁495d 495f▷
  val push : addr:address -> assertion -> brtl block
  val store_pop : addr:address -> assertion -> brtl block

⟨generic expansion operations for stack machines 495f⟩+≡ (495m 493e) ◁495e 495g▷
  val push_cvt : operator -> width -> addr:address -> assertion -> brtl block
  val store_pop_cvt : operator -> width -> addr:address -> assertion -> brtl block

⟨generic expansion operations for stack machines 495g⟩+≡ (495m 493e) ◁495f 495h▷
  val push_cvt_rm : operator -> warg -> width -> addr:address -> assertion
    -> brtl block
  val store_pop_cvt_rm : operator -> warg -> width -> addr:address -> assertion
    -> brtl block

⟨generic expansion operations for stack machines 495h⟩+≡ (495m 493e) ◁495g 495i▷
  module SlotTemp : sig
    val is : temp -> bool
    val push : temp -> brtl block
    val store_pop : temp -> brtl block
    val push_cvt : operator -> width -> temp -> brtl block
    val store_pop_cvt : operator -> width -> temp -> brtl block
    val push_cvt_rm : operator -> warg -> width -> temp -> brtl block
    val store_pop_cvt_rm : operator -> warg -> width -> temp -> brtl block
  end

⟨generic expansion operations for stack machines 495i⟩+≡ (495m 493e) ◁495h 495j▷
  val pushk : Rtl.Private.const -> brtl block
  val pushk_cvt : operator -> width -> Rtl.Private.const -> brtl block

⟨generic expansion operations for stack machines 495j⟩+≡ (495m 493e) ◁495i 495k▷
  val stack_op : operator -> brtl block
  val stack_op_rm : operator -> warg -> brtl block

⟨generic expansion operations for stack machines 495k⟩+≡ (495m 493e) ◁495j 495l▷
  val bc_stack : operator -> ifso:(brtl cbranch) -> ifnot:(brtl cbranch) -> brtl cbranch

⟨generic expansion operations for stack machines 495l⟩+≡ (495m 493e) ◁495k
  val stack_top_proxy : Rtl.loc
  val is_stack_top_proxy : Rtl.Private.loc -> bool

⟨postexpander.mli 495m⟩+≡ (492b) ◁492c
  module Nostack (Address : sig type t val reg : temp -> t end) : sig
    ⟨generic expansion operations for stack machines 495c⟩
  end

```


R.13.3 Implementation of the postexpander module

<exported utility functions for use by the generic expander 496a>≡ (492c) 496d▷

```
val remember_allocators : Talloc.Multiple.t -> Automaton.t -> unit
val forget_allocators   : unit -> unit
```

<postexpander.ml 496b>≡ (492a) 496c▷

```
module DG = Dag
module G   = Zipcfg
module R   = Rtl
module RU  = Rtlutil
type uid = int
<types for postexpanders 492d>
<signature of a postexpander 493e>
```

<postexpander.ml 496c>+≡ (492a) <496b 496e▷

```
module Alloc = struct
  let badslot : width -> int -> Automaton.loc =
    fun _ -> Impossible.impossible "slot allocator misconfigured"
  let badtemp : char -> width -> temp =
    fun _ _ -> Impossible.impossible "temporary allocator misconfigured"
  let valid = Reinit.ref false
  let theslot = Reinit.ref badslot
  let thetemp = Reinit.ref badtemp
  let slot ~width ~aligned = !theslot width aligned
  let temp c w = !thetemp c w
  let isValid () = !valid
end
let remember_allocators t s =
  if !Alloc.valid then
    Impossible.impossible "too many allocators";
    Alloc.valid := true;
    Alloc.thetemp := (fun c w -> Talloc.Multiple.reg c t w);
    Alloc.theslot := (fun w a -> Automaton.allocate s w "" a)
let forget_allocators () =
  if not !Alloc.valid then
    Impossible.impossible "too few allocators";
    Alloc.valid := false;
    Alloc.theslot := Alloc.badslot;
    Alloc.thetemp := Alloc.badtemp
```

<exported utility functions for use by the generic expander 496d>+≡ (492c) <496a

```
val remember_expanders :
  (exp block -> brtl block) -> (exp branch -> brtl branch) ->
  (exp cbranch -> brtl cbranch) -> unit
val forget_expanders : unit -> unit
```

<postexpander.ml 496e>+≡ (492a) <496c 497a▷

```
module Expand = struct
  let bad : exp block -> brtl block =
    fun _ -> Impossible.impossible "block expander misconfigured"
  let badb : exp branch -> brtl branch =
    fun _ -> Impossible.impossible "branch expander misconfigured"
  let badcb : exp cbranch -> brtl cbranch =
    fun _ -> Impossible.impossible "conditional branch expander misconfigured"
  let valid = Reinit.ref false
  let theblock = Reinit.ref bad
  let thebranch = Reinit.ref badb
  let thecbranch = Reinit.ref badcb
  let block b = !theblock b
  let branch b = !thebranch b
```

```

let cbranch b = !thecbranch b
let cbranch' e ~ifso ~ifnot =
  let rec upd_cbr = function
    | DG.Exit true  -> ifso
    | DG.Exit false -> ifnot
    | DG.Test (b, c) -> DG.Test (upd_block b, upd_cbi c)
    | DG.Shared (u, cbr) -> DG.Shared (u, upd_cbr cbr)
  and upd_block b = match b with
    | DG.Rtl _ | DG.Nop -> b
    | DG.Seq (b1, b2) -> DG.Seq (upd_block b1, upd_block b2)
    | DG.If (e, b1, b2) -> DG.If (e, upd_block b1, upd_block b2)
    | DG.While (cb, b) -> DG.While (upd_cbr cb, upd_block b)
  and upd_cbi (i, cb1, cb2) = (i, upd_cbr cb1, upd_cbr cb2) in
  upd_cbr (cbranch (DG.Test (DG.Nop, (e, DG.Exit true, DG.Exit false))))
end

let remember_expanders b br cb =
  if !Expand.valid then
    Impossible.impossible "too many expanders";
  Expand.valid := true;
  Expand.theblock := b;
  Expand.thebranch := br;
  Expand.thecbranch := cb
let forget_expanders () =
  if not !Expand.valid then
    Impossible.impossible "too few expanders";
  Expand.valid := false;
  Expand.theblock := Expand.bad;
  Expand.thebranch := Expand.badb;
  Expand.thecbranch := Expand.badcb

<postexpander.ml 497a>+≡ (492a) <496e 497b>
let (<:>) = DG.<:>

<postexpander.ml 497b>+≡ (492a) <497a 497c>
module R0 = struct
  let lobits w w' x = Rtl.app (Rtl.opr "lobits" [w;w';]) [x; ]
  let zx w w' x = Rtl.app (Rtl.opr "zx" [w;w';]) [x; ]
end

<postexpander.ml 497c>+≡ (492a) <497b 498a>
let warg_val = function
  | WTemp (Register.Reg t) -> R.fetch (R.reg t) (Register.width t)
  | WTemp (Register.Slice (w, lsb, t)) -> R.fetch (R.slice w ~lsb (R.reg t)) w
  | WBits b -> R.bits b (Bits.width b)

let with_hw ~hard ~soft ~temp block =
  match soft with
  | WTemp r when Register.eqx r hard -> block
  | _ ->
    let n = Register.widthx hard in
    let w = Register.width temp in
    let t = R.reg temp in
    let tval = R.fetch t w in
    let hardloc = match hard with
    | Register.Reg r -> R.reg r
    | Register.Slice (w, lsb, r) -> R.slice w ~lsb (R.reg r) in
    let hard = R.fetch hardloc n in
    let save = DG.Rtl (R.store t (R0.zx n w hard) w) in
    let set = DG.Rtl (R.store hardloc (warg_val soft) n) in
    let restore = DG.Rtl (R.store hardloc (R0.lobits w n tval) n) in
    Expand.block (save <:> set) <:> block <:> Expand.block restore

```

```

<postexpander.ml 498a>+≡ (492a) <497c
module Nostack (Address : sig type t val reg : temp -> t end) = struct
  let imposs = Impossible.impossible
  let opclass _ = Register
  let stack_depth = 0
  let stack_width = 0
  let converts_stack_to_temp _ = false
  let push ~addr _ = imposs "stack op on register machine"
  let store_pop ~addr _ = imposs "stack op on register machine"
  let push_cvt _ _ ~addr _ = imposs "stack op on register machine"
  let push_cvt_rm _ _ _ ~addr _ = imposs "stack op on register machine"
  let store_pop_cvt _ _ ~addr _ = imposs "stack op on register machine"
  let store_pop_cvt_rm _ _ _ ~addr _ = imposs "stack op on register machine"
  let pushk _ = imposs "stack op on register machine"
  let pushk_cvt _ _ _ = imposs "stack op on register machine"
  let stack_op _ = imposs "stack op on register machine"
  let stack_op_rm _ = imposs "stack op on register machine"
  let bc_stack _ ~ifso ~ifnot = imposs "stack op on register machine"
  let stack_top_proxy = Rtl.reg (('\'000', Rtl.Identity, Cell.of_size 0), 0, Rtl.C 0)
  let is_stack_top_proxy _ = false

  module SlotTemp = struct
    let is _ = false
    let push _ = imposs "stack op on register machine"
    let store_pop _ = imposs "stack op on register machine"
    let push_cvt _ _ _ = imposs "stack op on register machine"
    let push_cvt_rm _ _ _ _ = imposs "stack op on register machine"
    let store_pop_cvt _ _ _ = imposs "stack op on register machine"
    let store_pop_cvt_rm _ _ _ _ = imposs "stack op on register machine"
  end
end

```

R.13.4 A generic expander

```

<expander module type 498b>≡ (498)
module type S = sig
  val cfg : 'a -> Preast2ir.proc -> Preast2ir.proc * bool
  val machine : Preast2ir.basic_proc Target.machine
  val block :
    Preast2ir.basic_proc -> Rtl.exp Dag.block -> (Rtl.exp -> Rtl.rtl) Dag.block
  val goto :
    Preast2ir.basic_proc -> Rtl.exp Dag.branch -> (Rtl.exp -> Rtl.rtl) Dag.branch
  val cbranch :
    Preast2ir.basic_proc -> Rtl.exp Dag.cbranch -> (Rtl.exp -> Rtl.rtl) Dag.cbranch
  val call :
    Preast2ir.basic_proc -> Rtl.exp Dag.branch -> (Rtl.exp -> Rtl.rtl) Dag.branch
  val jump :
    Preast2ir.basic_proc -> Rtl.exp Dag.branch -> (Rtl.exp -> Rtl.rtl) Dag.branch
  val cut :
    Preast2ir.basic_proc -> Rtl.exp Dag.branch -> (Rtl.exp -> Rtl.rtl) Dag.branch
end

```

```

<expander.mli 498c>≡ (491c)
<expander module type 498b>
module IntFloatAddr (Post : Postexpander.S) : S

```

```

<expander.ml 498d>≡ (491b) 499a▷
open Nopoly
<expander module type 498b>

```

```

module A    = Automaton
module BO   = Bits.Ops
module DG   = Dag
module G    = Zipcfg
module GR   = Zipcfg.Rep
module O    = Opshape
module PA   = Preast2ir
module PX   = Postexpander
module R    = Rtl
module RP   = Rtl.Private
module RU   = Rtlutil
module Reg  = Register
module Rg   = Register
module S    = Space
module T    = Target

module Up   = Rtl.Up
module Dn   = Rtl.Dn

let upassn = Rtl.Up.assertion
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let unimpf  fmt = Printf.kprintf Impossible.unimp  fmt

(* claude: ops like pinf/mzero/pzero/minf/NaN have no hardware
 * implementation on x86 - they're bit patterns "implemented in the
 * simplifier using integer ops" (see Context.ml's comment on NaN) and are
 * only ever meant to reach the postexpander already folded into a Const.
 * That folding normally happens via Optimize.simplify_exps, which is only
 * wired in at -O3 (arch/x86/x86backend.ml's ~opt_level; -O0 is the
 * default, same gap the extract and %round_* fixes elsewhere in this file
 * work around), so try it locally wherever an unrecognized operator would
 * otherwise crash, rather than depend on that pass being plugged in.
 *)
let simplify_folds_to_const e =
  match Dn.exp (Simplify.exp (Up.exp e)) with
  | RP.Const _ as c -> Some c
  | _ -> None

⟨expander.ml 499a⟩+≡ (491b) <498d 499b>
  let fetch l = RP.Fetch (Dn.loc l, RU.Width.loc l)
  let width e = RU.Width.exp (Up.exp e)

⟨expander.ml 499b⟩+≡ (491b) <499a 499d>

⟨block-manipulation utilities 499c⟩≡ (501e)
  let (<:>) = DG.<:> in
  let (<::>) is (is', branch) = (is <:> is', branch) in
  let rec (<:::>) is cbranch = match cbranch with
  | DG.Exit b -> DG.Exit b
  | DG.Test (b, c) -> DG.Test (is <:> b, c)
  | DG.Shared (u, c) -> DG.Shared (u, is <:::> c) in

⟨expander.ml 499d⟩+≡ (491b) <499b 501a>
  module D = struct (* debugging *)
    let () = Debug.register "expander" "code expander"
    let eprintf = Printf.eprintf
    let sprintf = Printf.sprintf
    let strings =
      if Debug.on "expander" then

```

```

    (fun ss -> eprintf "%s" (String.concat "" ss))
else
    (fun ss -> ())
let int n = string_of_int n
let rtl r = Rtlutil.ToString.rtl r
let brtl b = "<function: exp -> rtl>"
let temp ((s, _, ms), n, c) =
    sprintf "$s[%d] : %d loc" (Char.escaped s) n (Cell.to_width ms c)
let exp = Rtlutil.ToString.exp
let pr_rtls rs = List.iter (fun r -> strings [" "; rtl r; "\n"]) (List.rev rs)
<more printing functions for the internal D module 500a>
(* renaming *)
let cbranch pa = if Debug.on "expander" then cbranch pa else fun _ -> "<cbranch>"
let pr_block pa = if Debug.on "expander" then pr_block pa " " else fun _ -> ()
let exp e = Rtlutil.ToString.exp (Up.exp e)
let exp' e = Rtlutil.ToString.exp e
let loc l = Rtlutil.ToString.loc (Up.loc l)
end

<more printing functions for the internal D module 500a>≡ (499d) 500b▷
let rec cbi pa c = match c with
| a, DG.Exit true, DG.Exit false -> pa a
| a, DG.Exit false, DG.Exit true -> sprintf "!(%s)" (pa a)
| a, DG.Exit true, p -> sprintf "(%s || %s)" (pa a) (cbranch pa p)
| a, p, DG.Exit false -> sprintf "(%s && %s)" (pa a) (cbranch pa p)
| a, p, q -> sprintf "(%s ? %s : %s)" (pa a) (cbranch pa p) (cbranch pa q)
and cbranch pa c = match c with
| DG.Exit p -> if p then "true" else "false"
| DG.Shared (_, c) -> sprintf "[%s]" (cbranch pa c)
| DG.Test (DG.Nop, c) -> cbi pa c
| DG.Test (b, c) -> sprintf "{%s; %s}" (compact_block pa b) (cbi pa c)
and compact_block pa b =
    let rec pr = function
    | DG.Rtl r -> sprintf "%s" (rtl r)
    | DG.Seq (b, b') -> sprintf "%s; %s" (pr b) (pr b')
    | DG.If (c, t, f) -> sprintf "if (%s) { %s; } else { %s; }"
        (cbranch pa c) (pr t) (pr f)
    | DG.While (c, b) -> sprintf "while (%s) { %s; }" (cbranch pa c) (pr b)
    | DG.Nop -> "skip" in
    pr b

<more printing functions for the internal D module 500b>+≡ (499d) <500a
let rec pr_block pa ind b =
    let rec pr = function
    | DG.Rtl r -> eprintf "%s%s;\n" ind (rtl r)
    | DG.Seq (b, b') -> pr b; pr b'
    | DG.If (DG.Exit p, t, f) -> pr (if p then t else f)
        (* true to semantics, but maybe not informative enough *)
    | DG.If (DG.Shared (_, c), t, f) -> pr (DG.If(c, t, f))
    | DG.If (DG.Test (DG.Nop, c), t, f) ->
        let ind' = ind ^ " " in
        eprintf "%sif (%s) {\n" ind (cbi pa c);
        pr_block pa ind' t;
        eprintf "%s} else {\n" ind;
        pr_block pa ind' f;
        eprintf "%s}\n" ind
    | DG.If (DG.Test (b, cbi), t, f) ->
        assert (Pervasives.<>) b DG.Nop);
        pr (DG.Seq (b, DG.If (DG.Test(DG.Nop, cbi), t, f)))
    | DG.While (c, b) -> (* not implemented really *)

```

```

    let ind' = ind ^ " " in
    eprintf "%swhile (%s) {\n" ind (cbranch pa c);
    pr_block pa ind' b;
    eprintf "%s}\n" ind
  | DG.Nop -> eprintf "%s<nop>\n" ind in
pr b

⟨expander.ml 501a⟩+≡ (491b) <499d 501b>
  let _ = Sys.set_signal Sys.sigterm (Sys.Signal_handle (fun _ -> exit 0))

⟨expander.ml 501b⟩+≡ (491b) <501a
  module IntFloatAddr (Post : Postexpander.S) = struct
    let pc_lhs = Dn.loc Post.pc_lhs
    let pc_rhs = Dn.loc Post.pc_rhs
    ⟨internal utilities for the generic expander 514b⟩
    ⟨generic expander 501c⟩
    ⟨generic flow-graph expander 517c⟩
  end

⟨generic expander 501c⟩≡ (501b) 501e>
  let alloc (allocate, check) w = allocate w
  let ok (allocate, check) reg = check reg

⟨definition of temp_in_context 501d⟩≡ (501e)
  let temp_in_context t context instructions =
    if ok context t then
      t, instructions
    else
      let t' = alloc context (Register.width t) in
      t', instructions <:> Post.move t' t in

⟨generic expander 501e⟩+≡ (501b) <501c
  let expand proc =
    let PA.T tgt = proc.Procedure.target in
    ⟨stack-slot allocation 517a⟩
    ⟨block-manipulation utilities 499c⟩
    ⟨functions for dealing with trivial guards 514e⟩
    let contextmap (allocator, checker) = (allocator proc.temps, checker) in
    let icontext = contextmap Post.icontext in
    let acontext = contextmap Post.acontext in
    let fcontext = contextmap Post.fcontext in
    ⟨rounding-mode checking 507a⟩
    ⟨context guessing 501f⟩
    let alloc_direct (allocator, _) = allocator proc.temps in
    ⟨definition of temp_in_context 501d⟩
    ⟨definitions of to_temp, rtl and all the other generic expander functions 502b⟩
    (block, branch', expand_cbranch, call', jump', cut')
  let number = 999

⟨context guessing 501f⟩≡ (501e) 501g>
  let cmp_context (oprname, ws) =
    if oprname.[0] <= 'f' then fcontext else icontext in

⟨context guessing 501g⟩+≡ (501e) <501f 502a>
  let guess_context = function
    | RP.Const (RP.Bits b) -> contextmap (Post.constant_context (Bits.width b))
    | RP.Const k -> icontext
    | RP.Fetch(RP.Reg (((('f'|'u)'), _, _), _, _), _) -> fcontext
    | RP.Fetch _ -> icontext
    | RP.App (op, args) -> contextmap (Post.result_context op) in

```

<context guessing 502a>+≡ (501e) *<501g*

```
let looks_like_stack_rhs = function
| RP.App (op, _) ->
  begin
    match Post.opclass op with
    | PX.Stack (_, _) -> true
    | PX.Register -> false
  end
| RP.Fetch (l, _) -> Post.is_stack_top_proxy l
| _ -> false in
```

<definitions of to_temp, rtl and all the other generic expander functions 502b>≡ (501e)

```
let rec to_temp' room context e =
  match e with
  <cases for converting a compile-time constant to a temporary 503a>
  <cases for converting a bare Fetch to a temporary 503b>
  <cases for sign extending or zero extending a temporary 504b>
  <cases for sign-extending and zero-extending loads 504a>
  <cases for sign extending or zero extending a narrow hardware register 504c>
  <cases for extracting from a wide temporary 503c>
  <cases for narrow weird value operators 504d>
  <cases for converting a Boolean to a value 505a>
  | RP.App ((_, [stackw; tempw]) as cvt, _) as e
  when Post.converts_stack_to_temp cvt ->
    let slot = exchange_slot tempw in
    let t, is = to_temp room context (fetch_slot slot tempw) in
    t, assign_slot room slot e tempw <:= is
  | RP.App (("f2f_implicit_round", ws), [x]) ->
    to_temp room context (RP.App (("f2f", ws), [x; RP.Fetch(rounding_mode, 2)]))
  | RP.App (op, args) as e -> (
    match simplify_folds_to_const e with
    | Some c -> to_temp' room context c
    | None ->
      <action for expanding op(args) into a temporary 502c>
    )
  <other generic expander functions 504f>
in
```

<action for expanding op(args) into a temporary 502c>≡ (502b)

```
match Post.opclass op with
| PX.Stack(dir, depth) ->
  let w = Post.stack_width in
  let slot = exchange_slot w in
  let t, is = to_temp room context (fetch_slot slot w) in
  t, assign_slot room slot e w <:= is
| PX.Register ->
  let w = width e in
  let t = alloc_direct (Post.result_context op) w in
  let temp e c = to_temp room (contextmap c) e in
  let warg e c = to_warg room (contextmap c) e in
  let is = match 0.capply temp warg op args (Post.arg_contexts op) with
  | 0.Binop ((x, b1), (y, b2)) -> b1 <:= b2 <:= Post.binop t op x y
  | 0.Unop ((x, b1)) -> b1 <:= Post.unop t op x
  | 0.Binrm ((x, b1), (y, b2), (r, b3)) -> b1 <:= b2 <:= b3 <:= Post.binrm t op x y r
  | 0.Unrm ((x, b1), (r, b2)) -> b1 <:= b2 <:= Post.unrm t op x r
  | 0.Fpcvt ((x, b1), (r, b2)) -> b1 <:= b2 <:= Post.unrm t op x r
  | 0.Dblop ((x, b1), (y, b2)) ->
    Impossible.unimp "double-width multiply into single temporary"
  | 0.Wrdop ((x, b1), (y, b2), (z, b3)) -> b1 <:= b2 <:= b3 <:= Post.wrdop t op x y z
  | 0.Wrdrop((x, b1), (y, b2), (z, b3)) ->
```

```

    impossf "single-bit result direct into temporary"
| O.Cmp _ | O.Width | O.Bool | O.Nullary ->
    impossf "operator %%%s reached postexpander" (fst op) in
temp_in_context t context is

⟨cases for converting a compile-time constant to a temporary 503a⟩≡ (502b)
| RP.Const k ->
    let w = width e in
    let t = alloc context w in
    t, Post.li t k
| RP.App (_, _) when is_compile_time_constant e ->
    let w = width e in
    let t = alloc context w in
    t, Post.lit t (Up.exp e)

⟨cases for converting a bare Fetch to a temporary 503b⟩≡ (502b)
| RP.Fetch (l, w) when Post.is_stack_top_proxy l ->
    let slot = exchange_slot w in
    let t, is = to_temp room context (fetch_slot slot w) in
    t, assign_slot room slot e w <:> is
| RP.Fetch (RP.Mem (('m', agg, ms), c, addr, assn), w) ->
    let t = alloc context w in
    let a, is = address room addr in
    assert (agg == Post.byte_order);
    assert (w = Cell.to_width ms c);
    t, is <:> Post.load t a (upassn assn)
| RP.Fetch (RP.Reg r, w) ->
    assert (w = Register.width r);
    temp_in_context r context DG.Nop
| RP.Fetch _ ->
    impossf "memory-like access to non-memory space"

⟨cases for extracting from a wide temporary 503c⟩≡ (502b)
| RP.App (("lobits", [w;n]), [RP.App (("shrl", [w']), [e; RP.Const (RP.Bits b)])]) as orig ->
    let shamt = Bits.U.to_int b in
    assert (w = w' && shamt < w);
    (* claude: e can be wider than any temp this target supports (e.g.
       a bits64 fetch on x86, whose temps are all 32 bits) - to_temp
       below would then crash trying to allocate one just to shift and
       discard most of it. Simplify already knows how to turn "shift a
       memory/register fetch right by a whole cell, then truncate"
       into a directly narrower fetch at an offset address/index
       (elab/simplify.ml's lobits), so try that first; it's the same
       rewrite Optimize.simplify_exps would have applied earlier if
       that pass were enabled (it only runs at -O3; -O0 is the
       default - see arch/x86/x86backend.ml's ~opt_level). *)
    (match Dn.exp (Simplify.exp (Up.exp orig)) with
    | (RP.Fetch ((RP.Mem _ | RP.Reg _), _)) as simplified ->
        to_temp' room context simplified
    | _ ->
        let t = alloc context n in
        let src, is = to_temp room (guess_context e) e in
        t, is <:> Post.extract ~dst:t ~lsb:shamt ~src)
| RP.App (("lobits", [w;n]), [e]) as orig ->
    (match Dn.exp (Simplify.exp (Up.exp orig)) with
    | (RP.Fetch ((RP.Mem _ | RP.Reg _), _)) as simplified ->
        to_temp' room context simplified
    | _ ->
        let t = alloc context n in
        let src, is = to_temp room (guess_context e) e in
        t, is <:> Post.extract ~dst:t ~lsb:0 ~src)

```



```

⟨cases for sign-extending and zero-extending loads 504a⟩≡ (502b)
| RP.App (("sx", [n;w]), [RP.Fetch (RP.Mem (('m',agg,ms), c, addr, assn), n')]) ->
  assert (n = n' && n' = Cell.to_width ms c);
  let t = alloc icontext w in
  let a, is = address room addr in
  assert (agg == Post.byte_order);
  temp_in_context t context (is <:> Post.sxload t a n (upassn assn))
| RP.App (("zx", [n;w]), [RP.Fetch (RP.Mem (('m',agg,ms), c, addr, assn), n')]) ->
  assert (n = n' && n' = Cell.to_width ms c);
  let t = alloc icontext w in
  let a, is = address room addr in
  assert (agg == Post.byte_order);
  temp_in_context t context (is <:> Post.zxload t a n (upassn assn))

```

```

⟨cases for sign extending or zero extending a temporary 504b⟩≡ (502b)
| RP.App (("sx", [n; w]), [RP.App(("lobits", [w'; n']), [x])]) when w = w' ->
  if n <> n' then Impossible.impossible "ill-typed %sx(%lobits(...))";
  to_temp' room context (Dn.exp (Rewrite.sxlo n w (Up.exp x)))
| RP.App (("zx", [n; w]), [RP.App(("lobits", [w'; n']), [x])]) when w = w' ->
  if n <> n' then Impossible.impossible "ill-typed %zx(%lobits(...))";
  to_temp' room context (Dn.exp (Rewrite.zxlo n w (Up.exp x)))

```

```

⟨cases for sign extending or zero extending a narrow hardware register 504c⟩≡ (502b)
| RP.App (((("sx"|"zx" as o), [n; w]), [RP.Fetch(r, _)])) when RU.is_hardware r ->
  let r = RU.to_hardware r in
  let t = alloc icontext w in
  let fill = if o == "sx" then PX.HighS else PX.HighZ in
  t, Post.hwget ~dst:(fill,t) ~src:r

```

```

⟨cases for narrow weird value operators 504d⟩≡ (502b) 504e▷
| RP.App (((("sx"|"zx") as xop, [n; w]),
  [RP.App(("carry"|"borrow"), [w']) as wrdop, args])) ->
  if n <> 1 then Impossible.impossible "ill-typed %sx/%zx(...)";
  let signed = xop == "sx" in
  let x, y, z, is = compile_weird_args room wrdop args in
  let t = alloc_direct (Post.result_context wrdop) w in
  let fill = if signed then PX.HighS else PX.HighZ in
  t, is <:> Post.wrdrop ~dst:(fill,t) wrdop x y z

```

```

⟨cases for narrow weird value operators 504e⟩+≡ (502b) ◁504d
| RP.App (((("addc"|"subb"), [w]) as wrdop, args) ->
  let x, y, z, is = compile_weird_args room wrdop args in
  let t = alloc_direct (Post.result_context wrdop) w in
  t, is <:> Post.wrdop ~dst:t wrdop x y z

```

```

⟨other generic expander functions 504f⟩≡ (502b) 504g▷
and compile_weird_args room op args = match args, Post.arg_contexts op with
| [x; y; z], [xc; yc; zc] ->
  let x, xis = to_temp room (contextmap xc) x in
  let y, yis = to_temp room (contextmap yc) y in
  let z, zis = to_warg room (contextmap zc) z in
  x, y, z, xis <:> yis <:> zis
| _ -> impossf "wrong number of args or contexts to %%" (fst op)

```

```

⟨other generic expander functions 504g⟩+≡ (502b) ◁504f 505b▷
and to_warg room context e = match e with
| RP.Const (RP.Bits b) ->
  PX.WBits b, DG.Nop
(* claude: same constants as Simplify.round_* in elab/simplify.ml (the
x87/IEEE-754 rounding-mode control-word encoding), applied here as a

```

```

    fallback for %round_*(()) args that reach the expander unsimplified *)
| RP.App (("round_nearest", []), []) -> PX.WBits (Bits.U.of_int 0 2), DG.Nop
| RP.App (("round_down", []), []) -> PX.WBits (Bits.U.of_int 1 2), DG.Nop
| RP.App (("round_up", []), []) -> PX.WBits (Bits.U.of_int 2 2), DG.Nop
| RP.App (("round_zero", []), []) -> PX.WBits (Bits.U.of_int 3 2), DG.Nop
| RP.App (("lobits", [w; n]), [e]) ->
    let t, is = to_temp room context e in
    (PX.WTemp (Rg.Reg t)), is
| RP.App (((("carry"|"borrow"), [w]) as wrdop, args) ->
    let x, y, z, is = compile_weird_args room wrdop args in
    let t = alloc_direct (Post.result_context wrdop) w in
    let t, is =
        temp_in_context t context (is <:= Post.wrdrop (PX.HighAny, t) wrdop x y z) in
    (PX.WTemp (Rg.Reg t)), is
| RP.Fetch (RP.Reg r, w) ->
    (PX.WTemp (Rg.Reg r), DG.Nop)
| RP.Fetch (RP.Slice (w, lsb, (RP.Reg r)), w') ->
    assert (w = w');
    (PX.WTemp (Rg.Slice (w, lsb, r)), DG.Nop)
| RP.Fetch _ -> Impossible.unimp "narrow argument from memory"
| e -> impossf "trying to put %s into a weird temporary" (RU.ToString.exp (Up.exp e))

```

<cases for converting a Boolean to a value 505a>≡ (502b)

```

| RP.App (((("sx"|"zx") as op, [n; w]), [RP.App (("bit", []), [e])]) ->
    assert (n = 1);
    let t = alloc context w in
    let nonzero = if op == "sx" then -1 else 1 in
    let tbranch = Post.li t (RP.Bits (Bits.S.of_int nonzero w)) in
    let fbranch = Post.li t (RP.Bits (Bits.zero w)) in
    let e = Up.exp e in
    let cond = expand_cbranch (DG.cond e) in
    t, DG.If (cond, tbranch, fbranch)

```

<other generic expander functions 505b>+≡ (502b) <504g 505c>

```

and address room exp =
    let t, is = to_temp room acontext exp in
    R.fetch (R.reg t) (Register.width t), is

```

<other generic expander functions 505c>+≡ (502b) <505b 505d>

```

and is_compile_time_constant = function
| RP.Const (RP.Bits _ | RP.Late (_, _)) -> true
| RP.App (((("add"|"sub"), [w]), es) -> List.for_all is_compile_time_constant es
| _ -> false

```

<other generic expander functions 505d>+≡ (502b) <505c 507c>

```

and to_stack' room e =
    if room < 1 then
        impossf "machine stack overflow in code generation";
    if Rtlutil.Width.exp' e <> Post.stack_width then
        (Printf.eprintf "failing expression: %s\n" (Rtlutil.ToString.exp
            (Rtl.Up.exp e));
            Unsupported.stack_width ~have:(Rtlutil.Width.exp' e)
            ~want:Post.stack_width);
    Debug.eprintf "expander" "to_stack %s\n" (RU.ToString.exp (Up.exp e));
    match e with
    | RP.Const k -> Post.pushk k
    <cases for getting a Fetch on the stack 506a>
    | RP.App (("f2f_implicit_round", ws), [x]) ->
        to_stack room (RP.App (("f2f", ws), [x; RP.Fetch(rounding_mode, 2)]))
    <cases for converting pushes 507d>

```

```

| RP.App (("i2f", [iw; fw] as i2f), [e; rm]) ->
  let slot = exchange_slot iw in
  assign_slot room slot e iw <:>
  to_stack room (RP.App (i2f, [fetch_slot slot iw; rm]))
| RP.App (("f2f", [w; _] as f2f), [e; rm]) ->
  let slot = exchange_slot w in
  assign_slot room slot e w <:>
  to_stack room (RP.App (f2f, [fetch_slot slot w; rm]))
| RP.App (op, args) as e -> (
  match simplify_folds_to_const e with
  | Some c -> to_stack' room c
  | None ->
    ⟨action for expanding op(args) onto the stack 506b⟩
)

⟨cases for getting a Fetch on the stack 506a⟩≡ (505d)
| RP.Fetch (l, _) when Post.is_stack_top_proxy l ->
  DG.Nop
| RP.Fetch (RP.Mem (('m', agg, ms), c, addr, assn), w) ->
  let a, is = address room addr in
  assert (agg == Post.byte_order);
  assert (w = Cell.to_width ms c);
  is <:> Post.push a (upassn assn)
| RP.Fetch (RP.Reg src, w) when Post.SlotTemp.is src ->
  assert (w = Register.width src);
  Post.SlotTemp.push src
| RP.Fetch (RP.Reg _, w) ->
  let slot = exchange_slot w in
  assign_slot room slot e w <:> to_stack room (fetch_slot slot w)
| RP.Fetch _ ->
  impossf "memory-like access to non-memory space"

⟨action for expanding op(args) onto the stack 506b⟩≡ (505d)
Debug.eprintf "expander" "to_stack generic case for %s\n" (D.exp e);
match Post.opclass op with
| PX.Stack(dir, depth) ->
  let push room args =
    let args = match dir with PX.LeftFirst -> args | PX.RightFirst -> List.rev args in
    let (is, _) = List.fold_left (fun (is,r) e -> (is <:> to_stack r e, r-1))
      (DG.Nop,room) args in
    is in
  let temp e c = e in
  let warg e c = to_warg room (contextmap c) e in
  let compute room = match 0.capply temp warg op args (Post.arg_contexts op) with
  | 0.Binop (x, y) -> push room [x; y] <:> Post.stack_op op
  | 0.Unop (x) -> push room [x] <:> Post.stack_op op
  | 0.Binrm (x, y, (r, b)) -> push room [x; y] <:> b <:> Post.stack_op_rm op r
  | 0.Unrm (x, (r, b)) -> push room [x] <:> b <:> Post.stack_op_rm op r
  | 0.Fpcvt (x, (r, b)) -> impossf "general fpcvt on stack"
  | 0.Dblop _ | 0.Wrdop _ | 0.Wrdrop _ ->
    Impossible.unimp "weird value operator on stack"
  | 0.Cmp _ | 0.Width | 0.Bool | 0.Nullary ->
    impossf "operator %%%s reached postexpander" (fst op) in
  if room >= depth then
    compute room
  else
    save_stack room <:> compute Post.stack_depth <:> restore_stack 1 room
| PX.Register ->
  let r, is = to_temp room (guess_context e) e in
  is <:> to_stack room (RP.Fetch (RP.Reg r, Register.width r))

```

```

⟨rounding-mode checking 507a⟩≡ (501e)
  let rounding_mode = Dn.loc tgt.Target.rounding_mode in
  let is_not_rounding_mode arg = match arg with
  | RP.Fetch (1, 2) -> not (RU.Eq.loc 1 rounding_mode)
  | _ -> true in

⟨junk(expander.nw) 507b⟩≡
  let insist_rounding_mode arg =
    if is_not_rounding_mode arg then
      Unsupported.soft_rounding_mode()

⟨other generic expander functions 507c⟩+≡ (502b) <505d 507e>
  and save_stack room =
    let _number_to_push = Post.stack_depth - room in
    Impossible.unimp "saving an overflowing machine stack"
  and restore_stack number_to_keep number_to_restore =
    Impossible.unimp "restoring an overflowing machine stack"

⟨cases for converting pushes 507d⟩≡ (505d)
  | RP.App (((("sx"|"zx"), [n; w]) as op,
    [RP.Fetch (RP.Mem (('m',agg,ms), c, addr, assn), n')])) ->
    assert (n = n' && n' = Cell.to_width ms c);
    assert (agg == Post.byte_order);
    let a, is = address room addr in
    is <:> Post.push_cvt op n a (upassn assn)
  | RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Mem (('m',agg,ms), c, addr, assn), n'); rm])) ->
    assert (n = n' && n' = Cell.to_width ms c);
    assert (agg == Post.byte_order);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
    | _ -> impossf "rm context" in
    let r, is' = to_warg room (contextmap rcontext) rm in
    let a, is = address room addr in
    is <:> is' <:> Post.push_cvt_rm op r n a (upassn assn)
  | RP.App (((("sx"|"zx"), [n; w]) as op,
    [RP.Fetch (RP.Reg src, n')])) when Post.SlotTemp.is src ->
    assert (n = n' && n' = Register.width src);
    Post.SlotTemp.push_cvt op n src
  | RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Reg src, n'); rm])) when Post.SlotTemp.is src ->
    assert (n = n' && n' = Register.width src);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
    | _ -> impossf "rm context" in
    let r, is = to_warg room (contextmap rcontext) rm in
    is <:> Post.SlotTemp.push_cvt_rm op r n src

⟨other generic expander functions 507e⟩+≡ (502b) <507c 508a>
  and rtl' hr =
    let RP.Rtl gs = Dn.rtl hr in
    if trivially Post.don't_touch_me gs then
      DG.Rtl hr
    else
      match gs with
      | [(g, RP.Store (left, right, w))] -> guarded g left right w
      | [(g, RP.Kill _)] -> DG.Nop
      | [] -> DG.Nop (* a nop is a nop is a nop *)
      | ((_:::_:_) as effs) -> (
        ⟨handle RTL with multiple effects effs (as RTL) 514d⟩
      )
  and wrap_don't_touch f r =

```

```

let RP.Rtl gs = Dn.rtl r in
if trivially Post.don't_touch_me gs then
  DG.Nop, r
else f r
and pre_rtl_to_call r =
  match Dn.exp (T.boxmach.T.call.T.project r) with
  | RP.Const c -> Post.call c
  | e          -> let r, is = to_temp Post.stack_depth acontext e in
                  is <::> Post.callr r
and rtl_to_call' r = wrap_don't_touch pre_rtl_to_call r
(* JUMP IS A PROBLEM -- WE DON'T DEAL WITH IT IN THE PX INTERFACE AT ALL *)
and pre_rtl_to_jump r =
  pre_rtl_to_branch r
and rtl_to_jump' r = wrap_don't_touch pre_rtl_to_jump r
and pre_rtl_to_cut r =
  let cut_args = T.boxmach.T.cutto.T.project r in
  let to_t e =
    let e      = Dn.exp e in
    let t, is = to_temp Post.stack_depth (guess_context e) e in
    let t = R.reg t in
    R.fetch t (RU.Width.loc t), is in
  let sp_t, is1 = to_t cut_args.Mflow.new_sp in
  let pc_t, is2 = to_t cut_args.Mflow.new_pc in
  is1 <::> is2 <::> Post.cut_to {Mflow.new_sp = sp_t; Mflow.new_pc = pc_t}
and rtl_to_cut' r = wrap_don't_touch pre_rtl_to_cut r
and pre_rtl_to_branch r =
  match Dn.exp (T.boxmach.T.goto.T.project r) with
  | RP.Const c -> Post.b c
  | e          -> let r, is = to_temp Post.stack_depth acontext e in
                  is <::> Post.br r
and rtl_to_branch' r = wrap_don't_touch pre_rtl_to_branch r

```

⟨other generic expander functions 508a⟩+≡ (502b) <507e 508b>

```

and guarded g left right w =
  match g with
  | RP.Const (RP.Bool b) -> if b then assign left right w else DG.Nop
  | RP.Const _ -> impossf "non-bool constant as guard"
  | RP.Fetch _ -> impossf "fetch as guard"
  | RP.App (cmp, [_; _]) ->
    Printf.eprintf "guarded: %s\n" (Rtlutil.ToString.exp (Up.exp g));
    Impossible.unimp "guard on other than conditional branch"
  | RP.App (cmp, _) ->
    Printf.kprintf Impossible.unimp "non-binary comparison in guard: %s" (D.exp g)

```

⟨other generic expander functions 508b⟩+≡ (502b) <508a 509a>

```

and expand_cbinst g ktrue kfalse =
  let room = Post.stack_depth in
  let rec expand g ktrue kfalse =
    match g with
    | RP.Const (RP.Bool b) -> if b then ktrue else kfalse
    | RP.Const _ -> impossf "non-bool constant as conditional guard"
    | RP.Fetch _ -> impossf "fetch as conditional guard"
    | RP.App (("not", []), [g]) -> expand g kfalse ktrue
    | RP.App (("conjoin", []), [x;y]) ->
      let kfalse = DG.shared kfalse in expand x (expand y ktrue kfalse) kfalse
    | RP.App (("disjoin", []), [x;y]) ->
      let ktrue = DG.shared ktrue in expand x ktrue (expand y ktrue kfalse)
    | RP.App (("false", []), []) -> kfalse
    | RP.App (("true", []), []) -> ktrue
    | RP.App (("bool", []), [RP.App (("lobits", [w; 1]), [x])]) ->

```

```

(* claude: same rewrite as Simplify.bool, applied here as a fallback for
%bool(...) guards that reach the expander unsimplified (e.g. after
widening) *)
let is1 = RP.Const (RP.Bits (Bits.U.of_int 1 w)) in
let is0 = RP.Const (RP.Bits (Bits.U.of_int 0 w)) in
expand (RP.App (("ne", [w]), [RP.App (("and", [w]), [x; is1]); is0])) ktrue kfalse
| RP.App (cmp, ([x; y] as args)) ->
begin
  match Post.opclass cmp with
  | PX.Register ->
    let context = cmp_context cmp in
    let xt, xis = to_temp room context x in
    let yt, yis = to_temp room context y in
    xis <:> yis <:::> Post.bc_of_guard (Post.bc_guard xt cmp yt) ktrue kfalse
  | PX.Stack(dir, depth) ->
    let args = List.filter is_not_rounding_mode args in (* cheat!!! *)
    (* ROUNDING *)
    let args =
      match dir with PX.LeftFirst -> args | PX.RightFirst -> List.rev args in
    let push_args room =
      let is, _ =
        List.fold_left (fun (is,r) e -> (is <:> to_stack r e, r-1))
        (DG.Nop,room) args in
      is in
    let compute room ktrue kfalse =
      push_args room <:::> Post.bc_stack cmp ~ifso:ktrue ~ifnot:kfalse in
    if room >= depth then
      compute room ktrue kfalse
    else
      (* PLAUSIBLE BUT COMPLETELY UNTESTED. *)
      let t = restore_stack 0 room <:::> ktrue in
      let f = restore_stack 0 room <:::> kfalse in
      save_stack room <:::> compute Post.stack_depth t f
  end
| RP.App (cmp, _) ->
  Printf.eprintf "rtl exp: %s\n" (Rtlutil.ToString.exp (Up.exp g));
  Impossible.unimp "non-binary comparison in conditional guard" in
expand g ktrue kfalse

```

⟨other generic expander functions 509a⟩+≡ (502b) ◁508b 509b▷

```

and expand_cbranch' c = match c with
| DG.Exit x -> DG.Exit x
| DG.Shared (u, c) -> DG.Shared(u, expand_cbranch' c)
| DG.Test (b, (g, ktrue, kfalse)) ->
  let b = block b in
  let rec extend c = match c with
  | DG.Exit p -> DG.Exit p
  | DG.Test (b', c) -> DG.Test (b <:> b', c)
  | DG.Shared (u, c) -> DG.Shared(u, extend c) in
  extend (expand_cbinst (Dn.exp g) (expand_cbranch' ktrue) (expand_cbranch' kfalse))

```

⟨other generic expander functions 509b⟩+≡ (502b) ◁509a 514a▷

```

and assign_left_right w = assign_room Post.stack_depth left right w
and assign_room room left right w =
  match right with
  | RP.App (("f2f_implicit_round", ws), [x]) ->
    assign_left (RP.App (("f2f", ws), [x; RP.Fetch(rounding_mode, 2)])) w
  | _ ->
    if RU.Eq.loc left pc_lhs then (* unconditional branch *)
      impossf "assignment to pc is not a branch"

```

```

else if Post.is_stack_top_proxy left then
  to_stack room right
else
  match left with
  | RP.Mem (('m',agg,memsize) as mspace, c, addr, assn) -> (* a store *)
    let w = Cell.to_width memsize c in
    assert (agg == Post.byte_order || agg == Rtl.Identity);
    <definition of split_assignment 513d>
    let a, is' = address room addr in
    (* claudie: give unresolved constant-folding ops (pinf, NaN,
       ...) a chance to become a plain Const here, so a wide one
       hits the RP.Const case below (which already knows how to
       split it into two stores) instead of falling through to
       the generic to_temp fallback, which cannot allocate a
       temp wider than this target's registers. See
       simplify_folds_to_const's comment. *)
    let right =
      match simplify_folds_to_const right with
      | Some c -> c
      | None -> right in
    (match right with
    |
      <pattern -> action for low-bit store 511a>
    |
      <pattern -> action for splittable assignment 512d>
    |
      <pattern -> action for doubling weird value operator 513b>
    | _ ->
      if looks_like_stack_rhs right then
        (match right with
        |
          <pattern -> action for push-convert, then store-pop 512b>
        |
          <pattern -> action for converting store-pop 511c>
        | _ ->
          let is = to_stack room right in
          is <:> is' <:> Post.store_pop a (upassn assn))
      else
        match right with
        | RP.Fetch (RP.Mem (('m', _, _), ct, addr', assn'), w) ->
          let a', is = address room addr' in
          is <:> is' <:>
            Post.block_copy a (upassn assn) a' (upassn assn') w
        | RP.Fetch (RP.Reg src, _) ->
          is' <:> Post.store a src (upassn assn)
        | _ ->
          let r, is = to_temp room (guess_context right) right in
          is <:> is' <:> Post.store a r (upassn assn))
  | RP.Mem (_, _, _, _) ->
    Impossible.unimp "memory space other than 'm'"
  | RP.Reg dst when Post.SlotTemp.is dst -> (* store to a stack slot *)
    (*<definition of [[split_assignment_slot]]>*)
    (match right with
    |
      <pattern -> action for low-bit store to reg 511b>
    |
      <pattern -> action for splittable assignment to reg 513a>
    |
      <pattern -> action for doubling weird value operator to reg 513c>
    | _ ->

```

```

        if looks_like_stack_rhs right then
            match right with
            |
            <pattern -> action for push-convert, then store-pop to reg 512c>
            |
            <pattern -> action for converting store-pop to reg 512a>
            | _ ->
                let is = to_stack room right in
                is <:> Post.SlotTemp.store_pop dst
            else
                match right with
                | RP.Fetch (RP.Mem(('m', _, _), ct, addr, assn), w) ->
                    let a, is = address room addr in
                    is <:> Post.load dst a (upassn assn)
                | RP.Fetch (RP.Reg src, _) -> Post.move ~dst ~src
                | _ ->
                    let r, is = to_temp room (guess_context right) right in
                    is <:> Post.move dst r
            | RP.Reg dst -> (* computation *)
                (match right with
                | RP.Fetch (RP.Reg src, _) -> Post.move ~dst ~src
                | _ -> let r, is = to_temp room (guess_context right) right in
                    is <:> Post.move ~dst ~src:r
                )
            (* ROUNDING --- narrow register *)
            | RP.Slice (w, lsb, RP.Reg r) ->
                let t, is = to_warg room icontext right in
                is <:> Post.hwset (Rg.Slice (w, lsb, r)) t
            | RP.Slice (w, lsb, _) ->
                impossf "back end advertises slice of non-HW in %s" (D.loc left)
            | RP.Var (_,_,_) | RP.Global(_,_,_) ->
                impossf "variable passed to code expander"

```

<pattern -> action for low-bit store 511a>≡ (509b)

```

RP.App (("lobits", [w; n]), [rhs]) ->
    let r, is = to_temp room icontext rhs in
    is <:> is' <:> Post.lostore a r n (upassn assn)
| RP.Const (RP.Bits b) when w < Post.itempwidth ->
    let rhs = RP.Const (RP.Bits (Bits.Ops.zx Post.itempwidth b)) in
    let r, is = to_temp room icontext rhs in
    is <:> is' <:> Post.lostore a r w (upassn assn)

```

<pattern -> action for low-bit store to reg 511b>≡ (509b)

```

RP.App (("lobits", [w; n]), [rhs]) ->
    impossf "slot-temp := %%lobits(e)    [extend postexpander?]"
| RP.Const (RP.Bits b) when w < Post.itempwidth ->
    impossf "slot-temp := %%lobits(k)    [extend postexpander?]"

```

<pattern -> action for converting store-pop 511c>≡ (509b)

```

RP.App (((("sx" | "zx"), [wsr; wd; dst]) as op, [rhs]) ->
    (* what goes wrong if wsrc is not the width of the stack? *)
    let is = to_stack room rhs in
    is <:> is' <:> Post.store_pop_cvt op wd; dst a (upassn assn)
| RP.App (((("f2f" | "f2i" | "i2f"), [wsr; wd; dst]) as op, [rhs; rm]) ->
    (* what goes wrong if wsrc is not the width of the stack? *)
    let is = to_stack room rhs in
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is'' = to_warg room (contextmap rcontext) rm in
    is <:> is' <:> is'' <:> Post.store_pop_cvt_rm op r wd; dst a (upassn assn)

```



```

⟨pattern -> action for converting store-pop to reg 512a⟩≡ (509b)
  RP.App (((("sx" | "zx"), [wsrc; wdst]) as op, [rhs]) ->
    (* what goes wrong if wsrc is not the width of the stack? *)
    let is = to_stack room rhs in
    is <:> Post.SlotTemp.store_pop_cvt op wdst dst
  | RP.App (((("f2f" | "f2i" | "i2f"), [wsrc; wdst]) as op, [rhs; rm]) ->
    (* what goes wrong if wsrc is not the width of the stack? *)
    let is = to_stack room rhs in
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is'' = to_warg room (contextmap rcontext) rm in
    is <:> is'' <:> Post.SlotTemp.store_pop_cvt_rm op r wdst dst

⟨pattern -> action for push-convert, then store-pop 512b⟩≡ (509b)
  RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Mem ((_,agg,ms), c, srcaddr, srcasn), n'); rm]) ->
    assert (n = n' && n' = Cell.to_width ms c);
    assert (agg == Post.byte_order);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is' = to_warg room (contextmap rcontext) rm in
    let srca, is = address room srcaddr in
    is <:> is' <:> Post.push_cvt_rm op r n srca (upassn srcasn) <:>
    Post.store_pop a (upassn assn)
  | RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Reg src, n'); rm]) when Post.SlotTemp.is src ->
    assert (n = n' && n' = Register.width src);
    assert (agg == Post.byte_order);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is = to_warg room (contextmap rcontext) rm in
    is <:> Post.SlotTemp.push_cvt_rm op r n src <:> Post.store_pop a (upassn assn)

⟨pattern -> action for push-convert, then store-pop to reg 512c⟩≡ (509b)
  RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Mem ((_,agg,ms), c, srcaddr, srcasn), n'); rm]) ->
    assert (n = n' && n' = Cell.to_width ms c);
    assert (agg == Post.byte_order);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is' = to_warg room (contextmap rcontext) rm in
    let srca, is = address room srcaddr in
    is <:> is' <:> Post.push_cvt_rm op r n srca (upassn srcasn) <:>
    Post.SlotTemp.store_pop dst
  | RP.App (((("f2f"|"f2i"|"i2f"), [n; w]) as op,
    [RP.Fetch (RP.Reg src, n'); rm]) when Post.SlotTemp.is src ->
    assert (n = n' && n' = Register.width src);
    let rcontext = match Post.arg_contexts op with [_; c] -> c
                    | _ -> impossf "rm context" in
    let r, is = to_warg room (contextmap rcontext) rm in
    is <:> Post.SlotTemp.push_cvt_rm op r n src <:> Post.SlotTemp.store_pop dst

⟨pattern -> action for splittable assignment 512d⟩≡ (509b)
  ( RP.App (("or", [ww]), [RP.App (("zx", [nn;ww']), [lsw]);
    RP.App (("shl", [ww']),
      [RP.App (("zx", [dd;ww']), [msw]);
      RP.Const (RP.Bits nnb)])])
  | RP.App (("or", [ww]), [RP.App (("shl", [ww']),
    [RP.App (("zx", [dd;ww']), [msw]);
    RP.Const (RP.Bits nnb)])])

```

```

        RP.App (("zx", [nn;ww']), [lsw]))]
) when w > Post.itemwidth && Pervasives.(<>) agg Rtl.Identity && ww = ww' && ww = ww''
    && ww = ww''' && dd = ww - nn && B0.eq nnb (Bits.U.of_int nn ww) ->
    split_assignment ~lsw ~lw:nn ~msw ~mw:dd
| RP.Const (RP.Bits b) when w > Post.itemwidth ->
    let lw = Post.itemwidth in
    let mw = w - lw in
    let lsw = RP.Const (RP.Bits (B0.lobits lw b)) in
    let msw = RP.Const (RP.Bits (B0.lobits mw (B0.shrl b (Bits.U.of_int lw w)))) in
    split_assignment ~lsw ~lw ~msw ~mw

⟨pattern -> action for splittable assignment to reg 513a⟩≡ (509b)
( RP.App (("or", [ww]), [RP.App (("zx", [nn;ww']), [lsw]);
    RP.App (("shl", [ww']),
        [RP.App (("zx", [dd;ww''']), [msw]);
        RP.Const (RP.Bits nnb)]))]
| RP.App (("or", [ww]), [RP.App (("shl", [ww']),
    [RP.App (("zx", [dd;ww''']), [msw]);
    RP.Const (RP.Bits nnb)]);
    RP.App (("zx", [nn;ww']), [lsw]))]
) when w > Post.itemwidth && ww = ww' && ww = ww''
    && ww = ww''' && dd = ww - nn && B0.eq nnb (Bits.U.of_int nn ww) ->
    impossf "splittable assignment to stack-slot temporary"
| RP.Const (RP.Bits b) when w > Post.itemwidth ->
    unimpf "assigned wide constant to stack-slot temporary"

⟨pattern -> action for doubling weird value operator 513b⟩≡ (509b)
RP.App (((("mulx"|"mulux"), [nw]) as opr, [x; y]) when w = 2 * nw ->
    let xcon, ycon = match Post.arg_contexts opr with
    | [x; y] -> contextmap x, contextmap y
    | _ -> impossf "arity of extended multiply (arg contexts)" in
    let rcon = contextmap (Post.result_context opr) in
    let tx, i1 = to_temp room xcon x in
    let ty, i2 = to_temp room ycon y in
    let thi, tlo = alloc rcon nw, alloc rcon nw in
    let i3 = Post.dblop ~dsthi:thi ~dstlo:tlo opr tx ty in
    let v tmp = RP.Fetch (RP.Reg tmp, nw) in
    i1 <:> i2 <:> i3 <:> split_assignment ~lsw:(v tlo) ~lw:nw ~msw:(v thi) ~mw:nw
| RP.App (((("mulx"|"mulux"), _) as e ->
    impossf "unsupported extended multiply %s at width %d"
    (RU.ToUnreadableString.exp (Up.exp e)) w

⟨pattern -> action for doubling weird value operator to reg 513c⟩≡ (509b)
RP.App (((("mulx"|"mulux"), [nw]), [x; y]) when w = 2 * nw ->
    impossf "extended multiply into stack-slot temporary"
| RP.App (((("mulx"|"mulux"), _) as e ->
    impossf "unsupported extended multiply %s at width %d"
    (RU.ToUnreadableString.exp (Up.exp e)) w

⟨definition of split_assignment 513d⟩≡ (509b)
let split_assignment ~lsw ~lw ~msw ~mw =
    if Debug.on "expander" then
        Printf.eprintf "Splitting msw %s; lsw = %s\n" (D.exp msw) (D.exp lsw);
    let lc, mc = Cell.to_count memsize lw, Cell.to_count memsize mw in
    assert (Cell.divides memsize lw && w = lw + mw);
    let addr = Up.exp addr in
    let assn = Up.assertion assn in
    let lsw, msw = Up.exp lsw, Up.exp msw in
    let offset (R.C n) = RU.addk tgt.T.pointersize addr n in
    match agg with

```

```

| Rtl.LittleEndian ->
  rtl (R.par [R.store (R.mem assn mspace lc addr) lsw lw;
              R.store (R.mem assn mspace mc (offset lc)) msw mw])
| Rtl.BigEndian ->
  rtl (R.par [R.store (R.mem assn mspace mc addr) msw mw;
              R.store (R.mem assn mspace lc (offset mc)) lsw lw])
| Rtl.Identity ->
  impossf "bad aggregation in split assignment" in

<other generic expander functions 514a>+≡ (502b) <509b 515f>
and fetch_slot slot w = Dn.exp (A.fetch slot w)
and assign_slot room slot right w =
  match Dn.rtl (A.store slot (Up.exp right) w) with
  | RP.Rtl [(RP.Const (RP.Bool true), RP.Store (left, right, w))] ->
    assign_room room left right w
  | _ -> impossf "stack slot is not a simple store"

<internal utilities for the generic expander 514b>≡ (501b) 515a▷
let has_pc_on_left = function
  | RP.Store (pc, _, _) -> RU.Eq.loc pc pc_lhs
  | RP.Kill _ -> false
let has_pc_on_right = function
  | RP.Store (_, e, _) -> RU.Exists.Loc.exp (RU.Eq.loc pc_rhs) e
  | RP.Kill _ -> false

<handle RTL with multiple effects effs (as branch) 514c>≡
<support for call and cut to 515b>
let effs = strip_trivial_guards effs in
if Post.don't_touch_me effs then
  DG.Nop, R.par (List.map Up.effect effs)
else
  if List.exists has_pc_on_left effs then
    if List.exists has_pc_on_right effs then
      make_call effs
    else
      make_cut_to effs
  else
    impossf "call or cut to without reference to PC"

<handle RTL with multiple effects effs (as RTL) 514d>≡ (507e)
<support for shuffle 515d>
let effs = strip_trivial_guards effs in
if Post.don't_touch_me effs then
  DG.Rtl (R.par (List.map Up.effect effs))
else
  let _ = if List.exists has_pc_on_left effs then
    impossf "straight-line code has PC on left" in
  make_shuffle effs

<functions for dealing with trivial guards 514e>≡ (501e)
let strip_trivial_guards l =
  List.fold_right
    (fun (g, e) l ->
      match g with RP.Const (RP.Bool b) -> if b then e :: l else l
      | _ -> Impossible.unimp "multiple effects with a nontrivial guard") l [] in
let trivially p l =
  let rec t es' = function
    | [] -> p (List.rev es')
    | (g, e) :: ges ->
      match g with RP.Const (RP.Bool b) -> t (if b then e :: es' else es') ges
      | _ -> false in
  t [] l in

```

<internal utilities for the generic expander 515a>+≡ (501b) <514b 519a>

```
let (<<) f g x = f (g x)
```

<support for call and cut to 515b>≡ (514c) 515c>

```
let make_call effs =
  let others = List.filter (not << has_pc_on_left) effs in
  match List.filter has_pc_on_left effs with
  | [RP.Store(pc, RP.Const c, _)] -> Post.call c others
  | [RP.Store(pc, e, _)] ->
    let t, is = to_temp Post.stack_depth acontext e in
    is <:> Post.callr t others
  | _ -> impossf "multiple pc := e in call" in
```

<support for call and cut to 515c>+≡ (514c) <515b

```
let make_cut_to effs =
  let expand e (preds, effs) = match e with
  | RP.Store(l, r, w) ->
    let t, is = to_temp Post.stack_depth (guess_context r) r in
    (preds <:> is, RP.Store(l, RP.Fetch(RP.Reg t, w), w) :: effs)
  | RP.Kill l -> (preds, RP.Kill l :: effs) in
  let preds, effs = List.fold_right expand effs (DG.Nop, []) in
  preds <:> Post.cut_to effs in
```

<support for shuffle 515d>≡ (514d)

```
let make_shuffle effs =
  let strip_store = function
  | RP.Store(l, r, w) -> (l, r, w)
  | RP.Kill _ -> impossf "kill in shuffle" in
  let rec shuffle =
    let assign = noisy_assign in
    function
    | [(l, r, w)] -> assign l r w
    | [] -> DG.Nop
    | ((l, (r:RP.exp), w) :: rest) as effs ->
      <definition of try_first_effect 515e>
      try_first_effect effs
      (fun (l, r, w) rest -> assign l r w <:> shuffle rest)
      (fun () ->
        let t = alloc (guess_context r) w in
        (assign (RP.Reg t) r w <:>
          shuffle rest <:>
          assign l (RP.Fetch (RP.Reg t, w)) w))
  in shuffle (List.map strip_store effs) in
```

<definition of try_first_effect 515e>≡ (515d)

```
let try_first_effect effects succ fail =
  let rec maybe_bad = function
  | [] -> fail()
  | (l, r, w) :: rest ->
    let alias = RU.MayAlias.store_uses' l in
    if not (List.exists alias bad || List.exists alias rest) then
      succ (l, r, w) (List.rev_append bad rest)
    else
      maybe ((l, r, w) :: bad) rest in
  maybe [] effects in
```

<other generic expander functions 515f>+≡ (502b) <514a 516>

```
and block b = match b with
| DG.Rtl r -> rtl r
| DG.Seq (b, b') -> block b <:> block b'
```

```

| DG.If (c, t, f) -> DG.If (expand_cbranch c, block t, block f)
| DG.While (e, b) -> Impossible.unimp "expand loop"
| DG.Nop          -> DG.Nop
and branch' (b, r) = let b', r = rtl_to_branch r in (block b <:> b', r)
and call'    (b, r) = let b', r = rtl_to_call   r in (block b <:> b', r)
and jump'    (b, r) = let b', r = rtl_to_jump   r in (block b <:> b', r)
and cut'     (b, r) = let b', r = rtl_to_cut    r in (block b <:> b', r)

```

(other generic expander functions 516)+≡

(502b) <515f

```

and rtl r =
  let _ = D.strings ["Expanding "; D.rtl r; "\n"] in
  let is = rtl' r in
  let _ = D.strings ["Expanded "; D.rtl r; " into\n"]; D.pr_block D.brtl is in
  is
and rtl_to_jump r =
  let _ = D.strings ["Expanding "; D.rtl r; "\n"] in
  let is, b = rtl_to_jump' r in
  let _ = D.strings ["Expanded "; D.rtl r; " into\n"];
    D.pr_block D.brtl (is <:> DG.Rtl b) in
  is, b
and rtl_to_cut r =
  let _ = D.strings ["Expanding "; D.rtl r; "\n"] in
  let is, b = rtl_to_cut' r in
  let _ = D.strings ["Expanded "; D.rtl r; " into\n"];
    D.pr_block D.brtl (is <:> DG.Rtl b) in
  is, b
and rtl_to_call r =
  let _ = D.strings ["Expanding "; D.rtl r; "\n"] in
  let is, b = rtl_to_call' r in
  let _ = D.strings ["Expanded "; D.rtl r; " into\n"];
    D.pr_block D.brtl (is <:> DG.Rtl b) in
  is, b
and rtl_to_branch r =
  let _ = D.strings ["Expanding "; D.rtl r; "\n"] in
  let is, b = rtl_to_branch' r in
  let _ = D.strings ["Expanded "; D.rtl r; " into\n"];
    D.pr_block D.brtl (is <:> DG.Rtl b) in
  is, b
and expand_cbranch cb =
  let _ = D.strings ["Expanding conditional branch "; D.cbranch D.exp' cb; "\n"] in
  let is = expand_cbranch' cb in
  let _ =
    D.strings ["Expanded conditional branch "; " into "; D.cbranch D.brtl is; "\n"] in
  is
and noisy_assign l r w =
  let is = assign l r w in
  let r = R.store (Up.loc l) (Up.exp r) w in
  let _ = D.strings ["Shuffling "; D.rtl r; " into\n"];
    D.pr_block D.brtl is in
  is
and to_temp room context e =
  let _ = D.strings ["Targeting "; D.exp e; "... \n"] in
  let t, is = to_temp' room context e in
  let _ = D.strings ["Targeted "; D.exp e; " => "; D.temp t; " by \n"] in
  let _ = D.pr_block D.brtl is in
  t, is
and to_stack room e =
  let _ = D.strings ["Pushing "; D.exp e; "... \n"] in
  let is = to_stack' room e in
  let _ = D.strings ["Pushed "; D.exp e; " by \n"] in

```

```

let _ = D.pr_block D.brtl is in
is

<stack-slot allocation 517a>≡ (501e)
let exchange_slot =
  let slots = ref [] in
  function w ->
    try List.assoc w (!slots)
    with Not_found ->
      let slot = Postexpander.Alloc.slot w Post.exchange_alignment in
      slots := (w, slot) :: !slots;
      slot in

<ZZZ definitions of to_temp, rtl and all the other generic expander functions 517b>≡
let rec old_general_guard g left right w =
  (* this case is bogus *)
  let r, is = to_temp Post.stack_depth (guess_context right) right in
  let l, is' = old_loc left in
  is <:> is' <:> DG.Rtl (R.guard (Up.exp g) (R.store l (R.fetch (R.reg r) w) w))
and old_loc l =
  match l with
  | RP.Mem (('m',_,_) as mspace, w, addr, assn) ->
    let addr, is = to_temp Post.stack_depth (acontext) addr in
    Up.loc (RP.Mem (mspace, w, fetch (R.reg addr), assn)), is
  | _ -> Up.loc l, DG.Nop in

<generic flow-graph expander 517c>≡ (501b)
let expand f (proc : PA.basic_proc) =
  let modified = ref false in
  let PA.T tgt = proc.target in
  let machine = proc, tgt.Target.machine, proc.exp_of_lbl in
  <block-conversion functions 519b>
  let (expand_block, expand_branch, expand_cbranch,
    expand_call, expand_jump, expand_cut as expanders) = expand proc in
  let g =
    if Postexpander.Alloc.isValid () then
      f expanders (block_before, block2cfg, cbranch2cfg)
    else
      (Postexpander.remember_allocators proc.temps proc.priv;
       Postexpander.remember_expanders expand_block expand_branch expand_cbranch;
       let g = f expanders (block_before, block2cfg, cbranch2cfg) in
       Postexpander.forget_allocators();
       Postexpander.forget_expanders();
       g) in
  g, !modified

let cfg _ ((cfg, proc) : Preast2ir.proc) =
  let PA.T tgt = proc.target in
  let m = tgt.Target.machine in
  let expand_cfg (expand_block, expand_branch, expand_cbranch, expand_call,
    expand_jump, expand_cut)
    (block_before, block2cfg, cbranch2cfg) =
  let expand_middle n = match n with
    | GR.Instruction i -> G.unfocus (block2cfg (expand_block (DG.Rtl i)))
    | GR.Stack_adjust _ -> G.single_middle n in
  let expand_last n =
    let to_block (b, i) n =
      G.unfocus (block_before b (G.entry (G.single_last (G.new_rtl i n)))) in
    match n with
    | GR.Cbranch (i, ifso, ifnot) ->

```

```

    let be = (T.boxmach.T.branch.T.project i,
              DG.Exit true, DG.Exit false) in
    let b = expand_cbranch (DG.Test (DG.Nop, be)) in
    let g = cbranch2cfg b ~ifso ~ifnot (G.entry G.empty) in
    G.of_block_list (G.postorder_dfs (G.unfocus g)) (* trim exit node *)
  | GR.Return _ -> to_block (DG.Nop, m.Mflow.return) n
  | GR.Forbidden _ -> to_block (DG.Nop, m.T.forbidden) n
  | GR.Branch (r, _) | GR.Mbranch (r, _) -> to_block (expand_branch (DG.Nop, r)) n
  | GR.Jump (r, _, _) -> to_block (expand_jump (DG.Nop, r)) n
  | GR.Call {GR.cal_i = r} -> to_block (expand_call (DG.Nop, r)) n
  | GR.Cut (r, _, _) -> to_block (expand_cut (DG.Nop, r)) n
  | GR.Exit -> to_block (DG.Nop, GR.last_instr n) n in
  G.expand expand_middle expand_last cfg in
let (g, modified) = expand expand_cfg proc in
(g, proc), modified

let block   proc b = fst (expand (fun (exp, _, _, _, _) _ -> exp b) proc)
let goto    proc b = fst (expand (fun (_, exp, _, _, _) _ -> exp b) proc)
let cbranch proc b = fst (expand (fun (_, _, exp, _, _) _ -> exp b) proc)
let call    proc b = fst (expand (fun (_, _, _, exp, _) _ -> exp b) proc)
let jump    proc b = fst (expand (fun (_, _, _, _, exp, _) _ -> exp b) proc)
let cut     proc b = fst (expand (fun (_, _, _, _, _, exp) _ -> exp b) proc)

let machine =
  let boxexp f = { T.embed = (fun p e -> f p (DG.Nop, Box.Exp.box e))
                  ; T.project = (fun _ -> assert false) } in
  let cutto =
    { T.embed =
      (fun p ca -> cut p (DG.Nop, Box.ExpList.box [ca.Mflow.new_sp; ca.Mflow.new_pc]))
    ; T.project = (fun box -> match Box.ExpList.unbox box with
                          | [sp; pc] -> {Mflow.new_sp = sp; Mflow.new_pc = pc}
                          | _ -> assert false)} in
  let move p ~src ~dst = Post.move ~dst ~src in
  let spill p r l =
    let w = Reg.width r in
    match Dn.rtl (l.RU.store (R.fetch (R.reg r) w) w) with
    | RP.Rtl [(RP.Const (RP.Bool true),
                RP.Store (RP.Mem (_, _, addr, assn), _, _))] ->
      Post.store ~addr:(Up.exp addr) ~src:r (R.aligned assn)
    | _ -> Impossible.impossible "unexpected spill RTL" in
  let reload p l r =
    let w = Reg.width r in
    match Dn.exp (l.RU.fetch w) with
    | RP.Fetch (RP.Mem (_, _, addr, assn), _) ->
      Post.load ~dst:r ~addr:(Up.exp addr) (R.aligned assn)
    | _ -> Impossible.impossible "unexpected reload exp" in
  { T.bnegate = Post.bnegate
  ; T.goto = boxexp goto
  ; T.jump = boxexp jump
  ; T.call = boxexp call
  ; T.branch = { T.embed = (fun p g -> cbranch p (DG.cond g))
                ; T.project = (fun _ -> assert false) }
  ; T.retgt_br =
    (fun r -> Post.bc_of_guard (DG.Nop, Up.exp (branch_condition r))
      ~ifso:(DG.Exit true) ~ifnot:(DG.Exit false))
  ; T.move = move
  ; T.spill = spill
  ; T.reload = reload
  ; T.cutto = cutto
  ; Mflow.return = Post.return

```

```

; T.forbidden = Post.forbidden
}

<internal utilities for the generic expander 519a>+≡ (501b) <515a
let branch_condition rtl = match Dn.rtl rtl with
| RP.Rtl [(g, RP.Store (left, right, w))] ->
    if not (RU.Eq.loc left pc_lhs) then
        impossf "conditional branch assigns to non-PC";
    g
| _ -> impossf "ill-formed conditional branch"

<block-conversion functions 519b>≡ (517c)
let update (g, m) = (if m then modified := true; g) in
let block_before b g = update (G.block_before machine b g) in
let block2cfg b = update (G.block2cfg machine b) in
let cbranch2cfg c ~ifso ~ifnot g = update (G.cbranch2cfg machine c ifso ifnot g) in

<old Make statements 519c>≡
and guard state succ fail e =
    let rec guard' succ fail = function

        | RP.Const(RP.Bool(true))      -> succ
        | RP.Const(RP.Bool(false))     -> fail
        | RP.Const(_)                  -> assert false
        | RP.Fetch(_)                  -> assert false
    in
    <special cases that match floating-point comparisons 520a>
    | RP.App(("conjoin",_), [x;y]) ->
        let ll = Idgen.label state.proc in
        let ls = F.symbol env ll in
        let fail = G.gm_label state.cfg ll ls fail in
        let tail = guard' succ fail y in
        let head = guard' tail fail x in
        head
    | RP.App(("not",_), [x]) -> guard' fail succ x
    | RP.App(("disjoin",_), [x;y]) ->
        let ll = Idgen.label state.proc in
        let ls = F.symbol env ll in
        let succ = G.gm_label state.cfg ll ls succ in
        let y = guard' succ fail y in
        let lexp = Rtl.codesym ls state.target.T.pointersize in
        let lrtl = state.target.T.goto.T.embed lexp in
        let succ = G.gm_goto state.cfg lrtl [G.lookup state.cfg ll] in
        guard' succ y x
    | RP.App(opr, args) as e ->
        let sl = Idgen.label state.proc in
        let ss = F.symbol env sl in
        let succ = G.gm_label state.cfg sl ss succ in

        let fl = Idgen.label state.proc in
        let fs = F.symbol env fl in
        let fail = G.gm_label state.cfg fl fs fail in

        let oexp = Rtl.codesym fs state.target.T.pointersize in
        let ortl = state.target.T.goto.T.embed oexp in
        let fail = G.gm_goto state.cfg ortl [G.lookup state.cfg fl] in

        let e = R.Revert.exp e in
        let link = Rtl.codesym ss state.target.T.pointersize in
        let rtl = state.target.T.branch.T.embed (e,link) in
        G.gm_branch state.cfg rtl succ fail

```



```
in
  { state with node = guard' succ fail e }
```

⟨special cases that match floating-point comparisons 520a⟩≡ (519c)
 (* the evil empire *)

R.14 Things yet to be covered in this document

R.15 front_ir/opshape.nw

⟨front_ir/opshape.ml 520b⟩≡
⟨opshape.ml 521a⟩

⟨front_ir/opshape.mli 520c⟩≡
⟨opshape.mli 520d⟩

R.16 Operator-shape analysis

⟨opshape.mli 520d⟩≡ (520c) 520f▷
⟨exported type definitions(opshape.nw) 520e⟩

⟨exported type definitions(opshape.nw) 520e⟩≡ (521a 520d)

```
type opr = Rtl.Private.opr
type exp = Rtl.Private.exp
type 'a hi = Hi of 'a
type 'a lo = Lo of 'a
type ('temp, 'warg) t =
  | Binop  of 'temp * 'temp
  | Unop   of 'temp
  | Binrm  of 'temp * 'temp * 'warg
  | Unrm   of 'temp * 'warg
  | Cmp    of 'temp * 'temp
  | Dblop  of 'temp * 'temp
  | Wrdop  of 'temp * 'temp * 'warg
  | Wrdrop of 'temp * 'temp * 'warg
  | Width
  | Fpcvt  of 'temp * 'warg
  | Bool
  | Nullary
```

⟨opshape.mli 520f⟩+≡ (520c) ◁520d
 val of_opr : opr -> (unit, unit) t
 val capply :
 ('exp -> 'c -> 'temp) ->
 ('exp -> 'c -> 'warg) -> opr -> 'exp list -> 'c list -> ('temp, 'warg) t

R.16.1 Shapes

R.16.2 Obsolete, unsupported operators

```
<opshape.ml 521a>≡ (520b) 521g▷  
  module SM = Strutil.Map  
  module T = Types
```

```
  let impossf fmt = Printf.kprintf Impossible.impossible fmt
```

```
  <exported type definitions(opshape.nw) 520e>
```

```
<'a 521b>≡ (521)  
  T.Bits (T.Var 1)
```

```
<'b 521c>≡ (521)  
  T.Bits (T.Var 2)
```

```
<bits1 521d>≡ (521)  
  T.Bits (T.Const 1)
```

```
<bits2 521e>≡ (521)  
  T.Bits (T.Const 2)
```

```
<rm 521f>≡ (521)  
  <bits2 521e>
```

```
<opshape.ml 521g>+≡ (520b) ◁521a 522▷  
  let shape scheme = match scheme with  
  | [  
    <'a 521b>  
    ;  
    <'a 521b>  
    ],  
    <'a 521b>  
    -> Binop ((), ())  
  | [  
    <'a 521b>  
    ],  
    <'a 521b>  
    -> Unop ()  
  | [  
    <'a 521b>  
    ;  
    <'a 521b>  
    ;  
    <rm 521f>  
    ],  
    <'a 521b>  
    -> Binrm ((), (), ())  
  | [  
    <'a 521b>  
    ;  
    <rm 521f>  
    ],  
    <'a 521b>  
    -> Unrm ((), ())  
  | [  
    <'a 521b>
```

```

;
⟨'a 521b⟩
], T.Bool                                -> Cmp ((), ())
| [
⟨'a 521b⟩
;
⟨'a 521b⟩
], T.Bits (T.Double 1) -> Dblop ((), ())
| [
⟨'a 521b⟩
;
⟨'a 521b⟩
;
⟨bits1 521d⟩
],
⟨'a 521b⟩
                                -> Wrdop ((), (), ())
| [
⟨'a 521b⟩
;
⟨'a 521b⟩
;
⟨bits1 521d⟩
],
⟨bits1 521d⟩
                                -> Wrdrop ((), (), ())
| [
⟨'a 521b⟩
],
⟨'b 521c⟩
                                -> Width
| [
⟨'a 521b⟩
;
⟨rm 521f⟩
],
⟨'b 521c⟩
                                -> Fpcvt ((), ())
| [T.Bool |
⟨bits1 521d⟩
], (T.Bool |
⟨bits1 521d⟩
) -> Bool
| [T.Bool; T.Bool ], T.Bool -> Bool
| [ ], T.Bool -> Bool
| [
(
⟨'a 521b⟩
|
⟨bits2 521e⟩
) -> Nullary
| _ -> impossf "unrecognized operator type %s" (Types.scheme_string scheme)

let shapetab = Rtlop.fold (fun o t z -> SM.add o (shape t) z) SM.empty

let of_opr (o, _) =
  try SM.find o shapetab with _ -> impossf "operator %%%s has no shape" o

⟨opshape.ml 522⟩+≡ (520b) <521g 523>
let args () = impossf "wrong number of arguments to operator"

```

```

let appfun t w = function
| Binop _ -> (function [x; y]      -> Binop (t x, t y)      | _ -> args())
| Unop _ -> (function [x]          -> Unop (t x)            | _ -> args())
| Binrm _ -> (function [x; y; r]    -> Binrm (t x, t y, w r) | _ -> args())
| Unrm _ -> (function [x;r ]        -> Unrm (t x, w r)       | _ -> args())
| Cmp _ -> (function [x; y]         -> Cmp (t x, t y)        | _ -> args())
| Dblop _ -> (function [x; y]       -> Dblop (t x, t y)      | _ -> args())
| Wrdop _ -> (function [x; y; c]     -> Wrdop (t x, t y, w c) | _ -> args())
| Wrdrop _ -> (function [x; y; c]    -> Wrdrop (t x, t y, w c) | _ -> args())
| Width _ -> (function [x]          -> Width                | _ -> args())
| Fpcvt _ -> (function [x;r ]        -> Fpcvt (t x, w r)      | _ -> args())
| Bool _ -> (function _              -> Bool)
| Nullary -> (function []            -> Nullary                | _ -> args())

```

(opshape.ml 523)+≡

(520b) <522

```

let cappfun t w =
  let binop cs es =
    match es, cs with
    | [x; y], [xc; yc] ->
      let x = t xc x in
      let y = t yc y in
      Binop (x, y)
    | _ -> args() in
  let unop cs es =
    match es, cs with [x], [xc] -> Unop (t xc x) | _ -> args() in
  let binrm cs es =
    match es, cs with
    | [x; y; r], [xc; yc; rc] ->
      let x = t xc x in
      let y = t yc y in
      let r = w rc r in
      Binrm (x, y, r)
    | _ -> args() in
  let unrm cs es =
    match es, cs with
    | [x; r], [xc; rc] ->
      let x = t xc x in
      let r = w rc r in
      Unrm (x, r)
    | _ -> args() in
  let cmp cs es =
    match es, cs with
    | [x; y], [xc; yc] ->
      let x = t xc x in
      let y = t yc y in
      Cmp (x, y)
    | _ -> args() in
  let dblop cs es =
    match es, cs with
    | [x; y], [xc; yc] ->
      let x = t xc x in
      let y = t yc y in
      Dblop (x, y)
    | _ -> args() in
  let wrdop cs es =
    match es, cs with
    | [x; y; z], [xc; yc; zc] ->
      let x = t xc x in
      let y = t yc y in
      let z = w zc z in

```

```

      Wrdop(x, y, z)
    | _ -> args() in
let wrdrop cs es =
  match es, cs with
  | [x; y; z], [xc; yc; zc] ->
    let x = t xc x in
    let y = t yc y in
    let z = w zc z in
    Wrdop(x, y, z)
  | _ -> args() in
let width cs es =
  match es, cs with [x], [xc] -> Width | _ -> args() in
let fpcvt cs es =
  match es, cs with
  | [x; r], [xc; rc] ->
    let x = t xc x in
    let r = w rc r in
    Fpcvt (x, r)
  | _ -> args() in
let nullary cs es =
  match es, cs with [], [] -> Width | _ -> args() in
function
| Binop _ -> binop
| Unop _ -> unop
| Binrm _ -> binrm
| Unrm _ -> unrm
| Cmp _ -> cmp
| Dblop _ -> dblop
| Wrdop _ -> wrdrop
| Wrdrop _ -> wrdrop
| Width _ -> width
| Fpcvt _ -> fpcvt
| Bool _ -> (fun _ _ -> Bool)
| Nullary _ -> nullary

let capply to_temp to_warg opr = cappfun to_temp to_warg (of_opr opr)

```

R.17 front_ir/preast2ir.nw

$\langle \text{front_ir/preast2ir.ml } 524a \rangle \equiv$
 $\langle \text{preast2ir.ml } 525a \rangle$

$\langle \text{front_ir/preast2ir.mli } 524b \rangle \equiv$
 $\langle \text{preast2ir.mli } 524d \rangle$

R.18 Target and proc definitions

$\langle \text{type defs } 524c \rangle \equiv$ (525a 524d)
 type tgt = T of (basic_proc, (Rtl.exp -> Automaton.t), Call.t) Target.t
 and basic_proc = (Automaton.t, unit, Call.t, tgt) Procedure.t
 type proc = Zipcfg.graph * basic_proc
 type old_proc = (Automaton.t, Rtl.rtl Cfgx.M.cfg, Call.t, tgt) Procedure.t

$\langle \text{preast2ir.mli } 524d \rangle \equiv$ (524b)
 $\langle \text{type defs } 524c \rangle$
 val tgt : basic_proc -> (basic_proc, (Rtl.exp -> Automaton.t), Call.t) Target.t

```

⟨preast2ir.ml 525a⟩≡ (524a)
  ⟨type defs 524c⟩
  let tgt {Procedure.target = T t} = t

```

R.19 front_ir/proc.nw

```

⟨front_ir/procedure.ml 525b⟩≡
  ⟨proc.ml 525f⟩

⟨front_ir/procedure.mli 525c⟩≡
  ⟨proc.mli 525e⟩

```

R.20 Procedure Intermediate Representation

```

⟨type t(proc.nw) 525d⟩≡ (525)
  type overflow = (Block.t list, Block.t list) Call.split_blocks
  type ('automaton, 'cfg, 'cc, 'tgt) t =
    { symbol:      Symbol.t           (* of procedure *)
    ; cc:          'cc                (* calling convention *)
    ; target:      'tgt               (* target of this procedure *)
    ; formals:     (int * Ast.bare_formal) list (* formal args w/ indices *)
    ; temps:       Talloc.Multiple.t  (* allocator for temporaries *)
    ; mk_symbol:   string -> Symbol.t (* allocator for symbols *)
    ; cfg:         'cfg               (* control-flow graph *)
    ; oldblocks:   overflow           (* stack - incoming parms, outgoing results *)
    ; youngblocks: overflow           (* stack - outgoing parms, incoming results *)
    ; stackd:      Block.t            (* stack - user stack data *)
    ; conts:       Block.t            (* pairs of pointers for conts *)
    ; sp:          Block.t            (* the 'standard' location of sp in body *)
    ; priv:        'automaton         (* stack - spill slots etc - still open *)
    ; eqns:        Rtleqn.t list      (* eqns for compile time consts *)
    ; vars:        int                (* number of local vars + parameters *)
    ; nvregs:      int                (* number of non-volatile registers *)
    ; var_map:     Automaton.loc option array (* where variables are stored *)
    ; global_map:  Automaton.loc array (* where global variables are stored *)
    ; bodylbl:     Zipcfg.uid * string (* beginning of procedure body *)
    ; headroom:    int                (* size of the headroom *)
    ; exp_of_lbl:  (Unique.uid * string) -> Rtl.exp (* exp of code label *)
    }

```

```

⟨proc.mli 525e⟩≡ (525c)
  ⟨type t(proc.nw) 525d⟩

```

```

⟨proc.ml 525f⟩≡ (525b)
  ⟨type t(proc.nw) 525d⟩

```

R.21 front_ir/rewrite.nw

```

⟨front_ir/rewrite.ml 525g⟩≡
  ⟨rewrite.ml 527d⟩

⟨front_ir/rewrite.mli 525h⟩≡
  ⟨rewrite.mli 526b⟩

```

R.22 Rewriting RTL operators (and related utilities)

```
<exported type abbreviations 526a>≡ (527d 526b)
type width = Rtl.width
type exp   = Rtl.exp
type loc   = Rtl.loc
type block = exp Postexpander.block
type temp  = Register.t

<rewrite.mli 526b>≡ (525h) 526c>
<exported type abbreviations 526a>
```

R.22.1 Basic operator interface

```
<rewrite.mli 526c>+≡ (525h) <526b 526d>
module Ops : sig
  <type signatures for applying standard operators 530>
  val signed : width -> int -> exp
  val unsigned : width -> int -> exp
end
```

R.22.2 Rewriting functions

```
<rewrite.mli 526d>+≡ (525h) <526c 526e>
val div' : width -> dst:loc -> exp -> exp -> quot:exp -> rem:exp -> block
  (* uses: comparisons, add, sub *)

val div2 : width -> dst:loc -> exp -> exp -> block
  (* always 2 conditional branches *)

val div1_3 : width -> dst:loc -> exp -> exp -> block
  (* 1--3 conditional branches *)

<rewrite.mli 526e>+≡ (525h) <526d 526f>
val div_overflows : width -> exp -> exp -> exp
  (* uses: conjoin, eq *)

val (mod) : width -> exp -> exp -> exp
  (* uses : div, mul, sub *)

val modu : width -> exp -> exp -> exp
  (* uses : divu, mul, sub *)

val rem : width -> exp -> exp -> exp
  (* uses : quot, mul, sub *)

<rewrite.mli 526f>+≡ (525h) <526e 526g>
val popcnt : width -> dst:loc -> exp -> block (* for 32 bits only *)

<rewrite.mli 526g>+≡ (525h) <526f 527a>
val sxlo : width -> width -> exp -> exp
  (* uses : shl, shra *)
val zxlo : width -> width -> exp -> exp
  (* uses : shl, shr1 *)
```

R.22.3 Other related utilities

```
<rewrite.mli 527a>+≡ (525h) <526g 527b>
  val regpair : hi:exp -> lo:exp -> exp

<rewrite.mli 527b>+≡ (525h) <527a 527c>
  val slice : width -> width -> lsb:int -> exp -> exp

<rewrite.mli 527c>+≡ (525h) <527b>
  type 'a pair = { hi : 'a; lo : 'a }
  val splits : width -> lsb:int -> exp -> exp pair
  val splitu : width -> lsb:int -> exp -> exp pair
```

R.22.4 Implementation

```
<rewrite.ml 527d>≡ (525g) 527e>
  module B = Bits
  module BO = Bits.Ops
  module DG = Dag
  module R = Rtl
  module RU = Rtlutil

  let (<:>) = DG.<:>

  <exported type abbreviations 526a>

  module Ops = struct
    <automatically generated implementations of operators 528e>
    let signed w n = R.bits (B.S.of_int n w) w
    let unsigned w n = R.bits (B.U.of_int n w) w
  end
  module O = Ops

  let div_overflows w x y =
    let minint = R.bits (BO.shl (B.U.of_int 1 w) (B.U.of_int (w-1) w)) w in
    O.conjoin (O.eq w x minint) (O.eq w y (O.signed w (-1)))

  let (mod) w x y = O.sub w x (O.mul w y (O.div w x y))
  let (modu) w x y = O.sub w x (O.mul w y (O.divu w x y))
  let (rem) w x y = O.sub w x (O.mul w y (O.quot w x y))

<rewrite.ml 527e>+≡ (525g) <527d 527f>
  let sxlo n w x =
    let shamt = O.unsigned w (w-n) in
    O.shra w (O.shl w x shamt) shamt

  let zxlo n w x =
    let shamt = O.unsigned w (w-n) in
    O.shrl w (O.shl w x shamt) shamt

<rewrite.ml 527f>+≡ (525g) <527e 528a>
  let popcnt w ~dst x =
    if w <> 32 then
      Unsupported.popcnt ~notok:w ~ok:32;
    let u = O.unsigned 32 in
    let (@:=) l r = DG.Rtl (R.store l r 32) in
    let (&) l r = O._and 32 l r in
    let (>>) l r = O.shrl 32 l (u r) in
```



```

let (-)  l r = 0.sub 32 l r in
let (+)  l r = 0.add 32 l r in
let ( * ) l r = 0.mul 32 l r in
let tmploc = dst in
let tmpval = R.fetch tmploc 32 in
let mask3  = u 0x33333333 in
let mask5  = R.bits (Bits.U.of_int64 0x55555555L 32) 32 in    (* Caml sign bit set *)
let maskF  = u 0x0F0F0F0F in
let mask01 = u 0x01010101 in
tmploc @:= x - ((x >> 1) & mask5) <:>
tmploc @:= (tmpval & mask3) + ((tmpval >> 2) & mask3) <:>
tmploc @:= ((tmpval + (tmpval >> 4)) & maskF) <:>
tmploc @:= ((tmpval * mask01) >> 24)    (* AMD guide *)

⟨rewrite.ml 528a⟩+≡ (525g) <527f 528b>
let regpair ~hi ~lo =
  let hw = RU.Width.exp hi in
  let lw = RU.Width.exp lo in
  let w = hw + lw in
  0.(or) w (0.shl w (0.zx hw w hi) (0.unsigned w lw)) (0.zx lw w lo)

⟨rewrite.ml 528b⟩+≡ (525g) <528a 528c>
let slice w n ~lsb e =
  if lsb = 0 then
    0.lobits w n e
  else
    0.lobits w n (0.shrl w e (0.unsigned w lsb))

⟨rewrite.ml 528c⟩+≡ (525g) <528b 528d>
type 'a pair = { hi : 'a; lo : 'a }

let splitu w ~lsb:n e =
  assert (0 < n && n < w);
  let lo = 0.zx n w (0.lobits w n e) in
  let n  = 0.unsigned w n in
  let hi = 0.shl w (0.shrl w e n) n in
  { hi = hi; lo = lo }

⟨rewrite.ml 528d⟩+≡ (525g) <528c 532b>
let splits w ~lsb:n e =
  assert (0 < n && n < w);
  let lo      = 0.sx n w (0.lobits w n e) in
  let adjust = 0.zx 1 w (slice w 1 ~lsb:(n-1) e) in
  let n      = 0.unsigned w n in
  let hi      = 0.shl w (0.add w (0.shrl w e n) adjust) n in
  { hi = hi; lo = lo }

```

R.23 Things not to be looked at

R.23.1 Automatically generated helper functions

```

⟨automatically generated implementations of operators 528e⟩≡ (527d)
(* This code generated automatically by Rtlop.Emit.creators *)
let _NaN w w' x = Rtl.app (Rtl.opr "NaN" [w;w';]) [x; ]
let add w x y = Rtl.app (Rtl.opr "add" [w;]) [x; y; ]
let addc w x y z = Rtl.app (Rtl.opr "addc" [w;]) [x; y; z; ]
let add_overflows w x y = Rtl.app (Rtl.opr "add_overflows" [w;]) [x; y; ]
let _and w x y = Rtl.app (Rtl.opr "and" [w;]) [x; y; ]

```

```

let bit x = Rtl.app (Rtl.opr "bit" []) [x; ]
let bool x = Rtl.app (Rtl.opr "bool" []) [x; ]
let borrow w x y z = Rtl.app (Rtl.opr "borrow" [w;]) [x; y; z; ]
let carry w x y z = Rtl.app (Rtl.opr "carry" [w;]) [x; y; z; ]
let com w x = Rtl.app (Rtl.opr "com" [w;]) [x; ]
let conjoin x y = Rtl.app (Rtl.opr "conjoin" []) [x; y; ]
let disjoin x y = Rtl.app (Rtl.opr "disjoin" []) [x; y; ]
let div w x y = Rtl.app (Rtl.opr "div" [w;]) [x; y; ]
let div_overflows w x y = Rtl.app (Rtl.opr "div_overflows" [w;]) [x; y; ]
let divu w x y = Rtl.app (Rtl.opr "divu" [w;]) [x; y; ]
let eq w x y = Rtl.app (Rtl.opr "eq" [w;]) [x; y; ]
let f2f w w' x y = Rtl.app (Rtl.opr "f2f" [w;w';]) [x; y; ]
let f2f_implicit_round w w' x = Rtl.app (Rtl.opr "f2f_implicit_round" [w;w';]) [x; ]
let f2i w w' x y = Rtl.app (Rtl.opr "f2i" [w;w';]) [x; y; ]
let fabs w x = Rtl.app (Rtl.opr "fabs" [w;]) [x; ]
let fadd w x y z = Rtl.app (Rtl.opr "fadd" [w;]) [x; y; z; ]
let fcmp w x y = Rtl.app (Rtl.opr "fcmp" [w;]) [x; y; ]
let fdiv w x y z = Rtl.app (Rtl.opr "fdiv" [w;]) [x; y; z; ]
let feq w x y = Rtl.app (Rtl.opr "feq" [w;]) [x; y; ]
let fge w x y = Rtl.app (Rtl.opr "fge" [w;]) [x; y; ]
let fgt w x y = Rtl.app (Rtl.opr "fgt" [w;]) [x; y; ]
let fle w x y = Rtl.app (Rtl.opr "fle" [w;]) [x; y; ]
let float_eq = Rtl.app (Rtl.opr "float_eq" []) []
let float_gt = Rtl.app (Rtl.opr "float_gt" []) []
let float_lt = Rtl.app (Rtl.opr "float_lt" []) []
let flt w x y = Rtl.app (Rtl.opr "flt" [w;]) [x; y; ]
let fmul w x y z = Rtl.app (Rtl.opr "fmul" [w;]) [x; y; z; ]
let fmulx w x y = Rtl.app (Rtl.opr "fmulx" [w;]) [x; y; ]
let fne w x y = Rtl.app (Rtl.opr "fne" [w;]) [x; y; ]
let fneg w x = Rtl.app (Rtl.opr "fneg" [w;]) [x; ]
let fordered w x y = Rtl.app (Rtl.opr "fordered" [w;]) [x; y; ]
let fsqrt w x y = Rtl.app (Rtl.opr "fsqrt" [w;]) [x; y; ]
let fsub w x y z = Rtl.app (Rtl.opr "fsub" [w;]) [x; y; z; ]
let funordered w x y = Rtl.app (Rtl.opr "funordered" [w;]) [x; y; ]
let ge w x y = Rtl.app (Rtl.opr "ge" [w;]) [x; y; ]
let geu w x y = Rtl.app (Rtl.opr "geu" [w;]) [x; y; ]
let gt w x y = Rtl.app (Rtl.opr "gt" [w;]) [x; y; ]
let gtu w x y = Rtl.app (Rtl.opr "gtu" [w;]) [x; y; ]
let i2f w w' x y = Rtl.app (Rtl.opr "i2f" [w;w';]) [x; y; ]
let le w x y = Rtl.app (Rtl.opr "le" [w;]) [x; y; ]
let leu w x y = Rtl.app (Rtl.opr "leu" [w;]) [x; y; ]
let lobits w w' x = Rtl.app (Rtl.opr "lobits" [w;w';]) [x; ]
let lt w x y = Rtl.app (Rtl.opr "lt" [w;]) [x; y; ]
let ltu w x y = Rtl.app (Rtl.opr "ltu" [w;]) [x; y; ]
let minf w = Rtl.app (Rtl.opr "minf" [w;]) []
let (mod) w x y = Rtl.app (Rtl.opr "mod" [w;]) [x; y; ]
let modu w x y = Rtl.app (Rtl.opr "modu" [w;]) [x; y; ]
let mul w x y = Rtl.app (Rtl.opr "mul" [w;]) [x; y; ]
let mulux w x y = Rtl.app (Rtl.opr "mulux" [w;]) [x; y; ]
let mulx w x y = Rtl.app (Rtl.opr "mulx" [w;]) [x; y; ]
let mul_overflows w x y = Rtl.app (Rtl.opr "mul_overflows" [w;]) [x; y; ]
let mulu_overflows w x y = Rtl.app (Rtl.opr "mulu_overflows" [w;]) [x; y; ]
let mzero w = Rtl.app (Rtl.opr "mzero" [w;]) []
let ne w x y = Rtl.app (Rtl.opr "ne" [w;]) [x; y; ]
let neg w x = Rtl.app (Rtl.opr "neg" [w;]) [x; ]
let not x = Rtl.app (Rtl.opr "not" []) [x; ]
let (or) w x y = Rtl.app (Rtl.opr "or" [w;]) [x; y; ]
let pinf w = Rtl.app (Rtl.opr "pinf" [w;]) []
let popcnt w x = Rtl.app (Rtl.opr "popcnt" [w;]) [x; ]
let pzero w = Rtl.app (Rtl.opr "pzero" [w;]) []

```

```

let quot w x y = Rtl.app (Rtl.opr "quot" [w;]) [x; y; ]
let quot_overflows w x y = Rtl.app (Rtl.opr "quot_overflows" [w;]) [x; y; ]
let rem w x y = Rtl.app (Rtl.opr "rem" [w;]) [x; y; ]
let rotl w x y = Rtl.app (Rtl.opr "rotl" [w;]) [x; y; ]
let rotr w x y = Rtl.app (Rtl.opr "rotr" [w;]) [x; y; ]
let round_down = Rtl.app (Rtl.opr "round_down" []) []
let round_nearest = Rtl.app (Rtl.opr "round_nearest" []) []
let round_up = Rtl.app (Rtl.opr "round_up" []) []
let round_zero = Rtl.app (Rtl.opr "round_zero" []) []
let shl w x y = Rtl.app (Rtl.opr "shl" [w;]) [x; y; ]
let shra w x y = Rtl.app (Rtl.opr "shra" [w;]) [x; y; ]
let shr1 w x y = Rtl.app (Rtl.opr "shr1" [w;]) [x; y; ]
let sub w x y = Rtl.app (Rtl.opr "sub" [w;]) [x; y; ]
let subb w x y z = Rtl.app (Rtl.opr "subb" [w;]) [x; y; z; ]
let sub_overflows w x y = Rtl.app (Rtl.opr "sub_overflows" [w;]) [x; y; ]
let sx w w' x = Rtl.app (Rtl.opr "sx" [w;w';]) [x; ]
let unordered = Rtl.app (Rtl.opr "unordered" []) []
let xor w x y = Rtl.app (Rtl.opr "xor" [w;]) [x; y; ]
let zx w w' x = Rtl.app (Rtl.opr "zx" [w;w';]) [x; ]
let bitExtract w w' x y = Rtl.app (Rtl.opr "bitExtract" [w;w';]) [x; y; ]
let bitInsert w w' x y z = Rtl.app (Rtl.opr "bitInsert" [w;w';]) [x; y; z; ]
let bitTransfer w x y z u v = Rtl.app (Rtl.opr "bitTransfer" [w;]) [x; y; z; u; v; ]

```

(type signatures for applying standard operators 530) ≡ (526c)

```

val _NaN : width -> width -> exp -> exp
val add : width -> exp -> exp -> exp
val addc : width -> exp -> exp -> exp -> exp
val add_overflows : width -> exp -> exp -> exp -> exp
val _and : width -> exp -> exp -> exp
val bit : exp -> exp
val bool : exp -> exp
val borrow : width -> exp -> exp -> exp -> exp
val carry : width -> exp -> exp -> exp -> exp
val com : width -> exp -> exp
val conjoin : exp -> exp -> exp
val disjoin : exp -> exp -> exp
val div : width -> exp -> exp -> exp
val div_overflows : width -> exp -> exp -> exp
val divu : width -> exp -> exp -> exp
val eq : width -> exp -> exp -> exp
val f2f : width -> width -> exp -> exp -> exp
val f2f_implicit_round : width -> width -> exp -> exp
val f2i : width -> width -> exp -> exp -> exp
val fabs : width -> exp -> exp
val fadd : width -> exp -> exp -> exp -> exp
val fcmp : width -> exp -> exp -> exp
val fdiv : width -> exp -> exp -> exp -> exp
val feq : width -> exp -> exp -> exp
val fge : width -> exp -> exp -> exp
val fgt : width -> exp -> exp -> exp
val fle : width -> exp -> exp -> exp
val float_eq : exp
val float_gt : exp
val float_lt : exp
val flt : width -> exp -> exp -> exp
val fmul : width -> exp -> exp -> exp -> exp
val fmulx : width -> exp -> exp -> exp
val fne : width -> exp -> exp -> exp
val fneg : width -> exp -> exp
val fordered : width -> exp -> exp -> exp

```

```

val fsqrt : width -> exp -> exp -> exp
val fsub : width -> exp -> exp -> exp -> exp
val funordered : width -> exp -> exp -> exp
val ge : width -> exp -> exp -> exp
val geu : width -> exp -> exp -> exp
val gt : width -> exp -> exp -> exp
val gtu : width -> exp -> exp -> exp
val i2f : width -> width -> exp -> exp -> exp
val le : width -> exp -> exp -> exp
val leu : width -> exp -> exp -> exp
val lobits : width -> width -> exp -> exp
val lt : width -> exp -> exp -> exp
val ltu : width -> exp -> exp -> exp
val minf : width -> exp
val ( mod ) : width -> exp -> exp -> exp
val modu : width -> exp -> exp -> exp
val mul : width -> exp -> exp -> exp
val mulux : width -> exp -> exp -> exp
val mulx : width -> exp -> exp -> exp
val mul_overflows : width -> exp -> exp -> exp
val mulu_overflows : width -> exp -> exp -> exp
val mzero : width -> exp
val ne : width -> exp -> exp -> exp
val neg : width -> exp -> exp
val not : exp -> exp
val ( or ) : width -> exp -> exp -> exp
val pinf : width -> exp
val popcnt : width -> exp -> exp
val pzero : width -> exp
val quot : width -> exp -> exp -> exp
val quot_overflows : width -> exp -> exp -> exp
val rem : width -> exp -> exp -> exp
val rotl : width -> exp -> exp -> exp
val rotr : width -> exp -> exp -> exp
val round_down : exp
val round_nearest : exp
val round_up : exp
val round_zero : exp
val shl : width -> exp -> exp -> exp
val shra : width -> exp -> exp -> exp
val shr1 : width -> exp -> exp -> exp
val sub : width -> exp -> exp -> exp
val subb : width -> exp -> exp -> exp -> exp
val sub_overflows : width -> exp -> exp -> exp
val sx : width -> width -> exp -> exp
val unordered : exp
val xor : width -> exp -> exp -> exp
val zx : width -> width -> exp -> exp

```

R.23.2 C code to test division rules

<divtest.c 531>≡

532a▷

```

#include <assert.h>
#include <stdio.h>
#include <stdlib.h>

int mosml_div(int x, int y) {
    assert(y != 0);
    if( y < 0 ) { x = - x; y = -y; }

```

```

if( x >= 0 ) { return x / y; }
else
    { x = - x;
      if( x % y == 0 )
          return - (x /y);
      else
          return - (x / y) - 1;
    }
}

// incorrect
int simplifiediv(int x, int y) {
    if( y < 0 ) { x = - x; y = -y; }
    if( x >= 0 )
        { return x / y; }
    else
        { return - (- x + (y - 1)) / y; // can fail because of overflow
        }
}

```

⟨divtest.c 532a⟩+≡ ◁531 532c▷

```

int njdiv(int x, int y) {
    if (y >= 0) {
        if (x >= 0) {
            return x / y;
        } else {
            return (x+1) / y - 1;
        }
    } else {
        if (x > 0) {
            return (x-1) / y - 1;
        } else {
            return x / y;
        }
    }
}

```

⟨rewrite.ml 532b⟩+≡ (525g) ▷528d 533a▷

```

let div2 w ~dst x y =
  let return e = DG.Rtl (R.store dst e w) in
  let one  = 0.signed w 1 in
  let zero = 0.signed w 0 in
  DG.If (DG.cond (0.ge w y zero),
    DG.If (DG.cond (0.ge w x zero),
      return (0.quot w x y),
      return (0.sub w (0.quot w (0.add w x one) y) one)),
    DG.If (DG.cond (0.gt w x zero),
      return (0.sub w (0.quot w (0.sub w x one) y) one),
      return (0.quot w x y)))

```

⟨divtest.c 532c⟩+≡ ◁532a 533b▷

```

int njfast (int x, int y) {
    if ((x ^ y) >= 0) {
        return x / y;
    } else if (y >= 0) {
        return (x+1) / y - 1;
    } else if (x == 0) {
        return 0;
    } else {

```

```

    return (x-1) / y - 1;
}
}

⟨rewrite.ml 533a⟩+≡ (525g) <532b 533c>
let div1_3 w ~dst x y =
  let return e = DG.Rtl (R.store dst e w) in
  let one = 0.signed w 1 in
  let zero = 0.signed w 0 in
  DG.If (DG.cond (0.ge w (0.xor w x y) zero),
    return (0.quot w x y),
  DG.If (DG.cond (0.ge w y zero),
    return (0.sub w (0.quot w (0.add w x one) y) one),
  DG.If (DG.cond (0.eq w x zero),
    return zero,
    return (0.sub w (0.quot w (0.sub w x one) y) one))))

⟨divtest.c 533b⟩+≡ <532c 533d>
int m3div (int x, int y) {
  int rem = x % y;
  int quot = x / y;
  if ((x ^ y) < 0 && rem != 0) {
    return quot - 1;
  } else {
    return quot;
  }
}

⟨rewrite.ml 533c⟩+≡ (525g) <533a
let div' w ~dst x y ~quot ~rem =
  let return e = DG.Rtl (R.store dst e w) in
  DG.If (DG.cond (0.conjoin (0.lt w (0.xor w x y) (0.signed w 0))
    (0.ne w rem (0.signed w 0))),
    return (0.sub w quot (0.signed w 1)),
    return quot)

⟨divtest.c 533d⟩+≡ <533b
#define nzsign(Y) ((Y) > 0 ? 1 : -1)

// incorrect
int signdiv(int x, int y) {
  if ((x ^ y) >= 0) {
    return x / y;
  } else {
    return (x - (y - nzsign(y))) / y; // can fail --- overflow
  }
}

// incorrect
int rounddiv(int x, int y) {
  if ((x ^ y) >= 0) {
    return x / y;
  } else if (y < 0) {
    return (x - y - 1) / y; // can fail --- overflow
  } else {
    return (x - y + 1) / y; // can fail --- overflow
  }
}

```

```

struct test { int ( * fun)(int, int); const char *name; };

struct test tests[] = {
    { mosml_div, "mosml_div" },
    // { simplediv, "simplediv" },
    { njdiv, "njdiv" },
    { njfast, "njfast" },
    { m3div, "m3div" },
    // { signdiv, "signdiv" },
    // { rounddiv, "rounddiv" },
};

int divtest(int x, int y) {
    int i, q;
    int failcount = 0;
    assert(y != 0);

    q = mosml_div(x, y);
    for (i = 0; i < sizeof(tests) / sizeof(tests[0]); i++) {
        int tq = tests[i].fun(x, y);
        printf("%12s(%3d, %3d) == %3d %s\n", tests[i].name, x, y, tq,
            tq == q ? "" : "<===== ERROR");
        if (tq != q) failcount++;
    }
    return failcount;
}

int randtest(unsigned n) {
    int *xs, *ys, i, rc;
    FILE *rand;
    int fails = 0;
    xs = malloc(n * sizeof(*xs));
    ys = malloc(n * sizeof(*ys));
    assert (xs != NULL && ys != NULL);
    rand = fopen("/dev/urandom", "r");
    if (!rand) {
        perror("Cannot open /dev/urandom --- need a Linux box?");
        exit(1);
    }
    rc = fread(xs, sizeof(*xs), n, rand);
    assert(n == rc);
    rc = fread(ys, sizeof(*ys), n, rand);
    assert(n == rc);
    fclose(rand);
    for (i = 0; i < n; i++)
        if (ys[i])
            fails += divtest(xs[i], ys[i]);
    return fails;
}

main() {
    int x, y;
    int fails = 0;
    printf("%d\n", njdiv(1606571677, -684103929));
    for (x = -10; x <= 10; x++)
        for (y = -10; y <= 10; y++)
            if (y != 0)
                fails += divtest(x, y);
}

```

```

fails += randtest(500000);
if (fails)
    printf("%d failures <====\n", fails);
else
    printf("All tests passed.\n");
}

```

R.24 front_ir/runtimedata.nw

```

<front_ir/runtimedata.ml 535a>≡
  <runtimedata.ml 535h>

```

```

<front_ir/runtimedata.mli 535b>≡
  <runtimedata.mli 535c>

```

R.25 Collecting and emitting data for the run-time system

R.25.1 The interface

```

<runtimedata.mli 535c>≡ (535b) 535d▷
  type spans = Spans.t
  exception DeadValue
  val upd_spans      : (Rtl.loc -> Rtl.loc) -> spans -> unit    (* may raise DeadValue *)
  val upd_all_spans :
    (Rtl.loc -> Rtl.loc) -> Zipcfg.graph -> unit    (* may raise DeadValue *)

```

```

<runtimedata.mli 535d>+≡ (535b) <535c 535e>
  type tgt = Preast2ir.tgt
  val print_reg_map : tgt -> unit

```

```

<runtimedata.mli 535e>+≡ (535b) <535d 535f>
  val emit_as_asm : tgt -> 'a Asm.assembler -> procsym:Symbol.t -> Zipcfg.graph -> unit

```

```

<runtimedata.mli 535f>+≡ (535b) <535e 535g>
  val user_spans : spans -> (Bits.bits * Reloc.t) list
  val stackdata  : spans -> Rtl.loc list

```

```

<runtimedata.mli 535g>+≡ (535b) <535f>
  val emit_global_properties : ('a, 'b, Call.t) Target.t -> 'c Asm.assembler -> unit

```

R.25.2 Imports, type definitions, and utilities

```

<runtimedata.ml 535h>≡ (535a) 536a▷
  module B = Bits
  module G = Zipcfg
  module GR = Zipcfg.Rep
  module CT = Ctypes
  module PA = Preast2ir
  module RM = Register.Map
  module RS = Register.Set
  module R = Rtl
  module RP = Rtl.Private
  module RU = Rtlutil
  module W = Rtlutil.Width
  module Dn = Rtl.Dn

```



```

module Up = Rtl.Up
module S = Spans
module T = Target
module IM = Map.Make (struct type t = int let compare = compare end)

type tgt = Preast2ir.tgt
type spans = S.t
exception DeadValue

let imposs = Impossible.impossible
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let unimp = Impossible.unimp

(* Don't touch the user-defined spans -- not necessary *)
let upd_spans upd spans =
  let spans = S.expose spans in
  let maybe_upd l = match l with
    | Some l -> (try Some (upd l) with DeadValue -> None)
    | None -> None in
  spans.S.inalloc <- upd spans.S.inalloc;
  spans.S.outalloc <- upd spans.S.outalloc;
  spans.S.ra <- upd spans.S.ra;
  spans.S.csregs <- List.map (fun (r,l) -> (r, maybe_upd l)) spans.S.csregs;
  spans.S.conts <- List.map (fun (pc,sp,vars) -> (pc, upd sp, vars)) spans.S.conts;
  spans.S.sds <- List.map upd spans.S.sds;
  Array.iteri (fun i l -> spans.S.vars.(i) <- maybe_upd l) spans.S.vars

let upd_all_spans upd g = Zipcfg.iter_spans (upd_spans upd) g

<runtime data.ml 536a>+≡ (535a) <535h 539b>
let emit_as_asm (PA.T target) asm ~procsym cfg =
  <gather the spans in order 536b>
  let wordsize = target.T.wordsizes in
  let to_bits i = Bits.S.of_int i wordsize in
  let zero = Bits.zero wordsize in
  let one = to_bits 1 in
  <define functions for emitting data 537f>
  <define emit_sd_spans and emit_site_spans 536c>
  let prev_section = asm#current in
  emit_sd_spans ();
  List.iter emit_site_spans spans;
  asm#section prev_section

<gather the spans in order 536b>≡ (536a)
let add_spans (first, _ as b) _ spans =
  let (_, last) = GR.goto_end (GR.unzip b) in
  let add_spans (_, lbl) = function Some ss -> (lbl, ss) :: spans | None -> spans in
  let spans = match first with
    | GR.Label (lbl, _, ss) -> add_spans lbl (!ss)
    | GR.Entry -> spans in
  match last with
  | GR.Call { GR.cal_spans = ss; GR.cal_contedges = {G.node = n} :: _ } ->
    add_spans n ss
  | _ -> spans in
let spans = List.rev (G.fold_layout add_spans [] cfg) in

<define emit_sd_spans and emit_site_spans 536c>≡ (536a) 537a>
let sd_label = asm#local (Idgen.label "stackdata") in

let emit_sd_spans () =

```

```

match spans with
| (_, spans)::_ ->
    let spans = S.expose spans in
    asm#section "pcmap_data";
    asm#label   sd_label;
    asm#value   (Bits.U.of_int (List.length spans.S.sds) target.T.wordsize);
    List.iter   emit_loc spans.S.sds
| [] -> () in

⟨define emit_sd_spans and emit_site_spans 537a⟩+≡ (536a) <536c
    let emit_site_spans (stop_l, spans) =
        let spans = S.expose spans in
        let frame_label = asm#local (Idgen.label "frame") in
        ⟨emit proc to run-time data map 537b⟩
        ⟨emit frame data 537c⟩
    in

⟨emit proc to run-time data map 537b⟩≡ (537a)
    asm#section "pcmap";
    asm#addr (Reloc.of_sym (asm#local stop_l, Rtl.codesym) wordsize);
    asm#addr (Reloc.of_sym (frame_label, Rtl.datasym) wordsize);

⟨emit frame data 537c⟩≡ (537a) 537d▷
    asm#section "pcmap_data";
    asm#label frame_label;
    emit_loc spans.S.inalloc;
    emit_loc spans.S.outalloc;
    emit_loc spans.S.ra;
    asm#addr (Reloc.of_sym (sd_label, Rtl.datasym) wordsize);
    emit_int (List.length (List.filter nonredundant spans.S.csregs));
    emit_int (Array.length spans.S.vars);

⟨emit frame data 537d⟩+≡ (537a) <537c 537e▷
    let users =
        let unsorted = List.map (fun (n,e) -> (Bits.S.to_int n,e)) spans.S.users in
        let compare ((n : int), _) ((n' : int), _) = compare n n' in
        List.stable_sort compare unsorted in
    let max_span_ix = match users with [] -> -1 | _ :: _ -> fst (Auxfuns.last users) in
    let rec emit_spans_from n spans =
        if n <= max_span_ix then
            match spans with
            | (n', e) :: tail when n > n' -> emit_spans_from n tail (* skip duplicate span *)
            | (n', e) :: tail when n = n' -> asm#addr e; emit_spans_from (n+1) tail
            | _ -> emit_int 0; emit_spans_from (n+1) spans in

⟨emit frame data 537e⟩+≡ (537a) <537d
    emit_int (max_span_ix + 1);
    emit_int (size_cont_block spans.S.conts);
    List.iter emit_csreg spans.S.csregs;
    Array.iter emit_maybe_loc spans.S.vars;
    emit_conts spans.S.conts;
    emit_spans_from 0 users;

⟨define functions for emitting data 537f⟩≡ (536a) 538a▷
    ⟨define simplify_exp to simplify rtl expressions 539a⟩
    let reg_ix ((s,_,_), i, _) as r) =
        try RM.find r (snd target.T.reg_ix_map)
        with Not_found -> imposs "Register not found in map" in
    let emit_dead_loc () = asm#value zero in
    let emit_link_const sym = asm#addr (Reloc.of_sym sym wordsize) in

```

```

let offset_w = wordsize - 1 in
let emit_offset bits = (* possibly an offset *)
  let value      = Bits.S.to_int bits in
  (try ignore (Bits.S.of_int value offset_w)
   with Bits.Overflow -> imposs "Offset from vfp is too large for run-time encoding");
  asm#value (B.Ops.or' (B.Ops.shl one (to_bits offset_w))
                    (B.Ops.shrl (B.Ops.shl (to_bits value) one) one)) in
let emit_int i = asm#value (Bits.U.of_int i wordsize) in
let emit_reg ix offset =
  let offset_w = wordsize - 2 in
  let w'       = offset_w / 2 in
  (try ignore (Bits.S.of_int ix w'); ignore(Bits.S.of_int offset w')
   with Bits.Overflow -> imposs "Reg ix or offset is too large for run-time encoding");
  asm#value (B.Ops.or' (B.Ops.shl one (to_bits offset_w))
                    (B.Ops.or' (B.Ops.shl (to_bits offset) (to_bits w'))
                               (to_bits ix))) in

⟨define functions for emitting data 538a⟩+≡ (536a) <537f 538b>
let emit_loc l =
  let mklink kind s w = Up.exp (RP.Const (RP.Link(s, kind, w))) in
  match Dn.loc (simplify_loc l) with
  | RP.Mem (_,_, RP.Const (RP.Link (sym,kind,_)),_) ->
    emit_link_const (sym, mklink kind)
  | RP.Mem (_,_, RP.Const (RP.Bits bs),_) -> emit_offset bs
  | RP.Reg r -> emit_reg (reg_ix r) 0
  | RP.Slice (_, i, RP.Reg r) -> emit_reg (reg_ix r) i
  | _ -> impossf "unexpected location for span data: %s" (RU.ToString.loc l) in
let emit_maybe_loc l = match l with
  | Some l -> emit_loc l
  | None -> emit_dead_loc () in

let nonredundant (reg, l) = match l with
  | Some r -> not (RU.Eq.loc (Dn.loc r) (RP.Reg reg))
  | None -> true in

let emit_csreg (reg, l) =
  if nonredundant (reg, l) then
    (emit_int (reg_ix reg); emit_maybe_loc l) in

⟨define functions for emitting data 538b⟩+≡ (536a) <538a 538c>
let size_cont_block conts =
  let n = List.length conts in
  let nvars = List.fold_left (fun rst (_,_,vars) -> List.length vars + rst) 0 conts in
  1 + 2 * nvars + 4 * n in

⟨define functions for emitting data 538c⟩+≡ (536a) <538b>
let emit_conts conts =
  let n = List.length conts in
  emit_int n;
  ignore (List.fold_left (fun offset (_,_,vs) -> emit_int offset;
                                offset + 2 * List.length vs + 3)
                    (n+1) conts);
let emit_cont_block (pc, sp, vars) =
  emit_int (List.length vars);
  asm#addr (Reloc.of_sym (asm#local pc, Rtl.datasym) wordsize);
  emit_loc sp;
  List.iter (fun (h,i,a) -> emit_int i; emit_int (Ctypes.enum_int h)) vars in
List.iter emit_cont_block conts in

```

```

⟨define simplify_exp to simplify rtl expressions 539a⟩≡ (537f)
let simplify_loc loc =
  let vfp = target.T.vfp in
  let check w = if w mod wordsize <> 0 || w < 0 then
    unimp (Printf.sprintf "unsupported size or alignment %d" w) in
  let is_vfp_offset e =
    let rec exp e = match e with
      | RP.Fetch (l,_) -> loc l
      | RP.App (_,exps) -> List.fold_left (fun v e -> v || exp e) false exps
      | RP.Const _ -> false
    and loc l =
      (* claude: this used to be stubbed out as "if false", because Vfp
       * then lived in layout/ (cmm_front_last), which depends on this
       * library - so it could not be called. That silently disabled the
       * flatten branch below which turns a vfp-relative address into a
       * plain frame offset, and span locations reached emit_loc still
       * shaped like bits32L[$V[0] + 4], which it rejects. The predicate
       * now lives in ir/vfp.ml and is reachable.
       *)
      if Vfp.is_vfp l
      then true
      else match l with
        | RP.Mem (_,_,e,_) -> exp e
        | RP.Slice (_,_,l) -> loc l
        | _ -> false in
    exp e in
  let rec flatten offset l = match l with
    | RP.Mem ((_,_,mcell as ms),_,e,ass) when is_vfp_offset e ->
      let w = Rtlutil.Width.exp (Up.exp e) in
      check offset; check w;
      (*V.eprintf verb "Adding an offset %d of width %d\n" offset w;*)
      let e' = Rtlutil.add w (R.app (R.opr "sub" [w]) [Up.exp e; vfp])
        (R.bits (Bits.S.of_int offset w) w) in
      R.mem (Up.assertion ass) ms (Cell.to_count mcell w) (Simplify.exp e')
    | RP.Mem (ms,w,e,ass) -> (* not a VFP offset *)
      R.mem (Up.assertion ass) ms w (Simplify.exp (Up.exp e))
    | RP.Reg r ->
      let w = Register.width r in
      check w; check offset;
      let l_up = Up.loc l in
      if offset <> 0 then
        R.slice w offset l_up
      else l_up
    | RP.Slice (_,i,l) -> flatten (offset + i) l
    | RP.Var _ | RP.Global _ -> Up.loc l in
  flatten 0 (Dn.loc loc) in

```

```

⟨runtimedata.ml 539b⟩+≡ (535a) <536a 539c>
let user_spans spans = (S.expose spans).S.users
let stackdata spans = (S.expose spans).S.sds

```

```

⟨runtimedata.ml 539c⟩+≡ (535a) <539b 539d>
let print_reg_map (PA.T tgt) =
  let (n, map) = tgt.T.reg_ix_map in
  Printf.printf "Target has %d registers:\n" n;
  RM.iter (fun ((s,_,_),i,_) n -> Printf.printf " %c[%d] -> %d\n" s i n) map

```

```

⟨runtimedata.ml 539d⟩+≡ (535a) <539c>
let emit_global_properties target (asm:'c Asm.assembler) =
  let cc = Call.get_cc target "C--" in

```

```

let growth = match cc.Call.stack_growth with Memalloc.Up -> 1 | Memalloc.Down -> -1 in
let wordsize = target.Target.wordsize in
let memsize = target.Target.memsize in
let sym = asm#export "Cmm_stack_growth" in
asm#section "data";
asm#align (wordsize / memsize);
asm#label sym;
asm#value (Bits.S.of_int growth wordsize)

```

R.26 front_ir/talloc.nw

```

⟨front_ir/talloc.ml 540a⟩≡
  ⟨talloc.ml 540e⟩

```

```

⟨front_ir/talloc.mli 540b⟩≡
  ⟨talloc.mli 540c⟩

```

R.27 Allocator for temporaries

```

⟨talloc.mli 540c⟩≡ (540b) 540d▷
  module Single : sig
    type t (* an allocator for one space -- a mutable type *)

    val for_space : Space.t -> t (* a fresh allocator *)
    val reg : t -> (*width*) int -> Register.t
    val loc : t -> (*width*) int -> Rtl.loc
  end

```

```

⟨talloc.mli 540d⟩+≡ (540b) <540c
  module Multiple : sig
    type t (* an allocator for multiple spaces -- a mutable type *)

    val for_spaces : Space.t list -> t (* a fresh allocator *)
    val reg : char -> t -> (*width*) int -> Register.t
    val loc : t -> char -> (*width*) int -> Rtl.loc
    val reg_like : t -> Register.t -> Register.t
  end

```

R.27.1 Implementation

```

⟨talloc.ml 540e⟩≡ (540a) 541c▷
  open Nopoly

  module S = Space
  ⟨auxiliaries(talloc.nw) 541b⟩
  module Single = struct
    type t = ((*width*)int -> Register.t) * ((*width*)int -> Rtl.loc)

    let for_space space =
      let next = ref 1 in
      let (_, _, cell) = space.S.space in
      let to_count = Cell.to_count cell in
      (fun width ->

```

```

    <use width to make k be the next index, updating next 541a>
    (space.S.space, k, to_count width)),
  (fun width ->
    <use width to make k be the next index, updating next 541a>
    Rtl.reg (space.S.space, k, to_count width))

  let _ = (for_space : Space.t -> t)

  let reg (reg, _) = reg
  let loc (_, loc) = loc
end

<use width to make k be the next index, updating next 541a>≡ (540)
  let () = if not (existsEq width space.S.widths) then
    Impossible.impossible ("Asked for temporary in space " ^
      space.S.doc ^ " with unsupported width " ^
      string_of_int width) in

  let Cell.C n = to_count width in
  let k = !next in
  let _ = next := k + n in

<auxiliaries(talloc.nw) 541b>≡ (540e) 542a▷
  let existsEq v =
    let rec exists = function
      | [] -> false
      | h :: t -> h = v || exists t
    in exists

<talloc.ml 541c>+≡ (540a) <540e
  module Multiple = struct
    let fail c =
      prerr_string ("Space " ^ Char.escaped c ^ " is not a temporary space\n");
      flush stderr;
      assert false

    let is_temp s = match s.S.classification with
      | S.Temp _ -> true
      | _ -> false

    type t = char -> Single.t

    let for_spaces spaces =
      List.fold_right
        (fun s rest ->
          if is_temp s then
            let a = Single.for_space s in
            let named_by (c', _, _) c = c <= c' in
            fun c -> if named_by s.S.space c then a else rest c
          else
            rest)
        spaces fail

    let _ = (for_spaces : Space.t list -> t)

    let reg c t = Single.reg (t c)
    let loc t c = Single.loc (t c)

    let reg_like (t:t) ((c, _, ms) as _space, _, ct) =
      Single.reg (t c) (Cell.to_width ms ct)
  end
end

```

$\langle \text{auxiliaries}(\text{talloc.nw}) \text{ 542a} \rangle \equiv$

$(\text{540e}) \triangleleft \text{541b}$

```
let divideby = function
| 32 -> fun n -> n / 32
| 8  -> fun n -> n / 8
| 64 -> fun n -> n / 64
| m  -> fun n -> n / m
```

R.28 front_ir/typecheck.nw

R.29 Type Checker for the Control-Flow Graph

$\langle \text{typecheck.mli 542b} \rangle \equiv$

```
val proc: Ast2ir.proc -> unit
```

R.29.1 Implementation

$\langle \text{typecheck.ml 542c} \rangle \equiv$

```
let rtl =
  fun rtl ->
    try
      Rtldebug.typecheck rtl
    with
      Rtldebug.TypeCheck r ->
        Printf.eprintf "Ill-typed RTL %s\n" (Rtlutil.ToUnreadableString.rtl rtl)

let proc (cfg, p) = Zipcfg.iter_rtls rtl cfg
```

Appendix S

asm

S.1 asm/astasm.nw

$\langle \text{assembler/astasm.ml } 543a \rangle \equiv$
 $\langle \text{astasm.ml } 543f \rangle$

$\langle \text{assembler/astasm.mli } 543b \rangle \equiv$
 $\langle \text{astasm.mli } 543e \rangle$

S.2 C-- Assembler

$\langle \text{PERSONALITY } 543c \rangle \equiv$ (543)
module type PERSONALITY = sig
 val wordsize: int
 val pointersize: int
 val memsize: int
 val byteorder: Rtl.aggregation
 val float: string
 val charset: string
 type proc
 val cfg2ast : proc -> Ast.proc
end

$\langle S(\text{astasm.nw}) 543d \rangle \equiv$ (543)
module type S = sig
 type proc
 val asm: out_channel -> proc Asm.assembler
end

$\langle \text{astasm.mli } 543e \rangle \equiv$ (543b)
 $\langle \text{PERSONALITY } 543c \rangle$
 $\langle S(\text{astasm.nw}) 543d \rangle$
module Make(P: PERSONALITY): S with type proc = P.proc

S.2.1 Implementation

$\langle \text{astasm.ml } 543f \rangle \equiv$ (543a)
module T = Target
module A = Ast
module Asm = Asm

 $\langle \text{PERSONALITY } 543c \rangle$


```

⟨S(astasm.nw) 543d⟩
module Make (P: PERSONALITY): S with type proc = P.proc = struct
  type proc = P.proc
  ⟨Make(astasm.nw) 544a⟩
end

⟨Make(astasm.nw) 544a⟩≡ (543f) 544b▷
let pointer      = A.BitsTy(P.pointersize)
let bits n       = A.BitsTy n
let int n        = A.Sint( string_of_int n, Some (bits P.wordsize))
let one          = int 1 (* wordsize *)
let zero         = A.Sint( "0" , Some (bits P.memsize)) (* memsize *)

exception Unsupported of string
let unsupported msg = raise (Unsupported msg)
type reloc = Reloc.t

⟨Make(astasm.nw) 544b⟩+≡ (543f) <544a 544c>
let spec =
  let reserved =
    [ "aborts"; "align"; "aligned"; "also"; "as"; "big"; "byteorder";
      "case"; "const"; "continuation"; "cut"; "cuts"; "else"; "equal";
      "export"; "foreign"; "goto"; "if"; "import"; "invariant"; "jump";
      "little"; "memsize"; "pragma"; "register"; "return"; "returns";
      "section"; "semi"; "span"; "stackdata"; "switch"; "target"; "targets";
      "to"; "typedef"; "unicode"; "unwinds"; "float"; "charset";
      "pointersize"; "wordsize"]
  in
  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '.'
    | '_'
    | '$'
    | '@'      -> true
    | _        -> false in
  let replace = function
    | x when id x -> x
    | _          -> '@'
  in
  { Mangle.preprocess = (fun x -> "sym:"^x)
    ; Mangle.replace   = replace
    ; Mangle.reserved  = reserved
    ; Mangle.avoid     = (fun x -> x ^ "$")
  }

⟨Make(astasm.nw) 544c⟩+≡ (543f) <544b 544d>
let reladdr (a:reloc) =
  let const b = Rtlutil.ToAST.expr (Rtl.bits b (Bits.width b)) in
  let sym (s,_) = A.Fetch (A.Name(None,s#mangled_text,None)) in
  let binop op l r = A.BinOp(l, op, r) in
  Reloc.fold ~const ~sym ~add:(binop "+") ~sub:(binop "-") a

⟨Make(astasm.nw) 544d⟩+≡ (543f) <544c 547a>
type init = unit

class asm (fd:out_channel): [proc] Asm.assembler =
object (this)
  val      _fd      = fd

```

```

val mutable _section = "this can't happen"
val mutable _exported = Strutil.Set.empty
val mutable _imported = Strutil.Set.empty
val mutable _toplevel = ([]: (string * A.section list) list) (* rev'ed *)
val mutable _actions = ([]: A.section list) (* reversed *)
val _mangle = Mangle.mk spec

method globals n = () (* probably could do better here... *)

(* We put every top-level thingie into its own section. This helps
the pretty printer. For John Dias *)
method private append (a:A.section) =
  _toplevel <- (_section, [a]) :: _toplevel

(* declare symbols *)
method import s =
  ( _imported <- Strutil.Set.add s _imported
  ; Symbol.with_mangler _mangle s
  )

method export s =
  ( _exported <- Strutil.Set.add s _exported
  ; Symbol.with_mangler _mangle s
  )

method local s =
  Symbol.with_mangler _mangle s

(* sections *)
(* section closes the current section and adds it to topLevel. It is
dropped in case it is empty. *)
method section s =
  ( match _actions with [] -> ()
  | _ :: _ -> _toplevel <- (_section, List.rev _actions) :: _toplevel
  )
  ; _section <- s
  ; _actions <- []

method current =
  _section

(* define symbols *)
method label (s: Symbol.t) =
  this#append (A.Datum(A.Label s#mangled_text))

method const (s: Symbol.t) (b:Bits.bits) =
  let i = Printf.sprintf "0x%Lx" (Bits.U.to_int64 b) in
  let bits = A.Uint(i,Some (bits (Bits.width b))) in
  this#append (A.Decl(A.Const(None,s#mangled_text,bits)))

(* set location counter *)
method org n =
  unsupported "no location counter in this implementation"

method align n =
  this#append (A.Datum(A.Align n))

method addloc n =
  let memsize = P.memsize in
  let ty = bits memsize in

```

```

    let size      = int n                      in
        this#append (A.Datum(A.MemDecl(ty,A.FixSize(size),None)))

method longjmp_size () =
    Impossible.unimp "longjmp size not set for AST -- needed for alternate returns"

(* instructions *)
method cfg_instr (proc:proc) =
    this#append (A.Procedure(P.cfg2ast proc))

(* emit data *)
method zeroes (n:int) =
    let ty      = bits P.memsize in
    let size    = int n              in
    let rec z   = function
        | 0 -> []
        | n -> zero :: z (n-1)      in
    let _init n = Some(A.InitExprs(z n)) in
    let init n  = None in (* fix for John's problem with SPEC benchmarks *)
        if n > 0 then
            this#append (A.Datum(A.MemDecl(ty,A.FixSize(size),init n)))
        else
            ()

method value (v:Bits.bits) =
    let ty  = bits (Bits.width v)          in
    let i   = Printf.sprintf "0x%Lx" (Bits.U.to_int64 v) in
    let init = A.InitExprs([A.Uint(i, Some ty)]) in
        this#append (A.Datum(A.MemDecl(ty, A.FixSize one, Some init)))

method addr (a: reloc) =
    let ty  = bits (Reloc.width a) in
    let init = A.InitExprs([reladdr a]) in
        this#append (A.Datum(A.MemDecl(ty, A.FixSize one, Some init)))

(* the AST has comments only at the statement level. However, we are
outside procedures here and thus cannot issue a comment. Should this
method call unsupported()? *)

method comment s = ()

method private imports =
    match Strutil.Set.elements _imported with
    | []      -> None
    | names -> Some (A.Import( Some (A.BitsTy P.pointersize)
                               , List.map (fun n -> None, n) names
                               ))

method private exports =
    match Strutil.Set.elements _exported with
    | []      -> None
    | names -> Some (A.Export( Some pointer
                               , List.map (fun n -> n, None) names
                               ))

(* Advertise pointer sizes and such for this assembler *)

method private personality =

```

```

[ A.Memsize P.memsize
; ( match P.byteorder with
  | Rtl.BigEndian    -> A.ByteorderBig
  | Rtl.LittleEndian -> A.ByteorderLittle
  | _                -> assert false
)
; A.PointerSize P.pointersize
; A.WordSize    P.wordsize
; A.Charset     P.charset
; A.FloatRepr   P.float
]

```

(* emit takes the declarations in toplevel and completes them to a C-- program. Imports and exports are announced according to the names registered in `_imported`, `_exported` *)

```

method emit =
  let target    = A.TopDecl(A.Target this#personality) in
  let section   = (_section, List.rev _actions)       in
  let toplevel  = List.rev (section :: _toplevel) in
  let sections  = List.map (fun (name,sect) -> A.Section(name,sect))
                        toplevel in
  let ast       = match this#imports, this#exports with
    | None , None   -> target :: sections
    | Some i, None   -> target :: A.TopDecl(i) :: sections
    | None , Some e  -> target :: A.TopDecl(e) :: sections
    | Some i, Some e -> target :: A.TopDecl(i) :: A.TopDecl(e)
                        :: sections in
  Astpp.emit _fd 72 ast
end

```

S.3 Assembler Constructor Function

<Make(astasm.nw) 547a> $\equiv$ (543f) <544d
`let asm fd = new asm fd`

S.4 assembler/cfgutil.nw

<assembler/cfgutil.ml 547b> $\equiv$
<cfgutil.ml 548a>

<assembler/cfgutil.mli 547c> $\equiv$
<cfgutil.mli 547d>

S.5 Control-Flow Graph Utilities

<cfgutil.mli 547d> $\equiv$ (547c)
`val cfg2dot : compress:bool -> live:bool -> name:string -> Zipcfg.graph -> string`
`val cfg2ast : (Rtl.rtl -> Ast.stmt) -> Zipcfg.graph -> name:string -> Ast.proc`
`val emit : Ast2ir.basic_proc -> Zipcfg.graph ->`
`(Zipcfg.Rep.call -> unit) -> (Rtl.rtl -> unit) ->`
`(string -> unit) -> unit`
`val block_name : Zipcfg.graph -> Zipcfg.uid -> string`
`val print_cfg: Zipcfg.graph -> unit`
`val pr_first : Zipcfg.Rep.first -> unit`

```

val pr_mid   : Zipcfg.Rep.middle -> unit
val pr_last  : Zipcfg.graph -> Zipcfg.Rep.last -> unit
val pr_last' : Zipcfg.Rep.last -> unit

type node = F of Zipcfg.Rep.first | M of Zipcfg.Rep.middle | L of Zipcfg.Rep.last
val numbered_layout_nodes : Zipcfg.graph -> (int * node) list
(* val delete : Zipcfg.zgraph -> Zipcfg.graph --- delete focus *)

```

S.5.1 Implementation

<cfgutil.ml 548a>≡ (547b) 548b▷

```

module A = Ast
module DG = Dag
module G = Zipcfg
module GR = Zipcfg.Rep
module OG = Cfgx.M
module P = Proc
module PA = Prest2ir
module T = Target
module UM = Unique.Map

```

<cfgutil.ml 548b>+≡ (547b) <548a 548c>

```

let block_name g uid =
  try
    (match UM.find uid (G.to_blocks g) with
     | GR.Entry, _ -> "<entry>"
     | GR.Label ((_, l), _, _) -> l)
  with Not_found -> "<unknown-uid>"

```

<cfgutil.ml 548c>+≡ (547b) <548b 549a>

```

let pr = Printf.eprintf
let pr_first = function
  | GR.Entry -> pr "<entry>\n"
  | GR.Label ((_, l), _, _) -> pr "%s:\n" l
let pr_mid m = match m with
  | GR.Instruction r -> pr "%s\n" (Rtlutil.ToString.rtl r)
  | GR.Stack_adjust r -> pr "%s\n" (Rtlutil.ToString.rtl r)
let lasttype = function
  | GR.Exit -> "exit"
  | GR.Branch _ -> "branch"
  | GR.Cbranch _ -> "cbranch"
  | GR.Mbranch _ -> "mbranch"
  | GR.Call _ -> "call"
  | GR.Cut _ -> "cut"
  | GR.Return _ -> "return"
  | GR.Jump _ -> "jump"
  | GR.Forbidden _ -> "forbidden"
let pr_last g l =
  let succ_names l = String.concat " " (List.map (block_name g) (GR.succs l)) in
  pr "<%s> %s [%s]\n" (lasttype l) (Rtlutil.ToString.rtl (GR.last_instr l))
  (succ_names l)
let pr_last' l =
  pr "<%s> %s\n" (lasttype l) (Rtlutil.ToString.rtl (GR.last_instr l))
let print_block g (first, tail) =
  pr_first first;
  let rec pr_tail = function
    | GR.Tail (m, tail) ->
      pr_mid m; pr_tail tail
    | GR.Last GR.Exit -> pr "<exit>\n"

```

```

    | GR.Last l -> pr_last g l in
pr_tail tail

let print_cfg g =
  let () = Printf.eprintf "\nBEGIN CFG\n" in
  let () =
    try List.iter (print_block g) (G.postorder_dfs g)
    with Not_found -> Unique.Map.iter (print_block g) (G.to_blocks g) in
  let () = Printf.eprintf "END CFG\n" in
  flush stderr

⟨cfgutil.ml 549a⟩+≡ (547b) <548c 549b>
let matches next = match next with
| None -> (fun _ -> false)
| Some (u, l) -> (fun (u', l') -> Unique.eq u u')

let emit_proc cfg call_emit rtl_emit sym_emit =
  let PA.T tgt = proc.Procedure.target in
  let rec seq_emit = function (* emit sequence of RTLs only *)
    | DG.Rtl r -> rtl_emit r
    | DG.Seq (a, b) -> seq_emit a; seq_emit b
    | DG.Nop -> ()
    | DG.If _ | DG.While _ -> Impossible.impossible "expected linear code sequence" in
  let block b next () =
    let first f () = match f with
    | GR.Entry -> ()
    | GR.Label (_, l), _, _ -> sym_emit l in
    let middle m () =
      if GR.is_executable m then
        rtl_emit (GR.mid_instr m) in
    let last l () = match l with
    | GR.Branch (_, tgt) when matches next tgt -> ()
    | GR.Cbranch (_, ttgt, ftgt) when not (matches next ftgt) ->
      rtl_emit (GR.last_instr l);
      let (b, r) = tgt.T.machine.T.goto.T.embed proc (proc.exp_of_lbl ftgt) in
      seq_emit b;
      rtl_emit r
    | GR.Call c -> call_emit c
    | _ -> rtl_emit (GR.last_instr l) in
    GR.fold_fwd_block first middle last b () in
  G.fold_layout block () cfg

⟨cfgutil.ml 549b⟩+≡ (547b) <549a 550>
let extend_with preds nodes =
  let rec add_predecessors visited candidates = match candidates with
  | [] -> visited
  | node :: candidates ->
    if List.exists (OG.eq node) visited then
      add_predecessors visited candidates
    else
      add_predecessors (node :: visited) (preds node @ candidates) in
  add_predecessors nodes (List.flatten (List.map preds nodes))

type node = F of GR.first | M of GR.middle | L of GR.last

let numbered_layout_nodes g =
  let add_block (f, t) next (i, nodes') =
    let (i, nodes') = (i+1, (i, F f) :: nodes') in
    let is_next (u', _) = match next with
    | Some (u, _) -> Unique.eq u u'

```

```

| None -> false in
let rec tail t (i, nodes') = match t with
| GR.Tail (m, t) -> tail t (i+1, (i, M m) :: nodes')
| GR.Last (GR.Branch (_, lbl)) when is_next lbl -> (i, nodes')
| GR.Last l -> (i+1, (i, L l) :: nodes') in
tail t (i, nodes') in
let _, nodes' = G.fold_layout add_block (0, []) g in
List.rev nodes'

let id_matches u = function
| F GR.Entry -> Unique.eq u GR.entry_uid
| F (GR.Label ((u', _), _, _)) -> Unique.eq u u'
| L _ | M _ -> false

let cfg2dot ~compress ~live ~name g =
let () = if live then assert false in
let nodes = numbered_layout_nodes g in
let number u = fst (List.find (fun (i, n) -> id_matches u n) nodes) in
let spr = Printf.sprintf in
let number_and_rtl (n, node) =
let instr rtl = spr "N%d: %s" n (Rtlutil.ToString.rtl rtl) in
match node with
| F GR.Entry -> "Entry"
| F (GR.Label ((_, l), _, _)) -> spr "N%d: %s" n l
| M m -> instr (GR.mid_instr m)
| L GR.Exit -> "Exit"
| L l -> instr (GR.last_instr l) in
let dotnode node = spr "N%d [label=%S]" (fst node) (number_and_rtl node) in
let edge_fwd from to' =
let edge = spr "N%d -> N%d" from to' in
if live then
(* edge ^ "[label=\"" ^ Register.SetX.to_string (L.live_in to') ^ "\"]\n" *)
assert false
else
edge ^ "\n" in
let _edge_bwd from to' = spr "N%d -> N%d [dir=back,style=dotted]\n" to' from in
let edges_leaving node tail = match snd node with
| F _ | M _ -> edge_fwd (fst node) (fst node + 1) :: tail
| L last ->
let add_edge dir edges u = dir (fst node) (number u) :: edges in
let tail = List.fold_left (add_edge edge_fwd) tail (GR.succs last) in
tail in
let oldv = Rtlutil.ToAST.verbosity Rtlutil.ToAST.Low in
let openg = "digraph " ^ name ^ " {\n" in
let closeg = "}\n" in
let pagesize = " page=\"8,10.5\"\n" in
let compress = if compress then " ratio=compress\n" else "" in
let body = List.fold_right (fun n t -> edges_leaving n t) nodes [closeg] in
let body = List.fold_right (fun n t -> dotnode n :: "\n" :: t) nodes body in
let _ = Rtlutil.ToAST.verbosity oldv in
String.concat "" (openg :: pagesize :: compress :: body)

```

<cfgutil.ml 550>+≡

(547b) <549b

```

let cfg2ast instr g ~name =
let block b next prev' =
let mid n prev' = instr (GR.mid_instr n) :: prev' in
let first n prev' = match n with
| GR.Label (l,_,_) -> A.LabelStmt (snd l) :: prev'
| GR.Entry -> prev' in
let last n prev' = match n with

```

```

| GR.Exit -> A.CommentStmt "exit node!" :: prev'
| GR.Branch (_, lbl) when matches next lbl -> prev'
| GR.Forbidden _ -> A.CommentStmt "forbidden to reach this point" :: prev'
| _ -> instr (GR.last_instr n) :: prev' in
let rec tail t prev' = match t with
| GR.Last l -> last l prev'
| GR.Tail (m, t) -> tail t (mid m prev') in
let (f, t) = b in
tail t (first f prev') in
let oldv = Rtlutil.ToAST.verbosity Rtlutil.ToAST.Low in
let stmts' = G.fold_layout block [] g in
let _ = Rtlutil.ToAST.verbosity oldv in
( None
, name
, []
, List.rev_map (fun s -> Ast.StmtBody s) stmts'
, Srcmap.null
)

```

S.6 assembler/dotasm.nw

⟨assembler/dotasm.ml 551a⟩≡
 ⟨dotasm.ml 551d⟩

⟨assembler/dotasm.mli 551b⟩≡
 ⟨dotasm.mli 551c⟩

S.7 Assembler Interface to DOT

⟨dotasm.mli 551c⟩≡ (551b)
 val asm : compress:bool -> live:bool -> out_channel -> Ast2ir.proc Asm.assembler

S.7.1 Implementation

⟨dotasm.ml 551d⟩≡ (551a)
 module Asm = Asm

```

exception Unsupported of string
let unsupported msg = raise (Unsupported msg)

```

⟨Make(dotasm.nw) 551e⟩
 let asm ~compress ~live fd = new asm (Cfgutil.cfg2dot ~compress ~live) fd

⟨Make(dotasm.nw) 551e⟩≡ (551d) 552▷
 let spec =
 let reserved = [] in
 let id = function
 | 'a'..'z'
 | '0'..'9'
 | 'A'..'Z'
 | '_' -> true
 | _ -> false in
 let replace = function
 | x when id x -> x
 | _ -> '_'


```

in
{ Mangle.preprocess = (fun x -> x)
; Mangle.replace    = replace
; Mangle.reserved    = reserved
; Mangle.avoid       = (fun x -> x ^ "_")
}

⟨Make(dotasm.nw) 552⟩+≡ (551d) <551e
class ['proc] asm cfg2dot (fd:out_channel) : ['proc] Asm.assembler =
object (this)
  val mutable _section = "this can't happen"

  (* declarations *)
  method import s = Symbol.unmangled s
  method export s = Symbol.unmangled s
  method local  s = Symbol.unmangled s

  method globals n = ()

  (* sections *)
  method section s =
    print_string "pad: Dotasm.section\n";
    _section <- s
  method current  =
    print_string "pad: Dotasm.current\n";
    _section

  (* definitions *)
  method label s  = ()
  method const s b = ()

  (* locations *)

  method org n    = ()
  method align n  = ()
  method addloc n = ()

  method longjmp_size () =
    print_string "pad: Dotasm.XXX\n";
    Impossible.unimp "longjmp size not set for dot -- needed for alternate returns"

  (* instructions *)
  method cfg_instr (proc : 'proc) =
    print_string "pad: Dotasm.cfg_instr\n";
    let (cfg, proc) = proc in
    let s    = proc.Procedure.symbol in
    let mangle = Mangle.mk spec in
    output_string fd (cfg2dot ~name:(mangle s#mangled_text) cfg)

  method zeroes n = ()
  method value v = ()
  method addr  a = ()
  method comment s = ()
  method emit = ()
end

```

S.8 assembler/dummyasm.nw

```
<assembler/dummyasm.ml 553a>≡  
  <dummyasm.ml 553d>
```

```
<assembler/dummyasm.mli 553b>≡  
  <dummyasm.mli 553c>
```

S.9 Dummy Assembler

```
<dummyasm.mli 553c>≡ (553b)  
  val asm : unit Asm.assembler
```

S.9.1 Implementation

```
<dummyasm.ml 553d>≡ (553a)  
  module Asm = Asm
```

```
  let debug s = prerr_string ("Dummyasm." ^ s ^ "\n")
```

```
  <Make(dummyasm.nw) 553e>  
  let asm = new asm ()
```

```
<Make(dummyasm.nw) 553e>≡ (553d)  
  class ['proc] asm () : ['proc] Asm.assembler =  
  object (this)
```

```
    (* declarations *)  
    method import s =  
      debug "import";  
      Symbol.unmangled s  
    method export s =  
      debug "export";  
      Symbol.unmangled s  
    method local s =  
      debug "local";  
      Symbol.unmangled s
```

```
    method globals n =  
      debug "globals";  
      ()
```

```
    (* sections *)  
    method section s =  
      debug "section";  
      ()
```

```
    method current =  
      debug "current";  
      "TODO"
```

```
    (* definitions *)  
    method label s =  
      debug "label";  
      ()  
    method const s b =  
      debug "const";
```

```

()

(* locations *)

method org n =
  debug "org";
  ()
method align n =
  debug "align";
  ()
method addloc n =
  debug "addloc";
  ()

method longjmp_size () =
  debug "longjmp_size";
  failwith "longjmp_size"

(* instructions *)
method cfg_instr (proc : 'proc) =
  debug "cfg_instr"

method zeroes n =
  debug "zeroes";
  ()
method value v =
  debug "value";
  ()
method addr a =
  debug "addr";
  ()
method comment s =
  debug "comment";
  ()
method emit =
  debug "emit";
  ()
end

```

S.10 assembler/mangle.nw

`<assembler/mangle.ml 554a>`≡
`<mangle.ml 555c>`

`<assembler/mangle.mli 554b>`≡
`<mangle.mli 554c>`

S.11 Name Mangling

`<mangle.mli 554c>`≡ `(554b) 555a▷`
`type t = string -> string`

```

⟨mangle.mli 555a⟩+≡ (554b) <554c 555b>
  type spec = { preprocess: string -> string
                ; replace:   char -> char
                ; reserved:  string list
                ; avoid:     string -> string
                }

⟨mangle.mli 555b⟩+≡ (554b) <555a
  val mk:      spec -> t      (* create a mangler *)

```

S.11.1 Implementation

```

⟨mangle.ml 555c⟩≡ (554a) 555d>
  type strset      = Strutil.Set.t
  type strmap      = (string,string) Hashtbl.t

  type t           = string -> string
  type spec        = { preprocess: string -> string
                      ; replace:   char -> char
                      ; reserved:  string list
                      ; avoid:     string -> string
                      }

⟨mangle.ml 555d⟩+≡ (554a) <555c 555e>
  type state      = { mutable used: strset
                    ; map:      strmap (* is also mutable *)
                    }

⟨mangle.ml 555e⟩+≡ (554a) <555d
  let mk spec =
    let state = { used = Strutil.Set.empty
                  ; map = Hashtbl.create 997 (* initial size *)
                  }
    in

    let rec avoid s =
      if Strutil.Set.mem s state.used
      then avoid (spec.avoid s)
      else s
    in

    let newMangle s =
      let s' = Bytes.of_string (spec.preprocess s) in
      let _ = for i = 0 to (Bytes.length s') - 1 do
        Bytes.set s' i (spec.replace (Bytes.get s' i))
      done in
      let s' = avoid (Bytes.to_string s') in
      ( state.used <- Strutil.Set.add s' state.used
        ; Hashtbl.add state.map s s'
        ; s'
        )
    in
    let mangle s =
      try Hashtbl.find state.map s
      with Not_found -> newMangle s
    in
    mangle

```

S.11.2 Example

```
<mangle.ml unused 556>≡
let simple =
  let reserved =
    [ "aborts"; "align"; "aligned"; "also"; "as"; "big"; "byteorder";
      "case"; "const"; "continuation"; "cut"; "cuts"; "else"; "equal";
      "export"; "foreign"; "goto"; "if"; "import"; "invariant"; "jump";
      "little"; "memsize"; "pragma"; "register"; "return"; "returns";
      "section"; "semi"; "span"; "stackdata"; "switch"; "target"; "targets";
      "to"; "typedef"; "unicode"; "unwinds"; "float"; "charset";
      "pointersize"; "wordsize"]
  in
  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '.'
    | '_'
    | '$'
    | '@'      -> true
    | _        -> false in
  let replace = function
    | x when id x -> x
    | _              -> '@' in
  { preprocess = (fun x -> x)
    ; replace   = replace
    ; reserved  = reserved
    ; avoid     = (fun x -> x ^ "$")
  }
```

Appendix T

front_last

T.1 front_last/automatonutil.nw

$\langle \text{front_last/automatonutil.ml } 557a \rangle \equiv$
 $\langle \text{automatonutil.ml } 557d \rangle$

$\langle \text{front_last/automatonutil.mli } 557b \rangle \equiv$
 $\langle \text{automatonutil.mli } 557c \rangle$

T.2 Automaton utilities

$\langle \text{automatonutil.mli } 557c \rangle \equiv$ (557b)
val alocs : Automaton.loc -> Rtl.width -> Rtl.loc list
val alloc : Automaton.loc -> Rtl.width -> Rtl.loc

T.2.1 Implementation

$\langle \text{automatonutil.ml } 557d \rangle \equiv$ (557a) 557e▷
module R = Rtl
module RP = Rtl.Private
module Up = Rtl.Up
module Dn = Rtl.Dn

$\langle \text{automatonutil.ml } 557e \rangle + \equiv$ (557a) ◁557d
let alocs alloc w =
 let RP.Rtl gs = Dn.rtl (Automaton.store alloc (Rtl.late "dummy" w) w) in
 let getloc = function _, RP.Store (l, _, _) -> Up.loc l | _, RP.Kill l -> Up.loc l in
 List.map getloc gs
let alloc a w = match alocs a w with
| [l] -> l
| _ -> Impossible.impossible "automaton split value across multiple locations"

T.3 front_last/callspec.nw

$\langle \text{front_last/callspec.ml } 557f \rangle \equiv$
 $\langle \text{callspec.ml } 559a \rangle$

$\langle \text{front_last/callspec.mli } 557g \rangle \equiv$
 $\langle \text{callspec.mli } 558a \rangle$

T.4 Constructing a Calling Convention

```

<callspec.mli 558a>≡ (557g) 558b>
  val overflow      : dealloc:Call.party -> alloc:Call.party -> Call.overflow
  val c_overflow    : Call.overflow
  val tail_overflow : Call.overflow

<callspec.mli 558b>+≡ (557g) <558a 558c>
  type nvr_saver = Talloc.Multiple.t -> Register.t -> Rtl.loc
  (* suggested by NR, but not used
   * val save_nvrs: Space.t list -> nvr_saver
   *)

<callspec.mli 558c>+≡ (557g) <558b 558e>
  module ReturnAddress: sig
    type style =
      | KeepInPlace      (* leave RA where it is      *)
      | SaveToTemp of char (* save RA in a temporary of this space *)
    (* <<suggested functions>> *)
  end

<suggested functions 558d>≡
  val enter_in_loc      : Rtl.loc -> Block.t -> Rtl.exp
  val save_in_temp      : Rtl.exp -> Talloc.Multiple.t -> Rtl.loc
  val save_as_is        : Rtl.exp -> Talloc.Multiple.t -> Rtl.loc
  val exit_in_temp      : Rtl.exp -> Block.t -> Talloc.Multiple.t -> Rtl.loc
  val exit_as_is        : Rtl.exp -> Block.t -> Talloc.Multiple.t -> Rtl.loc

<callspec.mli 558e>+≡ (557g) <558c 558g>
  <type t(callspec.nw) 558f>
  type t =
    { name          : string          (* name this CC *)
    ; stack_growth  : Memalloc.growth (* up or down *)
    ; overflow      : Call.overflow   (* overflow handling *)
    ; memspace      : Register.space
    ; sp            : Register.t      (* stack pointer register *)
    ; sp_align      : int             (* alignment of sp *)
    ; all_regs      : Register.Set.t  (* regs visible to allocator *)
    ; nv_regs       : Register.Set.t  (* preserved registers *)
    ; save_nvr      : nvr_saver       (* how to save registers *)
    ; ra            : Rtl.loc         (* where is RA, how to treat it *)
    }
    * ReturnAddress.style

<callspec.mli 558g>+≡ (557g) <558e
  val to_call: cutto:(unit, Mflow.cut_args) Target.map ->
    return:(int -> int -> ra:Rtl.exp -> Rtl.rtl) ->
    Automaton.cc_spec ->
    t -> Call.t

```

T.4.1 Implementation

```
<callspec.ml 559a>≡ (557f) 559b▷
module R = Rtl
module RU = Rtlutil
module W = Rtlutil.Width
module T = Talloc.Multiple
module C = Call
module A = Automaton

let overflow ~dealloc ~alloc =
  { C.parameter_deallocator = dealloc
  ; C.result_allocator      = alloc
  }

let tail_overflow =
  { C.parameter_deallocator = C.Callee
  ; C.result_allocator      = C.Caller
  }

let c_overflow =
  { C.parameter_deallocator = C.Caller
  ; C.result_allocator      = C.Caller
  }

type nvr_saver = Talloc.Multiple.t -> Register.t -> Rtl.loc

module ReturnAddress = struct
  type style =
    | KeepInPlace (* leave RA where it is *)
    | SaveToTemp of char (* save RA in a temporary *)
end

<type t(callspec.nw) 558f>

<callspec.ml 559b>+≡ (557f) <559a 559c>
let old_end wordsize growth block = match growth with
| Memalloc.Down -> RU.addk wordsize (Block.base block) (Block.size block)
| Memalloc.Up   -> Block.base block

let young_end wordsize growth block = match growth with
| Memalloc.Down -> Block.base block
| Memalloc.Up   -> RU.addk wordsize (Block.base block) (Block.size block)

let wordsize t = match t.sp with ((_,_,ms),_,c) -> Cell.to_width ms c
let memspace t = t.memspace
let byteorder t = let (_, b, _) = t.memspace in b
let vfp wordsize = Vfp.mk wordsize
let std_sp_location wordsize =
  RU.add wordsize (vfp wordsize) (R.late "minus frame size" wordsize)

let mk_automaton wordsize memspace block_name automaton = fun () ->
  Block.srelative (vfp wordsize) block_name (A.at memspace)
  automaton

<callspec.ml 559c>+≡ (557f) <559b 560a>
let ra t = fst t.ra
let style t = snd t.ra
```



```

<callspec.ml 560a>+≡ (557f) <559c 560b>
  let prolog t auto =
    let autosp = (fun a -> young_end (wordsize t) t.stack_growth a.A.overflow) in
    C.incoming ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(mk_automaton (wordsize t) (memspace t) "in call parms" auto.A.call)

    (* ~autosp:(fun _ -> vfp (wordsize t)) *)
    ~autosp:autosp
    ~postsp:(fun _ _ -> std_sp_location (wordsize t))
    ~insp:(fun a _ _ -> autosp a)

<callspec.ml 560b>+≡ (557f) <560a 560c>
  let epilog t auto =
    C.outgoing ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(mk_automaton (wordsize t) (memspace t)
        "out ovfl results" auto.A.results)
    ~autosp:(fun r -> std_sp_location (wordsize t))
    ~postsp:(match t.overflow with
      | {C.parameter_deallocator=C.Caller; C.result_allocator=C.Caller} ->
        (fun _ _ -> vfp (wordsize t))
      | {C.parameter_deallocator=C.Callee; C.result_allocator=C.Caller}
      | {C.parameter_deallocator=C.Caller; C.result_allocator=C.Callee}
      | {C.parameter_deallocator=C.Callee; C.result_allocator=C.Callee} ->
        (fun a _ ->
          young_end (wordsize t) t.stack_growth a.A.overflow)
    )

<callspec.ml 560c>+≡ (557f) <560b 560d>
  let call_actuals t auto =
    C.outgoing ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(mk_automaton (wordsize t) (memspace t) "out call parms" auto.A.call)
    ~autosp:(fun r -> std_sp_location (wordsize t))
    ~postsp:(fun a sp -> young_end (wordsize t) t.stack_growth a.A.overflow)

<callspec.ml 560d>+≡ (557f) <560c 560e>
  let call_results t auto =
    let autosp = (fun a -> young_end (wordsize t) t.stack_growth a.A.overflow) in
    C.incoming ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(mk_automaton (wordsize t) (memspace t) "in ovfl results" auto.A.results)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location (wordsize t))
    ~insp:(fun a _ _ -> autosp a)

<callspec.ml 560e>+≡ (557f) <560d 560f>
  let also_cuts_to t auto =
    let autosp = (fun r -> std_sp_location (wordsize t)) in
    C.incoming ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(mk_automaton (wordsize t) (memspace t) "in cont parms" auto.A.cutto)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location (wordsize t))
    ~insp:(fun a _ _ -> autosp a)

  let cut_actuals t auto base =
    C.outgoing ~growth:t.stack_growth ~sp:(R.reg t.sp)
      ~mkauto:(fun () -> A.at (memspace t) ~start:base auto.A.cutto)
    ~autosp:(fun r -> R.fetch (R.reg t.sp) (wordsize t))
    ~postsp:(fun _ _ -> R.fetch (R.reg t.sp) (wordsize t))

<callspec.ml 560f>+≡ (557f) <560e 561a>
  let ra_on_entry t block = R.fetch (ra t) (wordsize t)

```

```

⟨callspec.ml 561a⟩+≡ (557f) <560f
let ra_on_exit t saved_ra block temp = match style t with
| ReturnAddress.KeepInPlace -> saved_ra
| ReturnAddress.SaveToTemp s -> Talloc.Multiple.loc temp s (wordsize t)

let where_to_save_ra t = fun ra_on_entry temp -> match style t with
| ReturnAddress.KeepInPlace -> ra t
| ReturnAddress.SaveToTemp s -> Talloc.Multiple.loc temp s (wordsize t)

let to_call ~cutto ~return auto t =
  ⟨it's impossible for the stack pointer to be a useful register too 561b⟩
  ;
  let volregs = Register.Set.diff t.all_regs t.nv_regs in
  { C.name          = t.name
  ; C.overflow_alloc = t.overflow
  ; C.call_parms    = {C.in'=prolog      t auto; C.out=call_actuals t auto}
  ; C.cut_parms     = {C.in'=also_cuts_to t auto; C.out=cut_actuals  t auto}
  ; C.results       = {C.in'=call_results t auto; C.out=epilog      t auto}

  ; C.stack_growth  = t.stack_growth
  ; C.stable_sp_loc  = std_sp_location (wordsize t)
  ; C.jump_tgt_reg   = Rtl.reg (Register.Set.choose volregs)
  ; C.replace_vfp    = Vfp.replace_with ~sp:(R.reg t.sp)
  ; C.sp_align       = t.sp_align
  ; C.ra_on_entry    = ra_on_entry t
  ; C.where_to_save_ra = where_to_save_ra t
  ; C.ra_on_exit     = ra_on_exit t
  ; C.sp_on_unwind   = (fun _ -> Rtl.null)
  ; C.sp_on_jump     = (fun _ _ -> Rtl.null)
  ; C.pre_nvregs     = t.nv_regs
  ; C.volregs        = volregs
  ; C.saved_nvr      = t.save_nvr
  ; C.return         = return
  }

  ⟨it's impossible for the stack pointer to be a useful register too 561b⟩≡ (561a)
  if Register.Set.mem t.sp t.all_regs then
    Impossible.impossible
    (Printf.sprintf "In convention \"%s\", stack pointer is also an ordinary register"
      t.name)
  else
    ()

```

T.5 front_last/dataflow.nw

```

⟨front_last/dataflow.ml 561c⟩≡
  ⟨dataflow.ml 564c⟩

⟨front_last/dataflow.mli 561d⟩≡
  ⟨dataflow.mli 562d⟩

```

T.6 A polymorphic infrastructure for dataflow problems

```

⟨exported types(dataflow.nw) 561e⟩≡ (564c 562d) 562a▷
type 'a answer = Dataflow of 'a | Rewrite of Zipcfg.graph
type txlimit = int

```

```

<exported types(dataflow.nw) 562a>+≡ (564c 562d) <561e 562b>▷
type 'a fact = {
  fact_name : string; (* documentation *)
  init_info : 'a; (* lattice bottom element *)
  add_info : 'a -> 'a -> 'a; (* lattice join (least upper bound) *)
  changed : old:'a -> new':'a -> bool; (* is new one bigger? *)
  prop : 'a Unique.Prop.t; (* access to mutable state by uid *)
}

<exported types(dataflow.nw) 562b>+≡ (564c 562d) <562a
type 'a fact' = {
  fact_name' : string; (* documentation *)
  init_info' : 'a; (* lattice bottom element *)
  add_info' : 'a -> 'a -> 'a; (* lattice join (least upper bound) *)
  changed' : old:'a -> new':'a -> bool; (* is new one bigger? *)
  get' : Zipcfg.uid -> 'a;
  set' : Zipcfg.uid -> 'a -> unit;
}

<utilities(dataflow.nw) 562c>≡ (564c) 564d▷
let to_fact' f =
  { fact_name' = f.fact_name
  ; init_info' = f.init_info
  ; add_info' = f.add_info
  ; changed' = f.changed
  ; get' = P.get f.prop
  ; set' = P.set f.prop
  }
let to_fact f f' =
  {fact_name = f'.fact_name'; init_info = f'.init_info'; add_info = f'.add_info';
  changed = f'.changed'; prop = f.prop}

<dataflow.mli 562d>≡ (561d)
<exported types(dataflow.nw) 561e>
module B : sig
  <exported types for backward analyses 562e>
  <declarations of exported values for analyses 563f>
  <declarations of exported values for backward analyses 563h>
end
module F : sig
  <exported types for forward analyses 563c>
  <declarations of exported values for analyses 563f>
  <declarations of exported values for forward analyses 563i>
end
<declarations of shared exported values 564b>

```

T.6.1 Descriptions of dataflow passes

```

<exported types for backward analyses 562e>≡ (564c 562d) 563a▷
type ('i, 'o) computation =
{ name : string;
  last_in : Zipcfg.Rep.last -> 'o;
  middle_in : 'i -> Zipcfg.Rep.middle -> 'o;
  first_in : 'i -> Zipcfg.Rep.first -> 'o;
}

```

```

⟨exported types for backward analyses 563a⟩+≡ (564c 562d) <562e
type 'a analysis      = 'a fact * ('a, 'a) computation
type 'a analysis'     = 'a fact' * ('a, 'a) computation
type 'a transformation = ('a, Zipcfg.graph option) computation
type 'a pass          = 'a fact * ('a, txlimit -> 'a answer) computation
type 'a pass'         = 'a fact' * ('a, txlimit -> 'a answer) computation

⟨backward utilities 563b⟩≡ (564c) 565d▷
let to_analysis' (f, c) = (to_fact' f, c)
let y = (to_analysis' : 'a analysis -> 'a analysis')
let to_pass' (f, c) = (to_fact' f, c)

⟨exported types for forward analyses 563c⟩≡ (564c 562d) 563d▷
type 'a edge_fact_setter = (Zipcfg.uid -> 'a -> unit) -> unit

type ('i, 'om, 'ol) computation =
{ name      : string;
  middle_out : 'i -> Zipcfg.Rep.middle -> 'om;
  last_outs  : 'i -> Zipcfg.Rep.last -> 'ol;
}

⟨exported types for forward analyses 563d⟩+≡ (564c 562d) <563c
type 'a analysis = 'a fact * ('a, 'a, 'a edge_fact_setter) computation
type 'a analysis' = 'a fact' * ('a, 'a, 'a edge_fact_setter) computation
type 'a transformation = ('a, Zipcfg.graph option, Zipcfg.graph option) computation
type 'a pass =
  'a fact *
  ('a, txlimit -> 'a answer, txlimit -> 'a edge_fact_setter answer) computation
type 'a pass' =
  'a fact' *
  ('a, txlimit -> 'a answer, txlimit -> 'a edge_fact_setter answer) computation

⟨forward utilities 563e⟩≡ (564c) 569b▷
let to_analysis' (f, c) = (to_fact' f, c)
let to_pass' (f, c) = (to_fact' f, c)

⟨declarations of exported values for analyses 563f⟩≡ (562) 563g▷
val anal : 'a analysis -> 'a pass
val anal' : 'a analysis' -> 'a pass'
val a_t : 'a analysis -> 'a transformation -> 'a pass
val a_t' : 'a analysis' -> 'a transformation -> 'a pass'

⟨declarations of exported values for analyses 563g⟩+≡ (562) <563f
val debug' : ('a -> string) -> 'a pass' -> 'a pass'
val debug : ('a -> string) -> 'a pass -> 'a pass

⟨declarations of exported values for backward analyses 563h⟩≡ (562d) 563j▷
val run_anal : 'a analysis -> Zipcfg.graph -> unit
val run_anal' : 'a analysis' -> Zipcfg.graph -> unit

⟨declarations of exported values for forward analyses 563i⟩≡ (562d) 563k▷
val run_anal : 'a analysis -> entry_fact:'a -> Zipcfg.graph -> unit
val run_anal' : 'a analysis' -> entry_fact:'a -> Zipcfg.graph -> unit

⟨declarations of exported values for backward analyses 563j⟩+≡ (562d) <563h
val rewrite : 'a pass -> Zipcfg.graph -> Zipcfg.graph * bool
val rewrite' : 'a pass' -> Zipcfg.graph -> Zipcfg.graph * bool

⟨declarations of exported values for forward analyses 563k⟩+≡ (562d) <563i 564a▷
val rewrite : 'a pass -> entry_fact:'a -> Zipcfg.graph -> Zipcfg.graph * bool
val rewrite' : 'a pass' -> entry_fact:'a -> Zipcfg.graph -> Zipcfg.graph * bool

```

```

⟨declarations of exported values for forward analyses 564a⟩+≡ (562d) <563k
  val rewrite_solved : 'a pass -> entry_fact:'a -> Zipcfg.graph -> Zipcfg.graph * bool
  val rewrite_solved' : 'a pass' -> entry_fact:'a -> Zipcfg.graph -> Zipcfg.graph * bool
  val modify_solved' : 'a pass' -> entry_fact:'a -> Zipcfg.graph -> Zipcfg.graph * bool

⟨declarations of shared exported values 564b⟩≡ (562d)
  val limit_fun : ('a -> 'b -> 'c option) -> ('a -> 'b -> txlimit -> 'c option)

```

T.7 Implementation

```

⟨dataflow.ml 564c⟩≡ (561c)
  module G = Zipcfg
  module GR = Zipcfg.Rep
  module P = Unique.Prop
  module RS = Rtlutil.ToString
  module UM = Unique.Map
  let impossf fmt = Printf.kprintf Impossible.impossible fmt
  let unimpf fmt = Printf.kprintf Impossible.unimp fmt
  let dprintf fmt = Debug.eprintf "dataflow" fmt
  let _ = Debug.register "dataflow" "execution of generic dataflow engine"

  ⟨exported types(dataflow.nw) 561e⟩
  ⟨utilities(dataflow.nw) 562c⟩
  ⟨definitions of exported functions 568c⟩
  module B = struct
    ⟨exported types for backward analyses 562e⟩
    ⟨backward utilities 563b⟩
    ⟨backward stuff 565c⟩

    module XXX = struct
      ⟨backward, no txlim 572d⟩
    end

  end

  module F = struct
    ⟨exported types for forward analyses 563c⟩
    ⟨forward utilities 563e⟩
    ⟨forward stuff 569a⟩
  end
end

```

T.7.1 Utilities

```

⟨utilities(dataflow.nw) 564d⟩+≡ (564c) <562c 565a>
  let run dir name fact changed entry_fact do_block txlim blocks =
    let init () =
      List.iter (fun b -> fact.set' (GR.id b) fact.init_info') blocks;
      fact.set' GR.entry_uid entry_fact in
    let rec iterate n =
      let () = changed := false in
      let txlim = List.fold_left do_block txlim blocks in
      if !changed then
        if n < 1000 then iterate (n + 1) (* starts from original txlim *)
        else impossf "%s didn't converge in %n %s iterations" name n dir
      else
        (Debug.eprintf name "%s converged in %d %s iterations\n" name n dir; txlim) in
    Debug.eprintf name "starting %s dataflow pass %s\n" dir name;

```

```

init();
Debug.eprintf name "post-init %s dataflow pass %s\n" dir name;
iterate 1

<utilities(dataflow.nw) 565a>+≡ (564c) <564d 565b>
let update fact changed =
  fun u a ->
    let old_a = fact.get' u in
    let new_a = fact.add_info' a old_a in
    if fact.changed' ~old:old_a ~new:new_a then
      begin
        dprintf "Dataflow fact changed at unknown uid\n";
        fact.set' u new_a;
        if not (Unique.eq u GR.entry_uid) then (* no need to re-run on new entry *)
          changed := true
      end

let _ = (update : 'a fact' -> bool ref -> G.uid -> 'a -> unit)

<utilities(dataflow.nw) 565b>+≡ (564c) <565a 567b>
let without_changing_entry fact go =
  let restore =
    try
      let entry = fact.get' GR.entry_uid in
      fun () -> fact.set' GR.entry_uid entry
    with Not_found ->
      fun () -> () in
  let output = go () in
  let answer = fact.get' GR.entry_uid in
  restore();
  output, answer

```

T.7.2 Backward problems

```

<backward stuff 565c>≡ (564c) 566a>
let run_anal' (fact, comp) graph =
  let changed = ref false in
  let update = update fact changed in
  let set_block_fact () b =
    let h, l = GR.goto_end (GR.unzip b) in
    let block_in = (* 'in' fact for the block *)
      let rec head_in h out = match h with
        | GR.Head (h, m) -> head_in h (comp.middle_in out m)
        | GR.First f -> comp.first_in out f in
      head_in h (comp.last_in l) in
    update (GR.id b) block_in in
  let blocks = List.rev (G.postorder_dfs graph) in
  run "backward" comp.name fact changed fact.init_info' set_block_fact () blocks
let run_anal anal graph = run_anal' (to_analysis' anal) graph

<backward utilities 565d>+≡ (564c) <563b 567a>
let comp_with_exit comp exit_fact =
  let last_in l txlimit = match l with
    | GR.Exit -> Dataflow exit_fact
    | _ -> comp.last_in l txlimit in
  { comp with last_in = last_in }

```

```

⟨backward stuff 566a⟩+≡ (564c) <565c 566b>
let rec solve_graph fact comp txlim graph exit_fact =
  without_changing_entry fact (fun () ->
    general_backward fact (comp_with_exit comp exit_fact) txlim graph)
and general_backward fact comp txlim graph =
  let changed = ref false in
  let update = update fact changed in
  let set_block_fact txlim b =
    let txlim, block_in =
      let rec head_in txlim h out = match h with
        | GR.Head (h, m) ->
          (dprintf "Solving middle node %s\n" (RS.rtl (GR.mid_instr m)));
          match comp.middle_in out m txlim with
          | Dataflow a -> head_in txlim h a
          | Rewrite g ->
            let txlim, a = solve_graph fact comp (txlim-1) g out in
            head_in txlim h a)
        | GR.First f ->
          match comp.first_in out f txlim with
          | Dataflow a -> txlim, a
          | Rewrite g -> solve_graph fact comp (txlim-1) g out in
      let h, l = GR.goto_end (GR.unzip b) in
      match comp.last_in l txlim with
      | Dataflow a -> head_in txlim h a
      | Rewrite g ->
        let txlim, a = solve_graph fact comp (txlim-1) g fact.init_info' in
        head_in txlim h a in
    update (GR.id b) block_in;
  txlim in
let blocks = List.rev (G.postorder_dfs graph) in
run "backward" comp.name fact changed fact.init_info' set_block_fact txlim blocks

```

```

⟨backward stuff 566b⟩+≡ (564c) <566a 567d>
let rec solve_and_rewrite fact comp txlim graph exit_fact changed =
  let _, a = solve_graph fact comp txlim graph exit_fact in (* pass 1 *)
  let txlim, g, c = (* pass 2 *)
    backward_rewrite fact (comp_with_exit comp exit_fact) txlim graph changed in
  txlim, a, (g, c)
and backward_rewrite (fact : 'a fact') comp txlim graph changed =
  let rec rewrite_blocks txlim rewritten fresh changed : txlimit * G.graph * bool =
    match fresh with
    | [] -> txlim, G.of_block_list rewritten, changed
    | b :: bs ->
      let rec rewrite_next_block txlim =
        let h, l = GR.goto_end (GR.unzip b) in
        match comp.last_in l txlim with
        | Dataflow a -> propagate txlim h a (GR.Last l) rewritten changed
        | Rewrite g ->
          let txlim, a, (g, _) =
            solve_and_rewrite fact comp (txlim-1) g fact.init_info' changed in
          let t, g = G.remove_entry g in
          let rewritten = add_blocks (G.to_blocks g) rewritten in
          (* continue at entry of g *)
          propagate txlim h a t rewritten true
      and propagate : txlimit -> GR.head -> 'a -> GR.tail -> GR.block list -> bool ->
        txlimit * G.graph * bool =
        fun txlim h out tail rewritten changed -> match h with
        | GR.Head (h, m) -> (
          dprintf "Rewriting middle node %s\n" (RS.rtl (GR.mid_instr m));
          match comp.middle_in out m txlim with

```

```

| Dataflow a -> propagate txlim h a (GR.Tail (m, tail)) rewritten changed
| Rewrite g ->
  dprintf "Rewriting middle node...\n";
  let txlim, a, (g, _) =
    solve_and_rewrite fact comp (txlim-1) g out changed in
  dprintf "Rewrite of middle node completed\n";
  let t, g = G.splice_tail g tail in
  let rewritten = add_blocks (G.to_blocks g) rewritten in
  propagate txlim h a t rewritten true)
| GR.First f ->
  match comp.first_in out f txlim with
  | Dataflow a ->
    let b = (f, tail) in
    check_property_match fact a b;
    rewrite_blocks txlim (b :: rewritten) bs changed
  | Rewrite g -> impossf "rewriting a label in backward dataflow" in
  rewrite_next_block txlim in
  rewrite_blocks txlim [] (List.rev (G.postorder_dfs graph)) changed

let rewrite' (fact, comp) g =
  let txlim = Tx.remaining () in
  let txlim', _, gc = solve_and_rewrite fact comp txlim g fact.init_info' false in
  Tx.decrement ~name:comp.name ~from:txlim ~to':txlim';
  gc
let rewrite pass g = rewrite' (to_pass' pass) g

⟨backward utilities 567a⟩+≡ (564c) <565d
let check_property_match fact a block =
  match (fst block) with
  | GR.Entry -> () (* needn't match *)
  | GR.Label ((_, l), _, _) ->
    let old_a = fact.get' (GR.id block) in
    let new_a = fact.add_info' a old_a in
    if not (eqfact fact old_a new_a) then
      impossf "property at label '%s' changed after supposedly reaching fixed point" l

⟨utilities(dataflow.nw) 567b⟩+≡ (564c) <565b 567c>
let add_blocks map list =
  let add _ b bs = b :: bs in
  UM.fold add map list

⟨utilities(dataflow.nw) 567c⟩+≡ (564c) <567b 571a>
let ( << ) f g = fun x -> f (g x)

⟨backward stuff 567d⟩+≡ (564c) <566b 568a>
let debug' s (f, comp) =
  let pr = Printf.eprintf in
  let fact dir node a = pr "%s %s for %s = %s\n" f.fact_name' dir node (s a) in
  let rewr node g = pr "%s rewrites %s to <not-shown>\n" comp.name node in
  let wrap f nodestring node txlim =
    let answer = f node txlim in
    let () = match answer with
    | Dataflow a -> fact "in " (nodestring node) a
    | Rewrite g -> rewr (nodestring node) g in
    answer in
  let wrapout f nodestring out node txlim =
    fact "out" (nodestring node) out;
    wrap (f out) nodestring node txlim in
  let last_in = wrap comp.last_in (RS.rtl << GR.last_instr) in
  let middle_in = wrapout comp.middle_in (RS.rtl << GR.mid_instr) in

```



```

let first_in =
  let first = function GR.Entry -> "<entry>" | GR.Label ((u, l), _, _) -> l in
  wrapout comp.first_in first in
  f, { comp with last_in = last_in; middle_in = middle_in; first_in = first_in; }
let debug s ((f, comp) as pass) =
  let (f', comp') = debug' s (to_pass' pass) in
  (to_fact f f', comp')

```

<backward stuff 568a>+≡ (564c) <567d 568b>

```

let anal (fact, comp) =
  let wrap f node txlim = Dataflow (f node) in
  let wrap2 f out node txlim = Dataflow (f out node) in
  fact,
  { name = comp.name; last_in = wrap comp.last_in;
    middle_in = wrap2 comp.middle_in; first_in = wrap2 comp.first_in; }
let anal' = anal

```

<backward stuff 568b>+≡ (564c) <568a 568d>

```

let a_t' (fact, comp) tx =
  let last_in l txlim =
    if txlim > 0 then
      match tx.last_in l with
      | Some g -> Rewrite g
      | None -> Dataflow (comp.last_in l)
    else
      Dataflow (comp.last_in l) in
  let middle_in out m txlim =
    if txlim > 0 then
      match tx.middle_in out m with
      | Some g -> Rewrite g
      | None -> Dataflow (comp.middle_in out m)
    else
      Dataflow (comp.middle_in out m) in
  let first_in out f txlim =
    if txlim > 0 then
      match tx.first_in out f with
      | Some g -> Rewrite g
      | None -> Dataflow (comp.first_in out f)
    else
      Dataflow (comp.first_in out f) in
  fact,
  { name = Printf.sprintf "%s and %s" comp.name tx.name;
    last_in = last_in; middle_in = middle_in; first_in = first_in;
  }
let a_t x tx = a_t' x tx

```

<definitions of exported functions 568c>≡ (564c)

```

let limit_fun f i n txlim = if txlim > 0 then f i n else None
let limit_last f n txlim = if txlim > 0 then f n else None

```

<backward stuff 568d>+≡ (564c) <568b>

```

let limit_anal (fact, comp) =
  fact,
  { name = comp.name;
    first_in = (fun i n _ -> comp.first_in i n);
    middle_in = (fun i n _ -> comp.middle_in i n);
    last_in = (fun l _ -> comp.last_in l);
  }
let limit_tx tx =
  { name = tx.name;

```

```

first_in = limit_fun tx.first_in;
middle_in = limit_fun tx.middle_in;
last_in = limit_last tx.last_in;
}

```

T.7.3 Forward

```

⟨forward stuff 569a⟩≡ (564c) 569c▷
let run_anal' (fact, comp) ~entry_fact graph =
  let changed = ref false in
  let update = update fact changed in
  let set_successor_facts () b =
    let rec forward in' t = match t with
      | GR.Tail (m, t) -> forward (comp.middle_out in' m) t
      | GR.Last l -> comp.last_outs in' l update in
    let f, t = b in
    let blockname =
      match f with GR.Entry -> "<entry>" | GR.Label ((_, l), _, _) -> l in
    dprintf "Setting successor fact of block %s\n" blockname;
    forward (fact.get' (GR.fid f)) t in
  let blocks = G.postorder_dfs graph in
  run "forward" comp.name fact changed entry_fact set_successor_facts () blocks
let run_anal anal ~entry_fact graph = run_anal' (to_analysis' anal) entry_fact graph

```

```

⟨forward utilities 569b⟩+≡ (564c) <563e 571b>
let comp_with_exit comp exit_fact_ref =
  let last_outs in' l txlimit = match l with
  | GR.Exit -> Dataflow (fun set -> exit_fact_ref := in')
  | _ -> comp.last_outs in' l txlimit in
  { comp with last_outs = last_outs }

```

```

⟨forward stuff 569c⟩+≡ (564c) <569a 570>
let rec solve_graph fact comp txlim graph in_fact =
  let exit_fact_ref = ref fact.init_info' in
  let txlim =
    general_forward fact (comp_with_exit comp exit_fact_ref) txlim in_fact graph in
  txlim, !exit_fact_ref
and general_forward fact comp txlim entry_fact graph =
  let changed = ref false in
  let update = update fact changed in
  let set_successor_facts txlim b =
    let rec set_tail_facts txlim in' t = match t with
      | GR.Tail (m, t) ->
        (dprintf "Solving middle node %s\n" (RS.rtl (GR.mid_instr m)));
        match comp.middle_out in' m txlim with
        | Dataflow a -> set_tail_facts txlim a t
        | Rewrite g ->
          let txlim, g = solve_graph fact comp (txlim-1) g in' in
          set_tail_facts txlim g t
      | GR.Last l ->
        match comp.last_outs in' l txlim with
        | Dataflow setter -> (setter update; txlim)
        | Rewrite g -> fst (solve_graph fact comp (txlim-1) g in') in
    let f, t = b in
    let in' = match f with (* CHANGE TO USE GR.id ? *)
    | GR.Entry -> entry_fact
    | GR.Label ((u, _), _, _) -> fact.get' u in
    set_tail_facts txlim in' t in
  let blocks = G.postorder_dfs graph in
  run "forward" comp.name fact changed entry_fact set_successor_facts txlim blocks

```

```

⟨forward stuff 570⟩+≡ (564c) <569c 571c>
let expectD = function
  | Dataflow d -> d
  | Rewrite _ -> impossf "expected dataflow result"
let rec solve_and_rewrite modify fact comp txlim graph in_fact changed =
  let _ = solve_graph fact comp txlim graph in_fact in (* pass 1 *)
  let exit_ref = ref fact.init_info' in
  let txlim, g, c = forward_rewrite modify fact
    (comp_with_exit comp exit_ref) txlim graph in_fact changed in
  txlim, !exit_ref, (g,c)
and forward_rewrite modify (fact : 'a fact') comp txlim graph entry_fact changed =
  let rec rewrite_blocks txlim rewritten fresh changed : txlimit * G.graph * bool =
    match fresh with
    | [] -> txlim, G.of_block_list rewritten, changed
    | b :: bs ->
      let rec rewrite_next_block txlim =
        let f, t = b in
        let in' = match f with
          | GR.Entry -> entry_fact
          | GR.Label ((u, _), _, _) -> fact.get' u in
        propagate txlim (GR.First f) in' t rewritten changed
      and propagate :
        txlimit -> GR.head -> 'a -> GR.tail -> GR.block list -> bool ->
          txlimit * G.graph * bool =
      fun txlim h in' t rewritten changed -> match t with
      | GR.Tail (m, t) -> (
        dprintf "Rewriting middle node %s\n" (RS.rtl (GR.mid_instr m));
        match comp.middle_out in' m txlim with
        | Dataflow a -> propagate txlim (GR.Head (h, m)) a t rewritten changed
        | Rewrite g ->
          dprintf "Rewriting middle node...\n";
          let txlim, a, (g, _) =
            if modify then
              (txlim-1, expectD (comp.middle_out in' m 0), (g, false))
            else solve_and_rewrite modify fact comp (txlim-1) g in' changed in
          dprintf "Rewrite of middle node completed\n";
          let g, h = G.splice_head h g in
          propagate txlim h a t (add_blocks (G.to_blocks g) rewritten) true)
      | GR.Last l ->
        dprintf "Rewriting last node %s\n" (RS.rtl (GR.last_instr l));
        match comp.last_outs in' l txlim with
        | Dataflow set ->
          if not modify then set (check_property_match fact);
          let b = GR.zip (h, GR.Last l) in
          rewrite_blocks txlim (b :: rewritten) bs changed
        | Rewrite g ->
          (* could test here that [[exits g = exits (GR.Entry, GR.Last l)]] *)
          if Debug.on "rewrite-last" then
            dprintf "Last node %s rewritten to:\n" (RS.rtl (GR.last_instr l));
            let k (txlim, _, (g, changed)) =
              let g = G.to_blocks (G.splice_head_only h g) in
              rewrite_blocks txlim (add_blocks g rewritten) bs changed in
            if modify then
              k (txlim-1, expectD (comp.last_outs in' l 0), (g, true))
            else
              k (solve_and_rewrite modify fact comp (txlim-1) g in' true) in
          rewrite_next_block txlim in
      rewrite_blocks txlim [] (G.postorder_dfs graph) changed

let _ =

```

```

Debug.register "rewrite-last" "show what happens when last node is rewritten forward"

let rewrite_solved'' modify (fact, comp) ~entry_fact g =
  let txlim = Tx.remaining () in
  let txlim', g, changed = forward_rewrite modify fact comp txlim g entry_fact false in
  Tx.decrement ~name:comp.name ~from:txlim ~to':txlim';
  g, changed
let rewrite_solved pass ~entry_fact g =
  rewrite_solved'' false (to_pass' pass) entry_fact g
let rewrite_solved' pass ~entry_fact g = rewrite_solved'' false pass entry_fact g
let modify_solved' pass ~entry_fact g = rewrite_solved'' true pass entry_fact g

let rewrite' (fact, comp) ~entry_fact g =
  let txlim = Tx.remaining () in
  let txlim', _, gc = solve_and_rewrite false fact comp txlim g entry_fact false in
  Tx.decrement ~name:comp.name ~from:txlim ~to':txlim';
  gc
let rewrite pass ~entry_fact g = rewrite' (to_pass' pass) entry_fact g

<utilities(dataflow.nw) 571a>+≡ (564c) <567c
let eqfact fact a a' = (* poor man's approximation of equality *)
  not (fact.changed' a a' or fact.changed' a' a)

<forward utilities 571b>+≡ (564c) <569b
let check_property_match fact u a =
  let old_a = fact.get' u in
  let new_a = fact.add_info' a old_a in
  if not (eqfact fact old_a new_a) then
    impossf "property '%s' changed after supposedly reaching fixed point"
      fact.fact_name'

<forward stuff 571c>+≡ (564c) <570 572a>
let debug' s (f, comp) =
  let pr = Printf.eprintf in
  let fact dir node a = pr "%s %s for %s = %s\n" f.fact_name' dir node (s a) in
  let setter dir node run_sets set =
    run_sets (fun u a -> pr "%s %s for %s = %s\n" f.fact_name' dir node (s a); set u a) in
  let rewr node g = pr "%s rewrites %s to <not-shown>\n" comp.name node in
  let wrap f nodestring wrap_answer in' node txlim =
    fact "in " (nodestring node) in';
    wrap_answer (nodestring node) (f in' node txlim)
  and wrap_fact n answer =
    let () = match answer with
    | Dataflow a -> fact "out" n a
    | Rewrite g -> rewr n g in
    answer
  and wrap_setter n answer =
    match answer with
    | Dataflow set -> Dataflow (setter "out" n set)
    | Rewrite g -> (rewr n g; Rewrite g) in
  let middle_out = wrap comp.middle_out (RS.rtl << GR.mid_instr) wrap_fact in
  let last_outs = wrap comp.last_outs (RS.rtl << GR.last_instr) wrap_setter in
  f, { comp with last_outs = last_outs; middle_out = middle_out; }
let debug s ((f, comp) as pass) =
  let (f', comp') = debug' s (to_pass' pass) in
  ({fact_name = f'.fact_name'; init_info = f'.init_info'; add_info = f'.add_info';
    changed = f'.changed'; prop = f'.prop}, comp')

```

<forward stuff 572a>+≡ (564c) <571c 572b>

```
let limit_anal (fact, comp) =
  fact,
  { name = comp.name;
    middle_out = (fun i n _ -> comp.middle_out i n);
    last_outs = (fun i n _ -> comp.last_outs i n);
  }
let limit_tx tx =
  { name = tx.name;
    middle_out = limit_fun tx.middle_out;
    last_outs = limit_fun tx.last_outs;
  }
```

<forward stuff 572b>+≡ (564c) <572a 572c>

```
let anal (fact, comp) =
  let wrap f in' node txlim = Dataflow (f in' node) in
  fact,
  { name = comp.name;
    last_outs = wrap comp.last_outs; middle_out = wrap comp.middle_out; }
let anal' = anal
```

<forward stuff 572c>+≡ (564c) <572b

```
let a_t' (fact, comp) tx =
  let last_outs in' l txlim =
    if txlim > 0 then
      match tx.last_outs in' l with
      | Some g -> Rewrite g
      | None -> Dataflow (comp.last_outs in' l)
    else
      Dataflow (comp.last_outs in' l) in
  let middle_out in' m txlim =
    if txlim > 0 then
      match tx.middle_out in' m with
      | Some g -> Rewrite g
      | None -> Dataflow (comp.middle_out in' m)
    else
      Dataflow (comp.middle_out in' m) in
  fact,
  { name = Printf.sprintf "%s and %s" comp.name tx.name;
    last_outs = last_outs; middle_out = middle_out;
  }
let a_t x tx = a_t' x tx
```

<backward, no txlim 572d>≡ (564c) 573▷

```
let comp_with_exit comp exit_fact =
  let last_in l = match l with
  | GR.Exit -> Dataflow exit_fact
  | _ -> comp.last_in l in
  { comp with last_in = last_in }
```

```
let run f c e d blocks = run "backward" "comp.name" f c e (fun () b -> d b) () blocks
```

```
let rec solve_graph fact comp graph exit_fact =
  general_backward fact (comp_with_exit comp exit_fact) graph;
  fact.get' GR.entry_uid
and general_backward fact comp graph =
  let changed = ref false in
  let update = update fact changed in
  let set_block_fact b =
    let block_in =
```

```

let rec head_in h out = match h with
| GR.Head (h, m) ->
    let a = match comp.middle_in out m with
    | Dataflow a -> a
    | Rewrite g -> solve_graph fact comp g out in
    head_in h a
| GR.First f ->
    match comp.first_in out f with
    | Dataflow a -> a
    | Rewrite g -> solve_graph fact comp g out in
let h, l = GR.goto_end (GR.unzip b) in
let a = match comp.last_in l with
| Dataflow a -> a
| Rewrite g -> solve_graph fact comp g fact.init_info' in
head_in h a in
update (GR.id b) block_in in
let blocks = List.rev (G.postorder_dfs graph) in
run fact changed fact.init_info' set_block_fact blocks

⟨backward, no txlim 573⟩+≡ (564c) <572d 574a>
let rec solve_and_rewrite fact comp graph exit_fact changed =
    let a = solve_graph fact comp graph exit_fact in (* pass 1 *)
    let g, c = (* pass 2 *)
        backward_rewrite fact (comp_with_exit comp exit_fact) graph changed in
    a, (g, c)
and backward_rewrite fact comp graph changed =
    let rec rewrite_blocks rewritten fresh changed : G.graph * bool =
        match fresh with
        | [] -> G.of_block_list rewritten, changed
        | b :: bs ->
            let rec rewrite_next_block () =
                let h, l = GR.goto_end (GR.unzip b) in
                match comp.last_in l with
                | Dataflow a -> propagate h a (GR.Last l) rewritten changed
                | Rewrite g ->
                    let a, (g, _) =
                        solve_and_rewrite fact comp g fact.init_info' changed in
                    let t, g = G.remove_entry g in
                    let rewritten = add_blocks (G.to_blocks g) rewritten in
                    (* continue at entry of g *)
                    propagate h a t rewritten true
            and propagate : GR.head -> 'a -> GR.tail -> GR.block list -> bool ->
                G.graph * bool =
            fun h out tail rewritten changed -> match h with
            | GR.Head (h, m) -> (
                match comp.middle_in out m with
                | Dataflow a -> propagate h a (GR.Tail (m, tail)) rewritten changed
                | Rewrite g ->
                    let a, (g, _) =
                        solve_and_rewrite fact comp g out changed in
                    let t, g = G.splice_tail g tail in
                    let rewritten = add_blocks (G.to_blocks g) rewritten in
                    propagate h a t rewritten true)
            | GR.First f ->
                match comp.first_in out f with
                | Dataflow a ->
                    let b = (f, tail) in
                    rewrite_blocks (b :: rewritten) bs changed
                | Rewrite g -> impossf "rewriting a label in backward dataflow" in
            rewrite_next_block () in

```

```

rewrite_blocks [] (List.rev (G.postorder_dfs graph)) changed
⟨backward, no txlim 574a⟩≡ (564c) ◁573
module Unique = struct
  module Map = struct
    let empty = G.of_blocks Unique.Map.empty
    let union g1 g2 = G.of_blocks (Unique.Map.union (G.to_blocks g1) (G.to_blocks g2))
    let add k v g = G.of_blocks (Unique.Map.add k v (G.to_blocks g))
  end
end

let rec solve_and_rewrite fact comp graph exit_fact =
  let a = solve_graph fact comp graph exit_fact in (* pass 1 *)
  let g = (* pass 2 *)
    backward_rewrite fact (comp_with_exit comp exit_fact) graph in
  a, g
and backward_rewrite fact comp graph =
  let rec rewrite_blocks rewritten fresh =
    match fresh with
    | [] -> rewritten
    | b :: bs ->
      let rec rewrite_next_block () =
        let h, l = GR.goto_end (GR.unzip b) in
        match comp.last_in l with
        | Dataflow a -> propagate h a (GR.Last l) rewritten
        | Rewrite g ->
          let a, g = solve_and_rewrite fact comp g fact.init_info' in
          let t, g = G.remove_entry g in
          let rewritten = Unique.Map.union g rewritten in
          (* continue at entry of g *)
          propagate h a t rewritten
      and propagate : GR.head -> 'a -> GR.tail -> G.graph -> G.graph =
        fun h out tail rewritten -> match h with
        | GR.Head (h, m) -> (
          match comp.middle_in out m with
          | Dataflow a -> propagate h a (GR.Tail (m, tail)) rewritten
          | Rewrite g ->
            let a, g = solve_and_rewrite fact comp g out in
            let t, g = G.splice_tail g tail in
            let rewritten = Unique.Map.union g rewritten in
            propagate h a t rewritten)
        | GR.First f ->
          match comp.first_in out f with
          | Dataflow a ->
            let b = (f, tail) in
            rewrite_blocks (Unique.Map.add (GR.id b) b rewritten) bs
          | Rewrite g -> impossf "rewriting a label in backward dataflow" in
      rewrite_next_block () in
  rewrite_blocks Unique.Map.empty (List.rev (G.postorder_dfs graph))

```

T.8 front_last/mvalidate.nw

```

⟨front_last/mvalidate.ml 574b⟩≡
  ⟨mvalidate.ml 575b⟩
⟨front_last/mvalidate.mli 574c⟩≡
  ⟨mvalidate.mli 575a⟩

```

T.9 RTL Validation

```
<mvalidate.mli 575a>≡ (574c)
  val rtl : ('a, 'b, 'c) Target.t -> Rtl.rtl -> string option

<mvalidate.ml 575b>≡ (574b) 575c>
  open Nopoly

  module RP    = Rtl.Private
  module Down  = Rtl.Dn
  module Up    = Rtl.Up

<mvalidate.ml 575c>+≡ (574b) <575b 575d>
  exception RTLInvalid of string

<mvalidate.ml 575d>+≡ (574b) <575c
  let () = Debug.register "mvalidate" "debug machine-environment validator"
  let rtl t r =
    let remove_bits = function
      | Types.Bits n -> n
      | Types.Bool   -> 1
    in
    let impossf fmt = Printf.kprintf Impossible.impossible fmt in
    let imposs = Impossible.impossible in
    let m      = List.map Rtl.Dn.opr t.Target.capabilities.Target.operators in
    let lops   = List.map Rtl.Dn.opr t.Target.capabilities.Target.litops in
    let w      = t.Target.wordsize in
    let ns     = t.Target.capabilities.Target.literals in
    Debug.eprintf "mvalidate" "ns is %d length\n" (List.length ns);
    let spaces = Vfp.mk_space w :: t.Target.spaces in
    let name   = t.Target.name in
    let widths ops opname =
      let all = List.find_all (fun (opname',_) -> opname'=$= opname') ops in
      List.map (fun (opname, ws) -> ws) all in
    let machine_widths = widths m in
    let literal_widths = widths lops in
    let xy_or_z tostr xyz =
      match List.rev xyz with
      | [] -> impossf "No literal sizes specified for target '%s'" name
      | [z] -> tostr z
      | z::yx ->
          let xy = List.rev (List.map tostr yx) in
          let xy' = String.concat ", " xy in
          xy'^" or "^tostr z in
    let sprintf = Printf.sprintf in
    let reject fmt = Printf.kprintf (fun s -> raise (RTLInvalid s)) fmt in
    let spacename (s, _, _) = s in
    let reg s n p1 p2 =
      let space =
        try List.find (fun x -> spacename x.Space.space <= s) spaces
        with Not_found -> reject "%s" (p1 s name) in
      List.mem n space.Space.widths
      || reject "%s" (p2 s (xy_or_z string_of_int space.Space.widths) n) in
    let rec loc = function
      | RP.Mem(_,n,e,_) -> exp e (* not clear what else to do *)
      | RP.Reg((s,_,cell),_,c) ->
          reg s (Cell.to_width cell c)
          (sprintf "Space '%c' not found in target '%s'")
          (sprintf "Space '%c' only supports width %s; asked for %d")
      | RP.Var(x,_,n) -> n <= w || n mod t.Target.memsize == 0
```



```

|| reject
  "variable %s of type bits%d should either be at most %d bits wide or else have a width a multiple of
| RP.Global(s,_,n) ->
  if s == "System.rounding_mode" || s == "rm" then
    n = 2 || reject "System.rounding_mode must be 2 bits, not %d" n
  else
    reg 'r' n (fun c s -> imposs "Space 'r' must be available")
    (fun c -> sprintf "Globals only available at %s bits; asked for %d bits")
| RP.Slice(n,_,l) -> loc l (* what to check here? *)
and exp e = exp' None e
and exp' litcontext e =
  let nonliteral = match litcontext with
  | None -> (fun s -> s)
  | Some op -> (fun s -> reject "operator %%%s may be applied only to \
                                compile-time constant expressions" op) in
let rec exp = function
| RP.Fetch(l, n) -> nonliteral (loc l)
| RP.App(("or", [resw]), [RP.App(("zx", _), [e]);
                        RP.App(("shl", _), [RP.App(("zx", _), [e]); k])])
  when resw mod w = 0 && exp e && exp e' && exp k -> true
| RP.App(("lobits", [argw; resw]), [RP.App(("shrl", _), [e; k])])
  when resw mod w = 0 && exp e && exp k -> true
  (* should also check that k is a constant multiple of wordsize *)
| RP.App(("lobits", [argw; resw]), [e]) when resw mod w = 0 && exp e -> true
| RP.App((opname, ws), es) ->
  let str_of_opty (opname, ws) =
    let (argw, resw) = Rtl.op mono (Rtl.opr opname ws) in
    let argw = List.map remove_bits argw in
    let resw = remove_bits resw in
    String.concat "->" (List.map string_of_int (argw @ [resw])) in
  (* like the widener, we give up on operations not in M *)
  let wss' = machine_widths opname in
  let op_possible wss =
    (* An op is possible if some form exists where each argument is narrower than required.
    * and if we have the right number of arguments. *)
    try List.exists (List.for_all2 (fun w w' -> w <= w') ws) wss
    with Invalid_argument _ -> false in
  if Debug.on "mvalidate" then
    (Printf.eprintf "Validating %%%s::%s..." opname (str_of_opty (opname, ws));
    if op_possible wss' then
      Debug.eprintf "mvalidate" "matches at %s\n"
        (xy_or_z str_of_opty (List.map (fun x -> (opname, x)) wss'))
    else Debug.eprintf "mvalidate" "impossible\n");
  if op_possible wss' then
    List.for_all exp es
  else
    let lit_wss = literal_widths opname in
    if op_possible lit_wss then
      List.for_all (exp' (Some opname)) es
    else
      reject "No acceptable widths for %%%s on target '%s'; \
              asked for %s, target accepts %s" opname name
        (str_of_opty (opname, ws))
        (match wss' with [] -> "nothing!"
         | _ -> xy_or_z str_of_opty (List.map (fun x -> (opname, x)) wss'))
| RP.Const c -> const c
and const c =
  let string c = Rtlutil.ToString.exp (Up.exp (RP.Const c)) in
  match c with
  | (RP.Link(_,_,n)) as c -> nonliteral (List.exists (fun w -> n <= w) ns)

```

```

    || reject "Constant %s is %d bits but was expected at <= %s bits"
      (string c) n (xy_or_z string_of_int ns)
  | RP.Diff(x,y) -> nonliteral (const x && const y)
  | (RP.Late(_,n)) as c -> nonliteral (List.exists (fun w -> n <= w) ns)
    || reject "Constant %s is %d bits but was expected at <= %s bits"
      (string c) n (xy_or_z string_of_int ns)
  | (RP.Bits b) as c -> List.exists (fun w -> Bits.width b <= w) ns
    || reject "Constant %s is %d bits but was expected at <= %s bits"
      (string c) (Bits.width b) (xy_or_z string_of_int ns)
  | RP.Bool _ -> true in
exp e in
let check (g, eff) =
  let eff' =
    match eff with
    | RP.Store(l, r, w) -> loc l && exp r
    | RP.Kill l -> loc l
  in
  eff' && exp g
in
let RP.Rtl es = Down.rtl r in
try
  if List.for_all check es then None
  else Some "No explanation"
with RTLInvalid s -> Some s

```

T.10 front_last/placevar.nw

$\langle \text{front_last/placevar.ml } 577a \rangle \equiv$
 $\langle \text{placevar.ml } 577e \rangle$

$\langle \text{front_last/placevar.mli } 577b \rangle \equiv$
 $\langle \text{placevar.mli } 577c \rangle$

T.11 Placing variables into suitable temporary locations

$\langle \text{placevar.mli } 577c \rangle \equiv$ (577b) 577d▷
 val mk_automaton : warn:(width:int -> alignment:int -> kind:string -> unit) ->
 vfp:Rtl.exp ->
 memspace:Rtl.space ->
 (temps:(char -> Automaton.stage) -> Automaton.stage) ->
 (Ast2ir.proc -> Automaton.t)
 val context : (Ast2ir.proc -> Automaton.t) -> 'e -> Ast2ir.proc -> Ast2ir.proc * bool

$\langle \text{placevar.mli } 577d \rangle + \equiv$ (577b) <577c
 val replace_globals : 'a -> Ast2ir.proc -> Ast2ir.proc * bool

$\langle \text{placevar.ml } 577e \rangle \equiv$ (577a) 578d▷
 open Nopoly

```

module A = Automaton
module G = Zipcfg
module GR = Zipcfg.Rep
module P = Proc
module PA = Preast2ir
module R = Rtl
module RT = Runtimedata

```

```

module RU = Rtlutil
module RP = Rtl.Private

<replace var 578b>

<fetch and store globals 578a>≡ (581a)
  let store_global exp (s, i, w) = proc.global_map.(i).A.store (R.Up.exp exp) w in
  let fetch_global      (s, i, w) = proc.global_map.(i).A.fetch      w in

<replace var 578b>≡ (577e)
  let replace_var store_var fetch_var store_global fetch_global r =
    <walk RTL 578c>
    walkRtl r

<walk RTL 578c>≡ (578b)
  let rec walkLoc loc =
    match loc with
    | RP.Mem (sp, w, e, ass) -> RP.Mem (sp, w, walkExp e, ass)
    | RP.Reg (sp, i, w) as r -> r
    | RP.Var (s, i, w) -> Impossible.unimp "slice of variable"
    | RP.Global(s, i, w) -> Impossible.unimp "slice of global variable"
    | RP.Slice (w, i, l) -> RP.Slice (w, i, walkLoc l)

  and walkExp exp = match exp with
  | RP.Const _ -> exp
  | RP.Fetch (RP.Var (s,i,w), _) -> R.Dn.exp (fetch_var (s,i,w))
  | RP.Fetch (RP.Global (s,i,w), _) -> R.Dn.exp (fetch_global (s,i,w))
  | RP.Fetch (l, w) -> RP.Fetch (walkLoc l, w)
  | RP.App (op, exps) -> RP.App(op, List.map walkExp exps)
  and upExp e = R.Up.exp (walkExp e)
  and upLoc l = R.Up.loc (walkLoc l)

  and walkEffect effect = match effect with
  | RP.Store (RP.Var (s,i,w), e, _) -> store_var (walkExp e) (s,i,w)
  | RP.Store (RP.Global (s,i,w), e, _) -> store_global (walkExp e) (s,i,w)
  | RP.Store (l, e, w) -> R.store (upLoc l) (upExp e) w
  | RP.Kill (RP.Var _ | RP.Global _) -> Impossible.unimp "killing variables"
  | RP.Kill l -> R.kill (upLoc l) in

  let walkGuard (exp, effect) = R.guard (upExp exp) (walkEffect effect) in

  let walkRtl r = match Rtl.Dn.rtl r with
  | RP.Rtl gs -> R.par (List.map walkGuard gs) in

```

T.11.1 Variable Placement by Execution Estimate

```

<placevar.ml 578d>+≡ (577a) <577e 578e>
  module IntMod = struct type t = int let compare = compare end
  module IM = Map.Make (IntMod)
  module NM = Unique.Map
  module SM = Strutil.Map

<placevar.ml 578e>+≡ (577a) <578d 581c>
  <define context count structures 579a>
  let context autmtn _ (g, (Procedure.{ target = PA.T tgt; formals = formals;
                                var_map = tMap} as proc)) =

    let changed = ref false in
    let autmtn = autmtn (g, proc) in
    <initialize context count structures 579b>

```

```

  <get operator contexts 579c>
  <estimate exec_counts for graph nodes 579d>
  <count uses in each context 579e>
  <choose a context for each variable 580a>
  <replace variables with temps 581a>
  <freeze autmtn and make sure its overflow block is empty 581d>
  <add variable placements to spans 581f>
  (g, proc), !changed

<define context count structures 579a>≡ (578e)
  type counts = { mutable intc : float
                  ; mutable floatc : float
                  ; mutable addrc : float
                  ; mutable boolc : float
                  }

<initialize context count structures 579b>≡ (578e)
  let new_count _ = { intc = 0.0; floatc = 0.0; addrc = 0.0; boolc = 0.0} in
  let var_counts = Array.init proc.vars new_count in
  let inc_int i f = let record = var_counts.(i) in
                    record.intc <- record.intc +. f in
  let inc_float i f = let record = var_counts.(i) in
                      record.floatc <- record.floatc +. f in
  let inc_addr i f = let record = var_counts.(i) in
                     record.addrc <- record.addrc +. f in
  let inc_bool i f = Error.warningPrt "variable found in boolean context" in
  let inc_rm i f = Error.warningPrt "variable found in rounding mode context" in

<get operator contexts 579c>≡ (578e)
  let ops =
    let context = Context.full inc_int inc_float inc_rm inc_bool [] in
    let add map (op, parms, res) = SM.add op (parms, res) map in
    List.fold_left add SM.empty context in
  let ctxt_parms op = try Some (fst (SM.find op ops)) with Not_found -> None in
  let ctxt_result op = try Some (snd (SM.find op ops)) with Not_found -> None in

<estimate exec_counts for graph nodes 579d>≡ (578e)
  let exec_counts = G.fold_blocks (fun b map -> NM.add (GR.id b) 1.0 map) NM.empty g in
  let get_exec_count b = try NM.find (GR.id b) exec_counts
                        with Not_found -> Impossible.impossible "node not counted" in

<count uses in each context 579e>≡ (578e)
  let count (block : GR.block) =
    <define counting functions 579f>
    let rec count = function
      | GR.Tail (m, t) -> begin count_rtl (GR.mid_instr m); count t end
      | GR.Last l -> count_rtl (GR.last_instr l) in
    let (first, tail) = block in
    count tail in
  let () = G.iter_blocks count g in

<define counting functions 579f>≡ (579e)
  let rec count_loc loc ctxt = match loc with
  | RP.Mem (_, _, e, _) -> count_exp e (Some inc_addr)
  | RP.Slice (_, _, l) -> count_loc l ctxt
  | RP.Reg _ -> ()
  | RP.Global _ -> () (* cannot be placed based on context *)
  | RP.Var (_, i, _) ->
    (match ctxt with
     | None -> ())

```

```

    | Some c -> c i (get_exec_count block))

and count_exp exp ctxt = match exp with
| RP.Const _      -> ()
| RP.Fetch (l, _) -> count_loc l ctxt
| RP.App ((op, _), exps) ->
    let ctxts = match ctxt_parms op with
        | None      -> List.map (fun s -> None)    exps
        | Some cs -> List.map (fun c -> Some c) cs  in
    try List.iter2 count_exp exps ctxts
    with Invalid_argument _ -> Impossible.impossible "bad context in placevar" in

let count_effect effect = match effect with
| RP.Store (l, (RP.App ((op, _), _) as e), _) ->
    (count_exp e None ; count_loc l (ctxt_result op))
| RP.Store (l, e, _) -> (count_exp e None ; count_loc l None)
| RP.Kill   l      -> count_loc l None in

let count_guard (exp, effect) = (count_effect effect ; count_exp exp None) in

let rec count_rtl r = match Rtl.Dn.rtl r with
| RP.Rtl gs -> List.iter count_guard gs in

⟨choose a context for each variable 580a⟩≡ (578e)
let default_kind = "int" in
let ctxt_map =
    let elect i =
        let cnts = var_counts.(i) in
        let icnt = cnts.intc      in
        let fcnt = cnts.floatc   in
        let acnt = cnts.addrc     in
        let (cnt, ctxt) = if icnt >= fcnt then (icnt, "int") else (fcnt, "float") in
        if cnt >= acnt then ctxt else "address" in
    Array.init proc.vars elect in

⟨context replace var 580b⟩≡ (581a)
let formal_arr = Array.make proc.vars None in
let () = List.iter (fun (i,v) -> formal_arr.(i) <- Some v) formals in
let choose_ty i elected_kind =
    let space_name = function "signed" | "unsigned" | "" -> "int"
                        | n          -> n          in
match formal_arr.(i) with
| Some (Some ("address" as h), _, _, n, a) when elected_kind =?= "int" ->
    if tgt.Target.distinct_addr_sp then
        Impossible.unimp "Var placer does not distinguish addr and int spaces"
    else h, a
| Some (Some h, _, _, n, a) when Pervasives.<> (space_name h) elected_kind ->
    let votes = var_counts.(i) in
    begin
        (if not (elected_kind =?= default_kind) || votes.intc <> 0.0 then
            let error_str =
                ( Printf.sprintf
                    "Kind \"%s\" on formal parameter %s differs from inferred kind \"%s\":\n"
                    h n elected_kind
                ^ Printf.sprintf " {int: %f, float: %f, addr: %f}"
                    votes.intc votes.floatc votes.addrc) in
            Debug.eprintf "placevar" "%s" error_str;
            h, a
        end
    | _ -> elected_kind, None in

```

```

let get_placer i w =
  match tMap.(i) with
  | Some a -> a
  | None    -> let kind, aligned = choose_ty i ctxt_map.(i) in
                let alloc = A.allocate autmtn w kind (Auxfun.Option.get 1 aligned) in
                (changed := true;
                 tMap.(i) <- Some alloc;
                 alloc) in
let store_var exp (s, i, w) = (get_placer i w).A.store (R.Up.exp exp) w in
let fetch_var      (s, i, w) = (get_placer i w).A.fetch      w in
<replace variables with temps 581a>≡ (578e)
<context replace var 580b>
<fetch and store globals 578a>
let update rtl = replace_var store_var fetch_var store_global fetch_global rtl in
let g = G.map_rtls update g in
<verbosely dump tMap 581b>
<verbosely dump tMap 581b>≡ (581a)
let () =
  if Debug.on "placevar" then
    let loc = function
      | None -> "<??none??"
      | Some l -> RU.ToString.exp (l.A.fetch 99) in
    Printf.eprintf "Varmap: Var i -> Loc\n";
    Array.iteri (fun i l -> Printf.eprintf " %d -> %s\n" i (loc l)) tMap in
<placevar.ml 581c>+≡ (577a) <578e 581e>
let () = Debug.register "placevar" "variable placer"
<freeze autmtn and make sure its overflow block is empty 581d>≡ (578e)
let arestult = A.freeze autmtn in
let _ = if Block.size arestult.A.overflow <> 0 then
  Impossible.impossible "nonempty overflow from placing variables" in
<placevar.ml 581e>+≡ (577a) <581c 582a>
let ( *> ) = A.( *> )

let from_temps proc space = (* stage to allocate a temporary location *)
  let allocator = Talloc.Multiple.loc proc.Procedure.temps space in
  let alloc ~width:w ~alignment:a ~kind:h = A.of_loc (allocator w) in
  A.wrap (fun methods -> {A.allocate = alloc; A.freeze = methods.A.freeze})

let debug lbl w h a ctr =
  Printf.eprintf "Placevar %s w=%d h=%s a=%d ctr=%d\n" lbl w h a ctr

let mk_automaton ~warn ~vfp ~memspace mk_stage ((g, proc) : Ast2ir.proc) =
  let warn methods =
    let alloc ~width:w ~alignment:a ~kind:h =
      warn ~width:w ~alignment:a ~kind:h;
      methods.A.allocate w a h in
    {A.allocate = alloc; A.freeze = methods.A.freeze} in
  Block.srelative vfp "variables placed in memory" (A.at memspace)
  ( A.wrap warn *> mk_stage ~temps:(from_temps proc) *> A.as_stage proc.priv )
<add variable placements to spans 581f>≡ (578e)
let upd_var e = match R.Dn.loc e with
  | RP.Var (_,i,_) -> (match tMap.(i) with
    | None -> raise RT.DeadValue
    | Some l -> (Automatonutil.aloc 1 tgt.Target.wordsize))
  | _ -> e in
RT.upd_all_spans upd_var g;

```

```

⟨placevar.ml 582a⟩+≡ (577a) <581e
let replace_globals _ (g, (Procedure.{ target = tgt; global_map = gmap} as proc)) =
  let subst = RU.Subst.alloc ~guard:(function RP.Global _ -> true | _ -> false)
    ~map:(function RP.Global (_, i, _) -> gmap.(i)
      | 1 -> Impossible.impossible "global replacement") in
  let subst =
    if Debug.on "placevar" then
      (fun rtl ->
        let rtl' = subst rtl in
        Printf.eprintf "Placing vars maps\n %s\nto\n %s\n"
          (RU.ToString.rtl rtl) (RU.ToString.rtl rtl');
        rtl')
    else
      subst in
  (G.map_rtls subst g, proc), true (* probably changed :-) *)

```

T.12 layout/vfp.nw

```

⟨layout/vfprewrite.ml 582b⟩≡
⟨vfp.ml 582e⟩

⟨front_last/vfprewrite.mli 582c⟩≡
⟨vfp.mli 582d⟩

```

T.13 Virtual frame pointer

```

⟨vfp.mli 582d⟩≡ (582c)
(* claude: mk, mk_space and is_vfp are now in ir/vfp.ml *)
val replace_with : sp:Rtl.loc -> Zipcfg.graph -> Zipcfg.graph * bool

⟨vfp.ml 582e⟩≡ (582b) 582f▷
module D = Dataflow
module G = Zipcfg
module GR = Zipcfg.Rep
module P = Property
module R = Rtl
module RP = Rtl.Private
module RU = Rtlutil
module Dn = Rtl.Dn
module Up = Rtl.Up

let impossf fmt = Printf.kprintf Impossible.impossible fmt

(* claude: mk, is_vfp and mk_space moved to ir/vfp.ml, which needs only
 * Rtl/Cell/Space and is therefore visible to the modules that could not
 * reach this one - see the header there. What is left is the rewrite, which
 * genuinely needs Zipcfg and Dataflow.
 *)
let is_vfp = Vfp.is_vfp

⟨vfp.ml 582f⟩+≡ (582b) <582e 583▷
let unknown = max_int
let matcher = { P.embed = (fun a -> P.Vfp a);
  P.project = (function P.Vfp a -> Some a | _ -> None);
  P.is = (function P.Vfp a -> true | _ -> false);
}

```

```

let prop = Unique.Prop.prop matcher

let fact sp = {
  D.fact_name = "vfp location";
  D.init_info = (sp, unknown);
  D.add_info =
    (fun (vfp, k as a) (vfp', k' as a') ->
      let () = Debug.eprintf "vfp" "updating vfp dataflow fact\n" in
      if k = k' then a
      else if k = unknown then a'
      else if k' = unknown then a
      else (* accept inconsistency at exit and entry points to cut and unwind contn's *)
        if false (* G.kind node == G.Exit || G.is_non_local_join node *) then a
        else impossf "inconsistent stack-pointer location: %d and %d" k k');
  D.changed = (fun ~old:(_, k) ~new':(_, k') -> k <> k');
  D.prop = prop;
}

⟨vfp.ml 583⟩+≡ (582b) <582f 584b>
let replace_with ~sp =
  let w = RU.Width.loc sp in
  let spval = R.fetch sp w in
  let sp = Dn.loc sp in
  let sp_plus = RU.addk w spval in
  let sp_plus k = Dn.exp (sp_plus k) in
  ⟨supporting functions 584c⟩
  let fact = fact (Dn.exp spval) in
  let middle_out (vfp, k) m txlim =
    ⟨definition of simp, which is verbose 584a⟩
    let i = GR.mid_instr m in
    let down = Dn.rtl i in
    if RU.Exists.Loc.rtl is_vfp down then
      let simp = match m with GR.Stack_adjust _ -> simp | _ -> Simplify.rtl in
      let rtl = simp (replace_vfp vfp i) in
      D.Rewrite (G.single_middle (G.new_rtlm rtl m))
    else
      D.Dataflow (note_sp_changes down vfp k) in

let last_outs (vfp, k) l txlim =
  let upd vfp rtl = Simplify.rtl (replace_vfp vfp rtl) in
  let set_succs (vfp, k as a) =
    let upd_cedges ces =
      let upd ce = note_sp_changes (Dn.rtl ce.G.assertion) vfp k in
      D.Dataflow (fun set -> List.iter (fun ce -> set (fst ce.G.node) (upd ce))
        ces) in
    match l with
    | GR.Cut (_, ces, _) -> upd_cedges ces
    | GR.Call c -> upd_cedges c.GR.cal_contedges
    | _ -> D.Dataflow (fun set -> GR.iter_succs (fun u -> set u a) l) in
  let rewrite_assertions l fail =
    let upd_cedges vfp k cedges succ =
      if List.exists (fun ce -> RU.Exists.Loc.rtl is_vfp (Dn.rtl ce.G.assertion))
        cedges then
        succ (List.map (fun ce -> {ce with G.assertion = upd vfp ce.G.assertion})
          cedges)
      else fail () in
    match l with
    | GR.Cut (i, ces, r) ->
      let upd ces = D.Rewrite (G.single_last (GR.Cut (i, ces, r))) in

```



```

    upd_cedges vfp k ces upd
  | GR.Call c ->
    let upd ces =
      D.Rewrite (G.single_last (GR.Call { c with GR.cal_contedges = ces })) in
    let vfp, k = note_sp_changes (Dn.rtl (GR.last_instr l)) vfp k in
    upd_cedges vfp k c.GR.cal_contedges upd
  | _ -> fail () in
let i = GR.last_instr l in
let down = Dn.rtl (GR.last_instr l) in
let propagate () = match l with
  | GR.Cut _ -> set_succs (vfp, k)
  | _ -> set_succs (note_sp_changes down vfp k) in
if RU.Exists.Loc.rtl is_vfp down then
  let l = G.new_rtl1 (upd vfp i) l in
  rewrite_assertions l (fun () -> D.Rewrite (G.single_last l))
else rewrite_assertions l propagate in
let repl =
  { D.F.name = "replace vfp";
    D.F.middle_out = middle_out; D.F.last_outs = last_outs; } in
D.F.rewrite (fact, repl) ~entry_fact:(Dn.exp spval, 0)

```

<definition of simp, which is verbose 584a>≡ (583)

```

let simp rtl =
  let str = Rtlutil.ToString.rtl in
  let rtl' = Simplify.Unsafe.rtl rtl in
  Debug.eprintf "vfp"
    "Simplified stack adjustment from %s to %s\n" (str rtl) (str rtl');
  rtl' in

```

<vfp.ml 584b>+≡ (582b) <583 585>
 let () = Debug.register "vfp" "stack adjustments for virtual frame pointer"

<supporting functions 584c>≡ (583) 584d>

```

let replace_vfp value =
  let is_vfp = function
    | RP.Fetch (v, _) -> is_vfp v
    | _ -> false in
  RU.Subst.exp ~guard:is_vfp ~map:(fun _ -> value) in

```

<supporting functions 584d>+≡ (583) <584c

```

let rec note_sp_changes rtl vfp k =
  if RU.Exists.Loc.rtl (RU.Eq.loc sp) rtl then
    let k' = find_k'_added_to_sp rtl in
    let post_k = k - k' in
    sp_plus post_k, post_k
  else
    vfp, k
and find_k'_added_to_sp (RP.Rtl ges) =
  let rec find found k' = function
    | [] -> k' (* could be zero if only assignment is guarded *)
    | (RP.Const (RP.Bool b), RP.Store (sp', e, w)) :: ges when RU.Eq.loc sp sp' ->
      if not b then
        find found k' ges
      else if found then
        Impossible.impossible "multiple assignments to stack pointer"
      else
        (match e with
         | RP.App (("add", [_]), [RP.Fetch(sp', _); RP.Const (RP.Bits k')])
           when RU.Eq.loc sp' sp -> find true (Bits.S.to_int k') ges
         | RP.App (("add", [_]), [RP.Const (RP.Bits k'); RP.Fetch(sp', _)])

```

```

    when RU.Eq.loc sp' sp -> find true (Bits.S.to_int k') ges
  | RP.App (("sub", [_]), [RP.Fetch(sp', _); RP.Const (RP.Bits k')])
    when RU.Eq.loc sp' sp -> find true (- (Bits.S.to_int k')) ges
  | RP.Fetch (sp', _)
    when RU.Eq.loc sp' sp -> find true 0 ges
  | _ -> Printf.printf "illegal sp assignment: sp := %s\n" (RU.ToString.exp (Up.exp e)) ; 0)
(* RRO temp
    | _ -> Impossible.impossible ("sp assigned other than sp + k: " ^
      RU.ToString.exp (Up.exp e)))
*)
  | (g, RP.Store (sp', e, w)) :: ges when RU.Eq.loc sp sp' ->
    Impossible.impossible ("assigned sp with nontrivial guard " ^
      Rtlutil.ToString.exp (Rtl.Up.exp g))
  | (g, RP.Kill sp') :: ges when RU.Eq.loc sp sp' ->
    Impossible.impossible "killed sp"
  | _ :: ges -> find found k' ges in
find false 0 ges in

```

<vfp.ml 585>+≡

(582b) <584b

Appendix U

arch/dummy

U.1 arch/dummy/dummy.nw

`<arch/dummy/dummy.ml 586a>≡`
`<dummy.ml 586e>`

`<arch/dummy/dummy.mli 586b>≡`
`<dummy.mli 586c>`

U.2 The Dummy Target

`<dummy.mli 586c>≡` (586b) 586d▷

```
module type ARCH = sig
  val arch:          string (* name of this architecture *)
  val byte_order:    Rtl.aggregation
  val wordsize:      int
  val pointersize:   int
  val memsize:       int
end

module Make(A: ARCH): sig
  val target' :      Ast2ir.tgt
end
```

`<dummy.mli 586d>+≡` (586b) <586c

```
val dummy32l': Ast2ir.tgt
val dummy32b': Ast2ir.tgt
```

U.2.1 Implementation

`<dummy.ml 586e>≡` (586a)

```
module AN      = Automaton
module C       = Call
module PA      = Preast2ir
module R       = Rtl
module RP      = Rtl.Private
module RU      = Rtlutil
module Up      = Rtl.Up
module Dn      = Rtl.Dn
module SS32    = Space.Standard32
module T       = Target
module RS      = Register.Set
```

```

module type ARCH = sig
  val arch:      string (* name of this architecture *)
  val byte_order: Rtl.aggregation
  val wordsize:   int
  val pointersize: int
  val memsize:    int
end

let ( *> ) = AN.( *> )

(* to simplify the machine environment definition *)
let b i = Types.Bits i

module Make(A: ARCH) = struct
  let id          = Rtl.Identity
  let pointersize = A.pointersize
  let memsize     = A.memsize

  <module Spaces 589b>

  (* important registers *)
  let {SS32.cc = eflags} = SS32.locations Spaces.c
  let locations          = SS32.locations Spaces.c
  let pc                 = locations.SS32.pc
  let cc                 = locations.SS32.cc
  let npc                = locations.SS32.npc
  let fp_mode            = locations.SS32.fp_mode
  let fp_fcmp            = locations.SS32.fp_fcmp

  let rspace = ('r', Rtl.Identity, Cell.of_size A.wordsize)
  let fspace = ('f', Rtl.Identity, Cell.of_size A.wordsize)
  let mspace = ('m', A.byte_order, Cell.of_size 8)
  let reg n   = R.reg (rspace,n,R.C 1)
  let sp      = reg 31          (* stack pointer *)
  let ra      = reg 30          (* return address *)

  let fetch l   = R.fetch l   A.wordsize
  let store l e = R.store l e A.wordsize
  let assign    = store
  let return    = store pc (fetch ra)

  <global register allocation 589a>
  <module Flow 589c>
  <module Spill 590a>
  <module CC for calling conventions 590b>

  let ccspecs = {AN.call=CC.simple; AN.results=CC.simple; AN.cutto=CC.simple}
    (* the new-style target *)
  let fmach = Flow.machine sp
  let target' =
    let PA.T x86_tgt = X86.target in
    let spaces = [ Spaces.m
                  ; Spaces.c
                  ; Spaces.r
                  ; Spaces.f
                  ; Spaces.t
                  ; Spaces.u
                  ] in
    { T.name      = A.arch

```

```

; T.cc_specs      = [ "C--", ccspecs
;                  ; "C"  , ccspecs
;                  ]
; T.cc_spec_to_auto = (fun cname spec -> CC.conv "C--" false spec)
; T.is_instruction = (fun _ -> true)
; T.tx_ast = (fun secs -> secs)
(* Using the same M as x86 so that tests don't fail.
Not sure what the right M is for dummy. KR *)
; T.capabilities = x86_tgt.T.capabilities
; T.byteorder    = A.byte_order
; T.wordsize     = A.wordsize
; T.pointersize  = A.pointersize
; T.vfp          = SS32.vfp
; T.memspace     = mspace
; T.max_unaligned_load = R.C 1
; T.alignment    = 1
; T.memsize      = 8
; T.float        = Float.ieee754
; T.globals      = globals
; T.rounding_mode = fp_mode
; T.named_locs   = Strutil.assoc2map
;                  ["IEEE 754 rounding mode", fp_mode
;                  ; "IEEE 754 rounding results", fp_fcmp
;                  ]
; T.spaces       = spaces
; T.reg_ix_map   = T.mk_reg_ix_map spaces
; T.distinct_addr_sp = false
; T.machine = { T.bnegate    = fmach.T.bnegate
;               ; T.goto     = fmach.T.goto
;               ; T.jump     = fmach.T.jump
;               ; T.call     = fmach.T.call
;               ; T.cutto    = cutto
;               ; Mflow.return = fmach.Mflow.return
;               ; T.branch   = fmach.T.branch
;               ; T.move     = (fun _ -> assert false)
;               ; T.spill    = (fun _ -> assert false)
;               ; T.reload   = (fun _ -> assert false)
;               ; T.retgt_br = fmach.T.retgt_br
;               ; T.forbidden =
;                   Rtl.store (Rtl.mem Rtl.none mspace (Rtl.C 1)
;                             (Rtl.bits (Bits.zero 32) 32))
;                   (Rtl.bits (Bits.zero 8) 8) 8
;               }
(* bogus *)
; T.charset      = "latin1"
; T.data_section = "data"
}

let target' = PA.T target'
end

module Dummy32l = Make(struct
  let arch      = "dummy32l"
  let byte_order = Rtl.LittleEndian
  let wordsize  = 32
  let pointersize = 32
  let memsize   = 8
end)

module Dummy32b = Make(struct
  let arch      = "dummy32b"

```

```

let byte_order = Rtl.BigEndian
let wordsize    = 32
let pointersize = 32
let memsize     = 8
end)

```

```

let dummy32l' = Dummy32l.target'
let dummy32b' = Dummy32b.target'

```

U.2.2 Global Register Allocation

```

⟨global register allocation 589a⟩≡ (586e)
let globals base =
  let width w      = if w <= 8 then 8 else if w <= 16 then 16 else
    Auxfuns.round_up_to 32 w in
  AN.at ~start:base mspace
  (AN.widen width
   *> AN.align_to (function 8 -> 1 | 16 -> 2 | _ -> 4)
   *> AN.overflow ~growth:Memalloc.Down ~max_alignment:4)

```

U.2.3 Spaces

```

⟨module Spaces 589b⟩≡ (586e)
module Spaces = struct
  let m      = SS32.m A.byte_order [8; 16; 32]
  let c      = SS32.c 2 id [32]
  let r      = SS32.r 32 id [32]
  let f      = SS32.f 32 id [32]
  let t      = SS32.t id 32
  let u      = SS32.u id 32
end

```

U.2.4 Control Flow

```

⟨module Flow 589c⟩≡ (586e)
module Flow = Mflow.MakeStandard
  (struct
    let pc_lhs      = pc
    let pc_rhs      = pc
    let ra_reg      = R.reg (rspace, 30, R.C 1)
    let ra_offset    = 4 (* just guessing *)
  end)

let cutto =
  { T.embed      = (fun _ { Mflow.new_pc=newpc
    ; Mflow.new_sp=newsp
    } ->
    (Dag.Nop, Rtl.par [assign (reg 31) newsp; assign pc newpc]))
  ; T.project    = (fun r -> match Dn.rtl r with
    | RP.Rtl [ (_, RP.Store(_, nsp, _))
    ; (_, RP.Store(_, npc, _))] ->
    { Mflow.new_sp =Up.exp nsp
    ; Mflow.new_pc= Up.exp npc
    }
    | _ -> Impossible.impossible "projected non-cutto")
  }

```

U.2.5 Spills and Reloads

```
⟨module Spill 590a⟩≡ (586e)
module Spill = struct
  let assign lookup ~src ~dst =
    if Register.width src <> Register.width dst then
      Impossible.impossible "Dummy.assign source and destination width don't match"
    else
      let src_loc = Rtl.reg src in
      let dst_loc = Rtl.reg dst in
      Rtl.store dst_loc (Rtl.fetch src_loc (Register.width src)) (Register.width dst)

  let spill lookup reg loc =
    let w = Register.width reg in
    [Automaton.store loc (Rtl.fetch (Rtl.reg reg) w) w]

  let reload lookup reg loc =
    let w = Register.width reg in
    [Rtl.store (Rtl.reg reg) (Automaton.fetch loc w) w]
end
```

U.2.6 Calling Conventions

```
⟨module CC for calling conventions 590b⟩≡ (586e)
module CC = struct
  ⟨calling convention 590c⟩
end
```

U.2.7 New-Style Calling Convention

```
⟨calling convention 590c⟩≡ (590b) 590d▷
let reg n = (rspace, n, R.C 1)
let freg n = (fspace, n, R.C 1)
let vfp = Vfp.mk A.wordsize
let regs = List.map reg [0;1;2;3;4;5;6;7;8;9;10;11;12;13;14;15]
let fregs = List.map freg [0;1;2;3;4;5;6;7;8;9;10;11;12;13;14;15]

⟨calling convention 590d⟩+≡ (590b) <590c 590e▷
let volregs = RS.union (RS.of_list regs) (RS.of_list fregs)
let nvolregs = RS.empty

⟨calling convention 590e⟩+≡ (590b) <590d 590f▷
let spval = fetch sp
let sp_align = 4
let growth = Memalloc.Down
let std_sp_location = RU.add pointersize
  vfp (R.late "minus frame size" pointersize)

⟨calling convention 590f⟩+≡ (590b) <590e 591a▷
let simple =
  AN.choice
    [ AN.is_kind "float", AN.widen (Auxfuns.round_up_to ~multiple_of:32)
      *> AN.widths [32]
      *> AN.useregs fregs false
    ; AN.is_any, AN.widen (Auxfuns.round_up_to ~multiple_of:32)
      *> AN.widths [32]
      *> AN.useregs regs false
    ] *>
  AN.overflow ~growth:Memalloc.Down ~max_alignment:sp_align
```

```

<calling convention 591a>+≡ (590b) <590f 591b>
let return k n ~ra =
  if k = 0 && n = 0
  then store pc ra
  else Impossible.impossible "alternate return is unsupported"

<calling convention 591b>+≡ (590b) <591a 592a>
let saved_nvr temps =
  let t = Talloc.Multiple.loc temps 't' in
  let u = Talloc.Multiple.loc temps 'u' in
  function
  | (('r', _, _),_,_) as reg -> t (Register.width reg)
  | (('f', _, _),_,_) as reg -> u (Register.width reg)
  | ((s,_,_),i,_) ->
    Impossible.impossible (Printf.sprintf "cannot save $%c%d" s i)

<transformations(dummy.nw) 591c>≡ (592a)
let autoAt = AN.at mspace in
let call_actuals =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out call parms" autoAt specs.AN.call)
  ~autosp:(fun r -> Block.base r.AN.overflow)
  ~postsp:(fun _ sp -> sp) in

let prolog =
  let autosp = (fun _ -> vfp) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in call parms" autoAt specs.AN.call)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let call_results =
  let autosp = (fun r -> Block.base r.AN.overflow) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt specs.AN.results)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location) (* irrelevant? *)
  ~insp:(fun a _ _ -> autosp a) in

let epilg =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt specs.AN.results)
  ~autosp:(fun r -> Block.base r.AN.overflow)
  ~postsp:(fun _ _ -> vfp) (* irrelevant *) in

let also_cuts_to =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in cont parms" autoAt specs.AN.cutto)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let cut_actuals base =
  C.outgoing ~growth ~sp ~mkauto:(fun () -> autoAt base specs.AN.cutto)
  ~autosp:(fun r -> spval)
  ~postsp:(fun _ _ -> spval) in

```



```

<calling convention 592a>+≡ (590b) <591b 592b>
let conv name jump specs =
  <transformations(dummy.nw) 591c>
  { C.name           = name
  ; C.overflow_alloc = { C.parameter_deallocator = if jump then C.Callee else C.Caller
                        ; C.result_allocator      = C.Caller
                        }
  ; C.call_parms     = { C.in' = prolog;          C.out = call_actuals}
  ; C.cut_parms      = { C.in' = also_cuts_to; C.out = cut_actuals}
  ; C.results        = { C.in' = call_results; C.out = epilog}

  ; C.stack_growth   = Memalloc.Down
  ; C.stable_sp_loc   = std_sp_location
  ; C.replace_vfp     = Vfprewrite.replace_with ~sp
  ; C.jump_tgt_reg    = R.reg (reg 7)
  ; C.sp_align        = sp_align    (* alignment of stack pointer at call/cut *)
  ; C.pre_nvregs      = nvregs      (* registers preserved across calls *)
  ; C.volregs         = volregs     (* registers not preserved across calls *)
  ; C.saved_nvr       = saved_nvr
  ; C.return          = return
  ; C.ra_on_entry     = (fun _      -> R.fetch ra A.wordsize)
  ; C.where_to_save_ra = (fun _ t    -> Talloc.Multiple.loc t 't' A.wordsize)
  ; C.ra_on_exit      = (fun t b _ -> t)
  ; C.sp_on_unwind    = (fun e -> RU.store sp e)
  ; C.sp_on_jump      = (fun _ _ -> Rtl.null)
  }

```

```

<calling convention 592b>+≡ (590b) <592a>
let cmm = conv "C--" false
      { AN.call = simple ; AN.results = simple ; AN.cutto = simple }

```

U.3 arch/dummy/dummyexpander.nw

U.4 Code-Expander for the Dummy Target

U.4.1 Code Expansion with BURG

```

<dummyexpander.mlb 592c>≡
%head {:
  <utilities(dummyexpander.nw) 592d>
  :}
<terminal types(dummyexpander.nw) 594a>
%tail {:
  <tail(dummyexpander.nw) 598c>
  :}
%%
<rules(dummyexpander.nw) 594b>

```

U.4.2 Utilities for BURG actions

```

<utilities(dummyexpander.nw) 592d>≡ (592c) 593a>
module R = Rtl
module RP = Rtl.Private
module RU = Rtlutil
module S = Space
module T = Target

```

```

module C = Camlborg
module D = Rtl.Dn      (* Convert Down to private repr. *)
module U = Rtl.Up      (* Convert Up to abstract repr. *)
module W = Rtlutil.Width
module G = Cfg         (* for expanding all RTLs in a CFG *)
module GU = Cfgutil

⟨utilities(dummyexpander.nw) 593a⟩+≡ (592c) <592d 593b>
exception Error of string
let error msg = raise (Error msg)

⟨utilities(dummyexpander.nw) 593b⟩+≡ (592c) <593a 593c>
type state = R.rtl list * Talloc.Multiple.t
type 'a m = state -> ('a * state)

⟨utilities(dummyexpander.nw) 593c⟩+≡ (592c) <593b 593d>
let exec rtl = fun (rtls,tmp) -> (((), (rtl :: rtls,tmp)))

⟨utilities(dummyexpander.nw) 593d⟩+≡ (592c) <593c 593e>
let return a = fun s -> (a, s)
let (>=>) (m: 'a m) (f: 'a -> 'b m) = fun s -> let (a,s) = m s in f a s

⟨utilities(dummyexpander.nw) 593e⟩+≡ (592c) <593d 593f>
let dtmp = function
| 2 -> fun(rtls, tmp as state) -> (R.reg ('d', 1, 2), state)
| w -> Impossible.impossible
(Printf.sprintf "unsupported width for d-space register: %d" w)

let itmp = function
| 32 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 't' 32, state)
(*
| 8 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 'v' 8, state)
*)
| 8 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 't' 32, state)
| w -> Impossible.impossible
(Printf.sprintf "unsupported width for int register: %d" w)

let ftmp = function
| 64 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 'u' 32, state)
| 32 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 'u' 32, state)
| 8 -> fun (rtls,tmp as state) -> (Talloc.Multiple.loc tmp 'u' 32, state)
| w -> Impossible.impossible
(Printf.sprintf "unsupported width for float register: %d" w)

⟨utilities(dummyexpander.nw) 593f⟩+≡ (592c) <593e 593g>
module StringSet = Set.Make(struct type t = string let compare = compare end)

⟨utilities(dummyexpander.nw) 593g⟩+≡ (592c) <593f 593h>
let bool_ops2 = (* boolean valued primitive with non-boolean args *)
List.fold_right StringSet.add
["eq"; "ge"; "geu"; "gt"; "gtu"; "le"; "leu"; "lt"; "ltu"; "ne"]
StringSet.empty

let is_bool_op2 op = StringSet.mem op bool_ops2
let is_bool_op1 op = op = "bool"

⟨utilities(dummyexpander.nw) 593h⟩+≡ (592c) <593g 594d>
let fetch t = R.fetch t (W.loc t)

```

U.5 Code Expansion Rules

```

⟨terminal types(dummyexpander.nw) 594a⟩≡ (592c)
%term k width opr exps agg ass index bool bits link late x

⟨rules(dummyexpander.nw) 594b⟩≡ (592c) 594c▷
const:      Const(k)
            {: return (U.exp (RP.Const(k))) :}

-- r := k
ireg:      const [1]
            {: const                                >>= fun k ->
              itmp (W.exp k)                        >>= fun t ->
              exec (R.store t k (W.exp k))          >>= fun () ->
              return t
            :}

-- r := $m[addr]
ireg:      Fetch(mem,width) [1]
            {: mem                                >>= fun m ->
              itmp width                          >>= fun t ->
              exec (R.store t (R.fetch m width) width) >>= fun () ->
              return t
            :}

-- f := $m[addr]
freg:      Fetch(mem,width) [1]
            {: mem                                >>= fun m ->
              ftmp width                          >>= fun u ->
              exec (R.store u (R.fetch m width) width) >>= fun () ->
              return u
            :}

⟨rules(dummyexpander.nw) 594c⟩+≡ (592c) <594b 595a▷
ireg:      Fetch(icell,width) {: icell :}
freg:      Fetch(fcell,width) {: fcell :}
dreg:      Fetch(dcell,width) {: dcell :}

⟨utilities(dummyexpander.nw) 594d⟩+≡ (592c) <593h 596b▷
let bits2 n = R.bits (Bits.U.of_int n 2) 2

let app0 tmp s x =
  let opr  = R.opr s x in
  let exp  = R.app opr [] in
  let width = W.exp exp in
  tmp width >>= fun t ->
  (*
  exec (R.store t exp width) >>= fun () ->
  *)
  return t

let app1 tmp s x arg1 =
  arg1 >>= fun a1 ->
  let opr  = R.opr s x in
  let exp  = R.app opr [fetch a1] in
  let width = W.exp exp in
  tmp width >>= fun t ->
  exec (R.store t exp width) >>= fun () ->
  return t

```

```

let app2 tmp s x arg1 arg2 =
  arg1 >>= fun a1 -> arg2 >>= fun a2 ->
  let opr    = R.opr s x in
  let exp    = R.app opr [fetch a1; fetch a2] in
  let width  = W.exp exp in
  tmp width >>= fun t ->
  exec (R.store t exp width) >>= fun () ->
  return t

let app3 tmp s x arg1 arg2 arg3 =
  arg1 >>= fun a1 -> arg2 >>= fun a2 -> arg3 >>= fun a3 ->
  let opr    = R.opr s x in
  let exp    = R.app opr [fetch a1; fetch a2; fetch a3] in
  let width  = W.exp exp in
  tmp width >>= fun t ->
  exec (R.store t exp width) >>= fun () ->
  return t

```

$\langle \text{rules}(\text{dummyexpander.nw}) \text{ 595a} \rangle + \equiv \quad (592c) \triangleleft 594c \text{ 595b} \triangleright$

```

dreg: App0("float_eq", x)      {: app0 dtmp "float_eq" x :}
dreg: App0("float_gt", x)      {: app0 dtmp "float_gt" x :}
dreg: App0("float_lt", x)      {: app0 dtmp "float_lt" x :}
dreg: App0("unordered", x)     {: app0 dtmp "unordered" x :}

```

```

const: App0("round_down", x)    {: return (bits2 3) :}
const: App0("round_nearest", x) {: return (bits2 1) :}
const: App0("round_up", x)      {: return (bits2 2) :}
const: App0("round_zero", x)    {: return (bits2 0) :}

```

```

ireg: App1("com", x, r1:ireg)    {: app1 itmp "com" x r1 :}
ireg: App1("lobits",x,r1:ireg)   {: app1 itmp "lobits" x r1 :}
ireg: App1("neg", x, r1:ireg)    {: app1 itmp "neg" x r1 :}
ireg: App1("sx", x, r1:ireg)     {: app1 itmp "sx" x r1 :}
ireg: App1("zx", x, r1:ireg)     {: app1 itmp "zx" x r1 :}

```

$\langle \text{rules}(\text{dummyexpander.nw}) \text{ 595b} \rangle + \equiv \quad (592c) \triangleleft 595a \text{ 596a} \triangleright$

```

ireg: App2("add", x, r1:ireg, r2:ireg) {: app2 itmp "add" x r1 r2 :}
ireg: App2("and", x, r1:ireg, r2:ireg) {: app2 itmp "and" x r1 r2 :}
ireg: App2("div", x, r1:ireg, r2:ireg) {: app2 itmp "div" x r1 r2 :}
ireg: App2("divu", x, r1:ireg, r2:ireg) {: app2 itmp "divu" x r1 r2 :}
ireg: App2("mod", x, r1:ireg, r2:ireg) {: app2 itmp "mod" x r1 r2 :}
ireg: App2("modu", x, r1:ireg, r2:ireg) {: app2 itmp "modu" x r1 r2 :}
ireg: App2("mul", x, r1:ireg, r2:ireg) {: app2 itmp "mul" x r1 r2 :}
ireg: App2("or", x, r1:ireg, r2:ireg)  {: app2 itmp "or" x r1 r2 :}
ireg: App2("quot", x, r1:ireg, r2:ireg) {: app2 itmp "quot" x r1 r2 :}
ireg: App2("rem", x, r1:ireg, r2:ireg)  {: app2 itmp "rem" x r1 r2 :}
ireg: App2("rotl", x, r1:ireg, r2:ireg) {: app2 itmp "rotl" x r1 r2 :}
ireg: App2("rotr", x, r1:ireg, r2:ireg) {: app2 itmp "rotr" x r1 r2 :}
ireg: App2("shl", x, r1:ireg, r2:ireg)  {: app2 itmp "shl" x r1 r2 :}
ireg: App2("shra", x, r1:ireg, r2:ireg)  {: app2 itmp "shra" x r1 r2 :}
ireg: App2("shrl", x, r1:ireg, r2:ireg)  {: app2 itmp "shrl" x r1 r2 :}
ireg: App2("sub", x, r1:ireg, r2:ireg)  {: app2 itmp "sub" x r1 r2 :}
ireg: App2("xor", x, r1:ireg, r2:ireg)  {: app2 itmp "xor" x r1 r2 :}

```

```

freg: App1("fneg", x, r1:freg)      {: app1 ftmp "fneg" x r1 :}

```

```

freg: App2("f2i", x, r1:freg, r2:dreg) {: app2 itmp "f2i" x r1 r2 :}
freg: App2("i2f", x, r1:ireg, r2:dreg) {: app2 ftmp "i2f" x r1 r2 :}
freg: App2("f2f", x, r1:freg, r2:dreg) {: app2 ftmp "f2f" x r1 r2 :}

```

```

-- FP operations use a rounding mode
freg:  App3("fadd", x, r1:freg, r2:freg, c:dreg) {:app3 ftmp "fsub" x r1 r2 c:}
freg:  App3("fsub", x, r1:freg, r2:freg, c:dreg) {:app3 ftmp "fadd" x r1 r2 c:}
freg:  App3("fdiv", x, r1:freg, r2:freg, c:dreg) {:app3 ftmp "fdiv" x r1 r2 c:}
freg:  App3("fmul", x, r1:freg, r2:freg, c:dreg) {:app3 ftmp "fmul" x r1 r2 c:}

⟨rules(dummyexpander.nw) 596a⟩+≡ (592c) <595b 596c>
--
-- Floating point comparison results are ALWAYS stored in a hardware
-- register. This is a hack.
--

dreg:  App2("fcmp", x, f1:freg, f2:freg) [1]
      {:
        f1 >>= fun f1 -> f2 >>= fun f2 ->
        let o      = R.opr "fcmp" x in
        let exp     = R.app o [fetch f1; fetch f2] in
        let fpcond n = R.reg ('d', n, 2) in
        let r       = fpcond 1 in
        exec (R.store r exp 2) >>= fun () ->
        return r
      :}

⟨utilities(dummyexpander.nw) 596b⟩+≡ (592c) <594d 597a>
  let mem sp agg width addr ass=
    addr >>= fun a -> return (R.mem (U.assertion ass) sp agg width a)

  let reg sp i width = return (R.reg (sp, i, width))

⟨rules(dummyexpander.nw) 596c⟩+≡ (592c) <596a 596d>
  addr:  ireg  {: ireg >>= fun r -> return (fetch r) :}

⟨rules(dummyexpander.nw) 596d⟩+≡ (592c) <596c 596e>
  mem:  Mem('m', agg, width, addr, ass) {: mem 'm' agg width addr ass :}

⟨rules(dummyexpander.nw) 596e⟩+≡ (592c) <596d 596f>
  icell:  Reg('r', index, width) {: reg 'r' index width :}
  icell:  Reg('t', index, width) {: reg 't' index width :}
  icell:  Reg('g', index, width) {: reg 'g' index width :}
  icell:  Reg('v', index, width) {: reg 'v' index width :}
  fcell:  Reg('f', index, width) {: reg 'f' index width :}
  fcell:  Reg('u', index, width) {: reg 'u' index width :}
  dcell:  Reg('d', index, width) {: reg 'd' index width :}
  ccell:  Reg('c', index, width) {: reg 'c' index width :}

  error:  Reg(x:char, index, width)
          {: error (Printf.sprintf "register in space '%c'" x) :}

  error:  Mem(x:char, agg, width, addr, ass) [10]
          {: error (Printf.sprintf "cell in space '%c'" x) :}

⟨rules(dummyexpander.nw) 596f⟩+≡ (592c) <596e 597b>
  -- t := f
  ireg:  freg [10]
        {:
          freg >>= fun u ->
          let w = W.loc u in
          itmp w >>= fun t ->

```

```

    exec (R.store t (R.fetch u w) w)    >>= fun () ->
    return t
:}

```

```
-- f := t
```

```

freg:    ireg [10]
{:
    ireg                                >>= fun t ->
    let w = W.loc t in
    ftmp w                               >>= fun u ->
    exec (R.store u (R.fetch t w) w)    >>= fun () ->
    return u
:}

```

$\langle \text{utilities}(\text{dummyexpander.nw}) \text{ 597a} \rangle + \equiv \quad (592c) \triangleleft 596b$

```

let store left right width =
  left >>= fun l -> right >>= fun r ->
  return (R.store l (R.fetch r width) width)

```

```

let store' left right width = (* right is a value, not a location *)
  left >>= fun l -> right >>= fun r ->
  return (R.store l r width)

```

$\langle \text{rules}(\text{dummyexpander.nw}) \text{ 597b} \rangle + \equiv \quad (592c) \triangleleft 596f \text{ 597c} \triangleright$

```

stmt:    Store(mem,    ireg, width)      {: store mem    ireg  width :}
stmt:    Store(mem,    freg, width)      {: store mem    freg  width :}
stmt:    Store(icell,  ireg, width)      {: store icell  ireg  width :}
stmt:    Store(fcell,  freg, width)      {: store fcell  freg  width :}
stmt:    Store(ccell,  ireg, width) [1]  {: store ccell  ireg  width :}
stmt:    Store(dcell,  dreg, width)      {: store dcell  dreg  width :}
stmt:    Store(dcell,  const, width)     {: store' dcell const width :}
stmt:    Store(ccell,  const, width)     {: store' ccell const width :}

```

$\langle \text{rules}(\text{dummyexpander.nw}) \text{ 597c} \rangle + \equiv \quad (592c) \triangleleft 597b \text{ 598a} \triangleright$

```

guard:    True()  {: return (R.bool true)  :}
guard:    False() {: return (R.bool false) :}

guard:    App2(o:string, x, r1:ireg, r2:ireg)
    [{: if is_bool_op2 o then 0 else C.inf_cost :}]
{:
  r1 >>= fun r1 -> r2 >>= fun r2 ->
  let opr = R.opr o x in
  return (R.app opr [fetch r1; fetch r2])
:}

```

```

-- we need an extra rule for floating point results that always
-- stay in 'd' space.

```

```

guard:    App2(o:string, x, r1:dreg, r2:dreg)
    [{: if is_bool_op2 o then 0 else C.inf_cost :}]
{:
  r1 >>= fun r1 -> r2 >>= fun r2 ->
  let opr = R.opr o x in
  return (R.app opr [fetch r1; fetch r2])
:}

```

```

guard:    App1(o:string, x, r1:ireg)
    [{: if is_bool_op1 o then 0 else C.inf_cost :}]
{:
  r1 >>= fun r1 ->

```

```

        let opr = R.opr o x in
        return (R.app opr [fetch r1])
    :}

gstmt:    GStmt(guard,stmt)
    {:
        guard >>= fun g ->
        stmt  >>= fun s ->
        return (R.guard g s)
    :}

gstmts:   Nil() {: return [] :}
gstmts:   Cons(gstmt,gstmts)
    {: gstmt  >>= fun s  -> gstmts >>= fun ss -> return (s::ss) :}

⟨rules(dummyexpander.nw) 598a⟩+≡ (592c) <597c 598b⟩
    rtl:    Rtl(gstmts) [1]
    {:
        gstmts          >>= fun rtls ->
        exec (R.par rtls) >>= fun ()  ->
        return ()
    :}

⟨rules(dummyexpander.nw) 598b⟩+≡ (592c) <598a
    error:    Kill()  {: error "cannot handle Kill" :}
    error:    Var()   {: error "Var constructor" :}
    error:    Slice() {: error "Slice constructor" :}

```

U.5.1 Interfacing RTLs with the Expander

```

⟨tail(dummyexpander.nw) 598c⟩≡ (592c) 598d⟩
    let rec map f = function
        | []          -> conNil ()
        | x::xs       -> conCons (f x) (map f xs)

⟨tail(dummyexpander.nw) 598d⟩+≡ (592c) <598c 599a⟩
    let rec exp = function
        | RP.Const(RP.Bool(true))  -> conTrue()
        | RP.Const(RP.Bool(false)) -> conFalse()
        | RP.Const(k)              -> conConst(k)
        | RP.Fetch(l,w)            -> conFetch (loc l) w
        | RP.App((o,x),[])         -> conApp0 o x
        | RP.App((o,x),[e1])       -> conApp1 o x (exp e1)
        | RP.App((o,x),[e1;e2])    -> conApp2 o x (exp e1) (exp e2)
        | RP.App((o,x),[e1;e2;e3]) -> conApp3 o x (exp e1) (exp e2) (exp e3)
        | _                        -> error "application with too many args"

    and loc = function
        | RP.Mem(char, aff, w, e, ass) -> conMem char aff w (exp e) ass
        | RP.Reg(sp, i, w)              -> conReg sp i w
        | RP.Var(s, i, w)               -> conVar ()
        | RP.Slice(w,i,loc)             -> conSlice ()

    and stmt = function
        | RP.Store(l,e,w)              -> conStore (loc l) (exp e) w
        | RP.Kill(loc)                 -> conKill ()

    and guarded (e,eff)                = conGStmt (exp e) (stmt eff)

```

```

and rtl = function
  | RP.Rtl(gs)                -> conRtl (map guarded gs)
<tail(dummyexpander.nw) 599a>+≡ (592c) <598d 599b>
let expand tmps (r: R.rtl) =
  let plan = rtl (D.rtl r) in
  try
    match plan.rtl.Camlburg.action () ([],tmps) with
    | (), (rtls,_) -> rtls
  with
  Camlborg.Uncovered ->
    ( List.iter prerr_endline
      [ "cannot expand this RTL:"
        ; RU.ToUnreadableString.rtl r
      ]
    ; assert false
    )

```

U.5.2 Expanding an entire CFG

```

<tail(dummyexpander.nw) 599b>+≡ (592c) <599a 599c>
let cfg proc =
  let nodes      = GU.fold_bwd proc.Proc.cfg (fun n ns -> n::ns) []
  and tmps       = proc.Proc.temps (* source for temporaries *)
  and insert n i = G.gm_insert_assign_before i n
  and modified   = ref false in
  let expnd node =
    match expand tmps (G.instr node) with
    | last :: preds ->
      ( ignore (List.fold_left insert node preds)
        ; G.upd_instr node (fun _ -> last)
        ; modified := (preds <> [])
      )
    | [] -> assert false
  in
  ( List.iter expnd nodes
    ; !modified
  )

```

U.5.3 Exporting the Expander to Lua

```

<tail(dummyexpander.nw) 599c>+≡ (592c) <599b>
module Make (ProcT: Lua.Lib.TYPEVIEW with type 'a t = Proc.t):
  Lua.Lib.USERCODE with type 'a userdata' = 'a ProcT.combined =
struct
  type 'a userdata' = 'a ProcT.combined
  module M (C : Lua.Lib.CORE with type 'a V.userdata' = 'a userdata') =
  struct
    module V = C.V

    let ( **-> ) = V.( **-> )
    let proc     = ProcT.makemap V.userdata V.projection
    let init =
      C.register_module "Expander"
      [ "dummy", V.efunc (V.value **-> proc **-> V.result V.bool) (fun _ -> cfg)
      ]
  (* FIX -- want proper stage *)
  end (*M*)
end (*Make*)

```


Appendix V

arch/x86

V.1 arch/x86/x86.nw

$\langle \text{arch/x86/x86.ml } 600a \rangle \equiv$
 $\langle \text{x86.ml } 600d \rangle$

$\langle \text{arch/x86/x86.mli } 600b \rangle \equiv$
 $\langle \text{x86.mli } 600c \rangle$

V.2 Back end for a Intel x86 (32-bit subset)

$\langle \text{x86.mli } 600c \rangle \equiv$ (600b)
module X : Expander.S
val target : Ast2ir.tgt
val placevars : Ast2ir.proc -> Automaton.t

V.3 Implementation

$\langle \text{x86.ml } 600d \rangle \equiv$ (600a) 601a▷
open Nopoly

module A = Automaton
module C = Context
module DG = Dag
module PA = Preast2ir
module PX = Postexpander
module R = Rtl
module RP = Rtl.Private
module RU = Rtlutil
module Up = Rtl.Up
module Dn = Rtl.Dn
module Rg = X86regs
module RO = Rewrite.Ops
module T = Target

let impossf fmt = Printf.kprintf Impossible.impossible fmt
let unimpf fmt = Printf.kprintf Impossible.unimp fmt

```

<x86.ml 601a>+≡ (600a) <600d 601b>
(* pad: don't need lua *)
(*
let _ = Backplane.M.register "x86 invariant"
    ["The x86 machine invariant. For more explanation, see '<machine> invariant'."]
*)

```

V.3.1 Storage spaces

```

<x86.ml 601b>+≡ (600a) <601a 601c>
module SS = Space.Standard32
module Spaces = struct
  let bo = Rtl.LittleEndian
  let id = Rtl.Identity
  let m = SS.m bo [8; 16; 32]
  let r = SS.r 7 id [32]
  let t = SS.t id 32
  let c = SS.c 3 id [32]
end

```

V.3.2 Postexpander

```

<x86.ml 601c>+≡ (600a) <601b 601d>
let {SS.pc = pc; SS.cc = eflags} = SS.locations Spaces.c
let rounding_model = R.regx Rg.fpround
let rounding_mode = R.fetch rounding_model 2

```

```

<x86.ml 601d>+≡ (600a) <601c 601e>
let tempwidth = Register.width
let temploc t = R.reg t
let tempval t = R.fetch (temploc t) (tempwidth t)

```

```

<x86.ml 601e>+≡ (600a) <601d 602a>
let byte_order = Spaces.bo
let mcell = Cell.of_size 8
let mcount = Cell.to_count mcell
let mspace = ('m', byte_order, mcell)
let mem addr = R.mem R.none mspace (R.C 4) addr (* single word in memory *)
let exchange_alignment = 4 (* not required, but faster *)

let pc = pc
let esp = temploc Rg.esp
let espval = tempval Rg.esp
let add = Rtlutil.add 32
let sub x y = R0.sub 32 x y
let const n = R0.signed 32 n
let fetch l = R.fetch l 32
let store' l r = R.store l r 32
let pop_with f = (* rtl got by popping (f e), where e is top of stack *)
  let top = fetch (mem espval) in
  R.par [ f top; store' esp (add espval (const 4)) ]

let push' e = (* effect of pushing e *)
  let next_sp = sub espval (const 4) in
  R.par [ store' (mem next_sp) e; store' esp next_sp ]

```

```

⟨x86.ml 602a⟩+≡ (600a) <601e 602b>
module TY = Types
let (→) = TY.proc
module FS = Mflow.MakeStandard (
  struct
    let pc_lhs = pc
    let pc_rhs = pc
    let ra_reg =
      temploc (('?', Rtl.Identity, Cell.of_size 0), 99, R.C 1) (* not used *)
    let ra_offset = 33 (* not used *)
  end)
module F = struct
  include FS
  let fmach = FS.machine esp
  let call =
    { T.embed = (fun _ e -> (DG.Nop, R.par [R.store pc e 32; push' (fetch pc)]))
    ; T.project = (fun r -> match Dn.rtl r with
      | RP.Rtl [(_, RP.Store(_, e, _)); _, _] -> Up.exp e
      | _ -> Impossible.impossible (Printf.sprintf "projected non-call: %s"
        (RU.ToString.rtl r)))
    }
  let return = pop_with (fun ra -> store' pc ra)
end

⟨x86.ml 602b⟩+≡ (600a) <602a 612a>
module Post = struct
  ⟨x86 postexpander 602c⟩
end

⟨x86 postexpander 602c⟩≡ (602b) 602d>
type temp      = Register.t
type rtl       = Rtl.rtl
type address   = Rtl.exp
type width     = Rtl.width
type assertion = Rtl.assertion
type operator  = Rtl.Private.opr

⟨x86 postexpander 602d⟩+≡ (602b) <602c 602e>
let byte_order      = byte_order
let exchange_alignment = exchange_alignment

⟨x86 postexpander 602e⟩+≡ (602b) <602d 602f>
let talloc = Postexpander.Alloc.temp

⟨x86 postexpander 602f⟩+≡ (602b) <602e 602g>
let icontext = C.of_space Spaces.t
let itempwidth = 32
let fcontext = icontext (* oddity documented above *)
let acontext = icontext
let rcontext = icontext
let constant_context w = icontext
let overrides = []
let arg_contexts, result_context =
  C.functions (C.nonbool icontext fcontext rcontext overrides)

⟨x86 postexpander 602g⟩+≡ (602b) <602f 603a>
let (<:>) = DG.<:>
let rtl r = DG.Rtl r
let tempstore t e = rtl (R.store (temploc t) e (tempwidth t))

```

```

<x86 postexpander 603a>+≡ (602b) <602g 603b>
    let shift dst op n = tempstore dst (op 32 (tempval dst) (R0.unsigned 32 n))

<x86 postexpander 603b>+≡ (602b) <603a 603c>
    let eax = temploc Rg.eax
    let ecx = temploc Rg.ecx
    let edx = temploc Rg.edx
    let esp = temploc Rg.esp
    let esi = temploc Rg.esi
    let edi = temploc Rg.edi

    let ah_val = R.fetch Rg.ah 8
    let carrybit = R.app (R.opr "x86_carrybit" []) [RU.fetch eflags]

<x86 postexpander 603c>+≡ (602b) <603b 603d>
    let sahf = rtl (R.store eflags (R.app (R.opr "x86_ah2flags" []) [ah_val]) 16)
    let lahf = rtl (R.store Rg.ah (R.app (R.opr "x86_flags2ah" []) [R.fetch eflags 16]) 8)

<x86 postexpander 603d>+≡ (602b) <603c 603e>
    let c = carrybit
    let addflags x y w = R.store eflags (R.app (R.opr "x86_addflags" [w]) [x; y]) 32
    let adcflags x y w = R.store eflags (R.app (R.opr "x86_adcflags" [w]) [x; y; c]) 32
    let logicflags exp w = R.store eflags (R.app (R.opr "x86_logicflags" [w]) [exp]) 32
    let mulflags x y w = R.store eflags (R.app (R.opr "x86_mulflags" [w]) [x; y]) 32
    let muluxflags x y w = R.store eflags (R.app (R.opr "x86_muluxflags" [w]) [x; y]) 32
    let mulxflags x y w = R.store eflags (R.app (R.opr "x86_mulxflags" [w]) [x; y]) 32
    let negflags x w = R.store eflags (R.app (R.opr "x86_negflags" [w]) [x]) 32
    let subflags x y w = R.store eflags (R.app (R.opr "x86_subflags" [w]) [x; y]) 32
    let sbbflags x y w = R.store eflags (R.app (R.opr "x86_sbbflags" [w]) [x; y; c]) 32
    let undefflags = R.store eflags (R.app (R.opr "x86_undefflags" []) []) 32
    let shflags sh x y w =
        let flagsop = "x86_" ^ sh ^ "flags" in
        R.store eflags (R.app (R.opr flagsop [w]) [x; y]) 32

<x86 postexpander 603e>+≡ (602b) <603d 603f>
    let load ~dst ~addr assn =
        tempstore dst (R.fetch (R.mem assn mspace (R.C 4) addr) 32)
    let store ~addr ~src assn =
        rtl (R.store (R.mem assn mspace (R.C 4) addr) (tempval src) 32)

<x86 postexpander 603f>+≡ (602b) <603e 603g>
    let sxload ~dst ~addr n assn =
        tempstore dst (R0.sx n 32 (R.fetch (R.mem assn mspace (mcount n) addr) n))
    let zxload ~dst ~addr n assn =
        tempstore dst (R0.zx n 32 (R.fetch (R.mem assn mspace (mcount n) addr) n))
    let lostore ~addr ~src n assn =
        rtl (R.store eax (tempval src) 32) <:>
        rtl (R.store (R.mem assn mspace (mcount n) addr) (R0.lobits 32 n (R.fetch eax n)) n)

<x86 postexpander 603g>+≡ (602b) <603f 603h>
    let move ~dst ~src = if Register.eq dst src then DG.Nop else tempstore dst (tempval src)

<x86 postexpander 603h>+≡ (602b) <603g 604a>
    let extract ~dst ~lsb ~src =
        let w = tempwidth src in
        let n = tempwidth dst in
        let v = if lsb = 0 then tempval src else R0.shrl w (tempval src) (R0.unsigned w lsb) in
        tempstore dst (if n = w then v else R0.lobits w n v)
    let aggregate ~dst ~src = impossf "aggregate on x86"

```

```

<x86 postexpander 604a>+≡ (602b) <603h 604b>
  let li ~dst const = tempstore dst (Up.const const)
  let lix ~dst e = tempstore dst e

<x86 postexpander 604b>+≡ (602b) <604a 604c>
  let exp_with_flags setflags dst exp x y w =
    rtl (R.par [R.store (temploc dst) exp w; setflags x y w])

  let with_flags setflags dst op x y w =
    let x = tempval x in
    let y = tempval y in
    let exp = R.app op [x; y] in
    exp_with_flags setflags dst exp x y w

  let with_lflags dst op x y w =
    let exp = R.app op [x; y] in
    rtl (R.par [R.store (temploc dst) exp w; logicflags exp w])

<x86 postexpander 604c>+≡ (602b) <604b 604d>
  let at32 = function
    | opr, [32] -> opr
    | opr, [n] -> unimpf "operator %%%s%d(...)" opr n
    | opr, ws -> impossf "operator %%%s specialized to %d widths" opr (List.length ws)

<x86 postexpander 604d>+≡ (602b) <604c 605d>
  <definition of function llr 604e>
  let binop ~dst op x y = match at32 op with
  <cases in binop to handle integer division 604f>
  | _ -> move dst x <:> llr op dst y

<definition of function llr 604e>≡ (604d)
  let llr op x y = match at32 op with
  | "add" -> with_flags addflags x (Up.opr op) x y 32
  | "sub" -> with_flags subflags x (Up.opr op) x y 32
  | "mul" -> with_flags mulflags x (Up.opr op) x y 32
  <more cases for llr 605a>

<cases in binop to handle integer division 604f>≡ (604d)
  | "mod" ->
    PX.Expand.block (tempstore dst (Rewrite.(mod) 32 (tempval x) (tempval y)))
  | ("div" | "divu" | "quot" | "modu" | "rem") as opname ->
    let unsigned = opname =~= "divu" || opname =~= "modu" in
    let w = 32 in
    let sethi hreg lreg = (* set hreg to make divide come out right *)
      if unsigned then
        tempstore hreg (R0.unsigned w 0)
      else
        tempstore hreg (tempval lreg) <:> shift hreg R0.shra (w-1) in
    let regpair = Rewrite.regpair ~hi:(tempval Rg.edx) ~lo:(tempval Rg.eax) in
    let q, r = if unsigned then "divu", "modu" else "quot", "rem" in
    let div = R.par [R.store eax (R.app (R.opr q [w]) [regpair; tempval y]) w;
      R.store edx (R.app (R.opr r [w]) [regpair; tempval y]) w;
      undefflags] in
    let finish = match opname with
    | "quot" | "divu" -> move dst Rg.eax
    | "rem" | "modu" -> move dst Rg.edx
    | "div" -> let d = Rewrite.div' w ~dst:(R.reg dst) (tempval x) (tempval y)
      ~quot:(tempval Rg.eax) ~rem:(tempval Rg.edx) in
      PX.Expand.block d
    | _ -> impossf "division operator %%%s?" opname in
    move Rg.eax x <:> sethi Rg.edx Rg.eax <:> rtl div <:> finish

```

```

⟨more cases for llr 605a⟩≡ (604e) 605b▷
  | ("shl" | "shrl" | "shra" | "rotl" | "rotr") as shop ->
    let cl = R0.zx 8 32 (R.fetch Rg.cl 8) in
    let sh = exp_with_flags (shflags shop) x (R.app (Up.opr op) [tempval x; cl])
      (tempval x) cl 32 in
    move Rg.ecx y <:> sh

⟨more cases for llr 605b⟩+≡ (604e) <605a 605c>
  | ("and"|"or"|"xor") -> with_lflags x (Up.opr op) (tempval x) (tempval y) 32

⟨more cases for llr 605c⟩+≡ (604e) <605b
  | op -> impossf "non-binary operator %%%s in x86 binop" op

⟨x86 postexpander 605d⟩+≡ (602b) <604d 605e>
  let inplace op op32 x = match op32 with
  | "neg" -> rtl (R.par [R.store (temploc x) (R.app (Up.opr op) [tempval x]) 32;
    negflags (tempval x) 32])
  | "com" -> tempstore x (R.app (Up.opr op) [tempval x])
  | op -> impossf "non-unary operator %%%s in x86 unop" op

  let unop ~dst op x = match at32 op with
  | "popcnt" -> PX.Expand.block (Rewrite.popcnt 32 ~dst:(temploc dst) (tempval x))
  | op32 -> move dst x <:> inplace op op32 dst

⟨x86 postexpander 605e⟩+≡ (602b) <605d 605f>
  let unrm ~dst op x = impossf "operator %%%s in register" (fst op)
  let binrm ~dst op x y = impossf "operator %%%s in register" (fst op)

⟨x86 postexpander 605f⟩+≡ (602b) <605e 605g>
  let extended_multiply op y =
    let opfun, flagfun = match op with
    | "mulx" -> R0.mulx, mulxflags
    | "mulux" -> R0.mulux, muluxflags
    | _ -> impossf "non-multiplying dblop" in
    let product = opfun 32 (tempval Rg.eax) (tempval y) in
    rtl (R.par [R.store edx (Rewrite.slice 64 32 ~lsb:32 product) 32;
      R.store eax (Rewrite.slice 64 32 ~lsb:0 product) 32;
      flagfun (tempval Rg.eax) (tempval y) 32])

⟨x86 postexpander 605g⟩+≡ (602b) <605f 605h>
  let dblop ~dsthi ~dstlo op x y =
    move Rg.eax x <:>
    extended_multiply (at32 op) y <:>
    move dsthi Rg.edx <:>
    move dstlo Rg.eax

⟨x86 postexpander 605h⟩+≡ (602b) <605g 605i>
  let weird_tmp tmp z = match z with
  | PX.WTemp (Register.Reg t) -> t, DG.Nop
  | PX.WTemp (Register.Slice (w, 0, t)) -> t, DG.Nop
  | PX.WTemp (Register.Slice (w, lsb, t)) ->
    tmp, move tmp t <:> tempstore tmp (R0.shrl 32 (tempval tmp) (R0.unsigned 32 lsb))
  | PX.WBits b -> tmp, tempstore tmp (R.bits (Bits.Ops.zx 32 b) 32)

⟨x86 postexpander 605i⟩+≡ (602b) <605h 606a>
  let set_carry ~tmp z =
    let t, is = weird_tmp tmp z in
    let bit0 = R0.lobits 32 1 (tempval t) in
    is <:>
    rtl (R.store eflags (R.app (R.opr "x86_setcarry" []) [RU.fetch eflags; bit0]) 32)

```

```

⟨x86 postexpander 606a⟩+≡ (602b) <605i 606b>
let wrdop ~dst op x y z =
  let dv, yv = tempval dst, tempval y in
  let is_add =
    match at32 op with "addc" -> true | "subb" -> false | _ -> impossf "wrdop" in
  let flags = (if is_add then adcflags else sbbflags) dv yv 32 in
  let arith = R.store (temploc dst) (R.app (Up.opr op) [dv; yv; carrybit]) 32 in
  set_carry ~tmp:dst z <:> move dst x <:> rtl (R.par [arith; flags])

⟨x86 postexpander 606b⟩+≡ (602b) <606a 606c>
let wrdrop ~dst:(fill, dt) op x y z =
  let is_add =
    match at32 op with "carry" -> true | "borrow" -> false | _ -> impossf "wrdrop" in
  let t = talloc 't' 32 in
  wrdop ~dst:t ((if is_add then "addc" else "subb"), [32]) x y z <:> (* sets carry *)
  lahf <:> (* load ah from flags; carry is now lsb of ah *)
  shift Rg.eax R0.shrl 8 <:> (* shift carry into bit 0 of eax *)
  tempstore dt (R.fetch eax 32) <:>
  match fill with
  | PX.HighAny -> DG.Nop
  | PX.HighZ -> shift dt R0.shl 31 <:> shift dt R0.shrl 31 (* zxlo *)
  | PX.HighS -> shift dt R0.shl 31 <:> shift dt R0.shra 31 (* sxlo *)

⟨x86 postexpander 606c⟩+≡ (602b) <606b 606d>
let rmask_clear = R.bits (Bits.Ops.com (Bits.U.of_int 0xc00 32)) 32 (* 0xfffff3ff *)
let rmask_set = R.bits (Bits.U.of_int 0xc00 32) 32
let and32 = R.opr "and" [32]
let or32 = R.opr "or" [32]

⟨x86 postexpander 606d⟩+≡ (602b) <606c 606e>
let hwget ~dst:(fill,dt) ~src =
  if Register.eqx src Rg.fround then
    let slot = PX.Alloc.slot 16 2 in
    rtl (slot.A.store (tempval Rg.fpuctl) 16) <:> (* fnstcw *)
    tempstore dt (R0.zx 16 32 (slot.A.fetch 16)) <:> (* movzwl *)
    match fill with
    | PX.HighAny -> shift dt R0.shrl 10
    | PX.HighZ -> shift dt R0.shl 20 <:> shift dt R0.shrl 30
    | PX.HighS -> shift dt R0.shl 20 <:> shift dt R0.shra 30
  else
    impossf "getting unexposed hardware register"

⟨x86 postexpander 606e⟩+≡ (602b) <606d 606f>
let hwset ~dst ~src =
  if Register.eqx dst Rg.fround then
    let slot = PX.Alloc.slot 16 2 in
    let t = PX.Alloc.temp 't' 32 in
    let t2, ldsrc = weird_tmp (PX.Alloc.temp 't' 32) src in
    rtl (slot.A.store (tempval Rg.fpuctl) 16) <:> (* fnstcw *)
    tempstore t (R0.zx 16 32 (slot.A.fetch 16)) <:> (* movzwl *)
    with_lflags t and32 (tempval t) rmask_clear 32 <:> (* mask out round bits *)
    ldsrc <:>
    shift t2 R0.shl 10 <:>
    with_lflags t or32 (tempval t) (tempval t2) 32 <:>
    rtl (slot.A.store (R0.lobits 32 16 (tempval t)) 16) <:>
    tempstore Rg.fpuctl (slot.A.fetch 16) (* fldcw *)
  else
    impossf "setting unexposed hardware register"

⟨x86 postexpander 606f⟩+≡ (602b) <606e 607a>
let pc_lhs = pc
let pc_rhs = pc

```

```

⟨x86 postexpander 607a⟩+≡ (602b) <606f 607b>
  let br ~tgt = DG.Nop, R.store pc_lhs (tempval tgt) 32
  let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) 32

⟨x86 postexpander 607b⟩+≡ (602b) <607a 607c>
  let effects = List.map Up.effect
  let call ~tgt =
    DG.Nop, R.par [R.store pc_lhs (Up.const tgt) 32 ; push' (fetch pc)]
  let callr ~tgt =
    DG.Nop, R.par [R.store pc_lhs (tempval tgt) 32 ; push' (fetch pc)]

⟨x86 postexpander 607c⟩+≡ (602b) <607b 607d>
  let cut_to cut_args = F.fmach.T.cutto.T.embed () cut_args

⟨x86 postexpander 607d⟩+≡ (602b) <607c 607e>
  let return = F.return

  let interrupt n = (* very approximate *)
    let handler = R.app (R.opr "x86_idt_pc" []) [R0.unsigned 32 n] in
    R.par [store' pc handler; push' (R.fetch pc 32)]
  let () = Rtlop.add_operator ~name:"x86_idt_pc" ~result_is_float:false
    ([TY.fixbits 32] --> TY.fixbits 32)
  let forbidden = interrupt 3

⟨x86 postexpander 607e⟩+≡ (602b) <607d 607f>
  let don't_touch_me _ = false

⟨x86 postexpander 607f⟩+≡ (602b) <607e 609a>
  ⟨important functions on x86 operators 608a⟩
  let cmp x y =
    R.store eflags (R.app (R.opr "x86_subflags" [32]) [tempval x; tempval y]) 32

  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt 32)
  let bc_x86_cond x86opname = R.app (R.opr x86opname [32]) [R.fetch eflags 32]
  let bc_x86 x86opname ~ifso ~ifnot = (brtl (bc_x86_cond x86opname), ifso, ifnot)

  let bc x (opr, ws) y ~ifso ~ifnot =
    assert (ws == [32]);
    match opr with
    | "div_overflows" | "quot_overflows" ->
      let ov = Rewrite.div_overflows 32 (tempval x) (tempval y) in
      (*PX.Expand.cbranch DG.Test (DG.Nop, (ov, ifso, ifnot))*
      PX.Expand.cbranch' ov ifso ifnot
    | _ -> DG.Test (cmpfun opr x y, bc_x86 (cmpopr opr) ifso ifnot)
  let bc_guard x (opr, ws) y =
    assert (ws == [32]);
    match opr with
    | "div_overflows" | "quot_overflows" ->
      (DG.Nop, R.app (R.opr opr ws) [tempval x; tempval y])
    | _ -> (cmpfun opr x y, bc_x86_cond (cmpopr opr))
  let bc_of_guard (setup, guard) ~ifso ~ifnot =
    match Dn.exp guard with
    | RP.App (((("div_overflows" | "quot_overflows"), _), [x;y]) ->
      let ov = Rewrite.div_overflows 32 (Up.exp x) (Up.exp y) in
      PX.Expand.cbranch' ov ifso ifnot
    | _ -> DG.Test (setup, (brtl guard, ifso, ifnot))

  let bnegate r = match Dn.rtl r with
  | RP.Rtl [RP.App((cop, [32]), [RP.Fetch (flags, 32)]), RP.Store (pc, tgt, 32)]
    when RU.Eq.loc pc (Dn.loc pc_lhs) && RU.Eq.loc flags (Dn.loc eflags) ->

```



```

Up.rtl (RP.Rtl [RP.App((cmpneg cop, [32]), [RP.Fetch (flags, 32)]),
RP.Store (pc, tgt, 32)])
| _ -> impossf "ill-formed x86 conditional branch"

```

⟨important functions on x86 operators 608a⟩≡ (607f) 608b▷

```

let cmpopr = function
| "eq" -> "x86_e"
| "ne" -> "x86_ne"
| "lt" -> "x86_l"
| "le" -> "x86_le"
| "gt" -> "x86_g"
| "ge" -> "x86_ge"
| "ltu" -> "x86_b"
| "leu" -> "x86_be"
| "gtu" -> "x86_a"
| "geu" -> "x86_ae"
| "add_overflows"
| "div_overflows"
| "mul_overflows"
| "mulu_overflows"
| "sub_overflows" -> "x86_o"
| _ -> impossf "non-comparison in x86 conditional branch"

```

⟨important functions on x86 operators 608b⟩+≡ (607f) <608a 608c>

```

let cmpneg = function
| "x86_ne" -> "x86_e"
| "x86_e" -> "x86_ne"
| "x86_ge" -> "x86_l"
| "x86_g" -> "x86_le"
| "x86_le" -> "x86_g"
| "x86_l" -> "x86_ge"
| "x86_ae" -> "x86_b"
| "x86_a" -> "x86_be"
| "x86_be" -> "x86_a"
| "x86_b" -> "x86_ae"
| "x86_np" -> "x86_p"
| "x86_p" -> "x86_np"
| "x86_nc" -> "x86_c"
| "x86_c" -> "x86_nc"
| "x86_nz" -> "x86_z"
| "x86_z" -> "x86_nz"
| "x86_o" -> "x86_no"
| "x86_no" -> "x86_o"
| _ -> impossf "bad comparison in expanded x86 conditional branch"

```

⟨important functions on x86 operators 608c⟩+≡ (607f) <608b>

```

let cmpfun =
let set_flags op x y = binop (talloc 't' 32) (op, [32]) x y
and cmp x y = rtl (subflags (tempval x) (tempval y) 32) in
function
| "eq"
| "ne"
| "lt"
| "le"
| "gt"
| "ge"
| "ltu"
| "leu"
| "gtu"
| "geu" -> cmp

```

```

| "add_overflows" -> set_flags "add"
| "mul_overflows" -> set_flags "mul"
| "mulu_overflows" ->
  (fun x y ->
    if Register.eq Rg.eax y then
      extended_multiply "mulux" x
    else
      move Rg.eax x <:> extended_multiply "mulux" y)
| "sub_overflows" -> set_flags "sub"
| ("div_overflows" | "quot_overflows" as o) ->
  impossf "operator %%%s not rewritten in x86 postexpander" o
| o -> impossf "non-comparison %%%s in x86 conditional branch" o

```

V.3.3 Stack operations

```

⟨x86 postexpander 609a⟩+≡ (602b) <607f 609b>
let opclass (op, _) =
  match op with
  | "i2f" | "f2f" | "f2f_implicit_round" | "f2i" | "fabs" | "fneg" | "fsqrt" ->
    PX.Stack(PX.LeftFirst, 1)
  | "fadd" | "fcmp" | "fdiv" | "fmul" | "fmulx" | "fsub"
  | "feq" | "fge" | "fgt" | "fne"
  | "fordered" | "funordered" -> PX.Stack(PX.RightFirst, 2)
  | "fle" | "flt" -> PX.Stack(PX.LeftFirst, 2) (* computed by swapping *)
  | _ ->
    PX.Register

```

```

let converts_stack_to_temp (op, _) = op =~= "f2i" || op =~= "f2f"

```

```

⟨x86 postexpander 609b⟩+≡ (602b) <609a 609c>
let stspace = ('F', Rtl.Identity, Cell.of_size 3)
let fspace = ('f', Rtl.Identity, Cell.of_size 80)
let st = temploc (stspace, 0, R.C 1)
let stval = R.fetch st 3
let freg_at a = R.mem R.none fspace (R.C 1) a
let streg = freg_at stval
let stadd = Rtlutil.addk 3 stval
let stsub n = R0.sub 3 stval (R0.unsigned 3 n)
let stregplus n = freg_at (stadd n)
let ffetch l = R.fetch l 80

```

```

⟨x86 postexpander 609c⟩+≡ (602b) <609b 610a>
let st_pop_with f = (* rtl got by popping (f e), where e is top of floating-pt stack *)
  let top = ffetch streg in
  R.par [ f top; R.store st (stadd 1) 3 ]

let st_push e = (* effect of pushing e *)
  let next_st = stsub 1 in
  R.par [ R.store (freg_at next_st) e 80; R.store st next_st 3 ]

let stack_depth = 8
let stack_width = 80
let stack_top_proxy_reg = X86call.stack_top_proxy_reg
let stack_top_proxy = R.reg stack_top_proxy_reg
let is_stack_top_proxy = function
  | RP.Reg r -> Register.eq r stack_top_proxy_reg
  | _ -> false

```

```

<x86 postexpander 610a>+≡ (602b) <609c 610b>
let push ~addr assn = impossf "push 80-bit float without conversion"

let store_pop ~addr assn =
  let store regval = R.store (R.mem assn mspace (mcount 80) addr) regval 80 in
  rtl (st_pop_with store) (* bound to break *)

let push_cvt op w ~addr assn =
  assert (Cell.divides mcell w);
  let memval = R.fetch (R.mem assn mspace (mcount w) addr) w in
  let regval = R.app (Up.opr op) [memval; rounding_mode] in
  rtl (st_push regval)

let store_pop_cvt op w ~addr assn =
  assert (Cell.divides mcell w);
  let memval regval = R.app (Up.opr op) [regval; rounding_mode] in
  let store regval = R.store (R.mem assn mspace (mcount w) addr) (memval regval) w in
  rtl (st_pop_with store)

<x86 postexpander 610b>+≡ (602b) <610a 610c>
module SlotTemp = struct
  let unimpf fmt = Printf.kprintf Impossible.unimpf fmt
  let is _ = false
  let push src = impossf "push 80-bit float without conversion"
  let store_pop dst = unimpf "store-pop"
  let push_cvt op w src = unimpf "push-cvt"
  let store_pop_cvt op w dst = unimpf "store-pop-cvt"
  let push_cvt_rm op rm w src = unimpf "push-cvt"
  let store_pop_cvt_rm op rm w dst = unimpf "store-pop-cvt"
end

<x86 postexpander 610c>+≡ (602b) <610b 610d>
let pushk _ = impossf "load 80-bit floating-point constant"
let pushk_cvt _ _ _ = Impossible.unimpf "load floating-point constant"

<x86 postexpander 610d>+≡ (602b) <610c 610e>
let stack_op op = match opclass op with
| PX.Register -> impossf "passed register operator to stack_op"
| PX.Stack(first, depth) -> (* by lucky accident, depth is arity *)
  let positions = Auxfuns.from 0 (depth-1) in
  let positions = match first with PX.RightFirst -> positions
  | PX.LeftFirst -> List.rev positions in
  let argvals = List.map (fun n -> ffetch (stregplus n)) positions in
  let argvals = match op with
  | ("fadd"|"fsub"|"fdiv"|"fmul"|"f2f"|"f2i"|"i2f"), _ ->
    argvals @ [rounding_mode]
  | _ -> argvals in
  let result = R.app (Up.opr op) argvals in
  let next_st = stadd (depth-1) in
  if depth = 1 then
    rtl ( R.store (freg_at next_st) result 80)
  else
    rtl (R.par [ R.store (freg_at next_st) result 80; R.store st next_st 3])

<x86 postexpander 610e>+≡ (602b) <610d 611a>
let with_rm rm block = PX.with_hw ~hard:Rg.fpround ~soft:rm ~temp:(talloc 't' 32) block

let push_cvt_rm op rm w ~addr assn = with_rm rm (push_cvt op w ~addr assn)
let store_pop_cvt_rm op rm w ~addr assn =
  with_rm rm (store_pop_cvt op w ~addr assn)
let stack_op_rm op rm = with_rm rm (stack_op op)

```

```

<x86 postexpander 611a>+≡ (602b) <610e 611b>
let fpcmpopr = function
| "feq" -> "x86_z" (* AH is zero when xor'ed with the flag pattern for feq *)
| "fne" -> "x86_nz" (* AH is nonzero when xor'ed with the flag pattern for feq *)
| "flt" | "fgt" -> "x86_a" (* flt is swapped fgt *)
| "fle" | "fge" -> "x86_nc" (* fle is swapped fge *)
| "fordered" -> "x86_np"
| "funordered" -> "x86_p"
| _ -> impossf "floating-point comparison operator"

let bc_stack op ~ifso ~ifnot = match op with
| ("fordered"|"funordered"|"flt"|"fgt"|"fle"|"fge"|"feq"|"fne") as opname, [w] ->
  let positions = Auxfuns.from 0 1 in
  let argvals = List.map (fun n -> ffetch (stregplus n)) positions in
  let result = R.app (R.opr "x86_fcmp" [w]) argvals in
  let next_st = stadd 2 in
  let setcc = rtl (R.par [R.store Rg.fpcc result 2; R.store st next_st 3]) in
  let fstsw = rtl (R.store Rg.ax (R.fetch Rg.fpustatus 16) 16) in
  (match opname with
  | "feq" | "fne" ->
    let int8 = R0.unsigned 8 in
    let logic f l r = rtl (R.par [f l r 8; logicflags r 8]) in
    let clear_extra = logic R.store Rg.ah (R0._and 8 ah_val (int8 0x45)) in
    let xor_with_eq_pat = logic R.store Rg.ah (R0.xor 8 ah_val (int8 0x40)) in
    (* at this point, ZF = 0 iff the flags showed floating eq *)
    DG.Test (setcc <:> fstsw <:> clear_extra <:> xor_with_eq_pat,
             bc_x86 (fpcmpopr opname) ifso ifnot)
  | _ ->
    DG.Test (setcc <:> fstsw <:> sahf, bc_x86 (fpcmpopr opname) ifso ifnot))
| _ -> impossf "strange operator in x86 bc_stack"

<x86 postexpander 611b>+≡ (602b) <611a
let i2f_64_80 = ("i2f", [64; 80])
let f2i_80_64 = ("f2i", [80; 64])
let block_copy ~dst dassn ~src sassn w =
  match w with
  | 16 | 8 ->
    let t = talloc 't' 32 in
    zxload t src w sassn <:> lostore dst t w dassn
  | 32 ->
    let t = talloc 't' 32 in
    load t src sassn <:> store dst t dassn
  | 64 ->
    push_cvt i2f_64_80 64 src sassn <:> store_pop_cvt f2i_80_64 64 dst dassn
  | n ->
    let bytes = n / 8
    and const n = R0.signed 32 n
    and sub r c = R0.sub 32 (R.fetch r 32) c
    and byte_at r = R.mem R.none mspace (R.C 1) (R.fetch r 32) in
    let repmovsb =
      R.par [ R.store ecx (sub ecx (const 1)) 32
            ; R.store esi (sub esi (const bytes)) 32
            ; R.store edi (sub edi (const bytes)) 32
            ; R.store (byte_at esi) (R.fetch (byte_at edi) 8) 8
            ; R.guard (R.fetch ecx 32) (R.store pc (R.fetch pc 32) 32)
            ] in
    rtl (R.store esi src 32) <:>
    rtl (R.store edi dst 32) <:>
    rtl (R.store ecx (const bytes) 32) <:>
    rtl repmovsb

```

V.3.4 Building the target

```
<x86.ml 612a>+≡ (600a) <602b 612b>
module P = Post
module X = Expander.IntFloatAddr (Post)
(* expansion here once caused infinite loop *)
let spill_expand p r = [r]
let spill p t l =
  spill_expand p (Automaton.store l (tempval t) (tempwidth t))
let reload p t l =
  spill_expand p (R.store (temploc t) (Automaton.fetch l (tempwidth t)) (tempwidth t))

<x86.ml 612b>+≡ (600a) <612a 615a>
let downrtl = Dn.rtl
let uploc = Up.loc
let upexp = Up.exp

let ( *> ) = A.( *> )
let globals base =
  let width w = if w <= 8 then 8 else if w <= 16 then 16 else Auxfun.round_up_to 32 w in
  let align = function 8 -> 1 | 16 -> 2 | _ -> 4 in
  A.at mspace ~start:base
    (A.widen width *> A.align_to align *>
     A.overflow ~growth:Memalloc.Up ~max_alignment:4)

let fmach = F.machine esp
let tgt =
  let spaces = [Spaces.m; Spaces.r; Spaces.t; Spaces.c] in
  { T.name = "x86"
  ; T.memspace = mspace
  ; T.max_unaligned_load = R.C 4
  (* basic metrics and spaces are OK *)
  ; T.byteorder = P.byte_order
  ; T.wordsize = 32
  ; T.pointersize = 32
  ; T.vfp = SS.vfp
  ; T.alignment = 1
  ; T.memsize = Cell.to_width mcell (R.C 1)
  ; T.spaces = spaces
  ; T.reg_ix_map = T.mk_reg_ix_map spaces
  ; T.distinct_addr_sp = false

  (* Does the Pentium really implement IEEE 754? I think so *)
  ; T.float = Float.ieee754

  (* control flow is solid, except [[cutto]] is a lie *)
  ; T.machine = X.machine

  ; T.cc_specs = X86cc.cc_specs
  ; T.cc_spec_to_auto = X86call.cconv
    ~return_to:(fun ra -> pop_with (fun ra -> store' pc ra))
    { T.embed = fmach.T.cutto.T.embed
    ; T.project = fmach.T.cutto.T.project}

  ; T.is_instruction = X86rec.M.is_instruction
  ; T.tx_ast = (fun secs -> secs)
  ; T.capabilities = {T.memory = [8;16;32]; T.block_copy = true;
    T.items = [32]; T.ftemps = [32; 64];
    T.iwiden = true; T.fwiden = true;
    T.operators = List.map Up.opr [
```

```

    "sx",      [ 8; 32]
; "sx",      [ 1; 32]
; "sx",      [16; 32]
; "zx",      [ 1; 32]
; "zx",      [ 8; 32]
; "zx",      [16; 32]
; "lobits",  [32;  1]
; "lobits",  [32;  8]
; "lobits",  [32; 16]
; "lobits",  [32; 32]
; "add",     [32]
; "addc",    [32]
; "and",     [32]
; "borrow",  [32]
; "carry",   [32]
; "com",     [32]
; "div",     [32]
; "divu",    [32]
; "false",   []
; "mod",     [32]
; "modu",    [32]
; "mul",     [32]
; "mulx",    [32]
; "mulux",   [32]
; "neg",     [32]
; "or",      [32]
; "quot",    [32]
; "popcnt",  [32]
; "rem",     [32]
; "rotr",    [32]
; "rotr",    [32]
; "shl",     [32]
; "shra",    [32]
; "shrl",    [32]
; "sub",     [32]
; "subb",    [32]
; "true",    []
; "xor",     [32]
; "eq",      [32]
; "ge",      [32]
; "geu",     [32]
; "gt",      [32]
; "gtu",     [32]
; "le",      [32]
; "leu",     [32]
; "lt",      [32]
; "ltu",     [32]
; "ne",      [32]
; "fabs",    [32]
; "fadd",    [32]
; "fdiv",    [32]
; "feq",     [32]
; "fge",     [32]
; "fgt",     [32]
; "fle",     [32]
; "flt",     [32]
; "fne",     [32]
; "fordered", [32]
; "fmul",    [32]
; "fneg",    [32]

```

```

; "funordered", [32]
; "fsqrt", [32]
; "fsub", [32]
; "fabs", [64]
; "fadd", [64]
; "fdiv", [64]
; "feq", [64]
; "fge", [64]
; "fgt", [64]
; "fle", [64]
; "flt", [64]
; "fne", [64]
; "fordered", [64]
; "fmul", [64]
; "fneg", [64]
; "funordered", [64]
; "fsqrt", [64]
; "fsub", [64]
; "f2f", [32; 64]
; "f2f", [64; 32]
; "i2f", [32; 64]
; "f2i", [64; 32]
; "minf", [32] (* from simplifier *)
; "minf", [64] (* from simplifier *)
; "pinf", [32] (* from simplifier *)
; "pinf", [64] (* from simplifier *)
; "mzero", [32] (* from simplifier *)
; "mzero", [64] (* from simplifier *)
; "pzero", [32] (* from simplifier *)
; "pzero", [64] (* from simplifier *)
; "round_up", [] (* from simplifier *)
; "round_down", [] (* from simplifier *)
; "round_zero", [] (* from simplifier *)
; "round_nearest", [] (* from simplifier *)
; "NaN", [23; 32] (* from simplifier *)
; "add_overflows", [32]
; "div_overflows", [32]
; "mul_overflows", [32]
; "mulu_overflows", [32]
; "quot_overflows", [32]
; "sub_overflows", [32]
; "not", []
; "bool", []
; "disjoin", []
; "conjoin", []
; "bit", []
]; T.litops = List.map Up.opr [
  "NaN", [52; 64] (* from simplifier *)
];
T.literals = [32;64]}

```

```
(* global or hardware registers *)
```

```
; T.globals = globals
```

```
; T.rounding_mode = rounding_model
```

```
; T.named_locs = Strutil.assoc2map
```

```
  ["IEEE 754 rounding mode", rounding_model]
```

```
(* bogosity *)
```

```
; T.data_section = "data" (* NOT REALLY A PROPERTY OF THE TARGET MACHINE... *)
```

```
; T.charset = "latin1" (* THIS IS NONSENSE!! NOT A PROPERTY OF THE MACHINE?? *)
```

```

}
let target = PA.T tgt

type tgt = Ast2ir.tgt

```

V.3.5 Variable placement

```

⟨x86.ml 615a⟩+≡ (600a) <612b
let warning s = Printf.eprintf "backend warning: %s\n" s
let placevars =
  let is_float      w kind _ = w = 80 || (kind=$="float" && (w = 32 || w = 64)) in
  let strange_float w kind  = w = 80 && Pervasives.<> kind "float" in
  let strange_int   w kind  = kind=$= "float" && not (is_float w kind ()) in
  let warn ~width:w ~alignment:a ~kind:k =
    if strange_float w k then
      warning "80-bit variable not kinded float but will go as float anyway"
    else if strange_int w k then
      warning
        (Printf.sprintf "%d-bit variable kinded float but will go as integer" w) in
  let mk_stage ~temps =
    A.choice
      [ is_float,          A.widen (Auxfuns.round_up_to ~multiple_of: 32);
        (fun w k a -> w <= 32), A.widen (fun _ -> 32) *> temps 't';
        A.is_any,          A.widen (Auxfuns.round_up_to ~multiple_of: 8);
      ] in
  Placevar.mk_automaton ~warn ~vfp:tgt.T.vfp ~memspace:mspace mk_stage

```

V.4 Tentative example Lua code

```

⟨example Lua 615b⟩≡
X86.Regis = { r = Register.space('r', 8, 32), f = Register.space('f', 8, 80),
              c = Register.space('c', 4, 32) }
Register.addnames(X86.Regis, X86.Regis.r,
  { 'eax', 'ecx', 'edx', 'ebx', 'esp', 'ebp', 'esi', 'edi' })

X86.CC = Call.make(X86.target)

function X86.CC.results(rs)
  local CC = X86.CC
  return
    CC.choice ( 'float', { CC.widen(CC.multiple_of(80))
                          , CC.widths(80, "x86 FP too wide")
                          , CC.useregs(X86.Regis.f[0], "internal error")
                        },
    , {}, { CC.widen(CC.multiple_of(32))
          , CC.widths(32, "x86 return too wide")
          , CC.useregs(rs, "internal error")
        }
    )
end

function X86.setccs()
  local vfp = Vfp.mk(32)
  local R = X86.Regis
  local eax, ecx, edx, ebx, esi, edi, ebp =
    R.eax, R.ecx, R.edx, R.ebx, R.esi, R.edi, R.ebp

```



```

local allregs = {eax, ecx, edx, ebx, esi, edi, ebp}
local volregs = {eax, ecx, edx}
local nvregs  = {ebx, esi, edi, ebp}

local CC = X86.CC
local c_results  = CC.results(eax)
local cmm_results = CC.results(volregs)
local c_args     = { CC.widen(CC.multiple_of(32)), CC.overflow("up", 4)}
local cut_args   =
  { CC.widen(CC.multiple_of(32))
    , CC.widths(32, "at most 32 bits in cut-to argument")
    , CC.useregs(allregs)
    , CC.overflow("up", 4)
  }
local cmm_args =
  { CC.widen(CC.multiple_of(32))
    , CC.useregs(allregs)
    , CC.overflow("up", 4)
  }
end

```

V.5 arch/x86/x86all.nw

V.6 arch/x86/x86asm.nw

`<arch/x86/x86asm.ml 616a>≡`
`<x86asm.ml 616d>`

`<arch/x86/x86asm.mli 616b>≡`
`<x86asm.mli 616c>`

V.7 X86 Assembler

`<x86asm.mli 616c>≡` (616b)

```

val make :
  (('a, 'b, 'c, 'd) Procedure.t -> 'cfg -> (Zipcfg.Rep.call -> unit) -> (Rtl.rtl -> unit) -> (string -> unit)) ->
  out_channel -> ('cfg * ('a, 'b, 'c, 'd) Procedure.t) Asm.assembler
(* pass Cfgutil.emit *)

```

V.7.1 Implementation

`<x86asm.ml 616d>≡` (616a)

```

module G = Zipcfg
module GR = Zipcfg.Rep
module SM = Strutil.Map
<utilities(x86asm.nw) 616e>
<definitions(x86asm.nw) 616f>
let make emitter fd = new asm emitter fd

```

`<utilities(x86asm.nw) 616e>≡` (616d) 619a▷

```

let fprintf = Printf.fprintf

```

`<definitions(x86asm.nw) 616f>≡` (616d) 617b▷

<definition of manglespec (for the name mangler)(x86asm.nw) 617a>≡ (617b)

```
let spec =
  let reserved = [] in          (* list reserved words here so we can avoid them *)
  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '.'
    | '_'      -> true
    | _        -> false in
  let replace = function
    | x when id x -> x
    | _          -> '_'
  in
  { Mangler.preprocess = (fun x -> x)
    ; Mangler.replace   = replace
    ; Mangler.reserved  = reserved
    ; Mangler.avoid     = (fun x -> x ^ "_")
  }
```

<definitions(x86asm.nw) 617b>+≡ (616d) <616f

<definition of manglespec (for the name mangler)(x86asm.nw) 617a>

```
class ['cfg, 'a, 'b, 'c, 'd] asm emitter fd
: ['cfg * ('a, 'b, 'c, 'd) Procedure.t] Asm.assembler =
object (this)
  val _fd      = fd
  val _mangle  = (Mangler.mk spec)
  val mutable _syms = SM.empty
  method globals _ = ()
  method private new_symbol name =
    let s = Symbol.with_mangler _mangle name in
    _syms <- SM.add name s _syms;
    s

  <private assembly state(x86asm.nw) 618d>
  method private print l = List.iter (output_string _fd) l

  <assembly methods(x86asm.nw) 617c>
end
```

<assembly methods(x86asm.nw) 617c>≡ (617b) 617d▷

```
method import s = this#new_symbol s
```

<assembly methods(x86asm.nw) 617d>+≡ (617b) <617c 617e▷

```
method export s =
  let sym = this#new_symbol s in
  Printf.fprintf _fd ".globl %s\n" sym#mangled_text;
  sym
```

<assembly methods(x86asm.nw) 617e>+≡ (617b) <617d 618a▷

```
method local s = this#new_symbol s
```

<X86 assembly interface procedures 617f>≡ 618b▷

```
static AsmSymbol asm_common(name, size, align, section)
char *name; int size; int align; char *section;
{
  AsmSymbol s;
  assert(section == NULL);
  if (solaris)
```

```

    print(".common %s,%d,%d\n", name, size, align);
else
    print(".common %s,%d\n", name, size);
s = asm_sym_insert(asmtab, name, ASM_COMMON);
s->u.common.size = size;
s->u.common.align = align;
return s;
}

⟨assembly methods(x86asm.nw) 618a⟩+≡ (617b) <617e 618c>
method label (s: Symbol.t) = fprintf _fd "%s:\n" s#mangled_text

⟨X86 assembly interface procedures 618b⟩+≡ <617f
static void asm_function (name) char *name; {
    print(".type %s,@function", name);
}

⟨assembly methods(x86asm.nw) 618c⟩+≡ (617b) <618a 618e>
method section name =
    _section <- name;
(* claude: the pcmap sections must carry the ALLOC flag, or the data
 * ends up in the file but not in any PT_LOAD segment, so the run-time
 * system reads zeroes and every Cmm_lookup_entry fails. GNU as gives
 * default flags only to the standard section names; for others,
 * ".section .pcmap" alone yields a non-allocated section. Upstream's
 * own objects have the same problem - the 2004-era linker merged
 * pcmap.ld's ".rodata { *(.pcmap) }" into the real .rodata output
 * section, whereas a modern one creates a second, non-allocated
 * .rodata.
 *)
match name with
| "pcmap" | "pcmap_data" ->
    fprintf _fd ".section %s,\"a\",@progbits\n" name
| _ ->
    fprintf _fd ".section %s\n" name
method current = _section

⟨private assembly state(x86asm.nw) 618d⟩≡ (617b)
val mutable _section = "bogus section"

⟨assembly methods(x86asm.nw) 618e⟩+≡ (617b) <618c 618f>
method org n = Impossible.unimp "no .org in x86 assembler"

⟨assembly methods(x86asm.nw) 618f⟩+≡ (617b) <618e 618g>
method align n =
    if n <> 1 then fprintf _fd ".align %d\n" n
method addloc n =
    if n <> 0 then fprintf _fd ".skip %d\n" n

⟨assembly methods(x86asm.nw) 618g⟩+≡ (617b) <618f 618h>
method zeroes (n:int) = fprintf _fd ".skip %d, 0\n" n

⟨assembly methods(x86asm.nw) 618h⟩+≡ (617b) <618g 619b>
method value (v:Bits.bits) =
    let altfmt = Bits.to_hex_or_decimal_string ~declimit:256 in
    match Bits.width v with
    | 8 -> fprintf _fd ".byte %Ld\n" (Bits.S.to_int64 v)
    | 16 -> fprintf _fd ".word %s\n" (altfmt v)
    | 32 -> fprintf _fd ".long %s\n" (altfmt v)
    | 64 ->
        let i = Bits.U.to_int64 v in
        fprintf _fd ".long 0x%Lx\n" (Int64.logand i mask32);
        fprintf _fd ".long 0x%Lx\n" (Int64.shift_right_logical i 32)
    | w -> Impossible.unimp ("emission width " ^ string_of_int w ^ " in x86 assembler")

```

```

<utilities(x86asm.nw) 619a>+≡ (616d) <616e
  let mask32 = Int64.pred (Int64.shift_left Int64.one 32)

<assembly methods(x86asm.nw) 619b>+≡ (617b) <618h 619c>
  method addr a =
    match Reloc.if_bare a with
    | Some b -> this#value b
    | None -> let const bits = Printf.sprintf "0x%Lx" (Bits.U.to_int64 bits) in
               assert (Reloc.width a = 32);
               fprintf _fd ".long %s\n" (Asm.reloc_string const a)

<assembly methods(x86asm.nw) 619c>+≡ (617b) <619b 619d>
  method emit = ()

<assembly methods(x86asm.nw) 619d>+≡ (617b) <619c 619e>
  method comment s = fprintf _fd "/* %s */\n" s

  method const (s: Symbol.t) (b:Bits.bits) =
    fprintf _fd ".set %s, 0x%Lx" s#mangled_text (Bits.U.to_int64 b)

<assembly methods(x86asm.nw) 619e>+≡ (617b) <619d 619f>
  method longjmp_size () = 5
  val call_size = 4

<assembly methods(x86asm.nw) 619f>+≡ (617b) <619e 619g>
  method private instruction rtl =
    output_string _fd "\t";
    output_string _fd (X86rec.M.to_asm rtl);
    output_string _fd "\n"

<assembly methods(x86asm.nw) 619g>+≡ (617b) <619f
  method private call node =
    let longjmp edge = fprintf _fd "\t.byte 0xe9\n\t.long %s-.-%d\n"
                              (_mangle (snd edge.G.node)) call_size in
    let rec output_altret_jumps n edges = (* emit n jumps *)
        if n > 0 then
          match edges with
          | edge :: edges -> (longjmp edge; output_altret_jumps (n-1) edges)
          | [] -> Impossible.impossible "contedge count" in
    begin
      output_string _fd "\t";
      output_string _fd (X86rec.M.to_asm node.GR.cal_i);
      output_string _fd "\n";
      output_altret_jumps node.GR.cal_altrets (List.tl node.GR.cal_contedges);
    end

  method cfg_instr (cfg.proc) =
    (* We have to emit a label/whatever for the procedure's
       entry point. This is what symbol is for. Simply use this#label? *)
    let symbol = proc.Procedure.symbol in
    let label l = this#label (try SM.find l _syms with Not_found -> this#local l) in
    this#label symbol;
    (emitter proc cfg (this#call) (this#instruction) label : unit)

<unused utilities 619h>≡
  module Asm = X86rec.M

```

V.8 arch/x86/x86call.nw

```

<arch/x86/x86call.ml 619i>≡
  <x86call.ml 620d>

```

```

<arch/x86/x86call.mli 620a>≡
  <x86call.mli 620b>

```

V.9 X86 calling conventions

```

<x86call.mli 620b>≡ (620a) 620c>
  val cconv :
    return_to:(Rtl.exp -> Rtl.rtl) ->
    (unit, Mflow.cut_args) Target.map ->
    string -> Automaton.cc_spec ->
    Call.t

```

```

<x86call.mli 620c>+≡ (620a) <620b
  val stack_top_proxy_reg : Register.t

```

V.10 Implementation of X86 calling conventions

```

<x86call.ml 620d>≡ (619i) 620f>
  module A = Automaton
  module C = Call
  module R = Rtl
  module RO = Rewrite.Ops
  module RP = Rtl.Private
  module RS = Register.Set
  module RU = Rtlutil
  module T = Target
  let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

```

<x86 calling convention automata in Lua 620e>≡ 621a>
  A = Automaton
  X86 = X86 or {}
  X86.cc = X86.cc or {}
  X86.cc["C" ] = X86.cc["C" ] or {}
  X86.cc["C--" ] = X86.cc["C--" ] or {}
  X86.cc["notail"] = X86.cc["notail"] or {}
  X86.cc["gc"] = X86.cc["gc"] or {}
  X86.cc["C-- thread"] = X86.cc["C-- thread"] or {}

```

```

<x86call.ml 620f>+≡ (619i) <620d 620g>
  let rspace = X86regs.rspace
  let eax = X86regs.eax
  let ecx = X86regs.ecx
  let edx = X86regs.edx
  let ebx = X86regs.ebx
  let esp = X86regs.esp
  let ebp = X86regs.ebp
  let esi = X86regs.esi
  let edi = X86regs.edi
  let vfp = Vfp.mk 32

```

```

<x86call.ml 620g>+≡ (619i) <620f 620h>
  let volregs = RS.of_list [eax; ecx; edx]
  let nvregs = RS.of_list [ebx; esi; edi; ebp]
  let allregs = RS.union volregs nvregs

```

```

<x86call.ml 620h>+≡ (619i) <620g 621b>
  let saved_nvr temps =
    let t = Talloc.Multiple.loc temps 't' in fun r -> t (Register.width r)

```

```

<x86 calling convention automata in Lua 621a>+≡ <620e 621f>
X86.sp_align = 4
X86.wordsize = 32

function reg(sp, i, w)
    return { space = sp, cellsize = w, index = i, count = 1 }
end
function r(i) return reg("r", i, X86.wordsize) end

X86.eax = r(0) X86.ecx = r(1) X86.edx = r(2)
X86.ebx = r(3) X86.esp = r(4) X86.ebp = r(5)
X86.esi = r(6) X86.edi = r(7)

X86.stack_top_proxy_reg = reg("\0", 0, 80)
X86.all_regs = {X86.eax, X86.ecx, X86.edx, X86.ebx, X86.esi, X86.edi, X86.ebp}

<x86call.ml 621b>+≡ (619i) <620h 621c>
let sp = R.reg esp
let spval = R.fetch sp 32
let sp_align = 4
let growth = Memalloc.Down
let bo = R.LittleEndian
let memsize = 8

<x86call.ml 621c>+≡ (619i) <621b 621d>
let ( *> ) = A.( *> )

<x86call.ml 621d>+≡ (619i) <621c 621e>
let badwidth what w = impossf "Unsupported (rounded) width %d in x86 %s" w what
let imp _ = Impossible.impossible "grave miscalculation in automaton"

<x86call.ml 621e>+≡ (619i) <621d 622e>
let stack_top_proxy_reg = (('000', Rtl.Identity, Cell.of_size 80), 0, Rtl.C 1)

<x86 calling convention automata in Lua 621f>+≡ <621a 621g>
X86.cc["C"].results =
    A.choice { "float" , { A.widen(80)
                          , A.useregs { X86.stack_top_proxy_reg }
                          }
    , A.is_any, { A.widen(32, 'multiple')
                , A.useregs { X86.eax, X86.edx }
                }
    }

<x86 calling convention automata in Lua 621g>+≡ <621f 621h>
X86.cc["C"].call =
    { A.widen(32, "multiple")
    , A.overflow { growth = "up", max_alignment = X86.sp_align }
    }

<x86 calling convention automata in Lua 621h>+≡ <621g 622a>
X86.cc["C"].cutto =
    { A.widen(32)
    , A.useregs({X86.edx, X86.ebx, X86.esi, X86.edi, X86.ebp})
    , A.overflow { growth = "up", max_alignment = X86.sp_align }
    }

```

```

<x86 calling convention automata in Lua 622a>+≡ <621h 622b>
-- for now it appears that the C-- results automaton looks like the C one
-- with an overflow stage tagged on the end (although ecx was
-- commented out as an additional results register)
X86.cc["C--"].results =
  { X86.cc["C"].results
    , A.overflow { growth = "down", max_alignment = X86.sp_align }
  }

X86.cc["C--"].call =
  { A.widen(32, "multiple")
    , A.useregs { X86.eax }
    , A.overflow { growth = "up", max_alignment = X86.sp_align }
  }
-- all the calling conventions share the same cutto automaton specs.
X86.cc["C--"].cutto = X86.cc["C"].cutto

<x86 calling convention automata in Lua 622b>+≡ <622a 622c>
X86.cc["C-- thread"].results = { }
X86.cc["C-- thread"].call = A.overflow { growth = "up", max_alignment = 4 }
X86.cc["C-- thread"].cutto = { }

<x86 calling convention automata in Lua 622c>+≡ <622b 622d>
X86.cc["gc"].results =
  { A.choice { "float" , { A.widen(80)
                          , A.useregs { X86.stack_top_proxy_reg }
                        }
    , A.is_any, { A.widen(32, 'multiple')
                , A.useregs { X86.eax }
                }
  }
  , A.overflow { growth = "down", max_alignment = X86.sp_align }
}

X86.cc["gc"].call =
  { A.widen(32, "multiple")
    , A.overflow { growth = "up", max_alignment = X86.sp_align }
  }

X86.cc["gc"].cutto =
  { A.widen(32)
    , A.overflow { growth = "up", max_alignment = X86.sp_align }
  }

X86.layout["gc"] = X86.layout["C"]

<x86 calling convention automata in Lua 622d>+≡ <622c 625>
-- uncomment the next two lines to make "gc" a synonym for "C--"
-- X86.cc["gc"] = X86.cc["C"]
-- X86.layout["gc"] = X86.layout["C"]

<x86call.ml 622e>+≡ (619i) <621e 622f>
let mspace = ('m', bo, Cell.of_size 8)
let amem = R.mem (R.aligned 4) mspace (R.C 4)
let ra = amem vfp
let const x _ = x

<x86call.ml 622f>+≡ (619i) <622e 624a>
let addk = RU.addk 32
let add = RU.add 32
let std_sp_location = add vfp (R.late "minus frame size" 32)

```

```

⟨functions to transform automata 623a⟩≡ (624a) 623b▷
  let autoAt = A.at mspace in
  let call_actuals =
    C.outgoing ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "out call parms" autoAt specs.A.call)
    ~autosp:(fun r -> Block.base r.A.overflow)
    ~postsp:(fun _ sp -> sp) in (* was ~postsp:(fun _ -> std_sp_location) *)
  let prolog =
    let autosp = (fun _ -> vfp) in
    C.incoming ~growth ~sp
    ~mkauto:(fun () -> autoAt (addk vfp 4) specs.A.call)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location)
    ~insp:(fun a _ _ -> autosp a) in
    (* alternate: ~mkauto:Block.relative vfp "in call parms" specs.A.call *)
    (* ~autosp:(fun r -> addk (Block.base r.A.overflow) (-4)) *)

⟨functions to transform automata 623b⟩+≡ (624a) <623a 623c▷
  let c_call_results =
    let autosp = (fun r -> Block.base r.A.overflow) in
    C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt specs.A.results)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location) (* irrelevant? *)
    ~insp:(fun a _ _ -> autosp a) in
  let c_returns_struct_call_results =
    let autosp = (fun r -> addk (Block.base r.A.overflow) 4) in
    C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt specs.A.results)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location) (* irrelevant? *)
    ~insp:(fun a _ _ -> autosp a) in
  let cmm_call_results =
    C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt specs.A.results)
    ~autosp:(fun r -> Block.base r.A.overflow)
    ~postsp:(fun _ _ -> std_sp_location) (* irrelevant? *)
    ~insp:(fun a _ out_parm_block -> Block.base a.A.overflow) in
  let epilog =
    C.outgoing ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt specs.A.results)
    ~autosp:(fun r -> Block.base r.A.overflow)
    ~postsp:(fun _ r -> addk r (-4)) in
  let epilog_returns_struct =
    C.outgoing ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt specs.A.results)
    ~autosp:(fun r -> addk (Block.base r.A.overflow) 4)
    ~postsp:(fun _ r -> addk r (-4)) in

⟨functions to transform automata 623c⟩+≡ (624a) <623b
  let also_cuts_to =
    let autosp = (fun _ -> std_sp_location) in
    C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in cont parms" autoAt specs.A.cutto)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location)
    ~insp:(fun a _ _ -> autosp a) in

  let cut_actuals cont =
    C.outgoing ~growth ~sp

```



```

~mkauto:(fun () -> autoAt (Contn.ovblock_exp cont 8 32 sp_align) specs.A.cutto)
~autosp:(fun r -> spval) (* should this be std_sp_location? *)
~postsp:(fun _ -> spval) in

<x86call.ml 624a>+≡ (619i) <622f 624b>
let rtn return_to k n ~ra =
  if k > n then impossf "Return <k/n> has k:%d > n:%d\n" k n
  else return_to ra

let conv ~preserved ~jump ~struct' ~altret name specs ~return_to cut =
  <functions to transform automata 623a>
  let return k n ~ra =
    if altret then rtn return_to k n ~ra
    else if k = 0 & n = 0 then return_to ra
    else impossf "alternate return not allowed in %s calling convention" name in
  let nvregs = preserved in
  let volregs = RS.diff allregs preserved in
  { C.name = name
  ; C.overflow_alloc =
    { C.parameter_deallocator = if jump then C.Callee else C.Caller
    ; C.result_allocator = C.Caller
    }
  ; C.call_parms = { C.in' = prolog; C.out = call_actuals}
  ; C.cut_parms = { C.in' = also_cuts_to; C.out = cut_actuals}
  ; C.results = { C.in' = if jump then cmm_call_results
                  else if struct' then c_returns_struct_call_results
                  else c_call_results
                  ; C.out = if struct' then epilogs_returns_struct else epilogs}

  ; C.stack_growth = Memalloc.Down
  ; C.stable_sp_loc = std_sp_location
  ; C.jump_tgt_reg = Rtl.reg edx
  ; C.replace_vfp = Vfpwrite.replace_with ~sp
  ; C.sp_align = 4 (* alignment of stack pointer at call/cut *)
  ; C.ra_on_entry = const (R.fetch ra 32) (* where return address is on entry *)
  ; C.where_to_save_ra = (fun _ t -> Talloc.Multiple.loc t 't' 32)
    (* can't afford to leave ra where it is, even for the C convention,
    because we might make a tail call from C to C-- *)
  ; C.ra_on_exit =
    (let adjust = if struct' then 0 else -4 in
    (fun _ b _ -> amem (addk (Block.base b) adjust)))
  ; C.sp_on_unwind = (fun exp -> RU.store sp exp)
  ; C.sp_on_jump =
    if jump then (fun b _ -> RU.store sp (addk (Block.base b) (-4)))
    else (fun _ _ -> impossf "tail calls not supported by %s calling convention" name)
  ; C.pre_nvregs = nvregs (* registers preserved across calls *)
  ; C.volregs = volregs (* registers not preserved across calls *)
  ; C.saved_nvr = saved_nvr
  ; C.return = return
  }

<x86call.ml 624b>+≡ (619i) <624a 626a>
let thread cut specs =
  let outgoing _ _ = impossf "called out using thread convention" in
  let nocut _ _ = impossf "cut to or continuation using thread convention" in
  let noreturn _ _ = impossf "return using thread convention" in
  let incoming types formals = match types, formals with
  | [_; _], [_; _] ->
    let a = A.at mspace vfp specs.A.call in
    let crank effects' (w, k, al) formal =

```

```

    let l = A.allocate a w k al in
    A.store formal (A.fetch l w) w :: effects' in
let shuffle = R.par (List.rev (List.fold_left2 crank [] types formals)) in
let a = A.freeze a in
let _autosp = vfp in
let postsp = std_sp_location in
let _insp   = vfp in
let setsp   = RU.store sp postsp in
{ C.overflow = a.A.overflow
; C.insp     = (fun b -> RU.store sp vfp)
; C.regs     = a.A.regs_used
; C.shuffle  = shuffle
; C.post_sp  = R.guard (R0.ne 32 spval postsp) setsp
; C.pre_sp   = R.guard (R0.gt 32 spval postsp) setsp
}

| _ -> impossf "thread convention with %d parameters" (List.length formals) in
let _nvregs = RS.empty in
let _volregs = allregs in
{ C.name = "C-- thread"
; C.overflow_alloc =
    { C.parameter_deallocator = C.Caller
    ; C.result_allocator      = C.Caller
    }
; C.call_parms = { C.in' = incoming; C.out = outgoing}
; C.cut_parms  = { C.in' = nocut;   C.out = (fun _ -> nocut)}
; C.results    = { C.in' = noreturn; C.out = noreturn }
; C.stack_growth = Memalloc.Down
; C.stable_sp_loc = std_sp_location
; C.jump_tgt_reg  = Rtl.reg edx
; C.replace_vfp   = Vfprewrite.replace_with ~sp
; C.sp_align      = 4 (* alignment of stack pointer at call/cut *)
; C.ra_on_entry   = const (R.fetch ra 32) (* where return address is on entry *)
; C.where_to_save_ra = (fun _ t -> Talloc.Multiple.loc t 't' 32)
; C.ra_on_exit    = (fun _ _ t -> Talloc.Multiple.loc t 't' 32)
; C.sp_on_unwind  = (fun exp -> RU.store sp exp)
; C.sp_on_jump    = (fun _ _ -> impossf "tail calls not supported by C-- thread calling convention")
; C.pre_nvregs    = RS.empty (* registers preserved across calls *)
; C.volregs       = allregs (* registers not preserved across calls *)
; C.saved_nvr     = saved_nvr
; C.return        = (fun _ _ ~ra -> impossf "return in C-- thread convention")
}

```

<x86 calling convention automata in Lua 625>+≡ <622d

```

X86.cc["paranoid C"]      = X86.cc["C"]
X86.layout["paranoid C"]  = X86.layout["C"]
X86.cc["C returns struct"] = X86.cc["C"]
X86.layout["C returns struct"] = X86.layout["C"]

```

```

-- layout for "notail" is defined in luacompile.nw (uses the "C" layout)
X86.cc["notail"]          = X86.cc["C--"]

```

```

A.register_cc(Backend.x86.target,"C"      , X86.cc["C"      ])
A.register_cc(Backend.x86.target,"paranoid C",X86.cc["paranoid C"])
A.register_cc(Backend.x86.target,"C returns struct", X86.cc["C returns struct"])
A.register_cc(Backend.x86.target,"C'"      , X86.cc["C"      ])
A.register_cc(Backend.x86.target,"C--"     , X86.cc["C--"    ])
A.register_cc(Backend.x86.target,"C-- thread", X86.cc["C-- thread"])
A.register_cc(Backend.x86.target,"notail", X86.cc["notail"])
A.register_cc(Backend.x86.target,"gc", X86.cc["gc"])

```

```

<x86call.ml 626a>+≡ (619i) <624b
let cconv ~return_to cut ccname stage =
  let cconv = conv ~struct':false in
  let f =
    match ccname with
    | "C--"          -> cconv ~jump:true ~altret:true ~preserved:nvregs
    | "C-- thread"   -> (fun ccname stage ~return_to cut -> thread cut stage)
    | "lightweight"  -> cconv ~jump:true ~altret:true ~preserved:nvregs
    | "notail"       -> cconv ~jump:false ~altret:true ~preserved:nvregs
    | "paranoid C"   -> cconv ~jump:false ~altret:false ~preserved:RS.empty
    | "C"            -> cconv ~jump:false ~altret:false ~preserved:nvregs
    | "C returns struct" -> conv ~struct':true ~jump:false ~altret:false ~preserved:nvregs
    | "gc"           -> cconv ~jump:false ~altret:false
                        ~preserved:(RS.of_list [ecx;edx;ebx;esi;edi;ebp])
    | _ -> Unsupported.calling_convention ccname
  in f ccname stage ~return_to cut

```

V.10.1 Remove? Implementation using Callspec

```

<move before cconv to use again 626b>≡
module CS = Callspec
let template = (* conservative spec *)
  { CS.name          = "callspec"      (* override this! *)
  ; CS.stack_growth  = Memalloc.Down
  ; CS.overflow      = CS.overflow C.Caller C.Caller
  ; CS.memspace      = mspace
  ; CS.sp            = esp
  ; CS.sp_align      = 4
  ; CS.all_regs      = RS.union volregs nvregs
  ; CS.nv_regs       = nvregs
  ; CS.save_nvr      = saved_nvr
  ; CS.ra            = (ra, CS.ReturnAddress.KeepInPlace)
  }

let cc auto return_to cut spec =
  let t      = CS.to_call cut (rtn return_to) auto spec          in
  let jump   = spec.CS.overflow.C.parameter_deallocator = C.Callee in
  { t with (* fix what Callspec got wrong *)
    C.ra_on_exit = if jump
                    then (fun _ b _ -> amem (addk (Block.base b) (-4)))
                    else (fun _ _ _ -> ra)
    ; C.sp_on_unwind = (fun e -> RU.store sp e)
  }

let c' ccname auto ~return_to cut = cc auto return_to cut
  { template with CS.name = ccname }

```

```

<stack-frame layout functions 626c>≡ 626d>
X86 = X86 or {}
X86.layout = { creates='no late consts' }

function X86.layout.fn(dummy,proc) --- dispatch on cc name
  return X86.layout[Stack.ccname(proc)](dummy, proc)
end

```

```

<stack-frame layout functions 626d>+≡ <626c 628>
function X86.layout["C"](dummy,proc) --- really for a C convention only
  local blocks = Stack.blocks(proc)
  local slots

```

```

-- proc, slots = Stack.replace_slot_temporaries(proc)
blocks.ra          = Block.relative(blocks.vfp, "return address", 4, 4)

-- We need to leave space for the return address on each tail call.
local i, block = 1, blocks.oldblocks.callee[1]
while block do
    local tailcall_ra =
        Block.relative(blocks.vfp, "tail call return address" .. i, 4, 4)
    blocks.oldblocks.callee[i] = Block.cat(32, {blocks.oldblocks.callee[i],
                                                tailcall_ra})

    i = i + 1
    block = blocks.oldblocks.callee[i]
end

-- OVERLAP_HIGH CONSIDERED SUSPECT
blocks.oldblocks.callee = Block.overlap_high(32, blocks.oldblocks.callee)
blocks.oldblocks.caller  = Block.overlap_low (32, blocks.oldblocks.caller)
blocks.youngblocks.callee = Block.overlap_high(32, blocks.youngblocks.callee)
blocks.youngblocks.caller = Block.overlap_low (32, blocks.youngblocks.caller)

-- It is _vital_ that there be no padding inserted between the stack pointer
-- and the overflow areas. If there is padding, then the callee has no hope
-- of matching the caller's expectations.
-- Hence, we concatenate these blocks before attaching them to the rest of the
-- procedure, which might introduce strange alignment requirements.
local pre_ra_tail = Block.cat(32, { blocks.ra, blocks.vfp })
local ra_tail     = Block.overlap_high(32, {pre_ra_tail, blocks.oldblocks.callee})
local old_end     = Block.cat(32, { blocks.oldblocks.caller
                                   , ra_tail
                                   })

local young_end = Block.cat(32, { blocks.youngblocks.caller
                                   , blocks.sp
                                   , blocks.youngblocks.callee
                                   })

local layout =
{
    old_end          -- <-- high addresses
    , slots.confined
    , slots.unconfined
    , blocks.stackdata
    , blocks.continuations
    , blocks.spills
    , young_end      -- <-- low addresses
}

if Debug.stack then
    write('==== using stack layout for C/notail =====\n')
    write('***** cc name = ', Stack.ccname(proc), '\n')
    Debug.showblocks (blocks, {'oldblocks', 'ra', 'vfp',
                               'stackdata', 'continuations', 'spills',
                               'youngblocks', 'sp'})
end

local block = Block.cat(32, layout)
if Debug.stack then
    Debug.showblocks({frame=block}, {'frame'})
end
if Debug.framesize then Debug.showframesize(block,proc) end

proc = Stack.freeze(proc,block)
return proc, 1
end

```

```
X86.layout["notail"] = X86.layout["C"]
```

<stack-frame layout functions 628>+≡

<626d 629>

```
function X86.layout["C--"](dummy,proc)
  local blocks = Stack.blocks(proc)

  if Debug.stack then
    write('==== using stack layout for C-- with tail calls =====\n')
    local i = 1
    local keys = { }
    while blocks.oldblocks.callee[i] do keys[i] = i; i = i + 1 end
    write('oldblocks.callee (before ra is added):\n')
    Debug.showblocks(blocks.oldblocks.callee, keys)
    write('.....\n')
  end

  blocks.ra = Block.relative(blocks.vfp, "return address", 4, 4)

  -- first old block is always the incoming parms
  blocks.oldblocks.callee[1] =
    Block.cat(32, {blocks.oldblocks.callee[1], blocks.ra, blocks.vfp})

  -- We need to leave space for the return address on each tail call.
  local i, block = 2, blocks.oldblocks.callee[2]
  while block do
    local tailcall_ra =
      Block.relative(blocks.vfp, "tail call return address" .. i, 4, 4)
    blocks.oldblocks.callee[i] = Block.cat(32, {blocks.oldblocks.callee[i],
      tailcall_ra})

    i = i + 1
    block = blocks.oldblocks.callee[i]
  end

  blocks.oldblocks.caller = Block.overlap_low (32, blocks.oldblocks.caller)
  blocks.oldblocks.callee = Block.overlap_high(32, blocks.oldblocks.callee)
  blocks.youngblocks.caller = Block.overlap_low (32, blocks.youngblocks.caller)
  blocks.youngblocks.callee = Block.overlap_high(32, blocks.youngblocks.callee)

  -- It is _vital_ that there be no padding inserted between the stack pointer
  -- and the overflow areas. If there is padding, then the callee has no hope
  -- of matching the caller's expectations.
  -- Hence, we concatenate these blocks before attaching them to the rest of the
  -- procedure, which might introduce strange alignment requirements.
  local old_end = Block.cat(32, { blocks.oldblocks.caller
    -- On return, the ra must end up here,
    -- immediately followed by the sp
    , blocks.oldblocks.callee
    -- , blocks.ra -- joined to incoming block
    -- , blocks.vfp
    })

  local young_end = Block.cat(32, { blocks.youngblocks.caller
    , blocks.sp
    , blocks.youngblocks.callee
    })

  local layout =
    { old_end -- <-- high addresses
    , blocks.stackdata
    , blocks.continuations
    , blocks.spills
```

```

        , young_end          -- <-- low addresses
    }
if Debug.stack then
    Debug.showblocks (blocks, {'oldblocks', 'ra', 'vfp',
                               'stackdata', 'continuations', 'spills',
                               'youngblocks', 'sp'})
end
local block = Block.cat(32, layout)
if Debug.framesize then Debug.showframesize(block,proc) end
proc = Stack.freeze(proc,block)
return proc, 1
end

<stack-frame layout functions 629>+≡ <628
function X86.layout["C-- thread"](dummy,proc)
    local blocks = Stack.blocks(proc)

    if Debug.stack then
        write('==== using stack layout for C-- thread =====\n')
        local i = 1
        local keys = { }
        while blocks.oldblocks.callee[i] do keys[i] = i; i = i + 1 end
        write('oldblocks.callee (before ra is added):\n')
        Debug.showblocks(blocks.oldblocks.callee, keys)
        write('.....\n')
    end

    -- first old block is always the incoming parms
    blocks.oldblocks.caller[1] =
        Block.cat(32, {blocks.oldblocks.caller[1], blocks.vfp})
    blocks.oldblocks.caller    = Block.overlap_low (32, blocks.oldblocks.caller)
    blocks.oldblocks.callee    = Block.overlap_high(32, blocks.oldblocks.callee)
    blocks.youngblocks.caller   = Block.overlap_low (32, blocks.youngblocks.caller)
    blocks.youngblocks.callee  = Block.overlap_high(32, blocks.youngblocks.callee)
    local old_end    = Block.cat(32, { blocks.oldblocks.caller
                                       , blocks.oldblocks.callee
                                       })
    local young_end  = Block.cat(32, { blocks.youngblocks.caller
                                       , blocks.sp
                                       , blocks.youngblocks.callee
                                       })

    local layout =
        { old_end          -- <-- high addresses
          , blocks.stackdata
          , blocks.continuations
          , blocks.spills
          , young_end       -- <-- low addresses
        }

    if Debug.stack then
        Debug.showblocks (blocks, {'oldblocks', 'ra', 'vfp',
                                    'stackdata', 'continuations', 'spills',
                                    'youngblocks', 'sp'})
    end
    local block = Block.cat(32, layout)
    if Debug.framesize then Debug.showframesize(block,proc) end
    proc = Stack.freeze(proc,block)
    return proc, 1
end
end

```

V.11 arch/x86/x86rec.nw

`<arch/x86/x86rec.mli 630a>≡`
`<x86rec.mli 630b>`

V.12 Intel x86 Recognizer

`<x86rec.mli 630b>≡` (630a)
 module M : sig
 val is_instruction : Rtl.rtl -> bool
 val to_asm : Rtl.rtl -> string
 end

`<x86rec.mli 630c>≡`
 %head {:
 <modules(x86rec.nw) 631a>
 module M = struct
 <code to precede the labeler(x86rec.nw) 632a>
 :}
 %tail {:
 <code to follow the labeler(x86rec.nw) 642a>
 end (* of M *)
 :}

 %term
 <names of types of terminals(x86rec.nw) 633c>

 %type ah {: string :}
 %type ahval {: string :}
 %type any {: string :}
 %type ax {: string :}
 %type const {: string :}
 %type const8 {: string :}
 %type disp {: string :}
 %type eaddr {: string :}
 %type eaddri {: string :}
 %type eaddr1 {: string :}
 %type eaddr_shr_k {: string * int :}
 %type eax {: unit :}
 %type ecx {: unit :}
 %type edx {: unit :}
 %type edx_eax {: unit :}
 %type edi {: unit :}
 %type eflags {: unit :}
 %type eight {: unit :}
 %type eip {: unit :}
 %type esi {: unit :}
 %type esp {: unit :}
 %type espl {: unit :}
 %type fiadd {: int * string :}
 %type i2f_mem {: int * string :}
 %type four {: unit :}
 %type fpcc1 {: unit :}
 %type fppop {: unit :}
 %type fppop2 {: unit :}
 %type fppush {: unit :}
 %type fpssp {: unit :}
 %type fpstack1 {: unit :}

```

%type fpstack1l {: unit :}
%type fpstacknext {: unit :}
%type fpstacknext1l {: unit :}
%type fpstacktop {: unit :}
%type fpstacktop1l {: unit :}
%type fpuct1l {: unit :}
%type fpuct1l1l {: unit :}
%type fpustatus {: unit :}
%type fpustatus1l {: unit :}
%type immed {: string :}
%type immed8 {: string :}
%type inst {: string :}
%type inthandler {: string :}
%type lconst {: string :}
%type lateconst {: string :}
%type mem {: string :}
%type mem1l {: string :}
%type minusfour {: unit :}
%type one3 {: unit :}
%type pop {: unit :}
%type push {: unit :}
%type reg {: string :}
%type reg8 {: string :}
%type regabcd {: string :}
%type regabcd1l {: string :}
%type reg1l {: string :}
%type reg18 {: string :}
%type regpair {: string :}
%type regpair1l {: string :}
%type shamt {: int :}
%type slot {: string :}
%type slotaddr {: string :}
%type stacknext {: unit :}
%type stacktop {: unit :}
%type target {: string :}
%type two3 {: unit :}
%type vfp {: unit :}
%type vfpl {: unit :}
%type zero {: unit :}

```

```

%%
<rules(x86rec.nw) 633d>

```

```

<modules(x86rec.nw) 631a>≡ (630c)
open Nopoly

```

```

module Rg = X86regs
module RP = Rtl.Private
module RU = Rtlutil
module SS = Space.Standard32
module Dn = Rtl.Dn      (* Convert Dn   to private repr. *)
module Up = Rtl.Up      (* Convert Up   to abstract repr. *)

let ( =/ ) = RU.Eq.loc
let ( =// ) = RU.Eq.exp

```

```

<unused code for logging the results of comparisons 631b>≡
let fd = open_out "qc--.log"
let () = at_exit (fun () -> close_out fd)

```



```

let log eq s x y=
  let answer = eq x y in
  Printf.fprintf fd "%s %s %s\n" (s x) (if answer then "==" else "!=") (s y);
  answer

let ( =/ ) = log RU.Eq.loc (fun l -> RU.ToString.loc (Up.loc l))
let ( =// ) = log RU.Eq.exp (fun e -> RU.ToString.exp (Up.exp e))

<code to precede the labeler(x86rec.nw) 632a>≡ (630c) 632b▷
  exception Error of string
  let error msg = raise (Error msg)

<code to precede the labeler(x86rec.nw) 632b>+≡ (630c) <632a 632c>▷
  let sprintf = Printf.sprintf
  let s      = Printf.sprintf

```

V.12.1 Utilities

```

<code to precede the labeler(x86rec.nw) 632c>+≡ (630c) <632b 632d>▷
  let weirdb = Bits.S.of_int (-4) 32
  let weird = Nativeint.to_string (Bits.S.to_native weirdb)
  (* let _ = List.iter prerr_string ["Weird integer is "; weird; "\n"] *)
  let native' w b =
    assert (Bits.width b = w);
    Nativeint.to_string (Bits.U.to_native b)
  let native = native' 32
  let cat = String.concat ""
  let is_shamt b =
    let w = Bits.width b in
    w >= 5 && Bits.Ops.lt (Bits.zero w) b && Bits.Ops.lt b (Bits.U.of_int 32 w)

<code to precede the labeler(x86rec.nw) 632d>+≡ (630c) <632c 632e>▷
  let infinity = Camlburg.inf_cost
  let guard b = if b then 0 else infinity

<code to precede the labeler(x86rec.nw) 632e>+≡ (630c) <632d 633a>▷
  let suffix w = match w with
  | 8 -> "b"
  | 16 -> "w"
  | 32 -> "l"
  | _ -> Impossible.impossible "width in x86 not 8/16/32"
  let fsuffix w = match w with
  | 32 -> "s"
  | 64 -> "l"
  | 80 | 96 -> "t"
  | _ -> Impossible.impossible "floating-point width in x86 not 32/64/80"
  let fisuffix w = match w with
  | 16 -> "w"
  | 32 -> "l"
  | 64 -> "q"
  | _ -> Impossible.impossible "int to/from fregs width in x86 not 16/32/64"

```

V.12.2 Register names

```
<code to precede the labeler(x86rec.nw) 633a>+≡ (630c) <632e 633b>  
let r8_names = [| "%al"; "%cl"; "%dl"; "%bl"; "%ah"; "%ch"; "%dh"; "%bh" |]  
let r16_names = [| "%ax"; "%cx"; "%dx"; "%bx"; "%sp"; "%bp"; "%si"; "%di" |]  
let r32_names = [| "%eax"; "%ecx"; "%edx"; "%ebx"; "%esp"; "%ebp"; "%esi"; "%edi" |]  
let reg_names w = match w with  
| 8 -> r8_names  
| 16 -> r16_names  
| 32 -> r32_names  
| _ -> Impossible.impossible "x86 register width not 8/16/32"  
let regname w = Array.get (reg_names w)  
let hregname r = sprintf "%c" r.[2] (* pass %eax, get back %ah *)
```

V.12.3 Other machine info

```
<code to precede the labeler(x86rec.nw) 633b>+≡ (630c) <633a  
let fpcc = Dn.loc Rg.fpcc  
let fpustatus = Dn.loc Rg.fpustatus  
let fpuctl = RP.Reg Rg.fpuctl
```

V.12.4 Recognizer terminals, nonterminals, and constructors

```
<names of types of terminals(x86rec.nw) 633c>≡ (630c)  
n w c bits symbol
```

V.12.5 Recognizer Rules

```
<rules(x86rec.nw) 633d>≡ (630c) 633e>  
lconst : Link(symbol, w) {: symbol#mangled_text :}  
pic : Diff(c1:lconst,c2:lconst) {: c1 ^ " - " ^ c2 :}  
const : Bits(b:bits) [{: guard (Bits.width b = 32) :}] {: native b :}  
const8 : Bits(b:bits) [{: guard (Bits.width b = 8) :}] {: native' 8 b :}  
shamt : Bits(b:bits) [{: guard (is_shamt b) :}] {: Bits.U.to_int b :}  
four : Bits(b:bits)  
[{: guard (Bits.width b > 3 && Bits.Ops.eq (Bits.U.of_int 4 (Bits.width b)) b) :}] {: ( ) :}  
minusfour : Bits(b:bits)  
[{: guard (Bits.width b > 3 && Bits.Ops.eq (Bits.S.of_int (-4) (Bits.width b)) b) :}] {: ( ) :}
```

```
<rules(x86rec.nw) 633e>+≡ (630c) <633d 633f>  
bare_reg : Reg('r', n) {: n :}  
regl : bare_reg {: regname 32 bare_reg :}  
bare_regabcd : Reg('r', n) [{: guard (n < 4) :}] {: n :}  
regabcdl : bare_regabcd {: regname 32 bare_regabcd :}  
regl8 : Slice(w, i:n, bare_regabcd) [{: guard (w=8) :}] {: Rg.regname8 i bare_regabcd :}  
regl8H : Slice(w, i:n, regabcdl) [{: guard (w=8 && i=8) :}] {: hregname regabcdl :}  
regl_ecx : Reg('r', n) [{: guard (n=1) :}] {: ( ) :}  
regpairl : RegPair(regl:regl,reg2:regl) {: reg2 :}  
meml : Mem(reg, c) {: "(" ^ reg ^ ")" :} -- indir  
meml : Mem(displ, c) {: displ :}  
meml : Mem(lconst, c) {: lconst :}  
displ : Add(reg, const) {: const ^ "(" ^ reg ^ ")" :}  
displ : Add(const, reg) {: const ^ "(" ^ reg ^ ")" :}
```

```
<rules(x86rec.nw) 633f>+≡ (630c) <633e 634a>  
regl : Reg('t', n) {: sprintf "temporary register %d" n :}  
vfpl : Reg('V', n) [{: guard (n=0) :}] {: ( ) :}  
esp : Fetch(espl, w) {: espl :}  
vfp : Fetch(vfpl, w) {: vfpl :}
```

```

⟨rules(x86rec.nw) 634a⟩+≡ (630c) ◁633f 634b▷
    eaddrl : regl {: regl :}
    eaddrl : meml {: meml :}

⟨rules(x86rec.nw) 634b⟩+≡ (630c) ◁634a 634c▷
    reg      : Fetch(regl      , w) {: regl      :}
    regabcd  : Fetch(regabcdl, w) {: regabcdl :}
    reg8     : Lobits(Fetch(regl8, w), nw:w) [{: guard (nw=8) :}] {: regl8      :}
    reg8H    : BitExtract(eight, regabcd, w) [{: guard (w=8) :}] {: hregname regabcd :}
    reg8H    : Fetch(regl8H, w) {: regl8H :}
    reg_cl   : Lobits(Fetch(regl_ecx, w), nw:w) [{: guard (nw=8) :}] {: () :}
    reg_cll  : Slice(sw:w, n, regl_ecx) [{: guard (sw=8 && n=0) :}] {: () :}
    reg_cl   : Fetch(reg_cll, w) {: () :}
    regpair  : Fetch(regpairl, w) {: regpairl :}
    mem      : Fetch(meml      , w) {: meml      :}
    eaddr    : Fetch(eaddrl   , w) {: eaddrl   :}
    eaddri   : eaddr          {: eaddr      :}
    eaddri   : immed          {: immed      :}
    -- eaddr : lconst {: lconst :} -- absolute addressing mode
    -- indexed modes to come

⟨rules(x86rec.nw) 634c⟩+≡ (630c) ◁634b 634d▷
    -- immed : lconst {: lconst :}
    immed    : const   {: "$" ^ const :}
    immed8   : const8  {: "$" ^ const8 :}
    -- immed : Add(lconst, const) {: lconst ^ "+" ^ const :}
    -- immed : Add(const, lconst) {: lconst ^ "+" ^ const :}

⟨rules(x86rec.nw) 634d⟩+≡ (630c) ◁634c 634e▷
    slotaddr : Add(vfp, const) {: sprintf "%vfp+%s" const :}
    slotaddr : vfp {: "%vfp" :}

    const : Late(string, w) {: string :}
    const : Add(l:const, r:const) {: sprintf "%s+%s" l r :}

    inst : Guarded(Cmp(string, esp, slotaddr), Store(espl, s2:slotaddr, w))
           {: "adjust %esp" :}
    inst : Store(dst:regl, slotaddr, w) {: sprintf "%s := %s" dst slotaddr :}

    slot : Mem(slotaddr, c) {: slotaddr :}

    inst : Store(slot, Lobits(reg, lw:w), nw:w)
           {: s "bits%d[%s] := %%lobits%d(%s)" nw slot nw reg :}

    -- inst : Store(dst:eaddrl, lateconst, w) {: sprintf "%s := %s" dst lateconst :}

    disp : Add(vfp, const) {: const ^ "(%vfp)" :}
    disp : Add(const, vfp) {: const ^ "(%vfp)" :}

⟨rules(x86rec.nw) 634e⟩+≡ (630c) ◁634d 635▷
    -- pic code
    inst : Store(dst:eaddrl, pic, w)
           {: s "mov%s %s,%s" (suffix w) pic dst :}

    inst : Store(dst:eaddrl, src:immed, w)
           {: s "mov%s %s,%s" (suffix w) src dst :}

    inst : Store(dst:regl, src:lconst, w)
           {: s "lea%s %s,%s" (suffix w) src dst :}

```

```

-- straight load/store/move
inst : Store(dst:eaddrl, src:reg, w)
  { : s "mov%s %s,%s" (suffix w) src dst : }

inst : Store(dst:regl, src:eaddri, w)
  { : s "mov%s %s,%s" (suffix w) src dst : }

-- sign extension
inst : Store(dst:eaddrl, Sx(Fetch(src:regl, nw:w)), w)
  { : s "movs%s%s %s,%s" (suffix nw) (suffix w) src dst : }

inst : Store(dst:regl, Sx(Fetch(src:eaddrl, nw:w)), w)
  { : s "movs%s%s %s,%s" (suffix nw) (suffix w) src dst : }

-- zero extension
inst : Store(dst:eaddrl, Zx(Fetch(src:regl, nw:w)), w)
  { : s "movz%s%s %s,%s" (suffix nw) (suffix w) src dst : }

inst : Store(dst:regl, Zx(Fetch(src:eaddrl, nw:w)), w)
  { : s "movz%s%s %s,%s" (suffix nw) (suffix w) src dst : }

-- moving low bits (relies on proper name of source)
inst : Store(dst:eaddrl, Lobits(Fetch(Reg('r', n), fw:w), lw:w), nw:w)
  { : s "mov%s %s,%s" (suffix nw) (regname nw n) dst : }

inst : Store(dst:regl, Lobits(src:mem, sw:w), nw:w)
  { : s "mov%s %s,%s" (suffix nw) src dst : }

-- block copy
inst : Par(Llr(ecx1:ecx, "sub", n0:const, w1:w),
  Par(Llr(esi1:esi, "sub", n1:const, w2:w),
  Par(Llr(edi1:edi, "sub", n2:const, w3:w),
  Par(Store(Mem(Fetch(esi2:esi, w4:w), c11:c),
    Fetch(Mem(Fetch(edi2:edi, w5:w), c6:c), w7:w), w8:w),
    Guarded(Fetch(ecx2:ecx, w9:w), Goto(Fetch(eip, w10:w)))
  ))))
  { : "rep movsb" : }

```

$\langle \text{rules}(x86rec.nw) \text{ 635} \rangle + \equiv \quad (630c) \triangleleft 634e \text{ 636} \triangleright$

```

inst : Store(dst:regl, disp, w) [{:guard (w = 32):}]
  { : s "leal %s, %s" disp dst : }

inst : Llr(dst:regl, "add", const, w) [{:guard (w=32):}]
  { : s "leal %s(%s), %s" const dst dst : }

inst : Withcarryflags(dst:eaddrl, "addc", src:reg, w, "x86_adcflags")
  { : s "adc%s %s,%s" (suffix w) src dst : }

inst : Withcarryflags(dst:eaddrl, "subb", src:reg, w, "x86_sbbflags")
  { : s "sbb%s %s,%s" (suffix w) src dst : }

inst : Withcarryzero(dst:eaddrl, "addc", src:reg, w, "x86_adcflags")
  { : s "add%s %s,%s" (suffix w) src dst : }

inst : Withaflags(dst:eaddrl, "add", src:reg, w, "x86_addflags")
  { : s "add%s %s,%s" (suffix w) src dst : }

inst : Withaflags(dst:regl, "add", src:eaddri, w, "x86_addflags")
  { : s "add%s %s,%s" (suffix w) src dst : }

```

```

inst : Withaflags(dst:eaddrl, "sub", src:reg, w, "x86_subflags")
  {: s "sub%s %s,%s" (suffix w) src dst :}

inst : Withaflags(dst:regl, "sub", src:eaddri, w, "x86_subflags")
  {: s "sub%s %s,%s" (suffix w) src dst :}

inst : Withaflagsunary("neg", dst:eaddrl, w, "x86_negflags")
  {: s "neg%s %s" (suffix w) dst :}

inst : Withaflags(dst:eaddrl, "mul", src:reg, w, "x86_mulflags")
  {: s "imul %s %s, %s" (suffix w) src dst :}

inst : Withaflags(dst:regl, "mul", src:eaddri, w, "x86_mulflags")
  {: s "imul%s %s,%s" (suffix w) src dst :}

inst : Pairdestwithflags(edx, eax, Fetch(eax,w), "mulx", eaddr, "x86_mulxflags")
  {: s "imull %s" eaddr :}

inst : Pairdestwithflags(edx, eax, Fetch(eax,w), "mulux", eaddr, "x86_muluxflags")
  {: s "mull %s" eaddr :}

edx_eax : RegPair(edx, eax) {: () :}

inst : Withundefflags(dst:edx_eax, "quot", "rem", src:eaddr, w)
  {: sprintf "idiv%s %s, %%eax" (suffix w) src :}

-- suffix w looks wrong
inst : Withundefflags(dst:edx_eax, "divu", "modu", src:eaddr, w)
  {: sprintf "div%s %s, %%eax" (suffix w) src :}

inst : Withaflags(dst:regl, "shl", Zx(reg_cl), w, "x86_shlflags")
  {: sprintf "shl%s %%cl, %s" (suffix w) dst :}

inst : Withaflags(dst:regl, "shra", Zx(reg_cl), w, "x86_shraflags")
  {: sprintf "sar%s %%cl, %s" (suffix w) dst :}

inst : Withaflags(dst:regl, "shrl", Zx(reg_cl), w, "x86_shrlflags")
  {: sprintf "shr%s %%cl, %s" (suffix w) dst :}

inst : Withaflags(dst:regl, "rotl", Zx(reg_cl), w, "x86_rotlflags")
  {: sprintf "rol%s %%cl, %s" (suffix w) dst :}

inst : Withaflags(dst:regl, "rotr", Zx(reg_cl), w, "x86_rotrflags")
  {: sprintf "ror%s %%cl, %s" (suffix w) dst :}

inst : Llr(dst:regl, "shl", immed, w)
  {: sprintf "shl%s %s, %s" (suffix w) immed dst :}

inst : Llr(dst:regl, "shra", immed, w)
  {: sprintf "sar%s %s, %s" (suffix w) immed dst :}

inst : Llr(dst:regl, "shrl", immed, w)
  {: sprintf "shr%s %s, %s" (suffix w) immed dst :}

```

```

⟨rules(x86rec.nw) 636⟩+≡ (630c) ◁635 637a▷
inst : Setflags(X86_subflags(l:eaddr, r:reg))
  {: s "cmp%s %s,%s" (suffix 32) r l :} -- POSSIBLE BUG IN WIDTH
inst : Setflags(X86_subflags(l:reg, r:eaddri))

```

```

{: s "cmp%s %s,%s" (suffix 32) r 1 :} -- POSSIBLE BUG IN WIDTH

inst : UnaryInPlace(dst:regl, "com", w) {: s "not%s %s" (suffix w) dst :}

inst : Withlflags(dst:eaddrl, logical:string, src:reg, w, "x86_logicflags")
  [{: match logical with "and" | "or" | "xor" -> 0 | _ -> infinity :}]
  {: s "%s%s %s,%s" logical (suffix w) src dst :}

inst : Withlflags(dst:regl, logical:string, src:eaddri, w, "x86_logicflags")
  [{: match logical with "and" | "or" | "xor" -> 0 | _ -> infinity :}]
  {: s "%s%s %s,%s" logical (suffix w) src dst :}

inst : Withlflags(dst:regl8H, logical:string, src:immed8, w, "x86_logicflags")
  [{: (guard (w=8)) + match logical with "and" | "or" | "xor" -> 0 | _ -> infinity :}]
  {: sprintf "%s%s %s,%s" logical (suffix 8) src dst :}

inst : Withlflags8H(dst:regabcdl, logical:string, src:reg8H, "x86_logicflags")
  [{: match logical with "and" | "or" | "xor" -> 0 | _ -> infinity :}]
  {: sprintf "%s%s %s,%s" logical (suffix 8) src (hregname dst) :}

inst : Withlflags8H(dst:regabcdl, logical:string, src:immed8, "x86_logicflags")
  [{: match logical with "and" | "or" | "xor" -> 0 | _ -> infinity :}]
  {: sprintf "%s%s %s,%s" logical (suffix 8) src (hregname dst) :}

<rules(x86rec.nw) 637a>+≡ (630c) <636 637b>
eflags : Reg('c', n) [{: guard (n = SS.indices.SS.cc):}] {: () :}
eaddrbit0 : Lobits(eaddr, w) [{: guard (w=1) :}] {: eaddr :}
eaddrbitk : Lobits(eaddr_shr_k, w) [{: guard (w=1) :}] {: eaddr_shr_k :}
eaddr_shr_k : Shift("shra", eaddr, shamt) {: eaddr, shamt :}
eaddr_shr_k : Shift("shrl", eaddr, shamt) {: eaddr, shamt :}
inst : Llr(eflags, "x86_setcarry", eaddrbit0, w)
  {: sprintf "bt%s $0,%s" (suffix 32) eaddrbit0 :}

inst : Llr(eflags, "x86_setcarry", zero, w) {: "clc" :}

<rules(x86rec.nw) 637b>+≡ (630c) <637a 637c>
target : Fetch(eaddrl, w) {: "*" ^ eaddrl :}
target : lconst {: lconst :}
inst : Goto(target) {: "jmp " ^ target :}
inst : Jcc(cc:string, lconst) {: "j" ^ cc ^ " " ^ lconst :}

<rules(x86rec.nw) 637c>+≡ (630c) <637b 637d>
espl : Reg('r', n) [{: guard (n = 4):}] {: () :}
eip : Reg('c', n) [{: guard (n = SS.indices.SS.pc):}] {: () :}

<rules(x86rec.nw) 637d>+≡ (630c) <637c 637e>
inst : Par(Goto(target), Store(espl, frame:eaddri, w))
  {: s "movl %s, %%esp; jmp %s" frame target :}

<rules(x86rec.nw) 637e>+≡ (630c) <637d 638a>
pop : Store(espl, Add(esp, four), sw:w) {: () :}
pop : Llr(espl, "add", four, w) {: () :}
push : Llr(espl, "sub", four, w) {: () :}
push : Llr(espl, "add", minusfour, w) {: () :}
stacktop : Mem(esp, mc:c) [{:guard (mc = 4):}] {: () :}
stacknext : Mem(Sub(esp, four), mc:c) [{:guard (mc = 4):}] {: () :}
inst : Par(Goto(Fetch(stacktop, w)), pop)
  {: "ret" :}
inst : Par(Goto(eaddr), Par(Store(stacknext, Fetch(eip, x:w), y:w), push))
  {: "call *" ^ eaddr :}
inst : Par(Goto(lconst), Par(Store(stacknext, Fetch(eip, x:w), y:w), push))
  {: "call " ^ lconst :}

```

```

⟨rules(x86rec.nw) 638a⟩+≡ (630c) <637e 638b>
  inst : Par(Goto(inthandler),Par(Store(stacknext, Fetch(eip, x:w), y:w), push))
    { : "int " ^ inthandler : }

  inthandler : X86IdtPc(immed) { : immed : }

⟨rules(x86rec.nw) 638b⟩+≡ (630c) <638a 638c>
  one3 : Bits(b:bits)
    [{ : guard (Bits.width b = 3 && Bits.Ops.eq (Bits.U.of_int 1 3) b) :}]
    { : () :}
  two3 : Bits(b:bits)
    [{ : guard (Bits.width b = 3 && Bits.Ops.eq (Bits.U.of_int 2 3) b) :}]
    { : () :}
  zero : Bits(b:bits) [{ : guard (Bits.Ops.eq (Bits.U.of_int 0 (Bits.width b)) b) :}]
    { : () :}
  eight : Bits(b:bits) [{ : guard (Bits.width b > 3 &&
    Bits.Ops.eq (Bits.U.of_int 8 (Bits.width b)) b) :}]
    { : () :}
  fpssp : Reg('F', n) [{ : guard (n = 0):}] { : () :}
  fppop : Llr(fpssp, "add", one3, w) { : () :}
  fppop2 : Llr(fpssp, "add", two3, w) { : () :}
  fppush : Llr(fpssp, "sub", one3, w) { : () :}

⟨rules(x86rec.nw) 638c⟩+≡ (630c) <638b 638d>
  fpstacktop1 : Fpreg(Fetch(fpssp, w)) { : () :}
  fpstacktop1 : Fpreg(Add(Fetch(fpssp, w), zero)) { : () :}
  fpstack11 : Fpreg(Add(Fetch(fpssp, w), one3)) { : () :}
  fpstacknext1 : Fpreg(Sub(Fetch(fpssp, w), one3)) { : () :}

  fpstacktop : Fetch(fpstacktop1, w) [{ : guard (w=80) :}] { : fpstacktop1 :}
  fpstack1 : Fetch(fpstack11, w) [{ : guard (w=80) :}] { : fpstack11 :}
  fpstacknext : Fetch(fpstacknext1, w) [{ : guard (w=80) :}] { : fpstacknext1 :}

⟨rules(x86rec.nw) 638d⟩+≡ (630c) <638c 638e>
  inst : Par(Store(fpstacknext1, F2f(s:w, d:w, mem), w2:w), fppush)
    { : sprintf "fld%s %s" (fsuffix s) mem :}
  inst : Par(Store(dst:mem1, F2f(s:w, d:w, fpstacktop), w2:w), fppop)
    { : sprintf "fstp%s %s" (fsuffix d) dst :}

⟨rules(x86rec.nw) 638e⟩+≡ (630c) <638d 638f>
  inst : Par(Store(fpstacknext1, I2f(s:w, d:w, mem), w2:w), fppush)
    { : sprintf "fild%s %s" (fisuffix s) mem :}
  inst : Par(Store(dst:mem1, F2i(s:w, d:w, fpstacktop), w2:w), fppop)
    { : sprintf "fistp%s %s" (fisuffix d) dst :}

⟨rules(x86rec.nw) 638f⟩+≡ (630c) <638e 638g>
  inst : Par(Store(fpstack11, Fadd(fpstacktop, fpstack1), w), fppop)
    { : "faddp" :}
  inst : Par(Store(fpstack11, Fsub(fpstacktop, fpstack1), w), fppop)
    { : "fsubp" :}
  inst : Par(Store(fpstack11, Fdiv(fpstacktop, fpstack1), w), fppop)
    { : "fdivp" :}
  inst : Par(Store(fpstack11, Fdiv(fpstack1, fpstacktop), w), fppop)
    { : "fdivrp" :}
  inst : Par(Store(fpstack11, Fmul(fpstacktop, fpstack1), w), fppop)
    { : "fmulp" :}

⟨rules(x86rec.nw) 638g⟩+≡ (630c) <638f 639a>
  inst : UnaryInPlace(fpstacktop1, "fabs", w) { : "fabs" :}
  inst : UnaryInPlace(fpstacktop1, "fsqrt", w) { : "fsqrt" :}
  inst : UnaryInPlace(fpstacktop1, "fneg", w) { : "fchs" :}
  inst : Store(fpstacktop1, Frnd(fpstacktop), w) { : "frndint" :}

```

```

⟨rules(x86rec.nw) 639a⟩+≡ (630c) <638g 639b>
  i2f_mem : I2f(s:w, d:w, mem) [{: guard (s = 32 || s = 16) :}] {: s, mem :}
  fiadd : Fadd(fpstacktop, i2f_mem) {: i2f_mem :}
  inst : Store(fpstacktop1, fiadd, w)
    {: let s, mem = fiadd in sprintf "fiadd%s %s" (suffix s) mem :}

⟨rules(x86rec.nw) 639b⟩+≡ (630c) <639a 639c>
  fpcc1 : Fpcc1() {: () :}
  fpustatus1 : Fpustatus1() {: () :}
  fpustatus : Fetch(fpustatus1, w) {: () :}
  fpuct11 : Fpuct11() {: () :}
  fpuct1 : Fetch(fpuct11, w) {: () :}
  inst : Store(fpcc1, Fcmp(fpstacktop, fpstack1), w)
    {: "fcom" :}
  inst : Par(Store(fpcc1, Fcmp(fpstacktop, fpstack1), w), fppop)
    {: "fcomp" :}
  inst : Par(Store(fpcc1, Fcmp(fpstacktop, fpstack1), w), fppop2)
    {: "fcompp" :}

  inst : Store(mem1, fpuct1, w) {: s "fstcw %s" mem1 :}
  inst : Store(fpuct11, mem, w) {: s "fldcw %s" mem :}

⟨rules(x86rec.nw) 639c⟩+≡ (630c) <639b 639d>
  eax : Reg('r', n) [{: guard (n = 0):}] {: () :}
  ecx : Reg('r', n) [{: guard (n = 1):}] {: () :}
  edx : Reg('r', n) [{: guard (n = 2):}] {: () :}
  esi : Reg('r', n) [{: guard (n = 6):}] {: () :}
  edi : Reg('r', n) [{: guard (n = 7):}] {: () :}
  ax : Slice(w, lsb:n, eax) [{: guard (w = 16 && lsb = 0):}] {: "%ax" :}
  ah : Slice(w, lsb:n, eax) [{: guard (w = 8 && lsb = 8):}] {: "%ah" :}
  ahval : BitExtract(eight, Fetch(eax, fw:w), w) [{: guard (w = 8) :}] {: "%ah" :}
  ahval : Fetch(ah, w) [{: guard (w=8) :}] {: ah :}

  inst : Store(eax, BitInsert(zero, Fetch(eax, w), fpustatus), sw:w) {: "fstsw %ax" :}
  inst : Store(ax, fpustatus, w) [{: guard (w = 16) :}] {: s "fstsw %s" ax :}
  inst : Setflags(Ah2flags(ahval)) {: "sahf" :}
  inst : Store(ah, Flags2ah(Fetch(eflags, fw:w)), w) {: "lahf" :}

⟨rules(x86rec.nw) 639d⟩+≡ (630c) <639c 639e>
  inst : Nop() {: "nop" :}

⟨rules(x86rec.nw) 639e⟩+≡ (630c) <639d>
  inst: any [100] {: "<" ^ any ^ ">" :}

  any : True () {: "True" :}
  any : False () {: "False" :}
  any : Link(symbol, w) {: "Link(" ^ symbol#mangled_text ^ "," ^ string_of_int w ^ ")" :}
  any : Late(string, w) {: "Late(" ^ string ^ "," ^ string_of_int w ^ ")" :}
  any : Diff(c1:any,c2:any) {: "Diff(" ^ c1 ^ "," ^ c2 ^ ")" :}
  any : Bits(bits) {: sprintf "Bits(%s)" (Bits.to_string bits) :}

  any : Fetch (any, w) {: "Fetch(" ^ any ^ "," ^ string_of_int w ^ ")" :}

  any : And(x:any, y:any) {: "And(" ^ x ^ "," ^ y ^ ")" :}
  any : Or(x:any, y:any) {: "Or(" ^ x ^ "," ^ y ^ ")" :}
  any : Xor(x:any, y:any) {: "Xor(" ^ x ^ "," ^ y ^ ")" :}
  any : Com(x:any) {: "Com(" ^ x ^ ")" :}

  any : Add(x:any, y:any) {: "Add(" ^ x ^ "," ^ y ^ ")" :}

```



```

any : Mul(x:any, y:any) {: "Mul(" ^ x ^ ", " ^ y ^ ")" :}
any : Fadd(x:any, y:any) {: sprintf "Fadd(%s, %s)" x y :}
any : Fsub(x:any, y:any) {: sprintf "Fsub(%s, %s)" x y :}
any : Fmul(x:any, y:any) {: sprintf "Fmul(%s, %s)" x y :}
any : Fdiv(x:any, y:any) {: sprintf "Fdiv(%s, %s)" x y :}

any : Singlebit(string, x:any, y:any, z:any)
    {: sprintf "Singlebit(%s, %s, %s)" string x y z :}

any : Fneg(any) {: sprintf "Fneg(%s)" any :}
any : Fabs(any) {: sprintf "Fabs(%s)" any :}

any : Fpcc1() {: "Fpcc1()" :}
any : Fpustatus1() {: "Fpustatus()" :}
any : Fpuctl1() {: "Fpuctl1()" :}
any : BitInsert(x:any, y:any, z:any) {: sprintf "BitInsert(%s, %s, %s)" x y z :}
any : BitExtract(lsb:any, y:any, n:w) {: sprintf "BitExtract(%s, %s, %d)" lsb y n :}
any : Slice(n:w, lsb:n, y:any) {: sprintf "Slice(%d, %d, %s)" n lsb y :}
any : Ah2flags(any) {: sprintf "Ah2flags(%s)" any :}
any : Flags2ah(any) {: sprintf "Flags2ah(%s)" any :}

any : Sx(any) {: "Sx(" ^ any ^ ")" :}
any : Zx(any) {: "Zx(" ^ any ^ ")" :}
any : F2f(n:w, w, any) {: sprintf "F2f(%d, %d, %s)" n w any :}
any : F2i(n:w, w, any) {: sprintf "F2i(%d, %d, %s)" n w any :}
any : I2f(n:w, w, any) {: sprintf "I2f(%d, %d, %s)" n w any :}
any : Lobits(any, w) {: "Lobits(" ^ any ^ ", " ^ string_of_int w ^ ")" :}
any : X86_subflags(l:any, r:any) {: "X86_subflags(" ^ l ^ ", " ^ r ^ ")" :}
any : X86_addflags(l:any, r:any) {: "X86_addflags(" ^ l ^ ", " ^ r ^ ")" :}
any : X86ccop(string, cc:any) {: "X86ccop(" ^ string ^ ", " ^ cc ^ ")" :}
any : X86op(string) {: "X86op(" ^ string ^ ")" :}

any : NonLlrOp(string) {: "NonLlrOp(" ^ string ^ ")" :}
any : Shift(string, x:any, y:any) {: sprintf "Shift(%s, %s, %s)" string x y :}

any : Mem(any, c) {: "Mem(" ^ any ^ ", " ^ string_of_int c ^ ")" :}
any : Reg (char, n) {: sprintf "Reg(%s, %d)" (Char.escaped char) n :}
any : Fpreg(any) {: sprintf "Fpreg(%s)" any :}

any : Store (dst:any, src:any, w)
    {: "Store(" ^ dst ^ ", " ^ src ^ ", " ^ string_of_int w ^ ")" :}
any : Kill(any) {: "Kill(" ^ any ^ ")" :}

any : Repmovs(esi:any, edi:any, ecx:any)
    {: "Repmovs(" ^ esi ^ ", " ^ edi ^ ", " ^ ecx ^ ")" :}

any : Guarded(guard:any, any) {: "Guarded(" ^ guard ^ ", " ^ any ^ ")" :}
any : Par(l:any, r:any) {: "Par(" ^ l ^ ", " ^ r ^ ")" :}
any : Goto(any) {: "Goto(" ^ any ^ ")" :}
any : Setflags(any) {: "Setflags(" ^ any ^ ")" :}
any : Llr(dst:any, string, src:any, w)
    {: "Llr(" ^ dst ^ ", " ^ string ^ ", " ^ src ^ ", " ^ string_of_int w ^ ")" :}
any : UnaryInPlace(dst:any, string, w)
    {: "UnaryInPlace(" ^ dst ^ ", " ^ string ^ ", " ^ string_of_int w ^ ")" :}

any : Withaflags(dst:any, string, src:any, w, fo:string)
    {: "Withaflags(" ^ dst ^ ", " ^ string ^ ", " ^ src ^ ", " ^ string_of_int w ^
        ", " ^ fo ^ ")" :}

any : Withlflags(dst:any, string, src:any, w, fo:string)

```

```

{: "Withlflags(" ^ dst ^ "," ^ string ^ "," ^ src ^ "," ^ string_of_int w ^
  "," ^ fo ^ ")" :}

any : Withlflags8H(dst:any, string, src:any, fo:string)
  {: "Withlflags8H(" ^ dst ^ "," ^ string ^ "," ^ src ^ "," ^ fo ^ ")" :}

any : Jcc(string, any) {: "Jcc(" ^ string ^ "," ^ any ^ ")" :}

```

V.12.6 Interfacing RTLs with the Expander

(special cases for particular operators 641)≡ (642b)

```

| RP.App(("and", [w]), [x; y]) -> conAnd (exp x) (exp y)
| RP.App(("or", [w]), [x; y]) -> conOr (exp x) (exp y)
| RP.App(("xor", [w]), [x; y]) -> conXor (exp x) (exp y)
| RP.App(("com", [w]), [x]) -> conCom (exp x)
| RP.App(("add", [w]), [x; y]) -> conAdd (exp x) (exp y)
| RP.App(("sub", [w]), [x; y]) -> conSub (exp x) (exp y)
| RP.App(("addc"|"subb"|"carry"|"borrow") as op, [w]), [x; y; z]) ->
  conSinglebit op (exp x) (exp y) (exp z)
(*| RP.App(("neg", [w]), [x]) -> conNeg (exp x)*)
| RP.App(("mul", [w]), [x; y]) -> conMul (exp x) (exp y)
| RP.App(("sx", [n;w]), [x]) -> conSx (exp x)
| RP.App(("zx", [n;w]), [x]) -> conZx (exp x)
| RP.App(("f2f", [n;w]), [x; rm]) -> conF2f n w (exp x) (* need to assert rm *)
| RP.App(("f2i", [n;w]), [x; rm]) -> conF2i n w (exp x) (* need to assert rm *)
| RP.App(("i2f", [n;w]), [x; rm]) -> conI2f n w (exp x) (* need to assert rm *)
| RP.App(("fadd", [w]), [x; y; rm]) -> conFadd (exp x) (exp y) (* need to assert rm *)
| RP.App(("fsub", [w]), [x; y; rm]) -> conFsub (exp x) (exp y) (* need to assert rm *)
| RP.App(("fmul", [w]), [x; y; rm]) -> conFmul (exp x) (exp y) (* need to assert rm *)
| RP.App(("fdiv", [w]), [x; y; rm]) -> conFdiv (exp x) (exp y) (* need to assert rm *)
| RP.App(("fneg", [w]), [x]) -> conFneg (exp x)
| RP.App(("fabs", [w]), [x]) -> conFabs (exp x)
| RP.App(("x86_fcmp", [w]), [x; y]) -> conFcmp (exp x) (exp y)
| RP.App(("x86_e"|"x86_ne"|"x86_l"|"x86_le"|"x86_g"|"x86_ge"
  | "x86_b"|"x86_be"|"x86_a"|"x86_ae") as o, _) ->
  conX86ccop o (exp x)
| RP.App(("lobits", [w;n]), [x]) -> conLobits (exp x) n
| RP.App(("x86_subflags", [w]), [x; y]) -> conX86_subflags (exp x) (exp y)
| RP.App(("x86_addflags", [w]), [x; y]) -> conX86_addflags (exp x) (exp y)
| RP.App(("x86_ah2flags", []), [x]) -> conAh2flags (exp x)
| RP.App(("x86_flags2ah", []), [x]) -> conFlags2ah (exp x)
| RP.App(("x86_repmovs", [w]), [x;y;z]) -> conRepmovs (exp x) (exp y) (exp z)
| RP.App(("bitInsert", [w; n]), [lsb; dst; src]) ->
  conBitInsert (exp lsb) (exp dst) (exp src)
| RP.App(("bitExtract", [w; n]), [lsb; src]) -> conBitExtract (exp lsb) (exp src) n
| RP.App(("gt"|"lt"|"ge"|"le"|"eq"|"ne"), [w]) as op, [l; r]) ->
  conCmp op (exp l) (exp r)
| RP.App(("shra"|"shrl"|"shl"|"rotr"|"rotr") as op, [w]), [x; y]) -> conShift op (exp x) (exp y)
| RP.App(("quot"|"rem"|"div"|"mod"|"divu"|"modu"|"mulx"|"mulux") as op, [w]), [x; y]) ->
  conNonLlrOp op
| RP.App(("neg") as op, [w]), [x]) -> conNonLlrOp op
| RP.App(("x86_idt_pc", []), [x]) -> conX86IdtPc (exp x)
| RP.App(("add"|"sub"|"mul"|"sx"|"zx"|"lobits"|"x86_subflags"|"bitInsert"|"
  bitExtract"|"fabs"|"fneg"|"fdiv"|"fmul"|"fsub"|"fadd"|"f2f"|"f2i"|"
  i2f"|"and"|"or"|"xor"|"com") as op, ws), xs) ->
  Impossible.impossible
(Printf.sprintf
  "operator %%%s specialized to %d widths & applied to %d arguments"
  op (List.length ws) (List.length xs))

```

```

⟨code to follow the labeler(x86rec.nw) 642a⟩≡ (630c) 642b▷
let rec const = function
| RP.Bool(true)          -> conTrue  ()
| RP.Bool(false)         -> conFalse ()
| RP.Link(s,_,w)         -> conLink s w
| RP.Diff(c1,c2)         -> conDiff (const c1) (const c2)
| RP.Late(s,w)           -> conLate s w
| RP.Bits(b)             -> conBits(b)

⟨code to follow the labeler(x86rec.nw) 642b⟩+≡ (630c) <642a 642c▷
let rec exp = function
| RP.Const(k)             -> const (k)
| RP.Fetch(l,w)           -> conFetch (loc l) w
⟨special cases for particular operators 641⟩
| RP.App((o,_,_) when String.length o > 4 && String.sub o 0 4 =$= "x86_" ->
    conX86op(o)
| RP.App((o,_,_)         -> error ("unknown operator " ^ o)

⟨code to follow the labeler(x86rec.nw) 642c⟩+≡ (630c) <642b 642d▷
and loc l = match l with
| RP.Mem(('m',_,_), (RP.C c), e, ass) -> conMem (exp e) c
| RP.Mem(('f',_,_), c, e, ass) -> conFpreg (exp e)
| (RP.Reg(_,_,_) | RP.Slice(_,_,_)) when RU.Eq.loc l fpcc -> conFpcc1 ()
| RP.Reg(sp, i, w) when RU.Eq.loc l fpustatus -> conFpustatus1()
| RP.Reg(sp, i, w) when RU.Eq.loc l fpuctl -> conFpuctl1()
| RP.Reg((sp,_,_), i, _) -> conReg sp i
| RP.Mem(_,_,_,_) -> error "non-mem, non-reg cell"
| RP.Var _ | RP.Global _ -> error "var found"
| RP.Slice(w,i,l) -> conSlice w i (loc l)

⟨code to follow the labeler(x86rec.nw) 642d⟩+≡ (630c) <642c 642e▷
and effect = function
| RP.Store(l, RP.App((o, _), [RP.Fetch(l',_); r]), w)
    when l =/ l' -> conLlr (loc l) o (exp r) w
| RP.Store(l, RP.App((o, [w']), [RP.Fetch(l',_)]), w)
    when l =/ l' && w = w' -> conUnaryInPlace (loc l) o w
| RP.Store(RP.Reg(('c',_,_), i, w), r, _)
    when (i = SS.indices.SS.pc) -> conGoto (exp r)
| RP.Store(RP.Reg(('c',_,_), i, w), r, _)
    when (i = SS.indices.SS.cc) -> conSetflags (exp r)
| RP.Store(RP.Reg(('c',_,_), i, _), r, w) -> error ("set $c["^string_of_int i^"]")
| RP.Store(l,e,w) -> conStore (loc l) (exp e) w
| RP.Kill(l) -> conKill (loc l)

⟨code to follow the labeler(x86rec.nw) 642e⟩+≡ (630c) <642d 642f▷
and regpair = function
| RP.App(("or",_), [ RP.App(("shl",_), [RP.App(("zx",_), [RP.Fetch(msw,_)]);_)]
    ; RP.App(("zx",_), [RP.Fetch(lsw,_)]))
    -> conRegPair (loc msw) (loc lsw)
| x -> Impossible.impossible "Argument is not a register pair"

⟨code to follow the labeler(x86rec.nw) 642f⟩+≡ (630c) <642e 643a▷
and rtl (RP.Rtl es) = geffects es
and geffects = function
| [] -> conNop ()
| [g, s] -> guarded g s
⟨horrible pattern match for Pairdestwithflags 643c⟩
⟨horrible pattern match for Withflags 643d⟩
⟨horrible pattern match for addc and subb 645⟩
| (g, s) :: t -> conPar (guarded g s) (geffects t)

```

```

and is8 b =
  Bits.width b > 3 & Bits.Ops.eq b (Bits.U.of_int 8 (Bits.width b))
and is32 b =
  Bits.width b > 5 & Bits.Ops.eq b (Bits.U.of_int 32 (Bits.width b))

⟨code to follow the labeler(x86rec.nw) 643a⟩+≡ (630c) ◁642f 648a▷
and guarded g eff = match g with
| RP.Const(RP.Bool b) -> if b then effect eff else conNop()
| RP.App((compare, [32]), [RP.Fetch(RP.Reg(('c', _, _), n, _), _)])
  when n = SS.indices.SS.cc && begins_x86_ compare ->
  (match eff with
  | RP.Store(RP.Reg(('c', _, _), i, _), r, w)
    when (i = SS.indices.SS.pc) ->
    conJcc (String.sub compare 4 (String.length compare - 4)) (exp r)
  | _ -> error "guard on x86 non-branch")
| _ -> conGuarded (exp g) (effect eff)
and begins_x86_ s =
  String.length s >= 4 &&
  s.[0] <= 'x' && s.[1] <= '8' && s.[2] <= '6' && s.[3] <= '_'

⟨truepat 643b⟩≡ (646 645 644 643)
  RP.Const(RP.Bool true)

⟨horrible pattern match for Pairdestwithflags 643c⟩≡ (642f)
  | [ (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for hi := hi32(1 o r at 32) 647d⟩
  )
  ; (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for lo := lo32(1' o' r' at 32) 647e⟩
  )
  ; (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for flags := fo (1'', r'') 647b⟩
  )
] when l == 1' && o == o' && r == r' && l == RP.Fetch(1'', 32) && r == r'
  && is32 k32 && flag_index = SS.indices.SS.cc ->
  conPairdestwithflags (loc hi) (loc lo) (exp l) o (exp r) fo

⟨horrible pattern match for Withflags 643d⟩≡ (642f) 644▷
  | [ (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for l := l' o r at w 646e⟩
  )
  ; (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for l2 := l2' o2 r2 at w2 646h⟩
  )
  ; (
    ⟨truepat 643b⟩
    ,
    ⟨pattern for flags := %x86_undeflags() 646c⟩
  )
] when l == l' && l2 == l2' && w == w2 && flag_index = SS.indices.SS.cc ->
  conWithundeflags (loc l) o o2 (exp r) w

```

```

⟨horrible pattern match for Withflags 644⟩+≡ (642f) <643d
| [ (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for l := l' o r at w 646e⟩
)
; (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for flags := fo (l'', r') 646j⟩
)
] when l =/ l' && l =/ l'' && r =// r' && flag_index = SS.indices.SS.cc ->
  conWithaflags (loc l) o (exp r) w fo
| [ (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for l := l' at w 646i⟩
)
; (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for flags := fo (l'', Const b) 647a⟩
)
] when l =/ l' && l =/ l'' && Bits.is_zero b && flag_index = SS.indices.SS.cc ->
  conWithaflags (loc l) "add" (exp (RP.Const (RP.Bits b))) w fo
| [ (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for l := o l' at w 647c⟩
)
; (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for flags := fo l'' 647f⟩
)
] when l =/ l' && l =/ l'' && flag_index = SS.indices.SS.cc ->
  conWithaflagsunary o (loc l) w fo
| [ (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for d := rp:l1/l2 o r at w 647g⟩
)
; (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for flags := fo (l1'/l2', r') 647i⟩
)
] when l1 =/ l1' && l2 =/ l2' && r =// r' && flag_index = SS.indices.SS.cc ->
  conWithaflags (regpair rp) o (exp r) w fo
| [ (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for d := rp:l1/l2 o r at w 647g⟩
)
; (
  ⟨truepat 643b⟩
  ,
  ⟨pattern for d' := rp':l1'/l2' o' r' at w' 647h⟩
)
; (

```

```

    <truepat 643b>
    ,
    <pattern for flags := %x86_undeflags() 646c>
  )
] when w = w' && r =// r' && rp =// rp' && flag_index = SS.indices.SS.cc ->
  conWithundeflags (regpair rp) o o' (exp r) w
| [ (
    <truepat 643b>
    ,
    <pattern for l := l' o r at w 646e>
  )
; (
    <truepat 643b>
    ,
    <pattern for flags := fo (l'' o' r') 647j>
  )
] when l =/ l' && l =/ l'' && r =// r' && o =/= o'
  && flag_index = SS.indices.SS.cc ->
  conWithlflags (loc l) o (exp r) w fo
| [ (
    <truepat 643b>
    ,
    <pattern for l := l' o r at w at H register 646b>
  )
; (
    <truepat 643b>
    ,
    <pattern for flags := fo flag_left' 646d>
  )
] when is8 eightbits && is8 eightbits' && l =/ l' && l =/ l''
  && flag_left =// flag_left' && flag_index = SS.indices.SS.cc ->
  conWithlflags8H (loc l) o (exp r) fo
| [ (
    <truepat 643b>
    ,
    <pattern for l := l' o r at w at H register 646b>
  )
; -
] when is8 eightbits && is8 eightbits' && l =/ l' && l =/ l'' ->
  conWithlflags8H (loc l) o (exp r) "x86_mumbojumbo"
(*
  $r[0] := %%bitInsert(8, $r[0], %%and(%%bitExtract(8, $r[0]), 69))
| $c[2] := %%x86_logicflags(%%and(%%bitExtract(8, $r[0]), 69))
*)

```

<horrible pattern match for addc and subb 645>≡ (642f) 646a▷

```

| [ (
    <truepat 643b>
    ,
    <pattern for l := o(l',r,CF) at w 646f>
  )
; (
    <truepat 643b>
    ,
    <pattern for flags := fo (l'', r', CF) 646k>
  )
] when l =/ l' && l =/ l'' && r =// r' && flag_index = SS.indices.SS.cc &&
  cf_index1 = flag_index && cf_index2 = flag_index ->
  conWithcarryflags (loc l) o (exp r) w fo

```

```

<horrible pattern match for addc and subb 646a>+≡ (642f) <645
  | [ (
    <truepat 643b>
    ,
    <pattern for l := o(l',r,z1) at w 646g>
  )
; (
  <truepat 643b>
  ,
  <pattern for flags := fo (l'', r', z2) 646l>
)
] when l =/ l' && l =/ l'' && r =// r' && flag_index = SS.indices.SS.cc &&
  Bits.is_zero z1 && Bits.is_zero z2 ->
  conWithcarryzero (loc l) o (exp r) w fo

<pattern for l := l' o r at w at H register 646b>≡ (644)
  RP.Store(l, RP.App(("bitInsert", [32; 8]),
    [RP.Const (RP.Bits eightbits);
     RP.Fetch(l', _);
     RP.App((o, _),
       [ RP.App(("bitExtract", [32; 8]),
         [RP.Const (RP.Bits eightbits'); RP.Fetch(l'',_)]);
        r]) as flag_left]),
    w)

<pattern for flags := %x86_undefflags() 646c>≡ (644 643d)
  RP.Store(RP.Reg(('c', _, _), flag_index, _), RP.App(("x86_undefflags", _), []), _)

<pattern for flags := fo flag_left' 646d>≡ (644)
  RP.Store(RP.Reg(('c', _, _), flag_index, _), RP.App((fo, _), [flag_left']), _)

<pattern for l := l' o r at w 646e>≡ (644 643d)
  RP.Store(l, RP.App((o, _), [RP.Fetch(l',_); r]), w)

<pattern for l := o(l',r,CF) at w 646f>≡ (645)
  RP.Store(l, RP.App((o, _), [RP.Fetch(l',_); r;
    <cfpat 1 647k>
  ]), w)

<pattern for l := o(l',r,z1) at w 646g>≡ (646a)
  RP.Store(l, RP.App((o, _), [RP.Fetch(l',_); r; RP.Const (RP.Bits z1)]), w)

<pattern for l2 := l2' o2 r2 at w2 646h>≡ (643d)
  RP.Store(l2, RP.App((o2, _), [RP.Fetch(l2',_); r2]), w2)

<pattern for l := l' at w 646i>≡ (644)
  RP.Store(l, RP.Fetch(l',_), w)

<pattern for flags := fo (l'', r') 646j>≡ (644)
  RP.Store(RP.Reg(('c', _, _), flag_index, _),
    RP.App((fo, _), [RP.Fetch(l'',_); r']), _)

<pattern for flags := fo (l'', r', CF) 646k>≡ (645)
  RP.Store(RP.Reg(('c', _, _), flag_index, _),
    RP.App((fo, _), [RP.Fetch(l'',_); r';
    <cfpat 2 647l>
  ]), _)

<pattern for flags := fo (l'', r', z2) 646l>≡ (646a)
  RP.Store(RP.Reg(('c', _, _), flag_index, _),
    RP.App((fo, _), [RP.Fetch(l'',_); r';RP.Const (RP.Bits z2)]), _)

```

$\langle \text{pattern for flags} := \text{fo } (l'', \text{Const } b) \text{ 647a} \rangle \equiv \quad (644)$
 $\text{RP.Store}(\text{RP.Reg}('c', _, _), \text{flag_index}, _,$
 $\quad \text{RP.App}(\text{fo}, _, [\text{RP.Fetch}(l'', _); \text{RP.Const } (\text{RP.Bits } b)]), _)$

$\langle \text{pattern for flags} := \text{fo } (l'', r'') \text{ 647b} \rangle \equiv \quad (643c)$
 $\text{RP.Store}(\text{RP.Reg}('c', _, _), \text{flag_index}, _,$
 $\quad \text{RP.App}(\text{fo}, _, [\text{RP.Fetch}(l'', _); r'']), _)$

$\langle \text{pattern for l} := \text{o } l' \text{ at w } \text{647c} \rangle \equiv \quad (644)$
 $\text{RP.Store}(l, \text{RP.App}(\text{o}, _, [\text{RP.Fetch}(l', _)]), w)$

$\langle \text{pattern for hi} := \text{hi32}(l \text{ o } r \text{ at } 32) \text{ 647d} \rangle \equiv \quad (643c)$
 $\text{RP.Store}(\text{hi}, \text{RP.App} ("lobits", [64; 32]),$
 $\quad [\text{RP.App} ("shrl", [64]),$
 $\quad \quad [\text{RP.App}(\text{o}, _, [l; r]);$
 $\quad \quad \text{RP.Const } (\text{RP.Bits } k32)]], 32)$

$\langle \text{pattern for lo} := \text{lo32}(l' \text{ o' } r' \text{ at } 32) \text{ 647e} \rangle \equiv \quad (643c)$
 $\text{RP.Store}(\text{lo}, \text{RP.App} ("lobits", [64; 32]), [\text{RP.App}(\text{o}', _, [l'; r'])]), 32)$

$\langle \text{pattern for flags} := \text{fo } l'' \text{ 647f} \rangle \equiv \quad (644)$
 $\text{RP.Store}(\text{RP.Reg}('c', _, _), \text{flag_index}, _,$
 $\quad \text{RP.App}(\text{fo}, _, [\text{RP.Fetch}(l'', _)]), _)$

$\langle \text{pattern for d} := \text{rp}:l1/12 \text{ o } r \text{ at w } \text{647g} \rangle \equiv \quad (644)$
 $\text{RP.Store}(d, \text{RP.App}(\text{o}, _,$
 $\quad [\text{RP.App}(\text{"or"}, _), [\text{RP.App}(\text{"shl"}, _,$
 $\quad \quad [\text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l1, _)]); _]);$
 $\quad \quad \text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l2, _)]]) \text{ as rp});$
 $\quad r]), w)$

$\langle \text{pattern for d'} := \text{rp}:l1'/12' \text{ o' } r' \text{ at w' } \text{647h} \rangle \equiv \quad (644)$
 $\text{RP.Store}(d', \text{RP.App}(\text{o}', _,$
 $\quad [\text{RP.App}(\text{"or"}, _), [\text{RP.App}(\text{"shl"}, _,$
 $\quad \quad [\text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l1', _)]); _]);$
 $\quad \quad \text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l2', _)]]) \text{ as rp}');$
 $\quad r']), w')$

$\langle \text{pattern for flags} := \text{fo } (l1'/12', r') \text{ 647i} \rangle \equiv \quad (644)$
 $\text{RP.Store}(\text{RP.Reg}('c', _, _), \text{flag_index}, _,$
 $\quad \text{RP.App}(\text{fo}, _,$
 $\quad \quad [\text{RP.App}(\text{"or"}, _), [\text{RP.App}(\text{"shl"}, _,$
 $\quad \quad \quad [\text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l1', _)]); _]);$
 $\quad \quad \quad \text{RP.App}(\text{"zx"}, _, [\text{RP.Fetch}(l2', _)]])$
 $\quad \quad ; r']), _)$

$\langle \text{pattern for flags} := \text{fo } (l'' \text{ o' } r') \text{ 647j} \rangle \equiv \quad (644)$
 $\text{RP.Store}(\text{RP.Reg}('c', _, _), \text{flag_index}, _,$
 $\quad \text{RP.App}(\text{fo}, _, [\text{RP.App}(\text{o}', _, [\text{RP.Fetch}(l'', _); r'])]), _)$

$\langle \text{cflat } 1 \text{ 647k} \rangle \equiv \quad (646f)$
 $\text{RP.App} ("x86_carrybit", _, [\text{RP.Fetch}(\text{RP.Reg}('c', _, _), \text{cf_index1}, _, _)])$

$\langle \text{cflat } 2 \text{ 647l} \rangle \equiv \quad (646k)$
 $\text{RP.App} ("x86_carrybit", _, [\text{RP.Fetch}(\text{RP.Reg}('c', _, _), \text{cf_index2}, _, _)])$

V.12.7 The exported recognizers

```
<code to follow the labeler(x86rec.nw) 648a>+≡ (630c) <643a
let errmsg r msg =
  List.iter prerr_string
    [ "recognizer error: "; msg; " on "; RU.ToString.rtl r; "\n" ]

let to_asm r =
  try
    let plan = rtl (Dn.rtl r) in
    plan.inst.Camlburg.action ()
  with
  | Camlborg.Uncovered -> s " not an instruction: %s" (RU.ToString.rtl r)
  | Error msg -> (errmsg r msg; " error in recognizer: " ^ msg)

let () = Debug.register "x86rec" "diagnose rejected instructions in x86 recognizer"

let is_instruction r =
  try
    let plan = rtl (Dn.rtl r) in
    if plan.inst.Camlburg.cost < 100 then (* should be true, but shade this... *)
      true
    else
      (if Debug.on "x86rec" then
        Printf.eprintf "x86rec: rejected RTL %s\n" (plan.inst.Camlburg.action());
        false)
  with
  | Camlborg.Uncovered -> false
  | Error msg -> (errmsg r msg; false)
```

V.13 arch/x86/x86regs.nw

```
<arch/x86/x86regs.ml 648b>≡
<x86regs.ml 649b>

<arch/x86/x86regs.mli 648c>≡
<x86regs.mli 648d>
```

V.14 X86 registers

```
<x86regs.mli 648d>≡ (648c) 648e▷
val rspace : Register.space
val eax : Register.t
val ecx : Register.t
val edx : Register.t
val ebx : Register.t
val esp : Register.t
val ebp : Register.t
val esi : Register.t
val edi : Register.t

<x86regs.mli 648e>+≡ (648c) <648d 648f▷
val ah : Rtl.loc
val ax : Rtl.loc
val cl : Rtl.loc

<x86regs.mli 648f>+≡ (648c) <648e 649a▷
val regname8 : lsb:int -> base:int -> string
```

```

<x86regs.mli 649a>+≡ (648c) <648f
  val fpuctl : Register.t
  val fputag : Rtl.loc
  val fpustatus : Rtl.loc
  val fpcc      : Rtl.loc
  val fpround   : Register.x

```

V.15 Implementation

```

<x86regs.ml 649b>≡ (648b) 649c▷
  module R = Rtl
  let rspace = ('r', Rtl.Identity, Cell.of_size 32)
  let eax = (rspace, 0, Register.C 1)
  let ecx = (rspace, 1, Register.C 1)
  let edx = (rspace, 2, Register.C 1)
  let ebx = (rspace, 3, Register.C 1)
  let esp = (rspace, 4, Register.C 1)
  let ebp = (rspace, 5, Register.C 1)
  let esi = (rspace, 6, Register.C 1)
  let edi = (rspace, 7, Register.C 1)

  let ah = R.slice 8 ~lsb:8 (R.reg eax)
  let ax = R.slice 16 ~lsb:0 (R.reg eax)
  let cl = R.slice 8 ~lsb:0 (R.reg ecx)

<x86regs.ml 649c>+≡ (648b) <649b 649d▷
  let espace = ('e', Rtl.Identity, Cell.of_size 16)
  let fpuc n = (espace, n, Rtl.C 1)
  let fpuctl = fpuc 0
  let fputag = Rtl.reg (fpuc 2)
  let fpustatus = Rtl.reg (fpuc 1)
  let fpcc = Rtl.slice 3 ~lsb:8 fpustatus (* ignores bit 14 == C3 [probably OK] *)
  let fpround = Register.Slice (2, 10, fpuctl)

<x86regs.ml 649d>+≡ (648b) <649c
  let r8_lo = [| "%al"; "%cl"; "%dl"; "%bl"|]
  let r8_hi = [| "%ah"; "%ch"; "%dh"; "%bh"|]

  let regname8 ~lsb ~base = match lsb with
  | 0 -> Array.get r8_lo base
  | 8 -> Array.get r8_hi base
  | _ -> Impossible.impossible "bad lsb in slice denoting 8-bit register"

```

Appendix W

arch/ppc

W.1 arch/ppc/ppc.nw

$\langle \text{arch/ppc/ppc.ml } 650a \rangle \equiv$
 $\langle \text{ppc.ml } 650d \rangle$

$\langle \text{arch/ppc/ppc.mli } 650b \rangle \equiv$
 $\langle \text{ppc.mli } 650c \rangle$

W.2 Module Structure

$\langle \text{ppc.mli } 650c \rangle \equiv$ (650b)
module Spaces : sig
 val m : Space.t
 val r : Space.t
 val t : Space.t
 val c : Space.t
 val f : Space.t
 val u : Space.t
end
module Post : Postexpander.S
module X : Expander.S
val target : Ast2ir.tgt
val placevars : Ast2ir.proc -> Automaton.t

$\langle \text{ppc.ml } 650d \rangle \equiv$ (650a)
open Nopoly

module A = Automaton
module B = Bits
module B0 = Bits.Ops
module DG = Dag
module PA = Preast2ir
module R = Rtl
module RP = Rtl.Private
module RS = Register.Set
module Up = Rtl.Up
module Dn = Rtl.Dn
module R0 = Rewrite.Ops
module RU = Rtlutil
module PX = Postexpander
module T = Target

```

let rtl r = DG.Rtl r
let (<:>) = DG.<:>
<spaces 651a>
let pc_lhs = nia
let pc_rhs = cia
module F = Mflow.MakeStandard (
  struct
    let pc_lhs = pc_lhs
    let pc_rhs = pc_rhs
    let ra_reg = creg 5
    let ra_offset = 4
  end)
<registers 659e>
module Post = struct
  <ppc postexpander 653a>
end
module X = Expander.IntFloatAddr (Post)
<calling conventions 659d>
<target spec 663a>
<variable placer 664a>

```

W.3 Spaces

<spaces 651a>≡ (650d) 651b▷

```

module Spaces = struct
  module S = Space
  module SS = Space.Standard32
  let bo = Rtl.BigEndian
  let id = Rtl.Identity
  let m = SS.m    bo [8;16;32]
  let r = SS.r 32 id [32]
  let t = SS.t    id 32
  let c = SS.c 7  id [32]

  let flt = { S.space = ('f', id, Cell.of_size 64)
             ; S.doc = "floating point registers"
             ; S.indexwidth = 5
             ; S.indexlimit = Some 32
             ; S.widths = [64]
             ; S.classification = S.Reg
             }

  let f = S.checked flt
  let u = S.checked { flt with
    S.indexwidth = 31
    ; S.space = ('u', id, Cell.of_size 64)
    ; S.indexlimit = None
    ; S.classification =
      S.Temp { S.stands_for = S.stands_for 'f' id 64
              ; S.set_doc = "floating point temporaries"
              }
  }

end

```

<spaces 651b>+≡ (650d) <651a 652a▷

```

let rspace = Spaces.r.Space.space
let fspace = Spaces.f.Space.space
let cspace = Spaces.c.Space.space
let (_, _, mcell) as mspace = Spaces.m.Space.space

```

```

let reg n = R.reg (rspace,n,R.C 1)
let freg n = R.reg (fspace,n,R.C 1)
let creg n = R.reg (cspace,n,R.C 1)
let nia    = creg 0  (* new (next) instr address      *)
let cia    = creg 1  (* current instr address (pc)    *)
let cr     = creg 2  (* condition register            *)
let fpscr  = creg 3  (* flt point condition register  *)
let xer    = creg 4  (* XER register                  *)
let lr     = creg 5  (* link register                  *)
let ctr    = creg 6  (* counter register              *)

(* what follows is true but doesn't work *)
let rmode = R.slice 2 ~lsb:30 fpscr

(* what follows is a lie but works *)
let dspace = ('d', Rtl.Identity, Cell.of_size 2)
let rmode  = Rtl.reg (dspace, 0, R.C 1)
let rresults = Rtl.reg (dspace, 1, R.C 1)

⟨spaces 652a⟩+≡ (650d) <651b 652b>
  let set_flag reg flag_map v flag =
    R.store (R.slice 1 (flag_map flag) reg)
      (R.bits (Bits.U.of_int v 1) 1) 1

⟨spaces 652b⟩+≡ (650d) <652a 652c>
  let crf n = R.slice 4 (n*4) cr  (* CR field      *)
  let crfval n = R.fetch (crf n) 4  (* CR field value *)

  type cr_flag = LT | GT | EQ | SO | FX | FEX | VX | OX
  let cr_flag_to_bit = function
    LT -> 0 | GT -> 1 | EQ -> 2 | SO -> 3
    | FX -> 4 | FEX -> 5 | VX -> 6 | OX -> 7
  let set_cr_flag = set_flag cr cr_flag_to_bit 1
  let clr_cr_flag = set_flag cr cr_flag_to_bit 0

⟨spaces 652c⟩+≡ (650d) <652b 652d>
  (*
  type fpscr_flag =
    FX | FEX | VX | OX | UX | ZX | XX
    | VXSNaN | VXISI | VXIDI | VXZDZ | VXIMZ | VXVC
    | FR | FI | VXSOFT | VXSQRT | VXCVI
    | VE | OE | UE | ZE | XE | NI | RN
  *)

⟨spaces 652d⟩+≡ (650d) <652c>
  type xer_flag = XER_SO | OV | CA
  let xer_flag_to_bit = function XER_SO -> 0 | OV -> 1 | CA -> 2
  let set_xer' = set_flag xer xer_flag_to_bit
  let set_xer_flag fl = function
    XER_SO -> R.par [ set_cr_flag SO ; set_xer' 1 fl ]
    | _ -> set_xer' 1 fl
  let clr_xer_flag fl = function
    XER_SO -> R.par [ clr_cr_flag SO ; set_xer' 0 fl ]
    | _ -> set_xer' 0 fl

```

W.4 Post Expander

```
<ppc postexpander 653a>≡ (650d) 653b▷
  let pc_lhs = pc_lhs
  let pc_rhs = pc_rhs
  let byte_order = R.BigEndian
  let exchange_alignment = 4
  let wordsize = 32
  let memsize = 8
  module Address = struct
    type t = Rtl.exp
    let reg r = R.fetch (R.reg r) (Register.width r)
  end

<ppc postexpander 653b>+≡ (650d) <653a 653c>
  let talloc = PX.Alloc.temp
  let salloc = PX.Alloc.slot

<ppc postexpander 653c>+≡ (650d) <653b 653d>
  module C = Context
  let itempwidth = 32
  let icontext = C.of_space Spaces.t
  let fcontext = C.of_space Spaces.u
  let acontext = icontext
  let rcontext = (fun x y -> Impossible.unimp "Unsupported soft rounding mode")
    , (function (('d',_,_), 0, R.C 1) -> true | _ -> false)

  let overrides =
    ["i2f", [icontext;rcontext], fcontext; (* legitimate exception *)
     "f2f", [icontext;rcontext], fcontext; (* "wrong" exception *)
    ]
  let operators = C.nonbool icontext fcontext rcontext overrides
  let arg_contexts, result_context = C.functions operators
  let constant_context w = if w > wordsize then fcontext else icontext

<ppc postexpander 653d>+≡ (650d) <653c 653e>
  let tloc t = Rtl.reg t
  let twidth = Register.width
  let tval t = R.fetch (tloc t) (twidth t)
  let tstore tmp exp = rtl (R.store (tloc tmp) exp (twidth tmp))

  let mem assn w addr = R.mem assn mspace (Cell.to_count mcell w) addr
  let memval assn w addr = R.fetch (mem assn w addr) w

<ppc postexpander 653e>+≡ (650d) <653d 653f>
  let set_xer2 opr x y = rtl (R.store xer (R.app opr [x; y]) 32)

<ppc postexpander 653f>+≡ (650d) <653e 654a>
  let load ~dst ~addr assn =
    let w = twidth dst in
    assert (w = 8 || w = 16 || w = 32 || w = 64);
    rtl (R.store (tloc dst) (memval assn w addr) w)

  let store ~addr ~src assn =
    let w = twidth src in
    assert (w = 8 || w = 16 || w = 32 || w = 64);
    rtl (R.store (mem assn w addr) (tval src) w)
```

```

<ppc postexpander 654a>+≡ (650d) <653f 654b>
  let lostore ~addr ~src w assn =
    let ext = R0.lobits (twidht src) w (tval src) in
    rtl (R.store (mem assn w addr) ext w)

  let zxload ~dst ~addr w assn =
    tstore dst (R0.zx w (twidht dst) (memval assn w addr))

  let sxload ~dst ~addr w assn =
    zxload dst addr w assn <:>
    tstore dst (R0.sx 8 (twidht dst) (R0.lobits (twidht dst) 8 (tval dst)))

<ppc postexpander 654b>+≡ (650d) <654a 654c>
  (* claude: mirrors arch/x86/x86.ml's move, which elides dst=src - without
   * this, ppcrec.mlb's regl production still fires (it has no guard
   * against dst=src) and prints a real but pointless "mr rN,rN". *)
  let move ~dst ~src = if Register.eq dst src then DG.Nop else tstore dst (tval src)

<ppc postexpander 654c>+≡ (650d) <654b 654d>
  let lix ~dst exp =
    let {Rewrite.hi = hi; Rewrite.lo = lo} = Rewrite.splits 32 16 exp in
    tstore dst hi <:>
    tstore dst (R0.add 32 (tval dst) lo)

  let li ~dst const = lix dst (Up.exp (RP.Const const))

<ppc postexpander 654d>+≡ (650d) <654c 654e>
  (* claude: machine-independent shift+mask, ported from arch/x86/x86.ml's
   * extract; ppcrec.mlb already covers shr1 and lobits so no new recognizer
   * pattern is needed. *)
  let extract ~dst ~lsb ~src =
    let w = twidht src in
    let n = twidht dst in
    let v = if lsb = 0 then tval src else R0.shr1 w (tval src) (R0.unsigned w lsb) in
    tstore dst (if n = w then v else R0.lobits w n v)
  let aggregate ~dst ~src = Impossible.unimp "aggregate"

<ppc postexpander 654e>+≡ (650d) <654d 654f>
  let hwset ~dst ~src = Impossible.unimp "setting hardware register"
  let hwget ~dst ~src = Impossible.unimp "getting hardware register"

<ppc postexpander 654f>+≡ (650d) <654e 656a>
  let machine_env =
    [ "NaN" , [52;64] (* float, [n] -> m *)
    ; "add" , [32] (* int, [n; n] -> n *)
    ; "addc" , [32] (* int, [n; n; 1] -> n *)
    ; "add_overflows" , [32] (* bool, [n; n] -> bool *)
    ; "and" , [32] (* int, [n; n] -> n *)
    ; "bit" , [ ] (* int, [bool] -> 1 *)
    ; "bool" , [ ] (* bool, [1] -> bool *)
    ; "borrow" , [32] (* int, [n; n; 1] -> 1 *)
    ; "carry" , [32] (* int, [n; n; 1] -> 1 *)
    ; "com" , [32] (* int, [n] -> n *)
    ; "conjoin" , [ ] (* bool, [bool; bool] -> bool *)
    ; "disjoin" , [ ] (* bool, [bool; bool] -> bool *)
    ; "div" , [32] (* int, [n; n] -> n *)
    ; "div_overflows" , [32] (* bool, [n; n] -> bool *)
    ; "divu" , [32] (* int, [n; n] -> n *)
    ; "eq" , [32] (* bool, [n; n] -> bool *)
    ; "f2f" , [32;64] (* float, [n; 2] -> m *)

```

```

(*)
; "f2f" , [64;32]
; "f2f_implicit_round" , [64;32] (* float, [n] -> m *)
*)
; "f2i" , [64;32] (* int, [n; 2] -> m *)
; "fabs" , [64] (* float, [n] -> n *)
; "fadd" , [64] (* float, [n; n; 2] -> n *)
; "fcmp" , [64] (* code2, [n; n] -> 2 *)
; "fdiv" , [64] (* float, [n; n; 2] -> n *)
; "feq" , [64] (* bool, [n; n] -> bool *)
; "fge" , [64] (* bool, [n; n] -> bool *)
; "fgt" , [64] (* bool, [n; n] -> bool *)
; "fle" , [64] (* bool, [n; n] -> bool *)
; "float_eq" , [ ] (* code2, [ ] -> 2 *)
; "float_gt" , [ ] (* code2, [ ] -> 2 *)
; "float_lt" , [ ] (* code2, [ ] -> 2 *)
; "flt" , [64] (* bool, [n; n] -> bool *)
; "fmul" , [64] (* float, [n; n; 2] -> n *)
; "fmulx" , [64] (* float, [n; n] -> 2n *)
; "fne" , [64] (* bool, [n; n] -> bool *)
; "fneg" , [64] (* float, [n] -> n *)
; "fordered" , [64] (* bool, [n; n] -> bool *)
; "fsqrt" , [64] (* float, [n; 2] -> n *)
; "fsub" , [64] (* float, [n; n; 2] -> n *)
; "funordered" , [64] (* bool, [n; n] -> bool *)
; "ge" , [32] (* bool, [n; n] -> bool *)
; "geu" , [32] (* bool, [n; n] -> bool *)
; "gt" , [32] (* bool, [n; n] -> bool *)
; "gtu" , [32] (* bool, [n; n] -> bool *)
; "i2f" , [32;64] (* float, [n; 2] -> m *)
; "le" , [32] (* bool, [n; n] -> bool *)
; "leu" , [32] (* bool, [n; n] -> bool *)
; "lobits" , [32; 8] (* int, [n] -> m *)
; "lobits" , [32;16]
; "lt" , [32] (* bool, [n; n] -> bool *)
; "ltu" , [32] (* bool, [n; n] -> bool *)
; "minf" , [64] (* float, [ ] -> n *)
; "mod" , [32] (* int, [n; n] -> n *)
; "modu" , [32] (* int, [n; n] -> n *)
; "mul" , [32] (* int, [n; n] -> n *)
(*)
; "mulux" , [00] (* int, [n; n] -> 2n *)
; "mulx" , [00] (* int, [n; n] -> 2n *)
*)
; "mul_overflows" , [32] (* bool, [n; n] -> bool *)
; "mulu_overflows" , [32] (* bool, [n; n] -> bool *)
; "mzero" , [64] (* float, [ ] -> n *)
; "ne" , [32] (* bool, [n; n] -> bool *)
; "neg" , [32] (* int, [n] -> n *)
; "not" , [ ] (* bool, [bool] -> bool *)
; "or" , [32] (* int, [n; n] -> n *)
; "pinf" , [64] (* float, [ ] -> n *)
; "popcnt" , [32] (* int, [n] -> 1 *)
; "pzero" , [64] (* float, [ ] -> n *)
; "quot" , [32] (* int, [n; n] -> n *)
; "quot_overflows" , [32] (* bool, [n; n] -> bool *)
; "rem" , [32] (* int, [n; n] -> n *)
; "rotl" , [32] (* int, [n; n] -> n *)
; "rotr" , [32] (* int, [n; n] -> n *)
; "round_down" , [ ] (* code2, [ ] -> 2 *)

```



```

; "round_nearest"      ,[ ]      (* code2, [] -> 2          *)
; "round_up"           ,[ ]      (* code2, [] -> 2          *)
; "round_zero"         ,[ ]      (* code2, [] -> 2          *)
; "shl"                ,[32]     (* int, [n; n] -> n      *)
; "shra"               ,[32]     (* int, [n; n] -> n      *)
; "shrl"               ,[32]     (* int, [n; n] -> n      *)
; "sub"                ,[32]     (* int, [n; n] -> n      *)
; "subb"               ,[32]     (* int, [n; n; 1] -> n    *)
; "sub_overflows"      ,[32]     (* bool, [n; n] -> bool  *)
; "sx"                 ,[8; 32]  (* int, [n] -> m         *)
; "sx"                 ,[16;32]  (* int, [n] -> m         *)
; "unordered"          ,[ ]      (* code2, [] -> 2          *)
; "xor"                ,[32]     (* int, [n; n] -> n      *)
; "zx"                 ,[8; 32]  (* int, [n] -> m         *)
; "zx"                 ,[16;32]  (* int, [n] -> m         *)
(*
; "bitExtract"         ,[00]     (* int, [n; n] -> m      *)
; "bitInsert"          ,[00]     (* int, [n; n; m] -> n    *)
; "bitTransfer"        ,[00]     (* int, [n; n; n; n; n] -> n *)
*)
]

```

<ppc postexpander 656a> +≡ (650d) <654f 656b>

```

let unimp_opr (op,ws) =
  let ws' = List.fold_left (fun s i -> string_of_int i ^ " ") "" ws in
  Impossible.unimp ("unimplemented operator " ^ op ^ ":" ^ ws')

let to_binop f = function
  [x;y] -> f 32 (tval x) (tval y)
  | _ -> Impossible.impossible "wrong number of args given to binop"

let rtlop ~dst op args =
  match op with
  | "mod", [32] -> PX.Expand.block (tstore dst (to_binop Rewrite.(mod) args))
  | "modu", [32] -> PX.Expand.block (tstore dst (to_binop Rewrite.modu args))
  | "rem", [32] -> PX.Expand.block (tstore dst (to_binop Rewrite.rem args))
  | opr, ws ->
    if List.mem op machine_env
    then tstore dst (R.app (Up.opr op) (List.map tval args))
    else unimp_opr op

let unop ~dst op x = rtlop dst op [x]
let binop ~dst op x y = rtlop dst op [x;y]
let dblop ~dsthi ~dstlo op x y = Unsupported.mulx_and_mulux()
let wrdop ~dst op x y z = Unsupported.singlebit ~op:(fst op)
let wrdrop ~dst op x y z = Unsupported.singlebit ~op:(fst op)
let unrm ~dst op x rm = Impossible.unimp "floating point"
let binrm ~dst op x y rm = Impossible.unimp "floating point"

```

<ppc postexpander 656b> +≡ (650d) <656a 657a>

```

let block_copy ~dst assn1 ~src assn2 width =
  (* claude: width, like every other width in this signature (e.g.
  * zxload's), is in bits - the copy loop below is byte-indexed, so it
  * must be converted once here. Without this, a plain N-bit scalar
  * memory-to-memory move (e.g. bits32B[dst] := bits32B[src], width=32)
  * was copying N*bytes* (here, 8 words instead of 1) - x86.ml's
  * block_copy does this same "n / 8" conversion for its analogous
  * byte-oriented fallback case. *)
  let width = width / 8 in
  let tmp = talloc 't' 32 in

```

```

let reg_sl = function
  | 32 -> tloc tmp
  | n -> R.slice n (32-n) (tloc tmp) in
let stx d w = rtl (R.store (mem assn1 w (RU.addk 16 dst d))
                        (R.fetch (reg_sl w) w) w)
and ld32 d = rtl (R.store (tloc tmp) (memval assn2 32 (RU.addk 16 src d)) 32) in
let rec copy d =
  match width - d with
  | 1 -> ld32 d <:> stx d 8
  | 2 -> ld32 d <:> stx d 16
  | 3 -> ld32 d <:> stx d 8 <:> stx d 16 (* LOOKING VERY SUSPECT *)
  | 4 -> ld32 d <:> stx d 32
  | n -> assert(n > 0); ld32 d <:> stx d 32 <:> copy (d + 4) in
copy 0

⟨ppc postexpander 657a⟩+≡ (650d) <656b 657b⟩
let br ~tgt = rtl (R.store lr (tval tgt) wordsize),
                  R.store pc_lhs (R.fetch lr wordsize) wordsize
let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) wordsize

⟨ppc postexpander 657b⟩+≡ (650d) <657a 658⟩
let xer_condition = function
  | "eq" | "ne" | "lt" | "le" | "gt" | "ge"
  | "ltu" | "leu" | "gtu" | "geu" -> None
  | "add_overflows"
  | "div_overflows"
  | "mul_overflows"
  | "mulu_overflows"
  | "sub_overflows" -> Some "ppc_xer_ov_set"
  | opr -> Impossible.unimp ("conditional branch on " ^ opr)

let with_xer z x opr y =
  let opr' = "ppc_xer_" ^ opr in
  rtl (R.par [R.store (tloc z) (R.app (R.opr opr [32]) [tval x; tval y]) 32;
              R.store xer (R.app (R.opr opr' []) [tval x; tval y]) 32;
            ])

let bc_setup opr x y = match opr with
  | "add_overflows" -> with_xer (talloc 't' 32) x "add" y
  | "sub_overflows" -> with_xer (talloc 't' 32) x "sub" y
  | "mul_overflows" -> with_xer (talloc 't' 32) x "mul" y
  | "mulu_overflows" -> with_xer (talloc 't' 32) x "mulu" y
  | "div_overflows" -> Impossible.unimp "%div_overflows"
  | _ -> Impossible.impossible "bc_setup"

let bc x ((opr, ws) as op) y ~ifso ~ifnot =
  assert (ws == [wordsize]);
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt 32) in
  match xer_condition opr with
  | None -> DG.Test (DG.Nop, (brtl (R.app (Up.opr op) [tval x; tval y]), ifso, ifnot))
  | Some tst ->
    DG.Test (bc_setup opr x y, (brtl (R.app (R.opr tst []) [R.fetch xer wordsize]),
                                ifso, ifnot))

let bc_guard x ((opr, ws) as op) y =
  assert (ws == [wordsize]);
  match xer_condition opr with
  | None -> (DG.Nop, (R.app (Up.opr op) [tval x; tval y]))
  | Some tst -> (bc_setup opr x y, (R.app (R.opr tst []) [R.fetch xer wordsize]))

let bc_of_guard (setup, guard) ~ifso ~ifnot =
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt 32) in

```

```
DG.Test (setup, (brtl guard, ifso, ifnot))
```

```
let negate = function
| "ne"          -> "eq"
| "eq"          -> "ne"
| "ge"          -> "lt"
| "gt"          -> "le"
| "le"          -> "gt"
| "lt"          -> "ge"
| "geu"         -> "ltu"
| "gtu"         -> "leu"
| "leu"         -> "gtu"
| "ltu"         -> "geu"
| "feq"
| "fne"
| "flt"
| "fle"
| "fgt"
| "fge"
| "fordered"
| "funordered"  -> Impossible.unimp "floating-point comparison"
| _             -> Impossible.impossible
                  "bad comparison in expanded MIPS conditional branch"
```

```
let bnegate r = match Dn.rtl r with
|          RP.Rtl [RP.App( (op,          [32]),[x;y]), RP.Store (pc,tgt,32)]
  when RU.Eq.loc pc (Dn.loc pc_lhs) ->
    Up.rtl (RP.Rtl [RP.App( (negate op,[32]),[x;y]), RP.Store (pc,tgt,32)])
| _ -> Impossible.impossible "ill-formed MIPS conditional branch"
```

<ppc postexpander 658>+≡ (650d) <657b 659a>

```
(* claude: "call = b" (unconditional branch, no return-address save)
* meant NO call on ppc could ever return to its actual caller -
* whatever function was called eventually did "blr", which returns to
* wherever lr last held a meaningful value, i.e. all the way back to
* the very first real hardware "bl" (deep in libc's startup code
* before it ever reaches this program's own entry point). That
* explains both demos/hello_ppc.c--'s wrong exit code (return(0) never
* executes; whatever raw register printf's own return left in r3
* becomes the "return value of main" instead) and the cut/altret
* family's garbage output. Fixed to build the same
* "Par(Goto(target), Store(lr, cia+4))" shape ppcrec.mlb's bl rule
* expects, mirroring x86.ml's call (R.par [store pc; push return
* addr]) - Goto first, lr-store second, since Par's constructor
* argument order matters for the grammar match. Root-caused via
* gdb-over-qemu-gdbserver on tests/cmm/altread.c--: a breakpoint right
* after a "b open" call site was never hit even though the real
* open(2) syscall plainly executed - it returned straight past this
* function's own dispatch logic. *)
```

```
let call ~tgt =
  DG.Nop, R.par [ R.store pc_lhs (Up.const tgt) wordsize
                  ; R.store lr (RU.addk wordsize (R.fetch cia wordsize) 4) wordsize ]
(* claude: callr (indirect call through a register) is likely the same
* bug - br sets lr := tgt then jumps through lr, which looks built for
* some other purpose (see cut_to below, which does something similar)
* rather than "save a return address, then jump". Not yet fixed: none
* of the cases investigated so far (cut, altret, foreign C calls) go
* through an indirect call, so there is no failing case yet to verify
* a fix against. *)
let callr = br
```

```

(* THIS IS SUSPECT -- WHY ARE WE SETTING THE LINK REGISTER? *)
let cut_to {Mflow.new_sp = sp'; Mflow.new_pc = pc'} =
  let effs = [R.store pc_lhs (R.fetch lr wordsize) wordsize; R.store sp sp' wordsize] in
  rtl (R.store lr pc' wordsize), R.par effs

<ppc postexpander 659a>+≡ (650d) <658 659b>
  let return = fmach.Mflow.return
  let forbidden = Rtl.par [] (* BOGUS: NEEDS TO BE A REAL FAULTING INSTRUCTION *)

<ppc postexpander 659b>+≡ (650d) <659a 659c>
  let don't_touch_me _ = false

<ppc postexpander 659c>+≡ (650d) <659b>
  include Postexpander.Nostack(Address)

```

W.5 Calling Conventions

```

<calling conventions 659d>≡ (650d) 660>
  module C = Call

<registers 659e>≡ (650d)
  let sp_reg = (rspace, 1, R.C 1)
  let sp      = R.reg sp_reg
  let vfp     = Vfp.mk 32
  let fmach   = F.machine sp

  let rreg n = (rspace, n, R.C 1)
  let regset s l = RS.of_list (List.map (fun r-> (s,r,R.C 1)) l)
  (* claude: r2 and r13 are dedicated/reserved in the 32-bit PowerPC SVR4
   * ABI, not general-purpose - r2 is a system-reserved pointer and r13 is
   * glibc/Linux's TLS base register, both expected to survive any call
   * unmodified by the caller. Including them here let the allocator hand
   * them out as ordinary scratch registers, so any call to external code
   * (e.g. printf) that itself read r2/r13 crashed on whatever we had left
   * in them. Root-caused via a 'lwz r11,-28680(r2)' segfault inside
   * glibc's own printf. *)
  let vregs = regset rspace [3;4;5;6;7;8;9;10;11;12]
  let nvregs = regset rspace [14;15;16;17;18;19;20;21;
                              22;23;24;25;26;27;28;29;30;31]
  let fvregs = regset fspace [0;1;2;3;4;5;6;7;8;9;10;11;12;13]
  let fnvregs = regset fspace [14;15;16;17;18;19;20;21;22;
                               23;24;25;26;27;28;29;30;31]
  let cvregs = regset cspace [5(*lr*); 6(*ctr*); 4(*xer*); 2(*cr*)]

  (* let volregs = RS.union (RS.union vregs fvregs) cvregs *)
  (* let nvolregs = RS.union nvregs fnvregs *)
  let volregs = RS.union vregs cvregs
  let nvolregs = nvregs

<transformations(ppc.nw) 659f>≡ (660)
  let autoAt = A.at mspace in
  let linkage_base r = addk (Block.base r.A.overflow) (-24) in
  let call_actuals =
    C.outgoing ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "out call parms" autoAt specs.A.call)
    ~autosp:linkage_base
    ~postsp:(fun _ sp -> sp)

```

```

and prolog =
  let autosp = (fun _ -> vfp) in
  C.incoming ~growth ~sp
    ~mkauto:(fun () -> autoAt (addk vfp 24) specs.A.call)
    (* specific address is redundant, but eqn solver should take in stride *)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location)
    ~insp:(fun a _ _ -> autosp a)
and call_results =
  C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt specs.A.results)
    ~autosp:linkage_base
    ~postsp:(fun _ _ -> std_sp_location) (* irrelevant? *)
    ~insp:(fun a _ _ -> linkage_base a)
and epilog =
  C.outgoing ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt specs.A.results)
    ~autosp:linkage_base
    ~postsp:(fun _ r -> r)
(* ~postsp:(fun _ r -> vfp)*) (* irrelevant *)
and also_cuts_to =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
    ~mkauto:(fun () -> Block.srelative vfp "in cont parms" autoAt specs.A.cutto)
    ~autosp
    ~postsp:(fun _ _ -> std_sp_location)
    ~insp:(fun a _ _ -> autosp a)
and cut_actuals base =
  C.outgoing ~growth ~sp ~mkauto:(fun () -> autoAt base specs.A.cutto)
    ~autosp:(fun r -> spval)
    ~postsp:(fun _ _ -> spval)
and saved_nvr temps =
  let t = Talloc.Multiple.loc temps 't' in
  fun r -> t (Register.width r)

```

<calling conventions 660>+≡

(650d) <659d

```

let cconv name specs =
  let ws = Post.wordsize
  and growth = Memalloc.Down
  and spval = R.fetch sp 32
  and addk = RU.addk 32
  and std_sp_location = RU.add 32 vfp (R.late "minus frame size" 32)
  in
  <transformations(ppc.nw) 659f>
  in
  { C.name = name
  ; C.overflow_alloc = { C.parameter_deallocator = C.Caller
                        ; C.result_allocator = C.Caller
                        }
  ; C.call_parms = { C.in' = prolog; C.out = call_actuals}
  ; C.cut_parms = { C.in' = also_cuts_to; C.out = cut_actuals}
  ; C.results = { C.in' = call_results; C.out = epilog}

  ; C.stack_growth = growth
  ; C.stable_sp_loc = std_sp_location
  ; C.jump_tgt_reg = R.reg (rreg 7)
  ; C.replace_vfp = Vfprewrite.replace_with ~sp
  ; C.sp_align = 4
  ; C.pre_nvregs = nvolsregs
  ; C.volregs = volsregs

```

```

; C.saved_nvr      = saved_nvr
; C.return         = (fun k n ~ra -> R.store nia (R.fetch lr ws) ws)
; C.ra_on_entry    = (fun b      -> R.fetch lr ws)
(* ; C.where_to_save_ra = (fun e t      -> Post.mem R.none ws sp_8) *)
; C.where_to_save_ra = (fun _ t      -> Talloc.Multiple.loc t 't' ws)
; C.ra_on_exit      = (fun l e t -> lr)
; C.sp_on_unwind    = (fun e      -> RU.store sp e)
(* claude: was "fun _ _ -> Rtl.null" - a genuine no-op, so a tail call
* ("jump") never restored sp before re-entering the target's own
* prologue, which unconditionally does its own "addi sp,sp,-N". Every
* tail-recursive iteration (tests/cmm/tail.c--'s "down") therefore
* shrank sp by another frame instead of reusing the current one,
* corrupting whatever was below the original frame after enough
* iterations - not an immediate crash, but silent argument corruption
* (confirmed via gdb: i/n read back as garbage after ~20 iterations)
* that looks like an infinite loop/hang from outside. Mirrors
* x86call.ml's sp_on_jump (RU.store sp (addk (Block.base b) -4)) using
* this backend's own -24 linkage-area constant, the same one
* call_actuals/call_results/epilog already use via linkage_base above
* for the equivalent non-tail-call case. *)
; C.sp_on_jump      = (fun b _      -> RU.store sp (addk (Block.base b) (-24)))
}

```

<PPC calling convention automata in Lua 661a>≡ 661b▷

```

A      = Automaton
PPC     = PPC          or {}
PPC.cc  = PPC.cc       or {}
PPC.cc["C"  ] = PPC.cc["C"  ] or {}
PPC.cc["C--" ] = PPC.cc["C--" ] or {}
PPC.cc["notail"] = PPC.cc["notail"] or {}

```

```

function PPC.r(i) return({ space = "r", index = i, cellsize = 32, count = 1 }) end
function PPC.f(i) return({ space = "f", index = i, cellsize = 64, count = 1 }) end

```

```

PPC.overflow = A.overflow { growth = "up", max_alignment = 4 }

```

<PPC calling convention automata in Lua 661b>+≡ <661a 662a▷

```

PPC.cc["C"].call =
{ A.widen(32, "multiple")
, A.bitcounter("bits")
, A.choice
  { "float", { A.widen(64),
    A.useregs(
      {PPC.f(1),PPC.f(2),PPC.f(3),PPC.f(4),PPC.f(5),
      PPC.f(6),PPC.f(7),PPC.f(8),PPC.f(9),PPC.f(10),
      PPC.f(11),PPC.f(12),PPC.f(13)}
      ,"reserve")
    }
  , A.is_any, A.regs_by_bits("bits",
    {PPC.r(3),PPC.r(4),PPC.r(5),PPC.r(6),
    PPC.r(7),PPC.r(8),PPC.r(9),PPC.r(10)}, "reserve")
  }
, PPC.overflow
}

```

```

PPC.cc["C"].results =
  A.choice { "float" , { A.widen(64), A.useregs { PPC.f(1) }}
    , A.is_any, { A.widen(32), A.useregs { PPC.r(3), PPC.r(4) }}
  }

```

```

PPC.cc["C"].cutto = { A.widen(32), PPC.overflow }

```

```

<PPC calling convention automata in Lua 662a>+≡ <661b 662b>
PPC.cc["C--"].call    = PPC.cc["C"].call
PPC.cc["C--"].results = { PPC.cc["C"].results, PPC.overflow }
PPC.cc["C--"].cutto   = PPC.cc["C"].cutto

<PPC calling convention automata in Lua 662b>+≡ <662a>
A.register_cc(Backend.ppc.target,"C"      ,PPC.cc["C"  ])
A.register_cc(Backend.ppc.target,"C'"     ,PPC.cc["C"  ])
A.register_cc(Backend.ppc.target,"C--"    ,PPC.cc["C--"])
A.register_cc(Backend.ppc.target,"notail",PPC.cc["C--"])

<PPC stack layout in Lua 662c>≡ 662d>
PPC = PPC or {}
PPC.layout = { creates='no late consts' }

<PPC stack layout in Lua 662d>+≡ <662c>
function PPC.layout.fn(dummy,proc) --- dispatch on cc name
    local layout = PPC.layout[Stack.ccname(proc)]
    if not layout then
        error('Convention "' .. Stack.ccname(proc) .. '" does not have a frame layout')
    end
    return layout(dummy, proc, 'varargs')
    -- might one day optimize and not always pass 'varargs'
end

function PPC.layout["C"](dummy, proc, varargs)
    local blocks = Stack.blocks(proc)

    local old, young = blocks.oldblocks, blocks.youngblocks
    old.callee = Block.overlap_high(32, old.callee)
    old.caller  = Block.overlap_low (32, old.caller)
    young.callee = Block.overlap_high(32, young.callee)
    young.caller = Block.overlap_low (32, young.caller)

    local linkage =
        { caller = Block.relative(blocks.vfp, "caller's linkage area", 24, 16)
          , callee = Block.relative(blocks.sp , "our linkage area"      , 24, 16)
          }      -- Mach-O Runtime Conventions for PowerPC, pp47--48

    local youngparms = Block.cat(32, {young.caller, young.callee})
    if varargs then
        youngparms = Block.overlap_low(32, {youngparms,
            Block.relative(blocks.vfp, "varargs flush area", 32, 4) })
    end

    local layout =
        { Block.cat (32, {old.caller, old.callee, linkage.caller})
          , blocks.vfp
          , blocks.spills
          , blocks.continuations
          , blocks.stackdata
          , Block.cat (32, {youngparms, linkage.callee})
          , blocks.sp
          }

    if Debug.stack then
        blocks.caller_linkage = linkage.caller
        blocks.callee_linkage = linkage.callee
        write('***** cc name = ', Stack.ccname(proc), '\n')
        Debug.showblocks (blocks, { 'oldblocks'

```

```

        , 'caller_linkage'
        , 'vfp'
        , 'spills'
        , 'continuations'
        , 'stackdata'
        , 'youngblocks'
        , 'callee_linkage'
        , 'sp'
    })

end

local block = Block.cat(32, layout)
block = Block.adjust(block)
Stack.freeze(proc, block)
return 1
end
PPC.layout["C--"] = PPC.layout["C"] -- flagrant waste
PPC.layout["notail"] = PPC.layout["C"] -- flagrant waste

```

W.6 Target Specification

```

⟨target spec 663a⟩≡ (650d) 663b▷

⟨target spec 663b⟩+≡ (650d) <663a 663c▷
let ( *> ) = A.( *> )
let globals base =
  let width w = if w <= 8 then 8 else if w <= 16 then 16 else Auxfuns.round_up_to 32 w in
  let align = function 8 -> 1 | 16 -> 2 | _ -> 4 in
  A.at mspace ~start:base
  (A.widen width *> A.align_to align *>
   A.overflow ~growth:Memalloc.Up ~max_alignment:4)

let spill lookup reg loc =
  let w = Register.width reg in
  [ Automaton.store loc (Rtl.fetch (Rtl.reg reg) w) w ]

let reload lookup reg loc =
  let w = Register.width reg in
  [ Rtl.store (Rtl.reg reg) (Automaton.fetch loc w) w ]

⟨target spec 663c⟩+≡ (650d) <663b
let target =
  let spaces = [ Spaces.m; Spaces.r; Spaces.t; Spaces.c; Spaces.f; Spaces.u ] in
  PA.T { T.name = "ppc"
    ; T.memspace = mspace
    ; T.max_unaligned_load = R.C 1
    (* basic metrics and spaces are OK *)
    ; T.byteorder = Post.byte_order
    ; T.wordsize = Post.wordsize
    ; T.pointersize = Post.wordsize
    ; T.vfp = Space.Standard32.vfp
    ; T.alignment = 1
    ; T.memsize = Post.memsize
    ; T.spaces = spaces
    ; T.reg_ix_map = T.mk_reg_ix_map spaces
    ; T.distinct_addr_sp = false

    (* Does the PPC really implement IEEE 754? I think so *)
  }

```



```

; T.float = Float.ieee754

(* control flow is solid, except [[cutto]] is a lie *)
; T.machine = X.machine
; T.cc_specs      = Ppccc.cc_specs
; T.cc_spec_to_auto = cconv

; T.is_instruction = Ppcrec.M.is_instruction
; T.tx_ast = (fun secs -> secs)
; T.capabilities = { T.operators = List.map Up.opr Post.machine_env;
                    T.litops = [];
                    T.literals = [32;64]; T.memory = [8;32];
                    T.block_copy = false; T.itemps = [32]; T.ftemps = [];
                    T.iwiden = true; T.fwiden = false; }

(* global or hardware registers *)
; T.globals = globals
; T.rounding_mode = rmode
; T.named_locs = Strutil.assoc2map
                  ["IEEE 754 rounding mode", rmode
                  ;"IEEE 754 rounding results", rresults
                  ]

(* bogosity *)
; T.data_section = "data" (* NOT REALLY A PROPERTY OF THE TARGET MACHINE... *)
; T.charset = "latin1" (* THIS IS NONSENSE!! NOT A PROPERTY OF THE MACHINE?? *)
}

```

W.7 Variable Placer

```

⟨variable placer 664a⟩≡ (650d)
let unimp      = Impossible.unimp
let impossible = Impossible.impossible

let placevars =
  let is_float w kind _ = w <= 32 && kind = $= "float" in
  let warn ~width:w ~alignment:a ~kind:k =
    if w > 64 then unimp (Printf.sprintf "%d-bit values not supported" w) in
  let mk_stage ~temps =
    A.choice
    [ is_float,      A.widen (Auxfuns.round_up_to ~multiple_of: 64);
      (fun w _ _ -> w <= 32), A.widen (fun _ -> 32) *> temps 't';
      A.is_any,      A.widen (Auxfuns.round_up_to ~multiple_of: 8);
    ] in
  Placevar.mk_automaton ~warn ~vfp ~memspace:mspace mk_stage

```

W.8 arch/ppc/ppcasm.nw

```

⟨arch/ppc/ppcasm.ml 664b⟩≡
  ⟨ppcasm.ml 665b⟩

⟨arch/ppc/ppcasm.mli 664c⟩≡
  ⟨ppcasm.mli 665a⟩

```

W.9 PPC Assembler

```
<ppcasm.mli 665a>≡ (664c)
val make :
  (('a, 'b, 'c, 'd) Proc.t -> 'cfg -> (Zipcfg.Rep.call -> unit) ->
    (Rtl.rtl -> unit) -> (string -> unit) -> unit) ->
  out_channel -> ('cfg * ('a, 'b, 'c, 'd) Proc.t) Asm.assembler
(* pass Cfgutil.emit *)
```

W.9.1 Implementation

```
<ppcasm.ml 665b>≡ (664b)
open Nopoly
```

```
module G = Zipcfg
module GR = Zipcfg.Rep
module SM = Strutil.Map
<utilities(ppcasm.nw) 665c>
<definitions(ppcasm.nw) 665e>
let make emitter fd = new asm emitter fd
```

```
<utilities(ppcasm.nw) 665c>≡ (665b) 667g▷
let fprintf = Printf.fprintf
```

```
<definition of manglespec (for the name mangler)(ppcasm.nw) 665d>≡ (665e)
let spec =
  let reserved = [] in (* list reserved words here so we can avoid them *)
  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '.'
    | '_' -> true
    | _ -> false in
  let replace = function
    | x when id x -> x
    | _ -> '_'
  in
  { Mangler.preprocess = (fun x -> "_" ^ x)
  ; Mangler.replace = replace
  ; Mangler.reserved = reserved
  ; Mangler.avoid = (fun x -> x ^ "_")
  }
```

```
<definitions(ppcasm.nw) 665e>≡ (665b)
<definition of manglespec (for the name mangler)(ppcasm.nw) 665d>
```

```
class ['cfg, 'a, 'b, 'c, 'd] asm emitter fd
: ['cfg * ('a, 'b, 'c, 'd) Proc.t] Asm.assembler =
object (this)
  val _fd = fd
  val _mangle = (Mangler.mk spec)
  val mutable _syms = SM.empty
  method globals _ = ()
  method private new_symbol name =
    let s = Symbol.with_mangler _mangle name in
    _syms <- SM.add name s _syms;
    s
```

```

    <private assembly state(ppcasm.nw) 667c>
    method private print l = List.iter (output_string _fd) l

    <assembly methods(ppcasm.nw) 666a>
end

<assembly methods(ppcasm.nw) 666a>≡ (665e) 666b▷
val imports = ref ([] : string list)

method import s =
    let sym = this#new_symbol s in
    imports := ("_" ^ s)::(!imports);
    output_string _fd ".picsymbol_stub\n";
    Printf.fprintf _fd "L_%s$stub:\n" s;
    Printf.fprintf _fd "\t.indirect_symbol _%s\n" s;
    output_string _fd "\tmflr r0\n";
    Printf.fprintf _fd "\tbcl 20,31,L_%s$pb\n" s;
    Printf.fprintf _fd "L_%s$pb:\n" s;
    output_string _fd "\tmflr r11\n";
    Printf.fprintf _fd "\taddis r11,r11,ha16(L_%s$lz-L_%s$pb)\n" s s;
    output_string _fd "\tmtlr r0\n";
    Printf.fprintf _fd "\tlwz r12,lo16(L_%s$lz-L_%s$pb)(r11)\n" s s;
    output_string _fd "\tmtctr r12\n";
    Printf.fprintf _fd "\taddi r11,r11,lo16(L_%s$lz-L_%s$pb)\n" s s;
    output_string _fd "\tbctr\n";
    output_string _fd ".lazy_symbol_pointer\n";
    Printf.fprintf _fd "L_%s$lz:\n" s;
    Printf.fprintf _fd "\t.indirect_symbol _%s\n" s;
    output_string _fd "\t.long dyld_stub_binding_helper\n";
    sym

<assembly methods(ppcasm.nw) 666b>+≡ (665e) <666a 666c>
method export s =
    let sym = this#new_symbol s in
    Printf.fprintf _fd ".globl %s\n" sym#mangled_text;
    sym

<assembly methods(ppcasm.nw) 666c>+≡ (665e) <666b 666e>
method local s = this#new_symbol s

<PPC assembly interface procedures 666d>≡ 667a▷
static AsmSymbol asm_common(name, size, align, section)
    char *name; int size; int align; char *section;
{
    AsmSymbol s;
    assert(section == NULL);
    if (solaris)
        print(".common %s,%d,%d\n", name, size, align);
    else
        print(".common %s,%d\n", name, size);
    s = asm_sym_insert(asmtab, name, ASM_COMMON);
    s->u.common.size = size;
    s->u.common.align = align;
    return s;
}

<assembly methods(ppcasm.nw) 666e>+≡ (665e) <666c 667b>▷
method label (s: Symbol.t) = fprintf _fd "%s:\n" s#mangled_text

```

```

<PPC assembly interface procedures 667a>+≡                                     <666d
    static void asm_function (name) char *name; {
        print(".type %s,@function", name);
    }

<assembly methods(ppcasm.nw) 667b>+≡                                     (665e) <666e 667d>
    method section name =
        _section <- name;
        if name != "text" then fprintf _fd ".text\n"
        else fprintf _fd ".section __DATA,%s\n" name
    method current = _section

<private assembly state(ppcasm.nw) 667c>≡                               (665e)
    val mutable _section = "bogus section"

<assembly methods(ppcasm.nw) 667d>+≡                                     (665e) <667b 667e>
    method org n = fprintf _fd ".org %d\n" n
    method align n =
        let rec lg = function
            | 0 -> 0
            | 1 -> 0
            | n -> 1 + (lg (n/2))
        in
        if n <> 1 then fprintf _fd ".align %d\n" (lg n)
    method addloc n =
        if n <> 0 then fprintf _fd ".space %d\n" n

<assembly methods(ppcasm.nw) 667e>+≡                                     (665e) <667d 667f>
    method zeroes (n:int) = fprintf _fd ".space %d, 0\n" n

<assembly methods(ppcasm.nw) 667f>+≡                                     (665e) <667e 667h>
    method value (v:Bits.bits) =
        let altfmt = Bits.to_hex_or_decimal_string ~declimit:256 in
        match Bits.width v with
        | 8 -> fprintf _fd ".byte %Ld\n" (Bits.S.to_int64 v)
        | 16 -> fprintf _fd ".short %s\n" (altfmt v)
        | 32 -> fprintf _fd ".long %s\n" (altfmt v)
        | 64 ->
            let i = Bits.U.to_int64 v in
            fprintf _fd ".long 0x%Lx\n" (Int64.shift_right_logical i 32);
            fprintf _fd ".long 0x%Lx\n" (Int64.logand i mask32)
        | w -> Impossible.unimp ("emission width " ^ string_of_int w ^ " in ppc assembler")

<utilities(ppcasm.nw) 667g>+≡                                           (665b) <665c
    let mask32 = Int64.pred (Int64.shift_left Int64.one 32)

<assembly methods(ppcasm.nw) 667h>+≡                                     (665e) <667f 667i>
    method addr a =
        match Reloc.if_bare a with
        | Some b -> this#value b
        | None -> let const bits = Printf.sprintf "0x%Lx" (Bits.U.to_int64 bits) in
            assert (Reloc.width a = 32);
            fprintf _fd ".long %s\n" (Asm.reloc_string const a)

<assembly methods(ppcasm.nw) 667i>+≡                                     (665e) <667h 667j>
    method emit = ()

<assembly methods(ppcasm.nw) 667j>+≡                                     (665e) <667i 668a>
    method comment s = fprintf _fd "; %s\n" s

    method const (s: Symbol.t) (b:Bits.bits) =
        fprintf _fd ".set %s, 0x%Lx" s#mangled_text (Bits.U.to_int64 b)

```

```

<assembly methods(ppcasm.nw) 668a>+≡ (665e) <667j
method private instruction rtl =
  output_string _fd "\t";
  output_string _fd (Ppcrec.M.to_asm rtl (!imports));
  output_string _fd "\n"

method longjmp_size () = 4
method private call node =
  let longjmp edge = fprintf _fd "\tb %s\n" (_mangle (snd edge.G.node)) in
  let rec output_altret_jumps n edges = (* emit n jumps *)
    if n > 0 then
      match edges with
      | edge :: edges -> (longjmp edge; output_altret_jumps (n-1) edges)
      | [] -> Impossible.impossible "contedge count" in
  begin
    fprintf _fd "%s\n" (Ppcrec.M.to_asm node.GR.cal_i []); (* NOTE BOGUS ARG [] *)
    output_altret_jumps node.GR.cal_altrets (List.tl node.GR.cal_contedges);
  end

method cfg_instr (cfg, proc) =
  let symbol = proc.Proc.symbol in
  let label l = this#label (try SM.find l _syms with Not_found -> this#local l) in
  this#label symbol;
  (emitter proc cfg (this#call) (this#instruction) label : unit)

```

W.10 arch/ppc/ppcrec.nw

```

<arch/ppc/ppcrec.mli 668b>≡
<ppcrec.mli 668c>

```

W.11 PPC Recognizer

```

<ppcrec.mli 668c>≡ (668b)
module M : sig
  val is_instruction : Rtl.rtl -> bool
  (* claude: elf defaults to false (Mach-0's bare "r3" register syntax);
   * ppcasm.ml passes ~elf:true for GNU as/clang's required "%r3". *)
  val to_asm : ?elf:bool -> Rtl.rtl -> string list -> string
end

<ppcrec.mlb 668d>≡
%head {:
  <modules(ppcrec.nw) 669a>
  module M = struct
    <code to precede the labeler(ppcrec.nw) 669b>
  :}
%tail {:
  <code to follow the labeler(ppcrec.nw) 673a>
  end (* of M *)
:}

%term
  <names of types of terminals(ppcrec.nw) 669e>
%%
  <rules(ppcrec.nw) 670a>

```

```

<modules(ppcrec.nw) 669a>≡ (668d)
open Nopoly
module BO = Bits.Ops
module RP = Rtl.Private
module RU = Rtlutil
module SS = Space.Standard32
module Down = Rtl.Dn (* Convert Down to private repr. *)
module Up = Rtl.Up (* Convert Up to abstract repr. *)

exception Error of string
let error msg = raise (Error msg)
let sprintf = Printf.sprintf

```

W.11.1 Utilities

```

<code to precede the labeler(ppcrec.nw) 669b>≡ (668d) 669c▷
let s = Printf.sprintf

<code to precede the labeler(ppcrec.nw) 669c>+≡ (668d) <669b 669d>
let infinity = Camlburg.inf_cost
let guard b = if b then 0 else infinity

<code to precede the labeler(ppcrec.nw) 669d>+≡ (668d) <669c
let imports = ref ([] : string list)

(* claude: GNU as/clang's integrated PowerPC assembler reject
 * bare register names (e.g. "mr r3,r3" errors as an
 * "unsupported relocation against r3", since without a
 * distinguishing prefix "r3" reads as a symbol); they require
 * "%r3". Mach-0's assembler wants the bare form instead, so
 * this is a runtime switch rather than a hardcoded string -
 * set from to_asm's elf argument (see ppcasm.ml vs
 * ppcmach.ml). *)
let elf_syntax = ref false
let reg_str n = if !elf_syntax then s "%r%d" n else s "r%d" n
let cr0_str () = if !elf_syntax then "%cr0" else "cr0"
let r0_str () = reg_str 0

(* claude: absolute (non-PIC) hi/lo halves of a symbol's
 * address - GNU as/clang want "sym@ha"/"sym@l"; Mach-0 wants
 * "ha16(sym)"/"lo16(sym)", same as the PIC-relative case below
 * uses for a symbol difference instead of a bare symbol. *)
let abs_ha sym = if !elf_syntax then s "%s@ha" sym else s "ha16(%s)" sym
let abs_lo sym = if !elf_syntax then s "%s@l" sym else s "lo16(%s)" sym

let ind_addr name =
  if List.exists ((=$=) name) (!imports) then "L" ^ name ^ "$stub" else name

let ppc_op = function
| "ltu" -> ("l", "lt")
| "leu" -> ("l", "le")
| "gtu" -> ("l", "gt")
| "geu" -> ("l", "ge")
| op -> ("", op)

```

W.11.2 Recognizer Rules

```

<names of types of terminals(ppcrec.nw) 669e>≡ (668d)
n w bits symbol

```

$\langle \text{rules}(\text{ppcrec.nw}) \text{ 670a} \rangle \equiv$ (668d) 670b $\triangleright$

```

const16: Bits(bits) [{: guard (Bits.S.fits 16 bits) :}]
           {: Bits.to_decimal_string bits :}

k15 : Bits(bits)
      [{: guard (Bits.width bits > 5 &&
                Bits.eq bits (Bits.U.of_int 15 (Bits.width bits))) :}]
      {: () :}

k16 : Bits(bits)
      [{: guard (Bits.width bits > 5 &&
                Bits.eq bits (Bits.U.of_int 16 (Bits.width bits))) :}]
      {: () :}

k4 : Bits(bits)
      [{: guard (Bits.width bits > 5 &&
                Bits.eq bits (Bits.U.of_int 4 (Bits.width bits))) :}]
      {: () :}

lconst: Link(symbol, w:int)      {: symbol#mangled_text :}
pic:    Diff(c1:lconst, c2:lconst) {: s "%s-%s" c1 c2 :}
pic:    Fetch(Mem(Diff(c1:lconst, c2:lconst)), w2:int) {: s "%s-%s" c1 c2 :}

```

$\langle \text{rules}(\text{ppcrec.nw}) \text{ 670b} \rangle + \equiv$ (668d) $\triangleleft$ 670a 670c $\triangleright$

```

pcl: Reg('c', 0) {: () :}
cial: Reg('c', 1) {: () :}
crl: Reg('c', 2) {: () :}
xerl: Reg('c', 4) {: () :}
lrl: Reg('c', 5) {: () :}
spl: Reg('r', 14) {: () :}

pc: Fetch(pcl, 32) {: () :}
cia: Fetch(cial, 32) {: () :}
cr: Fetch(crl, 32) {: () :}
lr: Fetch(lrl, 32) {: () :}
sp: Fetch(spl, 32) {: () :}

regl: Reg('r', n:int) [{: guard (n<>0) :}] {: reg_str n :}
reg: Fetch(regl, 32) {: regl :}

```

$\langle \text{rules}(\text{ppcrec.nw}) \text{ 670c} \rangle + \equiv$ (668d) $\triangleleft$ 670b 670d $\triangleright$

```

addr: const16      {: s "%s(%s)" const16 (r0_str ()) :}
addr: reg          {: s "0(%s)" reg :}
addr: Add(reg, const16) {: s "%s(%s)" const16 reg :}

ndx_addr: Add(reg1:reg, reg2:reg) {: s "%s,%s" reg1 reg2 :}

```

$\langle \text{rules}(\text{ppcrec.nw}) \text{ 670d} \rangle + \equiv$ (668d) $\triangleleft$ 670c 671a $\triangleright$

```

inst: Store(regl, reg, w:int) {: s "mr %s,%s" regl reg :}
inst: Store(regl, lr, w:int) {: s "mflr %s" regl :}
inst: Store(lrl, reg, w:int) {: s "mtlr %s" reg :}
(* claude: PPC has no direct memory<->lr transfer; mflr/mtlr need a GPR, so saving/restoring lr (Stack.freeze's
inst: Store(Mem(addr), lr, 32)      {: s "mflr %s\n\tstw %s,%s" (r0_str ()) (r0_str ()) addr :}
inst: Store(lrl, Fetch(Mem(addr),32), 32) {: s "lwz %s,%s\n\tmtlrl %s" (r0_str ()) addr (r0_str ()) :}
(* claude: cut/alternate-return continuation adjustment adds an offset to lr directly; same r0-mediated two-ins
inst: Store(lrl, Add(lr, x:reg), 32) {: s "mflr %s\n\tadd %s,%s,%s\n\tmtlrl %s" (r0_str ()) (r0_str ()) (r0_str
inst: Store(regl, cr, w:int) {: s "mfcrl %s" regl :}

inst: Store(regl, const16, 32) {: s "addi %s,0,%s" regl const16 :}

```

```

(* claude: a 32-bit constant too wide for const16, reaching the recognizer unsplit (e.g. via the generic move p
const32: Bits(bits)
  [{: guard (not (Bits.S.fits 16 bits)) :}]
  {: let i64 = Bits.U.to_int64 bits in
    ( Int64.to_int (Int64.logand (Int64.shift_right_logical i64 16) 0xffffL)
      , Int64.to_int (Int64.logand i64 0xffffL) )
  :}

inst : Store(regl, const32, 32)
  {: let (hi, lo) = const32 in
    s "addis %s,0,0x%x\n\tori %s,%s,0x%x" regl hi regl regl lo
  :}

inst: Store (regl, Fetch(Mem(addr      ),32), 32)      {: s "lwz %s,%s" regl addr :}
inst: Store (regl, Fetch(Mem(ndx_addr),32), 32)      {: s "lwzx %s,%s" regl ndx_addr :}
inst: Store (regl, Zx(Fetch(Mem(addr),8)), 32)       {: s "lbz %s,%s" regl addr :}
inst: Store (regl, Zx(Fetch(Mem(ndx_addr),8)), 32)   {: s "lbzx %s,%s" regl ndx_addr :}
inst: Store (regl, Zx(Fetch(Mem(addr),16)), 32)      {: s "lhz %s,%s" regl addr :}
inst: Store (regl, Zx(Fetch(Mem(ndx_addr),16)), 32)  {: s "lhzx %s,%s" regl ndx_addr :}
inst: Store (regl, Sxlo(reg, 8), 32)                 {: s "extsb %s,%s" regl reg :}
inst: Store (regl, Sxlo(reg, 16), 32)                {: s "extsh %s,%s" regl reg :}
inst: Store (regl, Sx(Fetch(Mem(addr),16)), 32)      {: s "lha %s,%s" regl addr :}
inst: Store (regl, Sx(Fetch(Mem(ndx_addr),16)), 32)  {: s "lhax %s,%s" regl ndx_addr :}

inst: Store (Mem(addr      ), reg, 32)              {: s "stw %s,%s" reg addr :}
inst: Store (Mem(ndx_addr), reg, 32)                {: s "stwx %s,%s" reg ndx_addr :}
inst: Store (Mem(addr      ), Lobits(reg, 8), 8)     {: s "stb %s,%s" reg addr :}
inst: Store (Mem(ndx_addr), Lobits(reg, 8), 8)       {: s "stbx %s,%s" reg ndx_addr :}
inst: Store (Mem(addr      ), Lobits(reg,16),16)     {: s "sth %s,%s" reg addr :}
inst: Store (Mem(ndx_addr), Lobits(reg,16),16)       {: s "sthx %s,%s" reg ndx_addr :}

⟨rules(ppcrec.nw) 671a⟩+≡ (668d) ◁670d 671b▷
inst : Store(regl, Add(reg, ha16), 32) {: s "addis %s,%s,%s" regl reg ha16 :}
inst : Store(regl, ha16, 32)          {: s "addis %s,0,%s" regl ha16 :}

inst : Store(regl, Add(reg, Sxlo(pic,16)),32) {: s "addi %s,%s,lo16(%s)" regl reg pic :}
inst : Store(regl, Sxlo(pic, 16), 32)         {: s "addi %s,0,lo16(%s)" regl pic :}

ha16: Ha16(pic) {: s "ha16(%s)" pic :}

(* claude: same shape as pic/ha16 above, but for a bare symbol address instead of a PIC-relative symbol difference
inst : Store(regl, Add(reg, absha16), 32) {: s "addis %s,%s,%s" regl reg absha16 :}
inst : Store(regl, absha16, 32)          {: s "addis %s,0,%s" regl absha16 :}

inst : Store(regl, Add(reg, Sxlo(lconst,16)),32) {: s "addi %s,%s,%s" regl reg (abs_lo lconst) :}
inst : Store(regl, Sxlo(lconst, 16), 32)         {: s "addi %s,0,%s" regl (abs_lo lconst) :}

absha16: Ha16(lconst) {: abs_ha lconst :}

⟨rules(ppcrec.nw) 671b⟩+≡ (668d) ◁671a 671c▷
inst: Goto(lconst) {: s "b %s" (ind_addr lconst) :}
inst: Goto(lr)      {: "blr" :}

⟨rules(ppcrec.nw) 671c⟩+≡ (668d) ◁671b 671d▷
inst : Par(Goto(lr),Store(regl,reg,w:int)) {: sprintf "mr %s, %s; blr" regl reg :}

⟨rules(ppcrec.nw) 671d⟩+≡ (668d) ◁671c 672a▷
next: Add(cia,k4) {: ( ) :}
inst: Par(Goto(lr      ), Store(lrl,next,32)) {: "blr1" :}
inst: Par(Goto(lconst), Store(lrl,next,32)) {: s "bl %s" (ind_addr lconst) :}

```



```

⟨rules(ppcrec.nw) 672a⟩+≡ (668d) <671d 672b>
cmp: Cmp(op:string, x:reg, y:reg)      {: ("", ppc_op op,x,y) :}
cmp: Cmp(op:string, x:reg, y:const16) {: ("i", ppc_op op,x,y) :}
inst: Guarded(cmp,Goto(lconst))
      {: let (i_, (l_, op), x, y) = cmp in
         s "cmp%sw%s %s,%s,%s\n\tb%s %s" l_ i_ (cr0_str ()) x y op lconst
      :}
inst : Guarded(0vSet(Fetch(xerl,32)), Goto(lconst))
      {: s "bo %s" lconst :}

⟨rules(ppcrec.nw) 672b⟩+≡ (668d) <672a 672c>
inst: Store(regl, Add(x:reg, y:reg), 32)      {: s "add %s,%s,%s" regl x y :}
inst: Store(regl, Add(x:reg, y:const16), 32)   {: s "addi %s,%s,%s" regl x y :}

inst: Store(regl, Unop (opr:string,x:reg),      32) {: s "%s %s,%s" opr regl x :}
(* claude: one's-complement has no single-opcode form on PPC, unlike Unop above *)
inst: Store(regl, Com(x:reg),                    32) {: s "nor %s,%s,%s" regl x x :}
inst: Store(regl, Binop(opr:string,x:reg,y:reg),32) {: s "%s %s,%s,%s" opr regl x y :}
inst: Store(regl, Binop(opr:string,x:reg,y:const16), 32)
      {: s "%si %s,%s,%s" opr regl x y :}

⟨rules(ppcrec.nw) 672c⟩+≡ (668d) <672b>
(* claude: without this, a compile-time-false guard's conNop() fell through to the any[100] catch-all as literal *)
inst : Nop() {: "nop" :}
inst : any [100] {: s "<%s>" any :}

any : True () {: "True" :}
any : False () {: "False" :}
any : Link(symbol, w:int) {: s "Link(%s,%d)" (symbol#mangled_text) w :}
any : Diff(c1:any, c2:any) {: s "Diff(%s, %s)" c1 c2 :}
any : Bits(bits)      {: sprintf "Bits(%s)" (Bits.to_string bits) :}

any : Fetch (any, w:int) {: s "Fetch(%s,%d)" any w :}

any : Sx(any)      {: s "Sx(%s)" any :}
any : Zx(any)      {: s "Zx(%s)" any :}
any : Sxlo(any,w:int)  {: s "Sxlo(%s,%d)" any w :}
any : Zxlo(any,w:int)  {: s "Zxlo(%s,%d)" any w :}
any : Add(x:any, y:any) {: s "Add(%s, %s)" x y :}

any: Ha16(any) {: s "Ha16(%s)" any :}

any : Unop (op:string, x:any)      {: s "Unop(%s,%s)" op x :}
(* claude: debug fallback for Com, mirrors the Unop/Binop ones above it *)
any : Com(x:any)      {: s "Com(%s)" x :}
any : Binop(op:string, x:any, y:any) {: s "Binop(%s,%s,%s)" op x y :}

any : Nop () {: "nop" :}

any : Lobits(any, w:int) {: s "Lobits(%s, %d)" any w :}
any : BitExtract(lsb:any, y:any, n:int) {: sprintf "BitExtract(%s, %s, %d)" lsb y n :}

any : Slice(w:int, n:int, y:any) {: sprintf "Slice(%d, %d, %s)" w n y :}

any : Mem(any) {: s "Mem(%s)" any :}
any : Reg(char, n:int) {: sprintf "Reg(%s, %d)" (Char.escaped char) n :}

any : Store (dst:any, src:any, w:int) {: s "Store(%s,%s,%d)" dst src w :}
any : Kill(any) {: s "Kill(%s)" any :}

```

```

any : Guarded(guard:any, any) {: s "Guarded(%s,%s)" guard any :}
any : Cmp(op:string, x:any, y:any) {: s "Cmp(%s,%s,%s)" op x y :}
any : Par(l:any, r:any) {: s "Par(%s,%s)" l r :}
any : Goto(any) {: s "Goto(%s)" any :}

```

W.11.3 Interfacing RTLs with the Expander

```

<code to follow the labeler(ppcrec.nw) 673a>≡ (668d) 673b▷
let rec const = function
| RP.Bool(true)      -> conTrue  ()
| RP.Bool(false)     -> conFalse ()
| RP.Link(s,_,w)      -> conLink s w
| RP.Diff(c1,c2)      -> conDiff (const c1) (const c2)
| RP.Late(s,w)        -> Impossible.impossible "Late constant in recognizer"
| RP.Bits(b)          -> conBits(b)

```

```

<code to follow the labeler(ppcrec.nw) 673b>+≡ (668d) <673a 674e>
let is_cmp (opr,ws) =
let cmp = ["eq";"ge";"geu";"gt";"gtu";"le";"leu";"lt";"ltu";"ne"] in
if not (List.mem opr cmp) then false
else match ws with
[32] -> true
| _ -> error "comparison not at 32 bits in PPC recognizer"

```

```

let rtl2ppc = function
| "and" -> "and"
| "divu" -> "divwu"
| "quot" -> "divw"
| "mul" -> "mullw"
| "neg" -> "neg"
| "or" -> "or"
| "shl" -> "slw"
| "shra" -> "sraw"
| "shrl" -> "srw"
| "sub" -> "sub"
| "xor" -> "xor"
| opr -> error (sprintf "Unsupported RTL operator \"%s\" " opr)

```

```

let rec exp = function
| RP.Const(k)          -> const (k)
| RP.Fetch(l,w)        -> conFetch (loc l) w
<case for ha16(e) 674a>
| RP.App(("sx", [n; _]), [RP.App ("lobits", [_;_]), [x]]) -> conSxlo (exp x) n
| RP.App(("zx", [n; _]), [RP.App ("lobits", [_;_]), [x]]) -> conZxlo (exp x) n
| RP.App(("sx", [8 ;32]), [x]) -> conSx (exp x)
| RP.App(("sx", [16;32]), [x]) -> conSx (exp x)
| RP.App(("zx", [8 ;32]), [x]) -> conZx (exp x)
| RP.App(("zx", [16;32]), [x]) -> conZx (exp x)
| RP.App(("add", [16]), [x; y]) -> conAdd (exp x) (exp y)
| RP.App(("add", [32]), [x; y]) -> conAdd (exp x) (exp y)

| RP.App(("ppc_xer_ov_set", []), [x]) -> conOvSet (exp x)
| RP.App(("bitExtract", [_; n]), [lsb; src]) -> conBitExtract (exp lsb) (exp src) n

| RP.App(("lobits", [32;w]), [x]) -> conLobits (exp x) w

```

(* claude: PPC has no single-opcode one's-complement; it is synthesized

```

* as nor rD,rA,rA (see the Com grammar rule below), unlike the generic
* Unop case which assumes a real one-operand-source instruction. *)
| RP.App(("com", [32]), [x])      -> conCom (exp x)

| RP.App(("opr", ws), [x])        -> conUnop (rtl2ppc opr) (exp x)
| RP.App(("opr", ws), [x;y])      -> if is_cmp(opr,ws) then conCmp opr (exp x) (exp y)
                                   else conBinop (rtl2ppc opr) (exp x) (exp y)

| RP.App(("o,_"),_) -> error (sprintf "unknown operator %s" o)

⟨case for ha16(e) 674a⟩≡ (673b)
| RP.App(("shl", [32]), [
  ⟨pic_hi16 + pic_15 674b⟩
; RP.Const (RP.Bits k16)])
when Bits.eq k16 (Bits.U.of_int 16 32) && Bits.eq k16' (Bits.U.of_int 16 32)
  && Bits.eq k15 (Bits.U.of_int 15 32) && RU.Eq.exp e e' ->
  conHa16 (exp e)

⟨pic_hi16 + pic_15 674b⟩≡ (674a)
RP.App(("add", [32]), [
  ⟨pic_hi16 674c⟩
;
  ⟨pic_15 674d⟩
])

⟨pic_hi16 674c⟩≡ (674b)
RP.App(("shrl", [32]), [e; RP.Const (RP.Bits k16')])

⟨pic_15 674d⟩≡ (674b)
RP.App(("zx", [1;32]),
  [RP.App(("lobits", [32;1]),
    [RP.App(("shrl", [32]), [e'; RP.Const (RP.Bits k15)])])])])

⟨code to follow the labeler(ppcrec.nw) 674e⟩+≡ (668d) <673b 675>▷
and loc l = match l with
| RP.Mem(('m',_,_), Rtl.C c, e, ass) -> conMem (exp e)
| RP.Reg((sp,_,_), i, w)             -> conReg sp i
| RP.Mem(.,_,_,_)                    -> error "non-mem, non-reg cell"
| RP.Var _ | RP.Global _             -> error "var found"
| RP.Slice(w,i,l)                   -> conSlice w i (loc l)

and effect = function
| RP.Store(RP.Reg(('c',_,_),i,_),r,_)
  when i = SS.indices.SS.pc          -> conGoto (exp r)
(* claude: Stack.freeze (shared, non-target-specific) emits raw
* "store a location's own fetched value back into itself" RTLs -
* seemingly a liveness/"protect across this point" marker rather
* than a real computation, since it never goes through
* Postexpander.move. ppcrec has no instruction for that shape, so
* it fell through to the any[100] catch-all as literal garbage
* text; eliding it here is what an assembler-level "mr rN,rN"
* would have done anyway, just without wasting an instruction. *)
| RP.Store(l, RP.Fetch(l',w'), w)
  when w =| w' && RU.Eq.loc l l'      -> conNop ()
| RP.Store(l,e,w)                    -> conStore (loc l) (exp e) w
| RP.Kill(l)                         -> conKill (loc l)

and guarded g eff =
  match g with
  | RP.Const(RP.Bool b) -> if b then effect eff else conNop()

```

```

| _                -> conGuarded (exp g) (effect eff)

and geffects = function
  | []              -> conNop()
  | [g, s]          -> guarded g s
  | (g, s) :: t     -> conPar (guarded g s) (geffects t)
and rtl (RP.Rtl es) = geffects es

```

W.11.4 The exported recognizers

```

<code to follow the labeler(ppcrec.nw) 675>+≡ (668d) <674e

let errmsg r msg =
  List.iter prerr_string
    [ "recognizer error: "; msg; " on "; RU.ToString.rtl r; "\n" ]

let to_asm ?(elf=false) r i =
  try
    let _ = imports := i in
    let _ = elf_syntax := elf in
    let plan = rtl (Down.rtl r) in
    plan.inst.Camlburg.action ()
  with
  | Camlborg.Uncovered -> " not an instruction: " ^ RU.ToString.rtl r
  | Error msg -> (errmsg r msg; " error in recognizer: " ^ msg)

let is_instruction r =
  try
    let plan = rtl (Down.rtl r) in
    plan.inst.Camlburg.cost < 100
  with
  | Camlborg.Uncovered -> false
  | Error msg -> (errmsg r msg; false)

```

Appendix X

arch/sparc

X.1 Back end for the SPARC

Required as part of the interface: postexpander, target, and variable placer. May add other pieces as required by the recognizer (the main client of this module).

```
<sparc.mli 676a>≡
  module X : Expander.S
  val target: Ast2ir.tgt
  val placevars : Ast2ir.proc -> Automaton.t

<sparc.ml 676b>≡
  <sparc.ml 676c>

<sparc.ml 676c>≡                                     (676b) 676d▷
  open Nopoly
  module PX = Postexpander
  module DG = Dag
  module S = Space
  module SS = S.Standard32
  module R = Rtl
  module RO = Rewrite.Ops
  module RU = Rtlutil
  module Up = R.Up
  module Dn = R.Dn
  module RP = R.Private
  module SM = Strutil.Map
  module A = Automaton
  module T = Target

  let rtl r = DG.Rtl r
  let (<:>) = DG.<:>
  let rstore l r w = rtl (R.store l r w)
  let astore l r w = rtl (A.store l r w)

<sparc.ml 676d>+≡                                     (676b) <676c 676e▷
  let byteorder = R.BigEndian
  let wordsize = 32
```

X.1.1 Storage spaces

```
<sparc.ml 676e>+≡                                     (676b) <676d 677a▷
  module Spaces = Sparcregs.Spaces
```

X.1.2 Registers

```
<sparc.ml 677a>+≡ (676b) <676e 677b>
let pc = Sparcregs.pc
let cc = Sparcregs.cc
let npc = Sparcregs.npc
(* Do we use this?
   let fp_mode = locations.SS.fp_mode
   let fp_fcmp = locations.SS.fcmp
*)
let vfp = Vfp.mk wordsize

let (_, _, mcell) as mspace = Spaces.m.S.space
let rspace = Spaces.r.S.space
let fspace = Spaces.f.S.space
let dspace = Spaces.d.S.space

let r n = (rspace, n, R.C 1)

let zero = r 0 (* always zero, r[0] is same as global[0] *)
let ra = r 31 (* return address set by CALL instruction, same as %o7 *)

let fp = r 30 (* conventionally the same as %fp or %i6 *)
let sp = r 14 (* conventionally the same as %sp or %o6 *)

let cwp = Sparcregs.cwp
let yreg = Sparcregs.y

let rounding_model = R.regx Sparcregs.fpround
let rounding_mode_reg = R.fetch rounding_model
(*
let rounding_mode_reg = (dspace, 0, R.C 1)
let rounding_model = Rtl.reg rounding_mode_reg
*)
let rounding_mode = R.fetch rounding_model 2
let rounding_results1 = Rtl.reg (dspace, 1, R.C 1)
```

X.1.3 Utilities

```
<sparc.ml 677b>+≡ (676b) <677a 677c>
let imposs = Impossible.impossible
let unimpf fmt = Printf.kprintf Impossible.unimp fmt
let impossf fmt = Printf.kprintf Impossible.impossible fmt
let mem w assn addr = R.mem assn mspace (Cell.to_count mcell w) addr
let rwidth = Register.width

let rfetch t = R.fetch (R.reg t) (rwidth t)
let tstore t e = R.store (R.reg t) e (rwidth t)

let shift dst op n = rtl (tstore dst (op 32 (rfetch dst) (R0.unsigned 32 n)))

let at32 = function
| opr, [32] -> opr
| opr, [n] -> unimpf "operator %%%s%d(...)" opr n
| opr, ws -> impossf "operator %%%s specialized to %d widths" opr (List.length ws)
```

X.1.4 Control flow

```
<sparc.ml 677c>+≡ (676b) <677b 678a>
```

```

module F = Mflow.MakeStandard
  (struct
    let pc_lhs    = npc
    let pc_rhs    = pc
    let ra_reg    = R.reg ra
    let ra_offset = 4
  end)
(* claude: needed both for Post.return (the generic Mflow-provided
 * default, same as ppc.ml's 'fmach.Mflow.return') and for the cutto
 * embed/project pair threaded into Sparccall.cconv below. *)
let fmach = F.machine (R.reg sp)

⟨sparc.ml 678a⟩+≡ (676b) <677c 678b>
  let return e =
    let one = R0.signed 32 1 in
    R.par [R.store npc e wordsize;
          R.store (R.reg cwp) (R.app (R.opr "add" [32]) [rfetch cwp; one]) wordsize]

```

X.1.5 Postexpander

```

⟨sparc.ml 678b⟩+≡ (676b) <678a 686b>
  module Post = struct
    let byte_order = byteorder
    let wordsize   = wordsize
    let exchange_alignment = 4
    module Address = struct
      type t = R.exp
      let reg r = R.fetch (R.reg r) (Register.width r)
    end
    type temp      = Register.t
    type rtl       = R.rtl
    type width     = R.width
    type assertion = R.assertion
    type operator  = RP.opr
    ⟨SPARC postexpander 678c⟩
    include Postexpander.Nostack(Address)
  end
end

```

Our allocators.

```

⟨SPARC postexpander 678c⟩≡ (678b) 678d>
  let talloc = Postexpander.Alloc.temp
  let salloc = Postexpander.Alloc.slot

```

There is some nasty cheating going on regarding the 'o' registers. For this reason, it is not useful to use `Context.of_space` and friends. (And what about 'l', 'i', ...?)

```

⟨SPARC postexpander 678d⟩+≡ (678b) <678c 679a>
  let icontext =
    Talloc.Multiple.reg 't',
    fun ((c,_,_), _, _) -> c <= 'r' || c <= 't' || c <= 'o'

  let fcontext =
    (fun t w ->
      if w = 32 then Talloc.Multiple.reg 'u' t 32
      else Talloc.Multiple.reg 'q' t 64),
    fun ((c,_,_), _, _) -> c <= 'f' || c <= 'u' || c <= 'q'
  let acontext = icontext
  let rcontext = (fun x y -> unimpf "Unsupported soft rounding mode")
    ,fun ((sp,_,_), i, _) -> sp <= 'd' && i = 0
  let itempwidth = 32

```

```

let operators = Context.nonbool icontext fcontext rcontext []
let arg_contexts, result_context = Context.functions operators
let constant_context w = icontext

⟨SPARC postexpander 679a⟩+≡ (678b) ◁678d 679b▷
let extend op n w e = R.app (R.opr op [n; w]) [e]
let lobits w n e = R.app (R.opr "lobits" [w; n]) [e]
let xload op ~dst ~addr n assn =
  let w = rwidth dst in
  assert (w = wordsize || w = wordsize * 2);
  rtl (R.store (R.reg dst)
    (extend op n w (R.fetch (R.mem assn mspace (Cell.to_count mcell n) addr) n)) w)

let sxload = xload "sx"
let zxload = xload "zx"

let lostore ~addr ~src n assn =
  let w = rwidth src in
  assert (w = wordsize || w = 2 * wordsize);
  rtl (R.store (R.mem assn mspace (Cell.to_count mcell n) addr)
    (lobits w n (rfetch src)) n)

⟨SPARC postexpander 679b⟩+≡ (678b) ◁679a 680a▷
let rec load ~dst ~addr assn =
  let w = rwidth dst in
  assert (w = wordsize || w = wordsize * 2);
  let a = Dn.assertion assn in
  let align_a_plus n = Alignment.alignment (Alignment.add n (Alignment.init a)) in
  let addr_plus n = R0.add 32 addr (R0.signed 32 n) in
  if a >= w / 8 then rtl (R.store (R.reg dst) (R.fetch (mem w assn addr) w) w)
  else
    (* warn about expensive unaligned loads? *)
    match dst with
    | (('t', _, cell), _, R.C 1) when Cell.to_width cell (R.C 1) = 32 ->
      let fit = align_a_plus (w / 8) in
      let w' = fit * 8 in
      let nloads = w / w' in
      let t, d = talloc 't' 32, talloc 't' 32 in
      let ld1 offset =
        zxload ~dst:t ~addr:(addr_plus offset) w' assn <:>
        rtl (tstore d (R0.shl 32 (rfetch d) (R0.unsigned 32 w')) <:>
        rtl (R.store (R.reg d) (R0.(or) 32 (rfetch d) (rfetch t)) 32)
      in
      let rec f acc n = if n = -1 then acc else f (ld1 (fit * n) <:> acc) (n - 1) in
      PX.Expand.block (f (rtl (R.store (R.reg dst) (rfetch d) 32)) (nloads - 1))
    | (('u', _, cell), _, R.C 1) when Cell.to_width cell (R.C 1) = 32 ->
      (* MUST go through memory -- stack slot is word aligned *)
      let slot = salloc w 4 in
      let d = talloc 't' 32 in
      load ~dst:d ~addr assn <:>
      rtl (A.store slot (rfetch d) w) <:>
      rtl (R.store (R.reg dst) (A.fetch slot w) w)
    | (('q', _, cell), i, R.C 1) ->
      assert (Cell.to_width cell (R.C 1) = 64);
      let u1, u2 = talloc 'u' 32, talloc 'u' 32 in
      let q1, q2 = R.slice 32 32 (R.reg dst), R.slice 32 0 (R.reg dst) in
      (* THIS CAUSES A HW REG TO BE SET ?!?! PX.Expand.block *)
      (load ~dst:u1 ~addr assn <:>
      load ~dst:u2 ~addr:(addr_plus 4) (Up.assertion (align_a_plus 4)) <:>

```



```

        rtl (R.store q1 (rfetch u1) 32) <:>
        rtl (R.store q2 (rfetch u2) 32))
| _ -> unimpf "SPARC unaligned load: %s := bits%i[%s]"
        (RU.ToString.reg dst) w (RU.ToString.exp addr)

⟨SPARC postexpander 680a⟩+≡ (678b) <679b 680b>
let store ~addr ~src assn =
  let w = rwidth src in
  assert (w = wordsize || w = wordsize * 2);
  rtl (R.store (mem w assn addr) (rfetch src) w)

⟨SPARC postexpander 680b⟩+≡ (678b) <680a 681a>
let rec block_copy ~dst dassn ~src sassn w =
  let decompose_f dst dassn src sassn x w f =
    if w - x <= 0 then impossf "block copy attempted to decompose move into %i bits" x
    else
      let d' = talloc 't' 32 in
      let s' = talloc 't' 32 in
      let i = R0.unsigned 32 (x / 8) in
      block_copy ~dst dassn ~src sassn x <:>
      rtl (R.store (R.reg d') (R0.add 32 dst i) 32) <:>
      rtl (R.store (R.reg s') (R0.add 32 src i) 32) <:>
      f d' s'
  in
  let decompose dst dassn src sassn x w =
    decompose_f dst dassn src sassn x w
    (fun d' s' -> block_copy ~dst:(rfetch d') dassn ~src:(rfetch s') sassn (w - x))
  in
  match w with
  | 0 -> DG.Nop
  | 8 | 16 ->
    let t = talloc 't' 32 in
    zxload t src w sassn <:> lstore dst t w dassn
  | 24 -> decompose dst dassn src sassn 8 24
  | 32 ->
    let t = talloc 't' 32 in
    load t src sassn <:> store dst t dassn
  | w when w > 32 && w < 64 && w mod 8 = 0 -> decompose dst dassn src sassn 32 w
  | 64 ->
    let q = talloc 'q' 64 in
    load ~dst:q ~addr:src sassn <:> store ~addr:dst ~src:q dassn
(* claude: was a runtime While-loop copying 64 bits at a time via
 * PX.cond, but Postexpander.cond isn't part of the current interface
 * (commented out in postexpander.mli - Dag.cond exists but at the
 * wrong representation level for a brtl block here). Left as a stub
 * like the catchall below rather than guessing at a replacement for
 * an already-obscure, seemingly untested path (block copies over 64
 * bits whose width isn't itself a multiple of 64). Original code,
 * kept in case it's worth reviving once a real cond/While path
 * exists again:
 *
 * | w when w > 64 && (w mod 64) mod 8 = 0 ->
 *   let wmod64 = w mod 64 in
 *   let f d' s' =
 *     let t = talloc 't' 32 in
 *     let i = R0.unsigned 32 (64 / 8) in
 *     let n64cpys = R0.unsigned 32 ((w - wmod64) / 8) in
 *     rtl (R.store (R.reg t) (R0.add 32 (rfetch d') n64cpys) 32) <:>
 *     DG.While(PX.cond (R0.eq 32 (rfetch d') (rfetch t)),
 *       block_copy ~dst:(rfetch d') dassn ~src:(rfetch s') sassn 64 <:>

```

```

*           rtl (R.store (R.reg d') (R0.add 32 (rfetch d') i) 32) <:>
*           rtl (R.store (R.reg s') (R0.add 32 (rfetch s') i) 32))
*   in
*   decompose_f dst dassn src sassn wmod64 w f
*)
| w when w > 64 && (w mod 64) mod 8 = 0 ->
    unimpf "Block copy of %i bits on SPARC (>64, not a multiple of 64)" w
| _ -> unimpf "Block copy of %i bits on SPARC" w

```

We write very strange code in `if` in case of the possibility that a temporary is *neither* a float nor an integer.

```

⟨SPARC postexpander 681a⟩+≡ (678b) <680b 681b>
let is_int = snd icontext
let is_float = snd fcontext
let move ~dst ~src =
  let w = rwidth src in
  assert (w = rwidth dst);
  if (if is_int src then is_float dst else is_float src && is_int dst) then
    (* move through stack -- doubleword aligned *)
    let slot = salloc w 8 in
    rtl (A.store slot (rfetch src) w) <:> rtl (R.store (R.reg dst) (A.fetch slot w) w)
  else if Register.eq src dst then
    DG.Nop
  else
    rtl (R.store (R.reg dst) (rfetch src) w)

```

```

⟨SPARC postexpander 681b⟩+≡ (678b) <681a 681c>
let extract ~dst ~lsb ~src =
  let w = rwidth src in
  let n = rwidth dst in
  let srcval =
    if lsb = 0 then
      R0.lobits w n (rfetch src)
    else if lsb = 32 then
      R0.lobits w n (R0.shrl w (rfetch src) (R0.unsigned w lsb))
    else
      impossf "bad lsb in sparc extract" in
  if is_int dst && is_float src then
    (* move through stack -- doubleword aligned *)
    let slot = salloc n 8 in
    rtl (A.store slot srcval n) <:> rtl (R.store (R.reg dst) (A.fetch slot n) n)
  else if is_float dst && is_float src then
    rtl (R.store (R.reg dst) srcval n)
  else
    impossf "unexpected extract in sparc postexpander"

```

```

let aggregate ~dst ~src = Impossible.unimp "aggregate"

```

```

⟨SPARC postexpander 681c⟩+≡ (678b) <681b 681d>
let weird_tmp tmp z = match z with
| PX.WTemp (Register.Reg t)          -> t, DG.Nop
| PX.WTemp (Register.Slice (w, 0, t)) -> t, DG.Nop
| PX.WTemp (Register.Slice (w, lsb, t)) ->
    tmp, move tmp t <:> rtl (tstore tmp (R0.shrl 32 (rfetch tmp) (R0.unsigned 32 lsb)))
| PX.WBits b -> tmp, rtl (tstore tmp (R.bits (Bits.Ops.zx 32 b) 32))

```

```

⟨SPARC postexpander 681d⟩+≡ (678b) <681c 682a>
let hwset ~dst ~src =
  let rmask_clear = R.bits (Bits.U.of_int64 (Int64.of_string "0x3fffffff") 32) 32 in
  if Register.eqx dst Sparcregs.fpround then
    let slot = salloc 32 4 in

```

```

let t          = talloc 't' 32 in
let t2, ldsrc = weird_tmp (talloc 't' 32) src in
rtl (slot.A.store (rfetch Sparcregs.fpctl) 32) <:>
rtl (tstore t (slot.A.fetch 32)) <:>
PX.Expand.block (rtl (tstore t (R0._and 32 (rfetch t) rmask_clear))) <:>
ldsrc <:>
rtl (tstore t2 (R0.shl 32 (rfetch t2) (R0.unsigned 32 30))) <:>
rtl (tstore t (R0.(or) 32 (rfetch t) (rfetch t2))) <:>
rtl (slot.A.store (rfetch t) 32) <:>
rtl (tstore Sparcregs.fpctl (slot.A.fetch 32))
else Impossible.unimp "setting hardware register"

```

```

let hwget ~dst:(fill, dt) ~src =
if Register.eqx src Sparcregs.fpround then
let slot = salloc 32 4 in
rtl (slot.A.store (rfetch Sparcregs.fpctl) 32) <:>
rtl (tstore dt (slot.A.fetch 32)) <:>
match fill with
| PX.HighAny | PX.HighZ -> shift dt R0.shrl 30
| PX.HighS              -> shift dt R0.shra 30
else Impossible.unimp "getting hardware register"

```

⟨SPARC postexpander 682a⟩+≡ (678b) ◁681d 682b▷

```

let li ~dst const =
match dst, const with
| (('u', _, _), _, R.C 1), _ -> (* we are forced to go through memory *)
let t = talloc 't' 32 in
let slot = salloc 32 4 in
rstore (R.reg t) (Up.const const) 32 <:>
astore slot (rfetch t) 32 <:>
rstore (R.reg dst) (A.fetch slot 32) 32
| (('t', _, _), _, R.C 1), _ ->
rstore (R.reg dst) (Up.const const) 32
| ((s, _, _), _, _) as r, RP.Bits b ->
unimpf "loading immediate %s to a %d-bit register in space %%%c"
(Bits.to_string b) (rwidth r) s
| ((s, _, _), _, _) as r, _ ->
unimpf "loading symbolic immediate to a %d-bit register in space %%%c"
(rwidth r) s
let lix ~dst e = rtl (R.store (R.reg dst) e (rwidth dst))
(* IS THIS REALLY RIGHT? *)

```

⟨SPARC postexpander 682b⟩+≡ (678b) ◁682a 682c▷

```

let unop ~dst op x =
match op with
| ("zx", _) | ("sx", _) | ("lobits", _) ->
impossf "%%%s%d reached sparc postexpander" (fst op) (List.nth (snd op) 1)
| _ -> rstore (R.reg dst) (R.app (Up.opr op) [rfetch x]) (rwidth dst)

```

sparc_mulx_hi is intended to do an extended multiply and return the high 32 bits.

⟨SPARC postexpander 682c⟩+≡ (678b) ◁682b 683a▷

```

let dblop ~dsthi ~dstlo (op, ws) x y =
let xv, yv = rfetch x, rfetch y in
rtl (R.par [R.store (R.reg dstlo) (R.app (R.opr "mul" [32]) [xv;yv]) 32;
R.store (R.reg yreg) (R.app (R.opr ("sparc_"^op^"_hi") [32])
[xv; yv]) 32])
<:> rstore (R.reg dsthi) (rfetch yreg) 32
(* Unsupported.mulx_and_mulux() *)

```

(SPARC postexpander 683a)+≡ (678b) <682c 683b>

```

let binop ~dst op x y =
  let dstw = rwidth dst in
  let xv, yv = rfetch x, rfetch y in
  match op with
  | "modu", [w] ->
    PX.Expand.block (rstore (R.reg dst) (Rewrite.modu w xv yv) dstw)
  | "rem", [w] ->
    PX.Expand.block (rstore (R.reg dst) (Rewrite.rem w xv yv) dstw)
  | "mul", [w] ->
    (* hoping that a later pass will eliminate the y to y move *)
    dblop ~dsthi:yreg ~dstlo:dst (Dn.opr (R.opr "mulx" [32])) x y
  | _ -> rstore (R.reg dst) (R.app (Up.opr op) [xv; yv]) dstw

```

Assuming 32-bit values.

(SPARC postexpander 683b)+≡ (678b) <683a 684a>

```

let carrybit = R.app (R.opr "sparc_carrybit" []) [RU.fetch cc]
let adcflags x y w =
  R.store cc (R.app (R.opr "sparc_adcflags" [w]) [x; y; carrybit]) 32
let sbbflags x y w =
  R.store cc (R.app (R.opr "sparc_sbbflags" [w]) [x; y; carrybit]) 32

```

```

let set_carry ~tmp z =
  let t, is = weird_tmp tmp z in
  let bit0 = R0.sx 1 32 (R0.lobits 32 1 (rfetch t)) in
  let d = talloc 't' 32 in
  is <:>
  PX.Expand.block (rtl (R.store (R.reg d) bit0 32)) <:>
  rtl (R.store cc (R.app (R.opr "sparc_addcc" [32])
    [rfetch d; R0.signed 32 1]) 32)

```

```

let wrdop ~dst op x y z =
  let is_add =
    match at32 op with
    | "addc" -> true
    | "subb" -> false
    | _ -> impossf "wrdop: %s" (fst op)
  in
  let dstv, xv, yv = rfetch dst, rfetch x, rfetch y in
  set_carry ~tmp:dst z <:>
  rtl (R.store (R.reg dst) (R.app (Up.opr op) [xv; yv; carrybit]) 32)

```

```

let wrdrop ~dst:(fill, dt) op x y z =
  let is_add =
    match at32 op with
    | "carry" -> true | "borrow" -> false | _ -> impossf "wrdrop: %s" (fst op) in
  let op = if is_add then "addc" else "subb" in
  let t = talloc 't' 32 in
  let xv, yv = rfetch x, rfetch y in
  let zero = R0.signed 32 0 in
  let flags = (if is_add then adcflags else sbbflags) xv yv 32 in
  set_carry ~tmp:dt z <:>
  rtl (R.par [R.store (R.reg dt) (R.app (R.opr op [32]) [xv; yv; carrybit]) 32;
    flags]) <:>
  rtl (R.store (R.reg t) zero 32) <:>
  rtl (R.store (R.reg dt) (R0.addc 32 (rfetch t) (rfetch t) carrybit) 32) <:>
  (match fill with
  | PX.HighAny -> DG.Nop
  | PX.HighZ -> shift dt R0.shl 31 <:> shift dt R0.shrl 31 (* zxlo *)
  | PX.HighS -> shift dt R0.shl 31 <:> shift dt R0.shra 31 (* sxlo *))

```

```

⟨SPARC postexpander 684a⟩+≡ (678b) <683b 684b>
let with_rm rm b =
  PX.with_hw ~hard:Sparcregs.fpround ~soft:rm ~temp:(talloc 't' 32) b

let f2i op dst x w = match dst, x with
| (('u', _, _), _, _), (('u', _, _), _, _) ->
  rstore (R.reg dst) (R.app op [rfetch x; rounding_mode]) w
| (('u', _, _), _, _), (('q', _, _), _, _) ->
  rstore (R.reg dst) (R.app op [rfetch x; rounding_mode]) w
| ((dstr, _, _), _, _), ((xr, _, _), _, _) ->
  impossf "'%c' := %f2i('%c'..): f2i only available single precision\
    or float-to-float space\n" dstr xr

let unrm ~dst op x rm =
  let w = rwidth dst in
  match op with
  | "f2i", _ -> with_rm rm (f2i (Up.opr op) dst x w)
  | _ ->
    with_rm rm (rstore (R.reg dst) (R.app (Up.opr op) [rfetch x; rounding_mode]) w)

let binrm ~dst op x y rm =
  let w = rwidth dst in
  with_rm rm
    (rstore (R.reg dst) (R.app (Up.opr op) [rfetch x; rfetch y; rounding_mode]) w)

⟨SPARC postexpander 684b⟩+≡ (678b) <684a 684c>
let pc_lhs = npc (* PC as assigned by branch *)
let pc_rhs = pc (* PC as captured by call *)

⟨SPARC postexpander 684c⟩+≡ (678b) <684b 684d>
let br ~tgt = DG.Nop, R.store pc_lhs (rfetch tgt) wordsize (* branch reg *)
let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) wordsize (* branch *)

Needs to be filled in

⟨SPARC postexpander 684d⟩+≡ (678b) <684c 684e>
(* claude: split from the old single 'bc' (kept in spirit, same RTLs)
 * into bc_guard/bc_of_guard, the shape Postexpander.S now requires -
 * see ppc.ml's bc_guard/bc_of_guard for the same split applied there. *)
let bc_guard x (opr, ws as op) y =
  (* Might be a 64-bit float *)
  assert (ws == [wordsize] || ws == [wordsize*2]);
  (rstore cc (R.app (R.opr "sparc_subcc" ws) [rfetch x; rfetch y]) 32,
   R.app (R.opr ("sparc_"~opr) ws) [R.fetch cc 32])
let bc_of_guard (setup, guard) ~ifso ~ifnot =
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt 32) in
  DG.Test (setup, (brtl guard, ifso, ifnot))

⟨SPARC postexpander 684e⟩+≡ (678b) <684d 685a>
let bnegate r =
  let zero = R0.signed 32 0 in
  let negate = function
    | "sparc_ne" -> "sparc_eq"
    | "sparc_eq" -> "sparc_ne"
    | "sparc_feq" -> "sparc_fne"
    | "sparc_fne" -> "sparc_feq"
    | _ -> imposs "ill-formed SPARC conditional branch" in
  match Dn.rtl r with
  | RP.Rtl [ RP.App( ( ("eq"|"ne" as op), [32] )
    , [RP.Fetch(RP.Reg(x),32); RP.Const(RP.Bits(b))] )
    )
    , RP.Store (pc, tgt, 32)

```

```

] when RU.Eq.loc pc (Dn.loc pc_lhs) && Bits.is_zero b ->
  R.guard (R.app (R.opr (negate op) [32]) [rfetch x; zero])
  (R.store pc_lhs (Up.exp tgt) wordsize)
| _ -> impose "ill-formed SPARC conditional branch"

```

```

⟨SPARC postexpander 685a⟩+≡ (678b) <684e 685b>
let effects = List.map Up.effect
(* claude: dropped ~others (extra live-in effects at the call site) -
 * Postexpander.S's call/callr no longer take it (see ppc.ml's call/
 * callr, also just ~tgt); nothing in the Expander machinery calling
 * these generically could have supplied it anyway. *)
(* claude: "call ~tgt = DG.Nop, R.store pc_lhs ... wordsize" (just a
 * Goto, no return-address save) is the exact same bug found and fixed
 * in ppc.ml's call: every call silently became a tail branch, so
 * nothing ever returned to its actual caller. sparcrec.mlb's real
 * "call" mnemonic is only matched by "Par(Goto(target), next)" where
 * "next: Store(regl, Add(pc, four), w)" (line ~487-489) - i.e. the
 * RTL must ALSO store pc+4 into some register (real hardware writes
 * %o7 as a side effect of the CALL instruction; ra=r31=%i7 is what
 * this register becomes once the callee's own 'save' rotates the
 * window). Mirrors ppc.ml's call/x86.ml's call (Goto first, ra-store
 * second - Par's constructor argument order matters for the grammar
 * match). callr uses the same fix via the "Par(Goto(reg), next)" rule
 * one line below the plain-symbol one. *)
let call ~tgt =
  DG.Nop, R.par [ R.store pc_lhs (Up.const tgt) wordsize
                  ; R.store (R.reg ra) (RU.addk wordsize (R.fetch pc wordsize) 4) wordsize ]
let callr ~tgt =
  DG.Nop, R.par [ R.store pc_lhs (rfetch tgt) wordsize
                  ; R.store (R.reg ra) (RU.addk wordsize (R.fetch pc wordsize) 4) wordsize ]

```

```

⟨SPARC postexpander 685b⟩+≡ (678b) <685a 686a>
(* claude: adapted from the old effect-list-based signature to the
 * current Mflow.cut_args record ({new_sp; new_pc}), same semantics:
 * reset the register-window pointer (cwp) then jump to the new pc with
 * sp set to the new sp. *)
let cut_to {Mflow.new_sp = sp'; Mflow.new_pc = pc'} =
  DG.Rtl (R.store (R.reg cwp) (R0.signed 32 0) 32),
  R.par [R.store pc_lhs pc' wordsize; R.store (R.reg sp) sp' wordsize]
(* claude: return/forbidden are required by Postexpander.S but had no
 * definition here. return was first set to ppc.ml's
 * 'fmach.Mflow.return' (the generic Mflow-provided default) by
 * analogy, but that is wrong for SPARC: codegen/expander.ml wires
 * this field directly into a generic "GR.Return" CFG node
 * (Mflow.return = Post.return), bypassing the calling-convention-
 * specific Call.t.return/Sparccall.rtn path entirely - and the
 * generic default is just "jump to whatever's in the ra register",
 * with no SPARC-specific "+8" return-address adjustment and no
 * register-window 'restore'. Empirically this is exactly what fired
 * for demos/hello_sparc.c--'s plain "return(0)": main's epilogue
 * landed back exactly on __libc_start_call_main's own "call %g1"
 * instruction (real hardware stores pc_of_call into the return
 * register, not pc_of_call+4) instead of past it, re-invoking main a
 * second time with %g1 no longer holding a valid function pointer
 * (clobbered as ordinary scratch by main's own body) - confirmed via
 * gdb-over-qemu-gdbserver, single-stepping with a $pc watchpoint from
 * inside printf out through main's return. Fixed by reusing this
 * file's own top-level 'return e' (also used by Sparccall.cconv's
 * ~return_to), which emits Store(pc_lhs, e, w) parred with the cwp
 * increment sparcrec.mlb's "restore" nonterminal matches - together

```

```

* exactly "Par(Goto(ra), restore)", recognized as "ret\n\trestore". *)
let return = return (R.fetch (R.reg ra) wordsize)
let forbidden = Rtl.par [] (* BOGUS: NEEDS TO BE A REAL FAULTING INSTRUCTION *)

```

If an instruction modifies the current window pointer, it's a save or restore, and the generic expander must not touch it.

```

<SPARC posterpander 686a>+≡ (678b) <685b
let don't_touch_me =
  let stores_to_cwp = function
    | RP.Store (RP.Reg maybe_cwp, _, _) when Register.eq maybe_cwp cwp -> true
    | _ -> false in
  List.exists stores_to_cwp

```

X.1.6 Expander

```

<sparc.ml 686b>+≡ (676b) <678b 686c>
module X = Expander.IntFloatAddr(Post)

```

X.1.7 Target

```

<sparc.ml 686c>+≡ (676b) <686b 686d>
let spill p t l = (* assert (rwidth t = Rtlutil.Width.loc l); *)
  [A.store l (rfetch t) (rwidth t)]
let reload p t l =
  let w = rwidth t in [R.store (R.reg t) (A.fetch l w) w]

```

```

<sparc.ml 686d>+≡ (676b) <686c 686e>
let ( *> ) = A.( *> )
let globals base =
  let width w = if w <= 8 then 8
    else if w <= 16 then 16
    else Auxfuns.round_up_to ~multiple_of:wordsize w in
  let align = function _ -> 8 in
  A.at mspace ~start:base (A.widen width *> A.align_to align *>)
  A.overflow ~growth:Memalloc.Up ~max_alignment:16)

```

```

<sparc.ml 686e>+≡ (676b) <686d 689a>
let target =
  let spaces = [ Spaces.m
    ; Spaces.r
    ; Spaces.f
    ; Spaces.t
    ; Spaces.u
    ; Spaces.q
    ; Spaces.c
    ; Spaces.k
  ] in
  (* claude: Ast2ir.tgt = Preast2ir.tgt = T of (...) Target.t - a
   * wrapped variant, not the bare record (see ppc.ml's 'PA.T {...}',
   * PA = Preast2ir). *)
  Preast2ir.T
  { T.name = "sparc"
  ; T.memspace = mspace
  ; T.max_unaligned_load = R.C 1
  ; T.byteorder = byteorder
  ; T.wordsize = wordsize
  ; T.pointersize = wordsize
  ; T.alignment = 4

```

```

; T.memsize           = 8
; T.spaces            = spaces
; T.reg_ix_map        = T.mk_reg_ix_map spaces
; T.distinct_addr_sp  = false
; T.float             = Float.ieee754

; T.vfp               = vfp
(* claude: T.spill/T.reload/T.bnegate/T.goto/T.jump/T.call/T.return/
 * T.branch aren't Target.t fields anymore (see target.mli) - they now
 * live inside the single T.machine record, built by the
 * Expander.IntFloatAddr functor from Post below, same as ppc.ml/
 * x86.ml. *)
; T.machine           = X.machine

; T.cc_specs          = Sparccc.cc_specs
; T.cc_spec_to_auto   = Sparccall.cconv ~return_to:return
                        { T.embed    = fmach.T.cutto.T.embed
                          ; T.project = fmach.T.cutto.T.project }

; T.is_instruction    = Sparcrec.is_instruction
; T.tx_ast = (fun secs -> secs)
; T.capabilities      = {T.memory = [32; 64]; T.block_copy = true;
                        T.itemps = [32]; T.ftemps = [32; 64];
                        T.iwiden = false; T.fwiden = false;
                        T.literals = [32; 64]; T.litops = [];
                        T.operators = List.map Up.opr [
                            "sx",      [ 1; 32]
                            ; "zx",      [ 1; 32]
                            ; "lobits", [32; 1]
                            ; "lobits", [32; 32]
                            ; "add",     [32]
                            ; "addc",    [32]
                            ; "and",     [32]
                            ; "borrow",  [32]
                            ; "carry",   [32]
                            ; "com",     [32]
                            ; "div",     [32]
                            ; "divu",    [32]
                            ; "mod",     [32]
                            ; "modu",    [32]
                            ; "mul",     [32]
                            ; "mulx",    [32]
                            ; "mulux",   [32]
                            ; "neg",     [32]
                            ; "or",      [32]
                            ; "quot",    [32]
                            ; "popcnt",  [32]
                            ; "rem",     [32]
                            ; "rotl",    [32]
                            ; "rotr",    [32]
                            ; "shl",     [32]
                            ; "shra",    [32]
                            ; "shrl",    [32]
                            ; "sub",     [32]
                            ; "subb",    [32]
                            ; "xor",     [32]
                            ; "eq",      [32]
                            ; "ge",      [32]
                            ; "geu",     [32]
                            ; "gt",      [32]

```



```

; "gtu",      [32]
; "le",       [32]
; "leu",      [32]
; "lt",       [32]
; "ltu",      [32]
; "ne",       [32]
; "fabs", [32]
; "fadd", [32]
; "fdiv", [32]
; "feq", [32]
; "fge", [32]
; "fgt", [32]
; "fle", [32]
; "flt", [32]
; "fne", [32]
; "fordered", [32]
; "fmul", [32]
; "fneg", [32]
; "funordered", [32]
; "fsqrt", [32]
; "fsub", [32]
; "fabs", [64]
; "fadd", [64]
; "fdiv", [64]
; "feq", [64]
; "fge", [64]
; "fgt", [64]
; "fle", [64]
; "flt", [64]
; "fne", [64]
; "fordered", [64]
; "fmul", [64]
; "fneg", [64]
; "funordered", [64]
; "fsqrt", [64]
; "fsub", [64]
; "f2f", [32; 64]
; "f2f", [64; 32]
; "i2f", [32; 64]
; "f2i", [64; 32]
; "minf", [32] (* from simplifier *)
; "minf", [64] (* from simplifier *)
; "pinf", [32] (* from simplifier *)
; "pinf", [64] (* from simplifier *)
; "mzero", [32] (* from simplifier *)
; "mzero", [64] (* from simplifier *)
; "pzero", [32] (* from simplifier *)
; "pzero", [64] (* from simplifier *)
; "round_up", [] (* from simplifier *)
; "round_down", [] (* from simplifier *)
; "round_zero", [] (* from simplifier *)
; "round_nearest", [] (* from simplifier *)
; "NaN", [23; 32] (* from simplifier *)
(* ; "NaN", [52; 64] from simplifier *)
; "add_overflows", [32]
; "div_overflows", [32]
; "mul_overflows", [32]
; "mulu_overflows", [32]
; "quot_overflows", [32]
; "sub_overflows", [32]

```

```

        ; "not",      []
        ; "bool",     []
        ; "disjoin",  []
        ; "conjoin",  []
        ; "bit",      []
    ]}
; T.globals          = globals
; T.rounding_mode    = rounding_model (* Rtl.reg rm_reg *)
; T.named_locs      = Strutil.assoc2map
                      ["IEEE 754 rounding mode",    rounding_model
                      ;"IEEE 754 rounding results", rounding_results1
                      ]
; T.data_section     = "data"
; T.charset          = "latin1" (* REMOVE THIS FROM TARGET.T *)
}

```

X.1.8 Variable Placement

If we get a 64-bit variable that is not kinded float, we could put it in a floating-point temporary, but we're not that aggressive.

```

⟨sparc.ml 689a⟩+≡ (676b) <686e
let warning s = Printf.eprintf "backend warning: %s\n" s
let placevars =
  let is_float w kind _ = kind = $= "float" in
  let warn ~width ~alignment ~kind = () in
  let mk_stage ~temps =
    A.choice
    [ is_float,          A.widen (Auxfuns.round_up_to ~multiple_of: 32)
    ; (fun w h _ -> w <= 32), A.widen (fun _ -> 32) *> temps 't'
    ; A.is_any,          A.widen (Auxfuns.round_up_to ~multiple_of: 8)
    ] in
  Placevar.mk_automaton ~warn ~vfp ~memspace:mSPACE mk_stage

```

X.2 SPARC Assembler

```

⟨sparcasm.mli 689b⟩≡
(* claude: node/make's shape used to be built on the older Cfgx.M
 * representation (Rtl.rtl Cfgx.M.node); Cfgutil.emit (the only real
 * caller, per the comment below) has since moved to the newer
 * Zipcfg.Rep-based "call" node, same shape arch/ppc/ppcasm.mli uses. *)
val make :
  (('a, 'b, 'c, 'd) Procedure.t -> 'cfg -> (Zipcfg.Rep.call -> unit) ->
   (Rtl.rtl -> unit) -> (string -> unit) -> unit) ->
  out_channel -> ('cfg * ('a, 'b, 'c, 'd) Procedure.t) Asm.assembler
(* pass Cfgutil.emit *)

```

X.2.1 Implementation

```

⟨sparcasm.ml 689c⟩≡
(* claude: was Cfgx.M (the older Cfg representation); Cfgutil.emit (the
 * only real caller, see sparcasm.mli) has since moved to Zipcfg/
 * Zipcfg.Rep, same as arch/ppc/ppcmach.ml already uses. *)
module G = Zipcfg
module GR = Zipcfg.Rep
module SM = Strutil.Map
⟨Sparcasm utilities 690a⟩

```

```

⟨Sparcasm definitions 690c⟩
let make emitter fd = new asm emitter fd

⟨Sparcasm utilities 690a⟩≡ (689c)
let fprintf = Printf.fprintf
let unimpf fmt = Printf.kprintf Impossible.unimp fmt

⟨definition of manglespec (for the name mangler) 690b⟩≡ (690c)
let spec =
  let reserved = [] in (* list reserved words here so we can avoid them *)
  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '.'
    | '_' -> true
    | _ -> false in
  let replace = function
    | x when id x -> x
    | _ -> '_'
  in
  { Mangle.preprocess = (fun x -> x)
    ; Mangle.replace = replace
    ; Mangle.reserved = reserved
    ; Mangle.avoid = (fun x -> x ^ "_")
  }

⟨Sparcasm definitions 690c⟩≡ (689c)
⟨definition of manglespec (for the name mangler) 690b⟩
class ['cfg, 'a, 'b, 'c, 'd] asm emitter fd
  : ['cfg * ('a, 'b, 'c, 'd) Procedure.t] Asm.assembler =
object (this)
  val _fd = fd
  val _mangle = (Mangle.mk spec)
  val mutable _syms = SM.empty
  method globals _ = ()
  method private new_symbol name =
    let s = Symbol.with_mangler _mangle name in
    _syms <- SM.add name s _syms;
    s

  ⟨private assembly state 692c⟩
  method private print l = List.iter (output_string _fd) l

  ⟨Sparcasm assembly methods 690d⟩
end

⟨Sparcasm assembly methods 690d⟩≡ (690c) 690e▷
method import s = this#new_symbol s

⟨Sparcasm assembly methods 690e⟩+≡ (690c) <690d 691a>
method export s =
  let sym = this#new_symbol s in
  Printf.fprintf _fd ".global %s\n" sym#mangled_text;
  sym
method local s = this#new_symbol s
method label (s: Symbol.t) = fprintf _fd "%s:\n" s#mangled_text
method section name =
  _section <- name;
  (* claude: pcap/pcmap_data must carry the ALLOC flag or the runtime

```

```

* data lands outside every PT_LOAD segment and Cmm_lookup_entry
* always reads zeroes - same fix/reasoning as arch/x86/x86asm.ml's
* and arch/ppc/ppcasm.ml's #section (GNU as only gives default
* flags to the standard section names). Confirmed via gdb: without
* this, the live pcmap table at Cmm_pc_map..Cmm_pc_map_limit was
* entirely zero bytes, not just missing one entry. *)
(match name with
| "pcmap" | "pcmap_data" ->
  fprintf _fd ".section \"%.s\", \"a\", @progbits\n" name
| _ ->
  fprintf _fd ".section \"%.s\", \"a\", @progbits\n" name)
method current = _section
method org n = Impossible.unimp "no .org in SPARC assembler"
method align n =
  if n <> 1 then fprintf _fd ".align %d\n" n;
  _a <- Alignment.align n _a
method addloc n =
  if n <> 0 then fprintf _fd ".skip %d\n" n;
  _a <- Alignment.add n _a
method zeroes (n:int) =
  fprintf _fd ".skip %d\n" n;
  _a <- Alignment.add n _a

⟨Sparcasm assembly methods 691a⟩+≡ (690c) <690e 691b>
method value (v:Bits.bits) =
  let altfmt = Bits.to_hex_or_decimal_string ~declimit:256 in
  let w = Bits.width v in
  if w / 8 <= Alignment.alignment _a && w <= 32 then
    (* we can emit the value directly *)
    match Bits.width v with
    | 8 -> fprintf _fd ".byte %Ld\n" (Bits.S.to_int64 v)
    | 16 -> fprintf _fd ".half %.s\n" (altfmt v)
    | 32 -> fprintf _fd ".word %.s\n" (altfmt v)
    | w -> unimpf "emission width %s in SPARC assembler" (string_of_int w)
  else
    (* have to emit as smaller chunks, assuming it is a power of 2 *)
    (this#value (Bits.Ops.lobits (w/2) (Bits.Ops.shrl v (Bits.U.of_int (w/2) w)));
     this#value (Bits.Ops.lobits (w/2) v))
  (*
  else
    (this#value (Bits.Ops.lobits 32 (Bits.Ops.shrl v (Bits.U.of_int (w - 32) w)));
     this#value (Bits.Ops.lobits (w - 32) v))
  *)

⟨Sparcasm assembly methods 691b⟩+≡ (690c) <691a 691c>
method addr a =
  match Reloc.if_bare a with
  | Some b -> this#value b
  | None -> let const bits = Printf.sprintf "0x%Lx" (Bits.U.to_int64 bits) in
    assert (Reloc.width a = 32);
    fprintf _fd ".word %.s\n" (Asm.reloc_string const a)

⟨Sparcasm assembly methods 691c⟩+≡ (690c) <691b 691d>
method emit = ()

⟨Sparcasm assembly methods 691d⟩+≡ (690c) <691c 692a>
method comment s = fprintf _fd "!. %.s\n" s

method const (s: Symbol.t) (b:Bits.bits) =
  Impossible.unimp "Don't know how to make a constant for SPARC"

```

For alternate returns, we need to know the size of a long jump.

```

<Sparcasm assembly methods 692a>+≡ (690c) <691d 692b>
method longjmp_size () = 8
(*
  Impossible.unimp "longjmp size not set for SPARC -- needed for alternate returns"
*)

<Sparcasm assembly methods 692b>+≡ (690c) <692a
method private instruction rtl =
  output_string _fd "\t";
  output_string _fd (Sparcrec.to_asm rtl);
  output_string _fd "\n"

(* claude: adapted to the Zipcfg.Rep.call node shape (cal_i/
 * cal_altrets/cal_contedges) - see arch/ppc/ppcmach.ml's own 'call'
 * method, same shape. The "ba ...\n\tnop" longjmp idiom (branch
 * always, with its mandatory delay-slot nop) is SPARC-specific,
 * kept from the original. *)
method private call (node : GR.call) =
  let longjmp edge = fprintf _fd "\tba %s\n\tnop\n" (_mangle (snd edge.G.node)) in
  let rec output_altret_jumps n edges =
    if n > 0 then
      match edges with
      | edge :: edges -> (longjmp edge; output_altret_jumps (n-1) edges)
      | [] -> Impossible.impossible "contedge count" in
  begin
    this#instruction node.GR.cal_i;
    output_altret_jumps node.GR.cal_altrets (List.tl node.GR.cal_contedges)
  end

(* this better be only used for printing out functions... *)
method cfg_instr (cfg, proc) =
  (* We have to emit a label for the procedure's
   entry point. This is what symbol is for. Simply use this#label? *)
  let symbol = proc.Procedure.symbol in
  let label l = this#label (try SM.find l _syms with Not_found -> this#local l) in
  this#label symbol;
  (emitter proc cfg (this#call) (this#instruction) label : unit)

<private assembly state 692c>≡ (690c)
val mutable _a = Alignment.init 32
val mutable _section = "bogus section"

```

X.3 SPARC calling conventions

```

<sparccall.mli 692d>≡
val cconv :
  return_to:(Rtl.exp -> Rtl.rtl) ->
  (unit, Mflow.cut_args) Target.map ->
  string -> Automaton.cc_spec ->
  Call.t

```

X.4 Implementation of SPARC calling conventions

```

<sparccall.ml 692e>≡
<sparccall.ml 693a>

```

```

(sparccall.ml 693a)≡ (692e) 693c▷
module R = Rtl
module RU = Rtlutil
module RS = Register.Set
module Rg = Sparcregs
module A = Automaton
module C = Call
let sprintf = Printf.sprintf
let impossf fmt = Printf.kprintf Impossible.impossible fmt

(SPARC calling convention automata in Lua 693b)≡ 694a▷
A = Automaton
Sparc = Sparc or {}
Sparc.cc = Sparc.cc or {}
Sparc.cc["C" ] = Sparc.cc["C"] or {}
Sparc.cc["notail"] = Sparc.cc["C"] or {}
Sparc.cc["C--"] = Sparc.cc["C--"] or {}

```

SPARC registers and their conventional uses Cheats, cheats, and more cheats...

```

(sparccall.ml 693c)+≡ (692e) <693a 694b▷
let rspace = Rg.Spaces.r.Space.space
let fspace = Rg.Spaces.f.Space.space
let dspace = Rg.Spaces.d.Space.space

let r i = (rspace, i, R.C 1)
let f i = (fspace, i, R.C 1)
let x i = (fspace, 8+2*i, R.C 2)
let oreg i = r (8 + i) (* %o(i) *)
let ireg i = r (24 + i) (* %i(i) *)

let ( *> ) = A.( *> )

(* claude: SPARC register windows mean the SAME automaton cannot
 * correctly serve both directions for integer/pointer results: the
 * callee produces its own return value via %i0/%i1 (see sparccc.ml's
 * c_results, used below by 'epilog' - correct for that direction), but
 * the CALLER reads a just-made call's result via %o0/%o1 - same
 * physical register file, different window-relative name depending on
 * which side of the 'save'/'restore' boundary you're on. Float results
 * have no such split (there is only one %f bank, not windowed), so only
 * the integer/pointer branch differs. Mirrors sparccc.ml's c_results
 * shape but swapped to o-registers; kept local here since it only
 * matters for this one incoming direction ('call_results' below, not
 * 'epilog'). Root-caused empirically: fork-tiger's runtime.c--'s
 * "getenv" call read its own %i0 (an unrelated incoming argument
 * register) instead of the freshly-returned %o0, so a NULL-returning
 * getenv looked non-NULL and the following atoi(garbage) segfaulted. *)
let call_results_via_o =
  A.choice
    [ A.is_kind "float", A.userregs (List.map f (Auxfuns.from 0 ~upto:7)) false
    ; A.is_any, A.widen (Auxfuns.round_up_to ~multiple_of:32)
      *> A.widths [32; 64]
      *> A.userregs [oreg 0; oreg 1] false
    ]

let vol_regs = RS.of_list (List.map r ([1;2;3;4;31] @ Auxfuns.from 16 ~upto:23))
let nv_regs = RS.of_list (List.map r [])
let vol_fp =
  RS.of_list (List.map f (Auxfuns.from 0 ~upto:7))

```

```

        @ List.map x (Auxfuns.from 0 ~upto:7)
      )
let all_regs      = List.fold_left (fun acc e -> RS.union acc e) RS.empty
                    [ vol_regs; nv_regs; vol_fp ]

```

```

let saved_nvr temps =
  let t = Talloc.Multiple.loc temps 't' in
  let u = Talloc.Multiple.loc temps 'u' in
  let q = Talloc.Multiple.loc temps 'q' in
  fun ((sp, _, _), i, _) as reg ->
    let bad () = impossf "cannot save %c%d" sp i in
    let w = Register.width reg in
    (match sp with 'r' -> t | 'f' -> u | 'x' -> q | _ -> bad()) w

```

Stack pointer is doubleword-aligned (which we actually force to be quad word aligned in case we are really on a sparc 9)

```

<SPARC calling convention automata in Lua 694a>+≡ <693b 695a>
Sparc.sp_align  = 16
Sparc.wordsize  = 32

```

```

function reg(sp, i, w, agg)
  return Register.create { space = sp, index = i, cellsize = w, agg = agg }
end
function r(i) return (reg("r", i, Sparc.wordsize)) end
function f(i) return (reg("f", i, Sparc.wordsize, 'big')) end
function x(i) return (reg("f", 8+2*i, Sparc.wordsize * 2, 'big')) end
function o(i) return (reg("o", i, Sparc.wordsize)) end
function i(i) return (reg("i", i, Sparc.wordsize)) end

```

```

Sparc.vol_fp = (f(0)..f(7)) .. (x(0)..x(7))

```

Conventions governing the stack

```

<sparccall.ml 694b>+≡ (692e) <693c 695e>

```

```

let ra      = R.reg (r 31)
let spr     = r 14
let sp      = R.reg spr
let spval   = R.fetch sp 32
let sp_align = 16
let growth  = Memalloc.Down
let bo      = R.BigEndian

```

```

(* claude: the exact same i/o direction split as call_results_via_o
 * above, but for ARGUMENTS rather than results: Sparccc.c_call's
 * %o0-%o5 are correct for OUTGOING call args (call_actuals below, i.e.
 * "I am making a call and placing my arguments"), but wrong for
 * INCOMING ones (prolog below, i.e. "I am a callee reading my own
 * parameters") - a procedure's own parameters arrive via %i0-%i5 after
 * its own 'save', not %o0-%o5 (which are its own, unrelated outgoing
 * scratch registers for calls it goes on to make). Root-caused
 * empirically, the same way as call_results_via_o: fork-tiger's
 * stdlibcmm.c--'s "new_string(bits32 size)" compiled to a bare
 * "mov %o0,%o0" (a no-op self-move of garbage) instead of
 * "mov %i0,%o0" when forwarding its own "size" parameter into a
 * tig_alloc call - main/tig_call_gc's own incoming parameters happened
 * to never be read for anything, so this exact same bug was already
 * present (visible as the same suspicious "mov %o0,%o0" in their own
 * disassembly) without ever being exercised until a parameter's value
 * actually got used. *)

```

```

let call_via_i =

```

```

A.widen (Auxfuns.round_up_to ~multiple_of:32)
*> A.useregs (List.map ireg (Auxfuns.from 0 ~upto:5)) false
*> A.overflow ~growth:Memalloc.Up ~max_alignment:sp_align

```

Automata for passing values—C convention

⟨SPARC calling convention automata in Lua 695a⟩+≡ ◁694a 695b▷

```

Sparc.cc["C"].results =
  A.choice { "float", A.useregs (f(0)..f(7)),
             A.is_any, { A.widen(32, "multiple")
                        , A.widths {32, 64}
                        , A.useregs { i(0), i(1) }
                        }
            }

```

```

Sparc.cc["C"].call =
  { A.widen(32, "multiple")
  , A.useregs (o(0)..o(5))
  , A.overflow { growth = "up", max_alignment = Sparc.sp_align }
  }

```

⟨SPARC calling convention automata in Lua 695b⟩+≡ ◁695a 695c▷

```

Sparc.cc["C"].cutto =
  { A.widen(32, "multiple")
  , A.useregs (r(24)..r(29))
  , A.overflow { growth = "up", max_alignment = Sparc.sp_align }
  }

```

⟨SPARC calling convention automata in Lua 695c⟩+≡ ◁695b 695d▷

```

Sparc.cc["C--"].call = Sparc.cc["C"].call
Sparc.cc["notail"].call = Sparc.cc["C"].call

Sparc.cc["C--"].results = {
  Sparc.cc["C"].results,
  A.overflow { growth = "up", max_alignment = Sparc.sp_align }
}

Sparc.cc["notail"].results = Sparc.cc["C--"].results

```

```

Sparc.cc["C--"].cutto = Sparc.cc["C"].cutto
Sparc.cc["notail"].cutto = Sparc.cc["C"].cutto

```

Finally we register the calling conventions.

⟨SPARC calling convention automata in Lua 695d⟩+≡ ◁695c

```

A.register_cc(Backend.sparc.target, "C", Sparc.cc["C"])
A.register_cc(Backend.sparc.target, "notail", Sparc.cc["notail"])
A.register_cc(Backend.sparc.target, "C--", Sparc.cc["C--"])
A.register_cc(Backend.sparc.target, "C-- thread", Sparc.cc["C--"]) -- damn lies

```

X.4.1 Implementation based on Callspec

⟨sparccall.ml 695e⟩+≡ (692e) ◁694b 696a▷

```

let wordsize = 32
let memsize = 8
let byteorder = bo
let mspace = Rg.Spaces.m.Space.space
let vfp = Vfp.mk wordsize
let std_sp_location = RU.add wordsize vfp (R.late "minus frame size" wordsize)
let mk_automaton block_name automaton () =
  Block.srelative vfp block_name (A.at mspace) automaton

```



```

let old_end block = RU.addk wordsize (Block.base block) (Block.size block)
let young_end block = Block.base block

⟨sparccall.ml 696a⟩+≡ (692e) <695e 696b>
let tail_overflow =
  { C.parameter_deallocator = C.Callee
    ; C.result_allocator      = C.Caller
  }

let c_overflow =
  { C.parameter_deallocator = C.Caller
    ; C.result_allocator      = C.Caller
  }

⟨sparccall.ml 696b⟩+≡ (692e) <696a 696c>
let prolog auto =
  let autosp a = young_end a.A.overflow in
  C.incoming ~growth:growth ~sp:sp
  ~mkauto:(mk_automaton "in call parms" call_via_i)
  ~autosp:autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a)

⟨sparccall.ml 696c⟩+≡ (692e) <696b 696d>
let epilog auto =
  C.outgoing ~growth:growth ~sp:sp
  ~mkauto:(mk_automaton "out ovfl results" auto.A.results)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun _ _ -> vfp)

⟨sparccall.ml 696d⟩+≡ (692e) <696c 696e>
let call_actuals auto =
  C.outgoing ~growth:growth ~sp:sp
  ~mkauto:(mk_automaton "out call parms" auto.A.call)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun a sp -> std_sp_location)

⟨sparccall.ml 696e⟩+≡ (692e) <696d 697a>
(* claude: unlike epilog below (which reuses auto.A.results, and so picks
 * up "C--"'s extra overflow_up stage via Sparccc.cmm_results), this
 * always uses the bare 2-register call_results_via_o with no overflow
 * stage - a call returning more values than fit in %o0/%o1 isn't
 * handled yet. Not believed to be exercised by anything simple-program-
 * shaped (a "C" convention runtime call returns at most one pointer/int
 * here); flagged rather than solved so a real failure is loud instead of
 * silently wrong. *)
let call_results auto =
  let autosp = (fun a -> std_sp_location) in
  C.incoming ~growth:growth ~sp:sp
  ~mkauto:(mk_automaton "in ovfl results" call_results_via_o)
  ~autosp:autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> std_sp_location)

```

These might be wrong. No tests yet.

```

(sparccall.ml 697a)+≡ (692e) <696e 697b>
  let also_cuts_to auto =
    let autosp = (fun r -> std_sp_location) in
    C.incoming ~growth:growth ~sp:sp
      ~mkauto:(mk_automaton "in cont parms" auto.A.cutto)
      ~autosp:autosp
      ~postsp:(fun _ -> std_sp_location)
      ~insp:(fun a _ -> autosp a)

  let cut_actuals auto base =
    C.outgoing ~growth:growth ~sp:sp
      ~mkauto:(fun () -> A.at mspace ~start:base auto.A.cutto)
      ~autosp:(fun r -> std_sp_location)
      ~postsp:(fun a sp -> std_sp_location)

(sparccall.ml 697b)+≡ (692e) <697a 697c>
  let ra_on_entry block = R.fetch ra wordsize
  let where_to_save_ra ra_on_entry temp = Talloc.Multiple.loc temp 't' 32
  (* ra *)

(sparccall.ml 697c)+≡ (692e) <697b 697d>
  let rtn ccname return_to k n ~ra =
    if k > n then impossf "Return <k/n> has k:%d > n:%d\n" k n
    else return_to ra
  (*
    if k = 0 && n = 0 then return_to ra
    else Impossible.impossible ("alternate return using "^ccname^" calling convention")
  *)

(sparccall.ml 697d)+≡ (692e) <697c 698b>
  (* Really intended for the C convention *)
  let to_call ~cutto ~return_to ~altret ccname auto =
    <Sparccall it's impossible for the stack pointer to be a useful register too 698a>
  ;
  let return k n ~ra =
    if altret then rtn ccname return_to k n ~ra
    else if k = 0 && n = 0 then return_to ra
    else impossf "alternate return not allowed in %s calling convention" ccname
  in
  { C.name          = ccname
  ; C.overflow_alloc = c_overflow
  ; C.call_parms    = {C.in'=prolog      auto; C.out=call_actuals auto}
  ; C.cut_parms     = {C.in'=also_cuts_to auto; C.out=cut_actuals auto}
  ; C.results       = {C.in'=call_results auto; C.out=epilog      auto}

  ; C.stack_growth  = growth
  ; C.stable_sp_loc  = std_sp_location
  (* claude: reserved for indirect jumps (ast2ir.ml stores the target here
   * before jumping through it); r5 is otherwise unused by this backend -
   * not zero/sp/fp/ra, not in vol_regs/nv_regs. Mirrors ppc.ml's rreg 7
   * and x86call.ml's edx, both similarly arbitrary volatile picks. *)
  ; C.jump_tgt_reg   = R.reg (r 5)
  ; C.replace_vfp     = Vfprewrite.replace_with ~sp:sp
  ; C.sp_align       = sp_align
  ; C.ra_on_entry     = ra_on_entry
  ; C.where_to_save_ra = where_to_save_ra
  ; C.ra_on_exit      = (fun _ _ t -> ra)
  ; C.sp_on_unwind    = (fun e -> RU.store sp e)
  (* claude: was "fun _ _ -> Rtl.null" - a genuine no-op, same bug

```

```

* ppc.ml's sp_on_jump had (see its own comment): a tail call
* ("jump") never restored sp before re-entering the target's own
* prologue, which unconditionally does its own "save %sp,-N,%sp".
* Every tail-recursive iteration would therefore shrink sp by
* another frame instead of reusing the current one. Mirrors
* ppc.ml's fix (RU.store sp (addk (Block.base b) (-24)), its own
* linkage-area constant) using this backend's -96 reserved
* register-window-save + minimum-outgoing-arg area (see
* arch/sparc/sparcbackend.ml's window_and_arg_area_size - not
* referenced directly to avoid a backwards dependency, but must
* stay in sync with it). *)
; C.sp_on_jump      = (fun b _ -> RU.store sp (RU.addk wordsize (Block.base b) (-96)))
; C.pre_nvregs      = nv_regs
; C.volregs         = Register.Set.diff all_regs nv_regs
; C.saved_nvr       = saved_nvr
; C.return          = return
}

```

⟨Sparccall *it's impossible for the stack pointer to be a useful register too* 698a⟩≡ (697d)

```

if Register.Set.mem spr all_regs then
  Impossible.impossible
  (Printf.sprintf "In convention \"%s\", stack pointer is also an ordinary register"
    ccname)
else
  ()

```

Here's our CC lookup function.

⟨sparccall.ml 698b⟩+≡ (692e) <697d

```

let c ~altret ~return_to cut ccname auto =
  to_call ~altret ~cutto:cut ~return_to ccname auto (* postprocess auto *)

let cconv ~return_to cut ccname stage =
  let f =
    match ccname with
    | "C--"      -> c ~altret:true
    | "notail"   -> c ~altret:true
    | "C"        -> c ~altret:false
    | "C-- thread" -> c ~altret:false (* damn lies *)
    | _         -> Impossible.unimp ccname
  in f ~return_to cut ccname stage

```

X.5 SPARC Recognizer

⟨sparcrec.mli 698c⟩≡

```

val to_asm : Rtl.rtl -> string
val is_instruction : Rtl.rtl -> bool

```

A few abbreviations.

⟨Sparcrec *modules* 698d⟩≡ (699a)

```

module R = Rtl
module RU = Rtlutil
module RP = Rtl.Private
module SS = Space.Standard32
module Down = Rtl.Dn
module Up = Rtl.Up

```

```

⟨sparcrec.mlb 699a⟩≡
  %head {:
    ⟨Sparcrec modules 698d⟩
    ⟨Sparcrec code to precede the labeler 699b⟩
  :}
  %tail {:
    ⟨Sparcrec code to follow the labeler 709a⟩
  :}

  %term
    ⟨Sparcrec names of types of terminals 700a⟩

  %%
  ⟨Sparcrec rules 700b⟩

```

X.5.1 Utilities

```

⟨Sparcrec code to precede the labeler 699b⟩≡ (699a)
  let infinity = Camlburg.inf_cost
  let guard b = if b then 0 else infinity

  let const32 b =
    assert (Bits.width b = 32);
    Nativeint.to_string (Bits.U.to_native b)

  let const64 b =
    assert (Bits.width b = 64);
    let hi32 = Bits.Ops.lobits 32 (Bits.Ops.shrl b (Bits.U.of_int 32 64)) in
    let lo32 = Bits.Ops.lobits 32 b in
    (Nativeint.to_string (Bits.U.to_native hi32),
     Nativeint.to_string (Bits.U.to_native lo32))

  exception Error of string
  let sprintf    = Printf.sprintf
  let s          = Printf.sprintf
  let error msg = raise (Error msg)

  let rspace = ('r', R.BigEndian, Cell.of_size 32)
  let spl = RP.Reg(rspace, 14, R.C 1)
  let sp  = RP.Fetch(spl, 32)
  let ral = RP.Reg(rspace, 31, R.C 1)
  let ra  = RP.Fetch(ral, 32)
  let yregl = RP.Reg Sparcregs.y
  let yreg = RP.Fetch(yregl, 32)

  let idiomatic_reg_name n =
    if n = 14 then "%sp"
    else if n = 30 then "%fp"
    else if n >= 0 && n < 8 then sprintf "%g%i" n
    else if n < 16 then sprintf "%o%i" (n - 8)
    else if n < 24 then sprintf "%l%i" (n - 16)
    else if n < 32 then sprintf "%i%i" (n - 24)
    else Impossible.impossible (sprintf "Register %r%i doesn't exist" n)

  let positive b = Bits.Ops.gt b (Bits.zero 32)
  let negative b = Bits.Ops.lt b (Bits.zero 32)

  let in_proc = Reinit.ref false

```

X.5.2 Recognizer terminals, nonterminals, and constructors

⟨Sparcrec names of types of terminals 700a⟩≡ (699a)
n w bits symbol

X.5.3 Recognizer Rules

Constants

⟨Sparcrec rules 700b⟩≡ (699a) 700c▷
lconst : Link(symbol, w) {: symbol#mangled_text :}
const : Bits(b:bits) [{: guard (Bits.width b = 32) :}] {: const32 b :}
pos : Bits(b:bits) [{: guard (Bits.width b = 32 && positive b) :}] {: const32 b :}
neg : Bits(b:bits) [{: guard (Bits.width b = 32 && negative b) :}] {: const32 b :}

constx : Bits(b:bits) [{: guard (Bits.width b = 64) :}] {: const64 b :}

four : Bits(bits) [{: guard (Bits.eq bits (Bits.S.of_int 4 32)) :}] {:():}
one : Bits(bits) [{: guard (Bits.eq bits (Bits.S.of_int 1 32)) :}] {:():}
zero : Bits(bits) [{: guard (Bits.eq bits (Bits.S.of_int 0 32)) :}] {:():}

Location Types

⟨Sparcrec rules 700c⟩+≡ (699a) <700b 701a▷
rreg : Fetch(rregl, w) {: rregl :}
rregl : Reg('r', n:int) {: idiomatic_reg_name n :}

freg : Fetch(fregl, w) {: fregl :}
fregl : Reg('f', n:int) {: sprintf "%f%i" n :}

dreg : Fetch(dregl, w) {: dregl :}
dregl : RegPair('f', n:int) {: sprintf "%f%i" n :}

dregnum : Fetch(dregnuml, w) {: dregnuml :}
dregnuml : RegPair('f', n:int) {: n :}

yreg : Fetch(yregl, w) {: yregl :}
yregl : Reg('y', n:int) {: "%y" :}

fsr : Fetch(fsrl, w) {: fsrl :}
fsrl : Reg('c', n:int) [{: guard (n = 4) :}] {: "%fsr" :}

-- this is stupid, but we don't know how to deal with
-- changing register windows at this point, so this allows
-- us to do an ugly hack.
result_reg : Fetch(Reg('i', n:int), w) {: sprintf "%o%i" n :}
result_regl : Reg('i', n:int) {: sprintf "%i%i" n :}

arg_reg : Fetch(Reg('o', n:int), w) {: sprintf "%i%i" n :}
arg_regl : Reg('o', n:int) {: sprintf "%o%i" n :}

regl : rregl {: rregl :}
regl : result_regl {: result_regl :}
regl : arg_regl {: arg_regl :}

reg : rreg {: rreg :}
reg : result_reg {: result_reg :}
reg : arg_reg {: arg_reg :}

mem : Fetch(meml, w) {: meml :}

```

meml : Mem(reg, w)    {: s "[%s]" reg :} -- indirect

reg_or_const : reg    {: reg :}
reg_or_const : const  {: const :}

target : lconst {: lconst :}

```

Special Locations

⟨Sparcrec rules 701a⟩+≡ (699a) ◁700c 701b▷

```

pcl: Reg('c', 0) {: () :}
npc1: Reg('c', 1) {: () :}
ccl: Reg('c', 2) {: () :}
spl: Reg('r', 14) {: () :}
fpl: Reg('r', 30) {: () :}
ral: Reg('r', 31) {: () :}
cwpl: Reg('k', 0) {: () :}

pc: Fetch(pcl, w) {: () :}
cc: Fetch(ccl, w) {: () :}
sp: Fetch(spl, w) {: () :}
ra: Fetch(ral, w) {: () :}
cwp: Fetch(cwpl, w) {: () :}

```

Data movement

⟨Sparcrec rules 701b⟩+≡ (699a) ◁701a 703▷

```

inst : Store(dst:fregl, Ftoi(src:freg), w)
  {: sprintf "fstoi %s, %s" src dst :}

inst : Store(dst:fregl, Ftoi(src:dreg), w)
  {: sprintf "fdtoi %s, %s" src dst :}

inst : Store(dst:fregl, Itof(src:freg), w)
  {: sprintf "fitos %s, %s" src dst :}
inst : Store(dst:dregl, Itof(src:freg), w)
  {: sprintf "fitod %s, %s" src dst :}

inst : Store(dst:fregl, Ftof(src:freg), w)
  {: sprintf "fmovs %s, %s" src dst :}
inst : Store(dst:dregl, Ftof(src:freg), w)
  {: sprintf "fstod %s, %s" src dst :}
inst : Store(dst:fregl, Ftof(src:dreg), w)
  {: sprintf "fdtos %s, %s" src dst :}

-- load symbol
inst : Store(dst:regl, src:lconst, w)
  {: sprintf "set %s, %s" src dst :}

-- load immediate
inst : Store(dst:regl, src:const, w)
  {: sprintf "set %s, %s" src dst :}

-- register move
inst : Store(dst:regl, src:reg, w)
  {: sprintf "mov %s, %s" src dst :}

inst : Store(dst:fregl, src:freg, w)
  {: sprintf "fmovs %s, %s" src dst :}

```

```

inst : Store(dst:regl, src:yreg, w)
{: sprintf "rd %s, %s" src dst :}

-- THIS SHOULD BE DONE IN THE POSTEXPANDER
inst : Store(dst:dregnuml, src:dregnum, w)
{: sprintf "fmovs %%f%d, %%f%d\nfmovs %%f%d, %%f%d" src dst (src+1) (dst+1):}

-- memory load
inst : Store(dst:regl, src:mem, w)
{: if w = 64 then sprintf "ldx %s, %s" src dst
  else sprintf "ld %s, %s" src dst :}
inst : Store(dst:regl, Zxbyte(src:mem), w)
{: sprintf "ldub %s, %s" src dst :}
inst : Store(dst:regl, Zxhalf(src:mem), w)
{: sprintf "lduh %s, %s" src dst :}

inst : Store(dst:regl, Sxbyte(src:mem), w)
{: sprintf "ldsb %s, %s" src dst :}
inst : Store(dst:regl, Sxhalf(src:mem), w)
{: sprintf "ldsh %s, %s" src dst :}

inst : Store(dst:fregl, src:mem, w)
{: sprintf "ld %s, %s" src dst :}
inst : Store(dst:dregl, src:mem, w)
{: sprintf "ldd %s, %s" src dst :}

inst : Store(dst:fsrl, src:mem, w)
{: sprintf "ld %s, %s" src dst :}

-- memory store
inst : Store(dst:meml, src:reg, w)
{: sprintf "st %s, %s" src dst :}
inst : Store(dst:meml, Lobyte(src:reg), w)
{: sprintf "stb %s, %s" src dst :}
inst : Store(dst:meml, Lohalf(src:reg), w)
{: sprintf "sth %s, %s" src dst :}

inst : Store(dst:meml, src:freg, w)
{: sprintf "st %s, %s" src dst :}
inst : Store(dst:meml, src:dreg, w)
{: sprintf "std %s, %s" src dst :}

inst : Store(dst:meml, src:fsr, w)
{: sprintf "st %s, %s" src dst :}

-- memory store to offset
-- note that y must be a 13-bit constant...probably need a different
-- rule that can split 32-bit constants.
inst : Store(Mem(Add(x:reg, y:const),mw:w), src:reg, w)
{: sprintf "st %s, [%s+%s]" src x y :}
inst : Store(Mem(Add(x:reg, y:const),mw:w), src:freg, w)
{: sprintf "st %s, [%s+%s]" src x y :}
inst : Store(Mem(Add(x:reg, y:const),mw:w), src:dreg, w)
{: sprintf "std %s, [%s+%s]" src x y :}

inst : Store(Mem(Add(x:reg, y:const), mw:w), src:fsr, w)
{: sprintf "st %s, [%s+%s]" src x y :}

-- memory load from offset

```

```

inst : Store(dst:regl, Fetch(Mem(Add(x:reg, y:const), mw:w), w), w)
{: sprintf "ld [%s+%s], %s" x y dst :}
inst : Store(dst:fregl, Fetch(Mem(Add(x:reg, y:const), mw:w), w), w)
{: sprintf "ld [%s+%s], %s" x y dst :}

inst : Store(dst:dregl, Fetch(Mem(Add(x:reg, y:const), mw:w), w), w)
{: sprintf "ldd [%s+%s], %s" x y dst :}

inst : Store(dst:fsrl, Fetch(Mem(Add(x:reg, y:const), mw:w), w), w)
{: sprintf "ld [%s+%s], %s" x y dst :}

```

Arithmetic

⟨Sparcrec *rules* 703⟩+≡ (699a) <701b 705a>

```

-- add
inst : Store(dst:regl, Add(x:reg_or_const, y:reg), w)
{: sprintf "add %s, %s, %s" y x dst :}
inst : Store(dst:regl, Add(x:reg, y:reg_or_const), w)
{: sprintf "add %s, %s, %s" x y dst :}

-- sub
inst : Store(dst:regl, Sub(x:reg, y:reg_or_const), w)
{: sprintf "sub %s, %s, %s" x y dst :}

-- mul
inst : Store(dst:regl, Mul(x:reg_or_const, y:reg), w)
{: sprintf "smul %s, %s, %s" y x dst :}
inst : Store(dst:regl, Mul(x:reg, y:reg_or_const), w)
{: sprintf "smul %s, %s, %s" x y dst :}

-- mulx
inst : Par(Store(dst:regl, Mul(x:reg_or_const, y:reg), w),
           Store(yregl, Sparcmulxhi(x2:reg_or_const, y2:reg), w))
-- [{: guard (x = x2 && y = y2) :}]
-- I'd like to check this, but Burg won't let me!
{: sprintf "smul %s, %s, %s" x y dst :}

-- mulux
inst : Par(Store(dst:regl, Mul(x:reg_or_const, y:reg), w),
           Store(yregl, Sparcmuluxhi(x2:reg_or_const, y2:reg), w))
-- [{: guard (x = x2 && y = y2) :}]
-- I'd like to check this, but Burg won't let me!
{: sprintf "umul %s, %s, %s" x y dst :}

-- quot
inst : Store(dst:regl, Quot(x:reg, y:reg_or_const), w)
{: sprintf "sdiv %s, %s, %s" x y dst :}

-- divu
inst : Store(dst:regl, Divu(x:reg, y:reg_or_const), w)
{: sprintf "udiv %s, %s, %s" x y dst :}

-- neg
inst : Store(dst:regl, Neg(x:reg_or_const), w)
{: sprintf "neg %s, %s" x dst :}

-- and
inst : Store(dst:regl, And(x:reg, y:reg_or_const), w)
{: sprintf "and %s, %s, %s" x y dst :}
inst : Store(dst:regl, And(x:reg_or_const, y:reg), w)

```



```

{: sprintf "and %s, %s, %s" y x dst :}

-- or
inst : Store(dst:regl, Or(x:reg, y:reg_or_const), w)
{: sprintf "or %s, %s, %s" x y dst :}
inst : Store(dst:regl, Or(x:reg_or_const, y:reg), w)
{: sprintf "or %s, %s, %s" y x dst :}

-- xor
inst : Store(dst:regl, Xor(x:reg, y:reg_or_const), w)
{: sprintf "xor %s, %s, %s" x y dst :}
inst : Store(dst:regl, Xor(x:reg_or_const, y:reg), w)
{: sprintf "xor %s, %s, %s" y x dst :}

-- com
inst : Store(dst:regl, Com(x:reg_or_const), w)
{: sprintf "not %s, %s" x dst :}

-- shl
inst : Store(dst:regl, Shl(x:reg, y:reg_or_const), w)
{: sprintf "sll %s, %s, %s" x y dst :}

-- shrL
inst : Store(dst:regl, Shrl(x:reg, y:reg_or_const), w)
{: sprintf "srl %s, %s, %s" x y dst :}

-- shra
inst : Store(dst:regl, Shra(x:reg, y:reg_or_const), w)
{: sprintf "sra %s, %s, %s" x y dst :}

-- fdiv
inst : Store(dst:fregl, Fdiv(x:freg, y:freg), w)
{: sprintf "fdivs %s, %s, %s" x y dst :}
inst : Store(dst:dregl, Fdiv(x:dreg, y:dreg), w)
{: sprintf "fdivd %s, %s, %s" x y dst :}

-- fmul
inst : Store(dst:fregl, Fmul(x:freg, y:freg), w)
{: sprintf "fmuls %s, %s, %s" x y dst :}
inst : Store(dst:dregl, Fmul(x:dreg, y:dreg), w)
{: sprintf "fmuld %s, %s, %s" x y dst :}

-- fadd
inst : Store(dst:fregl, Fadd(x:freg, y:freg), w)
{: sprintf "fadds %s, %s, %s" x y dst :}
inst : Store(dst:dregl, Fadd(x:dreg, y:dreg), w)
{: sprintf "faddd %s, %s, %s" x y dst :}

-- fsub
inst : Store(dst:fregl, Fsub(x:freg, y:freg), w)
{: sprintf "fsubs %s, %s, %s" x y dst :}
inst : Store(dst:dregl, Fsub(x:dreg, y:dreg), w)
{: sprintf "fsubd %s, %s, %s" x y dst :}

-- fneg
inst : Store(dst:fregl, Fneg(x:freg), w)
{: sprintf "fnegs %s, %s" x dst :}

```

To negate a double-precision floating-point value, we negate the most significant word, which contains the sign bit. The least significant word is simply copied. Strictly speaking, this ought to be handled in the postexpander.

That would be good, because then we could get rid of dregnum1.

```
⟨Sparcrec rules 705a⟩+≡ (699a) ◁703 705b▷  
inst : Store(dst:dregnum1, Fneg(x:dregnum), w)  
  {: sprintf "fnegs %%f%d, %%f%d\n\tfmovs %%f%d, %%f%d" x dst (x+1) (dst+1) :}
```

Control Flow

```
⟨Sparcrec rules 705b⟩+≡ (699a) ◁705a 706▷  
-- call  
next: Store(reg1, Add(pc, four), w) {: reg1 :}  
inst: Par(Goto(target), next)  
  {: sprintf "call %s, 0\n\tnop" target :}  
inst: Par(Goto(reg), next)  
  {: sprintf "call %s, 0\n\tnop" reg :}  
  
-- decrement register window and allocate space on the stack  
inst : Save(x:sp, y:neg)  
  {: if not !in_proc then (in_proc := true; sprintf "save %%sp, %s, %%sp" y)  
    else sprintf "! Evil recognizer deleted add %%sp, %s, %%sp" y :}  
  
inst : Save(x:neg, y:sp)  
  {: if not !in_proc then (in_proc := true; sprintf "save %%sp, %s, %%sp" x)  
    else sprintf "! Evil recognizer deleted add %%sp, %s, %%sp" x :}  
  
inst : Save(x:sp, y:pos)  
  {: sprintf "! Evil recognizer deleted add %%sp, %s, %%sp" y :}  
  
inst : Save(x:pos, y:sp)  
  {: sprintf "! Evil recognizer deleted add %%sp, %s, %%sp" x :}  
  
-- increment register window and deallocate space on the stack  
restore: Store(cwpl, Add(cwp, one), w) {: () :}  
inst : Par(Goto(ra), restore)  
  {: (in_proc := false; "ret\n\trestore") :}  
  
-- Trap 3 tells the OS to flush all reg window data  
inst: Store(cwpl, zero, w) {: "ta 3" :}  
  
-- cut to  
inst : Par(Goto(r:reg), Store(spl, reg, w))  
  {: sprintf "jmp %s\n\tmov %s, %%sp" r reg :}  
  
-- branches  
inst: Goto(target) {: sprintf "ba %s\n\tnop" target :}  
  
-- indirect branches  
inst: Goto(reg)    {: sprintf "jmp %s\n\tnop" reg :}  
  
-- carry and borrow  
inst: Store(ccl, Sparcaddcc(x:reg, y:reg_or_const), w)  
  {: sprintf "addcc %s, %s, %%g0" x y :}  
  
inst: Store(d:reg1, Addc(x:reg, y:reg_or_const, Sparccarrybit(cc)), w)  
  {: sprintf "addx %s, %s, %s" x y d :}  
  
inst: Store(d:reg1, Subb(x:reg, y:reg_or_const, Sparccarrybit(cc)), w)  
  {: sprintf "subx %s, %s, %s" x y d :}  
  
-- assumption: x and y are the same in both sub-expressions  
inst: Par(Store(d:reg1, Addc(x:reg, y:reg_or_const, Sparccarrybit(cc)), w),
```

```

        Store(ccl, Sparcadccflags(x:reg, y:reg_or_const, Sparccarrybit(cc)), w)
{: sprintf "addxcc %s, %s, %s" x y d :}

inst: Par(Store(d:regl, Subb(x:reg, y:reg_or_const, Sparccarrybit(cc)), w),
        Store(ccl, Sparcsbbflags(x:reg, y:reg_or_const, Sparccarrybit(cc)), w))
{: sprintf "subxcc %s, %s, %s" x y d :}

-- conditional branches
inst: Store(ccl, Sparcsubcc(x:reg, y:reg_or_const), w)
{: sprintf "subcc %s, %s, %g0" x y :}
inst: Store(ccl, Sparcsubcc(x:freg, y:freg), w)
{: sprintf "fcmps %s, %s\n\tnop" x y :}
inst: Store(ccl, Sparcsubcc(x:dreg, y:dreg), w)
{: sprintf "fcmpd %s, %s\n\tnop" x y :}

inst: Guarded(Sparceq(cc), Goto(target))
{: sprintf "be %s\n\tnop" target :}
inst: Guarded(Sparcne(cc), Goto(target))
{: sprintf "bne %s\n\tnop" target :}

inst: Guarded(Sparcge(cc), Goto(target))
{: sprintf "bge %s\n\tnop" target :}
inst: Guarded(Sparcgeu(cc), Goto(target))
{: sprintf "bgeu %s\n\tnop" target :}
inst: Guarded(Sparcgt(cc), Goto(target))
{: sprintf "bg %s\n\tnop" target :}
inst: Guarded(Sparcgtu(cc), Goto(target))
{: sprintf "bgu %s\n\tnop" target :}

inst: Guarded(Sparcle(cc), Goto(target))
{: sprintf "ble %s\n\tnop" target :}
inst: Guarded(Sparcleu(cc), Goto(target))
{: sprintf "bleu %s\n\tnop" target :}
inst: Guarded(Sparclt(cc), Goto(target))
{: sprintf "bl %s\n\tnop" target :}
inst: Guarded(Sparcltu(cc), Goto(target))
{: sprintf "blu %s\n\tnop" target :}

inst: Guarded(Sparcfeq(cc), Goto(target))
{: sprintf "fbe %s\n\tnop" target :}
inst: Guarded(Sparcfne(cc), Goto(target))
{: sprintf "fbne %s\n\tnop" target :}

inst: Guarded(Sparcfge(cc), Goto(target))
{: sprintf "fbge %s\n\tnop" target :}
inst: Guarded(Sparcfgt(cc), Goto(target))
{: sprintf "fbg %s\n\tnop" target :}

inst: Guarded(Sparcfle(cc), Goto(target))
{: sprintf "fble %s\n\tnop" target :}
inst: Guarded(Sparcflt(cc), Goto(target))
{: sprintf "fbl %s\n\tnop" target :}

```

Other Instructions Why is this here?

```

⟨Sparcrec rules 706⟩+≡ (699a) ◁705b 707a▷
inst : Nop() {: "! Why do you think there should be a nop? " :}

```

Diagnostic rules

$\langle \text{Sparcrec rules } 707a \rangle + \equiv$ (699a) $\triangleleft 706$
 (* claude: fetch %y then store it right back into itself - marks %y live/defined without moving it; a true no-op)
 inst: Store(yregl, yreg, w) {: s "! y register self-store (no-op)" :}
 inst: any [1000] {: s "<%s>" any :}

 any : True () {: "True" :}
 any : False () {: "False" :}
 any : Link(symbol, w) {: s "Link(%s,%d)" (symbol#mangled_text) w :}
 any : Late(string, w) {: s "Late(%s,%d)" string w :}
 any : Diff(c1:any,c2:any) {: s "Diff(%s,%s)" c1 c2 :}
 any : Bits(bits) {: sprintf "Bits(%s)" (Bits.to_string bits) :}

 any : Fetch (any, w) {: s "Fetch(%s,%d)" any w :}

 any : And(x:any, y:any) {: s "And(%s, %s)" x y :}
 any : Or(x:any, y:any) {: s "Or(%s, %s)" x y :}
 any : Xor(x:any, y:any) {: s "Xor(%s, %s)" x y :}
 any : Com(x:any) {: s "Com(%s)" x :}

 any : Add(x:any, y:any) {: s "Add(%s, %s)" x y :}
 any : Mul(x:any, y:any) {: s "Mul(%s, %s)" x y :}
 any : Fadd(x:any, y:any) {: sprintf "Fadd(%s, %s)" x y :}
 any : Fsub(x:any, y:any) {: sprintf "Fsub(%s, %s)" x y :}
 any : Fmul(x:any, y:any) {: sprintf "Fmul(%s, %s)" x y :}
 any : Fdiv(x:any, y:any) {: sprintf "Fdiv(%s, %s)" x y :}

 any : Fneg(any) {: sprintf "Fneg(%s)" any :}
 any : Fabs(any) {: sprintf "Fabs(%s)" any :}

 any : Loword(any) {: sprintf "Loword(%s)" any :}

 any : BitInsert(x:any, y:any, z:any) {: sprintf "BitInsert(%s, %s, %s)" x y z :}
 any : BitExtract(lsb:any, y:any, n:w) {: sprintf "BitExtract(%s, %s, %d)" lsb y n :}
 any : Slice(n:w, lsb:n, y:any) {: sprintf "Slice(%d, %d, %s)" n lsb y :}

 any : Sx(any) {: s "Sx(%s)" any :}
 any : Zx(any) {: s "Zx(%s)" any :}
 any : F2f(n:w, w, any) {: sprintf "F2f(%d, %d, %s)" n w any :}
 any : F2i(n:w, w, any) {: sprintf "F2i(%d, %d, %s)" n w any :}
 any : I2f(n:w, w, any) {: sprintf "I2f(%d, %d, %s)" n w any :}
 any : Lobits(any, w) {: s "Lobits(%s, %d)" any w :}
 any : Mem(any, w) {: s "Mem(%s)" any :}
 any : Reg (char, n:int) {: sprintf "Reg(%s, %d)" (Char.escaped char) n :}
 any : RegPair(char, n:int) {: sprintf "RegPair(%s, %d)" (Char.escaped char) n :}
 any : Store (dst:any, src:any, w)
 {: s "Store(%s,%s,%d)" dst src w :}
 any : Kill(any) {: s "Kill(%s)" any :}

 any : Guarded(guard:any, any) {: s "Guarded(%s,%s)" guard any :}
 any : Par(l:any, r:any) {: s "Par(%s,%s)" l r :}
 any : Goto(any) {: s "Goto(%s)" any :}

X.5.4 Interfacing RTLs with the Expander

$\langle \text{Sparcrec special cases for particular operators } 707b \rangle \equiv$ (709b)
 | RP.App(("sub", [w]), [x; y]) -> conSub (exp x) (exp y)
 | RP.App(("subb", [w]), [x; y; z]) -> conSubb (exp x) (exp y) (exp z)
 | RP.App(("add", [w]), [x; y]) -> conAdd (exp x) (exp y)

```

| RP.App(("addc", [w]), [x; y; z]) -> conAddc (exp x) (exp y) (exp z)
| RP.App(("mul", [32]), [x; y]) -> conMul (exp x) (exp y)
| RP.App(("neg", [w]), [x]) -> conNeg (exp x)
| RP.App(("quot", [w]), [x; y]) -> conQuot (exp x) (exp y)
| RP.App(("divu", [w]), [x; y]) -> conDivu (exp x) (exp y)
(* maybe can be implemented with mulx instruction for SPARC 9
| RP.App(("mulx", [32;64]), [x; y]) -> conMulx (exp x) (exp y)
*)

| RP.App(("and", [w]), [x; y]) -> conAnd (exp x) (exp y)
| RP.App(("or", [w]), [x; y]) -> conOr (exp x) (exp y)
| RP.App(("xor", [w]), [x; y]) -> conXor (exp x) (exp y)
| RP.App(("com", [w]), [x]) -> conCom (exp x)

| RP.App(("shl", [w]), [x; y]) -> conShl (exp x) (exp y)
| RP.App(("shrl", [w]), [x; y]) -> conShrl (exp x) (exp y)
| RP.App(("shra", [w]), [x; y]) -> conShra (exp x) (exp y)

| RP.App(("lobits", [32;8]), [x]) -> conLobyte (exp x)
| RP.App(("lobits", [32;16]), [x]) -> conLohalf (exp x)
| RP.App(("lobits", [64;32]), [x]) -> conLoword (exp x)
| RP.App(("zx", [8; 32]), [x]) -> conZxbyte (exp x)
| RP.App(("zx", [16;32]), [x]) -> conZxhalf (exp x)
| RP.App(("sx", [8; 32]), [x]) -> conSxbyte (exp x)
| RP.App(("sx", [16;32]), [x]) -> conSxhalf (exp x)

| RP.App(("sparc_subcc", [w]), [x; y]) -> conSparcsubcc (exp x) (exp y)
| RP.App(("sparc_addcc", [w]), [x; y]) -> conSparcaddcc (exp x) (exp y)
| RP.App(("sparc_mulx_hi", [w]), [x; y]) -> conSparcmulxhi (exp x) (exp y)
| RP.App(("sparc_mlux_hi", [w]), [x; y]) -> conSparcmuluxhi (exp x) (exp y)
| RP.App(("sparc_adcflags", [w]), [x; y; z]) -> conSparcadcflags (exp x) (exp y) (exp z)
| RP.App(("sparc_sbbflags", [w]), [x; y; z]) -> conSparcsbbflags (exp x) (exp y) (exp z)
| RP.App(("sparc_carrybit", _), [x]) -> conSparccarrybit (exp x)

| RP.App(("sparc_eq", [w]), [x]) -> conSparceq (exp x)
| RP.App(("sparc_ne", [w]), [x]) -> conSparcne (exp x)
| RP.App(("sparc_ge", [w]), [x]) -> conSparcge (exp x)
| RP.App(("sparc_geu", [w]), [x]) -> conSparcgeu (exp x)
| RP.App(("sparc_gt", [w]), [x]) -> conSparcgt (exp x)
| RP.App(("sparc_gtu", [w]), [x]) -> conSparcgtu (exp x)
| RP.App(("sparc_le", [w]), [x]) -> conSparcle (exp x)
| RP.App(("sparc_leu", [w]), [x]) -> conSparcleu (exp x)
| RP.App(("sparc_lt", [w]), [x]) -> conSparclt (exp x)
| RP.App(("sparc_ltu", [w]), [x]) -> conSparcltu (exp x)

| RP.App(("sparc_feq", [w]), [x]) -> conSparcfeq (exp x)
| RP.App(("sparc_fne", [w]), [x]) -> conSparcfne (exp x)
| RP.App(("sparc_fge", [w]), [x]) -> conSparcfge (exp x)
| RP.App(("sparc_fgt", [w]), [x]) -> conSparcfgt (exp x)
| RP.App(("sparc_fle", [w]), [x]) -> conSparcfle (exp x)
| RP.App(("sparc_flt", [w]), [x]) -> conSparcflt (exp x)

| RP.App(("fdiv", [w]), [x; y; rm]) -> conFdiv (exp x) (exp y)
| RP.App(("fmul", [w]), [x; y; rm]) -> conFmul (exp x) (exp y)
| RP.App(("fadd", [w]), [x; y; rm]) -> conFadd (exp x) (exp y)
| RP.App(("fsub", [w]), [x; y; rm]) -> conFsub (exp x) (exp y)
| RP.App(("fneg", [w]), [x]) -> conFneg (exp x)

| RP.App(("i2f", [w;w']), [x; rm]) -> conItof (exp x)
| RP.App(("f2i", [w;w']), [x; rm]) -> conFtoi (exp x)

```

```

(* MISSING ASSERTION: %f2i ALWAYS ROUNDS TO ZERO *)
| RP.App(("f2f", [w;w']), [x; rm]) -> conFtof (exp x)

| RP.App(("add"|"sub"|"mul"|"sx"|"zx"|"lobits"|"bitInsert"|"
  "bitExtract"|"fabs"|"fneg"|"fdiv"|"fmul"|"fsub"|"fadd"|"f2f"|"f2i"|"
  "i2f"|"and"|"or"|"xor"|"com") as op, ws), xs)->
Impossible.impossible
(Printf.sprintf
  "operator %%s specialized to [%s] & applied to %d arguments"
  op (String.concat "; " (List.map string_of_int ws)) (List.length xs))

```

And now we convert between RTLs and Burg constructors.

<Sparcrec code to follow the labeler 709a>≡ (699a) 709b▷

```

let unimp = Impossible.unimp
let const = function
| RP.Late(s,w)          -> unimp "sparc: late constants"
| RP.Bool(b)           -> unimp "sparc: bool"
| RP.Link(s,_,w)       -> conLink s w
| RP.Diff _            -> error "PIC not supported"
| RP.Bits(b)           -> conBits(b)

```

<Sparcrec code to follow the labeler 709b>+≡ (699a) <709a 709c>

```

let rec exp = function
| RP.Const(k)          -> const (k)
| RP.Fetch(l,w)        -> conFetch (loc l) w
<Sparcrec special cases for particular operators 707b>
| RP.App((o,_),_)      -> error ("unknown operator " ^ o)

```

<Sparcrec code to follow the labeler 709c>+≡ (699a) <709b 709d>

```

and loc l = match l with
| RP.Mem(('m', aff, _), w, e, ass) -> conMem (exp e) w
| RP.Reg((sp, _, _), i, R.C 1)     -> conReg    sp i
| RP.Reg((sp, _, _), i, R.C 2)     -> conRegPair sp i
| RP.Reg _                         -> unimp "quad registers and other large beasts"
| RP.Mem(_, _, _, _)              -> error "non-mem, non-reg cell"
| RP.Var _ | RP.Global _          -> error "var found"
| RP.Slice(w,i,l)                 -> unimp "sparc: slice locations"

```

We recognize some special forms of single effects. When is npc used and when is pc used?

How to handle returns:

ret is really jmpl retl is really jmpl

this is a bit of a problem for our code because the machinery has no idea that the return address is being placed in different registers maybe we should tell it that non-volatile register, so that it won't try to save and restore the value on the stack, which is pointless. Should be okay as long as is mentioned as volatile and isn't available to pass a parameter on a call. For now, we will just be using the “ret” instruction because we don't do the so-called “leaf optimization”.

<Sparcrec code to follow the labeler 709d>+≡ (699a) <709c 710a>

```

and effect = function
| RP.Store(RP.Reg(('c',_,_), i, c), r, _)
  when (i = 1 (* i = npc *)) -> conGoto (exp r)
(*)
| RP.Store(RP.Reg('c',i, _), r, w) -> error (sprintf "set $c[%d]" i)
*)
| RP.Store(maybe_spl, RP.App(("add",_), [x;y]), _)
  when (RU.Eq.loc maybe_spl spl && (RU.Eq.exp x sp || RU.Eq.exp y sp)) ->
    conSave (exp x) (exp y)
| RP.Store(l,e,w)          -> conStore (loc l) (exp e) w
| RP.Kill(l)               -> unimp "sparc: kill effect"

```

We attempt to recognize register pairs.

```

(Sparcrec code to follow the labeler 710a)+≡ (699a) ◁709d 710b▷
and regpair = function
  | _ -> Impossible.impossible "Argument is not a register pair"

(Sparcrec code to follow the labeler 710b)+≡ (699a) ◁710a 710c▷
and rtl (RP.Rtl es) = geffects es
and geffects = function
  | [] -> conNop ()
  | [g, s] -> guarded g s
  | (g, s) :: t -> conPar (guarded g s) (geffects t)
and guarded g eff = match g with
  | RP.Const(RP.Bool b) -> if b then effect eff else conNop()
  | _ -> conGuarded (exp g) (effect eff)

```

X.5.5 The exported recognizer

We try not to immediately halt if something goes wrong but instead drop error messages into the assembly language.

```

(Sparcrec code to follow the labeler 710c)+≡ (699a) ◁710b
let errmsg r msg =
  List.iter prerr_string
    [ "recognizer error: "; msg; " on "; RU.ToString.rtl r; "\n" ]

let to_asm r =
  try
    let plan = rtl (Down.rtl (Simplify.rtl r)) in
    plan.inst.Camlburg.action ()
  with
  | Camlborg.Uncovered -> " not an instruction: " ^ RU.ToString.rtl r
  | Error msg -> (errmsg r msg; " error in recognizer: " ^ msg)

let is_instruction r =
  try
    let plan = rtl (Down.rtl (Simplify.rtl r)) in
    plan.inst.Camlburg.cost < 100 (* should be true, but shade this... *)
  with
  | Camlborg.Uncovered -> false
  | Error msg -> (errmsg r msg; false)

```

X.6 SPARC spaces and registers

This module exports central information used by several parts of the SPARC back end.

```

(sparcregs.mli 710d)≡
module Spaces : sig
  val m : Space.t (* memory *)
  val r : Space.t (* integer regs *)
  val f : Space.t (* floating regs *)
  val k : Space.t (* register-window hardware *)
  val c : Space.t (* standard special registers *)

  val t : Space.t (* 32-bit integer temps *)
  val u : Space.t (* 32-bit floating temps *)
  val q : Space.t (* 64-bit floating temps *)

  val d : Space.t (* bogosity for rounding mode -- THIS MUST GO *)

```

```

end

val pc : Rtl.loc
val npc : Rtl.loc
val cc : Rtl.loc

val cwp : Register.t (* current window pointer *)
val y : Register.t (* y register, for multiply *)

val fpctl : Register.t
val fpround : Register.x

⟨sparcregs.ml 711a⟩≡
  ⟨sparcregs.ml 711b⟩

⟨sparcregs.ml 711b⟩≡ (711a) 712e▷
  open Nopoly

module R = Rtl
module S = Space
module SS = Space.Standard32

module Spaces = struct
  let byteorder = R.BigEndian
  ⟨SPARC spaces 711c⟩
end

```

m is a big-endian memory space with byte, halfword, word, and doubleword accesses.

```

⟨SPARC spaces 711c⟩≡ (711b) 711d▷
  let m = SS.m byteorder [8; 16; 32]

```

r is the general purpose integer register file with 32-bit registers. Some of these may be managed specially by register windows. Nyarggg. We pretend that it's actually split between 32 and 64 bit registers.

```

⟨SPARC spaces 711d⟩+≡ (711b) ◁711c 711e▷
  let r = SS.r 32 byteorder [32; 64]

```

f is the general purpose floating-point register file with 32 32-bit registers. Unlike the *r* space, these are all globals. (We have lies and more lies!!!)

```

⟨SPARC spaces 711e⟩+≡ (711b) ◁711d 711f▷
  let f = S.checked { S.space      = ('f', Rtl.BigEndian, Cell.of_size 32)
                    ; S.doc        = "floating-point registers"
                    ; S.indexwidth = 32
                    ; S.indexlimit = Some 8 (* something strange here *)
                    ; S.widths     = [32; 64]
                    ; S.classification = S.Reg
                    }

```

t is the temporary space for the *r* space.

```

⟨SPARC spaces 711f⟩+≡ (711b) ◁711e 711g▷
  let t = SS.t byteorder 32

```

u is the temporary space for the *f* space.

```

⟨SPARC spaces 711g⟩+≡ (711b) ◁711f 712a▷
  let u = S.checked { S.space      = ('u',
                                     Rtl.BigEndian,
                                     Cell.of_size 32)
                    ; S.doc        = "floating-point temporaries"
                    ; S.indexwidth = 32
                    ; S.indexlimit = None
                    ; S.widths     = [32]
                    ; S.classification =

```



```

        S.Temp { S.stands_for = S.stands_for 'f' Rtl.BigEndian 32
                  (* lies about index? *)
                  ; S.set_doc   = "floating-point temporaries"
                }
      }

let q = S.checked { S.space      = ('q', Rtl.Identity, Cell.of_size 64)
                    ; S.doc       = "64-bit floating-point temporaries"
                    ; S.indexwidth = 32
                    ; S.indexlimit = None
                    ; S.widths     = [64]
                    ; S.classification =
                      S.Temp { S.stands_for =
                                (fun ((c, _, _), i, R.C n) ->
                                  i land 0x1 = 0 && c <= 'f' && n = 2)
                                ; S.set_doc   = "64-bit floating-point temporaries"
                              }
                  }

```

c is the space for condition codes and the program counter.

<SPARC spaces 712a>+≡ (711b) <711g 712b>

```

let c = SS.c 6 Rtl.Identity [32] (* pc, npc, cc, ???, fp_mode, fp_fcmp *)

```

k is the space for register windows. The first cell represents the CWP (current window pointer), which is just a counter that we increment and decrement when saving and restoring windows. The rest of the space will eventually be used to represent saved window registers.

<SPARC spaces 712b>+≡ (711b) <712a 712c>

```

let k = S.checked { Space.space      = ('k', Rtl.Identity, Cell.of_size 32)
                    ; Space.doc       = "register windows"
                    ; Space.indexwidth = 32      (* what is indexwidth? *)
                    ; Space.indexlimit = Some 1
                    ; Space.widths     = [32]
                    ; Space.classification = Space.Fixed
                  }

```

The 'd' space is a piece of utter bogosity. It needs to be eliminated in favor of some truth about the location of the rounding mode.

<SPARC spaces 712c>+≡ (711b) <712b 712d>

```

let d = S.checked { S.space      = ('d', Rtl.Identity, Cell.of_size 2)
                    ; S.doc       = "bogus space for rounding mode"
                    ; S.indexwidth = 32
                    ; S.indexlimit = Some 1
                    ; S.widths     = [2]
                    ; S.classification = Space.Fixed
                  }

```

This is just for the Y register which is used for extended multiply.

<SPARC spaces 712d>+≡ (711b) <712c

```

let y = S.checked { Space.space      = ('y', Rtl.Identity, Cell.of_size 32)
                    ; Space.doc       = "Y register"
                    ; Space.indexwidth = 32
                    ; Space.indexlimit = Some 1
                    ; Space.widths     = [32]
                    ; Space.classification = Space.Fixed
                  }

```

<sparcregs.ml 712e>+≡ (711a) <711b 713>

```

let locations = SS.locations Spaces.c
let pc        = locations.SS.pc
let cc        = locations.SS.cc
let npc       = locations.SS.npc

```

⟨sparcregs.ml 713⟩+≡

```
let cwp      = (Spaces.k.Space.space, 0, R.C 1)
let y        = (Spaces.y.Space.space, 0, R.C 1)
let fpctl    = (Spaces.c.Space.space, 4, R.C 1)
let fpround  = Register.Slice(2, 0, fpctl)
```

(711a) <712e

Appendix Y

arch/alpha

Y.1 Back end for the Alpha

The Alpha is a 64 bit little-endian architecture. For informations about he architecture, we consulted the *True64 UNIX Assembly Language Programmer's Guide*, Version 5.1, published by Compaq Computer, Houston.

```
<alpha.mli 714a>≡
  module Post : Postexpander.S
  module X    : Expander.S

  val target: Ast2ir.tgt
  (* claudes: needed by alphabackend.ml's optimizer (Placevar.context
   * Alpha.placevars) - same export sparc.mli/ppc.mli have for their own
   * placevars. *)
  val placevars: Ast2ir.proc -> Automaton.t
```

Y.1.1 Name and storage spaces

```
<alpha.ml 714b>≡
  <alpha.ml 714c>

<alpha.ml 714c>≡ (714b) 714d▷
  (* claudes: == (list-of-widths equality) lives in Nopoly, not the
   * default Stdlib polymorphic (=) - same fix sparc.ml/ppc.ml needed. *)
  open Nopoly
  let arch      = "alpha"                (* architecture *)
  let byteorder = Rtl.LittleEndian
  let wordsize  = 64
```

We use the standard storage spaces, including the spaces for PC and condition codes.

```
<alpha.ml 714d>+≡ (714b) <714c 715a▷
  module SS = Space.Standard64
  module A  = Automaton
  module PX = Postexpander
  module DG = Dag
  module R  = Rtl
  module RU = Rtlutil
  module RP = Rtl.Private
  module Up = Rtl.Up
  module Dn = Rtl.Dn
  module SM = Strutil.Map
  module T  = Target

  module Spaces = struct
    let id = Rtl.Identity
```

```

let m = SS.m byteorder [8; 16; 32; 64] (* byte, word, longword, quadword *)
let r = SS.r 32 id [64]
let f = SS.f 32 id [64]
let t = SS.t id 64
let u = SS.u id 64
let c = SS.c 6 id [64] (* pc, npc, cc, _, fp_mode, fp_fcmp *)
end

```

Y.1.2 Registers

```

⟨alpha.ml 715a⟩+≡ (714b) ◁714d 715b▷
let locations = SS.locations Spaces.c
let pc        = locations.SS.pc
let cc        = locations.SS.cc
let npc       = locations.SS.npc
let fp_mode   = locations.SS.fp_mode
let fp_fcmp   = locations.SS.fp_fcmp
let vfp       = Vfp.mk wordsize

let rspace = Spaces.r.Space.space
let reg n   = (rspace,n,R.C 1)
let sp      = reg 30 (* stack pointer *)
let ra      = reg 26 (* return address *)
let zero    = reg 31 (* always zero *)
let gp      = reg 29 (* global pointer *)
let pv      = reg 27 (* procedure value *)

let pv_loc   = R.reg pv
let rm_reg   = (('d', Rtl.Identity, Cell.of_size 2), 0, Rtl.C 1)

```

Y.1.3 Utilities

```

⟨alpha.ml 715b⟩+≡ (714b) ◁715a 715c▷
let unimp      = Impossible.unimp
let impossible = Impossible.impossible

let (_, _, mcell) as mspace = Spaces.m.Space.space
let fetch_word l           = R.fetch l wordsize
let store_word l e         = R.store l e wordsize
let mem w addr             = R.mem R.none mspace (Cell.to_count mcell w) addr
let reg_width              = Register.width

```

Y.1.4 Control-flow RTLs

We generate standard control-flow RTLs. The `ast2ir.nw.nw` module inserts these into the CFG it builds.

```

⟨alpha.ml 715c⟩+≡ (714b) ◁715b 716a▷
let ra_offset = 4 (* instruction size *)
module F = Mflow.MakeStandard
  (struct
    let pc_lhs = pc
    let pc_rhs = pc
    let ra_reg = R.reg ra
    let ra_offset = ra_offset
  end)
(* claude: needed for T.cc_spec_to_auto's cutto embed/project pair below -
* same role as sparc.ml's fmach (see sparc.ml for the longer version of

```

```

    * this comment). *)
let fmach = F.machine (R.reg sp)

⟨alpha.ml 716a⟩+≡ (714b) <715c 716b>
let return e = R.store pc e wordsize

```

Y.1.5 Postexpander

```

⟨alpha.ml 716b⟩+≡ (714b) <716a 724c>
(* claude: PX.<:>/PX.Rtl don't exist - Nop/Rtl/Test/<:> live in Dag
 * (aliased DG above), not Postexpander; same stale-interface fix already
 * applied to sparcm1/ppcm1. *)
let <:> = DG.<:>
let rtl r = DG.Rtl r
module Post = struct
  ⟨Alpha postexpander 716c⟩
end

```

```

⟨Alpha postexpander 716c⟩≡ (716b) 716d>
let byte_order = byteorder
let wordsize = wordsize
let exchange_alignment = 8

type temp = Register.t
type rtl = Rtl.rtl
type width = Rtl.width
type assertion = Rtl.assertion
type operator = Rtl.Private.opr

```

The postexpander may need to allocate temporaries.

```

⟨Alpha postexpander 716d⟩+≡ (716b) <716c 716e>
let talloc = Postexpander.Alloc.temp

```

Contexts There is no distinction between an integer and an address.

```

⟨Alpha postexpander 716e⟩+≡ (716b) <716d 716f>
let icontext = Context.of_space Spaces.t
let fcontext = Context.of_space Spaces.u
let acontext = icontext
let rcontext = (fun x y -> unimp "Unsupported soft rounding mode"), Register.eq rm_reg

let operators = Context.nonbool icontext fcontext rcontext []
let arg_contexts, result_context = Context.functions operators
let constant_context w = icontext
let itempwidth = 64

```

Addressing modes

```

⟨Alpha postexpander 716f⟩+≡ (716b) <716e 717a>
module Address = struct
  type t = Rtl.exp
  let reg r = R.fetch (R.reg r) (Register.width r)
end
include Postexpander.Nostack(Address)

```

Load and Store

For now we assume every temporary is 64 bits wide. However, the alpha can handle 32 bit floating point values in its 64 bit floating point registers. I believe that one would always spill the 64 bit register.

⟨Alpha postexpander 717a⟩+≡ (716b) <716f 717b>

```
let twidth = reg_width
let tloc t = Rtl.reg t
let tval t = R.fetch (tloc t) (twidth t)

let load ~dst ~addr assn =
  let w = twidth dst in
  assert (w = wordsize);
  rtl (R.store (tloc dst) (R.fetch (mem w addr) w) w)

let store ~addr ~src assn =
  let w = twidth src in
  assert (w = wordsize);
  rtl (R.store (mem w addr) (tval src) w)
```

⟨Alpha postexpander 717b⟩+≡ (716b) <717a 717c>

```
let block_copy ~dst dassn ~src sassn w =
  match w with
  | 64 -> let t = talloc 't' w in load t src sassn <:> store dst t dassn
  | _ -> Impossible.unimp "general block copies on Alpha"
```

The Alpha provides sign- and zero-extending load operations for loading values smaller than `wordsiz`. Values are always extended to 64 bits.

⟨Alpha postexpander 717c⟩+≡ (716b) <717b 717d>

```
let extend op n e = R.app (R.opr op [n; wordsize]) [e]
let lobits n e = R.app (R.opr "lobits" [wordsiz; n]) [e]

let xload op ~dst ~addr n assn =
  let w = twidth dst in
  assert (w = wordsiz);
  assert (Cell.divides mcell n);
  rtl (R.store (tloc dst)
    (extend op n (R.fetch (R.mem assn mspace (Cell.to_count mcell n) addr) n)) w)

let sxload = xload "sx"
let zxload = xload "zx"

let lostore ~addr ~src n assn =
  assert (reg_width src = wordsiz);
  assert (Cell.divides mcell n);
  rtl
    (R.store (R.mem assn mspace (Cell.to_count mcell n) addr) (lobits n (tval src)) n)
```

The general move operation only works between temporaries of the same width. Load immediate loads a constant into a temporary.

⟨Alpha postexpander 717d⟩+≡ (716b) <717c 717e>

```
let move ~dst ~src =
  assert (reg_width src = reg_width dst);
  if Register.eq src dst then DG.Nop
  else rtl (R.store (tloc dst) (tval src) (twidth src))
```

⟨Alpha postexpander 717e⟩+≡ (716b) <717d 718a>

```
let extract ~dst ~lsb ~src = Impossible.unimp "extract"
let aggregate ~dst ~src = Impossible.unimp "aggregate"
```

⟨Alpha postexpander 718a⟩+≡ *(716b) <717e 718b>*

```
let hwset ~dst ~src = Impossible.unimp "setting hardware register"
let hwget ~dst ~src = Impossible.unimp "getting hardware register"
```

Immediate load, and extended immediate load. An extended load-immediate can take sums and differences of compile-time constants (including late compile-time constants).

⟨Alpha postexpander 718b⟩+≡ *(716b) <718a 718c>*

```
let li ~dst const = rtl (R.store (tloc dst) (Up.const const) (twidth dst))
let lix ~dst e      = rtl (R.store (tloc dst) e (twidth dst))
```

Binary and unary operators

None of the operators below expect or return boolean values. This implies, that comparison operators are not expanded here before they are used as guards for branches.

⟨Alpha postexpander 718c⟩+≡ *(716b) <718b 720a>*

```
let unop ~dst op x =
  rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x]) (twidth dst))
```

```
(* claude: Alpha has no integer divide instruction in any generation -
 * a deliberate RISC design choice, not a gap in this backend - so
 * "%quot" can't be a single alpharec.mlb rule the way every other
 * binop here is (see armrec.mlb/etc.'s own single Store(dst,App(op,
 * [x;y])) shape, which is exactly what the generic 'binop' case below
 * still handles for every OTHER operator). Real Alpha toolchains
 * handle this by calling a library routine (__divq, a non-standard
 * ABI: args in $24/$25, result in $27, return address in $23 not
 * $26 - confirmed by disassembling alpha-linux-gnu-gcc's own output)
 * but embedding that call from here would need this postexpander to
 * mark $24/$25/$27/$23 as clobbered mid-block, which nothing in this
 * fork's Dag/Expander machinery supports (RP.Kill reaches every
 * recognizer, including alpharec.mlb's own, as a hard "cannot handle
 * kill" error - it's assumed already consumed upstream, never
 * something a postexpander emits itself). A genuine runtime loop
 * (Dag.While) is also not an option: middle/expand/expander.ml's own
 * block-lowering has "| DG.While (e, b) -> Impossible.unimp "expand
 * loop"" - never implemented for any backend.
 *
```

```
* So this computes unsigned binary long division via DG.If instead,
 * unrolled 64 times at compile time (an "exp Dag.block", the same
 * representation - and the same PX.Expand.block lowering step - that
 * ppc.ml's/x86.ml's own Rewrite.(mod)/popcnt use for their own multi-
 * instruction operator expansions), on the operands' absolute values,
 * with the sign fixed up afterward to match %quot's truncating-
 * toward-zero convention. Each iteration is branchless: the "did this
 * bit make the remainder >= divisor" test is materialized as a 0/1
 * integer via zx(bit(cmp)) (the same structural shape codegen/
 * expander.ml's own boolean-to-value case handles for every backend,
 * confirmed working here) rather than a real conditional store, so
 * the whole 64-step loop is just adds/subs/muls/comparisons - real
 * Alpha instructions throughout, ~4 per iteration. 'neg e = 0 - e'
 * is used for the absolute value instead of a dedicated negate,
 * which is correct even for INT64_MIN: 0 - INT64_MIN wraps (mod
 * 2^64) to INT64_MIN's own bit pattern, which read as UNSIGNED is
 * exactly 2^63 - the correct magnitude, needing no special case. *)
```

```
module 0 = Rewrite.Ops
let quot64 ~dst x y =
  let w = 64 in
  let zero = 0.signed w 0 in
  let xval, yval = tval x, tval y in
```

```

let anloc = tloc (talloc 't' w) in (* shrinking |x|-derived working copy *)
let adloc = tloc (talloc 't' w) in (* |y|, fixed for the whole loop *)
let rloc = tloc (talloc 't' w) in (* running remainder *)
let qloc = tloc (talloc 't' w) in (* running quotient *)
let store loc e = DG.Rtl (R.store loc e w) in
let absto loc e =
  DG.If (DG.cond (0.lt w e zero), store loc (0.sub w zero e), store loc e) in
(* claude: every intermediate gets its OWN fresh temp, written by its
 * OWN instruction, in an order where a location is never READ
 * (even indirectly, via an embedded subexpression) by an
 * instruction SEQUENCED AFTER an earlier instruction in this same
 * step already overwrote it - each 'DG.Rtl' node here is lowered
 * independently, so an old 'nv'/'rv'/etc. reference embedded
 * inside a LATER store's source expression re-reads the location
 * as of THAT point, not as "captured" when the OCaml 'let' ran.
 * First version stored straight back into anloc/rloc/qloc while
 * still referencing their old values from a later store's nested
 * expression - confirmed via a standalone smoke test (7 %quot 7
 * came out 4, not 1) and by reading the emitted alpha .s, which
 * showed anloc's doubling computed twice: once into anloc itself,
 * then again from inside the next store's lowering, reading the
 * now-already-doubled value. *)
let step () =
  let nv, rv, qv, adv =
    R.fetch anloc w, R.fetch rloc w, R.fetch qloc w, R.fetch adloc w in
  let dnloc = tloc (talloc 't' w) in
  let cyloc = tloc (talloc 't' w) in
  let rbloc = tloc (talloc 't' w) in
  let geloc = tloc (talloc 't' w) in
  store dnloc (0.add w nv nv) <:>
  store cyloc (0.zx 1 w (0.bit (0.ltu w (R.fetch dnloc w) nv))) <:>
  store rbloc (0.add w (0.add w rv rv) (R.fetch cyloc w)) <:>
  store geloc (0.zx 1 w (0.bit (0.geu w (R.fetch rbloc w) adv))) <:>
  store rloc (0.sub w (R.fetch rbloc w) (0.mul w adv (R.fetch geloc w))) <:>
  store qloc (0.add w (0.add w qv qv) (R.fetch geloc w)) <:>
  store anloc (R.fetch dnloc w) in
let rec steps n = if n <= 0 then DG.Nop else steps (n-1) <:> step () in
let init =
  absto anloc xval <:> absto adloc yval <:> store rloc zero <:> store qloc zero in
let q, neg_q = R.fetch qloc w, 0.sub w zero (R.fetch qloc w) in
let fixup =
  DG.If (DG.cond (0.lt w xval zero),
    DG.If (DG.cond (0.lt w yval zero), store (tloc dst) q, store (tloc dst) neg_q),
    DG.If (DG.cond (0.lt w yval zero), store (tloc dst) neg_q, store (tloc dst) q)) in
init <:> steps 64 <:> fixup

let binop ~dst op x y = match op with
| "quot", [64] -> PX.Expand.block (quot64 ~dst x y)
| _ -> rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x;tval y]) (twidth dst))

let unrm ~dst op x rm = Impossible.unimp "floating point with rounding mode"
let binrm ~dst op x y rm = Impossible.unimp "floating point with rounding mode"
let dblop ~dsthi ~dstlo op x y = Unsupported.mulx_and_mulux()
let wrdop ~dst op x y z = Unsupported.singlebit ~op:(fst op)
let wrdrop ~dst op x y z = Unsupported.singlebit ~op:(fst op)

```

Control Flow

On the Alpha, we can read and write the program counter.


```

⟨Alpha postexpander 720a⟩+≡ (716b) <718c 720b>
let pc_lhs = pc (* PC as assigned by branch *)
let pc_rhs = pc (* PC as captured by call *)

```

Unconditional Branches

```

⟨Alpha postexpander 720b⟩+≡ (716b) <720a 720c>
let br ~tgt = DG.Nop, R.store pc_lhs (tval tgt) wordsize (* branch reg *)
let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) wordsize (* branch *)

```

Conditional Branches On the Alpha, a conditional branch is not a primitive instruction. Instead, a sequence of instructions implements a comparison, whose result is written to a register. The branch instruction reads the register, interprets it as a boolean, and branches accordingly.

For the results of the comparison instructions we would like to allocate new temporaries. However, the Postexpander interface does not allow for it. THIS COMMENT IS OBSOLETE—WE CAN ALLOCATE TEMPORARIES NOW. We therefore store the result of $x \oplus y$ and $\neg x$ in x .

Not all comparison operators are provided as primitives on the Alpha. We thus have to implement them as an instruction sequence. The result is in all cases stored in x .

`com` complements a temporay bitwise. As above, we could like to assign the result to a new temporary, but cannot allocate one. We thus mutate the argument.

```

⟨Alpha postexpander 720c⟩+≡ (716b) <720b 720d>
(* claude: "bit" is declared width-less ("bit", [] in T.capabilities
 * just below, matching every other backend's own declaration - ppc.ml/
 * sparc.ml/x86.ml/arm.ml/interp.ml all agree) and elab/simplify.ml's
 * own "bit" simplification rule hard-asserts "null w" on every
 * backend, not just this one. Was "R.opr "bit" [wordsize]" - matched
 * alpharec.mlb's own (equally wrong) recognizer pattern, so nothing
 * failed to compile, but any comparison actually reaching
 * elab/simplify.ml's generic constant-folding (not just alpharec.mlb's
 * instruction selection) tripped its "assert (null w)" instead. Fixed
 * together with alpharec.mlb's matching RP.App(("bit", [64]), ...)
 * pattern, now RP.App(("bit", []), ...). *)
let bit = R.opr "bit" []
let com x =
  let o = R.opr "com" [wordsize] in
  rtl (R.store (tloc x) (R.app o [tval x]) wordsize)
(* claude: logical negation of a clean 0/1 comparison result - NOT
 * "com" (bitwise complement): com(1) is -2 (0x...FE), still nonzero,
 * so every "<:> bnot dst" case below (ne/ge/le/geu) was silently
 * wrong whenever the underlying comparison held before this existed
 * (confirmed by hand: %le evaluated true unconditionally, including
 * %le(7,3)). "alpha_bnot" is an internal-only operator (like
 * "alpha_gp" below), not part of T.capabilities, recognized by
 * alpharec.mlb as "xor x, 1, dst". *)
let bnot x =
  let o = R.opr "alpha_bnot" [wordsize] in
  rtl (R.store (tloc x) (R.app o [tval x]) wordsize)

```

`relation` implements a binary relation, where the result is stored in x as a value.

```

⟨Alpha postexpander 720d⟩+≡ (716b) <720c 721a>
(* claude: dst is a FRESH temp (talloc 't'), never x or y - was
 * "R.store (tloc x) ...", storing the comparison result back into
 * one of its own two source operands. Two separate bugs came from
 * that, fixed in two stages: (1) for swapped-operand cases (gt/geu/
 * gtu/le, which reuse lt/ltu/leu with x and y exchanged) the result
 * landed in y's register instead of x's - fixed by passing ~dst:x
 * explicitly; but (2) even with dst pinned to x, reusing x's own

```

```

* register as the destination corrupts x for any LATER comparison
* against the same value - confirmed by hand on
* tests/tiger64/exceptions.c--'s handler-dispatch chain ("handle ex1
* ... handle ex2 ... handle ex3"), which compares the same raised-
* exception-id register against three different constants in
* sequence: cmpeq against the first constant overwrote that register
* with its own 0/1 result, so the second comparison in the chain was
* silently comparing the FIRST comparison's boolean instead of the
* original exception id, matching every subsequent constant as false
* regardless of what was actually raised. x86/ppc's own compare
* instructions set flags/a condition register instead of overwriting
* a source operand, so they never had this problem to begin with -
* it is specific to Alpha's "materialize the boolean into a GPR"
* instruction shape. *)

```

```

let relation op x y =
  let dst = talloc 't' wordsize in
  let o = R.opr op [wordsize] in
  dst, rtl (R.store (tloc dst) (R.app bit [R.app o [tval x;tval y]])) wordsize)

```

cmp implements the RTL comparison operators where the result is stored in the first argument. THE NEED FOR A COMPLEMENT HERE IS SUSPECT. MOST MACHINES MAKE IT POSSIBLE TO DO A COMPARISON IN THE SAME NUMBER OF INSTRUCTIONS REGARDLESS OF THE OPERATOR IN USE.

```

⟨Alpha postexpander 721a⟩+≡ (716b) ◁720d 721b▷
let cmp op x y = match op with
| "eq"      -> relation "eq"  x y
| "ne"      -> let d,s = relation "eq"  x y in d, s <:> bnot d
| "lt"      -> relation "lt"  x y
| "gt"      -> relation "lt"  y x
| "ge"      -> let d,s = relation "lt"  x y in d, s <:> bnot d
(* claude: was missing entirely - alpharec.mlb's own "cmp" set
 * (eq/ge/geu/gt/gtu/le/leu/lt/ltu/ne) already recognizes "le",
 * and there is no "lea" alpha instruction, so any signed <=
 * comparison (e.g. a for-loop bound) hit the "_ -> impossible"
 * case below instead ("bad comparison in expanded Alpha
 * conditional branch"). x<=y is NOT(y<x), same derivation as
 * "ge" just above (x>=y is NOT(x<y)). *)
| "le"      -> let d,s = relation "lt"  y x in d, s <:> bnot d
| "ltu"     -> relation "ltu" x y
| "leu"     -> relation "leu" x y
| "gtu"     -> relation "ltu" y x
| "geu"     -> relation "leu" y x
| "feq"     -
| "fne"     -
| "flt"     -
| "fle"     -
| "fgt"     -
| "fge"     -
| "forordered"
| "funordered" -> unimp "floating-point comparison"
| _         -> impossible
              "bad comparison in expanded Alpha conditional branch"

```

A conditional branch is a sequence of relational operators, followed by a cbranch on the result. COULD THE SENSE OF COMPARISON BE REVERSED HERE? OR PERHAPS THE COMPARISON SHOULD BE RETURNED FROM cmp?

```

⟨Alpha postexpander 721b⟩+≡ (716b) ◁721a 722a▷
(* claude: split from the old single 'bc' into bc_guard/bc_of_guard,
 * the shape Postexpander.S now requires (see sparc.ml/ppc.ml's own

```

```

* bc_guard/bc_of_guard for the same split, and alpharec.mlb's
* "cmp_zero: Cmp(op, reg, zero)" rule for why a register-vs-zero
* comparison is the only branch shape Alpha instructions recognize:
* there is no direct register-vs-register conditional branch in the
* ISA, only "b<cond> $reg, target" testing a register against zero.
* So a general 'x op y' compare has to happen in two steps: first
* 'cmp' computes the boolean into a fresh temp (one of the special
* App cases alpharec.mlb's exp function recognizes - eq/ne/lt/.../
* geu), then a second, separate zero-comparison of that result
* becomes the actual branch guard. claude: was "eq"-to-zero ("guard
* holds when cmp's result is 0, i.e. the comparison was FALSE") -
* the surrounding comment already flagged this exact line as
* unverified ("not re-derived from scratch... until a conditional
* actually runs"), and it was backwards: bc_of_guard's ~ifso/~ifnot
* convention takes the guard-true branch to ifso, so a guard that
* holds on FALSE sent every %gt/%le/%ge/%ne (the branch senses this
* exercises) to the wrong side - confirmed by hand (%gt(7,3)
* evaluated as 0, %gt(3,7) as -1, exactly inverted). "ne"-to-zero
* (guard holds when the comparison was TRUE) is correct. *)
let bc_guard x (opr, ws as op) y =
  assert (ws == [wordsize]);
  let dst, setup = cmp opr x y in
  setup, R.app (R.opr "ne" [wordsize]) [tval dst; R.bits (Bits.zero 64) 64]
let bc_of_guard (setup, guard) ~ifso ~ifnot =
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt wordsize) in
  DG.Test (setup, (brtl guard, ifso, ifnot))

```

bnegate inverts the condition in a conditional branch. Since we only compare for equality or inequality, this is easy. All the real work is done in the relational operators preceding the branch.

(Alpha postexpander 722a) $\equiv$ (716b) $\triangleleft$ 721b 723 $\triangleright$

```

let bnegate r =
  let zero = R.bits (Bits.zero 64) 64 in
  let negate = function
    | "ne" -> "eq"
    | "eq" -> "ne"
    | _ -> impossible "ill-formed Alpha conditional branch" in
  match Dn.rtl r with
  | RP.Rtl [ RP.App( ("eq"|"ne" as op), [64])
            , [RP.Fetch(RP.Reg(x), 64); RP.Const(RP.Bits(b))]]
    )
    , RP.Store (pc, tgt, 64)
  ] when RU.Eq.loc pc (Dn.loc pc_lhs) && Bits.is_zero b ->
    R.guard (R.app (R.opr (negate op) [64]) [tval x; zero])
    (R.store pc_lhs (Up.exp tgt) wordsize)
  | _ -> Impossible.impossible "ill-formed Alpha conditional branch"

```

Calls Calls and returns must follow a protocol to allow a procedure to maintain its global pointer: on entry, register \$27 contains the procedure's address, on exit \$26 contains the return address. Both registers are used to load the global pointer on entry and after a return:

(example for calling convention 722b) $\equiv$

```

...
lda $27, foo
jsr $26, ($27)      /* ra in $26, foo in $27 */
ldgp $gp, 0($26)    /* establish gp; $pc = $26 */

foo:
ldgp $gp, 0($27)    /* $27 = foo */

```

```
...
ret ($26)
```

The call and return instructions in general do not use the conventional registers. Here we expand them such that they do. We also add the `ldgp` instruction after a `jsr`. THIS IS PLAYING FAST AND LOOSE WITH THE COMPILER'S INTERNAL INVARIANTS. YOU'RE LYING TO THE COMPILER ABOUT THE CONTROL FLOW... The `ldgp` instruction at the beginning of a procedure is added by the assembler module `alphaasm.nw`. The calling convention ensures that the return address is in register \$26 and thus the protocol is followed.

The semantics of `ldgp` are captured by the target-specific operator `alpha_gp`; it takes as an argument the current PC and returns the global pointer. `ldgp_ra` assumes that the current PC is in register \$26. This is the case after a return, when the convention explained above was followed.

```
<Alpha postexpander 723>+≡ (716b) <722a 724a>
let alpha_gp = R.opr "alpha_gp" [] (* takes one argument *)
let ldgp_ra = R.store (R.reg gp)
                    (R.app alpha_gp [fetch_word (R.reg ra)]) wordsize

(* claude: original do_call/call/callr took an extra ~others param
 * (dropped: Postexpander.S's call/callr are just ~tgt now, see
 * sparc.ml's identical fix) and packed the WHOLE call sequence -
 * including the actual pc jump - into the "block" (pre-branch
 * effects) half of the return pair, leaving the "branch" (terminal
 * Rtl.rtl) half holding ldgp_ra alone: a gp reload, not a jump. That
 * can't be right structurally - Dag.branch = block * Rtl.rtl, and
 * only that second Rtl.rtl component ever reaches the recognizer as
 * the node's terminal instruction (see Dag.mli), so the call would
 * never actually transfer control, only ever reload gp. Also, per
 * the DEC Alpha ABI, 'pv'/$27 must hold the callee's address for ANY
 * call (direct or indirect) - alphaasm.ml already emits an
 * unconditional "ldgp $gp,0($27)" at the top of every function
 * (see its cfg_instr), which is how a NEW gp gets established once
 * inside the callee. Restructured so the branch half is the actual
 * "Par(Goto(reg=pv), next=store ra)" pattern alpharec.mlb's grammar
 * recognizes for "jsr" (same shape as sparc.ml's call/callr), with
 * loading the target into pv left in the block half as ordinary
 * setup. This drops the old code's ldgp_ra use (there is no slot in
 * the current call/callr signature for "run this after the callee
 * returns" - that belongs in the calling convention's own epilogue,
 * per this function's own original "SHOULD BE PART OF CALLING
 * CONVENTION" comment below, not here) - kept as dead code, unused,
 * since restoring $gp after a call is a real DEC Alpha ABI
 * requirement that alphacc.ml/alphacall.ml will need to pick up
 * later if/when a program actually needs gp again post-call (a
 * single-compilation-unit hello-world does not: every callee
 * reloads its own gp from pv on entry, and there's only one gp value
 * in play). *)
let effects = List.map Up.effect
let do_call tgt =
  rtl (R.store pv_loc tgt wordsize),
  R.par [ R.store pc_lhs (fetch_word pv_loc) wordsize
        ; R.store (R.reg ra) (RU.addk wordsize (R.fetch pc wordsize) ra_offset) wordsize
        ]
  (* ldgp_ra -- FLAGRANT LIE HERE---SHOULD BE PART OF CALLING CONVENTION *)

let call ~tgt = do_call (Up.const tgt)
let callr ~tgt = do_call (tval tgt)
```

Cut-To

```

⟨Alpha postexpander 724a⟩+≡ (716b) <723 724b>
(* claude: adapted from the old effect-list signature to the current
 * Mflow.cut_args record ({new_sp; new_pc}) - same fix as sparc.ml's
 * cut_to, minus sparc's register-window reset since Alpha has none. *)
let cut_to {Mflow.new_sp = sp'; Mflow.new_pc = pc'} =
  DG.Nop, R.par [R.store pc_lhs pc' wordsize; R.store (R.reg sp) sp' wordsize]

```

Sacred instructions

```

⟨Alpha postexpander 724b⟩+≡ (716b) <724a
let don't_touch_me es = false
(* claude: return/forbidden are required by Postexpander.S but had no
 * definition here (same gap sparc.ml had). Alpha has no register
 * windows, so - unlike sparc's return, which also has to bump cwp -
 * this is just "jump to whatever's in ra", the plain Mflow-style
 * default. *)
let return = return (R.fetch (R.reg ra) wordsize)
let forbidden = Rtl.par [] (* BOGUS: NEEDS TO BE A REAL FAULTING INSTRUCTION *)

```

Y.1.6 Expander

```

⟨alpha.ml 724c⟩+≡ (714b) <716b 724d>
module X = Expander.IntFloatAddr(Post)

```

Y.1.7 Spill and reload

The register allocator needs to spill and reload values; we have to provide the instructions.

```

⟨alpha.ml 724d⟩+≡ (714b) <724c 724e>
let spill p t l = [A.store l (Post.tval t) (Post.twidth t)]
let reload p t l =
  let w = Post.twidth t in [R.store (Post.tloc t) (Automaton.fetch l w) w]

```

Y.1.8 Global Variables

When a Global C-- variable names no hardware register to live in, the variable is passed through to following automaton to obtain its location.

THIS AUTOMATON SEEMS QUITE UNIVERSAL FOR 64 BIT ARCHITECTURES. MOVE IT TO Automaton.Standard64?

I am not sure what the best alignments are. Maybe every slot should be 8-byte aligned.

```

⟨alpha.ml 724e⟩+≡ (714b) <724d 725>
let ( *> ) = A.( *> )
let globals base =
  let width w = if w <= 8 then 8
                 else if w <= 16 then 16
                 else if w <= 32 then 32
                 else Auxfuns.round_up_to wordsize w in
  let align = function _ -> 8 in
  A.at ~start:base mspace (A.widen width *> A.align_to align *>
  A.overflow ~growth:Memalloc.Up ~max_alignment:8)

```

Y.1.9 The target record

```
<alpha.ml 725>+≡ (714b) <724e>
let target =
  let spaces = [ Spaces.m
                ; Spaces.r
                ; Spaces.f
                ; Spaces.t
                ; Spaces.u
                ; Spaces.c
                ] in
  (* claude: Ast2ir.tgt = Prest2ir.tgt = T of (...) Target.t - a wrapped
   * variant, not the bare record (see sparcm1/ppcm1's own PA.T{...}). *)
  Prest2ir.T
  { T.name           = "alpha"
  ; T.memspace       = mspace
  ; T.max_unaligned_load = R.C 1
  ; T.byteorder      = byteorder
  ; T.wordsize       = wordsize
  ; T.pointersize    = wordsize
  ; T.alignment      = 8          (* not sure *)
  ; T.memsize        = 8
  ; T.spaces         = spaces
  ; T.reg_ix_map     = T.mk_reg_ix_map spaces
  ; T.distinct_addr_sp = false
  ; T.float          = Float.ieee754

  ; T.vfp            = vfp
  (* claude: T.spill/T.reload/T.bnegate/T.goto/T.jump/T.call/T.return/
   * T.branch aren't Target.t fields anymore (see target.mli) - they now
   * live inside the single T.machine record, built by the
   * Expander.IntFloatAddr functor from Post above, same as sparcm1/
   * ppcm1/x86.m1. *)
  ; T.machine        = X.machine

  ; T.cc_specs       = Alphacc.cc_specs
  ; T.cc_spec_to_auto = Alphacall.cconv ~return_to:return
                        { T.embed      = fmach.T.cutto.T.embed
                        ; T.project    = fmach.T.cutto.T.project }

  ; T.is_instruction = Alpharec.is_instruction
  ; T.tx_ast = (fun secs -> secs)
  (* claude: T.incapable (empty operator/literal/memory lists) would
   * reject every operator this backend actually implements - see
   * alpharec.mlb's grammar (only add/sub/com/bit + the eq/ne/lt/gt/ge/
   * ltu/leu/gtu/geu comparisons are recognized as real instructions;
   * everything else falls through to the "<...>" catch-all/error path)
   * and alpha.ml's own Post.cmp (no float comparisons, no mul/div/and/
   * or/xor/shift - all Impossible.unimp/Unsupported so far). Scoped to
   * exactly that subset, at the one width this target's registers ever
   * have (64) - same rationale as sparcm1's own hand-written operator
   * list, just much shorter since alpharec.mlb recognizes far fewer
   * shapes today. Widen this list (and alpharec.mlb) together as more
   * operators get real camlborg rules. *)
  ; T.capabilities   = { T.operators = List.map Up.opr
                        [ "add",      [64]
                        ; "sub",      [64]
                        ; "mul",      [64]
                        ; "quot",     [64]
                        ; "and",      [64]
```

```

        ; "com",      [64]
        ; "eq",      [64]
        ; "ne",      [64]
        ; "lt",      [64]
        ; "le",      [64]
        ; "gt",      [64]
        ; "ge",      [64]
        ; "ltu",     [64]
        ; "leu",     [64]
        ; "gtu",     [64]
        ; "geu",     [64]
        ; "not",     []
        ; "bool",    []
        ; "disjoin", []
        ; "conjoin", []
        ; "bit",     []
        ; "sx",      [1;64]
        ; "zx",      [1;64]
        ; "lobits",  [64;8]
        ; "lobits",  [64;16]
        ; "lobits",  [64;32]
    ]
    ; T.litops      = []
    ; T.literals    = [64]
    ; T.memory      = [8; 16; 32; 64]
    ; T.block_copy  = true
    ; T.itemps      = [64]
    ; T.ftemps      = []
    ; T.iwiden      = false
    ; T.fwiden      = false
  }
  ; T.globals      = globals
  ; T.rounding_mode = Rtl.reg rm_reg
  ; T.named_locs   = Strutil.Map.empty
  ; T.data_section = "data"
  ; T.charset      = "latin1" (* REMOVE THIS FROM TARGET.T *)
}

```

```

(* claude: Placevar.context's automaton for deciding where a C--
 * "variable" (as opposed to a hardware register) lives - required by
 * alphabackend.ml's optimizer (Placevar.context Alpha.placevars), same
 * role/shape as sparc.ml's/ppc.ml's own placevars, just widened to 64
 * everywhere 32 appeared there (this backend's only integer temp width -
 * see Post.itempwidth above). *)

```

```

let placevars =
  let is_float w kind _ = kind = $= "float" in
  let warn ~width ~alignment ~kind = () in
  let mk_stage ~temps =
    A.choice
    [ is_float,          A.widen (Auxfun.round_up_to ~multiple_of: 64)
    ; (fun w h _ -> w <= 64), A.widen (fun _ -> 64) *> temps 't'
    ; A.is_any,          A.widen (Auxfun.round_up_to ~multiple_of: 8)
    ] in
  Placevar.mk_automaton ~warn ~vfp ~memspace:mSPACE mk_stage

```

Y.2 Alpha Assembler

This assembler emits assembly language for the Alpha platform. It implements the `Asm.assembler` interface.

```
<alphaasm.mli 727a>≡
(* claude: was built on the older Cfgx.M representation (Rtl.rtl
 * Cfgx.M.node); Cfgutil.emit (the only real caller) has since moved to
 * the newer Zipcfg.Rep-based "call" node, same shape arch/sparc/
 * sparcasm.mli/arch/ppc/ppcasm.mli use - see alphaasm.ml's #call/
 * #cfg_instr for the corresponding implementation fix. *)
val make :
  (('a, 'b, 'c, 'd) Procedure.t -> 'cfg -> (Zipcfg.Rep.call -> unit) ->
   (Rtl.rtl -> unit) -> (string -> unit) -> unit) ->
  out_channel -> ('cfg * ('a, 'b, 'c, 'd) Procedure.t) Asm.assembler
(* pass Cfgutil.emit *)
```

Y.2.1 Name Mangling

```
<Alphaasm name mangler specification 727b>≡ (727c)
let spec =
  let reserved = [] in          (* list reserved words here so we can
                                avoid them *)

  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '_'      -> true
    | _        -> false in
  let replace = function
    | x when id x -> x
    | _          -> '_'
  in
  { Mangle.preprocess = (fun x -> x)
  ; Mangle.replace    = replace
  ; Mangle.reserved    = reserved
  ; Mangle.avoid       = (fun x -> x ^ "_")
  }
```

Y.2.2 Implementation

No supprises here. See the sepecification of the interface in `asm.nw` for the semantics of the public methods.

```
<alphaasm.ml 727c>≡
(* claude: was "type node = Rtl.rtl Cfgx.M.node" (the older Cfg
 * representation) plus a #call/#cfg_instr pair built around it -
 * Cfgutil.emit (the only real caller, see alphaasm.mli) has since moved
 * to Zipcfg/Zipcfg.Rep, same as arch/ppc/ppcasm.ml (the closest
 * template: also a from-scratch Linux/ELF assembler, not a Mach-0/
 * windowed one) and arch/sparc/sparcasm.ml already use - see this file's
 * #call/#cfg_instr below for the corresponding fix. *)
module G = Zipcfg
module GR = Zipcfg.Rep
module SM = Strutil.Map

let fprintf = Printf.fprintf
let sprintf = Printf.sprintf
let unimp   = Impossible.unimp
let int64   = Bits.U.to_int64
```


⟨Alphaasm name mangler specification 727b⟩

```
(* claude: 'cfg * (...) Proc.t, not just (...) Proc.t - same fix as
 * ppcasm.ml/sparcasm.ml's class type (Asm.assembler's type param now
 * pairs the Zipcfg.graph alongside the Proc.t record; see #cfg_instr
 * below, which receives that pair). *)
class ['cfg, 'a, 'b, 'c, 'd] asm emitter fd
  : ['cfg * ('a, 'b, 'c, 'd) Procedure.t] Asm.assembler =
object (this)
  val      _fd      = fd
  val      _mangle  = (Mangle.mk spec)
  val mutable _syms  = SM.empty
  val mutable _section = "bogus section"

  method globals _ = ()
  method private new_symbol name =
    let s = Symbol.with_mangler _mangle name in
    _syms <- SM.add name s _syms;
    s

  method private print l = List.iter (output_string _fd) l

  ⟨Alphaasm assembly methods 728⟩
end
let make emitter fd = new asm emitter fd
```

Y.2.3 Public Methods

```
⟨Alphaasm assembly methods 728⟩≡ (727c) 729a▷
method import s = this#new_symbol s
method local  s = this#new_symbol s

method export s =
  let sym = this#new_symbol s in
  Printf.fprintf _fd ".globl %s\n" sym#mangled_text;
  sym

method label (s: Symbol.t) =
  fprintf _fd "%s:\n" s#mangled_text

method section name =
  _section <- name;
  (* claude: pcpmap/pcpmap_data must carry the ALLOC flag or the
   * runtime data lands outside every PT_LOAD segment and
   * Cmm_lookup_entry always reads zeroes - same fix/reasoning as
   * arch/x86/x86asm.ml's, arch/ppc/ppcasm.ml's, and
   * arch/sparc/sparcasm.ml's #section (GNU as only gives default
   * flags to the standard section names). Applied proactively here
   * (not rediscovered via a crash) since it is now a known,
   * recurring pattern across every ELF-targeting backend in this
   * fork. *)
  (match name with
   | "pcmap" | "pcmap_data" ->
     fprintf _fd ".section \"%.s\", \"a\", @progbits\n" name
   | _ ->
     fprintf _fd ":%s\n" name)

method current = _section
```

```

method org n      = unimp "no .org in Alpha assembler"

(* claude: GNU as's .align on Alpha ELF (like PowerPC/MIPS/SPARC) is a
 * power-of-two exponent, not a byte count - same convention
 * ppcasm.ml's #align uses. *)
method align n      =
  let rec lg = function
    | 0 -> 0
    | 1 -> 0
    | n -> 1 + (lg (n/2))
  in
  if n <> 1 then fprintf _fd ".align %d\n" (lg n)
method addloc n      = if n <> 0 then fprintf _fd ".space %d\n" n
method zeroes (n:int) = fprintf _fd ".space %d, 0\n" n

method value (v:Bits.bits) = match Bits.width v with
| 8 -> fprintf _fd ".byte %Ld\n" (int64 v)
| 16 -> fprintf _fd ".word %Ld\n" (int64 v)
| 32 -> fprintf _fd ".long %Ld\n" (int64 v)
| 64 -> fprintf _fd ".quad %Ld\n" (int64 v)
| w -> unimp (sprintf "unsupprted width %d in Alpha assembler" w)

method addr a =
  match Reloc.if_bare a with
  | Some b -> this#value b
  | None -> let const bits = Printf.sprintf "0x%Lx" (Bits.U.to_int64 bits) in
    assert (Reloc.width a = 64);
    fprintf _fd ".quad %s\n" (Asm.reloc_string const a)

method emit = ()

  For alternate returns, we need to know the size of a long jump.
  <Alphaasm assembly methods 729a>+≡ (727c) <728 729b>▷
  method longjmp_size () =
    Impossible.unimp "longjmp size not set for Alpha -- needed for alternate returns"

  <Alphaasm assembly methods 729b>+≡ (727c) <729a
  method comment s = fprintf _fd "/* %s */\n" s

  method const (s: Symbol.t) (b:Bits.bits) =
    fprintf _fd "%s = %Lx" s#mangled_text (int64 b)

  method private instruction rtl =
    output_string _fd (Alpharec.to_string rtl);
    output_string _fd "\n"

  (* claude: adapted to the Zipcfg.Rep.call node shape (cal_i/
   * cal_altrets/cal_contedges) - see arch/ppc/ppcasm.ml's/
   * arch/sparc/sparcasm.ml's own #call, same shape. Alpha has no
   * branch-delay slot (unlike SPARC's "ba ... \n\tnop"), so a longjmp
   * is just a plain unconditional branch. *)
  method private call (node : GR.call) =
    let longjmp edge = fprintf _fd "\tbr %s\n" (_mangle (snd edge.G.node)) in
    let rec output_altret_jumps n edges =
      if n > 0 then
        match edges with
        | edge :: edges -> (longjmp edge; output_altret_jumps (n-1) edges)
        | [] -> Impossible.impossible "contedge count" in
    begin
      this#instruction node.GR.cal_i;

```

```

    output_altret_jumps node.GR.cal_altrets (List.tl node.GR.cal_contedges)
end

(* claudex (cfg, proc) pair + emitter called as "emitter proc cfg ..."
 * - Cfgutil.emit's actual signature (see alphaasm.mli) - not the
 * bare "proc" + "emitter cfg ..." this took before, which is a
 * leftover from the older Cfgx.M-based Asm.assembler shape (see
 * this file's #call above and the class type declaration for the
 * rest of that fix). *)
method cfg_instr (cfg, proc) =
  let symbol = proc.Procedure.symbol in
  let label l = this#label (try SM.find l _syms
                          with Not_found -> this#local l) in

  Printf.fprintf _fd ".set noreorder /* HACK! alphaasm.nw did this */\n";
  Printf.fprintf _fd ".arch ev6 /* HACK! alphaasm.nw did this */\n";
  this#label symbol;
  Printf.fprintf _fd "\tldgp $gp,0($27) /* HACK! alphasm.nw did this */\n";
  (emitter proc cfg (this#call) (this#instruction) label : unit)

```

Y.3 Alpha calling conventions

This module implements calling conventions for the Alpha. The parameters represent the machine instructions to implement return and cut to.

```

<alphacall.mli 730a>≡
val cconv :
  return_to:(Rtl.exp -> Rtl.rtl) ->
  (unit, Mflow.cut_args) Target.map ->
  string -> Automaton.cc_spec ->
  Call.t

```

The implementation is based on *True64 UNIX Assembly Language Programmer's Guide*, Version 5.1, published by Compaq Computer, Houston.

It is not clear how to handle the requirements that on entry, register 27 is live and holds the address of the called procedure, or that on exit, register 26 is live and must hold the return address. It's probably easier to manage register 26.

Y.3.1 Implementation of Alpha calling conventions

```

<alphacall.ml 730b>≡
<alphacall.ml 730c>

<alphacall.ml 730c>≡ (730b) 731a▷
module A = Automaton
module C = Call
module R = Rtl
module RP = Rtl.Private
module RS = Register.Set
module RU = Rtlutil
module T = Target

let impossf fmt = Printf.kprintf Impossible.impossible fmt
let wordsize = 64

```

Registers A non-volatile register can be used in a procedure if its initial value is restored upon exit. Such a register is also called callee-saved. A volatile register can be used without saving and restoring. Registers that are neither volatile nor non-volatile are unavailable for register allocation.

The return address `ra` is volatile. It can be used for register allocation, but the call instruction always writes it (and it is live on entry).

```
<alphacall.ml 731a>+≡ (730b) <730c 731b>
let byteorder = R.LittleEndian
let mspace = ('m', byteorder, Cell.of_size 8)
(* claude: Rtl.Identity, not byteorder - must match alpha.ml's own
 * Spaces.r/Spaces.f (both "SS.r/f 32 id [64]", id = Rtl.Identity), since
 * this file can't import alpha.ml directly (alpha.ml depends on
 * Alphacall.cconv, so the reverse would be circular) and instead
 * hand-rolls its own copy of these space tuples for volregs/pre_nvregs.
 * Getting the aggregation field wrong here doesn't fail to compile - it
 * silently fails Target.fits's stands_for match (front_target/target.ml:
 * "agg == agg'") against alpha.ml's real Spaces.t/Spaces.u temp spaces
 * for EVERY register, since none of cc.Call.volregs/pre_nvregs's
 * registers then match any temp space's expected aggregation - which
 * regalloc/flowra.ml's get_regs_for_space silently turns into an empty
 * candidate-register list for any temp needing allocation, only
 * surfacing as "alloc_one: no registers to spill?" once something
 * actually contends for registers (confirmed: demos/hello_alpha.c--,
 * straight-line with no competing temps, never spills and so never hit
 * this; tests/tiger64/hello.c-- does). riscv64call.ml avoids this whole
 * class of bug by importing Riscv64regs.rspace/fspace directly instead
 * of re-deriving them - not done here since alpha has no equivalent
 * regs-only module to import from without restructuring. *)
let rspace = ('r', Rtl.Identity, Cell.of_size 64)
let fspace = ('f', Rtl.Identity, Cell.of_size 64)

let r n      = (rspace, n, R.C 1)
let f n      = (fspace, n, R.C 1)
let vfp      = Vfp.mk wordsize
```

Calling conventions treat floating point registers specially; therefore we have separate lists for them.

Can we use register \$26, \$27 and \$28, \$29? \$27 is used implicitly for sub-routine calls. If we make the use explicit in our call RTL, we could use it.

```
<alphacall.ml 731b>+≡ (730b) <731a 731c>
let vol_int = List.map r ((Auxfuns.from 0 ~upto:8)@(Auxfuns.from 16 ~upto:26))
let nvl_int = List.map r (Auxfuns.from 9 ~upto:15)
let vol_fp  = List.map f ([0;1] @ (Auxfuns.from 10 ~upto:30))
let nvl_fp  = List.map f (Auxfuns.from 2 ~upto:9)
```

Non-volatile registers are saved somewhere in the frame. Currently, we cannot provide dedicated locations as they are demanded in §6.3.3.5.

```
<alphacall.ml 731c>+≡ (730b) <731b 731d>
let saved_nvr temps =
  let t = Talloc.Multiple.loc temps 't' in
  let u = Talloc.Multiple.loc temps 'u' in
  function
  | (('r', _, _),_,_) as reg -> t (Register.width reg)
  | (('f', _, _),_,_) as reg -> u (Register.width reg)
  | ((s,_,_),i,_) -> impossf "cannot save $%c%d" s i
```

Conventions The size of the Alpha frame must be a multiple of 16. This cannot be specified here.

```
<alphacall.ml 731d>+≡ (730b) <731c 734b>
let ra      = R.reg (r 26) (* return address *)
```

```

let sp      = R.reg (r 30)          (* stack pointer *)
let spval   = R.fetch sp wordsize
let growth  = Memalloc.Down        (* stack grows down *)
let sp_align = 16                   (* SP always 16-byte aligned *)

```

```

let std_sp_location =
  RU.add wordsize vfp (R.late "minus frame size" wordsize)

```

```

let ( *> ) = A.( *> )

```

```

let badwidth (msg:string) (w:int) =
  impossf "unsupported (rounded) width %d in Alpha: %s" w msg

```

```

let fatal _ =
  impossf "fatal error in Alpha automaton"

```

And in Lua:

<Alpha calling convention automata in Lua 732a>≡ 732b▷

```

A      = Automaton
Alpha  = Alpha      or {}
Alpha.cc = Alpha.cc or {}
Alpha.cc["C"] = Alpha.cc["C"] or {}
Alpha.cc["cmm"] = Alpha.cc["cmm"] or {}

```

```

Alpha.sp_align = 16
Alpha.wordsize = 64

```

```

function reg(sp,i)
  return {space = sp, index = i, cellsize = Alpha.wordsize, count = 1}
end
function f(i) return(reg("f", i)) end
function r(i) return(reg("r", i)) end

```

```

Alpha.vol_fp = (f(0) .. f(1)) .. (f(10) .. f(30)) -- includes f0-f1, f10-f30
Alpha.vol_int = (r(0) .. r(8)) .. (r(16) .. r(26)) -- includes r0-f8, r16-f26

```

C return results A C function returns an integer (up to 64 bits wide) in \$0, a floating-point result (up to two 64-bit values) in \$f0 and \$f1.

old comment – THIS IS PROBABLY A BUG! `useregs` CREATES MUTABLE STATE, SO IF A COMPI-
LATION UNIT HAS MULTIPLE CALL SITES THAT RETURN THE SAME HINT, WE SHOULD SEE A
FAILURE HERE.

<Alpha calling convention automata in Lua 732b>+≡ <732a 732c>

```

Alpha.cc["C"].results =
  { A.widen(64, "multiple")
    , A.choice { "float" , A.useregs { f(0), f(1) }
                , A.is_any, A.useregs { r(0) }
    }
  }

```

As an alternative we provide calling convention `cmm` that returns every result in memory. This is for experi-
ments with custom calling conventions.

<Alpha calling convention automata in Lua 732c>+≡ <732b 733a>

```

Alpha.cc["cmm"].results =
  { A.widths { 64 }
    , A.overflow { growth = "up", max_alignment = Alpha.sp_align
    }
  }

```

C procedure parameters Table 6-3 summarizes the calling convention: the location of an argument is determined by its position and type:

Position	Integer	Floating-Point
1	\$16	f16
2	\$17	f17
3	\$18	f18
4	\$19	f19
5	\$20	f20
6	\$21	f21
7	stack	stack
...	...	...

The implementation below uses the *position* of a parameter to determine the register it is passed in. The **counter** stage counts the number of bits requested for allocation; the **regs_by_bits** stages looks at its list of registers, ignores as many registers as are needed to account for bits already allocated and then uses any remaining register to satisfy the request.

Note that the direction of **growth** in the overflow block is independent of the stack growth.

```

<Alpha calling convention automata in Lua 733a>+≡                                <732c 733b>
Alpha.cc["C"].call =
{ A.widen(64, "multiple")
, A.bitcounter("bits")
, A.choice { "float", A.regs_by_bits("bits", f(16) .. f(21))
              , A.is_any, A.regs_by_bits("bits", r(16) .. r(21))
            }
, A.overflow { growth = "up", max_alignment = Alpha.sp_align }
}

Alpha.cc["cmm"].call = Alpha.cc["C"].call

```

C cut-to parameters Since this is strictly an internal calling convention, we can use whatever we like. We use all volatile registers.

```

<Alpha calling convention automata in Lua 733b>+≡                                <733a 735a>
Alpha.cc["C"].cutto =
{ A.widen(64, "multiple")
, A.choice { "float", A.useregs(Alpha.vol_fp)
              , A.is_any, A.useregs(Alpha.vol_int)
            }
, A.overflow { growth = "down", max_alignment = Alpha.sp_align }
}

Alpha.cc["cmm"].cutto = Alpha.cc["C"].cutto

```

Y.3.2 Putting it together

The Alpha demands (§6.3.3.2) that a procedure loads the global pointer value (\$27) into *\$gp*, if the procedure uses large constants. We have to assume that this is always the case. How do we implement this requirement? Update: currently there is no appropriate mechanism. For the Alpha, work is distributed over the code expander, the assembler, and the recognizer.

I don't understand the meaning of **autosp** and **postsp**. On the Alpha, the stack pointer does not move during calls. It is once set in the prolog of a procedure and set back before the procedure returns. I hope the

code below reflects this.

```

⟨Alphacall transformations 734a⟩≡ (734b)
let autoAt = A.at mspace in
let prolog =
  let autosp = (fun _ -> vfp) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in call parms" autoAt spec.A.call)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let epilg =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt spec.A.results)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun _ _ -> vfp) in

let call_actuals =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out call parms" autoAt spec.A.call)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun _ sp -> sp) in

let call_results =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt spec.A.results)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let also_cuts_to =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in cont parms" autoAt spec.A.cutto)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let cut_actuals base =
  C.outgoing ~growth ~sp ~mkauto:(fun () -> autoAt base spec.A.cutto)
  ~autosp:(fun r -> spval)
  ~postsp:(fun _ _ -> spval) in

⟨alphacall.ml 734b⟩+≡ (730b) <731d 735b>
let rtn return_to k n ~ra =
  if k = 0 & n = 0 then return_to ra
  else impossf "alternate return using C calling convention"

let c ~return_to cut spec =
  ⟨Alphacall transformations 734a⟩
  { C.name = "C"
  ; C.overflow_alloc = { C.parameter_deallocator = C.Caller
                        ; C.result_allocator = C.Caller
                        }
  ; C.call_parms = { C.in' = prolog ; C.out = call_actuals }
  ; C.cut_parms = { C.in' = also_cuts_to ; C.out = cut_actuals }
  ; C.results = { C.in' = call_results ; C.out = epilg }

```

```

; C.stack_growth      = growth
; C.stable_sp_loc     = std_sp_location
(* claude: C.cutto (the newpc/newsp embed/project map) isn't a Call.t
 * field anymore - see call.mli, and sparccall.ml's identical drop of
 * "C.cutto = cut" (the 'cut' param is threaded through purely for
 * Alphacall.cconv's/alpha.ml's interface shape now, unused here,
 * same as sparc's). C.jump_tgt_reg is new: a hardware register
 * reserved for indirect jumps, since spilling a temp after the sp
 * has already moved would be unsafe. $28/at is the DEC Alpha ABI's
 * own "assembler temporary" register, set aside for exactly this
 * kind of compiler-internal scratch use - not a0-a5 (args), not
 * s0-s5 (callee-saved), not ra/pv/gp/sp/zero. Mirrors sparccall.ml's
 * r5 and ppc.ml's rreg 7, both similarly-motivated arbitrary-but-
 * unused picks for their own targets. *)
; C.jump_tgt_reg      = R.reg (r 28)
(* claude: Cfgx.Vfp renamed to Vfprewrite - same fix as sparccall.ml. *)
; C.replace_vfp       = Vfprewrite.replace_with ~sp
; C.sp_align          = sp_align
; C.pre_nvregs        = RS.union (RS.of_list nvl_int) (RS.of_list nvl_fp)
; C.volregs           = RS.union (RS.of_list vol_int) (RS.of_list vol_fp)
; C.saved_nvr         = saved_nvr
; C.return            = rtn return_to
; C.ra_on_entry       = (fun _      -> R.fetch ra wordsize)
; C.where_to_save_ra  = (fun _ t    -> Talloc.Multiple.loc t 't' wordsize)
; C.ra_on_exit        = (fun _ _ t -> ra)
; C.sp_on_unwind      = (fun e -> RU.store sp e)
; C.sp_on_jump        = (fun _ _ -> Rtl.null)
}

```

And in Lua we need only register the defined conventions.

```

<Alpha calling convention automata in Lua 735a>+≡ <733b
-- register the new calling conventions!
A.register_cc(Backend.alpha.target, "C"      , Alpha.cc["C"  ])
A.register_cc(Backend.alpha.target, "cmm0", Alpha.cc["C"  ])
A.register_cc(Backend.alpha.target, "cmm1", Alpha.cc["C"  ])
A.register_cc(Backend.alpha.target, "cmm2", Alpha.cc["C"  ])
A.register_cc(Backend.alpha.target, "cmm3", Alpha.cc["cmm"])

```

Test of new calling convention specification Module `callspec.nw` provides a new interface to build a calling convention. The convention returned acts as a template for us; we have to provide some small changes to make it fit. We have to fix the location of the return address on exit. It has to be in \$26. Using `SaveToTemp` also crashes the register allocator.

```

<alphacall.ml 735b>+≡ (730b) <734b 736>
(* claude: Callspec (module CS) isn't wired into this fork's build yet -
 * it still sits, untouched, in TODO/arch/callspec.{ml,mli} (unlike
 * Automaton/Call/etc, already integrated under front_ir/). Nothing in
 * Alphacc.cc_specs uses the "cmm0"/"cmm1"/"cmm2" names cconv below would
 * have dispatched to this Callspec-backed cc/template/cmm0/cmm1/cmm2/
 * cmm3 for - Alphacc.cc_specs only registers "C"/"C--"/"notail"/
 * "C-- thread" (mirrors sparccc.ml), all of which land on the plain 'c'
 * convention above via cconv's own "_ -> c" fallback just below. So this
 * was already dead code before Callspec went missing from the link line
 * - not a functional loss to comment out. Kept here in case Callspec is
 * integrated later and this is worth reviving as-is:
 *
 * module CS = Callspec
 *

```



```

* let template = (* conservative spec *)
*   { CS.name      = "cmm"
*   ; CS.stack_growth = Memalloc.Down
*   ; CS.overflow   = CS.overflow C.Caller C.Caller
*   ; CS.memspace   = mspace
*   ; CS.sp         = r 30
*   ; CS.sp_align   = sp_align
*   ; CS.all_regs    = RS.of_list (List.concat [nvl_int; nvl_fp;
*                                               vol_int; vol_fp])
*   ; CS.nv_regs     = RS.of_list (nvl_int @ nvl_fp)
*   ; CS.save_nvr    = saved_nvr
*   ; CS.ra          = (ra, CS.ReturnAddress.SaveToTemp 't')
*   }
*
* let cc auto return_to cut spec =
*   let t = CS.to_call cut (rtn return_to) auto spec in
*   { t with C.ra_on_exit = (fun _ _ t -> ra)
*   ;   C.sp_on_unwind = (fun e -> RU.store sp e)
*   }
*
* let cmm0 ~return_to cut ccspec = cc ccspec return_to cut
*   { template with CS.name      = "cmm0"
*   ;   CS.overflow = CS.overflow C.Caller C.Caller
*   }
* let cmm1 ~return_to cut ccspec = cc ccspec return_to cut
*   { template with CS.name      = "cmm1"
*   ;   CS.overflow = CS.overflow C.Caller C.Callee
*   }
* let cmm2 ~return_to cut ccspec = cc ccspec return_to cut
*   { template with CS.name      = "cmm2"
*   ;   CS.overflow = CS.overflow C.Callee C.Caller
*   }
* let cmm3 ~return_to cut ccspec = cc ccspec return_to cut
*   { template with CS.name      = "cmm3"
*   ;   CS.overflow = CS.overflow C.Callee C.Callee
*   }
*)

```

We build a `Call.t` value from a specification and do the final fixup for the return address.

Most variations we are interested in can be expressed in the specification. Here are calling conventions that we derive from our template.

And now the lookup function.

```

(αcall.ml 736)+≡ (730b) <735b
let cconv ~return_to cut cname spec =
  let f =
    match cname with
  (*
    | "cmm0" -> cmm0
    | "cmm1" -> cmm1
    | "cmm2" -> cmm2
  *)
  | _ -> c
  in f ~return_to cut spec

```

Y.4 Alpha Recognizer

This module provides a recognizer for Alpha RTLs. The recognizer has two interfaces. The first is a predicate that is true, if an RTL is a MIPS instruction. The second returns an assembly-language string representation

of the instruction.

```
<alpharec.mli 737a>≡  
  val is_instruction: Rtl.rtl -> bool  
  val to_string:      Rtl.rtl -> string
```

It is a checked run-time error to pass an RTL to `to_string` that is not an Alpha instruction.

Y.4.1 Implementation

The recognizer is generated from a BURG specification. The `head` part precedes code generated from `rules`, the `tail` part follows it.

```
<alpharec.mlb 737b>≡  
  %head {:  
    <Alpharec head 737c>  
  :}  
  %tail {:  
    <Alpharec tail 742c>  
  :}  
  %term  
    <Alpharec terminal types 738b>  
  %%  
  <Alpharec rules 738c>
```

The usual abbreviations for modules go into the `head` clause.

```
<Alpharec head 737c>≡ (737b) 737d▷  
  module RP = Rtl.Private  
  module RU = Rtlutil  
  module Up = Rtl.Up  
  module Dn = Rtl.Dn  
  module SS = Space.Standard64
```

We report a fatal error with `error`. We do not expect to recover from such an error; it is something that should not happen and is caused by an internal inconsistency.

```
<Alpharec head 737d>+≡ (737b) <737c 737e>  
  exception Error of string  
  let error msg = raise (Error msg)  
  let sprintf   = Printf.sprintf (* useful for formatting msg *)
```

The `guard` function turns a predicate into a cost function that can be used in a dynamic cost expression.

```
<Alpharec head 737e>+≡ (737b) <737d 738a>  
  let guard p = if p then 0 else Camlburg.inf_cost  
  (* claude: Alpha's ALU instructions (addq/subq/...) only accept an  
   * unsigned 8-bit literal operand (0..255) in their immediate form -  
   * anything else needs a real register, or (for the common "adjust sp  
   * by a small signed constant" case this guards) the "lda" pseudo-op,  
   * which takes a full signed 16-bit displacement instead. Confirmed by  
   * alpha-linux-gnu-as itself: "addq $30, -16, $30" (frame-size setup,  
   * a negative literal) fails to assemble with "operand out of range  
   * (-16 is not between 0 and 255)". See the "addq"/"lda" rule pair  
   * below in %% that this guards between. *)  
  let is_addq_lit b =  
    let n = Bits.S.to_int64 b in  
    Int64.compare n 0L >= 0 && Int64.compare n 255L <= 0
```

Some utilities for building strings.

```

⟨Alpharec head 738a⟩+≡ (737b) <737e
let int64 b =
  assert (Bits.width b = 64);
  Int64.to_string (Bits.U.to_int64 b)

let int32 b =
  assert (Bits.width b = 32);
  Nativeint.to_string (Bits.U.to_native b)

let cat = String.concat ""

let reg n    = "$" ^ string_of_int n
let freg n   = "$f" ^ string_of_int n

let suffix = function
  | 8  -> "b"
  | 16 -> "w"
  | 32 -> "l"
  | 64 -> "q"
  | w  -> error (sprintf "not an Alpha width: %d" w)

(* claude: Alpha's actual mnemonics for the unsigned cases are
 * "cmpult"/"cmpule" (unsigned-prefix, then lt/le), not "cmpltu"/
 * "cmpleu" (qc--'s own RTL operator names, ltu/leu, unsigned-suffix) -
 * a bare "cmp" ^ op concatenation produced an assembler-rejected
 * mnemonic for every unsigned compare until this remap (confirmed:
 * alpha-linux-gnu-as "unknown opcode cmpltu"). eq/lt/le pass through
 * unchanged - they already match. Defined here in %head, not %tail
 * (where "cmp" just below lives) - %% actions can only see %head
 * bindings, confirmed by is_addq_lit's own placement/use above. *)
let alpha_cmp_mnemonic = function
  | "ltu" -> "ult" | "leu" -> "ule" | op -> op

let width w = string_of_int w

```

Y.4.2 Recognizer Rules

Note that the MIPS assembler provides pseudo instructions that translate into multiple machine instructions. If we would emit binary instructions we would have to implement these pseudo instructions here.

We use `x` as an universal terminal type whenever we need one.

```

⟨Alpharec terminal types 738b⟩≡ (737b)
bits n x w

```

```

⟨Alpharec rules 738c⟩≡ (737b) 738d▷
const:    Bits(bits)      {: int64 bits      :}
zero:     Bits(bits)      [{: guard (Bits.eq bits (Bits.zero 64)) :}] {: () :}
four:     Bits(bits)      [{: guard (Bits.eq bits (Bits.S.of_int 4 64)) :}] {: () :}
symbol:   Link(x, w:int)  {: x#mangled_text   :}

```

Registers

```

⟨Alpharec rules 738d⟩+≡ (737b) <738c 739a▷
regl:     Reg('r', n:int) {: reg  n :}
fregl:    Reg('f', n:int) {: freg n :}

pcl:      Reg('c', 0)  {: () :}

```

```

spl:      Reg('r', 30) {: () :}
ral:      Reg('r', 26) {: () :}
pvl:      Reg('r', 27) {: () :}
gpl:      Reg('r', 29) {: () :}

```

```

reg:      Fetch(regl,w:int)  {: regl   :}
freg:     Fetch(fregl,w:int) {: fregl  :}

```

```

pc:      Fetch(pcl,64)      {: () :}
sp:      Fetch(spl,64)      {: () :}
ra:      Fetch(ral,64)      {: () :}
pv:      Fetch(pvl,64)      {: () :}
gp:      Fetch(gpl,64)      {: () :}

```

Addresses

```

⟨Alpharec rules 739a⟩+≡ (737b) <738d 739b>
meml:      Mem(addr)        {: addr :}
mem:       Fetch(meml,w:int) {: meml :}

```

(* claude: alpha-linux-gnu-as requires an explicit displacement digit before the parens - rejects "stq \$26, (\$3

```

addr:      reg              {: sprintf "0(%s)" reg      :}
addr:      imm              {: imm          :}
addr:      Add(imm,reg)     {: sprintf "%s(%s)" imm reg :}
addr:      Add(reg,imm)     {: sprintf "%s(%s)" imm reg :}
addr:      symbol          {: symbol        :}

```

Constant Expressions

```

⟨Alpharec rules 739b⟩+≡ (737b) <739a 739c>
imm:      const            {: const  :}
imm:      symbol           {: symbol :}
imm:      Add(symbol,imm)  {: sprintf "%s+%s" symbol imm :}
imm:      Add(imm,symbol)  {: sprintf "%s+%s" symbol imm :}

```

(* claude: the unsigned-8-bit-literal subset of const usable directly as an addq/subq/etc immediate - see is_ad

```

imm8:      Bits(bits)      [{: guard (is_addq_lit bits) :}] {: int64 bits :}

```

Data Movement Load register from memory. A load of a value smaller than 64 bits implies a zero or sign extension. Todo: add floating-point support.

```

⟨Alpharec rules 739c⟩+≡ (737b) <739b 740a>
inst:      Store(regl,imm,64)
           {: sprintf "lda %s, %s" regl imm :}

```

```

inst:      Store(regl,const,64)
           {: sprintf "ldi %s, %s" regl const :}

```

```

inst:      Store(regl,mem,64)
           {: sprintf "ldq %s, %s" regl mem :}

```

(* claude: was missing - only the store-to-memory direction ("sts", spilling a callee-saved float register) exi

```

inst:      Store(fregl,mem,64)
           {: sprintf "lds %s, %s" fregl mem :}

```

```

inst:      Store(regl, Sx(Fetch(mem,x:int)), w:int)
           {: sprintf "ld%s %s, %s" (suffix w) regl mem :}

inst:      Store(regl, Zx(Fetch(mem,x:int)), w:int)
           {: sprintf "ld%su %s, %s" (suffix w) regl mem :}

```

Write register to memory. No extension here; the 8, 32, or 64 bits are simply written to memory. Floating-point registers use their own load and store instructions.

```

⟨Alpharec rules 740a⟩+≡ (737b) ◁739c 740b▷
inst:      Store(meml, reg, w:int)
           {: sprintf "st%s %s, %s" (suffix w) reg meml :}

inst:      Store(meml, freg, w:int)
           {: sprintf "sts %s, %s" freg meml :}

```

Moves between registers. I cannot find the instructions to move between integer and floating-point registers in the *Assembly Language Programmer's Guide*, but they are describes in the *Alpha Architecture Handbook*, Sections 4.10.18 and 4.10.19 (according to Glenn Holloway). To make them work, we have to emit `.arch ev6`;

```

⟨Alpharec rules 740b⟩+≡ (737b) ◁740a 740c▷
inst:      Store(regl, reg, 64)
           {: sprintf "mov %s, %s" reg regl :}

inst:      Store(fregl, freg, 64)
           {: sprintf "fmov %s, %s" freg fregl :}

inst:      Store(fregl, reg, 64)
           {: sprintf "itofl %s, %s" reg fregl :}

inst:      Store(regl, freg, 64)
           {: sprintf "ftoit %s, %s" freg regl :}

```

Control Flow

```

⟨Alpharec rules 740c⟩+≡ (737b) ◁740b 740d▷
inst:      Goto(symbol)
           {: sprintf "br %s" symbol :}

inst:      Goto(reg)
           {: sprintf "jmp (%s)" reg :}

```

(* claude: was missing - recognizes Post.cut_to's jump-through-register-while-updating-sp RTL (a Tiger "cuts to

```

inst:      Par(Goto(target:reg), Store(spl, nsp:reg, 64))
           {: sprintf "mov %s, $30\n\tjmp (%s)" nsp target :}

```

The next rule covers storing the address of the next instruction in a register.

```

⟨Alpharec rules 740d⟩+≡ (737b) ◁740c 741a▷
next:      Store(regl,Add(pc,four),64)      {: regl :}

inst:      Par(Goto(reg),next)
           {: sprintf "jsr %s,(%s)" next reg :}

inst:      Store(gpl,GP(reg),64)
           {: sprintf "ldgp $gp, (%s)" reg :}

```

Here are conditional branches. The RTL operator names fit the Alpha assembly branch op-codes. We cannot inline the `Cmp` constructor because the `op` terminal symbol would be inaccessible. Only top-level terminals are in scope for the semantic action.

```

⟨Alpharec rules 741a⟩+≡ (737b) <740d 741b>
inst:      Guarded(cmp_zero,Goto(addr))
           {: match cmp_zero with (op,reg) ->
             sprintf "b%s %s, %s" op reg addr
           :}

```

Comparison

```

⟨Alpharec rules 741b⟩+≡ (737b) <741a 741c>
cmp:      Cmp(op:string,x:reg,y:reg)           {: (op, x, y) :}
cmp_zero: Cmp(op:string,reg,zero)              {: (op, reg)  :}

```

(* claude: 3-operand form (dst explicit, not implied by x) - alpha.ml's relation now always passes an explicit

```

inst:      Store(dst:regl,Bit(cmp),64)
           {: match cmp with (op, x, y) ->
             sprintf "cmp%s %s, %s, %s" (alpha_cmp_mnemonic op) x y dst
           :}

```

```

inst:      Store(dst:regl,Com(reg),64)
           {: sprintf "not %s, %s" dst reg :}

```

(* claude: logical negation of a clean 0/1 value (alpha.ml's bnot, used to negate a comparison result) - NOT th

```

inst:      Store(dst:regl,Bnot(reg),64)
           {: sprintf "xor %s, 1, %s" reg dst :}

```

Arithmetic

```

⟨Alpharec rules 741c⟩+≡ (737b) <741b 742a>
inst:      Store(dst:regl, Add(x:reg,y:reg), 64)
           {: sprintf "addq %s, %s, %s" x y dst :}

```

```

inst:      Store(dst:regl, Add(x:reg,y:imm8), 64)
           {: sprintf "addq %s, %s, %s" x y dst :}

```

(* claude: fallback when the immediate is out of addq's 0..255 literal range (e.g. sp+(-16)) or is a symbol - "

```

inst:      Store(dst:regl, Add(x:reg,y:imm), 64) [1]
           {: sprintf "lda %s, %s(%s)" dst y x :}

```

(* claude: "subq" - was entirely missing (only the "any : Sub" debug fallback existed), unlike Add which has fu

```

inst:      Store(dst:regl, Sub(x:reg,y:reg), 64)
           {: sprintf "subq %s, %s, %s" x y dst :}

```

(* claude: "mulq" is register-only, no immediate form - matching Add's own imm8/imm split would need a scratch-

```

inst:      Store(dst:regl, Mul(x:reg,y:reg), 64)
           {: sprintf "mulq %s, %s, %s" x y dst :}

```

(* claude: "and" takes only an 8-bit literal directly (like addq's imm8, not added here - nothing has needed it

```

inst:      Store(dst:regl, And(x:reg,y:reg), 64)
           {: sprintf "and %s, %s, %s" x y dst :}

```

```

inst:      Store(dst:regl, And(x:reg,y:imm), 64) [1]
           {: sprintf "ldiq $28, %s\n\tand %s, $28, %s" y x dst :}

```

Other Instructions

```
<Alpharec rules 742a>+≡ (737b) <741c 742b>
  inst:      Nop() {: "nop" :}
```

Y.4.3 Debugging Support

Uncomment the next rule to get a printout of the tree burg tries to match.

```
<Alpharec rules 742b>+≡ (737b) <742a>
  inst: any [100]      {: cat ["<;any;>"] :}

  any : True  ()      {: cat [ "True" ] :}
  any : False ()      {: cat [ "False" ] :}
  any : Link(x, w:int)  {: cat [ "Link(";x#mangled_text;",";width w;)" ] :}
  any : Late(string,w:int){: cat [ "Late(";string;",";width w;)" ] :}
  any : Bits(bits)     {: cat [ "Bits(b)" ] :}

  any : Fetch (any, w:int){: cat [ "Fetch(";any;",";width w;)" ] :}

  any : Add(x:any, y:any) {: cat [ "Add(";x;",";y;)" ] :}
  any : Sub(x:any, y:any) {: cat [ "Sub(";x;",";y;)" ] :}
  any : Mul(x:any, y:any) {: cat [ "Mul(";x;",";y;)" ] :}
  any : And(x:any, y:any) {: cat [ "And(";x;",";y;)" ] :}
  any : Sx(any)          {: cat [ "Sx(";any;)" ] :}
  any : Zx(any)          {: cat [ "Zx(";any;)" ] :}
  any : Lobits(any)      {: cat [ "Lobits(";any;)" ] :}

  any : Mem(any)         {: cat [ "Mem(";any;)" ] :}
  any : Reg(char, n:int)  {: cat [ "Reg('";Char.escaped char;",";width n;)" ] :}

  any : Store (dst:any, src:any, w:int)
        {: cat [ "Store(";dst;",";src;",";width w;)" ] :}
  any : Kill(any)        {: cat [ "Kill(";any;)" ] :}

  any : Guarded(guard:any, any)
        {: cat [ "Guarded(";guard;",";any;)" ] :}
  any : Cmp(op:string, x:any, y:any)
        {: cat [ "Cmp(";op;",";x;",";y;)" ] :}
  any : GP(any)          {: cat [ "GP(";any;)" ] :}
  any : Com(any)         {: cat [ "Com(";any;)" ] :}
  any : Bnot(any)        {: cat [ "Bnot(";any;)" ] :}
  any : Bit(any)         {: cat [ "Bit(";any;)" ] :}
  any : Par(l:any, r:any) {: cat [ "Par(";l;",";r;)" ] :}
  any : Goto(any)        {: cat [ "Goto(";any;)" ] :}
```

Y.4.4 Interfacing RTLs with the Expander

The principle interface is easy: RTL constructors are mapped to constructor functions of the same name. Because some transformations are difficult to express in BURG, several operators and effects are matched as special cases in OCaml.

```
<Alpharec tail 742c>≡ (737b) 743a>
  let const = function
    | RP.Bool _      -> error "boolean found"
    | RP.Link(s,_,w) -> conLink s w
    | RP.Diff _      -> error "PIC not supported"
    | RP.Bits(b)     -> conBits b
    | RP.Late(s,w)   -> error (sprintf "late constant %s found" s)
```

```

⟨Alpharec tail 743a⟩+≡ (737b) <742c 743b>
  ⟨Alpharec helpers for exp and loc 743e⟩
  let rec exp = function
    | RP.Const(k)          -> const k
    | RP.Fetch(l,w)        -> conFetch (loc l) w
    ⟨Alpharec special cases for App 743f⟩
    | RP.App((o,_),_)      -> error (sprintf "unknown operator %s" o)

  and loc = function
    | RP.Reg((sp,_,_),i,_) -> conReg sp i
    | RP.Mem(('m',_,_),w,e,ass) -> conMem (exp e)
    | RP.Mem((sp,_,_),_,_,_) -> error (sprintf "mem-space space %c" sp)
    | RP.Var (s,i,w)        -> error (sprintf "var %s found" s)
    | RP.Global(s,i,w)      -> error (sprintf "var %s found" s)
    | RP.Slice _            -> error "cannot handle slice"

⟨Alpharec tail 743b⟩+≡ (737b) <743a 743c>
  let effect = function
    ⟨Alpharec special cases for Store 743d⟩
    | RP.Store(l,e,w)      -> conStore (loc l) (exp e) w
    | RP.Kill(l)           -> error "cannot handle kill"

⟨Alpharec tail 743c⟩+≡ (737b) <743b 744>
  let guarded g stmt = match g with
    | RP.Const(RP.Bool b)  -> if b then effect stmt else conNop ()
    ⟨Alpharec special cases for guarded 743g⟩
    | _                    -> conGuarded (exp g) (effect stmt)

  let rec geffects = function
    | []                   -> conNop ()
    | [g, s]               -> guarded g s
    | (g, s) :: t          -> conPar (guarded g s) (geffects t)

  let rtl (RP.Rtl es) = geffects es

```

Y.4.5 Special cases

```

⟨Alpharec special cases for Store 743d⟩≡ (743b)
  | RP.Store(RP.Reg(('c',_,_),i,_,r,_) when i = SS.indices.SS.pc -> conGoto (exp r)

```

```

⟨Alpharec helpers for exp and loc 743e⟩≡ (743a)
  let cmp = Strutil.from_list ["eq";"ge";"geu";"gt";"gtu";"le";"leu";"lt";"ltu";"ne"]

```

```

⟨Alpharec special cases for App 743f⟩≡ (743a)
  | RP.App(("add", [w]), [x; y]) -> conAdd (exp x) (exp y)
  | RP.App(("sub", [w]), [x; y]) -> conSub (exp x) (exp y)
  | RP.App(("mul", [w]), [x; y]) -> conMul (exp x) (exp y)
  | RP.App(("and", [w]), [x; y]) -> conAnd (exp x) (exp y)
  | RP.App((op, [w]), [x; y])
    when Strutil.Set.mem op cmp -> conCmp op (exp x) (exp y)
  | RP.App(("bit", []), [x]) -> conBit (exp x)
  | RP.App(("alpha_gp", []), [x]) -> conGP (exp x)
  | RP.App(("alpha_bnot", [64]), [x]) -> conBnot (exp x)
  | RP.App(("com", [64]), [x]) -> conCom (exp x)
  (* claude: %lobits only ever wraps a value immediately feeding a narrow-width memory store (alpha.ml's Post.los
  | RP.App(("lobits", [_;_]), [x]) -> exp x

```

```

⟨Alpharec special cases for guarded 743g⟩≡ (743c)

```


Y.4.6 The exported recognizers

If an error occurs, we emit the error message to `stderr` and include it in the output. This will lead to errors with the assembler but makes debugging easier because we do not abort after the first problem. Revise this once the expander is more mature.

```
<Alpharec tail 744>+≡ (737b) <743c
let rtl_to_string = RU.ToString.rtl

let dump msg rtl =
  List.iter prerr_string
  [ "error in recognizer: "
    ; msg
    ; " on "
    ; rtl_to_string rtl
    ; "\n"
  ]

let to_string r =
  try
    let plan = rtl (Dn.rtl r) in
    sprintf "\t%s" (plan.inst.Camlburg.action ())
  with
    | Camlburg.Uncovered -> cat ["not an instruction: "
                                ; rtl_to_string r
                              ]
    | Error msg          -> ( dump msg r
                            ; sprintf "error: %s" (rtl_to_string r)
                            )

let is_instruction r =
  try
    let plan = rtl (Dn.rtl r) in
    plan.inst.Camlburg.cost < 100
  with
    | Camlburg.Uncovered -> false
    | Error msg          -> ( dump msg r
                            ; false
                            )
```

Appendix Z

arch/mips

Z.1 arch/mips/mips.nw

```
<arch/mips/mips.ml 745a>≡  
  <mips.ml 745d>
```

```
<arch/mips/mips.mli 745b>≡  
  <mips.mli 745c>
```

Z.2 Back end for the MIPS

```
<mips.mli 745c>≡ (745b)  
  module Post : Postexpander.S  
  module X    : Expander.S  
  
  val target: Ast2ir.tgt  
  val placevars : Ast2ir.proc -> Automaton.t
```

Z.2.1 Name and storage spaces

```
<mips.ml 745d>≡ (745a) 745e▷  
  (* claude: == / ==$= (list-of-widths / string equality) live in Nopoly, not  
   * the default Stdlib polymorphic (=) - same fix sparc.ml/alpha.ml needed. *)  
  open Nopoly  
  let arch      = "mips" (architecture *)  
  let wordsize  = 32
```

```
<mips.ml 745e>+≡ (745a) <745d 746a▷  
  module A = Automaton  
  module PX = Postexpander  
  module DG = Dag  
  module R = Rtl  
  module Rg = Mipsregs  
  module RP = Rtl.Private  
  module RU = Rtlutil  
  module Up = Rtl.Up  
  module Dn = Rtl.Dn  
  module S = Space  
  module SS = Space.Standard32  
  module SM = Strutil.Map  
  module T = Target
```

```

(* claude: PX.<:>/PX.Rtl don't exist - Nop/Rtl/Test/<:> live in Dag
 * (aliased DG above), not Postexpander; same stale-interface fix already
 * applied to sparc.ml/ppc.ml/alpha.ml. *)
let rtl r = DG.Rtl r
let <:> = DG.<:>

```

Z.2.2 Registers

```

<mips.ml 746a>+≡ (745a) <745e 746b>
let vfp          = Vfp.mk wordsize

let dspace = ('d', Rtl.Identity, Cell.of_size 2) (* rounding modes *)
let reg n    = (Rg.rspace,n,Rtl.C 1)
let sp       = reg 29 (* stack pointer *)
let ra       = reg 31 (* return address *)
let r0       = reg 0  (* register 0 *)
let rm_reg   = (dspace, 0, Rtl.C 1)

```

Z.2.3 Utilities

```

<mips.ml 746b>+≡ (745a) <746a 746c>
let unimp          = Impossible.unimp
let impossible     = Impossible.impossible

let fetch_word l   = R.fetch l wordsize
let store_word l e = R.store l e wordsize
let (_, byteorder, mcell) as mspace = Rg.mspace
let mcount = Cell.to_count mcell
let mem w addr     = R.mem R.none mspace (mcount w) addr

```

Z.2.4 Control-flow RTLs

```

<mips.ml 746c>+≡ (745a) <746b 746d>
let ra_offset = 4 (* size of call instruction *)
module F = Mflow.MakeStandard
  (struct
    let pc_lhs = Rg.pc (* should be PC, but recognizer is broken! *)
    let pc_rhs = Rg.pc
    let ra_reg = R.reg ra
    let ra_offset = ra_offset
  end)
(* claude: needed for T.cc_spec_to_auto's cutto embed/project pair below -
 * same role as sparc.ml's/alpha.ml's own fmach. *)
let fmach = F.machine (R.reg sp)

<mips.ml 746d>+≡ (745a) <746c 747a>
(* claude: R.store is loc -> exp -> width -> rtl (curried) - the original
 * was missing the trailing 'wordsize' argument, so 'return' was a
 * 'width -> rtl' function value hiding under a generic inferred type,
 * never applied since nothing referenced it before Post.return needed a
 * plain Rtl.rtl below. *)
let return = R.store Rg.pc (fetch_word (R.reg ra)) wordsize

```

Z.2.5 Postexpander

```
⟨mips.ml 747a⟩+≡ (745a) <746d 750c>
  module Post = struct
    ⟨MIPS postexpander 747b⟩
  end

⟨MIPS postexpander 747b⟩≡ (747a) 747c>
  let byte_order = byteorder
  let exchange_alignment = 4
  let wordsize = wordsize

  type temp = Register.t
  type rtl = Rtl.rtl
  type width = Rtl.width
  type assertion = Rtl.assertion
  type operator = Rtl.Private.opr

⟨MIPS postexpander 747c⟩+≡ (747a) <747b 747d>
  let talloc = Postexpander.Alloc.temp

⟨MIPS postexpander 747d⟩+≡ (747a) <747c 747e>
  let icontext = Context.of_space Rg.Spaces.t
  let fcontext = Context.of_space Rg.Spaces.u
  let acontext = icontext
  let rcontext = (fun x y -> unimp "unsupported soft rounding mode"), Register.eq rm_reg

  let operators = Context.nonbool icontext fcontext rcontext []
  let arg_contexts, result_context = Context.functions operators
  let itempwidth = 32
  let constant_context w = if w = wordsize then icontext else fcontext

⟨MIPS postexpander 747e⟩+≡ (747a) <747d 747f>
  module Address = struct
    type t = Rtl.exp
    let reg r = R.fetch (R.reg r) (Register.width r)
  end
  include Postexpander.Nostack(Address)

⟨MIPS postexpander 747f⟩+≡ (747a) <747e 748a>
  let tloc t = Rtl.reg t
  let tval t = R.fetch (tloc t) (Register.width t)
  let twidth = Register.width

  let load ~dst ~addr assn =
    let w = twidth dst in
    assert (w = wordsize); (* remove when we have 64-bit spaces *)
    rtl (R.store (tloc dst) (R.fetch (mem w addr) w) w)

  let store ~addr ~src assn =
    let w = twidth src in
    assert (w = wordsize); (* remove when we have 64-bit spaces *)
    rtl (R.store (mem w addr) (tval src) w)

  let block_copy ~dst dassn ~src sassn w =
    match w with
    | 32 -> let t = talloc 't' w in load t src sassn <:> store dst t dassn
    | _ -> Impossible.unimp "general block copies on Mips"
```

```

<MIPS postexpander 748a>+≡ (747a) <747f 748b>
let extend op n e = R.app (R.opr op [n; wordsize]) [e]
let lobits n e = R.app (R.opr "lobits" [wordsizes; n]) [e]

let xload op ~dst ~addr n assn =
  let w = twidth dst in
  assert (w = wordsize);
  rtl (R.store (tloc dst)
    (extend op n (R.fetch (R.mem assn mspace (mcount n) addr) n)) w)

let sxload = xload "sx"
let zxload = xload "zx"

let lostore ~addr ~src n assn =
  assert (Register.width src = wordsize);
  rtl (R.store (R.mem assn mspace (mcount n) addr) (lobits n (tval src)) n)

<MIPS postexpander 748b>+≡ (747a) <748a 748c>
let move ~dst ~src =
  assert (Register.width src = Register.width dst);
  if Register.eq src dst then DG.Nop
  else rtl (R.store (tloc dst) (tval src) (twidth src))

<MIPS postexpander 748c>+≡ (747a) <748b 748d>
let extract ~dst ~lsb ~src = Impossible.unimp "extract"
let aggregate ~dst ~src = Impossible.unimp "aggregate"

<MIPS postexpander 748d>+≡ (747a) <748c 748e>
let hwset ~dst ~src = Impossible.unimp "setting hardware register"
let hwget ~dst ~src = Impossible.unimp "getting hardware register"

<MIPS postexpander 748e>+≡ (747a) <748d 748f>
let li ~dst const = rtl (R.store (tloc dst) (Up.const const) (twidth dst))
let lix ~dst e = rtl (R.store (tloc dst) e (twidth dst))

<MIPS postexpander 748f>+≡ (747a) <748e 748g>
let unop ~dst op x =
  rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x]) (twidth dst))

let binop ~dst op x y =
  rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x;tval y]) (twidth dst))

let unrm ~dst op x rm = Impossible.unimp "floating point with rounding mode"
let binrm ~dst op x y rm = Impossible.unimp "floating point with rounding mode"
let dblop ~dsthi ~dstlo op x y = Unsupported.mulx_and_mulux()
let wrdop ~dst op x y z = Unsupported.singlebit ~op:(fst op)
let wrdrop ~dst op x y z = Unsupported.singlebit ~op:(fst op)

<MIPS postexpander 748g>+≡ (747a) <748f 748h>
let pc_lhs = Rg.pc (* PC as assigned by branch *)
let pc_rhs = Rg.pc (* PC as captured by call *)

<MIPS postexpander 748h>+≡ (747a) <748g 749a>
let br ~tgt = DG.Nop, R.store pc_lhs (tval tgt) wordsize (* branch reg *)
let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) wordsize (* branch *)

```

⟨MIPS postexpander 749a⟩+≡

(747a) <748h 749b>

```
let negate = function
| "ne"          -> "eq"
| "eq"          -> "ne"
| "ge"          -> "lt"
| "gt"          -> "le"
| "le"          -> "gt"
| "lt"          -> "ge"
| "geu"         -> "ltu"
| "gtu"         -> "leu"
| "leu"         -> "gtu"
| "ltu"         -> "geu"
| "feq"
| "fne"
| "flt"
| "fle"
| "fgt"
| "fge"
| "fordered"
| "funordered"  -> unimp "floating-point comparison"
| _             -> impossible
                  "bad comparison in expanded MIPS conditional branch"
```

⟨MIPS postexpander 749b⟩+≡

(747a) <749a 749c>

```
(* claude: split from the old single 'bc' into bc_guard/bc_of_guard, the
 * shape Postexpander.S now requires (see sparc.ml/ppc.ml/alpha.ml's own
 * bc_guard/bc_of_guard for the same split). Unlike alpha's ISA (which
 * has no direct register-vs-register conditional branch, only
 * reg-vs-zero), MIPS's "Guarded(cmp,Goto(addr))" grammar rule
 * recognizes a direct two-register comparison as the branch guard
 * itself, so no separate compare-instruction setup is needed - setup
 * stays DG.Nop, same as ppc.ml's own no-condition-register case. *)
let bc_guard x (opr, ws as op) y =
  assert (ws == [wordsize]);
  DG.Nop, R.app (Up.opr op) [tval x; tval y]
let bc_of_guard (setup, guard) ~ifso ~ifnot =
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt wordsize) in
  DG.Test (setup, (brtl guard, ifso, ifnot))
```

⟨MIPS postexpander 749c⟩+≡

(747a) <749b 749d>

```
let bnegate r = match Dn.rtl r with
|          RP.Rtl [ RP.App( op,          [32]), [x; y]], RP.Store (pc, tgt, 32)]
  when RU.Eq.loc pc (Dn.loc pc_lhs) ->
  Up.rtl (RP.Rtl [ RP.App( (negate op,[32]), [x; y]), RP.Store (pc, tgt, 32)])
| _ -> impossible "ill-formed MIPS conditional branch"
```

⟨MIPS postexpander 749d⟩+≡

(747a) <749c 750a>

```
let effects = List.map Up.effect
(* claude: original call/callr took an extra ~others param (dropped:
 * Postexpander.S's call/callr are just ~tgt now, see sparc.ml's/
 * alpha.ml's identical fix) and only ever stored pc_lhs, never storing
 * ra - so a "call" would jump but never leave a usable return address,
 * i.e. it could never actually return (same class of bug alpha.ml's
 * do_call had, though there the branch half held no jump at all rather
 * than no ra-store). mipsrec.mlb's grammar only recognizes a call as
 * "Par(Goto(addr),Store(ral,next,32))" ("jal"), where next=Add(pc,const)
 * is the return address, so both stores must land in the single Par
 * the recognizer sees - hence do_call below, mirroring alpha.ml's
 * restructured do_call/call/callr. No separate setup step is needed
 * (unlike alpha's pv-register indirection): MIPS has no pv/gp
```

```

* machinery in this backend, so the block half stays DG.Nop and the
* whole call sequence lives in the branch half. *)
let do_call tgt =
  DG.Nop,
  R.par [ R.store pc_lhs tgt wordsize
          ; R.store (R.reg ra) (RU.addk wordsize (R.fetch pc_rhs wordsize) ra_offset) wordsize
        ]
let call ~tgt = do_call (Up.const tgt)
let callr ~tgt = do_call (tval tgt)

```

⟨MIPS postexpander 750a⟩+≡ (747a) <749d 750b>

```

(* claude: adapted from the old effect-list signature to the current
* Mflow.cut_args record ({new_sp; new_pc}) - same fix as sparc.ml's/
* alpha.ml's cut_to. *)
let cut_to {Mflow.new_sp = sp'; Mflow.new_pc = pc'} =
  DG.Nop, R.par [R.store pc_lhs pc' wordsize; R.store (R.reg sp) sp' wordsize]

```

⟨MIPS postexpander 750b⟩+≡ (747a) <750a

```

let don't_touch_me es = false
(* claude: return/forbidden are required by Postexpander.S but had no
* definition here (same gap sparc.ml/alpha.ml had). *)
let return = return
let forbidden = Rtl.par [] (* BOGUS: NEEDS TO BE A REAL FAULTING INSTRUCTION *)

```

Z.2.6 Expander

⟨mips.ml 750c⟩+≡ (745a) <747a 750d>

```

module X = Expander.IntFloatAddr(Post)

```

Z.2.7 Spill and reload

⟨mips.ml 750d⟩+≡ (745a) <750c 750e>

```

let spill p t l = [A.store l (Post.tval t) (Post.twidth t)]
let reload p t l =
  let w = Post.twidth t in [R.store (Post.tloc t) (Automaton.fetch l w) w]

```

Z.2.8 Global Variables

⟨mips.ml 750e⟩+≡ (745a) <750d 750f>

```

let ( *> ) = A.( *> )
let globals base =
  let width w = if w <= 8 then 8
                 else if w <= 16 then 16
                 else Auxfun.round_up_to 32 w in
  let align = function 8 -> 1 | 16 -> 2 | _ -> 4 in
  A.at ~start:base mspace
  (A.widen width *> A.align_to align *>
   A.overflow ~growth:Memalloc.Up ~max_alignment:4)

```

⟨mips.ml 750f⟩+≡ (745a) <750e 751>

```

let placevars =
  let is_float w kind _ = w <= 32 && kind = $ = "float" in
  let warn ~width:w ~alignment:a ~kind:k =
    if w > 32 then
      unimp (Printf.sprintf "%d-bit values (because no block copies)" w) in
  let mk_stage ~temps =
    A.choice

```

```

[ is_float,          A.widen (Auxfun.round_up_to ~multiple_of: 32);
  (fun w h _ -> w <= 32), A.widen (fun _ -> 32) *> temps 't';
  A.is_any,          A.widen (Auxfun.round_up_to ~multiple_of: 8);
] in
Placevar.mk_automaton ~warn ~vfp ~memspace:mspace mk_stage

```

Z.2.9 The target record

```

<mips.ml 751>+≡ (745a) <750f
let target =
  let spaces = [ Rg.Spaces.m
                ; Rg.Spaces.r
                ; Rg.Spaces.f
                ; Rg.Spaces.t
                ; Rg.Spaces.u
                ; Rg.Spaces.c
                ] in
  (* claude: Ast2ir.tgt = Preast2ir.tgt = T of (...) Target.t - a wrapped
   * variant, not the bare record (see sparc.ml/ppc.ml/alpha.ml's own
   * PA.T{...}/Preast2ir.T{...}). *)
  Preast2ir.T
  { T.name           = "mips"
  ; T.memspace       = mspace
  ; T.max_unaligned_load = R.C 1
  ; T.byteorder      = byteorder
  ; T.wordsize       = wordsize
  ; T.pointersize    = wordsize
  ; T.alignment      = 4          (* not sure *)
  ; T.memsize        = 8
  ; T.spaces         = spaces
  ; T.reg_ix_map     = T.mk_reg_ix_map spaces
  ; T.distinct_addr_sp = false
  ; T.float          = Float.ieee754

  ; T.vfp            = vfp
  (* claude: T.spill/T.reload/T.bnegate/T.goto/T.jump/T.call/T.return/
   * T.branch aren't Target.t fields anymore (see target.mli) - they now
   * live inside the single T.machine record, built by the
   * Expander.IntFloatAddr functor from Post above, same as sparc.ml/
   * ppc.ml/alpha.ml/x86.ml. *)
  ; T.machine        = X.machine

  ; T.cc_specs       = Mipscc.cc_specs
  ; T.cc_spec_to_auto = Mipsccall.cconv
                        ~return_to:(fun ra -> (R.store Rg.pc ra wordsize))
                        { T.embed    = fmach.T.cutto.T.embed
                          ; T.project = fmach.T.cutto.T.project }

  ; T.is_instruction = Mipsrec.is_instruction
  ; T.tx_ast = (fun secs -> secs)
  (* claude: T.incapable (empty operator/literal/memory lists) would
   * reject every operator this backend actually implements - see
   * mipsrec.mlb's grammar (only add/sub + the eq/ne/lt/gt/ge/ltu/leu/
   * gtu/geu comparisons are recognized as real instructions; everything
   * else falls through to the "<...>" catch-all/error path) and
   * mips.ml's own Post (no mul/div/and/or/xor/shift, no float compute -
   * unrm/binrm/dblop/wrdop/wrdrop are all Impossible.unimp/Unsupported
   * so far). Scoped to exactly that subset, at this target's one integer
   * width (32) - same rationale as alpha.ml's own hand-written operator
   * list. Widen this list (and mipsrec.mlb) together as more operators

```



```

* get real camlburg rules. *)
; T.capabilities = { T.operators = List.map Up.opr
                    [ "add",      [32]
                      ; "sub",      [32]
                      ; "and",      [32]
                      ; "mul",      [32]
                      ; "quot",     [32]
                      ; "eq",       [32]
                      ; "ne",       [32]
                      ; "lt",       [32]
                      ; "gt",       [32]
                      ; "ge",       [32]
                      ; "ltu",      [32]
                      ; "leu",      [32]
                      ; "gtu",      [32]
                      ; "geu",      [32]
                      (* claude: %lobits8/%lobits16 feeding a narrow-width memory store (mips.ml's
                      ; "lobits",   [32;8]
                      ; "lobits",   [32;16]
                      ; "not",      []
                      ; "bool",     []
                      ; "disjoin",  []
                      ; "conjoin",  []
                    ]
                    ; T.litops      = []
                    ; T.literals    = [32]
                    ; T.memory      = [8; 16; 32]
                    ; T.block_copy  = true
                    ; T.itemps      = [32]
                    ; T.ftemps      = []
                    ; T.iwiden      = false
                    ; T.fwiden      = false
                    }
; T.globals      = globals
; T.rounding_mode = Rtl.reg rm_reg
; T.named_locs   = Strutil.assoc2map
                  ["v0", Rtl.reg (reg 2) (* for SPIM *)
                  ]
; T.data_section = "data"
; T.charset      = "latin1" (* REMOVE THIS FROM TARGET.T *)
}

```

Z.3 arch/mips/mipsasm.nw

$\langle \text{arch/mips/mipsasm.ml } 752a \rangle \equiv$
 $\langle \text{mipsasm.ml } 753b \rangle$

$\langle \text{arch/mips/mipsasm.mli } 752b \rangle \equiv$
 $\langle \text{mipsasm.mli } 752c \rangle$

Z.4 MIPS Assembler

$\langle \text{mipsasm.mli } 752c \rangle \equiv$ (752b)
 (* claude: was built on the older Cfgx.M representation (Rtl.rtl
 * Cfgx.M.node); Cfgutil.emit (the only real caller) has since moved to

```

* the newer Zipcfg.Rep-based "call" node, same shape arch/sparc/
* sparcasm.mli/arch/alpha/alphaasm.mli use - see mipsasm.ml's #call/
* #cfg_instr for the corresponding implementation fix. *)
val make :
  (('a, 'b, 'c, 'd) Procedure.t -> 'cfg -> (Zipcfg.Rep.call -> unit) ->
   (Rtl.rtl -> unit) -> (string -> unit) -> unit) ->
  out_channel -> ('cfg * ('a, 'b, 'c, 'd) Procedure.t) Asm.assembler
(* pass Cfgutil.emit *)

```

Z.4.1 Name Mangling

```

⟨name mangler specification 753a⟩≡ (753b)
let spec =
  let reserved = [] in (* list reserved words here so we can
                        avoid them *)

  let id = function
    | 'a'..'z'
    | '0'..'9'
    | 'A'..'Z'
    | '_'      -> true
    | _        -> false in

  let replace = function
    | x when id x -> x
    | _            -> '_'
  in
  { Mangle.preprocess = (fun x -> x)
  ; Mangle.replace     = replace
  ; Mangle.reserved    = reserved
  ; Mangle.avoid       = (fun x -> x ^ "_")
  }

```

Z.4.2 Implementation

```

⟨mipsasm.ml 753b⟩≡ (752a)
(* claude: was "type node = Rtl.rtl Cfgx.M.node" (the older Cfg
 * representation) plus a #call/#cfg_instr pair built around it -
 * Cfgutil.emit (the only real caller, see mipsasm.mli) has since moved
 * to Zipcfg/Zipcfg.Rep, same as arch/ppc/ppcasm.ml/arch/sparc/
 * sparcasm.ml/arch/alpha/alphaasm.ml - see this file's #call/#cfg_instr
 * below for the corresponding fix. *)
module G = Zipcfg
module GR = Zipcfg.Rep
module SM = Strutil.Map

let fprintf = Printf.fprintf
let sprintf = Printf.sprintf
let unimp   = Impossible.unimp
let int64   = Bits.U.to_int64

⟨name mangler specification 753a⟩

(* claude: 'cfg * (...) Proc.t, not just (...) Proc.t - same fix as
 * ppcasm.ml/sparcasm.ml/alphaasm.ml's class type (Asm.assembler's type
 * param now pairs the Zipcfg.graph alongside the Proc.t record; see
 * #cfg_instr below, which receives that pair). *)
class ['cfg, 'a, 'b, 'c, 'd] asm emitter fd
  : ['cfg * ('a, 'b, 'c, 'd) Procedure.t] Asm.assembler =
object (this)

```

```

val      _fd      = fd
val      _mangle   = (Mangle.mk spec)
val mutable _syms   = SM.empty
val mutable _section = "bogus section"

method globals _ = ()
method private new_symbol name =
  let s = Symbol.with_mangler _mangle name in
    _syms <- SM.add name s _syms;
    s

method private print l = List.iter (output_string _fd) l

<assembly methods(mipsasm.nw) 754>
end
let make emitter fd = new asm emitter fd

```

Z.4.3 Public Methods

```

<assembly methods(mipsasm.nw) 754>≡ (753b) 755▷
method import s = this#new_symbol s
method local s = this#new_symbol s

method export s =
  let sym = this#new_symbol s in
    Printf.fprintf _fd ".globl %s\n" sym#mangled_text;
    sym

method label (s: Symbol.t) =
  fprintf _fd "%s:\n" s#mangled_text

method section name =
  _section <- name;
  (* claude: pcmap/pcmap_data must carry the ALLOC flag or the
   * runtime data lands outside every PT_LOAD segment and
   * Cmm_lookup_entry always reads zeroes - same fix/reasoning as
   * arch/x86/x86asm.ml's, arch/ppc/ppcasm.ml's, arch/sparc/
   * sparcasm.ml's, and arch/alpha/alphaasm.ml's #section (GNU as
   * only gives default flags to the standard section names).
   * Applied proactively here (not rediscovered via a crash) since
   * it is now a known, recurring pattern across every ELF-targeting
   * backend in this fork. *)
  (match name with
   | "pcmap" | "pcmap_data" ->
     fprintf _fd ".section \"%.s\", \"a\", @progbits\n" name
   | _ ->
     fprintf _fd ":%s\n" name)

method current = _section
method org n = unimp "no .org in mips assembler"

(* claude: GNU as's .align on MIPS ELF (like PowerPC/SPARC/Alpha) is a
 * power-of-two exponent, not a byte count - same convention
 * ppcasm.ml's/alphaasm.ml's #align uses. *)
method align n =
  let rec lg = function
    | 0 -> 0
    | 1 -> 0
    | n -> 1 + (lg (n/2))
  in

```

```

in
  if n <> 1 then fprintf _fd ".align %d\n" (lg n)
method addloc n      = if n <> 0 then fprintf _fd ".space %d\n"  n
method zeroes (n:int) = fprintf _fd ".space %d, 0\n" n

method value (v:Bits.bits) = match Bits.width v with
  | 8 -> fprintf _fd ".byte %Ld\n"  (int64 v)
  | 16 -> fprintf _fd ".double %Ld\n" (int64 v)
  | 32 -> fprintf _fd ".word %Ld\n"  (int64 v)
  | w -> unimp (sprintf "unsupprrted width %d in mips assembler" w)

method addr a =
  match Reloc.if_bare a with
  | Some b -> this#value b
  | None -> let const bits = Printf.sprintf "0x%Lx" (Bits.U.to_int64 bits) in
    assert (Reloc.width a = 32);
    fprintf _fd ".word %s\n" (Asm.reloc_string const a)

method emit = ()

<assembly methods(mipsasm.nw) 755>+≡ (753b) <754
(* claude: this method used to be defined three times in a row (a plain
 * OCaml error, "this method is defined several times" - not just a
 * shadowing warning) - dropped the two later redefinitions, kept the
 * "#"-style comment since mips-linux-gnu-as (this ELF/Linux toolchain,
 * like ppcasm.ml's own "# %s") uses "#" for line comments. *)
method comment s = fprintf _fd "# %s\n" s

method const (s: Symbol.t) (b:Bits.bits) =
  fprintf _fd "%s = %Lx" s#mangled_text (int64 b)

method longjmp_size () =
  Impossible.unimp "longjmp size not set for mips -- needed for alternate returns"

method private instruction rtl =
  output_string _fd (Mipsrec.to_string rtl);
  output_string _fd "\n"

(* claude: adapted to the Zipcfg.Rep.call node shape (cal_i/
 * cal_altrets/cal_contedges) - see arch/ppc/ppcasm.ml's/arch/sparc/
 * sparcasm.ml's/arch/alpha/alphaasm.ml's own #call, same shape. MIPS
 * has a branch-delay slot (like SPARC, unlike Alpha/PPC), so a
 * longjmp needs an explicit trailing nop, same as sparcasm.ml's
 * "ba ... \n\tnop\n". *)
method private call (node : GR.call) =
  let longjmp edge = fprintf _fd "\tj %s\n\tnop\n" (_mangle (snd edge.G.node)) in
  let rec output_altret_jumps n edges =
    if n > 0 then
      match edges with
      | edge :: edges -> (longjmp edge; output_altret_jumps (n-1) edges)
      | [] -> Impossible.impossible "contedge count" in
  begin
    this#instruction node.GR.cal_i;
    output_altret_jumps node.GR.cal_altrets (List.tl node.GR.cal_contedges)
  end

(* claude: (cfg, proc) pair + emitter called as "emitter proc cfg ..."
 * - Cfgutil.emit's actual signature (see mipsasm.mli) - not the bare
 * "proc" + "emitter cfg ..." this took before, which is a leftover
 * from the older Cfgx.M-based Asm.assembler shape (see this file's

```

```

* #call above and the class type declaration for the rest of that
* fix). *)
method cfg_instr (cfg, proc) =
  let symbol = proc.Procedure.symbol in
  let label l = this#label (try SM.find l _syms
                           with Not_found -> this#local l) in
  Printf.fprintf _fd ".set noreorder /* HACK! mipsasm.nw did this */\n";
  (* claude: replaces ".abicalls"+"cpload $25" (which computed $gp from $25, on the assumption that every ca
  Printf.fprintf _fd ".option pic0\n";
  this#label symbol;
  (emitter proc cfg (this#call) (this#instruction) label : unit)

```

Z.5 arch/mips/mipsCALL.nw

```

⟨arch/mips/mipsCALL.ml 756a⟩≡
  ⟨mipsCALL.ml 756e⟩

⟨arch/mips/mipsCALL.mli 756b⟩≡
  ⟨mipsCALL.mli 756c⟩

```

Z.6 MIPS calling conventions

```

⟨mipsCALL.mli 756c⟩≡ (756b)
  val cconv :
    return_to:(Rtl.exp -> Rtl.rtl) ->
    (unit, Mflow.cut_args) Target.map ->
    string -> Automaton.cc_spec ->
    Call.t

⟨implementation of MIPS calling convention in LCC 756d⟩≡
  static Symbol argreg(int argno, int offset, int ty, int sz, int ty0) {
    assert((offset&3) == 0);
    if (offset > 12)
      return NULL;
    else if (argno == 0 && ty == F)
      return freg2[12];
    else if (argno == 1 && ty == F && ty0 == F)
      return freg2[14];
    else if (argno == 1 && ty == F && sz == 8)
      return d6; /* Pair! */
    else
      return ireg[(offset/4) + 4];
  }

```

Z.6.1 Implementation of MIPS calling conventions

```

⟨mipsCALL.ml 756e⟩≡ (756a) 757a▷
  module A = Automaton
  module C = Call
  module R = Rtl
  module Rg = Mipsregs
  module RP = Rtl.Private
  module RS = Register.Set
  module RU = Rtlutil
  module T = Target

  let impossf fmt = Printf.kprintf Impossible.impossible fmt

```

```

<mipsCALL.ml 757a>+≡ (756a) <756e 757b>
let r n      = (Rg.rspace, n, R.C 1)
let f n      = (Rg.fspace, n, R.C 1)
let vfp      = Vfp.mk 32

```

```

<mipsCALL.ml 757b>+≡ (756a) <757a 757c>
let vol_int  = List.map r (Auxfuns.from 2 ~upto:15 @ [24;25;31])
let nvl_int  = List.map r (Auxfuns.from 16 ~upto:23 @ [30])
(* claude: only the EVEN float registers are independently-named o32
 * registers (odd ones are the upper half of a double-precision pair, not
 * a separate save slot) - the original Auxfuns.from 0/20 ~upto:18/30
 * listed every register, odd included, which mipsel-linux-gnu-gcc's
 * default assembler flags reject outright ("float register should be
 * even") the moment any of these show up in the prologue/epilogue's
 * unconditional non-volatile-register save/restore boilerplate (mipsel-
 * linux-gnu-as run directly only warned; going through gcc's driver,
 * which adds its own default ABI flags, turns the same complaint into a
 * hard error - see demos/Makefile's "mips" target). *)
let evens regs = List.filter (fun n -> n mod 2 = 0) regs
let vol_fp    = List.map f (evens (Auxfuns.from 0 ~upto:18))
let nvl_fp    = List.map f (evens (Auxfuns.from 20 ~upto:30))

```

```

<mipsCALL.ml 757c>+≡ (756a) <757b 757e>
let saved_nvr temps =
  let t = Talloc.Multiple.loc temps 't' in
  let u = Talloc.Multiple.loc temps 'u' in
  function
  | (('r', _, _),_,_) as reg -> t (Register.width reg)
  | (('f', _, _),_,_) as reg -> u (Register.width reg)
  | ((s, _, _), i, _) -> impossf "cannot save $%c%d" s i

```

```

<MIPS calling convention automata in Lua 757d>≡ 758a>
A          = Automaton
Mips       = Mips      or {}
Mips.cc    = Mips.cc   or {}
Mips.cc["C"] = Mips.cc["C"] or {}

Mips.sp_align = 16
Mips.wordsize = 32

function reg(sp,i,agg)
  return Register.create { space = sp, index = i, cellsize = 32, agg=agg }
end
function f(i) return (reg("f", i, 'little')) end
function r(i) return (reg("r", i)) end

Mips.vol_fp = (f(0) .. f(18))
Mips.vol_int = (r(2) .. r(15)) .. { r(24), r(25), r(31) }

```

```

<mipsCALL.ml 757e>+≡ (756a) <757c 758c>
let ra      = R.reg (r 31)      (* return address *)
let sp      = R.reg (r 29)      (* stack pointer *)
let spval   = R.fetch sp 32
let growth  = Memalloc.Down    (* stack grows down *)
let sp_align = 16              (* SP always 16-byte aligned *)

let std_sp_location =
  RU.add 32 vfp (R.late "minus frame size" 32)

let ( *> ) = A.( *> )

```

```

let badwidth (msg:string) (w:int) =
  impossf "unsupported (rounded) width %d in MIPS: %s" w msg

let fatal _ = impossf "fatal error in MIPS automaton"

<MIPS calling convention automata in Lua 758a>+≡ <757d 758b>
  Mips.cc["C"].results =
    { A.widen (32, "multiple")
    , A.widths { 32, 64 }
    , A.choice { "float" , A.userregs(f(0) .. f(3))
                , A.is_any, A.userregs(r(2) .. r(3))
                }
    }

  }

<MIPS calling convention automata in Lua 758b>+≡ <758a 758d>
  -- note that there's some postprocessing magic going on in mipsCALL.nw too
  function Mips.alignf (size)
    if size == 64 then return 8 else return 4 end
  end

  Mips.cc["C"].call =
    { A.widen (32, "multiple")
    , A.widths { 32, 64 }
    , A.align_to(Mips.alignf)
    , A.bitcounter("bits")
    , A.pad("bits")
    , A.argcounter("args")
    , A.first_choice {
        "float" , A.choice {
            { "float", 64 }, A.regs_by_bits("bits", f(12)..f(15))
            , "float", A.regs_by_args("args", {f(12), f(14)})
            , A.is_any, {}
          },
        A.is_any, {}
      }
    , A.regs_by_bits("bits", r(4)..r(7))
    , A.overflow { growth = "up", max_alignment = Mips.sp_align }
    }

<mipsCALL.ml 758c>+≡ (756a) <757e 759b>
  let prefix16bytes result =
    let b = Block.relative vfp "16-byte block" Block.at ~size:16 ~alignment:4
    in
      { result with
        A.overflow = Block.cath1 result.A.overflow b
      }

  let postprocess cconv =
    { cconv with A.call = A.postprocess cconv.A.call prefix16bytes }

<MIPS calling convention automata in Lua 758d>+≡ <758b 760a>
  Mips.cc["C"].cutto =
    { A.widen (32, "multiple")
    , A.choice { "float" , A.userregs(Mips.vol_fp)
                , A.is_any, A.userregs(Mips.vol_int)
                }
    , A.overflow { growth = "up", max_alignment = Mips.sp_align }
    }

```

Z.6.2 Putting it together

```

⟨transformations(mipsCALL.nw) 759a⟩≡ (759b)
let autoAt = A.at Rg.mspace in
let prolog =
  let autosp = (fun _ -> vfp) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in call parms" autoAt stage.A.call)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let epilg =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out ovfl results" autoAt stage.A.results)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun _ _ -> vfp) in

let call_actuals =
  C.outgoing ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "out call parms" autoAt stage.A.call)
  ~autosp:(fun r -> std_sp_location)
  ~postsp:(fun a sp -> std_sp_location) in

let call_results =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in ovfl results" autoAt stage.A.results)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let also_cuts_to =
  let autosp = (fun r -> std_sp_location) in
  C.incoming ~growth ~sp
  ~mkauto:(fun () -> Block.srelative vfp "in cont parms" autoAt stage.A.cutto)
  ~autosp
  ~postsp:(fun _ _ -> std_sp_location)
  ~insp:(fun a _ _ -> autosp a) in

let cut_actuals base =
  C.outgoing ~growth ~sp ~mkauto:(fun () -> autoAt base stage.A.cutto)
  ~autosp:(fun r -> spval)
  ~postsp:(fun _ _ -> spval) in

⟨mipsCALL.ml 759b⟩+≡ (756a) ◁758c 760b▷
let c ~return_to cut stage =
  let stage = postprocess stage in
  ⟨transformations(mipsCALL.nw) 759a⟩
  let return k n ~ra =
    if k = 0 & n = 0 then return_to ra
    else impossf "alternate return using C calling convention" in
  { C.name = "C"
  ; C.overflow_alloc = { C.parameter_deallocator = C.Caller
                        ; C.result_allocator = C.Caller
                        }
  ; C.call_parms = { C.in' = prolog ; C.out = call_actuals }
  ; C.cut_parms = { C.in' = also_cuts_to ; C.out = cut_actuals }
  ; C.results = { C.in' = call_results ; C.out = epilg }

```



```

; C.stack_growth      = growth
; C.stable_sp_loc     = std_sp_location
(* claude: Cfgx.Vfp -> Vfprewrite rename, same fix as sparccall.ml/
 * alphacall.ml. *)
; C.replace_vfp       = Vfprewrite.replace_with ~sp
; C.sp_align          = sp_align
; C.pre_nvlregs       = RS.union (RS.of_list nvl_int) (RS.of_list nvl_fp)
; C.volregs           = RS.union (RS.of_list vol_int) (RS.of_list vol_fp)
; C.saved_nvr         = saved_nvr
(* claude: C.cutto (the newpc/newsp embed/project map) isn't a Call.t
 * field anymore - see call.mli, same drop as sparccall.ml/alphacall.ml
 * ("C.cutto = cut" removed; the 'cut' param is now threaded through
 * purely for Mipsccall.cconv's/mips.ml's interface shape, unused here).
 * C.jump_tgt_reg is new: a hardware register reserved for indirect
 * jumps, since spilling a temp after the sp has already moved would
 * be unsafe. $1/at is MIPS's own reserved assembler-temp register,
 * same role as alphacall.ml's pick of Alpha's $28/at. *)
; C.jump_tgt_reg      = R.reg (r 1)
; C.return            = return
; C.ra_on_entry        = (fun _      -> R.fetch ra 32)
; C.where_to_save_ra   = (fun _ t    -> Talloc.Multiple.loc t 't' 32)
; C.ra_on_exit         = (fun _ _ t -> ra)
; C.sp_on_unwind       = (fun e      -> RU.store sp e)
; C.sp_on_jump         = (fun _ _    -> Rtl.null)
}

```

⟨MIPS calling convention automata in Lua 760a⟩+≡ ◁758d

```

-- register the new calling conventions!
A.register_cc(Backend.mips.target, "C" , Mips.cc["C"])
A.register_cc(Backend.mips.target, "C'", Mips.cc["C"])
A.register_cc(Backend.mips.target, "C--", Mips.cc["C"])

```

Z.6.3 Implementation using Callspec

⟨mipsccall.ml 760b⟩+≡ (756a) ◁759b 761b▷

```

(* claude: Callspec (module CS) isn't wired into this fork's build yet - it
 * still sits, untouched, in TODO/arch/callspec.{ml,mli}. Nothing in
 * Mipscc.cc_specs (written to fix T.cc_specs = A.init_cc's empty-table bug,
 * same as sparccc.ml/alphacc.ml) dispatches to the "C" name this
 * Callspec-backed c' would have handled - Mipscc.cc_specs only registers
 * "C"/"C--"/"notail"/"C-- thread", all landing on the plain 'c' convention
 * above via cconv's own "_ -> c" fallback just below. So this was already
 * dead code before Callspec went missing from the link line - not a
 * functional loss to comment out. Kept here in case Callspec is integrated
 * later and this is worth reviving as-is:
 *
 * module CS = Callspec
 *
 * let rtn return_to k n ~ra =
 *   if k = 0 & n = 0 then return_to ra
 *   else impossf "alternate return using C calling convention"
 *
 * let c' ~return_to cut auto =
 *   let spec =
 *     { CS.name      = "C'"
 *     ; CS.stack_growth = Memalloc.Down
 *     ; CS.overflow   = CS.overflow C.Caller C.Caller
 *     ; CS.sp         = r 29
 *     ; CS.sp_align   = sp_align

```

```

*           ; CS.memspace      = Rg.mspace
*           ; CS.all_regs      = RS.of_list (List.concat [nvl_int; nvl_fp;
*                                           vol_int; vol_fp])
*           ; CS.nv_regs       = RS.of_list (nvl_int @ nvl_fp)
*           ; CS.save_nvr      = saved_nvr
*           ; CS.ra             = (ra, CS.ReturnAddress.SaveToTemp 't')
*       }
*
*   in
*   let t = CS.to_call cut (rtn return_to) auto spec in
*   { t with (* fix what callspec got wrong *)
*         C.ra_on_exit   = (fun _ _ t -> ra)
*       ; C.sp_on_unwind = (fun e -> RU.store sp e)
*   }
*)

```

$\langle \text{callspec specification } 761a \rangle \equiv$

```

let spec =
  { CS.name      = "C'"
  ; CS.stack_growth = Memalloc.Down
  ; CS.overflow   = CS.overflow C.Caller C.Caller
  ; CS.sp         = r 29
  ; CS.sp_align   = sp_align
  ; CS.memspace   = Rg.mspace
  ; CS.all_regs   = RS.of_list (List.concat [nvl_int; nvl_fp;
                                           vol_int; vol_fp])
  ; CS.nv_regs    = RS.of_list (nvl_int @ nvl_fp)
  ; CS.save_nvr   = saved_nvr
  ; CS.ra         = (ra, CS.ReturnAddress.SaveToTemp 't')
  }

```

$\langle \text{mipscall.ml } 761b \rangle + \equiv$ (756a) $\triangleleft 760b \ 761c \triangleright$

```

let cconv ~return_to cut ccname stage =
  let f =
    match ccname with
    | _ -> c
  in f ~return_to cut stage

```

$\langle \text{mipscall.ml } 761c \rangle + \equiv$ (756a) $\triangleleft 761b$

```

(*)
let cconv ~return_to cut ccname stage =
  let f =
    match ccname with
    | "C'" -> c'
    | _ -> c
  in f ~return_to cut stage
*)

```

Z.7 arch/mips/mipsrec.nw

$\langle \text{arch/mips/mipsrec.mli } 761d \rangle \equiv$

$\langle \text{mipsrec.mli } 761e \rangle$

Z.8 MIPS Recognizer

$\langle \text{mipsrec.mli } 761e \rangle \equiv$ (761d)

```

val is_instruction: Rtl.rtl -> bool
val to_string:      Rtl.rtl -> string

```

Z.8.1 Implementation

```
<mipsrec.mlb 762a>≡
  %head {:
    <head 762b>
  :}
  %tail {:
    <tail(mipsrec.nw) 766c>
  :}
  %term
    <terminal types(mipsrec.nw) 763b>
  %%
  <rules(mipsrec.nw) 763c>

<head 762b>≡ (762a) 762c▷
  (* claude: =\$= (string equality) lives in Nopoly, not the default Stdlib
   * polymorphic (=) - needed below by the "syscall" mangled-name guard. *)
  open Nopoly
  module RP = Rtl.Private
  module RU = Rtlutil
  module Up = Rtl.Up
  module Dn = Rtl.Dn
  module SS = Space.Standard32

<head 762c>+≡ (762a) <762b 762d▷
  exception Error of string
  let error msg = raise (Error msg)
  let sprintf = Printf.sprintf (* useful for formatting msg *)

<head 762d>+≡ (762a) <762c 762e▷
  let guard p = if p then 0 else Camlburg.inf_cost

<head 762e>+≡ (762a) <762d 763a▷
  (* claude: was Bits.U.to_native (unsigned) - mipsel-linux-gnu-as rejects
   * an unsigned 32-bit literal like "4294967264" as an addi/lw/sw 16-bit
   * signed-immediate operand ("operand out of range"), even though it is
   * bit-for-bit the same value as -32. GAS's imm16 range check looks at
   * the literal as written, it does not infer a negative interpretation
   * from a large unsigned decimal. Bits.S.to_native (signed) prints "-32"
   * instead, which addi/lw/sw's 16-bit field accepts directly, and li/la
   * (which the same const32 feeds too) don't care either way. *)
  let const32 b =
    assert (Bits.width b = 32);
    Nativeint.to_string (Bits.S.to_native b)

  let const64 b =
    assert (Bits.width b = 64);
    Int64.to_string (Bits.U.to_int64 b) (* signed or unsigned? *)

  let cat = String.concat ""
  let printf = Printf.printf
  let sprintf = Printf.sprintf

  let reg n = "$" ^ string_of_int n
  let freg n = "$f" ^ string_of_int n

  let suffix = function
    | 8 -> "b"
    | 16 -> "h"
    | 32 -> "w"
    | w -> error (sprintf "not a MIPS width: %d" w)
```

```

let zx      = "u"  (* to construct op-code *)
let sx      = ""
let width = string_of_int

⟨head 763a⟩+≡ (762a) <762e
  let lo b =
    assert (Bits.width b = 64);
    Bits.Ops.lobits 32 b

  let b32 = Bits.U.of_int 32 64
  let hi b =
    assert (Bits.width b = 64);
    Bits.Ops.lobits 32 (Bits.Ops.shr1 b b32)

```

Z.8.2 Recognizer Rules

```

⟨terminal types(mipsrec.nw) 763b⟩≡ (762a)
  bits n x w

```

```

⟨rules(mipsrec.nw) 763c⟩≡ (762a) 763d>
  const64: Bits(bits) [{: guard (Bits.width bits = 64) :}]{: bits :}
  const:   Bits(bits) [{: guard (Bits.width bits = 32) :}]{: const32 bits :}
  symbol:   Link(x, w:int)      {: x#mangled_text :}

```

```

⟨rules(mipsrec.nw) 763d⟩+≡ (762a) <763c 763e>
  f:      Reg('f', n:int)  {: n :}
  r:      Reg('r', n:int)  {: n :}

  regl:    r   {: reg r   :}
  fregl:   f   {: freg f :}

  pcl:     Reg('c', 0) {: () :}
  spl:     Reg('r', 29) {: () :}
  ral:     Reg('r', 31) {: () :}

  reg:     Fetch(regl,w:int)  {: regl   :}
  freg:    Fetch(fregl,w:int) {: fregl  :}

  pc:      Fetch(pcl,32)      {: () :}
  sp:      Fetch(spl,32)      {: () :}
  ra:      Fetch(ral,32)      {: () :}

```

```

⟨rules(mipsrec.nw) 763e⟩+≡ (762a) <763d 763f>
  meml:     Mem(addr)      {: addr :}
  mem:      Fetch(meml,w:int)  {: meml :}

  addr:     reg             {: cat ["(";reg;")"] :}
  addr:     imm             {: imm :}
  addr:     Add(imm,reg)     {: cat [imm;"(";reg;")"] :}
  addr:     Add(reg,imm)     {: cat [imm;"(";reg;")"] :}
  addr:     symbol          {: symbol :}

```

```

⟨rules(mipsrec.nw) 763f⟩+≡ (762a) <763e 764a>
  imm:      const           {: const :}
  imm:      symbol          {: symbol :}
  imm:      Add(symbol,imm)  {: cat [symbol;"+";imm] :}
  imm:      Add(imm,symbol)  {: cat [symbol;"+";imm] :}

```

⟨rules(mipsrec.nw) 764a⟩+≡ (762a) ◁763f 764c▷

```
inst:      Store(regl,imm,32)
           {: cat ["la"; " "; regl; ","; imm] :}

inst:      Store(regl,const,32)
           {: cat ["li"; " "; regl; ","; const] :}

inst:      Store(regl,mem,32)
           {: cat ["l"; suffix 32; " "; regl; ","; mem] :}

inst:      Store(regl, Sx(Fetch(mem,x:int)), w:int)
           {: cat ["l"; suffix w; sx; " "; regl; ","; mem] :}

inst:      Store(regl, Zx(Fetch(mem,x:int)), w:int)
           {: cat ["l"; suffix w; zx; " "; regl; ","; mem] :}
```

⟨rules we don't use 764b⟩≡

```
inst:      Store(fregl,const,32)
           {: cat ["li.s"; " "; fregl; ","; const] :}

inst:      Store(fregl,const,64)
           {: cat ["li.d"; " "; fregl; ","; const] :}
```

⟨rules(mipsrec.nw) 764c⟩+≡ (762a) ◁764a 764d▷

```
inst:      Store(fregl,const,32)
           {: sprintf "li $1, %s; mtc1 $1, %s" const fregl :}
```

(* claude: Store(meml, freg, w) (float store) already had a rule below ("s.s"); the mirror-image float load new

```
inst:      Store(fregl, mem, 32)
           {: cat ["l.s"; " "; fregl; ","; mem] :}
```

⟨rules(mipsrec.nw) 764d⟩+≡ (762a) ◁764c 764e▷

```
inst:      Store(f,b:const64,64)
           {: sprintf "li $1, %s; mtc1 $1, %s; li $1 %s; mtc1 $1, %s"
                    (const64 (lo b)) (freg f) (const64 (hi b)) (freg (f+1))
           :}
```

⟨rules(mipsrec.nw) 764e⟩+≡ (762a) ◁764d 764f▷

```
inst:      Store(meml, reg, w:int)
           {: cat ["s"; suffix w; " "; reg; ","; meml] :}

inst:      Store(meml, freg, w:int)
           {: cat ["s.s "; freg; ","; meml] :}
```

⟨rules(mipsrec.nw) 764f⟩+≡ (762a) ◁764e 764g▷

```
inst:      Store(regl, reg, 32)
           {: cat ["move"; " "; regl; ","; reg] :}

inst:      Store(fregl, freg, 32)
           {: cat ["mov.s"; " "; fregl; ","; freg] :}

inst:      Store(fregl, freg, 64)
           {: cat ["mov.d"; " "; fregl; ","; freg] :}
```

⟨rules(mipsrec.nw) 764g⟩+≡ (762a) ◁764f 765a▷

```
inst:      Store(fregl, reg, 32)
           {: cat ["nop; mtc1"; " "; reg; ","; fregl] :}

inst:      Store(regl, freg, 32)
           {: cat ["nop; mfc1"; " "; regl; ","; freg] :}
```

```

⟨rules(mipsrec.nw) 765a⟩+≡ (762a) <764g 765b>
(* claude: same missing-delay-slot bug as the Goto(reg)/jr rule below - unconditional "j" also has a delay slot
inst:      Goto(symbol)
           {: cat ["j"; " "; symbol; "\n\tnop"] :}

(* claude: this rule had no delay-slot filler either - the Guarded rule's comment below claiming "mirrors the j
inst:      Goto(reg)
           {: cat ["jr"; " "; reg; "\n\tnop"] :}

⟨rules(mipsrec.nw) 765b⟩+≡ (762a) <765a 765c>
next:      Add(pc,const)      {: () :}
(* claude: the original "-- s/addr/symbol/g ?" note was right to be suspicious - addr also matches reg (as "(re
inst:      Par(Goto(symbol),Store(ral,next,32))
           {: cat ["jal"; " "; symbol] :}

inst:      Par(Goto(target:reg),Store(ral,next,32))
           {: cat ["jalr"; " "; target] :}

⟨rules(mipsrec.nw) 765c⟩+≡ (762a) <765b 765d>
(* claude: recognizes Mips.Post.cut_to's jump-through-register-while-updating-sp RTL, the shape the tiger runti
inst:      Par(Goto(target:reg), Store(spl, nsp:reg, 32))
           {: sprintf "jr %s\n\tnmove $29, %s" target nsp :}

syscall:   Link(x, w:int) [{: guard (x#mangled_text == "syscall") :}] {: () :}
inst:      Par(Goto(syscall), Store(ral,pc,32))
           {: "syscall" :}

⟨rules(mipsrec.nw) 765d⟩+≡ (762a) <765c 765e>
cmp:      Cmp(op:string,x:reg,y:reg)      {: (op,x,y) :}
cmp:      Cmp(op:string,x:reg,y:const)    {: (op,x,y) :}
(* claude: this rule had no delay-slot filler at all - under mipsasm.ml's ".set noreorder" the assembler never
inst:      Guarded(cmp,Goto(addr))
           {: match cmp with
              | (op,x,y) -> cat ["b";op;" ";x;" ";y;" ";addr;"\n\tnop"]
              :}

⟨rules(mipsrec.nw) 765e⟩+≡ (762a) <765d 766a>
inst:      Store(dst:regl, Add(x:reg,y:reg), 32)
           {: cat ["add"; " "; dst; ","; x; ","; y] :}

inst:      Store(dst:regl, Add(x:reg,y:imm), 32)
           {: cat ["addi"; " "; dst; ","; x; ","; y] :}

(* claude: the tail's exp function already converts sub to conSub (same as conAdd), but no inst: rule ever matc
inst:      Store(dst:regl, Sub(x:reg,y:reg), 32)
           {: cat ["sub"; " "; dst; ","; x; ","; y] :}

(* claude: bitwise and, needed by fork-tiger's alloc.c-- (rounding a pointer/size to an alignment via a mask).
inst:      Store(dst:regl, And(x:reg,y:reg), 32)
           {: cat ["and"; " "; dst; ","; x; ","; y] :}

(* claude: no %mul rule existed at all - tiger array indexing multiplies by element size constantly, so this wa
inst:      Store(dst:regl, Mul(x:reg,y:reg), 32)
           {: cat ["mul"; " "; dst; ","; x; ","; y] :}

(* claude: %quot (needed by tiger's array bounds-check/stride math wherever a size divides, e.g. colmajor's row
inst:      Store(dst:regl, Quot(x:reg,y:reg), 32)
           {: cat ["div"; " "; x; ","; y; "\n\tnflo "; dst] :}

inst:      Store(dst:fregl, D2S(x:freg), 32)

```

```

{: cat ["cvt.s.d"; " "; dst; ",,"; x] :}

inst:      Store(dst:freg1, S2D(x:freg), 64)
{: cat ["cvt.d.s"; " "; dst; ",,"; x] :}

⟨rules(mipsrec.nw) 766a⟩+≡ (762a) ◁765e 766b▷
inst:      Nop() {: "nop" :}

```

Z.8.3 Debugging Support

```

⟨rules(mipsrec.nw) 766b⟩+≡ (762a) ◁766a
inst: any [100]      {: cat ["<";any;">"] :}

any : True  ()      {: cat [ "True" ] :}
any : False ()      {: cat [ "False" ] :}
any : Link(x, w:int) {: cat [ "Link(";x#mangled_text;",";width w;")" ] :}
any : Late(string,w:int){: cat [ "Late(";string;",";width w;")" ] :}
any : Bits(bits)    {: cat [ "Bits(b)" ] :}

any : Fetch (any, w:int){: cat [ "Fetch(";any;",";width w;")" ] :}

any : Add(x:any, y:any) {: cat [ "Add(";x;",";y;")" ] :}
any : Sub(x:any, y:any) {: cat [ "Sub(";x;",";y;")" ] :}
any : Sx(any)          {: cat [ "Sx(";any;")" ] :}
any : Zx(any)          {: cat [ "Zx(";any;")" ] :}
any : Lobits(any)      {: cat [ "Lobits(";any;")" ] :}

any : Mem(any)  {: cat [ "Mem(";any;")" ] :}
any : Reg(char, n:int) {: cat [ "Reg('";Char.escaped char;"',"; width n;")" ] :}

any : Store (dst:any, src:any, w:int)
      {: cat [ "Store(";dst;",";src;",";width w;")" ] :}
any : Kill(any)      {: cat [ "Kill(";any;")" ] :}

any : Guarded(guard:any, any)
      {: cat [ "Guarded(";guard;",";any;")" ] :}
any : Cmp(op:string, x:any, y:any)
      {: cat [ "Cmp(";op;",";x;",";y;")" ] :}
any : Par(l:any, r:any) {: cat [ "Par(";l;",";r;")" ] :}
any : Goto(any)        {: cat [ "Goto(";any;")" ] :}

```

Z.8.4 Interfacing RTLs with the Expander

```

⟨tail(mipsrec.nw) 766c⟩≡ (762a) 766d▷
let const = function
  | RP.Bool _      -> error "boolean found"
  | RP.Link(s,_,w) -> conLink s w
  | RP.Diff _      -> error "PIC not supported"
  | RP.Bits(b)     -> conBits b
  | RP.Late(s,w)   -> error (sprintf "late constant %s found" s)

⟨tail(mipsrec.nw) 766d⟩+≡ (762a) ◁766c 767a▷
⟨helpers for exp and loc 767d⟩
let rec exp = function
  | RP.Const(k)      -> const k
  | RP.Fetch(l,w)    -> conFetch (loc l) w
  | RP.App((o,_),_) -> error (sprintf "unknown operator %s" o)

```

```

and loc = function
  | RP.Reg((sp,_,_),i,w)      -> conReg sp i
  | RP.Mem(('m',_,_),w,e,ass) -> conMem (exp e)
  | RP.Mem((sp,_,_),_,_,_)   -> error (sprintf "mem-space space %c" sp)
  | RP.Var (s,i,w)           -> error (sprintf "var %s found" s)
  | RP.Global(s,i,w)         -> error (sprintf "var %s found" s)
  | RP.Slice _               -> error "cannot handle slice"

⟨tail(mipsrec.nw) 767a⟩+≡ (762a) <766d 767b>
  let effect = function
    ⟨Mipsrec special cases for Store 767c⟩
    | RP.Store(l,e,w)      -> conStore (loc l) (exp e) w
    | RP.Kill(l)           -> error "cannot handle kill"

⟨tail(mipsrec.nw) 767b⟩+≡ (762a) <767a 768a>
  let guarded g stmt = match g with
    | RP.Const(RP.Bool b)   -> if b then effect stmt else conNop ()
    ⟨Mipsrec special cases for guarded 767f⟩
    | _                     -> conGuarded (exp g) (effect stmt)

  let rec geffects = function
    | []                    -> conNop ()
    | [g, s]                -> guarded g s
    | (g, s) :: t           -> conPar (guarded g s) (geffects t)

  let rtl (RP.Rtl es) = geffects es

```

Z.8.5 Special cases

```

⟨Mipsrec special cases for Store 767c⟩≡ (767a)
  | RP.Store(RP.Reg(('c',_,_),i,w),r,_)
    when i = SS.indices.SS.pc -> conGoto (exp r)

```

```

⟨helpers for exp and loc 767d⟩≡ (766d)
  let cmp =
    Strutil.from_list ["eq";"ge";"geu";"gt";"gtu";"le";"leu";"lt";"ltu";"ne"]

```

```

⟨Mipsrec special cases for App 767e⟩≡ (766d)
  | RP.App(("add", [w]), [x; y])      -> conAdd (exp x) (exp y)
  | RP.App(("sub", [w]), [x; y])      -> conSub (exp x) (exp y)
  | RP.App(("and", [w]), [x; y])      -> conAnd (exp x) (exp y)
  | RP.App(("mul", [w]), [x; y])      -> conMul (exp x) (exp y)
  | RP.App(("quot", [w]), [x; y])     -> conQuot (exp x) (exp y)
  | RP.App(("f2f_implicit_round", [32;64]), [x]) -> conS2D (exp x)
  | RP.App(("f2f_implicit_round", [64;32]), [x]) -> conD2S (exp x)
  (* claude: %lobits only ever wraps a value immediately feeding a narrow-width memory store in this backend (see
  | RP.App(("lobits", [_;_]), [x])     -> exp x

  | RP.App((op, [w]), [x; y])
    when Strutil.Set.mem op cmp      -> conCmp op (exp x) (exp y)

```

```

⟨Mipsrec special cases for guarded 767f⟩≡ (767b)

```


Z.8.6 The exported recognizers

```
<tail(mipsrec.nw) 768a>+≡ (762a) <767b
let rtl_to_string = RU.ToString.rtl

let dump msg rtl =
  List.iter prerr_string
  [ "error in recognizer: "
    ; msg
    ; " on "
    ; rtl_to_string rtl
    ; "\n"
  ]

let to_string r =
  try
    let plan = rtl (Dn.rtl r) in
    Printf.sprintf "\t%s" (plan.inst.Camlburg.action ())
  with
    | Camlburg.Uncovered -> cat ["not an instruction: "
                                ; rtl_to_string r
                              ]
    | Error msg          -> ( dump msg r
                              ; sprintf "error: %s" (rtl_to_string r)
                              )

let is_instruction r =
  try
    let plan = rtl (Dn.rtl r) in
    plan.inst.Camlburg.cost < 100
  with
    | Camlburg.Uncovered -> false
    | Error msg          -> ( dump msg r
                              ; false
                              )
```

Z.9 arch/mips/mipsregs.nw

```
<arch/mips/mipsregs.ml 768b>≡
<mipsregs.ml 769a>

<arch/mips/mipsregs.mli 768c>≡
<mipsregs.mli 768d>
```

Z.10 MIPS spaces and registers

```
<mipsregs.mli 768d>≡ (768c)
module Spaces : sig
  val m : Space.t (* memory *)
  val r : Space.t (* integer regs *)
  val f : Space.t (* floating regs *)
  val c : Space.t (* special registers *)

  val t : Space.t (* 32-bit integer temps *)
  val u : Space.t (* 32-bit floating temps *)
end
```

```

val pc : Rtl.loc
val npc : Rtl.loc
val cc : Rtl.loc

val mspace : Rtl.space
val rspace : Rtl.space
val fspace : Rtl.space

<mipsregs.ml 769a>≡ (768b) 769b▷
  module R = Rtl
  module S = Space
  module SS = Space.Standard32

  let byteorder = Rtl.LittleEndian
  let mcell = Cell.of_size 8
  let mspace = ('m', byteorder, mcell)

<mipsregs.ml 769b>+≡ (768b) <769a 769c>▷
  let rspace = ('r', Rtl.Identity, Cell.of_size 32)
  let fspace = ('f', byteorder, Cell.of_size 32)
  module Spaces = struct
    let id = Rtl.Identity
    let m = SS.m byteorder [8; 16; 32]
    let r = SS.r 32 id [32]
    let f = SS.f 32 byteorder [32; 64]
    let t = SS.t id 32
    let u = SS.u byteorder 32
    let c = SS.c 6 id [32] (* pc, npc, cc, _, fp_mode, fp_fcmp *)
  end

<mipsregs.ml 769c>+≡ (768b) <769b>▷
  let locations = SS.locations Spaces.c
  let pc = locations.SS.pc
  let cc = locations.SS.cc
  let npc = locations.SS.npc

```

Appendix AA

arch/arm

AA.1 arch/arm/arm.nw

$\langle \text{arch/arm/arm.ml } 770a \rangle \equiv$
 $\langle \text{arm.ml } 770d \rangle$

$\langle \text{arch/arm/arm.mli } 770b \rangle \equiv$
 $\langle \text{arm.mli } 770c \rangle$

AA.2 Back end for the ARM

$\langle \text{arm.mli } 770c \rangle \equiv$ (770b)
module Post : Postexpander.S
module X : Expander.S

val target: Ast2ir.tgt
val placevars : Ast2ir.proc -> Automaton.t

AA.2.1 Abbreviations and utility functions

$\langle \text{arm.ml } 770d \rangle \equiv$ (770a) 771b▷
(* claude: == (list-of-widths equality) lives in Nopoly, not the default
* Stdlib polymorphic (=) - same fix arch/mips/mips.ml/arch/sparc/sparc.ml/
* arch/alpha/alpha.ml needed. *)
open Nopoly
module SS = Space.Standard32
module S = Space
module A = Automaton
let (*>) = A.(*>)
module PX = Postexpander
module DG = Dag
module R = Rtl
module RP = Rtl.Private
module RU = Rtlutil
module Up = Rtl.Up
module Dn = Rtl.Dn
module SM = Strutil.Map
module T = Target

let unimp = Impossible.unimp
let impossible = Impossible.impossible
(* claude: PX.Rtl/PX.<:> don't exist - Nop/Rtl/Test/<:> live in Dag

```

* (aliased DG above), not Postexpander; same stale-interface fix already
* applied to arch/mips/mips.ml/arch/sparc/sparc.ml/arch/alpha/alpha.ml
* (arm.ml was ported straight from upstream's arm.nw, which predates that
* interface change, so it still needs the same fix here). *)
let rtl r = DG.Rtl r
let (<:>) = DG.<:>

```

```

<utilities that depend on byteorder or wordsize 771a>≡ (771b)
let fetch_word l      = R.fetch l    wordsize
let store_word l e    = R.store l e wordsize
let mcell = Cell.of_size 8
let mspace = ('m', byteorder, mcell)
let mcount = Cell.to_count mcell
let mem w addr       = R.mem R.none mspace (mcount w) addr

```

AA.2.2 Name and storage spaces

```

<arm.ml 771b>+≡ (770a) <770d 771c>
let arch      = "arm" (* architecture *)
let byteorder = Rtl.LittleEndian
let wordsize  = 32
<utilities that depend on byteorder or wordsize 771a>

```

```

<arm.ml 771c>+≡ (770a) <771b 771d>
module Spaces = struct
  let id = Rtl.Identity
  let m  = SS.m byteorder [8; 16; 32]
  let r  = SS.r 16 id [32]
  let t  = SS.t id 32
  let c  = SS.c 3 id [32] (* pc, _, cc *)
end

```

AA.2.3 Registers

```

<arm.ml 771d>+≡ (770a) <771c 771e>
let locations = SS.locations Spaces.c
let pc        = locations.SS.pc
let cc        = locations.SS.cc
let vfp       = Vfp.mk wordsize

let rspace = ('r', Rtl.Identity, Cell.of_size 32)
let reg n  = (rspace, n, Rtl.C 1)
let sp     = reg 13 (* stack pointer *)
let ra     = reg 14 (* return address *)

```

AA.2.4 Variable Placer

```

<arm.ml 771e>+≡ (770a) <771d 772a>
let placevars =
  let warn ~width:w ~alignment:a ~kind:k =
    if w > 32 then unimp (Printf.sprintf "%d-bit values not supported" w) in
  let mk_stage ~temps =
    A.choice
    [ (fun w h _ -> w <= 32), A.widen (fun _ -> 32) *> temps 't';
      A.is_any, A.widen (Auxfun.round_up_to ~multiple_of: 8);
    ] in
  Placevar.mk_automaton ~warn ~vfp ~memspace:mspace mk_stage

```

AA.2.5 Control-flow RTLs

$\langle \text{arm.ml } 772a \rangle + \equiv$ (770a) $\triangleleft 771e \ 772b \triangleright$

```
module F = Mflow.MakeStandard
  (struct
    let pc_lhs    = pc
    let pc_rhs    = pc
    let ra_reg    = R.reg ra
    let ra_offset = 4 (* size of call instruction *)
  end)
```

$\langle \text{arm.ml } 772b \rangle + \equiv$ (770a) $\triangleleft 772a \ 772c \triangleright$

```
(* claude: was "R.store pc (fetch_word (R.reg ra))", missing its trailing
 * 'width' argument (R.store is curried loc -> exp -> width -> rtl), so
 * 'return' had the hidden function type 'width -> rtl' rather than the
 * plain 'Rtl.rtl' every actual use site below needs - same bug mips.ml's
 * own 'return' originally had. Turned into an explicit function of the
 * value to store into pc, matching alpha.ml's/sparc.ml's own top-level
 * 'return e', used both by Post.return below and by T.cc_spec_to_auto's
 * ~return_to. *)
let return e = R.store pc e wordsize
(* claude: needed for T.cc_spec_to_auto's cutto embed/project pair below -
 * same role as mips.ml's/sparc.ml's/alpha.ml's own fmach. *)
let fmach = F.machine (R.reg sp)
```

AA.2.6 Postexpander

$\langle \text{arm.ml } 772c \rangle + \equiv$ (770a) $\triangleleft 772b \ 776c \triangleright$

```
module Post = struct
   $\langle \text{ARM postexpander } 772d \rangle$ 
end
```

$\langle \text{ARM postexpander } 772d \rangle \equiv$ (772c) $772e \triangleright$

```
let byte_order  = byteorder
let wordsize    = wordsize
let exchange_alignment = 4

type temp      = Register.t
type rtl       = Rtl.rtl
type width     = Rtl.width
type assertion = Rtl.assertion
type operator   = Rtl.Private.opr
```

$\langle \text{ARM postexpander } 772e \rangle + \equiv$ (772c) $\triangleleft 772d \ 772f \triangleright$

```
let talloc = Postexpander.Alloc.temp
```

$\langle \text{ARM postexpander } 772f \rangle + \equiv$ (772c) $\triangleleft 772e \ 773a \triangleright$

```
let icontext = Context.of_space Spaces.t
let acontext = icontext
let itempwidth = 32
let fcontext = (fun x y -> unimp "no floating point on ARM"), fun _ -> false
let rcontext = (fun x y -> unimp "no rounding mode on ARM"), fun _ -> false
let constant_context w = icontext

let operators = Context.nonbool icontext fcontext rcontext []
let arg_contexts, result_context = Context.functions operators
```

```

<ARM postexpander 773a>+≡ (772c) <772f 773b>
  module Address = struct
    type t      = Rtl.exp
    let reg r = R.fetch (R.reg r) (Register.width r)
  end
  include Postexpander.Nostack(Address)

<ARM postexpander 773b>+≡ (772c) <773a 773c>
  let tloc t = Rtl.reg t
  let tval t = R.fetch (tloc t) (Register.width t)
  let twidth = Register.width

  let load ~dst ~addr assn =
    let w = twidth dst in
      assert (w = wordsize);
      rtl (R.store (tloc dst) (R.fetch (mem w addr) w) w)

  let store ~addr ~src assn =
    let w = twidth src in
      assert (w = wordsize);
      rtl (R.store (mem w addr) (tval src) w)

  let block_copy ~dst dassn ~src sassn w =
    match w with
    | 32 -> let t = talloc 't' w in load t src sassn <:> store dst t dassn
    | _   -> Impossible.unimp "general block copies on Arm"

<ARM postexpander 773c>+≡ (772c) <773b 773d>
  let extend op n e = R.app (R.opr op [n; wordsize]) [e]
  let lobits n e = R.app (R.opr "lobits" [wordsize; n]) [e]

  let xload op ~dst ~addr n assn =
    let w = twidth dst in
      assert (w = wordsize);
      rtl (R.store (tloc dst)
        (extend op n (R.fetch (R.mem assn mspace (mcount n) addr) n)) w)

  let sxload = xload "sx"
  let zxload = xload "zx"

  let lostore ~addr ~src n assn =
    assert (Register.width src = wordsize);
    rtl (R.store (R.mem assn mspace (mcount n) addr) (lobits n (tval src)) n)

<ARM postexpander 773d>+≡ (772c) <773c 773e>
  let move ~dst ~src =
    assert (Register.width src = Register.width dst);
    if Register.eq src dst then DG.Nop
    else rtl (R.store (tloc dst) (tval src) (twidth src))

<ARM postexpander 773e>+≡ (772c) <773d 773f>
  let extract ~dst ~lsb ~src = Impossible.unimp "extract"
  let aggregate ~dst ~src = Impossible.unimp "aggregate"

<ARM postexpander 773f>+≡ (772c) <773e 773g>
  let hwset ~dst ~src = Impossible.unimp "setting hardware register"
  let hwget ~dst ~src = Impossible.unimp "getting hardware register"

<ARM postexpander 773g>+≡ (772c) <773f 774a>
  let li ~dst const = rtl (R.store (tloc dst) (Up.const const) (twidth dst))
  let lix ~dst e = rtl (R.store (tloc dst) e (twidth dst))

```

```

(ARM postexpander 774a)+≡ (772c) <773g 774b>
  let subflags x y w = R.store cc (R.app (R.opr "arm_subcc" [w]) [x; y]) 32

  let unop ~dst op x =
    rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x]) (twidth dst))

  let binop ~dst op x y =
    rtl (R.store (tloc dst) (R.app (Up.opr op) [tval x; tval y]) (twidth dst))

  let unrm ~dst op x rm = Impossible.unimp "floating point with rounding mode"
  let binrm ~dst op x y rm = Impossible.unimp "floating point with rounding mode"

  let dblop ~dsthi ~dstlo op x y = Unsupported.mulx_and_mlux()
  let wrdop ~dst op x y z = Unsupported.singlebit ~op:(fst op)
  let wrdrop ~dst op x y z = Unsupported.singlebit ~op:(fst op)

(ARM postexpander 774b)+≡ (772c) <774a 774c>
  let pc_lhs = pc (* PC as assigned by branch *)
  let pc_rhs = pc (* PC as captured by call *)

(ARM postexpander 774c)+≡ (772c) <774b 774d>
  let br ~tgt = DG.Nop, R.store pc_lhs (tval tgt) wordsize (* branch reg *)
  let b ~tgt = DG.Nop, R.store pc_lhs (Up.const tgt) wordsize (* branch *)

(ARM postexpander 774d)+≡ (772c) <774c 775a>
  let cmp x y = rtl (subflags (tval x) (tval y) 32)

(* claude: split from the old single 'bc' (kept in spirit, same RTLs)
 * into bc_guard/bc_of_guard, the shape Postexpander.S now requires -
 * see arch/sparc/sparc.ml's bc_guard/bc_of_guard for the same split
 * applied to a target whose condition codes are also a single flags
 * pseudo-location (arm.ml's 'cc', like sparc.ml's), rather than
 * MIPS's direct register-vs-register branch or Alpha's register-vs-
 * zero branch. *)
let rec bc_guard x (opr, ws as op) y =
  assert (ws == [wordsize]);
  let cond c = R.app (R.opr c [32]) [R.fetch cc 32] in
  match opr with
  | "eq" | "ne" | "lt" | "le" | "gt" | "ge" | "leu" | "gtu" ->
    (rtl (subflags (tval x) (tval y) 32), cond (arm_cond opr))
  | "ltu" -> bc_guard y ("gtu", ws) x
  | "geu" -> bc_guard y ("leu", ws) x
  | _ -> Impossible.impossible
    "non-comparison in ARM conditional branch (or overflow not implemented)"
and bc_of_guard (setup, guard) ~ifso ~ifnot =
  let brtl cond tgt = R.guard cond (R.store pc_lhs tgt wordsize) in
  DG.Test (setup, (brtl guard, ifso, ifnot))
and arm_cond = function
  | "eq" -> "arm_eq"
  | "ne" -> "arm_ne"
  | "lt" -> "arm_lt"
  | "le" -> "arm_le"
  | "gt" -> "arm_gt"
  | "ge" -> "arm_ge"
  | "leu" -> "arm_ls"
  | "gtu" -> "arm_hi"
  | "add_overflows"
  | "div_overflows"
  | "mul_overflows"
  | "mulu_overflows"

```

```

| "sub_overflows" -> Impossible.unimp "ARM overflow tests"
| "ltu" | "geu" -> Impossible.impossible "ARM comparison not reversed"
| _ -> Impossible.impossible "non-comparison in ARM conditional branch"

⟨ARM postexpander 775a⟩+≡ (772c) <774d 775b>
let rec bnegate r = match Dn.rtl r with
| RP.Rtl [RP.App((cop, [32]), [RP.Fetch (bcodes, 32)]), RP.Store (pc, tgt, 32)]
  when RU.Eq.loc pc (Dn.loc pc_lhs) && RU.Eq.loc bcodes (Dn.loc cc) ->
    Up.rtl (RP.Rtl [RP.App((negate cop, [32]), [RP.Fetch (bcodes, 32)]),
      RP.Store (pc, tgt, 32)])
| _ -> Impossible.impossible "ill-formed ARM conditional branch"
and negate = function
| "ne" -> "eq"
| "eq" -> "ne"
| "ge" -> "lt"
| "gt" -> "le"
| "le" -> "gt"
| "lt" -> "ge"
| "geu" -> "ltu"
| "gtu" -> "leu"
| "leu" -> "gtu"
| "ltu" -> "geu"
| "arm_eq" -> "arm_ne"
| "arm_ne" -> "arm_eq"
| "arm_lt" -> "arm_ge"
| "arm_le" -> "arm_gt"
| "arm_gt" -> "arm_le"
| "arm_ge" -> "arm_lt"
| "arm_ls" -> "arm_hi"
| "arm_hi" -> "arm_ls"
| "arm_vs" -> "arm_vc"
| "arm_vc" -> "arm_vs"
| "feq" -> unimp "floating-point comparison"
| "fne" -> unimp "floating-point comparison"
| "flt" -> unimp "floating-point comparison"
| "fle" -> unimp "floating-point comparison"
| "fgt" -> unimp "floating-point comparison"
| "fge" -> unimp "floating-point comparison"
| "fordered" -> unimp "floating-point comparison"
| "funordered" -> unimp "floating-point comparison"
| _ -> impossible
      "bad comparison in expanded ARM conditional branch"

⟨ARM postexpander 775b⟩+≡ (772c) <775a 776a>
(* claude: original call/callr took an extra ~others param (dropped:
* Postexpander.S's call/callr are just ~tgt now, see arch/mips/
* mips.ml's/arch/sparc/sparc.ml's identical fix) and only ever stored
* pc_lhs, never storing ra - so a "call" would jump but never leave a
* usable return address, i.e. it could never actually return (same
* class of bug those backends' original call/callr had). armrec.mlb's
* grammar only recognizes a call as "Par(Goto(target),
* Store(ral,next,32))" ("bl"/"blx"), where next=Add(pc,const) is the
* return address, so both stores must land in the single Par the
* recognizer sees - hence do_call below, mirroring mips.ml's/sparc.ml's
* restructured do_call/call/callr. *)
let do_call tgt =
  DG.Nop,
  R.par [ R.store pc_lhs tgt wordsize
    ; R.store (R.reg ra) (RU.addk wordsize (R.fetch pc_rhs wordsize) 4) wordsize
  ]

```



```
let call ~tgt = do_call (Up.const tgt)
let callr ~tgt = do_call (tval tgt)
```

```
<ARM postexpander 776a>+≡ (772c) <775b 776b>
(* claude: adapted from the old effect-list signature to the current
 * Mflow.cut_args record ({new_sp; new_pc}) - same fix as arch/mips/
 * mips.ml's/arch/sparc/sparc.ml's/arch/alpha/alpha.ml's cut_to. *)
let cut_to {Mflow.new_sp = sp'; Mflow.new_pc = pc'} =
  DG.Nop, R.par [R.store pc_lhs pc' wordsize; R.store (R.reg sp) sp' wordsize]
```

```
<ARM postexpander 776b>+≡ (772c) <776a
let don't_touch_me es = false
(* claude: return/forbidden are required by Postexpander.S but had no
 * definition here (same gap arch/mips/mips.ml/arch/sparc/sparc.ml/
 * arch/alpha/alpha.ml originally had). ARM has no register windows
 * (unlike SPARC) and no gp/pv indirection (unlike Alpha), so this is
 * just "jump to whatever's in lr" - the plain Mflow-style default,
 * reusing this file's own top-level 'return e' (also used by
 * Armcalls.cconv's ~return_to). *)
let return = return (R.fetch (R.reg ra) wordsize)
let forbidden = Rtl.par [] (* BOGUS: NEEDS TO BE A REAL FAULTING INSTRUCTION *)
```

AA.2.7 Expander

```
<arm.ml 776c>+≡ (770a) <772c 776d>
module X = Expander.IntFloatAddr(Post)
```

AA.2.8 Spill and reload

```
<arm.ml 776d>+≡ (770a) <776c 776e>
let spill p t l = [A.store l (Post.tval t) (Post.twidth t)]
let reload p t l =
  let w = Post.twidth t in [R.store (Post.tloc t) (Automaton.fetch l w) w]
```

AA.2.9 Global Variables

```
<arm.ml 776e>+≡ (770a) <776d 776f>
let globals base =
  let width w = if w <= 8 then 8
                 else if w <= 16 then 16
                 else Auxfun.round_up_to 32 w in
  let align = function 8 -> 1 | 16 -> 2 | _ -> 4 in
  A.at mspace ~start:base (A.widen width *> A.align_to align *>
  A.overflow ~growth:Memalloc.Up ~max_alignment:4)
```

AA.2.10 The target record

```
<arm.ml 776f>+≡ (770a) <776e
(* claude: rewritten target record to match the current Target.t shape
 * (target.mli): T.spill/T.reload/T.bnegate/T.goto/T.jump/T.call/T.return/
 * T.branch aren't Target.t fields anymore - they now live inside the
 * single T.machine record, built by the Expander.IntFloatAddr functor
 * from Post above (module X), same as arch/mips/mips.ml/arch/sparc/
 * sparc.ml/arch/alpha/alpha.ml. Ast2ir.tgt = Preast2ir.tgt = T of (...)
 * Target.t is a wrapped variant, not the bare record, hence the
 * Preast2ir.T wrapper below (same as those three backends' own target). *)
```

```

let target =
  let spaces = [ Spaces.m
                  ; Spaces.r
                  ; Spaces.t
                  ; Spaces.c
                ] in
  Preast2ir.T
  { T.name           = "arm"
  ; T.memspace       = mspace
  ; T.max_unaligned_load = R.C 1
  ; T.byteorder      = byteorder
  ; T.wordsize       = wordsize
  ; T.pointersize    = wordsize
  ; T.alignment      = 4 (* strange rotations occur on unaligned loads *)
  ; T.memsize        = 8
  ; T.spaces         = spaces
  ; T.reg_ix_map     = T.mk_reg_ix_map spaces
  ; T.distinct_addr_sp = false
  ; T.float          = Float.none

  ; T.vfp            = vfp
  ; T.machine        = X.machine

  ; T.cc_specs       = Armcc.cc_specs
  ; T.cc_spec_to_auto = Armcall.cconv
                        ~return_to:return
                        { T.embed   = fmach.T.cutto.T.embed
                          ; T.project = fmach.T.cutto.T.project }

  ; T.is_instruction = Armrec.is_instruction
  ; T.tx_ast         = (fun secs -> secs)
  (* claude: T.incapable (empty operator/literal/memory lists) would
   * reject every operator this backend actually implements - see
   * armrec.mlb's grammar (add/sub/and/mul/quot + the eq/ne/lt/gt/ge/ltu/
   * leu/gtu/geu comparisons are recognized as real instructions;
   * everything else falls through to the "<...>" catch-all/error path)
   * and this file's own Post (no double-width mul, or/xor/shift, no
   * float compute - unrm/binrm/dblop/wrdop/wrdrop are all
   * Impossible.unimp/Unsupported so far). Scoped to exactly that
   * subset, at this target's one integer width (32) - same rationale as
   * arch/mips/mips.ml's/arch/alpha/alpha.ml's own hand-written operator
   * lists. mul/quot were added later, driven by tests/tiger/'s actual
   * operator usage (see notes_arm.txt) - widen this list (and
   * armrec.mlb) together as more operators get real camlburg rules. *)
  ; T.capabilities   = { T.operators = List.map Up.opr
                        [ "add",      [32]
                        ; "sub",      [32]
                        ; "and",      [32]
                        ; "mul",      [32]
                        ; "quot",     [32]
                        ; "eq",       [32]
                        ; "ne",       [32]
                        ; "lt",       [32]
                        ; "le",       [32]
                        ; "gt",       [32]
                        ; "ge",       [32]
                        ; "ltu",      [32]
                        ; "leu",      [32]
                        ; "gtu",      [32]
                        ; "geu",      [32]
                        ; "lobits",   [32;8]

```

```

; "lobits", [32;16]
; "not", []
; "bool", []
; "disjoin", []
; "conjoin", []
(* claude: "sx"/"zx" from 1 bit + "bit"
 * (empty width list, like not/bool/
 * disjoin/conjoin above - a "declared
 * supported generically" marker, not a
 * real per-width instruction) declared
 * so codegen/expander.ml's Sx/Zx(Bit(e))
 * boolean-materialization special case
 * (built purely from Post.li +
 * expand_cbranch, no armrec.mlb grammar
 * needed) doesn't print a spurious
 * "No acceptable widths" mvalidate
 * warning - same declaration sparc.ml's/
 * x86.ml's own capabilities make. *)
; "sx", [1;32]
; "zx", [1;32]
; "bit", []
]
; T.litops = []
; T.literals = [32]
; T.memory = [8; 16; 32]
; T.block_copy = true
; T.itemps = [32]
; T.ftemps = []
; T.iwiden = false
; T.fwiden = false
}

; T.globals = globals
; T.rounding_mode = R.reg (('?', Rtl.Identity, Cell.of_size 32), 99, R.C 1)
; T.named_locs = Strutil.assoc2map []
; T.data_section = "data"
; T.charset = "latin1" (* REMOVE THIS FROM TARGET.T *)
}

```

Appendix AB

runtime/

Appendix AC

.

AC.1 driver.nw

`<driver.ml 780a>≡`
`<driver.ml content 29a>`

AC.2 main2.nw

AC.3 Main

`<main.mli 780b>≡`

`<main.ml 780c>≡` `780d▷`
 `module E = Error`
 `module I = Lualink.I`
 `module V = I.Value`

```
let export_argv g strings =
  I.register_module "Sys"
  [ "argv", (V.list V.string).V.embed (List.tl strings);
    "cmd",   V.string.V.embed         (List.hd strings);
  ] g
```

`<main.ml 780d>+≡` `<780c 780e>`
 `let main () =`
 `let argv = Array.to_list Sys.argv in`
 `let state = I.mk () in (* fresh Lua state *)`
 `let evaluate e = ignore (I.dostring state e) in`
 `let bootscript = "Util.dosearchfile('qc--.lua', Boot)" in`
 `try`
 `( export_argv state argv (* export cmd line *)`
 `; evaluate bootscript (* boot interpreter *)`
 `; exit 0 (* success *)`
 `)`
 `with`
 `E.ErrorExn msg as e ->`
 `( E.errorPrt ("uncaught Error exn: " ^ msg)`
 `; raise e`
 `)`

`<main.ml 780e>+≡` `<780d`
 `let _ = main ()`

```

⟨main.ml unused 781a⟩≡
  let _ = try main () with e ->
    ( Printf.eprintf "%s\n" (Printexc.to_string e)
      ; Printf.eprintf "Please report this problem to bugs@cminusminus.org\n"
      ; Printf.eprintf "Exit with exit code 2\n"
      ; exit 2
    )

```

AC.4 This

```

⟨this.mli 781b⟩≡
  val name          : out_channel -> unit
  val version       : out_channel -> unit
  (* pad: TODO
  val boot          : string (* contents of "qc--.lua" *)
  val manual        : string (* manual page qc--(1) *)
  val byteorder     : string (* "big" | "little" *)
  val arch_os       : string (* "x86-linux" *)
  val install_dir   : string (* "/usr/local" *)
  *)

```

```

⟨this.in 781c⟩≡
  (* Do not edit - this file is created from this.in through mk(1)
  * If this file does not compile, check the following files:
  * (1) main2.nw - this.in is defined here
  * (2) mkfile target this.ml - constructs the boot string
  *)

```

781d▷

```

let system          = "@this@"

```

```

⟨this.in 781d⟩+≡
  let name channel =
    let s = try let minus = String.rindex system '-' in
      String.sub system 0 minus
    with Not_found -> "not configured"
  in
    output_string channel s

```

◁781c 781e▷

```

⟨this.in 781e⟩+≡
  let version channel =
    let s = try let minus = String.rindex system '-' in
      String.sub system (minus+1) (String.length system - minus - 1)
    with Not_found -> "not configured"
  in
    output_string channel s

```

◁781d

Appendix AD

Changelog

Appendix AE

Glossary

AST Abstract Syntax Tree

CFG Control-Flow Graph

NAST Normalized Abstract Syntax Tree

RTL Register Transfer List

IR Intermediate Representation

FENV Fat Environment

ELAB Elaboration

PAL Portable Assembly Language

Indexes

Bibliography

- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s)
- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s)
- [Ram12] Norman Ramsey. Noweb, 2012. <http://www.cs.tufts.edu/~nr/noweb/>. cited page(s)